

aggregator.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry_Research
4 # - Aggregator
5 # =====
6 # Description:
7 #   Централизованный агрегатор телеметрии, синхронизирующий фо
8 #   новый сбор данных
9 #
10 # Usage Examples:
11 #   Python API:
12 #       from apps.windows.telemetry_research.aggregator import T
13 #       telemetryAggregator
14 #
15 #       service = TelemetryAggregator()
16 #
17 # File: aggregator.py
18 # Project: ai-breadboard
19 # Package: apps.windows.telemetry_research
20 # Author: hypo69
21 # Copyright: © 2026 hypo69
22 # Updated: 2026-10-01 13:28:28
23 # =====
24
25 from __future__ import annotations
26 """Централизованный агрегатор телеметрии, синхронизирующий фон
27 овый сбор данных"""
28
29 import os
30 import threading
31 import time
32 from datetime import datetime, timedelta, timezone
33 from typing import Any, Dict, List, Optional
```

```

30
31 try:
32     from logger_ai.logger import logger
33 except ImportError:
34     from logger import logger
35 from apps.windows.telemetry.telemetry_config import TelemetryC
onfigManager
36 from apps.windows.telemetry.sensor_collector import SensorColl
ector
37 from apps.windows.telemetry.file_collector import FileCollecto
r
38 from apps.windows.telemetry.json_logger import TelemetryJsonLo
gger
39 from apps.windows.telemetry.sqlite import TelemetryStorage
40
41
42 class TelemetryAggregator:
43     """Централизованный агрегатор телеметрии и сенсорного сбор
а в базу данных и лог.
44
45     Осуществляет периодический сбор аппаратных метрик (через :
class:`SensorCollector`)
46     и файловых событий (через :class:`FileCollector`) с сохран
ением сырых измерений
47     в базу данных SQLite (:class:`TelemetryStorage`), а также
выполняет периодическую
48     почасовую агрегацию метрик.
49
50     Примечание:
51     Экземпляр :class:`TelemetryJsonLogger` сохранён в `__i
nit__` (атрибут `self.logger`)
52     для поддержки явного внедрения зависимостей (DI) и всп
омогательного/резервного
53     файлового JSON-логирования, в то время как основным по
стоянным хранилищем
54     телеметрии выступает SQLite база данных (:class:`Telem
etryStorage`).
55     """

```

```

56 |
57 |     def __init__(
58 |         self,
59 |         config_manager: Optional[TelemetryConfigManager] = Non
60 |     e,
61 |         log_dir: Optional[str] = None,
62 |         storage: Optional[TelemetryStorage] = None,
63 |         sensor_collector: Optional[SensorCollector] = None,
64 |         file_collector: Optional[FileCollector] = None,
65 |         telemetry_logger: Optional[TelemetryJsonLogger] = None
66 |     ,
67 |     ) -> None:
68 |         """Инициализирует агрегатор телеметрии.
69 |
70 |         Args:
71 |             config_manager: Менеджер конфигурации (если None,
72 | создается новый).
73 |             log_dir: Директория для логов (по умолчанию: %APPD
74 | ATA%\AI-Breadboard\apps\logs).
75 |             storage: Экземпляр постоянного SQLite хранилища Te
76 | lemetryStorage.
77 |             sensor_collector: Коллектор аппаратных сенсоров.
78 |             file_collector: Коллектор файловых событий.
79 |             telemetry_logger: Логгер JSON-потока телеметрии.
80 |         """
81 |         self.config_manager = config_manager or TelemetryConfi
82 | gManager()
83 |         self.log_dir = log_dir or self._get_default_log_dir()
84 |         self.storage = storage or TelemetryStorage.get_instanc
85 | e()
86 |
87 |         # Инициализация коллекторов с поддержкой явного внедре
88 | ния зависимостей (DI)
89 |         self.sensor_collector = sensor_collector or SensorColl
90 | ector(config_manager=self.config_manager)
91 |         self.file_collector = file_collector or FileCollector(
92 |             watch_dirs=self.config_manager._config.get("watch_
93 | directories", ["C:\\Users\\"])

```

```

84         )
85         self.logger = telemetry_logger or TelemetryJsonLogger(
86             log_dir=self.log_dir,
87             filename=self.config_manager._config.get("log_file
name", "ai_sensors_polls.json"),
88             max_file_size_mb=self.config_manager._config.get("
max_file_size_mb", 100),
89         )
90
91         self._running = False
92         self._thread: Optional[threading.Thread] = None
93         self._stop_event = threading.Event()
94
95         self._last_measurements: Dict[str, Any] = {}
96         self._measurement_count = 0
97         self._start_time = datetime.now(timezone.utc)
98         self._last_aggregation = self._start_time
99         self._last_audit_check = self._start_time
100        self._audit_check_interval = 3600.0 # Проверка аудито
в раз в час
101
102        # Ленивая инициализация чекера аудитов
103        self._audit_checker = None
104
105        @staticmethod
106        def _get_default_log_dir() -> str:
107            """Возвращает путь к директории логов по умолчанию.
108
109            Returns:
110                str: Путь к лог-директории.
111            """
112            appdata = os.environ.get("APPDATA", os.path.expanduser
("~\\AppData\\Roaming"))
113            return os.path.join(appdata, "AI-Breadboard", "apps",
"windows", "telemetry", "logs")
114
115        def start(self) -> bool:
116            """Запускает фоновый поток сбора телеметрии.

```

```
117 |
118 |     Returns:
119 |         bool: True если агрегатор успешно запущен.
120 |     """
121 |     if self._running:
122 |         logger.warning("Агрегатор телеметрии уже запущен."
123 | )
124 |         return False
125 |
126 |     self._stop_event.clear()
127 |     self._running = True
128 |
129 |     self._thread = threading.Thread(
130 |         target=self._worker_loop,
131 |         name="TelemetryAggregatorWorker",
132 |         daemon=True,
133 |     )
134 |     self._thread.start()
135 |
136 |     logger.info("Агрегатор телеметрии запущен")
137 |     return True
138 |
139 | def stop(self) -> bool:
140 |     """Останавливает фоновый поток сбора телеметрии.
141 |
142 |     Returns:
143 |         bool: True если агрегатор был остановлен.
144 |     """
145 |     if not self._running:
146 |         return False
147 |
148 |     self._running = False
149 |     self._stop_event.set()
150 |
151 |     if self._thread and self._thread.is_alive():
152 |         self._thread.join(timeout=3.0)
153 |
154 |     self._thread = None
```

```

154         self.file_collector.stop()
155         logger.info("Агрегатор телеметрии остановлен")
156         return True
157
158     def _worker_loop(self) -> None:
159         """Основной рабочий цикл агрегатора."""
160         while not self._stop_event.is_set():
161             try:
162                 self.poll_once()
163             except Exception as e:
164                 logger.error(f"Ошибка при сборе телеметрии: {e}")
165
166                 min_interval = self._get_min_interval()
167                 sleep_time = max(0.1, min_interval)
168
169                 if self._stop_event.wait(timeout=sleep_time):
170                     break
171
172     def _get_min_interval(self) -> float:
173         """Возвращает минимальный интервал среди всех сенсоров
174         , учитывая быстрый режим.
175
176         Если с момента запуска прошло меньше fast_duration_days,
177         используется fast_interval.
178         Иначе - обычный минимум среди включенных сенсоров.
179         """
180         now = datetime.now(timezone.utc)
181         fast_days = self.config_manager.get_fast_duration_days()
182
183         if (now - self._start_time).days < fast_days:
184             return self.config_manager.get_fast_interval()
185         # Обычная логика
186         intervals = []
187         for sensor_name in self.config_manager.get_enabled_sensors():
188             interval = self.config_manager.get_sensor_interval(sensor_name)

```

```

186         intervals.append(interval)
187     if not intervals:
188         return self.config_manager._default_interval
189     return min(intervals)
190
191     def poll_once(self) -> None:
192         """Проводит единовременный сбор всей телеметрии."""
193         self._measurement_count += 1
194         now_iso = datetime.now(timezone.utc).isoformat()
195
196         hw_snapshot = {}
197         file_events = []
198         try:
199             hw_snapshot = self.sensor_collector.get_hardware_s
200             napshot()
201             sensor_items = []
202             for s in hw_snapshot.get('sensors', []):
203                 val_entry = s.get('values', [{}])[0]
204                 sensor_items.append({
205                     'id': s.get('id'),
206                     'hardware_name': s.get('hardware_name'),
207                     'hardware_type': s.get('hardware_type'),
208                     'sensor_category': s.get('sensor_category'
209                     ),
210                     'sensor_name': s.get('sensor_name'),
211                     'unit': s.get('unit'),
212                     'value': val_entry.get('num')
213                 })
214             if sensor_items:
215                 self.storage.save_sensor_polls_batch(sensor_it
216                 ems, timestamp=now_iso)
217             except Exception as e:
218                 logger.error(f'Ошибка при сборе и сохранении сенсо
219                 ров: {e}')
220
221             try:
222                 file_events = self.file_collector.get_recent_event
223                 s()

```

```

219 |         except Exception as e:
220 |             logger.error(f'Ошибка при сборе файловых событий:
    | {e}')
221 |
222 |         self._last_measurements = {
223 |             'hardware': hw_snapshot,
224 |             'file_events': file_events,
225 |             'timestamp': now_iso,
226 |         }
227 |
228 |         # После обычного поллинга проверяем необходимость часо
    | вой агрегации
229 |         now = datetime.now(timezone.utc)
230 |         fast_days = self.config_manager.get_fast_duration_days
    | ()
231 |         if (now - self._start_time).days >= fast_days:
232 |             if (now - self._last_aggregation).total_seconds()
    | >= self.config_manager.get_aggregation_interval():
233 |                 self._aggregate_hourly()
234 |                 self._last_aggregation = now
235 |
236 |         # Периодическая проверка критических аудитов
237 |         if (now - self._last_audit_check).total_seconds() >= s
    | elf._audit_check_interval:
238 |             self._check_audits_periodic()
239 |             self._last_audit_check = now
240 |
241 |     def _check_audits_periodic(self) -> None:
242 |         """Периодическая проверка критических состояний систем
    | ы.
243 |
244 |         Проверяет:
245 |         - Pending reboot (требуется перезагрузка)
246 |         - Критическую загрузку CPU/RAM
247 |         - Проблемные драйверы (опционально)
248 |         """
249 |         try:
250 |             # Ленивая инициализация чекера

```



```

251         if self._audit_checker is None:
252             from .audit_startup_checker import AuditStartu
pChecker
253             self._audit_checker = AuditStartupChecker()
254
255             result = self._audit_checker.check_startup_health(
256                 check_integrity=True,
257                 check_performance=True,
258                 check_drivers=False,
259                 check_eventlog=False
260             )
261
262             if not result.is_healthy:
263                 logger.error(
264                     f"⚠️ AUDIT CHECK: Критические проблемы! "
265                     f"Critical: {result.critical_count}, Warni
ngs: {result.warning_count}"
266                 )
267                 for finding in result.findings:
268                     severity = finding.get('severity', 'unknow
n').upper()
269                     domain = finding.get('domain', 'unknown')
270                     title = finding.get('title', 'No title')
271                     logger.warning(f" - [{severity}] {domain}
: {title}")
272
273                 # Сохраняем инцидент в БД
274                 try:
275                     self.storage.save_event(
276                         event_type='audit_critical',
277                         event_details={
278                             'critical_count': result.critical_
count,
279                             'warning_count': result.warning_co
unt,
280                             'findings': result.findings
281                         },
282                         severity='critical' if result.critical

```

```

    _count > 0 else 'warning'
283         )
284         except Exception as save_err:
285             logger.debug(f"Не удалось сохранить инциде
инт аудита: {save_err}")
286
287             elif result.warning_count > 0:
288                 logger.info(
289                     f"  AUDIT CHECK: Предупреждения: {result.w
arning_count} "
290                     f"({result.duration_ms}ms)"
291                 )
292             else:
293                 logger.debug(f"AUDIT CHECK: OK ({result.durati
on_ms}ms)")
294
295         except Exception as e:
296             logger.warning(f"Ошибка при периодической проверке
аудитов: {e}")
297
298     def _aggregate_hourly(self) -> None:
299         """Агрегирует данные сенсоров за последний полный час
и сохраняет их в таблицу агрегатов."""
300         # Вычисляем границы последнего полного часа
301         now = datetime.now(timezone.utc)
302         period_end = now.replace(minute=0, second=0, microseco
nd=0)
303         period_start = period_end - timedelta(hours=1)
304
305         # Запрос к базе: агрегируем по sensor_id за указанный
период
306         query = """
307             SELECT sensor_id,
308                    AVG(value) as avg_value,
309                    MIN(value) as min_value,
310                    MAX(value) as max_value,
311                    COUNT(*) as count
312             FROM sensor_polls

```

```

313         WHERE timestamp >= ? AND timestamp < ?
314         GROUP BY sensor_id
315         """
316         try:
317             with self.storage._lock, self.storage._get_connect
ion() as conn:
318                 cursor = conn.cursor()
319                 cursor.execute(query, (period_start.isoformat(
), period_end.isoformat()))
320                 rows = cursor.fetchall()
321                 aggregates = []
322                 for row in rows:
323                     sensor_id, avg_val, min_val, max_val, cnt
= row
324                     aggregates.append({
325                         "sensor_id": sensor_id,
326                         "period_start": period_start.isoformat
(),
327                         "period_end": period_end.isoformat(),
328                         "avg": avg_val,
329                         "min": min_val,
330                         "max": max_val,
331                         "count": cnt,
332                     })
333                 if aggregates:
334                     saved = self.storage.save_sensor_aggregate
s_batch(aggregates)
335                     logger.info(f"Сохраниено {saved} агрегирова
нных записей за период {period_start} - {period_end}")
336                 except Exception as e:
337                     logger.error(f"Ошибка при часовой агрегации телеме
трии: {e}")
338
339     def get_status(self) -> Dict[str, Any]:
340         """Возвращает текущий статус агрегатора.
341
342         Returns:
343             Dict[str, Any]: Состояние агрегатора.

```

```
344 |         """
345 |         return {
346 |             "is_running": self._running,
347 |             "measurement_count": self._measurement_count,
348 |             "enabled_sensors": self.config_manager.get_enabled
|_sensors(),
349 |         }
350 |
351 |     def get_last_measurement(self) -> Optional[Dict[str, Any]]
|:
352 |         """Возвращает последнюю записанную телеметрию.
353 |
354 |         Returns:
355 |             Optional[Dict[str, Any]]: Последняя телеметрия или
|None.
356 |         """
357 |         return self._last_measurements
358 |
```

analyzer.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry_Research
  # - Analyzer
4 # =====
5 # Description:
6 #   Аналитический движок исследования логов телеметрии.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry_research.analyzer import Tel
  #       emetryResearcher
11 #
12 #       service = TelemetryResearcher()
13 #
14 # File: analyzer.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry_research
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-01 13:28:28
20 # =====
21
22 from __future__ import annotations
23 """Аналитический движок исследования логов телеметрии."""
24
25 import math
26 import statistics
27 import uuid
28 from datetime import datetime, timezone
29 from typing import Any, Dict, List, Optional, Tuple, Union
30
31 try:
```

```

32     from logger import logger
33 except ImportError:
34     from logger import logger
35
36 from .charts import TelemetryChartGenerator
37 from .extractor import TelemetryDataExtractor
38 from .models import (
39     AnomalyEvent,
40     ChartConfig,
41     CorrelationMatrixItem,
42     DeepResearchReport,
43     DeviceEventSummary,
44     HypothesisResult,
45     MetricPoint,
46     MetricStats,
47     ResearchScenarioRequest,
48     TelemetryResearchReport,
49     TimeSeriesDataset,
50 )
51
52
53 class TelemetryResearcher:
54     """Исследователь и аналитик логов системной и аппаратной т
55     елеметрии."""
56
57     def __init__(self, extractor: Optional[TelemetryDataExtrac
58     tor] = None) -> None:
59         """Инициализация аналитического движка.
60
61         Args:
62             extractor: Экземпляр загрузчика данных телеметрии.
63         """
64         self.extractor = extractor or TelemetryDataExtractor()
65
66     def analyze(self, source: Optional[Union[str, List[Dict[str,
67     Any]]]] = None) -> TelemetryResearchReport:
68         """Провести глубокое исследование логов телеметрии и с
69         формировать отчет.

```

```

66 |
67 |         Args:
68 |             source: Источник данных (директория, путь к файлу
или список записей).
69 |
70 |         Returns:
71 |             TelemetryResearchReport: Комплексный отчет исследо
вания.
72 |             """
73 |             records = self.extractor.load_all_records(source)
74 |             logger.info(f"Загружено {len(records)} записей телемет
рии для исследования.")
75 |             now_str = datetime.now(timezone.utc).isoformat()
76 |             if not records:
77 |                 return TelemetryResearchReport(
78 |                     report_id=f"rep-{uuid.uuid4().hex[:8]}",
79 |                     generated_at=now_str,
80 |                     records_analyzed=0,
81 |                     health_score=100.0,
82 |                     summary_conclusions=["Логи телеметрии не содер
жат записей для анализа."],
83 |                 )
84 |
85 |             time_series = self._extract_time_series(records)
86 |             stats_map: Dict[str, MetricStats] = {}
87 |             for metric_name, ds in time_series.items():
88 |                 vals = [pt.value for pt in ds.points]
89 |                 if vals:
90 |                     stats_map[metric_name] = self._calculate_stats
(vals, ds.unit)
91 |
92 |             device_summary = self._analyze_device_events(records)
93 |             anomalies = self._detect_anomalies(time_series, device
_summary)
94 |             health_score = self._compute_health_score(stats_map, a
nomalies, device_summary)
95 |             conclusions = self._generate_conclusions(stats_map, an
omalies, device_summary, health_score)

```

```

96
97         ts_list = [r.get("timestamp") for r in records if r.ge
t("timestamp")]
98         start_time = ts_list[0] if ts_list else None
99         end_time = ts_list[-1] if ts_list else None
100
101         return TelemetryResearchReport(
102             report_id=f"rep-{uuid.uuid4().hex[:8]}",
103             generated_at=now_str,
104             records_analyzed=len(records),
105             time_window_start=start_time,
106             time_window_end=end_time,
107             statistics=stats_map,
108             anomalies=anomalies,
109             device_summary=device_summary,
110             health_score=health_score,
111             summary_conclusions=conclusions,
112         )
113
114     def _extract_time_series(self, records: List[Dict[str, Any
]]) -> Dict[str, TimeSeriesDataset]:
115         """Извлечь структурированные временные ряды из разноро
дных записей."""
116         datasets: Dict[str, TimeSeriesDataset] = {
117             "cpu_load": TimeSeriesDataset(name="CPU Load", uni
t="%", color="#3B82F6"),
118             "ram_used_percent": TimeSeriesDataset(name="RAM Us
age", unit="%", color="#10B981"),
119             "gpu_load": TimeSeriesDataset(name="GPU Load", uni
t="%", color="#8B5CF6"),
120             "cpu_temp": TimeSeriesDataset(name="CPU Temperatur
e", unit="°C", color="#EF4444"),
121             "gpu_temp": TimeSeriesDataset(name="GPU Temperatur
e", unit="°C", color="#F59E0B"),
122             "disk_read_mb_s": TimeSeriesDataset(name="Disk Rea
d", unit="MB/s", color="#06B6D4"),
123             "disk_write_mb_s": TimeSeriesDataset(name="Disk Wr
ite", unit="MB/s", color="#EC4899"),

```



```

124         "net_recv_mb_s": TimeSeriesDataset(name="Network R
    ecv", unit="MB/s", color="#6366F1"),
125         "net_sent_mb_s": TimeSeriesDataset(name="Network S
    ent", unit="MB/s", color="#14B8A6"),
126     }
127
128     for r in records:
129         ts = r.get("timestamp", "")
130         # Поддержка вложенного формата psutil/SystemSnapsh
    ot
131         cpu = r.get("cpu") or {}
132         if isinstance(cpu, dict):
133             val = cpu.get("total_percent") or cpu.get("loa
    d_percent")
134             if val is not None:
135                 datasets["cpu_load"].points.append(MetricP
    oint(timestamp=ts, value=float(val)))
136             temp = cpu.get("temperature_celsius")
137             if temp is not None:
138                 datasets["cpu_temp"].points.append(MetricP
    oint(timestamp=ts, value=float(temp)))
139             if "cpu_total_percent" in r and r["cpu_total_perce
    nt"] is not None:
140                 datasets["cpu_load"].points.append(MetricPoint
    (timestamp=ts, value=float(r["cpu_total_percent"])))
141             elif "cpu_percent" in r and r["cpu_percent"] is no
    t None:
142                 datasets["cpu_load"].points.append(MetricPoint
    (timestamp=ts, value=float(r["cpu_percent"])))
143             elif "cpu_usage_pct" in r and r["cpu_usage_pct"] i
    s not None:
144                 datasets["cpu_load"].points.append(MetricPoint
    (timestamp=ts, value=float(r["cpu_usage_pct"])))
145
146         mem = r.get("memory") or {}
147         if isinstance(mem, dict):
148             val = mem.get("percent")
149             if val is not None:

```

```

150         datasets["ram_used_percent"].points.append
(MetricPoint(timestamp=ts, value=float(val)))
151         if "memory_percent" in r and r["memory_percent"] i
s not None:
152             datasets["ram_used_percent"].points.append(Met
ricPoint(timestamp=ts, value=float(r["memory_percent"])))
153             elif "memory_usage_pct" in r and r["memory_usage_p
ct"] is not None:
154                 datasets["ram_used_percent"].points.append(Met
ricPoint(timestamp=ts, value=float(r["memory_usage_pct"])))
155
156         gpu = r.get("gpu") or (r.get("gpus")[0] if isinsta
nce(r.get("gpus"), list) and r.get("gpus") else {})
157         if isinstance(gpu, dict):
158             val = gpu.get("load_percent")
159             if val is not None:
160                 datasets["gpu_load"].points.append(MetricP
oint(timestamp=ts, value=float(val)))
161             temp = gpu.get("temperature_celsius")
162             if temp is not None:
163                 datasets["gpu_temp"].points.append(MetricP
oint(timestamp=ts, value=float(temp)))
164             if "gpu_load_percent" in r and r["gpu_load_percent
"] is not None:
165                 datasets["gpu_load"].points.append(MetricPoint
(timestamp=ts, value=float(r["gpu_load_percent"])))
166             if "gpu_temp_c" in r and r["gpu_temp_c"] is not No
ne:
167                 datasets["gpu_temp"].points.append(MetricPoint
(timestamp=ts, value=float(r["gpu_temp_c"])))
168
169         disk_io = r.get("disk_io") or {}
170         if isinstance(disk_io, dict):
171             rb = disk_io.get("read_bytes_per_sec", 0.0) or
(disk_io.get("read_kb_s", 0.0) * 1024.0)
172             wb = disk_io.get("write_bytes_per_sec", 0.0) o
r (disk_io.get("write_kb_s", 0.0) * 1024.0)
173             if rb > 0:

```

```

174 |             datasets["disk_read_mb_s"].points.append(M
MetricPoint(timestamp=ts, value=round(rb / (1024 * 1024), 2)))
175 |             if wb > 0:
176 |                 datasets["disk_write_mb_s"].points.append(
MetricPoint(timestamp=ts, value=round(wb / (1024 * 1024), 2)))
177 |             if "disk_read_bytes_sec" in r and r["disk_read_byt
es_sec"] is not None:
178 |                 rb = float(r["disk_read_bytes_sec"])
179 |                 datasets["disk_read_mb_s"].points.append(Metri
cPoint(timestamp=ts, value=round(rb / (1024 * 1024), 2)))
180 |             if "disk_write_bytes_sec" in r and r["disk_write_b
ytes_sec"] is not None:
181 |                 wb = float(r["disk_write_bytes_sec"])
182 |                 datasets["disk_write_mb_s"].points.append(Metr
icPoint(timestamp=ts, value=round(wb / (1024 * 1024), 2)))
183 |
184 |             # Сеть
185 |             net = r.get("network")
186 |             if isinstance(net, list) and net:
187 |                 n0 = net[0]
188 |                 if isinstance(n0, dict):
189 |                     rs = n0.get("bytes_recv_sec", 0.0)
190 |                     ss = n0.get("bytes_sent_sec", 0.0)
191 |                     if rs > 0:
192 |                         datasets["net_recv_mb_s"].points.appen
d(MetricPoint(timestamp=ts, value=round(rs / (1024 * 1024), 2)
))
193 |                     if ss > 0:
194 |                         datasets["net_sent_mb_s"].points.appen
d(MetricPoint(timestamp=ts, value=round(ss / (1024 * 1024), 2)
))
195 |             if "network_recv_bytes_sec" in r and r["network_re
cv_bytes_sec"] is not None:
196 |                 rs = float(r["network_recv_bytes_sec"])
197 |                 datasets["net_recv_mb_s"].points.append(Metric
Point(timestamp=ts, value=round(rs / (1024 * 1024), 2)))
198 |             elif "net_bytes_recv_sec" in r and r["net_bytes_re
cv_sec"] is not None:

```

```

199         try:
200             rs = float(r["net_bytes_recv_sec"])
201             datasets["net_recv_mb_s"].points.append(MetricPoint(timestamp=ts, value=round(rs / (1024 * 1024), 2)))
202         except ValueError:
203             pass
204         if "network_sent_bytes_sec" in r and r["network_sent_bytes_sec"] is not None:
205             ss = float(r["network_sent_bytes_sec"])
206             datasets["net_sent_mb_s"].points.append(MetricPoint(timestamp=ts, value=round(ss / (1024 * 1024), 2)))
207         elif "net_bytes_sent_sec" in r and r["net_bytes_sent_sec"] is not None:
208             try:
209                 ss = float(r["net_bytes_sent_sec"])
210                 datasets["net_sent_mb_s"].points.append(MetricPoint(timestamp=ts, value=round(ss / (1024 * 1024), 2)))
211             except ValueError:
212                 pass
213
214         # Поддержка sensor_polls (LHM / LibreHardwareMonitor)
215         sensor_name = str(r.get("sensor_name", "")).lower()
216         sensor_type = str(r.get("sensor_type", "")).lower()
217         sensor_cat = str(r.get("sensor_category", "")).lower()
218         val_raw = r.get("value")
219         if val_raw is not None and val_raw != "":
220             try:
221                 val = float(val_raw)
222                 st = f"{sensor_name} {sensor_type} {sensor_cat}"
223                 if "cpu" in st and ("load" in st or "total" in st or "usage" in st):
224                     datasets["cpu_load"].points.append(MetricPoint(timestamp=ts, value=val))

```

```

225         elif "cpu" in st and ("temp" in st or "pac
| kage" in st):
226             datasets["cpu_temp"].points.append(Met
| ricPoint(timestamp=ts, value=val))
227             elif "gpu" in st and ("load" in st or "cor
| e" in st):
228                 datasets["gpu_load"].points.append(Met
| ricPoint(timestamp=ts, value=val))
229                 elif "gpu" in st and "temp" in st:
230                     datasets["gpu_temp"].points.append(Met
| ricPoint(timestamp=ts, value=val))
231                 elif ("ram" in st or "memory" in st) and (
| "used" in st or "percent" in st or "%" in str(r.get("unit", ""
| )))
232                     datasets["ram_used_percent"].points.ap
| pend(MetricPoint(timestamp=ts, value=val))
233             except (ValueError, TypeError):
234                 pass
235
236     return {k: ds for k, ds in datasets.items() if ds.poin
| ts}
237
238     def _calculate_stats(self, values: List[float], unit: str
| = "%") -> MetricStats:
239         """Рассчитать описательную статистику по массиву значе
| ний."""
240         if not values:
241             return MetricStats(unit=unit)
242         sorted_vals = sorted(values)
243         n = len(sorted_vals)
244         min_v = sorted_vals[0]
245         max_v = sorted_vals[-1]
246         avg_v = sum(sorted_vals) / n
247         median_v = statistics.median(sorted_vals)
248         p95_idx = int(math.ceil(0.95 * n)) - 1
249         p95_v = sorted_vals[min(p95_idx, n - 1)]
250         std_v = statistics.stdev(sorted_vals) if n > 1 else 0.

```

```

251         return MetricStats(
252             count=n,
253             min_val=round(min_v, 2),
254             max_val=round(max_v, 2),
255             avg_val=round(avg_v, 2),
256             median_val=round(median_v, 2),
257             p95_val=round(p95_v, 2),
258             std_dev=round(std_v, 2),
259             unit=unit,
260         )
261
262     def _analyze_device_events(self, records: List[Dict[str, Any]]) -> DeviceEventSummary:
263         """Собрать агрегированную статистику по аппаратным событиям и сбоям."""
264         summary = DeviceEventSummary()
265         flapping_set = set()
266         for r in records:
267             ev_type = r.get("event_type")
268             dev_id = r.get("device_instance_id") or r.get("friendly_name")
269             if not ev_type and not dev_id:
270                 continue
271             summary.total_events += 1
272             if ev_type:
273                 summary.by_event_type[ev_type] = summary.by_event_type.get(ev_type, 0) + 1
274             cat = r.get("category") or r.get("device_class") or "Другое"
275             summary.by_category[cat] = summary.by_category.get(cat, 0) + 1
276             if r.get("has_problem") or r.get("problem_code", 0) > 0:
277                 summary.error_count += 1
278             flap_count = r.get("flapping_count_in_window", 0)
279             if flap_count > 0 and dev_id:
280                 flapping_set.add(f"{dev_id} (всплесков: {flap_count})")

```

```

281 |         summary.flapping_devices = list(flapping_set)
282 |         return summary
283 |
284 |     def _detect_anomalies(
285 |         self, time_series: Dict[str, TimeSeriesDataset], device_summary: DeviceEventSummary
286 |     ) -> List[AnomalyEvent]:
287 |         """Обнаружить аномальные значения метрик и сбои оборудования."""
288 |         anomalies: List[AnomalyEvent] = []
289 |         if "cpu_load" in time_series:
290 |             for pt in time_series["cpu_load"].points:
291 |                 if pt.value >= 90.0:
292 |                     anomalies.append(
293 |                         AnomalyEvent(
294 |                             timestamp=pt.timestamp,
295 |                             metric="CPU Load",
296 |                             value=pt.value,
297 |                             threshold=90.0,
298 |                             severity="critical" if pt.value >=
299 |                             98.0 else "warning",
300 |                             description=f"Пиковая нагрузка процессора достигла {pt.value}%",
301 |                         )
302 |                     )
303 |                 if "ram_used_percent" in time_series:
304 |                     for pt in time_series["ram_used_percent"].points:
305 |                         if pt.value >= 90.0:
306 |                             anomalies.append(
307 |                                 AnomalyEvent(
308 |                                     timestamp=pt.timestamp,
309 |                                     metric="RAM Usage",
310 |                                     value=pt.value,
311 |                                     threshold=90.0,
312 |                                     severity="critical" if pt.value >=
313 |                                     95.0 else "warning",
314 |                                     description=f"Высокое потребление оперативной памяти: {pt.value}%",

```

```

313         )
314     )
315     if "cpu_temp" in time_series:
316         for pt in time_series["cpu_temp"].points:
317             if pt.value >= 80.0:
318                 anomalies.append(
319                     AnomalyEvent(
320                         timestamp=pt.timestamp,
321                         metric="CPU Temp",
322                         value=pt.value,
323                         threshold=80.0,
324                         severity="critical" if pt.value >=
325                             90.0 else "warning",
326                         description=f"Перегрев процессора:
327                             {pt.value}°C",
328                     )
329                 )
330             if device_summary.error_count > 0:
331                 anomalies.append(
332                     AnomalyEvent(
333                         timestamp=datetime.now(timezone.utc).isofo
334                         rmat(),
335                         metric="Device Failures",
336                         value=float(device_summary.error_count),
337                         threshold=0.0,
338                         severity="warning",
339                         description=f"Зафиксировано {device_summar
340                             y.error_count} сбоев в работе подключенных устройств",
341                     )
342                 )
343             return anomalies[-50:]
344
345     def _compute_health_score(
346         self, stats_map: Dict[str, MetricStats], anomalies: Li
347         st[AnomalyEvent], device_summary: DeviceEventSummary
348     ) -> float:
349         """Рассчитать интегральный индекс здоровья системы от
350         0.0 до 100.0."""

```



```

345 |         score = 100.0
346 |         if "cpu_load" in stats_map:
347 |             avg_cpu = stats_map["cpu_load"].avg_val
348 |             if avg_cpu > 70.0:
349 |                 score -= min(30.0, (avg_cpu - 70.0) * 1.0)
350 |         if "ram_used_percent" in stats_map:
351 |             avg_ram = stats_map["ram_used_percent"].avg_val
352 |             if avg_ram > 80.0:
353 |                 score -= min(25.0, (avg_ram - 80.0) * 1.25)
354 |         if "cpu_temp" in stats_map:
355 |             max_temp = stats_map["cpu_temp"].max_val
356 |             if max_temp > 80.0:
357 |                 score -= min(20.0, (max_temp - 80.0) * 1.5)
358 |         if device_summary.error_count > 0:
359 |             score -= min(25.0, device_summary.error_count * 5.
0)
360 |
361 |         crit_count = sum(1 for a in anomalies if a.severity ==
"critical")
362 |         score -= min(20.0, crit_count * 4.0)
363 |         return max(0.0, round(score, 1))
364 |
365 |     def _generate_conclusions(
366 |         self,
367 |         stats_map: Dict[str, MetricStats],
368 |         anomalies: List[AnomalyEvent],
369 |         device_summary: DeviceEventSummary,
370 |         health_score: float,
371 |     ) -> List[str]:
372 |         """Сформировать текстовые выводы и рекомендации исслед
ования."""
373 |         conclusions = []
374 |         if health_score >= 90.0:
375 |             conclusions.append("Состояние системы стабильное:
параметры утилизации ресурсов находятся в допустимых границах.
")
376 |             elif health_score >= 70.0:
377 |                 conclusions.append("Состояние системы удовлетворит

```

```

    ельное, но выявлены эпизодические пики нагрузки.")
378         else:
379             conclusions.append("Внимание: состояние системы де
градировало из-за высокой утилизации ресурсов или сбоев оборуд
ования.")
380
381         if "cpu_load" in stats_map:
382             cpu = stats_map["cpu_load"]
383             conclusions.append(f"Процессор (CPU): средняя нагр
узка {cpu.avg_val}%, пик {cpu.max_val}% (p95: {cpu.p95_val}%).
")
384         if "ram_used_percent" in stats_map:
385             ram = stats_map["ram_used_percent"]
386             conclusions.append(f"Оперативная память (RAM): сре
днее использование {ram.avg_val}%, максимум {ram.max_val}%.")
387         if device_summary.error_count > 0:
388             conclusions.append(f"Обнаружено {device_summary.er
ror_count} аппаратных сбоев или проблемных устройств.")
389         if device_summary.flapping_devices:
390             conclusions.append(f"Нестабильные устройства (флап
пинг): {' '.join(device_summary.flapping_devices)}.")
391         return conclusions
392
393     def run_deep_research(
394         self,
395         scenario: Optional[ResearchScenarioRequest] = None,
396         chart_generator: Optional[TelemetryChartGenerator] = N
one,
397     ) -> DeepResearchReport:
398         """Выполнить полный цикл исследовательского анализа те
леметрии (с корреляциями и гипотезами)."""
399         sc = scenario or ResearchScenarioRequest()
400         source = sc.records if sc.records is not None else sc.
source_path
401         c_gen = chart_generator or TelemetryChartGenerator()
402
403         raw_records = self.extractor.load_all_records(source)
404         filtered_records = self._filter_records(

```

```
405         records=raw_records, start_time=sc.start_time, end
|_time=sc.end_time, subsystems=sc.subsystems
406     )
407     logger.info(f"Запуск исследования телеметрии: обработка
|но {len(filtered_records)} записей (из исходных {len(raw_recor
|ds)}).")
408
409     base_report = self.analyze(filtered_records)
410     time_series_map = self._extract_time_series(filtered_r
|ecords)
411     base_report.charts = c_gen.generate_chart_configs(time
|_series_map, base_report)
412
413     correlations = self._calculate_correlations(time_serie
|s_map)
414     hypotheses: List[HypothesisResult] = []
415     if sc.enable_hypotheses_check:
416         hypotheses = self._evaluate_hypotheses(base_report
|, time_series_map, correlations)
417
418     recommendations = self._generate_recommendations(base_
|report, hypotheses, correlations)
419     summary = self._build_investigation_summary(base_repor
|t, hypotheses, len(filtered_records))
420
421     report_id = f"deep_research_{datetime.now(timezone.utc
|).strftime('%Y%m%d_%H%M%S')}-{uuid.uuid4().hex[:6]}"
422     now_iso = datetime.now(timezone.utc).isoformat()
423
424     return DeepResearchReport(
425         report_id=report_id,
426         generated_at=now_iso,
427         base_report=base_report,
428         correlations=correlations,
429         hypotheses=hypotheses,
430         investigation_summary=summary,
431         actionable_recommendations=recommendations,
432     )
```

```

433
434     # Алиас для совместимости
435     analyze_deep = run_deep_research
436
437     def _filter_records(
438         self,
439         records: List[Dict[str, Any]],
440         start_time: Optional[str] = None,
441         end_time: Optional[str] = None,
442         subsystems: Optional[List[str]] = None,
443     ) -> List[Dict[str, Any]]:
444         """Фильтрует записи по временному диапазону и составу
445         подсистем."""
446         if not records:
447             return []
448         start_dt = self._parse_iso_time(start_time) if start_t
449         ime else None
450         end_dt = self._parse_iso_time(end_time) if end_time el
451         se None
452         result: List[Dict[str, Any]] = []
453
454         for r in records:
455             ts_str = r.get("timestamp") or r.get("time") or r.
456             get("created_at")
457             if ts_str and (start_dt or end_dt):
458                 rec_dt = self._parse_iso_time(str(ts_str))
459                 if rec_dt:
460                     if start_dt and rec_dt < start_dt:
461                         continue
462                     if end_dt and rec_dt > end_dt:
463                         continue
464                 if subsystems:
465                     match_subsystem = False
466                     for sub in subsystems:
467                         sub_lower = sub.lower()
468                         if sub_lower in r or any(sub_lower in str(
469                             k).lower() for k in r.keys()):
470                             match_subsystem = True

```

```

466         break
467         if sub_lower == "devices" and ("device_ins
tance_id" in r or "event_type" in r):
468             match_subsystem = True
469             break
470         if not match_subsystem:
471             continue
472         result.append(r)
473     return result
474
475     @staticmethod
476     def _parse_iso_time(time_str: Optional[str]) -> Optional[da
atetime]:
477         """Парсит строку даты и времени в ISO формате."""
478         if not time_str:
479             return None
480         clean_str = time_str.replace("Z", "+00:00")
481         try:
482             return datetime.fromisoformat(clean_str)
483         except Exception:
484             return None
485
486     def _calculate_correlations(self, time_series_map: Dict[str,
TimeSeriesDataset]) -> List[CorrelationMatrixItem]:
487         """Вычисляет коэффициенты корреляции Пирсона между пар
ами системных метрик."""
488         keys = sorted(list(time_series_map.keys()))
489         correlations: List[CorrelationMatrixItem] = []
490
491         for i in range(len(keys)):
492             for j in range(i + 1, len(keys)):
493                 key_a = keys[i]
494                 key_b = keys[j]
495                 ds_a = time_series_map[key_a]
496                 ds_b = time_series_map[key_b]
497                 pts_a = ds_a.points if hasattr(ds_a, "points")
else ds_a
498                 pts_b = ds_b.points if hasattr(ds_b, "points")

```

```

    else ds_b
499         if not pts_a or not pts_b:
500             continue
501         val_a = [p.value if hasattr(p, "value") else f
load(p[1] if isinstance(p, (list, tuple)) else p) for p in pts
_a]
502         val_b = [p.value if hasattr(p, "value") else f
load(p[1] if isinstance(p, (list, tuple)) else p) for p in pts
_b]
503         min_len = min(len(val_a), len(val_b))
504         if min_len < 3:
505             continue
506         slice_a = val_a[:min_len]
507         slice_b = val_b[:min_len]
508         coeff = self._pearson_correlation(slice_a, sli
ce_b)
509         if math.isnan(coeff):
510             continue
511         interpretation = self._interpret_correlation(c
oeff, key_a, key_b)
512         correlations.append(
513             CorrelationMatrixItem(
514                 metric_a=key_a,
515                 metric_b=key_b,
516                 coefficient=round(coeff, 3),
517                 sample_size=min_len,
518                 interpretation=interpretation,
519             )
520         )
521         return sorted(correlations, key=lambda x: abs(x.coeffi
cient), reverse=True)
522
523     @staticmethod
524     def _pearson_correlation(x: List[float], y: List[float]) -
> float:
525         """Вычисляет коэффициент линейной корреляции Пирсона д
ля двух списков чисел."""
526         n = len(x)

```

```

527         if n == 0:
528             return 0.0
529         mean_x = sum(x) / n
530         mean_y = sum(y) / n
531         diff_prod = sum((x[k] - mean_x) * (y[k] - mean_y) for
k in range(n))
532         sq_diff_x = sum((x[k] - mean_x) ** 2 for k in range(n)
)
533         sq_diff_y = sum((y[k] - mean_y) ** 2 for k in range(n)
)
534         denominator = math.sqrt(sq_diff_x * sq_diff_y)
535         if denominator == 0.0:
536             return 0.0
537         return diff_prod / denominator
538
539     @staticmethod
540     def _interpret_correlation(coeff: float, metric_a: str, me
tric_b: str) -> str:
541         """Формирует текстовое объяснение коэффициента корреля
ции."""
542         abs_c = abs(coeff)
543         direction = "прямая" if coeff > 0 else "обратная"
544         if abs_c >= 0.8:
545             strength = f"Очень сильная {direction}"
546         elif abs_c >= 0.6:
547             strength = f"Заметная {direction}"
548         elif abs_c >= 0.3:
549             strength = f"Умеренная {direction}"
550         else:
551             strength = f"Слабая {direction}"
552         return f"{strength} связь между '{metric_a}' и '{metri
c_b}' (r={coeff:.2f}). {'Изменения происходят синхронно.' if c
oeff > 0.5 else ''}"
553
554     def _evaluate_hypotheses(
555         self,
556         report: TelemetryResearchReport,
557         time_series_map: Dict[str, TimeSeriesDataset],

```

```

558         correlations: List[CorrelationMatrixItem],
559     ) -> List[HypothesisResult]:
560         """Проверяет набор стандартных системных гипотез."""
561         hypotheses: List[HypothesisResult] = []
562
563         # 1. CPU Thermal stress
564         cpu_temp_stats = report.statistics.get("cpu_temp")
565         cpu_load_stats = report.statistics.get("cpu_load")
566         h1_confirmed = False
567         h1_evidence: List[str] = []
568         h1_conf = 0.5
569         if cpu_temp_stats and cpu_temp_stats.max_val >= 82.0:
570             h1_confirmed = True
571             h1_conf = 0.85
572             h1_evidence.append(f"Пиковая температура CPU дости
573 гла {cpu_temp_stats.max_val}°C (порог 82°C)")
574             if cpu_load_stats:
575                 h1_evidence.append(f"Средняя загрузка CPU сост
576 авляла {cpu_load_stats.avg_val:.1f}%")
577             else:
578                 h1_evidence.append("Температуры CPU находятся в пр
579 еделах нормы (ниже 82°C)")
580             hypotheses.append(
581                 HypothesisResult(
582                     hypothesis_id="H_CPU_THERMAL_STRESS",
583                     title="Термический стресс и перегрев CPU",
584                     description="Проверка риска троттлинга или дег
585 радации при высоких температурах процессора.",
586                     confirmed=h1_confirmed,
587                     confidence=h1_conf,
588                     evidence=h1_evidence,
589                     recommendation="Проверить термопасту, систему
590 охлаждения и отсутствие пыли в радиаторах CPU."

```



```

591 |         # 2. RAM Pressure
592 |         ram_stats = report.statistics.get("ram_used_percent")
593 |         h2_confirmed = False
594 |         h2_evidence: List[str] = []
595 |         h2_conf = 0.5
596 |         if ram_stats and ram_stats.max_val >= 88.0:
597 |             h2_confirmed = True
598 |             h2_conf = 0.9
599 |             h2_evidence.append(f"Пиковое потребление RAM соста
вилось {ram_stats.max_val}% (порог 88%)")
600 |             if ram_stats:
601 |                 h2_evidence.append(f"Средняя загрузка RAM: {ra
m_stats.avg_val:.1f}%")
602 |             else:
603 |                 h2_evidence.append("Потребление RAM находится в бе
зопасных пределах (ниже 88%)")
604 |             hypotheses.append(
605 |                 HypothesisResult(
606 |                     hypothesis_id="H_RAM_PRESSURE",
607 |                     title="Дефицит оперативной памяти (RAM Pressur
e)",
608 |                     description="Анализ рисков нехватки оперативно
й памяти и повышенного использования файла подкачки.",
609 |                     confirmed=h2_confirmed,
610 |                     confidence=h2_conf,
611 |                     evidence=h2_evidence,
612 |                     recommendation="Закрыть неиспользуемые фоновые
приложения или расширить объем ОЗУ."
613 |                     if h2_confirmed
614 |                     else "Памяти достаточно для текущей рабочей на
грузки.",
615 |                 )
616 |             )
617 |
618 |         # 3. Hardware Flapping
619 |         dev_sum = report.device_summary
620 |         h3_confirmed = dev_sum.error_count > 0 or len(dev_sum.
flapping_devices) > 0

```

```

621         h3_evidence: List[str] = []
622         if h3_confirmed:
623             if dev_sum.error_count > 0:
624                 h3_evidence.append(f"Зафиксировано {dev_sum.er
ror_count} сбоев и ошибок оборудования в логах")
625                 if dev_sum.flapping_devices:
626                     h3_evidence.append(f"Обнаружен флаппинг устрой
ств: {' ', ' '.join(dev_sum.flapping_devices)}")
627             else:
628                 h3_evidence.append("Ошибок оборудования и флаппинг
а устройств в исследованный период не обнаружено")
629             hypotheses.append(
630                 HypothesisResult(
631                     hypothesis_id="H_DEVICE_INSTABILITY",
632                     title="Нестабильность и сбои оборудования (Har
dware Flapping)",
633                     description="Проверка наличия сбоящих шин, отв
аливающихся USB/PCI контроллеров или конфликтов драйверов.",
634                     confirmed=h3_confirmed,
635                     confidence=0.95 if h3_confirmed else 0.7,
636                     evidence=h3_evidence,
637                     recommendation="Проверить подключение кабелей,
обновить драйверы сбоящих устройств или заменить порт."
638                     if h3_confirmed
639                     else "Оборудование работает стабильно.",
640                 )
641             )
642
643         # 4. GPU Thermal
644         gpu_temp = report.statistics.get("gpu_temp")
645         h4_confirmed = False
646         h4_evidence: List[str] = []
647         if gpu_temp and gpu_temp.max_val >= 83.0:
648             h4_confirmed = True
649             h4_evidence.append(f"Пиковая температура GPU: {gpu
_temp.max_val}°C")
650         else:
651             h4_evidence.append("Температура GPU в безопасной з

```

```

    one")
652         hypotheses.append(
653             HypothesisResult(
654                 hypothesis_id="H_GPU_THERMAL",
655                 title="Термическая стабильность графического у
скорителя (GPU)",
656                 description="Оценка температурных режимов граф
ического чипа при нагрузке.",
657                 confirmed=h4_confirmed,
658                 confidence=0.85 if h4_confirmed else 0.6,
659                 evidence=h4_evidence,
660                 recommendation="Настроить кривую вентиляторов
GPU или проверить вентиляцию корпуса."
661                 if h4_confirmed
662                 else "GPU работает в штатном тепловом режиме."
663             )
664         )
665
666     return hypotheses
667
668     def _generate_recommendations(
669         self,
670         report: TelemetryResearchReport,
671         hypotheses: List[HypothesisResult],
672         correlations: List[CorrelationMatrixItem],
673     ) -> List[str]:
674         """Генерирует практические рекомендации на основе анализ
а. """
675         recs: List[str] = []
676         for h in hypotheses:
677             if h.confirmed and h.recommendation:
678                 recs.append(f"[{h.title}]: {h.recommendation}")
679         )
680         for anom in report.anomalies:
681             if anom.severity in ("critical", "high"):
682                 recs.append(f"[Аномалия {anom.metric}]: {anom.
description}")

```

```
682         if not recs:
683             recs.append("Система работает стабильно. Регулярно
        проводите профилактический аудит телеметрии.")
684         return recs
685
686     @staticmethod
687     def _build_investigation_summary(
688         report: TelemetryResearchReport, hypotheses: List[Hypo
thesisResult], total_records: int
689     ) -> str:
690         """Формирует текстовое резюме проведенного исследовани
        я."""
691         confirmed_count = sum(1 for h in hypotheses if h.confir
med)
692         return (
693             f"Исследование завершено. Проанализировано {total_
records} записей телеметрии. "
694             f"Индекс здоровья системы: {report.health_score}/1
00. Обнаружено аномалий: {len(report.anomalies)}. "
695             f"Подтверждено проблемных гипотез: {confirmed_coun
t} из {len(hypotheses)}."
696         )
697
698
699 TelemetryAnalyzer = TelemetryResearcher
700
701 __all__ = [
702     "TelemetryResearcher",
703     "TelemetryAnalyzer",
704 ]
705
706
```

assistant.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry_Research
  # - Assistant
4 # =====
5 # Description:
6 #   Интеллектуальный AI-ассистент анализа системной телеметрии
  #   и построения графиков.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry_research.assistant import Te
  #       lemetryAssistant
11 #
12 #       service = TelemetryAssistant()
13 #
14 # File: assistant.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry_research
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-04 07:48:00
20 # =====
21
22 from __future__ import annotations
23 """Интеллектуальный AI-ассистент анализа системной телеметрии,
  #   ресурсов программ клиента и построения графиков."""
24
25 import asyncio
26 import json
27 from typing import Any, Dict, List, Optional, Union
28
29 try:
```

```

30     from logger import logger
31 except ImportError:
32     import logging
33     logger = logging.getLogger(__name__)
34
35 from .query_engine import TelemetryQueryEngine
36 from .client_resource_analyzer import ClientResourceAnalyzer
37
38 try:
39     from src.ai.orchestration.unified_chat import UnifiedChatModel
40 except Exception as ex:
41     logger.warning(f"Не удалось импортировать UnifiedChatModel
42     : {ex}")
43     UnifiedChatModel = None
44
45 class TelemetryAssistant:
46     """Ассистент для диалогового исследования телеметрии с генерацией графиков и SQL."""
47
48     SYSTEM_INSTRUCTION = """Ты – интеллектуальный эксперт по анализу системной телеметрии Windows и энергопотреблению компьютера в проекте AI-Breadboard.
49     Твоя задача – отвечать на вопросы пользователя четко, профессионально и на русском языке, анализировать показатели загрузки ЦП, видеокарты, дисков, температурных датчиков, энергопотребления и процессов.
50     Когда пользователю требуется визуализация (график, круговая диаграмма, столбчатая диаграмма), ты формируешь развернутый ответ с выводами, рекомендациями и структурированными данными.
51     Всегда приводи конкретные числовые значения (кВт·ч, Вт, %, ГГц, МБ) и объясняй их физический и системный смысл."""
52
53     def __init__(
54         self,
55         query_engine: Optional[TelemetryQueryEngine] = None,
56         chat_model: Optional[Any] = None,

```

```

57         client_analyzer: Optional[ClientResourceAnalyzer] = No
ne,
58     ) -> None:
59         """Инициализация ассистента.
60
61         Args:
62             query_engine: Экземпляр движка запросов telemetry.
db.
63             chat_model: Модель чата (по умолчанию UnifiedChatM
odel).
64             client_analyzer: Анализатор ресурсов программ клие
нта.
65         """
66         self.query_engine = query_engine or TelemetryQueryEngi
ne()
67         self._chat_model = chat_model
68         self.client_analyzer = client_analyzer or ClientResour
ceAnalyzer(
69             query_engine=self.query_engine, chat_model=self._c
hat_model
70         )
71
72     @property
73     def chat_model(self) -> Any:
74         """Получить активную модель LLM."""
75         if self._chat_model is None and UnifiedChatModel is no
t None:
76             try:
77                 self._chat_model = UnifiedChatModel(system_ins
truction=self.SYSTEM_INSTRUCTION)
78                 if self.client_analyzer and self.client_analyz
er._chat_model is None:
79                     self.client_analyzer._chat_model = self._c
hat_model
80             except Exception as ex:
81                 logger.warning(f"Не удалось инициализировать U
nifiedChatModel: {ex}")
82                 self._chat_model = None

```

```

83         return self._chat_model
84
85     async def handle_query(
86         self,
87         user_message: str,
88         source_path: Optional[str] = None,
89         history: Optional[List[Dict[str, Any]]] = None,
90     ) -> Dict[str, Any]:
91         """Обработать пользовательский запрос на естественном
92         языке.
93
94         Args:
95             user_message: Текст запроса пользователя.
96             source_path: Путь к выбранному файлу БД или логов.
97             history: История диалога.
98
99         Returns:
100             Dict[str, Any]: Ответ ассистента со структурирован
101             ным текстом, графиком и SQL.
102         """
103         msg_lower = user_message.strip().lower()
104
105         # 1. Распознавание интента: Ресурсы клиентских програм
106         # м и оценка Gemini
107         if self._is_client_resources_query(msg_lower):
108             return await self._handle_client_resources_query(u
109             ser_message, source_path)
110
111         # 2. Распознавание интента: График загрузки ЦПУ по вре
112         # мени
113         if self._is_cpu_timeline_query(msg_lower):
114             return await self._handle_cpu_timeline_query(user_
115             message, source_path)
116
117         # 3. Распознавание интента: Потребление энергии и суто
118         # чная круговая диаграмма
119         if self._is_power_query(msg_lower):
120             return await self._handle_power_consumption_query(

```



```

    user_message, source_path)
114
115     # 4. Распознавание интента: Топ процессов
116     if self._is_top_processes_query(msg_lower):
117         return await self._handle_top_processes_query(user
    _message, source_path)
118
119     # 5. Произвольный SELECT SQL запрос от пользователя
120     if msg_lower.startswith("select ") or msg_lower.starts
    with("with "):
121         return self._handle_direct_sql_query(user_message,
    source_path)
122
123     # 6. Общий аналитический запрос с обращением к LLM
124     return await self._handle_general_ai_query(user_messag
    e, source_path, history)
125
126     def _is_client_resources_query(self, msg: str) -> bool:
127         """Проверить, относится ли запрос к ресурсам программ
    клиента или оценке Gemini."""
128         has_prog = any(w in msg for w in ["программ", "приложе
    н", "софт", "клиент", "пользовател", "запущенн"])
129         has_res = any(w in msg for w in ["мест", "ресурс", "за
    нимает", "жрет", "памят", "диск", "объем", "тяжел", "неизвестн
    ", "gemini", "нормальн"])
130         return (has_prog and has_res) or ("сколько места" in m
    sg) or ("неизвестная программа" in msg)
131
132     def _is_cpu_timeline_query(self, msg: str) -> bool:
133         """Проверить, относится ли запрос к графику ЦП по врем
    ени."""
134         keywords = ["график", "таймлайн", "timeline", "динамик
    ", "истори", "по времени"]
135         targets = ["цпу", "cpu", "процессор", "загрузк", "нагр
    узк", "частот"]
136         has_kw = any(k in msg for k in keywords)
137         has_target = any(t in msg for t in targets)
138         return (has_kw and has_target) or ("загрузк" in msg an

```

```

139 | d ("цпу" in msg or "cpu" in msg or "процессор" in msg))
140 |     def _is_power_query(self, msg: str) -> bool:
141 |         """Проверить, относится ли запрос к энергопотреблению
142 |         или круговой диаграмме мощности."""
143 |         keywords = [
144 |             "энерг",
145 |             "электроэнерг",
146 |             "потреблен",
147 |             "ватт",
148 |             "квт",
149 |             "мощност",
150 |             "кругов",
151 |             "диаграмм",
152 |             "pie",
153 |             "doughnut",
154 |             "power",
155 |         ]
156 |         return any(k in msg for k in keywords) and any(
157 |             t in msg for t in ["компьют", "пк", "систем", "потреблен", "суточн", "кругов", "ватт", "энерг", "power"]
158 |         )
159 |     def _is_top_processes_query(self, msg: str) -> bool:
160 |         """Проверить, относится ли запрос к ресурсоемким процессам."""
161 |         return "процесс" in msg and any(k in msg for k in ["топ", "нагруж", "жрет", "груз", "памят", "cpu", "цпу", "ресурс"])
162 |
163 |     async def _handle_cpu_timeline_query(
164 |         self, user_message: str, source_path: Optional[str] = None
165 |     ) -> Dict[str, Any]:
166 |         """Сформировать ответ для графика загрузки ЦП."""
167 |         timeline_data = self.query_engine.get_cpu_timeline(limit=120, db_path=source_path)
168 |         if timeline_data.get("status") != "ok":

```

```

169         return {
170             "status": "error",
171             "reply": f"␣ Не удалось извлечь данные по загр
узке ЦП: {timeline_data.get('message')}",
172             "chart": None,
173             "sql_queries": [timeline_data.get("sql", "")],
174             "metrics_summary": {},
175             "quick_followups": [
176                 "Посчитай суточное потребление энергии (кр
уговая диаграмма)",
177                 "Какие процессы больше всего нагружают ЦП?",
178             ],
179         }
180
181     stats = timeline_data.get("statistics", {})
182     avg_cpu = stats.get("avg_cpu_percent", 0.0)
183     max_cpu = stats.get("max_cpu_percent", 0.0)
184     points_count = stats.get("points_count", 0)
185
186     # Формирование базового отчета
187     explanation = (
188         f"### ␣ Анализ динамики загрузки и частоты процес
сора (CPU)\n\n"
189         f"Отрисован временной ряд по последним **{points_c
ount} точкам замера** телеметрии:\n\n"
190         f"- **Средняя загрузка ЦП:** `{avg_cpu}%`\n"
191         f"- **Пиковая нагрузка ЦП:** `{max_cpu}%`\n"
192         f"- **Минимальная загрузка:** `{stats.get('min_cpu
_percent', 0.0)}%\n"
193         f"- **Временной интервал:** с `{stats.get('start_t
ime', '')[:19]}` по `{stats.get('end_time', '')[:19]}`\n\n"
194     )
195
196     if max_cpu > 85.0:
197         explanation += "␣ **Внимание:** Зафиксированы пико
вые всплески нагрузки свыше 85%. Рекомендуется проверить фонов
ые задачи и процессы в топе.\n"

```

```

198         else:
199             explanation += "\n\n **Статус:** Нагрузка на процессо
р находится в пределах нормы, троттлинг не выражен.\n"
200
201         # Дополнение от LLM
202         if self.chat_model and hasattr(self.chat_model, "ask")
:
203             try:
204                 llm_prompt = (
205                     f"Пользователь спросил: '{user_message}'.\n
n"
206                     f"Данные выборки ЦП: Средняя={avg_cpu}%, М
аксимум={max_cpu}%, Всего точек={points_count}.\n"
207                     f"Дай краткий аналитический комментарий (2
-3 предложения) о профиле нагрузки и энергоэффективности проце
ссора."
208                 )
209                 ai_comment = await asyncio.wait_for(self.chat_
model.ask(llm_prompt), timeout=3.5)
210                 if ai_comment and len(ai_comment.strip()) > 10
:
211                     explanation += f"\n\n **AI-анализ:** {ai_c
omment.strip()}\n"
212             except Exception as ex:
213                 logger.debug(f"LLM ask fallback: {ex}")
214
215         return {
216             "status": "ok",
217             "reply": explanation,
218             "chart": timeline_data.get("chart"),
219             "sql_queries": [timeline_data.get("sql", "")],
220             "metrics_summary": {
221                 "avg_cpu_percent": f"{avg_cpu}%",
222                 "max_cpu_percent": f"{max_cpu}%",
223                 "points_analyzed": points_count,
224             },
225             "quick_followups": [
226                 "Посчитай суточное потребление энергии (кругов

```

```

    ая диаграмма)",
227         "Топ процессов по нагрузке на ЦП",
228         "Были ли критические перезагрузки системы?",
229     ],
230 }
231
232 async def _handle_power_consumption_query(
233     self, user_message: str, source_path: Optional[str] =
None
234 ) -> Dict[str, Any]:
235     """Сформировать ответ для расчета энергопотребления и
    суточной круговой диаграммы."""
236     power_data = self.query_engine.calculate_power_consump
tion_24h(db_path=source_path, hours=24)
237     if power_data.get("status") != "ok":
238         return {
239             "status": "error",
240             "reply": f"❌ Ошибка расчета энергопотребления:
    {power_data.get('message')}",
241             "chart": None,
242             "sql_queries": [],
243             "metrics_summary": {},
244             "quick_followups": ["Покажи график загрузки ЦП
    У по времени"],
245         }
246
247     total_kwh = power_data.get("total_kwh", 0.0)
248     total_wh = power_data.get("total_watt_hours", 0.0)
249     avg_watts = power_data.get("avg_system_power_watts", 0
    .0)
250     cpu_w = power_data.get("cpu_package_power_watts", 0.0)
251     gpu_w = power_data.get("gpu_power_watts", 0.0)
252     breakdown = power_data.get("breakdown", [])
253
254     table_rows = []
255     for item in breakdown:
256         table_rows.append(
257             f"| {item['component']} | {item['avg_power_wat

```

```

    ts']] Вт | {item['watt_hours']] Вт·ч | {item['kwh']] кВт·ч | *
    *{item['percent']]%** | "
258         )
259         table_md = "\n".join(table_rows)
260
261         explanation = (
262             f"### □ Расчет суточного энергопотребления компьют
ера и круговая диаграмма\n\n"
263             f"На основе телеметрии аппаратных датчиков мощност
и (`sensor_polls`) и загрузки компонентов за **24 часа**:\n\n"
264             f"- □ **Общее суточное потребление:** **`{total_k
wh} кВт·ч`** (`{total_wh} Вт·ч`)\n"
265             f"- □ **Средняя мощность всей системы:** **`{avg_
watts} Вт`**\n"
266             f"- □ **Мощность процессора (CPU Package):** `{cp
u_w} Вт`\n"
267             f"- □ **Мощность видеокарты (GPU):** `{gpu_w} Вт`
\n\n"
268             f"#### □ Детализация энергопотребления по узлам с
истемы:\n\n"
269             f"| Компонент / Подсистема | Средняя мощность | За
24ч (Вт·ч) | Энергия (кВт·ч) | Доля (%) | \n"
270             f"| :--- | :--- | :--- | :--- | :--- | \n"
271             f"{table_md}\n\n"
272         )
273
274         est_cost_rub = round(total_kwh * 5.80, 2)
275         est_cost_usd = round(total_kwh * 0.15, 2)
276         explanation += (
277             f"□ *Примерная стоимость электроэнергии за сутки:
* ~**{est_cost_rub} ₺** (или ~**${est_cost_usd}** при среднем
тарифе).\n"
278         )
279
280         if self.chat_model and hasattr(self.chat_model, "ask")
:
281             try:
282                 llm_prompt = (

```

```

283         f"Пользователь спросил: '{user_message}'.\n"
284         f"Суточное энергопотребление ПК: Всего={total_kwh} кВт*ч, Средняя мощность={avg_watts} Вт (CPU={cpu_w}W, GPU={gpu_w}W).\n"
285         f"Дай краткий совет (2 предложения) по оптимизации энергопотребления и профилю электропитания Windows."
286     )
287     ai_comment = await asyncio.wait_for(self.chat_model.ask(llm_prompt), timeout=3.5)
288     if ai_comment and len(ai_comment.strip()) > 10:
289         explanation += f"\n    **AI-рекомендация:**\n    {ai_comment.strip()}\n"
290     except Exception as ex:
291         logger.debug(f"LLM ask fallback: {ex}")
292
293     return {
294         "status": "ok",
295         "reply": explanation,
296         "chart": power_data.get("chart"),
297         "sql_queries": [power_data.get("sql", "")],
298         "metrics_summary": {
299             "total_kwh_24h": f"{total_kwh} кВт·ч",
300             "avg_power_watts": f"{avg_watts} Вт",
301             "cpu_power_watts": f"{cpu_w} Вт",
302             "gpu_power_watts": f"{gpu_w} Вт",
303         },
304         "quick_followups": [
305             "Покажи график загрузки ЦПУ по времени",
306             "Топ процессов по нагрузке на ЦП",
307             "Какая средняя температура видеокарты и процессора?",
308         ],
309     }
310
311     async def _handle_top_processes_query(
312         self, user_message: str, source_path: Optional[str] =

```

```

None
313     ) -> Dict[str, Any]:
314         """Сформировать ответ по топ-процессам."""
315         proc_data = self.query_engine.get_top_processes_summary(limit=10, db_path=source_path)
316         processes = proc_data.get("processes", [])
317
318         if not processes:
319             return {
320                 "status": "ok",
321                 "reply": "В исследуемой выборке не обнаружено записей о процессах.",
322                 "chart": None,
323                 "sql_queries": [proc_data.get("sql", "")],
324                 "metrics_summary": {},
325                 "quick_followups": ["Покажи график загрузки ЦП
У по времени"],
326             }
327
328         rows_md = []
329         for idx, p in enumerate(processes, 1):
330             rows_md.append(f"| {idx} | **{p.get('name')}** | {
p.get('avg_cpu', 0)}% | {p.get('max_cpu', 0)}% | {p.get('avg_r
am_mb', 0)} MB |")
331         table_md = "\n".join(rows_md)
332
333         explanation = (
334             f"### □ Топ ресурсоемких процессов по загрузке ЦП
и памяти\n\n"
335             f"Рейтинг сформирован на основе снимков процессов
(`process_snapshots` / `process_rollups`):\n\n"
336             f"| № | Процесс | Средний CPU | Пиковый CPU | Памя
ть RAM | \n"
337             f"| :- | :--- | :--- | :--- | :--- | \n"
338             f"{table_md}\n\n"
339         )
340
341         return {

```



```

342         "status": "ok",
343         "reply": explanation,
344         "chart": proc_data.get("chart"),
345         "sql_queries": [proc_data.get("sql", "")],
346         "metrics_summary": {
347             "top_process_name": processes[0].get("name") i
f processes else "-",
348             "top_process_cpu": f"{processes[0].get('avg_cp
u', 0)}%" if processes else "0%",
349         },
350         "quick_followups": [
351             "Покажи график загрузки ЦПУ по времени",
352             "Посчитай суточное потребление энергии (кругов
ая диаграмма)",
353         ],
354     }
355
356     async def _handle_client_resources_query(
357         self, user_message: str, source_path: Optional[str] =
None
358     ) -> Dict[str, Any]:
359         """Сформировать подробный отчет по ресурсам программ к
лиента и оценке неизвестных процессов в Gemini."""
360         summary = await self.client_analyzer.analyze_client_re
sources(
361             db_path=source_path, ask_gemini_for_unknown=True,
limit=20
362         )
363         progs = summary.programs
364
365         if not progs:
366             return {
367                 "status": "ok",
368                 "reply": "В текущей телеметрии не обнаружен
о активных программ, запущенных клиентом.",
369                 "chart": None,
370                 "sql_queries": [],
371                 "metrics_summary": {"total_programs": 0},

```

```

372 |         "quick_followups": ["Покажи график загрузки ЦП
    | У по времени"],
373 |     }
374 |
375 |     # Таблица распределения ресурсов
376 |     rows_md = []
377 |     for idx, p in enumerate(progs[:12], 1):
378 |         disk_str = f"{p.disk_size_mb} MB" if p.disk_size_m
    | b else "N/A"
379 |         status_badge = "🟢 Известная" if p.is_known_softwar
    | e else ("🟡 Неизвестная (Heavy)" if p.is_resource_heavy else "i
    | Неизвестная")
380 |         rows_md.append(
381 |             f"| {idx} | **{p.display_name or p.name}** | `
    | {p.name}` | {p.cpu_percent}% | {p.memory_mb} MB | {disk_str} |
    | {status_badge} |"
382 |         )
383 |     table_md = "\n".join(rows_md)
384 |
385 |     explanation = (
386 |         f"### 📊 Ресурсы программ, запущенных клиентом\n\n
    | "
387 |         f"Проведен детальный аудит запущенного клиентского
    | ПО и занимаемых ресурсов (память RAM, нагрузка CPU, файлы на
    | диске):\n\n"
388 |         f"- 📊 **Всего клиентских программ:** `{summary.to
    | tal_client_programs}`\n"
389 |         f"- 📊 **Суммарно занято памяти RAM:** **`${summary
    | .total_ram_mb} MB`** (~`${round(summary.total_ram_mb / 1024, 2)
    | } GB`)\n"
390 |         f"- 📊 **Суммарная нагрузка на ЦП:** **`${summary.to
    | tal_cpu_percent}%`**\n"
391 |         f"- 📊 **Общий объем файлов на диске:** `{summary.
    | total_disk_size_mb} MB`\n"
392 |         f"- 📊 **Неизвестных ресурсоемких программ:** **`${
    | summary.unknown_heavy_programs_count}`**\n\n"
393 |         f"#### 📋 Детализация по программам:\n\n"
394 |         f"| № | Программа | Процесс | CPU (%) | RAM (MB) |

```

```

Диск (файл) | Статус |\n"
395         f"| :- | :--- | :--- | :--- | :--- | :--- | :--- |
\n"
396         f"{table_md}\n\n"
397     )
398
399     # Вывод оценок от Gemini для неизвестных тяжелых прогр
амм
400     if summary.evaluations:
401         explanation += "### □ Экспертное заключение модел
и Gemini по неизвестным ресурсоемким программам:\n\n"
402         for ev in summary.evaluations:
403             norm_icon = "□ Штатно" if ev.is_normal else "□
□ Аномалия / Риск"
404             explanation += (
405                 f"#### □ Программа: `{ev.program_name}` (
PID: {ev.pid})\n"
406                 f"- **Вердикт Gemini:** `{ev.verdict}` (
{norm_icon}, Риск: `{ev.risk_level.upper()}`)\n"
407                 f"- **Потребление ресурсов:** CPU `{ev.cpu
_percent}%`, RAM `{ev.memory_mb} MB`, Диск `{ev.disk_io_mb_s}
MB/s`\n"
408                 f"- **Путь запуска:** `{ev.executable_path
or 'N/A'}`\n"
409                 f"- **Анализ:** {ev.analysis}\n"
410             )
411             if ev.recommendations:
412                 recs_str = "\n".join([f"  * {r}" for r in
ev.recommendations])
413                 explanation += f"- **Рекомендации:**\n{rec
s_str}\n\n"
414             else:
415                 explanation += "□ **Все ресурсоемкие программы кла
сифицированы как известные и доверенные.** Аномальных неизвес
тных процессов не обнаружено.\n"
416
417     # Chart configuration for top client programs RAM & CP
U

```

```

418 |         chart_labels = [p.display_name or p.name for p in prog
| s[:8]]
419 |         chart_ram = [p.memory_mb for p in progs[:8]]
420 |         chart = {
421 |             "type": "bar",
422 |             "title": "Потребление оперативной памяти RAM клиен
| тскими программами (МБ)",
423 |             "labels": chart_labels,
424 |             "datasets": [
425 |                 {
426 |                     "label": "Память RAM (МБ)",
427 |                     "data": chart_ram,
428 |                     "backgroundColor": "rgba(99, 102, 241, 0.7
| )",
429 |                     "borderColor": "#6366f1",
430 |                     "borderWidth": 1,
431 |                 }
432 |             ],
433 |         }
434 |
435 |         return {
436 |             "status": "ok",
437 |             "reply": explanation,
438 |             "chart": chart,
439 |             "sql_queries": [],
440 |             "metrics_summary": {
441 |                 "total_client_programs": summary.total_client_
| programs,
442 |                 "total_ram_mb": f"{summary.total_ram_mb} MB",
443 |                 "total_cpu_percent": f"{summary.total_cpu_perc
| ent}%",
444 |                 "unknown_heavy_count": summary.unknown_heavy_p
| rograms_count,
445 |             },
446 |             "quick_followups": [
447 |                 "Покажи график загрузки ЦПУ по времени",
448 |                 "Посчитай суточное потребление энергии (кругов
| ая диаграмма)",

```

```

449         "Топ процессов по нагрузке на ЦП",
450     ],
451 }
452
453 def _handle_direct_sql_query(
454     self, sql_query: str, source_path: Optional[str] = Non
455     e
456     ) -> Dict[str, Any]:
457     """Выполнить прямой SELECT SQL-запрос."""
458     res = self.query_engine.execute_safe_sql(sql_query, db
459     _path=source_path)
460     if res.get("status") != "ok":
461         return {
462             "status": "error",
463             "reply": f"❌ Ошибка выполнения SQL-запроса:\n`
464             ``\n{res.get('message')}\n``",
465             "chart": None,
466             "sql_queries": [res.get("sql", sql_query)],
467             "metrics_summary": {},
468             "quick_followups": ["Покажи график загрузки ЦП
469             У по времени"],
470         }
471
472     cols = res.get("columns", [])
473     rows = res.get("rows", [])
474     count = res.get("count", 0)
475
476     header_md = "| " + " | ".join(cols) + " |"
477     sep_md = "| " + " | ".join([":---" * len(cols)] + " |"
478     "
479
480     data_md = []
481     for r in rows[:15]:
482         vals = [str(r.get(c, "")) for c in cols]
483         data_md.append("| " + " | ".join(vals) + " |")
484
485     table_str = f"{header_md}\n{sep_md}\n" + "\n".join(dat
486     a_md)

```

```

481         explanation = (
482             f"### □ Результаты SQL-запроса к telemetry.db\n\n
483             f"Возвращено строк: **{count}**\n\n"
484             f"{table_str}\n\n"
485         )
486         if count > 15:
487             explanation += f"*Отображены первые 15 строк из {c
488 out}\n"
489         return {
490             "status": "ok",
491             "reply": explanation,
492             "chart": None,
493             "sql_queries": [res.get("sql", "")],
494             "metrics_summary": {"rows_count": count},
495             "quick_followups": [
496                 "Покажи график загрузки ЦПУ по времени",
497                 "Посчитай суточное потребление энергии (кругов
498 ая диаграмма)",
499             ],
500         }
501     async def _handle_general_ai_query(
502         self,
503         user_message: str,
504         source_path: Optional[str] = None,
505         history: Optional[List[Dict[str, Any]]] = None,
506     ) -> Dict[str, Any]:
507         """Обработать общий запрос через LLM с контекстом схем
508 ы базы данных."""
509         schema_info = self.query_engine.get_schema_summary(db_
510 path=source_path)
511         context_prompt = (
512             f"Пользователь задал вопрос о телеметрии Windows:
513             '{user_message}'.\n"
514             f"Доступные таблицы в telemetry.db: {list(schema_i

```

```

nfo.get('tables', {}).keys())}.\\n"
513         f"Ответь информативно и структурированно на русско
м языке. "
514         f"Если пользователю нужны графики, предложи ему го
товые фразы ('покажи график загрузки цпу по времени' или 'посч
итай потребление энергии')."
515     )
516
517     reply_text = ""
518     if self.chat_model and hasattr(self.chat_model, "ask")
:
519         try:
520             reply_text = await asyncio.wait_for(self.chat_
model.ask(context_prompt), timeout=4.0)
521         except Exception as ex:
522             logger.warning(f"Ошибка вызова LLM: {ex}")
523
524     if not reply_text:
525         reply_text = (
526             f"Я готов помочь вам с исследованием системной
телеметрии Windows!\\n\\n"
527             f"**Доступные команды и возможности:**\\n"
528             f"- □ **График ЦП:** введите *«покажи график
загрузки цпу по времени»*\\n"
529             f"- □ **Энергопотребление:** введите *«Посчита
й примерное общее потребление электроэнергии компьютером и пок
ажи суточную круговую диаграмму»*\\n"
530             f"- □ **Топ процессов:** введите *«топ процес
сов по сру»*\\n"
531             f"- □ **SQL-запросы:** введите любой запрос,
начинающийся с `SELECT ...` к базе `telemetry.db`"
532         )
533
534     return {
535         "status": "ok",
536         "reply": reply_text,
537         "chart": None,
538         "sql_queries": [],

```

```
539 |         "metrics_summary": {},
540 |         "quick_followups": [
541 |             "Покажи график загрузки цпу по времени",
542 |             "Посчитай суточное потребление энергии (кругов
    | ая диаграмма)",
543 |             "Топ процессов по нагрузке на ЦП",
544 |         ],
545 |     }
546 |
```


audit_startup_checker.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry_Research
4 # - Audit Startup Checker
5 # Description:
6 #     Проверка критических аудитов при запуске приложения.
7 #
8 # Usage Examples:
9 #     Python API:
10 #         from apps.windows.telemetry_research.audit_startup_checker import StartupAuditResult
11 #
12 #         service = StartupAuditResult()
13 #
14 # File: audit_startup_checker.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry_research
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-01 13:28:28
20 # =====
21
22 from __future__ import annotations
23 """Проверка критических аудитов при запуске приложения."""
24
25 import time
26 from dataclasses import dataclass, field
27 from typing import Any, Dict, List, Optional
28
29 try:
30     from logger import logger
31 except ImportError:
```

```

32 | import logging
33 | logger = logging.getLogger(__name__)
34 |
35 |
36 | @dataclass
37 | class StartupAuditResult:
38 |     """Результат проверки аудитов при запуске."""
39 |     is_healthy: bool
40 |     critical_count: int = 0
41 |     warning_count: int = 0
42 |     findings: List[Dict[str, Any]] = field(default_factory=list)
43 |     checks_passed: List[str] = field(default_factory=list)
44 |     duration_ms: float = 0.0
45 |
46 |     def to_dict(self) -> Dict[str, Any]:
47 |         return {
48 |             'is_healthy': self.is_healthy,
49 |             'critical_count': self.critical_count,
50 |             'warning_count': self.warning_count,
51 |             'findings': self.findings,
52 |             'checks_passed': self.checks_passed,
53 |             'duration_ms': self.duration_ms
54 |         }
55 |
56 |
57 | class AuditStartupChecker:
58 |     """Проверяет критические состояния системы при запуске."""
59 |
60 |     def __init__(self) -> None:
61 |         """Инициализирует чекер с ленивой загрузкой коллекторов. """
62 |         self._integrity_collector = None
63 |         self._performance_collector = None
64 |         self._driver_collector = None
65 |         self._eventlog_collector = None
66 |
67 |     def _get_integrity_collector(self):

```

```

68 |         """Ленивая загрузка IntegrityCollector."""
69 |         if self._integrity_collector is None:
70 |             from apps.windows.sdk.core.audits.integrity_collec
tor import IntegrityCollector
71 |             self._integrity_collector = IntegrityCollector()
72 |             return self._integrity_collector
73 |
74 |     def _get_performance_collector(self):
75 |         """Ленивая загрузка PerformanceCollector."""
76 |         if self._performance_collector is None:
77 |             from apps.windows.sdk.core.audits.performance_coll
ector import PerformanceCollector
78 |             self._performance_collector = PerformanceCollector
79 |             ()
80 |             return self._performance_collector
81 |
82 |     def _get_driver_collector(self):
83 |         """Ленивая загрузка DriverCollector."""
84 |         if self._driver_collector is None:
85 |             from apps.windows.sdk.core.audits.driver_collector
import DriverCollector
86 |             self._driver_collector = DriverCollector()
87 |             return self._driver_collector
88 |
89 |     def _get_eventlog_collector(self):
90 |         """Ленивая загрузка EventLogCollector."""
91 |         if self._eventlog_collector is None:
92 |             from apps.windows.sdk.core.audits.eventlog_collect
or import EventLogCollector
93 |             self._eventlog_collector = EventLogCollector()
94 |             return self._eventlog_collector
95 |
96 |     def check_startup_health(
97 |         self,
98 |         check_integrity: bool = True,
99 |         check_performance: bool = True,
100 |         check_drivers: bool = False,
    check_eventlog: bool = False,

```

```

101         eventlog_hours: int = 1
102     ) -> StartupAuditResult:
103         """Выполняет быструю проверку критических состояний при запуске.
104
105         Args:
106             check_integrity: Проверять pending reboot и целостность системы.
107             check_performance: Проверять критическую загрузку CPU/RAM.
108             check_drivers: Проверять проблемные драйверы (медленнее).
109             check_eventlog: Проверять критические события в логах (медленнее).
110             eventlog_hours: Сколько часов логов анализировать.
111
112         Returns:
113             StartupAuditResult с результатами проверки.
114         """
115         start_t = time.perf_counter()
116         result = StartupAuditResult(is_healthy=True)
117
118         # 1. Проверка целостности (pending reboot) - быстро
119         if check_integrity:
120             try:
121                 integrity = self._get_integrity_collector().collect()
122                 pending_reboot = integrity.metrics.get('pending_reboot', False)
123
124                 if pending_reboot:
125                     result.findings.append({
126                         'domain': 'integrity',
127                         'severity': 'warning',
128                         'title': 'Требуется перезагрузка системы',
129                         'description': 'Обнаружены отложенные операции, требующие перезагрузки'

```

```

130         })
131         result.warning_count += 1
132     else:
133         result.checks_passed.append('integrity: no
pending reboot')
134
135     except Exception as e:
136         logger.warning(f'Ошибка при проверке целостнос
ти: {e}')
137         result.checks_passed.append('integrity: check
skipped')
138
139     # 2. Проверка производительности - быстро
140     if check_performance:
141         try:
142             perf = self._get_performance_collector().colle
ct()
143             cpu_pct = perf.metrics.get('cpu_percent', 0.0)
144             mem_pct = perf.metrics.get('memory_percent', 0
.0)
145
146             if cpu_pct >= 90.0:
147                 result.findings.append({
148                     'domain': 'performance',
149                     'severity': 'critical',
150                     'title': f'Критическая загрузка CPU: {
cpu_pct:.1f}%',
151                     'description': 'Процессор загружен бол
ее 90% при запуске'
152                 })
153                 result.critical_count += 1
154                 result.is_healthy = False
155             elif cpu_pct >= 75.0:
156                 result.findings.append({
157                     'domain': 'performance',
158                     'severity': 'warning',
159                     'title': f'Повышенная загрузка CPU: {c
pu_pct:.1f}%',

```

```

160         'description': 'Процессор загружен бол
ее 75% при запуске'
161     })
162     result.warning_count += 1
163     else:
164         result.checks_passed.append(f'performance:
CPU {cpu_pct:.1f}%')
165
166         if mem_pct >= 90.0:
167             result.findings.append({
168                 'domain': 'performance',
169                 'severity': 'critical',
170                 'title': f'Критическая загрузка RAM: {
mem_pct:.1f}%',
171                 'description': 'Оперативная память зап
олнена более 90% при запуске'
172             })
173             result.critical_count += 1
174             result.is_healthy = False
175         elif mem_pct >= 80.0:
176             result.findings.append({
177                 'domain': 'performance',
178                 'severity': 'warning',
179                 'title': f'Высокая загрузка RAM: {mem_
pct:.1f}%',
180                 'description': 'Оперативная память зап
олнена более 80%'
181             })
182             result.warning_count += 1
183         else:
184             result.checks_passed.append(f'performance:
RAM {mem_pct:.1f}%')
185
186     except Exception as e:
187         logger.warning(f'Ошибка при проверке производи
тельности: {e}')
188         result.checks_passed.append('performance: chec
k skipped')

```

```

189 |
190 |     # 3. Проверка драйверов - опционально, медленнее
191 |     if check_drivers:
192 |         try:
193 |             drivers = self._get_driver_collector().collect
194 |             (
195 |                 problem_count = sum(
196 |                     1 for f in drivers.findings
197 |                     if f.severity.value in ('critical', 'cauti
198 | on'))
199 |                 )
200 |             if problem_count > 0:
201 |                 result.findings.append({
202 |                     'domain': 'drivers',
203 |                     'severity': 'warning',
204 |                     'title': f'Обнаружено проблем с драйве
205 | рами: {problem_count}',
206 |                     'description': 'Проверьте раздел драйв
207 | еров для деталей'
208 |                 })
209 |                 result.warning_count += 1
210 |             else:
211 |                 result.checks_passed.append('drivers: no p
212 | roblems')
213 |
214 |         except Exception as e:
215 |             logger.warning(f'Ошибка при проверке драйверов
216 | : {e}')
217 |             result.checks_passed.append('drivers: check sk
218 | ipped')
219 |
220 |     # 4. Проверка критических событий в логах - опциональн
221 | о, медленнее
222 |     if check_eventlog:
223 |         try:
224 |             events = self._get_eventlog_collector().collec
225 | t(hours=eventlog_hours)

```

```

218 |         critical_events = [
219 |             f for f in events.findings
220 |             if f.severity.value == 'critical'
221 |         ]
222 |
223 |         if critical_events:
224 |             result.findings.append({
225 |                 'domain': 'eventlog',
226 |                 'severity': 'warning',
227 |                 'title': f'Критические события в логах
: {len(critical_events)}',
228 |                 'description': 'Обнаружены критические
события за последние часы'
229 |             })
230 |             result.warning_count += 1
231 |         else:
232 |             result.checks_passed.append('eventlog: no
critical events')
233 |
234 |         except Exception as e:
235 |             logger.warning(f'Ошибка при проверке логов: {e
}')
236 |             result.checks_passed.append('eventlog: check s
kipped')
237 |
238 |         result.duration_ms = round((time.perf_counter() - star
t_t) * 1000, 2)
239 |
240 |         # Логируем результат
241 |         if result.critical_count > 0:
242 |             logger.error(
243 |                 f'Startup audit: CRITICAL issues found: {resul
t.critical_count}, '
244 |                 f'warnings: {result.warning_count}'
245 |             )
246 |         elif result.warning_count > 0:
247 |             logger.warning(
248 |                 f'Startup audit: warnings found: {result.warni

```



```

    ng_count}, '
249         f'checks passed: {len(result.checks_passed)}'
250     )
251     else:
252         logger.info(
253             f'Startup audit: all checks passed ({result.du
ration_ms}ms)'
254         )
255
256     return result
257
258     def check_pending_reboot_only(self) -> bool:
259         """Быстрая проверка только pending reboot.
260
261         Returns:
262             True если требуется перезагрузка, False иначе.
263         """
264         try:
265             integrity = self._get_integrity_collector().collec
t()
266             return integrity.metrics.get('pending_reboot', Fal
se)
267         except Exception as e:
268             logger.warning(f'Ошибка при проверке pending reboo
t: {e}')
269             return False
270
271
272     def run_startup_audit(
273         check_integrity: bool = True,
274         check_performance: bool = True,
275         check_drivers: bool = False,
276         check_eventlog: bool = False
277     ) -> StartupAuditResult:
278         """Утилитарная функция для запуска проверки из точки входа
279         .
280
281         Args:

```

```
281 |         check_integrity: Проверять pending reboot.
282 |         check_performance: Проверять CPU/RAM.
283 |         check_drivers: Проверять драйверы.
284 |         check_eventlog: Проверять логи событий.
285 |
286 |     Returns:
287 |         StartupAuditResult с результатами.
288 |     """
289 |     checker = AuditStartupChecker()
290 |     return checker.check_startup_health(
291 |         check_integrity=check_integrity,
292 |         check_performance=check_performance,
293 |         check_drivers=check_drivers,
294 |         check_eventlog=check_eventlog
295 |     )
296 |
```

charts.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry_Research
4 # - Charts
5 # Description:
6 #   Генератор графиков и HTML/SVG визуализаций телеметрии.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry_research.charts import Telem
11 #       etryChartGenerator
12 #
13 #       service = TelemetryChartGenerator()
14 #
15 # File: charts.py
16 # Project: ai-breadboard
17 # Package: apps.windows.telemetry_research
18 # Author: hypo69
19 # Copyright: © 2026 hypo69
20 # Updated: 2026-10-01 13:28:28
21 # =====
22
23 from __future__ import annotations
24
25 """Генератор графиков и HTML/SVG визуализаций телеметрии."""
26
27 import json
28
29 from datetime import datetime
30 from typing import Any, Dict, List, Optional, Tuple
31
32 from .models import ChartConfig, DeviceEventSummary, MetricSta
33 ts, TelemetryResearchReport, TimeSeriesDataset
```

```

31
32 class TelemetryChartGenerator:
33     """Генератор графиков и визуальных отчетов телеметрии."""
34
35     def generate_chart_configs(self, time_series: Dict[str, TimeSeriesDataset], report: TelemetryResearchReport) -> List[ChartConfig]:
36         """Сформировать стандартный набор конфигураций графиков для отчета.
37
38         Args:
39             time_series: Словарь извлеченных временных рядов.
40             report: Отчет исследования.
41
42         Returns:
43             List[ChartConfig]: Список спецификаций графиков.
44         """
45         charts: List[ChartConfig] = []
46         res_keys = [k for k in ("cpu_load", "ram_used_percent", "gpu_load") if k in time_series]
47         if res_keys:
48             labels, datasets = self._merge_series_for_chart(time_series, res_keys)
49             charts.append(
50                 ChartConfig(
51                     id="chart_resource_utilization",
52                     title="Утилизация ресурсов (CPU, RAM, GPU)",
53                     chart_type="line",
54                     labels=labels,
55                     datasets=datasets,
56                     y_axis_label="Загрузка (%)",
57                     description="Динамика потребления вычислительных ресурсов системы во времени.",
58                 )
59             )
60
61         temp_keys = [k for k in ("cpu_temp", "gpu_temp") if k

```

```

    in time_series]
62         if temp_keys:
63             labels, datasets = self._merge_series_for_chart(ti
me_series, temp_keys)
64             charts.append(
65                 ChartConfig(
66                     id="chart_temperatures",
67                     title="Температурный профиль оборудования"
68                     ,
69                     chart_type="line",
70                     labels=labels,
71                     datasets=datasets,
72                     y_axis_label="Температура (°C)",
73                     description="Мониторинг нагрева процессора
и графического ускорителя.",
74                 )
75             )
76         disk_keys = [k for k in ("disk_read_mb_s", "disk_write
_mb_s") if k in time_series]
77         if disk_keys:
78             labels, datasets = self._merge_series_for_chart(ti
me_series, disk_keys)
79             charts.append(
80                 ChartConfig(
81                     id="chart_disk_io",
82                     title="Пропускная способность дисковой под
системы",
83                     chart_type="line",
84                     labels=labels,
85                     datasets=datasets,
86                     y_axis_label="Скорость (MB/s)",
87                     description="Потоки чтения и записи на нак
опителях.",
88                 )
89             )
90
91         net_keys = [k for k in ("net_recv_mb_s", "net_sent_mb_

```



```

123         description="Структура активности и сбоев
| подключенного оборудования.",
124     )
125 )
126
127     if report.statistics:
128         stat_labels = list(report.statistics.keys())
129         avg_vals = [report.statistics[k].avg_val for k in
| stat_labels]
130         p95_vals = [report.statistics[k].p95_val for k in
| stat_labels]
131         max_vals = [report.statistics[k].max_val for k in
| stat_labels]
132         charts.append(
133             ChartConfig(
134                 id="chart_eda_stats",
135                 title="Сравнительный профиль метрик (Avg,
| p95, Max)",
136                 chart_type="bar",
137                 labels=stat_labels,
138                 datasets=[
139                     {"label": "Среднее (Avg)", "data": avg
| _vals, "backgroundColor": "#3B82F6"},
140                     {"label": "95-й перцентиль (p95)", "da
| ta": p95_vals, "backgroundColor": "#F59E0B"},
141                     {"label": "Максимум (Max)", "data": ma
| x_vals, "backgroundColor": "#EF4444"},
142                 ],
143                 y_axis_label="Значение",
144                 description="Сравнение типичных и пиковых
| нагрузок по ключевым показателям.",
145             )
146         )
147
148     return charts
149
150     def _merge_series_for_chart(
151         self, time_series: Dict[str, TimeSeriesDataset], keys:

```

```

    List[str], max_points: int = 150
152     ) -> Tuple[List[str], List[Dict[str, Any]]]:
153         """Свести несколько временных рядов в синхронизированн
    ые списки для графика."""
154         all_timestamps = set()
155         for k in keys:
156             for pt in time_series[k].points:
157                 all_timestamps.add(pt.timestamp)
158
159         sorted_ts = sorted(list(all_timestamps))
160         if len(sorted_ts) > max_points:
161             step = max(1, len(sorted_ts) // max_points)
162             sample_ts = sorted_ts[::step]
163         else:
164             sample_ts = sorted_ts
165
166         labels = [self._format_ts_label(t) for t in sample_ts]
167         datasets = []
168         for k in keys:
169             ds = time_series[k]
170             pt_dict = {p.timestamp: p.value for p in ds.points
    }
171             data_points = [pt_dict.get(t, None) for t in sampl
    e_ts]
172             datasets.append(
173                 {
174                     "label": f"{ds.name} ({ds.unit})",
175                     "data": data_points,
176                     "borderColor": ds.color or "#3B82F6",
177                     "backgroundColor": (ds.color or "#3B82F6")
    + "22",
178                     "fill": False,
179                     "tension": 0.3,
180                 }
181             )
182         return (labels, datasets)
183
184     @staticmethod

```



```

185     def _format_ts_label(iso_ts: str) -> str:
186         """Сформировать короткую человекочитаемую подпись врем
ени."""
187         try:
188             dt = datetime.fromisoformat(iso_ts.replace("Z", "+
00:00"))
189             return dt.strftime("%H:%M:%S")
190         except Exception:
191             return iso_ts.split("T")[-1][:8] if "T" in iso_ts
else iso_ts[-8:]
192
193     def render_svg_chart(self, chart: ChartConfig, width: int
= 800, height: int = 400) -> str:
194         """Сгенерировать автономный векторный SVG график.
195
196         Args:
197             chart: Конфигурация графика.
198             width: Ширина SVG холста в пикселях.
199             height: Высота SVG холста в пикселях.
200
201         Returns:
202             str: XML-строка с SVG графиком.
203         """
204         padding = 50
205         graph_w = width - 2 * padding
206         graph_h = height - 2 * padding
207
208         lines_svg = [
209             f'<rect width="{width}" height="{height}" fill="#1
E293B" rx="8" />',
210             f'<text x="{width // 2}" y="30" fill="#F8FAFC" fon
t-size="16" font-family="sans-serif" font-weight="bold" text-a
nchor="middle">{chart.title}</text>',
211         ]
212
213         all_vals = []
214         for ds in chart.datasets:
215             for v in ds.get("data", []):

```

```

216         if v is not None and isinstance(v, (int, float
    ))):
217             all_vals.append(v)
218
219         max_v = max(all_vals) if all_vals else 100.0
220         min_v = 0.0
221         val_range = max(1.0, max_v - min_v)
222
223         for i in range(5):
224             y_pos = padding + graph_h - int(i / 4.0 * graph_h)
225             grid_val = round(min_v + i / 4.0 * val_range, 1)
226             lines_svg.append(f'<line x1="{padding}" y1="{y_pos
    }" x2="{width - padding}" y2="{y_pos}" stroke="#334155" stroke
    -dasharray="4" />')
227             lines_svg.append(f'<text x="{padding - 10}" y="{y_
    pos + 4}" fill="#94A3B8" font-size="11" font-family="sans-seri
    f" text-anchor="end">{grid_val}</text>')
228
229         n_points = len(chart.labels)
230         if n_points > 1:
231             x_step = graph_w / (n_points - 1)
232             for ds in chart.datasets:
233                 color = ds.get("borderColor", "#38BDF8")
234                 points_str = []
235                 for idx, v in enumerate(ds.get("data", [])):
236                     if v is not None:
237                         x = padding + idx * x_step
238                         norm_y = (v - min_v) / val_range
239                         y = padding + graph_h - norm_y * graph
    _h
240                 points_str.append(f"{x:.1f},{y:.1f}")
241                 lines_svg.append(f'<circle cx="{x:.1f}
    " cy="{y:.1f}" r="3" fill="{color}" />')
242                 if points_str:
243                     polyline = " ".join(points_str)
244                     lines_svg.append(f'<polyline fill="none" s
    troke="{color}" stroke-width="2.5" points="{polyline}" />')
245

```

```

246         content = "\n".join(lines_svg)
247         return f'<svg xmlns="http://www.w3.org/2000/svg" viewB
ox="0 0 {width} {height}" width="{width}" height="{height}">\n
{content}\n</svg>'
248
249     def render_html_dashboard(self, report: TelemetryResearchR
eport) -> str:
250         """Сгенерировать автономный интерактивный HTML-дашборд
с графиками."""
251         charts_json = json.dumps([c.model_dump() for c in repo
rt.charts], ensure_ascii=False)
252         score_color = "#10B981" if report.health_score >= 85 e
lse "#F59E0B" if report.health_score >= 60 else "#EF4444"
253
254         anomalies_rows = ""
255         for a in report.anomalies:
256             badge_cls = "badge-danger" if a.severity == "criti
cal" else "badge-warning"
257             anomalies_rows += f"""
258                 <tr>
259                     <td><span class="badge {badge_cls}">{a.severit
y.upper()}</span></td>
260                     <td>{a.timestamp}</td>
261                     <td><strong>{a.metric}</strong></td>
262                     <td>{a.value} (норм: {a.threshold})</td>
263                     <td>{a.description}</td>
264                 </tr>
265             """
266
267         conclusions_list = "".join(f"<li>{c}</li>" for c in re
port.summary_conclusions)
268
269         return f"""<!DOCTYPE html>
270 <html lang="ru">
271 <head>
272     <meta charset="UTF-8">
273     <meta name="viewport" content="width=device-width, initial
-scale=1.0">

```

```
274 |     <title>AI-Breadboard: Исследование телеметрии #{report.rep
    | ort_id}</title>
275 |     <script src="https://cdn.jsdelivr.net/npm/chart.js"></scri
    | pt>
276 |     <style>
277 |         :root {{
278 |             --bg: #0F172A;
279 |             --surface: #1E293B;
280 |             --surface-hover: #334155;
281 |             --border: #475569;
282 |             --text-main: #F8F AFC;
283 |             --text-muted: #94A3B8;
284 |             --primary: #38BDF8;
285 |             --success: #10B981;
286 |             --warning: #F59E0B;
287 |             --danger: #EF4444;
288 |         }}
289 |         * {{ box-sizing: border-box; margin: 0; padding: 0; }}
290 |         body {{
291 |             font-family: -apple-system, BlinkMacSystemFont, 'S
    | egoe UI', Roboto, Helvetica, Arial, sans-serif;
292 |             background-color: var(--bg);
293 |             color: var(--text-main);
294 |             padding: 24px;
295 |             line-height: 1.5;
296 |         }}
297 |         .header {{
298 |             display: flex;
299 |             justify-content: space-between;
300 |             align-items: center;
301 |             border-bottom: 1px solid var(--border);
302 |             padding-bottom: 20px;
303 |             margin-bottom: 24px;
304 |         }}
305 |         .health-card {{
306 |             background: var(--surface);
307 |             padding: 16px 24px;
308 |             border-radius: 12px;
```

```
309 |         border-left: 6px solid {score_color};
310 |         text-align: center;
311 |     }}
312 |     .health-value {{
313 |         font-size: 36px;
314 |         font-weight: 800;
315 |         color: {score_color};
316 |     }}
317 |     .grid-cards {{
318 |         display: grid;
319 |         grid-template-columns: repeat(auto-fit, minmax(280
px, 1fr));
320 |         gap: 16px;
321 |         margin-bottom: 24px;
322 |     }}
323 |     .card {{
324 |         background: var(--surface);
325 |         border-radius: 12px;
326 |         padding: 20px;
327 |         border: 1px solid var(--border);
328 |     }}
329 |     .charts-grid {{
330 |         display: grid;
331 |         grid-template-columns: repeat(auto-fit, minmax(500
px, 1fr));
332 |         gap: 24px;
333 |         margin-bottom: 24px;
334 |     }}
335 |     .chart-container {{
336 |         background: var(--surface);
337 |         border-radius: 12px;
338 |         padding: 20px;
339 |         border: 1px solid var(--border);
340 |         position: relative;
341 |     }}
342 |     table {{
343 |         width: 100%;
344 |         border-collapse: collapse;
```

```
345         margin-top: 12px;
346     }}
347     th, td {{
348         padding: 10px 14px;
349         text-align: left;
350         border-bottom: 1px solid var(--border);
351         font-size: 14px;
352     }}
353     th {{ color: var(--text-muted); font-weight: 600; }}
354     .badge {{
355         display: inline-block;
356         padding: 4px 8px;
357         border-radius: 6px;
358         font-size: 12px;
359         font-weight: bold;
360     }}
361     .badge-danger {{ background: rgba(239, 68, 68, 0.2); c
362     olor: #F87171; }}
363     .badge-warning {{ background: rgba(245, 158, 11, 0.2);
364     color: #FBBF24; }}
365     ul {{ padding-left: 20px; }}
366     li {{ margin-bottom: 8px; }}
367 </style>
368 </head>
369 <body>
370     <div class="header">
371         <div>
372             <h1>Исследование телеметрии Windows</h1>
373             <p style="color: var(--text-muted);">Отчет: {repor
374             t.report_id} | Создан: {report.generated_at}</p>
375         </div>
376         <div class="health-card">
377             <div style="font-size: 13px; color: var(--text-mut
378             ed); text-transform: uppercase;">Индекс здоровья</div>
379             <div class="health-value">{report.health_score}%</
380             div>
381         </div>
382     </div>
```

```

378
379     <div class="grid-cards">
380         <div class="card">
381             <h3 style="color: var(--primary); margin-bottom: 8
382 px;">□ Объем выборки</h3>
383             <p><strong>Записей в логах:</strong> {report.recor
384 ds_analyzed}</p>
385             <p><strong>Период:</strong> {report.time_window_st
386 art or 'N/A'} – {report.time_window_end or 'N/A'}</p>
387         </div>
388         <div class="card">
389             <h3 style="color: var(--warning); margin-bottom: 8
390 px;">□ Аномалии и сбои</h3>
391             <p><strong>Обнаружено аномалий:</strong> {len(repo
392 rt.anomalies)}</p>
393             <p><strong>Ошибок устройств:</strong> {report.devi
394 ce_summary.error_count}</p>
395         </div>
396         <div class="card">
397             <h3 style="color: var(--success); margin-bottom: 8
398 px;">□ Выводы исследования</h3>
399             <ul>{conclusions_list}</ul>
400         </div>
401     </div>
402
403     <h2 style="margin-bottom: 16px;">□ Графики телеметрии</h2>
404 >
405     <div class="charts-grid" id="chartsContainer"></div>
406
407     <div class="card" style="margin-top: 24px;">
408         <h2 style="margin-bottom: 12px;">□ Зафиксированные аномалии и выбросы</h2>
409         {f'<table><thead><tr><th>Уровень</th><th>Время</th><th>Метрика</th><th>Значение</th><th>Описание</th></tr></thead><tbody>{anomalies_rows}</tbody></table>' if report.anomalies else "<p style='color: var(--text-muted);'>Аномалий за исследуемый период не выявлено.</p>"}
410     </div>

```

```

403
404     <script>
405         const chartConfigs = {charts_json};
406         const container = document.getElementById('chartsConta
407 iner');
408         chartConfigs.forEach((cfg) => {{
409             const wrapper = document.createElement('div');
410             wrapper.className = 'chart-container';
411             wrapper.innerHTML = `
412                 <h3 style="margin-bottom: 4px;">${{cfg.title}}
413 </h3>
414                 <p style="color: var(--text-muted); font-size:
415 13px; margin-bottom: 16px;">${{cfg.description || ''}}</p>
416                 <canvas id="${{cfg.id}}"></canvas>
417             `;
418             container.appendChild(wrapper);
419             const ctx = document.getElementById(cfg.id).getCon
420 text('2d');
421             new Chart(ctx, {{
422                 type: cfg.chart_type,
423                 data: {{
424                     labels: cfg.labels,
425                     datasets: cfg.datasets
426                 }},
427                 options: {{
428                     responsive: true,
429                     maintainAspectRatio: true,
430                     plugins: {{
431                         legend: {{ labels: {{ color: '#F8FAFC'
432 }} }}
433                     }},
434                     scales: cfg.chart_type === 'doughnut' ? {{
435 }} : {{
436                     x: {{ ticks: {{ color: '#94A3B8' }} }, g
437 rid: {{ color: '#334155' }} }},
438                     y: {{ ticks: {{ color: '#94A3B8' }} }, g

```



```
rid: {{ color: '#334155' }}, title: {{ display: true, text: cf
g.y_axis_label, color: '#94A3B8' }} }}
434 |         }}
435 |         }}
436 |     }));
437 |     });
438 | </script>
439 </body>
440 </html>
441 ""
442
```

cli.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry_Research
4 # - Cli
5 # Description:
6 #   CLI интерфейс для исследования логов телеметрии и построения графиков.
7 #
8 # Usage Examples:
9 #   CLI:
10 #       python -m apps.windows.telemetry_research.cli
11 #   Python API:
12 #       from apps.windows.telemetry_research.cli import parse_args
13 #
14 #       res = parse_args()
15 #
16 # File: cli.py
17 # Project: ai-breadboard
18 # Package: apps.windows.telemetry_research
19 # Author: hypo69
20 # Copyright: © 2026 hypo69
21 # Updated: 2026-10-01 13:28:28
22 # =====
23
24 from __future__ import annotations
25 """CLI интерфейс для исследования логов телеметрии и построения графиков."""
26
27 import argparse
28 import json
29 import sys
```

```
30 from pathlib import Path
31
32 _root_dir = Path(__file__).resolve().parents[3]
33 if str(_root_dir) not in sys.path:
34     sys.path.insert(0, str(_root_dir))
35
36 try:
37     from logger import logger
38 except ImportError:
39     from logger import logger
40
41 from .analyzer import TelemetryResearcher
42 from .charts import TelemetryChartGenerator
43 from .models import ResearchScenarioRequest
44
45
46 def parse_args() -> argparse.Namespace:
47     """Парсинг аргументов командной строки."""
48     parser = argparse.ArgumentParser(
49         description="AI-Breadboard: Исследование логов телемет
50     рии и генерация графиков",
51         formatter_class=argparse.RawDescriptionHelpFormatter,
52     )
53     parser.add_argument(
54         "--source",
55         "-s",
56         type=str,
57         default=None,
58         help="Путь к файлу логов (.json, .jsonl, .db) или дире
59     ктории с логами телеметрии",
60     )
61     parser.add_argument(
62         "--output",
63         "-o",
64         type=str,
65         default=None,
66         help="Путь для сохранения результата (по умолчанию: te
67    lemetry_research_report.html)",
```

```

65 |     )
66 |     parser.add_argument(
67 |         "--format",
68 |         "-f",
69 |         choices=["html", "json", "svg", "summary"],
70 |         default="html",
71 |         help="Формат вывода: html, json, svg, summary",
72 |     )
73 |     return parser.parse_args()
74 |
75 |
76 | def main() -> int:
77 |     """Точка входа CLI."""
78 |     args = parse_args()
79 |     researcher = TelemetryResearcher()
80 |     chart_gen = TelemetryChartGenerator()
81 |
82 |     logger.info(
83 |         f"Запуск исследования логов телеметрии (источник: {arg
84 | s.source or 'системные директории по умолчанию'})..."
85 |     )
86 |
87 |     if args.format == "summary":
88 |         req = ResearchScenarioRequest(source_path=args.source)
89 |         deep_report = researcher.run_deep_research(scenario=req, chart_generator=chart_gen)
90 |         print("\n" + "=" * 70)
91 |         print(f"  Отчет исследования телеметрии: {deep_report
92 | .report_id}")
93 |         print("=" * 70)
94 |         print(f"Дата формирования: {deep_report.generated_at}"
95 | )
96 |         print(f"Проанализировано записей: {deep_report.base_re
97 | port.records_analyzed}")
98 |         print(f"Индекс здоровья системы: {deep_report.base_rep
99 | ort.health_score}/100")
100 |         print(f"Обнаружено аномалий: {len(deep_report.base_rep
101 | ort.anomalies)}")

```

```

96         print("\n--- Проверенные гипотезы ---")
97         for h in deep_report.hypotheses:
98             status_icon = "✅ [ПОДТВЕРЖДЕНА]" if h.confirmed else "❌ [В ПОРЯДКЕ]"
99             print(f"{status_icon} {h.title} (уверенность {h.confidence * 100:.0f}%)")
100             for ev in h.evidence:
101                 print(f"    • {ev}")
102         print("\n--- Рекомендации ---")
103         for rec in deep_report.actionable_recommendations:
104             print(f"    📌 {rec}")
105         print("=" * 70 + "\n")
106         return 0
107
108     report = researcher.analyze(args.source)
109     records = researcher.extractor.load_all_records(args.source)
110     ts_map = researcher._extract_time_series(records)
111     report.charts = chart_gen.generate_chart_configs(ts_map, report)
112
113     out_path_str = args.output or ("telemetry_research_report.html" if args.format == "html" else "report.json")
114     out_path = Path(out_path_str)
115     out_path.parent.mkdir(parents=True, exist_ok=True)
116
117     if args.format == "html":
118         html_content = chart_gen.render_html_dashboard(report)
119         out_path.write_text(html_content, encoding="utf-8")
120         print(f"✅ HTML дашборд с графиками сохранен в: {out_path.resolve()}")
121     elif args.format == "json":
122         json_content = json.dumps(report.model_dump(), ensure_ascii=False, indent=2)
123         out_path.write_text(json_content, encoding="utf-8")
124         print(f"✅ JSON отчет сохранен в: {out_path.resolve()}")
125     elif args.format == "svg":

```

```
126         for chart in report.charts:
127             svg_file = out_path.parent / f"{chart.id}.svg"
128             svg_content = chart_gen.render_svg_chart(chart)
129             svg_file.write_text(svg_content, encoding="utf-8")
130             print(f"    SVG график сохранен в: {svg_file.resolve  
    (())}")
131
132     print("\n--- Результаты исследования ---")
133     print(f"Проанализировано записей: {report.records_analyzed  
}")
134     print(f"Индекс здоровья системы (Health Score): {report.he  
alth_score}%")
135     print(f"Обнаружено аномалий: {len(report.anomalies)}")
136     print(f"Сгенерировано графиков: {len(report.charts)}")
137     for conclusion in report.summary_conclusions:
138         print(f"    • {conclusion}")
139
140     return 0
141
142
143 if __name__ == "__main__":
144     sys.exit(main())
145
```

client_resource_analyzer.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry_Research
4 # - Client Resource Analyzer
5 # =====
6 # Description:
7 #   Анализ ресурсов программ, запущенных клиентом (память, процессор, диск),
8 #   и экспертная оценка аномальных/неизвестных ресурсоемких процессов через модель Gemini.
9 #
10 # Usage Examples:
11 #   Python API:
12 #       from apps.windows.telemetry_research.client_resource_analyzer import ClientResourceAnalyzer
13 #       analyzer = ClientResourceAnalyzer()
14 #       summary = await analyzer.analyze_client_resources()
15 #
16 # File: client_resource_analyzer.py
17 # Project: ai-breadboard
18 # Package: apps.windows.telemetry_research
19 # Author: hypo69
20 # Copyright: © 2026 hypo69
21 # Updated: 2026-10-04 07:47:00
22 # =====
23
24 from __future__ import annotations
25 """Анализ ресурсов программ, запущенных клиентом, и оценка неизвестных процессов через Gemini."""
26
27 import asyncio
28 import json
```

```
29 import os
30 from datetime import datetime, timezone
31 from pathlib import Path
32 from typing import Any, Dict, List, Optional, Set, Tuple, Union
33
34 try:
35     from logger import logger
36 except ImportError:
37     import logging
38     logger = logging.getLogger(__name__)
39
40 from .models import ClientProgramResource, ClientResourceSummary, UnknownProgramEvaluation
41 from .query_engine import TelemetryQueryEngine
42
43 try:
44     from src.ai.orchestration.unified_chat import UnifiedChatModel
45 except Exception as ex:
46     logger.warning(f"Не удалось импортировать UnifiedChatModel: {ex}")
47     UnifiedChatModel = None
48
49 # Базовый список системных процессов ядра Windows (не являются программами клиента)
50 SYSTEM_PROCESS_NAMES: Set[str] = {
51     "system", "system idle process", "idle", "registry", "smss.exe", "csrss.exe",
52     "wininit.exe", "services.exe", "lsass.exe", "svchost.exe", "fontdrvhost.exe",
53     "dwm.exe", "spoolsv.exe", "conhost.exe", "sihost.exe", "taskhostw.exe",
54     "winlogon.exe", "memory compression", "werfault.exe", "wmiprvse.exe",
55     "wmiprvse.exe", "searchindexer.exe", "audiodg.exe", "dashost.exe",
56     "securityhealthservice.exe", "msmpeng.exe", "nissrv.exe",
```



```
"smartscreen.exe",
57     "lsiso.exe", "rundll32.exe", "dllhost.exe"
58 }
59
60 # Каталог известных доверенных клиентских программ
61 KNOWN_CLIENT_APPS: Dict[str, Dict[str, str]] = {
62     "chrome.exe": {"display_name": "Google Chrome", "publisher": "Google LLC", "category": "Браузер"},
63     "msedge.exe": {"display_name": "Microsoft Edge", "publisher": "Microsoft Corporation", "category": "Браузер"},
64     "firefox.exe": {"display_name": "Mozilla Firefox", "publisher": "Mozilla Foundation", "category": "Браузер"},
65     "brave.exe": {"display_name": "Brave Browser", "publisher": "Brave Software", "category": "Браузер"},
66     "opera.exe": {"display_name": "Opera Browser", "publisher": "Opera Norway", "category": "Браузер"},
67     "code.exe": {"display_name": "Visual Studio Code", "publisher": "Microsoft Corporation", "category": "Разработка"},
68     "devenv.exe": {"display_name": "Visual Studio", "publisher": "Microsoft Corporation", "category": "Разработка"},
69     "pycharm64.exe": {"display_name": "PyCharm", "publisher": "JetBrains s.r.o.", "category": "Разработка"},
70     "idea64.exe": {"display_name": "IntelliJ IDEA", "publisher": "JetBrains s.r.o.", "category": "Разработка"},
71     "telegram.exe": {"display_name": "Telegram Desktop", "publisher": "Telegram FZ-LLC", "category": "Мессенджер"},
72     "discord.exe": {"display_name": "Discord", "publisher": "Discord Inc.", "category": "Мессенджер"},
73     "slack.exe": {"display_name": "Slack", "publisher": "Slack Technologies", "category": "Мессенджер"},
74     "teams.exe": {"display_name": "Microsoft Teams", "publisher": "Microsoft Corporation", "category": "Мессенджер"},
75     "steam.exe": {"display_name": "Steam Client", "publisher": "Valve Corporation", "category": "Игры"},
76     "epicgameslauncher.exe": {"display_name": "Epic Games Launcher", "publisher": "Epic Games", "category": "Игры"},
77     "spotify.exe": {"display_name": "Spotify", "publisher": "Spotify AB", "category": "Медиа"},

```

```
78     "vlc.exe": {"display_name": "VLC Media Player", "publisher": "VideoLAN", "category": "Медиа"},
79     "explorer.exe": {"display_name": "Проводник Windows", "publisher": "Microsoft Corporation", "category": "Система"},
80     "powershell.exe": {"display_name": "PowerShell", "publisher": "Microsoft Corporation", "category": "Инструменты"},
81     "pwsh.exe": {"display_name": "PowerShell 7", "publisher": "Microsoft Corporation", "category": "Инструменты"},
82     "cmd.exe": {"display_name": "Командная строка", "publisher": "Microsoft Corporation", "category": "Инструменты"},
83     "git.exe": {"display_name": "Git", "publisher": "Software Freedom Conservancy", "category": "Разработка"},
84     "python.exe": {"display_name": "Python Interpreter", "publisher": "Python Software Foundation", "category": "Разработка"},
85     "py.exe": {"display_name": "Python Launcher", "publisher": "Python Software Foundation", "category": "Разработка"},
86     "node.exe": {"display_name": "Node.js Runtime", "publisher": "OpenJS Foundation", "category": "Разработка"},
87     "docker.exe": {"display_name": "Docker Desktop", "publisher": "Docker Inc.", "category": "Разработка"},
88     "postman.exe": {"display_name": "Postman", "publisher": "Postman Inc.", "category": "Разработка"},
89     "winrar.exe": {"display_name": "WinRAR", "publisher": "win.rar GmbH", "category": "Утилиты"},
90     "7z.exe": {"display_name": "7-Zip", "publisher": "Igor Pavlov", "category": "Утилиты"},
91     "7zfm.exe": {"display_name": "7-Zip File Manager", "publisher": "Igor Pavlov", "category": "Утилиты"},
92     "notepad.exe": {"display_name": "Блокнот", "publisher": "Microsoft Corporation", "category": "Утилиты"},
93     "notepad++.exe": {"display_name": "Notepad++", "publisher": "Don HO", "category": "Разработка"},
94 }
95
96
97 class ClientResourceAnalyzer:
98     """Анализатор потребления ресурсов клиентскими программами
```

```

    с диагностикой через Gemini."""
99
100     def __init__(
101         self,
102         query_engine: Optional[TelemetryQueryEngine] = None,
103         chat_model: Optional[Any] = None,
104         cpu_heavy_threshold: float = 15.0,
105         memory_heavy_mb_threshold: float = 500.0,
106         disk_io_heavy_mb_threshold: float = 10.0,
107     ) -> None:
108         """Инициализация анализатора клиентских ресурсов.
109
110         Args:
111             query_engine: Движок запросов к telemetry.db.
112             chat_model: Экземпляр модели AI (по умолчанию UnifiedChatModel с провайдером Gemini).
113             cpu_heavy_threshold: Порог загрузки процессора (%) для классификации как ресурсоемкий.
114             memory_heavy_mb_threshold: Порог памяти (МБ) для классификации как ресурсоемкий.
115             disk_io_heavy_mb_threshold: Порог дискового ввода-вывода (МБ/с).
116         """
117         self.query_engine = query_engine or TelemetryQueryEngine()
118         self._chat_model = chat_model
119         self.cpu_heavy_threshold = cpu_heavy_threshold
120         self.memory_heavy_mb_threshold = memory_heavy_mb_threshold
121         self.disk_io_heavy_mb_threshold = disk_io_heavy_mb_threshold
122         self._evaluations_cache: Dict[str, UnknownProgramEvaluation] = {}
123
124     @property
125     def chat_model(self) -> Any:
126         """Получить активный экземпляр языковой модели."""
127         if self._chat_model is None and UnifiedChatModel is no

```

```

t None:
128         try:
129             self._chat_model = UnifiedChatModel()
130         except Exception as ex:
131             logger.warning(f"Не удалось инициализировать U
UnifiedChatModel: {ex}")
132             self._chat_model = None
133         return self._chat_model
134
135     def is_client_program(self, proc_name: str, username: Opti
onal[str] = None, path: Optional[str] = None) -> bool:
136         """Определить, запущена ли программа клиентом (пользо
ателем), а не ядром системы.
137
138         Args:
139             proc_name: Имя исполняемого файла процесса.
140             username: Учетная запись владельца процесса.
141             path: Путь к исполняемому файлу.
142
143         Returns:
144             bool: True если процесс относится к пользовательск
им приложениям.
145         """
146         name_lower = (proc_name or "").strip().lower()
147         if not name_lower or name_lower in SYSTEM_PROCESS_NAME
S:
148             return False
149
150         # Если имя пользователя явно указывает на системные ак
каунты NT AUTHORITY
151         user_lower = (username or "").strip().lower()
152         if user_lower in {"nt authority\\system", "nt authorit
y\\local service", "nt authority\\network service", "system"}:
153             # Некоторые пользовательские программы могут запус
каться под SYSTEM как сервисы, но по умолчанию фильтруем
154             if name_lower not in KNOWN_CLIENT_APPS:
155                 return False
156

```

```

157         return True
158
159     def is_known_software(self, proc_name: str, path: Optional
160 [str] = None) -> bool:
161         """Проверить, является ли программа известным доверенн
162 ым ПО.
163
164         Args:
165             proc_name: Имя процесса.
166             path: Путь к файлу.
167
168         Returns:
169             bool: True если программа находится в базе известн
170 ых приложений или системных директорий.
171         """
172         name_lower = (proc_name or "").strip().lower()
173         if name_lower in KNOWN_CLIENT_APPS or name_lower in SY
174 STEM_PROCESS_NAMES:
175             return True
176
177         if path:
178             path_lower = path.lower()
179             # Программы из Program Files с известными вендорам
180 и
181             if ("program files" in path_lower or "windowsapps"
182 in path_lower) and not ("temp" in path_lower):
183                 return True
184
185         return False
186
187     def get_file_size_mb(self, path: Optional[str]) -> Optiona
188 l[float]:
189         """Получить размер исполняемого файла программы на дис
190 ке в МБ.
191
192         Args:
193             path: Путь к исполняемому файлу.
194

```

```

187 |         Returns:
188 |             Optional[float]: Размер файла в МБ или None если ф
айл недоступен.
189 |             """
190 |         if not path:
191 |             return None
192 |         try:
193 |             p = Path(path)
194 |             if p.exists() and p.is_file():
195 |                 return round(p.stat().st_size / (1024.0 * 1024
.0), 2)
196 |         except Exception:
197 |             pass
198 |         return None
199 |
200 |     def calculate_resource_score(self, cpu_pct: float, ram_mb:
float, disk_io_mbs: float) -> float:
201 |         """Рассчитать совокупный скор ресурсоемкости процесса
(0..100+).
202 |
203 |         Args:
204 |             cpu_pct: Загрузка ЦП в %.
205 |             ram_mb: Память в МБ.
206 |             disk_io_mbs: Дисковая активность в МБ/с.
207 |
208 |         Returns:
209 |             float: Индекс ресурсоемкости.
210 |             """
211 |         # Веса: 1% CPU = 1.0 балл, 100 МБ RAM = 1.0 балл, 5 МБ
/c Disk = 1.0 балл
212 |         score = (cpu_pct * 1.0) + (ram_mb / 100.0) + (disk_io_
mbs / 5.0)
213 |         return round(score, 2)
214 |
215 |     def is_heavy_resource_consumer(self, cpu_pct: float, ram_m
b: float, disk_io_mbs: float) -> bool:
216 |         """Определить, потребляет ли программа аномально много
системных ресурсов.

```

```

217 |
218 |     Args:
219 |         cpu_pct: Загрузка ЦП в %.
220 |         ram_mb: Память в МБ.
221 |         disk_io_mbs: Скорость диска в МБ/с.
222 |
223 |     Returns:
224 |         bool: True если процесс превышает установленные по
роговые значения.
225 |     """
226 |     if cpu_pct >= self.cpu_heavy_threshold:
227 |         return True
228 |     if ram_mb >= self.memory_heavy_mb_threshold:
229 |         return True
230 |     if disk_io_mbs >= self.disk_io_heavy_mb_threshold:
231 |         return True
232 |     return False
233 |
234 |     def load_raw_client_processes(self, db_path: Optional[Union[
str, Path]] = None, limit: int = 150) -> List[Dict[str, Any]]:
235 |         """Загрузить сырые данные о процессах из telemetry.db
и ЖИВОГО снимка.
236 |
237 |     Args:
238 |         db_path: Путь к базе данных SQLite.
239 |         limit: Лимит выборки записей.
240 |
241 |     Returns:
242 |         List[Dict[str, Any]]: Список словарей процессов.
243 |     """
244 |     target_db = self.query_engine.get_db_path(db_path)
245 |     raw_list: List[Dict[str, Any]] = []
246 |
247 |     if target_db and target_db.exists():
248 |         import sqlite3
249 |         try:
250 |             conn = sqlite3.connect(str(target_db), timeout

```

```

=5.0)
251         conn.row_factory = sqlite3.Row
252         cursor = conn.cursor()
253
254         # 1. Извлекаем последние снимки процессов из p
rocess_snapshots
255         cursor.execute("""
256             SELECT pid, name, cpu_percent, memory_mb,
memory_percent,
257             num_threads, num_handles, username,
read_bytes_sec, write_bytes_sec,
258             executable as executable_path, cmdl
ine as command_line
259             FROM process_snapshots
260             WHERE id IN (
261                 SELECT MAX(id) FROM process_snapshots
GROUP BY pid
262             )
263             ORDER BY cpu_percent DESC, memory_mb DESC
264             LIMIT ?
265             """, (limit,))
266         rows = cursor.fetchall()
267         for r in rows:
268             raw_list.append(dict(r))
269
270         # 2. Если таблица пуста, пробуем process_rollu
ps_daily / process_outliers
271         if not raw_list:
272             cursor.execute("""
273                 SELECT 0 as pid, name, avg_cpu_percent
as cpu_percent, avg_memory_mb as memory_mb,
274                 0.0 as memory_percent, 1 as num
_threads, 0 as num_handles,
275                 'Client' as username, 0.0 as re
ad_bytes_sec, 0.0 as write_bytes_sec,
276                 '' as executable_path, '' as co
mmand_line
277                 FROM process_rollups_daily

```



```

278         ORDER BY avg_cpu_percent DESC, avg_mem
    | ory_mb DESC
279         LIMIT ?
280         """ , (limit,))
281         for r in cursor.fetchall():
282             raw_list.append(dict(r))
283
284         conn.close()
285     except Exception as ex:
286         logger.warning(f"Ошибка чтения процессов из SQ
    | Lite базы {target_db}: {ex}")
287
288     # 3. Дополняем активными процессами через psutil при н
    | аличии
289     if not raw_list:
290         try:
291             import psutil
292             for p in psutil.process_iter(['pid', 'name', '
    | cpu_percent', 'memory_info', 'memory_percent', 'num_threads',
    | 'num_handles', 'username', 'exe', 'cmdline']):
293                 try:
294                     info = p.info
295                     mem_mb = (info.get('memory_info').rss
    | / (1024 * 1024)) if info.get('memory_info') else 0.0
296                     raw_list.append({
297                         'pid': info.get('pid') or 0,
298                         'name': info.get('name') or '',
299                         'cpu_percent': info.get('cpu_perce
    | nt') or 0.0,
300                         'memory_mb': round(mem_mb, 2),
301                         'memory_percent': round(info.get('
    | memory_percent') or 0.0, 2),
302                         'num_threads': info.get('num_threa
    | ds') or 0,
303                         'num_handles': info.get('num_handl
    | es') or 0,
304                         'username': info.get('username') o
    | r '',

```

```

305         'read_bytes_sec': 0.0,
306         'write_bytes_sec': 0.0,
307         'executable_path': info.get('exe')
    or '',
308         'command_line': ' '.join(info.get(
    'cmdline') or []),
309     })
310     except (psutil.NoSuchProcess, psutil.Access
    Denied):
311         continue
312     except Exception as ex:
313         logger.debug(f"psutil fallback недоступен: {ex
    }")
314
315     return raw_list
316
317     async def evaluate_program_with_gemini(self, program: Clie
    ntProgramResource) -> UnknownProgramEvaluation:
318         """Спросить у модели Gemini, насколько нормально такое
    потребление ресурсов для программы.
319
320         Args:
321             program: Объект с данными ресурсоемкости программы
    .
322
323         Returns:
324             UnknownProgramEvaluation: Структурированное заключ
    ение модели Gemini.
325         """
326         cache_key = f"{program.name}:{program.pid}:{round(prog
    ram.cpu_percent, -1)}:{round(program.memory_mb, -2)}"
327         if cache_key in self._evaluations_cache:
328             return self._evaluations_cache[cache_key]
329
330         disk_io = round((program.disk_read_bytes_sec + program
    .disk_write_bytes_sec) / (1024.0 * 1024.0), 2)
331         path_str = program.executable_path or "Путь не определ
    ен (запуск из памяти/скрипта)"

```

```
332 |         cmd_str = program.command_line or "Командная строка не
    | доступна"
333 |
334 |         prompt = (
335 |             f"Ты – ведущий эксперт по информационной безопасно
    | сти и оптимизации производительности Windows.\n"
336 |             f"Проанализируй запущенную на компьютере пользоват
    | еля программу и ответь, насколько нормально такое потребление
    | ресурсов:\n\n"
337 |             f"□  **Параметры программы:**\n"
338 |             f"- Имя процесса: `{program.name}` (PID: {program.
    | pid})\n"
339 |             f"- Путь к файлу: `{path_str}`\n"
340 |             f"- Аргументы запуска: `{cmd_str}`\n"
341 |             f"- Пользователь: `{program.username or 'Текущий к
    | лиент'}`\n"
342 |             f"- Загрузка ЦП (CPU): **{program.cpu_percent}%**\n
    | n"
343 |             f"- Использование RAM: **{program.memory_mb} МБ**
    | ({program.memory_percent}% от общей памяти)\n"
344 |             f"- Дисковый ввод-вывод (I/O): **{disk_io} МБ/с**\n
    | n"
345 |             f"- Размер исполняемого файла: **{program.disk_siz
    | e_mb or 'Неизвестен'}` МБ**\n"
346 |             f"- Поточков: {program.num_threads}, Дескрипторов:
    | {program.num_handles}\n\n"
347 |             f"Ответь строго в формате JSON со следующими полям
    | и:\n"
348 |             f"{{\n"
349 |             f'  "is_normal": true|false,\n'
350 |             f'  "confidence": 0.0-1.0,\n'
351 |             f'  "risk_level": "low"|"medium"|"high"|"critical"
    | ,\n'
352 |             f'  "verdict": "Краткий вывод на русском языке (до
    | 10 слов)",\n'
353 |             f'  "analysis": "Подробное объяснение на русском я
    | зыке: что это за программа, нормально ли для нее такое потребл
    | ение памяти/CPU/диска, может ли это быть майнер, утечка памяти
```

```

или вредоносная активность.",\n'
354         f'  "recommendations": ["Рекомендация 1", "Рекомен
дация 2"]\n'
355         f"}}"
356     )
357
358     model_name = "gemini-3.1-flash"
359     eval_result: Optional[UnknownProgramEvaluation] = None
360
361     if self.chat_model and hasattr(self.chat_model, "ask")
:
362         try:
363             raw_response = await asyncio.wait_for(self.cha
t_model.ask(prompt), timeout=6.0)
364             if raw_response:
365                 cleaned_json = raw_response.strip()
366                 if "```json" in cleaned_json:
367                     cleaned_json = cleaned_json.split("```
json")[1].split("```")[0].strip()
368                 elif "```" in cleaned_json:
369                     cleaned_json = cleaned_json.split("```
")[1].split("```")[0].strip()
370
371                 parsed = json.loads(cleaned_json)
372                 eval_result = UnknownProgramEvaluation(
373                     program_name=program.name,
374                     pid=program.pid,
375                     executable_path=program.executable_pat
h,
376                     command_line=program.command_line,
377                     cpu_percent=program.cpu_percent,
378                     memory_mb=program.memory_mb,
379                     disk_io_mb_s=disk_io,
380                     disk_size_mb=program.disk_size_mb,
381                     is_normal=bool(parsed.get("is_normal",
True)),
382                     confidence=float(parsed.get("confidenc
e", 0.85)),

```

```

383         risk_level=str(parsed.get("risk_level"
    , "medium"))),
384         verdict=str(parsed.get("verdict", "Ана
    лиз завершен")),
385         analysis=str(parsed.get("analysis", "Э
    кспертный анализ модели Gemini.")),
386         recommendations=list(parsed.get("recom
    mendations", [])),
387         evaluated_at=datetime.now(timezone.utc
    ).isoformat(),
388         model_used=model_name,
389     )
390     except Exception as ex:
391         logger.warning(f"Ошибка вызова Gemini для оцен
    ки программы {program.name}: {ex}")
392
393     # Детерминированный fallback если LLM недоступна
394     if eval_result is None:
395         is_suspicious_location = any(
396             loc in (program.executable_path or "").lower()
    for loc in ("\\temp\\", "\\appdata\\local\\temp", "\\users\\p
    ublic\\"))
397     )
398     is_high_risk = is_suspicious_location or (program.
    cpu_percent > 70.0 and not program.is_known_software)
399     risk = "high" if is_high_risk else ("medium" if no
    t program.is_known_software else "low")
400     verdict = "Подозрительная активность" if is_high_r
    isk else ("Штатная активность" if program.is_known_software el
    se "Неизвестное приложение")
401     analysis = (
402         f"Программа {program.name} потребляет {program
    .cpu_percent}% ЦП и {program.memory_mb} МБ RAM. "
403         + ("Запуск из временного каталога Temp требует
    повышенного внимания." if is_suspicious_location else "Рекоме
    ндуется проверить цифровую подпись исполняемого файла.")
404     )
405     recs = [

```

```

406 |         "Проверить контрольную сумму и происхождение ф
айла",
407 |         "При непрекращающейся высокой нагрузке заверши
ть процесс через диспетчер задач"
408 |     ]
409 |     eval_result = UnknownProgramEvaluation(
410 |         program_name=program.name,
411 |         pid=program.pid,
412 |         executable_path=program.executable_path,
413 |         command_line=program.command_line,
414 |         cpu_percent=program.cpu_percent,
415 |         memory_mb=program.memory_mb,
416 |         disk_io_mb_s=disk_io,
417 |         disk_size_mb=program.disk_size_mb,
418 |         is_normal=not is_high_risk,
419 |         confidence=0.75,
420 |         risk_level=risk,
421 |         verdict=verdict,
422 |         analysis=analysis,
423 |         recommendations=recs,
424 |         evaluated_at=datetime.now(timezone.utc).isofor
mat(),
425 |         model_used="deterministic-heuristics",
426 |     )
427 |
428 |     self._evaluations_cache[cache_key] = eval_result
429 |     return eval_result
430 |
431 |     async def analyze_client_resources(
432 |         self,
433 |         db_path: Optional[Union[str, Path]] = None,
434 |         ask_gemini_for_unknown: bool = True,
435 |         limit: int = 50,
436 |     ) -> ClientResourceSummary:
437 |         """Провести комплексный аудит ресурсов клиентских прог
рамм.
438 |
439 |         Args:

```

```
440         db_path: Путь к базе данных телеметрии.
441         ask_gemini_for_unknown: Опрашивать ли Gemini для н
известных ресурсоемких программ.
442         limit: Максимальное количество анализируемых прог
амм.
443
444     Returns:
445         ClientResourceSummary: Сводка по ресурсам всех про
грамм клиента и вердикты Gemini.
446     """
447     raw_procs = self.load_raw_client_processes(db_path=db_
path, limit=limit * 2)
448     client_programs: List[ClientProgramResource] = []
449     evaluations: List[UnknownProgramEvaluation] = []
450
451     seen_keys: Set[Tuple[str, int]] = set()
452
453     for raw in raw_procs:
454         p_name = raw.get("name") or "unknown.exe"
455         p_pid = int(raw.get("pid") or 0)
456         key = (p_name.lower(), p_pid)
457         if key in seen_keys:
458             continue
459         seen_keys.add(key)
460
461         path = raw.get("executable_path") or raw.get("path
")
462         username = raw.get("username")
463
464         # Проверяем, что это клиентская программа, а не си
стемная служба
465         if not self.is_client_program(p_name, username=use
rname, path=path):
466             continue
467
468         cpu_pct = float(raw.get("cpu_percent") or 0.0)
469         mem_mb = float(raw.get("memory_mb") or 0.0)
470         mem_pct = float(raw.get("memory_percent") or 0.0)
```

```
471         read_bytes = float(raw.get("read_bytes_sec") or 0.  
472         0)  
473         write_bytes = float(raw.get("write_bytes_sec") or  
474         0.0)  
475         disk_io_mbs = round((read_bytes + write_bytes) / (  
476         1024.0 * 1024.0), 2)  
477         disk_size = self.get_file_size_mb(path)  
478         is_known = self.is_known_software(p_name, path=pat  
479         h)  
480         is_heavy = self.is_heavy_resource_consumer(cpu_pct  
481         , mem_mb, disk_io_mbs)  
482         score = self.calculate_resource_score(cpu_pct, mem  
483         _mb, disk_io_mbs)  
484         known_meta = KNOWN_CLIENT_APPS.get(p_name.lower(),  
485         {})  
486         disp_name = known_meta.get("display_name") or p_na  
487         me  
488         publisher = known_meta.get("publisher")  
489         category = known_meta.get("category", "Клиентское  
490         приложение")  
491         prog = ClientProgramResource(  
492             pid=p_pid,  
493             name=p_name,  
494             display_name=disp_name,  
495             executable_path=path,  
496             command_line=raw.get("command_line"),  
497             username=username,  
498             cpu_percent=round(cpu_pct, 1),  
499             memory_mb=round(mem_mb, 1),  
500             memory_percent=round(mem_pct, 1),  
501             disk_read_bytes_sec=read_bytes,  
502             disk_write_bytes_sec=write_bytes,  
503             disk_size_mb=disk_size,  
504             num_threads=int(raw.get("num_threads") or 0),  
505             num_handles=int(raw.get("num_handles") or 0),
```



```

500         is_known_software=is_known,
501         is_client_program=True,
502         is_resource_heavy=is_heavy,
503         resource_score=score,
504         category=category,
505         publisher=publisher,
506     )
507
508     # Если программа неизвестная и ресурсоемкая – запра
509     # шиваем оценку Gemini
510     if is_heavy and not is_known and ask_gemini_for_un
511     known:
512         eval_item = await self.evaluate_program_with_g
513         emini(prog)
514         prog.gemini_evaluation = eval_item
515         evaluations.append(eval_item)
516
517         client_programs.append(prog)
518         if len(client_programs) >= limit:
519             break
520
521     # Сортировка по скорости ресурсоемкости (самые тяжелые в в
522     # ерху)
523     client_programs.sort(key=lambda p: p.resource_score, r
524     everse=True)
525
526     total_ram = round(sum(p.memory_mb for p in client_prog
527     rams), 1)
528     total_cpu = round(sum(p.cpu_percent for p in client_pr
529     ograms), 1)
530     total_disk = round(sum((p.disk_size_mb or 0.0) for p i
531     n client_programs), 1)
532     heavy_count = sum(1 for p in client_programs if p.is_r
533     esource_heavy)
534     unknown_heavy_count = sum(1 for p in client_programs i
535     f p.is_resource_heavy and not p.is_known_software)
536
537     return ClientResourceSummary(

```

```
528 |         total_client_programs=len(client_programs),
529 |         total_ram_mb=total_ram,
530 |         total_cpu_percent=total_cpu,
531 |         total_disk_size_mb=total_disk,
532 |         heavy_programs_count=heavy_count,
533 |         unknown_heavy_programs_count=unknown_heavy_count,
534 |         programs=client_programs,
535 |         evaluations=evaluations,
536 |         scanned_at=datetime.now(timezone.utc).isoformat(),
537 |     )
538 |
```

compactor.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry_Research
  # - Compactor
4 # =====
5 # Description:
6 #   Модуль логарифмического и многоуровневого сжатия (rollup)
  #   временных рядов
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry_research.compactor import Te
  #       lemetryCompactor
11 #
12 #       service = TelemetryCompactor()
13 #
14 # File: compactor.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry_research
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-01 13:28:28
20 # =====
21
22 from __future__ import annotations
23 """Модуль логарифмического и многоуровневого сжатия (rollup) в
  #   временных рядов"""
24
25 import math
26 from datetime import datetime, timezone
27 from typing import Any, Dict, List, Optional, Sequence, Tuple
28
29 from apps.windows.telemetry.models import ProcessMetricRollup,
```

SystemMetricRollup

```
30
31
32 def compute_percentile(values: Sequence[float], p: float = 95.
    0) -> float:
33     """Вычисляет перцентиль выборки методом линейной интерполя
        ции.
34
35     Args:
36         values: Последовательность числовых значений.
37         p: Желаемый перцентиль (0..100), по умолчанию 95.0.
38
39     Returns:
40         float: Значение перцентиля (0.0 при пустой выборке).
41     """
42     if not values:
43         return 0.0
44     sorted_vals = sorted(values)
45     n = len(sorted_vals)
46     if n == 1:
47         return float(sorted_vals[0])
48
49     p_clamped = max(0.0, min(100.0, p))
50     rank = (p_clamped / 100.0) * (n - 1)
51     lower = int(math.floor(rank))
52     upper = int(math.ceil(rank))
53     weight = rank - lower
54
55     return float(sorted_vals[lower] + weight * (sorted_vals[up
        per] - sorted_vals[lower]))
56
57
58 class TelemetryCompactor:
59     """Компаратор и агрегатор временных рядов телеметрии."""
60
61     @staticmethod
62     def get_tier_for_age(age_seconds: float) -> str:
63         """Определяет рекомендуемую гранулярность (tier) в зав
```

исимости от возраста данных.

```
64
65     Args:
66         age_seconds: Возраст данных в секундах.
67
68     Returns:
69         str: Наименование уровня агрегации ('1s', '2s', '5
70 s', '10s', '30s', '1m', '5m', '15m', '1h', '6h').
71         """
72     if age_seconds <= 60.0:
73         return "1s"
74     elif age_seconds <= 300.0: # 5 мин
75         return "2s"
76     elif age_seconds <= 900.0: # 15 мин
77         return "5s"
78     elif age_seconds <= 1800.0: # 30 мин
79         return "10s"
80     elif age_seconds <= 3600.0: # 1 час
81         return "30s"
82     elif age_seconds <= 10800.0: # 3 часа
83         return "1m"
84     elif age_seconds <= 43200.0: # 12 часов
85         return "5m"
86     elif age_seconds <= 86400.0: # 24 часа
87         return "15m"
88     elif age_seconds <= 604800.0: # 7 дней
89         return "1h"
90     else:
91         return "6h"
92
93     @staticmethod
94     def compact_system_metrics(
95         samples: List[Dict[str, Any]],
96         period_start: str,
97         period_end: str,
98         tier: str = "1m",
99     ) -> SystemMetricRollup:
100         """Агрегирует список сырых замеров хоста в один System
```

```

MetricRollup.
100
101     Args:
102         samples: Список словарей замеров (SystemSnapshot и
ли сжатых метрик).
103         period_start: ISO timestamp начала периода.
104         period_end: ISO timestamp окончания периода.
105         tier: Уровень гранулярности бакета.
106
107     Returns:
108         SystemMetricRollup: Сжатый бакет с расчетом avg, m
in, max, p95 и total.
109     """
110     count = len(samples)
111     if count == 0:
112         return SystemMetricRollup(
113             period_start=period_start,
114             period_end=period_end,
115             duration_seconds=0.0,
116             sample_count=0,
117             tier=tier,
118         )
119
120     # Вычисление длительности периода
121     duration_sec = 60.0
122     try:
123         t0 = datetime.fromisoformat(period_start)
124         t1 = datetime.fromisoformat(period_end)
125         duration_sec = max(1.0, (t1 - t0).total_seconds())
126     except Exception:
127         duration_sec = float(count)
128
129     cpu_vals: List[float] = []
130     ram_used_vals: List[float] = []
131     ram_pct_vals: List[float] = []
132     disk_read_rates: List[float] = []
133     disk_write_rates: List[float] = []
134     net_rx_rates: List[float] = []

```

```

135         net_tx_rates: List[float] = []
136
137         for s in samples:
138             # CPU
139             cpu_val = float(s.get("cpu_total_percent") or s.ge
140 t("cpu_percent") or 0.0)
141             cpu_vals.append(cpu_val)
142
143             # RAM
144             ram_used = float(s.get("memory_used_gb") or 0.0)
145             ram_pct = float(s.get("memory_percent") or 0.0)
146             ram_used_vals.append(ram_used)
147             ram_pct_vals.append(ram_pct)
148
149             # Disk (байты/сек переводим в МБ/с)
150             disk_r = float(s.get("disk_read_bytes_sec") or 0.0
151 ) / (1024.0 * 1024.0)
152             disk_w = float(s.get("disk_write_bytes_sec") or 0.
153 0) / (1024.0 * 1024.0)
154             disk_read_rates.append(disk_r)
155             disk_write_rates.append(disk_w)
156
157             # Net (байты/сек переводим в МБ/с)
158             net_rx = float(s.get("network_recv_bytes_sec") or
159 0.0) / (1024.0 * 1024.0)
160             net_tx = float(s.get("network_sent_bytes_sec") or
161 0.0) / (1024.0 * 1024.0)
162             net_rx_rates.append(net_rx)
163             net_tx_rates.append(net_tx)
164
165             # Расчет средних и тоталов (rate * dt per sample)
166             sample_dt = duration_sec / max(1, count)
167
168             cpu_avg = sum(cpu_vals) / count
169             cpu_min = min(cpu_vals)
170             cpu_max = max(cpu_vals)
171             cpu_p95 = compute_percentile(cpu_vals, 95.0)

```

```
168 |         ram_avg = sum(ram_used_vals) / count
169 |         ram_min = min(ram_used_vals)
170 |         ram_max = max(ram_used_vals)
171 |         ram_pct_avg = sum(ram_pct_vals) / count
172 |         ram_pct_max = max(ram_pct_vals)
173 |
174 |         disk_r_avg = sum(disk_read_rates) / count
175 |         disk_r_max = max(disk_read_rates)
176 |         disk_r_total = sum(disk_read_rates) * sample_dt
177 |
178 |         disk_w_avg = sum(disk_write_rates) / count
179 |         disk_w_max = max(disk_write_rates)
180 |         disk_w_total = sum(disk_write_rates) * sample_dt
181 |
182 |         net_rx_avg = sum(net_rx_rates) / count
183 |         net_rx_max = max(net_rx_rates)
184 |         net_rx_total = sum(net_rx_rates) * sample_dt
185 |
186 |         net_tx_avg = sum(net_tx_rates) / count
187 |         net_tx_max = max(net_tx_rates)
188 |         net_tx_total = sum(net_tx_rates) * sample_dt
189 |
190 |     return SystemMetricRollup(
191 |         period_start=period_start,
192 |         period_end=period_end,
193 |         duration_seconds=round(duration_sec, 2),
194 |         sample_count=count,
195 |         tier=tier,
196 |         cpu_avg=round(cpu_avg, 2),
197 |         cpu_min=round(cpu_min, 2),
198 |         cpu_max=round(cpu_max, 2),
199 |         cpu_p95=round(cpu_p95, 2),
200 |         ram_avg_gb=round(ram_avg, 3),
201 |         ram_min_gb=round(ram_min, 3),
202 |         ram_max_gb=round(ram_max, 3),
203 |         ram_percent_avg=round(ram_pct_avg, 2),
204 |         ram_percent_max=round(ram_pct_max, 2),
205 |         disk_read_avg_mbs=round(disk_r_avg, 3),
```



```

206         disk_read_max_mbs=round(disk_r_max, 3),
207         disk_read_total_mb=round(disk_r_total, 3),
208         disk_write_avg_mbs=round(disk_w_avg, 3),
209         disk_write_max_mbs=round(disk_w_max, 3),
210         disk_write_total_mb=round(disk_w_total, 3),
211         network_rx_avg_mbs=round(net_rx_avg, 3),
212         network_rx_max_mbs=round(net_rx_max, 3),
213         network_rx_total_mb=round(net_rx_total, 3),
214         network_tx_avg_mbs=round(net_tx_avg, 3),
215         network_tx_max_mbs=round(net_tx_max, 3),
216         network_tx_total_mb=round(net_tx_total, 3),
217     )
218
219     @staticmethod
220     def compact_process_metrics(
221         process_samples: List[Dict[str, Any]],
222         period_start: str,
223         period_end: str,
224     ) -> List[ProcessMetricRollup]:
225         """Группирует и агрегирует замеры процессов по имени/P
ID.
226
227         Args:
228             process_samples: Список замеров процессов.
229             period_start: Начало периода.
230             period_end: Конец периода.
231
232         Returns:
233             List[ProcessMetricRollup]: Агрегированные показате
ли каждого процесса.
234         """
235         duration_sec = 60.0
236         try:
237             t0 = datetime.fromisoformat(period_start)
238             t1 = datetime.fromisoformat(period_end)
239             duration_sec = max(1.0, (t1 - t0).total_seconds())
240         except Exception:
241             duration_sec = 60.0

```

```

242 |
243 |         # Группировка по (name, pid)
244 |         grouped: Dict[Tuple[str, Optional[int]], List[Dict[str
    | , Any]]] = {}
245 |         for p in process_samples:
246 |             key = (str(p.get("name", "unknown")), p.get("pid")
    | )
247 |             if key not in grouped:
248 |                 grouped[key] = []
249 |             grouped[key].append(p)
250 |
251 |         results: List[ProcessMetricRollup] = []
252 |         for (name, pid), p_list in grouped.items():
253 |             count = len(p_list)
254 |             cpu_vals = [float(item.get("cpu_percent") or 0.0)
    | for item in p_list]
255 |             ram_vals = [float(item.get("memory_mb") or 0.0) fo
    | r item in p_list]
256 |             read_bytes_sec = [float(item.get("read_bytes_sec")
    | or 0.0) for item in p_list]
257 |             write_bytes_sec = [float(item.get("write_bytes_sec
    | ") or 0.0) for item in p_list]
258 |             rx_bytes_sec = [float(item.get("rx_bytes_sec") or
    | 0.0) for item in p_list]
259 |             tx_bytes_sec = [float(item.get("tx_bytes_sec") or
    | 0.0) for item in p_list]
260 |
261 |             sample_dt = duration_sec / max(1, count)
262 |             path = p_list[0].get("path") or p_list[0].get("exe
    | ")
263 |
264 |             cpu_avg = sum(cpu_vals) / count
265 |             cpu_max = max(cpu_vals)
266 |             cpu_p95 = compute_percentile(cpu_vals, 95.0)
267 |
268 |             ram_avg = sum(ram_vals) / count
269 |             ram_max = max(ram_vals)
270 |

```

```
271 |         disk_r_tot = (sum(read_bytes_sec) * sample_dt) / (  
    | 1024.0 * 1024.0)  
272 |         disk_w_tot = (sum(write_bytes_sec) * sample_dt) /  
    | (1024.0 * 1024.0)  
273 |         net_rx_tot = (sum(rx_bytes_sec) * sample_dt) / (10  
    | 24.0 * 1024.0)  
274 |         net_tx_tot = (sum(tx_bytes_sec) * sample_dt) / (10  
    | 24.0 * 1024.0)  
275 |  
276 |         results.append(  
277 |             ProcessMetricRollup(  
278 |                 name=name,  
279 |                 pid=pid,  
280 |                 period_start=period_start,  
281 |                 period_end=period_end,  
282 |                 duration_seconds=round(duration_sec, 2),  
283 |                 sample_count=count,  
284 |                 path=path,  
285 |                 cpu_avg=round(cpu_avg, 2),  
286 |                 cpu_max=round(cpu_max, 2),  
287 |                 cpu_p95=round(cpu_p95, 2),  
288 |                 ram_avg_mb=round(ram_avg, 2),  
289 |                 ram_max_mb=round(ram_max, 2),  
290 |                 disk_read_total_mb=round(disk_r_tot, 3),  
291 |                 disk_write_total_mb=round(disk_w_tot, 3),  
292 |                 network_rx_total_mb=round(net_rx_tot, 3),  
293 |                 network_tx_total_mb=round(net_tx_tot, 3),  
294 |             )  
295 |         )  
296 |  
297 |     return results  
298 |
```

deep_diagnostics.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry_Research
4 # - Deep Diagnostics
5 # Description:
6 #   Модуль глубокой аппаратной и системной диагностики, поведе
7 #   нческой форензики и анализа утечек.
8 # Usage Examples:
9 #   Python API:
10 #   from apps.windows.telemetry_research.deep_diagnostics im
11 #   port LASTINPUTINFO
12 #   service = LASTINPUTINFO()
13 #
14 # File: deep_diagnostics.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry_research
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-08 04:11:30
20 # =====
21
22 from __future__ import annotations
23 """Модуль глубокой аппаратной и системной диагностики, поведе
24   нческой форензики и анализа утечек."""
25
26 import codecs
27 import ctypes
28 import ctypes.wintypes as wintypes
29 import os
30 import re
```

```
30 import struct
31 import subprocess
32 import sys
33 import time
34 from datetime import datetime, timezone, timedelta
35 from pathlib import Path
36 from typing import Any, Dict, List, Optional, Tuple
37
38 try:
39     import psutil
40     PSUTIL_AVAILABLE = True
41 except ImportError:
42     psutil = None
43     PSUTIL_AVAILABLE = False
44
45 try:
46     import winreg
47     WINREG_AVAILABLE = True
48 except ImportError:
49     winreg = None
50     WINREG_AVAILABLE = False
51
52 try:
53     from logger import logger
54 except ImportError:
55     from logger import logger
56
57 from apps.windows.telemetry.models import (
58     DefenderTelemetrySummary,
59     ExtendedSystemAuditReport,
60     ForensicsActivityReport,
61     KernelThrottlingReport,
62     PeripheralsNetworkReport,
63     ProcessLeakDiagnosticsReport,
64     ProcessLeakItem,
65     StartupTelemetrySummary,
66     StorageBatteryWearReport,
67     UserAccountsTelemetrySummary,
68     VssTelemetrySummary,
69 )
```

```

68
69 GR_GDIOBJECTS = 0
70 GR_USEROBJECTS = 1
71 PROCESS_QUERY_INFORMATION = 1024
72 PROCESS_VM_READ = 16
73
74
75 class LASTINPUTINFO(ctypes.Structure):
76     """Структура Win32 для GetLastInputInfo."""
77     _fields_ = [('cbSize', wintypes.UINT), ('dwTime', wintypes
78         .DWORD)]
79
80 class PROCESS_MEMORY_COUNTERS_EX(ctypes.Structure):
81     """Структура PSAPI для GetProcessMemoryInfo."""
82     _fields_ = [('cb', wintypes.DWORD), ('PageFaultCount', win
83         types.DWORD), ('PeakWorkingSetSize', ctypes.c_size_t), ('Worki
84         ngSetSize', ctypes.c_size_t), ('QuotaPeakPagedPoolUsage', ctyp
85         es.c_size_t), ('QuotaPagedPoolUsage', ctypes.c_size_t), ('Quot
86         aPeakNonPagedPoolUsage', ctypes.c_size_t), ('QuotaNonPagedPool
87         Usage', ctypes.c_size_t), ('PagefileUsage', ctypes.c_size_t),
88         ('PeakPagefileUsage', ctypes.c_size_t), ('PrivateUsage', ctype
89         s.c_size_t)]
90
91
92 class DeepDiagnosticsEngine:
93     """Движок глубокой системной диагностики и форензики."""
94
95     def __init__(self) -> None:
96         """Инициализация Win32 API интерфейсов."""
97         self._user32 = getattr(ctypes.windll, 'user32', None)
98         if os.name == 'nt' else None
99         self._kernel32 = getattr(ctypes.windll, 'kernel32', No
100         ne) if os.name == 'nt' else None
101         self._psapi = getattr(ctypes.windll, 'psapi', None) if
102         os.name == 'nt' else None
103
104     def collect_process_leaks(self, limit: int=100) -> Process

```

```

LeakDiagnosticsReport:
95     """Собрать метрики скрытых утечек процессов (Handles,
    GDI, USER, Page Faults).
96
97     Args:
98         limit: Максимальное количество процессов в общем с
    писке.
99
100    Returns:
101        ProcessLeakDiagnosticsReport: Отчет с ранжирование
    м подозрительных процессов.
102    """
103    if not PSUTIL_AVAILABLE:
104        return ProcessLeakDiagnosticsReport()
105    items: List[ProcessLeakItem] = []
106    suspicious_count = 0
107    for proc in psutil.process_iter(['pid', 'name', 'statu
    s', 'num_threads', 'cpu_percent', 'memory_info']):
108        try:
109            info = proc.info
110            pid = info.get('pid')
111            if not pid or pid <= 4:
112                continue
113            name = info.get('name') or 'unknown'
114            status = info.get('status') or 'running'
115            threads_count = info.get('num_threads') or 1
116            cpu_percent = info.get('cpu_percent') or 0.0
117            mem_info = info.get('memory_info')
118            memory_mb = round(mem_info.rss / (1024 * 1024)
    , 2) if mem_info else 0.0
119            handles_count = 0
120            gdi_objects = 0
121            user_objects = 0
122            page_faults = 0
123            peak_ws_mb = memory_mb
124            if self._kernel32 and self._user32:
125                h_process = self._kernel32.OpenProcess(PRO
    CESS_QUERY_INFORMATION | PROCESS_VM_READ, False, pid)

```

```

126         if h_process:
127             try:
128                 dw_handles = wintypes.DWORD()
129                 if self._kernel32.GetProcessHandle
Count(h_process, ctypes.byref(dw_handles)):
130                     handles_count = int(dw_handles
.value)
131                     gdi_objects = int(self._user32.Get
GuiResources(h_process, GR_GDIOBJECTS))
132                     user_objects = int(self._user32.Ge
tGuiResources(h_process, GR_USEROBJECTS))
133                     if self._psapi:
134                         counters = PROCESS_MEMORY_COUN
TERS_EX()
135                         counters.cb = ctypes.sizeof(PR
OCESS_MEMORY_COUNTERS_EX)
136                         if self._psapi.GetProcessMemor
yInfo(h_process, ctypes.byref(counters), counters.cb):
137                             page_faults = int(counters
.PageFaultCount)
138                             peak_ws_mb = round(counter
s.PeakWorkingSetSize / (1024 * 1024), 2)
139                             finally:
140                                 self._kernel32.CloseHandle(h_proce
ss)
141                     if handles_count == 0:
142                         try:
143                             handles_count = proc.num_handles() if
hasattr(proc, 'num_handles') else 0
144                         except Exception:
145                             handles_count = 0
146                     risk_reasons: List[str] = []
147                     risk_score = 'normal'
148                     if handles_count >= 4000:
149                         risk_score = 'critical'
150                         risk_reasons.append(f'Критическое число де
скрипторов: {handles_count}')
151                     elif handles_count >= 1500:

```



```
152         if risk_score != 'critical':
153             risk_score = 'warning'
154             risk_reasons.append(f'Высокое число дескрипторов: {handles_count}')
155         if gdi_objects >= 2000:
156             risk_score = 'critical'
157             risk_reasons.append(f'Угроза истощения GDI пула: {gdi_objects} (лимит 10 000)')
158         elif gdi_objects >= 800:
159             if risk_score != 'critical':
160                 risk_score = 'warning'
161                 risk_reasons.append(f'Повышенное использование GDI: {gdi_objects}')
162         if user_objects >= 500:
163             if risk_score != 'critical':
164                 risk_score = 'warning'
165                 risk_reasons.append(f'Много USER объектов: {user_objects}')
166         if page_faults >= 2000000:
167             risk_reasons.append(f'Высокая нагрузка на подкачку ({page_faults:,} Hard Faults)')
168         if memory_mb >= 2048:
169             risk_reasons.append(f'Высокое потребление памяти: {memory_mb:.1f} MB')
170         if risk_score != 'normal':
171             suspicious_count += 1
172         item = ProcessLeakItem(pid=pid, name=name, status=status, handles_count=handles_count, gdi_objects=gdi_objects, user_objects=user_objects, page_faults_total=page_faults, peak_working_set_mb=peak_ws_mb, memory_mb=memory_mb, threads_count=threads_count, cpu_percent=cpu_percent, leak_risk_score=risk_score, leak_risk_reasons=risk_reasons)
173         items.append(item)
174     except (psutil.NoSuchProcess, psutil.AccessDenied, psutil.ZombieProcess):
175         continue
176     except Exception as ex:
177         logger.debug(f'Ошибка сбора дескрипторов проце
```

```

cca: {ex}')
178         top_handles = sorted(items, key=lambda x: x.handles_co
unt, reverse=True)[:10]
179         top_gdi = sorted(items, key=lambda x: x.gdi_objects, r
everse=True)[:10]
180         top_page_faults = sorted(items, key=lambda x: x.page_f
aults_total, reverse=True)[:10]
181         risk_priority = {'critical': 0, 'warning': 1, 'normal'
: 2}
182         sorted_all = sorted(items, key=lambda x: (risk_priorit
y.get(x.leak_risk_score, 3), -x.handles_count))[:limit]
183         return ProcessLeakDiagnosticsReport(timestamp=datetime
.now(timezone.utc).isoformat(), total_processes=len(items), su
spicious_count=suspicious_count, top_handle_hogs=top_handles,
top_gdi_hogs=top_gdi, top_page_fault_hogs=top_page_faults, all
_processes=sorted_all)
184
185     def collect_forensics_activity(self) -> ForensicsActivityR
eport:
186         """Собрать данные активности пользователя, активного о
кна, доступа к сенсорам и UserAssist."""
187         fg_window: Dict[str, Any] = {'title': 'Рабочий стол /
Системный интерфейс', 'process_name': 'explorer.exe', 'pid': 0
}
188         idle_seconds = 0.0
189         if self._user32:
190             try:
191                 hwnd = self._user32.GetForegroundWindow()
192                 if hwnd:
193                     length = self._user32.GetWindowTextLengthW
(hwnd)
194                     buff = ctypes.create_unicode_buffer(length
+ 1)
195                     self._user32.GetWindowTextW(hwnd, buff, le
ngth + 1)
196                     title = buff.value.strip() or 'Без заголов
ка'
197                     lp_pid = wintypes.DWORD()

```

```

198         self._user32.GetWindowThreadProcessId(hwnd
    , ctypes.byref(lp_pid))
199         pid = int(lp_pid.value)
200         proc_name = 'unknown'
201         if PSUTIL_AVAILABLE and pid > 0:
202             try:
203                 proc_name = psutil.Process(pid).name()
204             except Exception:
205                 proc_name = f'PID {pid}'
206                 fg_window = {'title': title, 'process_name': proc_name, 'pid': pid}
207             except Exception as ex:
208                 logger.debug(f'Ошибка получения ForegroundWindow: {ex}')
209         if self._user32 and self._kernel32:
210             try:
211                 lii = LASTINPUTINFO()
212                 lii.cbSize = ctypes.sizeof(LASTINPUTINFO)
213                 if self._user32.GetLastInputInfo(ctypes.byref(lii)):
214                     tick_count = self._kernel32.GetTickCount()
215                     idle_ms = max(0, tick_count - lii.dwTime)
216                     idle_seconds = round(idle_ms / 1000.0, 1)
217             except Exception as ex:
218                 logger.debug(f'Ошибка расчета UserIdleTime: {ex}')
219         cam_apps, mic_apps = self._get_camera_mic_access()
220         userassist_apps = self._get_userassist_apps(limit=15)
221         return ForensicsActivityReport(timestamp=datetime.now(
    timezone.utc).isoformat(), foreground_window=fg_window, user_idle_seconds=idle_seconds, camera_active_apps=cam_apps, microphone_active_apps=mic_apps, userassist_top_apps=userassist_apps)
222
223     def _get_camera_mic_access(self) -> Tuple[List[Dict[str, Any]], List[Dict[str, Any]]]:
224         """Определить приложения, использующие камеру и микрофон через реестр Windows."""

```

```

225         cam_list: List[Dict[str, Any]] = []
226         mic_list: List[Dict[str, Any]] = []
227         if not WINREG_AVAILABLE:
228             return (cam_list, mic_list)
229         capabilities = [('webcam', cam_list), ('microphone', mic_list)]
230         base_path = 'Software\\Microsoft\\Windows\\CurrentVersion\\CapabilityAccessManager\\ConsentStore'
231         for cap_name, target_list in capabilities:
232             try:
233                 cap_key_path = f'{base_path}\\{cap_name}'
234                 with winreg.OpenKey(winreg.HKEY_CURRENT_USER,
235                                     cap_key_path, 0, winreg.KEY_READ) as key:
236                     num_subkeys, _, _ = winreg.QueryInfoKey(key)
237                     for i in range(num_subkeys):
238                         try:
239                             sub_name = winreg.EnumKey(key, i)
240                             if sub_name == 'NonPackaged':
241                                 non_pkg_path = f'{cap_key_path}\\NonPackaged'
242                                 with winreg.OpenKey(winreg.HKEY_CURRENT_USER, non_pkg_path, 0, winreg.KEY_READ) as np_key:
243                                     np_count, _, _ = winreg.QueryInfoKey(np_key)
244                                     for j in range(np_count):
245                                         try:
246                                             app_sub = winreg.EnumKey(np_key, j)
247                                             with winreg.OpenKey(np_key, app_sub, 0, winreg.KEY_READ) as app_key:
248                                                 stop_val, _ = winreg.QueryValueEx(app_key, 'LastUsedTimeStop')
249                                                 start_val, _ = winreg.QueryValueEx(app_key, 'LastUsedTimeStart')
250                                                 is_active = stop_val == 0 and start_val > 0
251                                                 clean_app_name = app_sub.replace(' ', '_').replace('-', '_')

```

```

    = app_sub.replace('#', '\\').split('\\')[-1]
251 |                                     target_list.ap
    pend({'app_name': clean_app_name, 'full_path': app_sub.replace
    ('#', '\\'), 'is_active_now': is_active, 'type': 'Win32 Execut
    able'})
252 |                                     except Exception:
253 |                                         continue
254 |                                     else:
255 |                                         with winreg.OpenKey(key, sub_n
    ame, 0, winreg.KEY_READ) as uwp_key:
256 |                                             try:
257 |                                                 stop_val, _ = winreg.Q
    ueryValueEx(uwp_key, 'LastUsedTimeStop')
258 |                                                 start_val, _ = winreg.
    QueryValueEx(uwp_key, 'LastUsedTimeStart')
259 |                                                 is_active = stop_val =
    = 0 and start_val > 0
260 |                                     target_list.append({'a
    pp_name': sub_name, 'full_path': sub_name, 'is_active_now': is
    _active, 'type': 'UWP / Windows App'})
261 |                                     except Exception:
262 |                                         continue
263 |                                     except Exception:
264 |                                         continue
265 |                                     except Exception as ex:
266 |                                         logger.debug(f'Ошибка проверки доступа к {cap_
    name}: {ex}')
267 |                                     return (cam_list, mic_list)
268 |
269 |     def _get_userassist_apps(self, limit: int=15) -> List[Dict
    [str, Any]]:
270 |         """Извлечь историю запусков и фокусное время из ключей
    UserAssist реестра (ROT13)."""
271 |         apps: List[Dict[str, Any]] = []
272 |         if not WINREG_AVAILABLE:
273 |             return apps
274 |         ua_path = 'Software\\Microsoft\\Windows\\CurrentVersio
    n\\Explorer\\UserAssist'

```

```

275         try:
276             with winreg.OpenKey(winreg.HKEY_CURRENT_USER, ua_p
ath, 0, winreg.KEY_READ) as ua_key:
277                 num_guids, _, _ = winreg.QueryInfoKey(ua_key)
278                 for i in range(num_guids):
279                     guid_name = winreg.EnumKey(ua_key, i)
280                     count_path = f'{ua_path}\\{guid_name}\\Cou
nt'
281                 try:
282                     with winreg.OpenKey(winreg.HKEY_CURREN
T_USER, count_path, 0, winreg.KEY_READ) as count_key:
283                         num_vals, _, _ = winreg.QueryInfoK
ey(count_key)
284                         for v_idx in range(num_vals):
285                             try:
286                                 val_name, val_data, val_ty
pe = winreg.EnumValue(count_key, v_idx)
287                                 decoded_name = codecs.deco
de(val_name, 'rot_13')
288                                 if decoded_name.startswith
('{') or not decoded_name.strip():
289                                     continue
290                                 run_count = 0
291                                 focus_seconds = 0
292                                 if val_type == winreg.REG_
BINARY and len(val_data) >= 72:
293                                     run_count = struct.unp
ack_from('<I', val_data, 4)[0]
294                                     focus_ms = struct.unpa
ck_from('<I', val_data, 12)[0]
295                                     focus_seconds = round(
focus_ms / 1000.0)
296                                     if run_count > 0:
297                                         app_display = Path(dec
oded_name).name if '\\\\' in decoded_name or '/' in decoded_name
else decoded_name
298                                         apps.append({'name': a
pp_display, 'path': decoded_name, 'run_count': run_count, 'foc

```

```

        us_seconds': focus_seconds, 'focus_formatted': f'{focus_seconds
s // 60} мин {focus_seconds % 60} с'})
299         except Exception:
300             continue
301         except Exception:
302             continue
303     except Exception as ex:
304         logger.debug(f'Ошибка UserAssist: {ex}')
305     apps.sort(key=lambda x: (x['run_count'], x['focus_seconds']), reverse=True)
306     return apps[:limit]
307
308     def collect_kernel_throttling(self) -> KernelThrottlingReport:
309         """Собрать данные по прерываниям DPC/ISR, троттлингу,
Uptime и сбоям."""
310         dpc_pct = 0.0
311         interrupt_pct = 0.0
312         uptime_sec = 0.0
313         uptime_str = '0 дн 0 ч'
314         thermal_throttling = False
315         power_throttling = False
316         bsod_list: List[Dict[str, Any]] = []
317         pcie_info: Dict[str, Any] = {'gpu_name': 'PCIe Device'
, 'current_link_speed': 'Gen 3/4', 'current_link_width': 'x16'
, 'status': 'Optimal'}
318         if PSUTIL_AVAILABLE:
319             try:
320                 boot_time = psutil.boot_time()
321                 uptime_sec = max(0.0, time.time() - boot_time)
322                 days = int(uptime_sec // 86400)
323                 hours = int(uptime_sec % 86400 // 3600)
324                 mins = int(uptime_sec % 3600 // 60)
325                 uptime_str = f'{days} дн {hours} ч {mins} мин'
326                 cpu_times = psutil.cpu_times_percent(interval=
0.1)
327                 dpc_pct = round(getattr(cpu_times, 'dpc', 0.0)
or 0.0, 2)

```

```

328         interrupt_pct = round(getattr(cpu_times, 'interrupt', 0.0) or 0.0, 2)
329     except Exception as ex:
330         logger.debug(f'Ошибка сбора cpu_times: {ex}')
331     dpc_status = 'optimal'
332     if dpc_pct > 5.0 or interrupt_pct > 3.0:
333         dpc_status = 'severe'
334     elif dpc_pct > 1.5 or interrupt_pct > 1.0:
335         dpc_status = 'elevated'
336     minidump_dir = Path(os.environ.get('SystemRoot', 'C:\\Windows')) / 'Minidump'
337     if minidump_dir.exists() and minidump_dir.is_dir():
338         try:
339             dumps = sorted(minidump_dir.glob('*.dmp'), key
340                             =lambda p: p.stat().st_mtime, reverse=True)[:5]
341             for d in dumps:
342                 mtime = datetime.fromtimestamp(d.stat().st_mtime, tz=timezone.utc).isoformat()
343                 bsod_list.append({'file_name': d.name, 'timestamp': mtime, 'size_kb': round(d.stat().st_size / 1024, 1),
344                                     'type': 'Crash Dump (BugCheck)'})
345             except Exception as ex:
346                 logger.debug(f'Ошибка проверки Minidump: {ex}')
347         )
348     try:
349         from apps.windows.telemetry.sensors import get_hardware_sensors
350         sensors = get_hardware_sensors()
351         for s in sensors:
352             name_l = s.name.lower()
353             if 'prochot' in name_l or 'thermal throttling'
354             in name_l:
355                 if s.value > 0:
356                     thermal_throttling = True
357             if 'power limit' in name_l or 'pl1' in name_l
358             or 'pl2' in name_l:
359                 if s.value > 0:
360                     power_throttling = True

```



```

356         except Exception:
357             pass
358         return KernelThrottlingReport(timestamp=datetime.now(t
imezone.utc).isoformat(), dpc_latency_pct=dpc_pct, interrupt_l
atency_pct=interrupt_pct, dpc_status=dpc_status, thermal_throt
tling_detected=thermal_throttling, power_limit_throttling_dete
cted=power_throttling, system_uptime_seconds=uptime_sec, uptim
e_formatted=uptime_str, last_bsod_crashes=bsod_list, gpu_pcie_
link=pcie_info)
359
360     def collect_storage_battery_wear(self) -> StorageBatteryWe
arReport:
361         """Собрать данные износа SSD/NVMe и батареи питания."""
362         disks: List[Dict[str, Any]] = []
363         battery_data: Dict[str, Any] = {'has_battery': False,
'design_capacity_mwh': 0.0, 'full_charge_capacity_mwh': 0.0, '
wear_level_pct': 0.0, 'cycle_count': 0, 'charge_rate_mw': 0.0,
'is_charging': False, 'percent': 0.0, 'power_source': 'AC Mai
ns (Стационарное питание)'}
364         try:
365             from apps.windows.sdk.modules.storage_manager.core
.windows_storage_sensor import WindowsStorageSensor
366             sensor = WindowsStorageSensor()
367             snapshot = sensor.collect_snapshot(sync=True)
368             raw_partitions = snapshot.get('sources', {}).get('
partitions', [])
369             raw_logical = snapshot.get('sources', {}).get('log
ical_disks', [])
370
371             # Извлечем сведения о логических дисках (буква том
а -> ФС и свободное место)
372             logical_info: Dict[str, Dict[str, Any]] = {}
373             for log_d in raw_logical:
374                 if isinstance(log_d, dict):
375                     ld_id = str(log_d.get('DeviceID') or '').u
pper().rstrip('\\')
376                     if ld_id:

```

```

377 |             logical_info[ld_id] = {
378 |                 'fs': log_d.get('FileSystem') or '
| NTFS',
379 |                 'free_bytes': int(log_d.get('FreeS
| pace') or 0),
380 |             }
381 |
382 |     part_map: Dict[str, List[str]] = {}
383 |     part_free_map: Dict[str, float] = {}
384 |     for p in raw_partitions:
385 |         if isinstance(p, dict):
386 |             dn = p.get('DiskNumber')
387 |             dl = p.get('DriveLetter')
388 |             if dn is not None and dl:
389 |                 dn_str = str(dn)
390 |                 letter = f'{dl}:' .upper()
391 |                 fs_name = logical_info.get(letter, {}).
| .get('fs', 'NTFS')
392 |                 part_map.setdefault(dn_str, []).append
| (f'{letter} [{fs_name}]')
393 |                 free_b = logical_info.get(letter, {}).
| get('free_bytes', 0)
394 |                 if free_b > 0:
395 |                     part_free_map[dn_str] = part_free_
| map.get(dn_str, 0.0) + (free_b / 1024 ** 3)
396 |
397 |     io_counters = psutil.disk_io_counters(perdisk=True
| ) if PSUTIL_AVAILABLE and hasattr(psutil, 'disk_io_counters')
| else {}
398 |     now_dt = datetime.now(timezone.utc)
399 |     physical_disks = sensor.get_physical_disks(sync=Tr
| ue)
400 |     if physical_disks:
401 |         for d in physical_disks:
402 |             dev_num_str = ''.join(filter(str.isdigit,
| d.device_id))
403 |             mounted = part_map.get(dev_num_str, [])
404 |             has_free = dev_num_str in part_free_map

```

```

405         free_total = part_free_map.get(dev_num_str
    , 0.0)
406         if not has_free and PSUTIL_AVAILABLE and m
ounted:
407             for part_label in mounted:
408                 m_letter = part_label.split()[0] i
f part_label else ''
409                 if m_letter and ':' in m_letter:
410                     try:
411                         u = psutil.disk_usage(f'{m
_letter}\\')
412                         free_total += (u.free / 10
24 ** 3)
413                         has_free = True
414                     except Exception:
415                         pass
416                 io_key = f'PhysicalDrive{dev_num_str}'
417                 io = io_counters.get(io_key)
418                 bytes_read = io.read_bytes if io else (d.l
ifetime_read_bytes or 0)
419                 bytes_written = io.write_bytes if io else
(d.lifetime_write_bytes or 0)
420                 read_count = io.read_count if io else (d.r
ead_errors_total or 0)
421                 write_count = io.write_count if io else (d
.write_errors_total or 0)
422                 poh = d.power_on_hours
423                 first_on_str = (now_dt - timedelta(hours=i
nt(poh))).strftime('%Y-%m-%d') if poh and poh > 0 else None
424                 health_pct = 100.0
425                 if d.wear_percentage is not None and d.wea
r_percentage > 0:
426                     health_pct = max(0.0, min(100.0, 100.0
- float(d.wear_percentage)))
427                     elif d.health_status and d.health_status.l
ower() in ('warning', 'caution'):
428                         health_pct = 70.0
429                     elif d.health_status and d.health_status.l

```

```

ower() in ('unhealthy', 'critical', 'failed'):
430         health_pct = 20.0
431         status_str = 'Healthy (SMART OK)'
432         if d.health_status and d.health_status not
in ('Healthy', 'OK', '0', 'PASSED'):
433             status_str = d.health_status
434             disks.append({
435                 'device_id': f'Disk {dev_num_str}' if
dev_num_str else d.device_id,
436                 'name': d.model or d.friendly_name or
'Физический накопитель',
437                 'model': d.model or d.friendly_name,
438                 'serial_number': d.serial_number if d.
serial_number and d.serial_number != 'N/A' else None,
439                 'bus_type': d.bus_type or 'Unknown',
440                 'media_type': d.media_type or 'SSD',
441                 'partitions': ', '.join(mounted) if mo
unted else '-',
442                 'total_gb': round(d.size_gb, 1),
443                 'free_gb': round(free_total, 1) if has
_free else None,
444                 'health_pct': round(health_pct, 1),
445                 'wear_level_pct': d.wear_percentage or
0.0,
446                 'power_on_hours': poh,
447                 'first_power_on': first_on_str,
448                 'bytes_written': bytes_written,
449                 'bytes_read': bytes_read,
450                 'read_count': read_count,
451                 'write_count': write_count,
452                 'temperature_c': d.temperature_c,
453                 'status': status_str,
454             })
455         disks.sort(key=lambda x: int(''.join(filter(st
r.isdigit, str(x.get('device_id', '0')))) or 0))
456     except Exception as ex:
457         logger.debug(f'Ошибка сбора физических дисков чере
з WindowsStorageSensor: {ex}')

```

```

458         if not disks and PSUTIL_AVAILABLE:
459             try:
460                 for part in psutil.disk_partitions(all=False):
461                     try:
462                         usage = psutil.disk_usage(part.mountpo
int)
463                         disks.append({'device_id': part.device
, 'name': f'Tom {part.device}', 'model': part.device, 'serial_
number': None, 'bus_type': part.fstype, 'media_type': 'Drive',
'partitions': part.mountpoint, 'total_gb': round(usage.total
/ 1024 ** 3, 1), 'free_gb': round(usage.free / 1024 ** 3, 1),
'health_pct': 100.0, 'wear_level_pct': 0.0, 'temperature_c': N
one, 'power_on_hours': None, 'status': 'Healthy (SMART OK)'})
464                     except Exception:
465                         continue
466             except Exception as ex:
467                 logger.debug(f'Ошибка сбора дисков (psutil fal
lback): {ex}')
468         if PSUTIL_AVAILABLE and hasattr(psutil, 'sensors_batte
ry'):
469             try:
470                 bat = psutil.sensors_battery()
471                 if bat is not None:
472                     battery_data['has_battery'] = True
473                     battery_data['percent'] = round(bat.percen
t, 1)
474                     battery_data['is_charging'] = bat.power_pl
ugged
475                     battery_data['power_source'] = 'AC Adapter
' if bat.power_plugged else 'Battery Discharge'
476                     battery_data['secsleft'] = bat.secsleft if
bat.secsleft != psutil.POWER_TIME_UNLIMITED else -1
477             except Exception as ex:
478                 logger.debug(f'Ошибка опроса батареи: {ex}')
479             if battery_data['has_battery']:
480                 try:
481                     ps_cmd = 'Get-CimInstance -Namespace root/wmi
-ClassName BatteryStaticData -ErrorAction SilentlyContinue | S

```

```

    elect-Object DesignedCapacity, FullChargeCapacity | ConvertTo-
    Json'
482         res = subprocess.run(f'powershell.exe -NoProfi
    le -NonInteractive -Command "{ps_cmd}"', capture_output=True,
    text=True, timeout=5, shell=True)
483         if res.returncode == 0 and res.stdout.strip():
484             import json
485             b_json = json.loads(res.stdout.strip())
486             if isinstance(b_json, dict):
487                 des = float(b_json.get('DesignedCapaci
    ty') or 0.0)
488                 full = float(b_json.get('FullChargeCap
    acity') or 0.0)
489                 if des > 0 and full > 0:
490                     battery_data['design_capacity_mwh'
    ] = des
491                     battery_data['full_charge_capacity
    _mwh'] = full
492                     wear = max(0.0, min(100.0, (1.0 -
    full / des) * 100.0))
493                     battery_data['wear_level_pct'] = r
    ound(wear, 1)
494             except Exception:
495                 pass
496             return StorageBatteryWearReport(timestamp=datetime.now
    (timezone.utc).isoformat(), disks_wear=disks, battery_wear=bat
    tery_data)
497
498     def collect_peripherals_network(self) -> PeripheralsNetwor
    kReport:
499         """Собрать данные по USB устройствам, беспроводной сет
    и Wi-Fi и аудиоустройствам."""
500         usb_devices: List[Dict[str, Any]] = []
501         wifi_data: Dict[str, Any] = {'is_connected': False, 's
    id': 'Не подключено', 'bssid': '--', 'signal_pct': 0, 'rssi_d
    bm': -100, 'channel': 0, 'radio_type': '--', 'auth_cipher': '-
    -'}
502         audio_endpoints: List[Dict[str, Any]] = []

```

```

503         try:
504             from apps.windows.telemetry.win32_ffi.setupapi import SetupAPI
505             pnp = SetupAPI()
506             devs = pnp.get_all_devices()
507             for d in devs:
508                 hw_id = d.hardware_id or ''
509                 if 'USB' in hw_id.upper() or 'USB' in (d.device_class or '').upper():
510                     vid_match = re.search('VID_([0-9A-Fa-f]{4})', hw_id)
511                     pid_match = re.search('PID_([0-9A-Fa-f]{4})', hw_id)
512                     vid = vid_match.group(1) if vid_match else '----'
513                     pid = pid_match.group(1) if pid_match else '----'
514                     usb_devices.append({'name': d.friendly_name or 'USB Устройство', 'device_id': d.device_instance_id, 'vendor_id': vid, 'product_id': pid, 'device_class': d.device_class, 'has_problem': d.has_problem, 'status': 'Error' if d.has_problem else 'Connected & Active'})
515             except Exception as ex:
516                 logger.debug(f'Ошибка сбора USB через SetupAPI: {ex}')
517         try:
518             res = subprocess.run('netsh wlan show interfaces', capture_output=True, text=True, timeout=4, shell=True)
519             if res.returncode == 0 and res.stdout:
520                 out = res.stdout
521                 state_match = re.search('State\s*:\s*connected|Состояние\s*:\s*подключено', out, re.IGNORECASE)
522                 if state_match:
523                     wifi_data['is_connected'] = True
524                     ssid_m = re.search('SSID\s*:\s*(.+)', out)
525                     bssid_m = re.search('BSSID\s*:\s*(.+)', out)

```

```

526 |             sig_m = re.search('Signal|Сигнал\\s*:\\s*(
|\\d+)%', out)
527 |             chan_m = re.search('Channel|Канал\\s*:\\s*(
|\\d+)', out)
528 |             radio_m = re.search('Radio type|Тип радиом
одуля\\s*:\\s*(.+)', out)
529 |             auth_m = re.search('Authentication|Проверк
а подлинности\\s*:\\s*(.+)', out)
530 |             if ssid_m:
531 |                 wifi_data['ssid'] = ssid_m.group(1).st
rip()
532 |             if bssid_m:
533 |                 wifi_data['bssid'] = bssid_m.group(1).
strip()
534 |             if sig_m:
535 |                 sig_pct = int(sig_m.group(1))
536 |                 wifi_data['signal_pct'] = sig_pct
537 |                 wifi_data['rssi_dbm'] = int(sig_pct /
2.0 - 100)
538 |             if chan_m:
539 |                 wifi_data['channel'] = int(chan_m.grou
p(1))
540 |             if radio_m:
541 |                 wifi_data['radio_type'] = radio_m.grou
p(1).strip()
542 |             if auth_m:
543 |                 wifi_data['auth_cipher'] = auth_m.grou
p(1).strip()
544 |         except Exception as ex:
545 |             logger.debug(f'Ошибка Wi-Fi netsh: {ex}')
546 |         try:
547 |             audio_endpoints.append({'name': 'Основное устройст
во воспроизведения (Default Playback)', 'type': 'Speakers / He
adphones', 'is_default': True, 'status': 'Active'})
548 |             audio_endpoints.append({'name': 'Основное устройст
во записи (Default Recording)', 'type': 'Microphone', 'is_defa
ult': True, 'status': 'Active'})
549 |         except Exception:

```



```

550         pass
551         return PeripheralsNetworkReport(timestamp=datetime.now
| (timezone.utc).isoformat(), usb_devices=usb_devices, wifi_tele
| metry=wifi_data, audio_endpoints=audio_endpoints)
552
553     def collect_defender_telemetry(self) -> DefenderTelemetryS
| ummary:
554         """Собрать сводку телеметрии Windows Defender (CFA, AS
| R, исключения, угрозы)."""
555         summary = DefenderTelemetrySummary()
556         try:
557             from apps.windows.sdk.modules.defender.core.cfa_ma
| nager import ControlledFolderAccessManager
558             cfa_mgr = ControlledFolderAccessManager()
559             cfa_info = cfa_mgr.get_cfa_status()
560             summary.cfa_enabled = bool(cfa_info.enabled)
561             summary.cfa_mode = str(cfa_info.mode.value if hasa
| ttr(cfa_info.mode, 'value') else cfa_info.mode)
562             summary.cfa_protected_folders_count = len(cfa_info
| .protected_folders)
563         except Exception as ex:
564             logger.debug(f'Ошибка сбора статуса CFA Defender:
| {ex}')
565
566         try:
567             from apps.windows.sdk.modules.defender.core.asr_ma
| nager import ASRManager
568             asr_mgr = ASRManager()
569             rules = asr_mgr.get_asr_rules()
570             summary.asr_rules_count = len(rules)
571             summary.asr_enabled_count = len([r for r in rules
| if getattr(r, 'state', None) and str(r.state).lower() in ('ena
| bled', 'block', 'audit', 'warn')])
572         except Exception as ex:
573             logger.debug(f'Ошибка сбора правил ASR Defender: {
| ex}')
574
575         try:

```

```

576         from apps.windows.sdk.modules.defender.core.exclusions_auditor import ExclusionsAuditor
577         excl_auditor = ExclusionsAuditor()
578         excl = excl_auditor.audit_exclusions()
579         paths = getattr(excl, 'path_exclusions', []) or []
580         processes = getattr(excl, 'process_exclusions', []) or []
581         summary.path_exclusions = [str(p.value if hasattr(p, 'value') else p) for p in paths]
582         summary.process_exclusions = [str(p.value if hasattr(p, 'value') else p) for p in processes]
583         summary.exclusions_count = len(summary.path_exclusions) + len(summary.process_exclusions)
584         except Exception as ex:
585             logger.debug(f'Ошибка сбора исключений Defender: {ex}')
586
587         try:
588             from apps.windows.sdk.modules.defender.core.threat_manager import ThreatManager
589             threat_mgr = ThreatManager()
590             threat_history = threat_mgr.get_threats_history(limit=50)
591             active_threats = [t for t in threat_history if str(getattr(t, 'status', '')).lower() == 'active']
592             summary.active_threats_count = len(active_threats)
593             summary.threat_history_count = len(threat_history)
594         except Exception as ex:
595             logger.debug(f'Ошибка сбора угроз Defender: {ex}')
596
597         return summary
598
599     def collect_startup_telemetry(self) -> StartupTelemetrySummary:
600         """Собрать срез точек автозапуска и персистентности программ."""
601         summary = StartupTelemetrySummary()
602         try:

```

```

603         from apps.windows.startup.core.scanner import Star
        tupScanner
604         scanner = StartupScanner()
605         entries = scanner.scan_all()
606         summary.total_entries = len(entries)
607         serialized_entries: List[Dict[str, Any]] = []
608         for e in entries:
609             loc_type = str(getattr(e, 'location_type', ''))
        )
610             if 'REGISTRY' in loc_type:
611                 summary.registry_run_count += 1
612             elif 'STARTUP_FOLDER' in loc_type:
613                 summary.startup_folders_count += 1
614             elif 'SCHEDULED_TASK' in loc_type:
615                 summary.scheduled_tasks_count += 1
616             elif 'SERVICE' in loc_type:
617                 summary.startup_services_count += 1
618
619             serialized_entries.append({
620                 'name': getattr(e, 'name', 'Unknown'),
621                 'command': getattr(e, 'command_line', ''),
622                 'location_type': loc_type,
623                 'is_enabled': getattr(e, 'is_enabled', True)
        e),
624                 'publisher': getattr(e, 'publisher', ''),
625             })
626             summary.entries = serialized_entries
627     except Exception as ex:
628         logger.debug(f'Ошибка сбора автозагрузки: {ex}')
629     return summary
630
631     def collect_vss_telemetry(self) -> VssTelemetrySummary:
632         """Собрать состояние теневых копий томов VSS."""
633         summary = VssTelemetrySummary()
634         try:
635             from apps.windows.sdk.modules.backup_manager.core.
        vss_manager import VssManager
636             vss = VssManager()

```

```

637         snapshots = vss.list_snapshots()
638         summary.total_snapshots_count = len(snapshots)
639         vols: List[str] = []
640         serialized_snaps: List[Dict[str, Any]] = []
641         for s in snapshots:
642             vol = getattr(s, 'original_volume', '')
643             if vol and vol not in vols:
644                 vols.append(vol)
645                 serialized_snaps.append({
646                     'snapshot_id': getattr(s, 'snapshot_id', '
647                     '),
648                     'original_volume': vol,
649                     'creation_time': getattr(s, 'creation_time
650                     ', None),
651                     'shadow_volume_path': getattr(s, 'shadow_v
652                     olume_path', None),
653                     })
654                 summary.protected_volumes = vols
655                 summary.snapshots = serialized_snaps
656                 if snapshots and hasattr(snapshots[-1], 'creation_
657                 time'):
658                     summary.latest_snapshot_time = snapshots[-1].c
659                     reation_time
660                 except Exception as ex:
661                     logger.debug(f'Ошибка сбора теневых копий VSS: {ex
662                     }')
663                 return summary
664
665     def collect_users_telemetry(self) -> UserAccountsTelemetry
666     Summary:
667         """Собрать аудит локальных пользователей и привилегий.
668         """
669         summary = UserAccountsTelemetrySummary()
670         try:
671             from apps.windows.sysadmin.src.user_collector impo
672             rt WindowsUserCollector
673             collector = WindowsUserCollector()
674             users = collector.get_all_users()

```

```

666         summary.total_users_count = len(users)
667         admins: List[str] = []
668         logged_in: List[str] = []
669         locked_count = 0
670         serialized_users: List[Dict[str, Any]] = []
671         for u in users:
672             uname = getattr(u, 'name', '')
673             is_admin = getattr(u, 'is_admin', False)
674             is_logged = getattr(u, 'is_logged_in', False)
675             is_locked = getattr(u, 'lockout_status', False
) or not getattr(u, 'enabled', True)
676             if is_admin:
677                 admins.append(uname)
678             if is_logged:
679                 logged_in.append(uname)
680             if is_locked:
681                 locked_count += 1
682             serialized_users.append({
683                 'name': uname,
684                 'full_name': getattr(u, 'full_name', ''),
685                 'is_admin': is_admin,
686                 'is_logged_in': is_logged,
687                 'enabled': getattr(u, 'enabled', True),
688                 'account_type': getattr(u, 'account_type',
'Local'),
689                 'last_logon': getattr(u, 'last_logon', Non
e),
690             })
691         summary.admin_users_count = len(admins)
692         summary.admin_usernames = admins
693         summary.logged_in_users = logged_in
694         summary.locked_users_count = locked_count
695         summary.users = serialized_users
696     except Exception as ex:
697         logger.debug(f'Ошибка сбора локальных пользователе
й: {ex}')
698     return summary
699

```

```
700 |     def collect_extended_system_audit(self) -> ExtendedSystemA
      | uditReport:
701 |         """Собрать комплексный расширенный аудит безопасности,
      | автозагрузки, VSS и пользователей."""
702 |         import socket
703 |         return ExtendedSystemAuditReport(
704 |             timestamp=datetime.now(timezone.utc).isoformat(),
705 |             hostname=socket.gethostname(),
706 |             defender=self.collect_defender_telemetry(),
707 |             startup=self.collect_startup_telemetry(),
708 |             vss=self.collect_vss_telemetry(),
709 |             users=self.collect_users_telemetry(),
710 |         )
711 |
```

diagnostic_engine.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry_Research
4 # - Diagnostic Engine
5 # Description:
6 #    Универсальный диагностический движок и системный анализатор
7 #    телеметрии Windows.
8 # Usage Examples:
9 #    Python API:
10 #    from apps.windows.telemetry_research.diagnostic_engine i
11 #    mport DiagnosticEngine
12 #    service = DiagnosticEngine()
13 #
14 # File: diagnostic_engine.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry_research
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-01 13:28:28
20 # =====
21
22 from __future__ import annotations
23 """Универсальный диагностический движок и системный анализатор
24 телеметрии Windows."""
25
26 import json
27 from abc import ABC, abstractmethod
28 from datetime import datetime, timezone
29 from typing import Any, Dict, List, Optional, Tuple
```

```

30 try:
31     from logger import logger
32 except ImportError:
33     from logger import logger
34
35 from apps.windows.telemetry.models import AnomalyItem, SystemD
    iagnosticReport, SystemSnapshot, TelemetryIncident
36 from apps.windows.telemetry_research.grouped_telemetry import
    (
37     GroupDiagnosticResult,
38     GroupedTelemetryBuilder,
39     SynthesisDiagnosticResult,
40     TelemetryGroupInfo,
41 )
42 from apps.windows.telemetry_research.incident_detector import
    IncidentDetector
43
44
45 class DiagnosticEngine(ABC):
46     """Абстрактный базовый класс для диагностических движков т
    елеметрии."""
47
48     def __init__(self, chat_model: Optional[Any] = None) -> No
    ne:
49         self.chat_model = chat_model
50
51     @abstractmethod
52     def evaluate_heuristics(self, data: Any) -> Tuple[int, Lis
    t[AnomalyItem], List[str]]:
53         """Выполнение детерминированных эвристических проверок
    ."""
54         pass
55
56     def build_prompt(self, data: Any, score: int, anomalies: L
    ist[AnomalyItem], summary_text: str) -> str:
57         """Построение контекстного промпта для языковой модели
    ."""
58         return (

```



```

59         "Проанализируйте обнаруженные системные аномалии и
    метрики хоста:\n\n"
60         f"Аномалии: {[a.model_dump() for a in anomalies]}\n"
    n"
61         f"Текущий статус: {summary_text}\n\n"
62         "ТРЕБОВАНИЯ К ОТВЕТУ:\n"
63         "1. Немедленно предоставьте краткую диагностическую
    оценку (2-3 предложения) и 2-3 ключевые меры по устранению у
    зких мест.\n"
64         "2. Выведите готовый аналитический ответ сразу на
    русском языке без шаблонных вводных фраз."
65     )
66
67     async def diagnose(self, data: Any) -> SystemDiagnosticRep
    ort:
68         """Запуск эвристического анализа с последующим опциона
    льным LLM-синтезом."""
69         stages: List[Dict[str, Any]] = [
70             {
71                 "stage": "init",
72                 "title": "Сбор телеметрии",
73                 "message": "□ Получение актуального среза тел
    еметрии и сенсоров хоста...",
74                 "details": "WMI, LibreHardwareMonitor, системн
    ые процессы и ресурсы хранилища",
75             }
76         ]
77         system_instruction = (
78             "Вы – ведущий инженер по системной диагностике и а
    нализу производительности хоста. "
79             "Ваша задача: на основе переданных сенсоров и теле
    метрии немедленно предоставить краткую экспертную оценку и пра
    ктические рекомендации по оптимизации ресурсов. "
80             "Сразу выведите готовый аналитический ответ на рус
    ском языке."
81         )
82         model_name = "Heuristic Analyzer"
83         generated_prompt: Optional[str] = None

```

```

84 |         raw_response: Optional[str] = None
85 |         error_msg: Optional[str] = None
86 |
87 |         # 1. Эвристический анализ
88 |         try:
89 |             score, anomalies, recommendations = self.evaluate_
|heuristics(data)
90 |             summary_text = (
91 |                 f"Интегральный индекс здоровья: {score}/100. "
92 |                 f"Обнаружено аномалий: {len(anomalies)}. "
93 |                 + (f"Критические узкие места: {'', ' '.join([a.ti
|tle for a in anomalies if a.severity == 'critical'])}." if ano
|malies else "Система в стабильном состоянии.")
94 |             )
95 |         except Exception as ex:
96 |             logger.error(f"Ошибка эвристического анализа: {ex}
|")
97 |             score = 50
98 |             anomalies = [AnomalyItem(subsystem="Diagnostics",
|severity="warning", title="Evaluation Error", description=str(
|ex))]
99 |             recommendations = ["Проверьте доступность сенсоров
|и службы сбора телеметрии."]
100 |             summary_text = f"Ошибка оценки: {ex}"
101 |
102 |         # 2. Нейросетевой анализ при наличии активной модели
103 |         executor: Optional[Any] = self.chat_model
104 |         if executor is not None:
105 |             if hasattr(executor, "active_provider"):
106 |                 model_name = getattr(executor, "active_provide
|r", "Unified AI")
107 |             elif hasattr(executor, "model_id"):
108 |                 model_name = f"Gemini CLI ({getattr(executor,
|'model_id', 'gemini-3.1-flash-lite')})"
109 |             else:
110 |                 model_name = "AI Model"
111 |
112 |             prompt = self.build_prompt(data, score, anomalies,

```

```

summary_text)
113         generated_prompt = prompt
114         stages.append({
115             "stage": "prompt",
116             "title": "Формирование контекста сенсоров",
117             "message": "□ Передача состояния всех сенсоро
в и параметров в AI-модель...",
118             "details": "Телеметрия, датчики, накопители, п
роцессы и память",
119             "generated_prompt": prompt,
120             "system_instruction": system_instruction,
121         })
122
123         try:
124             if hasattr(executor, "ask"):
125                 response = await executor.ask(prompt, syst
em_instruction=system_instruction)
126             elif hasattr(executor, "chat"):
127                 response = await executor.chat(prompt)
128             elif callable(executor):
129                 response = await executor(prompt)
130             else:
131                 response = str(executor)
132
133             raw_response = str(response).strip()
134             if raw_response:
135                 summary_text = raw_response
136         except Exception as ai_err:
137             logger.warning(f"Ошибка вызова языковой модели
: {ai_err}")
138             error_msg = str(ai_err)
139
140         return SystemDiagnosticReport(
141             health_score=score,
142             summary=summary_text,
143             anomalies=anomalies,
144             recommendations=recommendations,
145             ai_model_used=model_name,

```

```

146         system_instruction=system_instruction,
147         generated_prompt=generated_prompt,
148         raw_response=raw_response,
149         error=error_msg,
150         stages=stages,
151     )
152
153
154 class SystemDiagnosticEngine(DiagnosticEngine):
155     """Диагностический движок для комплексного анализа телемет
156     рии хоста и сенсоров."""
157
158     def evaluate_heuristics(self, snapshot: SystemSnapshot) ->
159     Tuple[int, List[AnomalyItem], List[str]]:
160         """Выполнение детерминированного эвристического анализ
161         а сенсоров и телеметрии хоста.
162
163         Args:
164             snapshot: Актуальный срез телеметрии системы (Syst
165             emSnapshot).
166
167         Returns:
168             Tuple[int, List[AnomalyItem], List[str]]: Кортеж (
169             индекс_здоровья, список_аномалий, рекомендации).
170
171         """
172         score = 100
173         anomalies: List[AnomalyItem] = []
174         recommendations: List[str] = []
175
176         # ЦП
177         cpu = snapshot.cpu
178         if cpu and cpu.total_percent is not None:
179             if cpu.total_percent >= 90.0:
180                 score -= 25
181                 anomalies.append(AnomalyItem(subsystem="CPU",
182                 severity="critical", title="Critical CPU Saturation", descript
183                 ion=f"Overall CPU usage is at {cpu.total_percent}%, causing sy
184                 stem lag.))

```

```

176         recommendations.append("Inspect top CPU proces
ses and consider terminating unresponsive tasks.")
177         elif cpu.total_percent >= 75.0:
178             score -= 10
179             anomalies.append(AnomalyItem(subsystem="CPU",
severity="warning", title="Elevated CPU Load", description=f"C
PU load is elevated ({cpu.total_percent}%)."))
180
181         # Оперативная память
182         mem = snapshot.memory
183         if mem and mem.percent is not None:
184             if mem.percent >= 92.0:
185                 score -= 30
186                 anomalies.append(AnomalyItem(subsystem="RAM",
severity="critical", title="RAM Exhaustion Danger", descriptio
n=f"System RAM is {mem.percent}% full ({mem.used_gb}/{mem.tota
l_gb} GB)."))
187                 recommendations.append("Close memory-heavy app
lications or increase virtual memory paging size.")
188                 elif mem.percent >= 80.0:
189                     score -= 10
190                     anomalies.append(AnomalyItem(subsystem="RAM",
severity="warning", title="High Memory Pressure", description=
f"RAM utilization is at {mem.percent}%."))
191
192         # Дисковые разделы
193         for disk in snapshot.disks or []:
194             d_pct = getattr(disk, "percent", getattr(disk, "lo
ad_percent", 0.0))
195             if d_pct >= 95.0:
196                 score -= 15
197                 anomalies.append(AnomalyItem(subsystem="Disk",
severity="critical", title=f"Drive {disk.device} Almost Full"
, description=f"Volume {disk.mountpoint} is {d_pct}% full with
only {disk.free_gb} GB remaining."))
198                 recommendations.append(f"Free up disk space on
drive {disk.device} to prevent write failures.")
199                 elif d_pct >= 85.0:

```

```

200 |             score -= 5
201 |             anomalies.append(AnomalyItem(subsystem="Disk",
      |             severity="warning", title=f"Low Space on {disk.device}", desc
      |             ription=f"Volume {disk.mountpoint} is at {d_pct}% capacity."))
202 |
203 |             # Физические накопители и SMART
204 |             for p_disk in snapshot.physical_disks or []:
205 |                 if p_disk.health_status and p_disk.health_status.l
      |                 ower() in ["warning", "unhealthy", "error", "bad"]:
206 |                     score -= 25
207 |                     anomalies.append(AnomalyItem(subsystem="Storag
      |                     e", severity="critical", title=f"SMART Warning on {p_disk.mode
      |                     l}", description=f"Physical drive {getattr(p_disk, 'device_id'
      |                     , '')} ({p_disk.model}) reports health status: {p_disk.health_
      |                     status}."))
208 |                     recommendations.append(f"Perform disk backup a
      |                     nd replace failing drive {p_disk.model}.")
209 |                     if p_disk.temperature_celsius and p_disk.temperatu
      |                     re_celsius >= 60.0:
210 |                         score -= 10
211 |                         anomalies.append(AnomalyItem(subsystem="Storag
      |                         e", severity="warning", title=f"Overheating Storage: {p_disk.m
      |                         odel}", description=f"Drive temperature {p_disk.temperature_ce
      |                         lsius}°C exceeds recommended operating limits."))
212 |
213 |             # Температурные сенсоры LHM
214 |             for sensor in snapshot.sensors or []:
215 |                 if getattr(sensor, "category", "") == "temperature
      |                 " or getattr(sensor, "sensor_category", "") == "Temperatures":
216 |                     val = getattr(sensor, "value", 0.0)
217 |                     s_name = getattr(sensor, "name", getattr(senso
      |                     r, "sensor_name", "Sensor"))
218 |                     if val >= 85.0:
219 |                         score -= 20
220 |                         anomalies.append(AnomalyItem(subsystem="Th
      |                         ermals", severity="critical", title=f"High Temperature on {s_n
      |                         ame}", description=f"Sensor {s_name} reading {val}°C exceeds s
      |                         afe operating threshold."))

```

```

221         recommendations.append("Check chassis airf
low, fan speeds, and clean heatsink dust filters.")
222         elif val >= 75.0:
223             score -= 5
224             anomalies.append(AnomalyItem(subsystem="Th
ermals", severity="warning", title=f"Elevated Temperature on {
s_name}", description=f"Sensor {s_name} reading {val}°C is ele
vated."))
225
226         # Топ-процессы
227         ignored_processes = {"system idle process", "system",
"idle"}
228         for proc in (snapshot.top_processes or [])[:10]:
229             p_name_lower = (proc.name or "").strip().lower()
230             if proc.pid in [0, 4] or p_name_lower in ignored_p
rocesses:
231                 continue
232             if proc.cpu_percent and proc.cpu_percent >= 60.0:
233                 anomalies.append(AnomalyItem(subsystem="Proces
s", severity="warning", title=f"High CPU Consumer: {proc.name}
(PID {proc.pid})", description=f"Process {proc.name} is consu
ming {proc.cpu_percent:.1f}% CPU single-handedly."))
234             if proc.memory_mb and proc.memory_mb >= 6144.0:
235                 anomalies.append(AnomalyItem(subsystem="Proces
s", severity="warning", title=f"High Memory Consumer: {proc.na
me} (PID {proc.pid})", description=f"Process {proc.name} is us
ing {proc.memory_mb:.0f} MB RAM."))
236
237         # Стабильность ОС и обновления
238         if snapshot.alerts and getattr(snapshot.alerts, "reboo
t_pending", False):
239             anomalies.append(AnomalyItem(subsystem="Updates",
severity="warning", title="System Reboot Pending", description
="Windows Updates or system components require a restart."))
240             recommendations.append("Schedule a restart to fina
lize pending Windows system updates.")
241
242         score = max(0, min(100, score))

```

```

243         if not recommendations:
244             recommendations.append("System telemetry is within
healthy operating parameters.")
245         return (score, anomalies, recommendations)
246
247     def build_prompt(self, snapshot: Any, score: int, anomalie
s: List[AnomalyItem], summary_text: str) -> str:
248         """Построение обогащенного диагностического контекста
в формате JSON для языковой модели."""
249         if not isinstance(snapshot, SystemSnapshot):
250             return super().build_prompt(snapshot, score, anoma
lies, summary_text)
251
252         builder = GroupedTelemetryBuilder()
253         groups = builder.build_all_groups(snapshot)
254         groups_payload = {g.group_id: g.payload for g in group
s}
255
256         return (
257             "Вы – ведущий инженер по анализу системной телемет
рии и надежности Windows.\n"
258             "Ниже приведен структурированный срез телеметрии х
оста по 4 ключевым доменам:\n\n"
259             f"```\njson\n{json.dumps(groups_payload, ensure_asci
i=False, indent=2)}\n```\n\n"
260             f"Обнаруженные детерминированные аномалии: {[a.mod
el_dump() for a in anomalies]}\n"
261             f"Расчетный индекс здоровья (Heuristic Score): {sc
ore}/100\n\n"
262             "ИНСТРУКЦИИ:\n"
263             "1. Немедленно выдайте краткое аналитическое резюм
е состояния хоста (2-3 предложения).\n"
264             "2. Перечислите 2-4 конкретных практических шага п
о оптимизации и устранению найденных узких мест.\n"
265             "3. Отвечайте строго на русском языке."
266         )
267
268     async def diagnose_group(

```



```

269         self,
270         group_id: str = "",
271         payload: Optional[Dict[str, Any]] = None,
272         title: str = "",
273         group_info: Optional[TelemetryGroupInfo] = None,
274     ) -> GroupDiagnosticResult:
275         """Диагностика одной изолированной группы телеметрии с
помощью эвристики и LLM."""
276         gid = group_id or (group_info.group_id if group_info e
lse "general")
277         gtitle = title or (group_info.title if group_info else
gid)
278         gpayload = payload if payload is not None else (group_
info.payload if group_info else {})
279
280         status = "ok"
281         key_metrics: Dict[str, Any] = {}
282         anomalies: List[str] = []
283         recommendations: List[str] = []
284
285         if gid == "compute_thermals":
286             cpu = gpayload.get("cpu", {})
287             cpu_load = cpu.get("load_percent", 0.0)
288             pkg_temp = cpu.get("package_temperature_celsius")
or 0.0
289             key_metrics["cpu_load"] = f"{cpu_load}%"
290             key_metrics["package_temp"] = f"{pkg_temp}°C" if p
kg_temp else "N/A"
291             if cpu_load >= 85.0 or pkg_temp >= 85.0:
292                 status = "critical"
293                 anomalies.append(f"Критическая нагрузка/нагрев
CPU: {cpu_load}%, {pkg_temp}°C")
294                 recommendations.append("Проверьте систему охла
ждения и фоновые процессы.")
295             elif cpu_load >= 70.0 or pkg_temp >= 75.0:
296                 status = "warning"
297                 anomalies.append(f"Повышенная нагрузка CPU: {c
pu_load}%")

```

```
298         elif gid == "memory_processes":
299             ram = gpayload.get("ram", {})
300             ram_used = ram.get("used_percent", 0.0)
301             key_metrics["ram_used_percent"] = f"{ram_used}%"
302             if ram_used >= 90.0:
303                 status = "critical"
304                 anomalies.append(f"Память исчерпана: {ram_used}
305 | %")
306                 recommendations.append("Закройте неиспользуемы
307 | е программы.")
308             elif ram_used >= 75.0:
309                 status = "warning"
310                 anomalies.append(f"Высокое потребление памяти:
311 | {ram_used}%")
312             elif gid == "storage_smart":
313                 parts = gpayload.get("storage_partitions", [])
314                 max_d = max([p.get("used_percent", 0.0) for p in p
315 | arts] or [0.0])
316                 key_metrics["max_disk_used"] = f"{max_d}%"
317                 if max_d >= 95.0:
318                     status = "critical"
319                     anomalies.append(f"Том заполнен на {max_d}%")
320                     recommendations.append("Очистите дисковое прос
321 | транство.")
322                 elif max_d >= 85.0:
323                     status = "warning"
324                     anomalies.append(f"Низкий объем свободного мес
325 | та на диске ({max_d}%")
326             elif gid == "system_network":
327                 reboot = gpayload.get("system_updates", {}).get("r
328 | eboot_pending", False)
329                 key_metrics["reboot_pending"] = reboot
330                 if reboot:
331                     status = "warning"
332                     anomalies.append("Ожидается перезагрузка для з
333 | авершения обновлений ОС.")
334                 recommendations.append("Запланируйте перезагру
335 | зку компьютера.")
```

```

327
328         summary = f"Подсистема '{gtitle}' находится в статусе:
        {status.upper()}."
329         model_name = "Heuristic Analyzer"
330         raw_response = None
331
332         executor = self.chat_model
333         if executor is not None:
334             if hasattr(executor, "active_provider"):
335                 model_name = getattr(executor, "active_provide
r", "Unified AI")
336             elif hasattr(executor, "model_id"):
337                 model_name = f"Gemini CLI ({getattr(executor,
'model_id', 'gemini-3.1-flash-lite')})"
338             else:
339                 model_name = "AI Model"
340
341         prompt = (
342             f"Домен: {gtitle}\n"
343             f"Данные телеметрии:\n```json\n{json.dumps(gpa
yload, ensure_ascii=False, indent=2)}\n```\n\n"
344             "Сформулируйте краткий вердикт и 1-2 рекоменда
ции."
345         )
346         try:
347             if hasattr(executor, "ask"):
348                 res = await executor.ask(prompt)
349             elif hasattr(executor, "chat"):
350                 res = await executor.chat(prompt)
351             elif callable(executor):
352                 res = await executor(prompt)
353             else:
354                 res = str(executor)
355             raw_response = str(res).strip()
356             summary = raw_response
357         except Exception as ex:
358             logger.warning(f"Ошибка LLM анализа группы {gi
d}: {ex}")

```

```

359 |
360 |         return GroupDiagnosticResult(
361 |             group_id=gid,
362 |             title=gtitle,
363 |             icon="bi-cpu",
364 |             status=status,
365 |             summary=summary,
366 |             anomalies=anomalies,
367 |             recommendations=recommendations,
368 |             key_metrics=key_metrics,
369 |             raw_response=raw_response,
370 |             ai_model_used=model_name,
371 |         )
372 |
373 |     async def synthesize_final_report(self, groups: List[Group
DiagnosticResult]) -> SynthesisDiagnosticResult:
374 |         """Синтез общего вердикта по результатам всех групп."""
375 |         has_critical = any(g.status == "critical" for g in gro
ups)
376 |         has_warning = any(g.status == "warning" for g in group
s)
377 |         health_score = 60 if has_critical else (78 if has_warn
ing else 100)
378 |         status_label = "Критическое" if has_critical else ("Вн
имание" if has_warning else "Отличное")
379 |
380 |         actions = []
381 |         for g in groups:
382 |             actions.extend(g.recommendations)
383 |
384 |         summary = f"Общий вердикт: синтез завершен по {len(gro
ups)} доменам. Индекс здоровья: {health_score}/100 ({status_la
bel})."
385 |         model_name = "Heuristic Analyzer"
386 |
387 |         executor = self.chat_model
388 |         if executor is not None:

```

```

389         if hasattr(executor, "active_provider"):
390             model_name = getattr(executor, "active_provider", "Unified AI")
391             prompt = f"Синтезируйте общий вердикт по группам: {[g.model_dump() for g in groups]}"
392             try:
393                 if hasattr(executor, "ask"):
394                     res = await executor.ask(prompt)
395                     summary = str(res).strip()
396             except Exception:
397                 pass
398
399         return SynthesisDiagnosticResult(
400             health_score=health_score,
401             status_label=status_label,
402             executive_summary=summary,
403             critical_actions=actions[:4] if actions else ["Система функционирует штатно."],
404             ai_model_used=model_name,
405             groups_evaluated=len(groups),
406         )
407
408     async def synthesize_all_groups(self, groups: List[GroupDiagnosticResult]) -> SynthesisDiagnosticResult:
409         """Алиас для synthesize_final_report."""
410         return await self.synthesize_final_report(groups)
411
412     async def diagnose_incident(self, incident: TelemetryIncident) -> SystemDiagnosticReport:
413         """Формирует экспертный диагностический отчет по инциденту аномалии с привлечением LLM.
414
415         Args:
416             incident: Зафиксированный объект TelemetryIncident.
417
418         Returns:
419             SystemDiagnosticReport: Диагностический отчет с ре

```

```

    комендациями.
420         """
421         score = 40 if incident.severity == 'critical' else 65
422         anomalies = [
423             AnomalyItem(
424                 subsystem=f'Incident [{incident.trigger_type}]
',
425                 severity=incident.severity,
426                 title=incident.title,
427                 description=incident.description,
428             )
429         ]
430         recommendations = [
431             f"Проверьте процессы, вызвавшие триггер ({', '.join([p.get('name', '') for p in incident.suspect_processes]) or
'системная активность'}).",
432             "Изучите сопутствующие системные события и сетевые
соединения в окне инцидента.",
433         ]
434
435         summary_text = f"Инцидент {incident.incident_id} [{incident.severity.upper()}]: {incident.title}. {incident.description}"
436         model_name = "Incident Heuristic Analyzer"
437         prompt = IncidentDetector.format_incident_markdown_for_
_llm(incident)
438         raw_response = None
439         error_msg = None
440
441         executor = self.chat_model
442         if executor is not None:
443             if hasattr(executor, "active_provider"):
444                 model_name = getattr(executor, "active_provider", "Unified AI")
445             elif hasattr(executor, "model_id"):
446                 model_name = f"Gemini CLI ({getattr(executor, 'model_id', 'gemini-3.1-flash-lite'))"
447             else:

```

```
448         model_name = "AI Model"
449
450     try:
451         if hasattr(executor, "ask"):
452             res = await executor.ask(prompt)
453         elif hasattr(executor, "chat"):
454             res = await executor.chat(prompt)
455         elif callable(executor):
456             res = await executor(prompt)
457         else:
458             res = str(executor)
459         raw_response = str(res).strip()
460         if raw_response:
461             summary_text = raw_response
462     except Exception as ex:
463         logger.warning(f"Ошибка вызова LLM для инциден
та {incident.incident_id}: {ex}")
464         error_msg = str(ex)
465
466     return SystemDiagnosticReport(
467         health_score=score,
468         summary=summary_text,
469         anomalies=anomalies,
470         recommendations=recommendations,
471         ai_model_used=model_name,
472         generated_prompt=prompt,
473         raw_response=raw_response,
474         error=error_msg,
475     )
476
```

extractor.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry_Research
4 # - Extractor
5 # =====
6 # Description:
7 #   Загрузчик и парсер логов телеметрии из файлов и директорий
8 #   .
9 #
10 # Usage Examples:
11 #   Python API:
12 #       from apps.windows.telemetry_research.extractor import TelemetryDataExtractor
13 #
14 #       service = TelemetryDataExtractor()
15 #
16 # File: extractor.py
17 # Project: ai-breadboard
18 # Package: apps.windows.telemetry_research
19 # Author: hypo69
20 # Copyright: © 2026 hypo69
21 # Updated: 2026-10-01 13:28:28
22 # =====
23
24
25 from __future__ import annotations
26 """Загрузчик и парсер логов телеметрии из файлов и директорий.
27 """
28
29 import json
30 import os
31 from datetime import datetime, timezone
32 from pathlib import Path
33 from typing import Any, Dict, Generator, List, Optional, Union
```



```

30
31 try:
32     from logger import logger
33 except ImportError:
34     from logger import logger
35
36 try:
37     from apps.windows.telemetry.sqlite import TelemetryStorage
38 except ImportError:
39     TelemetryStorage = None
40
41
42 class TelemetryDataExtractor:
43     """Извлекатель и нормализатор данных из базы данных и файло
44     в логов телеметрии."""
45
46     def __init__(
47         self,
48         default_log_dirs: Optional[List[Union[str, Path]]] = None,
49         storage: Optional[Any] = None,
50     ) -> None:
51         """Инициализация извлекателя логов.
52
53         Args:
54             default_log_dirs: Список базовых директорий для по
55             иска логов по умолчанию.
56             storage: Экземпляр хранилища SQLite базы данных.
57         """
58         self._storage = storage
59         base_proj = Path(__file__).resolve().parents[3]
60         appdata = os.environ.get("APPDATA", str(Path.home() /
61         "AppData" / "Roaming"))
62         self.default_log_dirs = (
63             [Path(p) for p in default_log_dirs]
64             if default_log_dirs
65             else [
66                 base_proj / "logs" / "telemetry",

```

```

64 |             Path(appdata) / "AI-Breadboard" / "apps" / "lo
| gs",
65 |             Path(appdata) / "AI-Breadboard" / "apps" / "wi
| ndows" / "telemetry" / "logs",
66 |             base_proj / "data" / "telemetry" / "hardware_a
| rchives",
67 |         ]
68 |     )
69 |
70 |     @property
71 |     def storage(self) -> Optional[Any]:
72 |         """Получить экземпляр SQLite хранилища телеметрии."""
73 |         if self._storage is None and TelemetryStorage is not N
| one:
74 |             try:
75 |                 self._storage = TelemetryStorage.get_instance(
| )
76 |             except Exception:
77 |                 self._storage = None
78 |         return self._storage
79 |
80 |     def discover_log_files(self, search_dir: Optional[Union[str, Path]] = None) -> List[Path]:
81 |         """Обнаружить все доступные файлы логов и базы данных
| телеметрии.
82 |
83 |         Args:
84 |             search_dir: Опциональная конкретная директория для
| поиска.
85 |
86 |         Returns:
87 |             List[Path]: Список обнаруженных путей к файлам лог
| ов.
88 |         """
89 |         dirs_to_scan = [Path(search_dir)] if search_dir else s
| elf.default_log_dirs
90 |         found_files: List[Path] = []
91 |         for d in dirs_to_scan:

```

```

92         if not d.exists() or not d.is_dir():
93             continue
94         for pattern in ("*.db", "*.json", "*.jsonl", "*.log", "*.csv"):
95             try:
96                 for file_path in d.glob(pattern):
97                     if file_path.is_file() and file_path.stat().st_size > 0:
98                         found_files.append(file_path)
99             except Exception as ex:
100                 logger.warning(f"Ошибка сканирования директории {d} по шаблону {pattern}: {ex}")
101
102     seen = set()
103     unique_files = []
104     for f in found_files:
105         resolved = str(f.resolve())
106         if resolved not in seen:
107             seen.add(resolved)
108             unique_files.append(f)
109     return unique_files
110
111     def parse_file(self, file_path: Union[str, Path]) -> List[Dict[str, Any]]:
112         """Распарсить конкретный файл логов или базу данных телеметрии.
113
114         Args:
115             file_path: Путь к файлу лога (.db, .json, .jsonl, .csv).
116
117         Returns:
118             List[Dict[str, Any]]: Список нормализованных записей с метками времени.
119         """
120         path = Path(file_path)
121         if not path.exists() or not path.is_file():
122             logger.warning(f"Файл логов не найден: {path}")

```

[illegible]

```

155 |                 entries.append(self._n
| normalize_entry(item, source_file=path.name))
156 |                 elif "records" in content and isin
| stance(content["records"], list):
157 |                     for item in content["records"]
| :
158 |                         if isinstance(item, dict):
159 |                             entries.append(self._n
| ormalize_entry(item, source_file=path.name))
160 |                     else:
161 |                         entries.append(self._normalize
| _entry(content, source_file=path.name))
162 |                 except json.JSONDecodeError:
163 |                     f.seek(0)
164 |                     for line in f:
165 |                         line_str = line.strip()
166 |                         if not line_str:
167 |                             continue
168 |                         try:
169 |                             data = json.loads(line_str)
170 |                             if isinstance(data, dict):
171 |                                 entries.append(self._norma
| lize_entry(data, source_file=path.name))
172 |                     except Exception:
173 |                         continue
174 |             except Exception as err:
175 |                 logger.error(f"Ошибка чтения файла логов {path}: {
| err}")
176 |             return entries
177 |
178 |     def get_default_db_path(self) -> Optional[Path]:
179 |         """Получить путь к активной базе данных telemetry.db."
| ""
180 |         if self.storage and hasattr(self.storage, "db_path") a
| nd self.storage.db_path and Path(self.storage.db_path).exists(
| ):
181 |             return Path(self.storage.db_path)
182 |         appdata = os.environ.get("APPDATA") or os.environ.get(

```

```

"LOCALAPPDATA") or str(Path.home() / "AppData" / "Roaming")
183     candidates = [
184         Path(appdata) / "AI-Breadboard" / "apps" / "window
s" / "telemetry" / "logs" / "telemetry.db",
185         Path(__file__).resolve().parents[3] / "logs" / "te
lemetry" / "telemetry.db",
186         Path(appdata) / "AI-Breadboard" / "apps" / "logs"
/ "telemetry.db",
187     ]
188     for p in candidates:
189         if p.exists() and p.is_file():
190             return p
191     return None
192
193     def get_current_system_state(
194         self, db_path: Optional[Union[str, Path]] = None, limi
t: int = 100
195     ) -> Dict[str, Any]:
196         """Получить актуальное состояние системы и временные р
яды последних замеров из SQLite БД.
197
198         Args:
199             db_path: Путь к файлу telemetry.db (если не указан
, используется база по умолчанию).
200             limit: Количество последних точек временного ряда
(по умолчанию 100).
201
202         Returns:
203             Dict[str, Any]: Словарь с текущим срезом метрик, в
ременными рядами, сенсорами и процессами.
204         """
205         target_db = Path(db_path) if db_path else self.get_def
ault_db_path()
206         if not target_db or not target_db.exists():
207             return {
208                 "status": "not_found",
209                 "message": "База данных telemetry.db не найден
a",

```

```

210         "db_path": str(target_db) if target_db else No
ne,
211         "latest_snapshot": {},
212         "time_series": {},
213         "sensors": [],
214         "top_processes": [],
215         "recent_events": [],
216     }
217
218     try:
219         import sqlite3
220         conn = sqlite3.connect(str(target_db), timeout=5.0
)
221         conn.row_factory = sqlite3.Row
222         cursor = conn.cursor()
223
224         cursor.execute("SELECT name FROM sqlite_master WHE
RE type='table'")
225         tables = {row[0] for row in cursor.fetchall()}
226
227         # 1. Последние системные снелшоты (хронологический
порядок для графиков)
228         snapshots: List[Dict[str, Any]] = []
229         total_snapshots = 0
230         if "system_snapshots" in tables:
231             cursor.execute("SELECT count(*) FROM system_sn
apshots")
232             total_snapshots = cursor.fetchone()[0]
233
234             cursor.execute(
235                 "SELECT * FROM system_snapshots ORDER BY i
d DESC LIMIT ?", (max(10, min(limit, 2000)),)
236             )
237             raw_snaps = [dict(r) for r in cursor.fetchall(
)]
238             # Переворачиваем, чтобы временной ряд шел от с
тарых к новым
239             snapshots = list(reversed(raw_snaps))

```

```

240
241         latest = snapshots[-1] if snapshots else {}
242
243         # 2. Формирование временных рядов
244         time_series = {
245             "labels": [],
246             "cpu_load": [],
247             "cpu_freq_ghz": [],
248             "ram_percent": [],
249             "ram_used_gb": [],
250             "gpu_load": [],
251             "gpu_temp": [],
252             "disk_read_mb": [],
253             "disk_write_mb": [],
254             "net_recv_kb": [],
255             "net_sent_kb": [],
256         }
257
258         for s in snapshots:
259             ts_str = str(s.get("timestamp", ""))
260             # Короткая метка времени HH:MM:SS
261             short_time = ts_str[11:19] if len(ts_str) >= 1
262             9 else ts_str
263             time_series["labels"].append(short_time)
264             time_series["cpu_load"].append(round(float(s.get("cpu_total_percent") or 0.0), 1))
265             freq_mhz = float(s.get("cpu_frequency_mhz") or
266             0.0)
267             time_series["cpu_freq_ghz"].append(round(freq_
268             mhz / 1000.0, 2) if freq_mhz > 0 else 0.0)
269             time_series["ram_percent"].append(round(float(
270             s.get("memory_percent") or 0.0), 1))
271             time_series["ram_used_gb"].append(round(float(
272             s.get("memory_used_gb") or 0.0), 2))
273             time_series["gpu_load"].append(round(float(s.g
274             et("gpu_load_percent") or 0.0), 1))
275             time_series["gpu_temp"].append(round(float(s.g
276             et("gpu_temp_c") or 0.0), 1))

```



```

270 |
271 |         # Диск: байты/сек -> MB/s
272 |         rb = float(s.get("disk_read_bytes_sec") or 0.0
273 |         )
274 |         wb = float(s.get("disk_write_bytes_sec") or 0.
275 |         0)
276 |         time_series["disk_read_mb"].append(round(rb /
277 |         (1024 * 1024), 2))
278 |         time_series["disk_write_mb"].append(round(wb /
279 |         (1024 * 1024), 2))
280 |
281 |         # Сеть: байты/сек -> KB/s
282 |         nr = float(s.get("network_recv_bytes_sec") or
283 |         0.0)
284 |         ns = float(s.get("network_sent_bytes_sec") or
285 |         0.0)
286 |         time_series["net_recv_kb"].append(round(nr / 1
287 |         024, 1))
288 |         time_series["net_sent_kb"].append(round(ns / 1
289 |         024, 1))
290 |
291 |     # 3. Актуальные сенсоры (LHM / Hardware)
292 |     sensors: List[Dict[str, Any]] = []
293 |     if "sensor_polls" in tables:
294 |         cursor.execute(
295 |             """
296 |             SELECT sensor_name, sensor_category, unit,
297 |             value, hardware_name, timestamp
298 |             FROM sensor_polls
299 |             ORDER BY id DESC LIMIT 50
300 |             """
301 |         )
302 |         seen_sensors = set()
303 |         for r in cursor.fetchall():
304 |             s_dict = dict(r)
305 |             key = f"{s_dict.get('hardware_name')}_{s_d
306 |             ict.get('sensor_name')}"
307 |             if key not in seen_sensors:

```

```

298 |             seen_sensors.add(key)
299 |             sensors.append(s_dict)
300 |
301 |     # 4. Топ активных процессов
302 |     top_processes: List[Dict[str, Any]] = []
303 |     if "process_snapshots" in tables and latest.get("i
    | d"):
304 |         cursor.execute(
305 |             """
306 |             SELECT pid, name, cpu_percent, memory_mb,
    | memory_percent, num_threads, username, status
307 |             FROM process_snapshots
308 |             WHERE snapshot_id = ?
309 |             ORDER BY cpu_percent DESC LIMIT 15
310 |             """,
311 |             (latest["id"],),
312 |         )
313 |         top_processes = [dict(r) for r in cursor.fetch
    | all()]
314 |
315 |     if not top_processes and "process_outliers" in tab
    | les:
316 |         cursor.execute(
317 |             """
318 |             SELECT pid, name, cpu_percent, memory_mb,
    | memory_percent, num_threads, username, status
319 |             FROM process_outliers
320 |             ORDER BY id DESC LIMIT 15
321 |             """,
322 |         )
323 |         top_processes = [dict(r) for r in cursor.fetch
    | all()]
324 |
325 |     # 5. Последние события
326 |     recent_events: List[Dict[str, Any]] = []
327 |     if "device_events" in tables:
328 |         cursor.execute(
329 |             """

```

```

330         SELECT timestamp, event_type, friendly_name, category, has_problem, problem_code
331         FROM device_events
332         ORDER BY id DESC LIMIT 8
333         """
334     )
335     for r in cursor.fetchall():
336         d = dict(r)
337         d["source"] = "device"
338         recent_events.append(d)
339
340     if "w64_events" in tables:
341         cursor.execute(
342             """
343             SELECT timestamp, event_type, path, name,
pid, provider
344             FROM w64_events
345             ORDER BY id DESC LIMIT 8
346             """
347         )
348         for r in cursor.fetchall():
349             d = dict(r)
350             d["source"] = "system_activity"
351             recent_events.append(d)
352
353     recent_events.sort(key=lambda x: str(x.get("timestamp", "")), reverse=True)
354     recent_events = recent_events[:15]
355
356     stat_info = target_db.stat()
357     conn.close()
358
359     return {
360         "status": "ok",
361         "db_path": str(target_db.resolve()),
362         "db_name": target_db.name,
363         "db_size_bytes": stat_info.st_size,
364         "db_size_mb": round(stat_info.st_size / (1024

```

```

* 1024), 2),
365         "total_snapshots": total_snapshots,
366         "returned_points": len(snapshots),
367         "latest_snapshot": latest,
368         "time_series": time_series,
369         "sensors": sensors,
370         "top_processes": top_processes,
371         "recent_events": recent_events,
372     }
373     except Exception as ex:
374         logger.error(f"Ошибка получения текущего состояния
из telemetry.db: {ex}")
375         return {
376             "status": "error",
377             "message": str(ex),
378             "db_path": str(target_db) if target_db else No
ne,
379             "latest_snapshot": {},
380             "time_series": {},
381             "sensors": [],
382             "top_processes": [],
383             "recent_events": [],
384         }
385
386     def load_from_sqlite_db(self, db_path: Union[str, Path], l
imit: int = 5000) -> List[Dict[str, Any]]:
387         """Загрузить исторические записи телеметрии напрямую и
з произвольного SQLite файла.
388
389         Args:
390             db_path: Путь к файлу базы данных SQLite.
391             limit: Максимальное число извлекаемых записей.
392
393         Returns:
394             List[Dict[str, Any]]: Список нормализованных запис
ей из БД.
395         """
396         records: List[Dict[str, Any]] = []

```

```
397 |         try:
398 |             import sqlite3
399 |             conn = sqlite3.connect(str(db_path))
400 |             conn.row_factory = sqlite3.Row
401 |             cursor = conn.cursor()
402 |
403 |             # Проверка наличия таблиц
404 |             cursor.execute("SELECT name FROM sqlite_master WHE
| RE type='table'")
405 |             tables = {row[0] for row in cursor.fetchall()}
406 |
407 |             if "system_snapshots" in tables:
408 |                 cursor.execute("SELECT * FROM system_snapshots
| ORDER BY id DESC LIMIT ?", (limit,))
409 |                 for row in cursor.fetchall():
410 |                     row_dict = dict(row)
411 |                     row_dict["source_file"] = f"{Path(db_path)
| .name}:system_snapshots"
412 |                     records.append(self._normalize_entry(row_d
| ict, source_file=f"{Path(db_path).name}:system_snapshots"))
413 |
414 |             if "sensor_polls" in tables:
415 |                 cursor.execute("SELECT * FROM sensor_polls ORD
| ER BY id DESC LIMIT ?", (limit,))
416 |                 for row in cursor.fetchall():
417 |                     row_dict = dict(row)
418 |                     row_dict["source_file"] = f"{Path(db_path)
| .name}:sensor_polls"
419 |                     records.append(self._normalize_entry(row_d
| ict, source_file=f"{Path(db_path).name}:sensor_polls"))
420 |
421 |             if "telemetry_events" in tables:
422 |                 cursor.execute("SELECT * FROM telemetry_events
| ORDER BY id DESC LIMIT ?", (limit,))
423 |                 for row in cursor.fetchall():
424 |                     row_dict = dict(row)
425 |                     row_dict["source_file"] = f"{Path(db_path)
| .name}:telemetry_events"
```

```

426         records.append(self._normalize_entry(row_d
| ict, source_file=f"{Path(db_path).name}:telemetry_events"))
427
428         conn.close()
429     except Exception as ex:
430         logger.warning(f"Ошибка загрузки телеметрии из БД
| SQLite {db_path}: {ex}")
431     return records
432
433     def load_from_database(self, limit: int = 2000) -> List[Di
| ct[str, Any]]:
434         """Загрузить исторические записи телеметрии из стандар
| тного хранилища.
435
436         Args:
437             limit: Максимальное число извлекаемых записей.
438
439         Returns:
440             List[Dict[str, Any]]: Список нормализованных запис
| ей из БД.
441         """
442         if self.storage is None:
443             return []
444         records: List[Dict[str, Any]] = []
445         try:
446             snapshots = self.storage.get_snapshots(limit=limit
| )
447             for snap in snapshots:
448                 rec = dict(snap) if not isinstance(snap, dict)
| and hasattr(snap, "__dict__") else (snap if isinstance(snap,
| dict) else {})
449                 rec["source_file"] = "sqlite:system_snapshots"
450                 records.append(self._normalize_entry(rec, sour
| ce_file="sqlite:system_snapshots"))
451             sensor_polls = self.storage.get_sensor_history(lim
| it=limit)
452             for s in sensor_polls:
453                 rec = dict(s) if not isinstance(s, dict) and h

```

```

    asattr(s, "__dict__") else (s if isinstance(s, dict) else {}))
454         rec["source_file"] = "sqlite:sensor_polls"
455         records.append(self._normalize_entry(rec, source_file="sqlite:sensor_polls"))
456         events = self.storage.get_events(limit=limit)
457         for ev in events:
458             rec = dict(ev) if not isinstance(ev, dict) and
hasattr(ev, "__dict__") else (ev if isinstance(ev, dict) else
                {})
459             rec["source_file"] = "sqlite:telemetry_events"
460             records.append(self._normalize_entry(rec, source_file="sqlite:telemetry_events"))
461         except Exception as ex:
462             logger.warning(f"Ошибка загрузки телеметрии из базы данных: {ex}")
463         return records
464
465     def load_all_records(
466         self,
467         source: Optional[Union[str, Path, List[Dict[str, Any]]]] = None,
468         include_database: bool = True,
469     ) -> List[Dict[str, Any]]:
470         """Загрузить все записи телеметрии из источника, базы данных или найденных файлов.
471
472         Args:
473             source: Путь к директории, файлу или готовый список записей.
474             include_database: Загружать ли записи из базы данных SQLite.
475
476         Returns:
477             List[Dict[str, Any]]: Отсортированный по времени список записей телеметрии.
478
479         """
479         if isinstance(source, list):
480             return [self._normalize_entry(x, source_file="memo

```

```

    ry") for x in source if isinstance(x, dict)]
481
482     all_records: List[Dict[str, Any]] = []
483     if source:
484         source_path = Path(source)
485         if source_path.is_file():
486             all_records.extend(self.parse_file(source_path
487         ))
488         elif source_path.is_dir():
489             for f in self.discover_log_files(source_path):
490                 all_records.extend(self.parse_file(f))
491     else:
492         if include_database:
493             all_records.extend(self.load_from_database())
494             for f in self.discover_log_files():
495                 all_records.extend(self.parse_file(f))
496
497     all_records.sort(key=lambda item: item.get("timestamp"
498     , ""))
499     return all_records
500
501     def _normalize_entry(self, raw: Dict[str, Any], source_file: str = "") -> Dict[str, Any]:
502         """Привести сырую запись телеметрии к единому стандарту.
503
504         Args:
505             raw: Исходный словарь данных.
506             source_file: Имя исходного файла.
507
508         Returns:
509             Dict[str, Any]: Нормализованная запись.
510
511         """
512         normalized = dict(raw)
513         normalized.setdefault("source_file", source_file)
514         ts = raw.get("timestamp") or raw.get("time") or raw.get("created_at") or raw.get("recorded_at")
515         if not ts:

```



```
513 |         ts = datetime.now(timezone.utc).isoformat()
514 |     elif isinstance(ts, (int, float)):
515 |         try:
516 |             ts = datetime.fromtimestamp(ts, tz=timezone.ut
517 | c).isoformat()
518 |         except Exception:
519 |             ts = str(ts)
520 |     else:
521 |         ts = str(ts)
522 |     normalized["timestamp"] = ts
523 |     return normalized
```

grouped_telemetry.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry_Research
4 # - Grouped Telemetry
5 # Description:
6 #   Разбиение телеметрии хоста на специализированные доменные
7 #   группы для LLM-анализа.
8 # Usage Examples:
9 #   Python API:
10 #   from apps.windows.telemetry_research.grouped_telemetry i
11 #   mport TelemetryGroupInfo
12 #   service = TelemetryGroupInfo()
13 #
14 # File: grouped_telemetry.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry_research
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-08 12:00:00
20 # =====
21
22 from __future__ import annotations
23 """Разбиение телеметрии хоста на специализированные доменные г
24   руппы для LLM-анализа."""
25
26 from typing import Any, Dict, List, Optional
27 from pydantic import BaseModel, Field
28 from apps.windows.telemetry.models import SystemSnapshot
29
```

```

30 class TelemetryGroupInfo(BaseModel):
31     """Описание и структурированные данные одной диагностическ
ой группы."""
32     group_id: str = Field(..., description="Уникальный идентиф
икатор группы")
33     title: str = Field(..., description="Человекочитаемый заго
ловок домена")
34     icon: str = Field(default="bi-cpu", description="Bootstrap
-иконка")
35     description: str = Field(..., description="Краткое описани
е охватываемых подсистем")
36     order: int = Field(default=1, description="Порядковый номе
р выполнения в цепочке")
37     payload: Dict[str, Any] = Field(default_factory=dict, desc
ription="Срез данных телеметрии и сенсоров")
38
39
40 class GroupedTelemetryBuilder:
41     """Генератор изолированных пакетов телеметрии по функциона
льным группам."""
42
43     @staticmethod
44     def build_compute_thermals_group(snapshot: SystemSnapshot)
-> TelemetryGroupInfo:
45         """Формирует группу 1: Вычислительные ресурсы (CPU/GPU
) и тепловой режим."""
46         try:
47             sensors = snapshot.sensors or []
48             temps = {s.name: s.value for s in sensors if s.name
and (s.category == "temperature" or "temp" in (s.sensor_id or
"").lower())}
49             fans = {s.name: f"{s.value} {s.unit}" for s in sen
sors if s.name and (s.category == "fan" or "fan" in (s.sensor_
id or "").lower())}
50             voltages = {s.name: f"{s.value} {s.unit}" for s in
sensors if s.name and (s.category == "voltage" or "voltage" i
n (s.sensor_id or "").lower())}
51             powers = {s.name: f"{s.value} {s.unit}" for s in s

```

```

    ensors if s.name and (s.category == "power" or "power" in (s.s
    ensor_id or "").lower()))}
52         gpus_data = [
53             {
54                 "name": g.name or "GPU",
55                 "load_percent": g.load_percent,
56                 "memory_total_gb": g.memory_total_gb,
57                 "memory_used_gb": g.memory_used_gb,
58                 "memory_free_gb": g.memory_free_gb,
59                 "temperature_celsius": g.temperature_celsi
    us,
60             }
61             for g in snapshot.gpus or []
62         ]
63         cpu = snapshot.cpu
64         cpu_model = cpu.model if cpu else ""
65         cpu_arch = cpu.architecture if cpu else "x86_64"
66         cpu_load = round(cpu.total_percent, 1) if cpu and
    cpu.total_percent is not None else 0.0
67         p_cores = cpu.physical_cores if cpu else 1
68         l_cores = cpu.logical_cores if cpu else 1
69         freq = cpu.frequency_mhz if cpu else 0.0
70         pkg_temp = cpu.temperature_celsius if cpu else Non
    e
71         per_core = [round(c, 1) for c in cpu.per_core_perc
    ent or [] if c is not None] if cpu else []
72
73         payload: Dict[str, Any] = {
74             "cpu": {
75                 "model": cpu_model,
76                 "architecture": cpu_arch,
77                 "load_percent": cpu_load,
78                 "physical_cores": p_cores,
79                 "logical_cores": l_cores,
80                 "frequency_mhz": freq,
81                 "package_temperature_celsius": pkg_temp,
82                 "per_core_percent": per_core,
83             },

```

```

84         "gpu": gpu_data,
85         "sensors": {
86             "temperatures_celsius": temps,
87             "fans_speed": fans,
88             "voltages": voltages,
89             "powers_watt": powers,
90         },
91     }
92     except Exception:
93         payload = {
94             "cpu": {"model": "CPU", "load_percent": 0.0, "
physical_cores": 1, "logical_cores": 1},
95             "gpu": [],
96             "sensors": {"temperatures_celsius": {}, "fans_
speed": {}, "voltages": {}, "powers_watt": {}},
97         }
98     return TelemetryGroupInfo(
99         group_id="compute_thermals",
100         title="CPU, GPU и Охлаждение",
101         icon="bi-fire",
102         description="Анализ загрузки процессора, ядер, вид
еокарт, температурных датчиков и кулеров",
103         order=1,
104         payload=payload,
105     )
106
107     @staticmethod
108     def build_memory_processes_group(snapshot: SystemSnapshot)
-> TelemetryGroupInfo:
109         """Формирует группу 2: Оперативная память, Swap и акти
вные процессы."""
110         try:
111             ignored_processes = {"system idle process", "system", "idle"}
112             procs = snapshot.top_processes or []
113             active_processes = [
114                 {
115                     "pid": p.pid,

```

```

116         "name": p.name or "process",
117         "cpu_percent": round(p.cpu_percent, 1) if
p.cpu_percent is not None else 0.0,
118         "memory_mb": round(p.memory_mb, 1) if p.me
mory_mb is not None else 0.0,
119         "memory_percent": round(p.memory_percent,
1) if p.memory_percent is not None else 0.0,
120     }
121     for p in procs
122         if p.pid not in [0, 4] and (p.name or "").lowe
r() not in ignored_processes
123         ][:12]
124         mem = snapshot.memory
125         ram_total = mem.total_gb if mem else 0.0
126         ram_used = mem.used_gb if mem else 0.0
127         ram_avail = mem.available_gb if mem else 0.0
128         ram_pct = mem.percent if mem else 0.0
129         swap_total = mem.swap_total_gb if mem else 0.0
130         swap_used = mem.swap_used_gb if mem else 0.0
131         swap_pct = mem.swap_percent if mem else 0.0
132         ram_modules = [
133             {
134                 "bank_label": r.bank_label,
135                 "capacity_gb": r.capacity_gb,
136                 "speed_mhz": r.speed_mhz,
137                 "manufacturer": r.manufacturer,
138             }
139             for r in snapshot.ram_sticks or []
140         ]
141         payload: Dict[str, Any] = {
142             "ram": {"total_gb": ram_total, "used_gb": ram_
used, "available_gb": ram_avail, "used_percent": ram_pct},
143             "swap_paging_file": {"total_gb": swap_total, "
used_gb": swap_used, "used_percent": swap_pct},
144             "ram_modules": ram_modules,
145             "top_active_processes": active_processes,
146         }
147     except Exception:

```

```

148         payload = {
149             "ram": {"total_gb": 0.0, "used_gb": 0.0, "available_gb": 0.0, "used_percent": 0.0},
150             "swap_paging_file": {"total_gb": 0.0, "used_gb": 0.0, "used_percent": 0.0},
151             "ram_modules": [],
152             "top_active_processes": [],
153         }
154     return TelemetryGroupInfo(
155         group_id="memory_processes",
156         title="Память и Процессы",
157         icon="bi-memory",
158         description="Анализ объема и утилизации RAM, Swap-файла и топ-процессов потребителей ресурсов",
159         order=2,
160         payload=payload,
161     )
162
163     @staticmethod
164     def build_storage_smart_group(snapshot: SystemSnapshot) -> TelemetryGroupInfo:
165         """Формирует группу 3: Дисковые разделы, SMART-здоровье и Disk I/O."""
166         try:
167             partitions = [
168                 {
169                     "device": d.device or "",
170                     "mountpoint": d.mountpoint or "",
171                     "fstype": d.fstype or "",
172                     "total_gb": round(d.total_gb, 2) if d.total_gb is not None else 0.0,
173                     "used_gb": round(d.used_gb, 2) if d.used_gb is not None else 0.0,
174                     "free_gb": round(d.free_gb, 2) if d.free_gb is not None else 0.0,
175                     "used_percent": round(d.percent, 1) if d.percent is not None else 0.0,
176                 }

```

```

177         for d in snapshot.disks or []
178     ]
179     physical_disks = [
180         {
181             "model": p.model or "Disk",
182             "media_type": p.media_type or "SSD",
183             "size_gb": p.size_gb or 0.0,
184             "smart_health": p.health_status or "Health
185 y",
186             "temperature_celsius": p.temperature_celsi
187 us,
188             "interface_type": getattr(p, "interface_ty
189 pe", None) or "Unknown",
190         }
191     ]
192     for p in snapshot.physical_disks or []
193 ]
194     dio = snapshot.disk_io
195     r_mb = round((dio.read_bytes_per_sec or 0.0) / (10
196 24 * 1024), 2) if dio else 0.0
197     w_mb = round((dio.write_bytes_per_sec or 0.0) / (1
198 024 * 1024), 2) if dio else 0.0
199     r_iops = round(dio.read_count_per_sec or 0.0, 1) i
200 f dio else 0.0
201     w_iops = round(dio.write_count_per_sec or 0.0, 1)
202 if dio else 0.0
203     payload: Dict[str, Any] = {
204         "storage_partitions": partitions,
205         "physical_drives_smart": physical_disks,
206         "disk_io_rates": {"read_mb_per_sec": r_mb, "wr
207 ite_mb_per_sec": w_mb, "read_iops": r_iops, "write_iops": w_iops},
208     },
209     }
210 except Exception:
211     payload = {
212         "storage_partitions": [],
213         "physical_drives_smart": [],
214         "disk_io_rates": {"read_mb_per_sec": 0.0, "wri
215 te_mb_per_sec": 0.0, "read_iops": 0.0, "write_iops": 0.0},

```



```

205         }
206         return TelemetryGroupInfo(
207             group_id="storage_smart",
208             title="Диски и Хранилище",
209             icon="bi-hdd",
210             description="Оценка свободного места на томах, SMA
RT-состояния накопителей и дискового I/O",
211             order=3,
212             payload=payload,
213         )
214
215     @staticmethod
216     def build_system_network_group(snapshot: SystemSnapshot) -
> TelemetryGroupInfo:
217         """Формирует группу 4: Сеть, обновления Windows и точк
и восстановления."""
218         try:
219             backup_info: Dict[str, Any] = {"system_protection"
: "Enabled", "restore_points_count": 0, "latest_restore_point"
: "None"}
220             try:
221                 from apps.windows.sdk.core.system_restore impo
rt WindowsSystemRestoreManager
222                 restore_mgr = WindowsSystemRestoreManager(time
out_seconds=3)
223                 prot = restore_mgr.check_protection_status()
224                 backup_info["system_protection"] = "Enabled" i
f prot.get("system_protection_enabled") else "Disabled"
225                 points = restore_mgr.list_restore_points()
226                 backup_info["restore_points_count"] = len(poin
ts)
227                 if points:
228                     backup_info["latest_restore_point"] = poin
ts[0].get("creation_time") or points[0].get("description", "")
229             except Exception:
230                 pass
231
232         networks = [

```

```

233 |         {
234 |             "name": n.name or "Adapter",
235 |             "is_up": n.is_up,
236 |             "speed_mbps": n.speed_mbps or 0,
237 |             "recv_kb_per_sec": round((n.bytes_recv_per
    | _sec or 0.0) / 1024, 1),
238 |             "sent_kb_per_sec": round((n.bytes_sent_per
    | _sec or 0.0) / 1024, 1),
239 |         }
240 |         for n in snapshot.network or []
241 |     ]
242 |     ports = [
243 |         {"port": p.port, "process_name": p.process_name or "", "protocol": p.protocol or "TCP", "pid": p.pid}
    |         for p in (snapshot.listening_ports or [])[:15]
244 |     ]
245 |     payload: Dict[str, Any] = {
246 |         "host_info": {
247 |             "hostname": snapshot.hostname or "localhost",
248 |             "os": f"{snapshot.os_name or 'Windows'} {s
    | nAPSHOT.os_build or ''}".strip(),
249 |             "uptime_hours": round((snapshot.uptime_seconds or 0.0) / 3600, 1),
250 |             },
251 |         "system_updates": {
252 |             "reboot_pending": bool(snapshot.alerts and
    | snapshot.alerts.reboot_pending),
253 |             "status": snapshot.updates.status if snapshot.updates else "Up to date",
254 |             "installed_kb_count": snapshot.updates.installed_kb_count if snapshot.updates else 0,
255 |             },
256 |         "backup_and_restore": backup_info,
257 |         "network_adapters": networks,
258 |         "listening_ports_sample": ports,
259 |     }
260 | except Exception:
261 |

```

```

262         payload = {
263             "host_info": {"hostname": "localhost", "os": "
Windows", "uptime_hours": 0.0},
264             "system_updates": {"reboot_pending": False, "s
tatus": "Up to date", "installed_kb_count": 0},
265             "backup_and_restore": {"system_protection": "E
nabled", "restore_points_count": 0, "latest_restore_point": "N
one"},
266             "network_adapters": [],
267             "listening_ports_sample": [],
268         }
269         return TelemetryGroupInfo(
270             group_id="system_network",
271             title="Сеть и Стабильность ОС",
272             icon="bi-shield-check",
273             description="Проверка сети, обновлений ОС, отложен
ных перезагрузок и точек восстановления",
274             order=4,
275             payload=payload,
276         )
277
278     def build_all_groups(self, snapshot: SystemSnapshot) -> Li
st[TelemetryGroupInfo]:
279         """Формирует полный упорядоченный список диагностическ
их групп."""
280         return [
281             self.build_compute_thermals_group(snapshot),
282             self.build_memory_processes_group(snapshot),
283             self.build_storage_smart_group(snapshot),
284             self.build_system_network_group(snapshot),
285         ]
286
287
288 class GroupDiagnoseRequest(BaseModel):
289     """Запрос на AI-диагностику одной конкретной группы телеме
трии."""
290     group_id: str = Field(..., description="Идентификатор груп
пы (compute_thermals, memory_processes, storage_smart, system_

```

```

network)")
291     title: Optional[str] = Field(default="", description="Опци
ональный заголовок группы")
292     payload: Dict[str, Any] = Field(..., description="Данные т
елеметрии группы")
293
294
295 class GroupDiagnosticResult(BaseModel):
296     """Результат AI-анализа одной специализированной группы."""
297     group_id: str = Field(..., description="Идентификатор груп
пы")
298     title: str = Field(..., description="Заголовок группы")
299     icon: str = Field(default="bi-cpu", description="Bootstrap
-иконка")
300     status: str = Field(default="ok", description="Статус: ok,
warning, critical")
301     summary: str = Field(default="", description="Экспертное з
аключение AI-модели по данной группе")
302     anomalies: List[str] = Field(default_factory=list, descrip
tion="Выявленные отклонения и узкие места")
303     recommendations: List[str] = Field(default_factory=list, d
escription="Рекомендации по оптимизации домена")
304     key_metrics: Dict[str, Any] = Field(default_factory=dict,
description="Ключевые метрики для карточки UI")
305     raw_response: Optional[str] = Field(default=None, descript
ion="Сырой ответ языковой модели")
306     generated_prompt: Optional[str] = Field(default=None, desc
ription="Точный компактный JSON-промт, отправленный модели")
307     system_instruction: Optional[str] = Field(default=None, de
scription="Системная инструкция для модели")
308     ai_model_used: str = Field(default="AI Model", description
="Использованный AI-провайдер/модель")
309
310
311 class SynthesisRequest(BaseModel):
312     """Запрос на синтез итогового вердикта по всем завершенным
группам."""

```

```
313     groups: List[GroupDiagnosticResult] = Field(..., description="Результаты анализа всех 4 групп")
314
315
316 class SynthesisDiagnosticResult(BaseModel):
317     """Итоговый синтез состояния всей системы с интегральным Health Score."""
318     health_score: int = Field(default=100, ge=0, le=100, description="Интегральный индекс здоровья (0-100)")
319     status_label: str = Field(default="Отличное", description="Метка статуса (Отличное, Внимание, Критическое)")
320     executive_summary: str = Field(..., description="Итоговое резюме инженера по всей системе")
321     critical_actions: List[str] = Field(default_factory=list, description="Приоритетные шаги для администратора")
322     raw_response: Optional[str] = Field(default=None, description="Сырой ответ модели синтеза")
323     generated_prompt: Optional[str] = Field(default=None, description="Сформированный компактный JSON промпт синтеза")
324     system_instruction: Optional[str] = Field(default=None, description="Системная инструкция синтеза")
325     ai_model_used: str = Field(default="AI Model", description="Использованный провайдер для синтеза")
326     groups_evaluated: int = Field(default=4, description="Количество оцененных доменов")
327
```

hardware_auditor.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry_Research
4 # - Hardware Auditor
5 # Description:
6 #   Движок аудита оборудования, драйверов и привязки датчиков.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry_research.hardware_auditor import HardwareAuditor
11 #
12 #       service = HardwareAuditor()
13 #
14 # File: hardware_auditor.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry_research
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-04 03:10:00
20 # =====
21
22 from __future__ import annotations
23 """Движок аудита оборудования, драйверов и привязки датчиков."""
24
25 import os
26 import platform
27 import re
28 from datetime import datetime, timezone
29 from typing import Any, Dict, List, Optional
30 from logger import logger
```

```

31 from apps.windows.telemetry.win32_fffi.setupapi import SetupAPI
32 from apps.windows.telemetry.models import DriverInfo, Hardware
   AuditReport, HardwareDeviceAudit, HardwareSensor
33 from apps.windows.telemetry.sensors import get_hardware_sensor
   s
34
35
36 class HardwareAuditor:
37     """Аудитор аппаратного обеспечения, драйверов и параметров
   устройств."""
38
39     def __init__(self, setupapi_client: Optional[SetupAPI] = N
   one, storage: Optional[Any] = None) -> None:
40         """Инициализация аудитора оборудования.
41
42         Args:
43             setupapi_client: Опциональный экземпляр нативного
   клиента SetupAPI.
44             storage: Опциональный экземпляр хранилища Telemetr
   yStorage.
45         """
46         self._setupapi = setupapi_client or (SetupAPI() if os.
   name == 'nt' else None)
47         self._storage = storage
48
49     def _parse_wmi_date(self, date_str: Optional[str], context
   : Optional[str] = None) -> Optional[datetime]:
50         """Парсинг даты из формата WMI (YYYYMMDDHHMMSS.mmmmmmm+
   UUU) или ISO.
51
52         Args:
53             date_str: Строка с датой.
54             context: Опциональное описание контекста (имя устр
   оинства/драйвера).
55
56         Returns:
57             Optional[datetime]: Объект datetime или None.
58         """

```

```

59         if not date_str:
60             return None
61         raw = str(date_str).strip()
62         if not raw or raw.lower() == 'none':
63             return None
64         if len(raw) >= 8 and raw[:8].isdigit():
65             try:
66                 year = int(raw[0:4])
67                 month = int(raw[4:6])
68                 day = int(raw[6:8])
69                 return datetime(year, month, day, tzinfo=timezone.
one.utc)
70             except Exception:
71                 pass
72         iso_match = re.match('^(\\d{4})-(\\d{2})-(\\d{2})', raw)
73         if iso_match:
74             try:
75                 return datetime(int(iso_match.group(1)), int(iso_match.group(2)), int(iso_match.group(3)), tzinfo=timezone.utc)
76             except Exception:
77                 pass
78         ru_match = re.match('^(\\d{2})\\.(\\d{2})\\.(\\d{4})', raw)
79         if ru_match:
80             try:
81                 return datetime(int(ru_match.group(3)), int(ru_match.group(2)), int(ru_match.group(1)), tzinfo=timezone.utc)
82             except Exception:
83                 pass
84         target_info = f' для {context}' if context else ''
85         logger.debug(f'Не удалось распарсить дату WMI{target_info}: \\{date_str}\\')
86         return None
87
88     def _evaluate_driver_currency(self, driver_date_dt: Optional[datetime], provider: str, device_class: str) -> tuple[str,

```



```

Optional[int]]:
89     """Расчет актуальности драйвера на основе даты выпуска
    и поставщика.
90
91     Args:
92         driver_date_dt: Дата выпуска драйвера.
93         provider: Поставщик драйвера (NVIDIA, Intel, Micro
    soft и др.).
94         device_class: Класс оборудования (Display, Net, Pr
    ocessor и др.).
95
96     Returns:
97         tuple[str, Optional[int]]: (Статус актуальности, в
    озраст в днях).
98     """
99     now = datetime.now(timezone.utc)
100     age_days: Optional[int] = None
101     if driver_date_dt:
102         age_days = max(0, (now - driver_date_dt).days)
103     is_ms_inbox = 'microsoft' in provider.lower()
104     if age_days is not None:
105         if age_days > 1825:
106             if is_ms_inbox and device_class in ('System',
    'Volume', 'Processor'):
107                 return ('Стандартный системный драйвер', a
    ge_days)
108             return ('Критически устарел (> 5 лет)', age_da
    ys)
109         elif age_days > 730:
110             if is_ms_inbox and device_class in ('System',
    'Volume', 'Processor'):
111                 return ('Стандартный системный драйвер', a
    ge_days)
112             return ('Устарел (> 2 лет)', age_days)
113         else:
114             return ('Актуален', age_days)
115     if is_ms_inbox:
116         return ('Базовый драйвер ОС', age_days)

```

```

117         return ('Не определена', age_days)
118
119     def _get_drivers_from_wmi(self) -> Dict[str, Dict[str, Any
120 ]]:
121         """Получение карты драйверов из WMI Win32_PnPSignedDri
122 ver.
123
124         Returns:
125             Dict[str, Dict[str, Any]]: Словарь с метаданными д
126 райверов по DeviceID.
127
128         """
129         drivers_map: Dict[str, Dict[str, Any]] = {}
130         if os.name != 'nt':
131             return drivers_map
132         try:
133             import pythoncom
134             pythoncom.CoInitialize()
135             import wmi
136             w = wmi.WMI()
137             for d in w.Win32_PnPSignedDriver():
138                 dev_id = getattr(d, 'DeviceID', None)
139                 if not dev_id:
140                     continue
141                 norm_id = str(dev_id).strip().upper()
142                 drivers_map[norm_id] = {'name': getattr(d, 'De
143 viceName', '') or getattr(d, 'Description', '') or '', 'driver
144 _version': getattr(d, 'DriverVersion', '') or '', 'driver_date
145 _raw': getattr(d, 'DriverDate', None), 'provider': getattr(d,
146 'DriverProviderName', 'Unknown') or 'Unknown', 'inf_name': get
147 attr(d, 'InfName', None), 'is_signed': bool(getattr(d, 'IsSign
148 ed', True))}
149         except Exception:
150             pass
151         return drivers_map
152
153     def _get_device_install_date_registry(
154         self,
155         device_instance_id: str,

```

```

146         device_name: Optional[str] = None,
147         device_class: Optional[str] = None
148     ) -> Optional[str]:
149         """Чтение даты установки устройства из системного реес
150 тра или базы данных.
151
152         Если дата в системном реестре не найдена, выполняется
153 поиск в базе данных.
154
155         Если устройство уже есть в базе данных с датой – возвр
156 ащается существующая дата (без перезаписи).
157
158         Если устройства в базе данных нет – записывается текущ
159 ая дата.
160
161     Args:
162         device_instance_id: Идентификатор PnP устройства.
163         device_name: Человекочитаемое имя устройства.
164         device_class: Класс оборудования.
165
166     Returns:
167         Optional[str]: Строка даты установки в формате DD.
168 MM.YYYY HH:MM или None.
169
170     """
171     if not device_instance_id:
172         return None
173
174     reg_date: Optional[str] = None
175     if os.name == 'nt':
176         try:
177             import winreg
178             key_path = f'SYSTEM\\CurrentControlSet\\Enum\\
179 {device_instance_id}'
180             with winreg.OpenKey(winreg.HKEY_LOCAL_MACHINE,
181 key_path) as key:
182                 for val_name in ['InstallDate', 'FirstInst
183 allDate']:
184                     try:
185                         val, reg_type = winreg.QueryValueE
186 x(key, val_name)

```

```

175         if isinstance(val, int) and val >
176         0:
177             reg_date = datetime.fromtimestamp
178             amp(val, tz=timezone.utc).strftime('%d.%m.%Y %H:%M')
179             break
180         except OSError:
181             pass
182     except Exception:
183         pass
184
185     storage = self._storage
186     if storage is None:
187         try:
188             from apps.windows.telemetry.sqlite import Tele
189             metryStorage
190             storage = TelemetryStorage.get_instance()
191         except Exception:
192             storage = None
193
194     if reg_date:
195         if storage:
196             try:
197                 storage.get_or_create_device_install_date(
198                     device_instance_id=device_instance_id,
199                     friendly_name=device_name,
200                     device_class=device_class,
201                     default_date=reg_date
202                 )
203             except Exception as ex:
204                 logger.debug(f'Ошибка сохранения даты устр
205 ойства в БД: {ex}')
206                 return reg_date
207
208     dev_label = f"'{device_name}' [{device_instance_id}]"
209     if device_name else f"[{device_instance_id}]"
210     logger.debug(f'Дата установки для устройства {dev_labe
211 l} не найдена в реестре')
212

```

```

207         if storage:
208             try:
209                 now_str = datetime.now(timezone.utc).strftime(
                '%d.%m.%Y %H:%M')
210                 return storage.get_or_create_device_install_da
te(
211                     device_instance_id=device_instance_id,
212                     friendly_name=device_name,
213                     device_class=device_class,
214                     default_date=now_str
215                 )
216             except Exception as ex:
217                 logger.debug(f'Ошибка работы с БД для даты уст
ройства: {ex}')
218
219         return None
220
221     def _bind_sensors_to_device(self, device: HardwareDeviceAu
dit, all_sensors: List[HardwareSensor]) -> List[HardwareSensor
]:
222         """Привязка релевантных датчиков телеметрии к аппаратн
ому устройству.
223
224         Args:
225             device: Аудируемое устройство.
226             all_sensors: Полный список активных аппаратных сен
соров.
227
228         Returns:
229             List[HardwareSensor]: Список датчиков, относящихся
к данному устройству.
230         """
231         matched: List[HardwareSensor] = []
232         dev_class = device.device_class.lower()
233         dev_name = device.name.lower()
234         for s in all_sensors:
235             s_name = s.name.lower()
236             s_id = s.sensor_id.lower()

```

```

237         if dev_class in ('display', 'gpu') or 'nvidia' in
dev_name or 'geforce' in dev_name or ('radeon' in dev_name):
238             if 'gpu' in s_id or 'gpu' in s_name or 'nvidia
' in s_name:
239                 matched.append(s)
240                 continue
241             if dev_class in ('processor', 'cpu') or 'intel' in
dev_name or 'amd' in dev_name or ('ryzen' in dev_name):
242                 if 'cpu' in s_id or 'cpu' in s_name or 'therma
l' in s_id or ('acpi' in s_id):
243                     matched.append(s)
244                     continue
245                 if dev_class in ('diskdrive', 'storage', 'nvme') o
r 'ssd' in dev_name or 'hdd' in dev_name:
246                     if 'disk' in s_id or 'drive' in s_id or 'smart
' in s_id or ('storage' in s_id):
247                         matched.append(s)
248                         continue
249                 if dev_class in ('net', 'network') or 'ethernet' i
n dev_name or 'wi-fi' in dev_name or ('wireless' in dev_name):
250                     if 'net_' in s_id or 'internet_' in s_id:
251                         matched.append(s)
252                         continue
253             return matched
254
255     def audit_hardware(self, sensors: Optional[List[HardwareSe
nsor]]=None) -> HardwareAuditReport:
256         """Проведение полного аудита оборудования, драйверов и
привязки датчиков.
257
258         Args:
259             sensors: Опциональный список предварительно собран
ных сенсоров.
260
261         Returns:
262             HardwareAuditReport: Полный отчет по оборудованию.
263         """
264         all_sensors = sensors if sensors is not None else get_

```

```

hardware_sensors()
265     wmi_drivers = self._get_drivers_from_wmi()
266     device_audits: List[HardwareDeviceAudit] = []
267     problem_count = 0
268     outdated_count = 0
269     if self._setupapi:
270         try:
271             raw_devices = self._setupapi.get_all_devices(o
nly_present=True)
272             for dev in raw_devices:
273                 norm_id = dev.device_instance_id.strip().u
pper()
274                 driver_data = wmi_drivers.get(norm_id)
275                 driver_info: Optional[DriverInfo] = None
276                 if driver_data:
277                     date_dt = self._parse_wmi_date(
278                         driver_data.get('driver_date_raw')
,
279                         context=f"'{dev.friendly_name}' [{
norm_id}]"
280                     )
281                     date_str = date_dt.strftime('%Y-%m-%d'
) if date_dt else None
282                     provider = driver_data.get('provider',
'Unknown')
283                     currency_status, age_days = self._eval
uate_driver_currency(driver_date_dt=date_dt, provider=provider
, device_class=dev.device_class or 'Device')
284                     if 'устарел' in currency_status.lower(
):
285                         outdated_count += 1
286                         driver_info = DriverInfo(name=driver_d
ata.get('name') or dev.friendly_name, driver_version=driver_da
ta.get('driver_version', ''), driver_date=date_str, provider=p
rovider, inf_name=driver_data.get('inf_name'), is_signed=drive
r_data.get('is_signed', True), is_inbox='microsoft' in provide
r.lower(), age_days=age_days, currency_status=currency_status)
287                         install_date = self._get_device_install_da

```

```

te_registry(dev.device_instance_id, dev.friendly_name, dev.device_class)
288         if not install_date and driver_info and driver_info.driver_date:
289             install_date = driver_info.driver_date
290             status_str = 'Problem' if dev.has_problem
else 'OK'
291         if dev.has_problem:
292             problem_count += 1
293             audit_item = HardwareDeviceAudit(device_id=dev.device_instance_id, name=dev.friendly_name, device_class=dev.device_class or 'Device', manufacturer=dev.manufacturer or 'Unknown', install_date=install_date, status=status_str, problem_code=dev.problem_code, driver=driver_info, properties={'hardware_id': dev.hardware_id, 'status_code': dev.status_code})
294             audit_item.sensors = self._bind_sensors_to_device(audit_item, all_sensors)
295             device_audits.append(audit_item)
296         except Exception as ex:
297             logger.debug(f'Ошибка при аудите через SetupAPI: {ex}')
298         if not device_audits:
299             dummy_cpu = HardwareDeviceAudit(device_id='ROOT/CP
U/0000', name=platform.processor() or 'System Processor', device_class='Processor', manufacturer='Generic', install_date=datetime.now(timezone.utc).strftime('%d.%m.%Y %H:%M'), status='OK', problem_code=0, driver=DriverInfo(name='Processor Driver', driver_version='10.0.26100.1', driver_date='2024-05-01', provider='Microsoft', is_signed=True, is_inbox=True, age_days=100, currency_status='Актуален'))
300             dummy_cpu.sensors = self._bind_sensors_to_device(dummy_cpu, all_sensors)
301             device_audits.append(dummy_cpu)
302             summary_msg = f'Аудит завершен: обнаружено устройств: {len(device_audits)}, проблемных PnP: {problem_count}, с устаревшими драйверами: {outdated_count}'
303             return HardwareAuditReport(devices_count=len(device_audits), problem_devices_count=problem_count, outdated_drivers_c

```



```
ount=outdated_count, devices=device_audits, summary=summary_ms  
g)
```

304

hardware_history_manager.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry_Research
4 # - Hardware History Manager
5 # =====
6 # Description:
7 # Менеджер истории, архивации и детектора изменений аппаратного обеспечения.
8 #
9 # Usage Examples:
10 # Python API:
11 # from apps.windows.telemetry_research.hardware_history_manager import HardwareHistoryManager
12 #
13 # service = HardwareHistoryManager()
14 #
15 # File: hardware_history_manager.py
16 # Project: ai-breadboard
17 # Package: apps.windows.telemetry_research
18 # Author: hypo69
19 # Copyright: © 2026 hypo69
20 # Updated: 2026-10-01 13:28:28
21 # =====
22
23 from __future__ import annotations
24
25 """Менеджер истории, архивации и детектора изменений аппаратного обеспечения."""
26
27 import json
28 import os
29 import uuid
30
31 from datetime import datetime, timezone
32 from pathlib import Path
```

```

30 from typing import Any, Dict, List, Optional
31 from logger import logger
32 from apps.windows.telemetry.models import HardwareArchiveEntry
   , HardwareAuditReport, HardwareChangeItem, HardwareDeviceAudit
33
34
35 class HardwareHistoryManager:
36     """Менеджер архивов и отслеживания аппаратных изменений."""
   "
37
38     def __init__(self, archive_dir: Optional[Path] = None, max
   _archives: int = 100, use_file_index: bool = True) -> None:
39         """Инициализация менеджера истории оборудования.
40
41         Args:
42             archive_dir: Каталог для хранения архивных снимков
   (по умолчанию data/telemetry/hardware_archives).
43             max_archives: Максимальное количество хранимых арх
   ивов для ротации.
44             use_file_index: Использовать ли индексный файл арх
   ивов.
45         """
46         if archive_dir is None:
47             base_dir = Path(__file__).resolve().parent.parent.
   parent.parent
48             self.archive_dir = base_dir / 'data' / 'telemetry'
   / 'hardware_archives'
49         else:
50             self.archive_dir = Path(archive_dir)
51             self.max_archives = max(10, max_archives)
52             self.archive_dir.mkdir(parents=True, exist_ok=True)
53             self.index_file = self.archive_dir / 'archives_index.j
   sonl'
54             self._cached_latest_report: Optional[HardwareAuditRepo
   rt] = None
55             self.use_file_index = use_file_index
56
57     def detect_changes(self, current_report: HardwareAuditRepo

```

```

    rt, baseline_report: Optional[HardwareAuditReport]=None) -> Li
    st[HardwareChangeItem]:
58         """Сравнение двух снимков оборудования и выявление все
    х изменений (Diff Engine).
59
60         Args:
61             current_report: Текущий срез аудита оборудования.
62             baseline_report: Базовый срез для сравнения (если
    None, берется последний сохраненный).
63
64         Returns:
65             List[HardwareChangeItem]: Список зафиксированных и
    зменений.
66         """
67         base = baseline_report
68         if base is None:
69             latest_entry = self.get_latest_archive()
70             if latest_entry:
71                 base = latest_entry.report
72         if base is None:
73             return []
74         changes: List[HardwareChangeItem] = []
75         now_ts = datetime.now(timezone.utc).isoformat()
76         base_devs: Dict[str, HardwareDeviceAudit] = {d.device_
    id.upper(): d for d in base.devices}
77         curr_devs: Dict[str, HardwareDeviceAudit] = {d.device_
    id.upper(): d for d in current_report.devices}
78         for dev_id, dev in curr_devs.items():
79             if dev_id not in base_devs:
80                 changes.append(HardwareChangeItem(change_type=
    'added', device_id=dev.device_id, device_name=dev.name, descri
    ption=f'Подключено новое устройство: {dev.name} (Класс: {dev.d
    evice_class}, Производитель: {dev.manufacturer})', previous_va
    lue=None, current_value=dev.model_dump(), timestamp=now_ts))
81         for dev_id, dev in base_devs.items():
82             if dev_id not in curr_devs:
83                 changes.append(HardwareChangeItem(change_type=
    'removed', device_id=dev.device_id, device_name=dev.name, desc

```

```

ription=f'Отключено или удалено устройство: {dev.name} (Класс:
{dev.device_class})', previous_value=dev.model_dump(), current
t_value=None, timestamp=now_ts))
84         for dev_id, dev in curr_devs.items():
85             if dev_id in base_devs:
86                 old_dev = base_devs[dev_id]
87                 old_drv = old_dev.driver
88                 new_drv = dev.driver
89                 if old_drv and new_drv:
90                     if old_drv.driver_version != new_drv.driver_
r_version or old_drv.driver_date != new_drv.driver_date:
91                         desc = f"Обновлен драйвер для '{dev.name}': версия с '{old_drv.driver_version}' (от {old_drv.driver_
date or 'N/A'}) на '{new_drv.driver_version}' (от {new_drv.dri
ver_date or 'N/A'})"
92                         changes.append(HardwareChangeItem(change_type='driver_updated', device_id=dev.device_id, device_name
=dev.name, description=desc, previous_value=old_drv.model_dump
(), current_value=new_drv.model_dump(), timestamp=now_ts))
93                     elif not old_drv and new_drv:
94                         changes.append(HardwareChangeItem(change_t
ype='driver_updated', device_id=dev.device_id, device_name=dev
.name, description=f"Установлен драйвер для '{dev.name}': {new
_drv.driver_version} ({new_drv.provider})", previous_value=Non
e, current_value=new_drv.model_dump(), timestamp=now_ts))
95                     if old_dev.status != dev.status or old_dev.pro
blem_code != dev.problem_code:
96                         desc = f"Изменен статус устройства '{dev.n
ame}': было '{old_dev.status}' (код {old_dev.problem_code}) ->
стало '{dev.status}' (код {dev.problem_code})"
97                         changes.append(HardwareChangeItem(change_t
ype='status_changed', device_id=dev.device_id, device_name=dev
.name, description=desc, previous_value={'status': old_dev.sta
tus, 'problem_code': old_dev.problem_code}, current_value={'st
atus': dev.status, 'problem_code': dev.problem_code}, timestam
p=now_ts))
98         return changes
99

```

```

100     def archive_report(self, report: HardwareAuditReport, auto
|_diff: bool=True) -> HardwareArchiveEntry:
101         """Сохранение снимка аудита оборудования в архив с фик
|сацией изменений.
102
103         Args:
104             report: Отчет аудита оборудования.
105             auto_diff: Выполнять ли автоматическое вычисление
|изменений.
106
107         Returns:
108             HardwareArchiveEntry: Созданная запись архива.
109         """
110         if auto_diff:
111             changes = self.detect_changes(report)
112             report.changes_since_last_archive = changes
113             if changes:
114                 logger.info(f'Зафиксировано {len(changes)} изм
|енений в аппаратной конфигурации Windows!')
115                 for c in changes:
116                     logger.info(f' [Железо] {c.change_type.upp
|er(): {c.description}')
117                 archive_id = f"hw_{datetime.now(timezone.utc).strftime
|('%Y%m%d_%H%M%S')}-{uuid.uuid4().hex[:6]}"
118                 timestamp = datetime.now(timezone.utc).isoformat()
119                 entry = HardwareArchiveEntry(archive_id=archive_id, ti
|mestamp=timestamp, devices_count=report.devices_count, changes
|_count=len(report.changes_since_last_archive), report=report)
120                 archive_file = self.archive_dir / f'{archive_id}.json'
121                 try:
122                     with open(archive_file, 'w', encoding='utf-8') as
|f:
123                         json.dump(entry.model_dump(mode='json'), f, en
|sure_ascii=False, indent=2)
124                 except Exception as ex:
125                     logger.error(f'Ошибка сохранения архива оборудован
|ия в файл: {ex}')
126                 index_entry = {'archive_id': archive_id, 'timestamp':

```

```

timestamp, 'devices_count': report.devices_count, 'problem_dev
ices_count': report.problem_devices_count, 'outdated_drivers_c
ount': report.outdated_drivers_count, 'changes_count': len(rep
ort.changes_since_last_archive), 'file_name': f'{archive_id}.j
son'}
127         if self.use_file_index:
128             try:
129                 with open(self.index_file, 'a', encoding='utf-
130                     8') as f:
131                     f.write(json.dumps(index_entry, ensure_asc
132                         ii=False) + '\n')
133             except Exception as ex:
134                 logger.error(f'Ошибка записи индекса архивов о
135                     борудования: {ex}')
136             else:
137                 logger.warning('Запись в файл-индекс отключена кон
138                     фигурацией.')
139             self._cached_latest_report = report
140             self._rotate_archives()
141             return entry
142
143     def _rotate_archives(self) -> None:
144         """Ротация старых архивных файлов при превышении лимит
145             а."""
146         try:
147             json_files = sorted(self.archive_dir.glob('hw_*.js
148                 on'), key=os.path.getmtime)
149             if len(json_files) > self.max_archives:
150                 to_delete = json_files[:len(json_files) - self
151                     .max_archives]
152                 for p in to_delete:
153                     try:
154                         p.unlink()
155                     except OSError:
156                         pass
157         except Exception as ex:
158             logger.debug(f'Ошибка при ротации архивов оборудо
159                 вания: {ex}')

```

```

152
153     def get_latest_archive(self) -> Optional[HardwareArchiveEn
154 | try]:
155         """Получение самого свежего архива оборудования из хра
156 | нища.
157
158         Returns:
159         Optional[HardwareArchiveEntry]: Последний архив ил
160 | и None.
161         """
162         try:
163             json_files = sorted(self.archive_dir.glob('hw_*.js
164 | on'), key=os.path.getmtime, reverse=True)
165             if json_files:
166                 latest_path = json_files[0]
167                 with open(latest_path, 'r', encoding='utf-8')
168 | as f:
169                     data = json.load(f)
170                     return HardwareArchiveEntry.model_validate
171 | (data)
172             except Exception as ex:
173                 logger.error(f'Ошибка чтения последнего архива обо
174 | рудования: {ex}', exc_info=True)
175                 logger.error('Функция get_latest_archive вернула пусто
176 | й результат')
177                 return None
178
179     def get_history(self, limit: int=50) -> List[Dict[str, Any
180 | ]]:
181         """Получение сводного списка сохраненных архивов обо
182 | рудования.
183
184         Args:
185         limit: Максимальное количество записей архива.
186
187         Returns:
188         List[Dict[str, Any]]: Список метаданных архивов.
189         """

```



```

180         records: List[Dict[str, Any]] = []
181         if not self.index_file.exists():
182             return records
183         try:
184             with open(self.index_file, 'r', encoding='utf-8')
as f:
185                 for line in f:
186                     line = line.strip()
187                     if line:
188                         try:
189                             records.append(json.loads(line))
190                         except json.JSONDecodeError:
191                             continue
192         except Exception as ex:
193             logger.debug(f'Ошибка чтения индекса архивов: {ex}
')
194         return list(reversed(records))[:limit]
195
196     def get_archive_by_id(self, archive_id: str) -> Optional[HardwareArchiveEntry]:
197         """Получение конкретного полного архивного снимка по ID.
198
199         Args:
200             archive_id: Идентификатор архива.
201
202         Returns:
203             Optional[HardwareArchiveEntry]: Найденный архив или None.
204         """
205         safe_id = Path(archive_id).name
206         target = self.archive_dir / f'{safe_id}.json'
207         if not target.exists():
208             target = self.archive_dir / safe_id
209         if target.exists():
210             try:
211                 with open(target, 'r', encoding='utf-8') as f:
212                     data = json.load(f)

```

```

213         return HardwareArchiveEntry.model_validate
    (data)
214         except Exception as ex:
215             logger.debug(f'Ошибка загрузки архива {archive
_id}: {ex}')
216             logger.error('Функция get_archive_by_id вернула пустой
результат')
217             return None
218
219     def get_change_timeline(self, limit: int=100) -> List[Hard
wareChangeItem]:
220         """Получение сводного таймлайна всех изменений оборудо
вания из архивов.
221
222         Args:
223             limit: Лимит изменений для возврата.
224
225         Returns:
226             List[HardwareChangeItem]: Хронологический список з
афиксированных изменений.
227         """
228         all_changes: List[HardwareChangeItem] = []
229         json_files = sorted(self.archive_dir.glob('hw_*.json')
, key=os.path.getmtime, reverse=True)
230         for p in json_files:
231             try:
232                 with open(p, 'r', encoding='utf-8') as f:
233                     data = json.load(f)
234                     entry = HardwareArchiveEntry.model_validat
e(data)
235                     if entry.report and entry.report.changes_s
ince_last_archive:
236                         all_changes.extend(entry.report.change
s_since_last_archive)
237                     if len(all_changes) >= limit:
238                         break
239             except Exception:
240                 continue

```

```
241 |         return all_changes[:limit]
242 |
```

incident_detector.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry_Research
4 # - Incident Detector
5 # Description:
6 #   Модуль непрерывного анализа аномалий и автоматического выяв
7 #   ления сбоев системы.
8 # Usage Examples:
9 #   Python API:
10 #   from apps.windows.telemetry_research.incident_detector i
11 #   mport IncidentDetector
12 #   service = IncidentDetector()
13 #
14 # File: incident_detector.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry_research
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-01 13:28:28
20 # =====
21
22 from __future__ import annotations
23 """Модуль непрерывного анализа аномалий и автоматического выяв
24 ления сбоев системы."""
25
26 import time
27 import uuid
28 from datetime import datetime, timezone
29 from typing import Any, Dict, List, Optional, Tuple
```

```

30 try:
31     from logger import logger
32 except ImportError:
33     from logger import logger
34
35 from apps.windows.telemetry.models import TelemetryIncident
36 from apps.windows.telemetry.ring_buffer import TelemetryRingBu
37 ffer
38
39 class IncidentDetector:
40     """Детектор аномалий в потоках метрик и событий с фиксации инцидентов."""
41
42     def __init__(
43         self,
44         cpu_threshold: float = 90.0,
45         disk_write_burst_mb: float = 100.0,
46         net_burst_mb: float = 50.0,
47         ring_buffer: Optional[TelemetryRingBuffer] = None,
48     ) -> None:
49         """Инициализирует детектор пороговыми значениями и ссылкой на кольцевой буфер.
50
51         Args:
52             cpu_threshold: Порог срабатывания по CPU % (по умолчанию 90.0).
53             disk_write_burst_mb: Порог дисковой записи в МБ/с (по умолчанию 100.0).
54             net_burst_mb: Порог сетевого трафика в МБ/с (по умолчанию 50.0).
55             ring_buffer: Ссылка на экземпляр кольцевого буфера памяти.
56         """
57         self.cpu_threshold = cpu_threshold
58         self.disk_write_burst_mb = disk_write_burst_mb
59         self.net_burst_mb = net_burst_mb
60         self.ring_buffer = ring_buffer or TelemetryRingBuffer(

```

```

)
61         self._last_incident_epoch: float = 0.0
62         self._cooldown_seconds: float = 60.0 # Не спамить оди
наковыми инцидентами чаще раза в минуту
63
64     def check_anomaly(
65         self,
66         system_snapshot: Dict[str, Any],
67         processes: Optional[List[Dict[str, Any]]] = None,
68         recent_events: Optional[List[Dict[str, Any]]] = None,
69     ) -> Optional[TelemetryIncident]:
70         """Анализирует срез метрик и возвращает TelemetryIncident при обнаружении аномалии.
71
72         Args:
73             system_snapshot: Текущий снимок системы (SystemSnapshot dict).
74             processes: Список процессов текущего среза.
75             recent_events: Последние дискретные события (W64, ETW, Defender).
76
77         Returns:
78             Optional[TelemetryIncident]: Объект инцидента или None при штатной работе.
79         """
80         now = time.time()
81         cpu_pct = float(system_snapshot.get("cpu_total_percent") or system_snapshot.get("cpu_percent") or 0.0)
82         disk_w_mbs = float(system_snapshot.get("disk_write_bytes_sec") or 0.0) / (1024.0 * 1024.0)
83         net_tx_mbs = float(system_snapshot.get("network_sent_bytes_sec") or 0.0) / (1024.0 * 1024.0)
84         net_rx_mbs = float(system_snapshot.get("network_recv_bytes_sec") or 0.0) / (1024.0 * 1024.0)
85
86         trigger_type: Optional[str] = None
87         severity = "warning"
88         title = ""

```

```

89         description = ""
90         suspects: List[Dict[str, Any]] = []
91
92         # 1. Проверка CPU Spike
93         if cpu_pct >= self.cpu_threshold:
94             trigger_type = "cpu_spike"
95             severity = "critical" if cpu_pct >= 95.0 else "war
ning"
96             title = f"Критическая нагрузка на процессор: {cpu_
pct:.1f}%"
97             description = f"Общая загрузка CPU превысила порог
{self.cpu_threshold}%, достигнув {cpu_pct:.1f}%."
98
99         # 2. Проверка Disk Write Burst
100        elif disk_w_mbs >= self.disk_write_burst_mb:
101            trigger_type = "disk_burst"
102            severity = "warning"
103            title = f"Аномальная скорость записи на диск: {dis
k_w_mbs:.1f} МБ/с"
104            description = f"Скорость дисковой записи превысила
порог {self.disk_write_burst_mb} МБ/с."
105
106        # 3. Проверка Network Burst
107        elif (net_tx_mbs + net_rx_mbs) >= self.net_burst_mb:
108            trigger_type = "network_burst"
109            severity = "warning"
110            title = f"Всплеск сетевой активности: {(net_tx_mbs
+ net_rx_mbs):.1f} МБ/с"
111            description = f"Суммарный сетевой трафик достиг {(
net_tx_mbs + net_rx_mbs):.1f} МБ/с (TX: {net_tx_mbs:.1f}, RX:
{net_rx_mbs:.1f})."
112
113        # 4. Проверка подозрительных процессов (AppData, Temp,
Public)
114        if processes:
115            for p in processes:
116                exe_path = str(p.get("path") or p.get("exe") o
r "").lower()

```

```

117 |         p_cpu = float(p.get("cpu_percent") or 0.0)
118 |         is_suspicious_location = any(
119 |             loc in exe_path for loc in ("\\temp\\", "\\
|\\appdata\\local\\temp", "\\users\\public\\")
120 |         )
121 |         if is_suspicious_location and (p_cpu > 15.0 or
|trigger_type is not None):
122 |             suspects.append({
123 |                 "pid": p.get("pid"),
124 |                 "name": p.get("name"),
125 |                 "path": exe_path,
126 |                 "cpu_percent": p_cpu,
127 |                 "memory_mb": p.get("memory_mb"),
128 |                 "reason": "Запуск из временного катало
|га с повышенной активностью",
129 |             })
130 |             if trigger_type is None and p_cpu > 30.0:
131 |                 trigger_type = "suspicious_process"
132 |                 severity = "critical"
133 |                 title = f"Подозрительный процесс из Te
|mp: {p.get('name')} (PID {p.get('pid')})"
134 |                 description = f"Процесс {p.get('name')}
|} выполняется из {exe_path} и потребляет {p_cpu:.1f}% CPU."
135 |
136 |             elif p_cpu >= 50.0:
137 |                 suspects.append({
138 |                     "pid": p.get("pid"),
139 |                     "name": p.get("name"),
140 |                     "path": exe_path,
141 |                     "cpu_percent": p_cpu,
142 |                     "memory_mb": p.get("memory_mb"),
143 |                     "reason": f"Высокое индивидуальное пот
|ребление CPU: {p_cpu:.1f}%",
144 |                 })
145 |
146 |     # 5. Проверка событий безопасности (Defender / ETW)
147 |     matched_events: List[Dict[str, Any]] = []
148 |     if recent_events:

```



```
149         for ev in recent_events:
150             ev_type = str(ev.get("event_type") or "").lower()
151             ev_str = str(ev).lower()
152             if "defender" in ev_type or "threat" in ev_str
or "malware" in ev_str:
153                 matched_events.append(ev)
154                 if trigger_type is None:
155                     trigger_type = "defender_alert"
156                     severity = "critical"
157                     title = "Срабатывание Windows Defender
/ Защиты от угроз"
158                     description = f"Зафиксировано событие
безопасности: {ev.get('name') or ev_type}"
159
160             if not trigger_type:
161                 return None
162
163             # Проверка кулдауна, чтобы не генерировать дубликаты е
жесекундно
164             if (now - self._last_incident_epoch) < self._cooldown_
seconds:
165                 return None
166
167             self._last_incident_epoch = now
168             inc_id = f"INC-{{datetime.now(timezone.utc).strftime('%
Y%m%d%H%M%S')}}-{{uuid.uuid4().hex[:4].upper()}}"
169
170             # Захватываем срез из кольцевого буфера (например, 5 м
инут до события)
171             raw_window = self.ring_buffer.get_window(center_timest
amp=now, before_seconds=300.0, after_seconds=0.0)
172
173             incident = TelemetryIncident(
174                 incident_id=inc_id,
175                 timestamp=datetime.now(timezone.utc).isoformat(),
176                 trigger_type=trigger_type,
177                 severity=severity,
```

```

178         title=title,
179         description=description,
180         trigger_metrics={
181             "cpu_percent": cpu_pct,
182             "disk_write_mbs": disk_w_mbs,
183             "network_tx_mbs": net_tx_mbs,
184             "network_rx_mbs": net_rx_mbs,
185         },
186         suspect_processes=suspects,
187         related_events=matched_events or (recent_events[:1
188 0] if recent_events else []),
189         metrics_summary={
190             "buffer_samples_captured": len(raw_window),
191             "peak_cpu": cpu_pct,
192             "peak_disk_write_mbs": disk_w_mbs,
193         },
194         raw_window=raw_window,
195     )
196     logger.warning(f"Зафиксирован инцидент телеметрии [{incident.incident_id}]: {incident.title}")
197     return incident
198
199     @staticmethod
200     def format_incident_markdown_for_llm(incident: TelemetryIncident) -> str:
201         """Форматирует инцидент в компактный структурированный
202         Markdown для AI / LLM анализа.
203
204         Args:
205             incident: Объект TelemetryIncident.
206
207         Returns:
208             str: Готовый markdown блок для контекста языковой
209             модели.
210
211         """
212         lines = [
213             f"### □ ДИАГНОСТИЧЕСКИЙ ИНЦИДЕНТ: [{incident.incident_id}]

```

```

dent_id}]",
211         f"- **Тип триггера:** `{incident.trigger_type}` |
    **Критичность:** `{incident.severity.upper()}`",
212         f"- **Время:** {incident.timestamp}",
213         f"- **Сводка:** {incident.title}",
214         f"- **Описание:** {incident.description}",
215         "",
216         "#### □ Метрики в момент инцидента:",
217         f"- CPU: `{incident.trigger_metrics.get('cpu_perce
nt', 0.0)}%`",
218         f"- Диск (Запись): `{incident.trigger_metrics.get(
'disk_write_mbs', 0.0)} МБ/с`",
219         f"- Сеть (TX / RX): `{incident.trigger_metrics.get
('network_tx_mbs', 0.0)} МБ/с` / `{incident.trigger_metrics.ge
t('network_rx_mbs', 0.0)} МБ/с`",
220         f"- Сохранено секундных замеров в буфере: `{incide
nt.metrics_summary.get('buffer_samples_captured', 0)}`",
221         "",
222     ]
223
224     if incident.suspect_processes:
225         lines.append("#### □ Подозрительные процессы:")
226         lines.append("| PID | Имя | Путь | CPU % | Причина
|")
227         lines.append("|---|---|---|---|---|")
228         for sp in incident.suspect_processes:
229             lines.append(
230                 f"| {sp.get('pid', '-')} | `{sp.get('name'
, '-')}` | `{sp.get('path', '-')}` | {sp.get('cpu_percent', 0.
0)}% | {sp.get('reason', '-')} |"
231             )
232             lines.append("")
233
234     if incident.related_events:
235         lines.append("#### □ Сопутствующие события систем
ы (Events):")
236         for ev in incident.related_events[:5]:
237             ev_name = ev.get("name") or ev.get("event_type")

```

```
) or "event"
238         ev_time = ev.get("timestamp", "")
239         lines.append(f"- `{ev_time}`: **{ev_name}** ({
ev.get('path') or ev.get('details') or ''})")
240         lines.append("")
241
242         lines.append("Инструкция: Оцените потенциальную угрозу
(вирус/майнер/зависание/утечка) и предложите конкретные меры
противодействия.")
243         return "\n".join(lines)
244
```

main.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry_Research
  - Main
4 # =====
5 # Description:
6 #   Главная точка входа для запуска Web GUI и FastAPI сервиса
  исследования телеметрии.
7 #
8 # Usage Examples:
9 #   CLI:
10 #       python -m apps.windows.telemetry_research.main
11 #   Python API:
12 #       from apps.windows.telemetry_research.main import parse_a
  rgs
13 #
14 #       res = parse_args()
15 #
16 # File: main.py
17 # Project: ai-breadboard
18 # Package: apps.windows.telemetry_research
19 # Author: hypo69
20 # Copyright: © 2026 hypo69
21 # Updated: 2026-10-01 13:28:28
22 # =====
23
24 from __future__ import annotations
25 """Главная точка входа для запуска Web GUI и FastAPI сервиса и
  сследования телеметрии."""
26
27 import argparse
28 import sys
29 from pathlib import Path
```

```

30
31 _project_root = str(Path(__file__).resolve().parents[3])
32 if _project_root not in sys.path:
33     sys.path.insert(0, _project_root)
34
35 try:
36     from logger import logger
37 except ImportError:
38     from logger import logger
39
40
41 def parse_args() -> argparse.Namespace:
42     """Парсинг аргументов запуска веб-сервера."""
43     parser = argparse.ArgumentParser(
44         description="Запуск Web GUI и FastAPI сервера исследов
45 ания телеметрии",
46         formatter_class=argparse.RawDescriptionHelpFormatter,
47     )
48     parser.add_argument("--host", type=str, default="127.0.0.1",
49 help="Хост для биндинга (по умолчанию: 127.0.0.1)")
50     parser.add_argument("--port", type=int, default=8090, help=
51 "Порт для веб-сервера (по умолчанию: 8090)")
52     parser.add_argument("--reload", action="store_true", help=
53 "Включить автоперезагрузку при изменении кода")
54     return parser.parse_args()
55
56
57 def main() -> int:
58     """Запуск FastAPI сервера через Uvicorn."""
59     args = parse_args()
60     try:
61         import uvicorn
62     except ImportError:
63         logger.error("Для запуска Web GUI необходим пакет 'uvi
64 corn'. Установите его: pip install uvicorn")
65         print("❌ Ошибка: пакет 'uvicorn' не установлен. Устано
66 вите: pip install uvicorn")
67     return 1

```

```
62 |
63 |     logger.info("=" * 65)
64 |     logger.info(f"    Запуск Web GUI исследования телеметрии на
    | http://{args.host}:{args.port}")
65 |     logger.info("=" * 65)
66 |     print("\n" + "=" * 65)
67 |     print(f"    Web GUI исследования телеметрии доступен по адр
    | есу:")
68 |     print(f"    http://{args.host}:{args.port}")
69 |     print("=" * 65 + "\n")
70 |
71 |     uvicorn.run(
72 |         "apps.windows.telemetry_research.server:app",
73 |         host=args.host,
74 |         port=args.port,
75 |         reload=args.reload,
76 |         log_level="info",
77 |     )
78 |     return 0
79 |
80 |
81 | if __name__ == "__main__":
82 |     sys.exit(main())
83 |
```

models.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry_Research
4 # - Models
5 # Description:
6 #   Модели данных для исследования телеметрии и построения гра
7 #   фиков.
8 # Usage Examples:
9 #   Python API:
10 #   from apps.windows.telemetry_research.models import Metri
11 #   cPoint
12 #   service = MetricPoint()
13 #
14 # File: models.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry_research
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-04 07:47:00
20 # =====
21
22 from __future__ import annotations
23 """Модели данных для исследования телеметрии, ресурсов клиентс
24   ких программ и построения графиков."""
25
26 from datetime import datetime
27 from typing import Any, Dict, List, Optional
28 from pydantic import BaseModel, Field
29
```



```

30 class MetricPoint(BaseModel):
31     """Единичная точка временного ряда для метрики."""
32     timestamp: str = Field(..., description="Временная метка т
очки (ISO-8601 или formatted)")
33     value: float = Field(..., description="Числовое значение м
етрики")
34     label: Optional[str] = Field(default=None, description="Оп
циональная подпись или тег")
35
36
37 class TimeSeriesDataset(BaseModel):
38     """Набор данных временного ряда для графика."""
39     name: str = Field(..., description="Имя метрики или серии
данных")
40     unit: str = Field(default="%", description="Единица измере
ния (% , MB , °C , MB/s , ops/s)")
41     points: List[MetricPoint] = Field(default_factory=list, de
scription="Список точек временного ряда")
42     color: Optional[str] = Field(default=None, description="HE
X-код цвета серии для визуализации")
43
44
45 class AnomalyEvent(BaseModel):
46     """Зафиксированная аномалия или всплеск нагрузки в логах."
"""
47     timestamp: str = Field(..., description="Время фиксации ан
омалии")
48     metric: str = Field(..., description="Название метрики с а
номалией")
49     value: float = Field(..., description="Фактическое значени
е в момент всплеска")
50     threshold: float = Field(..., description="Пороговое или н
ормальное значение")
51     severity: str = Field(default="warning", description="Уров
ень критичности (info, warning, critical)")
52     description: str = Field(..., description="Понятное описан
ие выявленной проблемы")
53

```

```

54
55 class MetricStats(BaseModel):
56     """Сводная описательная статистика по метрике (EDA)."""
57     count: int = Field(default=0, description="Количество заме
ров")
58     min_val: float = Field(default=0.0, description="Минимальн
ое значение")
59     max_val: float = Field(default=0.0, description="Максималь
ное значение")
60     avg_val: float = Field(default=0.0, description="Среднее а
рифметическое значение")
61     median_val: float = Field(default=0.0, description="Медиан
ное значение")
62     p95_val: float = Field(default=0.0, description="95-й перц
ентиль")
63     std_dev: float = Field(default=0.0, description="Стандартн
ое отклонение")
64     unit: str = Field(default="%", description="Единица измере
ния")
65
66
67 class DeviceEventSummary(BaseModel):
68     """Сводная информация по событиям устройств (флаппинг, отк
лючения, сбои)."""
69     total_events: int = Field(default=0, description="Всего со
бытий устройств")
70     error_count: int = Field(default=0, description="Количеств
о событий со сбоями")
71     by_category: Dict[str, int] = Field(default_factory=dict,
description="Распределение по категориям устройств")
72     by_event_type: Dict[str, int] = Field(default_factory=dict
, description="Распределение по типам событий")
73     flapping_devices: List[str] = Field(default_factory=list,
description="Список устройств с частыми переподключениями")
74
75
76 class ChartConfig(BaseModel):
77     """Конфигурация графика для фронтенда / Chart.js."""

```

```

78     id: str = Field(..., description="Уникальный идентификатор
    графика")
79     title: str = Field(..., description="Заголовок графика")
80     chart_type: str = Field(default="line", description="Тип г
    рафика (line, bar, doughnut, radar, heatmap)")
81     labels: List[str] = Field(default_factory=list, descriptio
    n="Подписи оси X (временные метки)")
82     datasets: List[Dict[str, Any]] = Field(default_factory=lis
    t, description="Массив датасетов с данными и стилями")
83     y_axis_label: str = Field(default="", description="Подпись
    оси Y")
84     description: Optional[str] = Field(default=None, descripti
    on="Описание или аналитические выводы по графику")
85
86
87 class CorrelationMatrixItem(BaseModel):
88     """Элемент матрицы корреляции между парой метрик."""
89     metric_a: str = Field(description="Идентификатор первой ме
    трики")
90     metric_b: str = Field(description="Идентификатор второй ме
    трики")
91     coefficient: float = Field(description="Коэффициент коррел
    яции Пирсона (-1.0 ... 1.0)")
92     sample_size: int = Field(description="Число общих точек вр
    еменного ряда")
93     interpretation: str = Field(description="Текстовая интерпр
    етация взаимосвязи")
94
95
96 class HypothesisResult(BaseModel):
97     """Результат проверки аналитической гипотезы о поведении с
    истемы."""
98     hypothesis_id: str = Field(description="Уникальный идентиф
    икатор гипотезы")
99     title: str = Field(description="Краткое название гипотезы"
    )
100     description: str = Field(description="Подробная формулиров
    ка гипотезы")

```

```

101     confirmed: bool = Field(description="Подтверждена ли гипот
еза на основе собранных данных")
102     confidence: float = Field(description="Уровень уверенности
(0.0 - 1.0)")
103     evidence: List[str] = Field(default_factory=list, descript
ion="Список подтверждающих или опровергающих фактов")
104     recommendation: Optional[str] = Field(default=None, descri
ption="Практическая рекомендация по устранению проблемы")
105
106
107 class ResearchScenarioRequest(BaseModel):
108     """Параметры запроса для проведения исследовательского сце
нария."""
109     source_path: Optional[str] = Field(default=None, descripti
on="Путь к файлу логов, директории или шаблону поиска")
110     records: Optional[List[Dict[str, Any]]] = Field(default=No
ne, description="Массив сырых записей телеметрии, переданный н
апрямую")
111     start_time: Optional[str] = Field(default=None, descriptio
n="Начало временного интервала фильтрации (ISO 8601)")
112     end_time: Optional[str] = Field(default=None, description=
"Конец временного интервала фильтрации (ISO 8601)")
113     subsystems: Optional[List[str]] = Field(default=None, desc
ription="Список исследуемых подсистем (cpu, ram, gpu, disk, ne
twork, devices, sensors)")
114     cpu_anomaly_threshold: float = Field(default=90.0, descrip
tion="Порог загрузки CPU (%) для детекции аномалий")
115     temp_anomaly_threshold: float = Field(default=85.0, descri
ption="Порог температуры (°C) для детекции перегрева")
116     enable_hypotheses_check: bool = Field(default=True, descri
ption="Флаг активации автоматической проверки системных гипоте
з")
117
118
119 class TelemetryResearchReport(BaseModel):
120     """Итоговый отчет исследования логов телеметрии."""
121     report_id: str = Field(..., description="Идентификатор отч
ета исследования")

```

```
122     generated_at: str = Field(..., description="Время формирова  
ания отчета")  
123     records_analyzed: int = Field(default=0, description="Коли  
чество проанализированных записей логов")  
124     time_window_start: Optional[str] = Field(default=None, des  
cription="Начало исследуемого временного окна")  
125     time_window_end: Optional[str] = Field(default=None, descr  
iption="Окончание исследуемого временного окна")  
126     statistics: Dict[str, MetricStats] = Field(default_factory  
=dict, description="Статистические профили метрик")  
127     anomalies: List[AnomalyEvent] = Field(default_factory=list  
, description="Список выявленных аномалий")  
128     device_summary: DeviceEventSummary = Field(default_factory  
=DeviceEventSummary, description="Сводка событий устройств")  
129     charts: List[ChartConfig] = Field(default_factory=list, de  
scription="Список сгенерированных графиков")  
130     health_score: float = Field(default=100.0, description="Ит  
оговый индекс здоровья системы (0-100)")  
131     summary_conclusions: List[str] = Field(default_factory=lis  
t, description="Ключевые аналитические выводы исследования")  
132  
133  
134 class DeepResearchReport(BaseModel):  
135     """Расширенный отчет глубокого исследования телеметрии."""  
136     report_id: str = Field(..., description="Уникальный иденти  
фикатор отчета исследования")  
137     generated_at: str = Field(..., description="Время формирова  
ния отчета (ISO 8601)")  
138     base_report: TelemetryResearchReport = Field(..., descript  
ion="Базовый отчет статистического анализа телеметрии")  
139     correlations: List[CorrelationMatrixItem] = Field(default_  
factory=list, description="Матрица корреляций между ключевыми  
системными метриками")  
140     hypotheses: List[HypothesisResult] = Field(default_factory  
=list, description="Результаты автоматической проверки гипотез  
производительности")  
141     investigation_summary: str = Field(..., description="Сводн  
ое экспертное заключение исследования")
```

```
142     actionable_recommendations: List[str] = Field(default_factory=list, description="Список практических рекомендаций по оптимизации и обслуживанию системы")
143
144
145 class UnknownProgramEvaluation(BaseModel):
146     """Оценка неизвестной или ресурсоемкой программы языковой моделью Gemini."""
147     program_name: str = Field(..., description="Имя исполняемого файла или процесса")
148     pid: Optional[int] = Field(default=None, description="Идентификатор процесса (PID)")
149     executable_path: Optional[str] = Field(default=None, description="Полный путь к исполняемому файлу")
150     command_line: Optional[str] = Field(default=None, description="Командная строка запуска")
151     cpu_percent: float = Field(default=0.0, description="Потребление ЦП (%)")
152     memory_mb: float = Field(default=0.0, description="Использование оперативной памяти (МБ)")
153     disk_io_mb_s: float = Field(default=0.0, description="Интенсивность дискового ввода-вывода (МБ/с)")
154     disk_size_mb: Optional[float] = Field(default=None, description="Размер файла на диске (МБ)")
155     is_normal: bool = Field(default=True, description="Является ли такое поведение штатным и нормальным")
156     confidence: float = Field(default=0.85, description="Уверенность модели в оценке (0.0 - 1.0)")
157     risk_level: str = Field(default="low", description="Уровень риска: low, medium, high, critical")
158     verdict: str = Field(..., description="Краткий вердикт модели (например: 'Штатная нагрузка', 'Подозрительный процесс')")
159     analysis: str = Field(..., description="Развернутое экспертное заключение модели о потреблении ресурсов")
160     recommendations: List[str] = Field(default_factory=list, description="Список практических рекомендаций для пользователя")
161     evaluated_at: str = Field(default_factory=lambda: datetime
```

```

    .now().isoformat(), description="Временная метка проведения оц
енки")
162     model_used: str = Field(default="gemini-3.1-flash", descri
ption="Использованная модель ИИ")
163
164
165 class ClientProgramResource(BaseModel):
166     """Снимок ресурсоемкости отдельной клиентской/пользователь
ской программы."""
167     pid: int = Field(..., description="PID процесса")
168     name: str = Field(..., description="Имя процесса (например
, chrome.exe, code.exe)")
169     display_name: Optional[str] = Field(default=None, descript
ion="Понятное отображаемое имя программы")
170     executable_path: Optional[str] = Field(default=None, descr
iption="Путь к исполняемому файлу")
171     command_line: Optional[str] = Field(default=None, descript
ion="Командная строка запуска")
172     username: Optional[str] = Field(default=None, description=
"Учетная запись пользователя, запустившего программу")
173     cpu_percent: float = Field(default=0.0, description="Текущ
ая / средняя загрузка ЦП (%)")
174     memory_mb: float = Field(default=0.0, description="Использ
ование оперативной памяти RAM (МБ)")
175     memory_percent: float = Field(default=0.0, description="До
ля от общей оперативной памяти (%)")
176     disk_read_bytes_sec: float = Field(default=0.0, descriptio
n="Скорость чтения с диска (байт/с)")
177     disk_write_bytes_sec: float = Field(default=0.0, descripti
on="Скорость записи на диск (байт/с)")
178     disk_size_mb: Optional[float] = Field(default=None, descri
ption="Размер исполняемого файла на диске (МБ)")
179     num_threads: int = Field(default=0, description="Количеств
о потоков процесса")
180     num_handles: int = Field(default=0, description="Количеств
о дескрипторов процесса")
181     is_known_software: bool = Field(default=True, description=
"Флаг: известное доверенное приложение")

```

```
182     is_client_program: bool = Field(default=True, description=
"Флаг: программа запущена клиентом/пользователем")
183     is_resource_heavy: bool = Field(default=False, description
="Флаг: повышенное потребление ресурсов (CPU/RAM/Disk)")
184     resource_score: float = Field(default=0.0, description="Ин
тегральный скор ресурсоемкости")
185     category: str = Field(default="Пользовательское приложение
", description="Категория ПО")
186     publisher: Optional[str] = Field(default=None, description
="Издатель / разработчик программы")
187     gemini_evaluation: Optional[UnknownProgramEvaluation] = Fi
eld(default=None, description="Оценка от Gemini при аномалиях"
)
188
189
190 class ClientResourceSummary(BaseModel):
191     """Сводный отчет по ресурсам программ, запущенных клиентом
. """
192     total_client_programs: int = Field(default=0, description=
"Общее число клиентских процессов")
193     total_ram_mb: float = Field(default=0.0, description="Сумм
арно занятая память RAM клиентскими программами (МБ)")
194     total_cpu_percent: float = Field(default=0.0, description=
"Суммарная загрузка ЦП клиентскими программами (%)")
195     total_disk_size_mb: float = Field(default=0.0, description
="Суммарный объем файлов на диске (МБ)")
196     heavy_programs_count: int = Field(default=0, description="
Число ресурсоемких программ")
197     unknown_heavy_programs_count: int = Field(default=0, descr
iption="Число неизвестных ресурсоемких программ")
198     programs: List[ClientProgramResource] = Field(default_fact
ory=list, description="Список программ с детальными метриками"
)
199     evaluations: List[UnknownProgramEvaluation] = Field(defaul
t_factory=list, description="Список полученных оценок от Gemin
i")
200     scanned_at: str = Field(default_factory=lambda: datetime.n
ow().isoformat(), description="Время проведения аудита")
```


201 |

202 |

query_engine.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry_Research
4 # - Query Engine
5 # Description:
6 #   Движок SQL-запросов и аналитических расчетов по базе данны
7 #   x telemetry.db.
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry_research.query_engine import
11 #         TelemetryQueryEngine
12 #       service = TelemetryQueryEngine()
13 #
14 # File: query_engine.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry_research
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-04 07:47:00
20 # =====
21
22 from __future__ import annotations
23 """Движок SQL-запросов и аналитических расчетов по базе данных
24   telemetry.db."""
25
26 import sqlite3
27 from datetime import datetime, timezone
28 from pathlib import Path
29 from typing import Any, Dict, List, Optional, Union
```

```

30 try:
31     from logger import logger
32 except ImportError:
33     import logging
34     logger = logging.getLogger(__name__)
35
36 from .extractor import TelemetryDataExtractor
37
38
39 class TelemetryQueryEngine:
40     """Аналитический движок для выборки данных и выполнения SQL-запросов к telemetry.db."""
41
42     def __init__(self, extractor: Optional[TelemetryDataExtractor] = None) -> None:
43         """Инициализация движка запросов.
44
45         Args:
46             extractor: Экземпляр извлекателя данных телеметрии
47             .
48             """
49         self.extractor = extractor or TelemetryDataExtractor()
50
51     def get_db_path(self, custom_path: Optional[Union[str, Path]] = None) -> Optional[Path]:
52         """Получить актуальный путь к файлу SQLite базы данных
53         .
54
55         Args:
56             custom_path: Пользовательский путь к БД.
57
58         Returns:
59             Optional[Path]: Валидный путь к файлу базы данных.
60             """
61         if custom_path:
62             p = Path(custom_path)
63             if p.exists() and p.is_file():
64                 return p

```

```

63         return self.extractor.get_default_db_path()
64
65     def execute_safe_sql(
66         self, sql_query: str, db_path: Optional[Union[str, Path]] = None, limit: int = 500
67     ) -> Dict[str, Any]:
68         """Безопасное выполнение произвольного SELECT SQL-запроса к базе телеметрии.
69
70         Args:
71             sql_query: Строка SQL-запроса (только SELECT).
72             db_path: Опциональный путь к базе данных.
73             limit: Максимальное число возвращаемых строк.
74
75         Returns:
76             Dict[str, Any]: Результат запроса со столбцами, строками и метаданными.
77         """
78         target_db = self.get_db_path(db_path)
79         if not target_db:
80             return {
81                 "status": "error",
82                 "message": "База данных telemetry.db не найдена",
83                 "columns": [],
84                 "rows": [],
85                 "count": 0,
86             }
87
88         cleaned_sql = sql_query.strip().rstrip(";")
89         lower_sql = cleaned_sql.lower()
90
91         # Разрешаем только SELECT запросы
92         if not lower_sql.startswith("select") and not lower_sql.startswith("with"):
93             return {
94                 "status": "error",
95                 "message": "Разрешены только запросы на чтение

```

```

    данных (SELECT)",
96         "columns": [],
97         "rows": [],
98         "count": 0,
99     }
100
101     # Ограничиваем количество записей
102     if "limit" not in lower_sql:
103         cleaned_sql = f"{cleaned_sql} LIMIT {min(limit, 10
104         00)}"
105
106     try:
107         conn = sqlite3.connect(str(target_db), timeout=5.0
108         )
109         conn.row_factory = sqlite3.Row
110         cursor = conn.cursor()
111         cursor.execute(cleaned_sql)
112
113         rows = cursor.fetchall()
114         columns = [col[0] for col in cursor.description] i
115         f cursor.description else []
116         data = [dict(r) for r in rows]
117         conn.close()
118
119         return {
120             "status": "ok",
121             "sql": cleaned_sql,
122             "columns": columns,
123             "rows": data,
124             "count": len(data),
125         }
126     except Exception as ex:
127         logger.warning(f"Ошибка выполнения SQL-запроса '{c
128         leaned_sql}': {ex}")
129         return {
130             "status": "error",
131             "sql": cleaned_sql,
132             "message": str(ex),

```

```

129         "columns": [],
130         "rows": [],
131         "count": 0,
132     }
133
134     def get_cpu_timeline(
135         self, limit: int = 120, db_path: Optional[Union[str, Path]] = None
136     ) -> Dict[str, Any]:
137         """Получить временной ряд загрузки процессора и частоты для построения графика.
138
139         Args:
140             limit: Количество точек замера (по умолчанию 120).
141             db_path: Путь к базе данных.
142
143         Returns:
144             Dict[str, Any]: Структурированные временные ряды, метки времени и статистика.
145         """
146         target_db = self.get_db_path(db_path)
147         if not target_db:
148             return {"status": "error", "message": "telemetry.db не найдена"}
149
150         sql = """
151             SELECT id, timestamp, cpu_total_percent, cpu_frequency_mhz, uptime_seconds
152             FROM system_snapshots
153             ORDER BY id DESC
154             LIMIT ?
155         """
156         try:
157             conn = sqlite3.connect(str(target_db), timeout=5.0)
158
159             conn.row_factory = sqlite3.Row
160             cursor = conn.cursor()
161             cursor.execute(sql, (max(10, min(limit, 2000)),))

```

```

161         raw_rows = [dict(r) for r in cursor.fetchall()]
162         conn.close()
163
164         if not raw_rows:
165             return {
166                 "status": "empty",
167                 "message": "В таблице system_snapshots нет
записей",
168                 "labels": [],
169                 "cpu_load": [],
170                 "cpu_freq_ghz": [],
171                 "sql": sql.strip(),
172             }
173
174         # Хронологический порядок: от старых к новым
175         chronological = list(reversed(raw_rows))
176         labels: List[str] = []
177         cpu_load: List[float] = []
178         cpu_freq_ghz: List[float] = []
179
180         for row in chronological:
181             ts_str = str(row.get("timestamp") or "")
182             time_label = ts_str[11:19] if len(ts_str) >= 1
9 else ts_str
183             labels.append(time_label)
184
185             load = round(float(row.get("cpu_total_percent"
) or 0.0), 1)
186             cpu_load.append(load)
187
188             mhz = float(row.get("cpu_frequency_mhz") or 0.
0)
189             freq_ghz = round(mhz / 1000.0, 2) if mhz > 0 e
lse 0.0
190             cpu_freq_ghz.append(freq_ghz)
191
192             avg_cpu = round(sum(cpu_load) / len(cpu_load), 1)
if cpu_load else 0.0

```

```

193 |         max_cpu = max(cpu_load) if cpu_load else 0.0
194 |         min_cpu = min(cpu_load) if cpu_load else 0.0
195 |
196 |         return {
197 |             "status": "ok",
198 |             "sql": sql.strip(),
199 |             "labels": labels,
200 |             "cpu_load": cpu_load,
201 |             "cpu_freq_ghz": cpu_freq_ghz,
202 |             "chart": {
203 |                 "type": "line",
204 |                 "title": f"График загрузки и частоты ЦП по
| времени (Ср: {avg_cpu}%, Макс: {max_cpu}%)",
205 |                 "labels": labels,
206 |                 "datasets": [
207 |                     {
208 |                         "label": "Загрузка ЦП (%)",
209 |                         "data": cpu_load,
210 |                         "borderColor": "#38bdf8",
211 |                         "backgroundColor": "rgba(56, 189,
| 248, 0.15)",
212 |                         "borderWidth": 2,
213 |                         "fill": True,
214 |                         "tension": 0.3,
215 |                         "yAxisID": "y",
216 |                     },
217 |                     {
218 |                         "label": "Частота ЦП (GHz)",
219 |                         "data": cpu_freq_ghz,
220 |                         "borderColor": "#f59e0b",
221 |                         "backgroundColor": "transparent",
222 |                         "borderWidth": 2,
223 |                         "borderDash": [4, 4],
224 |                         "tension": 0.2,
225 |                         "yAxisID": "y1",
226 |                     },
227 |                 ],
228 |             },

```



```

229         "statistics": {
230             "avg_cpu_percent": avg_cpu,
231             "max_cpu_percent": max_cpu,
232             "min_cpu_percent": min_cpu,
233             "points_count": len(labels),
234             "start_time": chronological[0].get("timestamp"),
235             "end_time": chronological[-1].get("timestamp"),
236         },
237     }
238     except Exception as ex:
239         logger.error(f"Ошибка извлечения таймлайна CPU: {ex}")
240         return {"status": "error", "message": str(ex), "sql": sql.strip()}
241
242     def calculate_power_consumption_24h(
243         self, db_path: Optional[Union[str, Path]] = None, hours: int = 24
244     ) -> Dict[str, Any]:
245         """Рассчитать общее потребление электроэнергии и подготовить суточную круговую диаграмму.
246
247         Интегрирует показания сенсоров мощности (Вт) из sensor_polls во времени
248         и рассчитывает общее количество ватт-часов (Вт·ч) и кВт·ч, а также долю каждого компонента.
249
250         Args:
251             db_path: Путь к базе данных.
252             hours: Период анализа в часах (по умолчанию 24).
253
254         Returns:
255             Dict[str, Any]: Аналитический отчет, кВт·ч, разбивка по компонентам для круговой диаграммы.
256         """
257         target_db = self.get_db_path(db_path)

```



```

290         AVG(gpu_load_percent) as avg_gpu_load
291     FROM (
292         SELECT * FROM system_snapshots
293         ORDER BY id DESC LIMIT 1000
294     )
295     """
296     cursor.execute(sql_snapshots)
297     snap_stat = dict(cursor.fetchone() or {})
298     conn.close()
299
300     # 3. Анализ и распределение мощности
301     cpu_cores_w = 0.0
302     cpu_package_w = 0.0
303     cpu_memory_w = 0.0
304     gpu_w = 0.0
305     other_sensors_w = 0.0
306
307     for s in sensor_power_rows:
308         s_name = (s.get("sensor_name") or "").lower()
309         avg_w = float(s.get("avg_watts") or 0.0)
310         if "cpu cores" in s_name or "core power" in s_
name:
311             cpu_cores_w = max(cpu_cores_w, avg_w)
312             elif "cpu package" in s_name or "package power
" in s_name:
313                 cpu_package_w = max(cpu_package_w, avg_w)
314                 elif "cpu memory" in s_name or "dram" in s_nam
e:
315                     cpu_memory_w = max(cpu_memory_w, avg_w)
316                     elif "gpu" in s_name or "graphics" in s_name:
317                         gpu_w = max(gpu_w, avg_w)
318                     else:
319                         other_sensors_w += avg_w
320
321     avg_cpu_load = float(snap_stat.get("avg_cpu") or 1
5.0)
322     if cpu_package_w <= 0.0:
323         cpu_total_power = round(10.0 + 55.0 * (avg_cpu

```

```

    _load / 100.0), 2)
324         cpu_cores_w = round(cpu_total_power * 0.75, 2)
325         cpu_uncore_w = round(cpu_total_power * 0.25, 2
    )
326     else:
327         cpu_total_power = round(cpu_package_w, 2)
328         if cpu_cores_w <= 0.0:
329             cpu_cores_w = round(cpu_package_w * 0.75,
    2)
330             cpu_uncore_w = max(0.0, round(cpu_package_w -
    cpu_cores_w, 2))
331
332         if gpu_w <= 0.0:
333             avg_gpu_load = float(snap_stat.get("avg_gpu_lo
    ad") or 5.0)
334             gpu_w = round(5.0 + 35.0 * (avg_gpu_load / 100
    .0), 2)
335         else:
336             gpu_w = round(gpu_w, 2)
337
338         avg_ram_gb = float(snap_stat.get("avg_ram_gb") or
    8.0)
339         ram_power_w = round(max(2.0, avg_ram_gb * 0.4), 2)
    # ~0.4W на 1 GB RAM
340         motherboard_chipset_w = 12.0 # Чипсет материнской
    платы и VRM
341         disks_and_fans_w = 8.0          # Накопители и кулер
    ы
342
343         system_idle_base_w = round(ram_power_w + motherboa
    rd_chipset_w + disks_and_fans_w, 2)
344
345         # Итоговая средняя мощность системы (Вт)
346         total_avg_watts = round(cpu_total_power + gpu_w +
    system_idle_base_w, 2)
347
348         # Энергия за период (hours)
349         total_watt_hours = round(total_avg_watts * hours,

```

2)

```
350     total_kwh = round(total_watt_hours / 1000.0, 3)
351
352     # Доли компонентов для круговой диаграммы
353     pie_labels = [
354         "Ядра процессора (CPU Cores)",
355         "Контроллеры и Uncore ЦП",
356         "Видеокарта (GPU)",
357         "Оперативная память (RAM)",
358         "Материнская плата и чипсет",
359         "Накопители (SSD/HDD) и кулеры",
360     ]
361     pie_values = [
362         round(cpu_cores_w * hours, 2),
363         round(cpu_uncore_w * hours, 2),
364         round(gpu_w * hours, 2),
365         round(ram_power_w * hours, 2),
366         round(motherboard_chipset_w * hours, 2),
367         round(disks_and_fans_w * hours, 2),
368     ]
369     pie_colors = [
370         "#38bdf8", # Ядра CPU (голубой)
371         "#0284c7", # Uncore (синий)
372         "#a855f7", # GPU (фиолетовый)
373         "#10b981", # RAM (зеленый)
374         "#f59e0b", # Материнская плата (янтарный)
375         "#ec4899", # Накопители и кулеры (розовый)
376     ]
377
378     total_wh_sum = sum(pie_values) or 1.0
379     pie_percentages = [round((v / total_wh_sum) * 100.
380 0, 1) for v in pie_values]
381
382     breakdown_table = []
383     for lbl, val_wh, pct in zip(pie_labels, pie_values
384 , pie_percentages):
385         breakdown_table.append({
386             "component": lbl,
```

```

385         "watt_hours": val_wh,
386         "kwh": round(val_wh / 1000.0, 3),
387         "percent": pct,
388         "avg_power_watts": round(val_wh / hours, 2
    ),
389     })
390
391     return {
392         "status": "ok",
393         "period_hours": hours,
394         "total_kwh": total_kwh,
395         "total_watt_hours": total_watt_hours,
396         "avg_system_power_watts": total_avg_watts,
397         "cpu_package_power_watts": cpu_total_power,
398         "gpu_power_watts": gpu_w,
399         "system_board_power_watts": system_idle_base_w
    ,
400         "chart": {
401             "type": "doughnut",
402             "title": f"Суточная круговая диаграмма эне
    ргопотребления (Всего: {total_kwh} кВт·ч, {total_avg_watts} Вт
    )",
403             "labels": pie_labels,
404             "datasets": [
405                 {
406                     "data": pie_values,
407                     "backgroundColor": pie_colors,
408                     "borderWidth": 2,
409                     "borderColor": "#131b2e",
410                 }
411             ],
412         },
413         "breakdown": breakdown_table,
414         "sensor_readings": sensor_power_rows,
415         "sql": f"{sql_sensors.strip()}\n-- и статистик
    а нагрузки:\n{sql_snapshots.strip()}",
416     }
417 except Exception as ex:

```

```

418         logger.error(f"Ошибка расчета энергопотребления: {
ex}")
419         return {"status": "error", "message": str(ex)}
420
421     def get_top_processes_summary(
422         self, limit: int = 10, db_path: Optional[Union[str, Pa
th]] = None
423     ) -> Dict[str, Any]:
424         """Получить сводку самых ресурсоемких процессов по CPU
и памяти.
425
426         Args:
427             limit: Количество процессов.
428             db_path: Путь к базе данных.
429
430         Returns:
431             Dict[str, Any]: Список топ-процессов и SQL-запрос.
432         """
433         target_db = self.get_db_path(db_path)
434         if not target_db:
435             return {"status": "error", "message": "telemetry.d
b не найдена"}
436
437         sql = """
438             SELECT name,
439                    ROUND(AVG(cpu_percent), 1) as avg_cpu,
440                    ROUND(MAX(cpu_percent), 1) as max_cpu,
441                    ROUND(AVG(memory_mb), 1) as avg_ram_mb,
442                    COUNT(*) as samples_count
443             FROM (
444                 SELECT name, cpu_percent, memory_mb FROM proce
ss_snapshots
445                 ORDER BY id DESC LIMIT 500
446             )
447             GROUP BY name
448             ORDER BY avg_cpu DESC
449             LIMIT ?
450         """

```

```

451         try:
452             conn = sqlite3.connect(str(target_db), timeout=5.0
453         )
454             conn.row_factory = sqlite3.Row
455             cursor = conn.cursor()
456             cursor.execute(sql, (limit,))
457             rows = [dict(r) for r in cursor.fetchall()]
458             conn.close()
459
460             # Если snapshots пусты, попробуем process_rollups_da
ily
461             if not rows:
462                 fallback_sql = """
463                     SELECT name, avg_cpu_percent as avg_cpu, m
ax_cpu_percent as max_cpu, avg_memory_mb as avg_ram_mb, sample
_count as samples_count
464                     FROM process_rollups_daily
465                     ORDER BY avg_cpu_percent DESC LIMIT ?
466                 """
467                 conn = sqlite3.connect(str(target_db), timeout
=5.0)
468                 conn.row_factory = sqlite3.Row
469                 cursor = conn.cursor()
470                 cursor.execute(fallback_sql, (limit,))
471                 rows = [dict(r) for r in cursor.fetchall()]
472                 conn.close()
473                 sql = fallback_sql
474
475                 chart_labels = [r["name"] for r in rows]
476                 chart_cpu = [r["avg_cpu"] for r in rows]
477
478             return {
479                 "status": "ok",
480                 "sql": sql.strip(),
481                 "processes": rows,
482                 "count": len(rows),
483                 "chart": {
484                     "type": "bar",

```



```

484         "title": "Топ процессов по средней нагрузк
е на ЦП (%)",
485         "labels": chart_labels,
486         "datasets": [
487             {
488                 "label": "Средняя загрузка ЦП (%)"
489             ,
490                 "data": chart_cpu,
491                 "backgroundColor": "rgba(56, 189,
248, 0.7)",
492                 "borderColor": "#38bdf8",
493                 "borderWidth": 1,
494             }
495         ],
496     },
497     except Exception as ex:
498         logger.error(f"Ошибка выборки топ-процессов: {ex}"
499 )
500         return {"status": "error", "message": str(ex), "sq
l": sql.strip()}
501
502     def get_schema_summary(self, db_path: Optional[Union[str,
Path]] = None) -> Dict[str, Any]:
503         """Получить краткую схему таблиц telemetry.db для конт
екста AI-модели.
504
505         Args:
506             db_path: Путь к базе данных.
507
508         Returns:
509             Dict[str, Any]: Словарь таблиц с их полями и колич
еством строк.
510
511         """
512         target_db = self.get_db_path(db_path)
513         if not target_db:
514             return {"status": "error", "message": "telemetry.d
b не найдена"}

```

```

513 |
514 |         try:
515 |             conn = sqlite3.connect(str(target_db), timeout=5.0
516 | )
517 |             cursor = conn.cursor()
518 |             cursor.execute("SELECT name FROM sqlite_master WHE
519 | RE type='table'")
520 |             tables = [r[0] for r in cursor.fetchall() if r[0]
521 | != "sqlite_sequence"]
522 |
523 |             schema: Dict[str, Any] = {}
524 |             for t in tables:
525 |                 cursor.execute(f"PRAGMA table_info({t})")
526 |                 cols = [col[1] for col in cursor.fetchall()]
527 |                 cursor.execute(f"SELECT count(*) FROM {t}")
528 |                 cnt = cursor.fetchone()[0]
529 |                 schema[t] = {"columns": cols, "rows_count": cn
530 | t}
531 |
532 |             conn.close()
533 |             return {"status": "ok", "tables": schema}
534 |         except Exception as ex:
535 |             return {"status": "error", "message": str(ex)}
536 |
537 |     def get_client_processes_data(
538 |         self, db_path: Optional[Union[str, Path]] = None, limi
539 | t: int = 50
540 |     ) -> Dict[str, Any]:
541 |         """Получить данные о ресурсах клиентских процессов из
542 | SQLite.
543 |
544 |     Args:
545 |         db_path: Путь к файлу базы данных telemetry.db.
546 |         limit: Максимальное число процессов.
547 |
548 |     Returns:
549 |         Dict[str, Any]: Словарь с процессами, SQL-запросом
550 | и статусом.

```

```

544         """
545         target_db = self.get_db_path(db_path)
546         if not target_db:
547             return {"status": "error", "message": "telemetry.d
b не найдена", "processes": []}
548
549         sql = """
550             SELECT pid, name, cpu_percent, memory_mb, memory_p
ercent,
551                 num_threads, num_handles, username, read_by
tes_sec, write_bytes_sec,
552                 executable as executable_path, cmdline as c
ommand_line
553             FROM process_snapshots
554             WHERE id IN (
555                 SELECT MAX(id) FROM process_snapshots GROUP BY
pid
556             )
557             ORDER BY cpu_percent DESC, memory_mb DESC
558             LIMIT ?
559         """
560         try:
561             conn = sqlite3.connect(str(target_db), timeout=5.0
)
562             conn.row_factory = sqlite3.Row
563             cursor = conn.cursor()
564             cursor.execute(sql, (limit,))
565             rows = [dict(r) for r in cursor.fetchall()]
566             conn.close()
567             return {"status": "ok", "processes": rows, "count"
: len(rows), "sql": sql.strip()}
568         except Exception as ex:
569             logger.error(f"Ошибка выборки клиентских процессов
: {ex}")
570             return {"status": "error", "message": str(ex), "pr
ocesses": [], "sql": sql.strip()}
571
572

```

reboot_analyzer.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry_Research
  # - Reboot Analyzer
4 # =====
5 # Description:
6 #   Модуль глубокого анализа и корреляции причин перезагрузок
  Windows.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry_research.reboot_analyzer imp
  ort WindowsRebootAnalyzer
11 #
12 #       service = WindowsRebootAnalyzer()
13 #
14 # File: reboot_analyzer.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry_research
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-01 13:28:28
20 # =====
21
22 from __future__ import annotations
23 """Модуль глубокого анализа и корреляции причин перезагрузок W
  indows."""
24
25 import os
26 import re
27 import socket
28 import subprocess
29 import time
```

```
30 from datetime import datetime, timezone, timedelta
31 from typing import Any, Dict, List, Optional, Tuple
32
33 from logger import logger
34 from apps.windows.telemetry.win32_fffi.wevtapi import WevtAPI
35 from apps.windows.telemetry.win32_fffi.error_decoder import decoder_bugcheck_code, format_system_error
36 from apps.windows.telemetry.models import (
37     RebootAnalysisReport,
38     RebootCorrelatedEvent,
39     RebootSession,
40     ShutdownType,
41 )
42 from apps.windows.telemetry.sqlite import TelemetryStorage
43
44 # Целевые Event ID для корреляции жизненного цикла загрузки и
45 # выключения
46 TARGET_EVENT_IDS = {
47     1074: ('User32', 'Плановый перезапуск или выключение иници-
48     ировано процессом/пользователем'),
49     41: ('Kernel-Power', 'Система перезагрузилась без корректн-
50     ого предварительного выключения'),
51     6008: ('EventLog', 'Предыдущее завершение работы системы б-
52     ыло неожиданным'),
53     1001: ('WER-SystemErrorReporting', 'Критический системный
54     сбой ОС (BSOD / BugCheck)'),
55     19: ('WindowsUpdateClient', 'Успешная установка обновления
56     Windows Update'),
57     20: ('WindowsUpdateClient', 'Ошибка установки обновления W-
58     indows Update'),
59     21: ('WindowsUpdateClient', 'Завершена установка обновлени-
60     й, требуется перезапуск'),
61     12: ('Kernel-General', 'Запуск ядра операционной системы'),
62     ,
63     13: ('Kernel-General', 'Завершение работы операционной сис-
64     темы'),
65     6005: ('EventLog', 'Запуск службы журнала событий Windows
66     (система загружена)'),
```

```

56     6006: ('EventLog', 'Остановка службы журнала событий Windo
ws (штатное исключение)'),
57     7045: ('Service Control Manager', 'В системе была установл
ена новая служба'),
58     109: ('Kernel-Power', 'Переход ядра в режим гибернации / F
ast Startup'),
59 }
60
61
62 class WindowsRebootAnalyzer:
63     """Анализатор жизненного цикла перезагрузок и исключений о
перационной системы."""
64
65     def __init__(
66         self,
67         storage: Optional[TelemetryStorage] = None,
68         wevtapi: Optional[WevtAPI] = None,
69     ) -> None:
70         """Инициализация анализатора перезагрузок.
71
72         Args:
73             storage: Хранилище телеметрии TelemetryStorage.
74             wevtapi: Экземпляр низкоуровневого API WevtAPI.
75         """
76         self.storage = storage or TelemetryStorage.get_instanc
e()
77         self.wevtapi = wevtapi or WevtAPI()
78
79     def get_current_boot_info(self) -> Tuple[datetime, float]:
80         """Получить время текущей загрузки ОС и текущий аптайм
.
81
82         Returns:
83             Tuple[datetime, float]: (время старта, аптайм в се
кундах).
84         """
85         try:
86             import psutil

```

```

87         b_epoch = psutil.boot_time()
88         boot_dt = datetime.fromtimestamp(b_epoch, tz=timezone.
one.utc)
89         uptime_sec = max(0.0, time.time() - b_epoch)
90         return boot_dt, uptime_sec
91     except Exception:
92         # Fallback
93         boot_dt = datetime.now(timezone.utc)
94         return boot_dt, 0.0
95
96     def collect_reboot_history(
97         self,
98         limit: int = 20,
99         hours: int = 720,
100         persist_to_storage: bool = True,
101     ) -> RebootAnalysisReport:
102         """Сбор и глубокая корреляция истории перезагрузок за
103         указанный период.
104
105         Args:
106             limit: Максимальное количество анализируемых сессии
107             й перезагрузки.
108             hours: Глубина выборки событий в часах (по умолчанию 30
109             дней = 720 ч).
110             persist_to_storage: Сохранять ли выявленные новые
111             сессии в БД SQLite.
112
113         Returns:
114             RebootAnalysisReport: Полный аналитический отчет с
115             сессиями и оценкой стабильности.
116         """
117         cur_boot_dt, cur_uptime = self.get_current_boot_info()
118         cur_boot_str = cur_boot_dt.strftime('%Y-%m-%d %H:%M:%S
119         ')
120
121         hostname = socket.gethostname()
122
123         # 1. Извлечение событий из EventLog
124         raw_events = self._query_system_events(hours=hours)

```

```

118 |
119 |         # 2. Корреляция событий в хронологические сессии
120 |         sessions = self._correlate_events_into_sessions(raw_events, limit=limit)
121 |
122 |         # 3. Сохранение в постоянное хранилище при необходимости
123 |         if persist_to_storage and self.storage:
124 |             for session in sessions:
125 |                 try:
126 |                     self.storage.save_reboot_session(session)
127 |                 except Exception as ex:
128 |                     logger.debug(f'Ошибка сохранения сессии при перезагрузки {session.boot_id}: {ex}')
129 |
130 |         # 4. Подсчет статистики стабильности
131 |         planned_cnt = sum(1 for s in sessions if s.shutdown_type in (
132 |             ShutdownType.PLANNED, ShutdownType.UPDATE_RESTART,
133 |             ShutdownType.USER_INITIATED, ShutdownType.HYBRID_SHUTDOWN
134 |         ))
135 |         unexpected_cnt = sum(1 for s in sessions if s.shutdown_type in (
136 |             ShutdownType.UNEXPECTED, ShutdownType.DIRTY_POWER_LOSS, ShutdownType.HARDWARE_POWER_BUTTON
137 |         ))
138 |         bsod_cnt = sum(1 for s in sessions if s.shutdown_type == ShutdownType.CRASH_BSOD)
139 |         update_cnt = sum(1 for s in sessions if s.shutdown_type == ShutdownType.UPDATE_RESTART)
140 |         total_cnt = len(sessions)
141 |
142 |         if total_cnt > 0:
143 |             stability_score = round(max(0.0, 100.0 - (unexpected_cnt * 25.0 + bsod_cnt * 35.0) / total_cnt), 1)
144 |         else:
145 |             stability_score = 100.0

```



```

146 |         # Формирование текстового резюме
147 |         summary_parts = [
148 |             f'Проанализировано {total_cnt} перезапусков Window
| s.',
149 |             f'Плановых: {planned_cnt}, Обновлений: {update_cnt
| }, Аварийных/внезапных: {unexpected_cnt}, BSOD: {bsod_cnt}.',
150 |             f'Индекс стабильности завершений работы: {stabilit
| y_score}%/100%. ',
151 |         ]
152 |         if bsod_cnt > 0:
153 |             summary_parts.append(f'ВНИМАНИЕ: Зафиксировано {bs
| od_cnt} критических сбоев синего экрана (BSOD).')
154 |             elif unexpected_cnt > 0:
155 |                 summary_parts.append(f'ПРЕДУПРЕЖДЕНИЕ: Зафиксирова
| но {unexpected_cnt} внезапных обесточиваний или зависаний.')
156 |             else:
157 |                 summary_parts.append('Все недавние перезагрузки си
| стемы завершены в штатном запланированном режиме.')
158 |
159 |         return RebootAnalysisReport(
160 |             hostname=hostname,
161 |             current_boot_time=cur_boot_str,
162 |             current_uptime_seconds=round(cur_uptime, 1),
163 |             current_uptime_human=self._format_duration(cur_upt
| ime),
164 |             total_reboots_analyzed=total_cnt,
165 |             planned_count=planned_cnt,
166 |             unexpected_count=unexpected_cnt,
167 |             bsod_count=bsod_cnt,
168 |             update_reboot_count=update_cnt,
169 |             stability_score=stability_score,
170 |             sessions=sessions,
171 |             summary_ru=' '.join(summary_parts),
172 |         )
173 |
174 |     def analyze_current_boot(self) -> Optional[RebootSession]:
175 |         """Быстрый анализ причины завершения предыдущей сессии
| перед текущим запуском.

```

```

176
177         Returns:
178             Optional[RebootSession]: Сессия текущего запуска с
            исследованием предшествующего выключения.
179         """
180         report = self.collect_reboot_history(limit=1, hours=72
            , persist_to_storage=True)
181         if report.sessions:
182             return report.sessions[0]
183         return None
184
185     def _query_system_events(self, hours: int = 720) -> List[D
            ict[str, Any]]:
186         """Запрос событий из журналов Windows через WevtAPI ил
            и PowerShell fallback."""
187         events: List[Dict[str, Any]] = []
188
189         # 1. Попытка нативного сбора через WevtAPI
190         if self.wevtapi and self.wevtapi.is_available():
191             try:
192                 for eid in TARGET_EVENT_IDS.keys():
193                     # Читаем System
194                     sys_evs = self.wevtapi.read_events(channel
            ='System', limit=100, event_id=eid, hours=hours)
195                     events.extend(sys_evs)
196                     # Читаем Application
197                     if eid in (1001, 1074):
198                         app_evs = self.wevtapi.read_events(cha
            nnel='Application', limit=50, event_id=eid, hours=hours)
199                         events.extend(app_evs)
200                     # Читаем WindowsUpdateClient
201                     if eid in (19, 20, 21):
202                         wu_evs = self.wevtapi.read_events(
203                             channel='Microsoft-Windows-Windows
            UpdateClient/Operational',
204                             limit=50,
205                             event_id=eid,
206                             hours=hours,

```

```

207         )
208         events.extend(wu_evs)
209     except Exception as ex:
210         logger.debug(f'Ошибка нативного чтения WevtAPI
для перезагрузок: {ex}')
211
212     # 2. Если событий мало или WevtAPI недоступен, выполня
ем надежный PowerShell сбор
213     if len(events) < 2:
214         ps_events = self._query_events_powershell(hours=ho
urs)
215         events.extend(ps_events)
216
217     # 3. Дедупликация и сортировка от самых новых к старым
218     unique_events: List[Dict[str, Any]] = []
219     seen_keys = set()
220     for ev in events:
221         t = ev.get('timestamp', '')
222         eid = ev.get('event_id', 0)
223         prov = ev.get('provider', '')
224         key = f'{t}_{eid}_{prov}'
225         if key not in seen_keys:
226             seen_keys.add(key)
227             unique_events.append(ev)
228
229     # Сортировка по времени (по убыванию)
230     def _parse_ts(e: Dict[str, Any]) -> str:
231         return e.get('timestamp', '')
232
233     unique_events.sort(key=_parse_ts, reverse=True)
234     return unique_events
235
236     def _query_events_powershell(self, hours: int = 720) -> Li
st[Dict[str, Any]]:
237         """Fallback сбор событий через PowerShell Get-WinEvent
. """
238         if os.name != 'nt':
239             return []

```

```

240
241     ids_str = ','.join(str(k) for k in TARGET_EVENT_IDS.keys())
242     ps_script = f"""
243     $ErrorActionPreference = 'SilentlyContinue'
244     $startTime = (Get-Date).AddHours(-{hours})
245     $filter = @{{
246         LogName = 'System', 'Application'
247         Id = @({ids_str})
248         StartTime = $startTime
249     }}
250     $events = Get-WinEvent -FilterHashtable $filter -MaxEvents 300
251     $result = @()
252     foreach ($e in $events) {{
253         $result += [PSCustomObject]@{{
254             timestamp = $e.TimeCreated.ToString("yyyy-MM-dd HH:mm:ss")
255             event_id = $e.Id
256             provider = $e.ProviderName
257             level = $e.LevelDisplayName
258             message = $e.Message
259             event_data = @{{}}
260         }}
261     }}
262     $result | ConvertTo-Json -Depth 3 -Compress
263     """
264     try:
265         res = subprocess.run(
266             ['powershell.exe', '-NoProfile', '-NonInteractive', '-Command', ps_script],
267             capture_output=True,
268             text=True,
269             timeout=12,
270         )
271         if res.returncode == 0 and res.stdout.strip():
272             import json
273             data = json.loads(res.stdout)

```

```

274         if isinstance(data, list):
275             return data
276         elif isinstance(data, dict):
277             return [data]
278     except Exception as ex:
279         logger.debug(f'Ошибка выполнения PowerShell для со
бытий перезагрузки: {ex}')
280     return []
281
282     def _correlate_events_into_sessions(
283         self,
284         events: List[Dict[str, Any]],
285         limit: int = 20,
286     ) -> List[RebootSession]:
287         """Связывает сырые события в структурированные сессии
перезагрузок."""
288         # Находим маркеры запуска ОС (Event 12, Event 6005, ил
и Event 41 как признак нового старта)
289         boot_events: List[Dict[str, Any]] = []
290         for ev in events:
291             eid = int(ev.get('event_id', 0))
292             if eid in (12, 6005, 41):
293                 boot_events.append(ev)
294
295         # Если маркеров старта нет, но есть события выключения
(1074, 6006, 13), группируем вокруг них
296         if not boot_events:
297             for ev in events:
298                 eid = int(ev.get('event_id', 0))
299                 if eid in (1074, 6006, 13, 6008):
300                     boot_events.append(ev)
301
302         # Кластеризация событий по сессиям
303         sessions: List[RebootSession] = []
304         session_idx = 1
305
306         # Отслеживаем временные границы сессий
307         processed_boots = set()

```

```

308
309     for boot_ev in boot_events:
310         boot_time_str = boot_ev.get('timestamp', '')
311         if not boot_time_str:
312             continue
313
314         # Нормализация времени для дедупликации близких ме
ток старта (в пределах 2 минут)
315         boot_dt = self._parse_iso_time(boot_time_str)
316         if not boot_dt:
317             continue
318
319         is_duplicate = False
320         for seen_dt in processed_boots:
321             if abs((boot_dt - seen_dt).total_seconds()) <
120:
322                 is_duplicate = True
323                 break
324         if is_duplicate:
325             continue
326
327         processed_boots.add(boot_dt)
328
329         # Находим события в окне [-15 мин; +3 мин] вокруг
момента старта / выключения
330         window_start = boot_dt - timedelta(minutes=15)
331         window_end = boot_dt + timedelta(minutes=3)
332
333         session_events = [
334             ev for ev in events
335             if self._is_time_in_window(ev.get('timestamp',
''), window_start, window_end)
336         ]
337
338         session = self._analyze_session_window(boot_dt, se
ssion_events, session_idx)
339         sessions.append(session)
340         session_idx += 1

```

```

341
342         if len(sessions) >= limit:
343             break
344
345         # Рассчитываем uptime между последовательными сессиями
346         for i in range(len(sessions)):
347             curr = sessions[i]
348             if i + 1 < len(sessions):
349                 prev = sessions[i + 1]
350                 curr.previous_boot_time = prev.boot_time
351                 prev_dt = self._parse_iso_time(prev.boot_time)
352                 curr_dt = self._parse_iso_time(curr.boot_time)
353                 if prev_dt and curr_dt and curr_dt > prev_dt:
354                     diff_sec = (curr_dt - prev_dt).total_seconds()
355                     curr.uptime_seconds = round(diff_sec, 1)
356                     curr.uptime_human = self._format_duration(
357                         diff_sec)
358
359         return sessions
360
361     def _analyze_session_window(
362         self,
363         boot_dt: datetime,
364         window_events: List[Dict[str, Any]],
365         session_idx: int,
366     ) -> RebootSession:
367         """Анализирует события в окне сессии и строит вывод о
368         причине перезагрузки."""
369         boot_id = f"reboot-{boot_dt.strftime('%Y%m%d-%H%M%S')}"
370
371         boot_time_str = boot_dt.strftime('%Y-%m-%d %H:%M:%S')
372
373         # Извлечение специфических событий
374         ev_1074 = self._find_event(window_events, 1074)
375         ev_41 = self._find_event(window_events, 41)
376         ev_6008 = self._find_event(window_events, 6008)
377         ev_1001 = self._find_event(window_events, 1001)

```

```

375 |         ev_19 = self._find_event(window_events, 19)
376 |         ev_7045 = self._find_event(window_events, 7045)
377 |         ev_109 = self._find_event(window_events, 109)
378 |
379 |         evidence: List[str] = []
380 |         events_chain: List[RebootCorrelatedEvent] = []
381 |
382 |         for ev in window_events:
383 |             eid = int(ev.get('event_id', 0))
384 |             prov = ev.get('provider', '')
385 |             desc = TARGET_EVENT_IDS.get(eid, (prov, 'Системное
| событие'))[1]
386 |             events_chain.append(
387 |                 RebootCorrelatedEvent(
388 |                     event_id=eid,
389 |                     provider=prov or 'System',
390 |                     timestamp=ev.get('timestamp', ''),
391 |                     level=ev.get('level') or 'Information',
392 |                     description=desc,
393 |                     details=ev.get('event_data') or {},
394 |                 )
395 |             )
396 |
397 |         # 1. Извлечение полей из Event 1074 (User32)
398 |         initiating_proc = None
399 |         initiating_user = None
400 |         shutdown_action = None
401 |         reason_text = None
402 |         reason_code = None
403 |
404 |         if ev_1074:
405 |             msg = ev_1074.get('message', '')
406 |             data = ev_1074.get('event_data', {})
407 |             initiating_proc = data.get('param1') or self._extr
| act_regex(msg, r'process\s+([^\s]+)\s+has', r'процесс\s+([^\s]
| +)\s+инициировал')
408 |             initiating_user = data.get('param7') or data.get('
| param2') or self._extract_regex(msg, r'on behalf of user\s+([^\s]

```



```

\s]+)', r'от имени пользователя\s+([\s]+)')
409 shutdown_action = data.get('param5') or self._extract_regex(msg, r'Shutdown Type:\s*([\r\n]+)', r'Тип выключения:\s*([\r\n]+)')
410 reason_text = data.get('param3') or self._extract_regex(msg, r'following reason:\s*([\r\n]+)', r'следующей причине:\s*([\r\n]+)')
411 reason_code = data.get('param4') or self._extract_regex(msg, r'Reason Code:\s*([0-9a-fx]+)', r'Код причины:\s*([0-9a-fx]+)')
412 evidence.append(f'Event 1074 (User32): Инициатор={initiating_proc or "N/A"}, Пользователь={initiating_user or "N/A"}, Действие={shutdown_action or "restart"}')
413 if reason_code:
414     decoded_reason = format_system_error(reason_code)
415     if decoded_reason:
416         evidence.append(f'Код причины {reason_code}: {decoded_reason}')
417
418 # 2. Извлечение полей из Event 41 (Kernel-Power)
419 bugcheck_code = None
420 bugcheck_params: List[str] = []
421 power_button_ts = None
422
423 if ev_41:
424     data = ev_41.get('event_data', {})
425     bc_raw = data.get('BugcheckCode', '0')
426     try:
427         bc_int = int(bc_raw)
428         if bc_int != 0:
429             bugcheck_code = f'0x{bc_int:08X}'
430     except ValueError:
431         pass
432     p_btn = data.get('PowerButtonTimestamp', '0')
433     try:
434         p_btn_int = int(p_btn)
435         if p_btn_int > 0:

```

```

436         power_button_ts = p_btn_int
437     except ValueError:
438         pass
439     for i in range(1, 5):
440         p_val = data.get(f'BugcheckParameter{i}')
441         if p_val:
442             bugcheck_params.append(str(p_val))
443         evidence.append(f'Event 41 (Kernel-Power): BugChec
k={bugcheck_code or "none"}, PowerButtonTS={power_button_ts or
"none"}')
444
445     # 3. Извлечение полей из Event 6008 (EventLog)
446     if ev_6008:
447         evidence.append('Event 6008 (EventLog): Неожиданно
е завершение предыдущей сессии')
448
449     # 4. Извлечение полей из Event 1001 (BSOD / WER)
450     if ev_1001:
451         msg = ev_1001.get('message', '')
452         bc_match = self._extract_regex(msg, r'0x[0-9a-fA-F
]{8}')
453         if bc_match and not bugcheck_code:
454             bugcheck_code = bc_match
455         evidence.append(f'Event 1001 (WER/BugCheck): Зафик
сирован аварийный дамп {bugcheck_code or ""}')
456
457     # 5. Извлечение полей из Event 19 (WindowsUpdateClient
)
458     kb_number = None
459     if ev_19:
460         msg = ev_19.get('message', '')
461         kb_number = self._extract_regex(msg, r'KB\d+')
462         evidence.append(f'Event 19 (WindowsUpdateClient):
Установка обновления {kb_number or "KB"}')
463
464     # 6. Извлечение полей из Event 7045 (SCM)
465     service_name = None
466     if ev_7045:

```

```

467 |         msg = ev_7045.get('message', '')
468 |         service_name = self._extract_regex(msg, r'Service
| Name:\s*([\r\n]+)', r'Имя службы:\s*([\r\n]+)')
469 |         evidence.append(f'Event 7045 (SCM): Установка служ
| бы {service_name or "Unknown"}')
470 |
471 |         # -----
| -----
472 |         # Дерево принятия решений (Decision Gate) для классифи
| кации
473 |         # -----
| -----
474 |         shutdown_type = ShutdownType.UNKNOWN
475 |         likely_class = 'unknown'
476 |         conclusion = ''
477 |
478 |         # Сценарий А: BSOD / Синий экран
479 |         if ev_1001 or (bugcheck_code and bugcheck_code != '0x0
| 0000000'):
480 |             shutdown_type = ShutdownType.CRASH_BSOD
481 |             likely_class = 'bsod_crash'
482 |             bc_info = decode_bugcheck_code(bugcheck_code) if b
| ugcheck_code else {}
483 |             sym = bc_info.get('symbol')
484 |             desc = bc_info.get('description_ru')
485 |             if sym and sym != 'UNKNOWN_BUGCHECK':
486 |                 conclusion = f'Критический сбой операционной с
| истемы (BSOD): {sym} ({bugcheck_code}) – {desc}'
487 |                 elif desc and desc != 'Критический сбой ядра Windo
| ws.':
488 |                     conclusion = f'Критический сбой операционной с
| истемы (BSOD): {bugcheck_code} – {desc}'
489 |                 else:
490 |                     conclusion = f'Критический сбой операционной с
| истемы (BSOD). Код ошибки: {bugcheck_code or "Неизвестен"}.'
491 |
492 |         # Сценарий Б: Принудительное выключение физической кно
| пкой питания

```

```

493         elif power_button_ts and power_button_ts > 0:
494             shutdown_type = ShutdownType.HARDWARE_POWER_BUTTON
495             likely_class = 'hardware_power_button'
496             conclusion = 'Принудительное выключение долгим наж
ати́ем аппаратной кнопки питания.'
497
498         # Сценарий В: Внезапное отключение электропитания / за
висание (Event 41 / 6008 без 1074)
499         elif (ev_41 or ev_6008) and not ev_1074:
500             shutdown_type = ShutdownType.DIRTY_POWER_LOSS
501             likely_class = 'dirty_power_loss'
502             conclusion = 'Внезапная потеря электропитания или
мгновенный аппаратный сброс без сохранения состояния.'
503
504         # Сценарий Г: Плановый рестарт после обновления Window
s Update
505         elif ev_1074 and (
506             ev_19
507             or (initiating_proc and any(k in initiating_proc.l
ower() for k in ('svchost', 'trustedinstaller', 'tiworker', 'w
uaucflt'))))
508             or (reason_text and any(k in reason_text.lower() f
or k in ('update', 'обновлен', 'security', 'безопасн'))))
509             or (reason_code and '80020010' in reason_code.lowe
r())
510         ):
511             shutdown_type = ShutdownType.UPDATE_RESTART
512             likely_class = 'update_related_restart'
513             kb_str = f'({kb_number})' if kb_number else ''
514             conclusion = f'Плановый перезапуск службы обновлен
ия Windows Update{kb_str} от имени {initiating_user or "SYSTEM
"}.'
515
516         # Сценарий Д: Перезапуск после установки программы / с
лужбы
517         elif ev_1074 and (ev_7045 or (reason_text and any(k in
reason_text.lower() for k in ('install', 'установк', 'установ
')))):

```

```
518         shutdown_type = ShutdownType.PLANNED
519         likely_class = 'software_installation_restart'
520         svc_str = f' (служба: {service_name})' if service_
name else ''
521         conclusion = f'Плановый перезапуск после инсталляц
ии программного обеспечения{svc_str}.'
522
523         # Сценарий Е: Пользовательское штатное выключение / пе
резапуск
524         elif ev_1074:
525             shutdown_type = ShutdownType.USER_INITIATED if (in
itiating_proc and 'explorer' in initiating_proc.lower()) else
ShutdownType.PLANNED
526             likely_class = 'planned_clean_reboot'
527             act = shutdown_action or 'перезапуск'
528             conclusion = f'Штатный {act} инициирован процессом
{initiating_proc or "Система"} (пользователь: {initiating_use
r or "Пользователь"}).'
529
530         # Сценарий Ж: Быстрый запуск / Fast Startup (Kernel-Po
wer 109)
531         elif ev_109:
532             shutdown_type = ShutdownType.HYBRID_SHUTDOWN
533             likely_class = 'hybrid_fast_startup'
534             conclusion = 'Штатное завершение работы в режиме г
ибридного сна (Fast Startup / Hiberboot).'
535
536         # Fallback по умолчанию: Корректный перезапуск
537         else:
538             shutdown_type = ShutdownType.PLANNED
539             likely_class = 'clean_restart'
540             conclusion = 'Штатный запуск операционной системы
без зафиксированных системных ошибок.'
541
542         return RebootSession(
543             boot_id=boot_id,
544             boot_time=boot_time_str,
545             shutdown_type=shutdown_type,
```

```

546         likely_class=likely_class,
547         conclusion=conclusion,
548         initiating_process=initiating_proc,
549         initiating_user=initiating_user,
550         shutdown_action=shutdown_action,
551         reason_text=reason_text,
552         reason_code=reason_code,
553         bugcheck_code=bugcheck_code,
554         bugcheck_params=bugcheck_params,
555         power_button_timestamp=power_button_ts,
556         windows_update_kb=kb_number,
557         service_installed=service_name,
558         evidence=evidence,
559         events_chain=events_chain,
560     )
561
562     def _find_event(self, events: List[Dict[str, Any]], target
563     _id: int) -> Optional[Dict[str, Any]]:
564         """Находит первое событие с заданным ID среди списка."""
565         ""
566         for e in events:
567             if int(e.get('event_id', 0)) == target_id:
568                 return e
569         return None
570
571     def _extract_regex(self, text: str, *patterns: str) -> Opt
572     ional[str]:
573         """Извлекает подстроку по первому совпавшему регулярно
574         му выражению."""
575         if not text:
576             return None
577         for pat in patterns:
578             match = re.search(pat, text, re.IGNORECASE)
579             if match:
580                 return match.group(1).strip() if match.groups(
581                 ) else match.group(0).strip()
582         return None

```

```

579     def _parse_iso_time(self, ts_str: str) -> Optional[datetime
| e]:
580         """Парсинг строки времени в datetime UTC."""
581         if not ts_str:
582             return None
583         clean_ts = ts_str.replace('Z', '').split('.')[0]
584         for fmt in ('%Y-%m-%d %H:%M:%S', '%Y-%m-%dT%H:%M:%S',
| '%d.%m.%Y %H:%M:%S'):
585             try:
586                 dt = datetime.strptime(clean_ts, fmt)
587                 return dt.replace(tzinfo=timezone.utc)
588             except ValueError:
589                 pass
590         return None
591
592     def _is_time_in_window(self, ts_str: str, start: datetime,
| end: datetime) -> bool:
593         """Проверяет, попадает ли временная метка в заданный и
| нтервал."""
594         dt = self._parse_iso_time(ts_str)
595         if not dt:
596             return False
597         return start <= dt <= end
598
599     def _format_duration(self, seconds: Optional[float]) -> str:
| r:
600         """Форматирует секунды в читаемый вид (напр. '7h 25m')
| ."""
601         if seconds is None or seconds < 0:
602             return 'N/A'
603         mins, secs = divmod(int(seconds), 60)
604         hours, mins = divmod(mins, 60)
605         days, hours = divmod(hours, 24)
606         if days > 0:
607             return f'{days}д {hours}ч {mins}м'
608         if hours > 0:
609             return f'{hours}ч {mins}м'
610         return f'{mins}м {secs}с'

```

```
611 |
612 |
613 | RebootAnalyzer = WindowsRebootAnalyzer
614 |
615 | __all__ = [
616 |     "WindowsRebootAnalyzer",
617 |     "RebootAnalyzer",
618 |     "TARGET_EVENT_IDS",
619 | ]
620 |
621 |
```


server.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry_Research
  - Server
4 # =====
5 # Description:
6 #   FastAPI сервер и веб-интерфейс для исследования телеметрии
  с поддержкой AI-чата.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry_research.server import Telem
  etryChatRequest
11 #
12 #       service = TelemetryChatRequest()
13 #
14 # File: server.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry_research
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-04 07:48:00
20 # =====
21
22 from __future__ import annotations
23 """FastAPI сервер и веб-интерфейс для исследования телеметрии
  с поддержкой AI-чата."""
24
25 import json
26 from pathlib import Path
27 from typing import Any, Dict, List, Optional
28
29 from fastapi import FastAPI, HTTPException, Query
```

```
30 from fastapi.middleware.cors import CORSMiddleware
31 from fastapi.responses import HTMLResponse, JSONResponse
32 from fastapi.staticfiles import StaticFiles
33 from pydantic import BaseModel, Field
34
35 try:
36     from logger import logger
37 except ImportError:
38     import logging
39     logger = logging.getLogger(__name__)
40
41 from .analyzer import TelemetryResearcher
42 from .assistant import TelemetryAssistant
43 from .charts import TelemetryChartGenerator
44 from .client_resource_analyzer import ClientResourceAnalyzer
45 from .extractor import TelemetryDataExtractor
46 from .models import (
47     ClientProgramResource,
48     ClientResourceSummary,
49     DeepResearchReport,
50     ResearchScenarioRequest,
51     TelemetryResearchReport,
52     UnknownProgramEvaluation,
53 )
54 from .query_engine import TelemetryQueryEngine
55
56
57 class TelemetryChatRequest(BaseModel):
58     """Схема запроса к AI-ассистенту телеметрии."""
59
60     message: str = Field(..., description="Текст запроса пользо-  
вателя")
61     source_path: Optional[str] = Field(None, description="Путь  
к файлу базы данных или логов")
62     history: Optional[List[Dict[str, Any]]] = Field(None, desc-  
ription="История диалога")
63
64
```

```
65 class SqlQueryRequest(BaseModel):
66     """Схема запроса для выполнения SELECT SQL."""
67
68     sql: str = Field(..., description="Текст SQL-запроса (SELECT)")
69     db_path: Optional[str] = Field(None, description="Опциональный путь к telemetry.db")
70     limit: int = Field(500, ge=1, le=1000, description="Максимум возвращаемых строк")
71
72
73 app = FastAPI(
74     title="Telemetry Research API & Web GUI",
75     description="Веб-интерфейс, API и AI-ассистент для аналитического исследования телеметрии Windows",
76     version="1.1.0",
77 )
78
79 # CORS
80 app.add_middleware(
81     CORSMiddleware,
82     allow_origins=["*"],
83     allow_credentials=True,
84     allow_methods=["*"],
85     allow_headers=["*"],
86 )
87
88 _extractor = TelemetryDataExtractor()
89 _query_engine = TelemetryQueryEngine(extractor=_extractor)
90 _client_analyzer = ClientResourceAnalyzer(query_engine=_query_engine)
91 _assistant = TelemetryAssistant(query_engine=_query_engine, client_analyzer=_client_analyzer)
92 _researcher = TelemetryResearcher(extractor=_extractor)
93 _chart_gen = TelemetryChartGenerator()
94 _web_dir = Path(__file__).parent / "web"
95
96 if _web_dir.exists():
```

```

97     app.mount("/static", StaticFiles(directory=str(_web_dir)),
    name="static")
98
99
100 @app.get("/", response_class=HTMLResponse)
101 async def get_index() -> HTMLResponse:
102     """Отдает главную страницу Web GUI."""
103     index_file = _web_dir / "index.html"
104     if not index_file.exists():
105         raise HTTPException(status_code=404, detail="Web GUI t
    emplate not found")
106     return HTMLResponse(content=index_file.read_text(encoding=
    "utf-8"))
107
108
109 @app.get("/api/health")
110 async def health_check() -> Dict[str, str]:
111     """Проверка доступности сервиса."""
112     return {"status": "ok", "service": "telemetry_research"}
113
114
115 @app.post("/api/chat")
116 @app.post("/apps/telemetry_research/chat")
117 async def chat_endpoint(request: TelemetryChatRequest) -> Dict
    [str, Any]:
118     """Диалоговый эндпоинт AI-ассистента для аналитики и визуа
    лизации телеметрии."""
119     if not request.message or not request.message.strip():
120         raise HTTPException(status_code=400, detail="Текст зап
    роса не может быть пустым")
121
122     try:
123         response = await _assistant.handle_query(
124             user_message=request.message,
125             source_path=request.source_path,
126             history=request.history,
127         )
128         return response

```

```

129     except Exception as ex:
130         logger.error(f"Ошибка в AI-чате телеметрии: {ex}", exc
            _info=True)
131         return {
132             "status": "error",
133             "reply": f"⚠️ Произошла внутренняя ошибка при обраб
отке запроса: {str(ex)}",
134             "chart": None,
135             "sql_queries": [],
136             "metrics_summary": {},
137             "quick_followups": ["Покажи график загрузки ЦПУ по
времени"],
138         }
139
140
141 @app.get("/api/telemetry-sql/power")
142 @app.get("/apps/telemetry_research/tools/power")
143 async def get_power_data_endpoint(
144     db_path: Optional[str] = Query(None),
145     hours: int = Query(24, ge=1, le=168),
146 ) -> Dict[str, Any]:
147     """Возвращает расчет энергопотребления и конфигурацию круг
овой диаграммы."""
148     try:
149         return _query_engine.calculate_power_consumption_24h(d
b_path=db_path, hours=hours)
150     except Exception as ex:
151         logger.error(f"Ошибка вычисления энергопотребления: {e
x}")
152         raise HTTPException(status_code=500, detail=str(ex))
153
154
155 @app.get("/api/telemetry-sql/cpu-timeline")
156 @app.get("/apps/telemetry_research/tools/cpu-timeline")
157 async def get_cpu_timeline_endpoint(
158     db_path: Optional[str] = Query(None),
159     limit: int = Query(120, ge=10, le=2000),
160 ) -> Dict[str, Any]:

```

```
161     """Возвращает временной ряд загрузки и частоты процессора
    для графика."""
162     try:
163         return _query_engine.get_cpu_timeline(limit=limit, db_
    path=db_path)
164     except Exception as ex:
165         logger.error(f"Ошибка вычисления таймлайна CPU: {ex}")
166         raise HTTPException(status_code=500, detail=str(ex))
167
168
169 @app.post("/api/telemetry-sql/query")
170 @app.post("/apps/telemetry_research/tools/query")
171 async def execute_sql_endpoint(request: SqlQueryRequest) -> Di
    ct[str, Any]:
172     """Выполняет безопасный SELECT SQL-запрос к базе telemetry
    .db."""
173     try:
174         return _query_engine.execute_safe_sql(
175             sql_query=request.sql, db_path=request.db_path, li
    mit=request.limit
176         )
177     except Exception as ex:
178         logger.error(f"Ошибка выполнения SQL: {ex}")
179         raise HTTPException(status_code=500, detail=str(ex))
180
181
182 @app.get("/api/sources")
183 @app.get("/apps/telemetry_research/sources")
184 async def get_sources() -> List[Dict[str, Any]]:
185     """Возвращает список обнаруженных файлов и баз данных теле
    метрии."""
186     files = _researcher.extractor.discover_log_files()
187     result = []
188     for f in files:
189         try:
190             stat = f.stat()
191             result.append(
192                 {
```

```

193         "name": f.name,
194         "path": str(f.resolve()),
195         "size_bytes": stat.st_size,
196         "modified_at": stat.st_mtime,
197     }
198 )
199 except Exception:
200     continue
201 result.sort(key=lambda x: x.get("modified_at", 0), reverse
= True)
202 return result
203
204
205 @app.get("/api/current-state")
206 @app.get("/apps/telemetry_research/current-state")
207 async def get_current_state_endpoint(
208     db_path: Optional[str] = Query(None),
209     limit: int = Query(100, ge=10, le=2000),
210 ) -> Dict[str, Any]:
211     """Возвращает актуальное состояние системы и последние зам
еры из telemetry.db."""
212     try:
213         data = _researcher.extractor.get_current_system_state(
db_path=db_path, limit=limit)
214         return data
215     except Exception as ex:
216         logger.error(f"Ошибка получения текущего состояния: {e
x}")
217         raise HTTPException(status_code=500, detail=str(ex))
218
219
220 @app.post("/api/run-research", response_model=DeepResearchRepo
rt)
221 @app.post("/apps/telemetry_research/run-research", response_mo
del=DeepResearchReport)
222 async def run_research_endpoint(request: Optional[ResearchScen
arioRequest] = None) -> DeepResearchReport:
223     """Запускает глубокое аналитическое исследование телеметри

```

```

и и проверку гипотез."""
224     try:
225         report = _researcher.run_deep_research(scenario=request, chart_generator=_chart_gen)
226         return report
227     except Exception as ex:
228         logger.error(f"Ошибка аналитического исследования телеметрии: {ex}")
229         raise HTTPException(status_code=500, detail=str(ex))
230
231
232 @app.get("/api/records")
233 @app.get("/apps/telemetry_research/records")
234 async def get_records_endpoint(
235     source_path: Optional[str] = Query(None),
236     page: int = Query(1, ge=1),
237     page_size: int = Query(20, ge=1, le=100),
238     query: Optional[str] = Query(None),
239 ) -> Dict[str, Any]:
240     """Возвращает пагинированный список сырых записей телеметрии."""
241     try:
242         all_records = _researcher.extractor.load_all_records(source_path)
243         if query:
244             q_lower = query.lower()
245             all_records = [r for r in all_records if q_lower in json.dumps(r, ensure_ascii=False).lower()]
246
247         total = len(all_records)
248         start_idx = (page - 1) * page_size
249         end_idx = start_idx + page_size
250         items = all_records[start_idx:end_idx]
251
252         return {
253             "total": total,
254             "page": page,
255             "page_size": page_size,

```



```

256         "items": items,
257     }
258 except Exception as ex:
259     logger.error(f"Ошибка чтения записей: {ex}")
260     raise HTTPException(status_code=500, detail=str(ex))
261
262
263 @app.get("/api/dashboard", response_class=HTMLResponse)
264 @app.get("/apps/telemetry_research/dashboard", response_class=
HTMLResponse)
265 async def get_dashboard_html(source_path: Optional[str] = Quer
y(None)) -> HTMLResponse:
266     """Генерирует и возвращает автономный HTML-отчет с графика
ми."""
267     try:
268         report = _researcher.analyze(source_path)
269         records = _researcher.extractor.load_all_records(sourc
e_path)
270         ts_map = _researcher._extract_time_series(records)
271         report.charts = _chart_gen.generate_chart_configs(ts_m
ap, report)
272         html = _chart_gen.render_html_dashboard(report)
273         return HTMLResponse(content=html)
274     except Exception as ex:
275         logger.error(f"Ошибка генерации HTML дашборда: {ex}")
276         raise HTTPException(status_code=500, detail=str(ex))
277
278
279 @app.get("/api/client-resources", response_model=ClientResourc
eSummary)
280 @app.get("/apps/telemetry_research/client-resources", response
_model=ClientResourceSummary)
281 async def get_client_resources_endpoint(
282     db_path: Optional[str] = Query(None),
283     ask_gemini: bool = Query(True),
284     limit: int = Query(50, ge=1, le=200),
285 ) -> ClientResourceSummary:
286     """Возвращает сводный аудит ресурсов клиентских программ с

```

```

оценками от Gemini."""
287     try:
288         summary = await _client_analyzer.analyze_client_resour
ces(
289             db_path=db_path, ask_gemini_for_unknown=ask_gemini
, limit=limit
290         )
291         return summary
292     except Exception as ex:
293         logger.error(f"Ошибка аудита ресурсов клиентских прог
амм: {ex}")
294         raise HTTPException(status_code=500, detail=str(ex))
295
296
297 @app.post("/api/client-resources/evaluate", response_model=Unk
nownProgramEvaluation)
298 @app.post("/apps/telemetry_research/client-resources/evaluate"
, response_model=UnknownProgramEvaluation)
299 async def evaluate_program_endpoint(program: ClientProgramReso
urce) -> UnknownProgramEvaluation:
300     """Запрашивает экспертную оценку неизвестной ресурсоемкой
программы у модели Gemini."""
301     try:
302         evaluation = await _client_analyzer.evaluate_program_w
ith_gemini(program)
303         return evaluation
304     except Exception as ex:
305         logger.error(f"Ошибка запроса оценки к Gemini для {pro
gram.name}: {ex}")
306         raise HTTPException(status_code=500, detail=str(ex))
307
308

```

startup_history_manager.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry_Research
  # - Startup History Manager
4 # =====
5 # Description:
6 #   Менеджер истории, архивации и детектора изменений автозагрузки
  #   Windows.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry_research.startup_history_manager
  #       import StartupHistoryManager
11 #
12 #       service = StartupHistoryManager()
13 #
14 # File: startup_history_manager.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry_research
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-06 13:56:00
20 # =====
21
22 from __future__ import annotations
23 """Менеджер истории, архивации и детектора изменений автозагрузки
  #   Windows."""
24
25 import json
26 import os
27 import uuid
28 from datetime import datetime, timezone
29 from pathlib import Path
```

```

30 from typing import Any, Dict, List, Optional
31 from logger import logger
32 from apps.windows.sdk.modules.startup.core.models import (
33     AuditReport,
34     StartupArchiveEntry,
35     StartupChangeItem,
36     StartupEntry,
37 )
38
39
40 class StartupHistoryManager:
41     """Менеджер архивных снимков и отслеживания изменений точек автозапуска."""
42
43     def __init__(
44         self,
45         archive_dir: Optional[Path] = None,
46         storage_dir: Optional[Path] = None,
47         max_archives: int = 100,
48         use_file_index: bool = True,
49     ) -> None:
50         """Инициализация менеджера истории автозапуска.
51
52         Args:
53             archive_dir: Каталог для хранения архивных снимков
54             (по умолчанию data/telemetry/startup_archives).
55             storage_dir: Алиас для archive_dir.
56             max_archives: Максимальное количество хранимых архивов для ротации.
57             use_file_index: Использовать ли индексный файл архивов.
58
59         """
60         target_dir = archive_dir if archive_dir is not None else storage_dir
61         if target_dir is None:
62             base_dir = Path(__file__).resolve().parent.parent.parent.parent
63             self.archive_dir = base_dir / 'data' / 'telemetry'

```

```

    / 'startup_archives'
62         else:
63             self.archive_dir = Path(target_dir)
64             self.max_archives = max(10, max_archives)
65             self.archive_dir.mkdir(parents=True, exist_ok=True)
66             self.index_file = self.archive_dir / 'startup_index.js
onl'
67             self._cached_latest_report: Optional[AuditReport] = No
ne
68             self.use_file_index = use_file_index
69
70         def detect_changes(
71             self,
72             current_report: AuditReport,
73             baseline_report: Optional[AuditReport] = None,
74         ) -> List[StartupChangeItem]:
75             """Сравнение двух снимков автозапуска и выявление всех
отличий (Diff Engine).
76
77             Args:
78                 current_report: Текущий срез аудита автозапуска.
79                 baseline_report: Базовый срез для сравнения (если
None, берется последний сохраненный).
80
81             Returns:
82                 List[StartupChangeItem]: Список зафиксированных из
менений.
83             """
84             base = baseline_report
85             if base is None:
86                 if self._cached_latest_report is not None:
87                     base = self._cached_latest_report
88                 else:
89                     latest_entry = self.get_latest_archive()
90                     if latest_entry:
91                         base = latest_entry.report
92             if base is None:
93                 return []

```

```

94 |
95 |     changes: List[StartupChangeItem] = []
96 |     now_ts = datetime.now(timezone.utc).isoformat()
97 |
98 |     if isinstance(base, dict):
99 |         raw_base_entries = base.get('entries', [])
100 |         base_entries_list = [StartupEntry.model_validate(e
    | ) if isinstance(e, dict) else e for e in raw_base_entries]
101 |         elif hasattr(base, 'entries'):
102 |             base_entries_list = [StartupEntry.model_validate(e
    | ) if isinstance(e, dict) else e for e in base.entries]
103 |         else:
104 |             return []
105 |
106 |         base_entries: Dict[str, StartupEntry] = {e.id: e for e
    | in base_entries_list}
107 |
108 |     if isinstance(current_report, dict):
109 |         raw_curr_entries = current_report.get('entries', [
    | ])
110 |         curr_entries_list = [StartupEntry.model_validate(e
    | ) if isinstance(e, dict) else e for e in raw_curr_entries]
111 |         elif hasattr(current_report, 'entries'):
112 |             curr_entries_list = [StartupEntry.model_validate(e
    | ) if isinstance(e, dict) else e for e in current_report.entrie
    | s]
113 |         else:
114 |             return []
115 |
116 |         curr_entries: Dict[str, StartupEntry] = {e.id: e for e
    | in curr_entries_list}
117 |
118 |         # 1. Новые записи в автозагрузке
119 |         for entry_id, entry in curr_entries.items():
120 |             if entry_id not in base_entries:
121 |                 status_str = 'включен' if entry.is_enabled els
    | e 'отключен'
122 |                 desc = (

```

```

123 |                 f"Добавлен новый элемент автозапуска: «{en
| try.name}» "
124 |                 f"({entry.location_type.value if hasattr(e
| ntry.location_type, 'value') else entry.location_type}, {statu
| s_str}). "
125 |                 f"Команда: {entry.command or entry.executa
| ble_path}"
126 |             )
127 |             changes.append(
128 |                 StartupChangeItem(
129 |                     change_type='added',
130 |                     entry_id=entry.id,
131 |                     name=entry.name,
132 |                     description=desc,
133 |                     previous_value=None,
134 |                     current_value=entry.model_dump(mode='j
| son'),
135 |                     timestamp=now_ts,
136 |                 )
137 |             )
138 |
139 |     # 2. Удаленные записи из автозагрузки
140 |     for entry_id, entry in base_entries.items():
141 |         if entry_id not in curr_entries:
142 |             desc = f"Удален элемент из автозапуска: «{entr
| y.name}» (Расположение: {entry.location_type.value if hasattr(
| entry.location_type, 'value') else entry.location_type})"
143 |             changes.append(
144 |                 StartupChangeItem(
145 |                     change_type='removed',
146 |                     entry_id=entry.id,
147 |                     name=entry.name,
148 |                     description=desc,
149 |                     previous_value=entry.model_dump(mode='
| json'),
150 |                     current_value=None,
151 |                     timestamp=now_ts,
152 |                 )

```

```

153         )
154
155     # 3. Изменения параметров существующих записей
156     for entry_id, entry in curr_entries.items():
157         if entry_id in base_entries:
158             old_entry = base_entries[entry_id]
159
160             # Изменение статуса активности (ВКЛ / ОТКЛ)
161             if old_entry.is_enabled != entry.is_enabled:
162                 old_st = 'ВКЛЮЧЕН' if old_entry.is_enabled
163                 new_st = 'ВКЛЮЧЕН' if entry.is_enabled else 'ОТКЛЮЧЕН'
164                 desc = f"Изменен статус активности «{entry.name}»: было {old_st} -> стало {new_st}"
165                 changes.append(
166                     StartupChangeItem(
167                         change_type='state_changed',
168                         entry_id=entry.id,
169                         name=entry.name,
170                         description=desc,
171                         previous_value={'is_enabled': old_entry.is_enabled},
172                         current_value={'is_enabled': entry.is_enabled},
173                         timestamp=now_ts,
174                     )
175                 )
176
177             # Изменение исполняемого пути или команды
178             if (old_entry.executable_path != entry.executable_path) or (old_entry.command != entry.command):
179                 desc = f"Изменена команда запуска для «{entry.name}»: «{old_entry.command}» -> «{entry.command}»"
180                 changes.append(
181                     StartupChangeItem(
182                         change_type='path_changed',
183                         entry_id=entry.id,

```



```

184 |             name=entry.name,
185 |             description=desc,
186 |             previous_value={'command': old_ent
ry.command, 'executable_path': old_entry.executable_path},
187 |             current_value={'command': entry.co
mmand, 'executable_path': entry.executable_path},
188 |             timestamp=now_ts,
189 |         )
190 |     )
191 |
192 |     # Изменение уровня риска
193 |     old_risk = old_entry.risk_level.value if hasat
tr(old_entry.risk_level, 'value') else str(old_entry.risk_level)
194 |     new_risk = entry.risk_level.value if hasattr(e
ntry.risk_level, 'value') else str(entry.risk_level)
195 |     if old_risk != new_risk:
196 |         desc = f"Изменен уровень риска «{entry.nam
e}»: было {old_risk.upper()} -> стало {new_risk.upper()}"
197 |         changes.append(
198 |             StartupChangeItem(
199 |                 change_type='risk_changed',
200 |                 entry_id=entry.id,
201 |                 name=entry.name,
202 |                 description=desc,
203 |                 previous_value={'risk_level': old_
risk, 'reasons': old_entry.risk_reasons},
204 |                 current_value={'risk_level': new_r
isk, 'reasons': entry.risk_reasons},
205 |                 timestamp=now_ts,
206 |             )
207 |         )
208 |
209 |     return changes
210 |
211 |     def archive_report(
212 |         self,
213 |         report: AuditReport,

```

```

214         auto_diff: bool = True,
215     ) -> StartupArchiveEntry:
216         """Сохранение снимка аудита автозапуска в архив с фикс
ацией изменений.
217
218         Args:
219             report: Отчет аудита автозапуска.
220             auto_diff: Выполнять ли автоматическое вычисление
изменений.
221
222         Returns:
223             StartupArchiveEntry: Созданная запись архива.
224         """
225         changes: List[StartupChangeItem] = []
226         if auto_diff:
227             changes = self.detect_changes(report)
228             if changes:
229                 logger.info(f"Зафиксировано {len(changes)} изм
енений в автозагрузке Windows!")
230                 for c in changes:
231                     logger.info(f" [Автозапуск] {c.change_type
.upper()}: {c.description}")
232
233             archive_id = f"startup_{datetime.now(timezone.utc).str
ftime('%Y%m%d_%H%M%S')}__{uuid.uuid4().hex[:6]}"
234             timestamp = datetime.now(timezone.utc).isoformat()
235             entry = StartupArchiveEntry(
236                 archive_id=archive_id,
237                 timestamp=timestamp,
238                 total_entries=len(report.entries),
239                 health_score=report.summary.health_score if report
.summary else 100,
240                 changes_count=len(changes),
241                 changes=changes,
242                 report=report,
243             )
244
245             archive_file = self.archive_dir / f'{archive_id}.json'

```

```

246         try:
247             with open(archive_file, 'w', encoding='utf-8') as
f:
248                 json.dump(entry.model_dump(mode='json'), f, en
sure_ascii=False, indent=2)
249             except Exception as ex:
250                 logger.error(f'Ошибка сохранения архива автозапуск
а в файл: {ex}')
251
252         index_entry = {
253             'archive_id': archive_id,
254             'timestamp': timestamp,
255             'total_entries': entry.total_entries,
256             'health_score': entry.health_score,
257             'changes_count': entry.changes_count,
258             'file_name': f'{archive_id}.json',
259         }
260         if self.use_file_index:
261             try:
262                 with open(self.index_file, 'a', encoding='utf-
8') as f:
263                     f.write(json.dumps(index_entry, ensure_asc
ii=False) + '\n')
264             except Exception as ex:
265                 logger.error(f'Ошибка записи индекса архивов а
втозапуска: {ex}')
266
267         self._cached_latest_report = report
268         self._rotate_archives()
269         return entry
270
271     def record_snapshot(
272         self,
273         report: AuditReport,
274         auto_diff: bool = True,
275     ) -> tuple[StartupArchiveEntry, List[StartupChangeItem]]:
276         """Сохранение снимка автозапуска с возвратом кортежа (
архивная запись, список изменений).

```

```

277 |
278 |     Args:
279 |         report: Отчет аудита автозапуска.
280 |         auto_diff: Выполнять ли вычисление изменений.
281 |
282 |     Returns:
283 |         tuple[StartupArchiveEntry, List[StartupChangeItem]
284 | ]: Кортеж архива и списка изменений.
285 |     """
286 |     archive_entry = self.archive_report(report, auto_diff=
287 | auto_diff)
288 |     return archive_entry, archive_entry.changes
289 |
290 |     def _rotate_archives(self) -> None:
291 |         """Ротация старых архивных файлов при превышении лимит
292 | а."""
293 |         try:
294 |             json_files = sorted(self.archive_dir.glob('startup
295 | _*.json'), key=os.path.getmtime)
296 |             if len(json_files) > self.max_archives:
297 |                 to_delete = json_files[:len(json_files) - self
298 | .max_archives]
299 |                 for p in to_delete:
300 |                     try:
301 |                         p.unlink()
302 |                     except OSError:
303 |                         pass
304 |             except Exception as ex:
305 |                 logger.debug(f'Ошибка при ротации архивов автозапу
306 | ска: {ex}')
307 |
308 |     def get_latest_archive(self) -> Optional[StartupArchiveEnt
309 | ry]:
310 |         """Получение самого свежего архива автозапуска из хран
311 | илища.
312 |
313 |     Returns:
314 |         Optional[StartupArchiveEntry]: Последний архив или

```

```

None.
307     """
308     try:
309         json_files = sorted(self.archive_dir.glob('startup
_**.json'), key=os.path.getmtime, reverse=True)
310         if json_files:
311             latest_path = json_files[0]
312             with open(latest_path, 'r', encoding='utf-8')
as f:
313                 data = json.load(f)
314                 return StartupArchiveEntry.model_validate(
data)
315     except Exception as ex:
316         logger.error(f'Ошибка чтения последнего архива авт
озапуска: {ex}', exc_info=True)
317     return None
318
319     def get_history(self, limit: int = 50) -> List[Dict[str, A
ny]]:
320         """Получение сводного списка сохраненных архивов автоз
апуска.
321
322         Args:
323             limit: Максимальное количество записей архива.
324
325         Returns:
326             List[Dict[str, Any]]: Список метаданных архивов.
327         """
328         records: List[Dict[str, Any]] = []
329         if not self.index_file.exists():
330             return records
331         try:
332             with open(self.index_file, 'r', encoding='utf-8')
as f:
333                 for line in f:
334                     line = line.strip()
335                     if line:
336                         try:

```

```

337         records.append(json.loads(line))
338     except json.JSONDecodeError:
339         continue
340     except Exception as ex:
341         logger.debug(f'Ошибка чтения индекса архивов автоз
апуска: {ex}')
342     return list(reversed(records))[:limit]
343
344     def get_archive_by_id(self, archive_id: str) -> Optional[S
tartupArchiveEntry]:
345         """Получение конкретного полного архивного снимка по I
D.
346
347         Args:
348             archive_id: Идентификатор архива.
349
350         Returns:
351             Optional[StartupArchiveEntry]: Найденный архив или
None.
352         """
353         safe_id = Path(archive_id).name
354         target = self.archive_dir / f'{safe_id}.json'
355         if not target.exists():
356             target = self.archive_dir / safe_id
357         if target.exists():
358             try:
359                 with open(target, 'r', encoding='utf-8') as f:
360                     data = json.load(f)
361                     return StartupArchiveEntry.model_validate(
data)
362             except Exception as ex:
363                 logger.debug(f'Ошибка загрузки архива автозапу
ска {archive_id}: {ex}')
364             return None
365
366     def get_change_timeline(self, limit: int = 100) -> List[St
artupChangeItem]:
367         """Получение сводного списка всех изменений автозапуск

```

а из архивов.

```
368
369     Args:
370         limit: Лимит изменений для возврата.
371
372     Returns:
373         List[StartupChangeItem]: Хронологический список за
374         фиксированных изменений.
375         """
376         all_changes: List[StartupChangeItem] = []
377         json_files = sorted(self.archive_dir.glob('startup_*.j
378         son'), key=os.path.getmtime, reverse=True)
379         for p in json_files:
380             try:
381                 with open(p, 'r', encoding='utf-8') as f:
382                     data = json.load(f)
383                     entry = StartupArchiveEntry.model_validate
384                     (data)
385                     if entry.changes:
386                         all_changes.extend(entry.changes)
387                         if len(all_changes) >= limit:
388                             break
389             except Exception:
390                 continue
391         return all_changes[:limit]
392
393     # Алиас для удобства
394     get_changes = get_change_timeline
```

app.js

```
1 /**
2  * =====
3  * Process Name: Windows Web - App Script
4  * =====
5  * Description:
6  *   Клиентский скрипт управления интерфейсом модуля app.
7  *
8  * Usage Examples:
9  *   HTML Integration:
10 *     <script src="/windows/telemetry_research/web/app.js?v=2
11 *       0261001_v1" type="module"></script>
12 *
13 * File: app.js
14 * Project: ai-breadboard
15 * Package: windows/telemetry_research/web
16 * Author: hypo69
17 * Copyright: © 2026 hypo69
18 * Updated: 2026-10-04 07:49:00
19 * =====
20 */
21 /**
22 * Клиентский контроллер Web GUI Исследования и Мониторинга Те
23 * леметрии Windows.
24 * Поддерживает живой мониторинг telemetry.db, ресурсы програм
25 * м клиента, глубокий анализ и AI-ассистента с Gemini.
26 */
27
28 let currentReport = null;
29 let currentClientSummary = null;
30 let researchCharts = {};
31 let liveCharts = {};
32 let aiDynamicChart = null;
```



```

31 let currentPage = 1;
32 const pageSize = 20;
33 let totalPages = 1;
34 let searchQuery = "";
35 let liveRefreshTimer = null;
36
37 document.addEventListener("DOMContentLoaded", () => {
38     initTabs();
39     bindEvents();
40     initAiChat();
41     loadSources();
42     initLiveCharts();
43     loadCurrentState();
44     setupAutoRefresh();
45     runResearch();
46 });
47
48 /**
49  * Инициализация переключения вкладок.
50  */
51 function initTabs() {
52     const tabBtns = document.querySelectorAll(".tab-btn");
53     tabBtns.forEach((btn) => {
54         btn.addEventListener("click", () => {
55             tabBtns.forEach((b) => b.classList.remove("active"));
56             document.querySelectorAll(".tab-content").forEach((c) =>
57                 c.classList.remove("active"));
58
59             btn.classList.add("active");
60             const targetId = btn.getAttribute("data-tab");
61             const targetContent = document.getElementById(targetId);
62             if (targetContent) {
63                 targetContent.classList.add("active");
64             }
65
66             if (targetId === "tab-current") {
67                 loadCurrentState();
68             } else if (targetId === "tab-client-resources") {

```

```
68 |         loadClientResources(false);
69 |     } else if (targetId === "tab-records") {
70 |         loadRecords();
71 |     }
72 | });
73 | });
74 | }
75 |
76 | /**
77 |  * Привязка событий элементов управления.
78 |  */
79 | function bindEvents() {
80 |     const btnRun = document.getElementById("btn-run");
81 |     if (btnRun) {
82 |         btnRun.onclick = () => runResearch();
83 |     }
84 |
85 |     const selectSource = document.getElementById("select-source"
86 | );
87 |     if (selectSource) {
88 |         selectSource.onchange = () => {
89 |             runResearch();
90 |             loadCurrentState();
91 |             loadClientResources(false);
92 |         };
93 |     }
94 |
95 |     const btnDashboard = document.getElementById("btn-dashboard"
96 | );
97 |     if (btnDashboard) {
98 |         btnDashboard.onclick = () => openHtmlDashboard();
99 |     }
100 |
101 |     const btnJson = document.getElementById("btn-json");
102 |     if (btnJson) {
103 |         btnJson.onclick = () => downloadJsonReport();
104 |     }
105 | }
```

```
104 |   const btnRefreshCurrent = document.getElementById("btn-refre
    | sh-current");
105 |   if (btnRefreshCurrent) {
106 |     btnRefreshCurrent.onclick = () => loadCurrentState();
107 |   }
108 |
109 |   const btnRefreshClient = document.getElementById("btn-refres
    | h-client-procs");
110 |   if (btnRefreshClient) {
111 |     btnRefreshClient.onclick = () => loadClientResources(false
    | );
112 |   }
113 |
114 |   const btnAskGeminiAll = document.getElementById("btn-ask-gem
    | ini-all-unknown");
115 |   if (btnAskGeminiAll) {
116 |     btnAskGeminiAll.onclick = () => loadClientResources(true);
117 |   }
118 |
119 |   const inputSearchClient = document.getElementById("input-sea
    | rch-client-procs");
120 |   if (inputSearchClient) {
121 |     inputSearchClient.oninput = (e) => filterClientProcsTable(
    | e.target.value);
122 |   }
123 |   if (btnRefreshCurrent) {
124 |     btnRefreshCurrent.onclick = () => loadCurrentState();
125 |   }
126 |
127 |   const rangeSelect = document.getElementById("current-range-s
    | elect");
128 |   if (rangeSelect) {
129 |     rangeSelect.onchange = () => loadCurrentState();
130 |   }
131 |
132 |   const autoRefreshToggle = document.getElementById("auto-refr
    | esh-toggle");
133 |   if (autoRefreshToggle) {
```

```
134     autoRefreshToggle.onChange = () => setupAutoRefresh();
135 }
136
137 const searchInput = document.getElementById("input-search");
138 if (searchInput) {
139     searchInput.onkeydown = (e) => {
140         if (e.key === "Enter") {
141             searchQuery = searchInput.value.trim();
142             currentPage = 1;
143             loadRecords();
144         }
145     };
146 }
147
148 const prevBtn = document.getElementById("btn-prev-page");
149 if (prevBtn) {
150     prevBtn.onclick = () => {
151         if (currentPage > 1) {
152             currentPage--;
153             loadRecords();
154         }
155     };
156 }
157
158 const nextBtn = document.getElementById("btn-next-page");
159 if (nextBtn) {
160     nextBtn.onclick = () => {
161         if (currentPage < totalPages) {
162             currentPage++;
163             loadRecords();
164         }
165     };
166 }
167 }
168
169 /**
170  * Инициализация обработчиков AI-чата и панели ввода.
171  */
```

```
172 function initAiChat() {
173   const form = document.getElementById("ai-query-form");
174   const input = document.getElementById("ai-query-input");
175   const btnClear = document.getElementById("btn-ai-clear");
176   const btnClose = document.getElementById("btn-ai-close");
177   const responseCard = document.getElementById("ai-response-card");
178
179   if (input) {
180     input.addEventListener("input", () => {
181       if (btnClear) {
182         btnClear.style.display = input.value.length > 0 ? "block" : "none";
183       }
184     });
185
186     input.addEventListener("keydown", (e) => {
187       if (e.key === "Enter" && !e.shiftKey) {
188         e.preventDefault();
189         submitAiQuery(input.value.trim());
190       }
191     });
192   }
193
194   if (btnClear && input) {
195     btnClear.addEventListener("click", () => {
196       input.value = "";
197       btnClear.style.display = "none";
198       input.focus();
199     });
200   }
201
202   if (form && input) {
203     form.addEventListener("submit", (e) => {
204       e.preventDefault();
205       submitAiQuery(input.value.trim());
206     });
207   }
```

```
208
209 if (btnClose && responseCard) {
210     btnClose.addEventListener("click", () => {
211         responseCard.style.display = "none";
212     });
213 }
214
215 // Привязка быстрых чипов
216 const chips = document.querySelectorAll(".prompt-chip");
217 chips.forEach((chip) => {
218     chip.addEventListener("click", () => {
219         const q = chip.getAttribute("data-query");
220         if (q) {
221             if (input) input.value = q;
222             submitAiQuery(q);
223         }
224     });
225 });
226 }
227
228 /**
229  * Отправка запроса к AI-ассистенту телеметрии.
230  */
231 async function submitAiQuery(queryText) {
232     if (!queryText) return;
233
234     const input = document.getElementById("ai-query-input");
235     const btnSend = document.getElementById("btn-ai-send");
236     const btnText = document.getElementById("btn-ai-send-text");
237     const spinner = document.getElementById("btn-ai-spinner");
238     const responseCard = document.getElementById("ai-response-card");
239     const selectSource = document.getElementById("select-source");
240     const sourcePath = selectSource ? selectSource.value : null;
241
242     // Установка состояния загрузки
243     if (btnSend) btnSend.disabled = true;
```

```

244 |   if (btnText) btnText.style.display = "none";
245 |   if (spinner) spinner.style.display = "inline-block";
246 |
247 |   try {
248 |     const res = await fetch("/api/chat", {
249 |       method: "POST",
250 |       headers: { "Content-Type": "application/json" },
251 |       body: JSON.stringify({
252 |         message: queryText,
253 |         source_path: sourcePath || null,
254 |       }),
255 |     });
256 |
257 |     if (!res.ok) throw new Error(`HTTP ${res.status}`);
258 |     const data = await res.json();
259 |
260 |     renderAiResponse(queryText, data);
261 |   } catch (err) {
262 |     console.error("Ошибка при выполнении AI запроса:", err);
263 |     if (responseCard) {
264 |       responseCard.style.display = "block";
265 |       const replyBody = document.getElementById("ai-reply-body
266 | ");
267 |       if (replyBody) {
268 |         replyBody.innerHTML = `<div style="color: var(--accent
269 | -red); padding: 10px;">❌ Ошибка соединения с сервером AI-ассис
270 | тента: ${err.message}</div>`;
271 |       }
272 |     }
273 |   } finally {
274 |     if (btnSend) btnSend.disabled = false;
275 |     if (btnText) btnText.style.display = "inline";
276 |     if (spinner) spinner.style.display = "none";
277 |   }
278 | }
279 |
280 | /**
281 |  * Рендеринг ответа AI-ассистента, графиков Chart.js и выполне

```

```
        нных SQL.
279    */
280    function renderAiResponse(queryText, data) {
281        const responseCard = document.getElementById("ai-response-card");
282        const titleEl = document.getElementById("ai-response-title");
283        const chartWrapper = document.getElementById("ai-chart-wrapper");
284        const chartTitle = document.getElementById("ai-chart-title");
285        const metricsRow = document.getElementById("ai-metrics-row");
286        const replyBody = document.getElementById("ai-reply-body");
287        const sqlDetails = document.getElementById("ai-sql-details");
288        const sqlCode = document.getElementById("ai-sql-code");
289        const followupRow = document.getElementById("ai-followup-row");
290        const followupChips = document.getElementById("ai-followup-chips");
291
292        if (!responseCard) return;
293        responseCard.style.display = "block";
294
295        if (titleEl) {
296            titleEl.textContent = queryText.length > 60 ? queryText.substring(0, 60) + "..." : queryText;
297        }
298
299        // 1. Форматирование текста ответа
300        if (replyBody) {
301            replyBody.innerHTML = formatMarkdownToHtml(data.reply || "");
302        }
303
304        // 2. Рендеринг динамического графика Chart.js
305        if (data.chart && data.chart.datasets && data.chart.datasets
```



```
.length > 0) {
306     chartWrapper.style.display = "block";
307     if (chartTitle) chartTitle.textContent = data.chart.title
    || "□ Визуализация данных";
308
309     renderDynamicAiChart(data.chart);
310 } else {
311     chartWrapper.style.display = "none";
312     if (aiDynamicChart) {
313         aiDynamicChart.destroy();
314         aiDynamicChart = null;
315     }
316 }
317
318 // 3. Метрики KPI
319 if (data.metrics_summary && Object.keys(data.metrics_summary
).length > 0) {
320     metricsRow.style.display = "flex";
321     metricsRow.innerHTML = "";
322     Object.entries(data.metrics_summary).forEach(([k, v]) => {
323         const pill = document.createElement("div");
324         pill.className = "ai-metric-pill";
325         const cleanKey = k.replace(/_/g, " ");
326         pill.innerHTML = `<span>${cleanKey}</span> <strong>${v}
</strong>`;
327         metricsRow.appendChild(pill);
328     });
329 } else {
330     metricsRow.style.display = "none";
331 }
332
333 // 4. SQL-запросы
334 if (data.sql_queries && data.sql_queries.length > 0 && data.
sql_queries[0]) {
335     sqlDetails.style.display = "block";
336     if (sqlCode) {
337         sqlCode.textContent = data.sql_queries.join("\n\n");
338     }
```

```

339 | } else {
340 |     sqlDetails.style.display = "none";
341 | }
342 |
343 | // 5. Быстрые уточняющие вопросы
344 | if (data.quick_followups && data.quick_followups.length > 0)
    | {
345 |     followupRow.style.display = "flex";
346 |     if (followupChips) {
347 |         followupChips.innerHTML = "";
348 |         data.quick_followups.forEach((fText) => {
349 |             const btn = document.createElement("button");
350 |             btn.className = "prompt-chip";
351 |             btn.textContent = fText;
352 |             btn.onclick = () => {
353 |                 const input = document.getElementById("ai-query-inpu
    | t");
354 |                 if (input) input.value = fText;
355 |                 submitAiQuery(fText);
356 |             };
357 |             followupChips.appendChild(btn);
358 |         });
359 |     }
360 | } else {
361 |     followupRow.style.display = "none";
362 | }
363 |
364 | // Прокрутка к ответу
365 | responseCard.scrollToView({ behavior: "smooth", block: "st
    | art" });
366 | }
367 |
368 | /**
369 |  * Динамический рендеринг любого типа графика через Chart.js.
370 |  */
371 | function renderDynamicAiChart(chartConfig) {
372 |     const canvas = document.getElementById("chart-ai-dynamic");
373 |     if (!canvas) return;

```

```
374 |
375 | if (aiDynamicChart) {
376 |     aiDynamicChart.destroy();
377 |     aiDynamicChart = null;
378 | }
379 |
380 | const ctx = canvas.getContext("2d");
381 | const chartType = chartConfig.type || "line";
382 |
383 | const options = {
384 |     responsive: true,
385 |     maintainAspectRatio: false,
386 |     animation: { duration: 500 },
387 |     plugins: {
388 |         legend: {
389 |             position: chartType === "doughnut" || chartType === "pie" ? "right" : "top",
390 |             labels: { color: "#94a3b8", font: { size: 12 } },
391 |         },
392 |         tooltip: {
393 |             backgroundColor: "#131b2e",
394 |             titleColor: "#38bdf8",
395 |             bodyColor: "#f1f5f9",
396 |             borderColor: "#23304d",
397 |             borderWidth: 1,
398 |         },
399 |     },
400 | };
401 |
402 | if (chartType === "line" || chartType === "bar") {
403 |     options.scales = {
404 |         x: {
405 |             grid: { color: "rgba(35, 48, 77, 0.4)" },
406 |             ticks: { color: "#94a3b8", maxRotation: 0, autoSkip: true, maxTicksLimit: 10 },
407 |         },
408 |         y: {
409 |             grid: { color: "rgba(35, 48, 77, 0.4)" },
```

```
410         ticks: { color: "#94a3b8" },
411     },
412 };
413 }
414
415 aiDynamicChart = new Chart(ctx, {
416     type: chartType,
417     data: {
418         labels: chartConfig.labels,
419         datasets: chartConfig.datasets,
420     },
421     options: options,
422 });
423 }
424
425 /**
426  * Простой и надежный Markdown парсер для форматирования ответов
427  * AI.
428  */
429 function formatMarkdownToHtml(md) {
430     if (!md) return "";
431     let html = md
432         .replace(/&/g, "&")
433         .replace(/</g, "<")
434         .replace(/>/g, ">");
435
436     // Заголовки
437     html = html.replace(/^### (.*)/gim, "<h3>$1</h3>");
438     html = html.replace(/^#### (.*)/gim, "<h4>$1</h4>");
439     html = html.replace(/^## (.*)/gim, "<h2>$1</h2>");
440     html = html.replace(/^# (.*)/gim, "<h1>$1</h1>");
441
442     // Жирный шрифт и курсив
443     html = html.replace(/\*(.*?)\*/gim, "<strong>$1</strong>");
444     html = html.replace(/_(.*?)_/gim, "<em>$1</em>");
445     html = html.replace(/`([^\`]+)`/gim, "<code>$1</code>");
```

```

446 // Таблицы Markdown
447 const tableRegex = /(?:\|[\^\\n]+\|\\r?\n)((?:\|[\^\\n]+\|\\r?\n)+
    )/g;
448 html = html.replace(tableRegex, (match) => {
449     const lines = match.trim().split("\n");
450     if (lines.length < 2) return match;
451
452     let tableHtml = "<table>";
453     let isHeader = true;
454
455     for (let line of lines) {
456         if (line.includes("---")) {
457             isHeader = false;
458             continue;
459         }
460         const cells = line
461             .split("|")
462             .slice(1, -1)
463             .map((c) => c.trim());
464
465         tableHtml += "<tr>";
466         for (let cell of cells) {
467             const tag = isHeader ? "th" : "td";
468             tableHtml += `<${tag}>${cell}</${tag}>`;
469         }
470         tableHtml += "</tr>";
471         isHeader = false;
472     }
473     tableHtml += "</table>";
474     return tableHtml;
475 });
476
477 // Списки
478 html = html.replace(/^\\- (.*)/gim, "<li>$1</li>");
479 html = html.replace(/(<li>.*<\\li>)/gim, "<ul>$1</ul>");
480 html = html.replace(/<\\ul>\\s*<ul>/gim, "");
481
482 // Абзацы и переводы строк

```

```
483 |   html = html.replace(/\n\n+/g, "<br/><br/>");
484 |
485 |   return html;
486 | }
487 |
488 | /**
489 |  * Настройка периодического автообновления живого мониторинга.
490 |  */
491 | function setupAutoRefresh() {
492 |   if (liveRefreshTimer) {
493 |     clearInterval(liveRefreshTimer);
494 |     liveRefreshTimer = null;
495 |   }
496 |   const toggle = document.getElementById("auto-refresh-toggle"
497 | );
498 |   if (toggle && toggle.checked) {
499 |     liveRefreshTimer = setInterval(() => {
500 |       const currentTab = document.getElementById("tab-current"
501 | );
502 |       if (currentTab && currentTab.classList.contains("active"
503 | )) {
504 |         loadCurrentState(true);
505 |       }
506 |     }, 3000);
507 |   }
508 | }
509 |
510 | /**
511 |  * Загрузка списка доступных файлов и БД телеметрии.
512 |  */
513 | async function loadSources() {
514 |   const select = document.getElementById("select-source");
515 |   if (!select) return;
516 |
517 |   try {
518 |     const res = await fetch("/api/sources");
519 |     if (!res.ok) return;
520 |     const files = await res.json();
```

```

518     select.innerHTML = '<option value="">Все доступные источни
ки (telemetry.db + логи)</option>';
519     files.forEach((f) => {
520         const opt = document.createElement("option");
521         opt.value = f.path;
522         const sizeMb = (f.size_bytes / (1024 * 1024)).toFixed(1)
;
523         opt.textContent = `${f.name} (${sizeMb} MB)`;
524         select.appendChild(opt);
525     });
526 } catch (err) {
527     console.warn("Ошибка загрузки источников:", err);
528 }
529 }
530
531 /**
532  * Инициализация пустых экземпляров графиков Chart.js для теку
щего состояния.
533  */
534 function initLiveCharts() {
535     const commonOptions = {
536         responsive: true,
537         maintainAspectRatio: false,
538         animation: { duration: 300 },
539         interaction: { intersect: false, mode: "index" },
540         plugins: {
541             legend: { labels: { color: "#94a3b8", font: { size: 11 }
} },
542             tooltip: { backgroundColor: "#131b2e", titleColor: "#38b
df8", bodyColor: "#f1f5f9", borderColor: "#23304d", borderWidt
h: 1 }
543         },
544         scales: {
545             x: { grid: { color: "rgba(35, 48, 77, 0.4)" }, ticks: {
color: "#94a3b8", font: { size: 10 }, maxRotation: 0, autoSkip
: true, maxTicksLimit: 8 } },
546             y: { grid: { color: "rgba(35, 48, 77, 0.4)" }, ticks: {
color: "#94a3b8", font: { size: 10 } } }

```

```

547     }
548 };
549
550 const createChart = (id, datasets, extraY = {}) => {
551     const ctx = document.getElementById(id);
552     if (!ctx) return null;
553     const opts = JSON.parse(JSON.stringify(commonOptions));
554     if (Object.keys(extraY).length > 0) {
555         opts.scales = { ...opts.scales, ...extraY };
556     }
557     return new Chart(ctx.getContext("2d"), {
558         type: "line",
559         data: { labels: [], datasets: datasets },
560         options: opts
561     });
562 };
563
564 liveCharts.cpu = createChart("chart-live-cpu", [
565     { label: "CPU Load (%)", data: [], borderColor: "#38bdf8",
566       backgroundColor: "rgba(56, 189, 248, 0.15)", borderWidth: 2,
567       fill: true, tension: 0.3, yAxisID: "y" },
568     { label: "Частота (GHz)", data: [], borderColor: "#f59e0b"
569       , backgroundColor: "transparent", borderWidth: 2, borderDash:
570       [4, 4], tension: 0.2, yAxisID: "y1" }
571 ], {
572     y: { min: 0, max: 100, ticks: { color: "#38bdf8" } },
573     y1: { position: "right", grid: { drawOnChartArea: false },
574         ticks: { color: "#f59e0b" } }
575 });
576
577 liveCharts.ram = createChart("chart-live-ram", [
578     { label: "RAM (%)", data: [], borderColor: "#10b981", back
579       groundColor: "rgba(16, 185, 129, 0.15)", borderWidth: 2, fill:
580       true, tension: 0.3, yAxisID: "y" },
581     { label: "Занято (GB)", data: [], borderColor: "#a855f7",
582       backgroundColor: "transparent", borderWidth: 2, tension: 0.2,
583       yAxisID: "y1" }
584 ], {

```



```
576     y: { min: 0, max: 100, ticks: { color: "#10b981" } },
577     y1: { position: "right", grid: { drawOnChartArea: false },
    ticks: { color: "#a855f7" } }
578   });
579
580   liveCharts.gpu = createChart("chart-live-gpu", [
581     { label: "GPU Load (%)", data: [], borderColor: "#a855f7",
    backgroundColor: "rgba(168, 85, 247, 0.15)", borderWidth: 2,
    fill: true, tension: 0.3, yAxisID: "y" },
582     { label: "GPU Temp (°C)", data: [], borderColor: "#ef4444"
    , backgroundColor: "transparent", borderWidth: 2, tension: 0.2
    , yAxisID: "y1" }
583   ], {
584     y: { min: 0, max: 100, ticks: { color: "#a855f7" } },
585     y1: { position: "right", grid: { drawOnChartArea: false },
    ticks: { color: "#ef4444" } }
586   });
587
588   liveCharts.disk = createChart("chart-live-disk", [
589     { label: "Чтение (MB/s)", data: [], borderColor: "#06b6d4"
    , backgroundColor: "rgba(6, 182, 212, 0.15)", borderWidth: 2,
    fill: true, tension: 0.3 },
590     { label: "Запись (MB/s)", data: [], borderColor: "#ec4899"
    , backgroundColor: "rgba(236, 72, 153, 0.15)", borderWidth: 2,
    fill: true, tension: 0.3 }
591   ]);
592
593   liveCharts.net = createChart("chart-live-net", [
594     { label: "Входящий (KB/s)", data: [], borderColor: "#6366f
    1", backgroundColor: "rgba(99, 102, 241, 0.15)", borderWidth:
    2, fill: true, tension: 0.3 },
595     { label: "Исходящий (KB/s)", data: [], borderColor: "#14b8
    a6", backgroundColor: "rgba(20, 184, 166, 0.15)", borderWidth:
    2, fill: true, tension: 0.3 }
596   ]);
597 }
598
599 /**
```

```

600 | * Загрузка актуального среза состояния и построение графиков
    | из telemetry.db.
601 | */
602 | async function loadCurrentState(silent = false) {
603 |   const rangeSelect = document.getElementById("current-range-s
    | elect");
604 |   const limit = rangeSelect ? rangeSelect.value : 100;
605 |   const selectSource = document.getElementById("select-source"
    | );
606 |   const dbPath = selectSource && selectSource.value && selectS
    | ource.value.endsWith(".db") ? selectSource.value : "";
607 |
608 |   try {
609 |     const url = `/api/current-state?limit=${limit}${dbPath ? `
    | &db_path=${encodeURIComponent(dbPath)}` : ""}`;
610 |     const res = await fetch(url);
611 |     if (!res.ok) throw new Error(`HTTP ${res.status}`);
612 |     const data = await res.json();
613 |
614 |     if (data.status !== "ok") {
615 |       document.getElementById("live-db-status").textContent =
    | "❏ База данных не обнаружена";
616 |       document.getElementById("live-db-sub").textContent = dat
    | a.message || "Нет данных";
617 |       return;
618 |     }
619 |
620 |     renderCurrentStatusBar(data);
621 |     renderCurrentKPI(data.latest_snapshot);
622 |     updateLiveCharts(data.time_series);
623 |     renderLiveSensors(data.sensors);
624 |     renderLiveProcesses(data.top_processes);
625 |     renderLiveActivity(data.recent_events);
626 |   } catch (err) {
627 |     if (!silent) {
628 |       console.error("Ошибка загрузки текущего состояния:", err
    | );
629 |     }

```

```

630 | }
631 | }
632 |
633 | /**
634 |  * Отображение статуса базы данных telemetry.db.
635 |  */
636 | function renderCurrentStatusBar(data) {
637 |     const statusEl = document.getElementById("live-db-status");
638 |     const subEl = document.getElementById("live-db-sub");
639 |     if (statusEl) {
640 |         statusEl.innerHTML = `❏ <strong>${data.db_name || "teleme
641 |         try.db"}</strong> (${data.db_size_mb || 0} MB)`;
642 |     }
643 |     if (subEl) {
644 |         const latestTs = data.latest_snapshot && data.latest_snap
645 |         hot.timestamp ? data.latest_snapshot.timestamp.replace("T", "
646 |         ").substring(0, 19) : "н/д";
647 |         subEl.textContent = `Всего записей: ${data.total_snapshots
648 |         .toLocaleString()} • Отображено точек: ${data.returned_points}
649 |         • Время последнего замера: ${latestTs}`;
650 |     }
651 | }
652 |
653 | /**
654 |  * Обновление мгновенных карточек текущего состояния.
655 |  */
656 | function renderCurrentKPI(snap) {
657 |     if (!snap) return;
658 |
659 |     const cpuVal = document.getElementById("curr-cpu-val");
660 |     const cpuFreq = document.getElementById("curr-cpu-freq");
661 |     const cpuMeter = document.getElementById("curr-cpu-meter");
662 |     if (cpuVal) {
663 |         const cpuPct = Number(snap.cpu_total_percent || 0).toFixed
664 |         (1);
665 |         cpuVal.textContent = `${cpuPct}%`;
666 |         if (cpuMeter) cpuMeter.style.width = `${Math.min(100, cpuP
667 |         ct)}%`;

```

```

661 |     }
662 |     if (cpuFreq) {
663 |         const mhz = Number(snap.cpu_frequency_mhz || 0);
664 |         cpuFreq.textContent = mhz > 0 ? `Частота: ${mhz / 1000}.toFixed(2)} GHz` : "Частота: --";
665 |     }
666 |
667 |     const ramVal = document.getElementById("curr-ram-val");
668 |     const ramGb = document.getElementById("curr-ram-gb");
669 |     const ramMeter = document.getElementById("curr-ram-meter");
670 |     if (ramVal) {
671 |         const ramPct = Number(snap.memory_percent || 0).toFixed(1);
672 |         ramVal.textContent = `${ramPct}%`;
673 |         if (ramMeter) ramMeter.style.width = `${Math.min(100, ramPct)}%`;
674 |     }
675 |     if (ramGb) {
676 |         const used = Number(snap.memory_used_gb || 0).toFixed(1);
677 |         const total = Number(snap.memory_total_gb || 0).toFixed(1);
678 |         ramGb.textContent = `${used} / ${total} GB (Swap: ${snap.swap_percent || 0}%)`;
679 |     }
680 |
681 |     const gpuVal = document.getElementById("curr-gpu-val");
682 |     const gpuTemp = document.getElementById("curr-gpu-temp");
683 |     const gpuMeter = document.getElementById("curr-gpu-meter");
684 |     if (gpuVal) {
685 |         const gpuPct = Number(snap.gpu_load_percent || 0).toFixed(1);
686 |         gpuVal.textContent = `${gpuPct}%`;
687 |         if (gpuMeter) gpuMeter.style.width = `${Math.min(100, gpuPct)}%`;
688 |     }
689 |     if (gpuTemp) {
690 |         const t = snap.gpu_temp_c;
691 |         gpuTemp.textContent = t ? `Температура: ${Number(t).toFixed(1)} °C` : "Температура: --";

```

```

    d(1)}°C` : "Температура: --";
692     }
693
694     const diskVal = document.getElementById("curr-disk-val");
695     if (diskVal) {
696         const rb = ((snap.disk_read_bytes_sec || 0) / (1024 * 1024
        |)).toFixed(1);
697         const wb = ((snap.disk_write_bytes_sec || 0) / (1024 * 102
        |4)).toFixed(1);
698         diskVal.textContent = `R: ${rb} / W: ${wb} MB/s`;
699     }
700
701     const netVal = document.getElementById("curr-net-val");
702     const uptimeVal = document.getElementById("curr-uptime-val")
    ;
703     if (netVal) {
704         const nr = ((snap.network_recv_bytes_sec || 0) / 1024).toF
        |ixed(1);
705         const ns = ((snap.network_sent_bytes_sec || 0) / 1024).toF
        |ixed(1);
706         netVal.textContent = `↓ ${nr} / ↑ ${ns} KB/s`;
707     }
708     if (uptimeVal) {
709         const upSec = Number(snap.uptime_seconds || 0);
710         const hours = Math.floor(upSec / 3600);
711         const mins = Math.floor((upSec % 3600) / 60);
712         uptimeVal.textContent = upSec > 0 ? `Аптайм: ${hours}ч ${m
        |ins}м` : "Аптайм: --";
713     }
714 }
715
716 /**
717  * Плавное обновление временных рядов Chart.js.
718  */
719 function updateLiveCharts(ts) {
720     if (!ts || !ts.labels) return;
721     const labels = ts.labels;
722

```

```
723 | if (liveCharts.cpu) {
724 |     liveCharts.cpu.data.labels = labels;
725 |     liveCharts.cpu.data.datasets[0].data = ts.cpu_load || [];
726 |     liveCharts.cpu.data.datasets[1].data = ts.cpu_freq_ghz ||
    | [];
727 |     liveCharts.cpu.update("none");
728 | }
729 |
730 | if (liveCharts.ram) {
731 |     liveCharts.ram.data.labels = labels;
732 |     liveCharts.ram.data.datasets[0].data = ts.ram_percent || [
    | ];
733 |     liveCharts.ram.data.datasets[1].data = ts.ram_used_gb || [
    | ];
734 |     liveCharts.ram.update("none");
735 | }
736 |
737 | if (liveCharts.gpu) {
738 |     liveCharts.gpu.data.labels = labels;
739 |     liveCharts.gpu.data.datasets[0].data = ts.gpu_load || [];
740 |     liveCharts.gpu.data.datasets[1].data = ts.gpu_temp || [];
741 |     liveCharts.gpu.update("none");
742 | }
743 |
744 | if (liveCharts.disk) {
745 |     liveCharts.disk.data.labels = labels;
746 |     liveCharts.disk.data.datasets[0].data = ts.disk_read_mb ||
    | [];
747 |     liveCharts.disk.data.datasets[1].data = ts.disk_write_mb |
    | | [];
748 |     liveCharts.disk.update("none");
749 | }
750 |
751 | if (liveCharts.net) {
752 |     liveCharts.net.data.labels = labels;
753 |     liveCharts.net.data.datasets[0].data = ts.net_recv_kb || [
    | ];
754 |     liveCharts.net.data.datasets[1].data = ts.net_sent_kb || [
```

```

    ];
755     liveCharts.net.update("none");
756   }
757 }
758
759 /**
760  * Отображение плиток датчиков и сенсоров.
761  */
762 function renderLiveSensors(sensors) {
763   const container = document.getElementById("live-sensors-cont
ainer");
764   if (!container) return;
765   if (!sensors || sensors.length === 0) {
766     container.innerHTML = '<div style="color: var(--text-secon
dary); grid-column: span 2;">Сенсоры LHM не зафиксированы в БД
</div>';
767     return;
768   }
769   container.innerHTML = "";
770   sensors.slice(0, 16).forEach((s) => {
771     const tile = document.createElement("div");
772     tile.style.background = "rgba(11, 15, 25, 0.6)";
773     tile.style.border = "1px solid var(--border-color)";
774     tile.style.borderRadius = "8px";
775     tile.style.padding = "8px 10px";
776     tile.style.display = "flex";
777     tile.style.justifyContent = "space-between";
778     tile.style.alignItems = "center";
779
780     const name = `${s.hardware_name ? s.hardware_name + " - "
: ""}${s.sensor_name}`;
781     const val = `${s.value} ${s.unit || ""}`;
782     tile.innerHTML = `
783       <div style="color: var(--text-secondary); font-size: 11p
x; max-width: 140px; overflow: hidden; text-overflow: ellipsis
; white-space: nowrap;" title="${name}">${name}</div>
784       <div style="font-weight: 700; color: var(--accent-blue);
">${val}</div>

```

```

785     `;
786     container.appendChild(tile);
787 });
788 }
789
790 /**
791  * Отображение таблицы топ активных процессов.
792  */
793 function renderLiveProcesses(processes) {
794     const tbody = document.getElementById("live-processes-body")
795     ;
796     if (!tbody) return;
797     if (!processes || processes.length === 0) {
798         tbody.innerHTML = '<tr><td colspan="5" style="text-align:
799         center; color: var(--text-secondary);">Нет записей о процессах
800         в выборке</td></tr>';
801         return;
802     }
803     tbody.innerHTML = "";
804     processes.forEach((p) => {
805         const tr = document.createElement("tr");
806         const cpuPct = Number(p.cpu_percent || 0).toFixed(1);
807         const memMb = Number(p.memory_mb || 0).toFixed(1);
808         tr.innerHTML = `
809             <td><code>${p.pid}</code></td>
810             <td><strong>${p.name}</strong></td>
811             <td style="color: ${cpuPct > 20 ? 'var(--accent-amber)'
812             : 'inherit'}; font-weight: ${cpuPct > 20 ? '700' : 'normal'};"
813             >${cpuPct}%</td>
814             <td>${memMb} MB</td>
815             <td>${p.num_threads || 1}</td>
816         `;
817         tbody.appendChild(tr);
818     });
819 }
820
821 /**
822  * Отображение ленты системных событий.

```



```

818 | */
819 | function renderLiveActivity(events) {
820 |     const list = document.getElementById("live-activity-list");
821 |     if (!list) return;
822 |     if (!events || events.length === 0) {
823 |         list.innerHTML = '<li style="color: var(--text-secondary);
      | text-align: center; padding: 10px;">События не зафиксированы<
      | /li>';
824 |         return;
825 |     }
826 |     list.innerHTML = "";
827 |     events.forEach((ev) => {
828 |         const li = document.createElement("li");
829 |         li.className = "activity-feed-item";
830 |         const ts = (ev.timestamp || "").replace("T", " ").substri
      | ng(11, 19);
831 |         const tagClass = ev.source === "device" ? "activity-tag-de
      | vice" : "activity-tag-system";
832 |         const desc = ev.friendly_name || ev.name || ev.path || ev.
      | event_type || "Событие";
833 |         li.innerHTML = `
834 |             <div class="activity-feed-left">
835 |                 <span style="font-weight: 600;">${desc}</span>
836 |                 <span style="color: var(--text-secondary); font-size:
      | 11px;">${ev.category || ev.provider || ev.event_type || ""}</s
      | pan>
837 |             </div>
838 |             <div style="text-align: right;">
839 |                 <span class="activity-tag ${tagClass}">${ev.event_type
      | || "EVT"}</span>
840 |                 <div style="font-size: 10px; color: var(--text-seconda
      | ry); margin-top: 4px;">${ts}</div>
841 |             </div>
842 |         `;
843 |         list.appendChild(li);
844 |     });
845 | }
846 |

```

```
847 /**
848  * Запуск глубокого исследования телеметрии.
849  */
850 async function runResearch() {
851     const btnRun = document.getElementById("btn-run");
852     const selectSource = document.getElementById("select-source"
853 );
854
855     const sourcePath = selectSource ? selectSource.value : null;
856
857     if (btnRun) {
858         btnRun.disabled = true;
859         btnRun.innerHTML = "⏸ Анализ...";
860     }
861
862     try {
863         const res = await fetch("/api/run-research", {
864             method: "POST",
865             headers: { "Content-Type": "application/json" },
866             body: JSON.stringify({ source_path: sourcePath || null })
867         },
868     );
869
870     if (!res.ok) throw new Error(`HTTP ${res.status}`);
871     const report = await res.json();
872     currentReport = report;
873
874     renderKPI(report);
875     renderResearchCharts(report.base_report.charts);
876     renderHypotheses(report.hypotheses);
877     renderCorrelations(report.correlations);
878     renderAnomalies(report.base_report.anomalies, report.base_
879 report.summary_conclusions);
880 } catch (err) {
881     console.error("Ошибка при исследовании телеметрии:", err);
882 } finally {
883     if (btnRun) {
884         btnRun.disabled = false;
885         btnRun.innerHTML = "⏸ Анализ";
886     }
887 }
```

```
882     }
883   }
884 }
885
886 /**
887  * Рендеринг сводных KPI карточек исследования.
888  */
889 function renderKPI(report) {
890   const base = report.base_report;
891   const healthEl = document.getElementById("kpi-health");
892   const recordsEl = document.getElementById("kpi-records");
893   const cpuEl = document.getElementById("kpi-cpu-val");
894   const ramEl = document.getElementById("kpi-ram-val");
895   const anomaliesEl = document.getElementById("kpi-anomalies")
896   ;
897   const devicesEl = document.getElementById("kpi-devices");
898   if (healthEl) {
899     healthEl.textContent = `${base.health_score}%`;
900     healthEl.style.color = base.health_score >= 80 ? "var(--accent-green)" : base.health_score >= 50 ? "var(--accent-amber)"
901     : "var(--accent-red)";
902   }
903   if (recordsEl) {
904     recordsEl.textContent = base.records_analyzed.toLocaleString();
905   }
906   if (cpuEl) {
907     const stats = base.statistics || {};
908     const cpuStat = stats.cpu_load || stats.cpu_total_percent;
909     if (cpuStat && cpuStat.count > 0) {
910       cpuEl.textContent = `CPU: ${cpuStat.avg_val}% (Max: ${cpuStat.max_val})`;
911     } else {
912       cpuEl.textContent = "CPU: --%";
913     }
914   }
```

```
915 | }
916 |
917 | if (ramEl) {
918 |     const stats = base.statistics || {};
919 |     const ramStat = stats.ram_used_percent || stats.memory_per
cent;
920 |     if (ramStat && ramStat.count > 0) {
921 |         ramEl.textContent = `RAM: ${ramStat.avg_val}% (Max: ${ra
mStat.max_val}%)`;
922 |     } else {
923 |         ramEl.textContent = "RAM: --%";
924 |     }
925 | }
926 |
927 | if (anomaliesEl) {
928 |     const count = (base.anomalies ? base.anomalies.length : 0)
+ (base.device_summary ? base.device_summary.error_count : 0)
;
929 |     anomaliesEl.textContent = count;
930 |     anomaliesEl.style.color = count === 0 ? "var(--accent-gree
n)" : count > 5 ? "var(--accent-red)" : "var(--accent-amber)";
931 | }
932 |
933 | if (devicesEl) {
934 |     const dev = base.device_summary;
935 |     if (dev && dev.error_count > 0) {
936 |         devicesEl.textContent = `Сбоев оборудования: ${dev.error
_count}`;
937 |         devicesEl.style.color = "var(--accent-red)";
938 |     } else {
939 |         devicesEl.textContent = "Оборудование в норме";
940 |         devicesEl.style.color = "var(--text-secondary)";
941 |     }
942 | }
943 | }
944 |
945 | /**
946 |  * Рендеринг графиков вкладки Исследования.
```

```
947 | */
948 | function renderResearchCharts(charts) {
949 |     const container = document.getElementById("charts-container"
    | );
950 |     if (!container) return;
951 |
952 |     Object.values(researchCharts).forEach((chart) => chart.destr
    | oy());
953 |     researchCharts = {};
954 |     container.innerHTML = "";
955 |
956 |     if (!charts || charts.length === 0) {
957 |         container.innerHTML = '<div class="kpi-card" style="grid-c
    | olumn: 1 / -1; text-align: center; padding: 40px;">Нет данных
    | для построения графиков исследования</div>';
958 |         return;
959 |     }
960 |
961 |     charts.forEach((cfg, idx) => {
962 |         const card = document.createElement("div");
963 |         card.className = "chart-card";
964 |
965 |         const header = document.createElement("div");
966 |         header.className = "chart-header";
967 |         header.innerHTML = `<h3 class="chart-title">${cfg.title}</
    | h3>`;
968 |
969 |         const wrapper = document.createElement("div");
970 |         wrapper.className = "chart-canvas-wrapper";
971 |
972 |         const canvas = document.createElement("canvas");
973 |         canvas.id = `research-chart-${idx}`;
974 |
975 |         wrapper.appendChild(canvas);
976 |         card.appendChild(header);
977 |         card.appendChild(wrapper);
978 |         container.appendChild(card);
979 |     });
```

```

980 |     const ctx = canvas.getContext("2d");
981 |     const chartDatasets = cfg.datasets.map((ds) => ({
982 |         label: ds.label,
983 |         data: ds.data,
984 |         borderColor: ds.borderColor || "#38bdf8",
985 |         backgroundColor: ds.backgroundColor || "rgba(56, 189, 24
8, 0.1)",
986 |         borderWidth: 2,
987 |         fill: true,
988 |         tension: 0.3,
989 |     }));
990 |
991 |     researchCharts[canvas.id] = new Chart(ctx, {
992 |         type: cfg.chart_type === "bar" ? "bar" : "line",
993 |         data: { labels: cfg.labels, datasets: chartDatasets },
994 |         options: {
995 |             responsive: true,
996 |             maintainAspectRatio: false,
997 |             scales: {
998 |                 x: { grid: { color: "rgba(35, 48, 77, 0.4)" }, ticks
: { color: "#94a3b8", maxRotation: 0, autoSkip: true, maxTicks
Limit: 8 } },
999 |                 y: { grid: { color: "rgba(35, 48, 77, 0.4)" }, ticks
: { color: "#94a3b8" } },
1000 |             },
1001 |         },
1002 |     });
1003 | });
1004 | }
1005 |
1006 | /**
1007 |  * Рендеринг гипотез.
1008 |  */
1009 | function renderHypotheses(hypotheses) {
1010 |     const container = document.getElementById("hypotheses-contai
ner");
1011 |     if (!container) return;
1012 |     container.innerHTML = "";

```

```

1013
1014   if (!hypotheses || hypotheses.length === 0) {
1015       container.innerHTML = '<div class="kpi-card" style="grid-c
column: 1 / -1; text-align: center; padding: 40px;">Гипотезы не
проверялись</div>';
1016       return;
1017   }
1018
1019   hypotheses.forEach((h) => {
1020       const card = document.createElement("div");
1021       card.className = "hypothesis-card";
1022       const statusClass = h.confirmed ? "status-confirmed" : "st
atus-rejected";
1023       const statusText = h.confirmed ? "Подтверждена" : "Отвергн
ута";
1024
1025       card.innerHTML = `
1026           <div class="hyp-header">
1027               <h4 style="font-size: 15px; font-weight: 600;">${h.nam
e}</h4>
1028               <span class="hyp-status ${statusClass}">${statusText}<
/span>
1029           </div>
1030           <p style="font-size: 13px; color: var(--text-secondary);
margin-bottom: 8px;">${h.description}</p>
1031           <div style="font-size: 12px; color: var(--text-primary);
margin-bottom: 8px;">
1032               <strong>Обоснование:</strong> ${h.evidence}
1033           </div>
1034           <div style="font-size: 11px; color: var(--accent-blue);
background: var(--bg-main); padding: 6px 10px; border-radius:
6px;">
1035               □ <strong>Рекомендация:</strong> ${h.recommendation}
1036           </div>
1037       `;
1038       container.appendChild(card);
1039   });
1040 }

```

```

1041 |
1042 | /**
1043 |  * Рендеринг корреляций Пирсона.
1044 |  */
1045 | function renderCorrelations(correlations) {
1046 |     const container = document.getElementById("correlations-cont
1047 | ainer");
1048 |     if (!container) return;
1049 |     container.innerHTML = "";
1050 |     if (!correlations || correlations.length === 0) {
1051 |         container.innerHTML = '<p style="color: var(--text-seconda
1052 | ry);">Недостаточно данных для матрицы корреляций</p>';
1053 |         return;
1054 |     }
1055 |     correlations.forEach((c) => {
1056 |         const item = document.createElement("div");
1057 |         item.className = "corr-item";
1058 |         const coeff = Number(c.pearson_coeff).toFixed(2);
1059 |         const isStrong = Math.abs(c.pearson_coeff) >= 0.7;
1060 |         const coeffClass = isStrong ? "corr-strong" : "corr-med";
1061 |
1062 |         item.innerHTML = `
1063 |             <div>
1064 |                 <strong>${c.metric_a}</strong> ↔ <strong>${c.metric_b}
1065 | </strong>
1066 |                 <div style="font-size: 12px; color: var(--text-seconda
1067 | ry); margin-top: 2px;">${c.interpretation}</div>
1068 |             </div>
1069 |             <div class="corr-coeff ${coeffClass}">${coeff}</div>
1070 |         `;
1071 |         container.appendChild(item);
1072 |     });
1073 | }
1074 | /**
1075 |  * Рендеринг аномалий и выводов.

```



```

1075 */
1076 function renderAnomalies(anomalies, conclusions) {
1077     const list = document.getElementById("conclusions-list");
1078     if (list) {
1079         list.innerHTML = "";
1080         if (conclusions && conclusions.length > 0) {
1081             conclusions.forEach((c) => {
1082                 const li = document.createElement("li");
1083                 li.style.marginBottom = "6px";
1084                 li.textContent = c;
1085                 list.appendChild(li);
1086             });
1087         } else {
1088             list.innerHTML = "<li>Аномалий не выявлено.</li>";
1089         }
1090     }
1091
1092     const tbody = document.getElementById("anomalies-table-body"
1093 );
1094     if (tbody) {
1095         tbody.innerHTML = "";
1096         if (anomalies && anomalies.length > 0) {
1097             anomalies.forEach((a) => {
1098                 const tr = document.createElement("tr");
1099                 const isCrit = a.severity === "critical";
1100                 const tagColor = isCrit ? "var(--accent-red)" : "var(-
1101 -accent-amber)";
1102                 tr.innerHTML = `
1103                     <td><span style="color: ${tagColor}; font-weight: 70
1104 0; text-transform: uppercase;">${a.severity}</span></td>
1105                     <td>${a.timestamp || "--"}</td>
1106                     <td><strong>${a.metric}</strong></td>
1107                     <td>${a.value}</td>
1108                     <td>${a.description}</td>
1109                 `;
1110                 tbody.appendChild(tr);
1111             });
1112         } else {

```

```
1110 |         tbody.innerHTML = '<tr><td colspan="5" style="text-align  
| : center; color: var(--accent-green);">Критических аномалий в  
| исследуемой выборке не обнаружено</td></tr>';  
1111 |     }  
1112 | }  
1113 | }  
1114 |  
1115 | /**  
1116 |  * Загрузка сырых записей телеметрии с пагинацией.  
1117 |  */  
1118 | async function loadRecords() {  
1119 |     const tbody = document.getElementById("records-table-body");  
1120 |     const selectSource = document.getElementById("select-source"  
| );  
1121 |     const sourcePath = selectSource ? selectSource.value : null;  
1122 |  
1123 |     if (tbody) {  
1124 |         tbody.innerHTML = '<tr><td colspan="4" style="text-align:  
| center; color: var(--text-secondary);">Загрузка записей...</td  
| ></tr>';  
1125 |     }  
1126 |  
1127 |     try {  
1128 |         const params = new URLSearchParams({  
1129 |             page: currentPage,  
1130 |             page_size: pageSize,  
1131 |         });  
1132 |         if (sourcePath) params.append("source_path", sourcePath);  
1133 |         if (searchQuery) params.append("query", searchQuery);  
1134 |  
1135 |         const res = await fetch(`/api/records?${params.toString()}  
| `);  
1136 |         if (!res.ok) throw new Error(`HTTP ${res.status}`);  
1137 |         const data = await res.json();  
1138 |  
1139 |         totalPages = Math.ceil(data.total / pageSize) || 1;  
1140 |         const pageLabel = document.getElementById("page-label");  
1141 |         if (pageLabel) {
```

```
1142     pageLabel.textContent = `Страница ${data.page} из ${totalPages} (Всего: ${data.total})`;
1143   }
1144
1145   if (tbody) {
1146     tbody.innerHTML = "";
1147     if (data.items.length === 0) {
1148       tbody.innerHTML = '<tr><td colspan="4" style="text-align: center; color: var(--text-secondary);">Записи не найдены</td></tr>';
1149       return;
1150     }
1151
1152     data.items.forEach((item) => {
1153       const tr = document.createElement("tr");
1154       const ts = item.timestamp || item.created_at || "--";
1155       const source = item.source_file || "sqlite";
1156       const type = item.event_type || (item.cpu_total_percent !== undefined ? "system_snapshot" : item.sensor_name ? "sensor_poll" : "custom");
1157
1158       const displayObj = { ...item };
1159       delete displayObj.source_file;
1160       delete displayObj.timestamp;
1161
1162       tr.innerHTML = `
1163         <td><code>${ts}</code></td>
1164         <td><span style="color: var(--accent-blue);">${source}</span></td>
1165         <td><span style="font-weight: 600;">${type}</span></td>
1166         <td><pre style="max-height: 80px; overflow-y: auto; font-size: 11px; background: rgba(0,0,0,0.2); padding: 4px; border-radius: 4px;">${JSON.stringify(displayObj, null, 2)}</pre></td>
1167       `;
1168       tbody.appendChild(tr);
1169     });
```

```

1170     }
1171 } catch (err) {
1172     if (tbody) {
1173         tbody.innerHTML = `|<td colspan="4" style="color: var
1174         (--accent-red); text-align: center;">Ошибка загрузки записей:
1175         ${err.message}</td></tr>`;
1176     }
1177 }
1178 /**
1179  * Открытие автономного HTML отчета в новой вкладке.
1180  */
1181 function openHtmlDashboard() {
1182     const selectSource = document.getElementById("select-source"
1183     );
1184     const sourcePath = selectSource ? selectSource.value : "";
1185     const url = `/api/dashboard${sourcePath ? `?source_path=${en
1186     codeURIComponent(sourcePath)}` : ""}`;
1187     window.open(url, "_blank");
1188 }
1189 /**
1190  * Скачивание JSON-отчета исследования.
1191  */
1192 function downloadJsonReport() {
1193     if (!currentReport) {
1194         alert("Сначала выполните анализ для генерации отчета");
1195         return;
1196     }
1197     const blob = new Blob([JSON.stringify(currentReport, null, 2
1198     )], { type: "application/json" });
1199     const url = URL.createObjectURL(blob);
1200     const a = document.createElement("a");
1201     a.href = url;
1202     a.download = `telemetry_research_${Date.now()}.json`;
1203     a.click();
1204     URL.revokeObjectURL(url);

|  |

```

```

1203 }
1204
1205 /**
1206  * Загрузка и рендеринг ресурсов клиентских программ.
1207  */
1208 async function loadClientResources(askGemini = false) {
1209     const tbody = document.getElementById("client-procs-table-bo
1210     dy");
1211     const statusText = document.getElementById("client-status-te
1212     xt");
1213     const statusSub = document.getElementById("client-status-sub
1214     ");
1215
1216     if (statusText) statusText.textContent = askGemini ? "Запрос
1217     к Gemini AI..." : "Сканирование программ клиента...";
1218     if (statusSub) statusSub.textContent = askGemini ? "Анализи
1219     рuem неизвестные ресурсоемкие программы через LLM..." : "Сбор м
1220     етрик памяти, ЦП и диска...";
1221
1222     if (tbody) {
1223         tbody.innerHTML = `|<td colspan="8" style="text-align:
1224         center; color: var(--accent-blue); padding: 20px;">␣ ${askGemi
1225         ni ? "Оценка неизвестных процессов через Gemini AI..." : "Загр
1226         узка данных о клиентских программах..."}</td></tr>`;
1227     }
1228
1229     const selectSource = document.getElementById("select-source"
1230     );
1231     const sourcePath = selectSource ? selectSource.value : "";
1232     const queryUrl = `/api/client-resources?ask_gemini=${askGemi
1233     ni}&limit=50${sourcePath ? `&db_path=${encodeURIComponent(sour
1234     cePath)}` : ""}`;
1235
1236     try {
1237         const res = await fetch(queryUrl);
1238         if (!res.ok) throw new Error(`HTTP ${res.status}`);
1239         const data = await res.json();
1240         currentClientSummary = data;

|  |

```

```
1229 |
1230 |     // Обновление KPI
1231 |     const kpiTotal = document.getElementById("client-kpi-total
    | ");
1232 |     const kpiRam = document.getElementById("client-kpi-ram");
1233 |     const kpiCpu = document.getElementById("client-kpi-cpu");
1234 |     const kpiUnknown = document.getElementById("client-kpi-unk
    | nown");
1235 |
1236 |     if (kpiTotal) kpiTotal.textContent = data.total_client_pro
    | grams || 0;
1237 |     if (kpiRam) kpiRam.textContent = `${data.total_ram_mb || 0
    | } MB`;
1238 |     if (kpiCpu) kpiCpu.textContent = `${data.total_cpu_percent
    | || 0}%`;
1239 |     if (kpiUnknown) kpiUnknown.textContent = data.unknown_heav
    | y_programs_count || 0;
1240 |
1241 |     renderClientProcsTable(data.programs || []);
1242 |     renderGeminiEvaluations(data.evaluations || []);
1243 |
1244 |     if (statusText) statusText.textContent = "Аудит программ к
    | лиента завершен";
1245 |     if (statusSub) statusSub.textContent = `Обнаружено ${data.
    | total_client_programs} программ (${data.unknown_heavy_programs
    | _count} неизвестных ресурсоемких)`;
1246 | } catch (err) {
1247 |     if (statusText) statusText.textContent = "Ошибка аудита";
1248 |     if (statusSub) statusSub.textContent = err.message;
1249 |     if (tbody) {
1250 |         tbody.innerHTML = `|<td colspan="8" style="color: var
    | (--accent-red); text-align: center;">Не удалось загрузить данн
    | ые: ${err.message}</td></tr>`;
1251 |     }
1252 | }
1253 | }
1254 |
1255 | /**
|  |

```

```
1256 * Отрисовка строк таблицы клиентских программ.
1257 */
1258 function renderClientProcsTable(programs) {
1259     const tbody = document.getElementById("client-procs-table-bo
1260     dy");
1261     if (!tbody) return;
1262     tbody.innerHTML = "";
1263     if (!programs || programs.length === 0) {
1264         tbody.innerHTML = `<tr><td colspan="8" style="text-align:
1265         center; color: var(--text-secondary);">Активных клиентских про
1266         грамм не обнаружено.</td></tr>`;
1267         return;
1268     }
1269     programs.forEach((p, idx) => {
1270         const tr = document.createElement("tr");
1271         const isHeavy = p.is_resource_heavy;
1272         const isKnown = p.is_known_software;
1273         let statusBadge = `<span class="badge badge-normal">□ Дове
1274         ренная</span>`;
1275         if (!isKnown && isHeavy) {
1276             statusBadge = `<span class="badge badge-critical">□ Неи
1277             звестная (Heavy)</span>`;
1278         } else if (!isKnown) {
1279             statusBadge = `<span class="badge badge-warning">i Неизв
1280             естная</span>`;
1281         } else if (isHeavy) {
1282             statusBadge = `<span class="badge badge-warning">□ Нагру
1283             зка</span>`;
1284         }
1285         const diskMb = p.disk_size_mb ? `${p.disk_size_mb} MB` : "
1286         —";
1287         const diskIo = roundVal(((p.disk_read_bytes_sec || 0) + (p
1288         .disk_write_bytes_sec || 0)) / (1024 * 1024), 2);
1289         const diskIoStr = diskIo > 0 ? ` (${diskIo} MB/s)` : "";
```

```

1285 |
1286 |     const pathOrCmd = p.executable_path || p.command_line || "
    | -";
1287 |
1288 |     let actionBtn = "";
1289 |     if (!isKnown || isHeavy) {
1290 |         actionBtn = `
```



```

OrCmd)}}">
1311     <span style="font-size: 11px; color: var(--text-second
    ary);">${escapeHtml(pathOrCmd)}</span>
1312     </td>
1313     <td>${statusBadge}</td>
1314     <td>${actionBtn}</td>
1315     `;
1316     tbody.appendChild(tr);
1317 });
1318 }
1319
1320 /**
1321  * Фильтрация таблицы клиентских программ.
1322  */
1323 function filterClientProcsTable(query) {
1324     if (!currentClientSummary || !currentClientSummary.programs)
1325         return;
1326     const q = (query || "").trim().toLowerCase();
1327     if (!q) {
1328         renderClientProcsTable(currentClientSummary.programs);
1329         return;
1330     }
1331     const filtered = currentClientSummary.programs.filter((p) =>
1332     {
1333         return (
1334             (p.name || "").toLowerCase().includes(q) ||
1335             (p.display_name || "").toLowerCase().includes(q) ||
1336             (p.executable_path || "").toLowerCase().includes(q) ||
1337             (p.category || "").toLowerCase().includes(q)
1338         );
1339     });
1340     renderClientProcsTable(filtered);
1341 }
1342
1343 /**
1344  * Отрисовка карточек с экспертным мнением Gemini.
1345  */
1346 function renderGeminiEvaluations(evaluations) {

```

```

1345 |   const container = document.getElementById("gemini-evaluation
      | s-container");
1346 |   const listEl = document.getElementById("gemini-evals-list");
1347 |   const badgeEl = document.getElementById("gemini-evals-count-
      | badge");
1348 |   if (!container || !listEl) return;
1349 |
1350 |   if (!evaluations || evaluations.length === 0) {
1351 |     container.style.display = "none";
1352 |     listEl.innerHTML = "";
1353 |     return;
1354 |   }
1355 |
1356 |   container.style.display = "block";
1357 |   if (badgeEl) badgeEl.textContent = `${evaluations.length} or
      | четов`;
1358 |   listEl.innerHTML = "";
1359 |
1360 |   evaluations.forEach((ev) => {
1361 |     const card = document.createElement("div");
1362 |     card.className = "gemini-card";
1363 |
1364 |     const isNorm = ev.is_normal;
1365 |     const badgeClass = isNorm ? "gemini-verdict-normal" : "gem
      | ini-verdict-risk";
1366 |     const badgeText = isNorm ? `✅ ${ev.verdict}` : `❌ ${ev.ve
      | rdict}`;
1367 |
1368 |     const recsHtml = (ev.recommendations || []).map((r) => `

```

```

    апуск из памяти/скрипта'}}</div>
1375     </div>
1376     <span class="gemini-verdict-badge ${badgeClass}">${bad
    geText}</span>
1377 </div>
1378 <div class="gemini-card-metrics">
1379     <span class="gemini-metric-tag">CPU: <b>${ev.cpu_perce
    nt}%</b></span>
1380     <span class="gemini-metric-tag">RAM: <b>${ev.memory_mb
    } MB</b></span>
1381     <span class="gemini-metric-tag">Диск IO: <b>${ev.disk_
    io_mb_s} MB/s</b></span>
1382     <span class="gemini-metric-tag">Риск: <b>${escapeHtml(
    ev.risk_level.toUpperCase())}</b></span>
1383 </div>
1384 <div class="gemini-card-analysis">
1385     ${escapeHtml(ev.analysis)}
1386 </div>
1387     ${recsHtml ? `<ul class="gemini-card-recs">${recsHtml}</
    ul>` : ""}
1388     `;
1389     listEl.appendChild(card);
1390 });
1391 }
1392
1393 /**
1394  * Оценка конкретной программы в Gemini по клику кнопки в табл
    ице.
1395  */
1396 window.evaluateSingleProgramInGemini = async function (pid, en
    codedName) {
1397     const name = decodeURIComponent(encodedName);
1398     if (!currentClientSummary || !currentClientSummary.programs)
    return;
1399
1400     const prog = currentClientSummary.programs.find((p) => p.pid
    === pid || p.name === name);
1401     if (!prog) {

```

```
1402 |     alert("Программа не найдена в текущем снимке");
1403 |     return;
1404 | }
1405 |
1406 | try {
1407 |     const res = await fetch("/api/client-resources/evaluate",
1408 | {
1409 |         method: "POST",
1410 |         headers: { "Content-Type": "application/json" },
1411 |         body: JSON.stringify(prog),
1412 |     });
1413 |     if (!res.ok) throw new Error(`HTTP ${res.status}`);
1414 |     const evaluation = await res.json();
1415 |
1416 |     prog.gemini_evaluation = evaluation;
1417 |     const existing = currentClientSummary.evaluations || [];
1418 |     const withoutSame = existing.filter((e) => e.program_name
1419 |     !== evaluation.program_name || e.pid !== evaluation.pid);
1420 |     withoutSame.unshift(evaluation);
1421 |     currentClientSummary.evaluations = withoutSame;
1422 |
1423 |     renderGeminiEvaluations(currentClientSummary.evaluations);
1424 |     document.getElementById("gemini-evaluations-container)?.s
1425 |     crollIntoView({ behavior: "smooth" });
1426 | } catch (err) {
1427 |     alert(`Ошибка оценки Gemini: ${err.message}`);
1428 | }
```

index.html

```
1 <!--
2 =====
3 |=====
4 Process Name: Windows Web Interface - Index View
5 |=====
6 Description:
7 |Шаблон клиентского веб-интерфейса index.
8 |
9 Usage Examples:
10 |    Web Route:
11 |        /windows/telemetry_research/web/index.html
12 |
13 |    Tab Frame Integration:
14 |        <iframe src="/windows/telemetry_research/web/index.html"
15 |        class="w-100 h-100 border-0"></iframe>
16 |
17 File: index.html
18 Project: ai-breadboard
19 Package: windows/telemetry_research/web
20 Author: hypo69
21 Copyright: © 2026 hypo69
22 Updated: 2026-10-01 13:04:40
23 |=====
24 |=====
25 -->
26
27 <!-- Updated: 2026-10-04 07:49:00 -->
28 <!DOCTYPE html>
29 <html lang="ru">
30 <head>
31 |    <meta charset="UTF-8">
32 |    <meta name="viewport" content="width=device-width, initial-s
33 |    cale=1.0">
34 |    <title>Исследование телеметрии Windows | Web GUI</title>
35 |    <link rel="stylesheet" href="/static/style.css?v=20261004_v1
```

```
    ">
32    <script src="https://cdn.jsdelivr.net/npm/chart.js"></script
    >
33 </head>
34 <body>
35     <header class="app-header">
36         <div class="logo-section">
37             <div class="logo-icon">❏ </div>
38             <div class="logo-text">
39                 <h1>Исследование телеметрии Windows</h1>
40                 <p>Анализ долговременной телеметрии, EDA, ресурсы клие
    нтских программ и оценка Gemini</p>
41             </div>
42         </div>
43         <div class="header-controls">
44             <select id="select-source" class="source-select">
45                 <option value="">Загрузка источников...</option>
46             </select>
47             <button id="btn-run" class="btn btn-primary">❏ Анализ</
    button>
48             <button id="btn-dashboard" class="btn btn-secondary">❏
    HTML Дашборд</button>
49             <button id="btn-json" class="btn btn-secondary">❏ JSON<
    /button>
50         </div>
51     </header>
52
53     <main class="main-container">
54         <!-- KPI Summary Grid -->
55         <section class="kpi-grid">
56             <div class="kpi-card">
57                 <div class="kpi-title">Индекс здоровья (Health Score)<
    /div>
58                 <div class="kpi-value" id="kpi-health">--%</div>
59                 <div class="kpi-sub">Интегральный показатель стабильно
    сти</div>
60             </div>
61             <div class="kpi-card">
```

```

62 |         <div class="kpi-title">Записей в выборке</div>
63 |         <div class="kpi-value" id="kpi-records">0</div>
64 |         <div class="kpi-sub">Точек замеров телеметрии</div>
65 |     </div>
66 |     <div class="kpi-card">
67 |         <div class="kpi-title">ЦП и Память (Avg / Max)</div>
68 |         <div class="kpi-value" style="font-size: 16px; margin-
top: 6px;" id="kpi-cpu-val">CPU: --%</div>
69 |         <div class="kpi-sub" id="kpi-ram-val">RAM: --%</div>
70 |     </div>
71 |     <div class="kpi-card">
72 |         <div class="kpi-title">Аномалии и сбои</div>
73 |         <div class="kpi-value" id="kpi-anomalies" style="color
: var(--accent-amber);">0</div>
74 |         <div class="kpi-sub" id="kpi-devices">Оборудование в н
орме</div>
75 |     </div>
76 | </section>
77 |
78 | <!-- Navigation Tabs -->
79 | <nav class="tabs-nav">
80 |     <button class="tab-btn active" data-tab="tab-current">□
Текущее состояние (telemetry.db)</button>
81 |     <button class="tab-btn" data-tab="tab-client-resources">
□ Ресурсы программ клиента</button>
82 |     <button class="tab-btn" data-tab="tab-charts">□ Графики
исследования</button>
83 |     <button class="tab-btn" data-tab="tab-hypotheses">□ Сис
темные гипотезы</button>
84 |     <button class="tab-btn" data-tab="tab-anomalies">□ Аном
алии и выводы</button>
85 |     <button class="tab-btn" data-tab="tab-correlations">□ К
орреляции</button>
86 |     <button class="tab-btn" data-tab="tab-records">□ Журнал
записей</button>
87 | </nav>
88 |
89 | <!-- Tab: Current State (telemetry.db) -->

```

```

90 |     <div id="tab-current" class="tab-content active">
91 |         <!-- Live Status & Controls Bar -->
92 |         <div class="live-status-bar">
93 |             <div class="status-badge-container">
94 |                 <div class="pulse-dot"></div>
95 |                 <div>
96 |                     <div class="status-badge-text" id="live-db-status"
>Подключение к telemetry.db...</div>
97 |                     <div class="status-badge-sub" id="live-db-sub">Заг
рузка информации о базе данных...</div>
98 |                 </div>
99 |             </div>
100 |             <div class="live-controls">
101 |                 <label class="toggle-label">
102 |                     <input type="checkbox" id="auto-refresh-toggle" ch
ecked>
103 |                     <span>⏻ Автообновление (3с)</span>
104 |                 </label>
105 |                 <select id="current-range-select" class="source-sele
ct" style="min-width: 160px;">
106 |                     <option value="50">50 замеров</option>
107 |                     <option value="100" selected>100 замеров</option>
108 |                     <option value="300">300 замеров</option>
109 |                     <option value="500">500 замеров</option>
110 |                     <option value="1000">1000 замеров</option>
111 |                 </select>
112 |                 <button id="btn-refresh-current" class="btn btn-seco
ndary">⏻ Обновить</button>
113 |             </div>
114 |         </div>
115 |
116 |         <!-- Current Instant KPI Cards -->
117 |         <div class="current-kpi-grid">
118 |             <div class="current-kpi-card">
119 |                 <div class="current-kpi-header">
120 |                     <span class="kpi-title">Процессор (CPU)</span>
121 |                     <span class="current-kpi-icon">⏻ </span>
122 |                 </div>

```



```
123 |         <div class="kpi-value" id="curr-cpu-val">--%</div>
124 |         <div class="kpi-sub" id="curr-cpu-freq">Частота: --
    | GHz</div>
125 |         <div class="mini-meter-bg"><div class="mini-meter-fi
    | ll" id="curr-cpu-meter" style="width: 0%;"></div></div>
126 |     </div>
127 |
128 |     <div class="current-kpi-card">
129 |         <div class="current-kpi-header">
130 |             <span class="kpi-title">Память (RAM)</span>
131 |             <span class="current-kpi-icon">📊 </span>
132 |         </div>
133 |         <div class="kpi-value" id="curr-ram-val">--%</div>
134 |         <div class="kpi-sub" id="curr-ram-gb">-- / -- GB</di
    | v>
135 |         <div class="mini-meter-bg"><div class="mini-meter-fi
    | ll" id="curr-ram-meter" style="width: 0%; background: var(--ac
    | cent-green);"></div></div>
136 |     </div>
137 |
138 |     <div class="current-kpi-card">
139 |         <div class="current-kpi-header">
140 |             <span class="kpi-title">Видеокарта (GPU)</span>
141 |             <span class="current-kpi-icon">📊 </span>
142 |         </div>
143 |         <div class="kpi-value" id="curr-gpu-val">--%</div>
144 |         <div class="kpi-sub" id="curr-gpu-temp">Темп: --°C</
    | div>
145 |         <div class="mini-meter-bg"><div class="mini-meter-fi
    | ll" id="curr-gpu-meter" style="width: 0%; background: var(--ac
    | cent-purple);"></div></div>
146 |     </div>
147 |
148 |     <div class="current-kpi-card">
149 |         <div class="current-kpi-header">
150 |             <span class="kpi-title">Дисковая активность</span>
151 |             <span class="current-kpi-icon">📊 </span>
152 |         </div>
```

```
153 |         <div class="kpi-value" style="font-size: 18px;" id="
| curr-disk-val">R: 0 / W: 0 MB/s</div>
154 |         <div class="kpi-sub" id="curr-disk-sub">Чтение и зап
| ись на диск</div>
155 |     </div>
156 |
157 |     <div class="current-kpi-card">
158 |         <div class="current-kpi-header">
159 |             <span class="kpi-title">Сетевой трафик</span>
160 |             <span class="current-kpi-icon">⬇️ </span>
161 |         </div>
162 |         <div class="kpi-value" style="font-size: 18px;" id="
| curr-net-val">⬇️ 0 / ⬆️ 0 KB/s</div>
163 |         <div class="kpi-sub" id="curr-uptime-val">Аптайм: --
| </div>
164 |     </div>
165 | </div>
166 |
167 | <!-- Current Charts Grid -->
168 | <div class="charts-grid">
169 |     <div class="chart-card">
170 |         <div class="chart-header">
171 |             <h3 class="chart-title">⬇️ Нагрузка и частота ЦП (
| CPU)</h3>
172 |         </div>
173 |         <div class="chart-canvas-wrapper"><canvas id="chart-
| live-cpu"></canvas></div>
174 |     </div>
175 |
176 |     <div class="chart-card">
177 |         <div class="chart-header">
178 |             <h3 class="chart-title">⬇️ Оперативная память (RAM
| )</h3>
179 |         </div>
180 |         <div class="chart-canvas-wrapper"><canvas id="chart-
| live-ram"></canvas></div>
181 |     </div>
182 |
```

```

183 |         <div class="chart-card">
184 |             <div class="chart-header">
185 |                 <h3 class="chart-title">⬢ Нагрузка и температура
GPU</h3>
186 |             </div>
187 |             <div class="chart-canvas-wrapper"><canvas id="chart-
live-gpu"></canvas></div>
188 |         </div>
189 |
190 |         <div class="chart-card">
191 |             <div class="chart-header">
192 |                 <h3 class="chart-title">⬢ Дисковая скорость (Read
/ Write MB/s)</h3>
193 |             </div>
194 |             <div class="chart-canvas-wrapper"><canvas id="chart-
live-disk"></canvas></div>
195 |         </div>
196 |
197 |         <div class="chart-card">
198 |             <div class="chart-header">
199 |                 <h3 class="chart-title">⬢ Сетевая пропускная спос
обность (KB/s)</h3>
200 |             </div>
201 |             <div class="chart-canvas-wrapper"><canvas id="chart-
live-net"></canvas></div>
202 |         </div>
203 |
204 |         <div class="chart-card">
205 |             <div class="chart-header">
206 |                 <h3 class="chart-title">⬢ Аппаратные сенсоры (LHM
/ Hardware)</h3>
207 |             </div>
208 |             <div class="chart-canvas-wrapper" style="overflow-y:
auto; max-height: 280px;">
209 |                 <div id="live-sensors-container" style="display: g
rid; grid-template-columns: 1fr 1fr; gap: 8px; font-size: 12px
;"></div>
210 |             </div>

```

```

211         </div>
212     </div>
213
214     <!-- Split Grid: Top Processes + Activity Feed -->
215     <div class="split-grid">
216         <!-- Top Processes -->
217         <div class="table-wrapper">
218             <div class="table-header-bar">
219                 <h3 style="font-size: 15px;">☐ Топ активных проце
                ссов (telemetry.db)</h3>
220             </div>
221             <table>
222                 <thead>
223                     <tr>
224                         <th style="width: 70px;">PID</th>
225                         <th>Процесс</th>
226                         <th style="width: 80px;">CPU %</th>
227                         <th style="width: 90px;">RAM MB</th>
228                         <th style="width: 70px;">Потоки</th>
229                     </tr>
230                 </thead>
231                 <tbody id="live-processes-body"></tbody>
232             </table>
233         </div>
234
235         <!-- Activity Feed -->
236         <div class="table-wrapper">
237             <div class="table-header-bar">
238                 <h3 style="font-size: 15px;">☐ Системная активност
                ь и события</h3>
239             </div>
240             <div style="padding: 12px;">
241                 <ul class="activity-feed-list" id="live-activity-l
                ist"></ul>
242             </div>
243         </div>
244     </div>
245 </div>

```

```
246 |
247 |     <!-- Tab: Client Program Resources & Gemini AI -->
248 |     <div id="tab-client-resources" class="tab-content">
249 |         <!-- Client KPI Strip -->
250 |         <div class="kpi-grid" style="margin-bottom: 20px;">
251 |             <div class="kpi-card">
252 |                 <div class="kpi-title">Программ клиента</div>
253 |                 <div class="kpi-value" id="client-kpi-total">0</div>
254 |                 <div class="kpi-sub">Активных пользовательских проце
255 | ссов</div>
256 |             </div>
257 |             <div class="kpi-card">
258 |                 <div class="kpi-title">Занято памяти RAM</div>
259 |                 <div class="kpi-value" id="client-kpi-ram">0 MB</div>
260 |             </div>
261 |             <div class="kpi-card">
262 |                 <div class="kpi-title">Загрузка ЦП (CPU)</div>
263 |                 <div class="kpi-value" id="client-kpi-cpu">0%</div>
264 |                 <div class="kpi-sub">Суммарная нагрузка на ядра</div>
265 |             </div>
266 |             <div class="kpi-card">
267 |                 <div class="kpi-title">Неизвестных (Heavy)</div>
268 |                 <div class="kpi-value" id="client-kpi-unknown" style
269 | ="color: var(--accent-red);">0</div>
270 |                 <div class="kpi-sub">Требуют анализа в Gemini</div>
271 |             </div>
272 |
273 |     <!-- Controls & Actions Bar -->
274 |     <div class="live-status-bar" style="margin-bottom: 16px;
275 | ">
276 |         <div class="status-badge-container">
277 |             <div class="pulse-dot"></div>
278 |             <div>
```

```
278 |         <div class="status-badge-text" id="client-status-t
| ext">Аудит программ клиента</div>
279 |         <div class="status-badge-sub" id="client-status-su
| b">Отслеживание потребления ресурсов и проверка в Gemini AI</d
| iv>
280 |     </div>
281 | </div>
282 |     <div class="live-controls">
283 |         <button id="btn-refresh-client-procs" class="btn btn
| -secondary">⏻ Обновить список</button>
284 |         <button id="btn-ask-gemini-all-unknown" class="btn b
| tn-primary">⏻ Проверить неизвестные в Gemini</button>
285 |     </div>
286 | </div>
287 |
288 |     <!-- Gemini AI Evaluations Container -->
289 |     <div id="gemini-evaluations-container" style="margin-bot
| tom: 20px; display: none;">
290 |         <div class="kpi-card" style="border-left: 4px solid va
| r(--accent-purple);">
291 |             <div style="display: flex; justify-content: space-be
| tween; align-items: center; margin-bottom: 12px;">
292 |                 <h3 style="font-size: 16px; color: var(--accent-pu
| rple);">⏻ Экспертные заключения Gemini AI по неизвестным ресу
| рсоемким программам</h3>
293 |                 <span class="badge badge-warning" id="gemini-evals
| -count-badge">0 отчетов</span>
294 |             </div>
295 |             <div id="gemini-evals-list" class="gemini-evals-grid
| "></div>
296 |         </div>
297 |     </div>
298 |
299 |     <!-- Client Programs Table -->
300 |     <div class="table-wrapper">
301 |         <div class="table-header-bar">
302 |             <h3 style="font-size: 15px;">Программы, запущенные к
| лиентом (память, процессор, диск)</h3>
```

```

303 |         <input type="text" id="input-search-client-procs" cl
      | ass="search-input" placeholder="Фильтр по названию программы..
      | .">
304 |     </div>
305 |     <table>
306 |         <thead>
307 |             <tr>
308 |                 <th style="width: 220px;">Программа / Процесс</t
      | h>
309 |                 <th style="width: 80px;">PID</th>
310 |                 <th style="width: 110px;">CPU (%)</th>
311 |                 <th style="width: 140px;">RAM (МБ / %)</th>
312 |                 <th style="width: 130px;">Диск (файл / IO)</th>
313 |                 <th>Каталог запуска / Командная строка</th>
314 |                 <th style="width: 150px;">Статус / Доверие</th>
315 |                 <th style="width: 140px;">Действие</th>
316 |             </tr>
317 |         </thead>
318 |         <tbody id="client-procs-table-body">
319 |             <tr><td colspan="8" style="text-align: center; col
      | or: var(--text-secondary);">Загрузка данных о программах клиен
      | та...</td></tr>
320 |         </tbody>
321 |     </table>
322 | </div>
323 | </div>
324 |
325 | <!-- Tab: Charts (Research) -->
326 | <div id="tab-charts" class="tab-content">
327 |     <div class="charts-grid" id="charts-container">
328 |         <!-- Rendered by Chart.js -->
329 |     </div>
330 | </div>
331 |
332 | <!-- Tab: Hypotheses -->
333 | <div id="tab-hypotheses" class="tab-content">
334 |     <div class="hypotheses-grid" id="hypotheses-container">
335 |         <!-- Rendered by app.js -->

```

```

336 |         </div>
337 |     </div>
338 |
339 |     <!-- Tab: Anomalies & Conclusions -->
340 |     <div id="tab-anomalies" class="tab-content">
341 |         <div class="kpi-card" style="margin-bottom: 20px;">
342 |             <h3 style="font-size: 16px; margin-bottom: 12px; color
: var(--accent-green);">□ Сводные выводы исследования</h3>
343 |             <ul id="conclusions-list" style="padding-left: 20px; c
olor: var(--text-primary); font-size: 14px;"></ul>
344 |         </div>
345 |
346 |         <div class="table-wrapper">
347 |             <div class="table-header-bar">
348 |                 <h3 style="font-size: 15px;">Выявленные аномалии и к
ритические всплески</h3>
349 |             </div>
350 |             <table>
351 |                 <thead>
352 |                     <tr>
353 |                         <th>Уровень</th>
354 |                         <th>Время</th>
355 |                         <th>Метрика</th>
356 |                         <th>Значение</th>
357 |                         <th>Описание</th>
358 |                     </tr>
359 |                 </thead>
360 |                 <tbody id="anomalies-table-body"></tbody>
361 |             </table>
362 |         </div>
363 |     </div>
364 |
365 |     <!-- Tab: Correlations -->
366 |     <div id="tab-correlations" class="tab-content">
367 |         <div class="kpi-card">
368 |             <h3 style="font-size: 16px; margin-bottom: 12px; color
: var(--accent-blue);">Матрица корреляций между метриками (Пир
сон)</h3>

```



```

369         <div class="correlation-list" id="correlations-contain
er"></div>
370     </div>
371 </div>
372
373 <!-- Tab: Records -->
374 <div id="tab-records" class="tab-content">
375     <div class="table-wrapper">
376         <div class="table-header-bar">
377             <input type="text" id="input-search" class="search-i
nput" placeholder="Поиск по записям (Enter)...">
378             <div id="page-label" style="font-size: 13px; color:
var(--text-secondary);">Страница 1 из 1</div>
379         </div>
380         <table>
381             <thead>
382                 <tr>
383                     <th style="width: 180px;">Время</th>
384                     <th style="width: 200px;">Источник</th>
385                     <th style="width: 140px;">Тип</th>
386                     <th>Данные (JSON)</th>
387                 </tr>
388             </thead>
389             <tbody id="records-table-body"></tbody>
390         </table>
391         <div class="pagination-bar">
392             <button id="btn-prev-page" class="btn btn-secondary"
>⏪ Назад</button>
393             <button id="btn-next-page" class="btn btn-secondary"
>Вперед ⏩ </button>
394         </div>
395     </div>
396 </div>
397
398 <!-- AI Response Modal / Dynamic Presentation Card -->
399 <section id="ai-response-card" class="ai-response-card" st
yle="display: none;">
400     <div class="ai-response-header">

```

```
401         <div class="ai-response-title-group">
402             <span class="ai-badge"> AI Ассистент Телеметрии</span>
403             <h2 id="ai-response-title" class="ai-response-title">Отчет и визуализация</h2>
404         </div>
405         <div class="ai-header-actions">
406             <button id="btn-ai-close" class="btn-icon" title="Закрыть панель"></button>
407         </div>
408     </div>
409
410     <div class="ai-response-content">
411         <!-- Dynamic Chart Area -->
412         <div id="ai-chart-wrapper" class="ai-chart-wrapper" style="display: none;">
413             <div class="ai-chart-header">
414                 <h3 id="ai-chart-title" style="font-size: 15px; color: var(--accent-blue);"> Визуализация</h3>
415             </div>
416             <div class="ai-canvas-container">
417                 <canvas id="chart-ai-dynamic"></canvas>
418             </div>
419         </div>
420
421         <!-- KPI Metrics Badges -->
422         <div id="ai-metrics-row" class="ai-metrics-row" style="display: none;"></div>
423
424         <!-- Markdown Reply Text -->
425         <div id="ai-reply-body" class="ai-reply-body"></div>
426
427         <!-- SQL Queries Collapsible Badge -->
428         <details id="ai-sql-details" class="ai-sql-details" style="display: none;">
429             <summary> Выполненные SQL-запросы к <code>telemetry.db</code></summary>
430             <pre id="ai-sql-code" class="ai-sql-code"></pre>
```

```

431         </details>
432
433         <!-- Follow-up Chips -->
434         <div id="ai-followup-row" class="ai-followup-row" styl
e="display: none;">
435             <span style="font-size: 12px; color: var(--text-seco
ndary); margin-right: 8px;">□ Уточнить:</span>
436             <div id="ai-followup-chips" class="ai-chips-list"></
div>
437         </div>
438     </div>
439 </section>
440
441 <!-- Bottom AI Query Bar (Sticky / Fixed) -->
442 <footer class="bottom-ai-bar">
443     <!-- Quick Prompt Chips -->
444     <div class="ai-quick-chips">
445         <button class="prompt-chip" data-query="покажи график
загрузки цпу по времени">
446             □ График загрузки ЦПУ по времени
447         </button>
448         <button class="prompt-chip" data-query="Посчитай приме
рное общее потребление электроэнергии компьютером и покажи сут
очную круговую диаграмму">
449             □ Суточное потребление энергии (круговая диаграмма)
450         </button>
451         <button class="prompt-chip" data-query="топ процессов
по cpu">
452             □ Топ процессов по CPU
453         </button>
454         <button class="prompt-chip" data-query="SELECT timesta
mp, event_type, friendly_name FROM device_events ORDER BY id D
ESC LIMIT 5">
455             □ SQL: События устройств
456         </button>
457     </div>
458
459     <!-- Input and Send Button Form -->

```

```
460 |         <form id="ai-query-form" class="ai-query-form" onsubmit=
    | "return false;">
461 |         <div class="ai-input-wrapper">
462 |             <span class="ai-input-icon">❏ </span>
463 |             <input
464 |                 type="text"
465 |                 id="ai-query-input"
466 |                 class="ai-query-input"
467 |                 placeholder="Спросите ИИ о телеметрии (например: «
    | покажи график загрузки цпу по времени», «посчитай потребление
    | электроэнергии»)... "
468 |                 autocomplete="off"
469 |             />
470 |             <button type="button" id="btn-ai-clear" class="btn-c
    | lear" title="Очистить" style="display: none;">❏ </button>
471 |         </div>
472 |         <button type="submit" id="btn-ai-send" class="btn btn-
    | primary btn-ai-send">
473 |             <span id="btn-ai-send-text">Отправить</span>
474 |             <span id="btn-ai-spinner" class="spinner" style="dis
    | play: none;"></span>
475 |         </button>
476 |     </form>
477 | </footer>
478 | </main>
479 |
480 | <script src="/static/app.js?v=20261001_v2"></script>
481 | </body>
482 | </html>
483 |
```

style.css

```
1 /**
2  * =====
3  * Process Name: Windows Web Styles - Style Stylesheet
4  * =====
5  * Description:
6  *   Таблица стилей стиля и оформления интерфейса (style).
7  *
8  * Usage Examples:
9  *   HTML Link Tag:
10 *     <link rel="stylesheet" href="/windows/telemetry_research/web/style.css?v=20261001_v1">
11 *
12 * File: style.css
13 * Project: ai-breadboard
14 * Package: windows/telemetry_research/web
15 * Author: hypo69
16 * Copyright: © 2026 hypo69
17 * Updated: 2026-10-01 13:04:40
18 * =====
19 */
20
21 /* Стили для Web GUI Исследования Телеметрии */
22 :root {
23   --bg-main: #0b0f19;
24   --bg-card: #131b2e;
25   --bg-card-hover: #1c2742;
26   --border-color: #23304d;
27   --text-primary: #f1f5f9;
28   --text-secondary: #94a3b8;
29   --accent-blue: #38bdf8;
30   --accent-green: #10b981;
31   --accent-amber: #f59e0b;
32   --accent-red: #ef4444;
```

```
33 | --accent-purple: #a855f7;
34 | --radius-md: 10px;
35 | --radius-lg: 14px;
36 | --shadow-sm: 0 2px 8px rgba(0, 0, 0, 0.3);
37 | --shadow-md: 0 4px 16px rgba(0, 0, 0, 0.4);
38 | --shadow-lg: 0 10px 30px rgba(0, 0, 0, 0.6);
39 | }
40 |
41 | * {
42 |   box-sizing: border-box;
43 |   margin: 0;
44 |   padding: 0;
45 | }
46 |
47 | body {
48 |   font-family: -apple-system, BlinkMacSystemFont, "Segoe UI",
49 |   Roboto, "Helvetica Neue", Arial, sans-serif;
50 |   background-color: var(--bg-main);
51 |   color: var(--text-primary);
52 |   min-height: 100vh;
53 |   display: flex;
54 |   flex-direction: column;
55 |   padding-bottom: 140px; /* Отступ для фиксированной нижней па
56 |   нели */
57 | }
58 |
59 | .app-header {
60 |   background: var(--bg-card);
61 |   border-bottom: 1px solid var(--border-color);
62 |   padding: 16px 28px;
63 |   display: flex;
64 |   justify-content: space-between;
65 |   align-items: center;
66 |   position: sticky;
67 |   top: 0;
68 |   z-index: 100;
69 |   box-shadow: var(--shadow-sm);
70 | }
```

```
69 |
70 | .logo-section {
71 |   display: flex;
72 |   align-items: center;
73 |   gap: 12px;
74 | }
75 |
76 | .logo-icon {
77 |   font-size: 28px;
78 |   line-height: 1;
79 | }
80 |
81 | .logo-text h1 {
82 |   font-size: 20px;
83 |   font-weight: 700;
84 |   letter-spacing: -0.5px;
85 | }
86 |
87 | .logo-text p {
88 |   font-size: 12px;
89 |   color: var(--text-secondary);
90 | }
91 |
92 | .header-controls {
93 |   display: flex;
94 |   align-items: center;
95 |   gap: 12px;
96 | }
97 |
98 | .source-select {
99 |   background: var(--bg-main);
100 |   border: 1px solid var(--border-color);
101 |   color: var(--text-primary);
102 |   padding: 8px 14px;
103 |   border-radius: var(--radius-md);
104 |   font-size: 13px;
105 |   outline: none;
106 |   min-width: 260px;
```

```
107 |   cursor: pointer;
108 | }
109 |
110 | .source-select:focus {
111 |   border-color: var(--accent-blue);
112 | }
113 |
114 | .btn {
115 |   display: inline-flex;
116 |   align-items: center;
117 |   gap: 6px;
118 |   padding: 8px 16px;
119 |   border-radius: var(--radius-md);
120 |   font-size: 13px;
121 |   font-weight: 600;
122 |   cursor: pointer;
123 |   border: none;
124 |   transition: all 0.2s ease;
125 | }
126 |
127 | .btn-primary {
128 |   background: var(--accent-blue);
129 |   color: #0b0f19;
130 | }
131 |
132 | .btn-primary:hover {
133 |   background: #7dd3fc;
134 |   box-shadow: 0 0 12px rgba(56, 189, 248, 0.4);
135 | }
136 |
137 | .btn-secondary {
138 |   background: var(--bg-card-hover);
139 |   color: var(--text-primary);
140 |   border: 1px solid var(--border-color);
141 | }
142 |
143 | .btn-secondary:hover {
144 |   background: #273659;
```



```
145 |   border-color: var(--accent-blue);
146 | }
147 |
148 | .main-container {
149 |   max-width: 1400px;
150 |   width: 100%;
151 |   margin: 0 auto;
152 |   padding: 24px 28px;
153 |   display: flex;
154 |   flex-direction: column;
155 |   gap: 24px;
156 | }
157 |
158 | /* KPI Grid */
159 | .kpi-grid {
160 |   display: grid;
161 |   grid-template-columns: repeat(auto-fit, minmax(260px, 1fr));
162 |   gap: 16px;
163 | }
164 |
165 | .kpi-card {
166 |   background: var(--bg-card);
167 |   border: 1px solid var(--border-color);
168 |   border-radius: var(--radius-lg);
169 |   padding: 18px 20px;
170 |   box-shadow: var(--shadow-sm);
171 |   transition: transform 0.2s, box-shadow 0.2s;
172 | }
173 |
174 | .kpi-card:hover {
175 |   transform: translateY(-2px);
176 |   box-shadow: var(--shadow-md);
177 | }
178 |
179 | .kpi-title {
180 |   font-size: 12px;
181 |   font-weight: 600;
182 |   text-transform: uppercase;
```

```
183 | color: var(--text-secondary);
184 | letter-spacing: 0.5px;
185 | }
186 |
187 | .kpi-value {
188 |   font-size: 26px;
189 |   font-weight: 700;
190 |   margin-top: 8px;
191 |   color: var(--text-primary);
192 | }
193 |
194 | .kpi-sub {
195 |   font-size: 12px;
196 |   color: var(--text-secondary);
197 |   margin-top: 4px;
198 | }
199 |
200 | /* Tabs */
201 | .tabs-nav {
202 |   display: flex;
203 |   gap: 8px;
204 |   border-bottom: 1px solid var(--border-color);
205 |   padding-bottom: 8px;
206 |   overflow-x: auto;
207 | }
208 |
209 | .tab-btn {
210 |   background: transparent;
211 |   border: none;
212 |   color: var(--text-secondary);
213 |   padding: 8px 16px;
214 |   font-size: 13px;
215 |   font-weight: 600;
216 |   border-radius: var(--radius-md);
217 |   cursor: pointer;
218 |   transition: all 0.2s;
219 |   white-space: nowrap;
220 | }
```

```
221 |
222 | .tab-btn:hover {
223 |   color: var(--text-primary);
224 |   background: var(--bg-card);
225 | }
226 |
227 | .tab-btn.active {
228 |   color: var(--accent-blue);
229 |   background: rgba(56, 189, 248, 0.1);
230 |   border-bottom: 2px solid var(--accent-blue);
231 | }
232 |
233 | .tab-content {
234 |   display: none;
235 | }
236 |
237 | .tab-content.active {
238 |   display: block;
239 | }
240 |
241 | /* Live Status & Controls Bar */
242 | .live-status-bar {
243 |   display: flex;
244 |   justify-content: space-between;
245 |   align-items: center;
246 |   background: var(--bg-card);
247 |   border: 1px solid var(--border-color);
248 |   border-radius: var(--radius-lg);
249 |   padding: 12px 20px;
250 |   margin-bottom: 18px;
251 |   box-shadow: var(--shadow-sm);
252 |   flex-wrap: wrap;
253 |   gap: 12px;
254 | }
255 |
256 | .status-badge-container {
257 |   display: flex;
258 |   align-items: center;
```

```
259 | gap: 12px;
260 | }
261 |
262 | .pulse-dot {
263 |     width: 10px;
264 |     height: 10px;
265 |     background-color: var(--accent-green);
266 |     border-radius: 50%;
267 |     box-shadow: 0 0 8px var(--accent-green);
268 |     animation: pulse-animation 2s infinite;
269 | }
270 |
271 | @keyframes pulse-animation {
272 |     0% { transform: scale(0.95); box-shadow: 0 0 0 0 rgba(16, 18
273 |         5, 129, 0.7); }
274 |     70% { transform: scale(1); box-shadow: 0 0 0 8px rgba(16, 18
275 |         5, 129, 0); }
276 |     100% { transform: scale(0.95); box-shadow: 0 0 0 0 rgba(16,
277 |         185, 129, 0); }
278 | }
279 |
280 | .status-badge-text {
281 |     font-size: 14px;
282 |     font-weight: 600;
283 | }
284 |
285 | .status-badge-sub {
286 |     font-size: 11px;
287 |     color: var(--text-secondary);
288 |     margin-top: 2px;
289 | }
290 |
291 | .live-controls {
292 |     display: flex;
293 |     align-items: center;
294 |     gap: 12px;
295 | }
```

```
294 | .toggle-label {
295 |     display: flex;
296 |     align-items: center;
297 |     gap: 6px;
298 |     font-size: 12px;
299 |     color: var(--text-secondary);
300 |     cursor: pointer;
301 | }
302 |
303 | .toggle-label input {
304 |     cursor: pointer;
305 | }
306 |
307 | /* Current KPI Grid */
308 | .current-kpi-grid {
309 |     display: grid;
310 |     grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
311 |     gap: 14px;
312 |     margin-bottom: 20px;
313 | }
314 |
315 | .current-kpi-card {
316 |     background: var(--bg-card);
317 |     border: 1px solid var(--border-color);
318 |     border-radius: var(--radius-md);
319 |     padding: 14px 16px;
320 | }
321 |
322 | .current-kpi-header {
323 |     display: flex;
324 |     justify-content: space-between;
325 |     align-items: center;
326 | }
327 |
328 | .current-kpi-icon {
329 |     font-size: 18px;
330 | }
331 |
```

```
332 | .mini-meter-bg {
333 |     width: 100%;
334 |     height: 4px;
335 |     background: rgba(255, 255, 255, 0.1);
336 |     border-radius: 2px;
337 |     margin-top: 8px;
338 |     overflow: hidden;
339 | }
340 |
341 | .mini-meter-fill {
342 |     height: 100%;
343 |     background: var(--accent-blue);
344 |     transition: width 0.3s ease;
345 | }
346 |
347 | /* Charts Grid */
348 | .charts-grid {
349 |     display: grid;
350 |     grid-template-columns: repeat(auto-fit, minmax(440px, 1fr));
351 |     gap: 20px;
352 | }
353 |
354 | .chart-card {
355 |     background: var(--bg-card);
356 |     border: 1px solid var(--border-color);
357 |     border-radius: var(--radius-lg);
358 |     padding: 18px 20px;
359 |     box-shadow: var(--shadow-sm);
360 | }
361 |
362 | .chart-header {
363 |     margin-bottom: 14px;
364 | }
365 |
366 | .chart-title {
367 |     font-size: 15px;
368 |     font-weight: 600;
369 |     color: var(--text-primary);
```

```
370 }
371
372 .chart-canvas-wrapper {
373     position: relative;
374     height: 280px;
375     width: 100%;
376 }
377
378 /* Split Grid */
379 .split-grid {
380     display: grid;
381     grid-template-columns: 1.2fr 1fr;
382     gap: 20px;
383     margin-top: 20px;
384 }
385
386 /* Tables */
387 .table-wrapper {
388     background: var(--bg-card);
389     border: 1px solid var(--border-color);
390     border-radius: var(--radius-lg);
391     overflow: hidden;
392     box-shadow: var(--shadow-sm);
393 }
394
395 .table-header-bar {
396     padding: 14px 20px;
397     border-bottom: 1px solid var(--border-color);
398     display: flex;
399     justify-content: space-between;
400     align-items: center;
401 }
402
403 table {
404     width: 100%;
405     border-collapse: collapse;
406     font-size: 13px;
407     text-align: left;
```

```
408 }
409
410 th, td {
411     padding: 10px 16px;
412     border-bottom: 1px solid var(--border-color);
413 }
414
415 th {
416     background: rgba(11, 15, 25, 0.4);
417     color: var(--text-secondary);
418     font-size: 11px;
419     text-transform: uppercase;
420     font-weight: 600;
421 }
422
423 tr:hover {
424     background: var(--bg-card-hover);
425 }
426
427 .pagination-bar {
428     display: flex;
429     justify-content: flex-end;
430     gap: 8px;
431     padding: 12px 20px;
432     border-top: 1px solid var(--border-color);
433 }
434
435 .search-input {
436     background: var(--bg-main);
437     border: 1px solid var(--border-color);
438     color: var(--text-primary);
439     padding: 6px 12px;
440     border-radius: var(--radius-md);
441     font-size: 13px;
442     outline: none;
443     min-width: 280px;
444 }
445
```



```
446 | .search-input:focus {
447 |   border-color: var(--accent-blue);
448 | }
449 |
450 | /* Hypotheses Grid */
451 | .hypotheses-grid {
452 |   display: grid;
453 |   grid-template-columns: repeat(auto-fit, minmax(380px, 1fr));
454 |   gap: 16px;
455 | }
456 |
457 | .hypothesis-card {
458 |   background: var(--bg-card);
459 |   border: 1px solid var(--border-color);
460 |   border-radius: var(--radius-lg);
461 |   padding: 18px 20px;
462 |   box-shadow: var(--shadow-sm);
463 | }
464 |
465 | .hyp-header {
466 |   display: flex;
467 |   justify-content: space-between;
468 |   align-items: center;
469 |   margin-bottom: 8px;
470 | }
471 |
472 | .hyp-status {
473 |   padding: 2px 8px;
474 |   border-radius: 4px;
475 |   font-size: 11px;
476 |   font-weight: 700;
477 |   text-transform: uppercase;
478 | }
479 |
480 | .status-confirmed {
481 |   background: rgba(16, 185, 129, 0.2);
482 |   color: var(--accent-green);
483 | }
```

```
484 |
485 | .status-rejected {
486 |     background: rgba(239, 68, 68, 0.2);
487 |     color: var(--accent-red);
488 | }
489 |
490 | /* Correlations */
491 | .correlation-list {
492 |     display: flex;
493 |     flex-direction: column;
494 |     gap: 10px;
495 |     margin-top: 10px;
496 | }
497 |
498 | .corr-item {
499 |     display: flex;
500 |     justify-content: space-between;
501 |     align-items: center;
502 |     padding: 10px 14px;
503 |     background: var(--bg-main);
504 |     border: 1px solid var(--border-color);
505 |     border-radius: var(--radius-md);
506 |     font-size: 13px;
507 | }
508 |
509 | .corr-coeff {
510 |     font-weight: 700;
511 |     font-size: 14px;
512 |     padding: 2px 8px;
513 |     border-radius: 4px;
514 | }
515 |
516 | .corr-strong {
517 |     color: var(--accent-amber);
518 |     background: rgba(245, 158, 11, 0.15);
519 | }
520 |
521 | .corr-med {
```

```
522 | color: var(--accent-blue);
523 | background: rgba(56, 189, 248, 0.15);
524 | }
525 |
526 | /* Activity Feed */
527 | .activity-feed-list {
528 |   list-style: none;
529 |   display: flex;
530 |   flex-direction: column;
531 |   gap: 8px;
532 |   max-height: 380px;
533 |   overflow-y: auto;
534 |   padding-right: 6px;
535 | }
536 |
537 | .activity-feed-item {
538 |   display: flex;
539 |   justify-content: space-between;
540 |   align-items: flex-start;
541 |   padding: 10px 12px;
542 |   background: rgba(11, 15, 25, 0.6);
543 |   border: 1px solid var(--border-color);
544 |   border-radius: var(--radius-md);
545 |   font-size: 12px;
546 | }
547 |
548 | .activity-feed-item:hover {
549 |   background: var(--bg-card-hover);
550 | }
551 |
552 | .activity-feed-left {
553 |   display: flex;
554 |   flex-direction: column;
555 |   gap: 2px;
556 | }
557 |
558 | .activity-tag {
559 |   display: inline-block;
```

```
560 | padding: 2px 6px;
561 | border-radius: 4px;
562 | font-size: 10px;
563 | font-weight: 600;
564 | text-transform: uppercase;
565 | }
566 |
567 | .activity-tag-device {
568 |   background: rgba(56, 189, 248, 0.15);
569 |   color: var(--accent-blue);
570 | }
571 |
572 | .activity-tag-system {
573 |   background: rgba(168, 85, 247, 0.15);
574 |   color: var(--accent-purple);
575 | }
576 |
577 | /* -----
   | -- */
578 | /* AI Response Presentation Card & Dynamic Visualization
   | */
579 | /* -----
   | -- */
580 | .ai-response-card {
581 |   background: linear-gradient(180deg, #162038 0%, #101728 100%
   | );
582 |   border: 1px solid #2a3d66;
583 |   border-radius: var(--radius-lg);
584 |   padding: 22px 26px;
585 |   box-shadow: var(--shadow-lg);
586 |   margin-top: 10px;
587 |   margin-bottom: 20px;
588 |   position: relative;
589 |   animation: fadeInDown 0.3s ease;
590 | }
591 |
592 | @keyframes fadeInDown {
593 |   from { opacity: 0; transform: translateY(-10px); }
```

```
594 |   to { opacity: 1; transform: translateY(0); }
595 | }
596 |
597 | .ai-response-header {
598 |   display: flex;
599 |   justify-content: space-between;
600 |   align-items: center;
601 |   border-bottom: 1px solid #233355;
602 |   padding-bottom: 14px;
603 |   margin-bottom: 16px;
604 | }
605 |
606 | .ai-response-title-group {
607 |   display: flex;
608 |   align-items: center;
609 |   gap: 12px;
610 | }
611 |
612 | .ai-badge {
613 |   background: rgba(56, 189, 248, 0.15);
614 |   color: var(--accent-blue);
615 |   border: 1px solid rgba(56, 189, 248, 0.3);
616 |   font-size: 11px;
617 |   font-weight: 700;
618 |   padding: 4px 10px;
619 |   border-radius: 20px;
620 |   text-transform: uppercase;
621 |   letter-spacing: 0.5px;
622 | }
623 |
624 | .ai-response-title {
625 |   font-size: 17px;
626 |   font-weight: 700;
627 |   color: var(--text-primary);
628 | }
629 |
630 | .btn-icon {
631 |   background: transparent;
```

```
632 | border: none;
633 | color: var(--text-secondary);
634 | font-size: 18px;
635 | cursor: pointer;
636 | padding: 4px 8px;
637 | border-radius: 6px;
638 | transition: all 0.2s;
639 | }
640 |
641 | .btn-icon:hover {
642 |   color: var(--accent-red);
643 |   background: rgba(239, 68, 68, 0.15);
644 | }
645 |
646 | .ai-response-content {
647 |   display: flex;
648 |   flex-direction: column;
649 |   gap: 16px;
650 | }
651 |
652 | .ai-chart-wrapper {
653 |   background: rgba(11, 15, 25, 0.7);
654 |   border: 1px solid var(--border-color);
655 |   border-radius: var(--radius-md);
656 |   padding: 16px 18px;
657 | }
658 |
659 | .ai-canvas-container {
660 |   position: relative;
661 |   height: 320px;
662 |   max-width: 800px;
663 |   margin: 0 auto;
664 | }
665 |
666 | .ai-metrics-row {
667 |   display: flex;
668 |   flex-wrap: wrap;
669 |   gap: 10px;
```

```
670 }
671
672 .ai-metric-pill {
673   background: rgba(11, 15, 25, 0.6);
674   border: 1px solid var(--border-color);
675   border-radius: var(--radius-md);
676   padding: 8px 14px;
677   font-size: 12px;
678   display: flex;
679   align-items: center;
680   gap: 6px;
681 }
682
683 .ai-metric-pill strong {
684   color: var(--accent-blue);
685   font-size: 14px;
686 }
687
688 .ai-reply-body {
689   font-size: 14px;
690   line-height: 1.6;
691   color: var(--text-primary);
692 }
693
694 .ai-reply-body h3 {
695   font-size: 16px;
696   margin-bottom: 8px;
697   color: var(--accent-blue);
698 }
699
700 .ai-reply-body h4 {
701   font-size: 14px;
702   margin: 12px 0 6px 0;
703   color: var(--text-primary);
704 }
705
706 .ai-reply-body p {
707   margin-bottom: 8px;
```

```
708 }
709
710 .ai-reply-body ul, .ai-reply-body ol {
711     padding-left: 20px;
712     margin-bottom: 10px;
713 }
714
715 .ai-reply-body li {
716     margin-bottom: 4px;
717 }
718
719 .ai-reply-body table {
720     margin: 10px 0;
721     border-radius: 6px;
722     overflow: hidden;
723 }
724
725 .ai-reply-body code {
726     background: rgba(0, 0, 0, 0.3);
727     padding: 2px 6px;
728     border-radius: 4px;
729     color: var(--accent-blue);
730     font-family: Consolas, Monaco, monospace;
731     font-size: 12px;
732 }
733
734 .ai-sql-details {
735     background: rgba(11, 15, 25, 0.8);
736     border: 1px solid var(--border-color);
737     border-radius: var(--radius-md);
738     padding: 10px 14px;
739     font-size: 12px;
740     cursor: pointer;
741 }
742
743 .ai-sql-details summary {
744     font-weight: 600;
745     color: var(--accent-amber);
```



```
746 | outline: none;
747 | }
748 |
749 | .ai-sql-code {
750 |     margin-top: 10px;
751 |     padding: 10px;
752 |     background: #070a12;
753 |     border-radius: 6px;
754 |     font-family: Consolas, Monaco, monospace;
755 |     font-size: 12px;
756 |     color: #38bdf8;
757 |     overflow-x: auto;
758 |     white-space: pre-wrap;
759 | }
760 |
761 | .ai-followup-row {
762 |     display: flex;
763 |     align-items: center;
764 |     flex-wrap: wrap;
765 |     gap: 8px;
766 |     margin-top: 6px;
767 | }
768 |
769 | .ai-chips-list {
770 |     display: flex;
771 |     flex-wrap: wrap;
772 |     gap: 8px;
773 | }
774 |
775 | /* -----
776 | -- */
777 | /* Bottom Fixed AI Query Bar
778 | -- */
779 | /* -----
780 | -- */
781 |
782 | .bottom-ai-bar {
783 |     position: fixed;
784 |     bottom: 0;
```

```
781 | left: 0;
782 | right: 0;
783 | background: rgba(19, 27, 46, 0.95);
784 | backdrop-filter: blur(12px);
785 | -webkit-backdrop-filter: blur(12px);
786 | border-top: 1px solid #2a3a5e;
787 | padding: 14px 28px;
788 | z-index: 999;
789 | box-shadow: 0 -8px 24px rgba(0, 0, 0, 0.5);
790 | display: flex;
791 | flex-direction: column;
792 | gap: 10px;
793 | }
794 |
795 | .ai-quick-chips {
796 |   display: flex;
797 |   gap: 8px;
798 |   overflow-x: auto;
799 |   padding-bottom: 2px;
800 | }
801 |
802 | .prompt-chip {
803 |   background: rgba(11, 15, 25, 0.7);
804 |   border: 1px solid var(--border-color);
805 |   color: var(--text-secondary);
806 |   font-size: 12px;
807 |   font-weight: 500;
808 |   padding: 6px 12px;
809 |   border-radius: 20px;
810 |   cursor: pointer;
811 |   white-space: nowrap;
812 |   transition: all 0.2s ease;
813 | }
814 |
815 | .prompt-chip:hover {
816 |   background: rgba(56, 189, 248, 0.15);
817 |   border-color: var(--accent-blue);
818 |   color: var(--text-primary);
```

```
819 |   transform: translateY(-1px);
820 | }
821 |
822 | .ai-query-form {
823 |   display: flex;
824 |   gap: 12px;
825 |   max-width: 1400px;
826 |   width: 100%;
827 |   margin: 0 auto;
828 | }
829 |
830 | .ai-input-wrapper {
831 |   position: relative;
832 |   flex: 1;
833 |   display: flex;
834 |   align-items: center;
835 | }
836 |
837 | .ai-input-icon {
838 |   position: absolute;
839 |   left: 14px;
840 |   font-size: 16px;
841 |   pointer-events: none;
842 | }
843 |
844 | .ai-query-input {
845 |   width: 100%;
846 |   background: #0b0f19;
847 |   border: 1px solid #2a3a5e;
848 |   color: var(--text-primary);
849 |   padding: 12px 42px 12px 42px;
850 |   border-radius: 28px;
851 |   font-size: 14px;
852 |   outline: none;
853 |   box-shadow: inset 0 2px 4px rgba(0, 0, 0, 0.4);
854 |   transition: all 0.2s ease;
855 | }
856 |
```

```
857 | .ai-query-input:focus {
858 |   border-color: var(--accent-blue);
859 |   box-shadow: 0 0 0 3px rgba(56, 189, 248, 0.25);
860 |   background: #0e1424;
861 | }
862 |
863 | .btn-clear {
864 |   position: absolute;
865 |   right: 14px;
866 |   background: transparent;
867 |   border: none;
868 |   color: var(--text-secondary);
869 |   font-size: 14px;
870 |   cursor: pointer;
871 | }
872 |
873 | .btn-clear:hover {
874 |   color: var(--text-primary);
875 | }
876 |
877 | .btn-ai-send {
878 |   border-radius: 28px;
879 |   padding: 10px 24px;
880 |   font-size: 14px;
881 |   font-weight: 700;
882 |   display: flex;
883 |   align-items: center;
884 |   justify-content: center;
885 |   min-width: 130px;
886 | }
887 |
888 | .spinner {
889 |   width: 16px;
890 |   height: 16px;
891 |   border: 2px solid rgba(11, 15, 25, 0.3);
892 |   border-top-color: #0b0f19;
893 |   border-radius: 50%;
894 |   animation: spin 0.8s linear infinite;
```

```
895 | }
896 |
897 | @keyframes spin {
898 |   to { transform: rotate(360deg); }
899 | }
900 |
901 | @media (max-width: 768px) {
902 |   .ai-quick-chips {
903 |     display: none;
904 |   }
905 |   .app-header {
906 |     flex-direction: column;
907 |     gap: 12px;
908 |     align-items: flex-start;
909 |   }
910 |   .charts-grid {
911 |     grid-template-columns: 1fr;
912 |   }
913 | }
914 |
915 | /* Gemini Evaluations Grid & Cards */
916 | .gemini-evals-grid {
917 |   display: grid;
918 |   grid-template-columns: repeat(auto-fill, minmax(340px, 1fr))
919 |   ;
920 |   gap: 16px;
921 |   margin-top: 14px;
922 | }
923 | .gemini-card {
924 |   background: var(--bg-surface);
925 |   border: 1px solid var(--border-color);
926 |   border-radius: var(--radius-md);
927 |   padding: 16px;
928 |   display: flex;
929 |   flex-direction: column;
930 |   gap: 10px;
931 |   position: relative;
```

```
932 | transition: transform 0.15s ease, border-color 0.15s ease;
933 | }
934 |
935 | .gemini-card:hover {
936 |   transform: translateY(-2px);
937 |   border-color: var(--accent-purple);
938 | }
939 |
940 | .gemini-card-header {
941 |   display: flex;
942 |   justify-content: space-between;
943 |   align-items: flex-start;
944 |   gap: 8px;
945 | }
946 |
947 | .gemini-card-title {
948 |   font-size: 15px;
949 |   font-weight: 700;
950 |   color: var(--text-primary);
951 | }
952 |
953 | .gemini-verdict-badge {
954 |   padding: 4px 10px;
955 |   border-radius: 12px;
956 |   font-size: 11px;
957 |   font-weight: 700;
958 |   text-transform: uppercase;
959 | }
960 |
961 | .gemini-verdict-normal {
962 |   background: rgba(16, 185, 129, 0.15);
963 |   color: var(--accent-green);
964 |   border: 1px solid rgba(16, 185, 129, 0.3);
965 | }
966 |
967 | .gemini-verdict-risk {
968 |   background: rgba(239, 68, 68, 0.15);
969 |   color: var(--accent-red);
```

```
970 | border: 1px solid rgba(239, 68, 68, 0.3);
971 | }
972 |
973 | .gemini-card-metrics {
974 |   display: flex;
975 |   gap: 10px;
976 |   flex-wrap: wrap;
977 |   font-size: 12px;
978 |   color: var(--text-secondary);
979 | }
980 |
981 | .gemini-metric-tag {
982 |   background: var(--bg-base);
983 |   padding: 2px 8px;
984 |   border-radius: 4px;
985 |   border: 1px solid var(--border-color);
986 | }
987 |
988 | .gemini-card-analysis {
989 |   font-size: 13px;
990 |   line-height: 1.45;
991 |   color: var(--text-primary);
992 |   background: rgba(15, 23, 42, 0.4);
993 |   padding: 10px;
994 |   border-radius: var(--radius-sm);
995 |   border-left: 3px solid var(--accent-blue);
996 | }
997 |
998 | .gemini-card-recs {
999 |   font-size: 12px;
1000 |   color: var(--text-secondary);
1001 |   padding-left: 16px;
1002 |   margin: 0;
1003 | }
1004 |
1005 |
```

__init__.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry Analytics
  | - Package Root
4 # =====
5 # Description:
6 #   Аналитическое ядро телеметрии: агрегаторы, компакторы, дет
  | ектор инцидентов,
7 #   диагностический движок и сгруппированная телеметрия.
8 #
9 # Usage Examples:
10 #   Python API:
11 #       from apps.windows.telemetry.analytics import DiagnosticE
  | ngine, GroupedTelemetryBuilder
12 #
13 # File: __init__.py
14 # Project: ai-breadboard
15 # Package: apps.windows.telemetry.analytics
16 # Author: hypo69
17 # Copyright: © 2026 hypo69
18 # Updated: 2026-10-08 09:36:00
19 # =====
20
21 from __future__ import annotations
22 """Аналитический контур телеметрии, сжатие метрик и построение
  | сгруппированных срезов."""
23
24 from apps.windows.telemetry.analytics.diagnostic_engine import
  | DiagnosticEngine
25 from apps.windows.telemetry.analytics.grouped_telemetry import
  | GroupedTelemetryBuilder
26 from apps.windows.telemetry.analytics.incident_detector import
  | IncidentDetector
```



```
27 | from apps.windows.telemetry.analytics.reboot_analyzer import R
    | ebootAnalyzer
28 | from apps.windows.telemetry.analytics.aggregator import Teleme
    | tryAggregator
29 | from apps.windows.telemetry.analytics.analyzer import Telemetr
    | yAnalyzer
30 | from apps.windows.telemetry.analytics.compactor import Telemet
    | ryCompactor
31 |
32 | __all__ = [
33 |     "DiagnosticEngine",
34 |     "GroupedTelemetryBuilder",
35 |     "IncidentDetector",
36 |     "RebootAnalyzer",
37 |     "TelemetryAggregator",
38 |     "TelemetryAnalyzer",
39 |     "TelemetryCompactor",
40 | ]
41 |
```

event_catalog.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - Event C
  atalog
4 # =====
5 # Description:
6 #   Иерархический каталог провайдеров и каналов Windows Event
  Log,
7 #   классификация по 7 доменам телеметрии (Security, Process,
  Storage, Network,
8 #   Update, Hardware, Power) и Sysmon, движки построения графо
  в происхождения
9 #   (Process Provenance) и реконструкции жизненного цикла (Pow
  er Lifecycle).
10 #
11 # Usage Examples:
12 #   Python API:
13 #       from apps.windows.telemetry.event_catalog import Windows
  EventCatalogEngine
14 #
15 #       engine = WindowsEventCatalogEngine()
16 #       catalog = engine.get_full_catalog()
17 #       domains = engine.get_domains_summary()
18 #
19 # File: event_catalog.py
20 # Project: ai-breadboard
21 # Package: apps.windows.telemetry
22 # Author: hypo69
23 # Copyright: © 2026 hypo69
24 # Updated: 2026-10-08 02:35:00
25 # =====
26
27 from __future__ import annotations
```

```

28 """Иерархический каталог провайдеров и каналов Windows Event L
   |og по 7 доменам телеметрии."""
29
30 import os
31 import re
32 import subprocess
33 import time
34 from dataclasses import asdict, dataclass, field
35 from datetime import datetime, timedelta, timezone
36 from enum import Enum
37 from typing import Any, Dict, List, Optional, Set, Tuple
38
39 try:
40     from logger import logger
41 except ImportError:
42     import logging
43     logger = logging.getLogger(__name__)
44
45 from apps.windows.telemetry.win32_fffi.wevtapi import WevtAPI
46 from apps.windows.telemetry.sqlite import TelemetryStorage
47
48
49 class TelemetryDomain(str, Enum):
50     """7 доменов классификации Windows Event Logs."""
51     SECURITY_IDENTITY = 'security_identity'          # 1.  □  Secu
   |rity / Identity
52     PROCESS_SERVICES_TASKS = 'process_tasks'        # 2.  □  Proce
   |ss / Services / Tasks
53     STORAGE_FILESYSTEM = 'storage_io'               # 3.  □  Stor
   |age & Filesystem
54     NETWORK_COMMUNICATIONS = 'network_comms'        # 4.  □  Netw
   |ork & Sockets
55     WINDOWS_UPDATE = 'windows_update'               # 5.  □  Wind
   |ows Update
56     HARDWARE_PNP_DRIVERS = 'hardware_pnp'           # 6.  □  Hard
   |ware / Drivers / PnP
57     POWER_BOOT_SHUTDOWN = 'power_lifecycle'         # 7.  □  Powe
   |r / Boot / Shutdown

```

```

58     SYSMON_OBSERVABILITY = 'sysmon_deep'           # [] Sysmon
    Observability Layer
59
60
61 @dataclass
62 class EventProviderCatalogEntry:
63     """Запись каталога провайдеров и каналов Windows Event Log
    ."""
64     id: str
65     provider: str
66     channel: str
67     domain: TelemetryDomain
68     domain_title: str
69     key_events: Dict[int, str]
70     levels: List[str]
71     realtime_supported: bool
72     historical_supported: bool
73     privilege_required: str
74     volume_rating: str # Low, Medium, High, Extreme
75     useful_fields: List[str]
76     recommended_collector: str
77     storage_class: str
78     correlation_targets: List[str]
79     description: str = ''
80     is_present_on_host: bool = True
81     record_count: int = 0
82     size_mb: float = 0.0
83
84
85 # Полный реестр провайдеров по 7 доменам телеметрии
86 EVENT_PROVIDERS_REGISTRY: List[Dict[str, Any]] = [
87     # -----
    -----
88     # 1. [] Security / Identity
89     # -----
    -----
90     {
91         'id': 'sec_auditing',

```

```

92 |         'provider': 'Microsoft-Windows-Security-Auditing',
93 |         'channel': 'Security',
94 |         'domain': TelemetryDomain.SECURITY_IDENTITY,
95 |         'domain_title': '1. Security / Identity',
96 |         'key_events': {
97 |             4624: 'Успешный вход (Logon)',
98 |             4625: 'Неудачная попытка входа (Failed Logon)',
99 |             4634: 'Завершение сеанса пользователя (Logoff)',
100 |            4647: 'Пользователь инициировал выход',
101 |            4672: 'Назначение специальных привилегий (Admin/UA
      | C)',
102 |            4720: 'Создание пользователя',
103 |            4726: 'Удаление пользователя',
104 |            4732: 'Добавление пользователя в локальную группу'
      | ,
105 |            4738: 'Изменение атрибутов учетной записи',
106 |            4740: 'Блокировка учетной записи (Account Lockout)
      | ',
107 |            1102: 'Очистка журнала аудита безопасности',
108 |        },
109 |        'levels': ['Information', 'Audit Success', 'Audit Fail
      | ure'],
110 |        'realtime_supported': True,
111 |        'historical_supported': True,
112 |        'privilege_required': 'SeSecurityPrivilege (Admin)',
113 |        'volume_rating': 'High',
114 |        'useful_fields': ['SubjectUserName', 'TargetUserName',
      | 'LogonType', 'IpAddress', 'TargetSid'],
115 |        'recommended_collector': 'WevtAPI Incremental Bookmark
      | ',
116 |        'storage_class': 'security_events',
117 |        'correlation_targets': ['processes', 'user_sessions',
      | 'network_sockets'],
118 |        'description': 'Основной журнал аудита безопасности Wi
      | ndows: авторизация, аккаунты, группы и права.',
119 |    },
120 |    {
121 |        'id': 'sec_kerberos',

```

```
122         'provider': 'Microsoft-Windows-Kerberos-Key-Distributi
on-Center',
123         'channel': 'Security',
124         'domain': TelemetryDomain.SECURITY_IDENTITY,
125         'domain_title': '1. Security / Identity',
126         'key_events': {
127             4768: 'Выдача Kerberos TGT билета',
128             4769: 'Выдача Kerberos Service Ticket (TGS)',
129             4771: 'Сбой преаутентификации Kerberos (Неверный п
ароль)',
130             4776: 'NTLM валидация учетных данных',
131         },
132         'levels': ['Information', 'Audit Failure'],
133         'realtime_supported': True,
134         'historical_supported': True,
135         'privilege_required': 'Admin',
136         'volume_rating': 'Medium',
137         'useful_fields': ['TargetUserName', 'ServiceName', 'Ti
cketOptions', 'FailureCode', 'IpAddress'],
138         'recommended_collector': 'WevtAPI Incremental',
139         'storage_class': 'security_events',
140         'correlation_targets': ['domain_auth', 'failed_logons'
, 'lateral_movement'],
141         'description': 'Аудит протоколов Kerberos и NTLM аутен
тификации в корпоративной сети и локальном хосте.',
142     },
143     {
144         'id': 'sec_grouppolicy',
145         'provider': 'Microsoft-Windows-GroupPolicy',
146         'channel': 'Microsoft-Windows-GroupPolicy/Operational'
,
147         'domain': TelemetryDomain.SECURITY_IDENTITY,
148         'domain_title': '1. Security / Identity',
149         'key_events': {
150             5312: 'Список примененных политик Group Policy Obj
ects (GPO)',
151             5314: 'Начало обработки групповой политики',
152             7004: 'Ошибка применения политики пользователя/ком
```

```

пьютера',
153     },
154     'levels': ['Information', 'Warning', 'Error'],
155     'realtime_supported': True,
156     'historical_supported': True,
157     'privilege_required': 'Admin',
158     'volume_rating': 'Low',
159     'useful_fields': ['GPOList', 'PrincipalSamName', 'ElapsedTimeInMilliseconds', 'ErrorCode'],
160     'recommended_collector': 'WevtAPI Query',
161     'storage_class': 'security_events',
162     'correlation_targets': ['policy_drift', 'system_baseline'],
163     'description': 'Телеметрия применения групповых политик Windows и изменений конфигурации безопасности.',
164 },
165
166 # -----
167 # 2. □ Process / Services / Tasks
168 # -----
169 {
170     'id': 'proc_creation_4688',
171     'provider': 'Microsoft-Windows-Security-Auditing',
172     'channel': 'Security',
173     'domain': TelemetryDomain.PROCESS_SERVICES_TASKS,
174     'domain_title': '2. Process / Services / Tasks',
175     'key_events': {
176         4688: 'Создание нового процесса (с CommandLine и Parent PID)',
177         4689: 'Завершение работы процесса (Process Exit)',
178     },
179     'levels': ['Information', 'Audit Success'],
180     'realtime_supported': True,
181     'historical_supported': True,
182     'privilege_required': 'Admin',
183     'volume_rating': 'Extreme',

```

```
184         'useful_fields': ['NewProcessName', 'CommandLine', 'Pr
    ocessId', 'CreatorProcessId', 'ParentProcessName', 'SubjectUse
    rName'],
185         'recommended_collector': 'WevtAPI Incremental Tail',
186         'storage_class': 'security_events',
187         'correlation_targets': ['process_provenance_tree', 'cp
    u_spikes', 'file_access'],
188         'description': 'Глубокий аудит жизненного цикла процес
    сов: полный путь, аргументы запуска, родительский PID.',
189     },
190     {
191         'id': 'proc_scm_services',
192         'provider': 'Service Control Manager',
193         'channel': 'System',
194         'domain': TelemetryDomain.PROCESS_SERVICES_TASKS,
195         'domain_title': '2. Process / Services / Tasks',
196         'key_events': {
197             7036: 'Изменение состояния службы (Запущена / Оста
    новлена)',
198             7045: 'Установка новой системной службы (Service I
    nstall)',
199             7031: 'Аварийное завершение службы (Service Crash)
    ',
200             7034: 'Неожиданная остановка службы',
201         },
202         'levels': ['Information', 'Warning', 'Error'],
203         'realtime_supported': True,
204         'historical_supported': True,
205         'privilege_required': 'User / Admin',
206         'volume_rating': 'Medium',
207         'useful_fields': ['ServiceName', 'ImagePath', 'Service
    Type', 'StartType', 'AccountName'],
208         'recommended_collector': 'WevtAPI Incremental',
209         'storage_class': 'security_events',
210         'correlation_targets': ['process_provenance', 'service
    _matrix', 'boot_time'],
211         'description': 'Диспетчер служб Windows (SCM): запуск,
    остановка, установка новых сервисов и сбои.',
```



```

212     },
213     {
214         'id': 'proc_taskscheduler',
215         'provider': 'Microsoft-Windows-TaskScheduler',
216         'channel': 'Microsoft-Windows-TaskScheduler/Operationa
217     l',
218         'domain': TelemetryDomain.PROCESS_SERVICES_TASKS,
219         'domain_title': '2. Process / Services / Tasks',
220         'key_events': {
221             106: 'Регистрация новой задачи в планировщике',
222             107: 'Запуск задачи по расписанию или триггеру',
223             102: 'Успешное завершение задачи',
224             103: 'Сбой или ошибка выполнения задачи',
225             140: 'Изменение конфигурации задачи',
226             141: 'Удаление задачи из планировщика',
227         },
228         'levels': ['Information', 'Warning', 'Error'],
229         'realtime_supported': True,
230         'historical_supported': True,
231         'privilege_required': 'User / Admin',
232         'volume_rating': 'High',
233         'useful_fields': ['TaskName', 'ActionName', 'ResultCod
234     e', 'UserName', 'TaskInstanceId'],
235         'recommended_collector': 'WevtAPI Incremental',
236         'storage_class': 'security_events',
237         'correlation_targets': ['process_provenance', 'persist
238     ence_audit', 'scheduled_jobs'],
239         'description': 'Планировщик заданий: создание, триггер
240     ы, запуск задач и фоновая персистентность.',
241     },
242     {
243         'id': 'proc_wmi_activity',
244         'provider': 'Microsoft-Windows-WMI-Activity',
245         'channel': 'Microsoft-Windows-WMI-Activity/Operational
246     ',
247         'domain': TelemetryDomain.PROCESS_SERVICES_TASKS,
248         'domain_title': '2. Process / Services / Tasks',
249         'key_events': {

```

```
245         5857: 'Инициализация WMI провайдера',
246         5858: 'Ошибка выполнения WMI запроса (Query Failur
e)',
247         5861: 'Регистрация постоянного WMI Event Consumer
(Персистентность)',
248     },
249     'levels': ['Information', 'Warning', 'Error'],
250     'realtime_supported': True,
251     'historical_supported': True,
252     'privilege_required': 'Admin',
253     'volume_rating': 'Medium',
254     'useful_fields': ['Query', 'Namespace', 'ClientProcess
Id', 'User', 'ResultCode'],
255     'recommended_collector': 'WevtAPI Query',
256     'storage_class': 'security_events',
257     'correlation_targets': ['process_provenance', 'wmi_per
sistence'],
258     'description': 'Инфраструктура WMI: запросы процессов,
сбои провайдеров и механизмы постоянной подписки.',
259 },
260 {
261     'id': 'proc_powershell',
262     'provider': 'Microsoft-Windows-PowerShell',
263     'channel': 'Microsoft-Windows-PowerShell/Operational',
264     'domain': TelemetryDomain.PROCESS_SERVICES_TASKS,
265     'domain_title': '2. Process / Services / Tasks',
266     'key_events': {
267         4103: 'Выполнение командного блока (Pipeline Execu
tion)',
268         4104: 'Логирование содержимого скрипта (Script Blo
ck Logging)',
269         40961: 'Запуск консоли PowerShell хоста',
270     },
271     'levels': ['Information', 'Warning'],
272     'realtime_supported': True,
273     'historical_supported': True,
274     'privilege_required': 'User / Admin',
275     'volume_rating': 'High',
```

```

276 |         'useful_fields': ['ScriptBlockText', 'Path', 'UserId',
277 |         'ProcessId', 'HostName'],
278 |         'recommended_collector': 'WevtAPI Incremental',
279 |         'storage_class': 'security_events',
280 |         'correlation_targets': ['script_provenance', 'admin_audit'],
281 |         'description': 'Среда PowerShell: выполнение сценариев
282 |         автоматизации, командлетов и расширенный ScriptBlock аудит.',
283 |         },
284 |         # -----
285 |         # 3. Storage & Filesystem
286 |         # -----
287 |         {
288 |             'id': 'stor_ntfs',
289 |             'provider': 'Microsoft-Windows-Ntfs',
290 |             'channel': 'Microsoft-Windows-Ntfs/Operational',
291 |             'domain': TelemetryDomain.STORAGE_FILESYSTEM,
292 |             'domain_title': '3. Storage & Filesystem',
293 |             'key_events': {
294 |                 140: 'Сброс буфера файловой системы на диск',
295 |                 141: 'Внимание: ошибка метаданных тома NTFS',
296 |                 142: 'Активация подсистемы самовосстановления NTFS
297 |                 (Self-Healing)',
298 |             },
299 |             'levels': ['Information', 'Warning', 'Error'],
300 |             'realtime_supported': True,
301 |             'historical_supported': True,
302 |             'privilege_required': 'Admin',
303 |             'volume_rating': 'Medium',
304 |             'useful_fields': ['VolumeId', 'VolumeName', 'NtfsStatus', 'CorruptOffset'],
305 |             'recommended_collector': 'WevtAPI Incremental',
306 |             'storage_class': 'system_snapshots',
307 |             'correlation_targets': ['disk_io_spikes', 'file_corruption', 'smart_metrics'],

```

```
306         'description': 'Файловая система NTFS: целостность том  
ов, ошибки записи метаданных и самовосстановление.',  
307     },  
308     {  
309         'id': 'stor_disk_diag',  
310         'provider': 'Microsoft-Windows-DiskDiagnostic',  
311         'channel': 'Microsoft-Windows-DiskDiagnostic/Operation  
al',  
312         'domain': TelemetryDomain.STORAGE_FILESYSTEM,  
313         'domain_title': '3. Storage & Filesystem',  
314         'key_events': {  
315             1: 'Диагностика накопителя: предупреждение о скоро  
м выходе из строя (SMART Alert)',  
316             2: 'Результаты автоматической самодиагностики диск  
а',  
317         },  
318         'levels': ['Warning', 'Error', 'Critical'],  
319         'realtime_supported': True,  
320         'historical_supported': True,  
321         'privilege_required': 'Admin',  
322         'volume_rating': 'Low',  
323         'useful_fields': ['DeviceName', 'Model', 'SerialNumber  
, 'SmartAttribute', 'RawValue'],  
324         'recommended_collector': 'WevtAPI Query',  
325         'storage_class': 'system_snapshots',  
326         'correlation_targets': ['smart_sensors', 'hardware_hea  
lth'],  
327         'description': 'Диагностика накопителей: ранние аппара  
тные предупреждения S.M.A.R.T. о деградации диска.',  
328     },  
329     {  
330         'id': 'stor_spaces',  
331         'provider': 'Microsoft-Windows-StorageSpaces-Driver',  
332         'channel': 'Microsoft-Windows-StorageSpaces-Driver/Ope  
rational',  
333         'domain': TelemetryDomain.STORAGE_FILESYSTEM,  
334         'domain_title': '3. Storage & Filesystem',  
335         'key_events': {
```

```

336 |             100: 'Инициализация пула Storage Spaces',
337 |             200: 'Отказ физического диска в дисковом пространс
| тве',
338 |             201: 'Автоматическое восстановление зеркалирования
| (Repair/Resync)',
339 |         },
340 |         'levels': ['Information', 'Warning', 'Error', 'Critica
| l'],
341 |         'realtime_supported': True,
342 |         'historical_supported': True,
343 |         'privilege_required': 'Admin',
344 |         'volume_rating': 'Low',
345 |         'useful_fields': ['PoolId', 'SpaceId', 'PhysicalDiskId
| ', 'ErrorStatus'],
346 |         'recommended_collector': 'WevtAPI Query',
347 |         'storage_class': 'system_snapshots',
348 |         'correlation_targets': ['disk_arrays', 'raid_health'],
349 |         'description': 'Драйвер дисковых пространств Storage S
| paces: пулы дисков, сбои зеркал и ресинхронизация.',
350 |     },
351 |
352 |     # -----
| -----
353 |     # 4. □ Network & Sockets
354 |     # -----
| -----
355 |     {
356 |         'id': 'net_dns_client',
357 |         'provider': 'Microsoft-Windows-DNS-Client',
358 |         'channel': 'Microsoft-Windows-DNS-Client/Operational',
359 |         'domain': TelemetryDomain.NETWORK_COMMUNICATIONS,
360 |         'domain_title': '4. Network & Sockets',
361 |         'key_events': {
362 |             3008: 'Разрешение DNS запроса (Query Completed)',
363 |             3018: 'Таймаут обращения к DNS серверу',
364 |             3020: 'Сбой разрешения доменного имени (NXDOMAIN /
| Server Failure)',
365 |         },

```

```
366         'levels': ['Information', 'Warning', 'Error'],
367         'realtime_supported': True,
368         'historical_supported': True,
369         'privilege_required': 'User / Admin',
370         'volume_rating': 'High',
371         'useful_fields': ['QueryName', 'QueryType', 'QueryStat
us', 'ServerList'],
372         'recommended_collector': 'WevtAPI Incremental',
373         'storage_class': 'network_metrics',
374         'correlation_targets': ['process_sockets', 'latency_sp
ikes', 'web_activity'],
375         'description': 'Клиент DNS Windows: история запросов,
задержки DNS-серверов и сбои резолвинга.',
376     },
377     {
378         'id': 'net_firewall',
379         'provider': 'Microsoft-Windows-Windows Firewall With A
dvanced Security',
380         'channel': 'Microsoft-Windows-Windows Firewall With Ad
vanced Security/Firewall',
381         'domain': TelemetryDomain.NETWORK_COMMUNICATIONS,
382         'domain_title': '4. Network & Sockets',
383         'key_events': {
384             2004: 'Брандмауэр разрешил входящее/исходящее соедине
ние',
385             2005: 'Брандмауэр заблокировал пакет (Packet Dropp
ed)',
386             2006: 'Изменение правила брандмауэра',
387         },
388         'levels': ['Information', 'Warning'],
389         'realtime_supported': True,
390         'historical_supported': True,
391         'privilege_required': 'Admin',
392         'volume_rating': 'High',
393         'useful_fields': ['RuleId', 'ApplicationPath', 'LocalP
ort', 'RemotePort', 'Protocol', 'Direction'],
394         'recommended_collector': 'WevtAPI Incremental',
395         'storage_class': 'network_metrics',
```

```

396 |         'correlation_targets': ['process_provenance', 'blocked
    | _ports', 'security_events'],
397 |         'description': 'Сетевой экран Windows Defender Firewall:
    | блокировки пакетов, порты и изменение правил.',
398 |     },
399 |     {
400 |         'id': 'net_smb_client',
401 |         'provider': 'Microsoft-Windows-SMBClient',
402 |         'channel': 'Microsoft-Windows-SMBClient/Security',
403 |         'domain': TelemetryDomain.NETWORK_COMMUNICATIONS,
404 |         'domain_title': '4. Network & Sockets',
405 |         'key_events': {
406 |             31001: 'Подключение к сетевой папке SMB',
407 |             31017: 'Сбой авторизации на SMB сервере',
408 |             31020: 'Сброс сессии сетевого ресурса из-за сетево
    | го таймаута',
409 |         },
410 |         'levels': ['Information', 'Warning', 'Error'],
411 |         'realtime_supported': True,
412 |         'historical_supported': True,
413 |         'privilege_required': 'Admin',
414 |         'volume_rating': 'Low',
415 |         'useful_fields': ['ServerName', 'ShareName', 'UserName
    | ', 'Status'],
416 |         'recommended_collector': 'WevtAPI Query',
417 |         'storage_class': 'network_metrics',
418 |         'correlation_targets': ['network_shares', 'storage_io'
    | ],
419 |         'description': 'Сетевой клиент SMB: обращения к общим
    | сетевым папкам и сбои передачи файлов.',
420 |     },
421 |
422 |     # -----
    | -----
423 |     # 5. □ Windows Update
424 |     # -----
    | -----
425 |     {

```

```
426         'id': 'upd_client',
427         'provider': 'Microsoft-Windows-WindowsUpdateClient',
428         'channel': 'Microsoft-Windows-WindowsUpdateClient/Oper
ational',
429         'domain': TelemetryDomain.WINDOWS_UPDATE,
430         'domain_title': '5. Windows Update',
431         'key_events': {
432             19: 'Успешная установка обновления Windows Update
(KB)',
433             20: 'Ошибка при установке накопительного обновлени
я',
434             21: 'Установка пакета завершена, требуется перезап
уск ОС',
435             24: 'Начало скачивания компонентов обновления',
436             25: 'Начало установки пакета обновления',
437         },
438         'levels': ['Information', 'Warning', 'Error'],
439         'realtime_supported': True,
440         'historical_supported': True,
441         'privilege_required': 'User / Admin',
442         'volume_rating': 'Low',
443         'useful_fields': ['UpdateTitle', 'KBNumber', 'ErrorCod
e', 'SupportUrl', 'ClientApplicationID'],
444         'recommended_collector': 'WevtAPI Incremental',
445         'storage_class': 'system_snapshots',
446         'correlation_targets': ['reboots', 'servicing_integrit
y', 'disk_usage'],
447         'description': 'Клиент обновлений Windows Update: исто
рия загрузки, установки KB-пакетов и связанные перезагрузки.',
448     },
449     {
450         'id': 'upd_orchestrator',
451         'provider': 'Microsoft-Windows-UpdateOrchestrator',
452         'channel': 'Microsoft-Windows-UpdateOrchestrator/Opera
tional',
453         'domain': TelemetryDomain.WINDOWS_UPDATE,
454         'domain_title': '5. Windows Update',
455         'key_events': {
```



```

456 |         101: 'Update Orchestrator запустил проверку обновл
      | ений',
457 |         102: 'Обнаружены доступные обновления',
458 |         201: 'Запланирована принудительная перезагрузка си
      | стемы для завершения обновления',
459 |     },
460 |     'levels': ['Information', 'Warning'],
461 |     'realtime_supported': True,
462 |     'historical_supported': True,
463 |     'privilege_required': 'Admin',
464 |     'volume_rating': 'Low',
465 |     'useful_fields': ['ActionName', 'RebootDeadline', 'Sch
      | eduledTime'],
466 |     'recommended_collector': 'WevtAPI Query',
467 |     'storage_class': 'system_snapshots',
468 |     'correlation_targets': ['planned_reboots', 'update_cyc
      | le'],
469 |     'description': 'Диспетчер Update Orchestrator: планиро
      | вание обслуживания и дедлайны перезагрузок.',
470 | },
471 |
472 | # -----
      | -----
473 | # 6. □ Hardware / Drivers / Plug-and-Play
474 | # -----
      | -----
475 | {
476 |     'id': 'hw_kernel_pnp',
477 |     'provider': 'Microsoft-Windows-Kernel-PnP',
478 |     'channel': 'Microsoft-Windows-Kernel-PnP/Configuration
      | ',
479 |     'domain': TelemetryDomain.HARDWARE_PNP_DRIVERS,
480 |     'domain_title': '6. Hardware / Drivers / PnP',
481 |     'key_events': {
482 |         400: 'Устройство настроено (Device Configured)',
483 |         410: 'Устройство запущено (Device Started)',
484 |         411: 'Сбой запуска устройства (Device Start Failed
      | , Code 10 / 43)',

```

```
485 |         420: 'Удаление устройства (Device Removed / Unplug
    | ged)',
486 |         430: 'Требуется перезагрузка после установки драйв
    | ера',
487 |     },
488 |     'levels': ['Information', 'Warning', 'Error', 'Critica
    | l'],
489 |     'realtime_supported': True,
490 |     'historical_supported': True,
491 |     'privilege_required': 'Admin',
492 |     'volume_rating': 'Medium',
493 |     'useful_fields': ['DeviceInstanceId', 'DriverName', 'D
    | riverVersion', 'ProblemStatus', 'HardwareIds'],
494 |     'recommended_collector': 'WextAPI Incremental',
495 |     'storage_class': 'system_snapshots',
496 |     'correlation_targets': ['hardware_monitor', 'usb_perip
    | herals', 'pnp_errors'],
497 |     'description': 'Диспетчер оборудования Plug and Play:
    | подключение USB/PCIe устройств, загрузка драйверов и ошибки Co
    | de 10/43.',
498 | },
499 | {
500 |     'id': 'hw_device_setup',
501 |     'provider': 'Microsoft-Windows-DeviceSetupManager',
502 |     'channel': 'Microsoft-Windows-DeviceSetupManager/Admin
    | ',
503 |     'domain': TelemetryDomain.HARDWARE_PNP_DRIVERS,
504 |     'domain_title': '6. Hardware / Drivers / PnP',
505 |     'key_events': {
506 |         112: 'Успешная загрузка драйвера из Windows Update
    | Driver Store',
507 |         131: 'Ошибка сопоставления метаданных устройства',
508 |     },
509 |     'levels': ['Information', 'Warning', 'Error'],
510 |     'realtime_supported': True,
511 |     'historical_supported': True,
512 |     'privilege_required': 'Admin',
513 |     'volume_rating': 'Low',
```

```

514         'useful_fields': ['DeviceInstanceId', 'DriverPackage',
515         'Status'],
516         'recommended_collector': 'WevtAPI Query',
517         'storage_class': 'system_snapshots',
518         'correlation_targets': ['driver_store', 'peripherals']
519     },
520
521     # -----
522     # 7.  Power / Boot / Shutdown Lifecycle
523     # -----
524     {
525         'id': 'pwr_kernel_power',
526         'provider': 'Microsoft-Windows-Kernel-Power',
527         'channel': 'Microsoft-Windows-Kernel-Power/Operational
528     ',
529         'domain': TelemetryDomain.POWER_BOOT_SHUTDOWN,
530         'domain_title': '7. Power / Boot / Shutdown',
531         'key_events': {
532             41: 'КРИТИЧЕСКИЙ СБОЙ: Перезагрузка без предварите
533             льного чистого выключения (BSOD / Power Cut)',
534             42: 'Переход системы в спящий режим (Sleep / Moder
535             n Standby)',
536             107: 'Выход системы из спящего режима (Resume from
537             Sleep)',
538             109: 'Переход ядра в режим гибернации (Fast Startu
539             p / Hibernate)',
540             172: 'Изменение состояния подключения к электросет
541             и (AC / Battery)',
542         },
543         'levels': ['Information', 'Warning', 'Critical'],
544         'realtime_supported': True,
545         'historical_supported': True,
546         'privilege_required': 'User / Admin',

```

```
541         'volume_rating': 'Low',
542         'useful_fields': ['BugcheckCode', 'BugcheckParameter1'
543         , 'SleepTime', 'WakeTime', 'TargetState'],
544         'recommended_collector': 'WevtAPI Incremental',
545         'storage_class': 'system_snapshots',
546         'correlation_targets': ['reboot_analyzer', 'bsod_crash
547         es', 'uptime_timeline'],
548         'description': 'Управление питанием ядра: переходы в с
549         он/гибернацию, сбои питания (Event 41) и тайминги ACPI.',
550     },
551     {
552         'id': 'pwr_user32_shutdown',
553         'provider': 'User32',
554         'channel': 'System',
555         'domain': TelemetryDomain.POWER_BOOT_SHUTDOWN,
556         'domain_title': '7. Power / Boot / Shutdown',
557         'key_events': {
558             1074: 'Плановое выключение / перезагрузка иницииро
559             вана пользователем или процессом',
560         },
561         'levels': ['Information'],
562         'realtime_supported': True,
563         'historical_supported': True,
564         'privilege_required': 'User / Admin',
565         'volume_rating': 'Low',
566         'useful_fields': ['InitiatingProcess', 'CallingUser',
567         'ShutdownReason', 'ReasonCode'],
568         'recommended_collector': 'WevtAPI Incremental',
569         'storage_class': 'system_snapshots',
570         'correlation_targets': ['user_actions', 'reboot_proven
571         ance'],
572         'description': 'Инициация планового выключения/перезаг
573         рузки: кто, из какого процесса и по какой причине.',
574     },
575     {
576         'id': 'pwr_eventlog_lifecycle',
577         'provider': 'EventLog',
578         'channel': 'System',
```

```

572         'domain': TelemetryDomain.POWER_BOOT_SHUTDOWN,
573         'domain_title': '7. Power / Boot / Shutdown',
574         'key_events': {
575             6005: 'Служба журнала событий запущена (Система ус
пешно загрузилась)',
576             6006: 'Служба журнала событий остановлена (Штатное
выключение ОС)',
577             6008: 'Предыдущее выключение системы было неожидан
ным (Unexpected Shutdown)',
578         },
579         'levels': ['Information', 'Error'],
580         'realtime_supported': True,
581         'historical_supported': True,
582         'privilege_required': 'User / Admin',
583         'volume_rating': 'Low',
584         'useful_fields': ['TimeOfLastKnownGoodState', 'EventLo
gStatus'],
585         'recommended_collector': 'WevtAPI Incremental',
586         'storage_class': 'system_snapshots',
587         'correlation_targets': ['uptime_graph', 'crash_frequen
cy'],
588         'description': 'Метки времени загрузки и завершения ра
боты операционной системы для расчета надежности.',
589     },
590
591     # -----
592     # 8. Sysmon Observability Layer
593     # -----
594     {
595         'id': 'sysmon_deep_core',
596         'provider': 'Microsoft-Windows-Sysmon',
597         'channel': 'Microsoft-Windows-Sysmon/Operational',
598         'domain': TelemetryDomain.SYSMON_OBSERVABILITY,
599         'domain_title': 'Sysmon Observability Layer',
600         'key_events': {
601             1: 'Создание процесса с хэшами (SHA256, MD5, Paren

```

```

        tCommandLine)',
602         3: 'Сетевое соединение процесса (Source/Dest IP, P
        ort, Process)',
603         6: 'Загрузка драйвера ядра (Driver Loaded, Hashes)
        ',
604         7: 'Загрузка DLL модуля в процесс (Image Loaded)',
605         10: 'Доступ одного процесса к памяти другого (Proc
        ess Access / Injection)',
606         11: 'Создание файла на диске (File Created)',
607         12: 'Создание/удаление ключа реестра',
608         13: 'Изменение значения реестра (Registry Value Se
        t)',
609         22: 'DNS запрос процесса (DNS Query with Process I
        D)',
610     },
611     'levels': ['Information'],
612     'realtime_supported': True,
613     'historical_supported': True,
614     'privilege_required': 'Admin / System',
615     'volume_rating': 'Extreme',
616     'useful_fields': ['UtcTime', 'ProcessGuid', 'ProcessId
        ', 'Image', 'CommandLine', 'Hashes', 'ParentProcessId', 'Desti
        nationIp', 'DestinationPort'],
617     'recommended_collector': 'Sysmon Collector / WevtAPI',
618     'storage_class': 'security_events',
619     'correlation_targets': ['process_provenance_deep', 'ne
        twork_sockets', 'driver_hashes'],
620     'description': 'Microsoft System Monitor: непрерывная
        телеметрия процессов, сетевых сокетов, файловых операций и хэш
        ей драйверов.',
621     },
622 ]
623
624
625 class WindowsEventCatalogEngine:
626     """Движок иерархического каталога каналов Windows Event Lo
        g и корреляционных графов."""
627

```

```

628     def __init__(self) -> None:
629         """Инициализация движка каталога."""
630         self._wevtapi = WevtAPI()
631         self._storage = TelemetryStorage()
632         self._catalog_cache: Optional[List[EventProviderCatalogEntry]] = None
633         self._cache_timestamp: float = 0.0
634
635     def get_full_catalog(self, force_refresh: bool = False) ->
        List[EventProviderCatalogEntry]:
636         """Возвращает полный структурированный каталог провайдеров и каналов."""
637         now = time.time()
638         if not force_refresh and self._catalog_cache and (now
        - self._cache_timestamp < 30.0):
639             return self._catalog_cache
640
641         entries: List[EventProviderCatalogEntry] = []
642         for reg in EVENT_PROVIDERS_REGISTRY:
643             cname = reg['channel']
644             rec_cnt = self._wevtapi.get_channel_record_count(c
        name)
645
646             # Проверка доступности
647             is_present = True
648             size_mb = 0.0
649             try:
650                 # Оценка размера через wevtutil gli (быстро)
651                 info = self._wevtapi.check_channel_access(cnam
        e)
652                 is_present = info.get('accessible', True)
653             except Exception:
654                 is_present = False
655
656             entry = EventProviderCatalogEntry(
657                 id=reg['id'],
658                 provider=reg['provider'],
659                 channel=reg['channel'],

```

```

660         domain=reg['domain'],
661         domain_title=reg['domain_title'],
662         key_events=reg['key_events'],
663         levels=reg['levels'],
664         realtime_supported=reg['realtime_supported'],
665         historical_supported=reg['historical_supported
        ],
666         privilege_required=reg['privilege_required'],
667         volume_rating=reg['volume_rating'],
668         useful_fields=reg['useful_fields'],
669         recommended_collector=reg['recommended_collect
or'],
670         storage_class=reg['storage_class'],
671         correlation_targets=reg['correlation_targets']
        ,
672         description=reg.get('description', ''),
673         is_present_on_host=is_present,
674         record_count=rec_cnt,
675         size_mb=round(rec_cnt * 0.0006, 2), # примерн
ая оценка в МБ если нет прямого размера
676     )
677     entries.append(entry)
678
679     self._catalog_cache = entries
680     self._cache_timestamp = now
681     return entries
682
683     def get_domains_summary(self) -> Dict[str, Any]:
684         """Формирует сводную аналитику по 7 доменам телеметрии
        ."""
685         catalog = self.get_full_catalog()
686         domains_map: Dict[str, Dict[str, Any]] = {
687             TelemetryDomain.SECURITY_IDENTITY.value: {
688                 'id': TelemetryDomain.SECURITY_IDENTITY.value,
689                 'title': '1.  Security / Identity',
690                 'description': 'Кто вошел, кто вышел, права до
ступа, UAC и аудит учетных записей.',
691                 'channels_count': 0,

```



```

692         'total_records': 0,
693         'key_events': [4624, 4625, 4672, 4720, 1102],
694         'status': 'Active',
695     },
696     TelemetryDomain.PROCESS_SERVICES_TASKS.value: {
697         'id': TelemetryDomain.PROCESS_SERVICES_TASKS.v
alue,
698         'title': '2. □ Process / Services / Tasks',
699         'description': 'Жизненный цикл процессов, аргу
менты CommandLine, службы SCM, задачи и скрипты.',
700         'channels_count': 0,
701         'total_records': 0,
702         'key_events': [4688, 4689, 7036, 7045, 106, 41
04],
703         'status': 'Active',
704     },
705     TelemetryDomain.STORAGE_FILESYSTEM.value: {
706         'id': TelemetryDomain.STORAGE_FILESYSTEM.value
,
707         'title': '3. □ Storage & Filesystem',
708         'description': 'Ошибки томов NTFS, S.M.A.R.T.
предупреждения и целостность накопителей.',
709         'channels_count': 0,
710         'total_records': 0,
711         'key_events': [140, 142, 1, 2, 200],
712         'status': 'Active',
713     },
714     TelemetryDomain.NETWORK_COMMUNICATIONS.value: {
715         'id': TelemetryDomain.NETWORK_COMMUNICATIONS.v
alue,
716         'title': '4. □ Network & Sockets',
717         'description': 'DNS-запросы, блокировки брандм
ауэра и соединения по протоколам SMB/TCP.',
718         'channels_count': 0,
719         'total_records': 0,
720         'key_events': [3008, 3018, 2004, 2005, 31001],
721         'status': 'Active',
722     },

```

```
723 |         TelemetryDomain.WINDOWS_UPDATE.value: {
724 |             'id': TelemetryDomain.WINDOWS_UPDATE.value,
725 |             'title': '5. □ Windows Update',
726 |             'description': 'Установка накопительных пакетов
| в KB, ошибки патчей и дедлайны обслуживания.',
727 |             'channels_count': 0,
728 |             'total_records': 0,
729 |             'key_events': [19, 20, 21, 25, 201],
730 |             'status': 'Active',
731 |         },
732 |         TelemetryDomain.HARDWARE_PNP_DRIVERS.value: {
733 |             'id': TelemetryDomain.HARDWARE_PNP_DRIVERS.val
| ue,
734 |             'title': '6. □ Hardware / Drivers / PnP',
735 |             'description': 'Подключение USB/PCIe оборудова
| ния, ошибки драйверов Code 10/43 и DriverStore.',
736 |             'channels_count': 0,
737 |             'total_records': 0,
738 |             'key_events': [400, 410, 411, 420, 112],
739 |             'status': 'Active',
740 |         },
741 |         TelemetryDomain.POWER_BOOT_SHUTDOWN.value: {
742 |             'id': TelemetryDomain.POWER_BOOT_SHUTDOWN.valu
| e,
743 |             'title': '7. □ Power / Boot / Shutdown',
744 |             'description': 'Цепочка жизненного цикла: Boot
| -> Login -> Sleep -> Wake -> Shutdown/Crash.',
745 |             'channels_count': 0,
746 |             'total_records': 0,
747 |             'key_events': [41, 42, 107, 1074, 6005, 6008],
748 |             'status': 'Active',
749 |         },
750 |         TelemetryDomain.SYSMON_OBSERVABILITY.value: {
751 |             'id': TelemetryDomain.SYSMON_OBSERVABILITY.val
| ue,
752 |             'title': '□ Sysmon Observability Layer',
753 |             'description': 'Глубокая сквозная телеметрия с
| оздания процессов с хэшами, сокетов и файлов.',
```

```

754         'channels_count': 0,
755         'total_records': 0,
756         'key_events': [1, 3, 6, 7, 10, 11, 22],
757         'status': 'Optional',
758     },
759 }
760
761     for item in catalog:
762         dom_key = item.domain.value if hasattr(item.domain
763         , 'value') else str(item.domain)
764         if dom_key in domains_map:
765             domains_map[dom_key]['channels_count'] += 1
766             domains_map[dom_key]['total_records'] += item.
767             record_count
768
769         sysmon_info = self.check_sysmon_status()
770         if not sysmon_info['installed']:
771             domains_map[TelemetryDomain.SYSMON_OBSERVABILITY.v
772             alue]['status'] = 'Не установлен'
773         else:
774             domains_map[TelemetryDomain.SYSMON_OBSERVABILITY.v
775             alue]['status'] = 'Активен'
776
777     return {
778         'domains': list(domains_map.values()),
779         'total_registered_providers': len(catalog),
780         'sysmon_installed': sysmon_info['installed'],
781         'generated_at': datetime.now(timezone.utc).isoform
782         at(),
783     }
784
785     def reconstruct_process_provenance(
786         self,
787         process_name: Optional[str] = None,
788         pid: Optional[int] = None,
789         hours: int = 24,
790     ) -> Dict[str, Any]:
791         """Строит дерево происхождения процессов (Process Prov

```

enance Graph).

```
787
788     Цепочка: User -> Parent Process -> Process -> Child Pr
processes -> Service / Task.
789     """
790     # Считываем события 4688 и 4689 из Security
791     events_4688 = self._storage.get_security_events(event_
id=4688, process_name=process_name, pid=pid, limit=100)
792
793     # Считываем события служб SCM (System)
794     scm_events = self._wevtapi.read_events(channel='System
', limit=100, event_id=7045, hours=hours)
795     # Считываем события TaskScheduler
796     task_events = self._wevtapi.read_events(channel='Micro
soft-Windows-TaskScheduler/Operational', limit=100, event_id=1
07, hours=hours)
797
798     nodes: List[Dict[str, Any]] = []
799     links: List[Dict[str, Any]] = []
800     seen_pids: Set[int] = set()
801
802     for ev in events_4688:
803         curr_pid = ev.get('process_id') or 0
804         parent_pid = ev.get('parent_process_id') or 0
805         pname = ev.get('process_name') or 'unknown.exe'
806         parent_name = ev.get('parent_process_name') or (f"
PID {parent_pid}" if parent_pid else 'explorer.exe')
807         user = ev.get('subject_user') or 'SYSTEM'
808         cmd = ev.get('command_line') or ''
809         ts = ev.get('timestamp') or ''
810
811         node_id = f"proc_{curr_pid}_{pname}"
812         parent_node_id = f"proc_{parent_pid}_{parent_name}"
813
814         if curr_pid not in seen_pids:
815             nodes.append({
816                 'id': node_id,
```

```

817         'type': 'process',
818         'name': pname.split('\\')[-1],
819         'full_path': pname,
820         'pid': curr_pid,
821         'user': user,
822         'command_line': cmd,
823         'timestamp': ts,
824         'role': 'Target Process',
825     })
826     seen_pids.add(curr_pid)
827
828     if parent_pid and parent_pid not in seen_pids:
829         nodes.append({
830             'id': parent_node_id,
831             'type': 'parent_process',
832             'name': parent_name.split('\\')[-1],
833             'full_path': parent_name,
834             'pid': parent_pid,
835             'user': user,
836             'command_line': '',
837             'timestamp': ts,
838             'role': 'Parent Creator',
839         })
840         seen_pids.add(parent_pid)
841
842     if parent_pid:
843         links.append({
844             'source': parent_node_id,
845             'target': node_id,
846             'relationship': 'spawns',
847             'label': f"Fork ({ts.split(' ')[-1] if ' '
in ts else ts})",
848         })
849
850     # Связываем со службами
851     for scm in scm_events[:10]:
852         srv_name = scm.get('message', '').split(' ')[0] or
'System Service'

```

```

853         srv_id = f"srv_{scm.get('event_id')}_{'srv_name'}"
854         nodes.append({
855             'id': srv_id,
856             'type': 'service',
857             'name': srv_name,
858             'timestamp': scm.get('timestamp', ''),
859             'role': 'Windows Service',
860         })
861
862     return {
863         'target_filter': {'process_name': process_name, 'p
864         id': pid},
865         'nodes': nodes,
866         'links': links,
867         'total_nodes': len(nodes),
868         'total_links': len(links),
869     }
870
871     def reconstruct_power_lifecycle(self, hours: int = 72) ->
872     List[Dict[str, Any]]:
873         """Реконструирует хронологию жизненного цикла ОС: Boot
874         -> Login -> Sleep -> Wake -> Shutdown/Crash."""
875         # Используем WevtAPI для чтения Kernel-Power, User32,
876         EventLog
877         pwr_events = self._wevtapi.read_events(
878             channel='System',
879             limit=200,
880             hours=hours,
881             event_id=[41, 1074, 6005, 6006, 6008, 12, 13, 42,
882             107],
883         )
884
885         timeline: List[Dict[str, Any]] = []
886         for ev in pwr_events:
887             eid = ev.get('event_id', 0)
888             msg = ev.get('message', '')
889             ts = ev.get('timestamp', '')
890             provider = ev.get('provider', '')

```

```
886 |
887 |     phase = 'WORK'
888 |     icon = '□ '
889 |     severity = 'info'
890 |
891 |     if eid in (6005, 12):
892 |         phase = 'BOOT'
893 |         icon = '□ '
894 |         msg = 'Запуск операционной системы и служб ядр
a'
895 |     elif eid == 42:
896 |         phase = 'SLEEP'
897 |         icon = '□ '
898 |         msg = 'Переход системы в спящий режим (Sleep)'
899 |     elif eid == 107:
900 |         phase = 'WAKE'
901 |         icon = '□ '
902 |         msg = 'Пробуждение системы из спящего режима'
903 |     elif eid == 1074:
904 |         phase = 'SHUTDOWN_PLANNED'
905 |         icon = '□ '
906 |         severity = 'warning'
907 |     elif eid in (6006, 13):
908 |         phase = 'SHUTDOWN_CLEAN'
909 |         icon = '□ '
910 |         msg = 'Штатное завершение работы Windows'
911 |     elif eid in (41, 6008):
912 |         phase = 'CRASH_UNEXPECTED'
913 |         icon = '□ '
914 |         severity = 'critical'
915 |         msg = 'Критический сбой питания / BSOD (Внезап
ная перезагрузка)'
916 |
917 |     timeline.append({
918 |         'timestamp': ts,
919 |         'phase': phase,
920 |         'icon': icon,
921 |         'event_id': eid,
```

```

922         'provider': provider,
923         'severity': severity,
924         'message': msg,
925         'raw_data': ev.get('raw_data', ''),
926     })
927
928     timeline.sort(key=lambda x: x['timestamp'], reverse=True)
929     return timeline
930
931     def reconstruct_hardware_pnp_chain(self, hours: int = 72)
932     -> List[Dict[str, Any]]:
933         """Реконструирует цепочку событий оборудования и драйв
934         еров Plug and Play."""
935         pnp_events = self._wevtapi.read_events(
936             channel='Microsoft-Windows-Kernel-PnP/Configuratio
937             n',
938             limit=100,
939             hours=hours,
940         )
941
942         chain: List[Dict[str, Any]] = []
943         for ev in pnp_events:
944             eid = ev.get('event_id', 0)
945             msg = ev.get('message', '')
946             ts = ev.get('timestamp', '')
947
948             status = 'Configured'
949             if eid == 411:
950                 status = 'Failed (Code 10/43)'
951             elif eid == 420:
952                 status = 'Unplugged / Removed'
953             elif eid == 410:
954                 status = 'Started'
955
956             chain.append({
957                 'timestamp': ts,
958                 'event_id': eid,

```



```

956         'status': status,
957         'message': msg,
958         'level': ev.get('level', 'Information'),
959     })
960
961     return chain
962
963     def check_sysmon_status(self) -> Dict[str, Any]:
964         """Проверяет статус службы и журнала Sysmon в системе.
965         """
966         rec_cnt = self._wevtapi.get_channel_record_count('Microsoft-Windows-Sysmon/Operational')
967         has_channel = rec_cnt > 0
968
969         # Проверка службы
970         service_running = False
971         try:
972             res = subprocess.run(['sc', 'query', 'Sysmon64'],
973                                 capture_output=True, text=True, timeout=3)
974             if 'RUNNING' in res.stdout:
975                 service_running = True
976             else:
977                 res2 = subprocess.run(['sc', 'query', 'Sysmon'],
978                                     capture_output=True, text=True, timeout=3)
979                 if 'RUNNING' in res2.stdout:
980                     service_running = True
981         except Exception:
982             service_running = False
983
984         return {
985             'installed': has_channel or service_running,
986             'service_running': service_running,
987             'channel_name': 'Microsoft-Windows-Sysmon/Operational',
988             'record_count': rec_cnt,
989             'capabilities': [
990                 'Process Creation with Hashes (SHA256, MD5, IM
991                 PHASH)',

```

```
988 |         'Network Connection Telemetry (Process -> IP:P
    | ort)',
989 |         'File Creation & Modification Tracking',
990 |         'Driver & Kernel Module Signature Verification
    | ',
991 |         'Process Memory Access / Injection Detection',
992 |         'DNS Queries per Process ID',
993 |     ] if (has_channel or service_running) else [],
994 | }
995 |
996 |
997 | WindowsEventCatalog = WindowsEventCatalogEngine
998 |
999 | __all__ = [
1000 |     'TelemetryDomain',
1001 |     'EventProviderCatalogEntry',
1002 |     'WindowsEventCatalogEngine',
1003 |     'WindowsEventCatalog',
1004 |     'EVENT_PROVIDERS_REGISTRY',
1005 | ]
1006 |
1007 |
```

__init__.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry Catalog -
  Package Root
4 # =====
5 # Description:
6 #   Каталог событий Windows, реестр провайдеров и Sysmon слои.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.catalog.event_catalog import
  WindowsEventCatalog
11 #
12 # File: __init__.py
13 # Project: ai-breadboard
14 # Package: apps.windows.telemetry.catalog
15 # Author: hypo69
16 # Copyright: © 2026 hypo69
17 # Updated: 2026-10-08 09:36:00
18 # =====
19
20 from __future__ import annotations
21 """Каталог событийных журналов и реестр провайдеров ETW/Sysmon
  ."""
22
23 from apps.windows.telemetry.catalog.event_catalog import WindowsEventCatalog
24
25 __all__ = [
26     "WindowsEventCatalog",
27 ]
28
```

collector.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - Collector
4 # =====
5 # Description:
6 #   Движок сбора системных метрик, аппаратной структуры и показателей
7 #   производительности.
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.collector import SystemCollector
11 #
12 #       service = SystemCollector()
13 #
14 # File: collector.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-08 12:54:00
20 # =====
21
22 from __future__ import annotations
23 """Движок сбора системных метрик, аппаратной структуры и показателей
24   производительности."""
25
26 import asyncio
27 import ctypes
28 import getpass
29 import locale
30 import os
```

```
30 import platform
31 import socket
32 import time
33 from datetime import datetime, timezone
34 from typing import Any, Dict, List, Optional
35 try:
36     import psutil
37     PSUTIL_AVAILABLE = True
38 except ImportError:
39     psutil = None
40     PSUTIL_AVAILABLE = False
41 from logger import logger
42 from .models import (
43     AnomalyItem,
44     BatteryMetrics,
45     CloudStorageInfo,
46     CpuMetrics,
47     DiskIoMetrics,
48     DiskPartitionMetrics,
49     DriverInfo,
50     GpuMetrics,
51     HardwareArchiveEntry,
52     HardwareAuditReport,
53     HardwareChangeItem,
54     HardwareDeviceAudit,
55     HardwareNode,
56     HardwareSensor,
57     MemoryMetrics,
58     MonitorInfo,
59     NetworkInterfaceMetrics,
60     NetworkPortMetrics,
61     NpuMetrics,
62     OfficeSuiteInfo,
63     PhysicalDiskHealth,
64     ProcessLifecycleEvent,
65     ProcessMetrics,
66     ProcessNetworkActivity,
67     ProcessProvenanceInfo,
```

```

68     ProcessProvenanceReport,
69     RamStickInfo,
70     StartupArchiveEntry,
71     StartupChangeItem,
72     SystemCoreMetrics,
73     SystemHardwareQuick,
74     SystemHealthAlerts,
75     SystemSnapshot,
76     WindowsUpdateInfo,
77 )
78 from .sensors import get_hardware_sensors
79 from apps.windows.telemetry_research.hardware_auditor import HardwareAuditor
80 from apps.windows.telemetry_research.hardware_history_manager import HardwareHistoryManager
81 from apps.windows.telemetry_research.startup_history_manager import StartupHistoryManager
82 from .process_token_collector import ProcessTokenCollector
83 from .sqlite import TelemetryStorage
84
85 class SystemCollector:
86     """Telemetry collector for system load, hardware devices,
87     and processes."""
88
89     def __init__(
90         self,
91         auditor: Optional[HardwareAuditor] = None,
92         history_manager: Optional[HardwareHistoryManager] = None,
93         startup_history_manager: Optional[StartupHistoryManager] = None,
94         storage: Optional[TelemetryStorage] = None,
95     ) -> None:
96         """Initialize telemetry collector with timing, I/O baseline, hardware auditor, startup history, and storage."""
97         self._last_disk_io = psutil.disk_io_counters() if PSUTIL_AVAILABLE else None
98         self._last_net_io = psutil.net_io_counters(pernic=True)

```

```

    ) if PSUTIL_AVAILABLE else None
98         self._last_proc_net_io: Dict[int, Tuple[float, float,
float]] = {}
99         self._last_time = time.time()
100        self._cpu_model_cached: Optional[str] = None
101        self._identity_cached: Optional[Dict[str, Any]] = None
102        self._monitors_cached: Optional[List[MonitorInfo]] = N
one
103        self._updates_cached: Optional[WindowsUpdateInfo] = No
ne
104        self._office_cached: Optional[OfficeSuiteInfo] = None
105        self._ram_sticks_cached: Optional[List[RamStickInfo]]
= None
106        self._ram_sticks_cache_time: float = 0.0
107        self._physical_disks_cached: Optional[List[PhysicalDis
kHealth]] = None
108        self._physical_disks_cache_time: float = 0.0
109        self.storage = storage or TelemetryStorage.get_instanc
e()
110        self.auditor = auditor or HardwareAuditor(storage=self
.storage)
111        self.history_manager = history_manager or HardwareHist
oryManager()
112        self.startup_history_manager = startup_history_manager
or StartupHistoryManager()
113
114        def get_system_identity(self) -> Dict[str, Any]:
115            """Collect host identity, current user, system languag
e, and locale parameters.
116
117            Returns:
118                Dict[str, Any]: Detailed system identity and local
ization dictionary.
119            """
120            if self._identity_cached:
121                return self._identity_cached
122            hostname = socket.gethostname()
123            domain = os.environ.get('USERDOMAIN', '')

```

```

124 |         username_raw = os.environ.get('USERNAME') or getpass.g
| etuser()
125 |         full_username = f'{domain}\\{username_raw}' if domain
| and domain != hostname else username_raw
126 |         user_locale = 'ru-RU'
127 |         system_locale = 'ru-RU'
128 |         codepage = 'UTF-8'
129 |         input_languages: List[str] = []
130 |         if os.name == 'nt':
131 |             try:
132 |                 buf = ctypes.create_unicode_buffer(100)
133 |                 if ctypes.windll.kernel32.GetUserDefaultLocale
| Name(buf, 100):
134 |                     user_locale = buf.value or 'ru-RU'
135 |                 if ctypes.windll.kernel32.GetSystemDefaultLoca
| leName(buf, 100):
136 |                     system_locale = buf.value or 'ru-RU'
137 |                 acp = ctypes.windll.kernel32.GetACP()
138 |                 oemcp = ctypes.windll.kernel32.GetOEMCP()
139 |                 codepage = f'ACP: {acp} | OEM: {oemcp} (UTF-8)
| ,
140 |                 count = ctypes.windll.user32.GetKeyboardLayout
| List(0, None)
141 |                 if count > 0:
142 |                     hkls = (ctypes.c_void_p * count)()
143 |                     ctypes.windll.user32.GetKeyboardLayoutList
| (count, hkls)
144 |                     lang_map = {1049: 'Русский (RU)', 1033: 'E
| nglish (US)', 1037: 'օօօօօ (IL)', 1058: 'Українська (UA)', 103
| 1: 'Deutsch (DE)', 1036: 'Français (FR)'}
145 |                     for h in hkls:
146 |                         lang_id = (h.value if hasattr(h, 'valu
| e') and h.value else int(h)) & 65535
147 |                         lang_name = lang_map.get(lang_id, f'La
| yout 0x{lang_id:04X}')
148 |                         if lang_name not in input_languages:
149 |                             input_languages.append(lang_name)
150 |             except Exception as ex:

```



```

151 |         logger.debug(f'Failed to query native Windows
| locale: {ex}')
152 |     if not input_languages:
153 |         input_languages = ['Русский (RU)', 'English (US)']
154 |     try:
155 |         tz_offset = datetime.now().astimezone().strftime('%z')
156 |         tz_name = datetime.now().astimezone().tzname() or
| time.tzname[0]
157 |         timezone_str = f'{tz_name} (UTC{tz_offset[:3]}:{tz
| _offset[3:]})'
158 |     except Exception:
159 |         timezone_str = 'UTC+03:00'
160 |     def _format_lang_display(locale_tag: str) -> str:
161 |         if not locale_tag:
162 |             return ''
163 |         tag_clean = locale_tag.strip()
164 |         tag_lower = tag_clean.lower()
165 |         if 'ru' in tag_lower:
166 |             return f'Русский (Россия) [{tag_clean}]'
167 |         elif 'he' in tag_lower:
168 |             return f'עברית (ישראל) [{tag_clean}]'
169 |         elif 'en' in tag_lower:
170 |             return f'English (United States) [{tag_clean}]'
|
171 |         elif 'uk' in tag_lower:
172 |             return f'Українська (Україна) [{tag_clean}]'
173 |         elif 'de' in tag_lower:
174 |             return f'Deutsch (Deutschland) [{tag_clean}]'
175 |         elif 'fr' in tag_lower:
176 |             return f'Français (France) [{tag_clean}]'
177 |         elif 'es' in tag_lower:
178 |             return f'Español (España) [{tag_clean}]'
179 |         elif 'zh' in tag_lower:
180 |             return f'中文 (China) [{tag_clean}]'
181 |         elif 'ja' in tag_lower:
182 |             return f'日本語 (Japan) [{tag_clean}]'
183 |         return tag_clean

```

```

184
185     sys_lang_display = _format_lang_display(user_locale)
186     os_build = platform.version() or '10.0.26200'
187     os_install_date = ''
188     os_install_language = ''
189     if os.name == 'nt':
190         try:
191             import winreg
192             # Попытка получить оригинальный язык установки
из реестра (InstallLanguage)
193             for access in (winreg.KEY_READ | winreg.KEY_WO
W64_64KEY, winreg.KEY_READ):
194                 try:
195                     with winreg.OpenKey(winreg.HKEY_LOCAL_
MACHINE, r'SYSTEM\CurrentControlSet\Control\Nls\Language', 0,
access) as key:
196                         val, _ = winreg.QueryValueEx(key,
'InstallLanguage')
197                         if val:
198                             lcid = int(str(val), 16)
199                             buf_lcid = ctypes.create_unico
de_buffer(100)
200                             if ctypes.windll.kernel32.LCID
ToLocaleName(lcid, buf_lcid, 100, 0):
201                                 tag = buf_lcid.value
202                                 if tag:
203                                     os_install_language =
_format_lang_display(tag)
204                                     break
205                             except Exception:
206                                 pass
207             if not os_install_language:
208                 sys_ui_lang = ctypes.windll.kernel32.GetSy
stemDefaultUILanguage()
209                 buf_lcid = ctypes.create_unicode_buffer(10
0)
210                 if ctypes.windll.kernel32.LCIDToLocaleName
(sys_ui_lang, buf_lcid, 100, 0):

```

```

211 |             tag = buf_lcid.value
212 |             if tag:
213 |                 os_install_language = _format_lang
                _display(tag)
214 |             except Exception as ex:
215 |                 logger.debug(f'Failed to query Windows Install
                Language: {ex}')
216 |
217 |             try:
218 |                 import winreg
219 |                 for access in (winreg.KEY_READ | winreg.KEY_WO
                W64_64KEY, winreg.KEY_READ):
220 |                     try:
221 |                         with winreg.OpenKey(winreg.HKEY_LOCAL_
                MACHINE, r'SOFTWARE\Microsoft\Windows NT\CurrentVersion', 0, a
                ccess) as key:
222 |                             try:
223 |                                 val, _ = winreg.QueryValueEx(k
                ey, 'InstallDate')
224 |                                 if val and int(val) > 0:
225 |                                     os_install_date = datetime
                .fromtimestamp(int(val)).strftime('%d.%m.%Y %H:%M')
226 |                                     break
227 |                             except Exception:
228 |                                 pass
229 |                             try:
230 |                                 val, _ = winreg.QueryValueEx(k
                ey, 'InstallTime')
231 |                                 if val and int(val) > 0:
232 |                                     epoch_sec = (int(val) / 10
                000000.0) - 11644473600.0
233 |                                     if epoch_sec > 0:
234 |                                         os_install_date = date
                time.fromtimestamp(epoch_sec).strftime('%d.%m.%Y %H:%M')
235 |                                         break
236 |                             except Exception:
237 |                                 pass
238 |             except Exception:

```

```

239 |             continue
240 |         except Exception as ex:
241 |             logger.debug(f'Failed to query Windows Install
Date from registry: {ex}')
242 |             workgroup = 'WORKGROUP'
243 |             if os.name == 'nt':
244 |                 try:
245 |                     from ctypes import wintypes
246 |                     netapi32 = ctypes.windll.netapi32
247 |                     lp_buf = wintypes.LPWSTR()
248 |                     join_stat = wintypes.DWORD()
249 |                     if netapi32.NetGetJoinInformation(None, ctypes
.byref(lp_buf), ctypes.byref(join_stat)) == 0:
250 |                         if lp_buf.value:
251 |                             workgroup = lp_buf.value
252 |                             netapi32.NetApiBufferFree(lp_buf)
253 |                 except Exception as ex:
254 |                     logger.debug(f'Failed to query Windows Workgro
up: {ex}')
255 |             self._identity_cached = {
256 |                 'hostname': hostname,
257 |                 'workgroup': workgroup,
258 |                 'username': full_username,
259 |                 'os_build': os_build,
260 |                 'os_install_date': os_install_date,
261 |                 'system_language': sys_lang_display,
262 |                 'os_install_language': os_install_language,
263 |                 'user_locale': user_locale,
264 |                 'system_locale': system_locale,
265 |                 'timezone': timezone_str,
266 |                 'codepage': codepage,
267 |                 'input_languages': input_languages
268 |             }
269 |             return self._identity_cached
270 |
271 |     async def _resolve_cpu_model(self) -> str:
272 |         """Resolve CPU brand/model name from registry, platfor
m or WMI.

```

```

273
274     Returns:
275         str: Resolved CPU model name.
276     """
277     if self._cpu_model_cached:
278         return self._cpu_model_cached
279     model = ''
280     if os.name == 'nt':
281         try:
282             import winreg
283             with winreg.OpenKey(winreg.HKEY_LOCAL_MACHINE,
284                                 r'HARDWARE\DESCRIPTION\System\CentralProcessor\0') as key:
285                 val, _ = winreg.QueryValueEx(key, 'Process
286                 orNameString')
287                 if val and val.strip():
288                     model = val.strip()
289             except Exception:
290                 pass
291         if not model:
292             model = platform.processor() or ''
293         if os.name == 'nt' and (not model or 'Intel64' in mode
294                                l or 'AMD64' in model):
295             try:
296                 from src.utils.com_worker import com_worker
297
298                 def _get_cpu():
299                     import pythoncom
300                     pythoncom.CoInitialize()
301                     import wmi
302                     w = wmi.WMI()
303                     cpus = w.Win32_Processor()
304                     return str(cpus[0].Name).strip() if cpus e
305             lse None
306             res = await com_worker.run(_get_cpu)
307             if res:
308                 model = res
309         except Exception:
310             pass

```

```

307         if not model:
308             model = f'{platform.machine()} {os.cpu_count() or
1}-Core Processor'
309             self._cpu_model_cached = model
310             return model
311
312     async def get_cpu_metrics(self) -> CpuMetrics:
313         """Collect real-time CPU utilization and architecture
314         details.
315
316         Returns:
317             CpuMetrics: Current CPU performance metrics.
318         """
319         logical_cores = os.cpu_count() or 1
320         physical_cores = max(1, logical_cores // 2) if logical
321         _cores > 1 else 1
322         if PSUTIL_AVAILABLE:
323             per_core = psutil.cpu_percent(interval=None, percp
324             u=True)
325             total = round(sum(per_core) / max(len(per_core), 1
326             ), 1) if per_core else 0.0
327             physical = psutil.cpu_count(logical=False) or phys
328             ical_cores
329             logical = psutil.cpu_count(logical=True) or logica
330             l_cores
331             freq_mhz = 0.0
332             try:
333                 freq = psutil.cpu_freq()
334                 if freq:
335                     freq_mhz = round(freq.current, 1)
336             except Exception:
337                 pass
338             return CpuMetrics(model=await self._resolve_cpu_mo
339             del(), architecture=platform.machine(), physical_cores=physica
340             l, logical_cores=logical, total_percent=total, per_core_percen
341             t=per_core, frequency_mhz=freq_mhz)
342         return CpuMetrics(model=await self._resolve_cpu_model(
343         ), architecture=platform.machine(), physical_cores=physical_co

```

```

    res, logical_cores=logical_cores, total_percent=10.0, per_core
_percent=[10.0] * logical_cores, frequency_mhz=2400.0)
334
335     def get_memory_metrics(self) -> MemoryMetrics:
336         """Collect RAM and swap memory consumption.
337
338         Returns:
339             MemoryMetrics: Current memory usage statistics.
340         """
341         if PSUTIL_AVAILABLE:
342             vm = psutil.virtual_memory()
343             sm = psutil.swap_memory()
344             return MemoryMetrics(total_gb=round(vm.total / 102
4 ** 3, 2), available_gb=round(vm.available / 1024 ** 3, 2), u
sed_gb=round((vm.total - vm.available) / 1024 ** 3, 2), percen
t=round(vm.percent, 1), swap_total_gb=round(sm.total / 1024 **
3, 2), swap_used_gb=round(sm.used / 1024 ** 3, 2), swap_perce
nt=round(sm.percent, 1))
345             total_ram_gb = 16.0
346             used_ram_gb = 8.0
347             if os.name == 'nt':
348                 try:
349
350                     class MEMORYSTATUSEX(ctypes.Structure):
351                         _fields_ = [('dwLength', ctypes.c_ulong),
('dwMemoryLoad', ctypes.c_ulong), ('ullTotalPhys', ctypes.c_ul
onglong), ('ullAvailPhys', ctypes.c_ulonglong), ('ullTotalPage
File', ctypes.c_ulonglong), ('ullAvailPageFile', ctypes.c_ulon
glong), ('ullTotalVirtual', ctypes.c_ulonglong), ('ullAvailVir
tual', ctypes.c_ulonglong), ('sullAvailExtendedVirtual', ctype
s.c_ulonglong)]
352                         stat = MEMORYSTATUSEX()
353                         stat.dwLength = ctypes.sizeof(MEMORYSTATUSEX)
354                         if ctypes.windll.kernel32.GlobalMemoryStatusEx
(ctypes.byref(stat)):
355                             total_ram_gb = round(stat.ullTotalPhys / 1
024 ** 3, 2)
356                             avail_ram_gb = round(stat.ullAvailPhys / 1

```

```

024 ** 3, 2)
357         used_ram_gb = round(total_ram_gb - avail_r
am_gb, 2)
358         percent = round(stat.dwMemoryLoad, 1)
359         return MemoryMetrics(total_gb=total_ram_gb
, available_gb=avail_ram_gb, used_gb=used_ram_gb, percent=perc
ent, swap_total_gb=round(stat.ullTotalPageFile / 1024 ** 3, 2)
, swap_used_gb=round((stat.ullTotalPageFile - stat.ullAvailPag
eFile) / 1024 ** 3, 2), swap_percent=round((1.0 - stat.ullAvai
lPageFile / max(stat.ullTotalPageFile, 1)) * 100, 1))
360     except Exception:
361         pass
362     return MemoryMetrics(total_gb=total_ram_gb, available_
gb=total_ram_gb - used_ram_gb, used_gb=used_ram_gb, percent=50
.0, swap_total_gb=16.0, swap_used_gb=2.0, swap_percent=12.5)
363
364     def get_gpu_metrics(self) -> List[GpuMetrics]:
365         """Collect GPU accelerator load, VRAM, and capabilitie
s.
366
367         Returns:
368             List[GpuMetrics]: Detected GPU devices with teleme
try.
369         """
370         gpus: List[GpuMetrics] = []
371         try:
372             from apps.windows.sdk.modules.hardware.gpu_prober
import GpuProber, determine_gpu_compute_backends
373             prober = GpuProber()
374             gpu_list = prober.probe_all()
375             for g in gpu_list:
376                 vendor = getattr(g, 'vendor', '') or ''
377                 name = getattr(g, 'name', '') or ''
378                 caps = determine_gpu_compute_backends(
379                     vendor,
380                     name,
381                     memory_total_mb=g.memory_total_mb,
382                     dedicated_mb=g.dedicated_memory_mb or (g.m

```



```

memory_total_mb if getattr(g, 'gpu_type', '') == 'Discrete' else
None),
383         shared_mb=g.shared_memory_mb,
384     )
385     gpus.append(GpuMetrics(
386         name=g.name,
387         vendor=caps['vendor'],
388         gpu_type=caps['gpu_type'],
389         memory_total_gb=round((g.memory_total_mb or
r 0.0) / 1024.0, 2),
390         memory_used_gb=round((g.memory_used_mb or
0.0) / 1024.0, 2),
391         dedicated_memory_mb=caps['dedicated_memory
_mb'],
392         shared_memory_mb=caps['shared_memory_mb'],
393         memory_type=caps['memory_type'],
394         directx_version=caps['directx_version'],
395         load_percent=g.utilization_gpu_pct,
396         temperature_celsius=g.temperature_gpu_c,
397         has_cuda=caps['has_cuda'],
398         has_rocm=caps['has_rocm'],
399         has_oneapi=caps['has_oneapi'],
400         has_directml=caps['has_directml'],
401         has_vulkan=caps['has_vulkan'],
402         has_opengl=caps['has_opengl'],
403         compute_apis=caps['compute_apis'],
404         ai_backends=caps['ai_backends'],
405         engines=g.engines,
406     ))
407     except Exception as ex:
408         logger.debug(f'Ошибка сбора метрик GPU через GpuPr
ober: {ex}')
409     if not gpus:
410         gpus.append(GpuMetrics(
411             name='Integrated Display Controller',
412             vendor='Generic',
413             gpu_type='Integrated',
414             memory_total_gb=0.0,

```

```

415         dedicated_memory_mb=128.0,
416         shared_memory_mb=8192.0,
417         memory_type='Shared System Memory',
418         directx_version='DirectX 12 (FL 12_1)',
419         has_cuda=False,
420         has_rocm=False,
421         has_oneapi=False,
422         has_directml=True,
423         has_vulkan=True,
424         has_opengl=True,
425         compute_apis=['DirectML', 'Vulkan', 'OpenCL'],
426         ai_backends=['DirectML', 'Vulkan', 'CPU'],
427     ))
428     return gpus
429
430     def get_npu_metrics(self) -> List[NpuMetrics]:
431         """Collect NPU neural accelerator devices and AI capabilities.
432
433         Returns:
434             List[NpuMetrics]: Detected NPU devices.
435         """
436         npus: List[NpuMetrics] = []
437         try:
438             from apps.windows.sdk.modules.hardware.gpu_prober
439             import GpuProber
440             prober = GpuProber()
441             npu_list = prober.probe_npus()
442             for n in npu_list:
443                 npus.append(NpuMetrics(
444                     name=n.name,
445                     vendor=n.vendor,
446                     driver_version=n.driver_version,
447                     pnp_device_id=getattr(n, 'pnp_device_id',
448                                     ''),
449                     status=n.status,
450                     tops=n.tops,
451                     has_directml=n.has_directml,

```

```

450         has_qnn=n.has_qnn,
451         has_openvino=n.has_openvino,
452         ai_backends=n.ai_backends,
453     ))
454 except Exception as ex:
455     logger.debug(f'Ошибка сбора метрик NPU: {ex}')
456 return npus
457
458 def get_disk_metrics(self) -> tuple[List[DiskPartitionMetrics], DiskIoMetrics]:
459     """Collect disk partitions capacity and read/write rates.
460
461     Returns:
462         tuple[List[DiskPartitionMetrics], DiskIoMetrics]:
463         Partitions and I/O rates.
464     """
465     partitions: List[DiskPartitionMetrics] = []
466     io_metrics = DiskIoMetrics()
467     if PSUTIL_AVAILABLE:
468         for part in psutil.disk_partitions(all=False):
469             try:
470                 usage = psutil.disk_usage(part.mountpoint)
471                 vol_name = ''
472                 is_virtual = False
473                 drive_type = 'Fixed'
474                 if os.name == 'nt':
475                     try:
476                         import ctypes
477                         vol_buf = ctypes.create_unicode_buffer(1024)
478                         fs_buf = ctypes.create_unicode_buffer(1024)
479                         mount_root = part.mountpoint if part.mountpoint.endswith('\\') else f'{part.mountpoint}\\'
480                         if ctypes.windll.kernel32.GetVolumeInformationW(mount_root, vol_buf, 1024, None, None, None, fs_buf, 1024):

```

```

480         vol_name = vol_buf.value
481         dos_buf = ctypes.create_unicode_bu
ffer(1024)
482         clean_dev = part.device.rstrip('\\
')
483         ctypes.windll.kernel32.QueryDosDev
iceW(clean_dev, dos_buf, 1024)
484         dos_dev = dos_buf.value
485         if 'google' in vol_name.lower() or
'@' in vol_name or 'onedrive' in vol_name.lower() or ('dropbo
x' in vol_name.lower()) or ('volume{' in dos_dev.lower() and '
harddiskvolume' not in dos_dev.lower()):
486             is_virtual = True
487             drive_type = 'Google Drive' if
'google' in vol_name.lower() or '@' in vol_name else 'Cloud /
Virtual'
488         except Exception:
489             pass
490         partitions.append(DiskPartitionMetrics(dev
ice=part.device, mountpoint=part.mountpoint, fstype=part.fstyp
e, total_gb=round(usage.total / 1024 ** 3, 2), used_gb=round(u
sage.used / 1024 ** 3, 2), free_gb=round(usage.free / 1024 **
3, 2), percent=round(usage.percent, 1), volume_name=vol_name o
r None, is_virtual=is_virtual, drive_type=drive_type))
491         except (PermissionError, OSError):
492             continue
493         now = time.time()
494         elapsed = max(now - self._last_time, 0.1)
495         current_io = psutil.disk_io_counters()
496         if current_io and self._last_disk_io:
497             io_metrics.read_bytes_per_sec = max(0.0, (curr
ent_io.read_bytes - self._last_disk_io.read_bytes) / elapsed)
498             io_metrics.write_bytes_per_sec = max(0.0, (cur
rent_io.write_bytes - self._last_disk_io.write_bytes) / elapse
d)
499             io_metrics.read_count_per_sec = max(0.0, (curr
ent_io.read_count - self._last_disk_io.read_count) / elapsed)
500             io_metrics.write_count_per_sec = max(0.0, (cur

```

```

    rent_io.write_count - self._last_disk_io.write_count) / elapse
d)
501         self._last_disk_io = current_io
502     else:
503         drive = 'C:\\' if os.name == 'nt' else '/'
504         try:
505             import shutil
506             total, used, free = shutil.disk_usage(drive)
507             partitions.append(DiskPartitionMetrics(device=
'C:' if os.name == 'nt' else '/', mountpoint=drive, fstype='NT
FS' if os.name == 'nt' else 'ext4', total_gb=round(total / 102
4 ** 3, 2), used_gb=round(used / 1024 ** 3, 2), free_gb=round(
free / 1024 ** 3, 2), percent=round(used / max(total, 1) * 100
, 1)))
508         except Exception:
509             pass
510     return (partitions, io_metrics)
511
512     def get_network_metrics(self) -> List[NetworkInterfaceMetr
ics]:
513         """Collect network interface statuses and throughput r
ates.
514
515         Returns:
516             List[NetworkInterfaceMetrics]: Network adapters me
etrics.
517         """
518         interfaces: List[NetworkInterfaceMetrics] = []
519         adapter_stats_map = {}
520         try:
521             from apps.windows.network.network_usage import Win
dowsNetworkUsageCollector
522             usage_collector = WindowsNetworkUsageCollector()
523             stats_list = usage_collector.get_adapter_statistic
s()
524             adapter_stats_map = {s.name.lower(): s for s in st
ats_list}
525         except Exception as ex:

```

```

526         logger.debug(f'Failed to fetch adapter stats in Sy
stemCollector: {ex}')
527
528         if PSUTIL_AVAILABLE:
529             addrs = psutil.net_if_addrs()
530             stats = psutil.net_if_stats()
531             current_net_io = psutil.net_io_counters(pernic=True
e)
532             now = time.time()
533             elapsed = max(now - self._last_time, 0.1)
534             for name, addr_list in addrs.items():
535                 stat = stats.get(name)
536                 is_up = stat.isup if stat else True
537                 speed = stat.speed if stat else 0
538                 ips = [a.address for a in addr_list if a.famil
y == socket.AF_INET]
539                 sent_rate = 0.0
540                 recv_rate = 0.0
541                 if current_net_io and name in current_net_io a
nd self._last_net_io and (name in self._last_net_io):
542                     cur = current_net_io[name]
543                     prev = self._last_net_io[name]
544                     sent_rate = max(0.0, (cur.bytes_sent - pre
v.bytes_sent) / elapsed)
545                     recv_rate = max(0.0, (cur.bytes_recv - pre
v.bytes_recv) / elapsed)
546                     st_info = adapter_stats_map.get(name.lower())
547                     interfaces.append(NetworkInterfaceMetrics(name
=name, is_up=is_up, speed_mbps=speed, bytes_sent_per_sec=sent_
rate, bytes_recv_per_sec=recv_rate, ip_addresses=ips, statisti
cs=st_info))
548                 self._last_net_io = current_net_io
549             else:
550                 try:
551                     host_ip = socket.gethostbyname(socket.gethostn
ame())
552                 except Exception:
553                     host_ip = '127.0.0.1'

```

```

554         interfaces.append(NetworkInterfaceMetrics(name='Primary Network Adapter', is_up=True, speed_mbps=1000, bytes_sent_per_sec=0.0, bytes_recv_per_sec=0.0, ip_addresses=[host_ip]))
555     return interfaces
556
557     def get_network_usage_report(self, period_minutes: int = 1440):
558         """Получить глубокий отчет сетевого потребления за период для AI диагностики.
559
560         Args:
561             period_minutes: Интервал отчета в минутах.
562
563         Returns:
564             NetworkUsagePeriodReport: Полная сводка сетевой статистики за период.
565         """
566         from apps.windows.network.network_usage import WindowsNetworkUsageCollector
567         collector = WindowsNetworkUsageCollector()
568         return collector.get_traffic_period_summary(period_minutes=period_minutes)
569
570     def get_disk_usage_report(self, period_minutes: int = 1440, force_refresh_smart: bool = False):
571         """Получить глубокий отчет использования дискового I/O и SMART за период для AI диагностики.
572
573         Args:
574             period_minutes: Интервал отчета в минутах (по умолчанию 1440 = 24 часа).
575             force_refresh_smart: Принудительно опросить физические диски.
576
577         Returns:
578             DiskUsagePeriodReport: Полная сводка дискового I/O и SMART-показателей за период.

```

```

579         """
580         from apps.windows.telemetry.storage_usage import WindowsStorageUsageCollector
581         collector = WindowsStorageUsageCollector()
582         return collector.get_disk_usage_period_report(period_minutes=period_minutes, force_refresh_smart=force_refresh_smart)
583
584
585
586     @staticmethod
587     def _classify_launch_reason(
588         parent_name: Optional[str],
589         process_name: Optional[str] = None,
590         session_id: Optional[int] = None,
591         cmdline: Optional[str] = None,
592     ) -> str:
593         """Классификация причины/контекста запуска процесса по его родителю и окружению.
594
595         Args:
596             parent_name: Имя родительского процесса.
597             process_name: Имя текущего процесса.
598             session_id: Номер сессии Windows (0 = системные службы, 1+ = интерактивная сессия).
599             cmdline: Командная строка процесса (опционально).
600
601         Returns:
602             str: Категория запуска ('service', 'scheduled_task', 'shell_interactive', 'terminal_cli', 'ide_developer', 'system_boot', 'spawned_by_parent', 'unknown').
603         """
604         p_name = (parent_name or '').lower()
605         if p_name in ('services.exe',) or (p_name in ('svchost.exe',) and session_id == 0):
606             return 'service'
607         if p_name in ('taskeng.exe', 'taskhostw.exe', 'taskhost.exe') or (cmdline and '-task' in cmdline.lower()):

```



```

608         return 'scheduled_task'
609     if p_name in ('explorer.exe',):
610         return 'shell_interactive'
611     if p_name in ('cmd.exe', 'powershell.exe', 'pwsh.exe',
        'wt.exe', 'windowsterminal.exe', 'bash.exe', 'zsh.exe', 'conh
        ost.exe'):
612         return 'terminal_cli'
613     if p_name in ('code.exe', 'devenv.exe', 'pycharm64.exe
        ', 'idea64.exe', 'rider64.exe', 'cursor.exe', 'windsurf.exe'):
614         return 'ide_developer'
615     if p_name in ('winlogon.exe', 'userinit.exe', 'smss.ex
        e', 'csrss.exe', 'wininit.exe', 'lsass.exe'):
616         return 'system_boot'
617     if p_name:
618         clean_parent = p_name.replace('.exe', '')
619         return f'spawned_by_{clean_parent}'
620     return 'unknown'
621
622     def get_top_processes(self, limit: Any = 25, sort_by: str
        = 'cpu') -> List[ProcessMetrics]:
623         """Retrieve active processes sorted by resource consum
        ption, with token and provenance info.
624
625         Args:
626             limit: Maximum processes to return (0 for all).
627             sort_by: Attribute to sort by ('cpu', 'memory'/'ra
        m', 'handles'/'descriptors').
628
629         Returns:
630             List[ProcessMetrics]: Ranked process metrics, incl
        uding provenance, integrity_level and elevation.
631         """
632         limit_val = limit.default if hasattr(limit, "default")
        else (int(limit) if isinstance(limit, (int, float, str)) and
        str(limit).isdigit() else 25)
633         # Collect token information for all processes
634         token_collector = ProcessTokenCollector()
635         token_info_map = {info.pid: info for info in token_col
        lector.collect()}

```

```

635 |
636 |         # Классификация оконных процессов
637 |         from apps.windows.sdk.modules.process_manager.core.classifier import (
638 |             ProcessClassifier,
639 |             WINDOWS_SYSTEM_PROCESS_NAMES,
640 |             KNOWN_APP_FRIENDLY_NAMES,
641 |         )
642 |         classifier = ProcessClassifier()
643 |         windowed_pids, titles_map = classifier.get_windowed_pids_and_titles()
644 |
645 |         procs: List[ProcessMetrics] = []
646 |         if PSUTIL_AVAILABLE:
647 |             handle_attr = 'num_handles' if os.name == 'nt' else 'num_fds'
648 |             proc_dict: Dict[int, Dict[str, Any]] = {}
649 |             for p in psutil.process_iter(attrs=['pid', 'ppid', 'name', 'status', 'cpu_percent', 'memory_info', 'memory_percent', 'num_threads', 'username', 'create_time', 'exe', 'cmdline', 'cpu_times', 'io_counters', handle_attr]):
650 |                 try:
651 |                     proc_dict[p.info['pid']] = p.info
652 |                 except (psutil.NoSuchProcess, psutil.AccessDenied):
653 |                     continue
654 |
655 |             name_to_windowed = {
656 |                 (info.get('name') or '').lower(): True
657 |                 for p_id, info in proc_dict.items()
658 |                 if p_id in windowed_pids
659 |             }
660 |
661 |             for pid, info in proc_dict.items():
662 |                 try:
663 |                     p_name = info.get('name') or 'unknown'
664 |                     low_name = p_name.lower()
665 |                     exe_low = (info.get('exe') or '').lower()

```

```

666 |
667 |         mem_info = info.get('memory_info')
668 |         rss_bytes = int(mem_info.rss) if mem_info
and hasattr(mem_info, 'rss') else 0
669 |         rss_mb = round(rss_bytes / (1024 * 1024),
1)
670 |         private_bytes = int(getattr(mem_info, 'vms
', 0) or getattr(mem_info, 'private', 0) or 0) if mem_info els
e 0
671 |         page_faults = int(getattr(mem_info, 'num_p
age_faults', 0) or 0) if mem_info else 0
672 |
673 |         cpu_times = info.get('cpu_times')
674 |         user_time_ms = int(getattr(cpu_times, 'use
r', 0) * 1000) if cpu_times else 0
675 |         kernel_time_ms = int(getattr(cpu_times, 's
ystem', 0) * 1000) if cpu_times else 0
676 |
677 |         io_cnt = info.get('io_counters')
678 |         rb_total = int(getattr(io_cnt, 'read_bytes
', 0) or 0) if io_cnt else 0
679 |         wb_total = int(getattr(io_cnt, 'write_byte
s', 0) or 0) if io_cnt else 0
680 |         ro_total = int(getattr(io_cnt, 'read_count
', 0) or 0) if io_cnt else 0
681 |         wo_total = int(getattr(io_cnt, 'write_coun
t', 0) or 0) if io_cnt else 0
682 |
683 |         cpu_p = round(info.get('cpu_percent') or 0
.0, 1)
684 |         num_h = info.get(handle_attr) or 0
685 |         token = token_info_map.get(pid)
686 |         integrity = token.integrity_level if token
else None
687 |         elevation = token.elevation if token else
None
688 |         sid = token.sid if token else None
689 |         session_id = token.session_id if token els

```

```

        e None
690
691         ppid = info.get('ppid')
692         parent_info = proc_dict.get(ppid) if ppid
        else None
693         parent_name = parent_info.get('name') if p
        arent_info else None
694
695         # Построение цепочки предков
696         ancestors = [p_name or str(pid)]
697         curr_pid = ppid
698         visited = {pid}
699         while curr_pid and curr_pid in proc_dict a
        nd curr_pid not in visited:
700             visited.add(curr_pid)
701             curr_p = proc_dict[curr_pid]
702             ancestors.append(curr_p.get('name') or
        f'PID:{curr_pid}')
703             curr_pid = curr_p.get('ppid')
704             ancestors.reverse()
705             ancestor_chain = ' → '.join(ancestors)
706
707             create_time_val = info.get('create_time')
        or 0.0
708             creation_time_str = datetime.fromtimestamp
        (create_time_val, tz=timezone.utc).isoformat() if create_time_
        val > 0 else None
709             process_guid = f'proc_{pid}_{int(create_ti
        me_val)}'
710
711             raw_cmdline = info.get('cmdline')
712             cmdline_str = ' '.join(raw_cmdline) if isi
        nstance(raw_cmdline, list) else (str(raw_cmdline) if raw_cmdli
        ne else None)
713             launch_reason = self._classify_launch_reas
        on(parent_name, p_name, session_id, cmdline_str)
714
715             # Классификация категории

```

```

716         has_win = pid in windowed_pids
717         is_app = has_win or (low_name in name_to_w
indowed and low_name not in WINDOWS_SYSTEM_PROCESS_NAMES)
718         if is_app:
719             category = 'app'
720         elif low_name in WINDOWS_SYSTEM_PROCESS_NAMES or ('c:\\windows\\system32' in exe_low and not has_win):
721             category = 'windows'
722         else:
723             category = 'background'
724
725         win_titles = titles_map.get(pid, [])
726         first_title = win_titles[0] if win_titles
else None
727         friendly = classifier._get_friendly_name(p
_name, win_titles)
728
729         procs.append(ProcessMetrics(
730             pid=pid,
731             name=p_name,
732             status=info.get('status') or 'running'
,
733             cpu_percent=cpu_p,
734             memory_mb=rss_mb,
735             memory_percent=round(info.get('memory_
percent') or 0.0, 1),
736             num_threads=info.get('num_threads') or
1,
737             num_handles=num_h,
738             username=info.get('username'),
739             integrity_level=integrity,
740             elevation=elevation,
741             ppid=ppid,
742             parent_name=parent_name,
743             executable_path=info.get('exe'),
744             sid=sid,
745             session_id=session_id,
746             creation_time=creation_time_str,

```

```

747         process_guid=process_guid,
748         ancestor_chain=ancestor_chain,
749         launch_reason=launch_reason,
750         category=category,
751         is_app=is_app,
752         friendly_name=friendly,
753         window_title=first_title,
754         user_time_ms=user_time_ms,
755         kernel_time_ms=kernel_time_ms,
756         working_set_bytes=rss_bytes,
757         private_bytes=private_bytes,
758         page_faults_count=page_faults,
759         read_bytes_total=rb_total,
760         write_bytes_total=wb_total,
761         read_ops_total=ro_total,
762         write_ops_total=wo_total,
763     ))
764     except (psutil.NoSuchProcess, psutil.AccessDen
ied):
765         continue
766     else:
767         # Fallback for non-psutil environments
768         now_iso = datetime.now(timezone.utc).isoformat()
769         p = ProcessMetrics(
770             pid=os.getpid(),
771             name='python.exe',
772             status='running',
773             cpu_percent=1.5,
774             memory_mb=120.0,
775             memory_percent=1.0,
776             num_threads=8,
777             num_handles=42,
778             username=os.getenv('USERNAME', 'SYSTEM'),
779             ppid=os.getppid() if hasattr(os, 'getppid') el
se None,
780             parent_name='cmd.exe',
781             executable_path=os.path.abspath(__file__),
782             sid='S-1-5-21-mock-sid',

```

```

783         session_id=1,
784         creation_time=now_iso,
785         process_guid=f'proc_{os.getpid()}_{int(time.time())}',
786         ancestor_chain='cmd.exe → python.exe',
787         launch_reason='terminal_cli',
788         category='background',
789         is_app=False,
790         friendly_name='Python',
791     )
792     token = token_info_map.get(p.pid)
793     if token:
794         p.integrity_level = token.integrity_level
795         p.elevation = token.elevation
796         p.sid = token.sid
797         p.session_id = token.session_id
798     procs.append(p)
799
800     # Sorting according to requested key
801     sort_key = (sort_by or 'cpu').lower()
802     if sort_key in ('handles', 'descriptors'):
803         procs.sort(key=lambda x: x.num_handles, reverse=True)
804     elif sort_key in ('memory', 'ram'):
805         procs.sort(key=lambda x: x.memory_mb, reverse=True)
806     else:
807         procs.sort(key=lambda x: x.cpu_percent, reverse=True)
808
809     if limit_val is None or limit_val <= 0:
810         return procs
811     return procs[:limit_val]
812
813     def get_battery_metrics(self) -> BatteryMetrics:
814         """Collect laptop or UPS battery charge level and AC power status.
815

```

```

816 |         Returns:
817 |             BatteryMetrics: Current battery and power status.
818 |             """
819 |         if PSUTIL_AVAILABLE and hasattr(psutil, 'sensors_battery'):
820 |             try:
821 |                 bat = psutil.sensors_battery()
822 |                 if bat is not None:
823 |                     return BatteryMetrics(has_battery=True, percent=round(bat.percent, 1), power_plugged=bat.power_plugged, secs_left=bat.secsleft if bat.secsleft > 0 else None, power_profile='AC / Battery')
824 |             except Exception as ex:
825 |                 logger.debug(f'Battery probe failed: {ex}')
826 |             return BatteryMetrics(has_battery=False, percent=None, power_plugged=True, secs_left=None, power_profile='AC Mains / Desktop')
827 |
828 |     def get_physical_disks_health(self, force: bool = False) -> List[PhysicalDiskHealth]:
829 |         """Собрать данные о физических дисках, SMART, типах накопителей и здоровье.
830 |
831 |         В сценариях API (tc.ps1) данные читаются строго из SQLite базы данных telemetry.db
832 |         без вызова тяжелого опроса WMI/CIM.
833 |         Прямой опрос оборудования через WindowsStorageSensor выполняется только при force=True (в демоне tlm.ps1).
834 |
835 |         Args:
836 |             force: Принудительное обновление данных накопителя через WindowsStorageSensor (только для tlm.ps1).
837 |
838 |         Returns:
839 |             List[PhysicalDiskHealth]: Список обнаруженных физических накопителей.
840 |             """
841 |         now = time.time()

```



```

842         if not force and self._physical_disks_cached is not No
ne and now - self._physical_disks_cache_time < 43200.0:
843             return self._physical_disks_cached
844
845         # 1. Попытка прочитать физические диски из базы данных
SQLite
846         if not force and self.storage:
847             try:
848                 storage_data = self.storage.get_latest_storage
_data()
849                 if storage_data and storage_data.get("physical
_disks"):
850                     disks: List[PhysicalDiskHealth] = []
851                     for d in storage_data["physical_disks"]:
852                         disks.append(PhysicalDiskHealth(
853                             device_id=str(d.get("device_id")) o
r f"Disk {len(disks)}"),
854                             model=str(d.get("model") or d.get(
"friendly_name") or d.get("name") or "Physical Drive"),
855                             serial_number=str(d.get("serial_nu
mber")) if d.get("serial_number") and d.get("serial_number") !=
"N/A" else None,
856                             media_type=str(d.get("media_type")
or "SSD"),
857                             size_gb=round(float(d.get("size_gb
") or 0.0), 1),
858                             health_status=str(d.get("health_st
atus") or "Healthy"),
859                             operational_status=str(d.get("oper
ational_status") or "OK"),
860                             temperature_celsius=float(d.get("t
emperature_celsius") or d.get("temperature_c") or 0.0) if (d.g
et("temperature_celsius") is not None or d.get("temperature_c"
) is not None) else None,
861                             interface_type=str(d.get("interfac
e_type") or d.get("bus_type") or "NVMe"),
862                             lifetime_read_bytes=d.get("lifetim
e_read_bytes"),

```

```

863         lifetime_write_bytes=d.get("lifetime_
me_write_bytes"),
864         lifetime_read_tb=d.get("lifetime_r
ead_tb"),
865         lifetime_write_tb=d.get("lifetime_
write_tb"),
866         power_on_hours=d.get("power_on_hou
rs"),
867         wear_percentage=d.get("wear_perce
ntage"),
868     ))
869     if disks:
870         self._physical_disks_cached = disks
871         self._physical_disks_cache_time = now
872         return disks
873     except Exception as ex:
874         logger.debug(f"Не удалось прочитать physical_d
isks из БД: {ex}")
875
876     # 2. Прямой опрос через WindowsStorageSensor при force
=True или отсутствии дисков в БД
877     disks: List[PhysicalDiskHealth] = []
878     if os.name == 'nt':
879         try:
880             from apps.windows.sdk.modules.storage_manager.
core.windows_storage_sensor import WindowsStorageSensor
881             sensor = WindowsStorageSensor(ttl_sec=43200.0)
882             phys_disks = sensor.get_physical_disks(force_r
efresh=force, sync=True)
883             for d in phys_disks:
884                 disks.append(PhysicalDiskHealth(
885                     device_id=d.device_id,
886                     model=d.model or d.friendly_name or 'P
hysical Drive',
887                     serial_number=d.serial_number if d.ser
ial_number and d.serial_number != 'N/A' else None,
888                     media_type=d.media_type or 'SSD',
889                     size_gb=round(d.size_gb, 1) if d.size_

```

```

    gb else 0.0,
890         health_status=d.health_status or 'Healthy',
891         operational_status=d.operational_status or 'OK',
892         temperature_celsius=d.temperature_c,
893         interface_type=d.bus_type or 'NVMe',
894         lifetime_read_bytes=d.lifetime_read_bytes,
        tes,
895         lifetime_write_bytes=d.lifetime_write_bytes,
896         lifetime_read_tb=d.lifetime_read_tb,
897         lifetime_write_tb=d.lifetime_write_tb,
898         power_on_hours=d.power_on_hours,
899         wear_percentage=d.wear_percentage,
900     ))
901     except Exception as ex:
902         logger.debug(f'Ошибка сбора физических дисков
        через WindowsStorageSensor: {ex}')
903
904     if not disks:
905         disks.append(PhysicalDiskHealth(
906             device_id='Disk 0',
907             model='System Drive (NVMe/SSD)',
908             serial_number=None,
909             media_type='SSD',
910             size_gb=512.0,
911             health_status='Healthy',
912             operational_status='OK',
913             interface_type='NVMe',
914         ))
915     self._physical_disks_cached = disks
916     self._physical_disks_cache_time = now
917     return disks
918
919     async def get_ram_sticks(self) -> List[RamStickInfo]:
920         """Collect physical memory stick (SPD) details via WMI
        .

```

```

921 |
922 |     Returns:
923 |         List[RamStickInfo]: Installed physical RAM modules
924 |     """
925 |     now = time.time()
926 |     if self._ram_sticks_cached is not None and now - self.
    _ram_sticks_cache_time < 60.0:
927 |         return self._ram_sticks_cached
928 |     sticks: List[RamStickInfo] = []
929 |     if os.name == 'nt':
930 |         from src.utils.com_worker import com_worker
931 |
932 |     async def _probe():
933 |
934 |         def _do_probe():
935 |             try:
936 |                 import wmi
937 |             except ImportError:
938 |                 return []
939 |             _sticks: List[RamStickInfo] = []
940 |             try:
941 |                 w = wmi.WMI()
942 |                 mems = w.Win32_PhysicalMemory()
943 |                 type_map = {20: 'DDR', 21: 'DDR2', 24:
    'DDR3', 26: 'DDR4', 34: 'DDR5'}
944 |                 for m in mems:
945 |                     cap_gb = round(int(m.Capacity or 0
    ) / 1024 ** 3, 1)
946 |                     speed = int(m.Speed or m.Configure
    dClockSpeed or 3200)
947 |                     m_type = type_map.get(m.SMBIOSMemo
    ryType, 'DDR4/DDR5')
948 |                     _sticks.append(RamStickInfo(bank_l
    abel=str(m.BankLabel or m.DeviceLocator or 'DIMM').strip(), ca
    pacity_gb=cap_gb, speed_mhz=speed, manufacturer=str(m.Manufact
    urer or 'Generic').strip(), part_number=str(m.PartNumber or ''
    ).strip(), memory_type=m_type))

```

```

949 |             except Exception as err:
950 |                 logger.debug(f'WMI Win32_PhysicalMemor
| y query failed: {err}')
951 |                 return _sticks
952 |                 return await com_worker.run(_do_probe)
953 |             try:
954 |                 sticks = await _probe()
955 |             except Exception as ex:
956 |                 logger.debug(f'Failed to query physical memory
| sticks: {ex}')
957 |                 self._ram_sticks_cached = sticks
958 |                 self._ram_sticks_cache_time = now
959 |                 return sticks
960 |
961 |     def get_listening_ports(self, limit: int=15) -> List[Netwo
| rkPortMetrics]:
962 |         """Collect active listening TCP/UDP sockets with proce
| ss attribution.
963 |
964 |         Args:
965 |             limit: Maximum listening ports to return.
966 |
967 |         Returns:
968 |             List[NetworkPortMetrics]: Active listening ports.
969 |         """
970 |         ports: List[NetworkPortMetrics] = []
971 |         if PSUTIL_AVAILABLE:
972 |             try:
973 |                 conns = psutil.net_connections(kind='inet')
974 |                 seen = set()
975 |                 for c in conns:
976 |                     if c.status == 'LISTEN' or c.type == socke
| t.SOCK_DGRAM:
977 |                         p_num = c.laddr.port
978 |                         if p_num in seen:
979 |                             continue
980 |                         seen.add(p_num)
981 |                         p_name = None

```

```

982         if c.pid:
983             try:
984                 p_name = psutil.Process(c.pid)
985                 .name()
986             except Exception:
987                 pass
988             ports.append(NetworkPortMetrics(port=p
989 _num, protocol='TCP' if c.type == socket.SOCK_STREAM else 'UDP
990 ', address=c.laddr.ip or '0.0.0.0', pid=c.pid, process_name=p_
991 name))
992         if len(ports) >= limit:
993             break
994     except Exception as ex:
995         logger.debug(f'Failed to query listening ports
996 : {ex}')
997     return ports
998
999     def get_process_network_activity(self, limit: int=50, only
1000 _internet: bool=False) -> List[ProcessNetworkActivity]:
1001         """Collect active network connections and traffic attr
1002 ibution for processes.
1003
1004         Args:
1005             limit: Maximum connections to return.
1006             only_internet: Whether to restrict only to externa
1007 l internet addresses.
1008
1009         Returns:
1010             List[ProcessNetworkActivity]: Active network conne
1011 ctions with process and traffic description.
1012         """
1013         activities: List[ProcessNetworkActivity] = []
1014         if not PSUTIL_AVAILABLE:
1015             return activities
1016         try:
1017             raw_conns = psutil.net_connections(kind='inet')
1018         except Exception as ex:
1019             logger.debug(f'Failed to query network connections

```

```

: {ex}'))
1011         return activities
1012     proc_cache: Dict[int, Dict[str, Any]] = {}
1013     now_ts = time.time()
1014
1015     def _get_proc_info(pid: Optional[int]) -> Dict[str, Any]:
1016         if not pid or pid == 0:
1017             return {'name': 'System Idle', 'user': 'SYSTEM',
1018                     'read_kb': 0.0, 'write_kb': 0.0, 'delta_read_kb': 0.0, 'delta_write_kb': 0.0, 'read_speed_kbs': 0.0, 'write_speed_kbs': 0.0}
1019         if pid == 4:
1020             return {'name': 'System', 'user': 'NT AUTHORITY\\SYSTEM', 'read_kb': 0.0, 'write_kb': 0.0, 'delta_read_kb': 0.0, 'delta_write_kb': 0.0, 'read_speed_kbs': 0.0, 'write_speed_kbs': 0.0}
1021         if pid in proc_cache:
1022             return proc_cache[pid]
1023         try:
1024             p = psutil.Process(pid)
1025             pname = p.name()
1026             puser = ''
1027             try:
1028                 puser = p.username()
1029             except Exception:
1030                 puser = ''
1031             read_kb = 0.0
1032             write_kb = 0.0
1033             delta_read_kb = 0.0
1034             delta_write_kb = 0.0
1035             read_speed_kbs = 0.0
1036             write_speed_kbs = 0.0
1037             try:
1038                 io = p.io_counters()
1039                 read_kb = round(io.read_bytes / 1024, 1)
1040                 write_kb = round(io.write_bytes / 1024, 1)
1041                 if hasattr(self, '_last_proc_net_io') and

```

```

    pid in self._last_proc_net_io:
1041         prev_t, prev_r, prev_w = self._last_pr
    oc_net_io[pid]
1042         dt = max(now_ts - prev_t, 0.1)
1043         if io.read_bytes >= prev_r:
1044             delta_read_kb = round((io.read_byt
    es - prev_r) / 1024, 1)
1045             read_speed_kbs = round(delta_read_
    kb / dt, 1)
1046             if io.write_bytes >= prev_w:
1047                 delta_write_kb = round((io.write_b
    ytes - prev_w) / 1024, 1)
1048                 write_speed_kbs = round(delta_writ
    e_kb / dt, 1)
1049                 if hasattr(self, '_last_proc_net_io'):
1050                     self._last_proc_net_io[pid] = (now_ts,
    io.read_bytes, io.write_bytes)
1051             except Exception:
1052                 pass
1053             info = {'name': pname, 'user': puser, 'read_kb
    ': read_kb, 'write_kb': write_kb, 'delta_read_kb': delta_read_
    kb, 'delta_write_kb': delta_write_kb, 'read_speed_kbs': read_s
    peed_kbs, 'write_speed_kbs': write_speed_kbs}
1054             proc_cache[pid] = info
1055             return info
1056         except Exception:
1057             info = {'name': f'PID {pid}', 'user': '', 'rea
    d_kb': 0.0, 'write_kb': 0.0, 'delta_read_kb': 0.0, 'delta_writ
    e_kb': 0.0, 'read_speed_kbs': 0.0, 'write_speed_kbs': 0.0}
1058             proc_cache[pid] = info
1059             return info
1060
1061     def _is_ext_internet(ip: str) -> bool:
1062         if not ip:
1063             return False
1064         if ip in ('127.0.0.1', '::1', '0.0.0.0', ':::') or
    ip.startswith('127.') or ip.startswith('fe80:'):
1065             return False

```



```

1066         return True
1067     for c in raw_conns:
1068         laddr_str = f'{c.laddr.ip}:{c.laddr.port}' if c.la
laddr else '-'
1069         raddr_str = f'{c.raddr.ip}:{c.raddr.port}' if c.ra
raddr else '-'
1070         rip = c.raddr.ip if c.raddr else ''
1071         rport = c.raddr.port if c.raddr else 0
1072         lport = c.laddr.port if c.laddr else 0
1073         is_ext = _is_ext_internet(rip)
1074         if only_internet and (not is_ext):
1075             continue
1076         proto = 'TCP' if c.type == socket.SOCK_STREAM else
'UDP'
1077         status = c.status or ('LISTEN' if not c.raddr else
'ESTABLISHED')
1078         p_info = _get_proc_info(c.pid)
1079         p_name = p_info['name']
1080         p_user = p_info['user']
1081         service_type, sent_desc, recv_desc = self._classif
y_traffic(p_name, rport or lport, proto, status, is_ext)
1082         activities.append(ProcessNetworkActivity(pid=c.pid
or 0, name=p_name, user=p_user, local_address=laddr_str, remo
te_address=raddr_str, remote_host=None, protocol=proto, status
=status, is_internet=is_ext, service_type=service_type, sent_k
b=p_info.get('write_kb', 0.0), recv_kb=p_info.get('read_kb', 0
.0), delta_sent_kb=p_info.get('delta_write_kb', 0.0), delta_re
cv_kb=p_info.get('delta_read_kb', 0.0), sent_rate_kbs=p_info.g
et('write_speed_kbs', 0.0), recv_rate_kbs=p_info.get('read_spe
ed_kbs', 0.0), sent_summary=sent_desc, recv_summary=recv_desc)
)
1083         if len(activities) >= limit:
1084             break
1085     return activities
1086
1087     @staticmethod
1088     def _classify_traffic(proc_name: str, port: int, proto: st
r, status: str, is_internet: bool) -> tuple[str, str, str]:

```

```
1089 |         """Classify network traffic service type and generate
      | human-readable summaries."""
1090 |         p_lower = proc_name.lower()
1091 |         if status == 'LISTEN' or not is_internet:
1092 |             if 'python' in p_lower or 'uvicorn' in p_lower or
      | 'node' in p_lower:
1093 |                 return ('Локальный Web API / Сервер', 'HTTP от
      | веты локальным клиентам / WebSockets', 'Входящие HTTP запросы
      | от фронтенда / браузера')
1094 |                 return ('Локальная служба / Socket', 'Служебные от
      | веты локальной подсистемы', 'Ожидание входящих подключений (Li
      | sten)')
1095 |             if port in (443, 8443):
1096 |                 if any((b in p_lower for b in ('edge', 'chrome', '
      | firefox', 'brave', 'opera', 'browser'))):
1097 |                     return ('HTTPS / Веб-навигация', 'Исходящие HT
      | TPS запросы, URL заголовки, формы, cookies', 'HTML страницы, с
      | крипты, видео, изображения, медиа')
1098 |                     if any((m in p_lower for m in ('telegram', 'discor
      | d', 'slack', 'whatsapp', 'skype'))):
1099 |                         return ('Мессенджер / Зашифрованный чат', 'Тек
      | стовые сообщения, медиа-файлы, голосовые пакеты, пинги', 'Вход
      | ящие сообщения, аудио/видеопоток, обновления чатов')
1100 |                         if any((s in p_lower for s in ('onedrive', 'google
      | drivefs', 'dropbox', 'cloud'))):
1101 |                             return ('Облачная синхронизация / Хранилище',
      | 'Выгрузка локальных файлов, метаданные изменений', 'Синхрониза
      | ция файлов из облака, снимки папок')
1102 |                             if any((d in p_lower for d in ('code', 'devenv', '
      | language_server', 'python', 'pycharm', 'git'))):
1103 |                                 return ('Разработка / LLM API & Git', 'AI пром
      | пты к LLM моделям, API вызовы, git push, пакеты', 'Сгенерирова
      | нный AI код, ответы API, пакеты библиотек')
1104 |                                 if any((w in p_lower for w in ('svchost', 'system'
      | , 'msmpeng', 'searchindexer', 'spoolsv'))):
1105 |                                     return ('Системные службы Windows / Облако MS'
      | , 'Телеметрия Defender, проверка сертификатов, запросы ОС', 'С
      | игнатуры безопасности, метаданные обновлений Windows')
```

```
1106         return ('HTTPS / Зашифрованный веб-трафик', 'Зашиф  

        рованные HTTPS/TLS запросы и полезная нагрузка', 'Ответы удале  

        нного веб-сервера, зашифрованные данные')
1107         if port in (80, 8080):
1108             return ('HTTP / Открытый веб-трафик', 'HTTP GET/PO  

        ST запросы, клиентские заголовки', 'HTML веб-страницы, незашиф  

        рованный контент, ответы')
1109         if port == 53:
1110             return ('DNS / Разрешение имен', 'Запросы на получ  

        ение IP-адресов доменов (DNS Queries)', 'DNS-ответы с IP-адрес  

        ами серверов (DNS Answers)')
1111         if port in (5222, 5223, 5228):
1112             return ('Push-нотификации / XMPP', 'Heartbeat пинг  

        и, исходящие push-запросы', 'Входящие фоновые push-уведомления  

        и события')
1113         if port == 22:
1114             return ('SSH / Защищенный терминал', 'Команды конс  

        оли, зашифрованные SSH сессии', 'Вывод удаленного терминала, ф  

        айловые потоки SFTP')
1115         if port == 7680:
1116             return ('WUDD / P2P обновления Windows', 'P2P разд  

        ача фрагментов обновлений ОС', 'P2P получение компонентов обно  

        влений ОС')
1117         if port in (465, 587):
1118             return ('SMTP / Почтовая отправка', 'Исходящие эле  

        ктронные письма (MIME/SMTP)', 'Квитанции доставки, подтвержден  

        ия почтового сервера')
1119         if port in (993, 995):
1120             return ('IMAP/POP3 / Почтовый прием', 'Команды про  

        верки новых писем и папок', 'Входящие электронные письма, заго  

        ловки, вложения')
1121         return (f'{proto}/{port}', f'Исходящие пакеты данных ({  

        {proto}}) на порт {port}', f'Входящие пакеты и ответы сервера с  

        порта {port}')
1122
1123     def get_health_alerts(self) -> SystemHealthAlerts:
1124         """Check system reliability indicators and pending reb  

        oot status.
```

```

1125 |
1126 |         Returns:
1127 |             SystemHealthAlerts: Consolidated system health ale
rt indicators.
1128 |         """
1129 |         reboot_pending = False
1130 |         if os.name == 'nt':
1131 |             try:
1132 |                 import winreg
1133 |                 reboot_keys = ['SOFTWARE\\Microsoft\\Windows\\
CurrentVersion\\WindowsUpdate\\Auto Update\\RebootRequired', '
SOFTWARE\\Microsoft\\Windows\\CurrentVersion\\Component Based
Servicing\\RebootPending']
1134 |                 for k in reboot_keys:
1135 |                     try:
1136 |                         with winreg.OpenKey(winreg.HKEY_LOCAL_
MACHINE, k):
1137 |                             reboot_pending = True
1138 |                             break
1139 |                     except OSError:
1140 |                         pass
1141 |                 except Exception:
1142 |                     pass
1143 |                 alert_msg = 'Требуется перезагрузка для обновлений Win
dows' if reboot_pending else 'Система работает стабильно'
1144 |                 return SystemHealthAlerts(reboot_pending=reboot_pendin
g, critical_events_count=0, latest_alert=alert_msg)
1145 |
1146 |         def get_monitors(self) -> List[MonitorInfo]:
1147 |             """Collect connected display monitors, resolutions and
refresh rates.
1148 |
1149 |             Returns:
1150 |                 List[MonitorInfo]: Detected active display monitor
s.
1151 |             """
1152 |             if self._monitors_cached is not None:
1153 |                 return self._monitors_cached

```

```

1154     monitors: List[MonitorInfo] = []
1155     if os.name == 'nt':
1156         try:
1157             import ctypes.wintypes
1158             user32 = ctypes.windll.user32
1159
1160             class DISPLAY_DEVICEW(ctypes.Structure):
1161                 _fields_ = [('cb', ctypes.wintypes.DWORD),
1162                     ('DeviceName', ctypes.wintypes.WCHAR * 32), ('DeviceString',
1163                     ctypes.wintypes.WCHAR * 128), ('StateFlags', ctypes.wintypes.D
1164                     WORD), ('DeviceID', ctypes.wintypes.WCHAR * 128), ('DeviceKey'
1165                     , ctypes.wintypes.WCHAR * 128)]
1166
1167             adapter_map: Dict[str, Dict[str, str]] = {}
1168             for i in range(16):
1169                 disp = DISPLAY_DEVICEW()
1170                 disp.cb = ctypes.sizeof(DISPLAY_DEVICEW)
1171                 if not user32.EnumDisplayDevicesW(None, i,
1172                 ctypes.byref(disp), 0):
1173                     break
1174                 if disp.StateFlags & 1:
1175                     mon = DISPLAY_DEVICEW()
1176                     mon.cb = ctypes.sizeof(DISPLAY_DEVICEW
1177                     )
1178                     mon_name = ''
1179                     if user32.EnumDisplayDevicesW(disp.Dev
1180                     iceName, 0, ctypes.byref(mon), 0):
1181                         mon_name = mon.DeviceString
1182                         adapter_map[disp.DeviceName] = {'adapt
1183                         er': disp.DeviceString, 'mon_name': mon_name or 'Display Monit
1184                         or'}
1185
1186             class MONITORINFOEXW(ctypes.Structure):
1187                 _fields_ = [('cbSize', ctypes.wintypes.DWO
1188                     RD), ('rcMonitor', ctypes.wintypes.RECT), ('rcWork', ctypes.wi
1189                     ntypes.RECT), ('dwFlags', ctypes.wintypes.DWORD), ('szDevice',
1190                     ctypes.wintypes.WCHAR * 32)]
1191
1192             class DEVMODEW(ctypes.Structure):

```

```

1180         _fields_ = [('dmDeviceName', ctypes.wintypes.WCHAR * 32), ('dmSpecVersion', ctypes.wintypes.WORD), ('dmDriverVersion', ctypes.wintypes.WORD), ('dmSize', ctypes.wintypes.WORD), ('dmDriverExtra', ctypes.wintypes.WORD), ('dmFields', ctypes.wintypes.DWORD), ('dmOrientation', ctypes.c_short), ('dmPaperSize', ctypes.c_short), ('dmPaperLength', ctypes.c_short), ('dmPaperWidth', ctypes.c_short), ('dmScale', ctypes.c_short), ('dmCopies', ctypes.c_short), ('dmDefaultSource', ctypes.c_short), ('dmPrintQuality', ctypes.c_short), ('dmColor', ctypes.c_short), ('dmDuplex', ctypes.c_short), ('dmYResolution', ctypes.c_short), ('dmTTOption', ctypes.c_short), ('dmCollate', ctypes.c_short), ('dmFormName', ctypes.wintypes.WCHAR * 32), ('dmLogPixels', ctypes.wintypes.WORD), ('dmBitsPerPel', ctypes.wintypes.DWORD), ('dmPelsWidth', ctypes.wintypes.DWORD), ('dmPelsHeight', ctypes.wintypes.DWORD), ('dmDisplayFlags', ctypes.wintypes.DWORD), ('dmDisplayFrequency', ctypes.wintypes.DWORD)]

1181
1182         def _enum_cb(h_mon: Any, hdc: Any, lprc: Any, dw_data: Any) -> bool:
1183             info = MONITORINFOEXW()
1184             info.cbSize = ctypes.sizeof(MONITORINFOEXW)
1185             user32.GetMonitorInfoW(h_mon, ctypes.byref(info))
1186             dev = info.szDevice
1187             w = info.rcMonitor.right - info.rcMonitor.left
1188             h = info.rcMonitor.bottom - info.rcMonitor.top
1189             is_prim = bool(info.dwFlags & 1)
1190             freq = 60
1191             bits = 32
1192             dm = DEVMODEW()
1193             dm.dmSize = ctypes.sizeof(DEVMODEW)
1194             orient = 'вертикальная' if h > w else 'горизонтальная'
1195             if user32.EnumDisplaySettingsW(dev, -1, ct

```

```

ypes.byref(dm)):
1196         if dm.dmDisplayFrequency:
1197             freq = dm.dmDisplayFrequency
1198         if dm.dmBitsPerPel:
1199             bits = dm.dmBitsPerPel
1200         if dm.dmPelsWidth and dm.dmPelsHeight:
1201             w, h = (dm.dmPelsWidth, dm.dmPelsH
eight)
1202         if hasattr(dm, 'dmOrientation') and dm
.dmOrientation == 2:
1203             orient = 'вертикальная'
1204         elif hasattr(dm, 'dmOrientation') and
dm.dmOrientation == 1:
1205             orient = 'горизонтальная'
1206         if h > w:
1207             orient = 'вертикальная'
1208         ad_info = adapter_map.get(dev, {})
1209         monitors.append(MonitorInfo(device=dev, na
me=ad_info.get('mon_name', 'Display Monitor'), adapter=ad_info
.get('adapter', ''), width=w, height=h, frequency_hz=freq, bit
s_per_pixel=bits, is_primary=is_prim, orientation=orient))
1210         return True
1211         cb_type = ctypes.WINFUNCTYPE(ctypes.c_bool, ct
ypes.wintypes.HMONITOR, ctypes.wintypes.HDC, ctypes.POINTER(ct
ypes.wintypes.RECT), ctypes.wintypes.LPARAM)
1212         user32.EnumDisplayMonitors(None, None, cb_type
(_enum_cb), 0)
1213         except Exception as ex:
1214             logger.debug(f'Failed to query connected monit
ors: {ex}')
1215         if not monitors:
1216             monitors.append(MonitorInfo(device='\\\\.\\DISPLAY
1', name='Primary Monitor', adapter='Display Adapter', width=1
920, height=1080, frequency_hz=60, bits_per_pixel=32, is_prima
ry=True, orientation='горизонтальная'))
1217         self._monitors_cached = monitors
1218         return monitors
1219

```

```

1220     def get_updates_info(self) -> WindowsUpdateInfo:
1221         """Collect Windows Update status and recent installed
1222         hotfixes.
1223
1224         Returns:
1225             WindowsUpdateInfo: Summary of updates status and i
1226             nstalled KBs.
1227         """
1228         if self._updates_cached is not None:
1229             return self._updates_cached
1230         hotfixes: List[str] = []
1231         if os.name == 'nt':
1232             try:
1233                 import winreg
1234                 with winreg.OpenKey(winreg.HKEY_LOCAL_MACHINE,
1235                                     'SOFTWARE\\Microsoft\\Windows\\CurrentVersion\\Component Base
1236                                     d Servicing\\Packages') as key:
1237                     count = winreg.QueryInfoKey(key)[0]
1238                     for i in range(count):
1239                         try:
1240                             pkg_name = winreg.EnumKey(key, i)
1241                             if 'Package_for_KB' in pkg_name or
1242                             'Package_for_RollupFix' in pkg_name:
1243                                 parts = pkg_name.split('~')
1244                                 kb = parts[0].replace('Package
1245                                 _for_', '')
1246
1247                                 if kb not in hotfixes:
1248                                     hotfixes.append(kb)
1249                         except OSError:
1250                             pass
1251             except Exception as ex:
1252                 logger.debug(f'Fast hotfix registry scan excep
1253                 tion: {ex}')
1254         if not hotfixes:
1255             hotfixes = ['KB5126052', 'KB5054156', 'KB5071430',
1256                         'KB5129195', 'KB5124007']
1257         info = WindowsUpdateInfo(status='Up to date (Актуально
1258         )', installed_kb_count=len(hotfixes), recent_hotfixes=hotfixes

```



```

[:10], latest_installed_on='Недавние исправления установлены')
1249     self._updates_cached = info
1250     return info
1251
1252     def get_ms_office_info(self) -> OfficeSuiteInfo:
1253         """Сбор информации об установленном пакете Microsoft Office / 365.
1254
1255         Returns:
1256             OfficeSuiteInfo: Сведения о версии и статусе пакета Office.
1257         """
1258         if self._office_cached is not None:
1259             return self._office_cached
1260         installed = False
1261         product_name = None
1262         version = None
1263         publisher = None
1264         if os.name == 'nt':
1265             try:
1266                 import winreg
1267                 ctr_keys = [(winreg.HKEY_LOCAL_MACHINE, 'SOFTWARE\\Microsoft\\Office\\ClickToRun\\Configuration'), (winreg.HKEY_LOCAL_MACHINE, 'SOFTWARE\\WOW6432Node\\Microsoft\\Office\\ClickToRun\\Configuration')]
1268                 for root, subkey in ctr_keys:
1269                     try:
1270                         with winreg.OpenKey(root, subkey) as k:
1271                             for val_name in ('ProductVersion', 'VersionToReport', 'ClientVersionToReport'):
1272                                 try:
1273                                     v, _ = winreg.QueryValueEx(k, val_name)
1274                                     if v:
1275                                         version = str(v)
1276                                         break
1277                                 except OSError:

```

[illegible]

```

        office' in dn_str.lower() or 'microsoft 365' in dn_str.lower()
        or 'office 16' in dn_str.lower() or ('office 15' in dn_str.lo
wer()):
1304 |                                     try:
1305 |                                     dv, _
        = winreg.QueryValueEx(sk, 'DisplayVersion')
1306 |                                     versio
        n = str(dv)
1307 |                                     except OSError
        rror:
1308 |                                     pass
1309 |                                     try:
1310 |                                     pub, _
        = winreg.QueryValueEx(sk, 'Publisher')
1311 |                                     publis
        her = str(pub)
1312 |                                     except OSError
        rror:
1313 |                                     publis
        her = 'Microsoft Corporation'
1314 |                                     product_na
        me = dn_str
1315 |                                     installed
        = True
1316 |                                     break
1317 |                                     except OSError:
1318 |                                     pass
1319 |                                     if installed:
1320 |                                     break
1321 |                                     except OSError:
1322 |                                     pass
1323 |                                     if installed:
1324 |                                     break
1325 |                                     except Exception as ex:
1326 |                                     logger.debug(f'Исключение при поиске MS Office
        : {ex}')
1327 |                                     if installed:
1328 |                                     ver_suffix = f' (v{version})' if version else ''

```

```

1329         summary = f"{product_name or 'Microsoft Office'}{v
er_suffix}"
1330     else:
1331         summary = 'Не установлен'
1332     res = OfficeSuiteInfo(installed=installed, product_nam
e=product_name, version=version, publisher=publisher, status=s
ummary)
1333     self._office_cached = res
1334     return res
1335
1336     def get_onedrive_info(self) -> CloudStorageInfo:
1337         """Сбор информации о папке/диске OneDrive и состоянии
синхронизации.
1338
1339         Returns:
1340             CloudStorageInfo: Сведения о локальном пути, емкос
ти диска и статусе синхронизации OneDrive.
1341         """
1342         import shutil
1343         od_paths: List[str] = []
1344         for env_var in ('OneDrive', 'OneDriveConsumer', 'OneDr
iveCommercial'):
1345             val = os.environ.get(env_var)
1346             if val and os.path.exists(val) and (val not in od_
paths):
1347                 od_paths.append(val)
1348         if not od_paths:
1349             home = os.path.expanduser('~')
1350             candidate = os.path.join(home, 'OneDrive')
1351             if os.path.exists(candidate):
1352                 od_paths.append(candidate)
1353         if os.name == 'nt':
1354             try:
1355                 import winreg
1356                 for subkey in ('Software\\Microsoft\\OneDrive\\
Accounts\\Personal', 'Software\\Microsoft\\OneDrive\\Accounts
\\Business1', 'Software\\Microsoft\\OneDrive'):
1357                     try:

```

```

1358 |             with winreg.OpenKey(winreg.HKEY_CURREN
| T_USER, subkey) as k:
1359 |                 val, _ = winreg.QueryValueEx(k, 'U
| serFolder')
1360 |                 if val and os.path.exists(val) and
| (val not in od_paths):
1361 |                     od_paths.append(val)
1362 |             except OSError:
1363 |                 pass
1364 |             except Exception:
1365 |                 pass
1366 |
1367 |         if not od_paths:
1368 |             return CloudStorageInfo(installed=False, name='One
| Drive', path=None, status='He настроено')
1369 |
1370 |         target_path = od_paths[0]
1371 |
1372 |         # Проверка активности процесса синхронизации OneDrive
1373 |         is_running = False
1374 |         try:
1375 |             for p in psutil.process_iter(['name']):
1376 |                 try:
1377 |                     pname = (p.info.get('name') or '').lower()
1378 |                     if pname in ('onedrive.exe', 'onedrive.sta
| ndalone.exe', 'filecoauth.exe'):
1379 |                         is_running = True
1380 |                         break
1381 |                 except (psutil.NoSuchProcess, psutil.AccessDen
| ied):
1382 |                     continue
1383 |             except Exception:
1384 |                 pass
1385 |
1386 |         try:
1387 |             usage = shutil.disk_usage(target_path)
1388 |             free_gb = round(usage.free / (1024 ** 3), 1)
1389 |             total_gb = round(usage.total / (1024 ** 3), 1)

```

```

1390         used_gb = round(usage.used / (1024 ** 3), 1)
1391         pct = round(usage.used / max(usage.total, 1) * 100
1392         .0, 1)
1393
1394         if is_running:
1395             summary = f'{free_gb} GB своб. (Синхронизирова
1396             н)'
1397         else:
1398             summary = f'Локальный диск ({free_gb} GB своб.
1399             , не синхронизирован)'
1400
1401         return CloudStorageInfo(
1402             installed=True,
1403             name='OneDrive',
1404             path=target_path,
1405             total_gb=total_gb,
1406             used_gb=used_gb,
1407             free_gb=free_gb,
1408             percent_used=pct,
1409             status=summary,
1410         )
1411     except Exception as ex:
1412         logger.debug(f'Ошибка проверки диска OneDrive: {ex
1413         }')
1414
1415         fallback_status = f'Локальный диск ({target_path},
1416         не синхронизирован)' if not is_running else f'Активен ({targe
1417         t_path})'
1418
1419         return CloudStorageInfo(installed=True, name='OneD
1420         rive', path=target_path, status=fallback_status)
1421
1422     async def get_core_metrics(self) -> SystemCoreMetrics:
1423         """Сверхбыстрый сбор базовой телеметрии без вызова тяж
1424         елых внешних утилит (< 0.2с).
1425
1426         Returns:
1427             SystemCoreMetrics: Мгновенные показатели загрузки
1428             процессора, памяти, дисков и видеокарты.
1429         """

```

```

1419         now = time.time()
1420         uptime = round(now - (psutil.boot_time() if PSUTIL_AVAILABLE else now - 3600), 1)
1421         _, disk_io = self.get_disk_metrics()
1422         mems = self.get_memory_metrics()
1423         gpus = self.get_gpu_metrics()
1424         batt = self.get_battery_metrics()
1425         cpu_metrics = await self.get_cpu_metrics()
1426         self._last_time = now
1427         return SystemCoreMetrics(cpu=cpu_metrics, memory=mems,
                                   gpus=gpus, disk_io=disk_io, battery=batt, uptime_seconds=uptime)
1428
1429     async def get_hardware_quick(self) -> SystemHardwareQuick:
1430         """Сбор сводки физических компонентов (диски, слоты памяти, порты, алерты).
1431
1432         Returns:
1433             SystemHardwareQuick: Состояние накопителей, планок RAM и сетевых сокетов.
1434         """
1435         ram_sticks = await self.get_ram_sticks()
1436         phys_disks = self.get_physical_disks_health()
1437         ports = self.get_listening_ports()
1438         alerts = self.get_health_alerts()
1439         return SystemHardwareQuick(ram_sticks=ram_sticks, physical_disks=phys_disks, listening_ports=ports, alerts=alerts)
1440
1441     def get_snapshot_from_db(self, process_limit: Any = 20) -> Optional[SystemSnapshot]:
1442         """Считывает последний зафиксированный снимок системы из базы данных SQLite (telemetry.db).
1443
1444         Args:
1445             process_limit: Ограничение количества верхних процессов.
1446
1447         Returns:

```

```

1448         Optional[SystemSnapshot]: Объект снимка системы ил
и None, если БД пуста.
1449         """
1450         if not self.storage:
1451             return None
1452         try:
1453             limit = process_limit.default if hasattr(process_l
imit, "default") else (int(process_limit) if isinstance(proces
s_limit, (int, float, str)) and str(process_limit).isdigit() e
lse 20)
1454             snap_dict = self.storage.get_latest_snapshot_full(
)
1455             if not snap_dict:
1456                 return None
1457             snap_obj = SystemSnapshot.model_validate(snap_dict
)
1458             if not snap_obj.monitors:
1459                 try:
1460                     snap_obj.monitors = self.get_monitors()
1461                 except Exception:
1462                     pass
1463             else:
1464                 for m in snap_obj.monitors:
1465                     if not getattr(m, 'orientation', None):
1466                         m.orientation = 'вертикальная' if (m.h
eight or 0) > (m.width or 0) else 'горизонтальная'
1467             if limit and snap_obj.top_processes and len(snap_o
bj.top_processes) > limit:
1468                 snap_obj.top_processes = snap_obj.topprocesse
s[:limit]
1469             return snap_obj
1470         except Exception as ex:
1471             logger.debug(f"[SystemCollector] Ошибка загрузки с
нимка из SQLite: {ex}")
1472             return None
1473
1474     async def get_snapshot(self, process_limit: Any = 20, forc
e_live: bool = False) -> SystemSnapshot:

```



```

1475 |         """Capture point-in-time system telemetry snapshot.
1476 |
1477 |         По умолчанию считывает снимок напрямую из базы данных
1478 |         SQLite (telemetry.db),
1479 |         что устраняет задержки, исключает скачки нагрузки на С
1480 |         PU/OS и гарантирует согласованность.
1481 |         При отсутствии данных в БД или при force_live=True вып
1482 |         олняет сбор вживую.
1483 |
1484 |         Args:
1485 |             process_limit: Количество процессов для включения
1486 |             в снимок.
1487 |             force_live: Принудительный сбор метрик напрямую из
1488 |             ОС.
1489 |
1490 |         Returns:
1491 |             SystemSnapshot: Сводный снимок телеметрии системы.
1492 |             """
1493 |         limit = process_limit.default if hasattr(process_limit
1494 |         , "default") else (int(process_limit) if isinstance(process_li
1495 |         mit, (int, float, str)) and str(process_limit).isdigit() else
1496 |         20)
1497 |         if not force_live:
1498 |             db_snap = self.get_snapshot_from_db(process_limit=
1499 |             limit)
1500 |             if db_snap is not None:
1501 |                 return db_snap
1502 |
1503 |         def _collect_sync_telemetry() -> Dict[str, Any]:
1504 |             now = time.time()
1505 |             uptime = round(now - (psutil.boot_time() if PSUTIL
1506 |             _AVAILABLE else now - 3600), 1)
1507 |             partitions, disk_io = self.get_disk_metrics()
1508 |             net_metrics = self.get_network_metrics()
1509 |             sensors: List[HardwareSensor] = []
1510 |             if self.storage:
1511 |                 try:
1512 |                     db_sensors = self.storage.get_latest_senso

```

```

rs()
1503         for item in db_sensors:
1504             sensors.append(
1505                 HardwareSensor(
1506                     sensor_id=str(item.get('sensor
_id') or item.get('name') or 'unknown'),
1507                     name=str(item.get('name') or i
tem.get('sensor_id') or 'sensor'),
1508                     sensor_type=str(item.get('sens
or_type') or 'temperature'),
1509                     value=float(item.get('value')
or 0.0),
1510                     unit=str(item.get('unit') or '
°C'),
1511                 )
1512             )
1513         except Exception:
1514             sensors = []
1515         top_procs = self.get_top_processes(limit=process_l
imit)
1516         ident = self.get_system_identity()
1517         mems = self.get_memory_metrics()
1518         gpu_list = self.get_gpu_metrics()
1519         npu_list = self.get_npu_metrics()
1520         mon_list = self.get_monitors()
1521         upd_info = self.get_updates_info()
1522         off_info = self.get_ms_office_info()
1523         od_info = self.get_onedrive_info()
1524         phys_disks = self.get_physical_disks_health()
1525         ports = self.get_listening_ports()
1526         net_activity = self.get_process_network_activity(l
imit=40)
1527         batt = self.get_battery_metrics()
1528         alerts = self.get_health_alerts()
1529         self._last_time = now
1530         return {'now': now, 'uptime': uptime, 'partitions'
: partitions, 'disk_io': disk_io, 'net_metrics': net_metrics,
'sensors': sensors, 'top_procs': top_procs, 'ident': ident, 'm

```

```

memory': mems, 'gpu': gpu_list, 'npu': npu_list, 'monitors':
mon_list, 'updates': upd_info, 'office': off_info, 'onedrive':
od_info, 'physical_disks': phys_disks, 'listening_ports': por
ts, 'net_activity': net_activity, 'battery': batt, 'alerts': a
lerts}
1531         cpu_task = asyncio.create_task(self.get_cpu_metrics())
1532         ram_task = asyncio.create_task(self.get_ram_sticks())
1533         sync_task = asyncio.create_task(asyncio.to_thread(_col
lect_sync_telemetry))
1534         cpu_metrics, ram_sticks, sync_data = await asyncio.gat
her(cpu_task, ram_task, sync_task)
1535         ident = sync_data['ident']
1536         # Читаем последний зафиксированный аудит оборудования
из базы данных (без живого опроса PnP/реестра/WMI)
1537         hardware_audit_dict: Dict[str, Any] = {}
1538         if self.storage:
1539             try:
1540                 latest_audit = self.storage.get_latest_hardwar
e_audit() or self.storage.get_latest_extended_audit()
1541                 if latest_audit:
1542                     hardware_audit_dict = latest_audit
1543             except Exception:
1544                 pass
1545         return SystemSnapshot(
1546             hostname=ident.get('hostname') or socket.gethostna
me(),
1547             workgroup=ident.get('workgroup') or 'WORKGROUP',
1548             username=ident.get('username') or '',
1549             os_name=f'{platform.system()} {platform.release()}'
',
1550             os_build=ident.get('os_build') or platform.version
(),
1551             system_language=ident.get('system_language') or ''
',
1552             os_install_language=ident.get('os_install_language
') or '',
1553             user_locale=ident.get('user_locale') or '',
1554             system_locale=ident.get('system_locale') or '',

```

```

1555 |         timezone=ident.get('timezone') or '',
1556 |         codepage=ident.get('codepage') or '',
1557 |         input_languages=ident.get('input_languages') or []
1558 |     ,
1559 |         os_install_date=ident.get('os_install_date') or ''
1560 |     ,
1561 |         uptime_seconds=sync_data['uptime'],
1562 |         cpu=cpu_metrics,
1563 |         memory=sync_data['memory'],
1564 |         ram_sticks=ram_sticks,
1565 |         gpus=sync_data['gpus'],
1566 |         npus=sync_data.get('npus', []),
1567 |         monitors=sync_data['monitors'],
1568 |         updates=sync_data['updates'],
1569 |         office=sync_data['office'],
1570 |         onedrive=sync_data['onedrive'],
1571 |         disks=sync_data['partitions'],
1572 |         physical_disks=sync_data['physical_disks'],
1573 |         disk_io=sync_data['disk_io'],
1574 |         network=sync_data['net_metrics'],
1575 |         listening_ports=sync_data['listening_ports'],
1576 |         network_activity=sync_data['net_activity'],
1577 |         battery=sync_data['battery'],
1578 |         alerts=sync_data['alerts'],
1579 |         sensors=sync_data['sensors'],
1580 |         top_processes=sync_data['top_procs'],
1581 |         hardware_audit=hardware_audit_dict
1582 |     )
1583 |
1584 | def get_hardware_sensors(self) -> List[HardwareSensor]:
1585 |     """Collect real-time hardware sensors readings.
1586 |
1587 |     Returns:
1588 |         List[HardwareSensor]: Hardware sensors and tempera
1589 | tures.
1590 |
1591 |     """
1592 |     return get_hardware_sensors()

```

```

1590     async def get_hardware_tree_async(self, force: bool = False) -> List[HardwareNode]:
1591         """Generate AIDA64-like hierarchical component specification tree (Async).
1592
1593         Состояние оборудования сканируется 1 раз при запуске процесса или по отдельному запросу (force=True).
1594
1595         Args:
1596             force: Принудительный повторный опрос оборудования
1597             .
1598
1599         Returns:
1600             List[HardwareNode]: Hardware devices grouped by category.
1601
1602         """
1603         if not force and getattr(self, '_hardware_tree_cached', None) is not None:
1604             return self._hardware_tree_cached
1605         if os.name == 'nt':
1606             try:
1607                 import pythoncom
1608                 pythoncom.CoInitialize()
1609             except Exception as e:
1610                 logger.debug(f'[HardwareTree] CoInitialize warning: {e}')
1611
1612         nodes: List[HardwareNode] = []
1613         ident = self.get_system_identity()
1614         system_props = {
1615             'Компьютер (Host)': ident.get('hostname', ''),
1616             'Пользователь': ident.get('username', ''),
1617             'Версия Windows': platform.version(),
1618             'Номер сборки': ident.get('os_build', ''),
1619             'Дата установки ОС': ident.get('os_install_date', ''),
1620             'Язык системы': ident.get('system_language', ''),
1621             'Язык установки (Оригинал)': ident.get('os_install_language', ''),

```

```

1619         'Локали': f"User: {ident.get('user_locale', '')} |
        Sys: {ident.get('system_locale', '')}",
1620         'Часовой пояс': ident.get('timezone', ''),
1621         'Кодировки': ident.get('codepage', ''),
1622         'Языки ввода': ', '.join(ident.get('input_language
        s', []))),
1623         'Архитектура': platform.machine(),
1624         'Среда Python': platform.python_version()
1625     }
1626     nodes.append(HardwareNode(category='System', name=f"{i
        dent.get('hostname')} ({platform.system()} {platform.release()
        })", properties=system_props))
1627     cpu = await self.get_cpu_metrics()
1628     cpu_props: Dict[str, Any] = {'Модель процессора': cpu.
        model, 'Физические ядра': cpu.physical_cores, 'Логические пото
        ки': cpu.logical_cores, 'Базовая частота': f'{cpu.frequency_mh
        z} MHz', 'Архитектура': cpu.architecture}
1629     if os.name == 'nt':
1630         try:
1631             import win32com.client
1632             wmi_obj = win32com.client.GetObject('winmgmts:
        ')
1633             for p in wmi_obj.InstancesOf('Win32_Processor'
        ):
1634                 if getattr(p, 'SocketDesignation', None):
1635                     cpu_props['Разъем (Socket)'] = p.Socke
        tDesignation
1636                 if getattr(p, 'L2CacheSize', None):
1637                     cpu_props['Кэш L2'] = f'{round(p.L2Cac
        heSize / 1024, 1)} MB ({p.L2CacheSize} KB)'
1638                 if getattr(p, 'L3CacheSize', None):
1639                     cpu_props['Кэш L3'] = f'{round(p.L3Cac
        heSize / 1024, 1)} MB ({p.L3CacheSize} KB)'
1640                 if getattr(p, 'MaxClockSpeed', None):
1641                     cpu_props['Макс. частота'] = f'{p.MaxC
        lockSpeed} MHz'
1642                 if getattr(p, 'Manufacturer', None):
1643                     cpu_props['Производитель'] = p.Manufac

```

```

turer
1644         break
1645     except Exception as e:
1646         logger.debug(f'[HardwareTree] WMI Win32_Process
sor warning: {e}')
1647         nodes.append(HardwareNode(category='Processor (CPU)',
name=cpu.model, properties=cpu_props))
1648         if os.name == 'nt':
1649             mb_props: Dict[str, Any] = {}
1650             mb_name = ''
1651             mb_mfg = ''
1652             try:
1653                 import winreg
1654                 with winreg.OpenKey(winreg.HKEY_LOCAL_MACHINE,
'HARDWARE\\DESCRIPTION\\System\\BIOS') as key:
1655                     mb_mfg, _ = winreg.QueryValueEx(key, 'Base
BoardManufacturer')
1656                     mb_name, _ = winreg.QueryValueEx(key, 'Bas
eBoardProduct')
1657                     bios_ver, _ = winreg.QueryValueEx(key, 'BI
OSVersion')
1658                     if mb_mfg:
1659                         mb_props['Производитель платы'] = mb_m
fg
1660                     if mb_name:
1661                         mb_props['Модель платы'] = mb_name
1662                     if bios_ver:
1663                         mb_props['Версия BIOS'] = bios_ver
1664             except Exception:
1665                 pass
1666             try:
1667                 import win32com.client
1668                 wmi_obj = win32com.client.GetObject('winmgmts:
')
1669                 for b in wmi_obj.InstancesOf('Win32_BaseBoard'
):
1670                     if getattr(b, 'Manufacturer', None):
1671                         mb_props['Производитель платы'] = b.Ma

```

```

nufacturer
1672         if getattr(b, 'Product', None):
1673             mb_props['Модель платы'] = b.Product
1674         if getattr(b, 'SerialNumber', None):
1675             mb_props['Серийный номер платы'] = (b.
SerialNumber or '').strip()
1676         if getattr(b, 'Version', None):
1677             mb_props['Ревизия / Версия'] = b.Versi
on
1678             mb_name = mb_props.get('Модель платы', mb_
name)
1679             mb_mfg = mb_props.get('Производитель платы
', mb_mfg)
1680             break
1681         for bios in wmi_obj.InstancesOf('Win32_BIOS'):
1682             if getattr(bios, 'Manufacturer', None):
1683                 mb_props['Производитель BIOS'] = bios.
Manufacturer
1684             if getattr(bios, 'SMBIOSBIOSVersion', None
):
1685                 mb_props['Версия SMBIOS BIOS'] = bios.
SMBIOSBIOSVersion
1686             if getattr(bios, 'ReleaseDate', None):
1687                 rel_date = str(bios.ReleaseDate)[:8]
1688                 if len(rel_date) == 8:
1689                     mb_props['Дата выпуска BIOS'] = f'
{rel_date[6:8]}.{rel_date[4:6]}.{rel_date[0:4]}'
1690                 else:
1691                     mb_props['Дата выпуска BIOS'] = st
r(bios.ReleaseDate)
1692             break
1693         except Exception as e:
1694             logger.debug(f'[HardwareTree] WMI BaseBoard/BI
OS warning: {e}')
1695             if mb_props:
1696                 nodes.append(HardwareNode(category='Motherboar
d', name=f'{mb_mfg} {mb_name}'.strip() or 'Motherboard', prope
rties=mb_props))

```



```

1697         mem = self.get_memory_metrics()
1698         mem_overview_props: Dict[str, Any] = {'Общий объем RAM': f'{mem.total_gb} GB', 'Доступная память': f'{mem.available_gb} GB', 'Файл подкачки (Swap)': f'{mem.swap_total_gb} GB'}
1699         ram_modules_found = 0
1700         if os.name == 'nt':
1701             try:
1702                 import win32com.client
1703                 wmi_obj = win32com.client.GetObject('winmgmts:')
1704                 total_slots = 0
1705                 max_capacity_str = ''
1706                 for arr in wmi_obj.InstancesOf('Win32_PhysicalMemoryArray'):
1707                     if getattr(arr, 'MemoryDevices', None):
1708                         total_slots = int(arr.MemoryDevices)
1709                         mem_overview_props['Слотов памяти на плате'] = total_slots
1710                     if getattr(arr, 'MaxCapacity', None):
1711                         max_gb = round(int(arr.MaxCapacity) / 1024 ** 2, 1)
1712                         max_capacity_str = f'{max_gb} GB'
1713                         mem_overview_props['Макс. поддерживаемый объем'] = max_capacity_str
1714                         break
1715                 type_map = {20: 'DDR', 21: 'DDR2', 22: 'DDR2 FB-DIMM', 24: 'DDR3', 26: 'DDR4', 27: 'LPDDR', 28: 'LPDDR2', 29: 'LPDDR3', 30: 'LPDDR4', 34: 'DDR5', 35: 'LPDDR5'}
1716                 form_map = {8: 'DIMM (Desktop)', 12: 'SODIMM (Laptop)', 9: 'TSOP', 10: 'PGA', 11: 'RIMM'}
1717
1718                 raw_modules = list(wmi_obj.InstancesOf('Win32_PhysicalMemory'))
1719                 ram_modules_found = len(raw_modules)
1720                 module_speeds = [int(getattr(m, 'Speed', 0) or 0) for m in raw_modules if getattr(m, 'Speed', 0)]
1721                 module_clocks = [int(getattr(m, 'ConfiguredClockSpeed', 0) or getattr(m, 'Speed', 0) or 0) for m in raw_modules if getattr(m, 'ConfiguredClockSpeed', 0) or getattr(m, 'Speed', 0) or 0)]

```

```

les if (getattr(m, 'ConfiguredClockSpeed', 0) or getattr(m, 'S
peed', 0))]
1722         max_module_speed = max(module_speeds) if modul
e_speeds else 0
1723         min_module_speed = min(module_speeds) if modul
e_speeds else 0
1724
1725         for idx, m in enumerate(raw_modules):
1726             cap_bytes = int(getattr(m, 'Capacity', 0)
or 0)
1727             cap_gb = round(cap_bytes / 1024 ** 3, 1)
1728             smbios_type = int(getattr(m, 'SMBIOSMemory
Type', 0) or 0)
1729             mem_type = type_map.get(smbios_type, f'DDR
(Тип {smbios_type})' if smbios_type else 'DDR')
1730             form_code = int(getattr(m, 'FormFactor', 0
) or 0)
1731             form_name = form_map.get(form_code, 'DIMM'
)
1732             mfg = (getattr(m, 'Manufacturer', '') or '
').strip()
1733             if mfg == '04CB000080CE':
1734                 mfg_display = 'ADATA / Micron (04CB)'
1735             elif mfg:
1736                 mfg_display = mfg
1737             else:
1738                 mfg_display = 'Не определен'
1739             sn = (getattr(m, 'SerialNumber', '') or ''
).strip() or 'N/A'
1740             pn = (getattr(m, 'PartNumber', '') or '').
strip() or 'N/A'
1741             speed = getattr(m, 'Speed', 0) or 0
1742             clock = getattr(m, 'ConfiguredClockSpeed',
0) or speed
1743             locator = (getattr(m, 'DeviceLocator', '')
or f'DIMM {idx + 1}').strip()
1744             bank = (getattr(m, 'BankLabel', '') or '')
.strip()

```

```

1745 |
1746 |             # Определение статуса согласования частот
1747 |             и выявление узких мест
1748 |             if speed and clock and clock < speed:
1749 |                 compat_status = f'␣ Потенциал не раскр
1750 |                 ыт: планка рассчитана на {speed} MHz, но контроллер/BIOS ограничил до {clock} MHz (потеря {speed - clock} MHz)'
1751 |                 freq_warning = f' (␣ занижена на {speed - clock} MHz)'
1752 |                 eff_percent = f'{round((clock / speed) * 100, 1)}% от номинала планки'
1753 |                 elif max_module_speed and speed and speed < max_module_speed:
1754 |                     compat_status = f'␣ Узкое место: планка {speed} MHz медленнее других модулей ({max_module_speed} MHz), снижает общую частоту'
1755 |                     freq_warning = ''
1756 |                     eff_percent = '100% (номинал)'
1757 |                 else:
1758 |                     compat_status = f'␣ Оптимально: планка работает на паспортной частоте ({clock} MHz)'
1759 |                     freq_warning = ''
1760 |                     eff_percent = '100% (номинал)'
1761 |
1762 |             module_props: Dict[str, Any] = {
1763 |                 'Слот / Разъем (Locator)': locator,
1764 |                 'Объем планки': f'{cap_gb} GB ({cap_bytes:,} байт)',
1765 |                 'Номинальная частота': f'{speed} MHz' if speed else 'N/A',
1766 |                 'Текущая рабочая частота': f'{clock} MHz{freq_warning}' if clock else 'N/A',
1767 |                 'Эффективность планки': eff_percent,
1768 |                 'Состояние согласования частоты': compat_status,
1769 |                 'Тип памяти': f'{mem_type} (SMBIOS {smbios_type})',
1770 |                 'Форм-фактор': form_name,

```

```

1769 |             'Производитель': mfg_display,
1770 |             'Серийный номер (S/N)': sn,
1771 |             'Парт-номер (P/N)': pn,
1772 |         }
1773 |         if bank:
1774 |             module_props['Банк памяти (Bank)'] = b
1775 |         nodes.append(HardwareNode(category='Memory
Module (RAM)', name=f'Планка {idx + 1} ({locator}): {cap_gb}
GB {mem_type}-{speed or clock}', properties=module_props))
1776 |         except Exception as e:
1777 |             logger.debug(f'[HardwareTree] WMI PhysicalMemo
ry query warning: {e}')
1778 |         if ram_modules_found > 0:
1779 |             mem_overview_props['Установлено модулей'] = f'{ram
_modules_found} шт.'
1780 |             if total_slots > 0:
1781 |                 free_slots = max(0, total_slots - ram_modules_
found)
1782 |                 mem_overview_props['Конфигурация слотов'] = f'
Занято {ram_modules_found} из {total_slots} (свободно: {free_s
lots})'
1783 |                 if module_clocks and module_speeds:
1784 |                     min_c = min(module_clocks)
1785 |                     max_s = max(module_speeds)
1786 |                     if min_c < max_s:
1787 |                         mem_overview_props['Статус частоты памяти'
] = f'⚠ Частота шины занижена до {min_c} MHz (потенциал планок
: {max_s} MHz, проверьте XMP/BIOS)'
1788 |                     else:
1789 |                         mem_overview_props['Статус частоты памяти'
] = f'⚠ Оптимальный ({min_c} MHz, 100% номинала)'
1790 |             nodes.append(HardwareNode(category='System Memory', na
me=f'{mem.total_gb} GB Physical RAM', properties=mem_overview_
props))
1791 |             wmi_video_map: Dict[str, Dict[str, Any]] = {}
1792 |             if os.name == 'nt':
1793 |                 try:

```

```

1794         import win32com.client
1795         wmi_obj = win32com.client.GetObject('winmgmts:
1796         ')
1797         for v in wmi_obj.InstancesOf('Win32_VideoContr
1798         oller'):
1799             v_name = (getattr(v, 'Name', '') or '').st
1800             rip()
1801             v_pnp = (getattr(v, 'PNPDeviceID', '') or
1802             '').strip()
1803             v_mfg = (getattr(v, 'AdapterCompatibility'
1804             , '') or '').strip()
1805             v_drv_ver = (getattr(v, 'DriverVersion', '
1806             ') or '').strip()
1807             v_drv_date_raw = getattr(v, 'DriverDate',
1808             None)
1809             v_drv_date = ''
1810             if v_drv_date_raw:
1811                 ds = str(v_drv_date_raw)[:8]
1812                 if len(ds) == 8 and ds.isdigit():
1813                     v_drv_date = f'{ds[6:8]}.{ds[4:6]}
1814                     .{ds[0:4]}'
1815             v_proc = (getattr(v, 'VideoProcessor', '')
1816             or '').strip()
1817             v_mode = (getattr(v, 'VideoModeDescription
1818             ', '') or '').strip()
1819             v_refresh = getattr(v, 'CurrentRefreshRate
1820             ', None)
1821             v_status = (getattr(v, 'Status', '') or '0
1822             K').strip()
1823             info_dict = {'name': v_name, 'mfg': v_mfg,
1824             'drv_ver': v_drv_ver, 'drv_date': v_drv_date, 'proc': v_proc,
1825             'mode': v_mode, 'refresh': v_refresh, 'status': v_status, 'pn
1826             p': v_pnp}
1827             if v_name:
1828                 wmi_video_map[v_name.lower()] = info_d
1829             ict
1830         except Exception as e:
1831             logger.debug(f'[HardwareTree] WMI VideoControl

```

```

ler query warning: {e}')
1816         gpus = self.get_gpu_metrics()
1817         for idx, gpu in enumerate(gpus):
1818             v_extra = wmi_video_map.get(gpu.name.lower(), {})
1819             gpu_mfg = gpu.vendor or v_extra.get('mfg') or ('NVIDIA' if 'geforce' in gpu.name.lower() or 'nvidia' in gpu.name
IDIA' if 'geforce' in gpu.name.lower() or 'nvidia' in gpu.name
.lower() else 'Intel' if 'intel' in gpu.name.lower() else 'AMD
' if 'radeon' in gpu.name.lower() else 'Unknown')
1820             drv_ver = v_extra.get('drv_ver')
1821             drv_date = v_extra.get('drv_date')
1822             vproc = v_extra.get('proc')
1823             vmode = v_extra.get('mode')
1824
1825             # Форматирование памяти
1826             if gpu.dedicated_memory_mb and gpu.dedicated_memory_mb >= 1024:
1827                 ded_str = f'{round(gpu.dedicated_memory_mb / 1
024.0, 1)} GB'
1828             elif gpu.dedicated_memory_mb:
1829                 ded_str = f'{int(gpu.dedicated_memory_mb)} MB'
1830             else:
1831                 ded_str = f'{gpu.memory_total_gb} GB'
1832
1833             if gpu.shared_memory_mb and gpu.shared_memory_mb >
= 1024:
1834                 shared_str = f'{round(gpu.shared_memory_mb / 1
024.0, 1)} GB'
1835             elif gpu.shared_memory_mb:
1836                 shared_str = f'{int(gpu.shared_memory_mb)} MB'
1837             else:
1838                 shared_str = '0 MB'
1839
1840             total_mem_str = (
1841                 f'{round((gpu.dedicated_memory_mb + gpu.shared
_memory_mb) / 1024.0, 1)} GB'
1842                 if (gpu.dedicated_memory_mb + gpu.shared_memory_mb > 0)
1843                 else f'{gpu.memory_total_gb} GB'

```

```

1844 |         )
1845 |
1846 |         gpu_props: Dict[str, Any] = {
1847 |             'Индекс устройства': idx,
1848 |             'Модель видеокарты': gpu.name,
1849 |             'Производитель (Вендор)': gpu_mfg,
1850 |             'Тип GPU': gpu.gpu_type or 'Discrete',
1851 |             'Видеопамять (VRAM)': f'{gpu.memory_total_gb}
| GB',
1852 |             'Общая память GPU': total_mem_str,
1853 |             'Выделенная VRAM': ded_str,
1854 |             'Shared GPU Memory': shared_str,
1855 |             'Тип памяти': gpu.memory_type or 'Dedicated VR
| AM',
1856 |             'Поддержка DirectX': gpu.directx_version or 'D
| irectX 12',
1857 |         }
1858 |         if vproc:
1859 |             gpu_props['Видеопроцессор'] = vproc
1860 |         if drv_ver:
1861 |             gpu_props['Версия драйвера'] = drv_ver
1862 |         if drv_date:
1863 |             gpu_props['Дата драйвера'] = drv_date
1864 |         if vmode:
1865 |             gpu_props['Текущий видеорежим'] = vmode
1866 |         elif v_extra.get('refresh'):
1867 |             gpu_props['Частота обновления'] = f'{v_extra['
| refresh']}] Hz"
1868 |             gpu_props['Поддержка CUDA'] = 'Да' if gpu.has_cuda
| else 'Нет'
1869 |             gpu_props['Поддержка ROCm'] = 'Да' if gpu.has_rocm
| else 'Нет'
1870 |             gpu_props['Поддержка DirectML'] = 'Да' if gpu.has_
| directml else 'Нет'
1871 |             gpu_props['Поддержка Vulkan'] = 'Да' if gpu.has_vu
| lkan else 'Нет'
1872 |             gpu_props['Поддержка OpenCL'] = 'Да' if gpu.has_op
| encl else 'Нет'

```

```

1873         if gpu.has_oneapi:
1874             gpu_props['Поддержка oneAPI'] = 'Да'
1875         if gpu.compute_apis:
1876             gpu_props['Вычислительные API'] = ', '.join(gpu.compute_apis)
1877         if gpu.ai_backends:
1878             gpu_props['AI Бэкенды'] = ', '.join(gpu.ai_backends)
1879         gpu_props['Статус устройства'] = v_extra.get('status', 'OK (Активно)')
1880         if v_extra.get('pnp'):
1881             gpu_props['Идентификатор PnP'] = v_extra['pnp']
1882         nodes.append(HardwareNode(category='Display Adapter', name=gpu.name, properties=gpu_props))
1883         npus = self.get_npu_metrics()
1884         for idx, npu in enumerate(npus):
1885             npu_props: Dict[str, Any] = {
1886                 'Индекс устройства': idx,
1887                 'Модель NPU': npu.name,
1888                 'Производитель (Вендор)': npu.vendor,
1889                 'Тип устройства': 'Нейропроцессор (NPU / AI Accelerator)',
1890                 'Статус устройства': npu.status,
1891             }
1892             if npu.driver_version:
1893                 npu_props['Версия драйвера'] = npu.driver_version
1894             if npu.pnp_device_id:
1895                 npu_props['Идентификатор PnP'] = npu.pnp_device_id
1896             if npu.tops:
1897                 npu_props['Производительность (TOPS)'] = f'{npu.tops} TOPS'
1898             nodes.append(HardwareNode(category='Neural Processing Unit (NPU)', name=npu.name, properties=npu_props))
1899         for idx, mon in enumerate(self.get_monitors()):
1900             nodes.append(HardwareNode(category='Monitors & Dis

```



```

plays', name=f'{mon.name} ({mon.width}x{mon.height} @ {mon.frequency_hz}Hz, {mon.orientation})', properties={'Устройство': mon.device, 'Название дисплея': mon.name, 'Ориентация': mon.orientation, 'Подключенный видеоадаптер': mon.adapter or 'Default Adapter', 'Разрешение экрана': f'{mon.width} x {mon.height}', 'Частота развертки': f'{mon.frequency_hz} Hz', 'Глубина цвета': f'{mon.bits_per_pixel}-bit', 'Основной монитор': 'Да (Основной)' if mon.is_primary else 'Нет (Вторичный)'))
1901         if os.name == 'nt':
1902             try:
1903                 import win32com.client
1904                 wmi_obj = win32com.client.GetObject('winmgmts:
')
1905                 for d in wmi_obj.InstancesOf('Win32_DiskDrive'
):
1906                     size_bytes = int(getattr(d, 'Size', 0) or
0)
1907                     size_gb = round(size_bytes / 1024 ** 3, 1)
1908                     model = (getattr(d, 'Model', '') or 'Hard
Disk').strip()
1909                     iface = (getattr(d, 'InterfaceType', '') o
r 'SCSI/SATA').strip()
1910                     media = (getattr(d, 'MediaType', '') or 'F
ixed hard disk media').strip()
1911                     sn = (getattr(d, 'SerialNumber', '') or ''
).strip() or 'N/A'
1912                     firmware = (getattr(d, 'FirmwareRevision',
'') or '').strip()
1913                     parts = getattr(d, 'Partitions', 0) or 1
1914                     status = (getattr(d, 'Status', '') or 'OK'
).strip()
1915                     model_l = model.lower()
1916                     if 'samsung' in model_l:
1917                         disk_mfg = 'Samsung'
1918                     elif 'crucial' in model_l or model_l.start
swith('ct'):
1919                         disk_mfg = 'Crucial / Micron'
1920                     elif 'toshiba' in model_l or model_l.start

```

```

    with('hwd') or model_1.startswith('dt01'):
1921         disk_mfg = 'Toshiba'
1922         elif 'wdc' in model_1 or model_1.startswit
h('wd') or 'western digital' in model_1:
1923             disk_mfg = 'Western Digital'
1924             elif 'kingston' in model_1 or model_1.star
tswith('sa400') or model_1.startswith('skc'):
1925                 disk_mfg = 'Kingston'
1926                 elif 'seagate' in model_1 or model_1.start
swith('st'):
1927                     disk_mfg = 'Seagate'
1928                     elif 'sandisk' in model_1:
1929                         disk_mfg = 'SanDisk'
1930                     elif 'kioxia' in model_1:
1931                         disk_mfg = 'Kioxia'
1932                     elif 'intel' in model_1:
1933                         disk_mfg = 'Intel'
1934                     elif 'micron' in model_1:
1935                         disk_mfg = 'Micron'
1936                     elif 'sk hynix' in model_1 or 'hynix' in m
odel_1:
1937                         disk_mfg = 'SK Hynix'
1938                     else:
1939                         disk_mfg = 'Универсальный накопитель'
1940                         disk_props: Dict[str, Any] = {'Модель нако
пителя': model, 'Производитель (Бренд)': disk_mfg, 'Интерфейс
подключения': iface, 'Тип носителя': media, 'Емкость': f'{size
_gb} GB ({size_bytes:,} байт)', 'Серийный номер (S/N)': sn}
1941                         if firmware:
1942                             disk_props['Версия прошивки (Firmware)
'] = firmware
1943                             disk_props['Число разделов'] = parts
1944                             disk_props['Статус S.M.A.R.T.'] = status
1945                             nodes.append(HardwareNode(category='Physic
al Disk', name=f'{model} ({size_gb} GB)', properties=disk_prop
s))
1946                     except Exception as e:
1947                         logger.debug(f'[HardwareTree] WMI DiskDrive qu

```

```

    ery warning: {e}')
1948         partitions, _ = self.get_disk_metrics()
1949         for part in partitions:
1950             nodes.append(HardwareNode(category='Storage Volume
', name=f'Раздел {part.device} ({part.total_gb} GB)', properti
es={'Дисковый том': part.device, 'Точка монтирования': part.мо
untpoint, 'Файловая система': part.fstype, 'Общий объем': f'{p
art.total_gb} GB', 'Свободное место': f'{part.free_gb} GB', 'З
анято памяти': f'{part.used_gb} GB ({part.percent}%)'))
1951         wmi_net_adapters: Dict[str, Dict[str, Any]] = {}
1952         wmi_net_configs: Dict[str, Any] = {}
1953         if os.name == 'nt':
1954             try:
1955                 import win32com.client
1956                 wmi_obj = win32com.client.GetObject('winmgmts:
')
1957                 for cfg in wmi_obj.InstancesOf('Win32_NetworkA
dapterConfiguration'):
1958                     cdesc = (getattr(cfg, 'Description', '') o
r '').strip()
1959                     if cdesc:
1960                         wmi_net_configs[cdesc.lower()] = cfg
1961                 for a in wmi_obj.InstancesOf('Win32_NetworkAda
pter'):
1962                     net_id = (getattr(a, 'NetConnectionID', ''
) or '').strip()
1963                     aname = (getattr(a, 'Name', '') or '').str
ip()
1964                     adesc = (getattr(a, 'Description', '') or
'').strip()
1965                     apnp = (getattr(a, 'PNPDeviceID', '') or '
').strip()
1966                     amfg = (getattr(a, 'Manufacturer', '') or
'').strip()
1967                     amac = (getattr(a, 'MACAddress', '') or ''
).strip()
1968                     aspeed = getattr(a, 'Speed', None)
1969                     atype = (getattr(a, 'AdapterType', '') or

```



```

00)
2022         except Exception:
2023             pass
2024         if speed_mbps > 0:
2025             if speed_mbps >= 1000:
2026                 speed_str = f'{speed_mbps} Mbps ({round(
d(speed_mbps / 1000, 1)} Gbps)')
2027             else:
2028                 speed_str = f'{speed_mbps} Mbps'
2029         else:
2030             speed_str = 'Автоопределение (0 Mbps)' if
not is_up else '1000 Mbps (АВТО)'
2031             model_name = w_info.get('name') or w_info.get(
'desc') or if_name
2032             mfg_val = w_info.get('mfg')
2033             if not mfg_val:
2034                 if 'intel' in if_name.lower() or 'intel' i
n model_name.lower():
2035                     mfg_val = 'Intel Corporation'
2036                 elif 'realtek' in if_name.lower() or 'real
tek' in model_name.lower():
2037                     mfg_val = 'Realtek'
2038                 elif 'vethernet' in if_name.lower() or 'hy
per-v' in model_name.lower() or 'microsoft' in model_name.lowe
r() or ('local area connection*' in if_name.lower()):
2039                     mfg_val = 'Microsoft Corporation'
2040                 else:
2041                     mfg_val = 'Универсальный сетевой контр
оллер'
2042             name_l = if_name.lower()
2043             desc_l = (w_info.get('desc') or '').lower()
2044             if 'local area connection*' in name_l:
2045                 adapter_type = 'Виртуальный адаптер Wi-Fi
Direct'
2046             if model_name == if_name:
2047                 model_name = f'Microsoft Wi-Fi Direct
Virtual Adapter ({if_name})'
2048             elif 'wi-fi' in name_l or 'wireless' in desc_l

```

```

    or '802.11' in desc_l:
2049         adapter_type = 'Беспроводной адаптер (Wi-Fi 802.11)'
2050     elif 'bluetooth' in name_l or 'bluetooth' in desc_l:
2051         adapter_type = 'Bluetooth PAN адаптер'
2052     elif 'vethernet' in name_l or 'virtual' in desc_l or 'hyper-v' in desc_l:
2053         adapter_type = 'Виртуальный сетевой адаптер (Hyper-V / WSL)'
2054     elif 'loopback' in name_l:
2055         adapter_type = 'Программный интерфейс Loopback'
2056     elif 'tap' in name_l or 'tun' in name_l or 'vpn' in name_l:
2057         adapter_type = 'VPN / Туннельный адаптер'
2058     else:
2059         adapter_type = 'Сетевая карта Ethernet (LAN)'
2060     net_props: Dict[str, Any] = {'Имя подключения': if_name, 'Модель адаптера': model_name, 'Производитель': mfg_val, 'Тип адаптера': adapter_type, 'Физический (MAC) адрес': mac_addr or 'N/A', 'Состояние соединения': status_str, 'Скорость канала': speed_str}
2061     if stat and getattr(stat, 'duplex', None) is not None:
2062         d_val = getattr(stat.duplex, 'value', stat.duplex)
2063         if d_val == 2:
2064             net_props['Режим дуплекса'] = 'Full Duplex (Полный)'
2065         elif d_val == 1:
2066             net_props['Режим дуплекса'] = 'Half Duplex (Полудуплекс)'
2067     if stat and getattr(stat, 'mtu', None):
2068         net_props['Размер MTU'] = f'{stat.mtu} байт'
2069     net_props['IP-адреса (IPv4)'] = ', '.join(ipv4

```

```

_list) or 'N/A'
2070         if ipv6_list:
2071             net_props['IP-адреса (IPv6)'] = ', '.join(
ipv6_list[:3])
2072         if gateways:
2073             net_props['Основной шлюз'] = ', '.join(gat
eways)
2074         if dhcp_server or dhcp_enabled is not None:
2075             net_props['DHCP сервер'] = dhcp_server or
('Включен (DHCP)' if dhcp_enabled else 'Статический IP / Отклю
чен')
2076         if dns_servers:
2077             net_props['DNS серверы'] = ', '.join(dns_s
ervers)
2078         if w_info.get('pnp'):
2079             net_props['Идентификатор PnP'] = w_info['p
np']
2080             display_title = f'{if_name}: {model_name}' if
model_name != if_name else if_name
2081             nodes.append(HardwareNode(category='Network Ad
apter', name=display_title, properties=net_props))
2082             if os.name == 'nt':
2083                 try:
2084                     import win32com.client
2085                     wmi_obj = win32com.client.GetObject('winmgmts:
')
2086                     for s in wmi_obj.InstancesOf('Win32_SoundDevic
e'):
2087                         snd_name = (getattr(s, 'Caption', '') or g
getattr(s, 'Name', '') or '').strip()
2088                         snd_mfg = (getattr(s, 'Manufacturer', '')
or '').strip() or 'Аудиоустройство'
2089                         snd_status = (getattr(s, 'Status', '') or
'OK').strip()
2090                         snd_pnp = (getattr(s, 'PNPDeviceID', '') o
r '').strip()
2091                         if snd_name:
2092                             nodes.append(HardwareNode(category='Au

```



```

        dio Device', name=snd_name, properties={'Название устройства':
        snd_name, 'Производитель': snd_mfg, 'Состояние': f'{snd_status} (Работает)', 'Идентификатор PnP': snd_pnp or 'N/A'}))
2093         except Exception as e:
2094             logger.debug(f'[HardwareTree] WMI SoundDevice
        query warning: {e}')
2095         if os.name == 'nt':
2096             try:
2097                 import win32com.client
2098                 wmi_obj = win32com.client.GetObject('winmgmts:
        ')
2099                 for u in wmi_obj.InstancesOf('Win32_USBControl
        ler'):
2100                     usb_name = (getattr(u, 'Caption', '') or g
        etattr(u, 'Name', '') or '').strip()
2101                     usb_mfg = (getattr(u, 'Manufacturer', '')
        or '').strip() or 'USB Контроллер'
2102                     usb_status = (getattr(u, 'Status', '') or
        'OK').strip()
2103                     usb_pnp = (getattr(u, 'PNPDeviceID', '') o
        r '').strip()
2104                     if usb_name:
2105                         nodes.append(HardwareNode(category='US
        B Controller', name=usb_name, properties={'Название контроллер
        а': usb_name, 'Производитель': usb_mfg, 'Состояние': f'{usb_st
        atus} (Работает)', 'Идентификатор PnP': usb_pnp or 'N/A'}))
2106             except Exception as e:
2107                 logger.debug(f'[HardwareTree] WMI USBControlle
        r query warning: {e}')
2108             upd = self.get_updates_info()
2109             nodes.append(HardwareNode(category='Windows Updates',
        name=f'{upd.status} ({upd.installed_kb_count} KBs)', propertie
        s={'Статус обновлений': upd.status, 'Число установленных испра
        влений': upd.installed_kb_count, 'Последние KB пакеты': ', '.j
        oin(upd.recent_hotfixes) or 'N/A', 'Дата последней установки':
        upd.latest_installed_on or 'OK'}))
2110             self._hardware_tree_cached = nodes
2111             return nodes

```

```

2112
2113     def get_hardware_tree(self, force: bool = False) -> List[H
hardwareNode]:
2114         """Generate AIDA64-like hierarchical component specifi
cation tree.
2115
2116         Args:
2117             force: Принудительный повторный опрос оборудования
.
2118
2119         Returns:
2120             List[HardwareNode]: Hardware devices grouped by ca
tegory.
2121         """
2122         import asyncio
2123         return asyncio.run(self.get_hardware_tree_async(force=
force))
2124
2125     def get_hardware_audit(self) -> HardwareAuditReport:
2126         """Collect deep hardware devices audit with drivers, i
nstall dates, and attached sensors.
2127
2128         Returns:
2129             HardwareAuditReport: Complete hardware audit repor
t.
2130         """
2131         return self.auditor.audit_hardware()
2132
2133     def archive_hardware_state(self, auto_diff: bool=True) ->
HardwareArchiveEntry:
2134         """Capture current hardware audit and save it to histo
rical archive storage.
2135
2136         Args:
2137             auto_diff: Automatically compute diff with previou
s archive.
2138
2139         Returns:

```

```

2140         HardwareArchiveEntry: Persisted archive entry with
        detected changes.
2141         """
2142         report = self.get_hardware_audit()
2143         return self.history_manager.archive_report(report, auto_diff=auto_diff)
2144
2145     def get_hardware_history(self, limit: int=50) -> List[Dict
        [str, Any]]:
2146         """Retrieve historical hardware archive snapshots meta
        data.
2147
2148         Args:
2149             limit: Maximum count of archive entries.
2150
2151         Returns:
2152             List[Dict[str, Any]]: Chronological list of archive
        entries.
2153         """
2154         return self.history_manager.get_history(limit=limit)
2155
2156     def get_hardware_changes(self, limit: int=100) -> List[HardwareChangeItem]:
2157         """Retrieve historical timeline of all hardware configuration
        changes.
2158
2159         Args:
2160             limit: Maximum count of changes to return.
2161
2162         Returns:
2163             List[HardwareChangeItem]: Historical changes timeline.
2164         """
2165         return self.history_manager.get_change_timeline(limit=limit)
2166
2167     def get_startup_audit(self) -> Any:
2168         """Collect deep Windows startup and autoruns audit report

```

```

    ort.
2169
2170         Returns:
2171             AuditReport: Complete startup audit report with ri
sk ratings.
2172         """
2173         from apps.windows.sdk.modules.startup.core.auditor imp
ort StartupAuditor
2174         return StartupAuditor().run_audit()
2175
2176         def archive_startup_state(self, auto_diff: bool = True) ->
StartupArchiveEntry:
2177             """Capture current Windows startup state, compute chan
ges diff, and save snapshot to storage.
2178
2179             Args:
2180                 auto_diff: Automatically compute diff with previou
s archive snapshot.
2181
2182             Returns:
2183                 StartupArchiveEntry: Persisted archive entry with
detected changes.
2184             """
2185             report = self.get_startup_audit()
2186             entry = self.startup_history_manager.archive_report(re
port, auto_diff=auto_diff)
2187             if self.storage:
2188                 try:
2189                     self.storage.save_startup_archive(entry)
2190                 except Exception as ex:
2191                     logger.debug(f'Не удалось сохранить снимок авт
озапуска в БД: {ex}')
2192             return entry
2193
2194         def get_startup_history(self, limit: int = 50) -> List[Dic
t[str, Any]]:
2195             """Retrieve historical startup archive snapshots metad
ata.

```

```

2196 |
2197 |     Args:
2198 |         limit: Maximum count of archive entries.
2199 |
2200 |     Returns:
2201 |         List[Dict[str, Any]]: Chronological list of startup
2202 |         p archive entries.
2203 |         """
2204 |         if self.storage:
2205 |             try:
2206 |                 db_history = self.storage.get_startup_archives
2207 |                 (limit=limit)
2208 |                 if db_history:
2209 |                     return db_history
2210 |             except Exception:
2211 |                 pass
2212 |         return self.startup_history_manager.get_history(limit=
2213 |         limit)
2214 |
2215 |     def get_startup_changes(self, limit: int = 100) -> List[Di
2216 |     ct[str, Any]]:
2217 |         """Retrieve historical timeline of all startup configu
2218 |         ration changes.
2219 |
2220 |     Args:
2221 |         limit: Maximum count of changes to return.
2222 |
2223 |     Returns:
2224 |         List[Dict[str, Any]]: Historical changes timeline.
2225 |         """
2226 |         if self.storage:
2227 |             try:
2228 |                 db_changes = self.storage.get_startup_changes(
2229 |                 limit=limit)
2230 |                 if db_changes:
2231 |                     return db_changes
2232 |             except Exception:
2233 |                 pass

```

```

2228         return [c.model_dump(mode='json') for c in self.startu
p_history_manager.get_change_timeline(limit=limit)]
2229
2230     def get_latest_startup_audit(self) -> Optional[Dict[str, A
ny]]:
2231         """Retrieve latest recorded startup audit snapshot fro
m telemetry storage.
2232
2233         Returns:
2234             Optional[Dict[str, Any]]: Startup audit dict or No
ne.
2235         """
2236         if self.storage:
2237             try:
2238                 latest = self.storage.get_latest_startup_archi
ve()
2239                 if latest:
2240                     return latest
2241             except Exception:
2242                 pass
2243             latest_archive = self.startup_history_manager.get_late
st_archive()
2244             return latest_archive.model_dump(mode='json') if lates
t_archive else None
2245
2246     def save_snapshot_to_db(self, snapshot: Optional[SystemSna
pshot]=None, top_n: int=20) -> int:
2247         """Collect and save system telemetry snapshot directly
into SQLite database.
2248
2249         Args:
2250             snapshot: Optional existing SystemSnapshot (if Non
e, collected synchronously).
2251             top_n: Top processes count.
2252
2253         Returns:
2254             int: Created snapshot ID in database.
2255         """

```

```

2256         snap = snapshot
2257         if snap is None:
2258             import asyncio
2259             try:
2260                 loop = asyncio.get_event_loop()
2261             except RuntimeError:
2262                 loop = None
2263             if loop and loop.is_running():
2264                 snap = asyncio.run_coroutine_threadsafe(self.get_snapshot(process_limit=top_n), loop).result(timeout=5.0)
2265             else:
2266                 snap = asyncio.run(self.get_snapshot(process_limit=top_n))
2267         return self.storage.save_snapshot(snap, top_n=top_n)
2268
2269     def get_snapshots(self, limit: int=60, since_epoch: Optional[float]=None) -> List[Dict[str, Any]]:
2270         """Retrieve persisted system snapshots from SQLite database.
2271
2272         Args:
2273             limit: Maximum number of snapshots.
2274             since_epoch: Optional starting timestamp.
2275
2276         Returns:
2277             List[Dict[str, Any]]: List of system snapshots from database.
2278         """
2279         return self.storage.get_snapshots(limit=limit, since_epoch=since_epoch)
2280
2281     def get_latest_cpu_from_db(self) -> Optional[Dict[str, Any]]:
2282         """Извлекает последние метрики процессора из базы данных SQLite.
2283
2284         Returns:
2285             Optional[Dict[str, Any]]: Метрики процессора из SQ

```

```

    Lite.
2286     """
2287     return self.storage.get_latest_cpu()
2288
2289     def get_cpu_history_from_db(self, limit: int = 60, since_e
    poch: Optional[float] = None) -> List[Dict[str, Any]]:
2290         """Извлекает историю нагрузки процессора из базы данны
    x SQLite.
2291
2292         Args:
2293             limit: Количество записей.
2294             since_epoch: Начальное время UNIX epoch.
2295
2296         Returns:
2297             List[Dict[str, Any]]: Исторические срезы нагрузки
    CPU.
2298         """
2299         return self.storage.get_cpu_history(limit=limit, since
    _epoch=since_epoch)
2300
2301     def get_process_history(self, name: Optional[str]=None, pi
    d: Optional[int]=None, limit: int=100) -> List[Dict[str, Any]]
    :
2302         """Retrieve historical process metrics from SQLite dat
    abase.
2303
2304         Args:
2305             name: Substring of process name.
2306             pid: Process identifier.
2307             limit: Record limit.
2308
2309         Returns:
2310             List[Dict[str, Any]]: History of process metrics.
2311         """
2312         return self.storage.get_process_history(name=name, pid
    =pid, limit=limit)
2313
2314     def get_reboot_report(self, limit: int = 20, hours: int =

```



```

720) -> Any:
2315     """Сбор сводного отчета анализа перезагрузок и выключе
ний ОС.
2316
2317     Args:
2318         limit: Максимальное количество сессий.
2319         hours: Глубина выборки журналов в часах.
2320
2321     Returns:
2322         RebootAnalysisReport: Сводный аналитический отчет.
2323     """
2324     from apps.windows.telemetry_research.reboot_analyzer i
import WindowsRebootAnalyzer
2325     analyzer = WindowsRebootAnalyzer(storage=self.storage)
2326     return analyzer.collect_reboot_history(limit=limit, ho
urs=hours)
2327
2328     def get_latest_reboot(self) -> Optional[Any]:
2329         """Получение данных о последней перезагрузке системы.
2330
2331     Returns:
2332         Optional[RebootSession]: Информация о последнем пе
резапуске.
2333     """
2334     from apps.windows.telemetry_research.reboot_analyzer i
import WindowsRebootAnalyzer
2335     analyzer = WindowsRebootAnalyzer(storage=self.storage)
2336     return analyzer.analyze_current_boot()
2337
2338     def get_extended_system_audit(self) -> Any:
2339         """Сбор расширенного аудита безопасности (Defender), а
втозагрузки (Startup), VSS и пользователей.
2340
2341     Returns:
2342         ExtendedSystemAuditReport: Сводный отчет расширенн
ого аудита.
2343     """
2344     from apps.windows.telemetry_research.deep_diagnostics

```

```

import DeepDiagnosticsEngine
2345     engine = DeepDiagnosticsEngine()
2346     report = engine.collect_extended_system_audit()
2347     try:
2348         self.storage.save_extended_audit(report)
2349     except Exception as ex:
2350         logger.debug(f'Не удалось сохранить расширенный ау
аудит в БД: {ex}')
2351     return report
2352
2353     def get_latest_extended_audit(self) -> Optional[Dict[str,
Any]]:
2354         """Получение последнего сохраненного среза расширенног
о системного аудита.
2355
2356         Returns:
2357             Optional[Dict[str, Any]]: Словарь аудита или None.
2358         """
2359         return self.storage.get_latest_extended_audit()
2360
2361     def get_process_provenance_report(
2362         self,
2363         pid: Optional[int] = None,
2364         name_filter: Optional[str] = None,
2365         user_filter: Optional[str] = None,
2366         limit: int = 100,
2367     ) -> ProcessProvenanceReport:
2368         """Сформировать сводный отчет о происхождении и цепочк
ах запуска активных процессов.
2369
2370         Args:
2371             pid: Опциональный фильтр по PID.
2372             name_filter: Опциональный фильтр по имени процесса
.
2373             user_filter: Опциональный фильтр по пользователю/S
ID.
2374             limit: Максимальное количество процессов в отчете.
2375

```

```

2376 |         Returns:
2377 |             ProcessProvenanceReport: Полный отчет с детальным
        анализом происхождения процессов.
2378 |         """
2379 |         all_procs = self.get_top_processes(limit=0, sort_by='c
        pu')
2380 |         filtered_items: List[ProcessProvenanceInfo] = []
2381 |
2382 |         for p in all_procs:
2383 |             if pid is not None and p.pid != pid:
2384 |                 continue
2385 |             if name_filter and name_filter.lower() not in (p.n
        ame or '').lower():
2386 |                 continue
2387 |             if user_filter and user_filter.lower() not in (p.u
        sername or '').lower() and user_filter.lower() not in (p.sid o
        r '').lower():
2388 |                 continue
2389 |
2390 |             chain_list = [node.strip() for node in (p.ancestor
        _chain or "").split(" → ") if node.strip()]
2391 |             user_display = p.username or "Unknown"
2392 |             sid_display = p.sid or "N/A"
2393 |             session_display = f"Session {p.session_id}" if p.s
        ession_id is not None else "Session N/A"
2394 |             who_runs = f"{user_display} ({sid_display}, {sessi
        on_display})"
2395 |             who_created = f"{p.parent_name or 'Unknown'} (PPID
        : {p.ppid})"
2396 |             why_created = p.launch_reason or "unknown"
2397 |
2398 |             item = ProcessProvenanceInfo(
2399 |                 pid=p.pid,
2400 |                 ppid=p.ppid,
2401 |                 name=p.name,
2402 |                 executable_path=p.executable_path,
2403 |                 command_line=p.cmdline,
2404 |                 user=p.username,

```



```

2435 |             name=str(ev.get("name") or "unknown"),
2436 |             executable_path=ev.get("executable_path"),
2437 |             command_line=ev.get("command_line"),
2438 |             user=ev.get("user"),
2439 |             sid=ev.get("sid"),
2440 |             session_id=int(ev.get("session_id")) if ev
2441 |             .get("session_id") is not None else None,
2442 |             integrity_level=ev.get("integrity_level"),
2443 |             elevation=bool(ev.get("elevation")) if
2444 |             ev.get("elevation") is not None else None,
2445 |             parent_name=ev.get("parent_name"),
2446 |             ancestor_chain=ev.get("ancestor_chain"),
2447 |             launch_reason=ev.get("launch_reason"),
2448 |             source=str(ev.get("source") or "system"),
2449 |             raw_json=str(ev.get("details_json")) if
2450 |             ev.get("details_json") else None,
2451 |         ))
2452 |     except Exception as ex:
2453 |         logger.debug(f"Не удалось извлечь недавние события
2454 | provenance для отчета: {ex}")
2455 |
2456 |     summary = f"Всего активных процессов: {len(all_procs)}
2457 | . Проанализировано происхождение для {len(filtered_items)} про
2458 | цессов."
2459 |     return ProcessProvenanceReport(
2460 |         timestamp=now_iso,
2461 |         total_processes=len(all_procs),
2462 |         active_provenance=filtered_items,
2463 |         recent_events=recent_evs,
2464 |         summary=summary,
2465 |     )
2466 |
2467 | def get_process_provenance_history(
2468 |     self,

```

```

2463         guid: Optional[str] = None,
2464         pid: Optional[int] = None,
2465         name: Optional[str] = None,
2466         user: Optional[str] = None,
2467         event_type: Optional[str] = None,
2468         limit: int = 100,
2469     ) -> List[Dict[str, Any]]:
2470         """Получить исторические события жизненного цикла и пр
оисхождения процессов из БД.
2471
2472         Args:
2473             guid: Фильтр по уникальному идентификатору процесс
а.
2474             pid: Фильтр по PID.
2475             name: Фильтр по имени процесса.
2476             user: Фильтр по имени пользователя или SID.
2477             event_type: Фильтр по типу события ('ProcessCreate
d', 'ProcessTerminated').
2478             limit: Максимальное количество записей.
2479
2480         Returns:
2481             List[Dict[str, Any]]: Список исторических событий
происхождения процессов.
2482         """
2483         return self.storage.get_process_provenance_history(
2484             guid=guid,
2485             pid=pid,
2486             name=name,
2487             user=user,
2488             event_type=event_type,
2489             limit=limit,
2490         )
2491
2492     def get_process_lineage(self, guid_or_pid: str) -> List[Di
ct[str, Any]]:
2493         """Построить полную восходящую цепочку предков процесс
а (дерево запуска).
2494

```

```
2495 |         Args:
2496 |             guid_or_pid: Идентификатор process_guid или числов
    | ой PID.
2497 |
2498 |         Returns:
2499 |             List[Dict[str, Any]]: Упорядоченный список предков
    | от корневого процесса к целевому.
2500 |             """
2501 |         return self.storage.get_process_lineage(guid_or_pid=gu
    | id_or_pid)
2502 |
2503 |
2504 |
```

compactor.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - Compact
  or
4 # =====
5 # Description:
6 #   Модуль логарифмического и многоуровневого сжатия (rollup)
  временных рядов.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.compactor import TelemetryCo
  mpactor, compute_percentile
11 #
12 # File: compactor.py
13 # Project: ai-breadboard
14 # Package: apps.windows.telemetry
15 # Author: hypo69
16 # Copyright: © 2026 hypo69
17 # Updated: 2026-10-03 23:25:00
18 # =====
19
20 """Модуль `TelemetryCompactor` и вспомогательная функция `comp
  ute_percentile`.
21
22 Перенесены из `telemetry_research` без обертки.
23 """
24
25 from __future__ import annotations
26
27 import math
28 from datetime import datetime
29 from typing import Any, Dict, List, Optional, Sequence, Tuple
```



```

30
31 from apps.windows.telemetry.models import ProcessMetricRollup,
   SystemMetricRollup
32
33
34 def compute_percentile(values: Sequence[float], p: float = 95.
   0) -> float:
35     """Вычисляет перцентиль выборки методом линейной интерполя
   ции.
36
37     Args:
38         values: Последовательность числовых значений.
39         p: Желаемый перцентиль (0..100), по умолчанию 95.0.
40
41     Returns:
42         float: Значение перцентиля (0.0 при пустой выборке).
43     """
44     if not values:
45         return 0.0
46     sorted_vals = sorted(values)
47     n = len(sorted_vals)
48     if n == 1:
49         return float(sorted_vals[0])
50     p_clamped = max(0.0, min(100.0, p))
51     rank = (p_clamped / 100.0) * (n - 1)
52     lower = int(math.floor(rank))
53     upper = int(math.ceil(rank))
54     weight = rank - lower
55     return float(sorted_vals[lower] + weight * (sorted_vals[up
   per] - sorted_vals[lower]))
56
57
58 class TelemetryCompactor:
59     """Компаратор и агрегатор временных рядов телеметрии."""
60
61     @staticmethod
62     def get_tier_for_age(age_seconds: float) -> str:
63         """Определяет гранулярность (tier) в зависимости от во

```

зраста данных.

```
64 |
65 |     Args:
66 |         age_seconds: Возраст данных в секундах.
67 |
68 |     Returns:
69 |         str: Уровень агрегации ('1s', '2s', '5s', '10s', '
70 | 30s', '1m', '5m', '15m', '1h', '6h').
71 |         """
72 |     if age_seconds <= 60.0:
73 |         return "1s"
74 |     elif age_seconds <= 300.0:
75 |         return "2s"
76 |     elif age_seconds <= 900.0:
77 |         return "5s"
78 |     elif age_seconds <= 1800.0:
79 |         return "10s"
80 |     elif age_seconds <= 3600.0:
81 |         return "30s"
82 |     elif age_seconds <= 10800.0:
83 |         return "1m"
84 |     elif age_seconds <= 43200.0:
85 |         return "5m"
86 |     elif age_seconds <= 86400.0:
87 |         return "15m"
88 |     elif age_seconds <= 604800.0:
89 |         return "1h"
90 |     else:
91 |         return "6h"
92 |
93 | @staticmethod
94 | def compact_system_metrics(
95 |     samples: List[Dict[str, Any]],
96 |     period_start: str,
97 |     period_end: str,
98 |     tier: str = "1m",
99 | ) -> SystemMetricRollup:
    """Агрегирует список сырых замеров хоста в один System
```

```

MetricRollup.
100
101     Args:
102         samples: Список словарей замеров (SystemSnapshot и
ли сжатых метрик).
103         period_start: ISO timestamp начала периода.
104         period_end: ISO timestamp окончания периода.
105         tier: Уровень гранулярности бакета.
106
107     Returns:
108         SystemMetricRollup: Сжатый бакет с расчётом avg, m
in, max, p95 и total.
109     """
110     count = len(samples)
111     if count == 0:
112         return SystemMetricRollup(
113             period_start=period_start,
114             period_end=period_end,
115             duration_seconds=0.0,
116             sample_count=0,
117             tier=tier,
118         )
119     # Длительность периода
120     duration_sec = 60.0
121     try:
122         t0 = datetime.fromisoformat(period_start)
123         t1 = datetime.fromisoformat(period_end)
124         duration_sec = max(1.0, (t1 - t0).total_seconds())
125     except Exception:
126         duration_sec = float(count)
127     cpu_vals: List[float] = []
128     ram_used_vals: List[float] = []
129     ram_pct_vals: List[float] = []
130     disk_read_rates: List[float] = []
131     disk_write_rates: List[float] = []
132     net_rx_rates: List[float] = []
133     net_tx_rates: List[float] = []
134     for s in samples:

```

```
135 |         cpu_val = float(s.get("cpu_total_percent") or s.ge
| t("cpu_percent") or 0.0)
136 |         cpu_vals.append(cpu_val)
137 |         ram_used = float(s.get("memory_used_gb") or 0.0)
138 |         ram_pct = float(s.get("memory_percent") or 0.0)
139 |         ram_used_vals.append(ram_used)
140 |         ram_pct_vals.append(ram_pct)
141 |         disk_r = float(s.get("disk_read_bytes_sec") or 0.0
| ) / (1024.0 * 1024.0)
142 |         disk_w = float(s.get("disk_write_bytes_sec") or 0.
| 0) / (1024.0 * 1024.0)
143 |         disk_read_rates.append(disk_r)
144 |         disk_write_rates.append(disk_w)
145 |         net_rx = float(s.get("network_recv_bytes_sec") or
| 0.0) / (1024.0 * 1024.0)
146 |         net_tx = float(s.get("network_sent_bytes_sec") or
| 0.0) / (1024.0 * 1024.0)
147 |         net_rx_rates.append(net_rx)
148 |         net_tx_rates.append(net_tx)
149 |         sample_dt = duration_sec / max(1, count)
150 |         cpu_avg = sum(cpu_vals) / count
151 |         cpu_min = min(cpu_vals)
152 |         cpu_max = max(cpu_vals)
153 |         cpu_p95 = compute_percentile(cpu_vals, 95.0)
154 |         ram_avg = sum(ram_used_vals) / count
155 |         ram_min = min(ram_used_vals)
156 |         ram_max = max(ram_used_vals)
157 |         ram_pct_avg = sum(ram_pct_vals) / count
158 |         ram_pct_max = max(ram_pct_vals)
159 |         disk_r_avg = sum(disk_read_rates) / count
160 |         disk_r_max = max(disk_read_rates)
161 |         disk_r_total = sum(disk_read_rates) * sample_dt
162 |         disk_w_avg = sum(disk_write_rates) / count
163 |         disk_w_max = max(disk_write_rates)
164 |         disk_w_total = sum(disk_write_rates) * sample_dt
165 |         net_rx_avg = sum(net_rx_rates) / count
166 |         net_rx_max = max(net_rx_rates)
167 |         net_rx_total = sum(net_rx_rates) * sample_dt
```

```

168 |         net_tx_avg = sum(net_tx_rates) / count
169 |         net_tx_max = max(net_tx_rates)
170 |         net_tx_total = sum(net_tx_rates) * sample_dt
171 |         return SystemMetricRollup(
172 |             period_start=period_start,
173 |             period_end=period_end,
174 |             duration_seconds=round(duration_sec, 2),
175 |             sample_count=count,
176 |             tier=tier,
177 |             cpu_avg=round(cpu_avg, 2),
178 |             cpu_min=round(cpu_min, 2),
179 |             cpu_max=round(cpu_max, 2),
180 |             cpu_p95=round(cpu_p95, 2),
181 |             ram_avg_gb=round(ram_avg, 3),
182 |             ram_min_gb=round(ram_min, 3),
183 |             ram_max_gb=round(ram_max, 3),
184 |             ram_percent_avg=round(ram_pct_avg, 2),
185 |             ram_percent_max=round(ram_pct_max, 2),
186 |             disk_read_avg_mbs=round(disk_r_avg, 3),
187 |             disk_read_max_mbs=round(disk_r_max, 3),
188 |             disk_read_total_mb=round(disk_r_total, 3),
189 |             disk_write_avg_mbs=round(disk_w_avg, 3),
190 |             disk_write_max_mbs=round(disk_w_max, 3),
191 |             disk_write_total_mb=round(disk_w_total, 3),
192 |             network_rx_avg_mbs=round(net_rx_avg, 3),
193 |             network_rx_max_mbs=round(net_rx_max, 3),
194 |             network_rx_total_mb=round(net_rx_total, 3),
195 |             network_tx_avg_mbs=round(net_tx_avg, 3),
196 |             network_tx_max_mbs=round(net_tx_max, 3),
197 |             network_tx_total_mb=round(net_tx_total, 3),
198 |         )
199 |
200 |     @staticmethod
201 |     def compact_process_metrics(
202 |         process_samples: List[Dict[str, Any]],
203 |         period_start: str,
204 |         period_end: str,
205 |     ) -> List[ProcessMetricRollup]:

```

```

206 |         """Группирует и агрегирует замеры процессов по имени/P
    | ID.
207 |
208 |     Args:
209 |         process_samples: Список замеров процессов.
210 |         period_start: Начало периода.
211 |         period_end: Конец периода.
212 |
213 |     Returns:
214 |         List[ProcessMetricRollup]: Агрегированные показате
    | ли каждого процесса.
215 |         """
216 |         duration_sec = 60.0
217 |         try:
218 |             t0 = datetime.fromisoformat(period_start)
219 |             t1 = datetime.fromisoformat(period_end)
220 |             duration_sec = max(1.0, (t1 - t0).total_seconds())
221 |         except Exception:
222 |             duration_sec = 60.0
223 |         # Группировка по (name, pid)
224 |         grouped: Dict[Tuple[str, Optional[int]], List[Dict[str
    | , Any]]] = {}
225 |         for p in process_samples:
226 |             key = (str(p.get("name", "unknown")), p.get("pid")
    | )
227 |             grouped.setdefault(key, []).append(p)
228 |         results: List[ProcessMetricRollup] = []
229 |         for (name, pid), p_list in grouped.items():
230 |             count = len(p_list)
231 |             cpu_vals = [float(item.get("cpu_percent") or 0.0)
    | for item in p_list]
232 |             ram_vals = [float(item.get("memory_mb") or 0.0) fo
    | r item in p_list]
233 |             read_bytes_sec = [float(item.get("read_bytes_sec")
    | or 0.0) for item in p_list]
234 |             write_bytes_sec = [float(item.get("write_bytes_sec
    | ") or 0.0) for item in p_list]
235 |             rx_bytes_sec = [float(item.get("rx_bytes_sec") or

```

```

0.0) for item in p_list]
236         tx_bytes_sec = [float(item.get("tx_bytes_sec") or
0.0) for item in p_list]
237         sample_dt = duration_sec / max(1, count)
238         path = p_list[0].get("path") or p_list[0].get("exe
")
239         cpu_avg = sum(cpu_vals) / count
240         cpu_max = max(cpu_vals)
241         cpu_p95 = compute_percentile(cpu_vals, 95.0)
242         ram_avg = sum(ram_vals) / count
243         ram_max = max(ram_vals)
244         disk_r_tot = (sum(read_bytes_sec) * sample_dt) / (
1024.0 * 1024.0)
245         disk_w_tot = (sum(write_bytes_sec) * sample_dt) /
(1024.0 * 1024.0)
246         net_rx_tot = (sum(rx_bytes_sec) * sample_dt) / (10
24.0 * 1024.0)
247         net_tx_tot = (sum(tx_bytes_sec) * sample_dt) / (10
24.0 * 1024.0)
248         results.append(
249             ProcessMetricRollup(
250                 name=name,
251                 pid=pid,
252                 period_start=period_start,
253                 period_end=period_end,
254                 duration_seconds=round(duration_sec, 2),
255                 sample_count=count,
256                 path=path,
257                 cpu_avg=round(cpu_avg, 2),
258                 cpu_max=round(cpu_max, 2),
259                 cpu_p95=round(cpu_p95, 2),
260                 ram_avg_mb=round(ram_avg, 2),
261                 ram_max_mb=round(ram_max, 2),
262                 disk_read_total_mb=round(disk_r_tot, 3),
263                 disk_write_total_mb=round(disk_w_tot, 3),
264                 network_rx_total_mb=round(net_rx_tot, 3),
265                 network_tx_total_mb=round(net_tx_tot, 3),
266                 )

```

```
267 |         )
268 |     return results
269 |
```


ctl.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - CLI Con
  troller
4 # =====
5 # Description:
6 #   Консольный контроллер AITelemetryCtl для мониторинга, упра
  вления службой
7 #   Windows Service, проверки базы данных и выполнения диагнос
  тики.
8 #
9 # Usage Examples:
10 #   CLI:
11 #       AITelemetryCtl.exe status
12 #       AITelemetryCtl.exe start
13 #       AITelemetryCtl.exe stop
14 #       AITelemetryCtl.exe restart
15 #       AITelemetryCtl.exe diagnose
16 #       AITelemetryCtl.exe database
17 #
18 # File: ctl.py
19 # Project: ai-breadboard
20 # Package: apps.windows.telemetry
21 # Author: hypo69
22 # Copyright: © 2026 hypo69
23 # Updated: 2026-10-08 02:15:00
24 # =====
25
26 from __future__ import annotations
27 """Консольная утилита управления и диагностики службы AITeleme
  try."""
28
29 import argparse
```

```
30 import json
31 import os
32 import subprocess
33 import sys
34 from pathlib import Path
35 from typing import Any, Dict, List, Optional
36
37 _current_dir = Path(__file__).resolve().parent
38 _project_root = str(_current_dir.parents[2]) if len(_current_dir.parents) >= 3 else str(_current_dir)
39 if _project_root not in sys.path:
40     sys.path.insert(0, _project_root)
41
42
43 def get_service_status_scm(service_name: str = 'AITelemetry')
44     -> Dict[str, Any]:
45     """Запрашивает состояние службы через Windows SCM (sc.exe)
46     .
47
48     Args:
49         service_name: Имя службы в диспетчере Windows.
50
51     Returns:
52         Dict[str, Any]: Словарь со статусом и деталями службы.
53     """
54     try:
55         res = subprocess.run(
56             ['sc.exe', 'query', service_name],
57             capture_output=True,
58             text=True,
59             timeout=5
60         )
61         output = res.stdout
62         is_installed = res.returncode == 0
63         state = 'UNKNOWN'
64         if 'RUNNING' in output:
65             state = 'RUNNING'
66         elif 'STOPPED' in output:
```

```

65         state = 'STOPPED'
66     elif 'START_PENDING' in output:
67         state = 'START_PENDING'
68     elif 'STOP_PENDING' in output:
69         state = 'STOP_PENDING'
70     elif not is_installed:
71         state = 'NOT_INSTALLED'
72
73     return {
74         'installed': is_installed,
75         'state': state,
76         'raw_output': output.strip()
77     }
78     except Exception as ex:
79         return {'installed': False, 'state': 'ERROR', 'error':
80             str(ex)}
81
82 def cmd_status() -> int:
83     """Выводит подробный статус службы и базы данных."""
84     print("=" * 60)
85     print("        AITelemetry - Статус службы и мониторинга")
86     print("=" * 60)
87
88     scm_status = get_service_status_scm('AITelemetry')
89     state = scm_status.get('state', 'UNKNOWN')
90     state_icon = "■ " if state == 'RUNNING' else ("■ " if stat
91 e == 'STOPPED' else "■ ")
92     print(f"Служба Windows (SCM):    {state_icon} {state}")
93     print(f"Установлена в системе: {'Да' if scm_status.get('in
94 stalled') else 'Нет'}")
95
96     try:
97         from apps.windows.telemetry.telemetry_config import Te
98 lemetryConfigManager
99         cfg_mgr = TelemetryConfigManager()
100         cfg_path = cfg_mgr.config_path
101         print(f"Файл конфигурации:    {cfg_path} (существует:

```

```

    {cfg_path.exists()})")
99         print(f"Режим работы:           {cfg_mgr.get_mode()}")
100         print(f"Интервал замеров:       {cfg_mgr.get_interval_s
seconds()} сек")
101     except Exception as ex:
102         print(f"Конфигурация:           Ошибка чтения ({ex})")
103
104     try:
105         from apps.windows.telemetry.sqlite import TelemetrySto
rage
106         storage = TelemetryStorage.get_instance(read_only=True
)
107         db_path = storage.db_path
108         print(f"База данных SQLite:     {db_path} (размер: {db_
path.stat().st_size / 1024 / 1024:.2f} МБ)")
109         stats = storage.get_storage_stats()
110         print(f"Всего снимков (polls): {stats.get('snapshots_c
ount', 0)}")
111         print(f"Метрик процессов:       {stats.get('process_met
rics_count', 0)}")
112         print(f"Событий (events):       {stats.get('events_coun
t', 0)}")
113     except Exception as ex:
114         print(f"База данных:           Недоступна ({ex})")
115
116     print("=" * 60)
117     return 0
118
119
120 def cmd_control(action: str) -> int:
121     """Управляет службой через sc.exe / net.exe.
122
123     Args:
124         action: Действие ('start', 'stop', 'restart').
125
126     Returns:
127         int: Код возврата команды.
128     """

```

```
129 |     if action == 'start':
130 |         print("Запуск службы AITelemetry...")
131 |         res = subprocess.run(['net.exe', 'start', 'AITelemetry
|'])
132 |         return res.returncode
133 |     elif action == 'stop':
134 |         print("Остановка службы AITelemetry...")
135 |         res = subprocess.run(['net.exe', 'stop', 'AITelemetry'
|'])
136 |         return res.returncode
137 |     elif action == 'restart':
138 |         print("Перезапуск службы AITelemetry...")
139 |         subprocess.run(['net.exe', 'stop', 'AITelemetry'])
140 |         res = subprocess.run(['net.exe', 'start', 'AITelemetry
|'])
141 |         return res.returncode
142 |     return 1
143 |
144 |
145 | def cmd_database() -> int:
146 |     """Выводит детальную статистику таблиц SQLite."""
147 |     try:
148 |         from apps.windows.telemetry.sqlite import TelemetrySto
|rage
149 |         storage = TelemetryStorage.get_instance(read_only=True
|)
150 |         stats = storage.get_storage_stats()
151 |         print(json.dumps(stats, indent=2, ensure_ascii=False))
152 |         return 0
153 |     except Exception as ex:
154 |         print(f"Ошибка чтения базы данных: {ex}")
155 |         return 1
156 |
157 |
158 | def cmd_diagnose() -> int:
159 |     """Выполняет экспресс-диагностику системных метрик."""
160 |     print("Выполнение глубокой диагностики системы...")
161 |     try:
```

```

162         from apps.windows.telemetry.collector import SystemCol
lector
163         collector = SystemCollector()
164         snapshot = collector.collect_snapshot(top_processes_li
mit=10)
165         print(f"CPU: {snapshot.cpu.usage_percent:.1f}%")
166         print(f"Память: {snapshot.memory.percent_used:.1f}% (С
вободно: {snapshot.memory.available_mb / 1024:.2f} ГБ)")
167         print(f"Количество процессов: {len(snapshot.processes)
}")
168         print("Диагностика успешно выполнена.")
169         return 0
170     except Exception as ex:
171         print(f"Ошибка при диагностике: {ex}")
172         return 1
173
174
175 def cmd_security(args: argparse.Namespace) -> int:
176     """Обработчик группы команд ait security."""
177     from apps.windows.telemetry.security_collector import Wind
owsSecurityCollector
178     collector = WindowsSecurityCollector()
179     action = getattr(args, 'sec_action', 'status')
180
181     if action == 'status':
182         status = collector.check_access_and_audit()
183         print("=" * 60)
184         print("          AITelemetry - Windows Security Event Sub
system")
185         print("=" * 60)
186         acc_str = "□ Доступен" if status.accessible else f"□
Недоступен ({status.error})"
187         print(f"Журнал Security:          {acc_str}")
188         print(f"Всего событий в журнале ОС: {status.record_cou
nt:,}")
189         print(f"Событий в telemetry.db:      {status.total_inge
sted_events:,}")
190         print(f"Последний EventRecordID:    {status.last_recor

```

```

    d_id:,}")
191     cmd_audit_str = " Включен" if status.command_line_audit_enabled else " Отключен (ProcessCreationIncludeCmdLine_Enabled)"
192     print(f"Аудит командной строки 4688: {cmd_audit_str}")
193     print("=" * 60)
194     return 0
195
196     elif action == 'tail':
197         limit = getattr(args, 'limit', 20) or 20
198         eid = getattr(args, 'event_id', None)
199         search = getattr(args, 'search', '') or ''
200         events = collector.get_live_tail(limit=limit, event_id=eid, search=search)
201         print(f"\n[+] Последние {len(events)} событий Security (Live Tail):")
202         for ev in reversed(events):
203             print(f" [{ev.timestamp}] Event {ev.event_id:<4} | {ev.message}")
204         return 0
205
206     elif action == 'search':
207         eid = int(args.target) if str(args.target).isdigit() else None
208         limit = getattr(args, 'limit', 50) or 50
209         events = collector.storage.get_security_events(event_id=eid, limit=limit)
210         print(f"\n[+] Найдено {len(events)} событий для Event ID {eid}:")
211         for ev in events:
212             print(f" [{ev.get('timestamp')}] ID {ev.get('event_id')} | {ev.get('message')}")
213         return 0
214
215     elif action == 'user':
216         username = args.target
217         limit = getattr(args, 'limit', 50) or 50
218         events = collector.storage.get_security_events(user=username)

```

```

    urname, limit=limit)
219         print(f"\n[+] События безопасности для пользователя '{
    username}' ({len(events)} записей):")
220         for ev in events:
221             print(f"    [{ev.get('timestamp')}] Event {ev.get('e
    vent_id')} | {ev.get('message')}")
222         return 0
223
224     elif action == 'process':
225         pname = args.target
226         limit = getattr(args, 'limit', 50) or 50
227         events = collector.storage.get_security_process_creati
    ons(process_name=pname, limit=limit)
228         print(f"\n[+] Запуски процесса '{pname}' ({len(events)
    } записей):")
229         for ev in events:
230             print(f"    [{ev.get('timestamp')}] PID: {ev.get('pr
    ocess_id')} | User: {ev.get('subject_user')} | {ev.get('comman
    d_line')} or {ev.get('process_name')}")
231         return 0
232
233     elif action == 'failed-logons':
234         hours = getattr(args, 'hours', 24) or 24
235         events = collector.storage.get_security_failed_logons(
    hours=hours)
236         print(f"\n[+] Неудачные попытки входа за последние {ho
    urs} ч ({len(events)} записей):")
237         for ev in events:
238             print(f"    [{ev.get('timestamp')}] User: {ev.get('t
    arget_user')} | IP: {ev.get('source_ip')} | {ev.get('message')
    }")
239         return 0
240
241     elif action == 'collect':
242         batch_size = getattr(args, 'batch', 500) or 500
243         max_records = getattr(args, 'max', 5000) or 5000
244         save_raw = getattr(args, 'raw', False)
245         print(f"Запуск инкрементального сбора Security... (bat

```



```

ch={batch_size}, max={max_records}))")
246     report = collector.collect_incremental(batch_size=batch
h_size, max_records=max_records, save_raw=save_raw)
247     print(f"[+] Сбор завершен. Сохранено {report.total_eve
nts_ingested} событий. Последний RecordID: {report.last_record
_id}")
248     if report.events_by_id:
249         print("    Распределение по типам событий:")
250         for eid, count in sorted(report.events_by_id.items
()), key=lambda x: x[1], reverse=True)[:8]:
251             print(f"        Event {eid:<4}: {count} событий"
)
252     return 0
253
254     elif action == 'correlate':
255         pid = getattr(args, 'pid', None)
256         user = getattr(args, 'user', None)
257         corrs = collector.correlate_security_with_telemetry(pi
d=pid, user=user)
258         print(f"\n[+] Корреляция событий безопасности с телеме
трией ({len(corrs)} цепочек):")
259         for c in corrs[:20]:
260             notes_str = "; ".join(c.notes) if c.notes else "но
рмальная активность"
261             print(f"    * [{c.timestamp}] {c.event_type} | {c.us
er} -> {c.process_name} (PID {c.pid}) | {notes_str}")
262         return 0
263
264     return 0
265
266
267 def cmd_logs(args: argparse.Namespace) -> int:
268     """Обрабатывает подкоманды каталога журналов событий Windo
ws."""
269     from apps.windows.telemetry.event_catalog import WindowsEv
entCatalogEngine
270     engine = WindowsEventCatalogEngine()
271     action = getattr(args, 'logs_action', None) or 'domains'

```

```

272
273     if action == 'domains':
274         summary = engine.get_domains_summary()
275         print("\n" + "=" * 65)
276         print("      7 ДОМЕНОВ ТЕЛЕМЕТРИИ WINDOWS EVENT LOGS")
277         print("=" * 65)
278         print(f"Всего провайдеров в каталоге: {summary.get('total_registered_providers')}")
279         print(f"Служба Sysmon: {'☐ Активна' if summary.get('sysmon_installed') else '☐ Не обнаружена'}\n")
280         for dom in summary.get('domains', []):
281             print(f"      * {dom.get('title')} [{dom.get('status')}]]")
282             print(f"      Каналов: {dom.get('channels_count')} | Записей: {dom.get('total_records')}, } | ID: {dom.get('key_events')}")
283             print(f"      {dom.get('description')}\n")
284         return 0
285
286     elif action == 'catalog':
287         catalog = engine.get_full_catalog()
288         print("\n" + "=" * 65)
289         print(f"      КАТАЛОГ ПРОВАЙДЕРОВ WINDOWS EVENT LOG ({len(catalog)} источников)")
290         print("=" * 65)
291         for entry in catalog:
292             print(f"      [{entry.domain.value}] {entry.provider} -> {entry.channel}")
293             print(f"      Объем: {entry.volume_rating:<7} | Записей: {entry.record_count:<8}, } | ID: {list(entry.key_events.keys())}")
294             print(f"      {entry.description}\n")
295         return 0
296
297     elif action == 'provenance':
298         proc = getattr(args, 'process', None)
299         pid = getattr(args, 'pid', None)
300         graph = engine.reconstruct_process_provenance(process_

```

```

name=proc, pid=pid)
301     print("\n" + "=" * 65)
302     print("    ДЕРЕВО ПРОИСХОЖДЕНИЯ ПРОЦЕССОВ (PROCESS PROV
ENANCE)")
303     print("=" * 65)
304     print(f"Узлов: {graph.get('total_nodes')}, Связей: {gr
aph.get('total_links')}\n")
305     for node in graph.get('nodes', []):
306         print(f"    * [{node.get('type')}] {node.get('name')}
} (PID {node.get('pid')}) | User: {node.get('user')}")
307         if node.get('command_line'):
308             print(f"        CMD: {node.get('command_line')}")
    )
309     for link in graph.get('links', []):
310         print(f"    -> {link.get('source')} ==[{link.get(
'relationship')}]==> {link.get('target')}")
311     return 0
312
313     elif action == 'lifecycle':
314         hours = getattr(args, 'hours', 72) or 72
315         timeline = engine.reconstruct_power_lifecycle(hours=ho
urs)
316         print("\n" + "=" * 65)
317         print(f"    ЖИЗНЕННЫЙ ЦИКЛ ПИТАНИЯ ОС ЗА ПОСЛЕДНИЕ {hou
rs} Ч ({len(timeline)} событий)")
318         print("=" * 65)
319         for item in timeline:
320             print(f"    {item.get('icon')} [{item.get('timestamp
')}] {item.get('phase'):<18} | Event {item.get('event_id')} ({
item.get('provider')}) | {item.get('message')}")
321         return 0
322
323     elif action == 'sysmon':
324         status = engine.check_sysmon_status()
325         print("\n" + "=" * 65)
326         print("    СТАТУС И ВОЗМОЖНОСТИ SYSMON OBSERVABILITY LA
YER")
327         print("=" * 65)

```

```

328         print(f"Установлен: {' Да' if status.get('installle
|d') else ' Нет'}")
329         print(f"Канал логов: {status.get('channel_name')}")
330         print(f"Записей: {status.get('record_count'):,}")
331         print(f"Возможности: {' , '.join(status.get('capabilit
|ies', [])) or 'N/A'}")
332         print(f"Рекомендации: {status.get('recommended_config'
|)}}")
333         return 0
334
335     return 0
336
337
338 def main() -> int:
339     """Точка входа утилиты AITelemetryCtl."""
340     parser = argparse.ArgumentParser(description='AITelemetry
|CLI Controller')
341     subparsers = parser.add_subparsers(dest='command', help='К
|оманды управления')
342
343     subparsers.add_parser('status', help='Показать статус служ
|бы и БД')
344     subparsers.add_parser('start', help='Запустить службу')
345     subparsers.add_parser('stop', help='Остановить службу')
346     subparsers.add_parser('restart', help='Перезапустить служб
|у')
347     subparsers.add_parser('database', help='Показать статистик
|у базы данных')
348     subparsers.add_parser('diagnose', help='Выполнить экспресс
|-диагностику')
349
350     sec_parser = subparsers.add_parser('security', help='Управ
|ление и аналитика журнала безопасности Security.evtx')
351     sec_sub = sec_parser.add_subparsers(dest='sec_action', hel
|p='Действия security')
352
353     sec_sub.add_parser('status', help='Статус журнала безопасн
|ости и политик аудита')

```

```
354
355     tail_p = sec_sub.add_parser('tail', help='Свежие события б
езопасности в реальном времени')
356     tail_p.add_argument('--limit', type=int, default=20, help=
'Количество событий')
357     tail_p.add_argument('--event-id', type=int, default=None,
help='Фильтр по Event ID')
358     tail_p.add_argument('--search', type=str, default='', help
='Поисковая подстрока')
359
360     search_p = sec_sub.add_parser('search', help='Поиск по Eve
nt ID в сохраненной базе')
361     search_p.add_argument('target', help='Идентификатор событи
я (например 4624, 4688)')
362     search_p.add_argument('--limit', type=int, default=50, hel
p='Лимит записей')
363
364     user_p = sec_sub.add_parser('user', help='Поиск событий по
пользователю')
365     user_p.add_argument('target', help='Имя пользователя (напр
имер David, SYSTEM)')
366     user_p.add_argument('--limit', type=int, default=50, help=
'Лимит записей')
367
368     proc_p = sec_sub.add_parser('process', help='История созда
ния процессов (4688)')
369     proc_p.add_argument('target', help='Имя процесса (например
python.exe, cmd.exe)')
370     proc_p.add_argument('--limit', type=int, default=50, help=
'Лимит записей')
371
372     failed_p = sec_sub.add_parser('failed-logons', help='Неуда
чные попытки входа (4625)')
373     failed_p.add_argument('--hours', type=int, default=24, hel
p='Глубина выборки в часах')
374
375     coll_p = sec_sub.add_parser('collect', help='Запустить инк
рементальный сбор из Security.evtx')
```

```
376     coll_p.add_argument('--batch', type=int, default=500, help=  
    ='Размер пакета')  
377     coll_p.add_argument('--max', type=int, default=5000, help=  
    ='Максимум событий')  
378     coll_p.add_argument('--raw', action='store_true', help='Со  
    хранить сырой XML')  
379  
380     corr_p = sec_sub.add_parser('correlate', help='Корреляция  
    событий с метриками телеметрии')  
381     corr_p.add_argument('--pid', type=int, default=None, help=  
    ='Фильтр по PID')  
382     corr_p.add_argument('--user', type=str, default=None, help=  
    ='Фильтр по пользователю')  
383  
384     # Команда logs (Каталог 7 доменов, Provenance, Power Lifec  
    ycle)  
385     logs_p = subparsers.add_parser('logs', help='Иерархический  
    каталог Windows Event Logs, 7 доменов и графы происхождения')  
386     logs_sub = logs_p.add_subparsers(dest='logs_action', help=  
    'Действия logs')  
387     logs_sub.add_parser('domains', help='Сводка по 7 доменам т  
    елеметрии Windows Event Logs')  
388     logs_sub.add_parser('catalog', help='Полный реестр провайд  
    еров и каналов')  
389     prov_p = logs_sub.add_parser('provenance', help='Граф прои  
    схождения процессов (User -> Process -> Parent -> SCM)')  
390     prov_p.add_argument('--process', type=str, default=None, h  
    elp='Фильтр по имени процесса')  
391     prov_p.add_argument('--pid', type=int, default=None, help=  
    'Фильтр по PID')  
392     life_p = logs_sub.add_parser('lifecycle', help='Таймлайн ж  
    изненного цикла ОС (Boot/Sleep/Wake/Shutdown/Crash)')  
393     life_p.add_argument('--hours', type=int, default=72, help=  
    'Глубина выборки в часах')  
394     logs_sub.add_parser('sysmon', help='Проверить статус и воз  
    можности Sysmon')  
395  
396     args = parser.parse_args()
```

```
397 |
398 |     if not args.command or args.command == 'status':
399 |         return cmd_status()
400 |     elif args.command in ('start', 'stop', 'restart'):
401 |         return cmd_control(args.command)
402 |     elif args.command == 'database':
403 |         return cmd_database()
404 |     elif args.command == 'diagnose':
405 |         return cmd_diagnose()
406 |     elif args.command == 'security':
407 |         return cmd_security(args)
408 |     elif args.command == 'logs':
409 |         return cmd_logs(args)
410 |     return 0
411 |
412 |
413 | if __name__ == '__main__':
414 |     sys.exit(main())
415 |
416 |
```

deep_diagnostics.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - Deep Di
  agnostics
4 # =====
5 # Description:
6 #   Модуль глубокой аппаратной и системной диагностики, поведе
  нческой форензики и анализа утечек.
7 #
8 # Usage Examples:
9 #   Python API:
10 #
11 #   from apps.windows.telemetry.deep_diagnostics import DeepDi
  agnosticsEngine
12 #
13 #   service = DeepDiagnosticsEngine()
14 #
15 # File: deep_diagnostics.py
16 # Project: ai-breadboard
17 # Package: apps.windows.telemetry
18 # Author: hypo69
19 # Copyright: © 2026 hypo69
20 # Updated: 2026-10-08 04:11:30
21 # =====
22
23 """Модуль глубокой аппаратной и системной диагностики, поведе
  нческой форензики и анализа утечек.
24 """
25
26 from __future__ import annotations
27
28 import codecs
29 import ctypes
```



```
30 import ctypes.wintypes as wintypes
31 import os
32 import re
33 import struct
34 import subprocess
35 import sys
36 import time
37 from datetime import datetime, timezone, timedelta
38 from pathlib import Path
39 from typing import Any, Dict, List, Optional, Tuple
40
41 try:
42     import psutil
43     PSUTIL_AVAILABLE = True
44 except ImportError:
45     psutil = None
46     PSUTIL_AVAILABLE = False
47 try:
48     import winreg
49     WINREG_AVAILABLE = True
50 except ImportError:
51     winreg = None
52     WINREG_AVAILABLE = False
53 try:
54     from logger import logger
55 except ImportError:
56     from logger import logger
57
58 from apps.windows.telemetry.models import (
59     DefenderTelemetrySummary,
60     ExtendedSystemAuditReport,
61     ForensicsActivityReport,
62     KernelThrottlingReport,
63     PeripheralsNetworkReport,
64     ProcessLeakDiagnosticsReport,
65     ProcessLeakItem,
66     StartupTelemetrySummary,
67     StorageBatteryWearReport,
```

```

68     UserAccountsTelemetrySummary,
69     VssTelemetrySummary,
70 )
71
72 GR_GDIOBJECTS = 0
73 GR_USEROBJECTS = 1
74 PROCESS_QUERY_INFORMATION = 1024
75 PROCESS_VM_READ = 16
76
77 class LASTINPUTINFO(ctypes.Structure):
78     """Структура Win32 для GetLastInputInfo."""
79     _fields_ = [('cbSize', wintypes.UINT), ('dwTime', wintypes
80         .DWORD)]
81
82 class PROCESS_MEMORY_COUNTERS_EX(ctypes.Structure):
83     """Структура PSAPI для GetProcessMemoryInfo."""
84     _fields_ = [
85         ('cb', wintypes.DWORD),
86         ('PageFaultCount', wintypes.DWORD),
87         ('PeakWorkingSetSize', ctypes.c_size_t),
88         ('WorkingSetSize', ctypes.c_size_t),
89         ('QuotaPeakPagedPoolUsage', ctypes.c_size_t),
90         ('QuotaPagedPoolUsage', ctypes.c_size_t),
91         ('QuotaPeakNonPagedPoolUsage', ctypes.c_size_t),
92         ('QuotaNonPagedPoolUsage', ctypes.c_size_t),
93         ('PagefileUsage', ctypes.c_size_t),
94         ('PeakPagefileUsage', ctypes.c_size_t),
95         ('PrivateUsage', ctypes.c_size_t),
96     ]
97
98 class DeepDiagnosticsEngine:
99     """Движок глубокой системной диагностики и форензики."""
100
101     def __init__(self) -> None:
102         """Инициализация Win32 API интерфейсов."""
103         self._user32 = getattr(ctypes.windll, 'user32', None)
104         if os.name == 'nt' else None
105         self._kernel32 = getattr(ctypes.windll, 'kernel32', No

```

```

ne) if os.name == 'nt' else None
104         self._psapi = getattr(ctypes.windll, 'psapi', None) if
os.name == 'nt' else None
105
106     def collect_process_leaks(self, limit: int = 100) -> Proce
ssLeakDiagnosticsReport:
107         """Собрать метрики скрытых утечек процессов (Handles,
GDI, USER, Page Faults).
108
109         Args:
110             limit: Максимальное количество процессов в общем с
писке.
111
112         Returns:
113             ProcessLeakDiagnosticsReport: Отчет с ранжирование
м подозрительных процессов.
114         """
115         if not PSUTIL_AVAILABLE:
116             return ProcessLeakDiagnosticsReport()
117         items: List[ProcessLeakItem] = []
118         suspicious_count = 0
119         for proc in psutil.process_iter(['pid', 'name', 'statu
s', 'num_threads', 'cpu_percent', 'memory_info']):
120             try:
121                 info = proc.info
122                 pid = info.get('pid')
123                 if not pid or pid <= 4:
124                     continue
125                 name = info.get('name') or 'unknown'
126                 status = info.get('status') or 'running'
127                 threads_count = info.get('num_threads') or 1
128                 cpu_percent = info.get('cpu_percent') or 0.0
129                 mem_info = info.get('memory_info')
130                 memory_mb = round(mem_info.rss / (1024 * 1024)
, 2) if mem_info else 0.0
131                 handles_count = 0
132                 gdi_objects = 0
133                 user_objects = 0

```

```

134         page_faults = 0
135         peak_ws_mb = memory_mb
136         if self._kernel32 and self._user32:
137             h_process = self._kernel32.OpenProcess(PRO
CESS_QUERY_INFORMATION | PROCESS_VM_READ, False, pid)
138             if h_process:
139                 try:
140                     dw_handles = wintypes.DWORD()
141                     if self._kernel32.GetProcessHandle
Count(h_process, ctypes.byref(dw_handles)):
142                         handles_count = int(dw_handles
.value)
143                         gdi_objects = int(self._user32.Get
GuiResources(h_process, GR_GDIOBJECTS))
144                         user_objects = int(self._user32.Ge
tGuiResources(h_process, GR_USEROBJECTS))
145                         if self._psapi:
146                             counters = PROCESS_MEMORY_COUN
TERS_EX()
147                             counters.cb = ctypes.sizeof(PR
OCESS_MEMORY_COUNTERS_EX)
148                             if self._psapi.GetProcessMemor
yInfo(h_process, ctypes.byref(counters), counters.cb):
149                                 page_faults = int(counters
.PageFaultCount)
150                                 peak_ws_mb = round(counter
s.PeakWorkingSetSize / (1024 * 1024), 2)
151                                 finally:
152                                     self._kernel32.CloseHandle(h_proce
ss)
153                 if handles_count == 0:
154                     try:
155                         handles_count = proc.num_handles() if
hasattr(proc, 'num_handles') else 0
156                     except Exception:
157                         handles_count = 0
158                 risk_reasons: List[str] = []
159                 risk_score = 'normal'

```

```
160         if handles_count >= 4000:
161             risk_score = 'critical'
162             risk_reasons.append(f'Критическое число де
скрипторов: {handles_count}')
163         elif handles_count >= 1500:
164             if risk_score != 'critical':
165                 risk_score = 'warning'
166             risk_reasons.append(f'Высокое число дескри
пторов: {handles_count}')
167         if gdi_objects >= 2000:
168             risk_score = 'critical'
169             risk_reasons.append(f'Угроза исчерпания GD
I пула: {gdi_objects} (лимит 10 000)')
170         elif gdi_objects >= 800:
171             if risk_score != 'critical':
172                 risk_score = 'warning'
173             risk_reasons.append(f'Повышенное использо
вание GDI: {gdi_objects}')
174         if user_objects >= 500:
175             if risk_score != 'critical':
176                 risk_score = 'warning'
177             risk_reasons.append(f'Много USER объектов:
{user_objects}')
178         if page_faults >= 2000000:
179             risk_reasons.append(f'Высокая нагрузка на
подкачку ({page_faults:,} Hard Faults)')
180         if memory_mb >= 2048:
181             risk_reasons.append(f'Высокое потребление
памяти: {memory_mb:.1f} MB')
182         if risk_score != 'normal':
183             suspicious_count += 1
184         item = ProcessLeakItem(
185             pid=pid,
186             name=name,
187             status=status,
188             handles_count=handles_count,
189             gdi_objects=gdi_objects,
190             user_objects=user_objects,
```

```

191         page_faults_total=page_faults,
192         peak_working_set_mb=peak_ws_mb,
193         memory_mb=memory_mb,
194         threads_count=threads_count,
195         cpu_percent=cpu_percent,
196         leak_risk_score=risk_score,
197         leak_risk_reasons=risk_reasons,
198     )
199     items.append(item)
200     except (psutil.NoSuchProcess, psutil.AccessDenied,
201            psutil.ZombieProcess):
202         continue
203     except Exception as ex:
204         logger.debug(f'Ошибка сбора дескрипторов проце
205            cca: {ex}')
206     top_handles = sorted(items, key=lambda x: x.handles_co
207        unt, reverse=True)[:10]
208     top_gdi = sorted(items, key=lambda x: x.gdi_objects, r
209        everse=True)[:10]
210     top_page_faults = sorted(items, key=lambda x: x.page_f
211        aults_total, reverse=True)[:10]
212     risk_priority = {'critical': 0, 'warning': 1, 'normal'
213        : 2}
214     sorted_all = sorted(items, key=lambda x: (risk_priorit
215        y.get(x.leak_risk_score, 3), -x.handles_count))[:limit]
216     return ProcessLeakDiagnosticsReport(
217         timestamp=datetime.now(timezone.utc).isoformat(),
218         total_processes=len(items),
219         suspicious_count=suspicious_count,
220         top_handle_hogs=top_handles,
221         top_gdi_hogs=top_gdi,
222         top_page_fault_hogs=top_page_faults,
223         all_processes=sorted_all,
224     )
225
226     def collect_forensics_activity(self) -> ForensicsActivityR
227     eport:
228         """Собрать данные активности пользователя, активного о

```

```

    кна, доступа к сенсорам и UserAssist."""
221         fg_window: Dict[str, Any] = {'title': 'Рабочий стол /
Системный интерфейс', 'process_name': 'explorer.exe', 'pid': 0
    }
222         idle_seconds = 0.0
223         if self._user32:
224             try:
225                 hwnd = self._user32.GetForegroundWindow()
226                 if hwnd:
227                     length = self._user32.GetWindowTextLengthW
(hwnd)
228                     buff = ctypes.create_unicode_buffer(length
+ 1)
229                     self._user32.GetWindowTextW(hwnd, buff, le
ngth + 1)
230                     title = buff.value.strip() or 'Без заголов
ка'
231                     lp_pid = wintypes.DWORD()
232                     self._user32.GetWindowThreadProcessId(hwnd
, ctypes.byref(lp_pid))
233                     pid = int(lp_pid.value)
234                     proc_name = 'unknown'
235                     if PSUTIL_AVAILABLE and pid > 0:
236                         try:
237                             proc_name = psutil.Process(pid).na
me()
238                         except Exception:
239                             proc_name = f'PID {pid}'
240                     fg_window = {'title': title, 'process_name
': proc_name, 'pid': pid}
241             except Exception as ex:
242                 logger.debug(f'Ошибка получения ForegroundWind
ow: {ex}')
243         if self._user32 and self._kernel32:
244             try:
245                 lii = LASTINPUTINFO()
246                 lii.cbSize = ctypes.sizeof(LASTINPUTINFO)
247                 if self._user32.GetLastInputInfo(ctypes.byref(

```

```

l ii)):
248         tick_count = self._kernel32.GetTickCount()
249         idle_ms = max(0, tick_count - l ii.dwTime)
250         idle_seconds = round(idle_ms / 1000.0, 1)
251     except Exception as ex:
252         logger.debug(f'Ошибка расчета UserIdleTime: {e
x}')
253     cam_apps, mic_apps = self._get_camera_mic_access()
254     userassist_apps = self._get_userassist_apps(limit=15)
255     return ForensicsActivityReport(
256         timestamp=datetime.now(timezone.utc).isoformat(),
257         foreground_window=fg_window,
258         user_idle_seconds=idle_seconds,
259         camera_active_apps=cam_apps,
260         microphone_active_apps=mic_apps,
261         userassist_top_apps=userassist_apps,
262     )
263
264     def _get_camera_mic_access(self) -> Tuple[List[Dict[str, A
ny]], List[Dict[str, Any]]:
265         """Определить приложения, использующие камеру и микроф
он через реестр Windows."""
266         cam_list: List[Dict[str, Any]] = []
267         mic_list: List[Dict[str, Any]] = []
268         if not WINREG_AVAILABLE:
269             return (cam_list, mic_list)
270         capabilities = [('webcam', cam_list), ('microphone', m
ic_list)]
271         base_path = r'Software\Microsoft\Windows\CurrentVersio
n\CapabilityAccessManager\ConsentStore'
272         for cap_name, target_list in capabilities:
273             try:
274                 cap_key_path = f'{base_path}\\{cap_name}'
275                 with winreg.OpenKey(winreg.HKEY_CURRENT_USER,
cap_key_path, 0, winreg.KEY_READ) as key:
276                     num_subkeys, _, _ = winreg.QueryInfoKey(ke
y)
277                     for i in range(num_subkeys):

```



```

278         try:
279             sub_name = winreg.EnumKey(key, i)
280             if sub_name == 'NonPackaged':
281                 non_pkg_path = f'{cap_key_path
|}\\NonPackaged'
282                 with winreg.OpenKey(winreg.HKEY_
|CURRENT_USER, non_pkg_path, 0, winreg.KEY_READ) as np_key:
283                     np_count, _, _ = winreg.Qu
|eryInfoKey(np_key)
284                     for j in range(np_count):
285                         try:
286                             app_sub = winreg.E
|numKey(np_key, j)
287                             with winreg.OpenKe
|y(np_key, app_sub, 0, winreg.KEY_READ) as app_key:
288                                 stop_val, _ =
|winreg.QueryValueEx(app_key, 'LastUsedTimeStop')
289                                 start_val, _ =
|winreg.QueryValueEx(app_key, 'LastUsedTimeStart')
290                                 is_active = st
|op_val == 0 and start_val > 0
291                                 clean_app_name
|= app_sub.replace('#', '\\').split('\\')[-1]
292                                 target_list.ap
|pend({
293                                     'app_name'
|: clean_app_name,
294                                     'full_path
|': app_sub.replace('#', '\\'),
295                                     'is_active
|_now': is_active,
296                                     'type': 'W
|in32 Executable',
297                                 })
298             except Exception:
299                 continue
300         else:
301             with winreg.OpenKey(key, sub_n

```

```

    ame, 0, winreg.KEY_READ) as uwp_key:
302         try:
303             stop_val, _ = winreg.Q
QueryValueEx(uwp_key, 'LastUsedTimeStop')
304             start_val, _ = winreg.
QueryValueEx(uwp_key, 'LastUsedTimeStart')
305             is_active = stop_val =
= 0 and start_val > 0
306             target_list.append({
307                 'app_name': sub_na
me,
308                 'full_path': sub_n
ame,
309                 'is_active_now': i
s_active,
310                 'type': 'UWP / Win
dows App',
311             })
312         except Exception:
313             continue
314     except Exception:
315         continue
316     except Exception as ex:
317         logger.debug(f'Ошибка проверки доступа к {cap_
name}: {ex}')
318     return (cam_list, mic_list)
319
320     def _get_userassist_apps(self, limit: int = 15) -> List[Di
ct[str, Any]]:
321         """Извлечь историю запусков и фокусное время из ключей
UserAssist реестра (ROT13)."""
322         apps: List[Dict[str, Any]] = []
323         if not WINREG_AVAILABLE:
324             return apps
325         ua_path = r'Software\Microsoft\Windows\CurrentVersion\
Explorer\UserAssist'
326         try:
327             with winreg.OpenKey(winreg.HKEY_CURRENT_USER, ua_p

```



```

me,
352         'run_count': run_c
ount,
353         'focus_seconds': f
ocus_seconds,
354         'focus_formatted':
f'{focus_seconds // 60} мин {focus_seconds % 60} с',
355         })
356     except Exception:
357         continue
358     except Exception:
359         continue
360     except Exception as ex:
361         logger.debug(f'Ошибка UserAssist: {ex}')
362     apps.sort(key=lambda x: (x['run_count'], x['focus_seco
nds']), reverse=True)
363     return apps[:limit]
364
365     def collect_kernel_throttling(self) -> KernelThrottlingRep
ort:
366         """Собрать данные по прерываниям DPC/ISR, троттлингу,
Uptime и сбоям."""
367         dpc_pct = 0.0
368         interrupt_pct = 0.0
369         uptime_sec = 0.0
370         uptime_str = '0 дн 0 ч'
371         thermal_throttling = False
372         power_throttling = False
373         bsod_list: List[Dict[str, Any]] = []
374         pcie_info: Dict[str, Any] = {'gpu_name': 'PCIe Device'
, 'current_link_speed': 'Gen 3/4', 'current_link_width': 'x16'
, 'status': 'Optimal'}
375         if PSUTIL_AVAILABLE:
376             try:
377                 boot_time = psutil.boot_time()
378                 uptime_sec = max(0.0, time.time() - boot_time)
379                 days = int(uptime_sec // 86400)
380                 hours = int(uptime_sec % 86400 // 3600)

```

```

381         mins = int(uptime_sec % 3600 // 60)
382         uptime_str = f'{days} дн {hours} ч {mins} мин'
383         cpu_times = psutil.cpu_times_percent(interval=
0.1)
384         dpc_pct = round(getattr(cpu_times, 'dpc', 0.0)
    or 0.0, 2)
385         interrupt_pct = round(getattr(cpu_times, 'inte
rrupt', 0.0) or 0.0, 2)
386         except Exception as ex:
387             logger.debug(f'Ошибка сбора cpu_times: {ex}')
388         dpc_status = 'optimal'
389         if dpc_pct > 5.0 or interrupt_pct > 3.0:
390             dpc_status = 'severe'
391         elif dpc_pct > 1.5 or interrupt_pct > 1.0:
392             dpc_status = 'elevated'
393         minidump_dir = Path(os.environ.get('SystemRoot', 'C:\\
Windows')) / 'Minidump'
394         if minidump_dir.exists() and minidump_dir.is_dir():
395             try:
396                 dumps = sorted(minidump_dir.glob('*.dmp'), key
=lambda p: p.stat().st_mtime, reverse=True)[:5]
397                 for d in dumps:
398                     mtime = datetime.fromtimestamp(d.stat().st
_mtime, tz=timezone.utc).isoformat()
399                     bsod_list.append({
400                         'file_name': d.name,
401                         'timestamp': mtime,
402                         'size_kb': round(d.stat().st_size / 10
24, 1),
403                         'type': 'Crash Dump (BugCheck)',
404                     })
405             except Exception as ex:
406                 logger.debug(f'Ошибка проверки Minidump: {ex}'
    )
407         try:
408             from apps.windows.telemetry.sensors import get_har
dware_sensors
409             sensors = get_hardware_sensors()

```

```

410         for s in sensors:
411             name_l = s.name.lower()
412             if 'prochot' in name_l or 'thermal throttling'
in name_l:
413                 if s.value > 0:
414                     thermal_throttling = True
415                 if 'power limit' in name_l or 'pl1' in name_l
or 'pl2' in name_l:
416                     if s.value > 0:
417                         power_throttling = True
418         except Exception:
419             pass
420         return KernelThrottlingReport(
421             timestamp=datetime.now(timezone.utc).isoformat(),
422             dpc_latency_pct=dpc_pct,
423             interrupt_latency_pct=interrupt_pct,
424             dpc_status=dpc_status,
425             thermal_throttling_detected=thermal_throttling,
426             power_limit_throttling_detected=power_throttling,
427             system_uptime_seconds=uptime_sec,
428             uptime_formatted=uptime_str,
429             last_bsod_crashes=bsod_list,
430             gpu_pcie_link=pcie_info,
431         )
432
433     def collect_storage_battery_wear(self) -> StorageBatteryWe
arReport:
434         """Собрать данные износа SSD/NVMe и батареи питания."""
435
436         disks: List[Dict[str, Any]] = []
437         battery_data: Dict[str, Any] = {
438             'has_battery': False,
439             'design_capacity_mwh': 0.0,
440             'full_charge_capacity_mwh': 0.0,
441             'wear_level_pct': 0.0,
442             'cycle_count': 0,
443             'charge_rate_mw': 0.0,
444             'is_charging': False,

```

```

444         'percent': 0.0,
445         'power_source': 'AC Mains (Стационарное питание)',
446     }
447     try:
448         from apps.windows.sdk.modules.storage_manager.core
449         .windows_storage_sensor import WindowsStorageSensor
450         sensor = WindowsStorageSensor()
451         snapshot = sensor.collect_snapshot(sync=True)
452         raw_partitions = snapshot.get('sources', {}).get('
partitions', [])
453         raw_logical = snapshot.get('sources', {}).get('log
454         ical_disks', [])
455
456         # Извлечем сведения о логических дисках (буква том
457         а -> ФС и свободное место)
458         logical_info: Dict[str, Dict[str, Any]] = {}
459         for log_d in raw_logical:
460             if isinstance(log_d, dict):
461                 ld_id = str(log_d.get('DeviceID') or '').u
462                 pper().rstrip('\\')
463                 if ld_id:
464                     logical_info[ld_id] = {
465                         'fs': log_d.get('FileSystem') or '
466                         NTFS',
467                         'free_bytes': int(log_d.get('FreeS
468                         pace') or 0),
469                     }
470
471         part_map: Dict[str, List[str]] = {}
472         part_free_map: Dict[str, float] = {}
473         for p in raw_partitions:
474             if isinstance(p, dict):
475                 dn = p.get('DiskNumber')
476                 dl = p.get('DriveLetter')
477                 if dn is not None and dl:
478                     dn_str = str(dn)
479                     letter = f'{dl}:' .upper()
480                     fs_name = logical_info.get(letter, {})

```

```

    .get('fs', 'NTFS')
475         part_map.setdefault(dn_str, []).append
    (f'{letter} [{fs_name}]')
476         free_b = logical_info.get(letter, {}).
    get('free_bytes', 0)
477         if free_b > 0:
478             part_free_map[dn_str] = part_free_
    map.get(dn_str, 0.0) + (free_b / 1024 ** 3)
479
480         io_counters = psutil.disk_io_counters(perdisk=True
    ) if PSUTIL_AVAILABLE and hasattr(psutil, 'disk_io_counters')
    else {}
481         now_dt = datetime.now(timezone.utc)
482         physical_disks = sensor.get_physical_disks(sync=Tr
    ue)
483         if physical_disks:
484             for d in physical_disks:
485                 dev_num_str = ''.join(filter(str.isdigit,
    d.device_id))
486                 mounted = part_map.get(dev_num_str, [])
487                 has_free = dev_num_str in part_free_map
488                 free_total = part_free_map.get(dev_num_str
    , 0.0)
489                 if not has_free and PSUTIL_AVAILABLE and m
    ounted:
490                     for part_label in mounted:
491                         m_letter = part_label.split()[0] i
    f part_label else ''
492                         if m_letter and ':' in m_letter:
493                             try:
494                                 u = psutil.disk_usage(f'{m
    _letter}\\')
495                                 free_total += (u.free / 10
    24 ** 3)
496                                 has_free = True
497                             except Exception:
498                                 pass
499                 io_key = f'PhysicalDrive{dev_num_str}'

```



```

500         io = io_counters.get(io_key)
501         bytes_read = io.read_bytes if io else (d.l
ifetime_read_bytes or 0)
502         bytes_written = io.write_bytes if io else
(d.lifetime_write_bytes or 0)
503         read_count = io.read_count if io else (d.r
ead_errors_total or 0)
504         write_count = io.write_count if io else (d
.write_errors_total or 0)
505         poh = d.power_on_hours
506         first_on_str = (now_dt - timedelta(hours=i
nt(poh))).strftime('%Y-%m-%d') if poh and poh > 0 else None
507         health_pct = 100.0
508         if d.wear_percentage is not None and d.wear_percentage > 0:
509             health_pct = max(0.0, min(100.0, 100.0
- float(d.wear_percentage)))
510             elif d.health_status and d.health_status.l
ower() in ('warning', 'caution'):
511                 health_pct = 70.0
512             elif d.health_status and d.health_status.l
ower() in ('unhealthy', 'critical', 'failed'):
513                 health_pct = 20.0
514                 status_str = 'Healthy (SMART OK)'
515                 if d.health_status and d.health_status not
in ('Healthy', 'OK', '0', 'PASSED'):
516                     status_str = d.health_status
517                 disks.append({
518                     'device_id': f'Disk {dev_num_str}' if
dev_num_str else d.device_id,
519                     'name': d.model or d.friendly_name or
'Физический накопитель',
520                     'model': d.model or d.friendly_name,
521                     'serial_number': d.serial_number if d.
serial_number and d.serial_number != 'N/A' else None,
522                     'bus_type': d.bus_type or 'Unknown',
523                     'media_type': d.media_type or 'SSD',
524                     'partitions': ', '.join(mounted) if mo

```



```

    ** 3, 1),
556         'health_pct': 100.0,
557         'wear_level_pct': 0.0,
558         'temperature_c': None,
559         'power_on_hours': None,
560         'status': 'Healthy (SMART OK)',
561     })
562     except Exception:
563         continue
564     except Exception as ex:
565         logger.debug(f'Ошибка сбора дисков (psutil fal
lback): {ex}')
566     if PSUTIL_AVAILABLE and hasattr(psutil, 'sensors_batte
ry'):
567         try:
568             bat = psutil.sensors_battery()
569             if bat is not None:
570                 battery_data['has_battery'] = True
571                 battery_data['percent'] = round(bat.percen
t, 1)
572                 battery_data['is_charging'] = bat.power_pl
ugged
573                 battery_data['power_source'] = 'AC Adapter
' if bat.power_plugged else 'Battery Discharge'
574                 battery_data['secsleft'] = bat.secsleft if
bat.secsleft != psutil.POWER_TIME_UNLIMITED else -1
575             except Exception as ex:
576                 logger.debug(f'Ошибка опроса батареи: {ex}')
577             if battery_data['has_battery']:
578                 try:
579                     ps_cmd = 'Get-CimInstance -Namespace root/wmi
-ClassName BatteryStaticData -ErrorAction SilentlyContinue | S
elect-Object DesignedCapacity, FullChargeCapacity | ConvertTo-
Json'
580                     res = subprocess.run(f'powershell.exe -NoProfi
le -NonInteractive -Command "{ps_cmd}"', capture_output=True,
text=True, timeout=5, shell=True)
581                     if res.returncode == 0 and res.stdout.strip():

```

```

582         import json
583         b_json = json.loads(res.stdout.strip())
584         if isinstance(b_json, dict):
585             des = float(b_json.get('DesignedCapacity') or 0.0)
586             full = float(b_json.get('FullChargeCapacity') or 0.0)
587             if des > 0 and full > 0:
588                 battery_data['design_capacity_mwh']
589                 ] = des
590                 battery_data['full_charge_capacity_mwh'] = full
591                 wear = max(0.0, min(100.0, (1.0 -
592                 full / des) * 100.0))
593                 battery_data['wear_level_pct'] = round(wear, 1)
594             except Exception:
595                 pass
596             return StorageBatteryWearReport(timestamp=datetime.now(
597             (timezone.utc).isoformat(), disks_wear=disks, battery_wear=battery_data)
598
599     def collect_peripherals_network(self) -> PeripheralsNetworkReport:
600         """Собрать данные по USB устройствам, беспроводной сети Wi-Fi и аудиоустройствам."""
601         usb_devices: List[Dict[str, Any]] = []
602         wifi_data: Dict[str, Any] = {
603             'is_connected': False,
604             'ssid': 'Не подключено',
605             'bssid': '--',
606             'signal_pct': 0,
607             'rssi_dbm': -100,
608             'channel': 0,
609             'radio_type': '--',
610             'auth_cipher': '--',
611         }
612         audio_endpoints: List[Dict[str, Any]] = []

```

```

610         try:
611             from apps.windows.telemetry.win32_ffi.setupapi import SetupAPI
612             pnp = SetupAPI()
613             devs = pnp.get_all_devices()
614             for d in devs:
615                 hw_id = d.hardware_id or ''
616                 if 'USB' in hw_id.upper() or 'USB' in (d.device_class or '').upper():
617                     vid_match = re.search('VID_([0-9A-Fa-f]{4})', hw_id)
618                     pid_match = re.search('PID_([0-9A-Fa-f]{4})', hw_id)
619                     vid = vid_match.group(1) if vid_match else
620                         '----'
621                     pid = pid_match.group(1) if pid_match else
622                         '----'
623                     usb_devices.append({
624                         'name': d.friendly_name or 'USB Устройство',
625                         'device_id': d.device_instance_id,
626                         'vendor_id': vid,
627                         'product_id': pid,
628                         'device_class': d.device_class,
629                         'has_problem': d.has_problem,
630                         'status': 'Error' if d.has_problem else 'Connected & Active',
631                     })
632             except Exception as ex:
633                 logger.debug(f'Ошибка сбора USB через SetupAPI: {ex}')
634             try:
635                 res = subprocess.run('netsh wlan show interfaces',
636                                     capture_output=True, text=True, timeout=4, shell=True)
637                 if res.returncode == 0 and res.stdout:
638                     out = res.stdout
639                     state_match = re.search(r'State\s*:\s*connected|Состояние\s*:\s*подключено', out, re.IGNORECASE)

```

```

637         if state_match:
638             wifi_data['is_connected'] = True
639             ssid_m = re.search(r'SSID\s*:\s*(.+)', out
        )
640             bssid_m = re.search(r'BSSID\s*:\s*(.+)', out)
641             sig_m = re.search(r'Signal|Сигнал\s*:\s*(\
        d+)\%', out)
642             chan_m = re.search(r'Channel|Канал\s*:\s*(
        \d+)\%', out)
643             radio_m = re.search(r'Radio type|Тип радио
        модуля\s*:\s*(.+)', out)
644             auth_m = re.search(r'Authentication|Провер
        ка подлинности\s*:\s*(.+)', out)
645             if ssid_m:
646                 wifi_data['ssid'] = ssid_m.group(1).st
        rip()
647             if bssid_m:
648                 wifi_data['bssid'] = bssid_m.group(1).
        strip()
649             if sig_m:
650                 sig_pct = int(sig_m.group(1))
651                 wifi_data['signal_pct'] = sig_pct
652                 wifi_data['rssi_dbm'] = int(sig_pct /
        2.0 - 100)
653             if chan_m:
654                 wifi_data['channel'] = int(chan_m.grou
        p(1))
655             if radio_m:
656                 wifi_data['radio_type'] = radio_m.grou
        p(1).strip()
657             if auth_m:
658                 wifi_data['auth_cipher'] = auth_m.grou
        p(1).strip()
659         except Exception as ex:
660             logger.debug(f'Ошибка Wi-Fi netsh: {ex}')
661         try:
662             audio_endpoints.append({'name': 'Основное устройст

```

```

        во воспроизведения (Default Playback)', 'type': 'Speakers / He
adphones', 'is_default': True, 'status': 'Active'})
663         audio_endpoints.append({'name': 'Основное устройст
во записи (Default Recording)', 'type': 'Microphone', 'is_defa
ult': True, 'status': 'Active'})
664     except Exception:
665         pass
666     return PeripheralsNetworkReport(timestamp=datetime.now
(timezone.utc).isoformat(), usb_devices=usb_devices, wifi_tele
metry=wifi_data, audio_endpoints=audio_endpoints)
667
668     def collect_defender_telemetry(self) -> DefenderTelemetryS
ummary:
669         """Собрать сводку телеметрии Windows Defender (CFA, AS
R, исключения, угрозы)."""
670         summary = DefenderTelemetrySummary()
671         try:
672             from apps.windows.sdk.modules.defender.core.cfa_ma
nager import ControlledFolderAccessManager
673             cfa_mgr = ControlledFolderAccessManager()
674             cfa_info = cfa_mgr.get_cfa_status()
675             summary.cfa_enabled = bool(cfa_info.enabled)
676             summary.cfa_mode = str(cfa_info.mode.value if hasa
ttr(cfa_info.mode, 'value') else cfa_info.mode)
677             summary.cfa_protected_folders_count = len(cfa_info
.protected_folders)
678         except Exception as ex:
679             logger.debug(f'Ошибка сбора статуса CFA Defender:
{ex}')
680         try:
681             from apps.windows.sdk.modules.defender.core.asr_ma
nager import ASRManager
682             asr_mgr = ASRManager()
683             rules = asr_mgr.get_asr_rules()
684             summary.asr_rules_count = len(rules)
685             summary.asr_enabled_count = len([r for r in rules
if getattr(r, 'state', None) and str(r.state).lower() in ('ena
bled', 'block', 'audit', 'warn')])

```

```

686         except Exception as ex:
687             logger.debug(f'Ошибка сбора правил ASR Defender: {
ex}')
688         try:
689             from apps.windows.sdk.modules.defender.core.exclus
ions_auditor import ExclusionsAuditor
690             excl_auditor = ExclusionsAuditor()
691             excl = excl_auditor.audit_exclusions()
692             paths = getattr(excl, 'path_exclusions', []) or []
693             processes = getattr(excl, 'process_exclusions', []
) or []
694             summary.path_exclusions = [str(p.value if hasattr(
p, 'value') else p) for p in paths]
695             summary.process_exclusions = [str(p.value if hasat
tr(p, 'value') else p) for p in processes]
696             summary.exclusions_count = len(summary.path_exclus
ions) + len(summary.process_exclusions)
697         except Exception as ex:
698             logger.debug(f'Ошибка сбора исключений Defender: {
ex}')
699         try:
700             from apps.windows.sdk.modules.defender.core.threat
_manager import ThreatManager
701             threat_mgr = ThreatManager()
702             threat_history = threat_mgr.get_threats_history(li
mit=50)
703             active_threats = [t for t in threat_history if str
(getattr(t, 'status', '')).lower() == 'active']
704             summary.active_threats_count = len(active_threats)
705             summary.threat_history_count = len(threat_history)
706         except Exception as ex:
707             logger.debug(f'Ошибка сбора угроз Defender: {ex}')
708         return summary
709
710     def collect_startup_telemetry(self) -> StartupTelemetrySum
mary:
711         """Собрать срез точек автозапуска и персистентности пр
ограмм."""

```



```

712         summary = StartupTelemetrySummary()
713     try:
714         from apps.windows.startup.core.scanner import Star
715         tupScanner
716         scanner = StartupScanner()
717         entries = scanner.scan_all()
718         summary.total_entries = len(entries)
719         serialized_entries: List[Dict[str, Any]] = []
720         for e in entries:
721             loc_type = str(getattr(e, 'location_type', ''))
722             if 'REGISTRY' in loc_type:
723                 summary.registry_run_count += 1
724             elif 'STARTUP_FOLDER' in loc_type:
725                 summary.startup_folders_count += 1
726             elif 'SCHEDULED_TASK' in loc_type:
727                 summary.scheduled_tasks_count += 1
728             elif 'SERVICE' in loc_type:
729                 summary.startup_services_count += 1
730             serialized_entries.append({
731                 'name': getattr(e, 'name', 'Unknown'),
732                 'command': getattr(e, 'command_line', ''),
733                 'location_type': loc_type,
734                 'is_enabled': getattr(e, 'is_enabled', True),
735                 'publisher': getattr(e, 'publisher', ''),
736             })
737         summary.entries = serialized_entries
738     except Exception as ex:
739         logger.debug(f'Ошибка сбора автозагрузки: {ex}')
740     return summary
741
742 def collect_vss_telemetry(self) -> VssTelemetrySummary:
743     """Собрать состояние теневых копий томов VSS."""
744     summary = VssTelemetrySummary()
745     try:
746         from apps.windows.sdk.modules.backup_manager.core.
747         vss_manager import VssManager

```

```

746         vss = VssManager()
747         snapshots = vss.list_snapshots()
748         summary.total_snapshots_count = len(snapshots)
749         vols: List[str] = []
750         serialized_snaps: List[Dict[str, Any]] = []
751         for s in snapshots:
752             vol = getattr(s, 'original_volume', '')
753             if vol and vol not in vols:
754                 vols.append(vol)
755             serialized_snaps.append({
756                 'snapshot_id': getattr(s, 'snapshot_id', '
'),
757                 'original_volume': vol,
758                 'creation_time': getattr(s, 'creation_time
', None),
759                 'shadow_volume_path': getattr(s, 'shadow_v
olume_path', None),
760             })
761         summary.protected_volumes = vols
762         summary.snapshots = serialized_snaps
763         if snapshots and hasattr(snapshots[-1], 'creation_
time'):
764             summary.latest_snapshot_time = snapshots[-1].c
reation_time
765         except Exception as ex:
766             logger.debug(f'Ошибка сбора теневых копий VSS: {ex
}')
767         return summary
768
769     def collect_users_telemetry(self) -> UserAccountsTelemetry
Summary:
770         """Собрать аудит локальных пользователей и привилегий.
771         """
772         summary = UserAccountsTelemetrySummary()
773         try:
774             from apps.windows.sysadmin.src.user_collector impo
rt WindowsUserCollector
775             collector = WindowsUserCollector()

```

```

775 |         users = collector.get_all_users()
776 |         summary.total_users_count = len(users)
777 |         admins: List[str] = []
778 |         logged_in: List[str] = []
779 |         locked_count = 0
780 |         serialized_users: List[Dict[str, Any]] = []
781 |         for u in users:
782 |             uname = getattr(u, 'name', '')
783 |             is_admin = getattr(u, 'is_admin', False)
784 |             is_logged = getattr(u, 'is_logged_in', False)
785 |             is_locked = getattr(u, 'lockout_status', False
    | ) or not getattr(u, 'enabled', True)
786 |             if is_admin:
787 |                 admins.append(uname)
788 |             if is_logged:
789 |                 logged_in.append(uname)
790 |             if is_locked:
791 |                 locked_count += 1
792 |             serialized_users.append({
793 |                 'name': uname,
794 |                 'full_name': getattr(u, 'full_name', ''),
795 |                 'is_admin': is_admin,
796 |                 'is_logged_in': is_logged,
797 |                 'enabled': getattr(u, 'enabled', True),
798 |                 'account_type': getattr(u, 'account_type',
    | 'Local'),
799 |                 'last_logon': getattr(u, 'last_logon', Non
    | e),
800 |             })
801 |         summary.admin_users_count = len(admins)
802 |         summary.admin_usernames = admins
803 |         summary.logged_in_users = logged_in
804 |         summary.locked_users_count = locked_count
805 |         summary.users = serialized_users
806 |     except Exception as ex:
807 |         logger.debug(f'Ошибка сбора локальных пользователе
    | й: {ex}')
808 |     return summary

```

```
809 |
810 |     def collect_extended_system_audit(self) -> ExtendedSystemA
      | uditReport:
811 |         """Собрать комплексный расширенный аудит безопасности,
      | автозагрузки, VSS и пользователей."""
812 |         import socket
813 |         return ExtendedSystemAuditReport(
814 |             timestamp=datetime.now(timezone.utc).isoformat(),
815 |             hostname=socket.gethostname(),
816 |             defender=self.collect_defender_telemetry(),
817 |             startup=self.collect_startup_telemetry(),
818 |             vss=self.collect_vss_telemetry(),
819 |             users=self.collect_users_telemetry(),
820 |         )
821 |
822 |
```

device_flapping_sensor.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - Device
  Flapping Sensor
4 # =====
5 # Description:
6 #   Сенсор непрерывного мониторинга периферии, обнаружения дре
  безга и PnP-телеметрии.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.device_flapping_sensor impor
    t DeviceTransitionEvent
11 #
12 #       service = DeviceTransitionEvent()
13 #
14 # File: device_flapping_sensor.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-01 13:28:28
20 # =====
21
22 from __future__ import annotations
23 """Сенсор непрерывного мониторинга периферии, обнаружения дреб
  езга и PnP-телеметрии."""
24
25 import os
26 import threading
27 import time
28 from dataclasses import asdict, dataclass, field
29 from datetime import datetime, timezone
```

```

30 from typing import Any, Callable, Dict, List, Optional, Set
31 from logger import logger
32 from .win32_ffl.setupapi import PnPDeviceInfo, SetupAPI
33 from .sqlite import TelemetryStorage
34
35 @dataclass
36 class DeviceTransitionEvent:
37     """Модель события изменения состояния устройства в телемет
    | рии."""
38     timestamp: str
39     event_type: str
40     device_instance_id: str
41     friendly_name: str
42     device_class: str
43     category: str
44     has_problem: bool
45     problem_code: int
46     status_code: int
47     manufacturer: str
48     flapping_count_in_window: int = 0
49     uptime_seconds: float = 0.0
50
51     def to_dict(self) -> Dict[str, Any]:
52         """Преобразование события в словарь для сериализации."
    | ""
53         return asdict(self)
54
55 class DeviceFlappingSensor:
56     """Сенсор непрерывного мониторинга и отлова дребезга обору
    | дования."""
57     CATEGORY_MAPPING: Dict[str, str] = {'HIDClass': 'HID & Пер
    | иферия', 'Mouse': 'Мышь / Указатель', 'Keyboard': 'Клавиатура'
    | , 'DiskDrive': 'Дисконкей / Накопитель', 'Volume': 'Том накопи
    | теля', 'USB': 'USB Устройство / Контроллер', 'USBDevice': 'USB
    | Периферия', 'Monitor': 'HDMI / DP / Монитор', 'Display': 'Вид
    | еоадаптер / Дисплей', 'Printer': 'Принтер', 'PrintQueue': 'Оче
    | редь печати', 'Bluetooth': 'Bluetooth Адаптер / Устройство', '
    | Digitizer': 'Wacom / Графический планшет', 'Media': 'Аудио / В

```

```
идео / Микрофон', 'Camera': 'Веб-камера', 'Net': 'Сетевой адаптер']
```

58

```
59     def __init__(self, poll_interval_sec: float=1.0, flapping_
window_sec: float=60.0, flapping_threshold: int=3, storage: Op
tional[TelemetryStorage]=None, on_event_callback: Optional[Cal
lable[[DeviceTransitionEvent], None]]=None, json_logger: Any=None) -> None:
```

```
60         """Инициализирует сенсор телеметрии устройств.
```

61

```
62         Args:
```

```
63             poll_interval_sec: Интервал между опросами оборудо
вания в секундах.
```

```
64             flapping_window_sec: Временное окно для фиксации д
ребезга (сек).
```

```
65             flapping_threshold: Количество отключений за окно
для вызова тревоги FLAPPING_ALERT.
```

```
66             storage: Хранилище SQLite (по умолчанию использует
ся синглтон).
```

```
67             on_event_callback: Опциональный callback-обработчи
к для передачи событий в UI/RAG.
```

```
68             json_logger: Опциональный логгер JSONL для обратно
й совместимости и тестирования.
```

```
69         """
```

```
70         self.poll_interval = max(0.1, poll_interval_sec)
```

```
71         self.flapping_window = flapping_window_sec
```

```
72         self.flapping_threshold = flapping_threshold
```

```
73         self.on_event_callback = on_event_callback
```

```
74         self.json_logger = json_logger
```

```
75         self.storage = storage or TelemetryStorage.get_instanc
```

```
        e()
```

```
76         self._setupapi = SetupAPI() if os.name == 'nt' else No
ne
```

```
77         self._is_running = False
```

```
78         self._thread: Optional[threading.Thread] = None
```

```
79         self._lock = threading.Lock()
```

```
80         self._active_devices: Dict[str, PnPDeviceInfo] = {}
```

```
81         self._connected_since: Dict[str, float] = {}
```

```

82         self._disconnect_history: Dict[str, List[float]] = {}
83
84     def _resolve_category(self, device_class: str, friendly_name: str, instance_id: str) -> str:
85         """Определяет человекочитаемую категорию устройства.
86
87         Args:
88             device_class: PnP-класс из реестра SetupAPI.
89             friendly_name: Имя устройства.
90             instance_id: Идентификатор инстанса (VID/PID).
91
92         Returns:
93             str: Название категории.
94         """
95         combined = f'{device_class} {friendly_name} {instance_id}'.lower()
96         if 'wacom' in combined or 'digitizer' in combined or 'pen' in combined:
97             return 'Wacom / Графический планшет'
98         if 'hid' in combined and 'keyboard' in combined:
99             return 'Клавиатура'
100        if 'hid' in combined and 'mouse' in combined:
101            return 'Мышь / Указатель'
102        if 'bth' in combined or 'bluetooth' in combined:
103            return 'Bluetooth'
104        if 'monitor' in combined or 'display' in combined:
105            return 'HDMI / DP / Монитор'
106        if 'disk' in combined or 'usbstor' in combined:
107            return 'Дисконкей / Накопитель'
108        if 'print' in combined:
109            return 'Принтер'
110        return self.CATEGORY_MAPPING.get(device_class, device_class or 'Прочее оборудование')
111
112    def _get_current_devices_snapshot(self) -> Dict[str, PnPDeviceInfo]:
113        """Получает текущий снимок активных PnP-устройств.
114

```



```

115         Returns:
116             Dict[str, PnPDeviceInfo]: Словарь активных устройств по их Device Instance ID.
117             """
118         if self._setupapi is None:
119             return {}
120         try:
121             devices = self._setupapi.get_all_devices(only_present=True)
122             return {dev.device_instance_id: dev for dev in devices if dev.device_instance_id}
123         except Exception as e:
124             logger.error(f'Ошибка опроса SetupAPI в сенсоре устройств: {e}')
125             return {}
126
127     def _clean_flapping_history(self, now: float) -> None:
128         """Очищает устаревшие временные метки за пределами окна дребезга."""
129         cutoff = now - self.flapping_window
130         for dev_id in list(self._disconnect_history.keys()):
131             self._disconnect_history[dev_id] = [ts for ts in self._disconnect_history[dev_id] if ts >= cutoff]
132             if not self._disconnect_history[dev_id]:
133                 del self._disconnect_history[dev_id]
134
135     def _emit_event(self, event: DeviceTransitionEvent) -> None:
136         """Логирует событие телеметрии и оповещает внешние подписчики.
137
138         Args:
139             event: Экземпляр события изменения состояния устройства.
140             """
141         try:
142             self.storage.save_device_event(event.to_dict())
143         except Exception as ex:

```

```

144         logger.error(f'Ошибка сохранения device_event в БД
: {ex}')
145     if self.json_logger:
146         try:
147             if hasattr(self.json_logger, 'log_event'):
148                 self.json_logger.log_event(event.to_dict()
)
149             elif hasattr(self.json_logger, 'log'):
150                 self.json_logger.log(event.to_dict())
151         except Exception as ex:
152             logger.error(f'Ошибка сохранения device_event
в json_logger: {ex}')
153         log_msg = f"[{event.event_type}] [{event.category}] '{
event.friendly_name}' (ID: {event.device_instance_id}) | Ошибк
а: Code {event.problem_code} | Аптайм: {event.uptime_seconds:.
1f}s | Дребезгов: {event.flapping_count_in_window}"
154         if event.event_type in ('DISCONNECTED', 'FLAPPING_ALER
T') or event.has_problem:
155             logger.warning(log_msg)
156         else:
157             logger.info(log_msg)
158         if self.on_event_callback:
159             try:
160                 self.on_event_callback(event)
161             except Exception as ex:
162                 logger.error(f'Ошибка в on_event_callback: {ex
}')
163
164     def poll_once(self) -> List[DeviceTransitionEvent]:
165         """Выполняет один цикл опроса и детекции разницы состо
яний.
166
167         Returns:
168             List[DeviceTransitionEvent]: Список зафиксированны
х событий за текущий такт.
169         """
170         now = time.time()
171         iso_time = datetime.now(timezone.utc).isoformat()

```

```

172         events: List[DeviceTransitionEvent] = []
173         current_snapshot = self._get_current_devices_snapshot(
174     )
175         with self._lock:
176             self._clean_flapping_history(now)
177             if not self._active_devices and current_snapshot:
178                 self._active_devices = current_snapshot
179                 for dev_id in current_snapshot:
180                     self._connected_since[dev_id] = now
181                 logger.info(f'Сенсор инициализирован: обнаруже
но {len(current_snapshot)} активных устройств')
182                 return events
183                 current_ids = set(current_snapshot.keys())
184                 previous_ids = set(self._active_devices.keys())
185                 disconnected_ids = previous_ids - current_ids
186                 for dev_id in disconnected_ids:
187                     prev_dev = self._active_devices.pop(dev_id)
188                     connect_time = self._connected_since.pop(dev_i
d, now)
189                     uptime = max(0.0, now - connect_time)
190                     if dev_id not in self._disconnect_history:
191                         self._disconnect_history[dev_id] = []
192                         self._disconnect_history[dev_id].append(now)
193                         flapping_count = len(self._disconnect_history[
dev_id])
194                         category = self._resolve_category(prev_dev.dev
ice_class, prev_dev.friendly_name, dev_id)
195                         evt = DeviceTransitionEvent(timestamp=iso_time
, event_type='DISCONNECTED', device_instance_id=dev_id, friend
ly_name=prev_dev.friendly_name or prev_dev.device_class or dev
_id, device_class=prev_dev.device_class, category=category, ha
s_problem=prev_dev.has_problem, problem_code=prev_dev.problem_
code, status_code=prev_dev.status_code, manufacturer=prev_dev.
manufacturer, flapping_count_in_window=flapping_count, uptime_
seconds=uptime)
196                         events.append(evt)
197                         self._emit_event(evt)
198                         if flapping_count >= self.flapping_threshold:

```

```

198         flap_evt = DeviceTransitionEvent(timestamp
=iso_time, event_type='FLAPPING_ALERT', device_instance_id=dev
_id, friendly_name=prev_dev.friendly_name, device_class=prev_d
ev.device_class, category=category, has_problem=True, problem_
code=prev_dev.problem_code, status_code=prev_dev.status_code,
manufacturer=prev_dev.manufacturer, flapping_count_in_window=f
lapping_count, uptime_seconds=uptime)
199         events.append(flap_evt)
200         self._emit_event(flap_evt)
201     connected_ids = current_ids - previous_ids
202     for dev_id in connected_ids:
203         new_dev = current_snapshot[dev_id]
204         self._active_devices[dev_id] = new_dev
205         self._connected_since[dev_id] = now
206         flapping_count = len(self._disconnect_history.
get(dev_id, []))
207         category = self._resolve_category(new_dev.devi
ce_class, new_dev.friendly_name, dev_id)
208         evt = DeviceTransitionEvent(timestamp=iso_time
, event_type='CONNECTED', device_instance_id=dev_id, friendly_
name=new_dev.friendly_name or new_dev.device_class or dev_id,
device_class=new_dev.device_class, category=category, has_prob
lem=new_dev.has_problem, problem_code=new_dev.problem_code, st
atus_code=new_dev.status_code, manufacturer=new_dev.manufactur
er, flapping_count_in_window=flapping_count, uptime_seconds=0.
0)
209         events.append(evt)
210         self._emit_event(evt)
211     common_ids = current_ids & previous_ids
212     for dev_id in common_ids:
213         old_dev = self._active_devices[dev_id]
214         new_dev = current_snapshot[dev_id]
215         if old_dev.has_problem != new_dev.has_problem
or old_dev.problem_code != new_dev.problem_code:
216             self._active_devices[dev_id] = new_dev
217             connect_time = self._connected_since.get(d
ev_id, now)
218             category = self._resolve_category(new_dev.

```

```

device_class, new_dev.friendly_name, dev_id)
219         evt = DeviceTransitionEvent(timestamp=iso_
time, event_type='ERROR_STATE_CHANGED', device_instance_id=dev
_id, friendly_name=new_dev.friendly_name, device_class=new_dev
.device_class, category=category, has_problem=new_dev.has_prob
lem, problem_code=new_dev.problem_code, status_code=new_dev.st
atus_code, manufacturer=new_dev.manufacturer, uptime_seconds=m
ax(0.0, now - connect_time))
220         events.append(evt)
221         self._emit_event(evt)
222     return events
223
224     def _loop(self) -> None:
225         """Фоновый цикл непрерывного опроса."""
226         logger.info(f'Запущен фоновый поток сенсора устройств
(интервал: {self.poll_interval}с, окно дребезга: {self.flappin
g_window}с, порог: {self.flapping_threshold})')
227         while self._is_running:
228             try:
229                 self.poll_once()
230             except Exception as e:
231                 logger.error(f'Необработанная ошибка в цикле о
проса сенсора: {e}')
232                 time.sleep(self.poll_interval)
233
234     def start(self) -> None:
235         """Запускает непрерывный мониторинг в фоновом потоке."
"""
236         with self._lock:
237             if self._is_running:
238                 logger.warning('Сенсор устройств уже запущен')
239                 return
240             self._is_running = True
241             self._thread = threading.Thread(target=self._loop,
name='DeviceFlappingSensorThread', daemon=True)
242             self._thread.start()
243
244     def stop(self) -> None:

```

```
245 |         """Останавливает непрерывный мониторинг."""
246 |     with self._lock:
247 |         if not self._is_running:
248 |             return
249 |         self._is_running = False
250 |     if self._thread and self._thread.is_alive():
251 |         self._thread.join(timeout=3.0)
252 |         logger.info('Сенсор устройств успешно остановлен')
253 |
254 |     def get_status(self) -> Dict[str, Any]:
255 |         """Возвращает текущую статистику сенсора.
256 |
257 |         Returns:
258 |             Dict[str, Any]: Данные о количестве активных устройс
259 |             """
260 |         with self._lock:
261 |             return {'is_running': self._is_running, 'poll_inte
262 |             rval_sec': self.poll_interval, 'active_devices_count': len(sel
263 |             f._active_devices), 'flapping_devices_tracked': len(self._disc
264 |             onnect_history), 'devices_with_problems': sum((1 for d in self
265 |             ._active_devices.values() if d.has_problem))}
```

directory_watcher.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - Directory Watch Engine
4 # =====
5 # Description:
6 #   Движок фонового мониторинга файловой активности в контролируемых директориях
7 #   с привязкой к PID процессов (Win32 ReadDirectoryChangesW + Restart Manager / ETW).
8 #
9 # Usage Examples:
10 #   Python API:
11 #       from apps.windows.telemetry.directory_watcher import DirectoryWatchEngine
12 #
13 #       engine = DirectoryWatchEngine.get_instance()
14 #       engine.add_directory("C:\\Data\\Projects")
15 #       engine.start()
16 #
17 # File: directory_watcher.py
18 # Project: ai-breadboard
19 # Package: apps.windows.telemetry
20 # Author: hypo69
21 # Copyright: © 2026 hypo69
22 # Updated: 2026-10-08 11:57:00
23 # =====
24
25 from __future__ import annotations
26
27 """Движок мониторинга файловой активности в целевых директориях с привязкой к процессам по PID."""
28
```

```
29 import ctypes
30 from ctypes import wintypes
31 import json
32 import os
33 import platform
34 import threading
35 import time
36 from datetime import datetime, timezone
37 from pathlib import Path
38 from typing import Any, Dict, List, Optional, Set, Tuple
39
40 try:
41     from logger import logger
42 except ImportError:
43     import logging
44     logger = logging.getLogger(__name__)
45
46 from apps.windows.telemetry.sqlite import TelemetryStorage
47
48 # Win32 Константы для ReadDirectoryChangesW
49 FILE_SHARE_READ = 0x00000001
50 FILE_SHARE_WRITE = 0x00000002
51 FILE_SHARE_DELETE = 0x00000004
52 OPEN_EXISTING = 3
53 FILE_FLAG_BACKUP_SEMANTICS = 0x02000000
54 FILE_NOTIFY_CHANGE_FILE_NAME = 0x00000001
55 FILE_NOTIFY_CHANGE_DIR_NAME = 0x00000002
56 FILE_NOTIFY_CHANGE_ATTRIBUTES = 0x00000004
57 FILE_NOTIFY_CHANGE_SIZE = 0x00000008
58 FILE_NOTIFY_CHANGE_LAST_WRITE = 0x00000010
59
60 FILE_ACTION_ADDED = 1
61 FILE_ACTION_REMOVED = 2
62 FILE_ACTION_MODIFIED = 3
63 FILE_ACTION_RENAMED_OLD_NAME = 4
64 FILE_ACTION_RENAMED_NEW_NAME = 5
65
66 ACTION_MAP = {
```



```

67 |     FILE_ACTION_ADDED: 'CREATE',
68 |     FILE_ACTION_REMOVED: 'DELETE',
69 |     FILE_ACTION_MODIFIED: 'MODIFY',
70 |     FILE_ACTION_RENAMED_OLD_NAME: 'RENAME',
71 |     FILE_ACTION_RENAMED_NEW_NAME: 'RENAME',
72 | }
73 |
74 |
75 | class RM_UNIQUE_PROCESS(ctypes.Structure):
76 |     """Идентификатор процесса Restart Manager."""
77 |     _fields_ = [
78 |         ('dwProcessId', wintypes.DWORD),
79 |         ('ProcessStartTime', wintypes.FILETIME)
80 |     ]
81 |
82 |
83 | class RM_PROCESS_INFO(ctypes.Structure):
84 |     """Информация о процессе Restart Manager."""
85 |     _fields_ = [
86 |         ('Process', RM_UNIQUE_PROCESS),
87 |         ('strAppName', wintypes.WCHAR * 256),
88 |         ('strServiceShortName', wintypes.WCHAR * 64),
89 |         ('ApplicationType', wintypes.UINT),
90 |         ('AppStatus', wintypes.ULONG),
91 |         ('TSSessionId', wintypes.DWORD),
92 |         ('bRestartable', wintypes.BOOL)
93 |     ]
94 |
95 |
96 | def resolve_pid_for_file(file_path: str) -> Tuple[int, str]:
97 |     """Определяет PID и имя процесса, взаимодействующего с фай-
98 |     лом через Restart Manager Win32.
99 |
100 |     Args:
101 |         file_path: Абсолютный путь к файлу.
102 |
103 |     Returns:
104 |         Tuple[int, str]: Кортеж (PID, имя_процесса).

```

```

104     """
105     if platform.system() != 'Windows' or not file_path:
106         return (0, 'system')
107     try:
108         rstrtmgr = ctypes.windll.rstrtmgr
109         session_handle = wintypes.DWORD()
110         session_key = (wintypes.WCHAR * 33)()
111         if rstrtmgr.RmStartSession(ctypes.byref(session_handle
112 ), 0, session_key) != 0:
113             return (0, 'system')
114         try:
115             paths = (wintypes.LPCWSTR * 1)(file_path)
116             if rstrtmgr.RmRegisterResources(session_handle, 1,
117 paths, 0, None, 0, None) == 0:
118                 n_proc_needed = wintypes.UINT(0)
119                 n_proc = wintypes.UINT(5)
120                 reboot_reasons = wintypes.DWORD()
121                 proc_info = (RM_PROCESS_INFO * 5)()
122                 if rstrtmgr.RmGetList(
123                     session_handle,
124                     ctypes.byref(n_proc_needed),
125                     ctypes.byref(n_proc),
126                     proc_info,
127                     ctypes.byref(reboot_reasons)
128                 ) == 0 and n_proc.value > 0:
129                     p = proc_info[0]
130                     pid = int(p.Process.dwProcessId)
131                     name = str(p.strAppName).strip() or f'Proc
132 ess [{pid}]'
133                     return (pid, name)
134             finally:
135                 rstrtmgr.RmEndSession(session_handle)
136         except Exception as exc:
137             logger.debug(f'Ошибка определения процесса для файла {
138 file_path}: {exc}')
139             return (0, 'system')
140

```

```

138 class DirectoryWatchEngine:
139     """Движок мониторинга контролируемых папок с привязкой фай
        ловых событий к процессам по PID."""
140
141     _instance: Optional[DirectoryWatchEngine] = None
142     _lock = threading.RLock()
143
144     def __init__(self, config_path: Optional[Path] = None) ->
        None:
145         """Инициализирует наблюдатель за контролируемыми дирек
        ториями."""
146         self._config_path = config_path or (Path(__file__).res
        olve().parent / 'config.json')
147         self._tracked_directories: Set[str] = set()
148         self._watch_threads: Dict[str, threading.Thread] = {}
149         self._stop_events: Dict[str, threading.Event] = {}
150         self._running = False
151         self._load_configuration()
152
153     @classmethod
154     def get_instance(cls, config_path: Optional[Path] = None)
        -> DirectoryWatchEngine:
155         """Возвращает синглтон экземпляр DirectoryWatchEngine.
        """
156         with cls._lock:
157             if cls._instance is None:
158                 cls._instance = cls(config_path=config_path)
159             return cls._instance
160
161     def _load_configuration(self) -> None:
162         """Загружает список отслеживаемых папок из config.json
        ."""
163         if self._config_path.is_file():
164             try:
165                 with open(self._config_path, 'r', encoding='ut
                    f-8') as f:
166                     cfg = json.load(f)
167                     dirs = cfg.get('watch_directories', [])

```

```

168         for d in dirs:
169             if isinstance(d, str) and d.strip():
170                 self._tracked_directories.add(os.p
ath.normpath(d.strip()))
171         except Exception as exc:
172             logger.warning(f'[DirectoryWatchEngine] Ошибка
загрузки watch_directories: {exc}')
173
174     def _save_configuration(self) -> None:
175         """Сохраняет актуальный список отслеживаемых директори
й в config.json."""
176         if self._config_path.is_file():
177             try:
178                 with open(self._config_path, 'r', encoding='ut
f-8') as f:
179                     cfg = json.load(f)
180                     cfg['watch_directories'] = sorted(list(self._t
racked_directories))
181                 with open(self._config_path, 'w', encoding='ut
f-8') as f:
182                     json.dump(cfg, f, ensure_ascii=False, inde
nt=2)
183             except Exception as exc:
184                 logger.warning(f'[DirectoryWatchEngine] Ошибка
сохранения watch_directories: {exc}')
185
186     def get_tracked_directories(self) -> List[str]:
187         """Возвращает отсортированный список всех активных отс
леживаемых директорий."""
188         with self._lock:
189             return sorted(list(self._tracked_directories))
190
191     def add_directory(self, directory_path: str) -> bool:
192         """Добавляет директорию в список активного мониторинга
и запускает поток слежения.
193
194     Args:
195         directory_path: Путь к отслеживаемой папке.

```

```

196
197     Returns:
198         bool: True если директория успешно добавлена.
199     """
200     norm_path = os.path.normpath(directory_path.strip())
201     with self._lock:
202         if norm_path not in self._tracked_directories:
203             self._tracked_directories.add(norm_path)
204             self._save_configuration()
205             if self._running:
206                 self._start_watcher_thread(norm_path)
207             logger.info(f'[DirectoryWatchEngine] Добавлена
директория для мониторинга: {norm_path}')
208             return True
209         return False
210
211     def remove_directory(self, directory_path: str) -> bool:
212         """Удаляет директорию из списка отслеживания и останав
ливает поток.
213
214     Args:
215         directory_path: Путь к отслеживаемой папке.
216
217     Returns:
218         bool: True если директория удалена.
219     """
220     norm_path = os.path.normpath(directory_path.strip())
221     with self._lock:
222         if norm_path in self._tracked_directories:
223             self._tracked_directories.remove(norm_path)
224             self._save_configuration()
225             self._stop_watcher_thread(norm_path)
226             logger.info(f'[DirectoryWatchEngine] Удалена д
иректория из мониторинга: {norm_path}')
227             return True
228         return False
229
230     def start(self) -> None:

```

```

231 |         """Запускает мониторинг всех зарегистрированных директ
    | орий."""
232 |         with self._lock:
233 |             if self._running:
234 |                 return
235 |             self._running = True
236 |             for path in list(self._tracked_directories):
237 |                 self._start_watcher_thread(path)
238 |             logger.info(f'[DirectoryWatchEngine] Запущен монит
    | оринг {len(self._tracked_directories)} директорий')
239 |
240 |     def stop(self) -> None:
241 |         """Останавливает мониторинг всех директорий."""
242 |         with self._lock:
243 |             if not self._running:
244 |                 return
245 |             self._running = False
246 |             for path in list(self._watch_threads.keys()):
247 |                 self._stop_watcher_thread(path)
248 |             logger.info('[DirectoryWatchEngine] Мониторинг дир
    | екторий остановлен')
249 |
250 |     def _start_watcher_thread(self, path: str) -> None:
251 |         """Запускает поток Win32 ReadDirectoryChangesW для ука
    | занной директории."""
252 |         if path in self._watch_threads and self._watch_threads
    | [path].is_alive():
253 |             return
254 |         if not os.path.exists(path):
255 |             try:
256 |                 os.makedirs(path, exist_ok=True)
257 |             except Exception:
258 |                 pass
259 |
260 |         stop_evt = threading.Event()
261 |         self._stop_events[path] = stop_evt
262 |         t = threading.Thread(
263 |             target=self._watch_loop,

```

```

264         args=(path, stop_evt),
265         name=f'DirWatcher_{os.path.basename(path)}',
266         daemon=True
267     )
268     self._watch_threads[path] = t
269     t.start()
270
271     def _stop_watcher_thread(self, path: str) -> None:
272         """Останавливает фоновый поток слежения за директорией
273         ."""
274         stop_evt = self._stop_events.pop(path, None)
275         if stop_evt:
276             stop_evt.set()
277         t = self._watch_threads.pop(path, None)
278         if t and t.is_alive() and t != threading.current_thread():
279             t.join(timeout=1.0)
280
281     def _watch_loop(self, directory: str, stop_evt: threading.
282     Event) -> None:
283         """Основной рабочий цикл ReadDirectoryChangesW для ди-
284     ректории."""
285         if platform.system() != 'Windows':
286             return
287
288         kernel32 = ctypes.windll.kernel32
289         h_dir = kernel32.CreateFileW(
290             directory,
291             FILE_SHARE_READ | FILE_SHARE_WRITE | FILE_SHARE_DE
292             LETE,
293             FILE_SHARE_READ | FILE_SHARE_WRITE | FILE_SHARE_DE
294             LETE,
295             None,
296             OPEN_EXISTING,
297             FILE_FLAG_BACKUP_SEMANTICS,
298             None
299         )

```

```

296         if h_dir == -1 or h_dir == 0xFFFFFFFF:
297             logger.debug(f'[DirectoryWatchEngine] Не удалось о
    ткнуть дескриптор для: {directory}')
298             return
299
300         buffer = (ctypes.c_byte * 65536)()
301         bytes_returned = wintypes.DWORD()
302         flags = (
303             FILE_NOTIFY_CHANGE_FILE_NAME
304             | FILE_NOTIFY_CHANGE_DIR_NAME
305             | FILE_NOTIFY_CHANGE_ATTRIBUTES
306             | FILE_NOTIFY_CHANGE_SIZE
307             | FILE_NOTIFY_CHANGE_LAST_WRITE
308         )
309
310         try:
311             while not stop_evt.is_set() and self._running:
312                 success = kernel32.ReadDirectoryChangesW(
313                     h_dir,
314                     ctypes.byref(buffer),
315                     len(buffer),
316                     True,
317                     flags,
318                     ctypes.byref(bytes_returned),
319                     None,
320                     None
321                 )
322                 if not success or bytes_returned.value == 0:
323                     time.sleep(0.5)
324                     continue
325
326                 self._process_notifications(buffer, bytes_retu
    rned.value, directory)
327             except Exception as exc:
328                 logger.debug(f'[DirectoryWatchEngine] Исключение в
    цикле {directory}: {exc}')
329             finally:
330                 kernel32.CloseHandle(h_dir)

```



```

331
332     def _process_notifications(self, buffer: Any, buffer_len:
333 | int, target_directory: str) -> None:
334 |         """Парсит структуру FILE_NOTIFY_INFORMATION и записыва
335 | ет события в базу данных."""
336 |         offset = 0
337 |         events_to_save: List[Dict[str, Any]] = []
338 |         now_ts = datetime.now(timezone.utc).isoformat()
339 |
340 |         while offset < buffer_len:
341 |             next_entry_offset = int.from_bytes(bytes(buffer[of
342 | fset:offset + 4]), 'little')
343 |             action = int.from_bytes(bytes(buffer[offset + 4:of
344 | fset + 8]), 'little')
345 |             file_name_len = int.from_bytes(bytes(buffer[offset
346 | + 8:offset + 12]), 'little')
347 |             file_name_bytes = bytes(buffer[offset + 12:offset
348 | + 12 + file_name_len])
349 |             file_name = file_name_bytes.decode('utf-16le', err
350 | ors='ignore')
351 |
352 |             full_path = os.path.join(target_directory, file_na
353 | me)
354 |
355 |             action_type = ACTION_MAP.get(action, 'MODIFY')
356 |
357 |             # Определение PID и имени процесса
358 |             pid, proc_name = resolve_pid_for_file(full_path)
359 |
360 |             # Вычисление затронутого объема байт (если файл до
361 | ступен)
362 |
363 |             bytes_affected = 0
364 |             if action_type != 'DELETE' and os.path.isfile(full
365 | _path):
366 |                 try:
367 |                     bytes_affected = os.path.getsize(full_path
368 | )
369 |
370 |                 except Exception:
371 |                     pass

```

```
358 |
359 |         events_to_save.append({
360 |             'pid': pid,
361 |             'process_name': proc_name,
362 |             'action_type': action_type,
363 |             'target_directory': target_directory,
364 |             'file_path': full_path,
365 |             'bytes_affected': bytes_affected,
366 |             'timestamp': now_ts,
367 |             'created_at': time.time(),
368 |         })
369 |
370 |         if next_entry_offset == 0:
371 |             break
372 |         offset += next_entry_offset
373 |
374 |     if events_to_save:
375 |         try:
376 |             storage = TelemetryStorage.get_instance()
377 |             storage.insert_process_file_events(events_to_s
378 | ave)
379 |         except Exception as exc:
380 |             logger.debug(f'[DirectoryWatchEngine] Ошибка с
охранения файловых событий: {exc}')
```

event_catalog.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - Event C
  atalog
4 # =====
5 # Description:
6 #   Иерархический каталог провайдеров и каналов Windows Event
  Log,
7 #   классификация по 7 доменам телеметрии (Security, Process,
  Storage, Network,
8 #   Update, Hardware, Power) и Sysmon, движки построения графо
  в происхождения
9 #   (Process Provenance) и реконструкции жизненного цикла (Pow
  er Lifecycle).
10 #
11 # Usage Examples:
12 #   Python API:
13 #       from apps.windows.telemetry.event_catalog import Windows
  EventCatalogEngine
14 #
15 #       engine = WindowsEventCatalogEngine()
16 #       catalog = engine.get_full_catalog()
17 #       domains = engine.get_domains_summary()
18 #
19 # File: event_catalog.py
20 # Project: ai-breadboard
21 # Package: apps.windows.telemetry
22 # Author: hypo69
23 # Copyright: © 2026 hypo69
24 # Updated: 2026-10-08 02:35:00
25 # =====
26
27 from __future__ import annotations
```

```

28 """Иерархический каталог провайдеров и каналов Windows Event L
   |og по 7 доменам телеметрии."""
29
30 import os
31 import re
32 import subprocess
33 import time
34 from dataclasses import asdict, dataclass, field
35 from datetime import datetime, timedelta, timezone
36 from enum import Enum
37 from typing import Any, Dict, List, Optional, Set, Tuple
38
39 try:
40     from logger import logger
41 except ImportError:
42     import logging
43     logger = logging.getLogger(__name__)
44
45 from apps.windows.telemetry.win32_fffi.wevtapi import WevtAPI
46 from apps.windows.telemetry.sqlite import TelemetryStorage
47
48
49 class TelemetryDomain(str, Enum):
50     """7 доменов классификации Windows Event Logs."""
51     SECURITY_IDENTITY = 'security_identity'          # 1.  □  Secu
   |rity / Identity
52     PROCESS_SERVICES_TASKS = 'process_tasks'        # 2.  □  Proce
   |ss / Services / Tasks
53     STORAGE_FILESYSTEM = 'storage_io'               # 3.  □  Stor
   |age & Filesystem
54     NETWORK_COMMUNICATIONS = 'network_comms'        # 4.  □  Netw
   |ork & Sockets
55     WINDOWS_UPDATE = 'windows_update'               # 5.  □  Wind
   |ows Update
56     HARDWARE_PNP_DRIVERS = 'hardware_pnp'           # 6.  □  Hard
   |ware / Drivers / PnP
57     POWER_BOOT_SHUTDOWN = 'power_lifecycle'         # 7.  □  Powe
   |r / Boot / Shutdown

```

```

58     SYSMON_OBSERVABILITY = 'sysmon_deep'           # [] Sysmon
    Observability Layer
59
60
61 @dataclass
62 class EventProviderCatalogEntry:
63     """Запись каталога провайдеров и каналов Windows Event Log
    ."""
64     id: str
65     provider: str
66     channel: str
67     domain: TelemetryDomain
68     domain_title: str
69     key_events: Dict[int, str]
70     levels: List[str]
71     realtime_supported: bool
72     historical_supported: bool
73     privilege_required: str
74     volume_rating: str # Low, Medium, High, Extreme
75     useful_fields: List[str]
76     recommended_collector: str
77     storage_class: str
78     correlation_targets: List[str]
79     description: str = ''
80     is_present_on_host: bool = True
81     record_count: int = 0
82     size_mb: float = 0.0
83
84
85 # Полный реестр провайдеров по 7 доменам телеметрии
86 EVENT_PROVIDERS_REGISTRY: List[Dict[str, Any]] = [
87     # -----
    -----
88     # 1. [] Security / Identity
89     # -----
    -----
90     {
91         'id': 'sec_auditing',

```

```

92 |         'provider': 'Microsoft-Windows-Security-Auditing',
93 |         'channel': 'Security',
94 |         'domain': TelemetryDomain.SECURITY_IDENTITY,
95 |         'domain_title': '1. Security / Identity',
96 |         'key_events': {
97 |             4624: 'Успешный вход (Logon)',
98 |             4625: 'Неудачная попытка входа (Failed Logon)',
99 |             4634: 'Завершение сеанса пользователя (Logoff)',
100 |            4647: 'Пользователь инициировал выход',
101 |            4672: 'Назначение специальных привилегий (Admin/UA
      | C)',
102 |            4720: 'Создание пользователя',
103 |            4726: 'Удаление пользователя',
104 |            4732: 'Добавление пользователя в локальную группу'
      | ,
105 |            4738: 'Изменение атрибутов учетной записи',
106 |            4740: 'Блокировка учетной записи (Account Lockout)
      | ',
107 |            1102: 'Очистка журнала аудита безопасности',
108 |        },
109 |        'levels': ['Information', 'Audit Success', 'Audit Fail
      | ure'],
110 |        'realtime_supported': True,
111 |        'historical_supported': True,
112 |        'privilege_required': 'SeSecurityPrivilege (Admin)',
113 |        'volume_rating': 'High',
114 |        'useful_fields': ['SubjectUserName', 'TargetUserName',
      | 'LogonType', 'IpAddress', 'TargetSid'],
115 |        'recommended_collector': 'WevtAPI Incremental Bookmark
      | ',
116 |        'storage_class': 'security_events',
117 |        'correlation_targets': ['processes', 'user_sessions',
      | 'network_sockets'],
118 |        'description': 'Основной журнал аудита безопасности Wi
      | ndows: авторизация, аккаунты, группы и права.',
119 |    },
120 |    {
121 |        'id': 'sec_kerberos',

```

```
122 |         'provider': 'Microsoft-Windows-Kerberos-Key-Distributi
    | on-Center',
123 |         'channel': 'Security',
124 |         'domain': TelemetryDomain.SECURITY_IDENTITY,
125 |         'domain_title': '1. Security / Identity',
126 |         'key_events': {
127 |             4768: 'Выдача Kerberos TGT билета',
128 |             4769: 'Выдача Kerberos Service Ticket (TGS)',
129 |             4771: 'Сбой преаутентификации Kerberos (Неверный п
    | ароль)',
130 |             4776: 'NTLM валидация учетных данных',
131 |         },
132 |         'levels': ['Information', 'Audit Failure'],
133 |         'realtime_supported': True,
134 |         'historical_supported': True,
135 |         'privilege_required': 'Admin',
136 |         'volume_rating': 'Medium',
137 |         'useful_fields': ['TargetUserName', 'ServiceName', 'Ti
    | cketOptions', 'FailureCode', 'IpAddress'],
138 |         'recommended_collector': 'WevtAPI Incremental',
139 |         'storage_class': 'security_events',
140 |         'correlation_targets': ['domain_auth', 'failed_logons'
    | , 'lateral_movement'],
141 |         'description': 'Аудит протоколов Kerberos и NTLM аутен
    | тификации в корпоративной сети и локальном хосте.',
142 |     },
143 |     {
144 |         'id': 'sec_grouppolicy',
145 |         'provider': 'Microsoft-Windows-GroupPolicy',
146 |         'channel': 'Microsoft-Windows-GroupPolicy/Operational'
    | ,
147 |         'domain': TelemetryDomain.SECURITY_IDENTITY,
148 |         'domain_title': '1. Security / Identity',
149 |         'key_events': {
150 |             5312: 'Список примененных политик Group Policy Obj
    | ects (GPO)',
151 |             5314: 'Начало обработки групповой политики',
152 |             7004: 'Ошибка применения политики пользователя/ком
```

```

пьютера',
153     },
154     'levels': ['Information', 'Warning', 'Error'],
155     'realtime_supported': True,
156     'historical_supported': True,
157     'privilege_required': 'Admin',
158     'volume_rating': 'Low',
159     'useful_fields': ['GPOList', 'PrincipalSamName', 'ElapsedTimeInMilliseconds', 'ErrorCode'],
160     'recommended_collector': 'WevtAPI Query',
161     'storage_class': 'security_events',
162     'correlation_targets': ['policy_drift', 'system_baseline'],
163     'description': 'Телеметрия применения групповых политик Windows и изменений конфигурации безопасности.',
164 },
165
166 # -----
167 # 2. □ Process / Services / Tasks
168 # -----
169 {
170     'id': 'proc_creation_4688',
171     'provider': 'Microsoft-Windows-Security-Auditing',
172     'channel': 'Security',
173     'domain': TelemetryDomain.PROCESS_SERVICES_TASKS,
174     'domain_title': '2. Process / Services / Tasks',
175     'key_events': {
176         4688: 'Создание нового процесса (с CommandLine и Parent PID)',
177         4689: 'Завершение работы процесса (Process Exit)',
178     },
179     'levels': ['Information', 'Audit Success'],
180     'realtime_supported': True,
181     'historical_supported': True,
182     'privilege_required': 'Admin',
183     'volume_rating': 'Extreme',

```



```
184         'useful_fields': ['NewProcessName', 'CommandLine', 'ProcessId', 'CreatorProcessId', 'ParentProcessName', 'SubjectUserName'],
185         'recommended_collector': 'WevtAPI Incremental Tail',
186         'storage_class': 'security_events',
187         'correlation_targets': ['process_provenance_tree', 'cpu_spikes', 'file_access'],
188         'description': 'Глубокий аудит жизненного цикла процессов: полный путь, аргументы запуска, родительский PID.',
189     },
190     {
191         'id': 'proc_scm_services',
192         'provider': 'Service Control Manager',
193         'channel': 'System',
194         'domain': TelemetryDomain.PROCESS_SERVICES_TASKS,
195         'domain_title': '2. Process / Services / Tasks',
196         'key_events': {
197             7036: 'Изменение состояния службы (Запущена / Остановлена)',
198             7045: 'Установка новой системной службы (Service Install)',
199             7031: 'Аварийное завершение службы (Service Crash)',
200             7034: 'Неожиданная остановка службы',
201         },
202         'levels': ['Information', 'Warning', 'Error'],
203         'realtime_supported': True,
204         'historical_supported': True,
205         'privilege_required': 'User / Admin',
206         'volume_rating': 'Medium',
207         'useful_fields': ['ServiceName', 'ImagePath', 'ServiceType', 'StartType', 'AccountName'],
208         'recommended_collector': 'WevtAPI Incremental',
209         'storage_class': 'security_events',
210         'correlation_targets': ['process_provenance', 'service_matrix', 'boot_time'],
211         'description': 'Диспетчер служб Windows (SCM): запуск, остановка, установка новых сервисов и сбои.',
```

```

212     },
213     {
214         'id': 'proc_taskscheduler',
215         'provider': 'Microsoft-Windows-TaskScheduler',
216         'channel': 'Microsoft-Windows-TaskScheduler/Operationa
217     l',
218         'domain': TelemetryDomain.PROCESS_SERVICES_TASKS,
219         'domain_title': '2. Process / Services / Tasks',
220         'key_events': {
221             106: 'Регистрация новой задачи в планировщике',
222             107: 'Запуск задачи по расписанию или триггеру',
223             102: 'Успешное завершение задачи',
224             103: 'Сбой или ошибка выполнения задачи',
225             140: 'Изменение конфигурации задачи',
226             141: 'Удаление задачи из планировщика',
227         },
228         'levels': ['Information', 'Warning', 'Error'],
229         'realtime_supported': True,
230         'historical_supported': True,
231         'privilege_required': 'User / Admin',
232         'volume_rating': 'High',
233         'useful_fields': ['TaskName', 'ActionName', 'ResultCod
234     e', 'UserName', 'TaskInstanceId'],
235         'recommended_collector': 'WevtAPI Incremental',
236         'storage_class': 'security_events',
237         'correlation_targets': ['process_provenance', 'persist
238     ence_audit', 'scheduled_jobs'],
239         'description': 'Планировщик заданий: создание, триггер
240     ы, запуск задач и фоновая персистентность.',
241     },
242     {
243         'id': 'proc_wmi_activity',
244         'provider': 'Microsoft-Windows-WMI-Activity',
245         'channel': 'Microsoft-Windows-WMI-Activity/Operational
246     ',
247         'domain': TelemetryDomain.PROCESS_SERVICES_TASKS,
248         'domain_title': '2. Process / Services / Tasks',
249         'key_events': {

```

```
245 |             5857: 'Инициализация WMI провайдера',
246 |             5858: 'Ошибка выполнения WMI запроса (Query Failur
| e)',
247 |             5861: 'Регистрация постоянного WMI Event Consumer
| (Персистентность)',
248 |         },
249 |         'levels': ['Information', 'Warning', 'Error'],
250 |         'realtime_supported': True,
251 |         'historical_supported': True,
252 |         'privilege_required': 'Admin',
253 |         'volume_rating': 'Medium',
254 |         'useful_fields': ['Query', 'Namespace', 'ClientProcess
| Id', 'User', 'ResultCode'],
255 |         'recommended_collector': 'WevtAPI Query',
256 |         'storage_class': 'security_events',
257 |         'correlation_targets': ['process_provenance', 'wmi_per
| sistence'],
258 |         'description': 'Инфраструктура WMI: запросы процессов,
| сбои провайдеров и механизмы постоянной подписки.',
259 |     },
260 |     {
261 |         'id': 'proc_powershell',
262 |         'provider': 'Microsoft-Windows-PowerShell',
263 |         'channel': 'Microsoft-Windows-PowerShell/Operational',
264 |         'domain': TelemetryDomain.PROCESS_SERVICES_TASKS,
265 |         'domain_title': '2. Process / Services / Tasks',
266 |         'key_events': {
267 |             4103: 'Выполнение командного блока (Pipeline Execu
| tion)',
268 |             4104: 'Логирование содержимого скрипта (Script Blo
| ck Logging)',
269 |             40961: 'Запуск консоли PowerShell хоста',
270 |         },
271 |         'levels': ['Information', 'Warning'],
272 |         'realtime_supported': True,
273 |         'historical_supported': True,
274 |         'privilege_required': 'User / Admin',
275 |         'volume_rating': 'High',
```

```

276         'useful_fields': ['ScriptBlockText', 'Path', 'UserId',
    'ProcessId', 'HostName'],
277         'recommended_collector': 'WevtAPI Incremental',
278         'storage_class': 'security_events',
279         'correlation_targets': ['script_provenance', 'admin_audit'],
280         'description': 'Среда PowerShell: выполнение сценариев
    автоматизации, командлетов и расширенный ScriptBlock аудит.',
281     },
282
283     # -----
    -----
284     # 3. Storage & Filesystem
285     # -----
    -----
286     {
287         'id': 'stor_ntfs',
288         'provider': 'Microsoft-Windows-Ntfs',
289         'channel': 'Microsoft-Windows-Ntfs/Operational',
290         'domain': TelemetryDomain.STORAGE_FILESYSTEM,
291         'domain_title': '3. Storage & Filesystem',
292         'key_events': {
293             140: 'Сброс буфера файловой системы на диск',
294             141: 'Внимание: ошибка метаданных тома NTFS',
295             142: 'Активация подсистемы самовосстановления NTFS
    (Self-Healing)',
296         },
297         'levels': ['Information', 'Warning', 'Error'],
298         'realtime_supported': True,
299         'historical_supported': True,
300         'privilege_required': 'Admin',
301         'volume_rating': 'Medium',
302         'useful_fields': ['VolumeId', 'VolumeName', 'NtfsStatus', 'CorruptOffset'],
303         'recommended_collector': 'WevtAPI Incremental',
304         'storage_class': 'system_snapshots',
305         'correlation_targets': ['disk_io_spikes', 'file_corruption', 'smart_metrics'],

```

```
306         'description': 'Файловая система NTFS: целостность том  
ов, ошибки записи метаданных и самовосстановление.',  
307     },  
308     {  
309         'id': 'stor_disk_diag',  
310         'provider': 'Microsoft-Windows-DiskDiagnostic',  
311         'channel': 'Microsoft-Windows-DiskDiagnostic/Operation  
al',  
312         'domain': TelemetryDomain.STORAGE_FILESYSTEM,  
313         'domain_title': '3. Storage & Filesystem',  
314         'key_events': {  
315             1: 'Диагностика накопителя: предупреждение о скоро  
м выходе из строя (SMART Alert)',  
316             2: 'Результаты автоматической самодиагностики диск  
а',  
317         },  
318         'levels': ['Warning', 'Error', 'Critical'],  
319         'realtime_supported': True,  
320         'historical_supported': True,  
321         'privilege_required': 'Admin',  
322         'volume_rating': 'Low',  
323         'useful_fields': ['DeviceName', 'Model', 'SerialNumber  
, 'SmartAttribute', 'RawValue'],  
324         'recommended_collector': 'WevtAPI Query',  
325         'storage_class': 'system_snapshots',  
326         'correlation_targets': ['smart_sensors', 'hardware_hea  
lth'],  
327         'description': 'Диагностика накопителей: ранние аппара  
тные предупреждения S.M.A.R.T. о деградации диска.',  
328     },  
329     {  
330         'id': 'stor_spaces',  
331         'provider': 'Microsoft-Windows-StorageSpaces-Driver',  
332         'channel': 'Microsoft-Windows-StorageSpaces-Driver/Ope  
rational',  
333         'domain': TelemetryDomain.STORAGE_FILESYSTEM,  
334         'domain_title': '3. Storage & Filesystem',  
335         'key_events': {
```

```

336 |             100: 'Инициализация пула Storage Spaces',
337 |             200: 'Отказ физического диска в дисковом пространс
| тве',
338 |             201: 'Автоматическое восстановление зеркалирования
| (Repair/Resync)',
339 |         },
340 |         'levels': ['Information', 'Warning', 'Error', 'Critica
| l'],
341 |         'realtime_supported': True,
342 |         'historical_supported': True,
343 |         'privilege_required': 'Admin',
344 |         'volume_rating': 'Low',
345 |         'useful_fields': ['PoolId', 'SpaceId', 'PhysicalDiskId
| ', 'ErrorStatus'],
346 |         'recommended_collector': 'WevtAPI Query',
347 |         'storage_class': 'system_snapshots',
348 |         'correlation_targets': ['disk_arrays', 'raid_health'],
349 |         'description': 'Драйвер дисковых пространств Storage S
| paces: пулы дисков, сбои зеркал и ресинхронизация.',
350 |     },
351 |
352 |     # -----
| -----
353 |     # 4. □ Network & Sockets
354 |     # -----
| -----
355 |     {
356 |         'id': 'net_dns_client',
357 |         'provider': 'Microsoft-Windows-DNS-Client',
358 |         'channel': 'Microsoft-Windows-DNS-Client/Operational',
359 |         'domain': TelemetryDomain.NETWORK_COMMUNICATIONS,
360 |         'domain_title': '4. Network & Sockets',
361 |         'key_events': {
362 |             3008: 'Разрешение DNS запроса (Query Completed)',
363 |             3018: 'Таймаут обращения к DNS серверу',
364 |             3020: 'Сбой разрешения доменного имени (NXDOMAIN /
| Server Failure)',
365 |         },

```

```
366         'levels': ['Information', 'Warning', 'Error'],
367         'realtime_supported': True,
368         'historical_supported': True,
369         'privilege_required': 'User / Admin',
370         'volume_rating': 'High',
371         'useful_fields': ['QueryName', 'QueryType', 'QueryStat
us', 'ServerList'],
372         'recommended_collector': 'WevtAPI Incremental',
373         'storage_class': 'network_metrics',
374         'correlation_targets': ['process_sockets', 'latency_sp
ikes', 'web_activity'],
375         'description': 'Клиент DNS Windows: история запросов,
задержки DNS-серверов и сбои резолвинга.',
376     },
377     {
378         'id': 'net_firewall',
379         'provider': 'Microsoft-Windows-Windows Firewall With A
dvanced Security',
380         'channel': 'Microsoft-Windows-Windows Firewall With Ad
vanced Security/Firewall',
381         'domain': TelemetryDomain.NETWORK_COMMUNICATIONS,
382         'domain_title': '4. Network & Sockets',
383         'key_events': {
384             2004: 'Брандмауэр разрешил входящее/исходящее соедине
ние',
385             2005: 'Брандмауэр заблокировал пакет (Packet Dropp
ed)',
386             2006: 'Изменение правила брандмауэра',
387         },
388         'levels': ['Information', 'Warning'],
389         'realtime_supported': True,
390         'historical_supported': True,
391         'privilege_required': 'Admin',
392         'volume_rating': 'High',
393         'useful_fields': ['RuleId', 'ApplicationPath', 'LocalP
ort', 'RemotePort', 'Protocol', 'Direction'],
394         'recommended_collector': 'WevtAPI Incremental',
395         'storage_class': 'network_metrics',
```

```

396 |         'correlation_targets': ['process_provenance', 'blocked
    | _ports', 'security_events'],
397 |         'description': 'Сетевой экран Windows Defender Firewall:
    | блокировки пакетов, порты и изменение правил.',
398 |     },
399 |     {
400 |         'id': 'net_smb_client',
401 |         'provider': 'Microsoft-Windows-SMBClient',
402 |         'channel': 'Microsoft-Windows-SMBClient/Security',
403 |         'domain': TelemetryDomain.NETWORK_COMMUNICATIONS,
404 |         'domain_title': '4. Network & Sockets',
405 |         'key_events': {
406 |             31001: 'Подключение к сетевой папке SMB',
407 |             31017: 'Сбой авторизации на SMB сервере',
408 |             31020: 'Сброс сессии сетевого ресурса из-за сетево
    | го таймаута',
409 |         },
410 |         'levels': ['Information', 'Warning', 'Error'],
411 |         'realtime_supported': True,
412 |         'historical_supported': True,
413 |         'privilege_required': 'Admin',
414 |         'volume_rating': 'Low',
415 |         'useful_fields': ['ServerName', 'ShareName', 'UserName
    | ', 'Status'],
416 |         'recommended_collector': 'WevtAPI Query',
417 |         'storage_class': 'network_metrics',
418 |         'correlation_targets': ['network_shares', 'storage_io'
    | ],
419 |         'description': 'Сетевой клиент SMB: обращения к общим
    | сетевым папкам и сбои передачи файлов.',
420 |     },
421 |
422 |     # -----
    | -----
423 |     # 5. □ Windows Update
424 |     # -----
    | -----
425 |     {

```



```
426         'id': 'upd_client',
427         'provider': 'Microsoft-Windows-WindowsUpdateClient',
428         'channel': 'Microsoft-Windows-WindowsUpdateClient/Oper
ational',
429         'domain': TelemetryDomain.WINDOWS_UPDATE,
430         'domain_title': '5. Windows Update',
431         'key_events': {
432             19: 'Успешная установка обновления Windows Update
(KB)',
433             20: 'Ошибка при установке накопительного обновлени
я',
434             21: 'Установка пакета завершена, требуется перезап
уск ОС',
435             24: 'Начало скачивания компонентов обновления',
436             25: 'Начало установки пакета обновления',
437         },
438         'levels': ['Information', 'Warning', 'Error'],
439         'realtime_supported': True,
440         'historical_supported': True,
441         'privilege_required': 'User / Admin',
442         'volume_rating': 'Low',
443         'useful_fields': ['UpdateTitle', 'KBNumber', 'ErrorCod
e', 'SupportUrl', 'ClientApplicationID'],
444         'recommended_collector': 'WevtAPI Incremental',
445         'storage_class': 'system_snapshots',
446         'correlation_targets': ['reboots', 'servicing_integrit
y', 'disk_usage'],
447         'description': 'Клиент обновлений Windows Update: исто
рия загрузки, установки KB-пакетов и связанные перезагрузки.',
448     },
449     {
450         'id': 'upd_orchestrator',
451         'provider': 'Microsoft-Windows-UpdateOrchestrator',
452         'channel': 'Microsoft-Windows-UpdateOrchestrator/Opera
tional',
453         'domain': TelemetryDomain.WINDOWS_UPDATE,
454         'domain_title': '5. Windows Update',
455         'key_events': {
```

```

456 |         101: 'Update Orchestrator запустил проверку обновл
      | ений',
457 |         102: 'Обнаружены доступные обновления',
458 |         201: 'Запланирована принудительная перезагрузка си
      | стемы для завершения обновления',
459 |     },
460 |     'levels': ['Information', 'Warning'],
461 |     'realtime_supported': True,
462 |     'historical_supported': True,
463 |     'privilege_required': 'Admin',
464 |     'volume_rating': 'Low',
465 |     'useful_fields': ['ActionName', 'RebootDeadline', 'Sch
      | eduledTime'],
466 |     'recommended_collector': 'WevtAPI Query',
467 |     'storage_class': 'system_snapshots',
468 |     'correlation_targets': ['planned_reboots', 'update_cyc
      | le'],
469 |     'description': 'Диспетчер Update Orchestrator: планиро
      | вание обслуживания и дедлайны перезагрузок.',
470 | },
471 |
472 | # -----
      | -----
473 | # 6. □ Hardware / Drivers / Plug-and-Play
474 | # -----
      | -----
475 | {
476 |     'id': 'hw_kernel_pnp',
477 |     'provider': 'Microsoft-Windows-Kernel-PnP',
478 |     'channel': 'Microsoft-Windows-Kernel-PnP/Configuration
      | ',
479 |     'domain': TelemetryDomain.HARDWARE_PNP_DRIVERS,
480 |     'domain_title': '6. Hardware / Drivers / PnP',
481 |     'key_events': {
482 |         400: 'Устройство настроено (Device Configured)',
483 |         410: 'Устройство запущено (Device Started)',
484 |         411: 'Сбой запуска устройства (Device Start Failed
      | , Code 10 / 43)',

```

```
485 |         420: 'Удаление устройства (Device Removed / Unplug
    | ged)',
486 |         430: 'Требуется перезагрузка после установки драйв
    | ера',
487 |     },
488 |     'levels': ['Information', 'Warning', 'Error', 'Critica
    | l'],
489 |     'realtime_supported': True,
490 |     'historical_supported': True,
491 |     'privilege_required': 'Admin',
492 |     'volume_rating': 'Medium',
493 |     'useful_fields': ['DeviceInstanceId', 'DriverName', 'D
    | riverVersion', 'ProblemStatus', 'HardwareIds'],
494 |     'recommended_collector': 'WextAPI Incremental',
495 |     'storage_class': 'system_snapshots',
496 |     'correlation_targets': ['hardware_monitor', 'usb_perip
    | herals', 'pnp_errors'],
497 |     'description': 'Диспетчер оборудования Plug and Play:
    | подключение USB/PCIe устройств, загрузка драйверов и ошибки Co
    | de 10/43.',
498 | },
499 | {
500 |     'id': 'hw_device_setup',
501 |     'provider': 'Microsoft-Windows-DeviceSetupManager',
502 |     'channel': 'Microsoft-Windows-DeviceSetupManager/Admin
    | ',
503 |     'domain': TelemetryDomain.HARDWARE_PNP_DRIVERS,
504 |     'domain_title': '6. Hardware / Drivers / PnP',
505 |     'key_events': {
506 |         112: 'Успешная загрузка драйвера из Windows Update
    | Driver Store',
507 |         131: 'Ошибка сопоставления метаданных устройства',
508 |     },
509 |     'levels': ['Information', 'Warning', 'Error'],
510 |     'realtime_supported': True,
511 |     'historical_supported': True,
512 |     'privilege_required': 'Admin',
513 |     'volume_rating': 'Low',
```

```

514         'useful_fields': ['DeviceInstanceId', 'DriverPackage',
515         'Status'],
516         'recommended_collector': 'WevtAPI Query',
517         'storage_class': 'system_snapshots',
518         'correlation_targets': ['driver_store', 'peripherals']
519     },
520
521     # -----
522     # 7.  Power / Boot / Shutdown Lifecycle
523     # -----
524     {
525         'id': 'pwr_kernel_power',
526         'provider': 'Microsoft-Windows-Kernel-Power',
527         'channel': 'Microsoft-Windows-Kernel-Power/Operational
528     ',
529         'domain': TelemetryDomain.POWER_BOOT_SHUTDOWN,
530         'domain_title': '7. Power / Boot / Shutdown',
531         'key_events': {
532             41: 'КРИТИЧЕСКИЙ СБОЙ: Перезагрузка без предварите
533             льного чистого выключения (BSOD / Power Cut)',
534             42: 'Переход системы в спящий режим (Sleep / Moder
535             n Standby)',
536             107: 'Выход системы из спящего режима (Resume from
537             Sleep)',
538             109: 'Переход ядра в режим гибернации (Fast Startu
539             p / Hibernate)',
540             172: 'Изменение состояния подключения к электросет
541             и (AC / Battery)',
542         },
543         'levels': ['Information', 'Warning', 'Critical'],
544         'realtime_supported': True,
545         'historical_supported': True,
546         'privilege_required': 'User / Admin',

```

```
541         'volume_rating': 'Low',
542         'useful_fields': ['BugcheckCode', 'BugcheckParameter1'
543         , 'SleepTime', 'WakeTime', 'TargetState'],
544         'recommended_collector': 'WevtAPI Incremental',
545         'storage_class': 'system_snapshots',
546         'correlation_targets': ['reboot_analyzer', 'bsod_crash
547         es', 'uptime_timeline'],
548         'description': 'Управление питанием ядра: переходы в с
549         он/гибернацию, сбои питания (Event 41) и тайминги ACPI.',
550     },
551     {
552         'id': 'pwr_user32_shutdown',
553         'provider': 'User32',
554         'channel': 'System',
555         'domain': TelemetryDomain.POWER_BOOT_SHUTDOWN,
556         'domain_title': '7. Power / Boot / Shutdown',
557         'key_events': {
558             1074: 'Плановое выключение / перезагрузка иницииро
559             вана пользователем или процессом',
560         },
561         'levels': ['Information'],
562         'realtime_supported': True,
563         'historical_supported': True,
564         'privilege_required': 'User / Admin',
565         'volume_rating': 'Low',
566         'useful_fields': ['InitiatingProcess', 'CallingUser',
567         'ShutdownReason', 'ReasonCode'],
568         'recommended_collector': 'WevtAPI Incremental',
569         'storage_class': 'system_snapshots',
570         'correlation_targets': ['user_actions', 'reboot_proven
571         ance'],
572         'description': 'Инициация планового выключения/перезаг
573         рузки: кто, из какого процесса и по какой причине.',
574     },
575     {
576         'id': 'pwr_eventlog_lifecycle',
577         'provider': 'EventLog',
578         'channel': 'System',
```

```

572         'domain': TelemetryDomain.POWER_BOOT_SHUTDOWN,
573         'domain_title': '7. Power / Boot / Shutdown',
574         'key_events': {
575             6005: 'Служба журнала событий запущена (Система ус
пешно загрузилась)',
576             6006: 'Служба журнала событий остановлена (Штатное
выключение ОС)',
577             6008: 'Предыдущее выключение системы было неожидан
ным (Unexpected Shutdown)',
578         },
579         'levels': ['Information', 'Error'],
580         'realtime_supported': True,
581         'historical_supported': True,
582         'privilege_required': 'User / Admin',
583         'volume_rating': 'Low',
584         'useful_fields': ['TimeOfLastKnownGoodState', 'EventLo
gStatus'],
585         'recommended_collector': 'WevtAPI Incremental',
586         'storage_class': 'system_snapshots',
587         'correlation_targets': ['uptime_graph', 'crash_frequen
cy'],
588         'description': 'Метки времени загрузки и завершения ра
боты операционной системы для расчета надежности.',
589     },
590
591     # -----
592     # 8. Sysmon Observability Layer
593     # -----
594     {
595         'id': 'sysmon_deep_core',
596         'provider': 'Microsoft-Windows-Sysmon',
597         'channel': 'Microsoft-Windows-Sysmon/Operational',
598         'domain': TelemetryDomain.SYSMON_OBSERVABILITY,
599         'domain_title': 'Sysmon Observability Layer',
600         'key_events': {
601             1: 'Создание процесса с хэшами (SHA256, MD5, Paren

```

```

        tCommandLine)',
602         3: 'Сетевое соединение процесса (Source/Dest IP, P
        ort, Process)',
603         6: 'Загрузка драйвера ядра (Driver Loaded, Hashes)
        ',
604         7: 'Загрузка DLL модуля в процесс (Image Loaded)',
605         10: 'Доступ одного процесса к памяти другого (Proc
        ess Access / Injection)',
606         11: 'Создание файла на диске (File Created)',
607         12: 'Создание/удаление ключа реестра',
608         13: 'Изменение значения реестра (Registry Value Se
        t)',
609         22: 'DNS запрос процесса (DNS Query with Process I
        D)',
610     },
611     'levels': ['Information'],
612     'realtime_supported': True,
613     'historical_supported': True,
614     'privilege_required': 'Admin / System',
615     'volume_rating': 'Extreme',
616     'useful_fields': ['UtcTime', 'ProcessGuid', 'ProcessId
        ', 'Image', 'CommandLine', 'Hashes', 'ParentProcessId', 'Desti
        nationIp', 'DestinationPort'],
617     'recommended_collector': 'Sysmon Collector / WevtAPI',
618     'storage_class': 'security_events',
619     'correlation_targets': ['process_provenance_deep', 'ne
        twork_sockets', 'driver_hashes'],
620     'description': 'Microsoft System Monitor: непрерывная
        телеметрия процессов, сетевых сокетов, файловых операций и хэш
        ей драйверов.',
621 },
622 ]
623
624
625 class WindowsEventCatalogEngine:
626     """Движок иерархического каталога каналов Windows Event Lo
        g и корреляционных графов."""
627

```

```

628     def __init__(self) -> None:
629         """Инициализация движка каталога."""
630         self._wevtapi = WevtAPI()
631         self._storage = TelemetryStorage()
632         self._catalog_cache: Optional[List[EventProviderCatalogEntry]] = None
633         self._cache_timestamp: float = 0.0
634
635     def get_full_catalog(self, force_refresh: bool = False) ->
        List[EventProviderCatalogEntry]:
636         """Возвращает полный структурированный каталог провайдеров и каналов."""
637         now = time.time()
638         if not force_refresh and self._catalog_cache and (now
        - self._cache_timestamp < 30.0):
639             return self._catalog_cache
640
641         entries: List[EventProviderCatalogEntry] = []
642         for reg in EVENT_PROVIDERS_REGISTRY:
643             cname = reg['channel']
644             rec_cnt = self._wevtapi.get_channel_record_count(c
        name)
645
646             # Проверка доступности
647             is_present = True
648             size_mb = 0.0
649             try:
650                 # Оценка размера через wevtutil gli (быстро)
651                 info = self._wevtapi.check_channel_access(cnam
        e)
652                 is_present = info.get('accessible', True)
653             except Exception:
654                 is_present = False
655
656             entry = EventProviderCatalogEntry(
657                 id=reg['id'],
658                 provider=reg['provider'],
659                 channel=reg['channel'],

```



```

660         domain=reg['domain'],
661         domain_title=reg['domain_title'],
662         key_events=reg['key_events'],
663         levels=reg['levels'],
664         realtime_supported=reg['realtime_supported'],
665         historical_supported=reg['historical_supported
        ],
666         privilege_required=reg['privilege_required'],
667         volume_rating=reg['volume_rating'],
668         useful_fields=reg['useful_fields'],
669         recommended_collector=reg['recommended_collect
or'],
670         storage_class=reg['storage_class'],
671         correlation_targets=reg['correlation_targets']
        ,
672         description=reg.get('description', ''),
673         is_present_on_host=is_present,
674         record_count=rec_cnt,
675         size_mb=round(rec_cnt * 0.0006, 2), # примерн
ая оценка в МБ если нет прямого размера
676     )
677     entries.append(entry)
678
679     self._catalog_cache = entries
680     self._cache_timestamp = now
681     return entries
682
683     def get_domains_summary(self) -> Dict[str, Any]:
684         """Формирует сводную аналитику по 7 доменам телеметрии
        ."""
685         catalog = self.get_full_catalog()
686         domains_map: Dict[str, Dict[str, Any]] = {
687             TelemetryDomain.SECURITY_IDENTITY.value: {
688                 'id': TelemetryDomain.SECURITY_IDENTITY.value,
689                 'title': '1. □ Security / Identity',
690                 'description': 'Кто вошел, кто вышел, права до
ступа, UAC и аудит учетных записей.',
691                 'channels_count': 0,

```

```

692         'total_records': 0,
693         'key_events': [4624, 4625, 4672, 4720, 1102],
694         'status': 'Active',
695     },
696     TelemetryDomain.PROCESS_SERVICES_TASKS.value: {
697         'id': TelemetryDomain.PROCESS_SERVICES_TASKS.v
alue,
698         'title': '2. □ Process / Services / Tasks',
699         'description': 'Жизненный цикл процессов, аргу
менты CommandLine, службы SCM, задачи и скрипты.',
700         'channels_count': 0,
701         'total_records': 0,
702         'key_events': [4688, 4689, 7036, 7045, 106, 41
04],
703         'status': 'Active',
704     },
705     TelemetryDomain.STORAGE_FILESYSTEM.value: {
706         'id': TelemetryDomain.STORAGE_FILESYSTEM.value
,
707         'title': '3. □ Storage & Filesystem',
708         'description': 'Ошибки томов NTFS, S.M.A.R.T.
предупреждения и целостность накопителей.',
709         'channels_count': 0,
710         'total_records': 0,
711         'key_events': [140, 142, 1, 2, 200],
712         'status': 'Active',
713     },
714     TelemetryDomain.NETWORK_COMMUNICATIONS.value: {
715         'id': TelemetryDomain.NETWORK_COMMUNICATIONS.v
alue,
716         'title': '4. □ Network & Sockets',
717         'description': 'DNS-запросы, блокировки брандм
ауэра и соединения по протоколам SMB/TCP.',
718         'channels_count': 0,
719         'total_records': 0,
720         'key_events': [3008, 3018, 2004, 2005, 31001],
721         'status': 'Active',
722     },

```

```
723 |         TelemetryDomain.WINDOWS_UPDATE.value: {
724 |             'id': TelemetryDomain.WINDOWS_UPDATE.value,
725 |             'title': '5. □ Windows Update',
726 |             'description': 'Установка накопительных пакетов
| в KB, ошибки патчей и дедлайны обслуживания.',
727 |             'channels_count': 0,
728 |             'total_records': 0,
729 |             'key_events': [19, 20, 21, 25, 201],
730 |             'status': 'Active',
731 |         },
732 |         TelemetryDomain.HARDWARE_PNP_DRIVERS.value: {
733 |             'id': TelemetryDomain.HARDWARE_PNP_DRIVERS.val
| ue,
734 |             'title': '6. □ Hardware / Drivers / PnP',
735 |             'description': 'Подключение USB/PCIe оборудова
| ния, ошибки драйверов Code 10/43 и DriverStore.',
736 |             'channels_count': 0,
737 |             'total_records': 0,
738 |             'key_events': [400, 410, 411, 420, 112],
739 |             'status': 'Active',
740 |         },
741 |         TelemetryDomain.POWER_BOOT_SHUTDOWN.value: {
742 |             'id': TelemetryDomain.POWER_BOOT_SHUTDOWN.valu
| e,
743 |             'title': '7. □ Power / Boot / Shutdown',
744 |             'description': 'Цепочка жизненного цикла: Boot
| -> Login -> Sleep -> Wake -> Shutdown/Crash.',
745 |             'channels_count': 0,
746 |             'total_records': 0,
747 |             'key_events': [41, 42, 107, 1074, 6005, 6008],
748 |             'status': 'Active',
749 |         },
750 |         TelemetryDomain.SYSMON_OBSERVABILITY.value: {
751 |             'id': TelemetryDomain.SYSMON_OBSERVABILITY.val
| ue,
752 |             'title': '□ Sysmon Observability Layer',
753 |             'description': 'Глубокая сквозная телеметрия с
| оздания процессов с хэшами, сокетов и файлов.',
```

```

754         'channels_count': 0,
755         'total_records': 0,
756         'key_events': [1, 3, 6, 7, 10, 11, 22],
757         'status': 'Optional',
758     },
759 }
760
761     for item in catalog:
762         dom_key = item.domain.value if hasattr(item.domain
763         , 'value') else str(item.domain)
764         if dom_key in domains_map:
765             domains_map[dom_key]['channels_count'] += 1
766             domains_map[dom_key]['total_records'] += item.
767             record_count
768
769         sysmon_info = self.check_sysmon_status()
770         if not sysmon_info['installed']:
771             domains_map[TelemetryDomain.SYSMON_OBSERVABILITY.v
772             alue]['status'] = 'Не установлен'
773         else:
774             domains_map[TelemetryDomain.SYSMON_OBSERVABILITY.v
775             alue]['status'] = 'Активен'
776
777     return {
778         'domains': list(domains_map.values()),
779         'total_registered_providers': len(catalog),
780         'sysmon_installed': sysmon_info['installed'],
781         'generated_at': datetime.now(timezone.utc).isoform
782         at(),
783     }
784
785     def reconstruct_process_provenance(
786         self,
787         process_name: Optional[str] = None,
788         pid: Optional[int] = None,
789         hours: int = 24,
790     ) -> Dict[str, Any]:
791         """Строит дерево происхождения процессов (Process Prov

```

enance Graph).

```
787
788     Цепочка: User -> Parent Process -> Process -> Child Pr
processes -> Service / Task.
789     """
790     # Считываем события 4688 и 4689 из Security
791     events_4688 = self._storage.get_security_events(event_
id=4688, process_name=process_name, pid=pid, limit=100)
792
793     # Считываем события служб SCM (System)
794     scm_events = self._wevtapi.read_events(channel='System
', limit=100, event_id=7045, hours=hours)
795     # Считываем события TaskScheduler
796     task_events = self._wevtapi.read_events(channel='Micro
soft-Windows-TaskScheduler/Operational', limit=100, event_id=1
07, hours=hours)
797
798     nodes: List[Dict[str, Any]] = []
799     links: List[Dict[str, Any]] = []
800     seen_pids: Set[int] = set()
801
802     for ev in events_4688:
803         curr_pid = ev.get('process_id') or 0
804         parent_pid = ev.get('parent_process_id') or 0
805         pname = ev.get('process_name') or 'unknown.exe'
806         parent_name = ev.get('parent_process_name') or (f"
PID {parent_pid}" if parent_pid else 'explorer.exe')
807         user = ev.get('subject_user') or 'SYSTEM'
808         cmd = ev.get('command_line') or ''
809         ts = ev.get('timestamp') or ''
810
811         node_id = f"proc_{curr_pid}_{pname}"
812         parent_node_id = f"proc_{parent_pid}_{parent_name}"
813
814         if curr_pid not in seen_pids:
815             nodes.append({
816                 'id': node_id,
```

```

817         'type': 'process',
818         'name': pname.split('\\')[-1],
819         'full_path': pname,
820         'pid': curr_pid,
821         'user': user,
822         'command_line': cmd,
823         'timestamp': ts,
824         'role': 'Target Process',
825     })
826     seen_pids.add(curr_pid)
827
828     if parent_pid and parent_pid not in seen_pids:
829         nodes.append({
830             'id': parent_node_id,
831             'type': 'parent_process',
832             'name': parent_name.split('\\')[-1],
833             'full_path': parent_name,
834             'pid': parent_pid,
835             'user': user,
836             'command_line': '',
837             'timestamp': ts,
838             'role': 'Parent Creator',
839         })
840         seen_pids.add(parent_pid)
841
842     if parent_pid:
843         links.append({
844             'source': parent_node_id,
845             'target': node_id,
846             'relationship': 'spawns',
847             'label': f"Fork ({ts.split(' ')[-1] if ' '
in ts else ts})",
848         })
849
850     # Связываем со службами
851     for scm in scm_events[:10]:
852         srv_name = scm.get('message', '').split(' ')[0] or
'System Service'

```

```

853         srv_id = f"srv_{scm.get('event_id')}_{'srv_name'}"
854         nodes.append({
855             'id': srv_id,
856             'type': 'service',
857             'name': srv_name,
858             'timestamp': scm.get('timestamp', ''),
859             'role': 'Windows Service',
860         })
861
862     return {
863         'target_filter': {'process_name': process_name, 'p
864 id': pid},
865         'nodes': nodes,
866         'links': links,
867         'total_nodes': len(nodes),
868         'total_links': len(links),
869     }
870
871     def reconstruct_power_lifecycle(self, hours: int = 72) ->
872 List[Dict[str, Any]]:
873         """Реконструирует хронологию жизненного цикла ОС: Boot
874         -> Login -> Sleep -> Wake -> Shutdown/Crash."""
875         # Используем WevtAPI для чтения Kernel-Power, User32,
876         EventLog
877         pwr_events = self._wevtapi.read_events(
878             channel='System',
879             limit=200,
880             hours=hours,
881             event_id=[41, 1074, 6005, 6006, 6008, 12, 13, 42,
882 107],
883         )
884
885         timeline: List[Dict[str, Any]] = []
886         for ev in pwr_events:
887             eid = ev.get('event_id', 0)
888             msg = ev.get('message', '')
889             ts = ev.get('timestamp', '')
890             provider = ev.get('provider', '')

```

```
886 |
887 |     phase = 'WORK'
888 |     icon = '□ '
889 |     severity = 'info'
890 |
891 |     if eid in (6005, 12):
892 |         phase = 'BOOT'
893 |         icon = '□ '
894 |         msg = 'Запуск операционной системы и служб ядр
a'
895 |     elif eid == 42:
896 |         phase = 'SLEEP'
897 |         icon = '□ '
898 |         msg = 'Переход системы в спящий режим (Sleep)'
899 |     elif eid == 107:
900 |         phase = 'WAKE'
901 |         icon = '□ '
902 |         msg = 'Пробуждение системы из спящего режима'
903 |     elif eid == 1074:
904 |         phase = 'SHUTDOWN_PLANNED'
905 |         icon = '□ '
906 |         severity = 'warning'
907 |     elif eid in (6006, 13):
908 |         phase = 'SHUTDOWN_CLEAN'
909 |         icon = '□ '
910 |         msg = 'Штатное завершение работы Windows'
911 |     elif eid in (41, 6008):
912 |         phase = 'CRASH_UNEXPECTED'
913 |         icon = '□ '
914 |         severity = 'critical'
915 |         msg = 'Критический сбой питания / BSOD (Внезап
ная перезагрузка)'
916 |
917 |     timeline.append({
918 |         'timestamp': ts,
919 |         'phase': phase,
920 |         'icon': icon,
921 |         'event_id': eid,
```



```

922         'provider': provider,
923         'severity': severity,
924         'message': msg,
925         'raw_data': ev.get('raw_data', ''),
926     })
927
928     timeline.sort(key=lambda x: x['timestamp'], reverse=True)
929     return timeline
930
931     def reconstruct_hardware_pnp_chain(self, hours: int = 72)
932     -> List[Dict[str, Any]]:
933         """Реконструирует цепочку событий оборудования и драйв
934         еров Plug and Play."""
935         pnp_events = self._wevtapi.read_events(
936             channel='Microsoft-Windows-Kernel-PnP/Configuratio
937             n',
938             limit=100,
939             hours=hours,
940         )
941
942         chain: List[Dict[str, Any]] = []
943         for ev in pnp_events:
944             eid = ev.get('event_id', 0)
945             msg = ev.get('message', '')
946             ts = ev.get('timestamp', '')
947
948             status = 'Configured'
949             if eid == 411:
950                 status = 'Failed (Code 10/43)'
951             elif eid == 420:
952                 status = 'Unplugged / Removed'
953             elif eid == 410:
954                 status = 'Started'
955
956             chain.append({
957                 'timestamp': ts,
958                 'event_id': eid,

```

```

956         'status': status,
957         'message': msg,
958         'level': ev.get('level', 'Information'),
959     })
960
961     return chain
962
963     def check_sysmon_status(self) -> Dict[str, Any]:
964         """Проверяет статус службы и журнала Sysmon в системе.
965         """
966         rec_cnt = self._wevtapi.get_channel_record_count('Microsoft-Windows-Sysmon/Operational')
967         has_channel = rec_cnt > 0
968
969         # Проверка службы
970         service_running = False
971         try:
972             res = subprocess.run(['sc', 'query', 'Sysmon64'],
973                                 capture_output=True, text=True, timeout=3)
974             if 'RUNNING' in res.stdout:
975                 service_running = True
976             else:
977                 res2 = subprocess.run(['sc', 'query', 'Sysmon'],
978                                     capture_output=True, text=True, timeout=3)
979                 if 'RUNNING' in res2.stdout:
980                     service_running = True
981         except Exception:
982             service_running = False
983
984         return {
985             'installed': has_channel or service_running,
986             'service_running': service_running,
987             'channel_name': 'Microsoft-Windows-Sysmon/Operational',
988             'record_count': rec_cnt,
989             'capabilities': [
990                 'Process Creation with Hashes (SHA256, MD5, IM
991                 PHASH)',

```

```
988 |         'Network Connection Telemetry (Process -> IP:P
    | ort)',
989 |         'File Creation & Modification Tracking',
990 |         'Driver & Kernel Module Signature Verification
    | ',
991 |         'Process Memory Access / Injection Detection',
992 |         'DNS Queries per Process ID',
993 |     ] if (has_channel or service_running) else [],
994 | }
995 |
996 |
997 | __all__ = [
998 |     'TelemetryDomain',
999 |     'EventProviderCatalogEntry',
1000 |     'WindowsEventCatalogEngine',
1001 |     'EVENT_PROVIDERS_REGISTRY',
1002 | ]
1003 |
```

file_collector.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - File Collector
4 # =====
5 # Description:
6 #   Сбор событий файловой системы через DirectoryWatcher.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.file_collector import FileCollector
11 #
12 #       service = FileCollector()
13 #
14 # File: file_collector.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-03 23:06:40
20 # =====
21
22 from __future__ import annotations
23 """Сбор событий файловой системы через DirectoryWatcher."""
24
25 import threading
26 from datetime import datetime, timezone
27 from pathlib import Path
28 from typing import Any, Dict, List, Optional
29 try:
30     from logger import logger
31 except ImportError:
```

```

32     from logger import logger
33
34
35 def _get_directory_watcher_classes():
36     """Ленивый импорт DirectoryWatcher для избежания циклическ
их зависимостей."""
37     try:
38         from apps.windows.sdk.modules.sysadmin.src.directory_w
atcher import DirectoryWatcher, LiveFileEvent
39         return DirectoryWatcher, LiveFileEvent
40     except ImportError:
41         try:
42             from apps.windows.sysadmin.src.directory_watcher i
mport DirectoryWatcher, LiveFileEvent
43             return DirectoryWatcher, LiveFileEvent
44         except ImportError as e:
45             logger.warning(f'Не удалось импортировать Director
yWatcher: {e}')
46             return None, None
47
48
49 class FileCollector:
50     """Коллектор событий файловой системы."""
51
52     def __init__(self, watch_dirs: Optional[List[str]]=None, m
ax_history: int=200) -> None:
53         """Инициализирует коллектор файловых событий.
54
55         Args:
56             watch_dirs: Список директорий для мониторинга.
57             max_history: Размер истории событий.
58         """
59         self.watch_dirs = watch_dirs or [str(Path.home() / 'Do
cuments')]
60         self.max_history = max_history
61         self._watchers: Dict[str, Any] = {}
62         self._lock = threading.Lock()
63         self._init_watchers()

```

```

64 |
65 |     def _init_watchers(self) -> None:
66 |         """Инициализирует DirectoryWatcher для каждой директор
ии."""
67 |         DirectoryWatcher, _ = _get_directory_watcher_classes()
68 |         if DirectoryWatcher is None:
69 |             logger.warning('DirectoryWatcher недоступен')
70 |             return
71 |         for watch_dir in self.watch_dirs:
72 |             try:
73 |                 watcher = DirectoryWatcher(watch_dirs=[watch_d
ir], max_history=self.max_history)
74 |                 if watcher.start():
75 |                     self._watchers[watch_dir] = watcher
76 |                     logger.info(f'DirectoryWatcher запущен для
: {watch_dir}')
77 |             except Exception as e:
78 |                 logger.error(f'Ошибка инициализации DirectoryW
atcher для {watch_dir}: {e}')
79 |
80 |     def _convert_event(self, event: LiveFileEvent) -> Dict[str
, Any]:
81 |         """Преобразует LiveFileEvent в словарь для JSON.
82 |
83 |         Args:
84 |             event: Событие от DirectoryWatcher.
85 |
86 |         Returns:
87 |             Dict[str, Any]: Событие в формате словаря.
88 |         """
89 |         return {'timestamp': event.timestamp, 'action': event.
action, 'path': event.path, 'is_deletion': event.is_deletion,
'details': event.details}
90 |
91 |     def get_recent_events(self, limit: int=100) -> List[Dict[s
tr, Any]]:
92 |         """Получает недавние файловые события.
93 |

```

```

94 |         Args:
95 |             limit: Максимальное количество событий.
96 |
97 |         Returns:
98 |             List[Dict[str, Any]]: Список файловых событий.
99 |         """
100 |         with self._lock:
101 |             events: List[Dict[str, Any]] = []
102 |             for watch_dir, watcher in self._watchers.items():
103 |                 try:
104 |                     raw_events = watcher.get_recent_events(lim
it=limit)
105 |                     for event in raw_events:
106 |                         converted = self._convert_event(event)
107 |                         converted['watch_dir'] = watch_dir
108 |                         events.append(converted)
109 |                 except Exception as e:
110 |                     logger.error(f'Ошибка получения событий из
{watch_dir}: {e}')
111 |             events.sort(key=lambda x: x.get('timestamp', ''),
reverse=True)
112 |             return events[:limit]
113 |
114 |     def get_events_by_action(self, action: str, limit: int=50)
-> List[Dict[str, Any]]:
115 |         """Получает события определенного типа.
116 |
117 |         Args:
118 |             action: Тип действия (Created, Deleted, Modified).
119 |             limit: Максимальное количество событий.
120 |
121 |         Returns:
122 |             List[Dict[str, Any]]: Список событий.
123 |         """
124 |         all_events = self.get_recent_events(limit=limit * 2)
125 |         return [e for e in all_events if e.get('action') == ac
tion][:limit]
126 |

```

```

127 |     def get_events_by_type(self, event_type: str, limit: int=5
    | 0) -> List[Dict[str, Any]]:
128 |         """Получает события определенного типа (Created/Delete
    | d).
129 |
130 |         Args:
131 |             event_type: Тип события (Created или Deleted).
132 |             limit: Максимальное количество событий.
133 |
134 |         Returns:
135 |             List[Dict[str, Any]]: Список событий.
136 |         """
137 |         if event_type.lower() == 'created':
138 |             return self.get_events_by_action('Created', limit)
139 |         elif event_type.lower() == 'deleted':
140 |             return self.get_events_by_action('Deleted', limit)
141 |         return []
142 |
143 |     def stop(self) -> None:
144 |         """Останавливает все DirectoryWatcher."""
145 |         with self._lock:
146 |             for watcher in self._watchers.values():
147 |                 watcher.stop()
148 |             self._watchers.clear()
149 |             logger.info('FileCollector остановлен')
150 |
151 |     def restart(self) -> bool:
152 |         """Перезапускает DirectoryWatcher.
153 |
154 |         Returns:
155 |             bool: True если перезапуск успешен.
156 |         """
157 |         self.stop()
158 |         self._init_watchers()
159 |         return True
160 |
161 |     def add_watch_dir(self, watch_dir: str) -> bool:
162 |         """Добавляет директорию для мониторинга.

```



```

163
164     Args:
165         watch_dir: Путь к директории.
166
167     Returns:
168         bool: True если директория успешно добавлена.
169     """
170     DirectoryWatcher, _ = _get_directory_watcher_classes()
171     if DirectoryWatcher is None:
172         logger.warning('DirectoryWatcher недоступен')
173         return False
174     if watch_dir in self._watchers:
175         logger.warning(f'Директория уже отслеживается: {wa
176 tch_dir}')
177         return False
178     try:
179         watcher = DirectoryWatcher(watch_dirs=[watch_dir],
180 max_history=self.max_history)
181         if watcher.start():
182             self._watchers[watch_dir] = watcher
183             logger.info(f'Добавлена директория для мониторинга: {watch_dir}')
184             return True
185         except Exception as e:
186             logger.error(f'Ошибка добавления директории {watch
187 _dir}: {e}')
188             return False
189
190     def remove_watch_dir(self, watch_dir: str) -> bool:
191         """Удаляет директорию из мониторинга.
192
193     Args:
194         watch_dir: Путь к директории.
195
196     Returns:
197         bool: True если директория успешно удалена.
198     """
199     with self._lock:

```

```
197 |         if watch_dir in self._watchers:
198 |             watcher = self._watchers.pop(watch_dir)
199 |             watcher.stop()
200 |             logger.info(f'Удалена директория из мониторинг
a: {watch_dir}')
201 |             return True
202 |         else:
203 |             logger.warning(f'Директория не найдена в монит
оринге: {watch_dir}')
204 |             return False
```

grouped_telemetry.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - Grouped
  Telemetry
4 # =====
5 # Description:
6 #   Прямой экспорт сгруппированной телеметрии из аналитическог
  о ядра.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.grouped_telemetry import Gro
  upedTelemetryBuilder
11 #
12 # File: grouped_telemetry.py
13 # Project: ai-breadboard
14 # Package: apps.windows.telemetry
15 # Author: hypo69
16 # Copyright: © 2026 hypo69
17 # Updated: 2026-10-08 09:37:00
18 # =====
19
20 from __future__ import annotations
21 """Экспорт моделей и построителя сгруппированной телеметрии."""
22
23 from apps.windows.telemetry.analytics.grouped_telemetry import
  (
24     TelemetryGroupInfo,
25     GroupedTelemetryBuilder,
26     GroupDiagnoseRequest,
27     GroupDiagnosticResult,
28     SynthesisRequest,
```

```
29 |     SynthesisDiagnosticResult,  
30 | )  
31 |  
32 | __all__ = [  
33 |     "TelemetryGroupInfo",  
34 |     "GroupedTelemetryBuilder",  
35 |     "GroupDiagnoseRequest",  
36 |     "GroupDiagnosticResult",  
37 |     "SynthesisRequest",  
38 |     "SynthesisDiagnosticResult",  
39 | ]  
40 |
```

incident_detector.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - Incident Detector
4 # =====
5 # Description:
6 #   Модуль непрерывного анализа аномалий и автоматического выявления сбоев системы.
7 #
8 # Usage Examples:
9 #   Python API:
10 #   from apps.windows.telemetry.incident_detector import IncidentDetector
11 #
12 # File: incident_detector.py
13 # Project: ai-breadboard
14 # Package: apps.windows.telemetry
15 # Author: hypo69
16 # Copyright: © 2026 hypo69
17 # Updated: 2026-10-03 23:15:00
18 # =====
19
20 """Модуль `IncidentDetector`.
21
22 Содержит полную реализацию детектора аномалий, перенесённую из
23 `telemetry_research`.
24
25 """
26
27 from __future__ import annotations
28
29 import time
30 import uuid
31 from datetime import datetime, timezone
```

```

30 from typing import Any, Dict, List, Optional
31
32 # logger import handling
33 try:
34     from logger import logger
35 except ImportError:
36     from logger import logger
37
38 from apps.windows.telemetry.models import TelemetryIncident
39 from apps.windows.telemetry.ring_buffer import TelemetryRingBu
40 ffer
41
42 class IncidentDetector:
43     """Детектор аномалий в потоках метрик и событий с фиксац
44 и инцидентов."""
45
46     def __init__(
47         self,
48         cpu_threshold: float = 90.0,
49         disk_write_burst_mb: float = 100.0,
50         net_burst_mb: float = 50.0,
51         ring_buffer: Optional[TelemetryRingBuffer] = None,
52     ) -> None:
53         """Инициализирует детектор пороговыми значениями и ссы
54 лкой на кольцевой буфер.
55
56         Args:
57             cpu_threshold: Порог срабатывания по CPU % (по умо
58 лчанию 90.0).
59             disk_write_burst_mb: Порог дисковой записи в МБ/с
60 (по умолчанию 100.0).
61             net_burst_mb: Порог сетевого трафика в МБ/с (по ум
62 олчанию 50.0).
63             ring_buffer: Ссылка на экземпляр кольцевого буфера
64 памяти.
65         """
66         self.cpu_threshold = cpu_threshold

```

```

61         self.disk_write_burst_mb = disk_write_burst_mb
62         self.net_burst_mb = net_burst_mb
63         self.ring_buffer = ring_buffer or TelemetryRingBuffer(
64         )
65         self._last_incident_epoch: float = 0.0
66         self._cooldown_seconds: float = 60.0 # Не спамить оди
        наковыми инцидентами чаще раза в минуту
67
68     def check_anomaly(
69         self,
70         system_snapshot: Dict[str, Any],
71         processes: Optional[List[Dict[str, Any]]] = None,
72         recent_events: Optional[List[Dict[str, Any]]] = None,
73     ) -> Optional[TelemetryIncident]:
74         """Анализирует срез метрик и возвращает TelemetryIncident при обнаружении аномалии.
75
76         Args:
77             system_snapshot: Текущий снимок системы (SystemSnapshot dict).
78             processes: Список процессов текущего среза.
79             recent_events: Последние дискретные события (W64, ETW, Defender).
80
81         Returns:
82             Optional[TelemetryIncident]: Объект инцидента или None при штатной работе.
83
84         """
85         now = time.time()
86         cpu_pct = float(system_snapshot.get("cpu_total_percent") or system_snapshot.get("cpu_percent") or 0.0)
87         disk_w_mbs = float(system_snapshot.get("disk_write_bytes_sec") or 0.0) / (1024.0 * 1024.0)
88         net_tx_mbs = float(system_snapshot.get("network_sent_bytes_sec") or 0.0) / (1024.0 * 1024.0)
89         net_rx_mbs = float(system_snapshot.get("network_recv_bytes_sec") or 0.0) / (1024.0 * 1024.0)

```

```
89         trigger_type: Optional[str] = None
90         severity = "warning"
91         title = ""
92         description = ""
93         suspects: List[Dict[str, Any]] = []
94
95         # 1. Проверка CPU Spike
96         if cpu_pct >= self.cpu_threshold:
97             trigger_type = "cpu_spike"
98             severity = "critical" if cpu_pct >= 95.0 else "war
99 ning"
100             title = f"Критическая нагрузка на процессор: {cpu_
101 pct:.1f}%"
102             description = f"Общая загрузка CPU превысила порог
103 {self.cpu_threshold}%, достигнув {cpu_pct:.1f}%."
104
105         # 2. Проверка Disk Write Burst
106         elif disk_w_mbs >= self.disk_write_burst_mb:
107             trigger_type = "disk_burst"
108             severity = "warning"
109             title = f"Аномальная скорость записи на диск: {dis
110 k_w_mbs:.1f} МБ/с"
111             description = f"Скорость дисковой записи превысила
112 порог {self.disk_write_burst_mb} МБ/с."
113
114         # 3. Проверка Network Burst
115         elif (net_tx_mbs + net_rx_mbs) >= self.net_burst_mb:
116             trigger_type = "network_burst"
117             severity = "warning"
118             title = f"Всплеск сетевой активности: {(net_tx_mbs
119 + net_rx_mbs):.1f} МБ/с"
120             description = f"Суммарный сетевой трафик достиг {(
121 net_tx_mbs + net_rx_mbs):.1f} МБ/с (TX: {net_tx_mbs:.1f}, RX:
122 {net_rx_mbs:.1f})."
123
124         # 4. Проверка подозрительных процессов (AppData, Temp,
125 Public)
126         if processes:
```



```

118         for p in processes:
119             exe_path = str(p.get("path") or p.get("exe") or "").lower()
120             p_cpu = float(p.get("cpu_percent") or 0.0)
121             is_suspicious_location = any(
122                 loc in exe_path for loc in ("\\temp\\", "\\appdata\\local\\temp", "\\users\\public\\")
123             )
124             if is_suspicious_location and (p_cpu > 15.0 or trigger_type is not None):
125                 suspects.append({
126                     "pid": p.get("pid"),
127                     "name": p.get("name"),
128                     "path": exe_path,
129                     "cpu_percent": p_cpu,
130                     "memory_mb": p.get("memory_mb"),
131                     "reason": "Запуск из временного каталога с повышенной активностью",
132                 })
133                 if trigger_type is None and p_cpu > 30.0:
134                     trigger_type = "suspicious_process"
135                     severity = "critical"
136                     title = f"Подозрительный процесс из Temp: {p.get('name')} (PID {p.get('pid')})"
137                     description = f"Процесс {p.get('name')} выполняется из {exe_path} и потребляет {p_cpu:.1f}% CPU."
138
139                 elif p_cpu >= 50.0:
140                     suspects.append({
141                         "pid": p.get("pid"),
142                         "name": p.get("name"),
143                         "path": exe_path,
144                         "cpu_percent": p_cpu,
145                         "memory_mb": p.get("memory_mb"),
146                         "reason": f"Высокое индивидуальное потребление CPU: {p_cpu:.1f}%",
147                     })
148

```

```

149 |         # 5. Проверка событий безопасности (Defender / ETW)
150 |         matched_events: List[Dict[str, Any]] = []
151 |         if recent_events:
152 |             for ev in recent_events:
153 |                 ev_type = str(ev.get("event_type") or "").lower()
154 |                 ev_str = str(ev).lower()
155 |                 if "defender" in ev_type or "threat" in ev_str
or "malware" in ev_str:
156 |                     matched_events.append(ev)
157 |                     if trigger_type is None:
158 |                         trigger_type = "defender_alert"
159 |                         severity = "critical"
160 |                         title = "Срабатывание Windows Defender
/ Защиты от угроз"
161 |                         description = f"Зафиксировано событие
безопасности: {ev.get('name') or ev_type}"
162 |
163 |                     if not trigger_type:
164 |                         return None
165 |
166 |         # Проверка кулдауна, чтобы не генерировать дубликаты е
жесекундно
167 |         if (now - self._last_incident_epoch) < self._cooldown_
seconds:
168 |             return None
169 |
170 |         self._last_incident_epoch = now
171 |         inc_id = f"INC-{{datetime.now(timezone.utc).strftime('%
Y%m%d%H%M%S')}}-{{uuid.uuid4().hex[:4].upper()}}"
172 |
173 |         # Захватываем срез из кольцевого буфера (например, 5 м
инут до события)
174 |         raw_window = self.ring_buffer.get_window(center_timest
amp=now, before_seconds=300.0, after_seconds=0.0)
175 |
176 |         incident = TelemetryIncident(
177 |             incident_id=inc_id,

```

```

178         timestamp=datetime.now(timezone.utc).isoformat(),
179         trigger_type=trigger_type,
180         severity=severity,
181         title=title,
182         description=description,
183         trigger_metrics={
184             "cpu_percent": cpu_pct,
185             "disk_write_mbs": disk_w_mbs,
186             "network_tx_mbs": net_tx_mbs,
187             "network_rx_mbs": net_rx_mbs,
188         },
189         suspect_processes=suspects,
190         related_events=matched_events or (recent_events[:1
191 0] if recent_events else []),
191         metrics_summary={
192             "buffer_samples_captured": len(raw_window),
193             "peak_cpu": cpu_pct,
194             "peak_disk_write_mbs": disk_w_mbs,
195         },
196         raw_window=raw_window,
197     )
198
199     logger.warning(f"Зафиксирован инцидент телеметрии [{incident.incident_id}]: {incident.title}")
200     return incident
201
202     @staticmethod
203     def format_incident_markdown_for_llm(incident: TelemetryIncident) -> str:
204         """Форматирует инцидент в компактный структурированный
205         Markdown для AI / LLM анализа.
206
207         Args:
208             incident: Объект TelemetryIncident.
209
210         Returns:
211             str: Готовый markdown блок для контекста языковой
212             модели.

```

```

211         """
212         lines = [
213             f"### □ ДИАГНОСТИЧЕСКИЙ ИНЦИДЕНТ: [{incident.incident_id}]",
214             f"- **Тип триггера:** `{incident.trigger_type}` |",
215             f"- **Критичность:** `{incident.severity.upper()}`",
216             f"- **Время:** {incident.timestamp}",
217             f"- **Сводка:** {incident.title}",
218             f"- **Описание:** {incident.description}",
219             "",
220             f"#### □ Метрики в момент инцидента:",
221             f"- CPU: `{incident.trigger_metrics.get('cpu_percent', 0.0)}%`",
222             f"- Диск (Запись): `{incident.trigger_metrics.get('disk_write_mbs', 0.0)} МБ/с`",
223             f"- Сеть (TX / RX): `{incident.trigger_metrics.get('network_tx_mbs', 0.0)} МБ/с` / `{incident.trigger_metrics.get('network_rx_mbs', 0.0)} МБ/с`",
224             f"- Сохранено секундных замеров в буфере: `{incident.metrics_summary.get('buffer_samples_captured', 0)}%`",
225             "",
226         ]
227         if incident.suspect_processes:
228             lines.append("#### □ Подозрительные процессы:")
229             lines.append("| PID | Имя | Путь | CPU % | Причина")
230             lines.append("|---|---|---|---|---|")
231             for sp in incident.suspect_processes:
232                 lines.append(
233                     f"| {sp.get('pid', '-')} | `{sp.get('name', '-')}` | `{sp.get('path', '-')}` | {sp.get('cpu_percent', 0.0)}% | {sp.get('reason', '-')}"
234                 )
235             lines.append("")
236
237         if incident.related_events:
238             lines.append("#### □ Сопутствующие события систем")

```

```
    ы (Events):")
239         for ev in incident.related_events[:5]:
240             ev_name = ev.get("name") or ev.get("event_type")
                or "event"
241             ev_time = ev.get("timestamp", "")
242             lines.append(f"- `{ev_time}`: **{ev_name}** ({
ev.get('path') or ev.get('details') or ''})")
243             lines.append("")
244
245             lines.append("Инструкция: Оцените потенциальную угрозу
(вирус/майнер/зависание/утечка) и предложите конкретные меры
противодействия.")
246         return "\n".join(lines)
247
```

collector.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - Collector
4 # =====
5 # Description:
6 #   Движок сбора системных метрик, аппаратной структуры и показателей
7 #   производительности.
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.collector import SystemCollector
11 #
12 #       service = SystemCollector()
13 #
14 # File: collector.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-08 12:54:00
20 # =====
21
22 from __future__ import annotations
23 """Движок сбора системных метрик, аппаратной структуры и показателей
24   производительности."""
25
26 import asyncio
27 import ctypes
28 import getpass
29 import locale
30 import os
```

```
30 import platform
31 import socket
32 import time
33 from datetime import datetime, timezone
34 from typing import Any, Dict, List, Optional
35 try:
36     import psutil
37     PSUTIL_AVAILABLE = True
38 except ImportError:
39     psutil = None
40     PSUTIL_AVAILABLE = False
41 from logger import logger
42 from .models import (
43     AnomalyItem,
44     BatteryMetrics,
45     CloudStorageInfo,
46     CpuMetrics,
47     DiskIoMetrics,
48     DiskPartitionMetrics,
49     DriverInfo,
50     GpuMetrics,
51     HardwareArchiveEntry,
52     HardwareAuditReport,
53     HardwareChangeItem,
54     HardwareDeviceAudit,
55     HardwareNode,
56     HardwareSensor,
57     MemoryMetrics,
58     MonitorInfo,
59     NetworkInterfaceMetrics,
60     NetworkPortMetrics,
61     NpuMetrics,
62     OfficeSuiteInfo,
63     PhysicalDiskHealth,
64     ProcessLifecycleEvent,
65     ProcessMetrics,
66     ProcessNetworkActivity,
67     ProcessProvenanceInfo,
```

```

68     ProcessProvenanceReport,
69     RamStickInfo,
70     StartupArchiveEntry,
71     StartupChangeItem,
72     SystemCoreMetrics,
73     SystemHardwareQuick,
74     SystemHealthAlerts,
75     SystemSnapshot,
76     WindowsUpdateInfo,
77 )
78 from .sensors import get_hardware_sensors
79 from apps.windows.telemetry_research.hardware_auditor import HardwareAuditor
80 from apps.windows.telemetry_research.hardware_history_manager import HardwareHistoryManager
81 from apps.windows.telemetry_research.startup_history_manager import StartupHistoryManager
82 from .process_token_collector import ProcessTokenCollector
83 from .sqlite import TelemetryStorage
84
85 class SystemCollector:
86     """Telemetry collector for system load, hardware devices,
87     and processes."""
88
89     def __init__(
90         self,
91         auditor: Optional[HardwareAuditor] = None,
92         history_manager: Optional[HardwareHistoryManager] = None,
93         startup_history_manager: Optional[StartupHistoryManager] = None,
94         storage: Optional[TelemetryStorage] = None,
95     ) -> None:
96         """Initialize telemetry collector with timing, I/O baseline, hardware auditor, startup history, and storage."""
97         self._last_disk_io = psutil.disk_io_counters() if PSUTIL_AVAILABLE else None
98         self._last_net_io = psutil.net_io_counters(pernic=True)

```



```

    ) if PSUTIL_AVAILABLE else None
98         self._last_proc_net_io: Dict[int, Tuple[float, float,
float]] = {}
99         self._last_time = time.time()
100        self._cpu_model_cached: Optional[str] = None
101        self._identity_cached: Optional[Dict[str, Any]] = None
102        self._monitors_cached: Optional[List[MonitorInfo]] = N
one
103        self._updates_cached: Optional[WindowsUpdateInfo] = No
ne
104        self._office_cached: Optional[OfficeSuiteInfo] = None
105        self._ram_sticks_cached: Optional[List[RamStickInfo]]
= None
106        self._ram_sticks_cache_time: float = 0.0
107        self._physical_disks_cached: Optional[List[PhysicalDis
kHealth]] = None
108        self._physical_disks_cache_time: float = 0.0
109        self.storage = storage or TelemetryStorage.get_instanc
e()
110        self.auditor = auditor or HardwareAuditor(storage=self
.storage)
111        self.history_manager = history_manager or HardwareHist
oryManager()
112        self.startup_history_manager = startup_history_manager
or StartupHistoryManager()
113
114        def get_system_identity(self) -> Dict[str, Any]:
115            """Collect host identity, current user, system languag
e, and locale parameters.
116
117            Returns:
118                Dict[str, Any]: Detailed system identity and local
ization dictionary.
119            """
120            if self._identity_cached:
121                return self._identity_cached
122            hostname = socket.gethostname()
123            domain = os.environ.get('USERDOMAIN', '')

```

```

124 |         username_raw = os.environ.get('USERNAME') or getpass.g
| etuser()
125 |         full_username = f'{domain}\\{username_raw}' if domain
| and domain != hostname else username_raw
126 |         user_locale = 'ru-RU'
127 |         system_locale = 'ru-RU'
128 |         codepage = 'UTF-8'
129 |         input_languages: List[str] = []
130 |         if os.name == 'nt':
131 |             try:
132 |                 buf = ctypes.create_unicode_buffer(100)
133 |                 if ctypes.windll.kernel32.GetUserDefaultLocale
| Name(buf, 100):
134 |                     user_locale = buf.value or 'ru-RU'
135 |                 if ctypes.windll.kernel32.GetSystemDefaultLoca
| leName(buf, 100):
136 |                     system_locale = buf.value or 'ru-RU'
137 |                 acp = ctypes.windll.kernel32.GetACP()
138 |                 oemcp = ctypes.windll.kernel32.GetOEMCP()
139 |                 codepage = f'ACP: {acp} | OEM: {oemcp} (UTF-8)
| ,
140 |                 count = ctypes.windll.user32.GetKeyboardLayout
| List(0, None)
141 |                 if count > 0:
142 |                     hkls = (ctypes.c_void_p * count)()
143 |                     ctypes.windll.user32.GetKeyboardLayoutList
| (count, hkls)
144 |                     lang_map = {1049: 'Русский (RU)', 1033: 'E
| nglish (US)', 1037: 'օօօօօ (IL)', 1058: 'Українська (UA)', 103
| 1: 'Deutsch (DE)', 1036: 'Français (FR)'}
145 |                     for h in hkls:
146 |                         lang_id = (h.value if hasattr(h, 'valu
| e') and h.value else int(h)) & 65535
147 |                         lang_name = lang_map.get(lang_id, f'La
| yout 0x{lang_id:04X}')
148 |                         if lang_name not in input_languages:
149 |                             input_languages.append(lang_name)
150 |             except Exception as ex:

```

```

151 |         logger.debug(f'Failed to query native Windows
| locale: {ex}')
152 |     if not input_languages:
153 |         input_languages = ['Русский (RU)', 'English (US)']
154 |     try:
155 |         tz_offset = datetime.now().astimezone().strftime('%z')
156 |         tz_name = datetime.now().astimezone().tzname() or
| time.tzname[0]
157 |         timezone_str = f'{tz_name} (UTC{tz_offset[:3]}:{tz
| _offset[3:]})'
158 |     except Exception:
159 |         timezone_str = 'UTC+03:00'
160 |     def _format_lang_display(locale_tag: str) -> str:
161 |         if not locale_tag:
162 |             return ''
163 |         tag_clean = locale_tag.strip()
164 |         tag_lower = tag_clean.lower()
165 |         if 'ru' in tag_lower:
166 |             return f'Русский (Россия) [{tag_clean}]'
167 |         elif 'he' in tag_lower:
168 |             return f'עברית (ישראל) [{tag_clean}]'
169 |         elif 'en' in tag_lower:
170 |             return f'English (United States) [{tag_clean}]'
|
171 |         elif 'uk' in tag_lower:
172 |             return f'Українська (Україна) [{tag_clean}]'
173 |         elif 'de' in tag_lower:
174 |             return f'Deutsch (Deutschland) [{tag_clean}]'
175 |         elif 'fr' in tag_lower:
176 |             return f'Français (France) [{tag_clean}]'
177 |         elif 'es' in tag_lower:
178 |             return f'Español (España) [{tag_clean}]'
179 |         elif 'zh' in tag_lower:
180 |             return f'中文 (China) [{tag_clean}]'
181 |         elif 'ja' in tag_lower:
182 |             return f'日本語 (Japan) [{tag_clean}]'
183 |         return tag_clean

```

```

184
185     sys_lang_display = _format_lang_display(user_locale)
186     os_build = platform.version() or '10.0.26200'
187     os_install_date = ''
188     os_install_language = ''
189     if os.name == 'nt':
190         try:
191             import winreg
192             # Попытка получить оригинальный язык установки
из реестра (InstallLanguage)
193             for access in (winreg.KEY_READ | winreg.KEY_WO
W64_64KEY, winreg.KEY_READ):
194                 try:
195                     with winreg.OpenKey(winreg.HKEY_LOCAL_
MACHINE, r'SYSTEM\CurrentControlSet\Control\Nls\Language', 0,
access) as key:
196                         val, _ = winreg.QueryValueEx(key,
'InstallLanguage')
197                         if val:
198                             lcid = int(str(val), 16)
199                             buf_lcid = ctypes.create_unico
de_buffer(100)
200                             if ctypes.windll.kernel32.LCID
ToLocaleName(lcid, buf_lcid, 100, 0):
201                                 tag = buf_lcid.value
202                                 if tag:
203                                     os_install_language =
_format_lang_display(tag)
204                                     break
205                             except Exception:
206                                 pass
207             if not os_install_language:
208                 sys_ui_lang = ctypes.windll.kernel32.GetSy
stemDefaultUILanguage()
209                 buf_lcid = ctypes.create_unicode_buffer(10
0)
210                 if ctypes.windll.kernel32.LCIDToLocaleName
(sys_ui_lang, buf_lcid, 100, 0):

```

```

211 |             tag = buf_lcid.value
212 |             if tag:
213 |                 os_install_language = _format_lang
                _display(tag)
214 |             except Exception as ex:
215 |                 logger.debug(f'Failed to query Windows Install
                Language: {ex}')
216 |
217 |             try:
218 |                 import winreg
219 |                 for access in (winreg.KEY_READ | winreg.KEY_WO
                W64_64KEY, winreg.KEY_READ):
220 |                     try:
221 |                         with winreg.OpenKey(winreg.HKEY_LOCAL_
                MACHINE, r'SOFTWARE\Microsoft\Windows NT\CurrentVersion', 0, a
                ccess) as key:
222 |                             try:
223 |                                 val, _ = winreg.QueryValueEx(k
                ey, 'InstallDate')
224 |                                 if val and int(val) > 0:
225 |                                     os_install_date = datetime
                .fromtimestamp(int(val)).strftime('%d.%m.%Y %H:%M')
226 |                                     break
227 |                             except Exception:
228 |                                 pass
229 |                             try:
230 |                                 val, _ = winreg.QueryValueEx(k
                ey, 'InstallTime')
231 |                                 if val and int(val) > 0:
232 |                                     epoch_sec = (int(val) / 10
                000000.0) - 11644473600.0
233 |                                     if epoch_sec > 0:
234 |                                         os_install_date = date
                time.fromtimestamp(epoch_sec).strftime('%d.%m.%Y %H:%M')
235 |                                         break
236 |                             except Exception:
237 |                                 pass
238 |             except Exception:

```

```

239 |             continue
240 |         except Exception as ex:
241 |             logger.debug(f'Failed to query Windows Install
Date from registry: {ex}')
242 |             workgroup = 'WORKGROUP'
243 |             if os.name == 'nt':
244 |                 try:
245 |                     from ctypes import wintypes
246 |                     netapi32 = ctypes.windll.netapi32
247 |                     lp_buf = wintypes.LPWSTR()
248 |                     join_stat = wintypes.DWORD()
249 |                     if netapi32.NetGetJoinInformation(None, ctypes
.byref(lp_buf), ctypes.byref(join_stat)) == 0:
250 |                         if lp_buf.value:
251 |                             workgroup = lp_buf.value
252 |                             netapi32.NetApiBufferFree(lp_buf)
253 |                 except Exception as ex:
254 |                     logger.debug(f'Failed to query Windows Workgro
up: {ex}')
255 |                 self._identity_cached = {
256 |                     'hostname': hostname,
257 |                     'workgroup': workgroup,
258 |                     'username': full_username,
259 |                     'os_build': os_build,
260 |                     'os_install_date': os_install_date,
261 |                     'system_language': sys_lang_display,
262 |                     'os_install_language': os_install_language,
263 |                     'user_locale': user_locale,
264 |                     'system_locale': system_locale,
265 |                     'timezone': timezone_str,
266 |                     'codepage': codepage,
267 |                     'input_languages': input_languages
268 |                 }
269 |                 return self._identity_cached
270 |
271 |     async def _resolve_cpu_model(self) -> str:
272 |         """Resolve CPU brand/model name from registry, platfor
m or WMI.

```

```

273
274     Returns:
275         str: Resolved CPU model name.
276     """
277     if self._cpu_model_cached:
278         return self._cpu_model_cached
279     model = ''
280     if os.name == 'nt':
281         try:
282             import winreg
283             with winreg.OpenKey(winreg.HKEY_LOCAL_MACHINE,
284                                 r'HARDWARE\DESCRIPTION\System\CentralProcessor\0') as key:
285                 val, _ = winreg.QueryValueEx(key, 'Process
286                 orNameString')
287                 if val and val.strip():
288                     model = val.strip()
289             except Exception:
290                 pass
291         if not model:
292             model = platform.processor() or ''
293         if os.name == 'nt' and (not model or 'Intel64' in mode
294                                l or 'AMD64' in model):
295             try:
296                 from src.utils.com_worker import com_worker
297
298                 def _get_cpu():
299                     import pythoncom
300                     pythoncom.CoInitialize()
301                     import wmi
302                     w = wmi.WMI()
303                     cpus = w.Win32_Processor()
304                     return str(cpus[0].Name).strip() if cpus e
305             lse None
306             res = await com_worker.run(_get_cpu)
307             if res:
308                 model = res
309         except Exception:
310             pass

```

```

307         if not model:
308             model = f'{platform.machine()} {os.cpu_count() or
1}-Core Processor'
309             self._cpu_model_cached = model
310             return model
311
312     async def get_cpu_metrics(self) -> CpuMetrics:
313         """Collect real-time CPU utilization and architecture
314         details.
315
316         Returns:
317             CpuMetrics: Current CPU performance metrics.
318         """
319         logical_cores = os.cpu_count() or 1
320         physical_cores = max(1, logical_cores // 2) if logical
321         _cores > 1 else 1
322         if PSUTIL_AVAILABLE:
323             per_core = psutil.cpu_percent(interval=None, percp
324             u=True)
325             total = round(sum(per_core) / max(len(per_core), 1
326             ), 1) if per_core else 0.0
327             physical = psutil.cpu_count(logical=False) or phys
328             ical_cores
329             logical = psutil.cpu_count(logical=True) or logica
330             l_cores
331             freq_mhz = 0.0
332             try:
333                 freq = psutil.cpu_freq()
334                 if freq:
335                     freq_mhz = round(freq.current, 1)
336             except Exception:
337                 pass
338             return CpuMetrics(model=await self._resolve_cpu_mo
339             del(), architecture=platform.machine(), physical_cores=physica
340             l, logical_cores=logical, total_percent=total, per_core_percen
341             t=per_core, frequency_mhz=freq_mhz)
342         return CpuMetrics(model=await self._resolve_cpu_model(
343         ), architecture=platform.machine(), physical_cores=physical_co

```



```

    res, logical_cores=logical_cores, total_percent=10.0, per_core
_percent=[10.0] * logical_cores, frequency_mhz=2400.0)
334
335     def get_memory_metrics(self) -> MemoryMetrics:
336         """Collect RAM and swap memory consumption.
337
338         Returns:
339             MemoryMetrics: Current memory usage statistics.
340         """
341         if PSUTIL_AVAILABLE:
342             vm = psutil.virtual_memory()
343             sm = psutil.swap_memory()
344             return MemoryMetrics(total_gb=round(vm.total / 102
4 ** 3, 2), available_gb=round(vm.available / 1024 ** 3, 2), u
sed_gb=round((vm.total - vm.available) / 1024 ** 3, 2), percen
t=round(vm.percent, 1), swap_total_gb=round(sm.total / 1024 **
3, 2), swap_used_gb=round(sm.used / 1024 ** 3, 2), swap_perce
nt=round(sm.percent, 1))
345             total_ram_gb = 16.0
346             used_ram_gb = 8.0
347             if os.name == 'nt':
348                 try:
349
350                     class MEMORYSTATUSEX(ctypes.Structure):
351                         _fields_ = [('dwLength', ctypes.c_ulong),
('dwMemoryLoad', ctypes.c_ulong), ('ullTotalPhys', ctypes.c_ul
onglong), ('ullAvailPhys', ctypes.c_ulonglong), ('ullTotalPage
File', ctypes.c_ulonglong), ('ullAvailPageFile', ctypes.c_ulon
glong), ('ullTotalVirtual', ctypes.c_ulonglong), ('ullAvailVir
tual', ctypes.c_ulonglong), ('sullAvailExtendedVirtual', ctype
s.c_ulonglong)]
352                         stat = MEMORYSTATUSEX()
353                         stat.dwLength = ctypes.sizeof(MEMORYSTATUSEX)
354                         if ctypes.windll.kernel32.GlobalMemoryStatusEx
(ctypes.byref(stat)):
355                             total_ram_gb = round(stat.ullTotalPhys / 1
024 ** 3, 2)
356                             avail_ram_gb = round(stat.ullAvailPhys / 1

```

```

024 ** 3, 2)
357         used_ram_gb = round(total_ram_gb - avail_r
am_gb, 2)
358         percent = round(stat.dwMemoryLoad, 1)
359         return MemoryMetrics(total_gb=total_ram_gb
, available_gb=avail_ram_gb, used_gb=used_ram_gb, percent=perc
ent, swap_total_gb=round(stat.ullTotalPageFile / 1024 ** 3, 2)
, swap_used_gb=round((stat.ullTotalPageFile - stat.ullAvailPag
eFile) / 1024 ** 3, 2), swap_percent=round((1.0 - stat.ullAvai
lPageFile / max(stat.ullTotalPageFile, 1)) * 100, 1))
360     except Exception:
361         pass
362     return MemoryMetrics(total_gb=total_ram_gb, available_
gb=total_ram_gb - used_ram_gb, used_gb=used_ram_gb, percent=50
.0, swap_total_gb=16.0, swap_used_gb=2.0, swap_percent=12.5)
363
364     def get_gpu_metrics(self) -> List[GpuMetrics]:
365         """Collect GPU accelerator load, VRAM, and capabilitie
s.
366
367         Returns:
368             List[GpuMetrics]: Detected GPU devices with teleme
try.
369
370         """
371         gpus: List[GpuMetrics] = []
372         try:
373             from apps.windows.sdk.modules.hardware.gpu_prober
import GpuProber, determine_gpu_compute_backends
374             prober = GpuProber()
375             gpu_list = prober.probe_all()
376             for g in gpu_list:
377                 vendor = getattr(g, 'vendor', '') or ''
378                 name = getattr(g, 'name', '') or ''
379                 caps = determine_gpu_compute_backends(
380                     vendor,
381                     name,
382                     memory_total_mb=g.memory_total_mb,
dedicated_mb=g.dedicated_memory_mb or (g.m

```

```

memory_total_mb if getattr(g, 'gpu_type', '') == 'Discrete' else
None),
383         shared_mb=g.shared_memory_mb,
384     )
385     gpus.append(GpuMetrics(
386         name=g.name,
387         vendor=caps['vendor'],
388         gpu_type=caps['gpu_type'],
389         memory_total_gb=round((g.memory_total_mb or
r 0.0) / 1024.0, 2),
390         memory_used_gb=round((g.memory_used_mb or
0.0) / 1024.0, 2),
391         dedicated_memory_mb=caps['dedicated_memory
_mb'],
392         shared_memory_mb=caps['shared_memory_mb'],
393         memory_type=caps['memory_type'],
394         directx_version=caps['directx_version'],
395         load_percent=g.utilization_gpu_pct,
396         temperature_celsius=g.temperature_gpu_c,
397         has_cuda=caps['has_cuda'],
398         has_rocm=caps['has_rocm'],
399         has_oneapi=caps['has_oneapi'],
400         has_directml=caps['has_directml'],
401         has_vulkan=caps['has_vulkan'],
402         has_opengl=caps['has_opengl'],
403         compute_apis=caps['compute_apis'],
404         ai_backends=caps['ai_backends'],
405         engines=g.engines,
406     ))
407     except Exception as ex:
408         logger.debug(f'Ошибка сбора метрик GPU через GpuPr
ober: {ex}')
409     if not gpus:
410         gpus.append(GpuMetrics(
411             name='Integrated Display Controller',
412             vendor='Generic',
413             gpu_type='Integrated',
414             memory_total_gb=0.0,

```

```

415         dedicated_memory_mb=128.0,
416         shared_memory_mb=8192.0,
417         memory_type='Shared System Memory',
418         directx_version='DirectX 12 (FL 12_1)',
419         has_cuda=False,
420         has_rocm=False,
421         has_oneapi=False,
422         has_directml=True,
423         has_vulkan=True,
424         has_opengl=True,
425         compute_apis=['DirectML', 'Vulkan', 'OpenCL'],
426         ai_backends=['DirectML', 'Vulkan', 'CPU'],
427     ))
428     return gpus
429
430     def get_npu_metrics(self) -> List[NpuMetrics]:
431         """Collect NPU neural accelerator devices and AI capabilities.
432
433         Returns:
434             List[NpuMetrics]: Detected NPU devices.
435         """
436         npus: List[NpuMetrics] = []
437         try:
438             from apps.windows.sdk.modules.hardware.gpu_prober
439             import GpuProber
440             prober = GpuProber()
441             npu_list = prober.probe_npus()
442             for n in npu_list:
443                 npus.append(NpuMetrics(
444                     name=n.name,
445                     vendor=n.vendor,
446                     driver_version=n.driver_version,
447                     pnp_device_id=getattr(n, 'pnp_device_id',
448                     ''),
449                     status=n.status,
450                     tops=n.tops,
451                     has_directml=n.has_directml,

```

```

450         has_qnn=n.has_qnn,
451         has_openvino=n.has_openvino,
452         ai_backends=n.ai_backends,
453     ))
454 except Exception as ex:
455     logger.debug(f'Ошибка сбора метрик NPU: {ex}')
456 return npus
457
458 def get_disk_metrics(self) -> tuple[List[DiskPartitionMetrics], DiskIoMetrics]:
459     """Collect disk partitions capacity and read/write rates.
460
461     Returns:
462         tuple[List[DiskPartitionMetrics], DiskIoMetrics]:
463         Partitions and I/O rates.
464     """
465     partitions: List[DiskPartitionMetrics] = []
466     io_metrics = DiskIoMetrics()
467     if PSUTIL_AVAILABLE:
468         for part in psutil.disk_partitions(all=False):
469             try:
470                 usage = psutil.disk_usage(part.mountpoint)
471                 vol_name = ''
472                 is_virtual = False
473                 drive_type = 'Fixed'
474                 if os.name == 'nt':
475                     try:
476                         import ctypes
477                         vol_buf = ctypes.create_unicode_buffer(1024)
478                         fs_buf = ctypes.create_unicode_buffer(1024)
479                         mount_root = part.mountpoint if part.mountpoint.endswith('\\') else f'{part.mountpoint}\\'
480                         if ctypes.windll.kernel32.GetVolumeInformationW(mount_root, vol_buf, 1024, None, None, None, fs_buf, 1024):

```

```

480         vol_name = vol_buf.value
481         dos_buf = ctypes.create_unicode_bu
ffer(1024)
482         clean_dev = part.device.rstrip('\\
')
483         ctypes.windll.kernel32.QueryDosDev
iceW(clean_dev, dos_buf, 1024)
484         dos_dev = dos_buf.value
485         if 'google' in vol_name.lower() or
'@' in vol_name or 'onedrive' in vol_name.lower() or ('dropbo
x' in vol_name.lower()) or ('volume{' in dos_dev.lower() and '
harddiskvolume' not in dos_dev.lower()):
486             is_virtual = True
487             drive_type = 'Google Drive' if
'google' in vol_name.lower() or '@' in vol_name else 'Cloud /
Virtual'
488         except Exception:
489             pass
490         partitions.append(DiskPartitionMetrics(dev
ice=part.device, mountpoint=part.mountpoint, fstype=part.fstyp
e, total_gb=round(usage.total / 1024 ** 3, 2), used_gb=round(u
sage.used / 1024 ** 3, 2), free_gb=round(usage.free / 1024 **
3, 2), percent=round(usage.percent, 1), volume_name=vol_name o
r None, is_virtual=is_virtual, drive_type=drive_type))
491         except (PermissionError, OSError):
492             continue
493         now = time.time()
494         elapsed = max(now - self._last_time, 0.1)
495         current_io = psutil.disk_io_counters()
496         if current_io and self._last_disk_io:
497             io_metrics.read_bytes_per_sec = max(0.0, (curr
ent_io.read_bytes - self._last_disk_io.read_bytes) / elapsed)
498             io_metrics.write_bytes_per_sec = max(0.0, (cur
rent_io.write_bytes - self._last_disk_io.write_bytes) / elapse
d)
499             io_metrics.read_count_per_sec = max(0.0, (curr
ent_io.read_count - self._last_disk_io.read_count) / elapsed)
500             io_metrics.write_count_per_sec = max(0.0, (cur

```

```

    rent_io.write_count - self._last_disk_io.write_count) / elapse
d)
501         self._last_disk_io = current_io
502     else:
503         drive = 'C:\\' if os.name == 'nt' else '/'
504         try:
505             import shutil
506             total, used, free = shutil.disk_usage(drive)
507             partitions.append(DiskPartitionMetrics(device=
'C:' if os.name == 'nt' else '/', mountpoint=drive, fstype='NT
FS' if os.name == 'nt' else 'ext4', total_gb=round(total / 102
4 ** 3, 2), used_gb=round(used / 1024 ** 3, 2), free_gb=round(
free / 1024 ** 3, 2), percent=round(used / max(total, 1) * 100
, 1)))
508         except Exception:
509             pass
510     return (partitions, io_metrics)
511
512     def get_network_metrics(self) -> List[NetworkInterfaceMetr
ics]:
513         """Collect network interface statuses and throughput r
ates.
514
515         Returns:
516             List[NetworkInterfaceMetrics]: Network adapters me
etrics.
517         """
518         interfaces: List[NetworkInterfaceMetrics] = []
519         adapter_stats_map = {}
520         try:
521             from apps.windows.network.network_usage import Win
dowsNetworkUsageCollector
522             usage_collector = WindowsNetworkUsageCollector()
523             stats_list = usage_collector.get_adapter_statistic
s()
524             adapter_stats_map = {s.name.lower(): s for s in st
ats_list}
525         except Exception as ex:

```

```

526         logger.debug(f'Failed to fetch adapter stats in Sy
stemCollector: {ex}')
527
528         if PSUTIL_AVAILABLE:
529             addrs = psutil.net_if_addrs()
530             stats = psutil.net_if_stats()
531             current_net_io = psutil.net_io_counters(pernic=True
e)
532             now = time.time()
533             elapsed = max(now - self._last_time, 0.1)
534             for name, addr_list in addrs.items():
535                 stat = stats.get(name)
536                 is_up = stat.isup if stat else True
537                 speed = stat.speed if stat else 0
538                 ips = [a.address for a in addr_list if a.famil
y == socket.AF_INET]
539                 sent_rate = 0.0
540                 recv_rate = 0.0
541                 if current_net_io and name in current_net_io a
nd self._last_net_io and (name in self._last_net_io):
542                     cur = current_net_io[name]
543                     prev = self._last_net_io[name]
544                     sent_rate = max(0.0, (cur.bytes_sent - pre
v.bytes_sent) / elapsed)
545                     recv_rate = max(0.0, (cur.bytes_recv - pre
v.bytes_recv) / elapsed)
546                     st_info = adapter_stats_map.get(name.lower())
547                     interfaces.append(NetworkInterfaceMetrics(name
=name, is_up=is_up, speed_mbps=speed, bytes_sent_per_sec=sent_
rate, bytes_recv_per_sec=recv_rate, ip_addresses=ips, statisti
cs=st_info))
548                 self._last_net_io = current_net_io
549             else:
550                 try:
551                     host_ip = socket.gethostbyname(socket.gethostn
ame())
552                 except Exception:
553                     host_ip = '127.0.0.1'

```



```

554         interfaces.append(NetworkInterfaceMetrics(name='Primary Network Adapter', is_up=True, speed_mbps=1000, bytes_sent_per_sec=0.0, bytes_recv_per_sec=0.0, ip_addresses=[host_ip]))
555     return interfaces
556
557     def get_network_usage_report(self, period_minutes: int = 1440):
558         """Получить глубокий отчет сетевого потребления за период для AI диагностики.
559
560         Args:
561             period_minutes: Интервал отчета в минутах.
562
563         Returns:
564             NetworkUsagePeriodReport: Полная сводка сетевой статистики за период.
565         """
566         from apps.windows.network.network_usage import WindowsNetworkUsageCollector
567         collector = WindowsNetworkUsageCollector()
568         return collector.get_traffic_period_summary(period_minutes=period_minutes)
569
570     def get_disk_usage_report(self, period_minutes: int = 1440, force_refresh_smart: bool = False):
571         """Получить глубокий отчет использования дискового I/O и SMART за период для AI диагностики.
572
573         Args:
574             period_minutes: Интервал отчета в минутах (по умолчанию 1440 = 24 часа).
575             force_refresh_smart: Принудительно опросить физические диски.
576
577         Returns:
578             DiskUsagePeriodReport: Полная сводка дискового I/O и SMART-показателей за период.

```

```

579         """
580         from apps.windows.telemetry.storage_usage import WindowsStorageUsageCollector
581         collector = WindowsStorageUsageCollector()
582         return collector.get_disk_usage_period_report(period_minutes=period_minutes, force_refresh_smart=force_refresh_smart)
583
584
585
586     @staticmethod
587     def _classify_launch_reason(
588         parent_name: Optional[str],
589         process_name: Optional[str] = None,
590         session_id: Optional[int] = None,
591         cmdline: Optional[str] = None,
592     ) -> str:
593         """Классификация причины/контекста запуска процесса по его родителю и окружению.
594
595         Args:
596             parent_name: Имя родительского процесса.
597             process_name: Имя текущего процесса.
598             session_id: Номер сессии Windows (0 = системные службы, 1+ = интерактивная сессия).
599             cmdline: Командная строка процесса (опционально).
600
601         Returns:
602             str: Категория запуска ('service', 'scheduled_task', 'shell_interactive', 'terminal_cli', 'ide_developer', 'system_boot', 'spawned_by_parent', 'unknown').
603         """
604         p_name = (parent_name or '').lower()
605         if p_name in ('services.exe',) or (p_name in ('svchost.exe',) and session_id == 0):
606             return 'service'
607         if p_name in ('taskeng.exe', 'taskhostw.exe', 'taskhost.exe') or (cmdline and '-task' in cmdline.lower()):

```

```

608         return 'scheduled_task'
609     if p_name in ('explorer.exe',):
610         return 'shell_interactive'
611     if p_name in ('cmd.exe', 'powershell.exe', 'pwsh.exe',
        'wt.exe', 'windowsterminal.exe', 'bash.exe', 'zsh.exe', 'conh
        ost.exe'):
612         return 'terminal_cli'
613     if p_name in ('code.exe', 'devenv.exe', 'pycharm64.exe
        ', 'idea64.exe', 'rider64.exe', 'cursor.exe', 'windsurf.exe'):
614         return 'ide_developer'
615     if p_name in ('winlogon.exe', 'userinit.exe', 'smss.ex
        e', 'csrss.exe', 'wininit.exe', 'lsass.exe'):
616         return 'system_boot'
617     if p_name:
618         clean_parent = p_name.replace('.exe', '')
619         return f'spawned_by_{clean_parent}'
620     return 'unknown'
621
622     def get_top_processes(self, limit: int = 25, sort_by: str
        = 'cpu') -> List[ProcessMetrics]:
623         """Retrieve active processes sorted by resource consum
        ption, with token and provenance info.
624
625         Args:
626             limit: Maximum processes to return (0 for all).
627             sort_by: Attribute to sort by ('cpu', 'memory'/'ra
        m', 'handles'/'descriptors').
628
629         Returns:
630             List[ProcessMetrics]: Ranked process metrics, incl
        uding provenance, integrity_level and elevation.
631         """
632         # Collect token information for all processes
633         token_collector = ProcessTokenCollector()
634         token_info_map = {info.pid: info for info in token_col
        lector.collect()}
635
636         # Классификация оконных процессов
637         from apps.windows.sdk.modules.process_manager.core.cla

```

```

        classifier import (
637 |             ProcessClassifier,
638 |             WINDOWS_SYSTEM_PROCESS_NAMES,
639 |             KNOWN_APP_FRIENDLY_NAMES,
640 |         )
641 |         classifier = ProcessClassifier()
642 |         windowed_pids, titles_map = classifier.get_windowed_pids_and_titles()
643 |
644 |         procs: List[ProcessMetrics] = []
645 |         if PSUTIL_AVAILABLE:
646 |             handle_attr = 'num_handles' if os.name == 'nt' else 'num_fds'
647 |             proc_dict: Dict[int, Dict[str, Any]] = {}
648 |             for p in psutil.process_iter(attrs=['pid', 'ppid', 'name', 'status', 'cpu_percent', 'memory_info', 'memory_percent', 'num_threads', 'username', 'create_time', 'exe', 'cmdline', handle_attr]):
649 |                 try:
650 |                     proc_dict[p.info['pid']] = p.info
651 |                 except (psutil.NoSuchProcess, psutil.AccessDenied):
652 |                     continue
653 |
654 |             name_to_windowed = {
655 |                 (info.get('name') or '').lower(): True
656 |                 for p_id, info in proc_dict.items()
657 |                 if p_id in windowed_pids
658 |             }
659 |
660 |             for pid, info in proc_dict.items():
661 |                 try:
662 |                     p_name = info.get('name') or 'unknown'
663 |                     low_name = p_name.lower()
664 |                     exe_low = (info.get('exe') or '').lower()
665 |
666 |                     mem_info = info.get('memory_info')
667 |                     rss_mb = round(mem_info.rss / (1024 * 1024

```

```

    ), 1) if mem_info else 0.0
668         cpu_p = round(info.get('cpu_percent') or 0
    .0, 1)
669         num_h = info.get(handle_attr) or 0
670         token = token_info_map.get(pid)
671         integrity = token.integrity_level if token
    else None
672         elevation = token.elevation if token else
    None
673         sid = token.sid if token else None
674         session_id = token.session_id if token els
    e None
675
676         ppid = info.get('ppid')
677         parent_info = proc_dict.get(ppid) if ppid
    else None
678         parent_name = parent_info.get('name') if p
    arent_info else None
679
680         # Построение цепочки предков
681         ancestors = [p_name or str(pid)]
682         curr_pid = ppid
683         visited = {pid}
684         while curr_pid and curr_pid in proc_dict a
    nd curr_pid not in visited:
685             visited.add(curr_pid)
686             curr_p = proc_dict[curr_pid]
687             ancestors.append(curr_p.get('name') or
    f'PID:{curr_pid}')
688             curr_pid = curr_p.get('ppid')
689             ancestors.reverse()
690             ancestor_chain = ' → '.join(ancestors)
691
692             create_time_val = info.get('create_time')
    or 0.0
693             creation_time_str = datetime.fromtimestamp
    (create_time_val, tz=timezone.utc).isoformat() if create_time_
    val > 0 else None

```

```

694         process_guid = f'proc_{pid}_{int(create_time_val)}'
695
696         raw_cmdline = info.get('cmdline')
697         cmdline_str = ' '.join(raw_cmdline) if isinstance(raw_cmdline, list) else (str(raw_cmdline) if raw_cmdline else None)
698         launch_reason = self._classify_launch_reason(parent_name, p_name, session_id, cmdline_str)
699
700         # Классификация категории
701         has_win = pid in windowed_pids
702         is_app = has_win or (low_name in name_to_windowed and low_name not in WINDOWS_SYSTEM_PROCESS_NAMES)
703         if is_app:
704             category = 'app'
705         elif low_name in WINDOWS_SYSTEM_PROCESS_NAMES or ('c:\\windows\\system32' in exe_low and not has_win):
706             category = 'windows'
707         else:
708             category = 'background'
709
710         win_titles = titles_map.get(pid, [])
711         first_title = win_titles[0] if win_titles else None
712         friendly = classifier._get_friendly_name(p_name, win_titles)
713
714         procs.append(ProcessMetrics(
715             pid=pid,
716             name=p_name,
717             status=info.get('status') or 'running'
718             ,
719             cpu_percent=cpu_p,
720             memory_mb=rss_mb,
721             memory_percent=round(info.get('memory_percent') or 0.0, 1),
722             num_threads=info.get('num_threads') or

```

```

1,
722         num_handles=num_h,
723         username=info.get('username'),
724         integrity_level=integrity,
725         elevation=elevation,
726         ppid=ppid,
727         parent_name=parent_name,
728         executable_path=info.get('exe'),
729         sid=sid,
730         session_id=session_id,
731         creation_time=creation_time_str,
732         process_guid=process_guid,
733         ancestor_chain=ancestor_chain,
734         launch_reason=launch_reason,
735         category=category,
736         is_app=is_app,
737         friendly_name=friendly,
738         window_title=first_title,
739     ))
740     except (psutil.NoSuchProcess, psutil.AccessDen
ied):
741         continue
742     else:
743         # Fallback for non-psutil environments
744         now_iso = datetime.now(timezone.utc).isoformat()
745         p = ProcessMetrics(
746             pid=os.getpid(),
747             name='python.exe',
748             status='running',
749             cpu_percent=1.5,
750             memory_mb=120.0,
751             memory_percent=1.0,
752             num_threads=8,
753             num_handles=42,
754             username=os.getenv('USERNAME', 'SYSTEM'),
755             ppid=os.getppid() if hasattr(os, 'getppid') el
se None,
756             parent_name='cmd.exe',

```

```

757         executable_path=os.path.abspath(__file__),
758         sid='S-1-5-21-mock-sid',
759         session_id=1,
760         creation_time=now_iso,
761         process_guid=f'proc_{os.getpid()}_{int(time.time())}',
762         ancestor_chain='cmd.exe → python.exe',
763         launch_reason='terminal_cli',
764         category='background',
765         is_app=False,
766         friendly_name='Python',
767     )
768     token = token_info_map.get(p.pid)
769     if token:
770         p.integrity_level = token.integrity_level
771         p.elevation = token.elevation
772         p.sid = token.sid
773         p.session_id = token.session_id
774     procs.append(p)
775
776     # Sorting according to requested key
777     sort_key = (sort_by or 'cpu').lower()
778     if sort_key in ('handles', 'descriptors'):
779         procs.sort(key=lambda x: x.num_handles, reverse=True)
780     elif sort_key in ('memory', 'ram'):
781         procs.sort(key=lambda x: x.memory_mb, reverse=True)
782     else:
783         procs.sort(key=lambda x: x.cpu_percent, reverse=True)
784
785     if limit is None or limit <= 0:
786         return procs
787     return procs[:limit]
788
789 def get_battery_metrics(self) -> BatteryMetrics:
790     """Collect laptop or UPS battery charge level and AC p

```



```

ower status.
791
792     Returns:
793         BatteryMetrics: Current battery and power status.
794     """
795     if PSUTIL_AVAILABLE and hasattr(psutil, 'sensors_battery'):
796         try:
797             bat = psutil.sensors_battery()
798             if bat is not None:
799                 return BatteryMetrics(has_battery=True, percent=round(bat.percent, 1), power_plugged=bat.power_plugged, secs_left=bat.secsleft if bat.secsleft > 0 else None, power_profile='AC / Battery')
800         except Exception as ex:
801             logger.debug(f'Battery probe failed: {ex}')
802     return BatteryMetrics(has_battery=False, percent=None, power_plugged=True, secs_left=None, power_profile='AC Mains / Desktop')
803
804     def get_physical_disks_health(self, force: bool = False) -
805     > List[PhysicalDiskHealth]:
806         """Собрать данные о физических дисках, SMART, типах накопителей и здоровье.
807
808         В сценариях API (tc.ps1) данные читаются строго из SQLite базы данных telemetry.db
809         без вызова тяжелого опроса WMI/CIM.
810         Прямой опрос оборудования через WindowsStorageSensor выполняется только при force=True (в демоне tlm.ps1).
811
812     Args:
813         force: Принудительное обновление данных накопителя через WindowsStorageSensor (только для tlm.ps1).
814
815     Returns:
816         List[PhysicalDiskHealth]: Список обнаруженных физических накопителей.

```

```

816 |         """
817 |         now = time.time()
818 |         if not force and self._physical_disks_cached is not No
ne and now - self._physical_disks_cache_time < 43200.0:
819 |             return self._physical_disks_cached
820 |
821 |         # 1. Попытка прочитать физические диски из базы данных
SQLite
822 |         if not force and self.storage:
823 |             try:
824 |                 storage_data = self.storage.get_latest_storage
_data()
825 |                 if storage_data and storage_data.get("physical
_disks"):
826 |                     disks: List[PhysicalDiskHealth] = []
827 |                     for d in storage_data["physical_disks"]:
828 |                         disks.append(PhysicalDiskHealth(
829 |                             device_id=str(d.get("device_id")) o
r f"Disk {len(disks)}"),
830 |                             model=str(d.get("model") or d.get(
"friendly_name") or d.get("name") or "Physical Drive"),
831 |                             serial_number=str(d.get("serial_nu
mber")) if d.get("serial_number") and d.get("serial_number") != "N/A" else None,
832 |                             media_type=str(d.get("media_type")
or "SSD"),
833 |                             size_gb=round(float(d.get("size_gb
") or 0.0), 1),
834 |                             health_status=str(d.get("health_st
atus") or "Healthy"),
835 |                             operational_status=str(d.get("oper
ational_status") or "OK"),
836 |                             temperature_celsius=float(d.get("t
emperature_celsius") or d.get("temperature_c") or 0.0) if (d.g
et("temperature_celsius") is not None or d.get("temperature_c"
) is not None) else None,
837 |                             interface_type=str(d.get("interfac
e_type") or d.get("bus_type") or "NVMe"),

```

```

838         lifetime_read_bytes=d.get("lifetim
e_read_bytes"),
839         lifetime_write_bytes=d.get("lifeti
me_write_bytes"),
840         lifetime_read_tb=d.get("lifetime_r
ead_tb"),
841         lifetime_write_tb=d.get("lifetime_
write_tb"),
842         power_on_hours=d.get("power_on_hou
rs"),
843         wear_percentage=d.get("wear_perce
ntage"),
844     ))
845     if disks:
846         self._physical_disks_cached = disks
847         self._physical_disks_cache_time = now
848         return disks
849     except Exception as ex:
850         logger.debug(f"Не удалось прочитать physical_d
isks из БД: {ex}")
851
852     # 2. Прямой опрос через WindowsStorageSensor при force
= True или отсутствии дисков в БД
853     disks: List[PhysicalDiskHealth] = []
854     if os.name == 'nt':
855         try:
856             from apps.windows.sdk.modules.storage_manager.
core.windows_storage_sensor import WindowsStorageSensor
857             sensor = WindowsStorageSensor(ttl_sec=43200.0)
858             phys_disks = sensor.get_physical_disks(force_r
efresh=force, sync=True)
859             for d in phys_disks:
860                 disks.append(PhysicalDiskHealth(
861                     device_id=d.device_id,
862                     model=d.model or d.friendly_name or 'P
hysical Drive',
863                     serial_number=d.serial_number if d.ser
ial_number and d.serial_number != 'N/A' else None,

```

```

864 |             media_type=d.media_type or 'SSD',
865 |             size_gb=round(d.size_gb, 1) if d.size_
| gb else 0.0,
866 |             health_status=d.health_status or 'Heal
| thy',
867 |             operational_status=d.operational_statu
| s or 'OK',
868 |             temperature_celsius=d.temperature_c,
869 |             interface_type=d.bus_type or 'NVMe',
870 |             lifetime_read_bytes=d.lifetime_read_by
| tes,
871 |             lifetime_write_bytes=d.lifetime_write_
| bytes,
872 |             lifetime_read_tb=d.lifetime_read_tb,
873 |             lifetime_write_tb=d.lifetime_write_tb,
874 |             power_on_hours=d.power_on_hours,
875 |             wear_percentage=d.wear_percentage,
876 |         ))
877 |     except Exception as ex:
878 |         logger.debug(f'Ошибка сбора физических дисков
| через WindowsStorageSensor: {ex}')
879 |
880 |     if not disks:
881 |         disks.append(PhysicalDiskHealth(
882 |             device_id='Disk 0',
883 |             model='System Drive (NVMe/SSD)',
884 |             serial_number=None,
885 |             media_type='SSD',
886 |             size_gb=512.0,
887 |             health_status='Healthy',
888 |             operational_status='OK',
889 |             interface_type='NVMe',
890 |         ))
891 |     self._physical_disks_cached = disks
892 |     self._physical_disks_cache_time = now
893 |     return disks
894 |
895 |     async def get_ram_sticks(self) -> List[RamStickInfo]:

```

```

896 |         """Collect physical memory stick (SPD) details via WMI
897 |         .
898 |         Returns:
899 |             List[RamStickInfo]: Installed physical RAM modules
900 |         .
901 |         """
902 |         now = time.time()
903 |         if self._ram_sticks_cached is not None and now - self.
904 | _ram_sticks_cache_time < 60.0:
905 |             return self._ram_sticks_cached
906 |         sticks: List[RamStickInfo] = []
907 |         if os.name == 'nt':
908 |             from src.utils.com_worker import com_worker
909 |
910 |             async def _probe():
911 |                 def _do_probe():
912 |                     try:
913 |                         import wmi
914 |                     except ImportError:
915 |                         return []
916 |                     _sticks: List[RamStickInfo] = []
917 |                     try:
918 |                         w = wmi.WMI()
919 |                         mems = w.Win32_PhysicalMemory()
920 |                         type_map = {20: 'DDR', 21: 'DDR2', 24:
921 | 'DDR3', 26: 'DDR4', 34: 'DDR5'}
922 |                         for m in mems:
923 |                             cap_gb = round(int(m.Capacity or 0
924 | ) / 1024 ** 3, 1)
925 |                             speed = int(m.Speed or m.Configure
926 | dClockSpeed or 3200)
927 |                             m_type = type_map.get(m.SMBIOSMemo
928 | ryType, 'DDR4/DDR5')
929 |                             _sticks.append(RamStickInfo(bank_l
930 | abel=str(m.BankLabel or m.DeviceLocator or 'DIMM').strip(), ca
931 | pacity_gb=cap_gb, speed_mhz=speed, manufacturer=str(m.Manufact

```

```

    urer or 'Generic').strip(), part_number=str(m.PartNumber or ''
    ).strip(), memory_type=m_type))
925         except Exception as err:
926             logger.debug(f'WMI Win32_PhysicalMemor
y query failed: {err}')
927             return _sticks
928             return await com_worker.run(_do_probe)
929         try:
930             sticks = await _probe()
931         except Exception as ex:
932             logger.debug(f'Failed to query physical memory
sticks: {ex}')
933         self._ram_sticks_cached = sticks
934         self._ram_sticks_cache_time = now
935         return sticks
936
937     def get_listening_ports(self, limit: int=15) -> List[Netwo
rkPortMetrics]:
938         """Collect active listening TCP/UDP sockets with proce
ss attribution.
939
940         Args:
941             limit: Maximum listening ports to return.
942
943         Returns:
944             List[NetworkPortMetrics]: Active listening ports.
945         """
946         ports: List[NetworkPortMetrics] = []
947         if PSUTIL_AVAILABLE:
948             try:
949                 conns = psutil.net_connections(kind='inet')
950                 seen = set()
951                 for c in conns:
952                     if c.status == 'LISTEN' or c.type == socke
t.SOCK_DGRAM:
953                         p_num = c.laddr.port
954                         if p_num in seen:
955                             continue

```

```

956 |                 seen.add(p_num)
957 |                 p_name = None
958 |                 if c.pid:
959 |                     try:
960 |                         p_name = psutil.Process(c.pid)
961 |                         .name()
962 |                     except Exception:
963 |                         pass
964 |                 ports.append(NetworkPortMetrics(port=p
965 | _num, protocol='TCP' if c.type == socket.SOCK_STREAM else 'UDP
966 | ', address=c.laddr.ip or '0.0.0.0', pid=c.pid, process_name=p_
967 | name))
968 |                 if len(ports) >= limit:
969 |                     break
970 |             except Exception as ex:
971 |                 logger.debug(f'Failed to query listening ports
972 | : {ex}')
973 |         return ports
974 |
975 |     def get_process_network_activity(self, limit: int=50, only
976 | _internet: bool=False) -> List[ProcessNetworkActivity]:
977 |         """Collect active network connections and traffic attr
978 | ibution for processes.
979 |
980 |         Args:
981 |             limit: Maximum connections to return.
982 |             only_internet: Whether to restrict only to externa
983 | l internet addresses.
984 |
985 |         Returns:
986 |             List[ProcessNetworkActivity]: Active network conne
987 | ctions with process and traffic description.
988 |         """
989 |         activities: List[ProcessNetworkActivity] = []
990 |         if not PSUTIL_AVAILABLE:
991 |             return activities
992 |         try:
993 |             raw_conns = psutil.net_connections(kind='inet')

```

```

985 |         except Exception as ex:
986 |             logger.debug(f'Failed to query network connections
| : {ex}')
987 |             return activities
988 |         proc_cache: Dict[int, Dict[str, Any]] = {}
989 |         now_ts = time.time()
990 |
991 |         def _get_proc_info(pid: Optional[int]) -> Dict[str, An
| y]:
992 |             if not pid or pid == 0:
993 |                 return {'name': 'System Idle', 'user': 'SYSTEM
| ', 'read_kb': 0.0, 'write_kb': 0.0, 'delta_read_kb': 0.0, 'del
| ta_write_kb': 0.0, 'read_speed_kbs': 0.0, 'write_speed_kbs': 0
| .0}
994 |             if pid == 4:
995 |                 return {'name': 'System', 'user': 'NT AUTHORIT
| Y\\SYSTEM', 'read_kb': 0.0, 'write_kb': 0.0, 'delta_read_kb':
| 0.0, 'delta_write_kb': 0.0, 'read_speed_kbs': 0.0, 'write_spee
| d_kbs': 0.0}
996 |             if pid in proc_cache:
997 |                 return proc_cache[pid]
998 |             try:
999 |                 p = psutil.Process(pid)
1000 |                 pname = p.name()
1001 |                 puser = ''
1002 |                 try:
1003 |                     puser = p.username()
1004 |                 except Exception:
1005 |                     puser = ''
1006 |                 read_kb = 0.0
1007 |                 write_kb = 0.0
1008 |                 delta_read_kb = 0.0
1009 |                 delta_write_kb = 0.0
1010 |                 read_speed_kbs = 0.0
1011 |                 write_speed_kbs = 0.0
1012 |                 try:
1013 |                     io = p.io_counters()
1014 |                     read_kb = round(io.read_bytes / 1024, 1)

```



```

1015         write_kb = round(io.write_bytes / 1024, 1)
1016         if hasattr(self, '_last_proc_net_io') and
pid in self._last_proc_net_io:
1017             prev_t, prev_r, prev_w = self._last_pr
oc_net_io[pid]
1018             dt = max(now_ts - prev_t, 0.1)
1019             if io.read_bytes >= prev_r:
1020                 delta_read_kb = round((io.read_byt
es - prev_r) / 1024, 1)
1021                 read_speed_kbs = round(delta_read_
kb / dt, 1)
1022                 if io.write_bytes >= prev_w:
1023                     delta_write_kb = round((io.write_b
ytes - prev_w) / 1024, 1)
1024                     write_speed_kbs = round(delta_writ
e_kb / dt, 1)
1025                 if hasattr(self, '_last_proc_net_io'):
1026                     self._last_proc_net_io[pid] = (now_ts,
io.read_bytes, io.write_bytes)
1027             except Exception:
1028                 pass
1029             info = {'name': pname, 'user': puser, 'read_kb
': read_kb, 'write_kb': write_kb, 'delta_read_kb': delta_read_
kb, 'delta_write_kb': delta_write_kb, 'read_speed_kbs': read_s
peed_kbs, 'write_speed_kbs': write_speed_kbs}
1030             proc_cache[pid] = info
1031             return info
1032         except Exception:
1033             info = {'name': f'PID {pid}', 'user': '', 'rea
d_kb': 0.0, 'write_kb': 0.0, 'delta_read_kb': 0.0, 'delta_writ
e_kb': 0.0, 'read_speed_kbs': 0.0, 'write_speed_kbs': 0.0}
1034             proc_cache[pid] = info
1035             return info
1036
1037     def _is_ext_internet(ip: str) -> bool:
1038         if not ip:
1039             return False
1040         if ip in ('127.0.0.1', '::1', '0.0.0.0', ':::') or

```

```

        ip.startswith('127.') or ip.startswith('fe80:'):
1041         return False
1042         return True
1043         for c in raw_conns:
1044             laddr_str = f'{c.laddr.ip}:{c.laddr.port}' if c.la
laddr else '-'
1045             raddr_str = f'{c.raddr.ip}:{c.raddr.port}' if c.ra
raddr else '-'
1046             rip = c.raddr.ip if c.raddr else ''
1047             rport = c.raddr.port if c.raddr else 0
1048             lport = c.laddr.port if c.laddr else 0
1049             is_ext = _is_ext_internet(rip)
1050             if only_internet and (not is_ext):
1051                 continue
1052             proto = 'TCP' if c.type == socket.SOCK_STREAM else
'UDP'
1053             status = c.status or ('LISTEN' if not c.raddr else
'ESTABLISHED')
1054             p_info = _get_proc_info(c.pid)
1055             p_name = p_info['name']
1056             p_user = p_info['user']
1057             service_type, sent_desc, recv_desc = self._classif
y_traffic(p_name, rport or lport, proto, status, is_ext)
1058             activities.append(ProcessNetworkActivity(pid=c.pid
or 0, name=p_name, user=p_user, local_address=laddr_str, remo
te_address=raddr_str, remote_host=None, protocol=proto, status
=status, is_internet=is_ext, service_type=service_type, sent_k
b=p_info.get('write_kb', 0.0), recv_kb=p_info.get('read_kb', 0
.0), delta_sent_kb=p_info.get('delta_write_kb', 0.0), delta_re
cv_kb=p_info.get('delta_read_kb', 0.0), sent_rate_kbs=p_info.g
et('write_speed_kbs', 0.0), recv_rate_kbs=p_info.get('read_spe
ed_kbs', 0.0), sent_summary=sent_desc, recv_summary=recv_desc)
)
1059             if len(activities) >= limit:
1060                 break
1061         return activities
1062
1063     @staticmethod

```

```
1064     def _classify_traffic(proc_name: str, port: int, proto: str, status: str, is_internet: bool) -> tuple[str, str, str]:
1065         """Classify network traffic service type and generate human-readable summaries."""
1066         p_lower = proc_name.lower()
1067         if status == 'LISTEN' or not is_internet:
1068             if 'python' in p_lower or 'uvicorn' in p_lower or 'node' in p_lower:
1069                 return ('Локальный Web API / Сервер', 'HTTP ответы локальным клиентам / WebSockets', 'Входящие HTTP запросы от фронтенда / браузера')
1070                 return ('Локальная служба / Socket', 'Служебные ответы локальной подсистемы', 'Ожидание входящих подключений (Listen)')
1071             if port in (443, 8443):
1072                 if any((b in p_lower for b in ('edge', 'chrome', 'firefox', 'brave', 'opera', 'browser'))):
1073                     return ('HTTPS / Веб-навигация', 'Исходящие HTTP запросы, URL заголовки, формы, cookies', 'HTML страницы, скрипты, видео, изображения, медиа')
1074                 if any((m in p_lower for m in ('telegram', 'discord', 'slack', 'whatsapp', 'skype'))):
1075                     return ('Мессенджер / Зашифрованный чат', 'Текстовые сообщения, медиа-файлы, голосовые пакеты, пинги', 'Входящие сообщения, аудио/видеопоток, обновления чатов')
1076                 if any((s in p_lower for s in ('onedrive', 'google drivefs', 'dropbox', 'cloud'))):
1077                     return ('Облачная синхронизация / Хранилище', 'Выгрузка локальных файлов, метаданные изменений', 'Синхронизация файлов из облака, снимки папок')
1078                 if any((d in p_lower for d in ('code', 'devenv', 'language_server', 'python', 'pycharm', 'git'))):
1079                     return ('Разработка / LLM API & Git', 'AI промпты к LLM моделям, API вызовы, git push, пакеты', 'Сгенерированный AI код, ответы API, пакеты библиотек')
1080                 if any((w in p_lower for w in ('svchost', 'system', 'msmpeng', 'searchindexer', 'spoolsv'))):
1081                     return ('Системные службы Windows / Облако MS'
```

```
, 'Телеметрия Defender, проверка сертификатов, запросы ОС', 'С  
игнатуры безопасности, метаданные обновлений Windows')  
1082         return ('HTTPS / Зашифрованный веб-трафик', 'Зашиф  
рованные HTTPS/TLS запросы и полезная нагрузка', 'Ответы удале  
нного веб-сервера, зашифрованные данные')  
1083         if port in (80, 8080):  
1084             return ('HTTP / Открытый веб-трафик', 'HTTP GET/PO  
ST запросы, клиентские заголовки', 'HTML веб-страницы, незашиф  
рованный контент, ответы')  
1085             if port == 53:  
1086                 return ('DNS / Разрешение имен', 'Запросы на получ  
ение IP-адресов доменов (DNS Queries)', 'DNS-ответы с IP-адрес  
ами серверов (DNS Answers)')  
1087             if port in (5222, 5223, 5228):  
1088                 return ('Push-нотификации / XMPP', 'Heartbeat пинг  
и, исходящие push-запросы', 'Входящие фоновые push-уведомления  
и события')  
1089             if port == 22:  
1090                 return ('SSH / Защищенный терминал', 'Команды конс  
оли, зашифрованные SSH сессии', 'Вывод удаленного терминала, ф  
айловые потоки SFTP')  
1091             if port == 7680:  
1092                 return ('WUDO / P2P обновления Windows', 'P2P разд  
ача фрагментов обновлений ОС', 'P2P получение компонентов обно  
влений ОС')  
1093             if port in (465, 587):  
1094                 return ('SMTP / Почтовая отправка', 'Исходящие эле  
ктронные письма (MIME/SMTP)', 'Квитанции доставки, подтвержден  
ия почтового сервера')  
1095             if port in (993, 995):  
1096                 return ('IMAP/POP3 / Почтовый прием', 'Команды про  
верки новых писем и папок', 'Входящие электронные письма, заго  
ловки, вложения')  
1097             return (f'{proto}/{port}', f'Исходящие пакеты данных ({  
proto}) на порт {port}', f'Входящие пакеты и ответы сервера с  
порта {port}')  
1098  
1099     def get_health_alerts(self) -> SystemHealthAlerts:
```

```

1100 |         """Check system reliability indicators and pending rebo
oot status.
1101 |
1102 |         Returns:
1103 |             SystemHealthAlerts: Consolidated system health ale
rt indicators.
1104 |         """
1105 |         reboot_pending = False
1106 |         if os.name == 'nt':
1107 |             try:
1108 |                 import winreg
1109 |                 reboot_keys = ['SOFTWARE\\Microsoft\\Windows\\
CurrentVersion\\WindowsUpdate\\Auto Update\\RebootRequired', '
SOFTWARE\\Microsoft\\Windows\\CurrentVersion\\Component Based
Servicing\\RebootPending']
1110 |                 for k in reboot_keys:
1111 |                     try:
1112 |                         with winreg.OpenKey(winreg.HKEY_LOCAL_
MACHINE, k):
1113 |                             reboot_pending = True
1114 |                             break
1115 |                     except OSError:
1116 |                         pass
1117 |                 except Exception:
1118 |                     pass
1119 |                 alert_msg = 'Требуется перезагрузка для обновлений Win
dows' if reboot_pending else 'Система работает стабильно'
1120 |                 return SystemHealthAlerts(reboot_pending=reboot_pendin
g, critical_events_count=0, latest_alert=alert_msg)
1121 |
1122 |         def get_monitors(self) -> List[MonitorInfo]:
1123 |             """Collect connected display monitors, resolutions and
refresh rates.
1124 |
1125 |             Returns:
1126 |                 List[MonitorInfo]: Detected active display monitor
s.
1127 |             """

```

```

1128         if self._monitors_cached is not None:
1129             return self._monitors_cached
1130         monitors: List[MonitorInfo] = []
1131         if os.name == 'nt':
1132             try:
1133                 import ctypes.wintypes
1134                 user32 = ctypes.windll.user32
1135
1136                 class DISPLAY_DEVICEW(ctypes.Structure):
1137                     _fields_ = [('cb', ctypes.wintypes.DWORD),
1138                                ('DeviceName', ctypes.wintypes.WCHAR * 32), ('DeviceString',
1139                                ctypes.wintypes.WCHAR * 128), ('StateFlags', ctypes.wintypes.D
1140                                WORD), ('DeviceID', ctypes.wintypes.WCHAR * 128), ('DeviceKey'
1141                                , ctypes.wintypes.WCHAR * 128)]
1142
1143                 adapter_map: Dict[str, Dict[str, str]] = {}
1144                 for i in range(16):
1145                     disp = DISPLAY_DEVICEW()
1146                     disp.cb = ctypes.sizeof(DISPLAY_DEVICEW)
1147                     if not user32.EnumDisplayDevicesW(None, i,
1148                     ctypes.byref(disp), 0):
1149                         break
1150                     if disp.StateFlags & 1:
1151                         mon = DISPLAY_DEVICEW()
1152                         mon.cb = ctypes.sizeof(DISPLAY_DEVICEW
1153                     )
1154                         mon_name = ''
1155                         if user32.EnumDisplayDevicesW(disp.Dev
1156                     iceName, 0, ctypes.byref(mon), 0):
1157                             mon_name = mon.DeviceString
1158                             adapter_map[disp.DeviceName] = {'adapt
1159                     er': disp.DeviceString, 'mon_name': mon_name or 'Display Monit
1160                     or'}
1161
1162                 class MONITORINFOEXW(ctypes.Structure):
1163                     _fields_ = [('cbSize', ctypes.wintypes.DWO
1164                     RD), ('rcMonitor', ctypes.wintypes.RECT), ('rcWork', ctypes.wi
1165                     ntypes.RECT), ('dwFlags', ctypes.wintypes.DWORD), ('szDevice',
1166                     ctypes.wintypes.WCHAR * 32)]

```

```

1154
1155         class DEVMODEW(ctypes.Structure):
1156             _fields_ = [('dmDeviceName', ctypes.wintypes.WCHAR * 32), ('dmSpecVersion', ctypes.wintypes.WORD), ('dmDriverVersion', ctypes.wintypes.WORD), ('dmSize', ctypes.wintypes.WORD), ('dmDriverExtra', ctypes.wintypes.WORD), ('dmFields', ctypes.wintypes.DWORD), ('dmOrientation', ctypes.c_short), ('dmPaperSize', ctypes.c_short), ('dmPaperLength', ctypes.c_short), ('dmPaperWidth', ctypes.c_short), ('dmScale', ctypes.c_short), ('dmCopies', ctypes.c_short), ('dmDefaultSource', ctypes.c_short), ('dmPrintQuality', ctypes.c_short), ('dmColor', ctypes.c_short), ('dmDuplex', ctypes.c_short), ('dmYResolution', ctypes.c_short), ('dmTTOption', ctypes.c_short), ('dmCollate', ctypes.c_short), ('dmFormName', ctypes.wintypes.WCHAR * 32), ('dmLogPixels', ctypes.wintypes.WORD), ('dmBitsPerPel', ctypes.wintypes.DWORD), ('dmPelsWidth', ctypes.wintypes.DWORD), ('dmPelsHeight', ctypes.wintypes.DWORD), ('dmDisplayFlags', ctypes.wintypes.DWORD), ('dmDisplayFrequency', ctypes.wintypes.DWORD)]
1157
1158         def _enum_cb(h_mon: Any, hdc: Any, lprc: Any, dw_data: Any) -> bool:
1159             info = MONITORINFOEXW()
1160             info.cbSize = ctypes.sizeof(MONITORINFOEXW)
1161             user32.GetMonitorInfoW(h_mon, ctypes.byref(info))
1162             dev = info.szDevice
1163             w = info.rcMonitor.right - info.rcMonitor.left
1164             h = info.rcMonitor.bottom - info.rcMonitor.top
1165             is_prim = bool(info.dwFlags & 1)
1166             freq = 60
1167             bits = 32
1168             dm = DEVMODEW()
1169             dm.dmSize = ctypes.sizeof(DEVMODEW)
1170             orient = 'вертикальная' if h > w else 'гор

```

```

        'горизонтальная'
1171         if user32.EnumDisplaySettingsW(dev, -1, ctypes.byref(dm)):
1172             if dm.dmDisplayFrequency:
1173                 freq = dm.dmDisplayFrequency
1174             if dm.dmBitsPerPel:
1175                 bits = dm.dmBitsPerPel
1176             if dm.dmPelsWidth and dm.dmPelsHeight:
1177                 w, h = (dm.dmPelsWidth, dm.dmPelsHeight)
1178             if hasattr(dm, 'dmOrientation') and dm.dmOrientation == 2:
1179                 orient = 'вертикальная'
1180             elif hasattr(dm, 'dmOrientation') and dm.dmOrientation == 1:
1181                 orient = 'горизонтальная'
1182             if h > w:
1183                 orient = 'вертикальная'
1184             ad_info = adapter_map.get(dev, {})
1185             monitors.append(MonitorInfo(device=dev, name=ad_info.get('mon_name', 'Display Monitor'), adapter=ad_info.get('adapter', ''), width=w, height=h, frequency_hz=freq, bits_per_pixel=bits, is_primary=is_prim, orientation=orient))
1186             return True
1187         cb_type = ctypes.WINFUNCTYPE(ctypes.c_bool, ctypes.wintypes.HMONITOR, ctypes.wintypes.HDC, ctypes.POINTER(ctypes.wintypes.RECT), ctypes.wintypes.LPARAM)
1188         user32.EnumDisplayMonitors(None, None, cb_type(_enum_cb), 0)
1189     except Exception as ex:
1190         logger.debug(f'Failed to query connected monitors: {ex}')
1191     if not monitors:
1192         monitors.append(MonitorInfo(device='\\\\.\\DISPLAY1', name='Primary Monitor', adapter='Display Adapter', width=1920, height=1080, frequency_hz=60, bits_per_pixel=32, is_primary=True, orientation='горизонтальная'))
1193     self._monitors_cached = monitors

```



```

1194         return monitors
1195
1196     def get_updates_info(self) -> WindowsUpdateInfo:
1197         """Collect Windows Update status and recent installed
1198         hotfixes.
1199
1200         Returns:
1201             WindowsUpdateInfo: Summary of updates status and i
1202             nstalled KBs.
1203         """
1204         if self._updates_cached is not None:
1205             return self._updates_cached
1206         hotfixes: List[str] = []
1207         if os.name == 'nt':
1208             try:
1209                 import winreg
1210                 with winreg.OpenKey(winreg.HKEY_LOCAL_MACHINE,
1211                                     'SOFTWARE\\Microsoft\\Windows\\CurrentVersion\\Component Base
1212                                     d Servicing\\Packages') as key:
1213                     count = winreg.QueryInfoKey(key)[0]
1214                     for i in range(count):
1215                         try:
1216                             pkg_name = winreg.EnumKey(key, i)
1217                             if 'Package_for_KB' in pkg_name or
1218                             'Package_for_RollupFix' in pkg_name:
1219                                 parts = pkg_name.split('~')
1220                                 kb = parts[0].replace('Package
1221                                 _for_', '')
1222                                 if kb not in hotfixes:
1223                                     hotfixes.append(kb)
1224                         except OSError:
1225                             pass
1226             except Exception as ex:
1227                 logger.debug(f'Fast hotfix registry scan excep
1228                 tion: {ex}')
1229         if not hotfixes:
1230             hotfixes = ['KB5126052', 'KB5054156', 'KB5071430',
1231                         'KB5129195', 'KB5124007']

```



```

n)
1279 |                                     if 'microsoft
| office' in dn_str.lower() or 'microsoft 365' in dn_str.lower()
| or 'office 16' in dn_str.lower() or ('office 15' in dn_str.lo
| wer()):
1280 |                                     try:
1281 |                                         dv, _
| = winreg.QueryValueEx(sk, 'DisplayVersion')
1282 |                                         versio
| n = str(dv)
1283 |                                     except OSError
| rror:
1284 |                                         pass
1285 |                                     try:
1286 |                                         pub, _
| = winreg.QueryValueEx(sk, 'Publisher')
1287 |                                         publis
| her = str(pub)
1288 |                                     except OSError
| rror:
1289 |                                         publis
| her = 'Microsoft Corporation'
1290 |                                     product_na
| me = dn_str
1291 |                                     installed
| = True
1292 |                                     break
1293 |                                     except OSError:
1294 |                                         pass
1295 |                                     if installed:
1296 |                                         break
1297 |                                     except OSError:
1298 |                                         pass
1299 |                                     if installed:
1300 |                                         break
1301 |                                     except Exception as ex:
1302 |                                         logger.debug(f'Исключение при поиске MS Office
| : {ex}')

```

```

1303         if installed:
1304             ver_suffix = f' (v{version})' if version else ''
1305             summary = f"{product_name or 'Microsoft Office'}{v
er_suffix}"
1306         else:
1307             summary = 'Не установлен'
1308             res = OfficeSuiteInfo(installed=installed, product_nam
e=product_name, version=version, publisher=publisher, status=s
ummary)
1309             self._office_cached = res
1310             return res
1311
1312     def get_onedrive_info(self) -> CloudStorageInfo:
1313         """Сбор информации о папке/диске OneDrive и состоянии
синхронизации.
1314
1315         Returns:
1316             CloudStorageInfo: Сведения о локальном пути, емкос
ти диска и статусе синхронизации OneDrive.
1317         """
1318         import shutil
1319         od_paths: List[str] = []
1320         for env_var in ('OneDrive', 'OneDriveConsumer', 'OneDr
iveCommercial'):
1321             val = os.environ.get(env_var)
1322             if val and os.path.exists(val) and (val not in od_
paths):
1323                 od_paths.append(val)
1324         if not od_paths:
1325             home = os.path.expanduser('~')
1326             candidate = os.path.join(home, 'OneDrive')
1327             if os.path.exists(candidate):
1328                 od_paths.append(candidate)
1329         if os.name == 'nt':
1330             try:
1331                 import winreg
1332                 for subkey in ('Software\\Microsoft\\OneDrive\\
\\Accounts\\Personal', 'Software\\Microsoft\\OneDrive\\Accounts

```

```

    '\\Business1', 'Software\\Microsoft\\OneDrive'):
1333         try:
1334             with winreg.OpenKey(winreg.HKEY_CURREN
T_USER, subkey) as k:
1335                 val, _ = winreg.QueryValueEx(k, 'U
serFolder')
1336                 if val and os.path.exists(val) and
(val not in od_paths):
1337                     od_paths.append(val)
1338             except OSError:
1339                 pass
1340         except Exception:
1341             pass
1342
1343     if not od_paths:
1344         return CloudStorageInfo(installed=False, name='One
Drive', path=None, status='Не настроено')
1345
1346     target_path = od_paths[0]
1347
1348     # Проверка активности процесса синхронизации OneDrive
1349     is_running = False
1350     try:
1351         for p in psutil.process_iter(['name']):
1352             try:
1353                 pname = (p.info.get('name') or '').lower()
1354                 if pname in ('onedrive.exe', 'onedrive.sta
ndalone.exe', 'filecoauth.exe'):
1355                     is_running = True
1356                     break
1357             except (psutil.NoSuchProcess, psutil.AccessDen
ied):
1358                 continue
1359         except Exception:
1360             pass
1361
1362     try:
1363         usage = shutil.disk_usage(target_path)

```

```
1364         free_gb = round(usage.free / (1024 ** 3), 1)
1365         total_gb = round(usage.total / (1024 ** 3), 1)
1366         used_gb = round(usage.used / (1024 ** 3), 1)
1367         pct = round(usage.used / max(usage.total, 1) * 100
1368         .0, 1)
1369
1369         if is_running:
1370             summary = f'{free_gb} GB своб. (Синхронизирова
1371             н)'
1372         else:
1373             summary = f'Локальный диск ({free_gb} GB своб.
1374             , не синхронизирован)'
1375
1376         return CloudStorageInfo(
1377             installed=True,
1378             name='OneDrive',
1379             path=target_path,
1380             total_gb=total_gb,
1381             used_gb=used_gb,
1382             free_gb=free_gb,
1383             percent_used=pct,
1384             status=summary,
1385         )
1386     except Exception as ex:
1387         logger.debug(f'Ошибка проверки диска OneDrive: {ex
1388         }')
1389
1390         fallback_status = f'Локальный диск ({target_path},
1391         не синхронизирован)' if not is_running else f'Активен ({targe
1392         t_path})'
1393
1394         return CloudStorageInfo(installed=True, name='OneD
1395         rive', path=target_path, status=fallback_status)
1396
1397     async def get_core_metrics(self) -> SystemCoreMetrics:
1398         """Сверхбыстрый сбор базовой телеметрии без вызова тяж
1399         елых внешних утилит (< 0.2с).
1400
1401         Returns:
1402             SystemCoreMetrics: Мгновенные показатели загрузки
```

```

процессора, памяти, дисков и видеокарты.
1394     """
1395     now = time.time()
1396     uptime = round(now - (psutil.boot_time() if PSUTIL_AVAILABLE else now - 3600), 1)
1397     _, disk_io = self.get_disk_metrics()
1398     mems = self.get_memory_metrics()
1399     gpus = self.get_gpu_metrics()
1400     batt = self.get_battery_metrics()
1401     cpu_metrics = await self.get_cpu_metrics()
1402     self._last_time = now
1403     return SystemCoreMetrics(cpu=cpu_metrics, memory=mems,
                               gpus=gpus, disk_io=disk_io, battery=batt, uptime_seconds=uptime)
1404
1405     async def get_hardware_quick(self) -> SystemHardwareQuick:
1406         """Сбор сводки физических компонентов (диски, слоты памяти, порты, алерты).
1407
1408         Returns:
1409             SystemHardwareQuick: Состояние накопителей, планок RAM и сетевых сокетов.
1410
1411         """
1412         ram_sticks = await self.get_ram_sticks()
1413         phys_disks = self.get_physical_disks_health()
1414         ports = self.get_listening_ports()
1415         alerts = self.get_health_alerts()
1416         return SystemHardwareQuick(ram_sticks=ram_sticks, physical_disks=phys_disks, listening_ports=ports, alerts=alerts)
1417
1418     async def get_snapshot(self, process_limit: int=20) -> SystemSnapshot:
1419         """Capture full point-in-time system telemetry snapshot without blocking event loop.
1420
1421         Args:
1422             process_limit: Number of top active processes to include.

```



```

1422 |
1423 |         Returns:
1424 |             SystemSnapshot: Consolidated system and hardware s
napshot.
1425 |         """
1426 |
1427 |         def _collect_sync_telemetry() -> Dict[str, Any]:
1428 |             now = time.time()
1429 |             uptime = round(now - (psutil.boot_time() if PSUTIL
_AVAILABLE else now - 3600), 1)
1430 |             partitions, disk_io = self.get_disk_metrics()
1431 |             net_metrics = self.get_network_metrics()
1432 |             sensors: List[HardwareSensor] = []
1433 |             if self.storage:
1434 |                 try:
1435 |                     db_sensors = self.storage.get_latest_senso
rs()
1436 |                     for item in db_sensors:
1437 |                         sensors.append(
1438 |                             HardwareSensor(
1439 |                                 sensor_id=str(item.get('sensor
_id') or item.get('name') or 'unknown'),
1440 |                                 name=str(item.get('name') or i
tem.get('sensor_id') or 'sensor'),
1441 |                                 sensor_type=str(item.get('sens
or_type') or 'temperature'),
1442 |                                 value=float(item.get('value')
or 0.0),
1443 |                                 unit=str(item.get('unit') or '
°C'),
1444 |                             )
1445 |                         )
1446 |                 except Exception:
1447 |                     sensors = []
1448 |             top_procs = self.get_top_processes(limit=process_l
imit)
1449 |             ident = self.get_system_identity()
1450 |             mems = self.get_memory_metrics()

```

```

1451         gpu_list = self.get_gpu_metrics()
1452         npu_list = self.get_npu_metrics()
1453         mon_list = self.get_monitors()
1454         upd_info = self.get_updates_info()
1455         off_info = self.get_ms_office_info()
1456         od_info = self.get_onedrive_info()
1457         phys_disks = self.get_physical_disks_health()
1458         ports = self.get_listening_ports()
1459         net_activity = self.get_process_network_activity(l
limit=40)
1460         batt = self.get_battery_metrics()
1461         alerts = self.get_health_alerts()
1462         self._last_time = now
1463         return {'now': now, 'uptime': uptime, 'partitions'
: partitions, 'disk_io': disk_io, 'net_metrics': net_metrics,
'sensors': sensors, 'top_procs': top_procs, 'ident': ident, 'm
emory': mems, 'gpus': gpu_list, 'npus': npu_list, 'monitors':
mon_list, 'updates': upd_info, 'office': off_info, 'onedrive':
od_info, 'physical_disks': phys_disks, 'listening_ports': por
ts, 'net_activity': net_activity, 'battery': batt, 'alerts': a
lerts}
1464         cpu_task = asyncio.create_task(self.get_cpu_metrics())
1465         ram_task = asyncio.create_task(self.get_ram_sticks())
1466         sync_task = asyncio.create_task(asyncio.to_thread(_col
lect_sync_telemetry))
1467         cpu_metrics, ram_sticks, sync_data = await asyncio.gat
her(cpu_task, ram_task, sync_task)
1468         ident = sync_data['ident']
1469         # Читаем последний зафиксированный аудит оборудования
из базы данных (без живого опроса PnP/реестра/WMI)
1470         hardware_audit_dict: Dict[str, Any] = {}
1471         if self.storage:
1472             try:
1473                 latest_audit = self.storage.get_latest_hardwar
e_audit() or self.storage.get_latest_extended_audit()
1474                 if latest_audit:
1475                     hardware_audit_dict = latest_audit
1476             except Exception:

```

```
1477 |         pass
1478 |     return SystemSnapshot(
1479 |         hostname=ident.get('hostname') or socket.gethostna
me(),
1480 |         workgroup=ident.get('workgroup') or 'WORKGROUP',
1481 |         username=ident.get('username') or '',
1482 |         os_name=f'{platform.system()} {platform.release()}'
',
1483 |         os_build=ident.get('os_build') or platform.version
()),
1484 |         system_language=ident.get('system_language') or ''
',
1485 |         os_install_language=ident.get('os_install_language
') or '',
1486 |         user_locale=ident.get('user_locale') or '',
1487 |         system_locale=ident.get('system_locale') or '',
1488 |         timezone=ident.get('timezone') or '',
1489 |         codepage=ident.get('codepage') or '',
1490 |         input_languages=ident.get('input_languages') or []
',
1491 |         os_install_date=ident.get('os_install_date') or ''
',
1492 |         uptime_seconds=sync_data['uptime'],
1493 |         cpu=cpu_metrics,
1494 |         memory=sync_data['memory'],
1495 |         ram_sticks=ram_sticks,
1496 |         gpus=sync_data['gpus'],
1497 |         npus=sync_data.get('npus', []),
1498 |         monitors=sync_data['monitors'],
1499 |         updates=sync_data['updates'],
1500 |         office=sync_data['office'],
1501 |         onedrive=sync_data['onedrive'],
1502 |         disks=sync_data['partitions'],
1503 |         physical_disks=sync_data['physical_disks'],
1504 |         disk_io=sync_data['disk_io'],
1505 |         network=sync_data['net_metrics'],
1506 |         listening_ports=sync_data['listening_ports'],
1507 |         network_activity=sync_data['net_activity'],
```

```

1508         battery=sync_data['battery'],
1509         alerts=sync_data['alerts'],
1510         sensors=sync_data['sensors'],
1511         top_processes=sync_data['top_procs'],
1512         hardware_audit=hardware_audit_dict
1513     )
1514
1515     def get_hardware_sensors(self) -> List[HardwareSensor]:
1516         """Collect real-time hardware sensors readings.
1517
1518         Returns:
1519             List[HardwareSensor]: Hardware sensors and tempera
1520             tures.
1521             """
1522         return get_hardware_sensors()
1523
1524     async def get_hardware_tree_async(self, force: bool = False) -> List[HardwareNode]:
1525         """Generate AIDA64-like hierarchical component specification tree (Async).
1526
1527         Состояние оборудования сканируется 1 раз при запуске процесса или по отдельному запросу (force=True).
1528
1529         Args:
1530             force: Принудительный повторный опрос оборудования.
1531
1532         Returns:
1533             List[HardwareNode]: Hardware devices grouped by category.
1534             """
1535         if not force and getattr(self, '_hardware_tree_cached', None) is not None:
1536             return self._hardware_tree_cached
1537         if os.name == 'nt':
1538             try:
1539                 import pythoncom

```

```

1539         pythoncom.CoInitialize()
1540     except Exception as e:
1541         logger.debug(f'[HardwareTree] CoInitialize war
ning: {e}')
1542     nodes: List[HardwareNode] = []
1543     ident = self.get_system_identity()
1544     system_props = {
1545         'Компьютер (Host)': ident.get('hostname', ''),
1546         'Пользователь': ident.get('username', ''),
1547         'Версия Windows': platform.version(),
1548         'Номер сборки': ident.get('os_build', ''),
1549         'Дата установки ОС': ident.get('os_install_date',
),
1550         'Язык системы': ident.get('system_language', ''),
1551         'Язык установки (Оригинал)': ident.get('os_install
_language', ''),
1552         'Локали': f"User: {ident.get('user_locale', '')} |
Sys: {ident.get('system_locale', '')}",
1553         'Часовой пояс': ident.get('timezone', ''),
1554         'Кодировки': ident.get('codepage', ''),
1555         'Языки ввода': ', '.join(ident.get('input_language
s', [])),
1556         'Архитектура': platform.machine(),
1557         'Среда Python': platform.python_version()
1558     }
1559     nodes.append(HardwareNode(category='System', name=f"{i
dent.get('hostname')} ({platform.system()} {platform.release()
})", properties=system_props))
1560     cpu = await self.get_cpu_metrics()
1561     cpu_props: Dict[str, Any] = {'Модель процессора': cpu.
model, 'Физические ядра': cpu.physical_cores, 'Логические пото
ки': cpu.logical_cores, 'Базовая частота': f'{cpu.frequency_mh
z} MHz', 'Архитектура': cpu.architecture}
1562     if os.name == 'nt':
1563         try:
1564             import win32com.client
1565             wmi_obj = win32com.client.GetObject('winmgmts:
')

```

```

1566         for p in wmi_obj.InstancesOf('Win32_Processor'
1567         ):
1568             if getattr(p, 'SocketDesignation', None):
1569                 cpu_props['Разъем (Socket)'] = p.SocketDesignation
1570             if getattr(p, 'L2CacheSize', None):
1571                 cpu_props['Кэш L2'] = f'{round(p.L2CacheSize / 1024, 1)} MB ({p.L2CacheSize} KB)'
1572             if getattr(p, 'L3CacheSize', None):
1573                 cpu_props['Кэш L3'] = f'{round(p.L3CacheSize / 1024, 1)} MB ({p.L3CacheSize} KB)'
1574             if getattr(p, 'MaxClockSpeed', None):
1575                 cpu_props['Макс. частота'] = f'{p.MaxClockSpeed} MHz'
1576             if getattr(p, 'Manufacturer', None):
1577                 cpu_props['Производитель'] = p.Manufacturer
1578             break
1579         except Exception as e:
1580             logger.debug(f'[HardwareTree] WMI Win32_Processor warning: {e}')
1581         nodes.append(HardwareNode(category='Processor (CPU)',
1582         name=cpu.model, properties=cpu_props))
1583         if os.name == 'nt':
1584             mb_props: Dict[str, Any] = {}
1585             mb_name = ''
1586             mb_mfg = ''
1587             try:
1588                 import winreg
1589                 with winreg.OpenKey(winreg.HKEY_LOCAL_MACHINE,
1590                 'HARDWARE\\DESCRIPTION\\System\\BIOS') as key:
1591                     mb_mfg, _ = winreg.QueryValueEx(key, 'BaseBoardManufacturer')
1592                     mb_name, _ = winreg.QueryValueEx(key, 'BaseBoardProduct')
1593                     bios_ver, _ = winreg.QueryValueEx(key, 'BIOSVersion')
1594                     if mb_mfg:

```

```

1592         mb_props['Производитель платы'] = mb_m
    fg
1593         if mb_name:
1594             mb_props['Модель платы'] = mb_name
1595         if bios_ver:
1596             mb_props['Версия BIOS'] = bios_ver
1597     except Exception:
1598         pass
1599     try:
1600         import win32com.client
1601         wmi_obj = win32com.client.GetObject('winmgmts:
    ')
1602         for b in wmi_obj.InstancesOf('Win32_BaseBoard'
    ):
1603             if getattr(b, 'Manufacturer', None):
1604                 mb_props['Производитель платы'] = b.Ma
    nufacturer
1605             if getattr(b, 'Product', None):
1606                 mb_props['Модель платы'] = b.Product
1607             if getattr(b, 'SerialNumber', None):
1608                 mb_props['Серийный номер платы'] = (b.
    SerialNumber or '').strip()
1609             if getattr(b, 'Version', None):
1610                 mb_props['Ревизия / Версия'] = b.Versi
    on
1611             mb_name = mb_props.get('Модель платы', mb_
    name)
1612             mb_mfg = mb_props.get('Производитель платы
    ', mb_mfg)
1613             break
1614         for bios in wmi_obj.InstancesOf('Win32_BIOS'):
1615             if getattr(bios, 'Manufacturer', None):
1616                 mb_props['Производитель BIOS'] = bios.
    Manufacturer
1617             if getattr(bios, 'SMBIOSBIOSVersion', None
    ):
1618                 mb_props['Версия SMBIOS BIOS'] = bios.
    SMBIOSBIOSVersion

```



```

1646             mem_overview_props['Макс. поддерживаем
|ый объем'] = max_capacity_str
1647             break
1648             type_map = {20: 'DDR', 21: 'DDR2', 22: 'DDR2 F
|B-DIMM', 24: 'DDR3', 26: 'DDR4', 27: 'LPDDR', 28: 'LPDDR2', 29
|: 'LPDDR3', 30: 'LPDDR4', 34: 'DDR5', 35: 'LPDDR5'}
1649             form_map = {8: 'DIMM (Desktop)', 12: 'SODIMM (
|Laptop)', 9: 'TSOP', 10: 'PGA', 11: 'RIMM'}
1650
1651             raw_modules = list(wmi_obj.InstancesOf('Win32_
|PhysicalMemory'))
1652             ram_modules_found = len(raw_modules)
1653             module_speeds = [int(getattr(m, 'Speed', 0) or
|0) for m in raw_modules if getattr(m, 'Speed', 0)]
1654             module_clocks = [int(getattr(m, 'ConfiguredClo
|ckSpeed', 0) or getattr(m, 'Speed', 0) or 0) for m in raw_modu
|les if (getattr(m, 'ConfiguredClockSpeed', 0) or getattr(m, 'S
|peed', 0))]
1655             max_module_speed = max(module_speeds) if modul
|e_speeds else 0
1656             min_module_speed = min(module_speeds) if modul
|e_speeds else 0
1657
1658             for idx, m in enumerate(raw_modules):
1659                 cap_bytes = int(getattr(m, 'Capacity', 0)
|or 0)
1660                 cap_gb = round(cap_bytes / 1024 ** 3, 1)
1661                 smbios_type = int(getattr(m, 'SMBIOSMemory
|Type', 0) or 0)
1662                 mem_type = type_map.get(smbios_type, f'DDR
|(Тип {smbios_type})' if smbios_type else 'DDR')
1663                 form_code = int(getattr(m, 'FormFactor', 0
|) or 0)
1664                 form_name = form_map.get(form_code, 'DIMM'
|)
1665                 mfg = (getattr(m, 'Manufacturer', '') or '
|').strip()
1666                 if mfg == '04CB000080CE':

```

```

1667         mfg_display = 'ADATA / Micron (04CB)'
1668     elif mfg:
1669         mfg_display = mfg
1670     else:
1671         mfg_display = 'Не определен'
1672     sn = (getattr(m, 'SerialNumber', '') or ''
1673 | ).strip() or 'N/A'
1674     pn = (getattr(m, 'PartNumber', '') or '').
1675 | strip() or 'N/A'
1676     speed = getattr(m, 'Speed', 0) or 0
1677     clock = getattr(m, 'ConfiguredClockSpeed',
1678 | 0) or speed
1679     locator = (getattr(m, 'DeviceLocator', '')
1680 | or f'DIMM {idx + 1}').strip()
1681     bank = (getattr(m, 'BankLabel', '') or '')
1682 | .strip()
1683
1684     # Определение статуса согласования частот
1685 | и выявление узких мест
1686     if speed and clock and clock < speed:
1687         compat_status = f'⚠ Потенциал не раскр
1688 | ыт: планка рассчитана на {speed} MHz, но контроллер/BIOS ограничил до {clock} MHz (потеря {speed - clock} MHz)'
1689         freq_warning = f'⚠ занижена на {speed - clock} MHz)'
1690         eff_percent = f'{round((clock / speed) * 100, 1)}% от номинала планки'
1691     elif max_module_speed and speed and speed < max_module_speed:
1692         compat_status = f'⚠ Узкое место: планка {speed} MHz медленнее других модулей ({max_module_speed} MHz), снижает общую частоту'
1693         freq_warning = ''
1694         eff_percent = '100% (номинал)'
1695     else:
1696         compat_status = f'✅ Оптимально: планка работает на паспортной частоте ({clock} MHz)'
1697         freq_warning = ''

```

```

1691 |             eff_percent = '100% (номинал)'
1692 |
1693 |             module_props: Dict[str, Any] = {
1694 |                 'Слот / Разъем (Locator)': locator,
1695 |                 'Объем планки': f'{cap_gb} GB ({cap_by
tes:,} байт)',
1696 |                 'Номинальная частота': f'{speed} MHz'
| if speed else 'N/A',
1697 |                 'Текущая рабочая частота': f'{clock} M
Hz{freq_warning}' if clock else 'N/A',
1698 |                 'Эффективность планки': eff_percent,
1699 |                 'Состояние согласования частоты': comp
at_status,
1700 |                 'Тип памяти': f'{mem_type} (SMBIOS {sm
bios_type})',
1701 |                 'Форм-фактор': form_name,
1702 |                 'Производитель': mfg_display,
1703 |                 'Серийный номер (S/N)': sn,
1704 |                 'Парт-номер (P/N)': pn,
1705 |             }
1706 |             if bank:
1707 |                 module_props['Банк памяти (Bank)'] = b
ank
1708 |                 nodes.append(HardwareNode(category='Memory
Module (RAM)', name=f'Планка {idx + 1} ({locator}): {cap_gb}
GB {mem_type}-{speed or clock}', properties=module_props))
1709 |             except Exception as e:
1710 |                 logger.debug(f'[HardwareTree] WMI PhysicalMemo
ry query warning: {e}')
1711 |             if ram_modules_found > 0:
1712 |                 mem_overview_props['Установлено модулей'] = f'{ram
_modules_found} шт.'
1713 |                 if total_slots > 0:
1714 |                     free_slots = max(0, total_slots - ram_modules_
found)
1715 |                     mem_overview_props['Конфигурация слотов'] = f'
Занято {ram_modules_found} из {total_slots} (свободно: {free_s
lots})'

```

```

1716         if module_clocks and module_speeds:
1717             min_c = min(module_clocks)
1718             max_s = max(module_speeds)
1719             if min_c < max_s:
1720                 mem_overview_props['Статус частоты памяти'
] = f'␣ Частота шины занижена до {min_c} MHz (потенциал планок
: {max_s} MHz, проверьте XMP/BIOS)'
1721             else:
1722                 mem_overview_props['Статус частоты памяти'
] = f'␣ Оптимальный ({min_c} MHz, 100% номинала)'
1723         nodes.append(HardwareNode(category='System Memory', na
me=f'{mem.total_gb} GB Physical RAM', properties=mem_overview_
props))
1724         wmi_video_map: Dict[str, Dict[str, Any]] = {}
1725         if os.name == 'nt':
1726             try:
1727                 import win32com.client
1728                 wmi_obj = win32com.client.GetObject('winmgmts:
')
1729                 for v in wmi_obj.InstancesOf('Win32_VideoContr
oller'):
1730                     v_name = (getattr(v, 'Name', '') or '').st
rip()
1731                     v_pnp = (getattr(v, 'PNPDeviceID', '') or
'').strip()
1732                     v_mfg = (getattr(v, 'AdapterCompatibility'
, '') or '').strip()
1733                     v_drv_ver = (getattr(v, 'DriverVersion', '
') or '').strip()
1734                     v_drv_date_raw = getattr(v, 'DriverDate',
None)
1735                     v_drv_date = ''
1736                     if v_drv_date_raw:
1737                         ds = str(v_drv_date_raw)[:8]
1738                         if len(ds) == 8 and ds.isdigit():
1739                             v_drv_date = f'{ds[6:8]}.{ds[4:6]}
.{ds[0:4]}'
1740                     v_proc = (getattr(v, 'VideoProcessor', ''))

```

```

    or '').strip()
1741         v_mode = (getattr(v, 'VideoModeDescription
    ', '') or '').strip()
1742         v_refresh = getattr(v, 'CurrentRefreshRate
    ', None)
1743         v_status = (getattr(v, 'Status', '') or '0
    K').strip()
1744         info_dict = {'name': v_name, 'mfg': v_mfg,
    'drv_ver': v_drv_ver, 'drv_date': v_drv_date, 'proc': v_proc,
    'mode': v_mode, 'refresh': v_refresh, 'status': v_status, 'pn
    p': v_pnp}
1745         if v_name:
1746             wmi_video_map[v_name.lower()] = info_d
    ict
1747         except Exception as e:
1748             logger.debug(f'[HardwareTree] WMI VideoControl
    ler query warning: {e}')
1749         gpus = self.get_gpu_metrics()
1750         for idx, gpu in enumerate(gpus):
1751             v_extra = wmi_video_map.get(gpu.name.lower(), {})
1752             gpu_mfg = gpu.vendor or v_extra.get('mfg') or ('NV
    IDIA' if 'geforce' in gpu.name.lower() or 'nvidia' in gpu.name
    .lower() else 'Intel' if 'intel' in gpu.name.lower() else 'AMD
    ' if 'radeon' in gpu.name.lower() else 'Unknown')
1753             drv_ver = v_extra.get('drv_ver')
1754             drv_date = v_extra.get('drv_date')
1755             vproc = v_extra.get('proc')
1756             vmode = v_extra.get('mode')
1757
1758             # Форматирование памяти
1759             if gpu.dedicated_memory_mb and gpu.dedicated_memor
    y_mb >= 1024:
1760                 ded_str = f'{round(gpu.dedicated_memory_mb / 1
    024.0, 1)} GB'
1761             elif gpu.dedicated_memory_mb:
1762                 ded_str = f'{int(gpu.dedicated_memory_mb)} MB'
1763             else:
1764                 ded_str = f'{gpu.memory_total_gb} GB'

```

```

1765 |
1766 |         if gpu.shared_memory_mb and gpu.shared_memory_mb >
| = 1024:
1767 |             shared_str = f'{round(gpu.shared_memory_mb / 1
| 024.0, 1)} GB'
1768 |         elif gpu.shared_memory_mb:
1769 |             shared_str = f'{int(gpu.shared_memory_mb)} MB'
1770 |         else:
1771 |             shared_str = '0 MB'
1772 |
1773 |         total_mem_str = (
1774 |             f'{round((gpu.dedicated_memory_mb + gpu.shared
| _memory_mb) / 1024.0, 1)} GB'
1775 |             if (gpu.dedicated_memory_mb + gpu.shared_memor
| y_mb > 0)
1776 |             else f'{gpu.memory_total_gb} GB'
1777 |         )
1778 |
1779 |         gpu_props: Dict[str, Any] = {
1780 |             'Индекс устройства': idx,
1781 |             'Модель видеокарты': gpu.name,
1782 |             'Производитель (Вендор)': gpu.mfg,
1783 |             'Тип GPU': gpu.gpu_type or 'Discrete',
1784 |             'Видеопамять (VRAM)': f'{gpu.memory_total_gb}
| GB',
1785 |             'Общая память GPU': total_mem_str,
1786 |             'Выделенная VRAM': ded_str,
1787 |             'Shared GPU Memory': shared_str,
1788 |             'Тип памяти': gpu.memory_type or 'Dedicated VR
| AM',
1789 |             'Поддержка DirectX': gpu.directx_version or 'D
| irectX 12',
1790 |         }
1791 |         if vproc:
1792 |             gpu_props['Видеопроцессор'] = vproc
1793 |         if drv_ver:
1794 |             gpu_props['Версия драйвера'] = drv_ver
1795 |         if drv_date:

```

```
1796 |         gpu_props['Дата драйвера'] = drv_date
1797 |     if vmode:
1798 |         gpu_props['Текущий видеорежим'] = vmode
1799 |     elif v_extra.get('refresh'):
1800 |         gpu_props['Частота обновления'] = f"{v_extra['
refresh']}] Hz"
1801 |         gpu_props['Поддержка CUDA'] = 'Да' if gpu.has_cuda
else 'Нет'
1802 |         gpu_props['Поддержка ROCm'] = 'Да' if gpu.has_rocm
else 'Нет'
1803 |         gpu_props['Поддержка DirectML'] = 'Да' if gpu.has_
directml else 'Нет'
1804 |         gpu_props['Поддержка Vulkan'] = 'Да' if gpu.has_vu
lkan else 'Нет'
1805 |         gpu_props['Поддержка OpenCL'] = 'Да' if gpu.has_op
encl else 'Нет'
1806 |         if gpu.has_oneapi:
1807 |             gpu_props['Поддержка oneAPI'] = 'Да'
1808 |         if gpu.compute_apis:
1809 |             gpu_props['Вычислительные API'] = ', '.join(gp
u.compute_apis)
1810 |         if gpu.ai_backends:
1811 |             gpu_props['AI Бэкенды'] = ', '.join(gpu.ai_bac
kends)
1812 |         gpu_props['Статус устройства'] = v_extra.get('stat
us', 'OK (Активно)')
1813 |         if v_extra.get('pnp'):
1814 |             gpu_props['Идентификатор PnP'] = v_extra['pnp'
]
1815 |         nodes.append(HardwareNode(category='Display Adapte
r', name=gpu.name, properties=gpu_props))
1816 |     npus = self.get_npu_metrics()
1817 |     for idx, npu in enumerate(npus):
1818 |         npu_props: Dict[str, Any] = {
1819 |             'Индекс устройства': idx,
1820 |             'Модель NPU': npu.name,
1821 |             'Производитель (Вендор)': npu.vendor,
1822 |             'Тип устройства': 'Нейропроцессор (NPU / AI Ac
```

```

celerator)',
1823         'Статус устройства': npu.status,
1824     }
1825     if npu.driver_version:
1826         npu_props['Версия драйвера'] = npu.driver_vers
ion
1827     if npu.pnp_device_id:
1828         npu_props['Идентификатор PnP'] = npu.pnp_devic
e_id
1829     if npu.tops:
1830         npu_props['Производительность (TOPS)'] = f'{np
u.tops} TOPS'
1831     nodes.append(HardwareNode(category='Neural Process
ing Unit (NPU)', name=npu.name, properties=npu_props))
1832     for idx, mon in enumerate(self.get_monitors()):
1833         nodes.append(HardwareNode(category='Monitors & Dis
plays', name=f'{mon.name} ({mon.width}x{mon.height} @ {mon.fre
quency_hz}Hz, {mon.orientation})', properties={'Устройство': m
on.device, 'Название дисплея': mon.name, 'Ориентация': mon.ori
entation, 'Подключенный видеоадаптер': mon.adapter or 'Default
Adapter', 'Разрешение экрана': f'{mon.width} x {mon.height}',
'Частота развертки': f'{mon.frequency_hz} Hz', 'Глубина цвета
': f'{mon.bits_per_pixel}-bit', 'Основной монитор': 'Да (Основ
ной)' if mon.is_primary else 'Нет (Вторичный)'}))
1834     if os.name == 'nt':
1835         try:
1836             import win32com.client
1837             wmi_obj = win32com.client.GetObject('winmgmts:
')
1838             for d in wmi_obj.InstancesOf('Win32_DiskDrive'
):
1839                 size_bytes = int(getattr(d, 'Size', 0) or
0)
1840                 size_gb = round(size_bytes / 1024 ** 3, 1)
1841                 model = (getattr(d, 'Model', '') or 'Hard
Disk').strip()
1842                 iface = (getattr(d, 'InterfaceType', '') o
r 'SCSI/SATA').strip()

```



```

1843         media = (getattr(d, 'MediaType', '') or 'Fixed hard disk media').strip()
1844         sn = (getattr(d, 'SerialNumber', '') or '').strip() or 'N/A'
1845         firmware = (getattr(d, 'FirmwareRevision', '') or '').strip()
1846         parts = getattr(d, 'Partitions', 0) or 1
1847         status = (getattr(d, 'Status', '') or 'OK').strip()
1848         model_l = model.lower()
1849         if 'samsung' in model_l:
1850             disk_mfg = 'Samsung'
1851         elif 'crucial' in model_l or model_l.startswith('ct'):
1852             disk_mfg = 'Crucial / Micron'
1853         elif 'toshiba' in model_l or model_l.startswith('hdwd') or model_l.startswith('dt01'):
1854             disk_mfg = 'Toshiba'
1855         elif 'wdc' in model_l or model_l.startswith('wd') or 'western digital' in model_l:
1856             disk_mfg = 'Western Digital'
1857         elif 'kingston' in model_l or model_l.startswith('sa400') or model_l.startswith('skc'):
1858             disk_mfg = 'Kingston'
1859         elif 'seagate' in model_l or model_l.startswith('st'):
1860             disk_mfg = 'Seagate'
1861         elif 'sandisk' in model_l:
1862             disk_mfg = 'SanDisk'
1863         elif 'kioxia' in model_l:
1864             disk_mfg = 'Kioxia'
1865         elif 'intel' in model_l:
1866             disk_mfg = 'Intel'
1867         elif 'micron' in model_l:
1868             disk_mfg = 'Micron'
1869         elif 'sk hynix' in model_l or 'hynix' in model_l:
1870             disk_mfg = 'SK Hynix'

```

[illegible]

```

1894         for a in wmi_obj.InstancesOf('Win32_NetworkAda
1895 | pter'):
1896             net_id = (getattr(a, 'NetConnectionID', ''
1897 | ) or '').strip()
1898             aname = (getattr(a, 'Name', '') or '').stri
1899 | p()
1900             adesc = (getattr(a, 'Description', '') or
1901 | '').strip()
1902             apnp = (getattr(a, 'PNPDeviceID', '') or '
1903 | ').strip()
1904             amfg = (getattr(a, 'Manufacturer', '') or
1905 | '').strip()
1906             amac = (getattr(a, 'MACAddress', '') or ''
1907 | ).strip()
1908             aspeed = getattr(a, 'Speed', None)
1909             atype = (getattr(a, 'AdapterType', '') or
1910 | '').strip()
1911             astatus_num = getattr(a, 'NetConnectionSta
1912 | tus', None)
1913             aphys = getattr(a, 'PhysicalAdapter', Fals
1914 | e)
1915             item = {'net_id': net_id, 'name': aname, '
1916 | desc': adesc, 'pnp': apnp, 'mfg': amfg, 'mac': amac, 'speed':
1917 | aspeed, 'ad_type': atype, 'status_num': astatus_num, 'is_phys'
1918 | : aphys}
1919             if net_id:
1920                 wmi_net_adapters[net_id.lower()] = ite
1921 | m
1922             if aname:
1923                 wmi_net_adapters[aname.lower()] = item
1924             if adesc:
1925                 wmi_net_adapters[adesc.lower()] = item
1926     except Exception as e:
1927         logger.debug(f'[HardwareTree] WMI NetworkAdapt
1928 | er query warning: {e}')
1929     if PSUTIL_AVAILABLE:
1930         ps_addrs = psutil.net_if_addrs()
1931         ps_stats = psutil.net_if_stats()

```

```

1917         for if_name, addr_list in ps_addrs.items():
1918             stat = ps_stats.get(if_name)
1919             w_info = wmi_net_adapters.get(if_name.lower(),
1920             {})
1921             cfg = wmi_net_configs.get(w_info.get('desc', '
1922             ').lower()) or wmi_net_configs.get(if_name.lower())
1923             mac_addr = w_info.get('mac')
1924             if not mac_addr:
1925                 for a in addr_list:
1926                     fam_name = getattr(a.family, 'name', '
1927                     ')
1928                     if fam_name == 'AF_LINK' or a.family =
1929                     = -1 or a.family == getattr(psutil, 'AF_LINK', -1):
1930                         if a.address and len(a.address) in
1931                         (12, 17):
1932                             mac_addr = a.address.replace('
1933                             -', ':').upper()
1934                             break
1935             ipv4_list = [a.address for a in addr_list if a
1936             .family == socket.AF_INET]
1937             ipv6_list = [a.address.split('%')[0] for a in
1938             addr_list if hasattr(socket, 'AF_INET6') and a.family == socke
1939             t.AF_INET6]
1940             gateways = list(getattr(cfg, 'DefaultIPGateway
1941             ', [])) or [] if cfg else []
1942             dhcp_server = getattr(cfg, 'DHCP_Server', None)
1943             if cfg else None
1944             dhcp_enabled = getattr(cfg, 'DHCP_Enabled', Non
1945             e) if cfg else None
1946             dns_servers = list(getattr(cfg, 'DNS_Server_Sear
1947             chOrder', [])) or [] if cfg else []
1948             is_up = stat.isup if stat else True
1949             status_num = w_info.get('status_num')
1950             if status_num == 2 or (status_num is None and
1951             is_up):
1952                 status_str = 'Активно (Up)'
1953             elif status_num == 7:
1954                 status_str = 'Кабель не подключен (Disconn

```

```

ected)'
1941         elif status_num == 0:
1942             status_str = 'Отключено (Disabled)'
1943         elif not is_up:
1944             status_str = 'Не активно (Down)'
1945         else:
1946             status_str = 'Активно (Up)'
1947         speed_mbps = 0
1948         if stat and stat.speed and (stat.speed > 0) and (stat.speed < 1000000):
1949             speed_mbps = stat.speed
1950         elif w_info.get('speed'):
1951             try:
1952                 raw_spd = int(w_info['speed'])
1953                 if raw_spd < 1000000000000:
1954                     speed_mbps = round(raw_spd / 10000
00)
1955             except Exception:
1956                 pass
1957         if speed_mbps > 0:
1958             if speed_mbps >= 1000:
1959                 speed_str = f'{speed_mbps} Mbps ({round(speed_mbps / 1000, 1)} Gbps)'
1960             else:
1961                 speed_str = f'{speed_mbps} Mbps'
1962         else:
1963             speed_str = 'Автоопределение (0 Mbps)' if not is_up else '1000 Mbps (Авто)'
1964         model_name = w_info.get('name') or w_info.get('desc') or if_name
1965         mfg_val = w_info.get('mfg')
1966         if not mfg_val:
1967             if 'intel' in if_name.lower() or 'intel' in model_name.lower():
1968                 mfg_val = 'Intel Corporation'
1969             elif 'realtek' in if_name.lower() or 'realtek' in model_name.lower():
1970                 mfg_val = 'Realtek'

```

```

1971         elif 'vethernet' in if_name.lower() or 'hy
per-v' in model_name.lower() or 'microsoft' in model_name.lowe
r() or ('local area connection*' in if_name.lower()):
1972             mfg_val = 'Microsoft Corporation'
1973         else:
1974             mfg_val = 'Универсальный сетевой контр
оллер'
1975             name_l = if_name.lower()
1976             desc_l = (w_info.get('desc') or '').lower()
1977             if 'local area connection*' in name_l:
1978                 adapter_type = 'Виртуальный адаптер Wi-Fi
Direct'
1979                 if model_name == if_name:
1980                     model_name = f'Microsoft Wi-Fi Direct
Virtual Adapter ({if_name})'
1981                 elif 'wi-fi' in name_l or 'wireless' in desc_l
or '802.11' in desc_l:
1982                     adapter_type = 'Беспроводной адаптер (Wi-F
i 802.11)'
1983                 elif 'bluetooth' in name_l or 'bluetooth' in d
esc_l:
1984                     adapter_type = 'Bluetooth PAN адаптер'
1985                 elif 'vethernet' in name_l or 'virtual' in des
c_l or 'hyper-v' in desc_l:
1986                     adapter_type = 'Виртуальный сетевой адапте
р (Hyper-V / WSL)'
1987                 elif 'loopback' in name_l:
1988                     adapter_type = 'Программный интерфейс Loop
back'
1989                 elif 'tap' in name_l or 'tun' in name_l or 'vp
n' in name_l:
1990                     adapter_type = 'VPN / Туннельный адаптер'
1991                 else:
1992                     adapter_type = 'Сетевая карта Ethernet (LA
N)'
1993             net_props: Dict[str, Any] = {'Имя подключения'
: if_name, 'Модель адаптера': model_name, 'Производитель': mfg
_val, 'Тип адаптера': adapter_type, 'Физический (MAC) адрес':

```

```

mac_addr or 'N/A', 'Состояние соединения': status_str, 'Скорос
ть канала': speed_str}
1994         if stat and getattr(stat, 'duplex', None) is n
ot None:
1995             d_val = getattr(stat.duplex, 'value', stat
.duplex)
1996             if d_val == 2:
1997                 net_props['Режим дуплекса'] = 'Full Du
plex (Полный)'
1998             elif d_val == 1:
1999                 net_props['Режим дуплекса'] = 'Half Du
plex (Полудуплекс)'
2000             if stat and getattr(stat, 'mtu', None):
2001                 net_props['Размер MTU'] = f'{stat.mtu} бай
т'
2002                 net_props['IP-адреса (IPv4)'] = ', '.join(ipv4
_list) or 'N/A'
2003             if ipv6_list:
2004                 net_props['IP-адреса (IPv6)'] = ', '.join(
ipv6_list[:3])
2005             if gateways:
2006                 net_props['Основной шлюз'] = ', '.join(gat
eways)
2007             if dhcp_server or dhcp_enabled is not None:
2008                 net_props['DHCP сервер'] = dhcp_server or
('Включен (DHCP)' if dhcp_enabled else 'Статический IP / Отклю
чен')
2009             if dns_servers:
2010                 net_props['DNS серверы'] = ', '.join(dns_s
ervers)
2011             if w_info.get('pnp'):
2012                 net_props['Идентификатор PnP'] = w_info['p
np']
2013                 display_title = f'{if_name}: {model_name}' if
model_name != if_name else if_name
2014                 nodes.append(HardwareNode(category='Network Ad
apter', name=display_title, properties=net_props))
2015             if os.name == 'nt':

```

```

2016         try:
2017             import win32com.client
2018             wmi_obj = win32com.client.GetObject('winmgmts:
| ')
2019             for s in wmi_obj.InstancesOf('Win32_SoundDevic
| e'):
2020                 snd_name = (getattr(s, 'Caption', '') or g
| etattr(s, 'Name', '') or '').strip()
2021                 snd_mfg = (getattr(s, 'Manufacturer', '')
| or '').strip() or 'Аудиоустройство'
2022                 snd_status = (getattr(s, 'Status', '') or
| 'OK').strip()
2023                 snd_pnp = (getattr(s, 'PNPDeviceID', '') o
| r '').strip()
2024                 if snd_name:
2025                     nodes.append(HardwareNode(category='Au
| dio Device', name=snd_name, properties={'Название устройства':
| snd_name, 'Производитель': snd_mfg, 'Состояние': f'{snd_statu
| s} (Работает)', 'Идентификатор PnP': snd_pnp or 'N/A'}))
2026             except Exception as e:
2027                 logger.debug(f'[HardwareTree] WMI SoundDevice
| query warning: {e}')
2028             if os.name == 'nt':
2029                 try:
2030                     import win32com.client
2031                     wmi_obj = win32com.client.GetObject('winmgmts:
| ')
2032                     for u in wmi_obj.InstancesOf('Win32_USBControl
| ler'):
2033                         usb_name = (getattr(u, 'Caption', '') or g
| etattr(u, 'Name', '') or '').strip()
2034                         usb_mfg = (getattr(u, 'Manufacturer', '')
| or '').strip() or 'USB Контроллер'
2035                         usb_status = (getattr(u, 'Status', '') or
| 'OK').strip()
2036                         usb_pnp = (getattr(u, 'PNPDeviceID', '') o
| r '').strip()
2037                         if usb_name:

```



```

2038         nodes.append(HardwareNode(category='USB Controller', name=usb_name, properties={'Название контроллера': usb_name, 'Производитель': usb_mfg, 'Состояние': f'{usb_status} (Работает)', 'Идентификатор PnP': usb_pnp or 'N/A'}))
2039     except Exception as e:
2040         logger.debug(f'[HardwareTree] WMI USBController query warning: {e}')
2041     upd = self.get_updates_info()
2042     nodes.append(HardwareNode(category='Windows Updates', name=f'{upd.status} ({upd.installed_kb_count} KBs)', properties={'Статус обновлений': upd.status, 'Число установленных исправлений': upd.installed_kb_count, 'Последние KB пакеты': ', '.join(upd.recent_hotfixes) or 'N/A', 'Дата последней установки': upd.latest_installed_on or 'OK'}))
2043     self._hardware_tree_cached = nodes
2044     return nodes
2045
2046     def get_hardware_tree(self, force: bool = False) -> List[HardwareNode]:
2047         """Generate AIDA64-like hierarchical component specification tree.
2048
2049         Args:
2050             force: Принудительный повторный опрос оборудования
2051             .
2052
2053         Returns:
2054             List[HardwareNode]: Hardware devices grouped by category.
2055
2056         """
2057         import asyncio
2058         return asyncio.run(self.get_hardware_tree_async(force=force))
2059
2060     def get_hardware_audit(self) -> HardwareAuditReport:
2061         """Collect deep hardware devices audit with drivers, install dates, and attached sensors.

```

```

2061         Returns:
2062             HardwareAuditReport: Complete hardware audit repor
2063         t.
2064         """
2065         return self.auditor.audit_hardware()
2066
2067     def archive_hardware_state(self, auto_diff: bool=True) ->
2068     HardwareArchiveEntry:
2069         """Capture current hardware audit and save it to histo
2070         rical archive storage.
2071
2072         Args:
2073             auto_diff: Automatically compute diff with previou
2074             s archive.
2075
2076         Returns:
2077             HardwareArchiveEntry: Persisted archive entry with
2078             detected changes.
2079         """
2080         report = self.get_hardware_audit()
2081         return self.history_manager.archive_report(report, aut
2082         o_diff=auto_diff)
2083
2084     def get_hardware_history(self, limit: int=50) -> List[Dict
2085     [str, Any]]:
2086         """Retrieve historical hardware archive snapshots meta
2087         data.
2088
2089         Args:
2090             limit: Maximum count of archive entries.
2091
2092         Returns:
2093             List[Dict[str, Any]]: Chronological list of archiv
2094             e entries.
2095         """
2096         return self.history_manager.get_history(limit=limit)
2097
2098     def get_hardware_changes(self, limit: int=100) -> List[Har

```

```

    dwareChangeItem]:
2090         """Retrieve historical timeline of all hardware config
    uration changes.
2091
2092         Args:
2093             limit: Maximum count of changes to return.
2094
2095         Returns:
2096             List[HardwareChangeItem]: Historical changes timeline.
2097         """
2098         return self.history_manager.get_change_timeline(limit=
    limit)
2099
2100     def get_startup_audit(self) -> Any:
2101         """Collect deep Windows startup and autoruns audit report.
2102
2103         Returns:
2104             AuditReport: Complete startup audit report with risk ratings.
2105         """
2106         from apps.windows.sdk.modules.startup.core.auditor import StartupAuditor
2107         return StartupAuditor().run_audit()
2108
2109     def archive_startup_state(self, auto_diff: bool = True) ->
    StartupArchiveEntry:
2110         """Capture current Windows startup state, compute changes diff, and save snapshot to storage.
2111
2112         Args:
2113             auto_diff: Automatically compute diff with previous archive snapshot.
2114
2115         Returns:
2116             StartupArchiveEntry: Persisted archive entry with detected changes.

```

```

2117 |         """
2118 |         report = self.get_startup_audit()
2119 |         entry = self.startup_history_manager.archive_report(re
|port, auto_diff=auto_diff)
2120 |         if self.storage:
2121 |             try:
2122 |                 self.storage.save_startup_archive(entry)
2123 |             except Exception as ex:
2124 |                 logger.debug(f'Не удалось сохранить снимок авт
|озапуска в БД: {ex}')
2125 |         return entry
2126 |
2127 |     def get_startup_history(self, limit: int = 50) -> List[Dic
|t[str, Any]]:
2128 |         """Retrieve historical startup archive snapshots metad
|ata.
2129 |
2130 |         Args:
2131 |             limit: Maximum count of archive entries.
2132 |
2133 |         Returns:
2134 |             List[Dict[str, Any]]: Chronological list of startu
|p archive entries.
2135 |         """
2136 |         if self.storage:
2137 |             try:
2138 |                 db_history = self.storage.get_startup_archives
|(limit=limit)
2139 |                 if db_history:
2140 |                     return db_history
2141 |             except Exception:
2142 |                 pass
2143 |         return self.startup_history_manager.get_history(limit=
|limit)
2144 |
2145 |     def get_startup_changes(self, limit: int = 100) -> List[Di
|ct[str, Any]]:
2146 |         """Retrieve historical timeline of all startup configu

```

```

ration changes.
2147
2148     Args:
2149         limit: Maximum count of changes to return.
2150
2151     Returns:
2152         List[Dict[str, Any]]: Historical changes timeline.
2153     """
2154     if self.storage:
2155         try:
2156             db_changes = self.storage.get_startup_changes(
2157 limit=limit)
2158             if db_changes:
2159                 return db_changes
2160             except Exception:
2161                 pass
2162             return [c.model_dump(mode='json') for c in self.startu
2163 p_history_manager.get_change_timeline(limit=limit)]
2164
2165     def get_latest_startup_audit(self) -> Optional[Dict[str, A
2166 ny]]:
2167         """Retrieve latest recorded startup audit snapshot fro
2168 m telemetry storage.
2169
2170     Returns:
2171         Optional[Dict[str, Any]]: Startup audit dict or No
2172 ne.
2173     """
2174     if self.storage:
2175         try:
2176             latest = self.storage.get_latest_startup_archi
2177 ve()
2178             if latest:
2179                 return latest
2180             except Exception:
2181                 pass
2182             latest_archive = self.startup_history_manager.get_late
2183 st_archive()

```

```

2177         return latest_archive.model_dump(mode='json') if latest_archive else None
2178
2179     def save_snapshot_to_db(self, snapshot: Optional[SystemSnapshot]=None, top_n: int=20) -> int:
2180         """Collect and save system telemetry snapshot directly into SQLite database.
2181
2182         Args:
2183             snapshot: Optional existing SystemSnapshot (if None, collected synchronously).
2184             top_n: Top processes count.
2185
2186         Returns:
2187             int: Created snapshot ID in database.
2188         """
2189         snap = snapshot
2190         if snap is None:
2191             import asyncio
2192             try:
2193                 loop = asyncio.get_event_loop()
2194             except RuntimeError:
2195                 loop = None
2196             if loop and loop.is_running():
2197                 snap = asyncio.run_coroutine_threadsafe(self.get_snapshot(process_limit=top_n), loop).result(timeout=5.0)
2198             else:
2199                 snap = asyncio.run(self.get_snapshot(process_limit=top_n))
2200         return self.storage.save_snapshot(snap, top_n=top_n)
2201
2202     def get_snapshots(self, limit: int=60, since_epoch: Optional[float]=None) -> List[Dict[str, Any]]:
2203         """Retrieve persisted system snapshots from SQLite database.
2204
2205         Args:
2206             limit: Maximum number of snapshots.

```

```

2207         since_epoch: Optional starting timestamp.
2208
2209     Returns:
2210         List[Dict[str, Any]]: List of system snapshots from
2211         database.
2212         """
2213         return self.storage.get_snapshots(limit=limit, since_epoch=since_epoch)
2214
2215     def get_latest_cpu_from_db(self) -> Optional[Dict[str, Any]]:
2216         """Извлекает последние метрики процессора из базы данных SQLite.
2217
2218         Returns:
2219             Optional[Dict[str, Any]]: Метрики процессора из SQLite.
2220             """
2221         return self.storage.get_latest_cpu()
2222
2223     def get_cpu_history_from_db(self, limit: int = 60, since_epoch: Optional[float] = None) -> List[Dict[str, Any]]:
2224         """Извлекает историю нагрузки процессора из базы данных SQLite.
2225
2226         Args:
2227             limit: Количество записей.
2228             since_epoch: Начальное время UNIX epoch.
2229
2230         Returns:
2231             List[Dict[str, Any]]: Исторические срезы нагрузки CPU.
2232             """
2233         return self.storage.get_cpu_history(limit=limit, since_epoch=since_epoch)
2234
2235     def get_process_history(self, name: Optional[str]=None, pid: Optional[int]=None, limit: int=100) -> List[Dict[str, Any]]

```

```

:
2235     """Retrieve historical process metrics from SQLite dat
abase.
2236
2237     Args:
2238         name: Substring of process name.
2239         pid: Process identifier.
2240         limit: Record limit.
2241
2242     Returns:
2243         List[Dict[str, Any]]: History of process metrics.
2244     """
2245     return self.storage.get_process_history(name=name, pid
=pid, limit=limit)
2246
2247     def get_reboot_report(self, limit: int = 20, hours: int =
720) -> Any:
2248     """Сбор сводного отчета анализа перезагрузок и выключе
ний ОС.
2249
2250     Args:
2251         limit: Максимальное количество сессий.
2252         hours: Глубина выборки журналов в часах.
2253
2254     Returns:
2255         RebootAnalysisReport: Сводный аналитический отчет.
2256     """
2257     from apps.windows.telemetry_research.reboot_analyzer i
mport WindowsRebootAnalyzer
2258     analyzer = WindowsRebootAnalyzer(storage=self.storage)
2259     return analyzer.collect_reboot_history(limit=limit, ho
urs=hours)
2260
2261     def get_latest_reboot(self) -> Optional[Any]:
2262     """Получение данных о последней перезагрузке системы.
2263
2264     Returns:
2265         Optional[RebootSession]: Информация о последнем пе

```



```

резапуске.
2266     """
2267     from apps.windows.telemetry_research.reboot_analyzer i
import WindowsRebootAnalyzer
2268     analyzer = WindowsRebootAnalyzer(storage=self.storage)
2269     return analyzer.analyze_current_boot()
2270
2271     def get_extended_system_audit(self) -> Any:
2272         """Сбор расширенного аудита безопасности (Defender), а
втозагрузки (Startup), VSS и пользователей.
2273
2274         Returns:
2275             ExtendedSystemAuditReport: Сводный отчет расширенн
ого аудита.
2276         """
2277         from apps.windows.telemetry_research.deep_diagnostics
import DeepDiagnosticsEngine
2278         engine = DeepDiagnosticsEngine()
2279         report = engine.collect_extended_system_audit()
2280         try:
2281             self.storage.save_extended_audit(report)
2282         except Exception as ex:
2283             logger.debug(f'Не удалось сохранить расширенный ау
дит в БД: {ex}')
2284         return report
2285
2286     def get_latest_extended_audit(self) -> Optional[Dict[str,
Any]]:
2287         """Получение последнего сохраненного среза расширенног
о системного аудита.
2288
2289         Returns:
2290             Optional[Dict[str, Any]]: Словарь аудита или None.
2291         """
2292         return self.storage.get_latest_extended_audit()
2293
2294     def get_process_provenance_report(
2295         self,

```

```

2296         pid: Optional[int] = None,
2297         name_filter: Optional[str] = None,
2298         user_filter: Optional[str] = None,
2299         limit: int = 100,
2300     ) -> ProcessProvenanceReport:
2301         """Сформировать сводный отчет о происхождении и цепочк
ах запуска активных процессов.
2302
2303         Args:
2304             pid: Опциональный фильтр по PID.
2305             name_filter: Опциональный фильтр по имени процесса
.
2306             user_filter: Опциональный фильтр по пользователю/S
ID.
2307             limit: Максимальное количество процессов в отчете.
2308
2309         Returns:
2310             ProcessProvenanceReport: Полный отчет с детальным
анализом происхождения процессов.
2311         """
2312         all_procs = self.get_top_processes(limit=0, sort_by='c
pu')
2313         filtered_items: List[ProcessProvenanceInfo] = []
2314
2315         for p in all_procs:
2316             if pid is not None and p.pid != pid:
2317                 continue
2318             if name_filter and name_filter.lower() not in (p.n
ame or '').lower():
2319                 continue
2320             if user_filter and user_filter.lower() not in (p.u
sername or '').lower() and user_filter.lower() not in (p.sid o
r '').lower():
2321                 continue
2322
2323             chain_list = [node.strip() for node in (p.ancestor
_chain or '').split(" → ") if node.strip()]
2324             user_display = p.username or "Unknown"

```

```

2325         sid_display = p.sid or "N/A"
2326         session_display = f"Session {p.session_id}" if p.s
ession_id is not None else "Session N/A"
2327         who_runs = f"{user_display} ({sid_display}, {sessi
on_display})"
2328         who_created = f"{p.parent_name or 'Unknown'} (PPID
: {p.ppid})"
2329         why_created = p.launch_reason or "unknown"
2330
2331         item = ProcessProvenanceInfo(
2332             pid=p.pid,
2333             ppid=p.ppid,
2334             name=p.name,
2335             executable_path=p.executable_path,
2336             command_line=p.cmdline,
2337             user=p.username,
2338             sid=p.sid,
2339             session_id=p.session_id,
2340             integrity_level=p.integrity_level,
2341             elevation=bool(p.elevation) if p.elevation is
not None else None,
2342             creation_time=p.creation_time,
2343             process_guid=p.process_guid or f"proc_{p.pid}"
,
2344             parent_name=p.parent_name,
2345             ancestor_chain=chain_list,
2346             ancestor_chain_str=p.ancestor_chain or "",
2347             who_runs_it=who_runs,
2348             who_created_it=who_created,
2349             why_was_it_created=why_created,
2350         )
2351         filtered_items.append(item)
2352         if limit and len(filtered_items) >= limit:
2353             break
2354
2355         now_iso = datetime.now(timezone.utc).isoformat()
2356         recent_evs: List[ProcessLifecycleEvent] = []
2357         try:

```

```

2358         for ev in self.storage.get_process_provenance_hist
|ory(limit=20):
2359             if isinstance(ev, dict):
2360                 recent_evs.append(ProcessLifecycleEvent(
2361                     event_id=str(ev.get("event_id") or "")
|,
2362                     timestamp=str(ev.get("timestamp") or n
|ow_iso),
2363                     event_type=str(ev.get("event_type") or
|"ProcessCreated"),
2364                     pid=int(ev.get("pid") or 0),
2365                     ppid=int(ev["parent_pid"]) if ev.get("
|parent_pid") is not None else None,
2366                     process_guid=str(ev.get("process_guid"
|) or ""),
2367                     parent_guid=ev.get("parent_guid"),
2368                     name=str(ev.get("name") or "unknown"),
2369                     executable_path=ev.get("executable_pat
|h"),
2370                     command_line=ev.get("command_line"),
2371                     user=ev.get("user"),
2372                     sid=ev.get("sid"),
2373                     session_id=int(ev["session_id"]) if ev
|.get("session_id") is not None else None,
2374                     integrity_level=ev.get("integrityleve
|l"),
2375                     elevation=bool(ev.get("elevation")) if
|ev.get("elevation") is not None else None,
2376                     parent_name=ev.get("parent_name"),
2377                     ancestor_chain=ev.get("ancestor_chain"
|),
2378                     launch_reason=ev.get("launch_reason"),
2379                     source=str(ev.get("source") or "system
|"),
2380                     raw_json=str(ev.get("details_json")) i
|f ev.get("details_json") else None,
2381                 ))
2382         except Exception as ex:

```

```

2383         logger.debug(f"Не удалось извлечь недавние события
    provenance для отчета: {ex}")
2384
2385         summary = f"Всего активных процессов: {len(all_procs)}
    . Проанализировано происхождение для {len(filtered_items)} про
    цессов."
2386         return ProcessProvenanceReport(
2387             timestamp=now_iso,
2388             total_processes=len(all_procs),
2389             active_provenance=filtered_items,
2390             recent_events=recent_evs,
2391             summary=summary,
2392         )
2393
2394     def get_process_provenance_history(
2395         self,
2396         guid: Optional[str] = None,
2397         pid: Optional[int] = None,
2398         name: Optional[str] = None,
2399         user: Optional[str] = None,
2400         event_type: Optional[str] = None,
2401         limit: int = 100,
2402     ) -> List[Dict[str, Any]]:
2403         """Получить исторические события жизненного цикла и пр
    оисхождения процессов из БД.
2404
2405         Args:
2406             guid: Фильтр по уникальному идентификатору процесс
    а.
2407             pid: Фильтр по PID.
2408             name: Фильтр по имени процесса.
2409             user: Фильтр по имени пользователя или SID.
2410             event_type: Фильтр по типу события ('ProcessCreate
    d', 'ProcessTerminated').
2411             limit: Максимальное количество записей.
2412
2413         Returns:
2414             List[Dict[str, Any]]: Список исторических событий

```

происхождения процессов.

2415 """

2416 return self.storage.get_process_provenance_history(

2417 guid=guid,

2418 pid=pid,

2419 name=name,

2420 user=user,

2421 event_type=event_type,

2422 limit=limit,

2423)

2424

2425 def get_process_lineage(self, guid_or_pid: str) -> List[Dict[str, Any]]:

2426 """Построить полную восходящую цепочку предков процесса (дерево запуска).

2427

2428 Args:

2429 guid_or_pid: Идентификатор process_guid или числовой PID.

2430

2431 Returns:

2432 List[Dict[str, Any]]: Упорядоченный список предков от корневого процесса к целевому.

2433 """

2434 return self.storage.get_process_lineage(guid_or_pid=guid_or_pid)

2435

2436

2437

device_flapping_sensor.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - Device
  Flapping Sensor
4 # =====
5 # Description:
6 #   Сенсор непрерывного мониторинга периферии, обнаружения дре
  безга и PnP-телеметрии.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.device_flapping_sensor impor
  t DeviceTransitionEvent
11 #
12 #       service = DeviceTransitionEvent()
13 #
14 # File: device_flapping_sensor.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-01 13:28:28
20 # =====
21
22 from __future__ import annotations
23 """Сенсор непрерывного мониторинга периферии, обнаружения дреб
  езга и PnP-телеметрии."""
24
25 import os
26 import threading
27 import time
28 from dataclasses import asdict, dataclass, field
29 from datetime import datetime, timezone
```

```

30 from typing import Any, Callable, Dict, List, Optional, Set
31 from logger import logger
32 from .win32_ffi.setupapi import PnPDeviceInfo, SetupAPI
33 from .sqlite import TelemetryStorage
34
35 @dataclass
36 class DeviceTransitionEvent:
37     """Модель события изменения состояния устройства в телемет
    | рии."""
38     timestamp: str
39     event_type: str
40     device_instance_id: str
41     friendly_name: str
42     device_class: str
43     category: str
44     has_problem: bool
45     problem_code: int
46     status_code: int
47     manufacturer: str
48     flapping_count_in_window: int = 0
49     uptime_seconds: float = 0.0
50
51     def to_dict(self) -> Dict[str, Any]:
52         """Преобразование события в словарь для сериализации."
    | ""
53         return asdict(self)
54
55 class DeviceFlappingSensor:
56     """Сенсор непрерывного мониторинга и отлова дребезга обору
    | дования."""
57     CATEGORY_MAPPING: Dict[str, str] = {'HIDClass': 'HID & Пер
    | иферия', 'Mouse': 'Мышь / Указатель', 'Keyboard': 'Клавиатура'
    | , 'DiskDrive': 'Дисконкей / Накопитель', 'Volume': 'Том накопи
    | теля', 'USB': 'USB Устройство / Контроллер', 'USBDevice': 'USB
    | Периферия', 'Monitor': 'HDMI / DP / Монитор', 'Display': 'Вид
    | еоадаптер / Дисплей', 'Printer': 'Принтер', 'PrintQueue': 'Оче
    | редь печати', 'Bluetooth': 'Bluetooth Адаптер / Устройство', '
    | Digitizer': 'Wacom / Графический планшет', 'Media': 'Аудио / В

```



```
идео / Микрофон', 'Camera': 'Веб-камера', 'Net': 'Сетевой адаптер']
```

58

```
59     def __init__(self, poll_interval_sec: float=1.0, flapping_
window_sec: float=60.0, flapping_threshold: int=3, storage: Op
tional[TelemetryStorage]=None, on_event_callback: Optional[Cal
lable[[DeviceTransitionEvent], None]]=None, json_logger: Any=None) -> None:
```

```
60         """Инициализирует сенсор телеметрии устройств.
```

61

```
62         Args:
```

```
63             poll_interval_sec: Интервал между опросами оборудо
вания в секундах.
```

```
64             flapping_window_sec: Временное окно для фиксации д
ребезга (сек).
```

```
65             flapping_threshold: Количество отключений за окно
для вызова тревоги FLAPPING_ALERT.
```

```
66             storage: Хранилище SQLite (по умолчанию использует
ся синглтон).
```

```
67             on_event_callback: Опциональный callback-обработчи
к для передачи событий в UI/RAG.
```

```
68             json_logger: Опциональный логгер JSONL для обратно
й совместимости и тестирования.
```

```
69         """
```

```
70         self.poll_interval = max(0.1, poll_interval_sec)
```

```
71         self.flapping_window = flapping_window_sec
```

```
72         self.flapping_threshold = flapping_threshold
```

```
73         self.on_event_callback = on_event_callback
```

```
74         self.json_logger = json_logger
```

```
75         self.storage = storage or TelemetryStorage.get_instanc
```

```
e()
```

```
76         self._setupapi = SetupAPI() if os.name == 'nt' else No
ne
```

```
77         self._is_running = False
```

```
78         self._thread: Optional[threading.Thread] = None
```

```
79         self._lock = threading.Lock()
```

```
80         self._active_devices: Dict[str, PnPDeviceInfo] = {}
```

```
81         self._connected_since: Dict[str, float] = {}
```

```

82         self._disconnect_history: Dict[str, List[float]] = {}
83
84     def _resolve_category(self, device_class: str, friendly_name: str, instance_id: str) -> str:
85         """Определяет человекочитаемую категорию устройства.
86
87         Args:
88             device_class: PnP-класс из реестра SetupAPI.
89             friendly_name: Имя устройства.
90             instance_id: Идентификатор инстанса (VID/PID).
91
92         Returns:
93             str: Название категории.
94         """
95         combined = f'{device_class} {friendly_name} {instance_id}'.lower()
96         if 'wacom' in combined or 'digitizer' in combined or 'pen' in combined:
97             return 'Wacom / Графический планшет'
98         if 'hid' in combined and 'keyboard' in combined:
99             return 'Клавиатура'
100        if 'hid' in combined and 'mouse' in combined:
101            return 'Мышь / Указатель'
102        if 'bth' in combined or 'bluetooth' in combined:
103            return 'Bluetooth'
104        if 'monitor' in combined or 'display' in combined:
105            return 'HDMI / DP / Монитор'
106        if 'disk' in combined or 'usbstor' in combined:
107            return 'Дисконкей / Накопитель'
108        if 'print' in combined:
109            return 'Принтер'
110        return self.CATEGORY_MAPPING.get(device_class, device_class or 'Прочее оборудование')
111
112    def _get_current_devices_snapshot(self) -> Dict[str, PnPDeviceInfo]:
113        """Получает текущий снимок активных PnP-устройств.
114

```

```

115 |         Returns:
116 |             Dict[str, PnPDeviceInfo]: Словарь активных устройств по их Device Instance ID.
117 |             """
118 |         if self._setupapi is None:
119 |             return {}
120 |         try:
121 |             devices = self._setupapi.get_all_devices(only_present=True)
122 |             return {dev.device_instance_id: dev for dev in devices if dev.device_instance_id}
123 |         except Exception as e:
124 |             logger.error(f'Ошибка опроса SetupAPI в сенсоре устройств: {e}')
125 |             return {}
126 |
127 |     def _clean_flapping_history(self, now: float) -> None:
128 |         """Очищает устаревшие временные метки за пределами окна дребезга."""
129 |         cutoff = now - self.flapping_window
130 |         for dev_id in list(self._disconnect_history.keys()):
131 |             self._disconnect_history[dev_id] = [ts for ts in self._disconnect_history[dev_id] if ts >= cutoff]
132 |             if not self._disconnect_history[dev_id]:
133 |                 del self._disconnect_history[dev_id]
134 |
135 |     def _emit_event(self, event: DeviceTransitionEvent) -> None:
136 |         """Логирует событие телеметрии и оповещает внешние подписчики.
137 |
138 |         Args:
139 |             event: Экземпляр события изменения состояния устройства.
140 |         """
141 |         try:
142 |             self.storage.save_device_event(event.to_dict())
143 |         except Exception as ex:

```

```

144         logger.error(f'Ошибка сохранения device_event в БД
: {ex}')
145     if self.json_logger:
146         try:
147             if hasattr(self.json_logger, 'log_event'):
148                 self.json_logger.log_event(event.to_dict()
)
149             elif hasattr(self.json_logger, 'log'):
150                 self.json_logger.log(event.to_dict())
151         except Exception as ex:
152             logger.error(f'Ошибка сохранения device_event
в json_logger: {ex}')
153         log_msg = f"[{event.event_type}] [{event.category}] '{
event.friendly_name}' (ID: {event.device_instance_id}) | Ошибк
а: Code {event.problem_code} | Аптайм: {event.uptime_seconds:.
1f}s | Дребезгов: {event.flapping_count_in_window}"
154         if event.event_type in ('DISCONNECTED', 'FLAPPING_ALER
T') or event.has_problem:
155             logger.warning(log_msg)
156         else:
157             logger.info(log_msg)
158         if self.on_event_callback:
159             try:
160                 self.on_event_callback(event)
161             except Exception as ex:
162                 logger.error(f'Ошибка в on_event_callback: {ex
}')
163
164     def poll_once(self) -> List[DeviceTransitionEvent]:
165         """Выполняет один цикл опроса и детекции разницы состо
яний.
166
167         Returns:
168             List[DeviceTransitionEvent]: Список зафиксированны
х событий за текущий такт.
169         """
170         now = time.time()
171         iso_time = datetime.now(timezone.utc).isoformat()

```

```

172         events: List[DeviceTransitionEvent] = []
173         current_snapshot = self._get_current_devices_snapshot(
174     )
175         with self._lock:
176             self._clean_flapping_history(now)
177             if not self._active_devices and current_snapshot:
178                 self._active_devices = current_snapshot
179                 for dev_id in current_snapshot:
180                     self._connected_since[dev_id] = now
181                 logger.info(f'Сенсор инициализирован: обнаруже
но {len(current_snapshot)} активных устройств')
182                 return events
183                 current_ids = set(current_snapshot.keys())
184                 previous_ids = set(self._active_devices.keys())
185                 disconnected_ids = previous_ids - current_ids
186                 for dev_id in disconnected_ids:
187                     prev_dev = self._active_devices.pop(dev_id)
188                     connect_time = self._connected_since.pop(dev_i
d, now)
189                     uptime = max(0.0, now - connect_time)
190                     if dev_id not in self._disconnect_history:
191                         self._disconnect_history[dev_id] = []
192                         self._disconnect_history[dev_id].append(now)
193                         flapping_count = len(self._disconnect_history[
dev_id])
194                         category = self._resolve_category(prev_dev.dev
ice_class, prev_dev.friendly_name, dev_id)
195                         evt = DeviceTransitionEvent(timestamp=iso_time
, event_type='DISCONNECTED', device_instance_id=dev_id, friend
ly_name=prev_dev.friendly_name or prev_dev.device_class or dev
_id, device_class=prev_dev.device_class, category=category, ha
s_problem=prev_dev.has_problem, problem_code=prev_dev.problem_
code, status_code=prev_dev.status_code, manufacturer=prev_dev.
manufacturer, flapping_count_in_window=flapping_count, uptime_
seconds=uptime)
196                         events.append(evt)
197                         self._emit_event(evt)
198                         if flapping_count >= self.flapping_threshold:

```

```

198         flap_evt = DeviceTransitionEvent(timestamp
=iso_time, event_type='FLAPPING_ALERT', device_instance_id=dev
_id, friendly_name=prev_dev.friendly_name, device_class=prev_d
ev.device_class, category=category, has_problem=True, problem_
code=prev_dev.problem_code, status_code=prev_dev.status_code,
manufacturer=prev_dev.manufacturer, flapping_count_in_window=f
lapping_count, uptime_seconds=uptime)
199         events.append(flap_evt)
200         self._emit_event(flap_evt)
201         connected_ids = current_ids - previous_ids
202         for dev_id in connected_ids:
203             new_dev = current_snapshot[dev_id]
204             self._active_devices[dev_id] = new_dev
205             self._connected_since[dev_id] = now
206             flapping_count = len(self._disconnect_history.
get(dev_id, []))
207             category = self._resolve_category(new_dev.devi
ce_class, new_dev.friendly_name, dev_id)
208             evt = DeviceTransitionEvent(timestamp=iso_time
, event_type='CONNECTED', device_instance_id=dev_id, friendly_
name=new_dev.friendly_name or new_dev.device_class or dev_id,
device_class=new_dev.device_class, category=category, has_prob
lem=new_dev.has_problem, problem_code=new_dev.problem_code, st
atus_code=new_dev.status_code, manufacturer=new_dev.manufactur
er, flapping_count_in_window=flapping_count, uptime_seconds=0.
0)
209             events.append(evt)
210             self._emit_event(evt)
211             common_ids = current_ids & previous_ids
212             for dev_id in common_ids:
213                 old_dev = self._active_devices[dev_id]
214                 new_dev = current_snapshot[dev_id]
215                 if old_dev.has_problem != new_dev.has_problem
or old_dev.problem_code != new_dev.problem_code:
216                     self._active_devices[dev_id] = new_dev
217                     connect_time = self._connected_since.get(d
ev_id, now)
218                     category = self._resolve_category(new_dev.

```

```

device_class, new_dev.friendly_name, dev_id)
219         evt = DeviceTransitionEvent(timestamp=iso_
time, event_type='ERROR_STATE_CHANGED', device_instance_id=dev
_id, friendly_name=new_dev.friendly_name, device_class=new_dev
.device_class, category=category, has_problem=new_dev.has_prob
lem, problem_code=new_dev.problem_code, status_code=new_dev.st
atus_code, manufacturer=new_dev.manufacturer, uptime_seconds=m
ax(0.0, now - connect_time))
220         events.append(evt)
221         self._emit_event(evt)
222     return events
223
224     def _loop(self) -> None:
225         """Фоновый цикл непрерывного опроса."""
226         logger.info(f'Запущен фоновый поток сенсора устройств
(интервал: {self.poll_interval}с, окно дребезга: {self.flappin
g_window}с, порог: {self.flapping_threshold})')
227         while self._is_running:
228             try:
229                 self.poll_once()
230             except Exception as e:
231                 logger.error(f'Необработанная ошибка в цикле о
проса сенсора: {e}')
232                 time.sleep(self.poll_interval)
233
234     def start(self) -> None:
235         """Запускает непрерывный мониторинг в фоновом потоке."
"""
236         with self._lock:
237             if self._is_running:
238                 logger.warning('Сенсор устройств уже запущен')
239                 return
240             self._is_running = True
241             self._thread = threading.Thread(target=self._loop,
name='DeviceFlappingSensorThread', daemon=True)
242             self._thread.start()
243
244     def stop(self) -> None:

```

```
245 |         """Останавливает непрерывный мониторинг."""
246 |     with self._lock:
247 |         if not self._is_running:
248 |             return
249 |         self._is_running = False
250 |     if self._thread and self._thread.is_alive():
251 |         self._thread.join(timeout=3.0)
252 |         logger.info('Сенсор устройств успешно остановлен')
253 |
254 |     def get_status(self) -> Dict[str, Any]:
255 |         """Возвращает текущую статистику сенсора.
256 |
257 |         Returns:
258 |             Dict[str, Any]: Данные о количестве активных устройс
259 |             """
260 |         with self._lock:
261 |             return {'is_running': self._is_running, 'poll_inte
262 |             rval_sec': self.poll_interval, 'active_devices_count': len(sel
263 |             f._active_devices), 'flapping_devices_tracked': len(self._disc
264 |             onnect_history), 'devices_with_problems': sum((1 for d in self
265 |             ._active_devices.values() if d.has_problem))}
```


file_collector.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - File Collector
4 # =====
5 # Description:
6 #   Сбор событий файловой системы через DirectoryWatcher.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.file_collector import FileCollector
11 #
12 #       service = FileCollector()
13 #
14 # File: file_collector.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-03 23:06:40
20 # =====
21
22 from __future__ import annotations
23 """Сбор событий файловой системы через DirectoryWatcher."""
24
25 import threading
26 from datetime import datetime, timezone
27 from pathlib import Path
28 from typing import Any, Dict, List, Optional
29 try:
30     from logger import logger
31 except ImportError:
```

```

32     from logger import logger
33
34
35 def _get_directory_watcher_classes():
36     """Ленивый импорт DirectoryWatcher для избежания циклическ
их зависимостей."""
37     try:
38         from apps.windows.sdk.modules.sysadmin.src.directory_w
atcher import DirectoryWatcher, LiveFileEvent
39         return DirectoryWatcher, LiveFileEvent
40     except ImportError:
41         try:
42             from apps.windows.sysadmin.src.directory_watcher i
mport DirectoryWatcher, LiveFileEvent
43             return DirectoryWatcher, LiveFileEvent
44         except ImportError as e:
45             logger.warning(f'Не удалось импортировать Director
yWatcher: {e}')
46             return None, None
47
48
49 class FileCollector:
50     """Коллектор событий файловой системы."""
51
52     def __init__(self, watch_dirs: Optional[List[str]]=None, m
ax_history: int=200) -> None:
53         """Инициализирует коллектор файловых событий.
54
55         Args:
56             watch_dirs: Список директорий для мониторинга.
57             max_history: Размер истории событий.
58         """
59         self.watch_dirs = watch_dirs or [str(Path.home() / 'Do
cuments')]
60         self.max_history = max_history
61         self._watchers: Dict[str, Any] = {}
62         self._lock = threading.Lock()
63         self._init_watchers()

```

```

64 |
65 |     def _init_watchers(self) -> None:
66 |         """Инициализирует DirectoryWatcher для каждой директор
ии."""
67 |         DirectoryWatcher, _ = _get_directory_watcher_classes()
68 |         if DirectoryWatcher is None:
69 |             logger.warning('DirectoryWatcher недоступен')
70 |             return
71 |         for watch_dir in self.watch_dirs:
72 |             try:
73 |                 watcher = DirectoryWatcher(watch_dirs=[watch_d
ir], max_history=self.max_history)
74 |                 if watcher.start():
75 |                     self._watchers[watch_dir] = watcher
76 |                     logger.info(f'DirectoryWatcher запущен для
: {watch_dir}')
77 |             except Exception as e:
78 |                 logger.error(f'Ошибка инициализации DirectoryW
atcher для {watch_dir}: {e}')
79 |
80 |     def _convert_event(self, event: LiveFileEvent) -> Dict[str
, Any]:
81 |         """Преобразует LiveFileEvent в словарь для JSON.
82 |
83 |         Args:
84 |             event: Событие от DirectoryWatcher.
85 |
86 |         Returns:
87 |             Dict[str, Any]: Событие в формате словаря.
88 |         """
89 |         return {'timestamp': event.timestamp, 'action': event.
action, 'path': event.path, 'is_deletion': event.is_deletion,
'details': event.details}
90 |
91 |     def get_recent_events(self, limit: int=100) -> List[Dict[s
tr, Any]]:
92 |         """Получает недавние файловые события.
93 |

```

```

94 |         Args:
95 |             limit: Максимальное количество событий.
96 |
97 |         Returns:
98 |             List[Dict[str, Any]]: Список файловых событий.
99 |         """
100 |         with self._lock:
101 |             events: List[Dict[str, Any]] = []
102 |             for watch_dir, watcher in self._watchers.items():
103 |                 try:
104 |                     raw_events = watcher.get_recent_events(lim
105 | it=limit)
106 |                     for event in raw_events:
107 |                         converted = self._convert_event(event)
108 |                         converted['watch_dir'] = watch_dir
109 |                         events.append(converted)
110 |                     except Exception as e:
111 |                         logger.error(f'Ошибка получения событий из
112 | {watch_dir}: {e}')
113 |                         events.sort(key=lambda x: x.get('timestamp', ''),
114 | reverse=True)
115 |                         return events[:limit]
116 |
117 |     def get_events_by_action(self, action: str, limit: int=50)
118 |     -> List[Dict[str, Any]]:
119 |         """Получает события определенного типа.
120 |
121 |         Args:
122 |             action: Тип действия (Created, Deleted, Modified).
123 |             limit: Максимальное количество событий.
124 |
125 |         Returns:
126 |             List[Dict[str, Any]]: Список событий.
127 |         """
128 |         all_events = self.get_recent_events(limit=limit * 2)
129 |         return [e for e in all_events if e.get('action') == ac
130 | tion][:limit]

```

```

127 |     def get_events_by_type(self, event_type: str, limit: int=5
    | 0) -> List[Dict[str, Any]]:
128 |         """Получает события определенного типа (Created/Delete
    | d).
129 |
130 |         Args:
131 |             event_type: Тип события (Created или Deleted).
132 |             limit: Максимальное количество событий.
133 |
134 |         Returns:
135 |             List[Dict[str, Any]]: Список событий.
136 |         """
137 |         if event_type.lower() == 'created':
138 |             return self.get_events_by_action('Created', limit)
139 |         elif event_type.lower() == 'deleted':
140 |             return self.get_events_by_action('Deleted', limit)
141 |         return []
142 |
143 |     def stop(self) -> None:
144 |         """Останавливает все DirectoryWatcher."""
145 |         with self._lock:
146 |             for watcher in self._watchers.values():
147 |                 watcher.stop()
148 |             self._watchers.clear()
149 |             logger.info('FileCollector остановлен')
150 |
151 |     def restart(self) -> bool:
152 |         """Перезапускает DirectoryWatcher.
153 |
154 |         Returns:
155 |             bool: True если перезапуск успешен.
156 |         """
157 |         self.stop()
158 |         self._init_watchers()
159 |         return True
160 |
161 |     def add_watch_dir(self, watch_dir: str) -> bool:
162 |         """Добавляет директорию для мониторинга.

```

```

163
164     Args:
165         watch_dir: Путь к директории.
166
167     Returns:
168         bool: True если директория успешно добавлена.
169     """
170     DirectoryWatcher, _ = _get_directory_watcher_classes()
171     if DirectoryWatcher is None:
172         logger.warning('DirectoryWatcher недоступен')
173         return False
174     if watch_dir in self._watchers:
175         logger.warning(f'Директория уже отслеживается: {wa
176 tch_dir}')
177         return False
178     try:
179         watcher = DirectoryWatcher(watch_dirs=[watch_dir],
180 max_history=self.max_history)
181         if watcher.start():
182             self._watchers[watch_dir] = watcher
183             logger.info(f'Добавлена директория для мониторинга: {watch_dir}')
184             return True
185         except Exception as e:
186             logger.error(f'Ошибка добавления директории {watch
187 _dir}: {e}')
188             return False
189
190     def remove_watch_dir(self, watch_dir: str) -> bool:
191         """Удаляет директорию из мониторинга.
192
193     Args:
194         watch_dir: Путь к директории.
195
196     Returns:
197         bool: True если директория успешно удалена.
198     """
199     with self._lock:

```

```
197 |         if watch_dir in self._watchers:
198 |             watcher = self._watchers.pop(watch_dir)
199 |             watcher.stop()
200 |             logger.info(f'Удалена директория из мониторинг
a: {watch_dir}')
201 |             return True
202 |         else:
203 |             logger.warning(f'Директория не найдена в монит
оринге: {watch_dir}')
204 |             return False
```

internet_speed.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - Internet Speed
4 # =====
5 # Description:
6 #   Сенсор телеметрии скорости интернета и задержки сети на базе FAST.com (Netflix CDN).
7 #   Выполняет периодический замер (1 раз в час / 3600 сек) и кэширует результаты.
8 #
9 # Usage Examples:
10 #   Python API:
11 #       from apps.windows.telemetry.internet_speed import InternetSpeedSensor
12 #
13 #       sensor = InternetSpeedSensor()
14 #       metrics = sensor.measure_internet_speed()
15 #       print(metrics['download_mbps'], metrics['bufferbloat_ms'])
16 #
17 # File: internet_speed.py
18 # Project: ai-breadboard
19 # Package: apps.windows.telemetry
20 # Author: hypo69
21 # Copyright: © 2026 hypo69
22 # Updated: 2026-10-04 07:35:00
23 # =====
24
25 from __future__ import annotations
26 """Сенсор телеметрии скорости интернета и задержки сети на базе FAST.com."""
27
```



```

28 import asyncio
29 import threading
30 import time
31 from typing import Any, Dict, List, Optional
32
33 from logger import logger
34 from apps.windows.sdk.modules.network.speedtest import Network
    SpeedTester
35
36 _speed_cache: Optional[tuple[float, Dict[str, Any]]] = None
37 _CACHE_TTL = 3600 # 1 час между замерами скорости для снижени
    я нагрузки на сеть
38 _cache_lock = threading.Lock()
39 _bg_test_running = False
40 _bg_lock = threading.Lock()
41
42
43 class InternetSpeedSensor:
44     """Сенсор измерения скорости интернета и характеристик зад
    ержки (FAST.com)."""
45
46     def __init__(self, cache_ttl: int = _CACHE_TTL) -> None:
47         """Инициализация сенсора скорости интернета.
48
49         Args:
50             cache_ttl: Время жизни кеша замера в секундах (по
    умолчанию 3600 сек = 1 час).
51         """
52         self.cache_ttl = cache_ttl
53         self._speedtester = NetworkSpeedTester()
54
55     @classmethod
56     def clear_cache(cls) -> None:
57         """Очистить глобальный кеш замеров скорости интернета.
58         """
59
60         global _speed_cache
61         with _cache_lock:
62             _speed_cache = None

```

```

61 |
62 |     def _run_speedtest_sync(self) -> Dict[str, Any]:
63 |         """Синхронно выполнить полный цикл теста скорости FAST
64 |         .com."""
65 |         try:
66 |             loop = None
67 |             try:
68 |                 loop = asyncio.get_event_loop()
69 |             except RuntimeError:
70 |                 loop = None
71 |
72 |             if loop and loop.is_running():
73 |                 # Если вызывается внутри уже работающего цикла
74 |                 событий (например, в async worker)
75 |                 import concurrent.futures
76 |                 with concurrent.futures.ThreadPoolExecutor(max
77 |                 _workers=1) as executor:
78 |                     future = executor.submit(lambda: asyncio.r
79 |                     un(self._speedtester.run_full_speedtest()))
80 |                     return future.result(timeout=60.0)
81 |             else:
82 |                 return asyncio.run(self._speedtester.run_full_
83 |                 speedtest())
84 |         except Exception as ex:
85 |             logger.error(f'Ошибка выполнения теста скорости FA
86 |             ST.com в сенсоре телеметрии: {ex}')
87 |             return {
88 |                 'download': {'speed_mbps': 0.0, 'status': 'ERR
89 |                 OR'},
90 |                 'upload': {'speed_mbps': 0.0, 'status': 'ERROR
91 |                 '},
92 |                 'ping_ms': 0.0,
93 |                 'latency_unloaded_ms': 0.0,
94 |                 'latency_loaded_ms': 0.0,
95 |                 'bufferbloat_ms': 0.0,
96 |                 'meta': {'ip': 'N/A', 'isp': 'Unknown'},
97 |                 'provider': 'fast.com',
98 |                 'server': 'N/A'

```

```

91         }
92
93     def _perform_measurement_and_cache(self) -> Dict[str, Any]
94     :
95         """Выполняет замер скорости и сохраняет его в кэш."""
96         global _speed_cache, _bg_test_running
97         try:
98             report = self._run_speedtest_sync()
99             download_mbps = float(report.get('download', {}).g
100 et('speed_mbps', 0.0))
101             upload_mbps = float(report.get('upload', {}).get('
102 speed_mbps', 0.0))
103             ping_ms = float(report.get('ping_ms', report.get('
104 latency_unloaded_ms', 0.0)))
105             latency_loaded_ms = float(report.get('latency_load
106 ed_ms', 0.0))
107             bufferbloat_ms = float(report.get('bufferbloat_ms'
108 , 0.0))
109             meta = report.get('meta', {})
110
111             result: Dict[str, Any] = {
112                 'available': report.get('download', {}).get('s
113 tatus') == 'SUCCESS' or download_mbps > 0.0,
114                 'download_mbps': download_mbps,
115                 'upload_mbps': upload_mbps,
116                 'ping_ms': ping_ms,
117                 'latency_unloaded_ms': float(report.get('laten
118 cy_unloaded_ms', ping_ms)),
119                 'latency_loaded_ms': latency_loaded_ms,
120                 'bufferbloat_ms': bufferbloat_ms,
121                 'provider': 'fast.com',
122                 'server': report.get('server', 'FAST.com CDN')
123             },
124             'ip': meta.get('ip', 'N/A'),
125             'isp': meta.get('isp', 'Unknown ISP'),
126             'measured_at': time.time(),
127         }
128     }
129

```

```

120         with _cache_lock:
121             _speed_cache = (time.monotonic(), result)
122
123         logger.info(
124             f' [Телеметрия FAST.com] Скорость: ↓{download_
125             d_mbps:.1f} Mbps, '
126             f'↑{upload_mbps:.1f} Mbps, пинг: {ping_ms:.1f}
127             mc, bufferbloat: {bufferbloat_ms:.1f} mc'
128         )
129         return dict(result)
130     finally:
131         with _bg_lock:
132             _bg_test_running = False
133
134     def trigger_async_measure(self) -> None:
135         """Запускает измерение скорости в фоновом потоке, если
136         оно еще не выполняется."""
137         global _bg_test_running
138         with _bg_lock:
139             if _bg_test_running:
140                 return
141             _bg_test_running = True
142
143         thread = threading.Thread(
144             target=self._perform_measurement_and_cache,
145             name="InternetSpeedTestWorker",
146             daemon=True
147         )
148         thread.start()
149
150     def measure_internet_speed(self, force_refresh: bool = False,
151                               sync: bool = False) -> Dict[str, Any]:
152         """Измеряет профиль интернет-соединения через FAST.com
153         с кешированием на 1 час.
154
155         По умолчанию выполняется асинхронно в фоне и не блокир
156         ует вызывающий поток.

```

```

152 |         Args:
153 |             force_refresh: Принудительно инициировать новый за
мер.
154 |             sync: Выполнить замер синхронно с блокировкой теку
щего потока.
155 |
156 |         Returns:
157 |             Dict[str, Any]: Словарь метрик (download_mbps, upl
oad_mbps, ping_ms, bufferbloat_ms, etc.).
158 |             """
159 |             global _speed_cache
160 |             with _cache_lock:
161 |                 if not force_refresh and _speed_cache is not None:
162 |                     cached_at, cached_result = _speed_cache
163 |                     if time.monotonic() - cached_at < self.cache_t
tl:
164 |                         return dict(cached_result)
165 |
166 |             if sync:
167 |                 return self._perform_measurement_and_cache()
168 |
169 |             # Асинхронный запуск замера в фоне
170 |             self.trigger_async_measure()
171 |
172 |             with _cache_lock:
173 |                 if _speed_cache is not None:
174 |                     _, cached_result = _speed_cache
175 |                     return dict(cached_result)
176 |
177 |             # Возвращаем заглушку по умолчанию без блокировки пото
ка
178 |             return {
179 |                 'available': True,
180 |                 'download_mbps': 0.0,
181 |                 'upload_mbps': 0.0,
182 |                 'ping_ms': 0.0,
183 |                 'latency_unloaded_ms': 0.0,
184 |                 'latency_loaded_ms': 0.0,

```

```

185         'bufferbloat_ms': 0.0,
186         'provider': 'fast.com',
187         'server': 'FAST.com CDN (замер в фоне...)',
188         'ip': 'N/A',
189         'isp': 'Unknown ISP',
190         'measured_at': 0.0,
191     }
192
193
194 def get_internet_speed_sensors() -> List[Dict[str, Any]]:
195     """Получить показания сетевых метрик скорости в формате сп
иска сенсоров.
196
197     Returns:
198         List[Dict[str, Any]]: Список метрик сенсоров интернета
199     """
200     sensor = InternetSpeedSensor()
201     metrics = sensor.measure_internet_speed()
202     sensors = [
203         {
204             'sensor_id': 'internet_ping',
205             'name': 'Internet Ping Latency',
206             'category': 'network',
207             'value': metrics.get('ping_ms', 0.0),
208             'unit': 'ms'
209         },
210         {
211             'sensor_id': 'internet_download',
212             'name': 'Internet Download Speed',
213             'category': 'network',
214             'value': metrics.get('download_mbps', 0.0),
215             'unit': 'Mbps'
216         },
217         {
218             'sensor_id': 'internet_upload',
219             'name': 'Internet Upload Speed',
220             'category': 'network',

```

```
221         'value': metrics.get('upload_mbps', 0.0),
222         'unit': 'Mbps'
223     },
224     {
225         'sensor_id': 'internet_bufferbloat',
226         'name': 'Internet Bufferbloat',
227         'category': 'network',
228         'value': metrics.get('bufferbloat_ms', 0.0),
229         'unit': 'ms'
230     },
231     {
232         'sensor_id': 'internet_latency_loaded',
233         'name': 'Internet Loaded Latency',
234         'category': 'network',
235         'value': metrics.get('latency_loaded_ms', 0.0),
236         'unit': 'ms'
237     },
238 ]
239 return sensors
```

power_lifecycle.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - Power L
  ifecycle
4 # =====
5 # Description:
6 #   Движок сбора, разбора и корреляции событий жизненного цикл
  а питания Windows
7 #   (Boot, Clean Shutdown, Planned Restart, Unexpected Power L
  oss, BSOD, Sleep/Wake)
8 #   и автоматической реконструкции сессий питания (Power Sessi
  ons).
9 #
10 # Usage Examples:
11 #   Python API:
12 #       from apps.windows.telemetry.power_lifecycle import Power
  LifecycleEngine
13 #
14 #       engine = PowerLifecycleEngine()
15 #       summary = engine.scan_and_reconstruct(hours=720)
16 #       sessions = engine.get_sessions(limit=20)
17 #
18 # File: power_lifecycle.py
19 # Project: ai-breadboard
20 # Package: apps.windows.telemetry
21 # Author: hypo69
22 # Copyright: © 2026 hypo69
23 # Updated: 2026-10-08 02:51:00
24 # =====
25
26 from __future__ import annotations
27 """Движок анализа событий питания и реконструкции сессий жизне
  нного цикла Windows."""
```



```
28 |
29 | import json
30 | import os
31 | import re
32 | import socket
33 | import subprocess
34 | import time
35 | from datetime import datetime, timezone, timedelta
36 | from typing import Any, Dict, List, Optional, Tuple
37 |
38 | try:
39 |     from logger import logger
40 | except ImportError:
41 |     import logging
42 |     logger = logging.getLogger(__name__)
43 |
44 | from apps.windows.telemetry.win32_fffi.wevtapi import WevtAPI
45 | from apps.windows.telemetry.win32_fffi.error_decoder import decode_bugcheck_code, format_system_error
46 | from apps.windows.telemetry.models import (
47 |     PowerEventRecord,
48 |     PowerSessionRecord,
49 |     PowerLifecycleSummary,
50 | )
51 | from apps.windows.telemetry.sqlite import TelemetryStorage
52 |
53 |
54 | # Целевые Event ID для жизненного цикла питания и выключений
55 | POWER_LIFECYCLE_EVENT_IDS = {
56 |     1074: ('User32', 'Плановое выключение / перезагрузка иници
ирована пользователем или процессом'),
57 |     13: ('Microsoft-Windows-Kernel-General', 'Операционная сис
тема начала завершение работы'),
58 |     6006: ('EventLog', 'Служба Event Log корректно остановлена
(штатное чистое выключение)'),
59 |     6008: ('EventLog', 'Предыдущее завершение работы системы б
ыло неожиданным (Unexpected Shutdown)'),
60 |     41: ('Microsoft-Windows-Kernel-Power', 'КРИТИЧЕСКИЙ СБОЙ:
```

```

Перезагрузка без предварительного чистого выключения'),
61     12: ('Microsoft-Windows-Kernel-General', 'Операционная сис
тема запущена (Boot)'),
62     6005: ('EventLog', 'Служба Event Log запущена (ОС готова к
работе)'),
63     6009: ('EventLog', 'Windows записала информацию о версии я
дра при загрузке'),
64     1001: ('WER-SystemErrorReporting', 'Аварийный сбой ядра /
BSOD (BugCheck)'),
65     19: ('Microsoft-Windows-WindowsUpdateClient', 'Успешно уст
ановлено обновление Windows Update'),
66     42: ('Microsoft-Windows-Kernel-Power', 'Переход системы в
спящий режим (Sleep)'),
67     107: ('Microsoft-Windows-Kernel-Power', 'Пробуждение систе
мы из спящего режима (Resume/Wake)'),
68 }
69
70
71 def format_uptime_human(seconds: float) -> str:
72     """Форматирует секунды аптайма в человекочитаемую строку н
а русском языке.
73
74     Args:
75         seconds: Длительность в секундах.
76
77     Returns:
78         str: Строка вида '3д 14ч 35м 31с' или '14ч 35м 31с'.
79     """
80     total_sec = max(0, int(seconds))
81     days = total_sec // 86400
82     hours = (total_sec % 86400) // 3600
83     minutes = (total_sec % 3600) // 60
84     sec = total_sec % 60
85
86     parts = []
87     if days > 0:
88         parts.append(f"{days}д")
89     if hours > 0 or days > 0:

```

```

90         parts.append(f"{hours}ч")
91     if minutes > 0 or hours > 0 or days > 0:
92         parts.append(f"{minutes}м")
93     parts.append(f"{sec}с")
94
95     return ' '.join(parts)
96
97
98 class PowerLifecycleEngine:
99     """Движок глубокого анализа и реконструкции жизненного цик
100 ла питания ОС."""
101
102     def __init__(
103         self,
104         storage: Optional[TelemetryStorage] = None,
105         wevtapi: Optional[WevtAPI] = None,
106     ) -> None:
107         """Инициализация движка жизненного цикла питания.
108
109         Args:
110             storage: Экземпляр TelemetryStorage SQLite хранили
111 ща.
112             wevtapi: Низкоуровневая Ctypes-обертка WevtAPI.
113         """
114         self.storage = storage or TelemetryStorage.get_instanc
115 e()
116         self.wevtapi = wevtapi or WevtAPI()
117
118     def get_current_system_boot(self) -> Tuple[datetime, float
119 ]:
120         """Определяет точное время последней загрузки и текущи
121 й аптайм.
122
123         Returns:
124             Tuple[datetime, float]: (boot_time UTC, uptime в с
125 екундах).
126         """
127         try:

```

```

122         import psutil
123         b_epoch = psutil.boot_time()
124         boot_dt = datetime.fromtimestamp(b_epoch, tz=timezone.
one.utc)
125         uptime_sec = max(0.0, time.time() - b_epoch)
126         return boot_dt, uptime_sec
127     except Exception:
128         boot_dt = datetime.now(timezone.utc)
129         return boot_dt, 0.0
130
131     def parse_event_1074(self, ev: Dict[str, Any]) -> Dict[str
, Any]:
132         """Глубокий разбор события User32 1074 (инициатор, про
цесс, тип, причина).
133
134         Args:
135             ev: Словарь события из WeventAPI.
136
137         Returns:
138             Dict[str, Any]: Извлеченные нормализованные параме
тры.
139         """
140         msg = ev.get('message', '') or ev.get('formatted_messa
ge', '')
141         data = ev.get('event_data', {}) or {}
142
143         # 1. Попытка извлечения из XML Data (param1 ... param7
)
144         process = data.get('param1') or data.get('Process') or
''
145         computer = data.get('param2') or data.get('Computer')
or ev.get('computer', '')
146         reason = data.get('param3') or data.get('Reason') or '
'
147         reason_code = data.get('param4') or data.get('ReasonCo
de') or ''
148         shutdown_type = data.get('param5') or data.get('Shutdo
wnType') or ''

```

```

149 |         comment = data.get('param6') or data.get('Comment') or
150 |         ''
151 |         user = data.get('param7') or data.get('User') or ''
152 |         # 2. Если параметры пустые, применяем регулярные выраж
ения к сообщению
153 |         if not process and msg:
154 |             # Process regex
155 |             m_proc = re.search(r'(:process|процесс|процессом)
\s+([\s]+)', msg, re.IGNORECASE)
156 |             if m_proc:
157 |                 process = m_proc.group(1).rstrip('.,;:')
158 |
159 |             # User regex
160 |             m_user = re.search(r'(:user|пользователя|пользова
тель|имени)\s+([\s]+)', msg, re.IGNORECASE)
161 |             if m_user:
162 |                 user = m_user.group(1).rstrip('.,;:')
163 |
164 |             # Reason Code regex
165 |             m_code = re.search(r'(:Reason Code|код причины|ко
дом причины)[:\s]+(0x[0-9a-fA-F]+)', msg, re.IGNORECASE)
166 |             if m_code:
167 |                 reason_code = m_code.group(1)
168 |
169 |             # Shutdown Type regex
170 |             m_type = re.search(r'(:Shutdown Type|тип выключен
ия|тип завершения)[:\s]+([a-zA-Za-яA-Я0-9_-]+)', msg, re.IGNOR
ECASE)
171 |             if m_type:
172 |                 shutdown_type = m_type.group(1)
173 |
174 |             # Reason text regex
175 |             m_reason = re.search(r'(:reason|причине|причина)[
:\s]+(.*?)(:Reason Code|код причины|$)', msg, re.IGNORECASE)
176 |             if m_reason:
177 |                 reason = m_reason.group(1).strip()
178 |

```

```

179         # Comment regex
180         m_comment = re.search(r'(?::comment|комментарий)[:\s]+(.*?)$', msg, re.IGNORECASE)
181         if m_comment:
182             comment = m_comment.group(1).strip()
183
184         # Нормализация типа выключения
185         norm_type = 'Shutdown'
186         st_lower = str(shutdown_type).lower()
187         if 'restart' in st_lower or 'перезагр' in st_lower:
188             norm_type = 'Restart'
189         elif 'power off' in st_lower or 'выключ' in st_lower or 'poweroff' in st_lower:
190             norm_type = 'PowerOff'
191         elif 'hybrid' in st_lower or 'fast' in st_lower:
192             norm_type = 'HybridShutdown'
193
194         domain = ''
195         if '\\' in user:
196             domain, user_name = user.split('\\', 1)
197         else:
198             user_name = user
199
200         # Корреляция цепочки инициатора
201         initiator_chain: List[str] = []
202         proc_lower = process.lower()
203         if 'wmiprvse' in proc_lower:
204             initiator_chain = [
205                 f"Пользователь/Скрипт ({user or 'WMI Client'})",
206                 "WMI Remote/Local Call",
207                 "wmiprvse.exe",
208                 "Windows ExitWindowsEx API",
209             ]
210         elif 'shutdown.exe' in proc_lower:
211             initiator_chain = [
212                 f"Пользователь ({user or 'Console'})",
213                 "Командная строка / Task Scheduler",

```

```

214         "shutdown.exe",
215         "InitiateSystemShutdownEx API",
216     ]
217     elif 'explorer.exe' in proc_lower:
218         initiator_chain = [
219             f"Пользователь ({user or 'Interactive'})",
220             "Меню Пуск / Win+X / Alt+F4",
221             "explorer.exe",
222             "Shell Shutdown API",
223         ]
224     elif 'svchost.exe' in proc_lower:
225         initiator_chain = [
226             "Служба обновления Windows / System Service",
227             "svchost.exe",
228             "Automatic Maintenance Restart",
229         ]
230     elif process:
231         initiator_chain = [
232             f"Пользователь ({user or 'User'})",
233             f"Процесс ({process})",
234             "Shutdown API",
235         ]
236
237     return {
238         'process': process,
239         'computer': computer,
240         'reason': reason or 'Плановое завершение работы',
241         'reason_code': reason_code,
242         'shutdown_type': norm_type,
243         'comment': comment,
244         'user': user,
245         'user_name': user_name,
246         'domain': domain,
247         'initiator_chain': initiator_chain,
248     }
249
250     def read_raw_power_events(self, hours: int = 720, limit: i
nt = 500) -> List[Dict[str, Any]]:

```

```

251 |         """Извлекает сырые события питания из журнала System W
      | indows.
252 |
253 |     Args:
254 |         hours: Глубина выборки в часах.
255 |         limit: Максимальное количество событий каждого тип
      | а.
256 |
257 |     Returns:
258 |         List[Dict[str, Any]]: Список событий, упорядоченны
      | х хронологически.
259 |         """
260 |         target_ids = list(POWER_LIFECYCLE_EVENT_IDS.keys())
261 |         raw_events = self.wevtapi.read_events(
262 |             channel='System',
263 |             limit=limit,
264 |             hours=hours,
265 |             event_id=target_ids,
266 |             format_message=True,
267 |         )
268 |
269 |         normalized: List[Dict[str, Any]] = []
270 |         for ev in raw_events:
271 |             eid = ev.get('event_id', 0)
272 |             if eid not in POWER_LIFECYCLE_EVENT_IDS:
273 |                 continue
274 |
275 |             provider_info = POWER_LIFECYCLE_EVENT_IDS.get(eid,
      | ('System', ''))
276 |             provider = ev.get('provider') or provider_info[0]
277 |             ts = ev.get('timestamp') or ''
278 |             raw_xml = ev.get('raw_data') or ''
279 |             msg = ev.get('message') or ev.get('formatted_messa
      | ge') or provider_info[1]
280 |
281 |             event_type = 'event'
282 |             shutdown_type = None
283 |             user = None

```



```

284 |         domain = None
285 |         process = None
286 |         pid = ev.get('process_id')
287 |         reason = None
288 |         reason_code = None
289 |         comment = None
290 |         unexpected = False
291 |         bugcheck_code = None
292 |         bugcheck_params = None
293 |         details: Dict[str, Any] = {'message': msg}
294 |
295 |         if eid == 1074:
296 |             parsed_1074 = self.parse_event_1074(ev)
297 |             event_type = 'shutdown_planned'
298 |             shutdown_type = parsed_1074['shutdown_type']
299 |             user = parsed_1074['user']
300 |             domain = parsed_1074['domain']
301 |             process = parsed_1074['process']
302 |             reason = parsed_1074['reason']
303 |             reason_code = parsed_1074['reason_code']
304 |             comment = parsed_1074['comment']
305 |             details['initiator_chain'] = parsed_1074['init
iator_chain']
306 |             details['computer'] = parsed_1074['computer']
307 |
308 |         elif eid == 13:
309 |             event_type = 'shutdown_clean'
310 |             shutdown_type = 'Shutdown'
311 |             reason = 'Операционная система завершает работ
у ядра'
312 |
313 |         elif eid == 6006:
314 |             event_type = 'shutdown_clean'
315 |             shutdown_type = 'Shutdown'
316 |             reason = 'Служба Event Log корректно остано
влена'
317 |
318 |         elif eid in (41, 6008):

```

```

319 |         event_type = 'shutdown_unexpected'
320 |         unexpected = True
321 |         shutdown_type = 'Unexpected'
322 |         reason = 'Внезапное завершение работы (потеря
питания / зависание / сбой)'
323 |         if eid == 41:
324 |             ev_data = ev.get('event_data', {})
325 |             bc = ev_data.get('BugcheckCode')
326 |             if bc and str(bc) != '0':
327 |                 try:
328 |                     bc_int = int(bc)
329 |                     bugcheck_code = hex(bc_int)
330 |                     decoded = decode_bugcheck_code(bc_
int)
331 |                     details['bugcheck_info'] = decoded
332 |                     details['bugcheck_symbol'] = decod
ed.get('symbol', '')
333 |                 except Exception:
334 |                     bugcheck_code = str(bc)
335 |                     details['power_button_timestamp'] = ev_dat
a.get('PowerButtonTimestamp', '0')
336 |
337 |             elif eid in (12, 6005, 6009):
338 |                 event_type = 'boot'
339 |                 reason = 'Запуск операционной системы и систем
ных служб'
340 |
341 |             elif eid == 1001:
342 |                 event_type = 'bsod'
343 |                 reason = 'Критический сбой операционной систем
ы (BugCheck / BSOD)'
344 |                 ev_data = ev.get('event_data', {})
345 |                 bugcheck_code = ev_data.get('param1') or ev_da
ta.get('BugcheckCode')
346 |                 details['bugcheck_params'] = [ev_data.get(f'pa
ram{i}') for i in range(1, 6)]
347 |
348 |             elif eid == 19:

```

```

349         event_type = 'update_install'
350         reason = 'Успешно установлено обновление Windo
ws Update'
351
352     elif eid == 42:
353         event_type = 'sleep'
354         reason = 'Переход системы в спящий режим'
355
356     elif eid == 107:
357         event_type = 'wake'
358         reason = 'Выход системы из спящего режима'
359
360     record = {
361         'event_id': eid,
362         'provider': provider,
363         'channel': ev.get('channel', 'System'),
364         'timestamp': ts,
365         'created_at': self._parse_iso_to_epoch(ts),
366         'event_type': event_type,
367         'shutdown_type': shutdown_type,
368         'user': user,
369         'domain': domain,
370         'process': process,
371         'process_id': pid,
372         'reason': reason,
373         'reason_code': reason_code,
374         'comment': comment,
375         'unexpected': unexpected,
376         'bugcheck_code': bugcheck_code,
377         'bugcheck_params_json': json.dumps(bugcheck_pa
rams, ensure_ascii=False) if bugcheck_params else None,
378         'details_json': json.dumps(details, ensure_asc
ii=False),
379         'raw_xml': raw_xml,
380     }
381     normalized.append(record)
382
383     # Сортировка от старых к новым для корректной реконстр

```

укции

```
384         normalized.sort(key=lambda x: x['created_at'])
385         return normalized
386
387     def _parse_iso_to_epoch(self, ts_str: str) -> float:
388         """Преобразует строку даты/времени в unix epoch timestamp."""
389         if not ts_str:
390             return time.time()
391         for fmt in ('%Y-%m-%d %H:%M:%S', '%Y-%m-%dT%H:%M:%S',
392                    '%Y-%m-%d %H:%M:%S.%f', '%Y-%m-%dT%H:%M:%S.%fZ'):
393             try:
394                 dt = datetime.strptime(ts_str.split('.')[0].replace('T', ' '), '%Y-%m-%d %H:%M:%S')
395                 return dt.replace(tzinfo=timezone.utc).timestamp()
396             except Exception:
397                 pass
398         return time.time()
399
400     def reconstruct_power_sessions(self, events: List[Dict[str, Any]]) -> List[PowerSessionRecord]:
401         """Реконструирует цепочки Boot -> Power Session -> Shutdown/Crash.
402
403         Args:
404             events: Хронологически отсортированный список нормализованных событий.
405
406         Returns:
407             List[PowerSessionRecord]: Список собранных сессий питания.
408
409         """
410         sessions: List[PowerSessionRecord] = []
411         current_session: Optional[Dict[str, Any]] = None
412
413         for ev in events:
414             eid = ev.get('event_id', 0)
```

```

413         e_type = ev.get('event_type', '')
414         ts = ev.get('timestamp', '')
415         epoch = ev.get('created_at', 0.0)
416
417         # Событие запуска ОС (Boot)
418         if eid in (12, 6005, 6009) or e_type == 'boot':
419             # Если уже была открыта сессия без штатного за
420             # вершения, закрываем её как Unexpected
421             if current_session:
422                 s_id = current_session['session_id']
423                 uptime_sec = max(0.0, epoch - current_sess
424                 ion['boot_timestamp'])
425                 current_session['shutdown_time'] = ts
426                 current_session['shutdown_timestamp'] = ep
427                 och
428                 current_session['uptime_seconds'] = uptime
429                 _sec
430                 current_session['uptime_human'] = format_u
431                 ptime_human(uptime_sec)
432                 if not current_session.get('shutdown_type'
433                 ) or current_session.get('shutdown_type') == 'Active':
434                     current_session['shutdown_type'] = 'Un
435                     expected'
436                     current_session['unexpected_shutdown']
437                     = True
438                     current_session['clean_shutdown'] = Fa
439                     lse
440                     current_session['reason'] = current_se
441                     ssion['reason'] or 'Внезапный перезапуск без выключения (Kerne
442                     l-Power 41 / 6008)'
443                     sessions.append(PowerSessionRecord(**curre
444                     nt_session))
445
446             # Открываем новую сессию
447             session_id = f"PS-{ts.replace(':', '').replace
448             ('-', '').replace(' ', '-')}"
449             current_session = {
450                 'session_id': session_id,

```

```

438         'boot_time': ts,
439         'boot_timestamp': epoch,
440         'shutdown_time': None,
441         'shutdown_timestamp': None,
442         'uptime_seconds': 0.0,
443         'uptime_human': '',
444         'shutdown_type': 'Active',
445         'initiator': None,
446         'process': None,
447         'reason': None,
448         'reason_code': None,
449         'comment': None,
450         'clean_shutdown': True,
451         'unexpected_shutdown': False,
452         'bugcheck': None,
453         'boot_event_id': eid,
454         'shutdown_event_id': None,
455         'initiator_chain': [],
456         'events': [ev],
457         'created_at': epoch,
458     }
459
460     elif current_session:
461         current_session['events'].append(ev)
462
463         # Завершение сессии (1074, 13, 6006)
464         if eid == 1074:
465             current_session['shutdown_event_id'] = 107
466
467             4
468             current_session['shutdown_type'] = ev.get(
469                 'shutdown_type') or 'Restart'
470             current_session['initiator'] = ev.get('use
471             r')
472             current_session['process'] = ev.get('proce
473             ss')
474             current_session['reason'] = ev.get('reason
475             ')
476             current_session['reason_code'] = ev.get('r

```

```

eason_code')
471         current_session['comment'] = ev.get('comment')
472         current_session['clean_shutdown'] = True
473         details = json.loads(ev.get('details_json') or '{}') if isinstance(ev.get('details_json'), str) else (ev.get('details_json') or {})
474         if details.get('initiator_chain'):
475             current_session['initiator_chain'] = details['initiator_chain']
476
477         elif eid in (13, 6006):
478             if not current_session.get('shutdown_event_id'):
479                 current_session['shutdown_event_id'] = eid
480                 if current_session.get('shutdown_type') == 'Active':
481                     current_session['shutdown_type'] = 'Shutdown'
482                 current_session['clean_shutdown'] = True
483
484         elif eid in (41, 6008):
485             current_session['unexpected_shutdown'] = True
486             current_session['clean_shutdown'] = False
487             current_session['shutdown_type'] = 'Unexpected'
488             current_session['shutdown_event_id'] = eid
489             if ev.get('bugcheck_code'):
490                 current_session['bugcheck'] = ev.get('bugcheck_code')
491             current_session['reason'] = ev.get('reason') or 'Внезапное выключение питания (Kernel-Power 41)'
492
493         elif eid == 1001:
494             current_session['unexpected_shutdown'] = True
495     return

```

```

495         current_session['clean_shutdown'] = False
496         current_session['shutdown_type'] = 'BSOD'
497         current_session['bugcheck'] = ev.get('bugc
heck_code') or 'CRITICAL_PROCESS_DIED'
498
499         # Финализация последней активной сессии
500         if current_session:
501             cur_dt, cur_uptime = self.get_current_system_boot(
)
502             now_epoch = time.time()
503             if not current_session.get('shutdown_time'):
504                 current_session['shutdown_type'] = 'Active'
505                 current_session['uptime_seconds'] = max(0.0, n
ow_epoch - current_session['boot_timestamp'])
506                 current_session['uptime_human'] = format_uptime_human(current_session['uptime_seconds'])
507                 sessions.append(PowerSessionRecord(**current_session))
508
509         # Сортировка сессий от самых свежих к старым
510         sessions.sort(key=lambda s: s.boot_timestamp, reverse=
True)
511         return sessions
512
513     def scan_and_reconstruct(self, hours: int = 720, force: bo
ol = False) -> PowerLifecycleSummary:
514         """Выполняет сканирование журналов событий, парсинг и
сохранение сессий в SQLite.
515
516         Args:
517             hours: Глубина выборки в часах (по умолчанию 720ч
= 30 дней).
518             force: Принудительно выполнить пересчет.
519
520         Returns:
521             PowerLifecycleSummary: Сводный отчет жизненного ци
кла питания.
522         """

```



```

523         logger.info(f'[PowerLifecycle] Сканирование событий пи
        тания Windows за последние {hours}ч...')
524         raw_events = self.read_raw_power_events(hours=hours)
525
526         if raw_events:
527             self.storage.save_power_events(raw_events)
528             sessions = self.reconstruct_power_sessions(raw_eve
        nts)
529             self.storage.save_power_sessions(sessions)
530             logger.info(f'[PowerLifecycle] Реконструировано {l
        en(sessions)} сессий питания из {len(raw_events)} событий.')
531         else:
532             sessions = []
533
534         return self.get_summary()
535
536     def get_summary(self) -> PowerLifecycleSummary:
537         """Получает сводную статистику и последние сессии пита
        ния из БД SQLite.
538
539         Returns:
540             PowerLifecycleSummary: Сводный объект метрик.
541         """
542         db_summary = self.storage.get_power_summary()
543         db_sessions_raw = self.storage.get_power_sessions(limi
        t=30)
544         sessions_records = [PowerSessionRecord(**s) for s in d
        b_sessions_raw]
545
546         cur_dt, cur_uptime = self.get_current_system_boot()
547         cur_boot_str = cur_dt.strftime('%Y-%m-%d %H:%M:%S')
548         cur_uptime_human = format_uptime_human(cur_uptime)
549
550         return PowerLifecycleSummary(
551             current_boot_time=db_summary.get('current_boot_tim
        e') or cur_boot_str,
552             current_uptime_seconds=cur_uptime,
553             current_uptime_human=cur_uptime_human,

```

```

554         total_sessions_count=db_summary.get('total_session
| s_count', 0),
555         clean_shutdowns_count=db_summary.get('clean_shutdo
| wns_count', 0),
556         unexpected_shutdowns_count=db_summary.get('unexpec
| ted_shutdowns_count', 0),
557         bsod_count=db_summary.get('bsod_count', 0),
558         last_shutdown_type=db_summary.get('last_shutdown_t
| ype'),
559         last_initiator=db_summary.get('last_initiator'),
560         last_reason=db_summary.get('last_reason'),
561         sessions=sessions_records,
562     )
563
564     def get_sessions(
565         self,
566         limit: int = 50,
567         offset: int = 0,
568         shutdown_type: Optional[str] = None,
569         unexpected_only: bool = False,
570     ) -> List[PowerSessionRecord]:
571         """Возвращает список сессий питания с фильтрацией."""
572         items = self.storage.get_power_sessions(
573             limit=limit,
574             offset=offset,
575             shutdown_type=shutdown_type,
576             unexpected_only=unexpected_only,
577         )
578         return [PowerSessionRecord(**it) for it in items]
579
580     def get_events(
581         self,
582         limit: int = 100,
583         offset: int = 0,
584         event_id: Optional[Union[int, List[int]]] = None,
585     ) -> List[PowerEventRecord]:
586         """Возвращает список событий питания из БД."""
587         items = self.storage.get_power_events(limit=limit, off

```

```
    set=offset, event_id=event_id)
588         return [PowerEventRecord(**it) for it in items]
589
590
591 __all__ = [
592     'PowerLifecycleEngine',
593     'POWER_LIFECYCLE_EVENT_IDS',
594     'format_uptime_human',
595 ]
596
```

process_token_collector.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - Process
  Token Collector
4 # =====
5 # Description:
6 #   Коллектор токенов процессов, уровней целостности, прав UAC
  , SID и Session ID.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.process_token_collector impo
  rt ProcessTokenCollector
11 #
12 #       collector = ProcessTokenCollector()
13 #       tokens = collector.collect()
14 #
15 # File: process_token_collector.py
16 # Project: ai-breadboard
17 # Package: apps.windows.telemetry
18 # Author: hypo69
19 # Copyright: © 2026 hypo69
20 # Updated: 2026-10-04 01:45:00
21 # =====
22
23 """Коллектор токенов процессов, уровней целостности, прав UAC,
  SID и Session ID."""
24
25 from __future__ import annotations
26
27 import ctypes
28 import ctypes.wintypes as wtypes
29 import os
```

```
30 from typing import Dict, List, Optional
31 try:
32     import psutil
33     PSUTIL_AVAILABLE = True
34 except ImportError:
35     psutil = None
36     PSUTIL_AVAILABLE = False
37
38 from .models import ProcessTokenInfo
39
40 TOKEN_QUERY = 8
41 TokenUser = 1
42 TokenGroups = 2
43 TokenPrivileges = 3
44 TokenElevation = 20
45 TokenIntegrityLevel = 25
46
47
48 class TOKEN_ELEVATION(ctypes.Structure):
49     _fields_ = [('TokenIsElevated', ctypes.DWORD)]
50
51
52 class LUID(ctypes.Structure):
53     _fields_ = [('LowPart', ctypes.DWORD), ('HighPart', ctypes
54 .LONG)]
55
56 class LUID_AND_ATTRIBUTES(ctypes.Structure):
57     _fields_ = [('Luid', LUID), ('Attributes', ctypes.DWORD)]
58
59
60 class TOKEN_MANDATORY_LABEL(ctypes.Structure):
61     _fields_ = [('Label', LUID_AND_ATTRIBUTES)]
62
63
64 class SID_AND_ATTRIBUTES(ctypes.Structure):
65     _fields_ = [('Sid', ctypes.c_void_p), ('Attributes', ctypes
66 .DWORD)]
```

```

66 |
67 |
68 | class TOKEN_USER(ctypes.Structure):
69 |     _fields_ = [('User', SID_AND_ATTRIBUTES)]
70 |
71 |
72 | def _get_process_token(handle: Any) -> wtypes.HANDLE:
73 |     """Получить дескриптор токена процесса."""
74 |     token = wtypes.HANDLE()
75 |     if not ctypes.windll.advapi32.OpenProcessToken(handle, TOK
76 | EN_QUERY, ctypes.byref(token)):
77 |         raise ctypes.WinError()
78 |     return token
79 |
80 | def _get_token_elevation(token: Any) -> bool:
81 |     """Определить статус повышения прав UAC."""
82 |     elevation = TOKEN_ELEVATION()
83 |     size = wtypes.DWORD(ctypes.sizeof(elevation))
84 |     ret = ctypes.windll.advapi32.GetTokenInformation(
85 |         token, TokenElevation, ctypes.byref(elevation), size,
86 | ctypes.byref(size)
87 |     )
88 |     if not ret:
89 |         raise ctypes.WinError()
90 |     return bool(elevation.TokenIsElevated)
91 |
92 | def _get_integrity_level(token: Any) -> str:
93 |     """Получить уровень целостности процесса (Untrusted, Low,
94 | Medium, High, System)."""
95 |     size = wtypes.DWORD(0)
96 |     ctypes.windll.advapi32.GetTokenInformation(token, TokenInt
97 | egrityLevel, None, 0, ctypes.byref(size))
98 |     if size.value == 0:
99 |         return 'Medium'
100 |     buf = ctypes.create_string_buffer(size.value)
101 |     if not ctypes.windll.advapi32.GetTokenInformation(token, T

```

```

okenIntegrityLevel, buf, size, ctypes.byref(size)):
100     raise ctypes.WinError()
101     label = ctypes.cast(buf, ctypes.POINTER(TOKEN_MANDATORY_LA
BEL)).contents
102     integrity = label.Label.Luid.LowPart
103     if integrity < 4096:
104         return 'Untrusted'
105     elif integrity < 8192:
106         return 'Low'
107     elif integrity < 12288:
108         return 'Medium'
109     elif integrity < 16384:
110         return 'High'
111     else:
112         return 'System'
113
114
115 def _get_token_sid(token: Any) -> Optional[str]:
116     """Извлечь строковый SID пользователя из токена процесса."""
117     try:
118         size = wtypes.DWORD(0)
119         ctypes.windll.advapi32.GetTokenInformation(token, Toke
nUser, None, 0, ctypes.byref(size))
120         if size.value == 0:
121             return None
122         buf = ctypes.create_string_buffer(size.value)
123         if not ctypes.windll.advapi32.GetTokenInformation(token, TokenUser, buf, size, ctypes.byref(size)):
124             return None
125         user_struct = ctypes.cast(buf, ctypes.POINTER(TOKEN_US
ER)).contents
126         sid_ptr = user_struct.User.Sid
127         if not sid_ptr:
128             return None
129         str_sid = wtypes.LPWSTR()
130         if ctypes.windll.advapi32.ConvertSidToStringSidW(sid_p
tr, ctypes.byref(str_sid)):

```

```

131         sid_val = str_sid.value
132         ctypes.windll.kernel32.LocalFree(str_sid)
133         return sid_val
134     except Exception:
135         pass
136     return None
137
138
139 def get_process_session_id(pid: int) -> Optional[int]:
140     """Получить Session ID процесса (Session 0=Службы, Session
141     1+=Интерактивный пользователь)."""
142     if os.name != 'nt':
143         return None
144     try:
145         session_id = wtypes.DWORD(0)
146         if ctypes.windll.kernel32.ProcessIdToSessionId(pid, ct
147         ypes.byref(session_id)):
148             return int(session_id.value)
149     except Exception:
150         pass
151     return None
152
153 class ProcessTokenCollector:
154     """Коллектор сведений о безопасности токенов, правах UAC,
155     SID и сессиях процессов Windows."""
156
157     def get_process_token_info(self, pid: int, name: str = '')
158     -> Optional[ProcessTokenInfo]:
159         """Собрать информацию о токене для конкретного PID.
160
161         Args:
162             pid: Идентификатор процесса.
163             name: Имя процесса (опционально).
164
165         Returns:
166             Optional[ProcessTokenInfo]: Данные токена или None
167             при отсутствии доступа.

```



```

164         """
165         if os.name != 'nt':
166             return ProcessTokenInfo(pid=pid, name=name, elevat
ion=False, integrity_level='Medium', sid=None, session_id=1)
167
168         session_id = get_process_session_id(pid)
169         handle = ctypes.windll.kernel32.OpenProcess(1024, Fals
e, pid)
170         if not handle:
171             return ProcessTokenInfo(
172                 pid=pid,
173                 name=name,
174                 elevation=None,
175                 integrity_level=None,
176                 privileges=[],
177                 sid=None,
178                 session_id=session_id,
179             )
180         try:
181             token = _get_process_token(handle)
182             elevated = _get_token_elevation(token)
183             integrity = _get_integrity_level(token)
184             sid = _get_token_sid(token)
185             privileges: List[str] = []
186             return ProcessTokenInfo(
187                 pid=pid,
188                 name=name,
189                 elevation=bool(elevated),
190                 integrity_level=integrity,
191                 privileges=privileges,
192                 sid=sid,
193                 session_id=session_id,
194             )
195         except Exception:
196             return ProcessTokenInfo(
197                 pid=pid,
198                 name=name,
199                 elevation=None,

```

```

200         integrity_level=None,
201         privileges=[],
202         sid=None,
203         session_id=session_id,
204     )
205     finally:
206         ctypes.windll.kernel32.CloseHandle(handle)
207
208     def collect(self) -> List[ProcessTokenInfo]:
209         """Собрать сведения о токенах для всех активных процес
210         сов в системе.
211
212         Returns:
213             List[ProcessTokenInfo]: Список токенов процессов.
214         """
215         results: List[ProcessTokenInfo] = []
216         if not PSUTIL_AVAILABLE:
217             current_pid = os.getpid()
218             info = self.get_process_token_info(current_pid, 'p
219             ython.exe')
220             if info:
221                 results.append(info)
222             return results
223
224         for proc in psutil.process_iter(attrs=['pid', 'name']):
225             :
226             try:
227                 pid = proc.info['pid']
228                 name = proc.info['name'] or 'unknown'
229                 info = self.get_process_token_info(pid, name)
230                 if info:
231                     results.append(info)
232             except Exception:
233                 continue
234         return results

```

sampling_controller.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - Sampling
  # Controller
4 # =====
5 # Description:
6 #   Контроллер адаптивного изменения частоты сбора телеметрии
  # (Sampling Controller).
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.sampling_controller import S
  # amplingController
11 #
12 #       service = SamplingController()
13 #
14 # File: sampling_controller.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-06 05:34:00
20 # =====
21
22 from __future__ import annotations
23 """Контроллер адаптивного изменения частоты сбора телеметрии (
  # Sampling Controller)."""
24
25 import time
26 from datetime import datetime, timezone
27 from typing import Any, Dict, Optional
28
29 try:
```

```

30     import psutil
31     BOOT_TIME = psutil.boot_time()
32 except Exception:
33     BOOT_TIME = time.time()
34
35 try:
36     from logger import logger
37 except ImportError:
38     from logger import logger
39
40 from .models import SamplingMode
41
42
43 class SamplingController:
44     """Контроллер режимов сбора телеметрии и адаптивного интер
45 вала."""
46
47     def __init__(
48         self,
49         initial_mode: SamplingMode = SamplingMode.BOOT_AGGRESS
50 IVE,
51         boot_epoch: Optional[float] = None,
52     ) -> None:
53         """Инициализирует контроллер режимов сбора.
54
55         Args:
56             initial_mode: Начальный режим работы (по умолчанию
57 BOOT_AGGRESSIVE).
58             boot_epoch: Время старта ОС или сервиса в секундах
59 Unix epoch.
60         """
61         self._mode = initial_mode
62         self._boot_epoch = boot_epoch if boot_epoch is not Non
63 e else BOOT_TIME
64         self._start_epoch = time.time()
65         self._incident_expiry_epoch: Optional[float] = None
66         self._previous_mode: SamplingMode = SamplingMode.MONIT
67 ORING

```

```

62 |
63 |     @property
64 |     def mode(self) -> SamplingMode:
65 |         """Возвращает текущий режим сбора с учетом авто-истече
        ния инцидентов."""
66 |         self._check_incident_expiration()
67 |         return self._mode
68 |
69 |     def set_mode(self, mode: SamplingMode) -> None:
70 |         """Принудительно устанавливает режим сбора телеметрии.
71 |
72 |         Args:
73 |             mode: Новый режим работы.
74 |         """
75 |         self._mode = mode
76 |         if mode not in (SamplingMode.INCIDENT, SamplingMode.FO
        RENSIC):
77 |             self._incident_expiry_epoch = None
78 |             logger.info(f"Режим сбора телеметрии переключен на: {s
        elf._mode.value}")
79 |
80 |     def trigger_incident_mode(self, duration_seconds: float =
        1200.0) -> None:
81 |         """Переводит сборщик в режим INCIDENT (1-секундный зах
        ват) на заданную длительность.
82 |
83 |         Args:
84 |             duration_seconds: Длительность удержания режима ин
        цидента (по умолчанию 20 минут = 1200 сек).
85 |         """
86 |         if self._mode not in (SamplingMode.INCIDENT, SamplingM
        ode.FORENSIC):
87 |             self._previous_mode = self._mode
88 |             self._mode = SamplingMode.INCIDENT
89 |             self._incident_expiry_epoch = time.time() + max(0.01,
        duration_seconds)
90 |             logger.warning(f"Активирован режим INCIDENT на {durati
        on_seconds} сек.")

```

```
91 |
92 |     def trigger_forensic_mode(self, duration_seconds: float =
    | 300.0) -> None:
93 |         """Переводит сборщик в режим FORENSIC (250-мс глубокий
    | захват) на заданную длительность.
94 |
95 |         Args:
96 |             duration_seconds: Длительность удержания форензик-
    | режима в секундах (по умолчанию 5 минут = 300 сек).
97 |         """
98 |         if self._mode not in (SamplingMode.INCIDENT, SamplingM
    | ode.FORENSIC):
99 |             self._previous_mode = self._mode
100 |             self._mode = SamplingMode.FORENSIC
101 |             self._incident_expiry_epoch = time.time() + max(0.01,
    | duration_seconds)
102 |             logger.warning(f"Активирован высокоскоростной режим FO
    | RENSIC (250мс) на {duration_seconds} сек.")
103 |
104 |     def get_current_interval(self) -> float:
105 |         """Возвращает текущий рекомендуемый интервал опроса те
    | леметрии в секундах.
106 |
107 |         Returns:
108 |             float: Интервал в секундах.
109 |         """
110 |         self._check_incident_expiration()
111 |
112 |         if self._mode == SamplingMode.FORENSIC:
113 |             return 0.25
114 |
115 |         if self._mode == SamplingMode.INCIDENT:
116 |             return 1.0
117 |
118 |         if self._mode == SamplingMode.BOOT_AGGRESSIVE:
119 |             # Адаптивная кривая после старта Windows / сервиса
120 |             uptime = time.time() - self._boot_epoch
121 |             if uptime < 0:
```

```

122         uptime = time.time() - self._start_epoch
123
124         if uptime <= 600.0: # 0-10 минут: 1-секундные дан
ные
125             return 1.0
126         elif uptime <= 3600.0: # 10-60 минут: 5-секундные
127             return 5.0
128         elif uptime <= 86400.0: # 1-24 часа: 30-секундные
129             return 30.0
130         else:
131             # После 24 часов переходим в дежурный монитори
нг
132             return 60.0
133
134         if self._mode == SamplingMode.MONITORING:
135             return 30.0
136
137         if self._mode == SamplingMode.NORMAL:
138             return 60.0
139
140         return 30.0
141
142     def should_collect_per_process_details(self) -> bool:
143         """Определяет, требуется ли собирать глубокий срез по
всем процессам на текущем тике."""
144         return True
145
146     def _check_incident_expiration(self) -> None:
147         """Проверяет истечение времени режима инцидента/форенз
ики и возвращает предыдущий режим."""
148         if self._mode in (SamplingMode.INCIDENT, SamplingMode.
FORENSIC) and self._incident_expiry_epoch is not None:
149             if time.time() >= self._incident_expiry_epoch:
150                 logger.info("Время фиксации инцидента/форензик
и истекло. Возврат в штатный режим.")
151                 self._mode = self._previous_mode
152                 self._incident_expiry_epoch = None
153

```

```
154 |     def get_status(self) -> Dict[str, Any]:
155 |         """Возвращает статус контроллера опроса.
156 |
157 |         Returns:
158 |             Dict[str, Any]: Словарь с текущим режимом, интерва
лом и временем старта.
159 |         """
160 |         return {
161 |             "mode": self.mode.value,
162 |             "current_interval_sec": self.get_current_interval(
163 |             ),
164 |             "boot_epoch": self._boot_epoch,
165 |             "incident_active": self._mode in (SamplingMode.INC
IDENT, SamplingMode.FORENSIC),
166 |             "incident_remaining_sec": max(0.0, (self._incident
_expiry_epoch - time.time())) if self._incident_expiry_epoch e
lse 0.0,
167 |         }
```


security_collector.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - Security Collector
4 # =====
5 # Description:
6 #   Коллектор событий журнала безопасности Windows Security Event Log.
7 #   Реализует инкрементальный сбор на основе закладок (Bookmark), XPath-фильтрацию,
8 #   нормализацию и корреляцию с телеметрией системы.
9 #
10 # Usage Examples:
11 #   Python API:
12 #       from apps.windows.telemetry.security_collector import WindowsSecurityCollector
13 #
14 #       collector = WindowsSecurityCollector()
15 #       status = collector.check_access_and_audit()
16 #       report = collector.collect_incremental(batch_size=200)
17 #
18 # File: security_collector.py
19 # Project: ai-breadboard
20 # Package: apps.windows.telemetry
21 # Author: hypo69
22 # Copyright: © 2026 hypo69
23 # Updated: 2026-10-08 02:15:00
24 # =====
25
26 from __future__ import annotations
27 """Коллектор событий журнала безопасности Windows Security Event Log."""
28
```

```
29 import os
30 import sys
31 import time
32 import winreg
33 from datetime import datetime, timezone
34 from typing import Any, Callable, Dict, List, Optional, Set, T
   tuple
35
36 try:
37     from logger import logger
38 except ImportError:
39     import logging
40     logger = logging.getLogger(__name__)
41
42 from .models import (
43     SecurityAuditStatus,
44     SecurityBookmarkState,
45     SecurityCollectorReport,
46     SecurityCorrelationItem,
47     SecurityEventItem,
48     SecurityEventRaw,
49 )
50 from .security_normalizer import SecurityEventNormalizer
51 from .sqlite import TelemetryStorage
52 from .win32_fffi.wevtapi import WevtAPI
53
54
55 DEFAULT_SECURITY_EVENT_IDS: List[int] = [
56     4624, # Успешный вход
57     4625, # Неудачный вход
58     4634, # Завершение logon
59     4647, # Пользователь инициировал logoff
60     4672, # Специальные привилегии
61     4688, # Создание процесса
62     4689, # Завершение процесса
63     4697, # Установка службы
64     4698, # Создание Scheduled Task
65     4702, # Изменение Scheduled Task
```

```

66 | 4719, # Изменение аудита
67 | 4720, # Создание пользователя
68 | 4722, # Включение пользователя
69 | 4723, # Попытка изменения пароля
70 | 4724, # Сброс пароля
71 | 4725, # Отключение пользователя
72 | 4726, # Удаление пользователя
73 | 4732, # Добавление в локальную группу
74 | 4738, # Изменение пользователя
75 | 4740, # Блокировка пользователя
76 | 4768, # Kerberos TGT
77 | 4769, # Kerberos service ticket
78 | 4771, # Kerberos pre-auth failure
79 | 4776, # NTLM authentication
80 | 1102, # Очищен Security log
81 | ]
82 |
83 |
84 | class WindowsSecurityCollector:
85 |     """Инкрементальный сборщик событий журнала безопасности Wi
86 |     ndows (Security.evtx)."""
87 |
88 |     def __init__(
89 |         self,
90 |         storage: Optional[TelemetryStorage] = None,
91 |         wevtapi: Optional[WevtAPI] = None,
92 |         normalizer: Optional[SecurityEventNormalizer] = None,
93 |     ) -> None:
94 |         """Инициализация сборщика безопасности.
95 |
96 |         Args:
97 |             storage: Экземпляр хранилища TelemetryStorage (есл
98 |             и None, берется синглтон).
99 |             wevtapi: Экземпляр обертки WevtAPI.
100 |             normalizer: Экземпляр нормализатора событий.
101 |         """
102 |         self.storage = storage or TelemetryStorage.get_instanc
103 |         e()

```

```

101         self.wevtapi = wevtapi or WevtAPI()
102         self.normalizer = normalizer or SecurityEventNormalize
103         r()
104     def check_access_and_audit(self) -> SecurityAuditStatus:
105         """Проверить доступность журнала Security и статус пол
106         итик аудита.
107         Returns:
108             SecurityAuditStatus: Структурированный статус прав
109             и настроек.
110         """
111         access_info = self.wevtapi.check_channel_access('Secur
112         ity')
113         accessible = access_info.get('accessible', False)
114         record_count = access_info.get('record_count', 0)
115         error_msg = access_info.get('error')
116
117         # Проверка включения аудита командной строки в реестре
118         cmdline_audit = False
119         try:
120             with winreg.OpenKey(
121                 winreg.HKEY_LOCAL_MACHINE,
122                 r'SOFTWARE\Microsoft\Windows\CurrentVersion\Po
123                 licies\System\Audit',
124                 0,
125                 winreg.KEY_READ,
126             ) as key:
127                 val, _ = winreg.QueryValueEx(key, 'ProcessCrea
128                 tionIncludeCmdLine_Enabled')
129                 cmdline_audit = bool(val)
130         except OSError:
131             cmdline_audit = False
132
133         # Получаем статистику из базы данных
134         stats = self.storage.get_security_stats() if self.stor
135         age else {}
136         last_rec = stats.get('max_record_id', 0)

```

```

132         total_ingested = stats.get('total_events', 0)
133
134         return SecurityAuditStatus(
135             accessible=accessible,
136             record_count=record_count,
137             process_creation_audit_enabled=True, # Базовый ауд
138             command_line_audit_enabled=cmdline_audit,
139             error=error_msg,
140             last_record_id=last_rec,
141             total_ingested_events=total_ingested,
142         )
143
144     def _build_xpath_query(self, event_ids: Optional[List[int]
145 ] = None) -> str:
146         """Построить эффективный XPath фильтр для выборки толь
147         ко целевых Event ID.
148
149         Предотвращает переполнение предиката Win32 WevtAPI (ко
150         д 15001) при длинных списках,
151         используя диапазонную фильтрацию.
152         """
153         if event_ids is None:
154             # Стандартный диапазон охватывает все ключевые соб
155             ытия аудита безопасности (4624-4776 + 1102)
156             return '*[System[(EventID >= 4624 and EventID <= 4
157             776) or EventID=1102]]'
158         if not event_ids:
159             return '*'
160         if len(event_ids) <= 15:
161             clauses = ' or '.join(f'EventID={eid}' for eid in

```

```

162     def collect_incremental(
163         self,
164         batch_size: int = 500,
165         max_records: int = 5000,
166         filter_event_ids: Optional[List[int]] = None,
167         save_raw: bool = False,
168     ) -> SecurityCollectorReport:
169         """Выполнить инкрементальный сбор новых событий из жур
нала Security.
170
171         Args:
172             batch_size: Размер порции за один вызов EvtNext.
173             max_records: Максимальное количество событий для с
бора за один вызов.
174             filter_event_ids: Список опрашиваемых Event ID или
None для дефолтного набора.
175             save_raw: Сохранять ли параллельно сырой XML в sec
urity_events_raw.
176
177         Returns:
178             SecurityCollectorReport: Отчет о результатах сбора
.
179         """
180         report = SecurityCollectorReport(channel='Security')
181
182         # Проверка доступа к журналу
183         access_info = self.wevtapi.check_channel_access('Secur
ity')
184         if not access_info.get('accessible', False):
185             err = access_info.get('error') or 'Доступ к Secur
ity.evtx запрещен'
186             report.errors.append(err)
187             logger.warning(f'[SecurityCollector] {err}')
188             return report
189
190         # Загрузка сохраненной закладки
191         bm_dict = self.storage.get_security_bookmark('Security
') if self.storage else None

```

```

192 |         bookmark_xml = bm_dict.get('bookmark_xml') if bm_dict
    | else None
193 |         last_saved_record_id = bm_dict.get('last_record_id', 0
    | ) if bm_dict else 0
194 |
195 |         xpath = self._build_xpath_query(filter_event_ids)
196 |         collected_items: List[SecurityEventItem] = []
197 |         collected_raw: List[SecurityEventRaw] = []
198 |         events_by_id: Dict[int, int] = {}
199 |         max_record_id = last_saved_record_id
200 |         latest_timestamp = ''
201 |         total_fetched = 0
202 |
203 |         # Если закладки еще нет (первый запуск), читаем истори
    | ческий хвост последних событий в обратном порядке
204 |         if not bookmark_xml:
205 |             tail_limit = min(batch_size, max_records)
206 |             events_raw, init_bm, highest_rec_id = self.wevtapi
    | .read_events_incremental(
207 |                 channel='Security',
208 |                 query_xpath=xpath,
209 |                 bookmark_xml=None,
210 |                 batch_size=tail_limit,
211 |                 format_message=False,
212 |                 forward=False,
213 |             )
214 |             # Разворачиваем в хронологический порядок
215 |             events_raw = list(reversed(events_raw))
216 |             bookmark_xml = init_bm
217 |             max_record_id = highest_rec_id
218 |
219 |             for raw_dict in events_raw:
220 |                 item, raw_item = self.normalizer.normalize_eve
    | nt(raw_dict, extract_raw=save_raw)
221 |                 collected_items.append(item)
222 |                 if raw_item:
223 |                     collected_raw.append(raw_item)
224 |                 events_by_id[item.event_id] = events_by_id.get

```

```

    (item.event_id, 0) + 1
225         if item.timestamp:
226             latest_timestamp = item.timestamp
227             rec_id = item.event_record_id
228             if rec_id > max_record_id:
229                 max_record_id = rec_id
230
231         if self.storage and collected_items:
232             self.storage.save_security_events(collected_items, save_raw=save_raw, raw_events=collected_raw)
233             if bookmark_xml:
234                 self.storage.save_security_bookmark(
235                     channel='Security',
236                     last_record_id=max_record_id,
237                     bookmark_xml=bookmark_xml,
238                     last_timestamp=latest_timestamp,
239                 )
240             total_fetched += len(events_raw)
241
242         # Последующие запуски (или продолжение) читают строго
    вперед от закладки
243         while total_fetched < max_records and bookmark_xml:
244             cur_batch_limit = min(batch_size, max_records - total_fetched)
245             events_raw, new_bookmark_xml, highest_rec_id = self.wevtapi.read_events_incremental(
246                 channel='Security',
247                 query_xpath=xpath,
248                 bookmark_xml=bookmark_xml,
249                 batch_size=cur_batch_limit,
250                 format_message=False,
251                 forward=True,
252             )
253
254         if not events_raw:
255             break
256
257         batch_items: List[SecurityEventItem] = []

```



```
258         batch_raw: List[SecurityEventRaw] = []
259
260         for raw_dict in events_raw:
261             item, raw_item = self.normalizer.normalize_event(
262                 raw_dict, extract_raw=save_raw)
263             batch_items.append(item)
264             if raw_item:
265                 batch_raw.append(raw_item)
266                 events_by_id[item.event_id] = events_by_id.get(
267                     item.event_id, 0) + 1
268                 if item.timestamp:
269                     latest_timestamp = item.timestamp
270                     rec_id = item.event_record_id
271                     if rec_id > max_record_id:
272                         max_record_id = rec_id
273
274             if highest_rec_id > max_record_id:
275                 max_record_id = highest_rec_id
276
277             # Сохраняем порцию в SQLite
278             if self.storage and batch_items:
279                 self.storage.save_security_events(batch_items,
280                     save_raw=save_raw, raw_events=batch_raw)
281                 if new_bookmark_xml:
282                     self.storage.save_security_bookmark(
283                         channel='Security',
284                         last_record_id=max_record_id,
285                         bookmark_xml=new_bookmark_xml,
286                         last_timestamp=latest_timestamp,
287                     )
288
289             collected_items.extend(batch_items)
290             if save_raw:
291                 collected_raw.extend(batch_raw)
292
293             total_fetched += len(events_raw)
294             bookmark_xml = new_bookmark_xml
```

```

293 |         # Если вернулось меньше запрошенного batch_size, з
начит достигли текущего конца журнала
294 |         if len(events_raw) < cur_batch_limit:
295 |             break
296 |
297 |         report.total_events_ingested = len(collected_items)
298 |         report.last_record_id = max_record_id
299 |         report.bookmark_xml = bookmark_xml
300 |         report.events_by_id = events_by_id
301 |         report.events = collected_items[:100] # Ограничиваем
возвращаемый срез в отчете
302 |
303 |         logger.info(
304 |             f'[SecurityCollector] Инкрементально собрано {len(
collected_items)} событий Security. '
305 |             f'Последний RecordID: {max_record_id}'
306 |         )
307 |         return report
308 |
309 |     def get_live_tail(
310 |         self,
311 |         limit: int = 20,
312 |         event_id: Optional[int] = None,
313 |         search: str = '',
314 |     ) -> List[SecurityEventItem]:
315 |         """Получить самые свежие события безопасности в реальн
ом времени (live tail).
316 |
317 |         Args:
318 |             limit: Количество событий.
319 |             event_id: Опциональный фильтр по Event ID.
320 |             search: Поисковая подстрока.
321 |
322 |         Returns:
323 |             List[SecurityEventItem]: Список нормализованных со
бытий.
324 |         """
325 |         raw_events = self.wevtapi.read_events(

```

```

326         channel='Security',
327         limit=limit,
328         event_id=event_id or 0,
329         search=search,
330         format_message=False,
331     )
332     items: List[SecurityEventItem] = []
333     for raw in raw_events:
334         item, _ = self.normalizer.normalize_event(raw, extract_raw=False)
335         items.append(item)
336     return items
337
338     def subscribe_stream(
339         self,
340         callback: Callable[[SecurityEventItem], None],
341         filter_event_ids: Optional[List[int]] = None,
342     ) -> Optional[Any]:
343         """Оформить потоковую подписку на входящие события без
344         опасности через EvtSubscribe.
345
346         Args:
347             callback: Функция обратного вызова при получении события.
348
349             filter_event_ids: Список Event ID для фильтрации.
350
351         Returns:
352             Optional[HANDLE]: Дескриптор подписки или None.
353         """
354         xpath = self._build_xpath_query(filter_event_ids)
355
356         def _on_event(raw_dict: Dict[str, Any]) -> None:
357             item, raw_item = self.normalizer.normalize_event(raw_dict, extract_raw=True)
358             if self.storage:
359                 self.storage.save_security_events([item], save_raw=True, raw_events=[raw_item] if raw_item else None)
360             callback(item)

```

```

359
360         return self.wevtapi.subscribe_events(
361             channel='Security',
362             query_xpath=xpath,
363             callback=_on_event,
364             start_at_oldest=False,
365         )
366
367     def correlate_security_with_telemetry(
368         self,
369         time_window_seconds: float = 60.0,
370         pid: Optional[int] = None,
371         user: Optional[str] = None,
372         limit: int = 50,
373     ) -> List[SecurityCorrelationItem]:
374         """Выполнить корреляцию событий безопасности с метрика
ми телеметрии и процессами.
375
376         Связывает события создания процессов (4688), входов по
льзователей (4624) и
377         событий служб (4697) с реальной загрузкой CPU/RAM/диск
а и снимками процессов.
378
379         Args:
380             time_window_seconds: Временное окно корреляции в с
екундах.
381             pid: Фильтр по PID.
382             user: Фильтр по пользователю.
383             limit: Лимит записей.
384
385         Returns:
386             List[SecurityCorrelationItem]: Список коррелирован
ных событий с аналитикой.
387         """
388         if not self.storage:
389             return []
390
391         # 1. Извлекаем события безопасности

```

```

392         sec_events = self.storage.get_security_events(
393             user=user,
394             pid=pid,
395             limit=limit,
396         )
397         if not sec_events:
398             return []
399
400         # 2. Извлекаем последние снимки системы
401         snapshots = self.storage.get_snapshots(limit=30)
402         snap_by_time = {s.get('timestamp', ''): s for s in snapshots}
403
404         correlations: List[SecurityCorrelationItem] = []
405
406         for ev in sec_events:
407             eid = int(ev.get('event_id', 0) or 0)
408             p_name = ev.get('process_name', '')
409             p_short = p_name.replace('\\', '/').split('/')[-1]
410             if p_name else ''
411             p_pid = int(ev.get('process_id', 0) or 0)
412             u_name = ev.get('subject_user', '') or ev.get('target_user', '')
413             ts = ev.get('timestamp', '')
414             cmd = ev.get('command_line', '')
415             parent_name = ev.get('parent_process_name', '')
416             parent_short = parent_name.replace('\\', '/').split('/')[-1] if parent_name else ''
417
418             notes: List[str] = []
419             cpu_val: Optional[float] = None
420             ram_val: Optional[float] = None
421             disk_spike = False
422             net_spike = False
423
424             # Поиск близкого по времени снимка телеметрии
425             for s_ts, snap in snap_by_time.items():
426                 if ts and s_ts and abs(len(ts) - len(s_ts)) <

```

```

5:
426         cpu_total = float(snap.get('cpu_total_perc
ent', 0.0) or 0.0)
427         if cpu_total > 50.0:
428             cpu_val = cpu_total
429             notes.append(f'Повышенная загрузка CPU
системы в момент события: {cpu_total:.1f}%')
430
431         d_write = float(snap.get('disk_write_bytes
_sec', 0.0) or 0.0)
432         if d_write > 50 * 1024 * 1024:
433             disk_spike = True
434             notes.append(f'Всплеск дисковой записи
: {d_write / 1024 / 1024:.1f} МБ/с')
435
436         n_recv = float(snap.get('network_recv_byte
s_sec', 0.0) or 0.0)
437         if n_recv > 10 * 1024 * 1024:
438             net_spike = True
439             notes.append(f'Всплеск сетевого трафик
а: {n_recv / 1024 / 1024:.1f} МБ/с')
440             break
441
442         if eid == 4688:
443             ev_type = 'ProcessCreate'
444             if parent_short and ('powershell' in parent_sh
ort.lower() or 'cmd' in parent_short.lower()):
445                 notes.append(f'Запуск из интерактивной обо
лочки {parent_short}')
446                 if 'admin' in u_name.lower() or ev.get('elevat
ed_token') == 1:
447                     notes.append('Запуск с повышенными привиле
гиями (Elevated Token)')
448             elif eid == 4624:
449                 ev_type = 'UserLogon'
450                 ltype = ev.get('logon_type')
451                 if ltype == 10:
452                     notes.append('Удаленный вход по протоколу

```

```

RDP (LogonType 10)')
453         elif ltype == 2:
454             notes.append('Локальный интерактивный вход
(LogonType 2)')
455         elif ltype == 3:
456             notes.append('Сетевой вход SMB/RPC (LogonT
ype 3)')
457         elif eid == 4625:
458             ev_type = 'FailedLogon'
459             notes.append(f"Неудачная попытка входа: {ev.ge
t('status_code', '')}")
460         elif eid == 4697:
461             ev_type = 'ServiceInstall'
462             notes.append(f"Установка новой службы {ev.get(
'object_name', '')}")
463         else:
464             ev_type = f'Event_{eid}'
465
466         correlations.append(
467             SecurityCorrelationItem(
468                 timestamp=ts,
469                 event_type=ev_type,
470                 user=u_name,
471                 process_name=p_short or p_name,
472                 pid=p_pid,
473                 parent_process_name=parent_short or parent
_name,
474                 command_line=cmd,
475                 cpu_percent=cpu_val,
476                 ram_mb=ram_val,
477                 disk_spike=disk_spike,
478                 network_spike=net_spike,
479                 notes=notes,
480             )
481         )
482
483     return correlations
484

```

```
485 |  
486 | __all__ = [  
487 |     'WindowsSecurityCollector',  
488 |     'DEFAULT_SECURITY_EVENT_IDS',  
489 | ]  
490 |
```


security_normalizer.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - Security Normalizer
4 # =====
5 # Description:
6 #   Нормализатор событий журнала безопасности Windows Security Event Log.
7 #   Извлекает и структурирует атрибуты пользователей, процессов, сессий и политик.
8 #
9 # Usage Examples:
10 #   Python API:
11 #       from apps.windows.telemetry.security_normalizer import SecurityEventNormalizer
12 #
13 #       normalizer = SecurityEventNormalizer()
14 #       item, raw = normalizer.normalize_event(event_dict)
15 #
16 # File: security_normalizer.py
17 # Project: ai-breadboard
18 # Package: apps.windows.telemetry
19 # Author: hypo69
20 # Copyright: © 2026 hypo69
21 # Updated: 2026-10-08 02:15:00
22 # =====
23
24 from __future__ import annotations
25 """Нормализатор событий журнала безопасности Windows Security Event Log."""
26
27 import json
28 import time
```

```

29 import xml.etree.ElementTree as ET
30 from datetime import datetime, timezone
31 from typing import Any, Dict, List, Optional, Tuple
32
33 try:
34     from logger import logger
35 except ImportError:
36     import logging
37     logger = logging.getLogger(__name__)
38
39 from .models import SecurityEventItem, SecurityEventRaw
40
41
42 def _parse_int(val: Any, default: int = 0) -> int:
43     """Безопасный парсинг целого числа из строки, включая шест
    надцатеричные значения (0x...)."""
44     if val is None:
45         return default
46     if isinstance(val, int):
47         return val
48     s = str(val).strip()
49     if not s or s == '-':
50         return default
51     try:
52         if s.startswith(('0x', '0X')):
53             return int(s, 16)
54         return int(s)
55     except ValueError:
56         try:
57             return int(float(s))
58         except ValueError:
59             return default
60
61
62 class SecurityEventNormalizer:
63     """Нормализатор сырых XML и словарей событий Windows Secur
    ity в типизированные модели."""
64

```

```

65 |     EVENT_DESCRIPTIONS = {
66 |         4624: 'Успешный вход пользователя в систему',
67 |         4625: 'Неудачная попытка входа в систему',
68 |         4634: 'Завершение сеанса пользователя (Logoff)',
69 |         4647: 'Пользователь инициировал выход из системы',
70 |         4672: 'Назначение специальных привилегий администратор
    | а',
71 |         4688: 'Создание нового процесса',
72 |         4689: 'Завершение процесса',
73 |         4697: 'Установка новой системной службы',
74 |         4698: 'Создание запланированной задачи (Task Scheduler
    | )',
75 |         4702: 'Изменение параметров запланированной задачи',
76 |         4719: 'Изменение системной политики аудита безопасност
    | и',
77 |         4720: 'Создание учетной записи пользователя',
78 |         4722: 'Включение учетной записи пользователя',
79 |         4723: 'Попытка изменения пароля пользователя',
80 |         4724: 'Сброс пароля учетной записи пользователя',
81 |         4725: 'Отключение учетной записи пользователя',
82 |         4726: 'Удаление учетной записи пользователя',
83 |         4732: 'Добавление пользователя в локальную группу безо
    | пасности',
84 |         4738: 'Изменение параметров учетной записи пользовател
    | я',
85 |         4740: 'Блокировка учетной записи пользователя',
86 |         4768: 'Запрос билета Kerberos TGT (Authentication Tick
    | et)',
87 |         4769: 'Запрос сервисного билета Kerberos (Service Tick
    | et)',
88 |         4771: 'Сбой предварительной проверки подлинности Kerbe
    | ros',
89 |         4776: 'Проверка учетных данных контроллером (NTLM/MSV1
    | _0)',
90 |         1102: 'ВНИМАНИЕ: Журнал аудита безопасности был очищен
    | ',
91 |     }
92 |

```

```

93     def normalize_event(
94         self,
95         event_dict: Dict[str, Any],
96         extract_raw: bool = True,
97     ) -> Tuple[SecurityEventItem, Optional[SecurityEventRaw]]:
98         """Нормализовать срез события безопасности в типизированную
99         структуру.
100
101         Args:
102             event_dict: Словарь события от WevtAPI.
103             extract_raw: Формировать ли параллельно объект сырого XML.
104
105         Returns:
106             Tuple[SecurityEventItem, Optional[SecurityEventRaw]]:
107                 Кортеж нормализованного события и опционального сырого объекта.
108
109         """
110         event_id = _parse_int(event_dict.get('event_id', 0))
111         record_id = _parse_int(event_dict.get('record_id', 0))
112         timestamp = str(event_dict.get('timestamp', '') or '')
113         computer = str(event_dict.get('computer', '') or '')
114         channel = str(event_dict.get('channel', 'Security') or 'Security')
115         level = str(event_dict.get('level', 'Information') or 'Information')
116         raw_xml = str(event_dict.get('raw_data', '') or '')
117
118         # Получаем данные eventData
119         ed = event_dict.get('event_data', {})
120         if not ed and raw_xml:
121             ed = self._parse_event_data_xml(raw_xml)
122
123         now_iso = datetime.now(timezone.utc).isoformat()
124         now_epoch = time.time()
125
126         # Извлечение общих субъектов

```

```
125 |         subject_user = ed.get('SubjectUserName') or ed.get('Us
| erName') or ''
126 |         subject_domain = ed.get('SubjectDomainName') or ed.get
| ('UserDomain') or ''
127 |         subject_sid = ed.get('SubjectUserSid') or ed.get('User
| Sid') or ''
128 |
129 |         target_user = ed.get('TargetUserName') or ed.get('Memb
| erName') or ''
130 |         target_domain = ed.get('TargetDomainName') or ''
131 |         target_sid = ed.get('TargetUserSid') or ed.get('Member
| Sid') or ''
132 |
133 |         # Извлечение процессов
134 |         process_id = 0
135 |         process_name = ''
136 |         parent_process_id = 0
137 |         parent_process_name = ''
138 |         command_line = ''
139 |
140 |         if event_id == 4688:
141 |             process_id = _parse_int(ed.get('NewProcessId') or
| ed.get('ProcessId', 0))
142 |             process_name = str(ed.get('NewProcessName') or '')
143 |             parent_process_id = _parse_int(ed.get('ProcessId')
| or ed.get('ParentProcessId', 0))
144 |             parent_process_name = str(ed.get('ParentProcessNam
| e') or '')
145 |             command_line = str(ed.get('CommandLine') or '')
146 |         elif event_id == 4689:
147 |             process_id = _parse_int(ed.get('ProcessId', 0))
148 |             process_name = str(ed.get('ProcessName') or '')
149 |         else:
150 |             process_id = _parse_int(ed.get('ProcessId', 0))
151 |             process_name = str(ed.get('ProcessName') or '')
152 |
153 |         # Logon сессия и сеть
154 |         logon_id = str(ed.get('TargetLogonId') or ed.get('Subj
```

```

    ectLogonId') or '')
155         logon_type_val = ed.get('LogonType')
156         logon_type = _parse_int(logon_type_val, -1) if logon_t
type_val is not None else None
157         if logon_type == -1:
158             logon_type = None
159
160         elevated_token_val = ed.get('ElevatedToken') or ed.get
('TokenElevationType')
161         elevated_token = _parse_int(elevated_token_val, -1) if
elevated_token_val is not None else None
162         if elevated_token == -1:
163             elevated_token = None
164
165         source_ip = str(ed.get('IpAddress') or ed.get('Worksta
tionName') or '')
166         if source_ip == '-':
167             source_ip = ''
168         source_port_val = ed.get('IpPort')
169         source_port = _parse_int(source_port_val, -1) if sourc
e_port_val is not None else None
170         if source_port == -1:
171             source_port = None
172
173         # Объект и статусы
174         object_name = (
175             ed.get('ObjectName')
176             or ed.get('ServiceName')
177             or ed.get('TaskName')
178             or ed.get('TargetName')
179             or ''
180         )
181         status_code = str(ed.get('Status') or ed.get('SubStatu
s') or ed.get('ExitStatus') or '')
182
183         # Формирование русскоязычного сообщения
184         message = self._build_human_message(
185             event_id=event_id,

```

```
186 |         subject_user=subject_user,
187 |         target_user=target_user,
188 |         process_name=process_name,
189 |         process_id=process_id,
190 |         parent_process_name=parent_process_name,
191 |         parent_process_id=parent_process_id,
192 |         command_line=command_line,
193 |         logon_type=logon_type,
194 |         source_ip=source_ip,
195 |         object_name=object_name,
196 |         status_code=status_code,
197 |         default_msg=event_dict.get('message', ''),
198 |     )
199 |
200 |     item = SecurityEventItem(
201 |         event_record_id=record_id,
202 |         event_id=event_id,
203 |         timestamp=timestamp,
204 |         created_at=now_epoch,
205 |         computer=computer,
206 |         channel=channel,
207 |         level=level,
208 |         subject_user=subject_user,
209 |         subject_domain=subject_domain,
210 |         subject_sid=subject_sid,
211 |         target_user=target_user,
212 |         target_domain=target_domain,
213 |         target_sid=target_sid,
214 |         process_id=process_id,
215 |         process_name=process_name,
216 |         parent_process_id=parent_process_id,
217 |         parent_process_name=parent_process_name,
218 |         command_line=command_line,
219 |         logon_id=logon_id,
220 |         logon_type=logon_type,
221 |         elevated_token=elevated_token,
222 |         source_ip=source_ip,
223 |         source_port=source_port,
```

```

224         object_name=object_name,
225         status_code=status_code,
226         message=message,
227         event_data=ed,
228         ingested_at=now_iso,
229     )
230
231     raw_item = None
232     if extract_raw and raw_xml:
233         raw_item = SecurityEventRaw(
234             event_record_id=record_id,
235             event_id=event_id,
236             timestamp=timestamp,
237             created_at=now_epoch,
238             channel=channel,
239             raw_xml=raw_xml,
240             ingested_at=now_iso,
241         )
242
243     return item, raw_item
244
245     def _build_human_message(
246         self,
247         event_id: int,
248         subject_user: str,
249         target_user: str,
250         process_name: str,
251         process_id: int,
252         parent_process_name: str,
253         parent_process_id: int,
254         command_line: str,
255         logon_type: Optional[int],
256         source_ip: str,
257         object_name: str,
258         status_code: str,
259         default_msg: str = '',
260     ) -> str:
261         """Сформировать понятное русскоязычное описание события

```



```

я. """
262         p_short = process_name.replace('\\', '/').split('/')[-1] if process_name else ''
263         parent_short = parent_process_name.replace('\\', '/').split('/')[-1] if parent_process_name else ''
264
265         if event_id == 4688:
266             user_part = f"пользователем {subject_user}" if subject_user and subject_user != '-' else ""
267             parent_part = f", родитель: {parent_short} (PID {parent_process_id})" if parent_short else ""
268             cmd_part = f" [{command_line[:120]}...]" if len(command_line) > 120 else (f" [{command_line}]" if command_line else "")
269             return f"Создание процесса: {p_short} (PID {process_id}) {user_part}{parent_part}{cmd_part}".strip()
270
271         if event_id == 4689:
272             status_part = f", код завершения {status_code}" if status_code else ""
273             return f"Завершение процесса: {p_short} (PID {process_id}){status_part}"
274
275         if event_id == 4624:
276             user = target_user or subject_user or 'Неизвестно'
277             type_str = f", тип входа {logon_type}" if logon_type is not None else ""
278             ip_str = f", IP: {source_ip}" if source_ip else ""
279             return f"Успешный вход пользователя: {user}{type_str}{ip_str}"
280
281         if event_id == 4625:
282             user = target_user or subject_user or 'Неизвестно'
283             status_str = f", статус ошибки {status_code}" if status_code else ""
284             ip_str = f", IP: {source_ip}" if source_ip else ""
285             return f"Неудачная попытка входа: {user}{status_str}{ip_str}"

```

```
286 |
287 |         if event_id in (4634, 4647):
288 |             user = target_user or subject_user or 'Пользовател
    | б'
289 |             return f"Выход пользователя из системы: {user}"
290 |
291 |         if event_id == 4672:
292 |             return f"Назначение привилегий администратора поль
    | зователю {subject_user}"
293 |
294 |         if event_id == 4697:
295 |             return f"Установка новой службы Windows: {object_n
    | ame} ({p_short})"
296 |
297 |         if event_id == 4698:
298 |             return f"Создание запланированной задачи: {object_
    | name} пользователем {subject_user}"
299 |
300 |         if event_id == 4702:
301 |             return f"Обновление запланированной задачи: {objec
    | t_name}"
302 |
303 |         if event_id == 4719:
304 |             return f"Изменение системной политики аудита безоп
    | асности администратором {subject_user}"
305 |
306 |         if event_id == 4720:
307 |             return f"Создание учетной записи пользователя: {ta
    | rget_user} (инициатор: {subject_user})"
308 |
309 |         if event_id == 4722:
310 |             return f"Включение учетной записи: {target_user}"
311 |
312 |         if event_id in (4723, 4724):
313 |             return f"Сброс/изменение пароля пользователя: {tar
    | get_user}"
314 |
315 |         if event_id == 4725:
```

```
316         return f"Отключение учетной записи пользователя: {  
target_user}"  
317  
318         if event_id == 4726:  
319             return f"Удаление учетной записи пользователя: {ta  
rget_user}"  
320  
321         if event_id == 4732:  
322             return f"Добавление пользователя {target_user} в г  
руппу безопасности {object_name}"  
323  
324         if event_id == 4740:  
325             return f"Блокировка учетной записи пользователя: {  
target_user}"  
326  
327         if event_id == 4768:  
328             return f"Запрос Kerberos TGT для {target_user or s  
ubject_user}"  
329  
330         if event_id == 4771:  
331             return f"Сбой пре-аутентификации Kerberos для {tar  
get_user} (Код: {status_code})"  
332  
333         if event_id == 4776:  
334             return f"Проверка учетных данных NTLM для {target_  
user}"  
335  
336         if event_id == 1102:  
337             return f"ВНИМАНИЕ: Журнал аудита безопасности был  
очищен пользователем {subject_user}!"  
338  
339         if default_msg:  
340             return default_msg  
341  
342         desc = self.EVENT_DESCRIPTIONS.get(event_id, f'Событие  
безопасности {event_id}')  
343         return f"{desc} (субъект: {subject_user or target_user  
or '-'})"
```

```
344 |
345 |     def _parse_event_data_xml(self, raw_xml: str) -> Dict[str,
    |     str]:
346 |         """Извлечь структурированные пары ключ-значение из Eve
    | ntData XML."""
347 |         res: Dict[str, str] = {}
348 |         try:
349 |             root = ET.fromstring(raw_xml)
350 |             for elem in root.iter():
351 |                 tag = elem.tag.split('}')[1]
352 |                 if tag in ('Data', 'string', 'Value') and elem
    | .text:
353 |                     name_attr = elem.attrib.get('Name')
354 |                     if name_attr:
355 |                         res[name_attr] = elem.text.strip()
356 |         except Exception:
357 |             pass
358 |         return res
359 |
360 |
361 | __all__ = ['SecurityEventNormalizer', '_parse_int']
362 |
```

sensors.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - Sensors
4 # =====
5 # Description:
6 #   Движок считывания показаний температурных датчиков, напряж
   ений и вентиляторов.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.sensors import get_hardware_
   sensors
11 #
12 #       res = get_hardware_sensors()
13 #
14 # File: sensors.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-04 07:00:00
20 # =====
21
22 from __future__ import annotations
23 """Движок считывания показаний аппаратных сенсоров, сетевых ме
   трик и скорости FAST.com."""
24
25 import json
26 import os
27 import subprocess
28 from typing import List, Optional
29 try:
30     import wmi
```

```

31 |     _WMI_AVAILABLE = True
32 | except ImportError:
33 |     _WMI_AVAILABLE = False
34 | from logger import logger
35 | from .models import HardwareSensor
36 | from .internet_speed import InternetSpeedSensor
37 | from .sensor_registry import SensorProvider
38 |
39 | # NOTE: GPU temperature, power, and fan sensors are sourced fr
    | om:
40 | # 1. GpuProber._probe_nvidia() -> HardwareMonitor.get_gpu_me
    | trics()
41 | # 2. sensor_collector.extract_sensor_readings() normalizes G
    | puMetrics to sensor_id format
42 |
43 | def _probe_acpi_thermal_zones() -> List[HardwareSensor]:
44 |     """Сбор данных температурных зон Windows ACPI через CIM с
    | fallback на WMI.
45 |
46 |     Returns:
47 |         List[HardwareSensor]: Список показаний термальных зон
    | ACPI.
48 |     """
49 |     sensors: List[HardwareSensor] = []
50 |     if os.name != 'nt':
51 |         return sensors
52 |
53 |     # 1. Первичный сбор через CIM (Get-CimInstance)
54 |     try:
55 |         cmd = [
56 |             'powershell.exe', '-NoProfile', '-NonInteractive',
    | '-Command',
57 |             'Get-CimInstance -Namespace root/wmi -ClassName MS
    | Acpi_ThermalZoneTemperature -ErrorAction SilentlyContinue | '
58 |             'Select-Object InstanceName, CurrentTemperature |
    | ConvertTo-Json -Compress'
59 |         ]
60 |         res = subprocess.run(cmd, capture_output=True, text=Tr

```

```

ue, timeout=5)
61         if res.returncode == 0 and res.stdout.strip():
62             raw_data = json.loads(res.stdout.strip())
63             items = [raw_data] if isinstance(raw_data, dict) e
lse (raw_data if isinstance(raw_data, list) else [])
64             for idx, zone in enumerate(items):
65                 raw_temp = zone.get('CurrentTemperature')
66                 if raw_temp and raw_temp > 0:
67                     celsius = round(float(raw_temp) / 10.0 - 2
73.15, 1)
68                     if -20.0 <= celsius <= 125.0:
69                         zone_name = zone.get('InstanceName') o
r f'Thermal Zone {idx}'
70                         sensor = HardwareSensor(
71                             sensor_id=f'acpi_thermal_{idx}',
72                             name=f'ACPI {zone_name}',
73                             category='temperature',
74                             value=celsius,
75                             unit='°C'
76                         )
77                         sensor.provider = SensorProvider.CIM_S
YSTEM
78                         sensors.append(sensor)
79                     if sensors:
80                         return sensors
81             except Exception as ex:
82                 logger.debug(f'CIM ACPI thermal probe skipped or error
: {ex}')
83
84             # 2. Fallback / слой совместимости через WMI
85             try:
86                 import wmi
87                 import pythoncom
88                 pythoncom.CoInitialize()
89                 w = wmi.WMI(namespace='root\\wmi')
90                 for idx, zone in enumerate(w.MSAcpi_ThermalZoneTempera
ture()):
91                     raw_temp = getattr(zone, 'CurrentTemperature', Non

```

```

e)
92         if raw_temp and raw_temp > 0:
93             celsius = round(float(raw_temp) / 10.0 - 273.1
5, 1)
94             if -20.0 <= celsius <= 125.0:
95                 zone_name = getattr(zone, 'InstanceName',
f'Thermal Zone {idx}')
96                 sensor = HardwareSensor(
97                     sensor_id=f'acpi_thermal_{idx}',
98                     name=f'ACPI {zone_name}',
99                     category='temperature',
100                     value=celsius,
101                     unit='°C'
102                 )
103                 sensor.provider = SensorProvider.WMI_FALLB
ACK
104                 sensors.append(sensor)
105     except Exception as ex:
106         logger.debug(f'WMI ACPI thermal fallback probe skipped
: {ex}')
107
108     return sensors
109
110 def _probe_network_sensors() -> List[HardwareSensor]:
111     """Probe network interface metrics using Windows native to
ols.
112
113     Returns:
114         List[HardwareSensor]: Network interface telemetry as s
ensors with provider marker.
115     """
116     sensors: List[HardwareSensor] = []
117     try:
118         import psutil
119         current_net_io = psutil.net_io_counters(pernic=True)
120         if current_net_io:
121             for name, stats in current_net_io.items():
122                 # Create sensors with provider marker

```



```
123         s1 = HardwareSensor(sensor_id=f'net_{name}_byt
    es_recv', name=f'Network {name} Bytes Received', category='net
    work', value=round(stats.bytes_recv, 2), unit='B')
124         s1.provider = SensorProvider.NETWORK_SENSOR
125         sensors.append(s1)
126
127         s2 = HardwareSensor(sensor_id=f'net_{name}_byt
    es_sent', name=f'Network {name} Bytes Sent', category='network
    ', value=round(stats.bytes_sent, 2), unit='B')
128         s2.provider = SensorProvider.NETWORK_SENSOR
129         sensors.append(s2)
130
131         s3 = HardwareSensor(sensor_id=f'net_{name}_pac
    kets_recv', name=f'Network {name} Packets Received', category=
    'network', value=stats.packets_recv, unit='pkts')
132         s3.provider = SensorProvider.NETWORK_SENSOR
133         sensors.append(s3)
134
135         s4 = HardwareSensor(sensor_id=f'net_{name}_pac
    kets_sent', name=f'Network {name} Packets Sent', category='net
    work', value=stats.packets_sent, unit='pkts')
136         s4.provider = SensorProvider.NETWORK_SENSOR
137         sensors.append(s4)
138
139         s5 = HardwareSensor(sensor_id=f'net_{name}_err
    ors_recv', name=f'Network {name} Errors Received', category='n
    etwork', value=stats.errin, unit='errors')
140         s5.provider = SensorProvider.NETWORK_SENSOR
141         sensors.append(s5)
142
143         s6 = HardwareSensor(sensor_id=f'net_{name}_err
    ors_sent', name=f'Network {name} Errors Sent', category='netwo
    rk', value=stats.errout, unit='errors')
144         s6.provider = SensorProvider.NETWORK_SENSOR
145         sensors.append(s6)
146
147         s7 = HardwareSensor(sensor_id=f'net_{name}_dro
    pped_recv', name=f'Network {name} Dropped Received', category=
```

```

    'network', value=stats.dropin, unit='pkts')
148         s7.provider = SensorProvider.NETWORK_SENSOR
149         sensors.append(s7)
150
151         s8 = HardwareSensor(sensor_id=f'net_{name}_dro
pped_sent', name=f'Network {name} Dropped Sent', category='net
work', value=stats.dropout, unit='pkts')
152         s8.provider = SensorProvider.NETWORK_SENSOR
153         sensors.append(s8)
154     except Exception as ex:
155         logger.debug(f'Network sensor probe failed: {ex}')
156     return sensors
157
158 def _probe_internet_speed_sensors() -> List[HardwareSensor]:
159     """Собрать показания скорости интернета и характеристик се
ти FAST.com.
160
161     Returns:
162         List[HardwareSensor]: Метрики скорости интернета и зад
ержки с маркером провайдера.
163     """
164     sensors: List[HardwareSensor] = []
165     try:
166         sensor = InternetSpeedSensor()
167         metrics = sensor.measure_internet_speed()
168
169         s1 = HardwareSensor(
170             sensor_id='internet_ping',
171             name='Internet Ping',
172             category='network',
173             value=round(metrics.get('ping_ms', 0.0), 2),
174             unit='ms'
175         )
176         s1.provider = SensorProvider.INTERNET_SPEED
177         sensors.append(s1)
178
179         s2 = HardwareSensor(
180             sensor_id='internet_download',

```

```

181         name='Internet Download Speed',
182         category='network',
183         value=round(metrics.get('download_mbps', 0.0), 2),
184         unit='Mbps'
185     )
186     s2.provider = SensorProvider.INTERNET_SPEED
187     sensors.append(s2)
188
189     s3 = HardwareSensor(
190         sensor_id='internet_upload',
191         name='Internet Upload Speed',
192         category='network',
193         value=round(metrics.get('upload_mbps', 0.0), 2),
194         unit='Mbps'
195     )
196     s3.provider = SensorProvider.INTERNET_SPEED
197     sensors.append(s3)
198
199     s4 = HardwareSensor(
200         sensor_id='internet_bufferbloat',
201         name='Internet Bufferbloat',
202         category='network',
203         value=round(metrics.get('bufferbloat_ms', 0.0), 2)
204         ,
205         unit='ms'
206     )
207     s4.provider = SensorProvider.INTERNET_SPEED
208     sensors.append(s4)
209
210     s5 = HardwareSensor(
211         sensor_id='internet_latency_loaded',
212         name='Internet Loaded Latency',
213         category='network',
214         value=round(metrics.get('latency_loaded_ms', 0.0),
215         2),
216         unit='ms'
217     )
218     s5.provider = SensorProvider.INTERNET_SPEED

```

```

217         sensors.append(s5)
218     except Exception as ex:
219         logger.debug(f'Ошибка опроса сенсора скорости интернет
a: {ex}')
220     return sensors
221
222 def _probe_storage_sensors() -> List[HardwareSensor]:
223     """Сбор сенсоров температуры и надежности накопителей.
224
225     :return: Список объектов HardwareSensor для дисков с provi
der marker.
226     """
227     sensors: List[HardwareSensor] = []
228     if os.name != 'nt':
229         return sensors
230     try:
231         from apps.windows.sdk.modules.storage_manager.core.win
dows_storage_sensor import WindowsStorageSensor
232         sensor = WindowsStorageSensor(timeout_sec=30)
233         disks = sensor.get_physical_disks()
234         for disk in disks:
235             disk_slug = disk.device_id.replace('\\', '_').repl
ace('.', '_')
236             if disk.temperature_c is not None:
237                 s1 = HardwareSensor(sensor_id=f'disk_temp_{dis
k_slug}', name=f'{disk.friendly_name} Temperature', category='
temperature', value=float(disk.temperature_c), unit='°C')
238                 s1.provider = SensorProvider.WINDOWS_STORAGE
239                 sensors.append(s1)
240             if disk.wear_percentage is not None:
241                 s2 = HardwareSensor(sensor_id=f'disk_wear_{dis
k_slug}', name=f'{disk.friendly_name} Wear', category='wear',
value=float(disk.wear_percentage), unit='%')
242                 s2.provider = SensorProvider.WINDOWS_STORAGE
243                 sensors.append(s2)
244     except Exception as ex:
245         logger.debug(f'Ошибка сбора сенсоров накопителей: {ex}
')

```

```

246     return sensors
247
248 def _probe_cloud_storage_sensors() -> List[HardwareSensor]:
249     """Сбор сенсоров свободного и занятого места для облачного
    хранилища OneDrive.
250
251     Returns:
252         List[HardwareSensor]: Список сенсоров для дисков OneDr
ive с provider marker.
253     """
254     import shutil
255     sensors: List[HardwareSensor] = []
256     od_paths: List[str] = []
257     for env_var in ('OneDrive', 'OneDriveConsumer', 'OneDriveC
ommerical'):
258         val = os.environ.get(env_var)
259         if val and os.path.exists(val) and (val not in od_path
s):
260             od_paths.append(val)
261     if not od_paths:
262         home = os.path.expanduser('~')
263         candidate = os.path.join(home, 'OneDrive')
264         if os.path.exists(candidate):
265             od_paths.append(candidate)
266     if os.name == 'nt':
267         try:
268             import winreg
269             for subkey in ('Software\\Microsoft\\OneDrive\\Acc
ounts\\Personal', 'Software\\Microsoft\\OneDrive\\Accounts\\Bu
siness1', 'Software\\Microsoft\\OneDrive'):
270                 try:
271                     with winreg.OpenKey(winreg.HKEY_CURRENT_US
ER, subkey) as k:
272                         val, _ = winreg.QueryValueEx(k, 'UserF
older')
273                         if val and os.path.exists(val) and (va
l not in od_paths):
274                             od_paths.append(val)

```

```

275         except OSError:
276             pass
277     except Exception:
278         pass
279     for idx, od_path in enumerate(od_paths):
280         try:
281             usage = shutil.disk_usage(od_path)
282             free_gb = round(usage.free / 1024 ** 3, 2)
283             total_gb = round(usage.total / 1024 ** 3, 2)
284             used_pct = round(usage.used / max(usage.total, 1)
285 * 100.0, 1)
286             slug = f'_{idx + 1}' if len(od_paths) > 1 else ''
287             label_suffix = f' ({os.path.basename(od_path)})' i
288 f len(od_paths) > 1 else ''
289
290             s1 = HardwareSensor(sensor_id=f'onedrive{slug}_fre
291 e_gb', name=f'OneDrive Free Space{label_suffix}', category='st
292 orage', value=free_gb, unit='GB')
293             s1.provider = SensorProvider.SENSOR_COLLECTOR
294             sensors.append(s1)
295
296             s2 = HardwareSensor(sensor_id=f'onedrive{slug}_use
297 d_percent', name=f'OneDrive Used Percent{label_suffix}', categ
298 ory='storage', value=used_pct, unit='%')
299             s2.provider = SensorProvider.SENSOR_COLLECTOR
300             sensors.append(s2)
301
302             s3 = HardwareSensor(sensor_id=f'onedrive{slug}_tot
303 al_gb', name=f'OneDrive Total Capacity{label_suffix}', categor
304 y='storage', value=total_gb, unit='GB')
305             s3.provider = SensorProvider.SENSOR_COLLECTOR
306             sensors.append(s3)
307         except Exception as ex:
308             logger.debug(f'Ошибка сбора сенсора для OneDrive (
309 {od_path}): {ex}')
310     return sensors
311
312 def get_hardware_sensors() -> List[HardwareSensor]:

```

```
304 |     """Retrieve all available hardware sensors across all supp
    | orted backends.
305 |
306 |     NOTE: GPU sensors (temperature, power, fan) are NOT includ
    | ed here to avoid
307 |     duplication. They are collected via GpuProber in HardwareM
    | onitor.get_gpu_metrics()
308 |     and then normalized by SensorCollector.extract_sensor_read
    | ings().
309 |
310 |     Returns:
311 |         List[HardwareSensor]: Consolidated list of active hard
    | ware sensors.
312 |     """
313 |     all_sensors: List[HardwareSensor] = []
314 |     # GPU sensors excluded - collected via GpuProber to avoid
    | duplicate nvidia-smi calls
315 |     all_sensors.extend(_probe_acpi_thermal_zones())
316 |     all_sensors.extend(_probe_storage_sensors())
317 |     all_sensors.extend(_probe_cloud_storage_sensors())
318 |     all_sensors.extend(_probe_network_sensors())
319 |     all_sensors.extend(_probe_internet_speed_sensors())
320 |     return all_sensors
```

sensor_collector.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - Sensor
  Collector
4 # =====
5 # Description:
6 #   Сбор данных со всех доступных сенсоров и аппаратных провай
  деров.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.sensor_collector import Sens
    orCollector
11 #
12 #       service = SensorCollector()
13 #
14 # File: sensor_collector.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-06 08:10:00
20 # =====
21
22 from __future__ import annotations
23 """Сбор данных со всех доступных сенсоров и аппаратных провай
  деров."""
24
25 import re
26 import threading
27 import time
28 from datetime import datetime, timezone
29 from typing import Any, Dict, List, Optional
```



```

30 try:
31     from logger import logger
32 except ImportError:
33     from logger import logger
34 from .telemetry_config import TelemetryConfigManager
35 from .sensor_registry import SensorDeduplicator, SensorProvide
36 r
37 try:
38     from apps.windows.sdk.modules.hardware.hardware_monitor im
39     port HardwareMonitor
40     from apps.windows.sdk.modules.hardware.lhm_service import
41     LhmService
42     from .internet_speed import InternetSpeedSensor
43 except ImportError as e:
44     logger.warning(f'Не удалось импортировать зависимости сенс
45     оров: {e}')
46     HardwareMonitor = None
47     LhmService = None
48     InternetSpeedSensor = None
49
50 class SensorCollector:
51     """Коллектор данных со всех сенсоров с нормализованной схе
52     мой датчиков."""
53
54     def __init__(
55         self,
56         config_manager: Optional[TelemetryConfigManager] = Non
57         e,
58         lhm_service: Optional[Any] = None,
59     ) -> None:
60         """Инициализирует коллектор сенсоров.
61
62         Args:
63             config_manager: Менеджер конфигурации для настроек
64             и сенсоров.
65             lhm_service: Сервис LibreHardwareMonitor (опционал
66             ьно).
67         """

```

```

60         self.config_manager = config_manager
61         self._lhm_service = lhm_service or (LhmService() if Lh
mService else None)
62         self._hardware_monitor = HardwareMonitor() if Hardware
Monitor else None
63         self._internet_speed_sensor = InternetSpeedSensor() if
InternetSpeedSensor else None
64         self._last_hardware_snapshot: Optional[Dict[str, Any]]
= None
65         self._lock = threading.Lock()
66
67     def _get_hardware_data(self) -> Dict[str, Any]:
68         """Получает данные от HardwareMonitor.
69
70         Returns:
71             Dict[str, Any]: Данные HardwareMonitor.
72         """
73         if self._hardware_monitor is None:
74             return {}
75         try:
76             snapshot = self._hardware_monitor.get_snapshot(inc
lude_smart=True)
77             return snapshot.to_dict()
78         except Exception as e:
79             logger.error(f'Ошибка получения данных от Hardware
Monitor: {e}')
80             return {}
81
82     def _get_internet_speed_data(self) -> Dict[str, Any]:
83         """Получает данные о скорости интернета и задержке чер
ез FAST.com.
84
85         Returns:
86             Dict[str, Any]: Данные скорости интернета (downloa
d, upload, ping, bufferbloat, loaded latency).
87         """
88         if self._internet_speed_sensor is None:
89             return {'available': False, 'error': 'InternetSpee

```

```

    dSensor not available'}
90         try:
91             metrics = self._internet_speed_sensor.measure_inte
internet_speed()
92             return {
93                 'available': metrics.get('available', True),
94                 'ping_ms': metrics.get('ping_ms', 0.0),
95                 'download_mbps': metrics.get('download_mbps',
0.0),
96                 'upload_mbps': metrics.get('upload_mbps', 0.0)
,
97                 'latency_unloaded_ms': metrics.get('latency_un
loaded_ms', metrics.get('ping_ms', 0.0)),
98                 'latency_loaded_ms': metrics.get('latency_load
ed_ms', 0.0),
99                 'bufferbloat_ms': metrics.get('bufferbloat_ms'
, 0.0),
100                 'provider': metrics.get('provider', 'fast.com'
),
101                 'server': metrics.get('server', ''),
102                 'ip': metrics.get('ip', ''),
103                 'isp': metrics.get('isp', ''),
104             }
105         except Exception as e:
106             logger.error(f'Ошибка получения данных скорости ин
тернета: {e}')
107             return {'available': False, 'error': str(e)}
108
109     def _detect_provider(self, sensor_id: str, category: str,
hardware_type: str) -> SensorProvider:
110         """Определяет провайдера сенсора на основе его ID, кат
егории и источника.
111
112         Args:
113             sensor_id: Идентификатор сенсора.
114             category: Категория сенсора (Temperatures, Load, e
tc.).
115             hardware_type: Тип оборудования (cpu, gpu, etc.).

```

```
116 |
117 |     Returns:
118 |         SensorProvider: Провайдер сенсора.
119 |     """
120 |     # GPU sensors from GpuProber
121 |     if sensor_id.startswith('gpu_') and ('_temp' in sensor
122 | _id or '_load' in sensor_id):
123 |         return SensorProvider.GPU_PROBER
124 |
125 |     # Storage sensors
126 |     if sensor_id.startswith('disk_') or sensor_id.startswi
127 | th('storage_'):
128 |         return SensorProvider.WINDOWS_STORAGE
129 |
130 |     # Internet speed sensors
131 |     if sensor_id.startswith('internet_'):
132 |         return SensorProvider.INTERNET_SPEED
133 |
134 |     # Network sensors
135 |     if sensor_id.startswith('net_'):
136 |         return SensorProvider.NETWORK_SENSOR
137 |
138 |     # CPU/RAM utilization from sensor_collector
139 |     if sensor_id in ('cpu_util_total', 'ram_util_pct', 'sw
140 | ap_util_pct'):
141 |         return SensorProvider.SENSOR_COLLECTOR
142 |
143 |     if sensor_id.startswith('cpu_core_') and '_load' in se
144 | nsor_id:
145 |         return SensorProvider.SENSOR_COLLECTOR
146 |
147 |     if sensor_id.startswith('cpu_freq'):
148 |         return SensorProvider.SENSOR_COLLECTOR
149 |
150 |     # OneDrive sensors
151 |     if sensor_id.startswith('onedrive_'):
152 |         return SensorProvider.SENSOR_COLLECTOR
```

```

150         # Network rates
151         if sensor_id in ('network_sent_rate', 'network_recv_rate'):
152             return SensorProvider.SENSOR_COLLECTOR
153
154         # Storage usage
155         if sensor_id.startswith('storage_usage_') or sensor_id
156         in ('storage_read_rate', 'storage_write_rate'):
157             return SensorProvider.SENSOR_COLLECTOR
158
159         # Default based on category
160         if category in ('Temperatures', 'temperature'):
161             return SensorProvider.ACPI_THERMAL
162
163         return SensorProvider.SENSOR_COLLECTOR
164
165     def extract_sensor_readings(
166         self, hardware_data: Dict[str, Any], *args: Any, internet_data: Optional[Dict[str, Any]] = None, **kwargs: Any
167     ) -> List[Dict[str, Any]]:
168         """Извлекает нормализованный список всех текущих показаний датчиков с дедупликацией.
169
170         Каждый элемент содержит: id, hardware_name, hardware_type, sensor_category,
171         sensor_name, unit, value, _provider.
172
173         Args:
174             hardware_data: Данные HardwareMonitor.
175             *args: Опциональные extra_sensors_data и/или internet_data.
176             internet_data: Данные скорости интернета (если переданы явно).
177
178         Returns:
179             List[Dict[str, Any]]: Нормализованный и дедуплицированный список сенсоров.
180         """

```

```

180         extra_sensors_data = None
181         if len(args) == 1:
182             if isinstance(args[0], dict) and ("ping_ms" in arg
s[0] or "download_mbps" in args[0] or "available" in args[0]):
183                 internet_data = args[0]
184             else:
185                 extra_sensors_data = args[0]
186         elif len(args) >= 2:
187             extra_sensors_data = args[0]
188             internet_data = args[1]
189
190         raw_readings: List[Dict[str, Any]] = []
191
192         # Дополнительные внешние сенсоры
193         if extra_sensors_data and isinstance(extra_sensors_dat
a, dict) and "sensors" in extra_sensors_data:
194             for s in extra_sensors_data["sensors"]:
195                 if isinstance(s, dict):
196                     sensor_id = s.get("id", s.get("sensor_name
", "sensor"))
197                     category = s.get("sensor_category", "Metri
c")
198                     hardware_type = s.get("hardware_type", "se
nsor")
199                     provider = self._detect_provider(str(senso
r_id), category, hardware_type)
200                     raw_readings.append({
201                         "id": sensor_id,
202                         "hardware_name": s.get("hardware_name"
, "Hardware"),
203                         "hardware_type": hardware_type,
204                         "sensor_category": category,
205                         "sensor_name": s.get("sensor_name", "S
ensor"),
206                         "unit": s.get("unit", ""),
207                         "value": s.get("value_num", s.get("val
ue", 0.0)),
208                         "_provider": provider,

```

```

209         })
210
211         if 'cpu' in hardware_data:
212             cpu = hardware_data['cpu']
213             cpu_name = cpu.get('model') or cpu.get('model_name
214             ') or 'Intel Core Processor'
215             raw_readings.append({
216                 'id': 'cpu_util_total',
217                 'hardware_name': cpu_name,
218                 'hardware_type': 'cpu',
219                 'sensor_category': 'Load',
220                 'sensor_name': 'CPU Total',
221                 'unit': '%',
222                 'value': cpu.get('utilization_pct', 0.0),
223                 '_provider': SensorProvider.SENSOR_COLLECTOR
224             })
225             if 'per_core_pct' in cpu and isinstance(cpu['per_c
226             ore_pct'], list):
227                 for idx, core_val in enumerate(cpu['per_core_p
228                 ct']):
229                     raw_readings.append({
230                         'id': f'cpu_core_{idx}_load',
231                         'hardware_name': cpu_name,
232                         'hardware_type': 'cpu',
233                         'sensor_category': 'Load',
234                         'sensor_name': f'CPU Core #{idx}',
235                         'unit': '%',
236                         'value': core_val,
237                         '_provider': SensorProvider.SENSOR_COL
238                         LECTOR
239                     })
240             if 'frequency_current_mhz' in cpu and cpu['frequen
241             cy_current_mhz']:
242                 raw_readings.append({
243                     'id': 'cpu_freq_current',
244                     'hardware_name': cpu_name,
245                     'hardware_type': 'cpu',
246                     'sensor_category': 'Clocks',

```

```

242         'sensor_name': 'CPU Core Frequency',
243         'unit': 'MHz',
244         'value': cpu['frequency_current_mhz'],
245         '_provider': SensorProvider.SENSOR_COLLECT
OR
246     })
247
248     if 'memory' in hardware_data:
249         ram = hardware_data['memory']
250         raw_readings.append({
251             'id': 'ram_util_pct',
252             'hardware_name': 'System Memory',
253             'hardware_type': 'memory',
254             'sensor_category': 'Load',
255             'sensor_name': 'Memory Used',
256             'unit': '%',
257             'value': ram.get('utilization_pct', 0.0),
258             '_provider': SensorProvider.SENSOR_COLLECTOR
259         })
260         if 'swap_utilization_pct' in ram and ram['swap_utilization_pct'] is not None:
261             raw_readings.append({
262                 'id': 'swap_util_pct',
263                 'hardware_name': 'System Memory',
264                 'hardware_type': 'memory',
265                 'sensor_category': 'Load',
266                 'sensor_name': 'Swap Used',
267                 'unit': '%',
268                 'value': ram['swap_utilization_pct'],
269                 '_provider': SensorProvider.SENSOR_COLLECT
OR
270             })
271
272     # GPU metrics: temperature, load, memory, and subsystem engines
273     if 'gpus' in hardware_data and isinstance(hardware_data['gpus'], list):
274         for gpu in hardware_data['gpus']:

```



```

275 |         gpu_name = gpu.get('name', 'GPU')
276 |         gpu_idx = gpu.get('index', 0)
277 |         # GPU temperature - use hardware_data from Gpu
    | Prober
278 |         if 'temperature_gpu_c' in gpu and gpu['tempera
    | ture_gpu_c'] is not None:
279 |             raw_readings.append({
280 |                 'id': f'gpu_{gpu_idx}_temp',
281 |                 'hardware_name': gpu_name,
282 |                 'hardware_type': 'gpu',
283 |                 'sensor_category': 'Temperatures',
284 |                 'sensor_name': 'GPU Core Temperature',
285 |                 'unit': '°C',
286 |                 'value': gpu['temperature_gpu_c'],
287 |                 '_provider': SensorProvider.GPU_PROBER
288 |             })
289 |         # GPU load
290 |         if 'utilization_gpu_pct' in gpu and gpu['utili
    | zation_gpu_pct'] is not None:
291 |             raw_readings.append({
292 |                 'id': f'gpu_{gpu_idx}_load',
293 |                 'hardware_name': gpu_name,
294 |                 'hardware_type': 'gpu',
295 |                 'sensor_category': 'Load',
296 |                 'sensor_name': 'GPU Core Load',
297 |                 'unit': '%',
298 |                 'value': gpu['utilization_gpu_pct'],
299 |                 '_provider': SensorProvider.GPU_PROBER
300 |             })
301 |         # GPU VRAM / Memory
302 |         if 'memory_used_mb' in gpu and gpu['memory_use
    | d_mb'] is not None:
303 |             raw_readings.append({
304 |                 'id': f'gpu_{gpu_idx}_mem_used',
305 |                 'hardware_name': gpu_name,
306 |                 'hardware_type': 'gpu',
307 |                 'sensor_category': 'Data',
308 |                 'sensor_name': 'GPU Memory Used',

```

```

309         'unit': 'MB',
310         'value': float(gpu['memory_used_mb']),
311         '_provider': SensorProvider.GPU_PROBER
312     })
313     if 'memory_total_mb' in gpu and gpu['memory_to
tal_mb'] is not None:
314         raw_readings.append({
315             'id': f'gpu_{gpu_idx}_mem_total',
316             'hardware_name': gpu_name,
317             'hardware_type': 'gpu',
318             'sensor_category': 'Data',
319             'sensor_name': 'GPU Memory Total',
320             'unit': 'MB',
321             'value': float(gpu['memory_total_mb'])
322         },
323         '_provider': SensorProvider.GPU_PROBER
324     })
325     # GPU Subsystem Engines (3D, Video Decode, Vid
eo Processing, Copy, etc.)
326     engines = gpu.get('engines')
327     if engines and isinstance(engines, dict):
328         for eng_name, eng_val in engines.items():
329             clean_id = re.sub(r'^[a-zA-Z0-9]+', '_',
330             ', eng_name.lower()).strip('_')
331             raw_readings.append({
332                 'id': f'gpu_{gpu_idx}_eng_{clean_i
d}',
333                 'hardware_name': gpu_name,
334                 'hardware_type': 'gpu',
335                 'sensor_category': 'Load',
336                 'sensor_name': eng_name,
337                 'unit': '%',
338                 'value': float(eng_val),
339                 '_provider': SensorProvider.GPU_PR
OBER
340             })
341     if 'storage' in hardware_data:

```

```

341         storage = hardware_data['storage']
342         if 'partitions' in storage and isinstance(storage[
|'partitions'], list):
343             for part in storage['partitions']:
344                 dev = part.get('device', 'Disk')
345                 raw_readings.append({
346                     'id': f"storage_usage_{dev.replace(':', '
|', ' ').replace(chr(92), ' ')}",
347                     'hardware_name': f'Storage ({dev})',
348                     'hardware_type': 'storage',
349                     'sensor_category': 'Load',
350                     'sensor_name': f'Used Space {dev}',
351                     'unit': '%',
352                     'value': part.get('utilization_pct', 0
|.0),
353                     '_provider': SensorProvider.SENSOR_COL
|LECTOR
354                 })
355         if 'io_rates' in storage and storage['io_rates']:
356             io = storage['io_rates']
357             raw_readings.append({
358                 'id': 'storage_read_rate',
359                 'hardware_name': 'Storage',
360                 'hardware_type': 'storage',
361                 'sensor_category': 'Throughput',
362                 'sensor_name': 'Disk Read Rate',
363                 'unit': 'B/s',
364                 'value': io.get('read_bytes_sec', 0.0),
365                 '_provider': SensorProvider.SENSOR_COLLECT
|OR
366             })
367             raw_readings.append({
368                 'id': 'storage_write_rate',
369                 'hardware_name': 'Storage',
370                 'hardware_type': 'storage',
371                 'sensor_category': 'Throughput',
372                 'sensor_name': 'Disk Write Rate',
373                 'unit': 'B/s',

```

```

374         'value': io.get('write_bytes_sec', 0.0),
375         '_provider': SensorProvider.SENSOR_COLLECT
OR
376     })
377
378     # Network metrics: aggregated throughput rates
379     # Note: Detailed per-interface stats are collected by
    _probe_network_sensors() in sensors.py
380     if 'network' in hardware_data:
381         net = hardware_data['network']
382         raw_readings.append({
383             'id': 'network_sent_rate',
384             'hardware_name': 'Network',
385             'hardware_type': 'network',
386             'sensor_category': 'Throughput',
387             'sensor_name': 'Network Upload Speed',
388             'unit': 'B/s',
389             'value': net.get('bytes_sent_sec', 0.0),
390             '_provider': SensorProvider.SENSOR_COLLECTOR
391         })
392         raw_readings.append({
393             'id': 'network_recv_rate',
394             'hardware_name': 'Network',
395             'hardware_type': 'network',
396             'sensor_category': 'Throughput',
397             'sensor_name': 'Network Download Speed',
398             'unit': 'B/s',
399             'value': net.get('bytes_recv_sec', 0.0),
400             '_provider': SensorProvider.SENSOR_COLLECTOR
401         })
402
403     # Internet speed metrics
404     if internet_data and internet_data.get('available'):
405         raw_readings.append({
406             'id': 'internet_download_speed',
407             'hardware_name': 'Internet',
408             'hardware_type': 'network',
409             'sensor_category': 'Throughput',

```

```

410         'sensor_name': 'Internet Download Bandwidth',
411         'unit': 'Mbps',
412         'value': internet_data.get('download_mbps', 0.
0),
413         '_provider': SensorProvider.INTERNET_SPEED
414     })
415     raw_readings.append({
416         'id': 'internet_upload_speed',
417         'hardware_name': 'Internet',
418         'hardware_type': 'network',
419         'sensor_category': 'Throughput',
420         'sensor_name': 'Internet Upload Bandwidth',
421         'unit': 'Mbps',
422         'value': internet_data.get('upload_mbps', 0.0)
,
423         '_provider': SensorProvider.INTERNET_SPEED
424     })
425     raw_readings.append({
426         'id': 'internet_ping',
427         'hardware_name': 'Internet',
428         'hardware_type': 'network',
429         'sensor_category': 'Throughput',
430         'sensor_name': 'Internet Ping Latency',
431         'unit': 'ms',
432         'value': internet_data.get('ping_ms', 0.0),
433         '_provider': SensorProvider.INTERNET_SPEED
434     })
435     raw_readings.append({
436         'id': 'internet_bufferbloat',
437         'hardware_name': 'Internet',
438         'hardware_type': 'network',
439         'sensor_category': 'Throughput',
440         'sensor_name': 'Internet Bufferbloat',
441         'unit': 'ms',
442         'value': internet_data.get('bufferbloat_ms', 0
.0),
443         '_provider': SensorProvider.INTERNET_SPEED
444     })

```

```

445         raw_readings.append({
446             'id': 'internet_latency_loaded',
447             'hardware_name': 'Internet',
448             'hardware_type': 'network',
449             'sensor_category': 'Throughput',
450             'sensor_name': 'Internet Loaded Latency',
451             'unit': 'ms',
452             'value': internet_data.get('latency_loaded_ms'
    , 0.0),
453             '_provider': SensorProvider.INTERNET_SPEED
454         })
455
456     # Дедупликация базовых показаний по приоритету провайде
    ера
457     deduplicator = SensorDeduplicator()
458     unique_readings: List[Dict[str, Any]] = []
459
460     for reading in raw_readings:
461         sensor_id = str(reading.get('id', ''))
462         value = reading.get('value', 0.0)
463         provider = reading.get('_provider', SensorProvider
    .SENSOR_COLLECTOR)
464
465         if deduplicator.add(sensor_id, provider, value):
466             unique_readings.append(reading)
467
468     # Обогащение из LibreHardwareMonitor (добавляются ТОЛЬ
    КО сенсоры, отсутствующие в базовом пуле CIM/WMI)
469     if self._lhm_service and self._lhm_service.is_running(
    ):
470         try:
471             lhm_items = self._lhm_service.get_flattened_se
    nsors()
472             existing_ids = {str(r.get('id', '')) for r in
    unique_readings}
473             for item in lhm_items:
474                 hw_name = item.get("hardware_name", "Hardw
    are")

```

```

475         hw_type = item.get("hardware_type", "senso
r")
476         cat = item.get("sensor_category", "General
")
477         s_name = item.get("sensor_name", "Sensor")
478
479         if "Distance to TjMax" in s_name:
480             continue
481
482         val_num = item.get("value_num")
483         if val_num is None:
484             continue
485
486         vraw = str(item.get("value_raw") or "").lo
wer()
487         unit = ""
488         if "°c" in vraw or "c" in vraw:
489             unit = "°C"
490         elif "%" in vraw:
491             unit = "%"
492         elif "rpm" in vraw:
493             unit = "RPM"
494         elif "v" in vraw and "w" not in vraw:
495             unit = "V"
496         elif "w" in vraw:
497             unit = "W"
498         elif "mhz" in vraw:
499             unit = "MHz"
500         elif "b/s" in vraw:
501             unit = "B/s"
502         elif "gb" in vraw:
503             unit = "GB"
504
505         clean_hw = re.sub(r'^a-zA-Z0-9+', '_', h
w_name.lower()).strip('_')
506         clean_name = re.sub(r'^a-zA-Z0-9+', '_',
s_name.lower()).strip('_')
507         sid = f"lhm_{clean_hw}_{clean_name}"

```

```

508 |
509 |             # Пропускаем, если такой датчик уже есть и
    | ли это дубликат общей нагрузки CPU/RAM
510 |             if sid in existing_ids or (cat == "Load" a
    | nd "total" in clean_name and any("util_total" in eid for eid i
    | n existing_ids)):
511 |                 continue
512 |
513 |             existing_ids.add(sid)
514 |             unique_readings.append({
515 |                 'id': sid,
516 |                 'hardware_name': hw_name,
517 |                 'hardware_type': hw_type,
518 |                 'sensor_category': cat,
519 |                 'sensor_name': s_name,
520 |                 'unit': unit,
521 |                 'value': float(val_num),
522 |                 '_provider': SensorProvider.LHM_ENRICH
    | ED,
523 |                 })
524 |             except Exception as lhm_err:
525 |                 logger.debug(f'Ошибка обогащения сенсорами LHM
    | : {lhm_err}')
526 |
527 |             return unique_readings
528 |
529 |     def get_hardware_snapshot(self) -> Dict[str, Any]:
530 |         """Получает полный снапшот аппаратного состояния со сп
    | иском сенсоров.
531 |
532 |         Returns:
533 |             Dict[str, Any]: Снапшот со списком показаний сенсо
    | ров.
534 |         """
535 |         with self._lock:
536 |             now_iso = datetime.now(timezone.utc).isoformat()
537 |             hardware_data = self._get_hardware_data()
538 |             internet_data = self._get_internet_speed_data()

```



```

539         sensor_readings = self.extract_sensor_readings(har
|         dware_data, internet_data)
540         sensors_payload = []
541         for r in sensor_readings:
542             sensors_payload.append({'id': r.get('id'), 'ha
|             rdware_name': r.get('hardware_name'), 'hardware_type': r.get('
|             hardware_type'), 'sensor_category': r.get('sensor_category'),
|             'sensor_name': r.get('sensor_name'), 'unit': r.get('unit'), 'v
|             alues': [{'num': r.get('value', 0.0), 'time': now_iso}]}))
543             result = {'timestamp': now_iso, 'sensors': sensors
|             _payload, 'hardware_monitor': hardware_data if hardware_data e
|             lse {}, 'internet_speed': internet_data if internet_data else
|             {}}
544             self._last_hardware_snapshot = result
545             return result
546
547     def get_last_snapshot(self) -> Optional[Dict[str, Any]]:
548         """Возвращает последний собранный снапшот.
549
550         Returns:
551             Optional[Dict[str, Any]]: Последний снапшот или No
|         ne.
552         """
553         return self._last_hardware_snapshot
554
555     def collect_all_sensors(self) -> List[Dict[str, Any]]:
556         """Собирает показания всех доступных датчиков.
557
558         Returns:
559             List[Dict[str, Any]]: Список прочитанных сенсоров.
560         """
561         with self._lock:
562             hardware_data = self._get_hardware_data()
563             internet_data = self._get_internet_speed_data()
564             return self.extract_sensor_readings(hardware_data,
|             internet_data)
565
566     def collect_lhm_cpu_readings(self) -> List[Dict[str, Any]]

```

```

:
567 |         """Извлекает нормализованные показания температур ядер
      | CPU и Package из LHM.
568 |
569 |         Returns:
570 |             List[Dict[str, Any]]: Список показаний датчиков те
      | мператур.
571 |         """
572 |         from apps.windows.sdk.modules.hardware.lhm_service imp
      | ort LhmService
573 |         lhm = self._lhm_service or LhmService()
574 |         items = lhm.get_flattened_sensors()
575 |         results: List[Dict[str, Any]] = []
576 |         core_idx = 0
577 |         for item in items:
578 |             if item.get("hardware_type") != "cpu" or item.get(
      | "sensor_category") != "Temperatures":
579 |                 continue
580 |             name = item.get("sensor_name", "")
581 |             if "Distance to TjMax" in name:
582 |                 continue
583 |             val = item.get("value_num")
584 |             if val is None:
585 |                 continue
586 |             if "Package" in name:
587 |                 results.append({
588 |                     "id": "cpu_package_temp",
589 |                     "hardware_name": item.get("hardware_name",
      | "CPU"),
590 |                     "hardware_type": "cpu",
591 |                     "sensor_category": "Temperatures",
592 |                     "sensor_name": name,
593 |                     "unit": "°C",
594 |                     "value": val,
595 |                     "value_num": val,
596 |                 })
597 |             elif "Core" in name:
598 |                 results.append({

```

```
599 |         "id": f"cpu_core_{core_idx}_temp",
600 |         "hardware_name": item.get("hardware_name",
    |         "CPU"),
601 |         "hardware_type": "cpu",
602 |         "sensor_category": "Temperatures",
603 |         "sensor_name": name,
604 |         "unit": "°C",
605 |         "value": val,
606 |         "value_num": val,
607 |     })
608 |     core_idx += 1
609 | return results
610 |
```

sensor_registry.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - Sensor
  Registry
4 # =====
5 # Description:
6 #   Реестр сенсоров и политика дедупликации.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.sensor_registry import SensorProvider
11 #
12 #       service = SensorProvider()
13 #
14 # File: sensor_registry.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-04 07:00:00
20 # =====
21
22 from __future__ import annotations
23 """Реестр сенсоров и политика дедупликации."""
24
25 from dataclasses import dataclass, field
26 from enum import Enum
27 from typing import Dict, List, Optional, Set, Tuple
28 from datetime import datetime, timezone
29
30
31 class SensorProvider(Enum):
```

```

32 |     """Приоритеты и категории провайдеров сенсоров.
33 |
34 |     Приоритеты распределены так:
35 |     - INTERNET_SPEED (55): Выделенный speedtest
36 |     - CIM_SYSTEM (50): Основной современный нативный интерфейс
    |     Windows (CIM/MSFT)
37 |     - WINDOWS_STORAGE (45): WindowsStorageSensor (диски, SMART
    |     , температуры NVMe)
38 |     - GPU_PROBER (40): Прямой зонд GPU (NVIDIA/AMD/Intel)
39 |     - ACPI_THERMAL (35): Системные ACPI ThermalZones
40 |     - HARDWARE_MONITOR (35): Агрегатор аппаратных метрик
41 |     - SENSOR_COLLECTOR (30): Базовый коллектор системных метри
    |     к
42 |     - WMI_FALLBACK (30): WMI слой совместимости / fallback при
    |     недоступности CIM
43 |     - NETWORK_SENSOR (25): Сетевые интерфейсы psutil
44 |     - LHM_ENRICHED (20): Обогащение отсутствующими в CIM/WMI с
    |     енсорами из LibreHardwareMonitor
45 |     - NVIDIA_SMI_DIRECT (10): Прямой вызов утилит
46 |     """
47 |     INTERNET_SPEED = 55           # InternetSpeedSensor - приори
    |     тет для speedtest метрик
48 |     CIM_SYSTEM = 50               # Основной интерфейс CIM (Powe
    |     rShell / CimSession)
49 |     WINDOWS_STORAGE = 45         # WindowsStorageSensor (SMART,
    |     temps) - приоритет для storage
50 |     GPU_PROBER = 40               # GpuProber - приоритет для GP
    |     U метрик
51 |     ACPI_THERMAL = 35             # ACPI thermal zones
52 |     HARDWARE_MONITOR = 35        # HardwareMonitor (агрегатор)
53 |     SENSOR_COLLECTOR = 30        # SensorCollector.extract_sens
    |     or_readings()
54 |     WMI_FALLBACK = 30            # WMI слой совместимости
55 |     NETWORK_SENSOR = 25          # Network sensors через psutil
56 |     LHM_ENRICHED = 20            # Обогащение детальными датчик
    |     ами из LibreHardwareMonitor
57 |     NVIDIA_SMI_DIRECT = 10       # Прямой вызов nvidia-smi (leg
    |     асу)

```

```

58 |
59 |
60 | @dataclass
61 | class SensorDefinition:
62 |     """Определение сенсора с политикой дедупликации."""
63 |     sensor_id: str
64 |     category: str
65 |     preferred_provider: SensorProvider
66 |     description: str = ""
67 |     unit: str = ""
68 |     allow_multiple_sources: bool = False # Если True, дублика
ты разрешены с разными провайдерами
69 |     dedup_window_seconds: float = 1.0 # Окно дедупликации
по времени
70 |
71 |
72 | class SensorRegistry:
73 |     """Централизованный реестр сенсоров с политикой дедупликац
ии."""
74 |
75 |     # Реестр известных сенсоров с предпочтительными провайдера
ми
76 |     _registry: Dict[str, SensorDefinition] = {}
77 |
78 |     # Мappings категорий на предпочтительные провайдеры
79 |     _category_providers: Dict[str, SensorProvider] = {
80 |         "temperature": SensorProvider.ACPI_THERMAL,
81 |         "Temperatures": SensorProvider.ACPI_THERMAL,
82 |         "gpu_temp": SensorProvider.GPU_PROBER,
83 |         "gpu_load": SensorProvider.GPU_PROBER,
84 |         "gpu_power": SensorProvider.GPU_PROBER,
85 |         "gpu_fan": SensorProvider.GPU_PROBER,
86 |         "cpu_temp": SensorProvider.ACPI_THERMAL,
87 |         "cpu_load": SensorProvider.SENSOR_COLLECTOR,
88 |         "Load": SensorProvider.SENSOR_COLLECTOR, # CPU/RAM ut
ilization
89 |         "Clocks": SensorProvider.SENSOR_COLLECTOR,
90 |         "network_throughput": SensorProvider.SENSOR_COLLECTOR,

```

```

91         "network_bytes": SensorProvider.NETWORK_SENSOR,
92         "Throughput": SensorProvider.SENSOR_COLLECTOR,
93         "internet": SensorProvider.INTERNET_SPEED,
94         "storage_temp": SensorProvider.WINDOWS_STORAGE,
95         "storage_wear": SensorProvider.WINDOWS_STORAGE,
96         "storage_usage": SensorProvider.SENSOR_COLLECTOR,
97         "storage": SensorProvider.SENSOR_COLLECTOR,
98     }
99
100     # Паттерны дублирующихся sensor_id
101     _duplicate_patterns: Dict[str, List[str]] = {
102         # GPU метрики - собираются предпочтительно через GpuPr
103         "gpu_*": ["gpu_prober", "sensor_collector", "nvidia_s
104         mi_direct"],
105         # CPU метрики
106         "cpu_*": ["acpi_thermal", "sensor_collector"],
107         # Internet ping - дублируется в sensors.py и sensor_co
108         llector
109         "internet_ping": ["internet_speed", "sensor_collector"
110         ],
111         # Network throughput - дублируется в sensors.py и sens
112         or_collector
113         "network_*_rate": ["network_sensor", "sensor_collecto
114         r"],
115     }
116
117     @classmethod
118     def register(cls, definition: SensorDefinition) -> None:
119         """Регистрирует сенсор в реестре.
120
121         Args:
122             definition: Определение сенсора с политикой.
123         """
124         cls._registry[definition.sensor_id] = definition
125
126     @classmethod
127     def get_preferred_provider(cls, sensor_id: str, category:

```

```

    str) -> Optional[SensorProvider]:
123     """Возвращает предпочтительный провайдер для данного с
    енсора.
124
125     Args:
126         sensor_id: Идентификатор сенсора.
127         category: Категория сенсора.
128
129     Returns:
130         Preferred provider или None если не определен.
131     """
132     # 1. Проверяем явную регистрацию
133     if sensor_id in cls._registry:
134         return cls._registry[sensor_id].preferred_provider
135
136     # 2. Определяем по специфическому паттерну sensor_id
137     import re
138     for pattern, providers in cls._duplicate_patterns.items():
139         if re.match(pattern, sensor_id):
140             # Возвращаем первый из списка как предпочтительный
141             provider_name = providers[0].upper()
142             try:
143                 return SensorProvider(provider_name)
144             except KeyError:
145                 pass
146
147     # 3. Проверяем по категории
148     if category in cls._category_providers:
149         return cls._category_providers[category]
150
151     return None
152
153     @classmethod
154     def should_keep_sensor(
155         cls,
156         sensor_id: str,

```



```

157         provider: SensorProvider,
158         existing_sensors: List[Tuple[str, SensorProvider]]
159     ) -> bool:
160         """Определяет, нужно ли сохранить показание сенсора ил
и отклонить как дубликат.
161
162     Args:
163         sensor_id: Идентификатор сенсора.
164         provider: Провайдер, предоставивший показание.
165         existing_sensors: Список уже собранных (sensor_id,
provider) пар.
166
167     Returns:
168         True если сенсор нужно сохранить, False если это д
убликат.
169         """
170         # Проверяем, есть ли уже такой sensor_id
171         for existing_id, existing_provider in existing_sensors
:
172             if existing_id == sensor_id:
173                 # Дубликат найден - сравниваем приоритеты пров
айдеров
174                 preferred = cls.get_preferred_provider(sensor_
id, "")
175                 if preferred:
176                     # Сохраняем только от предпочтительного пр
овайдера
177                     return provider == preferred
178                 else:
179                     # Нет явной политики - сохраняем от провай
дера с высшим приоритетом
180                     return provider.value > existing_provider.
value
181
182         # Дубликатов нет - сохраняем
183         return True
184
185

```

```

186 class SensorDeduplicator:
187     """Дедупликатор показаний сенсоров."""
188
189     def __init__(self) -> None:
190         """Инициализирует дедупликатор с пустым кешем."""
191         self._seen: Dict[str, Tuple[float, SensorProvider, flo
192 at]] = {} # sensor_id -> (timestamp, provider, value)
193
194     def add(
195         self,
196         sensor_id: str,
197         provider: SensorProvider,
198         value: float,
199         timestamp: Optional[float] = None,
200         dedup_window: float = 1.0
201     ) -> bool:
202         """Пытается добавить показание сенсора с проверкой на
203 дубликаты.
204
205     Args:
206         sensor_id: Идентификатор сенсора.
207         provider: Провайдер показания.
208         value: Значение показания.
209         timestamp: Временная метка (epoch seconds).
210         dedup_window: Окно дедупликации в секундах.
211
212     Returns:
213         True если показание добавлено, False если отклонен
214 о как дубликат.
215
216     """
217     if timestamp is None:
218         timestamp = datetime.now(timezone.utc).timestamp()
219
220     # Проверяем наличие в кеше
221     if sensor_id in self._seen:
222         cached_ts, cached_provider, cached_value = self._s
223 een[sensor_id]

```

```

220         # Проверяем временное окно
221         if abs(timestamp - cached_ts) < dedup_window:
222             # В пределах окна - проверяем приоритеты
223             preferred = SensorRegistry.get_preferred_provi
224             der(sensor_id, "")
225             if preferred:
226                 # Если текущий провайдер предпочтительнее
227                 - обновляем
228                 if provider == preferred:
229                     self._seen[sensor_id] = (timestamp, pr
230                     ovider, value)
231                     return True
232                 else:
233                     # Отклоняем
234                     return False
235                 else:
236                     # Сравниваем по значению приоритета
237                     if provider.value > cached_provider.value:
238                         self._seen[sensor_id] = (timestamp, pr
239                         ovider, value)
240                         return True
241                     else:
242                         return False
243
244         # Нет дубликата или вне окна - добавляем
245         self._seen[sensor_id] = (timestamp, provider, value)
246         return True
247
248     def get_unique(self) -> Dict[str, Tuple[SensorProvider, fl
249     oat]]:
250         """Возвращает уникальные показания сенсоров.
251
252         Returns:
253             Словарь {sensor_id: (provider, value)}.
254         """
255         return {sid: (prov, val) for sid, (_, prov, val) in se
256         lf._seen.items()}

```

```
252     def clear(self) -> None:
253         """Очищает кеш дедупликатора."""
254         self._seen.clear()
255
256
257     def deduplicate_sensor_readings(
258         readings: List[Dict],
259         provider: SensorProvider = SensorProvider.SENSOR_COLLECTOR
260     ) -> List[Dict]:
261         """Дедуплицирует список показаний сенсоров.
262
263         Args:
264             readings: Список показаний с полями 'id' и 'value'.
265             provider: Провайдер показаний.
266
267         Returns:
268             Дедуплицированный список показаний.
269         """
270         deduplicator = SensorDeduplicator()
271         unique_readings: List[Dict] = []
272
273         for reading in readings:
274             sensor_id = reading.get('id', '')
275             value = reading.get('value', 0.0)
276
277             if deduplicator.add(sensor_id, provider, value):
278                 unique_readings.append(reading)
279
280         return unique_readings
281
```

storage_usage.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - Storage
  Usage
4 # =====
5 # Description:
6 #   Четырехуровневый стек сбора телеметрии накопителей Windows
  :
7 #   1. DiskInventoryCollector (Win32 + IOCTL Storage Property
  & Volumes)
8 #   2. DiskHealthCollector (NVMe SMART / Health / TBW)
9 #   3. DiskPerformanceCollector (IOPS, Latency, Throughput, Qu
  eue)
10 #   4. DiskProcessIOCollector (Попроцессный I/O анализ и агрег
  аты)
11 #
12 # Usage Examples:
13 #   Python API:
14 #       from apps.windows.telemetry.storage_usage import (
15 #           DiskInventoryCollector,
16 #           DiskHealthCollector,
17 #           DiskPerformanceCollector,
18 #           DiskProcessIOCollector,
19 #           WindowsStorageUsageCollector,
20 #       )
21 #
22 #       collector = WindowsStorageUsageCollector()
23 #       disks = collector.get_physical_disks()
24 #       report = collector.get_disk_usage_period_report()
25 #
26 # File: storage_usage.py
27 # Project: ai-breadboard
28 # Package: apps.windows.telemetry
29 # Author: hypo69
```

```
30 # Copyright: © 2026 hypo69
31 # Updated: 2026-10-06 01:35:00
32 # =====
33 |=====
34 from __future__ import annotations
35 """Четырехуровневый стек сбора телеметрии накопителей Windows.
36 |"""
37 import math
38 import os
39 import time
40 from typing import Any, Dict, List, Optional, Tuple
41 |
42 try:
43 |     import psutil
44 |     PSUTIL_AVAILABLE = True
45 except ImportError:
46 |     psutil = None
47 |     PSUTIL_AVAILABLE = False
48 |
49 from logger import logger
50 from apps.windows.sdk.modules.storage_manager.core.raw_disk_io
51 | import WindowsRawDiskIO
52 from apps.windows.sdk.modules.storage_manager.core.windows_storage_sensor import WindowsStorageSensor
53 from apps.windows.telemetry.models import (
54 |     AppDiskUsageItem,
55 |     DiskUsagePeriodReport,
56 |     PhysicalDiskHealth,
57 | )
58 |
59 def _format_bytes(num_bytes: float) -> str:
60 |     """Форматирование объема байт в человекочитаемый вид (B, K
61 |     B, MB, GB, TB).
62 |
63 |     Args:
```

```

63         num_bytes: Количество байт.
64
65     Returns:
66         str: Отформатированная строка.
67     """
68     if num_bytes <= 0 or math.isnan(num_bytes):
69         return '0 B'
70     units = ['B', 'KB', 'MB', 'GB', 'TB', 'PB']
71     idx = 0
72     val = float(num_bytes)
73     while val >= 1024.0 and idx < len(units) - 1:
74         val /= 1024.0
75         idx += 1
76     return f'{val:.2f} {units[idx]}' if idx > 0 else f'{int(val)} B'
77
78
79 class DiskInventoryCollector:
80     """Слой 1: Коллектор инвентаря физических накопителей, разделов и логических томов."""
81
82     def __init__(self, raw_io: Optional[WindowsRawDiskIO] = None) -> None:
83         """Инициализация коллектора инвентаря дисков."""
84         self.raw_io = raw_io or WindowsRawDiskIO()
85
86     def collect(self) -> Dict[str, Any]:
87         """Сбор полного инвентаря накопителей и томов.
88
89         Returns:
90             Dict[str, Any]: Данные физических дисков и логических томов.
91         """
92         disks = self.raw_io.list_physical_disks()
93         volumes = self.raw_io.get_logical_volumes()
94         return {
95             'disks_count': len(disks),
96             'disks': disks,

```

```

97         'volumes_count': len(volumes),
98         'volumes': volumes,
99         'timestamp': time.time(),
100     }
101
102
103 class DiskHealthCollector:
104     """Слой 2: Коллектор SMART-показателей здоровья, износа и
105     TBW накопителей."""
106
107     def __init__(
108         self,
109         raw_io: Optional[WindowsRawDiskIO] = None,
110         storage_sensor: Optional[WindowsStorageSensor] = None,
111     ) -> None:
112         """Инициализация коллектора здоровья дисков."""
113         self.raw_io = raw_io or WindowsRawDiskIO()
114         self.storage_sensor = storage_sensor or WindowsStorage
115         Sensor()
116
117     def collect(self, force_refresh: bool = False) -> List[Dic
118     t[str, Any]]:
119         """Сбор показателей здоровья всех доступных накопите
120         лей.
121
122         Returns:
123             List[Dict[str, Any]]: Показатели SMART, TBW, темпе
124             ратуры и ошибок.
125         """
126         results: List[Dict[str, Any]] = []
127         raw_disks = self.raw_io.list_physical_disks()
128
129         for d in raw_disks:
130             disk_id = d.get('disk_id', 0)
131             smart = d.get('smart_health', {})
132             results.append({
133                 'disk_id': disk_id,
134                 'device_path': d.get('device_path'),

```



```

130         'model': d.get('product'),
131         'bus_type': d.get('bus_type'),
132         'temperature_c': smart.get('temperature_c'),
133         'wear_percentage': smart.get('percentage_used'
    ),
134         'available_spare_percent': smart.get('available_spare_percent'),
135         'tbw_written_tb': smart.get('data_units_written_tb'),
136         'tbw_read_tb': smart.get('data_units_read_tb')
    ,
137         'power_on_hours': smart.get('power_on_hours'),
138         'unsafe_shutdowns': smart.get('unsafe_shutdowns'),
139         'media_errors': smart.get('media_errors'),
140         'raw_available': smart.get('raw_nvme_available', False),
141     })
142
143     # Fallback / дополнение через WindowsStorageSensor CIM
    если нативный IOCTL вернул None
144     needs_cim = any(r['temperature_c'] is None and r['wear_percentage'] is None for r in results)
145     if needs_cim:
146         try:
147             cim_disks = self.storage_sensor.get_physical_disks(force_refresh=force_refresh)
148             cim_map = {str(cd.device_id).replace(r'\\.\\PHYSICALDRIVE', '').replace('Disk', ''): cd for cd in cim_disks}
149             for r in results:
150                 str_id = str(r['disk_id'])
151                 if str_id in cim_map:
152                     cd = cim_map[str_id]
153                     if r['temperature_c'] is None and cd.temperature_c:
154                         r['temperature_c'] = cd.temperature_c
155                     if r['wear_percentage'] is None and cd

```

```

155 |         .wear_percentage:
156 |             r['wear_percentage'] = cd.wear_per
157 |             centage
158 |             if r['power_on_hours'] is None and cd.
159 |             power_on_hours:
160 |                 r['power_on_hours'] = cd.power_on_
161 |                 hours
162 |                 if r['tbw_written_tb'] is None and cd.
163 |                 lifetime_write_tb:
164 |                     r['tbw_written_tb'] = cd.lifetime_
165 |                     write_tb
166 |                     if r['tbw_read_tb'] is None and cd.lif
167 |                     etime_read_tb:
168 |                         r['tbw_read_tb'] = cd.lifetime_rea
169 |                         d_tb
170 |             except Exception as exc:
171 |                 logger.debug(f"Ошибка дополнения здоровья чере
172 |                 з CIM: {exc}")
173 |
174 |         return results
175 |
176 |
177 | class DiskPerformanceCollector:
178 |     """Слой 3: Коллектор текущей скорости, IOPS и времени откл
179 |     ика дисков."""
180 |
181 |     def __init__(self) -> None:
182 |         """Инициализация коллектора производительности дисков.
183 |         """
184 |
185 |         self._last_sample_time: float = time.time()
186 |         self._last_io_counters: Dict[str, Any] = {}
187 |
188 |     def collect(self) -> Dict[str, Any]:
189 |         """Сбор показателей пропускной способности и IOPS нако
190 |         пителей.
191 |
192 |         Returns:
193 |             Dict[str, Any]: Метрики скорости чтения/записи и о

```

пераций в секунду.

```
182         """
183         now = time.time()
184         dt = max(0.001, now - self._last_sample_time)
185         res: Dict[str, Any] = {
186             'total_read_bytes_sec': 0.0,
187             'total_write_bytes_sec': 0.0,
188             'total_read_iops': 0.0,
189             'total_write_iops': 0.0,
190             'per_disk': {},
191             'dt_sec': dt,
192         }
193
194         if not PSUTIL_AVAILABLE:
195             self._last_sample_time = now
196             return res
197
198         try:
199             per_disk_counters = psutil.disk_io_counters(perdis
200 k=True)
201             if per_disk_counters:
202                 for disk_name, cur in per_disk_counters.items(
203 ):
204                     prev = self._last_io_counters.get(disk_name)
205                     if prev:
206                         r_bytes = max(0, cur.read_bytes - prev
207 .read_bytes) / dt
208                         w_bytes = max(0, cur.write_bytes - prev
209 v.write_bytes) / dt
210                         r_count = max(0, cur.read_count - prev
211 .read_count) / dt
212                         w_count = max(0, cur.write_count - prev
213 v.write_count) / dt
214                         r_time = max(0, cur.read_time - prev.r
215 ead_time)
216                         w_time = max(0, cur.write_time - prev.
217 write_time)
```

```

210 |
211 |             res['total_read_bytes_sec'] += r_bytes
212 |             res['total_write_bytes_sec'] += w_byte
213 |         s
214 |             res['total_read_iops'] += r_count
215 |             res['total_write_iops'] += w_count
216 |
217 |             res['per_disk'][disk_name] = {
218 |                 'read_bytes_sec': round(r_bytes, 1
219 | ),
220 |                 'write_bytes_sec': round(w_bytes,
221 | 1),
222 |                 'read_iops': round(r_count, 1),
223 |                 'write_iops': round(w_count, 1),
224 |                 'read_time_ms': r_time,
225 |                 'write_time_ms': w_time,
226 |             }
227 |             self._last_io_counters = per_disk_counters
228 |         except Exception as exc:
229 |             logger.debug(f"Ошибка сбора счетчиков производител
230 | ьности дисков: {exc}")
231 |
232 |         self._last_sample_time = now
233 |         return res
234 |
235 | class DiskProcessIOCollector:
236 |     """Слой 4: Коллектор активности ввода-вывода процессов (To
237 | p Writers/Readers)."""
238 |
239 |     def __init__(self) -> None:
240 |         """Инициализация коллектора I/O процессов."""
241 |         self._last_process_io: Dict[int, Dict[str, Any]] = {}
242 |         self._last_poll_time: float = time.time()
243 |
244 |     def collect(self) -> Dict[str, Dict[str, Any]]:
245 |         """Опрос I/O всех активных процессов и вычисление дель
246 | т чтения/записи.
```

```

242
243     Returns:
244         Dict[str, Dict[str, Any]]: Агрегированные дельты п
о именам процессов.
245         """
246         if not PSUTIL_AVAILABLE:
247             return {}
248
249         now = time.time()
250         aggregated_by_name: Dict[str, Dict[str, Any]] = {}
251
252         try:
253             for p in psutil.process_iter(attrs=['pid', 'name']
):
254                 try:
255                     pid = p.info['pid']
256                     name = p.info.get('name') or f'PID_{pid}'
257                     io_cnt = p.io_counters()
258                     cur_r = io_cnt.read_bytes
259                     cur_w = io_cnt.write_bytes
260
261                     prev = self._last_process_io.get(pid)
262                     if prev:
263                         delta_r = max(0, cur_r - prev['read_by
tes'])
264                         delta_w = max(0, cur_w - prev['write_b
ytes'])
265                     else:
266                         delta_r = cur_r
267                         delta_w = cur_w
268
269                     self._last_process_io[pid] = {
270                         'name': name,
271                         'read_bytes': cur_r,
272                         'write_bytes': cur_w,
273                         'time': now,
274                     }
275

```

```

276         if name not in aggregated_by_name:
277             aggregated_by_name[name] = {
278                 'pid': pid,
279                 'read_bytes': 0,
280                 'write_bytes': 0,
281             }
282             aggregated_by_name[name]['read_bytes'] +=
283             delta_r
284             aggregated_by_name[name]['write_bytes'] +=
285             delta_w
286         except (psutil.NoSuchProcess, psutil.AccessDen
287         ired, AttributeError):
288             continue
289         except Exception as ex:
290             logger.debug(f'Ошибка выборки process I/O deltas:
291             {ex}')
292
293     self._last_poll_time = now
294     return aggregated_by_name
295
296 class WindowsStorageUsageCollector:
297     """Унифицированный диспетчер дисковой телеметрии, объединя
298     ющий все 4 слоя."""
299
300     def __init__(
301         self,
302         raw_io: Optional[WindowsRawDiskIO] = None,
303         storage_sensor: Optional[WindowsStorageSensor] = None,
304     ) -> None:
305         """Инициализация унифицированного коллектора дисковой
306         телеметрии."""
307
308         self.raw_io = raw_io or WindowsRawDiskIO()
309         self.storage_sensor = storage_sensor or WindowsStorage
310         Sensor()
311
312         self.inventory_collector = DiskInventoryCollector(self
313         .raw_io)

```

```

306         self.health_collector = DiskHealthCollector(self.raw_i
o, self.storage_sensor)
307         self.performance_collector = DiskPerformanceCollector(
)
308         self.process_io_collector = DiskProcessIOCollector()
309
310     def get_physical_disks(self, force_refresh: bool = False)
-> List[PhysicalDiskHealth]:
311         """Получить актуальный список физических накопителей с
показателями SMART и Lifetime."""
312         results: List[PhysicalDiskHealth] = []
313         try:
314             health_list = self.health_collector.collect(force_
refresh=force_refresh)
315             for h in health_list:
316                 results.append(PhysicalDiskHealth(
317                     device_id=f"Disk {h.get('disk_id', 0)}",
318                     model=h.get('model') or f"PhysicalDrive{h.
get('disk_id', 0)}",
319                     media_type='SSD' if 'SSD' in (h.get('model
', '')) + h.get('bus_type', '').upper() or h.get('bus_type') =
= 'NVMe' else 'HDD',
320                     size_gb=0.0,
321                     health_status='Healthy',
322                     operational_status='OK',
323                     temperature_celsius=h.get('temperature_c')
,
324                     interface_type=h.get('bus_type') or 'NVMe'
,
325                     lifetime_read_bytes=int(h['tbw_read_tb'] *
(1024 ** 4)) if h.get('tbw_read_tb') else None,
326                     lifetime_write_bytes=int(h['tbw_written_tb
'] * (1024 ** 4)) if h.get('tbw_written_tb') else None,
327                     lifetime_read_tb=h.get('tbw_read_tb'),
328                     lifetime_write_tb=h.get('tbw_written_tb'),
329                     power_on_hours=h.get('power_on_hours'),
330                     wear_percentage=h.get('wear_percentage'),
331                 ))

```

```

332         except Exception as ex:
333             logger.debug(f'Ошибка получения физических накопит
елей: {ex}')
334
335         if not results:
336             results.append(PhysicalDiskHealth(
337                 device_id='Disk 0',
338                 model='System Drive (NVMe/SSD)',
339                 media_type='SSD',
340                 size_gb=512.0,
341                 health_status='Healthy',
342                 operational_status='OK',
343                 interface_type='NVMe',
344             ))
345         return results
346
347     def poll_process_io_deltas(self) -> Dict[str, Dict[str, An
y]]:
348         """Опросить текущий I/O процессов и рассчитать накопле
нные дельты."""
349         return self.process_io_collector.collect()
350
351     def get_disk_usage_period_report(
352         self,
353         period_minutes: int = 1440,
354         force_refresh_smart: bool = False,
355     ) -> DiskUsagePeriodReport:
356         """Сформировать агрегированный отчет об использовании
диска за интервал."""
357         phys_disks = self.get_physical_disks(force_refresh=for
ce_refresh_smart)
358         proc_deltas = self.poll_process_io_deltas()
359
360         writers_list: List[AppDiskUsageItem] = []
361         readers_list: List[AppDiskUsageItem] = []
362
363         total_r = sum(item['read_bytes'] for item in proc_delt
as.values())

```



```

364         total_w = sum(item['write_bytes'] for item in proc_deltas.values())
365         total_all = total_r + total_w
366
367         # Ранжирование по записи
368         sorted_writers = sorted(proc_deltas.items(), key=lambda x: x[1]['write_bytes'], reverse=True)
369         for name, item in sorted_writers[:15]:
370             wb = item['write_bytes']
371             rb = item['read_bytes']
372             tot = wb + rb
373             if wb <= 0:
374                 continue
375             pct = round((wb / total_w) * 100.0, 1) if total_w > 0 else 0.0
376             writers_list.append(AppDiskUsageItem(
377                 process_name=name,
378                 pid=item.get('pid'),
379                 read_bytes=rb,
380                 write_bytes=wb,
381                 total_bytes=tot,
382                 read_formatted=_format_bytes(rb),
383                 write_formatted=_format_bytes(wb),
384                 share_percent=pct,
385             ))
386
387         # Ранжирование по чтению
388         sorted_readers = sorted(proc_deltas.items(), key=lambda x: x[1]['read_bytes'], reverse=True)
389         for name, item in sorted_readers[:15]:
390             wb = item['write_bytes']
391             rb = item['read_bytes']
392             tot = wb + rb
393             if rb <= 0:
394                 continue
395             pct = round((rb / total_r) * 100.0, 1) if total_r > 0 else 0.0
396             readers_list.append(AppDiskUsageItem(

```

```

397         process_name=name,
398         pid=item.get('pid'),
399         read_bytes=rb,
400         write_bytes=wb,
401         total_bytes=tot,
402         read_formatted=_format_bytes(rb),
403         write_formatted=_format_bytes(wb),
404         share_percent=pct,
405     ))
406
407     top_writer = sorted_writers[0][0] if sorted_writers and sorted_writers[0][1]['write_bytes'] > 0 else 'N/A'
408     top_reader = sorted_readers[0][0] if sorted_readers and sorted_readers[0][1]['read_bytes'] > 0 else 'N/A'
409
410     diag = (
411         f"За анализируемый период ({period_minutes} мин) записано {_format_bytes(total_w)}, "
412         f"прочитано {_format_bytes(total_r)}. Главный источник записи: {top_writer}, чтения: {top_reader}."
413     )
414
415     return DiskUsagePeriodReport(
416         period_minutes=period_minutes,
417         total_read_bytes=total_r,
418         total_write_bytes=total_w,
419         total_read_formatted=_format_bytes(total_r),
420         total_write_formatted=_format_bytes(total_w),
421         total_formatted=_format_bytes(total_all),
422         top_writers=writers_list,
423         top_readers=readers_list,
424         physical_disks=phys_disks,
425         summary_text=diag,
426     )
427
428
429 __all__ = [
430     'DiskInventoryCollector',

```

```
431 | 'DiskHealthCollector',  
432 | 'DiskPerformanceCollector',  
433 | 'DiskProcessIOCollector',  
434 | 'WindowsStorageUsageCollector',  
435 | ]  
436 |
```

w64_collector.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - W64 Collector
4 # =====
5 # Description:
6 #   AI Windows 64-bit Collector - модуль сбора детальных системных событий телеметрии.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.w64_collector import AIW64Collector
11 #
12 #       service = AIW64Collector()
13 #
14 # File: w64_collector.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-06 07:35:00
20 # =====
21
22 from __future__ import annotations
23 """AI Windows 64-bit Collector - модуль сбора детальных системных событий телеметрии."""
24
25 import json
26 import os
27 import re
28 import subprocess
29 import threading
```

```

30 import time
31 from datetime import datetime, timezone
32 from pathlib import Path
33 from typing import Any, Callable, Dict, List, Optional
34 from logger import logger
35 from .models import W64CollectorStatus, W64SystemEvent
36 from .sqlite import TelemetryStorage
37
38
39 class AIW64Collector:
40     """Максимально детальный сборщик событий Windows 64-bit дл
я подсистемы телеметрии.
41
42     Фиксирует непрерывные изменения системного состояния:
43     - Процессы (создание, завершение, активность)
44     - Файловая система (изменения в отслеживаемых путях)
45     - Реестр Windows (модификации ключей)
46     - Сетевая активность (новые соединения и открытые порты)
47     """
48
49     def __init__(
50         self,
51         enable_file_monitoring: bool = True,
52         enable_process_monitoring: bool = True,
53         enable_registry_monitoring: bool = True,
54         enable_network_monitoring: bool = True,
55         enable_event_log_monitoring: bool = True,
56         monitored_paths: Optional[List[str]] = None,
57         poll_interval_sec: float = 1.0,
58         storage: Optional[TelemetryStorage] = None,
59         on_event_callback: Optional[Callable[[Dict[str, Any]],
None]] = None,
60         **kwargs: Any,
61     ) -> None:
62         """Инициализация сборщика системных событий W64.
63
64         Args:
65             enable_file_monitoring: Флаг мониторинга изменений

```

```

        файлов.
66         enable_process_monitoring: Флаг мониторинга процес
сов.
67         enable_registry_monitoring: Флаг мониторинга реест
ра.
68         enable_network_monitoring: Флаг мониторинга сетевы
х соединений.
69         enable_event_log_monitoring: Флаг мониторинга журн
алов Windows.
70         monitored_paths: Список путей файловой системы для
отслеживания.
71         poll_interval_sec: Интервал между циклами сканиров
ания изменений (сек).
72         storage: Экземпляр хранилища TelemetryStorage для
сохранения в SQLite через буфер.
73         on_event_callback: Опциональный callback-обработчи
к каждого зафиксированного события.
74         **kwargs: Дополнительные параметры конфигурации.
75         """
76         self.enable_file_monitoring = enable_file_monitoring
77         self.enable_process_monitoring = enable_process_monito
ring
78         self.enable_registry_monitoring = enable_registry_moni
toring
79         self.enable_network_monitoring = enable_network_monito
ring
80         self.enable_event_log_monitoring = enable_event_log_mo
nitoring
81         self.poll_interval = max(0.1, poll_interval_sec)
82         self.storage = storage or TelemetryStorage.get_instanc
e()
83         self.on_event_callback = on_event_callback
84
85         self.monitored_paths = monitored_paths or [
86             os.path.expandvars(r'%USERPROFILE%\Documents'),
87             os.path.expandvars(r'%USERPROFILE%\Downloads'),
88             os.path.expandvars(r'%USERPROFILE%\Desktop'),
89             r'C:\Program Files',

```

```

90         r'C:\Program Files (x86)',
91         r'C:\Windows\System32',
92         r'C:\Windows\SysWOW64',
93     ]
94
95     self._running = False
96     self._thread: Optional[threading.Thread] = None
97     self._stop_event = threading.Event()
98     self._event_counter = 0
99     self._last_event_time: Optional[str] = None
100     self._last_baseline: Dict[str, Any] = {}
101     logger.debug('AIW64Collector инициализирован (буфериза
ция в оперативной памяти)')
102
103     def _log_event(self, event_type: str, event_data: Dict[str
, Any]) -> None:
104         """Зафиксировать событие телеметрии в оперативной памя
ти с пакетным сбросом в SQLite.
105
106         Args:
107             event_type: Тип события.
108             event_data: Словарь с атрибутами события.
109         """
110         self._event_counter += 1
111         now_iso = datetime.now(timezone.utc).isoformat()
112         event_data['event_id'] = f'evt_{int(time.time())}_{sel
f._event_counter}'
113         event_data['timestamp'] = now_iso
114         event_data['event_type'] = event_type
115         self._last_event_time = now_iso
116
117         # 1. Сохранение в буфер памяти TelemetryStorage
118         if self.storage:
119             try:
120                 self.storage.save_w64_event(event_data, provid
er='w64_collector')
121             except Exception as ex:
122                 logger.debug(f'Ошибка сохранения W64 события в

```

```

    БД: {ex}')
123
124     # 2. Передача в callback при наличии
125     if self.on_event_callback:
126         try:
127             self.on_event_callback(event_data)
128         except Exception as ex:
129             logger.debug(f'Ошибка в on_event_callback: {ex
    }')
130
131     def start(self) -> bool:
132         """Запустить фоновый поток мониторинга событий.
133
134         Returns:
135             bool: True если сборщик успешно запущен, False есл
и уже работал.
136         """
137         if self._running:
138             logger.warning('AIW64Collector уже запущен')
139             return False
140
141         self._running = True
142         self._stop_event.clear()
143         self._update_baseline()
144         self._thread = threading.Thread(target=self._monitorin
g_loop, name='AIW64CollectorWorker', daemon=True)
145         self._thread.start()
146         logger.info('AIW64Collector успешно запущен')
147         return True
148
149     def stop(self) -> bool:
150         """Остановить фоновый поток мониторинга.
151
152         Returns:
153             bool: True если сборщик остановлен, False если не
был активен.
154         """
155         if not self._running:

```



```

156         return False
157
158     self._running = False
159     self._stop_event.set()
160     if self._thread and self._thread.is_alive():
161         self._thread.join(timeout=5.0)
162     logger.info(f'AIW64Collector остановлен. Всего зафиксиро-
163 ровано событий: {self._event_counter}')
164     return True
165
166     def _update_baseline(self) -> None:
167         """Сформировать начальный снимок состояния системы для
168 сравнения."""
169         self._last_baseline = {
170             'processes': self._get_current_processes(),
171             'network_connections': self._get_current_network_c
172 onnections(),
173             'registry_keys': self._get_registry_keys(),
174             'file_hashes': self._get_file_hashes(),
175         }
176
177     def _monitoring_loop(self) -> None:
178         """Основной рабочий цикл мониторинга и выявления измен
179 ений."""
180         last_check = time.time()
181         while not self._stop_event.is_set():
182             try:
183                 current_time = time.time()
184                 if current_time - last_check >= self.poll_inte
185 rval:
186                     current_state = {
187                         'processes': self._get_current_process
188 es(),
189                         'network_connections': self._get_curre
190 nt_network_connections(),
191                         'registry_keys': self._get_registry_ke
192 ys(),
193                         'file_hashes': self._get_file_hashes()

```

```

186         }
187         self._detect_changes(self._last_baseline,
current_state)
188         self._last_baseline = current_state
189         last_check = current_time
190
191         self._stop_event.wait(timeout=min(0.5, self.po
ll_interval))
192     except Exception as ex:
193         logger.error(f'Ошибка в цикле мониторинга AIW6
4Collector: {ex}')
194
195     def _detect_changes(self, old_state: Dict[str, Any], new_s
tate: Dict[str, Any]) -> None:
196         """Сравнить предыдущее и текущее состояние и залогиров
ать изменения.
197
198         Args:
199             old_state: Предыдущий системный срез.
200             new_state: Текущий системный срез.
201         """
202         # Мониторинг процессов
203         if self.enable_process_monitoring:
204             old_procs = {p['pid']: p for p in old_state.get('p
rocesses', [])}
205             new_procs = {p['pid']: p for p in new_state.get('p
rocesses', [])}
206
207             for pid, proc in new_procs.items():
208                 if pid not in old_procs:
209                     self._log_event('process_start', proc)
210
211             for pid, proc in old_procs.items():
212                 if pid not in new_procs:
213                     self._log_event('process_stop', proc)
214
215         # Мониторинг сетевых соединений

```

```
216         if self.enable_network_monitoring:
217             old_conns = {c.get('key'): c for c in old_state.ge
218 t('network_connections', []) if c.get('key')}
219             new_conns = {c.get('key'): c for c in new_state.ge
220 t('network_connections', []) if c.get('key')}
221
222             for key, conn in new_conns.items():
223                 if key not in old_conns:
224                     self._log_event('network_open', conn)
225
226             for key, conn in old_conns.items():
227                 if key not in new_conns:
228                     self._log_event('network_close', conn)
229
230     # Мониторинг файлов
231     if self.enable_file_monitoring:
232         old_files = old_state.get('file_hashes', {})
233         new_files = new_state.get('file_hashes', {})
234
235         for path, h in new_files.items():
236             if path not in old_files:
237                 self._log_event('file_created', {'path': p
238 ath, 'hash': h})
239             elif old_files[path] != h:
240                 self._log_event('file_modified', {'path':
241 path, 'old_hash': old_files[path], 'new_hash': h})
242
243         for path, h in old_files.items():
244             if path not in new_files:
245                 self._log_event('file_deleted', {'path': p
246 ath, 'hash': h})
247
248     # Мониторинг реестра
249     if self.enable_registry_monitoring:
250         old_reg = old_state.get('registry_keys', {})
251         new_reg = new_state.get('registry_keys', {})
252
253         for key, val in new_reg.items():
```

```

249         if key not in old_reg:
250             self._log_event('registry_created', {'key'
: key, 'value': val})
251         elif old_reg[key] != val:
252             self._log_event('registry_modified', {'key
': key, 'old_value': old_reg[key], 'new_value': val})
253
254         for key, val in old_reg.items():
255             if key not in new_reg:
256                 self._log_event('registry_deleted', {'key'
: key, 'value': val})
257
258     def _get_current_processes(self) -> List[Dict[str, Any]]:
259         """Получить текущий список активных процессов в систем
e."""
260         processes = []
261         try:
262             import psutil
263             for proc in psutil.process_iter(['pid', 'name', 'e
xe', 'cmdline', 'username', 'create_time']):
264                 try:
265                     info = proc.info
266                     processes.append({
267                         'pid': info.get('pid'),
268                         'name': info.get('name') or '',
269                         'path': info.get('exe') or '',
270                         'cmdline': ' '.join(info.get('cmdline'
) or []) if info.get('cmdline') else '',
271                         'username': info.get('username') or ''
272                         ,
273                         'create_time': info.get('create_time')
or 0.0,
274                     })
275                 except (psutil.NoSuchProcess, psutil.AccessDen
ied):
276                     continue
277             except Exception as ex:
278                 logger.debug(f'Ошибка при получении списка процесс

```

```

    ов: {ex}')
278         return processes
279
280     def _get_current_network_connections(self) -> List[Dict[str, Any]]:
281         """Получить текущие активные сетевые подключения."""
282         connections = []
283         try:
284             import psutil
285             for conn in psutil.net_connections(kind='inet'):
286                 try:
287                     laddr = f'{conn.laddr.ip}:{conn.laddr.port}' if conn.laddr else ''
288                     raddr = f'{conn.raddr.ip}:{conn.raddr.port}' if conn.raddr else ''
289                     key = f'{conn.type}_{laddr}_{raddr}_{conn.pid}'
290                     connections.append({
291                         'key': key,
292                         'fd': conn.fd,
293                         'family': conn.family.name if hasattr(conn.family, 'name') else str(conn.family),
294                         'type': conn.type.name if hasattr(conn.type, 'name') else str(conn.type),
295                         'laddr': laddr,
296                         'raddr': raddr,
297                         'status': conn.status,
298                         'pid': conn.pid,
299                     })
300                 except Exception:
301                     continue
302             except Exception as ex:
303                 logger.debug(f'Ошибка при получении сетевых соединений: {ex}')
304         return connections
305
306     def _get_registry_keys(self) -> Dict[str, Any]:
307         """Получить контрольные ключи реестра автозагрузки и с

```

```

        ИСТЕМНЫХ ПОЛИТИК. """
308         keys = {}
309         if os.name != 'nt':
310             return keys
311         try:
312             import winreg
313             hives = [
314                 (winreg.HKEY_CURRENT_USER, r'Software\Microsof
315                 t\Windows\CurrentVersion\Run'),
316                 (winreg.HKEY_LOCAL_MACHINE, r'Software\Microso
317                 ft\Windows\CurrentVersion\Run'),
318                 (winreg.HKEY_LOCAL_MACHINE, r'Software\Microso
319                 ft\Windows\CurrentVersion\RunOnce'),
320             ]
321             for hive, subkey in hives:
322                 try:
323                     with winreg.OpenKey(hive, subkey) as k:
324                         idx = 0
325                         while True:
326                             try:
327                                 name, val, _ = winreg.EnumValu
328                                 e(k, idx)
329                                 key_path = f'{subkey}\\{name}'
330                                 keys[key_path] = str(val)
331                                 idx += 1
332                             except OSError:
333                                 break
334                         except OSError:
335                             continue
336                     except Exception as ex:
337                         logger.debug(f'Ошибка при чтении ключей реестра: {
338                         ex}')
339             return keys
340
341         def _get_file_hashes(self) -> Dict[str, str]:
342             """Получить отпечатки и размеры отслеживаемых файлов д
343             ля быстрого детекта изменений. """
344             hashes = {}

```

```

339         for root_path in self.monitored_paths:
340             p = Path(root_path)
341             if not p.exists():
342                 continue
343             try:
344                 # Ограничиваем сканирование первыми 100 файлам
и для предотвращения перегрузки
345                 count = 0
346                 for item in p.rglob('*'):
347                     if count > 100:
348                         break
349                     if item.is_file():
350                         try:
351                             stat = item.stat()
352                             hashes[str(item)] = f'{stat.st_siz
e}_{stat.st_mtime}'
353                             count += 1
354                         except OSError:
355                             continue
356                     except Exception as ex:
357                         logger.debug(f'Ошибка при проверке файлов в {r
oot_path}: {ex}')
358                 return hashes
359
360     def get_status(self) -> Dict[str, Any]:
361         """Получить диагностический статус сборщика.
362
363         Returns:
364             Dict[str, Any]: Словарь со статусом и счетчиками.
365         """
366         status = W64CollectorStatus(
367             running=self._running,
368             events_count=self._event_counter,
369             last_event_time=self._last_event_time,
370         )
371         return status.model_dump()
372
373     def get_events(self, event_type: Optional[str] = None, lim

```

```

        it: int = 100) -> List[Dict[str, Any]]:
374         """Получить список последних событий из SQLite хранилища телеметрии.
375
376         Args:
377             event_type: Фильтр по типу события.
378             limit: Максимальное число записей.
379
380         Returns:
381             List[Dict[str, Any]]: Список событий.
382         """
383         if self.storage:
384             return self.storage.get_w64_events(event_type=event_type, provider='w64_collector', limit=limit)
385         return []
386
387
388 _global_w64_collector: Optional[AIW64Collector] = None
389 _lock = threading.Lock()
390
391 def get_w64_collector(**kwargs: Any) -> AIW64Collector:
392     """Получить глобальный синглтон сборщика AIW64Collector.
393
394     Args:
395         **kwargs: Аргументы инициализации при первом создании.
396
397     Returns:
398         AIW64Collector: Синглтон сборщика.
399     """
400     global _global_w64_collector
401     with _lock:
402         if _global_w64_collector is None:
403             _global_w64_collector = AIW64Collector(**kwargs)
404         return _global_w64_collector
405
406 def start_w64_collector(**kwargs: Any) -> bool:
407     """Запустить глобальный сборщик AIW64Collector."""
408     collector = get_w64_collector(**kwargs)

```



```
409 |     return collector.start()
410 |
411 | def stop_w64_collector() -> bool:
412 |     """Остановить глобальный сборщик AIW64Collector."""
413 |     global _global_w64_collector
414 |     with _lock:
415 |         if _global_w64_collector:
416 |             res = _global_w64_collector.stop()
417 |             _global_w64_collector = None
418 |             return res
419 |     return False
420 |
```

w64_etw_collector.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - W64 Etw
  Collector
4 # =====
5 # Description:
6 #   ETW Collector для сбора низкоуровневых событий ядра и журна
  лов аудита Windows.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.w64_etw_collector import AIW
  64ETWCollector
11 #
12 #       service = AIW64ETWCollector()
13 #
14 # File: w64_etw_collector.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-06 07:35:00
20 # =====
21
22 from __future__ import annotations
23 """ETW Collector для сбора низкоуровневых событий ядра и журна
  лов аудита Windows."""
24
25 import json
26 import os
27 import threading
28 import time
29 from collections import deque
```

```

30 from datetime import datetime, timezone
31 from pathlib import Path
32 from typing import Any, Callable, Dict, List, Optional
33 from logger import logger
34 from .models import ETWTraceEvent, ProcessLifecycleEvent, W64C
    collectorStatus
35 from .sqlite import TelemetryStorage
36 from .win32_fffi.wevtapi import WevtAPI
37
38
39 class AIW64ETWCollector:
40     """Сборщик низкоуровневых событий через ETW и журналы ауди
    та Windows."""
41
42     def __init__(
43         self,
44         enable_process_trace: bool = True,
45         enable_disk_trace: bool = True,
46         enable_network_trace: bool = True,
47         enable_registry_trace: bool = True,
48         poll_interval_sec: float = 5.0,
49         storage: Optional[TelemetryStorage] = None,
50         on_event_callback: Optional[Callable[[Dict[str, Any]],
    None]] = None,
51         **kwargs: Any,
52     ) -> None:
53         """Инициализация ETW сборщика телеметрии.
54
55         Args:
56             enable_process_trace: Трассировка создания/заверше
    ния процессов (ID 4688).
57             enable_disk_trace: Трассировка операций файлового
    аудита (ID 4663).
58             enable_network_trace: Трассировка сетевых подключе
    ний Windows Filtering Platform.
59             enable_registry_trace: Трассировка изменений ключе
    й аудита реестра.
60             poll_interval_sec: Интервал между выборками событи

```

й ETW (сек).

```
61         storage: Экземпляр хранилища TelemetryStorage.
62         on_event_callback: Callback-обработчик каждого зафиксированного события.
63         **kwargs: Дополнительные параметры конфигурации.
64         """
65         self.enable_process_trace = enable_process_trace
66         self.enable_disk_trace = enable_disk_trace
67         self.enable_network_trace = enable_network_trace
68         self.enable_registry_trace = enable_registry_trace
69         self.poll_interval = max(1.0, poll_interval_sec)
70         self.storage = storage or TelemetryStorage.get_instance()
71         self.on_event_callback = on_event_callback
72
73         self.wevt_api = WevtAPI() if os.name == 'nt' else None
74         self._seen_records: set[tuple[str, int]] = set()
75         self._seen_queue: deque[tuple[str, int]] = deque(maxlen=5000)
76
77         self._running = False
78         self._thread: Optional[threading.Thread] = None
79         self._stop_event = threading.Event()
80         self._event_counter = 0
81         self._last_event_time: Optional[str] = None
82         logger.debug('AIW64ETWCollector инициализирован (буферизация в оперативной памяти)')
83
84     def start(self) -> bool:
85         """Запустить фоновый поток сбора ETW событий.
86
87         Returns:
88             bool: True если сборщик успешно запущен, False если уже работал.
89         """
90         if self._running:
91             logger.warning('AIW64ETWCollector уже запущен')
92             return False
```

```

93 |
94 |         self._running = True
95 |         self._stop_event.clear()
96 |         self._thread = threading.Thread(target=self._etw_loop,
    | name='AIW64ETWWorker', daemon=True)
97 |         self._thread.start()
98 |         logger.info('AIW64ETWCollector успешно запущен')
99 |         return True
100 |
101 |     def stop(self) -> bool:
102 |         """Остановить фоновый поток ETW сбора.
103 |
104 |         Returns:
105 |             bool: True если сборщик остановлен, False если не
    | был активен.
106 |         """
107 |         if not self._running:
108 |             return False
109 |
110 |         self._running = False
111 |         self._stop_event.set()
112 |         if self._thread and self._thread.is_alive():
113 |             self._thread.join(timeout=5.0)
114 |         logger.info(f'AIW64ETWCollector остановлен. Всего зафи
    | ксировано событий: {self._event_counter}')
115 |         return True
116 |
117 |     def _etw_loop(self) -> None:
118 |         """Основной рабочий цикл выборки событий ETW."""
119 |         while not self._stop_event.is_set():
120 |             try:
121 |                 self._collect_etw_events()
122 |                 self._stop_event.wait(timeout=self.poll_interv
    | al)
123 |             except Exception as ex:
124 |                 logger.error(f'Ошибка в цикле ETW сборщика: {e
    | x}')
125 |

```

```

126     def _collect_etw_events(self) -> None:
127         """Собрать порцию событий из включенных провайдеров."""
128         "
129         if os.name != 'nt' or not self.wevt_api or not self.we
130         vt_api.is_available():
131             return
132
133         if self.enable_process_trace:
134             self._collect_process_events()
135         if self.enable_disk_trace:
136             self._collect_file_events()
137         if self.enable_network_trace:
138             self._collect_network_events()
139         if self.enable_registry_trace:
140             self._collect_registry_events()
141
142     def _collect_security_events(self, event_ids: int | list[i
143     nt], event_type: str, max_events: int = 50) -> None:
144         """Собрать порцию событий из Security журнала Windows
145         через WevtAPI.
146
147         Args:
148             event_ids: Идентификатор или список идентификаторо
149             в событий.
150             event_type: Название типа события для логирования.
151             max_events: Максимальное количество событий в выбо
152             рке.
153
154         """
155         if not self.wevt_api or not self.wevt_api.is_available
156         ():
157             return
158
159         try:
160             raw_events = self.wevt_api.read_events(
161                 channel='Security',
162                 limit=max_events,
163                 event_id=event_ids,
164             )

```

```

157         for ev in raw_events:
158             rec_id = ev.get('record_id')
159             if rec_id:
160                 key = ('Security', rec_id)
161                 if key in self._seen_records:
162                     continue
163                 if len(self._seen_queue) == self._seen_queue.
ue.maxlen:
164                     oldest = self._seen_queue[0]
165                     self._seen_records.discard(oldest)
166                     self._seen_queue.append(key)
167                     self._seen_records.add(key)
168
169                 event_data = ev.get('event_data', {})
170                 if event_data:
171                     self._log_event(event_type, event_data)
172             except Exception as ex:
173                 logger.debug(f'Ошибка сбора событий {event_type} E
TW: {ex}')
174
175     def _collect_process_events(self) -> None:
176         """Собрать события создания процессов (Security ID 468
8)."""
177         self._collect_security_events(4688, 'etw_process_creat
e')
178
179     def _collect_file_events(self) -> None:
180         """Собрать события файловых операций (Security ID 4663
)."""
181         self._collect_security_events(4663, 'etw_file_access')
182
183     def _collect_network_events(self) -> None:
184         """Собрать события сетевых блокировок/подключений WFP
(Security ID 5156/5158)."""
185         self._collect_security_events([5156, 5158], 'etw_netwo
rk_connection')
186
187     def _collect_registry_events(self) -> None:

```

```

188 |         """Собрать события аудита реестра (Security ID 4657)."""
189 |         self._collect_security_events(4657, 'etw_registry_access')
190 |
191 |     def _log_event(self, event_type: str, event_data: Dict[str, Any]) -> None:
192 |         """Зафиксировать событие ETW в оперативной памяти с пакетным сбросом в SQLite.
193 |
194 |         Args:
195 |             event_type: Тип события ETW.
196 |             event_data: Словарь с полями из журнала событий.
197 |         """
198 |         self._event_counter += 1
199 |         now_iso = datetime.now(timezone.utc).isoformat()
200 |
201 |         payload = {
202 |             'event_id': f'etw_{int(time.time())}_{self._event_counter}',
203 |             'timestamp': now_iso,
204 |             'event_type': event_type,
205 |             'provider': 'Microsoft-Windows-Security-Auditing',
206 |             'data': event_data,
207 |         }
208 |         self._last_event_time = now_iso
209 |
210 |         # 1. Сохранение в SQLite TelemetryStorage через буфер
211 |         # памяти
212 |         if self.storage:
213 |             try:
214 |                 db_record = {
215 |                     'event_id': payload['event_id'],
216 |                     'timestamp': now_iso,
217 |                     'event_type': event_type,
218 |                     'path': event_data.get('ObjectName') or event_data.get('NewProcessName'),
219 |                     'pid': event_data.get('ProcessId') or event_data.get('ProcessName')

```



```

    t_data.get('NewProcessId'),
219 |             'name': event_data.get('ProcessName') or e
    vent_data.get('SubjectUserName'),
220 |             'details': event_data,
221 |         }
222 |         self.storage.save_w64_event(db_record, provide
    r='w64_etw_collector')
223 |
224 |         # Специфическая фиксация создания процессов дл
    я Process Provenance
225 |         if event_type == 'etw_process_create':
226 |             def _safe_pid(val: Any) -> int:
227 |                 if val is None:
228 |                     return 0
229 |                 if isinstance(val, int):
230 |                     return val
231 |                 s = str(val).strip()
232 |                 if s.startswith(('0x', '0X')):
233 |                     try:
234 |                         return int(s, 16)
235 |                     except ValueError:
236 |                         return 0
237 |                 try:
238 |                     return int(s)
239 |                 except ValueError:
240 |                     return 0
241 |
242 |             new_pid = _safe_pid(event_data.get('NewPro
    cessId') or event_data.get('ProcessId'))
243 |             parent_pid = _safe_pid(event_data.get('Pro
    cessId') or event_data.get('ParentProcessId'))
244 |             exe_path = event_data.get('NewProcessName'
    ) or event_data.get('ProcessName') or ''
245 |             proc_name = os.path.basename(exe_path) if
    exe_path else 'unknown'
246 |             parent_exe = event_data.get('ParentProcess
    Name') or ''
247 |             parent_name = os.path.basename(parent_exe)

```

```

    if parent_exe else None
248         u_name = event_data.get('SubjectUserName')
249         d_name = event_data.get('SubjectDomainName
    ')
250         full_user = f'{d_name}\\{u_name}' if d_name and u_name and d_name != '-' else u_name
251         u_sid = event_data.get('SubjectUserSid')
252         cmdline = event_data.get('CommandLine')
253         integrity = event_data.get('MandatoryLabel
    ')
254
255         prov_event = ProcessLifecycleEvent(
256             event_type='ProcessCreated',
257             process_guid=f'proc_{new_pid}_{int(time.
    time())}',
258             pid=new_pid,
259             name=proc_name,
260             executable_path=exe_path,
261             command_line=cmdline,
262             user=full_user,
263             sid=u_sid,
264             integrity_level=integrity,
265             parent_pid=parent_pid,
266             parent_name=parent_name,
267             created_at=now_iso,
268             details=event_data,
269         )
270         self.storage.save_process_provenance_event
    (prov_event)
271     except Exception as ex:
272         logger.debug(f'Ошибка сохранения ETW события в
    БД: {ex}')
273
274     # 2. Передача в callback
275     if self.on_event_callback:
276         try:
277             self.on_event_callback(payload)
278         except Exception as ex:

```

```

279         logger.debug(f'Ошибка в ETW on_event_callback:
    {ex}')
280
281     def get_status(self) -> Dict[str, Any]:
282         """Получить диагностический статус ETW сборщика.
283
284         Returns:
285             Dict[str, Any]: Словарь со статусом и счетчиками.
286         """
287         status = W64CollectorStatus(
288             running=self._running,
289             events_count=self._event_counter,
290             last_event_time=self._last_event_time,
291         )
292         return status.model_dump()
293
294     def get_events(self, event_type: Optional[str] = None, limit: int = 100) -> List[Dict[str, Any]]:
295         """Получить список последних событий ETW из SQLite хранилища.
296
297         Args:
298             event_type: Фильтр по типу события.
299             limit: Максимальное число записей.
300
301         Returns:
302             List[Dict[str, Any]]: Список событий.
303         """
304         if self.storage:
305             return self.storage.get_w64_events(event_type=event_type, provider='w64_etw_collector', limit=limit)
306         return []
307

```

__init__.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry Ingestion
4 # - Package Root
5 # =====
6 # Description:
7 #   Пакет коллекторов телеметрии (системные метрики, датчики,
8 #   ETW, токены процессов).
9 #
10 # Usage Examples:
11 #   Python API:
12 #       from apps.windows.telemetry.ingestion.collector import S
13 #       ystemCollector
14 #
15 # File: __init__.py
16 # Project: ai-breadboard
17 # Package: apps.windows.telemetry.ingestion
18 # Author: hypo69
19 # Copyright: © 2026 hypo69
20 # Updated: 2026-10-08 09:36:00
21 # =====
22
23 from __future__ import annotations
24 """Коллекторы сбора телеметрических данных операционной систем
25 ы."""
```

init_db.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - Init Db
4 # =====
5 # Description:
6 #   Модуль проверки, создания и валидации целостности структур
7 #   ы базы данных
8 # Usage Examples:
9 #   CLI:
10 #       python -m apps.windows.telemetry.init_db
11 #   Python API:
12 #       from apps.windows.telemetry.init_db import get_default_t
13 #       telemetry_db_path
14 #       res = get_default_telemetry_db_path()
15 #
16 # File: init_db.py
17 # Project: ai-breadboard
18 # Package: apps.windows.telemetry
19 # Author: hypo69
20 # Copyright: © 2026 hypo69
21 # Updated: 2026-10-06 07:50:00
22 # =====
23
24 from __future__ import annotations
25 """Модуль проверки, создания и валидации целостности структуры
26   базы данных"""
27
28 import argparse
29 import json
30 import os
31 from pathlib import Path
```

```
31 import sqlite3
32 import sys
33 from typing import Any, Dict, List, Optional, Union
34
35 _project_root = str(Path(__file__).resolve().parents[3])
36 if _project_root not in sys.path:
37     sys.path.insert(0, _project_root)
38 if __package__ in (None, ''):
39     __package__ = 'apps.windows.telemetry'
40
41 try:
42     from logger import logger
43 except ImportError:
44     import logging
45     logger = logging.getLogger(__name__)
46
47 from .sqlite import TelemetryStorage
48
49
50 def get_default_telemetry_db_path() -> Path:
51     """Возвращает стандартный путь к файлу базы данных telemetry.db.
52
53     Returns:
54         Path: Путь к файлу telemetry.db в каталоге логов приложения.
55     """
56     programdata = os.environ.get('ProgramData') or os.environ.get('ALLUSERSPROFILE')
57     if programdata and os.path.exists(programdata):
58         base_dir = Path(programdata)
59     else:
60         appdata = os.environ.get('APPDATA') or os.environ.get('LOCALAPPDATA')
61         if appdata and os.path.exists(appdata):
62             base_dir = Path(appdata)
63         else:
64             base_dir = Path.home() / '.config'
```

```

65 |     return base_dir / 'AI-Breadboard' / 'apps' / 'windows' / '
    | telemetry' / 'logs' / 'telemetry.db'
66 |
67 |
68 | def init_telemetry_database(
69 |     db_path: Optional[Union[str, Path]] = None,
70 |     force: bool = False,
71 |     check_integrity: bool = True,
72 | ) -> Dict[str, Any]:
73 |     """Инициализирует базу данных telemetry.db, создавая все т
    | аблицы и индексы при отсутствии.
74 |
75 |     Args:
76 |         db_path: Путь к целевому файлу SQLite базы данных (по
    | умолчанию стандартный путь).
77 |         force: Принудительно переинициализировать структуру та
    | блиц.
78 |         check_integrity: Выполнить проверку целостности PRAGMA
    | integrity_check.
79 |
80 |     Returns:
81 |         Dict[str, Any]: Словарь с результатами инициализации и
    | метаданными БД.
82 |     """
83 |     target_path = Path(db_path) if db_path else get_default_te
    |lemetry_db_path()
84 |     target_path.parent.mkdir(parents=True, exist_ok=True)
85 |
86 |     was_created = False
87 |     if not target_path.exists() or force:
88 |         logger.debug(f'Инициализация новой базы данных телемет
    |рии по пути: {target_path}')
89 |         storage = TelemetryStorage(db_path=target_path)
90 |         was_created = True
91 |     else:
92 |         storage = TelemetryStorage.get_instance(db_path=target
    |_path)
93 |

```

```

94 |     # Получение списка созданных таблиц
95 |     tables: List[str] = []
96 |     integrity_ok = True
97 |     integrity_msg = 'ok'
98 |
99 |     try:
100 |         with sqlite3.connect(str(target_path)) as conn:
101 |             cursor = conn.cursor()
102 |             cursor.execute("SELECT name FROM sqlite_master WHE
RE type='table' AND name NOT LIKE 'sqlite_%';")
103 |             tables = sorted([row[0] for row in cursor.fetchall
104 |                             (())])
105 |
106 |             if check_integrity:
107 |                 cursor.execute('PRAGMA integrity_check;')
108 |                 res = cursor.fetchone()
109 |                 if res and res[0] != 'ok':
110 |                     integrity_ok = False
111 |                     integrity_msg = str(res[0])
112 |     except Exception as ex:
113 |         logger.error(f'Ошибка проверки схемы и целостности БД
114 | {target_path}: {ex}')
115 |         integrity_ok = False
116 |         integrity_msg = str(ex)
117 |
118 |     size_bytes = target_path.stat().st_size if target_path.exi
119 |     sts() else 0
120 |     size_mb = round(size_bytes / (1024 * 1024), 3)
121 |
122 |     result = {
123 |         'status': 'success' if integrity_ok else 'warning',
124 |         'db_path': str(target_path),
125 |         'exists': target_path.exists(),
126 |         'created': was_created,
127 |         'size_bytes': size_bytes,
128 |         'size_mb': size_mb,
129 |         'tables_count': len(tables),
130 |         'tables': tables,

```



```

128         'integrity_ok': integrity_ok,
129         'integrity_message': integrity_msg,
130     }
131
132     if was_created:
133         logger.info(f'    База данных telemetry.db успешно созда
на: {len(tables)} таблиц ({size_mb} МБ)')
134     else:
135         logger.debug(f'База данных telemetry.db подтверждена:
{len(tables)} таблиц ({size_mb} МБ)')
136
137     return result
138
139
140 def main() -> int:
141     """Точка входа CLI для инициализации и проверки базы данны
х телеметрии."""
142     parser = argparse.ArgumentParser(
143         description='Инициализатор и верификатор SQLite базы д
анных системной телеметрии (telemetry.db)'
144     )
145     parser.add_argument('--db-path', type=str, default=None, h
elp='Пользовательский путь к telemetry.db')
146     parser.add_argument('--force', action='store_true', help='
Принудительно переинициализировать структуру')
147     parser.add_argument('--check-only', action='store_true', h
elp='Только проверить существующую БД без создания')
148     parser.add_argument('--json', action='store_true', help='В
ывод результата в формате JSON')
149
150     args = parser.parse_args()
151
152     target_path = Path(args.db_path) if args.db_path else get_
default_telemetry_db_path()
153
154     if args.check_only and not target_path.exists():
155         if args.json:
156             print(json.dumps({'status': 'error', 'message': f'

```

```

    Файл не найден: {target_path}', 'exists': False}, ensure_ascii
    =False))
157         else:
158             print(f'❌ База данных телеметрии не найдена: {target_path}')
159             return 1
160
161     res = init_telemetry_database(db_path=args.db_path, force=
    args.force)
162
163     if args.json:
164         print(json.dumps(res, ensure_ascii=False, indent=2))
165     else:
166         status_icon = '✅ ' if res['integrity_ok'] else '❌ '
167         print(f"\n{status_icon} База данных телеметрии: {res['
    db_path']}")
168         print(f"    • Существует:      {res['exists']}")
169         print(f"    • Вновь создана:  {res['created']}")
170         print(f"    • Размер файла:   {res['size_mb']} МБ ({res
    ['size_bytes']} байт)")
171         print(f"    • Таблиц в схеме: {res['tables_count']}")
172         print(f"    • Целостность:   {res['integrity_message']
    }")
173         if res['tables']:
174             print(f"    • Таблицы:           {'', '.join(res['table
    s'][:8])){'...' if len(res['tables']) > 8 else ''}")
175             print('')
176
177     return 0 if res['integrity_ok'] else 1
178
179
180 if __name__ == '__main__':
181     sys.exit(main())
182

```

internet_speed.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - Internet Speed
4 # =====
5 # Description:
6 #   Сенсор телеметрии скорости интернета и задержки сети на базе FAST.com (Netflix CDN).
7 #   Выполняет периодический замер (1 раз в час / 3600 сек) и кэширует результаты.
8 #
9 # Usage Examples:
10 #   Python API:
11 #       from apps.windows.telemetry.internet_speed import InternetSpeedSensor
12 #
13 #       sensor = InternetSpeedSensor()
14 #       metrics = sensor.measure_internet_speed()
15 #       print(metrics['download_mbps'], metrics['bufferbloat_ms'])
16 #
17 # File: internet_speed.py
18 # Project: ai-breadboard
19 # Package: apps.windows.telemetry
20 # Author: hypo69
21 # Copyright: © 2026 hypo69
22 # Updated: 2026-10-04 07:35:00
23 # =====
24
25 from __future__ import annotations
26 """Сенсор телеметрии скорости интернета и задержки сети на базе FAST.com."""
27
```

```

28 import asyncio
29 import threading
30 import time
31 from typing import Any, Dict, List, Optional
32
33 from logger import logger
34 from apps.windows.sdk.modules.network.speedtest import Network
    SpeedTester
35
36 _speed_cache: Optional[tuple[float, Dict[str, Any]]] = None
37 _CACHE_TTL = 3600 # 1 час между замерами скорости для снижени
    я нагрузки на сеть
38 _cache_lock = threading.Lock()
39 _bg_test_running = False
40 _bg_lock = threading.Lock()
41
42
43 class InternetSpeedSensor:
44     """Сенсор измерения скорости интернета и характеристик заде
    ржки (FAST.com)."""
45
46     def __init__(self, cache_ttl: int = _CACHE_TTL) -> None:
47         """Инициализация сенсора скорости интернета.
48
49         Args:
50             cache_ttl: Время жизни кеша замера в секундах (по
    умолчанию 3600 сек = 1 час).
51         """
52         self.cache_ttl = cache_ttl
53         self._speedtester = NetworkSpeedTester()
54
55     @classmethod
56     def clear_cache(cls) -> None:
57         """Очистить глобальный кеш замеров скорости интернета.
58         """
59
60         global _speed_cache
61         with _cache_lock:
62             _speed_cache = None

```

```

61 |
62 |     def _run_speedtest_sync(self) -> Dict[str, Any]:
63 |         """Синхронно выполнить полный цикл теста скорости FAST
64 |         .com."""
65 |         try:
66 |             loop = None
67 |             try:
68 |                 loop = asyncio.get_event_loop()
69 |             except RuntimeError:
70 |                 loop = None
71 |
72 |             if loop and loop.is_running():
73 |                 # Если вызывается внутри уже работающего цикла
74 |                 событий (например, в async worker)
75 |                 import concurrent.futures
76 |                 with concurrent.futures.ThreadPoolExecutor(max
77 |                 _workers=1) as executor:
78 |                     future = executor.submit(lambda: asyncio.r
79 |                     un(self._speedtester.run_full_speedtest()))
80 |                     return future.result(timeout=60.0)
81 |             else:
82 |                 return asyncio.run(self._speedtester.run_full_
83 |                 speedtest())
84 |         except Exception as ex:
85 |             logger.error(f'Ошибка выполнения теста скорости FA
86 |             ST.com в сенсоре телеметрии: {ex}')
87 |             return {
88 |                 'download': {'speed_mbps': 0.0, 'status': 'ERR
89 |                 OR'},
90 |                 'upload': {'speed_mbps': 0.0, 'status': 'ERROR
91 |                 '},
92 |                 'ping_ms': 0.0,
93 |                 'latency_unloaded_ms': 0.0,
94 |                 'latency_loaded_ms': 0.0,
95 |                 'bufferbloat_ms': 0.0,
96 |                 'meta': {'ip': 'N/A', 'isp': 'Unknown'},
97 |                 'provider': 'fast.com',
98 |                 'server': 'N/A'

```

```

91         }
92
93     def _perform_measurement_and_cache(self) -> Dict[str, Any]
94     :
95         """Выполняет замер скорости и сохраняет его в кэш."""
96         global _speed_cache, _bg_test_running
97         try:
98             report = self._run_speedtest_sync()
99             download_mbps = float(report.get('download', {}).g
100 et('speed_mbps', 0.0))
101             upload_mbps = float(report.get('upload', {}).get('
102 speed_mbps', 0.0))
103             ping_ms = float(report.get('ping_ms', report.get('
104 latency_unloaded_ms', 0.0)))
105             latency_loaded_ms = float(report.get('latency_load
106 ed_ms', 0.0))
107             bufferbloat_ms = float(report.get('bufferbloat_ms'
108 , 0.0))
109             meta = report.get('meta', {})
110
111             result: Dict[str, Any] = {
112                 'available': report.get('download', {}).get('s
113 tatus') == 'SUCCESS' or download_mbps > 0.0,
114                 'download_mbps': download_mbps,
115                 'upload_mbps': upload_mbps,
116                 'ping_ms': ping_ms,
117                 'latency_unloaded_ms': float(report.get('laten
118 cy_unloaded_ms', ping_ms)),
119                 'latency_loaded_ms': latency_loaded_ms,
120                 'bufferbloat_ms': bufferbloat_ms,
121                 'provider': 'fast.com',
122                 'server': report.get('server', 'FAST.com CDN')
123             },
124             'ip': meta.get('ip', 'N/A'),
125             'isp': meta.get('isp', 'Unknown ISP'),
126             'measured_at': time.time(),
127         }
128     }
129

```

```

120         with _cache_lock:
121             _speed_cache = (time.monotonic(), result)
122
123         logger.info(
124             f' [Телеметрия FAST.com] Скорость: ↓{download_mbps:.1f} Mbps, '
125             f'↑{upload_mbps:.1f} Mbps, пинг: {ping_ms:.1f} мс, bufferbloat: {bufferbloat_ms:.1f} мс'
126         )
127         return dict(result)
128     finally:
129         with _bg_lock:
130             _bg_test_running = False
131
132     def trigger_async_measure(self) -> None:
133         """Запускает измерение скорости в фоновом потоке, если
134         оно еще не выполняется."""
135         global _bg_test_running
136         with _bg_lock:
137             if _bg_test_running:
138                 return
139             _bg_test_running = True
140
141         thread = threading.Thread(
142             target=self._perform_measurement_and_cache,
143             name="InternetSpeedTestWorker",
144             daemon=True
145         )
146         thread.start()
147
148     def measure_internet_speed(self, force_refresh: bool = False, sync: bool = False) -> Dict[str, Any]:
149         """Измеряет профиль интернет-соединения через FAST.com
150         с кешированием на 1 час.
151
152         По умолчанию выполняется асинхронно в фоне и не блокирует вызывающий поток.

```

```

152 |         Args:
153 |             force_refresh: Принудительно инициировать новый за
мер.
154 |             sync: Выполнить замер синхронно с блокировкой теку
щего потока.
155 |
156 |         Returns:
157 |             Dict[str, Any]: Словарь метрик (download_mbps, upl
oad_mbps, ping_ms, bufferbloat_ms, etc.).
158 |             """
159 |         global _speed_cache
160 |         with _cache_lock:
161 |             if not force_refresh and _speed_cache is not None:
162 |                 cached_at, cached_result = _speed_cache
163 |                 if time.monotonic() - cached_at < self.cache_t
tl:
164 |                     return dict(cached_result)
165 |
166 |         if sync:
167 |             return self._perform_measurement_and_cache()
168 |
169 |         # Асинхронный запуск замера в фоне
170 |         self.trigger_async_measure()
171 |
172 |         with _cache_lock:
173 |             if _speed_cache is not None:
174 |                 _, cached_result = _speed_cache
175 |                 return dict(cached_result)
176 |
177 |         # Возвращаем заглушку по умолчанию без блокировки пото
ка
178 |         return {
179 |             'available': True,
180 |             'download_mbps': 0.0,
181 |             'upload_mbps': 0.0,
182 |             'ping_ms': 0.0,
183 |             'latency_unloaded_ms': 0.0,
184 |             'latency_loaded_ms': 0.0,

```



```

185         'bufferbloat_ms': 0.0,
186         'provider': 'fast.com',
187         'server': 'FAST.com CDN (замер в фоне...)',
188         'ip': 'N/A',
189         'isp': 'Unknown ISP',
190         'measured_at': 0.0,
191     }
192
193
194 def get_internet_speed_sensors() -> List[Dict[str, Any]]:
195     """Получить показания сетевых метрик скорости в формате сп
иска сенсоров.
196
197     Returns:
198         List[Dict[str, Any]]: Список метрик сенсоров интернета
199     """
200     sensor = InternetSpeedSensor()
201     metrics = sensor.measure_internet_speed()
202     sensors = [
203         {
204             'sensor_id': 'internet_ping',
205             'name': 'Internet Ping Latency',
206             'category': 'network',
207             'value': metrics.get('ping_ms', 0.0),
208             'unit': 'ms'
209         },
210         {
211             'sensor_id': 'internet_download',
212             'name': 'Internet Download Speed',
213             'category': 'network',
214             'value': metrics.get('download_mbps', 0.0),
215             'unit': 'Mbps'
216         },
217         {
218             'sensor_id': 'internet_upload',
219             'name': 'Internet Upload Speed',
220             'category': 'network',

```

```
221         'value': metrics.get('upload_mbps', 0.0),
222         'unit': 'Mbps'
223     },
224     {
225         'sensor_id': 'internet_bufferbloat',
226         'name': 'Internet Bufferbloat',
227         'category': 'network',
228         'value': metrics.get('bufferbloat_ms', 0.0),
229         'unit': 'ms'
230     },
231     {
232         'sensor_id': 'internet_latency_loaded',
233         'name': 'Internet Loaded Latency',
234         'category': 'network',
235         'value': metrics.get('latency_loaded_ms', 0.0),
236         'unit': 'ms'
237     },
238 ]
239 return sensors
```

json_logger.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - Json Lo
  gger
4 # =====
5 # Description:
6 #   Модуль JSON-логгера телеметрии Windows.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.json_logger import Telemetry
      JsonLogger
11 #
12 #       service = TelemetryJsonLogger()
13 #
14 # File: json_logger.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-01 13:28:28
20 # =====
21
22 from __future__ import annotations
23 """Модуль JSON-логгера телеметрии Windows."""
24
25 import json
26 import logging
27 import os
28 import time
29 from pathlib import Path
30 from typing import List, Optional
31
```

```

32 _logger = logging.getLogger(__name__)
33
34
35 class TelemetryJsonLogger:
36     """Записывает телеметрические события в JSON-Lines файл.
37
38     Параметры
39     -----
40     log_dir: str
41         Путь к директории, где будут храниться лог-файлы.
42     filename: str
43         Базовое имя лог-файла (без расширения).
44     max_file_size_mb: float, optional
45         Максимальный размер файла в мегабайтах до ротации. По
46         умолчанию 5 МБ.
47     """
48     def __init__(self, log_dir: str, filename: str, max_file_s
49         ize_mb: float = 5.0) -> None:
50         self.log_dir = Path(log_dir)
51         self.log_dir.mkdir(parents=True, exist_ok=True)
52         self.base_filename = filename if filename.endswith('.j
53         sonl') else f'{filename}.jsonl'
54         self.max_bytes = int(max_file_size_mb * 1024 * 1024)
55         self.current_path = self.log_dir / self.base_filename
56         _logger.debug('Инициализирован TelemetryJsonLogger: di
57         r=%s, file=%s, max_bytes=%d', self.log_dir, self.base_filename
58         , self.max_bytes)
59
60     def _rotate_if_needed(self) -> None:
61         """Ротирует текущий лог-файл, если его размер превышает
62         лимит.
63
64         Новый файл получает суффикс с меткой времени.
65     """
66     if self.current_path.exists() and self.current_path.st
67         at().st_size >= self.max_bytes:
68         timestamp = time.strftime('%Y%m%d%H%M%S')
69         rotated_name = f"{self.base_filename.rstrip('.json

```

```

l')}_{{timestamp}}.jsonl"
63         rotated_path = self.log_dir / rotated_name
64         try:
65             self.current_path.rename(rotated_path)
66             _logger.info('Файл лога ротирован: %s -> %s',
self.current_path.name, rotated_name)
67         except OSError as exc:
68             _logger.error('Не удалось ротировать лог-файл:
%s', exc)
69
70     def _write_record(self, record: dict) -> None:
71         """Записывает одну строку JSON в текущий файл.
72         Файл открывается в режиме добавления.
73         """
74         line = json.dumps(record, ensure_ascii=False)
75         with self.current_path.open('a', encoding='utf-8') as
f:
76             f.write(line + '\n')
77         _logger.debug('Записана запись в %s: %s', self.current
_path.name, line)
78
79     def log(self, record: dict) -> bool:
80         """Записать одну запись.
81
82         Возвращает ``True`` при успешной записи.
83         """
84         try:
85             self._rotate_if_needed()
86             self._write_record(record)
87             return True
88         except Exception as exc:
89             _logger.exception('Ошибка записи лога: %s', exc)
90             return False
91
92     def log_batch(self, records: List[dict]) -> int:
93         """Записать список записей.
94
95         Возвращает количество успешно записанных элементов.

```

```

96         """
97         count = 0
98         for rec in records:
99             if self.log(rec):
100                 count += 1
101         return count
102
103     def get_last_measurement(self) -> Optional[dict]:
104         """Вернуть последнюю запись из текущего лог-файла, если
и файл существует.
105         """
106         if not self.current_path.exists():
107             return None
108         try:
109             with self.current_path.open('rb') as f:
110                 f.seek(0, os.SEEK_END)
111                 offset = -1
112                 while -offset < f.tell():
113                     f.seek(offset, os.SEEK_END)
114                     if f.read(1) == b'\n' and offset != -1:
115                         break
116                     offset -= 1
117                 f.seek(offset + 1, os.SEEK_END)
118                 last_line = f.readline().decode('utf-8').strip
119             ()
120                 return json.loads(last_line) if last_line else
None
121         except Exception as exc:
122             _logger.exception('Не удалось прочитать последнюю
запись: %s', exc)
123             return None
124
125     def get_log_file_size(self) -> int:
126         """Вернуть размер текущего лог-файла в байтах.
127         """
128         return self.current_path.stat().st_size if self.curren
t_path.exists() else 0

```

main.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - Main
4 # =====
5 # Description:
6 #   Главная точка входа для сервиса сбора телеметрии Windows.
7 #
8 # Usage Examples:
9 #   CLI:
10 #       python -m apps.windows.telemetry.main
11 #   Python API:
12 #       from apps.windows.telemetry.main import parse_arguments
13 #
14 #       res = parse_arguments()
15 #
16 # File: main.py
17 # Project: ai-breadboard
18 # Package: apps.windows.telemetry
19 # Author: hypo69
20 # Copyright: © 2026 hypo69
21 # Updated: 2026-10-10 05:40:00
22 # =====
23
24 from __future__ import annotations
25 """Главная точка входа для сервиса сбора телеметрии Windows."""
26
27 import argparse
28 import os
29 import signal
30 import sys
31 import threading
32 import time
```

```

33 from datetime import datetime, timezone
34 from pathlib import Path
35 from typing import Any, Dict, List, Optional
36 _project_root = str(Path(__file__).resolve().parents[3])
37 if _project_root not in sys.path:
38     sys.path.insert(0, _project_root)
39 if __package__ in (None, ''):
40     __package__ = 'apps.windows.telemetry'
41 try:
42     from logger import logger
43 except ImportError:
44     from logger import logger
45 _stop_event = threading.Event()
46
47 def parse_arguments() -> argparse.Namespace:
48     """Парсит аргументы командной строки.
49
50     Returns:
51         argparse.Namespace: Распарсенные аргументы.
52     """
53     parser = argparse.ArgumentParser(description='Windows Tele
54     metry Service (Minimal, Hybrid, Full)', formatter_class=argpar
55     se.RawDescriptionHelpFormatter)
56     parser.add_argument('--mode', choices=['minimal', 'hybrid'
57     , 'full'], default=None, help='Режим работы: minimal (легковес
58     ный), hybrid (минимал + тяжелые периодически), full (полный ст
59     ек)')
60     parser.add_argument('--minimal', action='store_true', help
61     ='Запустить в легковесном режиме (эквивалентно --mode minimal)
62     ')
63     parser.add_argument('--interval', type=float, default=None
64     , help='Интервал сбора быстрой телеметрии в секундах (по умолч
65     анию из config.json)')
66     parser.add_argument('--heavy-interval', type=float, defaul
67     t=None, help='Интервал сбора тяжелых сенсоров для hybrid режим
68     а в секундах (по умолчанию из config.json)')
69     parser.add_argument('--top-processes', type=int, default=N
70     one, help='Количество сохраняемых процессов с наибольшей нагру

```



```

    зкой (по умолчанию 10)')
59     parser.add_argument('--config', type=str, default=None, help='Путь к файлу конфигурации config.json')
60     parser.add_argument('--log-dir', type=str, default=None, help='Путь к директории для логов и SQLite БД телеметрии')
61     parser.add_argument('--verbose', '-v', action='store_true', help='Включить подробный вывод (DEBUG)')
62     parser.add_argument('--once', action='store_true', help='Выполнить однократный опрос метрик и завершить работу')
63     return parser.parse_args()
64
65 def signal_handler(signum: int, frame: Optional[object]) -> None:
66     """Обработчик сигналов для корректной остановки сервиса.
67
68     Args:
69         signum: Номер сигнала.
70         frame: Стек вызовов.
71     """
72     logger.info(f'Получен сигнал {signum}, остановка телеметрии...')
73     _stop_event.set()
74
75 def _set_low_priority() -> None:
76     """Устанавливает пониженный приоритет процессу для экономии
77     и ресурсов CPU."""
78     try:
79         import psutil
80         p = psutil.Process()
81         p.nice(psutil.BELOW_NORMAL_PRIORITY_CLASS)
82         logger.debug('Установлен приоритет процесса: BELOW_NORMAL')
83     except Exception as ex:
84         logger.debug(f'Не удалось понизить приоритет процесса: {ex}')
85
86 def run_telemetry_service(
    mode: str = 'hybrid',

```

```

87     interval: float = 5.0,
88     heavy_interval: float = 60.0,
89     top_processes: int = 10,
90     heavy_collectors: Optional[Dict[str, bool]] = None,
91     log_dir: Optional[str] = None,
92     config_path: Optional[str] = None,
93     once: bool = False,
94 ) -> int:
95     """Запускает универсальный цикл сбора телеметрии (Minimal,
    Hybrid или Full).
96
97     Args:
98         mode: Режим сбора ('minimal', 'hybrid', 'full').
99         interval: Быстрый интервал сбора базовых метрик в секундах.
100        heavy_interval: Периодический интервал сбора тяжелых сенсоров (для hybrid).
101        top_processes: Количество процессов в топе.
102        heavy_collectors: Словарь активных тяжелых сенсоров.
103        log_dir: Каталог сохранения БД и логов.
104        config_path: Путь к файлу конфигурации.
105
106     Returns:
107         int: Код завершения.
108     """
109     import psutil
110     from apps.windows.sdk.modules.hardware.gpu_prober import GpuProber
111     from .models import CpuMetrics, DiskIoMetrics, DiskPartitionMetrics, GpuMetrics, MemoryMetrics, NetworkInterfaceMetrics, ProcessMetrics, SystemSnapshot
112     from .sqlite import TelemetryStorage
113     from .telemetry_config import TelemetryConfigManager
114     _set_low_priority()
115     cfg_mgr = TelemetryConfigManager(config_path=config_path)
116     resolved_log_dir = log_dir or str(cfg_mgr.get_data_path())
117     os.makedirs(resolved_log_dir, exist_ok=True)
118     db_path = os.path.join(resolved_log_dir, 'telemetry.db')

```

```
119     storage = TelemetryStorage(db_path=db_path)
120     gpu_prober = GpuProber()
121     hostname = os.environ.get('COMPUTERNAME', 'localhost')
122     boot_time = psutil.boot_time()
123     psutil.cpu_percent(interval=None)
124
125     custom_mode_set = mode is not None and ('--mode' in sys.ar
gv or '--minimal' in sys.argv)
126     custom_interval_set = interval is not None and ('--interva
l' in sys.argv or '-i' in sys.argv)
127     custom_heavy_interval_set = heavy_interval is not None and
 '--heavy-interval' in sys.argv
128     custom_top_processes_set = top_processes is not None and '
--top-processes' in sys.argv
129
130     if not custom_mode_set and cfg_mgr:
131         mode = cfg_mgr.get_mode()
132     if not custom_interval_set and cfg_mgr:
133         interval = cfg_mgr.get_interval_seconds()
134     if not custom_heavy_interval_set and cfg_mgr:
135         heavy_interval = cfg_mgr.get_heavy_interval_seconds()
136     if not custom_top_processes_set and cfg_mgr:
137         top_processes = cfg_mgr.get_effective_process_limit()
138
139     heavy_disk_interval_sec = cfg_mgr.get_heavy_disk_scan_inte
rval()
140     max_heavy_duration_sec = cfg_mgr.get_heavy_mode_max_durati
on_days() * 24.0 * 3600.0
141     auto_switch_enabled = cfg_mgr.is_heavy_auto_switch_enabled
()
142
143     h_collectors = heavy_collectors or cfg_mgr.get_heavy_colle
ctors()
144     sensor_collector = None
145     if mode in ('hybrid', 'full') and h_collectors.get('hardwa
re_sensors', True):
146         try:
147             from .sensor_collector import SensorCollector
```

```

148         sensor_collector = SensorCollector(config_manager=
| cfg_mgr)
149         logger.info('SensorCollector (LHM/Hardware) инициа
| лизирован для тяжелых опросов')
150         except Exception as sc_err:
151             logger.debug(f'SensorCollector не доступен: {sc_er
| r}')
152     system_collector = None
153     deep_diag = None
154     try:
155         from .collector import SystemCollector
156         from apps.windows.telemetry_research.deep_diagnostics
| import DeepDiagnosticsEngine
157         system_collector = SystemCollector(storage=storage)
158         deep_diag = DeepDiagnosticsEngine()
159         identity = system_collector.get_system_identity()
160         disks_health = system_collector.get_physical_disks_heal
| th(force=True)
161         import asyncio
162         hw_tree = asyncio.run(system_collector.get_hardware_tr
| ee_async(force=True))
163         logger.info(f"□ Хост: {identity.get('hostname')} | По
| льзователь: {identity.get('username')} | ОС: Windows {identity
| .get('os_build')}")
164         logger.info(f"□ Язык: {identity.get('system_language'
| )} | Локаль: {identity.get('user_locale')} | Раскладки: {' , '
| .join(identity.get('input_languages', []))}")
165         if disks_health:
166             d_summary = " , ".join([f"{d.model} ({d.media_type}
| , {d.size_gb}GB, SMART:{d.health_status})" for d in disks_heal
| th])
167             logger.info(f"□ Накопители SMART (Опрос при запус
| ке): {d_summary}")
168             if hw_tree:
169                 logger.info(f"□ [АППАРАТНАЯ КОНФИГУРАЦИЯ] Специфи
| кация всего железа ПК определена при старте процесса ({len(hw_
| tree)} компонентов)")
170     except Exception as init_err:

```

```

171         logger.debug(f"Инициализация системного коллектора: {i
nit_err}")
172
173     # Проверка критических аудитов при запуске
174     try:
175         from apps.windows.telemetry_research.audit_startup_che
cker import run_startup_audit
176         startup_audit = run_startup_audit(
177             check_integrity=True,
178             check_performance=True,
179             check_drivers=False,
180             check_eventlog=False
181         )
182         if not startup_audit.is_healthy:
183             logger.error(
184                 f"⚠ STARTUP AUDIT: Обнаружены критические про
блемы! "
185                 f"Critical: {startup_audit.critical_count}, Wa
rnings: {startup_audit.warning_count}"
186             )
187             for finding in startup_audit.findings:
188                 logger.warning(f" - [{finding['severity']}.upp
er()] {finding['domain']}: {finding['title']}")
189             elif startup_audit.warning_count > 0:
190                 logger.warning(
191                     f"⚠ STARTUP AUDIT: Предупреждения: {startup_au
dit.warning_count} "
192                     f"({startup_audit.duration_ms}ms)"
193                 )
194             else:
195                 logger.info(f"⚠ STARTUP AUDIT: Все проверки пройде
ны ({startup_audit.duration_ms}ms)")
196         except Exception as audit_err:
197             logger.debug(f"Проверка аудитов при запуске: {audit_er
r}")
198
199     # Инициализация W64/ETW сборщиков системных событий
200     w64_collector = None

```

```
201     w64_etw_collector = None
202     w64_cfg = cfg_mgr.get_config().get('w64_collector', {}) if
    cfg_mgr else {}
203     if w64_cfg.get('enabled', True):
204         try:
205             from .w64_collector import AIW64Collector
206             from .w64_etw_collector import AIW64ETWCollector
207             w64_collector = AIW64Collector(
208                 storage=storage,
209                 enable_file_monitoring=w64_cfg.get('enable_fil
    e_monitoring', True),
210                 enable_process_monitoring=w64_cfg.get('enable_
    process_monitoring', True),
211                 enable_registry_monitoring=w64_cfg.get('enable
    _registry_monitoring', True),
212                 enable_network_monitoring=w64_cfg.get('enable_
    network_monitoring', True),
213                 enable_event_log_monitoring=w64_cfg.get('enabl
    e_event_log_monitoring', True),
214             )
215             w64_collector.start()
216             w64_etw_collector = AIW64ETWCollector(
217                 storage=storage,
218                 enable_process_trace=w64_cfg.get('enable_proce
    ss_trace', True),
219                 enable_disk_trace=w64_cfg.get('enable_disk_tra
    ce', True),
220                 enable_network_trace=w64_cfg.get('enable_netwo
    rk_trace', True),
221                 enable_registry_trace=w64_cfg.get('enable_regi
    stry_trace', True),
222             )
223             w64_etw_collector.start()
224             logger.info('  Сборщики событий W64 и ETW запущен
    ы в составе телеметрии')
225         except Exception as w64_err:
226             logger.debug(f'Не удалось запустить W64 сборщики:
    {w64_err}')
```

```

227
228     logger.info('=' * 60)
229     logger.info(f'Запущен сервис телеметрии Windows [Режим: {mode.upper()}]')
230     logger.info(f'Быстрый интервал: {interval} с | Топ процесс
ов: {top_processes}')
231     if mode == 'hybrid':
232         logger.info(f'Периодический тяжелый опрос: каждые {heavy_interval} с')
233     logger.info(f'База данных: {db_path}')
234     logger.info('=' * 60)
235     tick = 0
236     prev_disk_io = psutil.disk_io_counters()
237     prev_net_io = psutil.net_io_counters()
238     prev_io_time = time.time()
239     last_gpu_probe_time: Optional[float] = None
240     cached_gpu_metrics: List[GpuMetrics] = []
241     last_partition_check: float = 0.0
242     cached_part_list: List[Any] = []
243     last_heavy_time = time.time()
244     last_heavy_disk_scan_time = time.time()
245     heavy_mode_start_time: Optional[float] = time.time() if mode.lower() in ('full', 'heavy') else None
246     heavy_poll_running = False
247     heavy_poll_lock = threading.Lock()
248     db_cleanup_interval_sec = cfg_mgr.get_db_cleanup_interval_seconds()
249     last_db_cleanup_time = time.time()
250     aggregation_interval_sec = getattr(cfg_mgr, 'get_aggregation_interval', lambda: 300.0)() if cfg_mgr else 300.0
251     last_aggregation_time = time.time()
252     known_pids: Dict[int, str] = {}
253     flashing_log_path = Path(resolved_log_dir) / 'flashing_processes.log'
254
255     try:
256         while not _stop_event.is_set():
257             loop_start = time.time()

```

```

258 |
259 |         # Динамическая перезагрузка параметров из config.j
    | son 'на лету'
260 |         cfg_mgr.check_tc_mode_auto_switch()
261 |         if cfg_mgr.check_and_reload():
262 |             if not custom_interval_set:
263 |                 interval = cfg_mgr.get_interval_seconds()
264 |             if not custom_heavy_interval_set:
265 |                 heavy_interval = cfg_mgr.get_heavy_interva
    | l_seconds()
266 |             if not custom_top_processes_set:
267 |                 top_processes = cfg_mgr.get_effective_proc
    | ess_limit()
268 |             if not custom_mode_set:
269 |                 mode = cfg_mgr.get_mode()
270 |                 heavy_disk_interval_sec = cfg_mgr.get_heavy_di
    | sk_scan_interval()
271 |                 db_cleanup_interval_sec = cfg_mgr.get_db_clean
    | up_interval_seconds()
272 |                 max_heavy_duration_sec = cfg_mgr.get_heavy_mod
    | e_max_duration_days() * 24.0 * 3600.0
273 |                 auto_switch_enabled = cfg_mgr.is_heavy_auto_sw
    | itch_enabled()
274 |                 h_collectors = cfg_mgr.get_heavy_collectors()
275 |                 logger.info(
276 |                     f" [Телеметрия CLI] Конфигурация config.
    | json обновлена 'на лету': "
277 |                     f"интервал={interval}с, тяжелый={heavy_int
    | erval}с, топ={top_processes}, режим={mode}, tel_mode={cfg_mgr.
    | get_telemetry_mode()}"
278 |                 )
279 |
280 |         now_dt = datetime.now(timezone.utc)
281 |         now_iso = now_dt.isoformat()
282 |         tick += 1
283 |         cpu_pct = psutil.cpu_percent(interval=None)
284 |         cpu_freq = psutil.cpu_freq()
285 |         freq_mhz = cpu_freq.current if cpu_freq else 0.0

```



```

286 |         cpu_model = ''
287 |         try:
288 |             cpu_model = os.environ.get('PROCESSOR_IDENTIFI
| ER', '')
289 |         except Exception:
290 |             pass
291 |         cpu_metrics = CpuMetrics(
292 |             model=cpu_model,
293 |             physical_cores=psutil.cpu_count(logical=False)
| or 1,
294 |             logical_cores=psutil.cpu_count(logical=True) o
| r 1,
295 |             total_percent=cpu_pct,
296 |             frequency_mhz=freq_mhz,
297 |             temperature_celsius=None,
298 |         )
299 |         mem = psutil.virtual_memory()
300 |         mem_metrics = MemoryMetrics(total_gb=round(mem.tot
| al / (1024 ** 3), 2), used_gb=round(mem.used / (1024 ** 3), 2)
| , available_gb=round(mem.available / (1024 ** 3), 2), percent=
| mem.percent)
301 |         now_sec = time.time()
302 |         if last_gpu_probe_time is None or now_sec - last_g
| pu_probe_time >= 10.0:
303 |             try:
304 |                 gpu_data = gpu_prober.probe()
305 |                 cached_gpu_metrics = [
306 |                     GpuMetrics(
307 |                         name=getattr(g, 'name', 'GPU'),
308 |                         vendor=getattr(g, 'vendor', ''),
309 |                         gpu_type=getattr(g, 'gpu_type', 'D
| iscrete'),
310 |                         load_percent=getattr(g, 'load_perc
| ent', None) or getattr(g, 'utilization_gpu_pct', None),
311 |                         memory_total_gb=round((getattr(g,
| 'memory_total_mb', 0.0) or 0.0) / 1024.0, 2),
312 |                         memory_used_gb=round((getattr(g, '
| memory_used_mb', 0.0) or 0.0) / 1024.0, 2),

```

```

313         dedicated_memory_mb=getattr(g, 'dedicated_memory_mb', 0.0),
314         shared_memory_mb=getattr(g, 'shared_memory_mb', 0.0),
315         memory_type=getattr(g, 'memory_type', 'Dedicated VRAM'),
316         directx_version=getattr(g, 'directx_version', 'DirectX 12'),
317         temperature_celsius=getattr(g, 'temperature_celsius', None) or getattr(g, 'temperature_gpu_c', None),
318         has_cuda=getattr(g, 'has_cuda', False),
319         has_rocm=getattr(g, 'has_rocm', False),
320         has_oneapi=getattr(g, 'has_oneapi', False),
321         has_directml=getattr(g, 'has_directml', True),
322         has_vulkan=getattr(g, 'has_vulkan', True),
323         has_opengl=getattr(g, 'has_opengl', True),
324         compute_apis=getattr(g, 'compute_apis', []),
325         ai_backends=getattr(g, 'ai_backends', []),
326         engines=getattr(g, 'engines', {}),
327     ) for g in gpu_data
328 ]
329 except Exception:
330     cached_gpu_metrics = []
331     last_gpu_probe_time = now_sec
332     gpu_metrics_list = cached_gpu_metrics
333     io_elapsed = max(0.001, now_sec - prev_io_time)
334     cur_disk_io = psutil.disk_io_counters()
335     disk_read_kb = 0.0
336     disk_write_kb = 0.0

```

```

337         if cur_disk_io and prev_disk_io:
338             disk_read_kb = round((cur_disk_io.read_bytes -
339 | prev_disk_io.read_bytes) / 1024.0 / io_elapsed, 1)
340             disk_write_kb = round((cur_disk_io.write_bytes
341 | - prev_disk_io.write_bytes) / 1024.0 / io_elapsed, 1)
342             prev_disk_io = cur_disk_io
343             if not cached_part_list or now_sec - last_partitio
344 | n_check >= 60.0:
345                 try:
346                     cached_part_list = psutil.disk_partitions(
347 | all=False)
348                     last_partition_check = now_sec
349                 except Exception:
350                     pass
351                 disk_partitions: List[DiskPartitionMetrics] = []
352                 for part in cached_part_list:
353                     try:
354                         usage = psutil.disk_usage(part.mountpoint)
355                         disk_partitions.append(DiskPartitionMetric
356 | s(device=part.device, mountpoint=part.mountpoint, fstype=part.
357 | fstype, total_gb=round(usage.total / 1024 ** 3, 2), used_gb=ro
358 | und(usage.used / 1024 ** 3, 2), free_gb=round(usage.free / 102
359 | 4 ** 3, 2), percent=usage.percent))
360                     except (PermissionError, OSError):
361                         continue
362                 cur_net_io = psutil.net_io_counters()
363                 net_recv_kb = 0.0
364                 net_sent_kb = 0.0
365                 if cur_net_io and prev_net_io:
366                     net_recv_kb = round((cur_net_io.bytes_recv - p
367 | rev_net_io.bytes_recv) / 1024.0 / io_elapsed, 1)
368                     net_sent_kb = round((cur_net_io.bytes_sent - p
369 | rev_net_io.bytes_sent) / 1024.0 / io_elapsed, 1)
370                 prev_net_io = cur_net_io
371                 prev_io_time = now_sec
372                 net_metrics_list = [NetworkInterfaceMetrics(interf
373 | ace_name='total', bytes_sent_sec=net_sent_kb * 1024.0, bytes_r
374 | ecv_sec=net_recv_kb * 1024.0, packets_sent_sec=0.0, packets_re

```

```

cv_sec=0.0, errors_in=cur_net_io.errin if cur_net_io else 0, e
rrors_out=cur_net_io.errout if cur_net_io else 0)]
363         raw_procs = []
364         current_proc_map: Dict[int, Dict[str, Any]] = {}
365         for p in psutil.process_iter(['pid', 'name', 'memo
ry_info', 'username']):
366             try:
367                 info = p.info
368                 pid = info.get('pid', 0)
369                 name = info.get('name') or 'unknown'
370                 if pid:
371                     current_proc_map[pid] = {'name': name,
'proc': p, 'info': info}
372                     mem_rss = (info.get('memory_info') or geta
ttr(p, 'memory_info', lambda: None()).rss if info.get('memory
_info') else 0
373                     raw_procs.append((mem_rss, p, info))
374             except (psutil.NoSuchProcess, psutil.AccessDen
ied):
375                 continue
376
377         # Детектор вспыхивающих процессов (новые PID, заре
гистрированные на текущем тике)
378         if tick > 1:
379             new_pids = set(current_proc_map.keys()) - set(
known_pids.keys())
380             if new_pids:
381                 for n_pid in new_pids:
382                     p_entry = current_proc_map[n_pid]
383                     p_obj = p_entry['proc']
384                     p_name = p_entry['name']
385                     exe_path = 'N/A'
386                     cmdline_str = 'N/A'
387                     parent_info = 'N/A'
388                     try:
389                         exe_path = p_obj.exe()
390                     except Exception:
391                         pass

```

```

392         try:
393             cmdline_str = " ".join(p_obj.cmdli
ne())
394         except Exception:
395             pass
396         try:
397             ppid = p_obj.ppid()
398             parent_name = psutil.Process(ppid)
.name() if ppid else 'Unknown'
399             parent_info = f"{parent_name} (PID
: {ppid})"
400         except Exception:
401             pass
402
403         alert_msg = (
404             f" [ВСПЫШКА / НОВЫЙ ПРОЦЕСС] PID:
{n_pid} | {p_name} | "
405             f"Путь: {exe_path} | Родитель: {pa
rent_info} | Команда: {cmdline_str}"
406         )
407         logger.warning(alert_msg)
408         try:
409             with open(flashes_log_path, 'a',
encoding='utf-8') as f_out:
410                 f_out.write(f"[{datetime.now()
.strftime('%Y-%m-%d %H:%M:%S')}] PID: {n_pid} | {p_name} | Exe
: {exe_path} | Parent: {parent_info} | Cmd: {cmdline_str}\n")
411         except Exception:
412             pass
413
414         known_pids = {pid: data['name'] for pid, data in c
urrent_proc_map.items()}
415
416         # Классификация процессов на Apps, Background и Wi
ndows
417         from apps.windows.sdk.modules.process_manager.core
.classifier import (
418             ProcessClassifier,

```

```

419         WINDOWS_SYSTEM_PROCESS_NAMES,
420         KNOWN_APP_FRIENDLY_NAMES,
421     )
422     classifier = ProcessClassifier()
423     windowed_pids, titles_map = classifier.get_windowe
d_pids_and_titles()
424
425     name_to_windowed = {
426         (data['name'] or '').lower(): True
427         for p_id, data in current_proc_map.items()
428         if p_id in windowed_pids
429     }
430
431     # Группировка процессов по категориям
432     app_groups: Dict[str, List[Dict[str, Any]]] = {}
433     bg_groups: Dict[str, List[Dict[str, Any]]] = {}
434     win_groups: Dict[str, List[Dict[str, Any]]] = {}
435
436     for mem_rss, p, info in raw_procs:
437         pid = info.get('pid', 0)
438         p_name = info.get('name') or 'unknown'
439         low_name = p_name.lower()
440         has_win = pid in windowed_pids
441         is_app = (
442             has_win or
443             (low_name in name_to_windowed and low_name
not in WINDOWS_SYSTEM_PROCESS_NAMES) or
444             (low_name in KNOWN_APP_FRIENDLY_NAMES and
low_name not in WINDOWS_SYSTEM_PROCESS_NAMES)
445         )
446
447         entry = {'pid': pid, 'name': p_name, 'mem_rss'
: mem_rss, 'proc': p, 'info': info, 'has_win': has_win}
448         if is_app:
449             app_groups.setdefault(p_name, []).append(e
ntry)
450         elif low_name in WINDOWS_SYSTEM_PROCESS_NAMES:
451             win_groups.setdefault(p_name, []).append(e

```

```

ntry)
452         else:
453             bg_groups.setdefault(p_name, []).append(en
try)
454
455         apps_count = len(app_groups)
456         bg_count = len(bg_groups)
457         win_count = len(win_groups)
458
459         raw_procs.sort(key=lambda x: x[0], reverse=True)
460         top_candidates = raw_procs[:top_processes]
461         top_procs: List[ProcessMetrics] = []
462         for mem_rss, p, info in top_candidates:
463             proc_cpu = 0.0
464             try:
465                 proc_cpu = p.cpu_percent(interval=None)
466             except Exception:
467                 pass
468             mem_mb = round(mem_rss / (1024 * 1024), 1)
469             raw_user = info.get('username')
470             clean_user = raw_user if isinstance(raw_user,
str) else ''
471             p_id = info.get('pid', 0)
472             p_name = info.get('name') or 'unknown'
473             low_name = p_name.lower()
474             has_win = p_id in windowed_pids
475             is_app = (
476                 has_win or
477                 (low_name in name_to_windowed and low_name
not in WINDOWS_SYSTEM_PROCESS_NAMES) or
478                 (low_name in KNOWN_APP_FRIENDLY_NAMES and
low_name not in WINDOWS_SYSTEM_PROCESS_NAMES)
479             )
480             cat = 'app' if is_app else ('windows' if low_n
ame in WINDOWS_SYSTEM_PROCESS_NAMES else 'background')
481             win_titles = titles_map.get(p_id, [])
482             friendly = classifier._get_friendly_name(p_nam
e, win_titles)

```

```

483
484         top_procs.append(ProcessMetrics(
485             pid=p_id,
486             name=p_name,
487             cpu_percent=proc_cpu,
488             memory_mb=mem_mb,
489             threads_count=1,
490             status='running',
491             username=clean_user,
492             category=cat,
493             is_app=is_app,
494             friendly_name=friendly,
495             window_title=win_titles[0] if win_titles else None,
496         ))
497
498     snapshot = SystemSnapshot(
499         timestamp=now_iso,
500         hostname=hostname,
501         uptime_seconds=round(time.time() - boot_time,
502             1),
503         cpu=cpu_metrics,
504         memory=mem_metrics,
505         gpus=gpu_metrics_list,
506         disks=disk_partitions,
507         network=net_metrics_list,
508         top_processes=top_procs,
509         apps_count=apps_count,
510         background_count=bg_count,
511         windows_count=win_count,
512     )
513     try:
514         storage.save_snapshot(snapshot, top_n=top_processes)
515     except Exception as db_err:
516         logger.warning(f'Ошибка сохранения снимка в SQLite: {db_err}')

```



```

517 |         # Формирование информативного пошагового лога тика
518 |         active_win_str = "N/A (Фоновый режим)"
519 |         if deep_diag:
520 |             try:
521 |                 forensics = deep_diag.collect_forensics_activity()
522 |                 win_info = forensics.foreground_window
523 |                 if win_info and win_info.get('title'):
524 |                     active_win_str = f"\{win_info.get('title')}\n" ({win_info.get('process_name') or 'N/A'}, PID:{win_info.get('pid', 0)}) | Простой: {forensics.user_idle_seconds:.1f}c"
525 |             except Exception:
526 |                 pass
527 |
528 |         disk_vol_str = ", ".join([f"{d.mountpoint} {d.used_gb:.0f}/{d.total_gb:.0f}GB ({d.percent}%)" for d in disk_partitions[:3]]) or "N/A"
529 |
530 |         # Формирование информативной сводки по Apps и Background processes
531 |         def _format_group_summary(groups: Dict[str, List[Dict[str, Any]]], top_n: int = 4) -> str:
532 |             sorted_g = sorted(
533 |                 groups.items(),
534 |                 key=lambda item: sum(x['mem_rss'] for x in item[1]),
535 |                 reverse=True
536 |            )[:top_n]
537 |             items = []
538 |             for name, p_list in sorted_g:
539 |                 total_mb = sum(x['mem_rss'] for x in p_list) / (1024 * 1024)
540 |                 cnt_str = f" ({len(p_list)})" if len(p_list) > 1 else ""
541 |                 friendly = classifier._get_friendly_name(name)
542 |                 if total_mb >= 1024:

```

```

543         mem_str = f"{total_mb / 1024:.1f} GB"
544     else:
545         mem_str = f"{total_mb:.0f} MB"
546     items.append(f"{friendly}{cnt_str} [{mem_s
tr}]]")
547     return ", ".join(items) if items else "Нет про
цессов"
548
549     apps_summary = _format_group_summary(app_groups, t
op_n=4)
550     bg_summary = _format_group_summary(bg_groups, top_
n=3)
551     win_summary = _format_group_summary(win_groups, to
p_n=2)
552
553     logger.info(f" [ТЕЛЕМЕТРИЯ #{tick}] Режим: {mode
.upper()} | {datetime.now().strftime('%H:%M:%S')}")
554     logger.info(f" └─ CPU: {cpu_pct:.1f}% @ {freq
_mhz:.0f}MHz ({cpu_metrics.physical_cores}P/{cpu_metrics.logic
al_cores}L) | RAM: {mem_metrics.used_gb:.1f}/{mem_metrics.tota
l_gb:.1f}GB ({mem_metrics.percent}%)"
555     if gpu_metrics_list:
556         g0 = gpu_metrics_list[0]
557         logger.info(f" └─ GPU: {g0.name} | Нагруз
ка: {g0.load_percent or 0.0}% | Температура: {g0.temperature_c
elsius or 0.0}°C")
558     logger.info(f" └─ Диски: Чтение {disk_read_kb
:.1f} KB/s | Запись {disk_write_kb:.1f} KB/s | Занято: {disk_v
ol_str}")
559     logger.info(f" └─ Сеть: ↓{net_recv_kb:.1f} KB
/s | ↑{net_sent_kb:.1f} KB/s | Активность: {active_win_str}")
560     logger.info(f" └─ Apps ({apps_count}): {apps_
summary}")
561     logger.info(f" └─ Background ({bg_count}): {b
g_summary}")
562     logger.info(f" └─ Windows ({win_count}): {win
_summary}")
563     logger.info(f" └─ Буфер БД: снимок #{tick} до

```

```

    бавлен (в буфере: {storage.get_buffered_count()} записей, БД:
    {Path(db_path).name}))")
564
565         # Автоматическое переключение из тяжелого режима в
    легкий при непрерывной работе 5 дней (432 000с)
566         if auto_switch_enabled and heavy_mode_start_time is
    not None and mode.lower() in ('full', 'heavy'):
567             if now_sec - heavy_mode_start_time >= max_heav
    y_duration_sec:
568                 days_run = round((now_sec - heavy_mode_sta
    rt_time) / (24 * 3600), 1)
569                 logger.warning(
570                     f" [АВТО-ПЕРЕКЛЮЧЕНИЕ] Телеметрия неп
    рывно работала {days_run} дн. в тяжелом режиме ({mode.upper(
    )}). "
571                     f"Для оптимизации ресурсов режим автом
    атически переключен в легкий (LIGHT).")
572             )
573             mode = 'light'
574             heavy_mode_start_time = None
575
576             is_heavy_due = mode.lower() in ('full', 'heavy') o
    r (mode.lower() == 'hybrid' and now_sec - last_heavy_time >= h
    eavy_interval)
577             if is_heavy_due:
578                 last_heavy_time = now_sec
579
580             def _execute_heavy_poll_job() -> None:
581                 nonlocal last_heavy_disk_scan_time, heavy_
    poll_running
582                 heavy_start = time.time()
583                 try:
584                     logger.info(f" [ТЯЖЕЛЫЙ ОПРОС] Запус
    к периодического опроса сенсоров LHM, SMART, портов и утечек..
    .")
585                     heavy_readings_total = []
586                     if sensor_collector and h_collectors.g
    et('hardware_sensors', True):

```

```

587 |             heavy_readings = sensor_collector.
| collect_all_sensors()
588 |             if heavy_readings:
589 |                 heavy_readings_total.extend(he
| avy_readings)
590 |                 try:
591 |                     storage.save_sensor_polls(
| heavy_readings)
592 |                 except Exception as sp_err:
593 |                     logger.debug(f'Ошибка сохр
| анения sensor_polls в БД: {sp_err}')
594 |
595 |                 # Разгруппировка и красивый вы
| вод LHM сенсоров в консоль
596 |                 lhm_temps = [f"{s.get('sensor_
| name')}: {s.get('value')}{s.get('unit')}" for s in heavy_readi
| ngs if s.get('sensor_category') in ('Temperatures', 'Temperatu
| re')]
597 |                 lhm_powers = [f"{s.get('sensor
| _name')}: {s.get('value')}{s.get('unit')}" for s in heavy_read
| ings if s.get('sensor_category') in ('Powers', 'Power')]
598 |                 lhm_fans = [f"{s.get('sensor_n
| ame')}: {s.get('value')}{s.get('unit')}" for s in heavy_readin
| gs if s.get('sensor_category') in ('Fans', 'Fan')]
599 |                 lhm_clocks = [f"{s.get('sensor
| _name')}: {s.get('value')}{s.get('unit')}" for s in heavy_read
| ings if s.get('sensor_category') in ('Clocks', 'Clock')]
600 |
601 |                 logger.info(f" [LHM СЕНСОРЫ]
| Опрошено {len(heavy_readings)} аппаратных датчиков:")
602 |                 if lhm_temps:
603 |                     logger.info(f"    └─ Темпер
| атуры ({len(lhm_temps)}): {' ', ' '.join(lhm_temps[:8])}")
604 |                 if lhm_powers:
605 |                     logger.info(f"    └─ Энерго
| потребление ({len(lhm_powers)}): {' ', ' '.join(lhm_powers[:6])}")
606 |                 if lhm_fans:
607 |                     logger.info(f"    └─ Вентил

```

```

яторы ({len(lhm_fans)}): {'', '.join(lhm_fans)}")
608         if lhm_clocks:
609             logger.info(f"    └ Частот
ы ({len(lhm_clocks)}): {'', '.join(lhm_clocks[:6])}")
610
611         # Детальный аудит портов, сетевых соединений и утечек дескрипторов
612         if system_collector:
613             try:
614                 ports = system_collector.get_listening_ports(limit=10)
615                 if ports:
616                     p_str = ", ".join([f"{p.port}/{p.protocol} ({p.process_name or 'System'})" for p in ports[:5]])
617                     logger.info(f"    [ТЯЖЕЛЫЙ ОПРОС] Открытые слушающие порты ({len(ports)}): {p_str}")
618             except Exception:
619                 pass
620             try:
621                 net_conns = system_collector.get_process_network_activity(limit=5, only_internet=True)
622                 if net_conns:
623                     c_str = ", ".join([f"{c.name}->{c.remote_address} ({c.service_type})" for c in net_conns[:3]])
624                     logger.info(f"    [ТЯЖЕЛЫЙ ОПРОС] Активные соединения с Интернет ({len(net_conns)}): {c_str}")
625             except Exception:
626                 pass
627
628         if deep_diag:
629             try:
630                 leaks = deep_diag.collect_process_leaks(limit=5)
631                 if leaks and leaks.top_handle_hogs:

```

```
632 |         top_h = leaks.top_handle_h  
    |     ogs[0]  
633 |         logger.info(f"[ТЯЖЕЛЫЙ  
    | ОПРОС] Топ процесса по дескрипторам: {top_h.name} (PID:{top_h.  
    | pid}) -> {top_h.handles_count} handles, {top_h.gdi_objects} GD  
    | I, {top_h.user_objects} USER")  
634 |         except Exception:  
635 |             pass  
636 |  
637 |             # Тяжелый опрос накопителей SMART и на  
    | дежности (2 раза в день / каждые 12 часов)  
638 |             h_now = time.time()  
639 |             if system_collector and h_collectors.g  
    | et('storage_smart', True) and (h_now - last_heavy_disk_scan_ti  
    | me >= heavy_disk_interval_sec):  
640 |                 try:  
641 |                     heavy_disks = system_collector  
    | .get_physical_disks_health(force=True)  
642 |                     if heavy_disks:  
643 |                         d_summary = ", ".join([f"{  
    | d.model} ({d.media_type}, {d.size_gb}GB, SMART:{d.health_statu  
    | s})" for d in heavy_disks])  
644 |                         logger.info(f"[ТЯЖЕЛЫЙ  
    | ОПРОС (SMART)] Накопители SMART: {d_summary}")  
645 |                         last_heavy_disk_scan_time = h_  
    | now  
646 |                     except Exception as d_err:  
647 |                         logger.debug(f"Ошибка тяжелого  
    | сканирования накопителей: {d_err}")  
648 |  
649 |             # Инкрементальный сбор событий журнала  
    | безопасности Windows (Security.evtx и Defender)  
650 |             if h_collectors.get('security_events',  
    | True):  
651 |                 try:  
652 |                     from apps.windows.telemetry.se  
    | curity_collector import WindowsSecurityCollector  
653 |                     sec_coll = WindowsSecurityColl
```

```

ector(storage=storage)
654         # Сбор журнала Security
655         sec_rep = sec_coll.collect_incremental(channel='Security', batch_size=500, max_records=2000,
        save_raw=False)
656         if sec_rep.total_events_ingested > 0:
657             logger.info(f" [SECURITY LOG] Инкрементально собрано {sec_rep.total_events_ingested} событий безопасности (RecordID: {sec_rep.last_record_id})")
658         # Сбор журнала Microsoft Defender
659         def_rep = sec_coll.collect_defender_events(batch_size=200, max_records=1000, save_raw=False)
660         if def_rep.total_events_ingested > 0:
661             logger.info(f" [DEFENDER LOG] Инкрементально собрано {def_rep.total_events_ingested} событий Defender (RecordID: {def_rep.last_record_id})")
662         except Exception as sec_err:
663             logger.debug(f"Ошибка сбора событий безопасности: {sec_err}")
664
665         heavy_dur = time.time() - heavy_start
666         logger.info(f" [{mode.upper()}] Периодический тяжелый опрос завершен за {heavy_dur:.2f}с (Всего сенсоров: {len(heavy_readings_total)})")
667         except Exception as h_err:
668             logger.error(f'Ошибка периодического тяжелого опроса: {h_err}')
669         finally:
670             with heavy_poll_lock:
671                 heavy_poll_running = False
672
673         if once:
674             _execute_heavy_poll_job()
675         else:
676             with heavy_poll_lock:

```

```

677         if not heavy_poll_running:
678             heavy_poll_running = True
679             worker_thread = threading.Thread(
680                 target=_execute_heavy_poll_job
681                 ,
682                 name="TelemetryHeavyPollWorker
683                 ",
684                 daemon=True
685             )
686             worker_thread.start()
687
688             # Фоновый периодический контроль размера SQLite ба
689             зы данных telemetry.db
690             if now_sec - last_db_cleanup_time >= db_cleanup_in
691             terval_sec:
692                 try:
693                     prune_res = storage.enforce_size_limit()
694                     if prune_res.get('pruned'):
695                         logger.warning(
696                             f" [КОНТРОЛЬ РАЗМЕРА БД] Усечени
697                             е базы данных telemetry.db: "
698                             f"{prune_res.get('initial_size_mb'
699                             )} МБ -> {prune_res.get('final_size_mb')} МБ "
700                             f"(удалено снимков: {prune_res.get
701                             ('deleted_snapshots')})")
702                     )
703                     last_db_cleanup_time = now_sec
704                     except Exception as db_clean_err:
705                         logger.debug(f"Ошибка периодического контр
706                         оля размера БД: {db_clean_err}")
707
708             # Фоновая периодическая многоуровневая SQL-агрегац
709             ия (hourly -> daily -> weekly -> monthly -> yearly)
710             if now_sec - last_aggregation_time >= aggregation_
711             interval_sec:
712                 try:
713                     storage.flush()
714                     agg_res = storage.run_aggregation_pipeline

```



```

705         h_cnt = agg_res.get('hourly', {}).get('buckets_aggregated', 0)
706         sp_cnt = agg_res.get('hourly', {}).get('spikes_recorded', 0)
707         if h_cnt > 0 or sp_cnt > 0:
708             logger.info(f" [SQL АГРЕГАЦИЯ] Обновлено бакеты: hourly={h_cnt}, зафиксировано всплесков={sp_cnt}"
709             )
710             last_aggregation_time = now_sec
711             except Exception as agg_err:
712                 logger.debug(f"Ошибка периодической SQL-агрегации: {agg_err}")
713
714         if once:
715             storage.flush()
716             try:
717                 storage.run_aggregation_pipeline()
718             except Exception:
719                 pass
720             logger.info(' Однократный опрос системной телеметрии успешно выполнен (--once)')
721             break
722
723         elapsed = time.time() - loop_start
724         sleep_time = max(0.05, interval - elapsed)
725         if _stop_event.wait(timeout=sleep_time):
726             logger.info('Получен сигнал остановки (_stop_event установлен)')
727             break
728         except KeyboardInterrupt:
729             logger.info('Прервано пользователем (KeyboardInterrupt)')
730         except Exception as ex:
731             logger.error(f'Критическая ошибка в цикле телеметрии: {ex}', exc_info=True)
732         finally:
733             if w64_collector:

```

```
733         try:
734             w64_collector.stop()
735         except Exception:
736             pass
737     if w64_etw_collector:
738         try:
739             w64_etw_collector.stop()
740         except Exception:
741             pass
742     logger.info('Процесс телеметрии успешно завершён')
743     return 0
744
745 def run_minimal_telemetry(interval: float=5.0, top_processes:
746     int=10, log_dir: Optional[str]=None) -> int:
747     """Обертка для обратной совместимости: запуск в режиме min
748     imal."""
749     return run_telemetry_service(mode='minimal', interval=inte
750     rval, top_processes=top_processes, log_dir=log_dir)
751
752 def main() -> int:
753     """Главная точка входа сервиса телеметрии.
754
755     Returns:
756         int: Код завершения.
757     """
758     args = parse_arguments()
759     if args.verbose:
760         logger.setLevel('DEBUG')
761     signal.signal(signal.SIGINT, signal_handler)
762     signal.signal(signal.SIGTERM, signal_handler)
763     from .telemetry_config import TelemetryConfigManager
764     try:
765         config_manager = TelemetryConfigManager(config_path=ar
766         gs.config)
767     except Exception as e:
768         logger.warning(f'Не удалось загрузить конфигурацию ({e
769         }), используются значения по умолчанию')
770     config_manager = None
```

```
766 |     if args.minimal:
767 |         mode = 'minimal'
768 |     elif args.mode:
769 |         mode = args.mode
770 |     elif config_manager:
771 |         mode = config_manager.get_mode()
772 |     else:
773 |         mode = 'hybrid'
774 |     interval = args.interval or (config_manager.get_interval_s
| econds() if config_manager else 5.0)
775 |     heavy_interval = args.heavy_interval or (config_manager.ge
| t_heavy_interval_seconds() if config_manager else 60.0)
776 |     top_processes = args.top_processes or (config_manager.get_
| top_processes() if config_manager else 10)
777 |     heavy_collectors = config_manager.get_heavy_collectors() i
| f config_manager else None
778 |     return run_telemetry_service(mode=mode, interval=interval,
|     heavy_interval=heavy_interval, top_processes=top_processes, h
| eavy_collectors=heavy_collectors, log_dir=args.log_dir, config
| _path=args.config, once=args.once)
779 | if __name__ == '__main__':
780 |     sys.exit(main())
```

models.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - Models
4 # =====
5 # Description:
6 #   Data models for system metrics, hardware specs, sensors, a
7 #   nd telemetry.
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.models import TelemetryProvi
11 #       der
12 #       service = TelemetryProvider()
13 #
14 # File: models.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-08 12:38:00
20 # =====
21
22 from __future__ import annotations
23 """Data models for system metrics, hardware specs, sensors, an
24 d telemetry."""
25
26 from abc import ABC, abstractmethod
27 from datetime import datetime, timezone
28 from enum import Enum
29 from typing import Any, Dict, List, Optional, Union
30 from pydantic import BaseModel, ConfigDict, Field
31 from apps.windows.sdk.modules.startup.core.models import Start
```

```

upArchiveEntry, StartupChangeItem
31
32
33
34 class CpuInventoryInfo(BaseModel):
35     """Паспорт физического процессора (CPU Inventory)."""
36     processor_id: int = 0
37     name: str = ''
38     vendor: str = ''
39     architecture: str = 'x86_64'
40     physical_cores: int = 1
41     logical_cores: int = 1
42     base_frequency_mhz: float = 0.0
43     max_frequency_mhz: float = 0.0
44     l2_cache_kb: Optional[int] = None
45     l3_cache_kb: Optional[int] = None
46     socket: str = ''
47     features: List[str] = Field(default_factory=list)
48
49
50 class CpuTelemetrySample(BaseModel):
51     """Срез телеметрии нагрузки и температур процессора."""
52     timestamp: str = ''
53     created_at: float = 0.0
54     processor_id: int = 0
55     total_percent: float = 0.0
56     user_percent: float = 0.0
57     kernel_percent: float = 0.0
58     frequency_mhz: float = 0.0
59     temperature_c: Optional[float] = None
60     package_power_w: Optional[float] = None
61     core_utilization: List[float] = Field(default_factory=list
62 )
63     core_temperatures: List[float] = Field(default_factory=lis
64 t)
65
66 class RamModuleInventoryInfo(BaseModel):

```

```

66 |     """Паспорт физического модуля (планки) оперативной памяти
    | (SPD)."""
67 |     slot_id: int = 0
68 |     bank_label: str = 'DIMM'
69 |     device_locator: str = ''
70 |     serial_number: str = ''
71 |     part_number: str = ''
72 |     manufacturer: str = ''
73 |     capacity_bytes: int = 0
74 |     capacity_gb: float = 0.0
75 |     speed_mhz: int = 0
76 |     memory_type: str = 'DDR4'
77 |     form_factor: str = 'DIMM'
78 |     configured_voltage: Optional[float] = None
79 |
80 |
81 | class RamTelemetrySample(BaseModel):
82 |     """Срез телеметрии использования физической и виртуальной
    | памяти."""
83 |     timestamp: str = ''
84 |     created_at: float = 0.0
85 |     total_bytes: int = 0
86 |     total_gb: float = 0.0
87 |     used_bytes: int = 0
88 |     used_gb: float = 0.0
89 |     available_bytes: int = 0
90 |     available_gb: float = 0.0
91 |     percent_used: float = 0.0
92 |     swap_total_gb: float = 0.0
93 |     swap_used_gb: float = 0.0
94 |     swap_percent: float = 0.0
95 |     pool_paged_mb: Optional[float] = None
96 |     pool_nonpaged_mb: Optional[float] = None
97 |
98 |
99 | class GpuInventoryInfo(BaseModel):
100 |     """Паспорт графического ускорителя (GPU Inventory)."""
101 |     gpu_id: int = 0

```

```

102     pci_device_id: str = ''
103     name: str = ''
104     vendor: str = ''
105     driver_version: str = ''
106     driver_date: str = ''
107     vram_bytes: int = 0
108     vram_gb: float = 0.0
109     pci_bus_id: str = ''
110     bios_version: str = ''
111     cuda_cores: Optional[int] = None
112     cuda_supported: bool = False
113     rocm_supported: bool = False
114     oneapi_supported: bool = False
115     directml_supported: bool = True
116     vulkan_supported: bool = True
117     opengl_supported: bool = True
118     compute_apis: List[str] = Field(default_factory=list, description='Поддерживаемые вычислительные API')
119
120
121 class GpuTelemetrySample(BaseModel):
122     """Срез телеметрии нагрузки, памяти и температур GPU."""
123     timestamp: str = ''
124     created_at: float = 0.0
125     gpu_id: int = 0
126     name: str = ''
127     load_percent: Optional[float] = None
128     memory_used_mb: Optional[float] = None
129     memory_total_mb: Optional[float] = None
130     memory_percent: Optional[float] = None
131     temperature_gpu_c: Optional[float] = None
132     temperature_memory_c: Optional[float] = None
133     fan_speed_pct: Optional[float] = None
134     power_draw_w: Optional[float] = None
135     clock_graphics_mhz: Optional[float] = None
136     clock_memory_mhz: Optional[float] = None
137
138

```

```

139 class NetworkAdapterInventoryInfo(BaseModel):
140     """Паспорт сетевого адаптера / интерфейса (Network Adapter
    Inventory)."""
141     adapter_id: int = 0
142     adapter_guid: str = ''
143     name: str = ''
144     interface_name: str = ''
145     mac_address: str = ''
146     adapter_type: str = 'Ethernet'
147     is_physical: bool = True
148     is_wireless: bool = False
149     max_speed_mbps: int = 0
150     driver_name: str = ''
151     driver_version: str = ''
152
153
154 class NetworkAdapterSample(BaseModel):
155     """Срез показателей сетевой активности и ошибок адаптера."
    ""
156     timestamp: str = ''
157     created_at: float = 0.0
158     adapter_name: str = ''
159     bytes_recv_sec: float = 0.0
160     bytes_sent_sec: float = 0.0
161     packets_recv_sec: float = 0.0
162     packets_sent_sec: float = 0.0
163     errors_in_sec: float = 0.0
164     errors_out_sec: float = 0.0
165     link_speed_mbps: int = 0
166     is_connected: bool = True
167
168 class TelemetryProvider(ABC):
169     """Абстрактный базовый класс для всех компонентов системы,
    предоставляющих телеметрию."""
170
171     @abstractmethod
172     def get_sensors(self) -> List[HardwareSensor]:
173         """Возвращает список сенсоров, предоставляемых данным

```



```

    КОМПОНЕНТОМ. """
174         pass
175
176 class CpuMetrics(BaseModel):
177     """CPU usage and architecture metrics."""
178     model: str = Field(default='', description='CPU model name
    ')
179     architecture: str = Field(default='x86_64', description='C
    PU architecture')
180     physical_cores: int = Field(default=1, description='Physic
    al core count')
181     logical_cores: int = Field(default=1, description='Logical
    core count (threads)')
182     total_percent: float = Field(default=0.0, description='Ove
    rall CPU usage percentage')
183     per_core_percent: List[float] = Field(default_factory=list
    , description='Usage percentage per logical core')
184     frequency_mhz: float = Field(default=0.0, description='Cur
    rent CPU frequency in MHz')
185     temperature_celsius: Optional[float] = Field(default=None,
    description='Package temperature in Celsius')
186
187 class MemoryMetrics(BaseModel):
188     """RAM and Swap memory metrics."""
189     total_gb: float = Field(default=0.0, description='Total ph
    ysical RAM in GB')
190     available_gb: float = Field(default=0.0, description='Avai
    lable RAM in GB')
191     used_gb: float = Field(default=0.0, description='Used RAM
    in GB')
192     percent: float = Field(default=0.0, description='Memory ut
    ilization percentage')
193     swap_total_gb: float = Field(default=0.0, description='Tot
    al swap space in GB')
194     swap_used_gb: float = Field(default=0.0, description='Used
    swap space in GB')
195     swap_percent: float = Field(default=0.0, description='Swap
    utilization percentage')

```

```
196
197 class GpuMetrics(BaseModel):
198     """GPU accelerator telemetry and compute backends."""
199     name: str = Field(default='Unknown GPU', description='GPU
device model name')
200     vendor: str = Field(default='', description='GPU vendor na
me (NVIDIA, AMD, Intel, etc.)')
201     gpu_type: str = Field(default='Discrete', description='Тип
GPU (Discrete / Integrated)')
202     load_percent: Optional[float] = Field(default=None, descri
ption='GPU core load percentage')
203     memory_total_gb: float = Field(default=0.0, description='T
otal VRAM in GB')
204     memory_used_gb: float = Field(default=0.0, description='Us
ed VRAM in GB')
205     memory_free_gb: float = Field(default=0.0, description='Fr
ee VRAM in GB')
206     dedicated_memory_mb: Optional[float] = Field(default=None,
description='Выделенная память VRAM (МБ)')
207     shared_memory_mb: Optional[float] = Field(default=None, de
scription='Общая системная память GPU (МБ)')
208     memory_type: str = Field(default='Dedicated VRAM', descrip
tion='Тип видеопамяти (Dedicated VRAM / Shared System Memory)'
)
209     directx_version: str = Field(default='DirectX 12 (FL 12_1)
', description='Версия DirectX')
210     temperature_celsius: Optional[float] = Field(default=None,
description='GPU core temperature in Celsius')
211     has_cuda: bool = Field(default=False, description='CUDA su
pport availability')
212     has_rocm: bool = Field(default=False, description='AMD ROC
m / HIP support availability')
213     has_oneapi: bool = Field(default=False, description='Intel
oneAPI support availability')
214     has_directml: bool = Field(default=True, description='Dire
ctML accelerator availability')
215     has_vulkan: bool = Field(default=True, description='Vulkan
API availability')
```

```

216     has_opengl: bool = Field(default=True, description='OpenGL
    API availability')
217     compute_apis: List[str] = Field(default_factory=list, desc
    ription='Supported compute APIs (CUDA, ROCm, DirectML, Vulkan,
    OpenCL, oneAPI)')
218     ai_backends: List[str] = Field(default_factory=list, descr
    iption='Supported AI inference backends')
219     engines: Dict[str, float] = Field(default_factory=dict, de
    scription='Подсистемы и движки GPU (3D, Decode, Copy, etc.)')
220
221
222 class NpuMetrics(BaseModel):
223     """NPU neural accelerator telemetry and AI capabilities."""
    "
224     name: str = Field(default='NPU Accelerator', description='
    NPU device model name')
225     vendor: str = Field(default='Intel', description='NPU vend
    or name (Intel, Qualcomm, AMD, etc.)')
226     driver_version: str = Field(default='', description='Drive
    r version')
227     pnp_device_id: str = Field(default='', description='PnP de
    vice identifier')
228     status: str = Field(default='OK (Активно)', description='D
    evice status')
229     tops: Optional[float] = Field(default=None, description='A
    I compute performance in TOPS')
230     has_directml: bool = Field(default=True, description='Dire
    ctML NPU support')
231     has_qnn: bool = Field(default=False, description='Qualcomm
    QNN support')
232     has_openvino: bool = Field(default=False, description='Int
    el OpenVINO support')
233     ai_backends: List[str] = Field(default_factory=list, descr
    iption='Supported AI backends')
234
235 class DiskPartitionMetrics(BaseModel):
236     """Storage partition metrics."""
237     device: str = Field(default='', description='Partition mou

```

```

    nt device or drive letter')
238     mountpoint: str = Field(default='', description='Mount poi
    nt path')
239     fstype: str = Field(default='', description='Filesystem ty
    pe')
240     total_gb: float = Field(default=0.0, description='Total ca
    pacity in GB')
241     used_gb: float = Field(default=0.0, description='Used spac
    e in GB')
242     free_gb: float = Field(default=0.0, description='Free spac
    e in GB')
243     percent: float = Field(default=0.0, description='Utilizati
    on percentage')
244     volume_name: Optional[str] = Field(default=None, descripti
    on='Метка тома или описание аккаунта')
245     is_virtual: bool = Field(default=False, description='Призн
    ак виртуального/облачного тома (Google Drive, OneDrive и др.)'
    )
246     drive_type: str = Field(default='Fixed', description='Тип
    диска (Fixed, Google Drive, Cloud / Virtual, Removable)')
247
248 class DiskIoMetrics(BaseModel):
249     """Disk read and write I/O rates."""
250     read_bytes_per_sec: float = Field(default=0.0, description
    ='Read throughput in bytes/s')
251     write_bytes_per_sec: float = Field(default=0.0, descriptio
    n='Write throughput in bytes/s')
252     read_count_per_sec: float = Field(default=0.0, description
    ='Read operations per second')
253     write_count_per_sec: float = Field(default=0.0, descriptio
    n='Write operations per second')
254
255 class NetworkAdapterStatistics(BaseModel):
256     """Накопительная статистика сетевого адаптера (Get-NetAdap
    terStatistics / MIB_IF_ROW2)."""
257     name: str = Field(default='', description='Имя адаптера/ин
    терфейса')
258     received_bytes: int = Field(default=0, description='Всего

```

```

получено байт')
259     sent_bytes: int = Field(default=0, description='Всего отпра
авлено байт')
260     received_packets: int = Field(default=0, description='Всег
о получено пакетов')
261     sent_packets: int = Field(default=0, description='Всего от
правлено пакетов')
262     received_discarded: int = Field(default=0, description='От
брошено входящих пакетов')
263     received_errors: int = Field(default=0, description='Ошибо
к входящих пакетов')
264     outbound_discarded: int = Field(default=0, description='От
брошено исходящих пакетов')
265     outbound_errors: int = Field(default=0, description='Ошибо
к исходящих пакетов')
266
267 class NetworkPerformanceCounter(BaseModel):
268     """Счётчики производительности интерфейса (Windows Perform
ance Counters)."""
269     interface_name: str = Field(default='', description='Имя с
етевого интерфейса в Performance Counters')
270     bytes_recv_per_sec: float = Field(default=0.0, description
='Скорость получения (байты/сек)')
271     bytes_sent_per_sec: float = Field(default=0.0, description
='Скорость отправки (байты/сек)')
272     bytes_total_per_sec: float = Field(default=0.0, descriptio
n='Суммарная скорость (байты/сек)')
273
274 class AppNetworkUsageItem(BaseModel):
275     """Потребление сетевого трафика конкретным приложением/про
цессом."""
276     process_name: str = Field(default='', description='Имя исп
олняемого файла процесса (например, chrome.exe)')
277     pid: Optional[int] = Field(default=None, description='Иден
тификатор процесса (если активен)')
278     rx_bytes: int = Field(default=0, description='Получено бай
т за период')
279     tx_bytes: int = Field(default=0, description='Отправлено б

```

```

айт за период')
280     total_bytes: int = Field(default=0, description='Всего пер
еданного трафика в байтах')
281     rx_formatted: str = Field(default='0 B', description='Отфо
рматированный объем полученных данных (MB/GB)')
282     tx_formatted: str = Field(default='0 B', description='Отфо
рматированный объем отправленных данных (MB/GB)')
283     share_percent: float = Field(default=0.0, description='Дол
я трафика от общего объема системы (%)')
284
285 class NetworkUsagePeriodReport(BaseModel):
286     """Агрегированный отчет об использовании сети за выбранный
интервал времени (10 мин, 1 час, 24 часа)."""
287     period_minutes: int = Field(default=1440, description='Дли
тельность периода в минутах (например, 1440 = 24 часа)')
288     total_rx_bytes: int = Field(default=0, description='Суммар
но получено байт за период')
289     total_tx_bytes: int = Field(default=0, description='Суммар
но отправлено байт за период')
290     total_rx_formatted: str = Field(default='0 B', description
='Человекочитаемый объем входящего трафика')
291     total_tx_formatted: str = Field(default='0 B', description
='Человекочитаемый объем исходящего трафика')
292     total_formatted: str = Field(default='0 B', description='С
уммарный объем трафика (RX + TX)')
293     adapter_stats: List[NetworkAdapterStatistics] = Field(defa
ult_factory=list, description='Статистика по адаптерам')
294     performance_counters: List[NetworkPerformanceCounter] = Fi
eld(default_factory=list, description='Текущие Performance Cou
nters')
295     top_apps: List[AppNetworkUsageItem] = Field(default_factor
y=list, description='Детализация по приложениям за период')
296     summary_text: str = Field(default='', description='Сводный
текстовый отчёт для LLM/ai-diagnostics')
297
298 class NetworkInterfaceMetrics(BaseModel):
299     """Network adapter telemetry."""
300     name: str = Field(default='', description='Adapter interfa

```

```

    ce name')
301     is_up: bool = Field(default=True, description='Interface l
ink status')
302     speed_mbps: int = Field(default=0, description='Link speed
in Mbps')
303     bytes_sent_per_sec: float = Field(default=0.0, description
='Upload rate in bytes/s')
304     bytes_recv_per_sec: float = Field(default=0.0, description
='Download rate in bytes/s')
305     ip_addresses: List[str] = Field(default_factory=list, desc
ription='Assigned IP addresses')
306     statistics: Optional[NetworkAdapterStatistics] = Field(def
ault=None, description='Накопительная статистика адаптера')
307
308 class ProcessMetrics(BaseModel):
309     """Метрики процесса с данными об идентичности и происхожде
нии (Process Identity & Provenance)."""
310     pid: int = Field(..., description='Process identifier')
311     name: str = Field(..., description='Process binary name')
312     status: str = Field(default='running', description='Proces
s execution status')
313     cpu_percent: float = Field(default=0.0, description='CPU u
sage percentage')
314     memory_mb: float = Field(default=0.0, description='Residen
t memory in Megabytes')
315     memory_percent: float = Field(default=0.0, description='Me
mory usage percentage')
316     num_threads: int = Field(default=1, description='Number of
active threads')
317     num_handles: int = Field(default=0, description='Количеств
о дескрипторов (handles/fds)')
318     username: Optional[str] = Field(default=None, description=
'Owner username')
319     cmdline: Optional[str] = Field(default=None, description='
Command line invocation')
320     read_bytes_sec: float = Field(default=0.0, description='Di
sk read rate in bytes/sec')
321     write_bytes_sec: float = Field(default=0.0, description='D

```

```

isk write rate in bytes/sec')
322     # Детализированные счетчики ресурсов (Per-PID Telemetry En
    gine)
323     user_time_ms: int = Field(default=0, description='User mod
    e CPU time in milliseconds')
324     kernel_time_ms: int = Field(default=0, description='Kernel
    mode CPU time in milliseconds')
325     working_set_bytes: int = Field(default=0, description='Wor
    king set memory in bytes')
326     private_bytes: int = Field(default=0, description='Private
    memory in bytes')
327     page_faults_count: int = Field(default=0, description='Num
    ber of page faults')
328     gpu_vram_bytes: int = Field(default=0, description='GPU VR
    AM consumption in bytes')
329     gpu_utilization: float = Field(default=0.0, description='G
    PU engine load percentage')
330     read_bytes_total: int = Field(default=0, description='Cum
    ulative bytes read')
331     write_bytes_total: int = Field(default=0, description='Cum
    ulative bytes written')
332     read_ops_total: int = Field(default=0, description='Cumula
    tive read operations count')
333     write_ops_total: int = Field(default=0, description='Cumul
    ative write operations count')
334     net_bytes_sent_total: int = Field(default=0, description='
    Cumulative network bytes sent')
335     net_bytes_recv_total: int = Field(default=0, description='
    Cumulative network bytes received')
336     gdi_objects: int = Field(default=0, description='GDI objec
    ts count')
337     user_objects: int = Field(default=0, description='USER obj
    ects count')
338     # Поля безопасности и токена
339     integrity_level: Optional[str] = Field(default=None, descr
    iption='Уровень целостности процесса (Low, Medium, High, Syste
    m)')
340     elevation: Optional[bool] = Field(default=None, descriptio

```



```

n='Статус повышения привилегий UAC')
341     sid: Optional[str] = Field(default=None, description='Иден
тификатор безопасности пользователя (User SID)')
342     session_id: Optional[int] = Field(default=None, descriptio
n='Идентификатор сессии Windows (0=Службы, 1=Интерактивная)')
343     # Поля происхождения и родословной (Process Provenance)
344     ppid: Optional[int] = Field(default=None, description='Иде
нтификатор родительского процесса (Parent PID)')
345     parent_name: Optional[str] = Field(default=None, descripti
on='Имя родительского процесса')
346     executable_path: Optional[str] = Field(default=None, descr
iption='Полный путь к исполняемому файлу процесса')
347     creation_time: Optional[str] = Field(default=None, descrip
tion='Время создания процесса (ISO 8601 UTC)')
348     process_guid: Optional[str] = Field(default=None, descript
ion='Уникальный GUID экземпляра процесса (PID + CreationTime)'
)
349     ancestor_chain: Optional[str] = Field(default=None, descri
ption='Цепочка предков запуска (напр. explorer -> pwsh -> pyth
on)')
350     launch_reason: Optional[str] = Field(default=None, descrip
tion='Определенный вектор/причина запуска (Служба, Задача, Тер
минал, Проводник)')
351     # Классификация процессов (Apps, Background processes, Win
dows processes)
352     category: str = Field(default='background', description='К
атегория процесса: app, background, windows')
353     is_app: bool = Field(default=False, description='Флаг инте
рактивного оконного приложения')
354     friendly_name: Optional[str] = Field(default=None, descrip
tion='Дружелюбное имя приложения (например Google Chrome)')
355     window_title: Optional[str] = Field(default=None, descript
ion='Заголовок активного окна процесса')
356     instance_count: int = Field(default=1, description='Количе
ство связанных экземпляров процесса')
357
358
359 class ProcessTokenInfo(BaseModel):

```

```

360     """Информация о токене процесса для сбора security-данных.
361     """
362     pid: int = Field(..., description='Идентификатор процесса'
363     )
364     name: str = Field(..., description='Имя процесса')
365     elevation: Optional[bool] = Field(default=None, description='Признак повышения привилегий (UAC)')
366     integrity_level: Optional[str] = Field(default=None, description='Уровень целостности процесса')
367     privileges: List[str] = Field(default_factory=list, description='Список привилегий токена')
368     sid: Optional[str] = Field(default=None, description='Идентификатор безопасности (SID)')
369     session_id: Optional[int] = Field(default=None, description='Идентификатор сессии Windows')
370
371 class ProcessProvenanceInfo(BaseModel):
372     """Детальный паспорт идентичности, происхождения и цепочки вызовов процесса (Process Provenance)."""
373     pid: int = Field(..., description='Идентификатор процесса (PID)')
374     ppid: Optional[int] = Field(default=None, description='Идентификатор родительского процесса (PPID)')
375     name: str = Field(..., description='Имя процесса (например, python.exe)')
376     executable_path: Optional[str] = Field(default=None, description='Полный путь к бинарному исполняемому файлу')
377     command_line: Optional[str] = Field(default=None, description='Полная командная строка запуска с аргументами')
378     user: Optional[str] = Field(default=None, description='Имя учетной записи (DOMAIN\\User)')
379     sid: Optional[str] = Field(default=None, description='Идентификатор безопасности пользователя (SID)')
380     session_id: Optional[int] = Field(default=None, description='Идентификатор сессии Windows (Session 0=Службы, Session 1+= Пользователь)')
381     integrity_level: Optional[str] = Field(default=None, description='Уровень целостности процесса')

```

```

    380 |         ipthion='Уровень целостности токена (Untrusted, Low, Medium, Hi
    381 |         gh, System)')
    382 |         elevation: Optional[bool] = Field(default=None, descriptio
    383 |         n='Признак повышенных привилегий администратора (Elevated)')
    384 |         creation_time: Optional[str] = Field(default=None, descrip
    385 |         tion='Время создания процесса в формате ISO 8601 UTC')
    386 |         parent_start_time: Optional[str] = Field(default=None, des
    387 |         cription='Время создания родительского процесса')
    388 |         process_guid: str = Field(default='', description='Уникаль
    389 |         ный GUID экземпляра процесса (исключает коллизии PID reuse)')
    390 |         parent_guid: Optional[str] = Field(default=None, descripti
    391 |         on='GUID родительского экземпляра процесса')
    392 |         parent_name: Optional[str] = Field(default=None, descripti
    393 |         on='Имя родительского процесса (например, WindowsTerminal.exe
    394 |         ')
    395 |         parent_cmdline: Optional[str] = Field(default=None, descri
    396 |         ption='Командная строка родителя')
    397 |         ancestor_chain: List[str] = Field(default_factory=list, de
    398 |         scription='Список узлов в цепочке предков (от корня к процессу
    399 |         )')
    400 |         ancestor_chain_str: str = Field(default='', description='С
    401 |         троковое представление цепочки предков (explorer → pwsh → pyth
    402 |         on)')
    403 |         who_runs_it: str = Field(default='', description='Кем испо
    404 |         лняется (User / SID / Token / Session)')
    405 |         who_created_it: str = Field(default='', description='Кто с
    406 |         оздал (Родительский процесс / Дерево предков)')
    407 |         why_was_it_created: str = Field(default='', description='П
    408 |         ричина/вектор создания (Служба, Задача, Интерактивно, IDE, CLI
    409 |         , Автозапуск)')
    410 |
    411 |
    412 | class ProcessLifecycleEvent(BaseModel):
    413 |     """Событие жизненного цикла процесса (создание, завершение
    414 |     , форензика)."""
    415 |     event_id: str = Field(default='', description='Уникальный
    416 |     идентификатор события')
    417 |     timestamp: str = Field(default_factory=lambda: datetime.no

```

```
w(timezone.utc).isoformat(), description='Время события UTC')
399     event_type: str = Field(..., description='Тип события: ProcessCreated, ProcessTerminated, Snapshot')
400     pid: int = Field(..., description='Идентификатор процесса')
401     ppid: Optional[int] = Field(default=None, description='Идентификатор родительского процесса')
402     process_guid: str = Field(default='', description='Уникальный GUID процесса')
403     parent_guid: Optional[str] = Field(default=None, description='GUID родительского процесса')
404     name: str = Field(..., description='Имя процесса')
405     executable_path: Optional[str] = Field(default=None, description='Путь к исполняемому файлу')
406     command_line: Optional[str] = Field(default=None, description='Аргументы командной строки')
407     user: Optional[str] = Field(default=None, description='Учетная запись пользователя')
408     sid: Optional[str] = Field(default=None, description='Идентификатор безопасности (SID)')
409     session_id: Optional[int] = Field(default=None, description='Идентификатор сессии')
410     integrity_level: Optional[str] = Field(default=None, description='Уровень целостности токена')
411     elevation: Optional[bool] = Field(default=None, description='Статус повышения привилегий')
412     parent_name: Optional[str] = Field(default=None, description='Имя родительского процесса')
413     parent_cmdline: Optional[str] = Field(default=None, description='Командная строка родителя')
414     ancestor_chain: Optional[str] = Field(default=None, description='Строка цепочки предков')
415     launch_reason: Optional[str] = Field(default=None, description='Определенный вектор запуска')
416     source: str = Field(default='system', description='Источник события: w64, etw, psutil, wevtapi')
417     raw_json: Optional[str] = Field(default=None, description='Сырые данные события в формате JSON')
```

```
418 |
419 |
420 | class ProcessProvenanceReport(BaseModel):
421 |     """Сводный аналитический отчет Process Provenance для AI D
422 |     iagnostics."""
423 |     timestamp: str = Field(default_factory=lambda: datetime.no
424 |     w(timezone.utc).isoformat(), description='Время формирования о
425 |     тчета')
426 |     total_processes: int = Field(default=0, description='Общее
427 |     число активных процессов')
428 |     active_provenance: List[ProcessProvenanceInfo] = Field(def
429 |     ault_factory=list, description='Список паспортов происхождения
430 |     процессов')
431 |     recent_events: List[ProcessLifecycleEvent] = Field(default
432 |     _factory=list, description='Недавние события создания/завершен
433 |     ия процессов')
434 |     summary: str = Field(default='', description='Аналитическа
435 |     я сводка для LLM/ai-diagnostics')
436 | class BatteryMetrics(BaseModel):
437 |     """Battery and power state telemetry."""
438 |     has_battery: bool = Field(default=False, description='Whet
439 |     her host has battery power')
440 |     percent: Optional[float] = Field(default=None, description
441 |     ='Battery charge percentage')
442 |     power_plugged: Optional[bool] = Field(default=None, descri
443 |     ption='Whether AC power is connected')
444 |     secs_left: Optional[int] = Field(default=None, description
445 |     ='Seconds of battery remaining')
446 |     power_profile: str = Field(default='Balanced', description
447 |     ='Active Windows power scheme')
448 |
449 |
450 |
451 | class AppDiskUsageItem(BaseModel):
452 |     """Потребление дискового I/O конкретным приложением/процес
453 |     сом."""
454 |     process_name: str = Field(default='', description='Имя исп
```

```

    оляемого файла процесса (например, chrome.exe)')
441     pid: Optional[int] = Field(default=None, description='Иден
тификатор процесса (если активен)')
442     read_bytes: int = Field(default=0, description='Прочитано
байт за период')
443     write_bytes: int = Field(default=0, description='Записано
байт за период')
444     total_bytes: int = Field(default=0, description='Всего дис
кового I/O в байтах')
445     read_formatted: str = Field(default='0 B', description='От
форматированный объем прочитанных данных')
446     write_formatted: str = Field(default='0 B', description='0
тформатированный объем записанных данных')
447     share_percent: float = Field(default=0.0, description='Дол
я дискового I/O от общего объема системы (%)')
448
449 class DiskUsagePeriodReport(BaseModel):
450     """Агрегированный отчет об использовании дискового I/O и S
MART накопителей за период."""
451     period_minutes: int = Field(default=1440, description='Дли
тельность периода в минутах (например, 1440 = 24 часа)')
452     total_read_bytes: int = Field(default=0, description='Сумм
арно прочитано байт за период')
453     total_write_bytes: int = Field(default=0, description='Сум
марно записано байт за период')
454     total_read_formatted: str = Field(default='0 B', descripti
on='Человекочитаемый объем прочитанных данных')
455     total_write_formatted: str = Field(default='0 B', descript
ion='Человекочитаемый объем записанных данных')
456     total_formatted: str = Field(default='0 B', description='C
уммарный объем I/O (Read + Write)')
457     top_writers: List[AppDiskUsageItem] = Field(default_factor
y=list, description='Топ процессов по объему записи за период'
)
458     top_readers: List[AppDiskUsageItem] = Field(default_factor
y=list, description='Топ процессов по объему чтения за период'
)
459     physical_disks: List[PhysicalDiskHealth] = Field(default_f

```

```

    actory=list, description='Состояние и SMART-метрики физических
    дисков')
460     smart_delta_write_bytes: Optional[int] = Field(default=None,
    description='Изменение объема записи в SMART за период (байт)')
461     smart_delta_write_formatted: Optional[str] = Field(default=
    None, description='Отформатированное изменение записи SMART')
462     io_difference_bytes: Optional[int] = Field(default=None, d
    escription='Разница между SMART Delta и суммарным I/O процессо
    в (байт)')
463     summary_text: str = Field(default='', description='Сводный
    текстовый отчёт для LLM/ai-diagnostics')
464
465 class PhysicalDiskHealth(BaseModel):
466     """Physical drive SMART and health telemetry."""
467     device_id: str = Field(default='', description='Drive iden
    tifier or disk index')
468     model: str = Field(default='Physical Drive', description='
    Drive model name')
469     serial_number: Optional[str] = Field(default=None, descrip
    tion='Серийный номер накопителя')
470     media_type: str = Field(default='SSD', description='Media
    type (NVMe, SSD, HDD)')
471     size_gb: float = Field(default=0.0, description='Drive tot
    al capacity in GB')
472     health_status: str = Field(default='Healthy', description=
    'Drive health status (Healthy, Warning, Unhealthy)')
473     operational_status: str = Field(default='OK', description=
    'Operational status')
474     temperature_celsius: Optional[float] = Field(default=None,
    description='Drive temperature if available')
475     interface_type: Optional[str] = Field(default=None, descri
    ption='Drive interface or bus type (NVMe, SATA, USB, etc.)')
476     lifetime_read_bytes: Optional[int] = Field(default=None, d
    escription='Всего прочитано накопителем за время жизни (байт)
    ')
477     lifetime_write_bytes: Optional[int] = Field(default=None,
    description='Всего записано накопителем за время жизни (байт)')

```

```

)
478     lifetime_read_tb: Optional[float] = Field(default=None, de
description='Всего прочитано накопителем за время жизни (TB)')
479     lifetime_write_tb: Optional[float] = Field(default=None, d
escription='Всего записано накопителем за время жизни (TB)')
480     power_on_hours: Optional[int] = Field(default=None, descri
ption='Время наработки диска в часах')
481     wear_percentage: Optional[float] = Field(default=None, des
cription='Процент износа накопителя (0-100%)')
482
483 class RamStickInfo(BaseModel):
484     """Physical RAM stick SPD details."""
485     bank_label: str = Field(default='DIMM', description='Memor
y slot label')
486     capacity_gb: float = Field(default=0.0, description='Modul
e capacity in GB')
487     speed_mhz: int = Field(default=0, description='Configured
clock speed in MHz/MTs')
488     manufacturer: str = Field(default='Generic', description='
Memory manufacturer')
489     part_number: str = Field(default='', description='Module p
art number')
490     memory_type: str = Field(default='DDR4/DDR5', description=
'Memory technology type')
491
492 class NetworkPortMetrics(BaseModel):
493     """Active listening TCP/UDP port item."""
494     port: int = Field(..., description='Listening port number'
)
495     protocol: str = Field(default='TCP', description='Protocol
: TCP / UDP')
496     address: str = Field(default='0.0.0.0', description='Bind
IP address')
497     pid: Optional[int] = Field(default=None, description='Owni
ng process PID')
498     process_name: Optional[str] = Field(default=None, descript
ion='Owning process executable name')
499

```



```
500 class ProcessNetworkActivity(BaseModel):
501     """Сетевая активность процесса (интернет-соединения, отпра
502     вка и прием трафика)."""
503     pid: int = Field(default=0, description='Process ID')
504     name: str = Field(default='', description='Process executab
505     le name')
506     user: str = Field(default='', description='Process owner u
507     sername')
508     local_address: str = Field(default='', description='Local
509     IP:Port')
510     remote_address: str = Field(default='', description='Remot
511     e IP:Port')
512     remote_host: Optional[str] = Field(default=None, descripti
513     on='Resolved remote hostname or service name')
514     protocol: str = Field(default='TCP', description='Protocol
515     : TCP / UDP')
516     status: str = Field(default='ESTABLISHED', description='Co
517     nnection state (ESTABLISHED, LISTEN, etc.)')
518     is_internet: bool = Field(default=True, description='Wheth
519     er remote address is external Internet vs local loopback')
520     service_type: str = Field(default='Web/HTTPS', description
521     ='Recognized protocol service: HTTPS, HTTP, DNS, Cloud Sync, e
522     tc.')
523     sent_kb: float = Field(default=0.0, description='Всего отп
524     равлено/записано процессом (КБ)')
525     recv_kb: float = Field(default=0.0, description='Всего пол
526     учено/скачано/прочитано процессом (КБ)')
527     delta_sent_kb: float = Field(default=0.0, description='Объ
528     ем отправленных данных за последний измеряемый период (КБ)')
529     delta_recv_kb: float = Field(default=0.0, description='Объ
530     ем скачанных/полученных данных за последний измеряемый период
531     (КБ)')
532     sent_rate_kbs: float = Field(default=0.0, description='Ско
533     рость отдачи/отправки данных (КБ/с)')
534     recv_rate_kbs: float = Field(default=0.0, description='Ско
535     рость скачивания/приема данных (КБ/с)')
536     sent_summary: str = Field(default='', description='Человек
537     опонятное описание отправляемого (что шлет)')
```

```

519     recv_summary: str = Field(default='', description='Человек
| опонятное описание принимаемого (что принимает)')
520
521 class SystemHealthAlerts(BaseModel):
522     """System reliability and event log alerts."""
523     reboot_pending: bool = Field(default=False, description='W
| hether a system reboot is pending')
524     critical_events_count: int = Field(default=0, description=
| 'Critical event logs in last 24h')
525     latest_alert: str = Field(default='Система стабильна', des
| cription='Summary of latest health event')
526
527 class HardwareSensor(BaseModel):
528     """Hardware sensor reading (AIDA64 style) with provider tr
| acking for deduplication."""
529     model_config = ConfigDict(extra='allow')
530
531     sensor_id: str = Field(..., description='Unique sensor ide
| ntifier')
532     name: str = Field(..., description='Human readable sensor
| name')
533     category: str = Field(default='temperature', description='
| Category: temperature, fan, voltage, power')
534     value: float = Field(..., description='Current numeric sen
| sor reading')
535     unit: str = Field(default='°C', description='Measurement u
| nit')
536     min_value: Optional[float] = Field(default=None, descripti
| on='Recorded minimum')
537     max_value: Optional[float] = Field(default=None, descripti
| on='Recorded maximum')
538     provider: Optional[Any] = Field(default=None, description=
| 'Provider name or ID for deduplication (SensorProvider enum or
| int/str)')
539
540 class HardwareNode(BaseModel):
541     """AIDA64-like hardware component tree item."""
542     category: str = Field(..., description='Component category

```

```

    (Motherboard, CPU, GPU, Memory, Storage)')
543     name: str = Field(..., description='Device or component na
me')
544     properties: Dict[str, Any] = Field(default_factory=dict, d
escription='Detailed hardware properties')
545     children: List[HardwareNode] = Field(default_factory=list,
description='Sub-components or devices')
546
547 class MonitorInfo(BaseModel):
548     """Connected display monitor details."""
549     device: str = Field(default='', description='Display devic
e name (e.g. \\.\.\DISPLAY1)')
550     name: str = Field(default='Monitor', description='Friendly
monitor name or model')
551     adapter: str = Field(default='', description='Display adap
ter name')
552     width: int = Field(default=1920, description='Display reso
lution width')
553     height: int = Field(default=1080, description='Display res
olution height')
554     frequency_hz: int = Field(default=60, description='Refresh
rate in Hz')
555     bits_per_pixel: int = Field(default=32, description='Color
bit depth')
556     is_primary: bool = Field(default=False, description='Wheth
er this is the primary display')
557     orientation: str = Field(default='горизонтальная', descrip
tion='Ориентация дисплея: горизонтальная или вертикальная')
558
559 class WindowsUpdateInfo(BaseModel):
560     """Windows Update status and recent hotfixes."""
561     status: str = Field(default='Up to date', description='Ove
rall update status')
562     installed_kb_count: int = Field(default=0, description='To
tal installed KBs')
563     recent_hotfixes: List[str] = Field(default_factory=list, d
escription='Recent KB identifiers')
564     latest_installed_on: Optional[str] = Field(default=None, d

```

```

    escription='Date of most recent update')
565
566 class OfficeSuiteInfo(BaseModel):
567     """Microsoft Office and productivity suite detection telem
etry."""
568     installed: bool = Field(default=False, description='Whethe
r an Office suite is installed')
569     product_name: Optional[str] = Field(default=None, descript
ion='Product suite name (e.g. Microsoft 365, Office 2021)')
570     version: Optional[str] = Field(default=None, description='
Office suite release version string')
571     publisher: Optional[str] = Field(default=None, description
='Software vendor/publisher name')
572     status: str = Field(default='Не установлен', description='
Human-readable office status summary')
573
574 class CloudStorageInfo(BaseModel):
575     """Cloud drive and synchronization storage telemetry (OneD
rive)."""
576     installed: bool = Field(default=False, description='Whethe
r cloud storage drive is detected and active')
577     name: str = Field(default='OneDrive', description='Cloud s
torage service name')
578     path: Optional[str] = Field(default=None, description='Loc
al synchronized root folder path')
579     total_gb: Optional[float] = Field(default=None, descriptio
n='Total storage volume capacity in GB')
580     used_gb: Optional[float] = Field(default=None, description
='Used space in GB')
581     free_gb: Optional[float] = Field(default=None, description
='Free available space in GB')
582     percent_used: Optional[float] = Field(default=None, descri
ption='Storage disk utilization percentage')
583     status: str = Field(default='Не настроено', description='H
uman-readable cloud storage status')
584
585 class SystemSnapshot(BaseModel):
586     """Complete system and hardware telemetry snapshot."""

```

```
587     timestamp: str = Field(default_factory=lambda: datetime.now(
    w(timezone.utc).isoformat(), description='ISO 8601 UTC timestamp of snapshot')
588     hostname: str = Field(default='', description='Machine host name')
589     workgroup: str = Field(default='WORKGROUP', description='Machine workgroup or domain')
590     username: str = Field(default='', description='Current logged in username')
591     os_name: str = Field(default='Windows', description='Operating system name and version')
592     os_build: str = Field(default='', description='Operating system build number')
593     system_language: str = Field(default='', description='System UI language')
594     os_install_language: str = Field(default='', description='Original OS install language')
595     user_locale: str = Field(default='', description='User locale')
596     system_locale: str = Field(default='', description='System locale')
597     timezone: str = Field(default='', description='System timezone')
598     codepage: str = Field(default='', description='Active system code pages')
599     input_languages: List[str] = Field(default_factory=list, description='Installed keyboard input layouts')
600     os_install_date: str = Field(default='', description='Operating system installation date')
601     uptime_seconds: float = Field(default=0.0, description='System uptime in seconds')
602     cpu: CpuMetrics = Field(default_factory=CpuMetrics, description='CPU metrics')
603     memory: MemoryMetrics = Field(default_factory=MemoryMetrics, description='RAM and Swap metrics')
604     ram_sticks: List[RamStickInfo] = Field(default_factory=list, description='Installed physical RAM modules')
605     gpus: List[GpuMetrics] = Field(default_factory=list, descr
```

```
    iption='Detected GPU accelerators')
606     npus: List[NpuMetrics] = Field(default_factory=list, descr
    iption='Detected NPU neural processors')
607     monitors: List[MonitorInfo] = Field(default_factory=list,
    description='Connected display monitors')
608     updates: WindowsUpdateInfo = Field(default_factory=Windows
    UpdateInfo, description='Windows Update information')
609     office: OfficeSuiteInfo = Field(default_factory=OfficeSuit
    eInfo, description='Office suite telemetry')
610     onedrive: CloudStorageInfo = Field(default_factory=CloudSt
    orageInfo, description='OneDrive cloud storage telemetry')
611     disks: List[DiskPartitionMetrics] = Field(default_factory=
    list, description='Disk partitions')
612     physical_disks: List[PhysicalDiskHealth] = Field(default_f
    actory=list, description='Physical storage disks and SMART hea
    lth')
613     disk_io: DiskIoMetrics = Field(default_factory=DiskIoMetri
    cs, description='Aggregate disk I/O rates')
614     network: List[NetworkInterfaceMetrics] = Field(default_fac
    tory=list, description='Network interfaces')
615     listening_ports: List[NetworkPortMetrics] = Field(default_
    factory=list, description='Active listening ports and sockets'
    )
616     network_activity: List[ProcessNetworkActivity] = Field(def
    ault_factory=list, description='Сетевая активность программ (с
    оединения и трафик)')
617     battery: BatteryMetrics = Field(default_factory=BatteryMet
    rics, description='Battery and power state')
618     alerts: SystemHealthAlerts = Field(default_factory=SystemH
    ealthAlerts, description='System health and reliability alerts
    ')
619     sensors: List[HardwareSensor] = Field(default_factory=list
    , description='Hardware sensor readings')
620     top_processes: List[ProcessMetrics] = Field(default_factor
    y=list, description='Top active processes')
621     apps_count: int = Field(default=0, description='Количество
    запущенных оконных приложений (Apps)')
622     background_count: int = Field(default=0, description='Коли
```

```

        чество фоновых процессов (Background processes)')
623     windows_count: int = Field(default=0, description='Количес
        тво системных процессов Windows (Windows processes)')
624     hardware_audit: Dict[str, Any] = Field(default_factory=dict, description='Full hardware audit data (manufacturer, serial
        , etc.)')
625
626 class WindowsFeaturesSnapshot(BaseModel):
627     """Снимок состояния Windows Optional Features при старте т
        елеметрии."""
628     timestamp: str = Field(default_factory=lambda: datetime.no
        w(timezone.utc).isoformat(), description='ISO-8601 метка време
        ни')
629     features: List[Dict[str, Any]] = Field(default_factory=list, description='Список компонентов Windows Optional Features')
630
631
632 class AnomalyItem(BaseModel):
633     """Specific detected anomaly or performance bottleneck."""
634     subsystem: str = Field(..., description='Subsystem (CPU, R
        AM, GPU, Disk, Network, Process)')
635     severity: str = Field(default='warning', description='Seve
        rity level: info, warning, critical')
636     title: str = Field(..., description='Brief anomaly summary
        ')
637     description: str = Field(..., description='Detailed diagno
        stic description')
638
639 class SystemDiagnosticReport(BaseModel):
640     """AI-powered diagnostic audit report."""
641     timestamp: str = Field(default_factory=lambda: datetime.no
        w(timezone.utc).isoformat(), description='Audit timestamp')
642     health_score: int = Field(default=100, description='System
        health score (0-100)')
643     summary: str = Field(..., description='High-level health a
        nd performance summary')
644     anomalies: List[AnomalyItem] = Field(default_factory=list,
        description='List of detected anomalies')

```

```

645     recommendations: List[str] = Field(default_factory=list, d
description='Actionable optimization suggestions')
646     ai_model_used: str = Field(default='heuristic', descriptio
n='AI Model identifier or heuristic engine')
647     system_instruction: Optional[str] = Field(default=None, de
scription='System instruction used for AI model')
648     generated_prompt: Optional[str] = Field(default=None, desc
ription='Exact prompt sent to AI model')
649     raw_response: Optional[str] = Field(default=None, descript
ion='Raw response text from AI provider')
650     error: Optional[str] = Field(default=None, description='Er
ror details if inference failed')
651     stages: List[Dict[str, Any]] = Field(default_factory=list,
description='Step-by-step audit stages')
652
653 class DriverInfo(BaseModel):
654     """Сведения об установленном драйвере устройства и его акт
уальности."""
655     name: str = Field(default='', description='Имя драйвера ил
и сервиса')
656     driver_version: str = Field(default='', description='Верси
я установленного драйвера')
657     driver_date: Optional[str] = Field(default=None, descripti
on='Дата релиза драйвера (ISO или YYYY-MM-DD)')
658     provider: str = Field(default='Unknown', description='Пост
авщик драйвера (NVIDIA, Intel, Microsoft и др.)')
659     inf_name: Optional[str] = Field(default=None, description=
'Имя INF-файла драйвера (oem*.inf)')
660     is_signed: bool = Field(default=True, description='Наличие
цифровой подписи WHQL')
661     is_inbox: bool = Field(default=False, description='Являетс
я ли драйвер стандартным (inbox) от Microsoft')
662     age_days: Optional[int] = Field(default=None, description=
'Возраст драйвера в днях')
663     currency_status: str = Field(default='Актуален', descripti
on='Статус актуальности: Актуален, Устарел, Базовый драйвер ОС
')
664

```



```

665 class HardwareDeviceAudit(BaseModel):
666     """Единица аудита аппаратного устройства с драйверами, дат
ами и сенсорами."""
667     device_id: str = Field(..., description='Уникальный иденти
фикатор инстанса устройства (PnP Instance ID)')
668     name: str = Field(..., description='Понятное наименование
устройства')
669     device_class: str = Field(default='Device', description='К
ласс устройства (Display, Processor, Net, DiskDrive и др.)')
670     manufacturer: str = Field(default='Unknown', description='
Производитель оборудования')
671     install_date: Optional[str] = Field(default=None, descript
ion='Дата первой/текущей установки устройства в системе')
672     status: str = Field(default='OK', description='Рабочий ста
тус устройства (OK, Problem, Error, Disabled)')
673     problem_code: int = Field(default=0, description='Код ошиб
ки PnP (0 если исправно, 10, 43, 28 при сбое)')
674     driver: Optional[DriverInfo] = Field(default=None, descrip
tion='Сведения об используемом драйвере')
675     sensors: List[HardwareSensor] = Field(default_factory=list
, description='Сенсоры, привязанные к данному оборудованию')
676     properties: Dict[str, Any] = Field(default_factory=dict, d
escription='Дополнительные аппаратные параметры')
677
678 class HardwareChangeItem(BaseModel):
679     """Зафиксированное изменение в аппаратной конфигурации или
драйверах."""
680     change_type: str = Field(..., description='Тип изменения:
added, removed, driver_updated, status_changed, hardware_alter
ed')
681     device_id: str = Field(..., description='Идентификатор зат
ронутного устройства')
682     device_name: str = Field(..., description='Наименование ус
тройства')
683     description: str = Field(..., description='Подробное описа
ние изменения на русском языке')
684     previous_value: Optional[Dict[str, Any]] = Field(default=N
one, description='Предыдущее состояние параметра')

```

```
685     current_value: Optional[Dict[str, Any]] = Field(default=None, description='Новое состояние параметра')
686     timestamp: str = Field(default_factory=lambda: datetime.now(timezone.utc).isoformat(), description='Временная метка обнаружения изменения')
687
688 class HardwareAuditReport(BaseModel):
689     """Полный отчет аудита аппаратного обеспечения с привязкой сенсоров и драйверов."""
690     timestamp: str = Field(default_factory=lambda: datetime.now(timezone.utc).isoformat(), description='Время проведения аудита (UTC)')
691     devices_count: int = Field(default=0, description='Общее количество обнаруженных устройств')
692     problem_devices_count: int = Field(default=0, description='Количество устройств с ошибками PnP')
693     outdated_drivers_count: int = Field(default=0, description='Количество устройств с устаревшими драйверами')
694     devices: List[HardwareDeviceAudit] = Field(default_factory=list, description='Список проаудированных устройств')
695     summary: str = Field(default='', description='Сводка состояния аппаратной части')
696     changes_since_last_archive: List[HardwareChangeItem] = Field(default_factory=list, description='Изменения, обнаруженные по сравнению с последним сохраненным архивом')
697
698 class HardwareArchiveEntry(BaseModel):
699     """Запись в архиве истории оборудования."""
700     archive_id: str = Field(..., description='Уникальный идентификатор архивного снимка')
701     timestamp: str = Field(..., description='Время создания архива')
702     devices_count: int = Field(default=0, description='Количество устройств в архиве')
703     changes_count: int = Field(default=0, description='Количество изменений, зафиксированных в этом архиве')
704     report: HardwareAuditReport = Field(..., description='Полный отчет аудита оборудования')
```

```

705
706 class ProcessLeakItem(BaseModel):
707     """Метрики скрытой диагностики и утечек отдельного процесс
a."""
708     pid: int = Field(..., description='Идентификатор процесса'
)
709     name: str = Field(..., description='Имя процесса')
710     status: str = Field(default='running', description='Статус
процесса')
711     handles_count: int = Field(default=0, description='Количес
тво открытых дескрипторов (Handles)')
712     gdi_objects: int = Field(default=0, description='Количеств
о GDI-объектов (шрифты, битмапы, кисти, лимит 10 000)')
713     user_objects: int = Field(default=0, description='Количес
тво USER-объектов (окна, меню, курсоры)')
714     page_faults_total: int = Field(default=0, description='Общ
ее число страничных ошибок (Page Faults)')
715     peak_working_set_mb: float = Field(default=0.0, descriptio
n='Пиковый рабочий набор памяти в МБ')
716     memory_mb: float = Field(default=0.0, description='Текущая
память (RSS) в МБ')
717     threads_count: int = Field(default=1, description='Количес
тво потоков процесса')
718     cpu_percent: float = Field(default=0.0, description='Нагру
зка на CPU %')
719     leak_risk_score: str = Field(default='normal', description
='Уровень риска утечки: normal, warning, critical')
720     leak_risk_reasons: List[str] = Field(default_factory=list,
description='Список признаков аномального поведения')
721
722 class ProcessLeakDiagnosticsReport(BaseModel):
723     """Сводный отчет скрытой диагностики процессов, утечек дес
крипторов и ресурсов UI."""
724     timestamp: str = Field(default_factory=lambda: datetime.no
w(timezone.utc).isoformat(), description='Время сбора среза')
725     total_processes: int = Field(default=0, description='Всего
проанализировано процессов')
726     suspicious_count: int = Field(default=0, description='Коли

```

```
чество процессов с признаками утечки')
727     top_handle_hogs: List[ProcessLeakItem] = Field(default_factory=list, description='Топ процессов по открытым дескрипторам')
728     top_gdi_hogs: List[ProcessLeakItem] = Field(default_factory=list, description='Топ процессов по GDI объектам')
729     top_page_fault_hogs: List[ProcessLeakItem] = Field(default_factory=list, description='Топ процессов по обращениям к файлу подкачки')
730     all_processes: List[ProcessLeakItem] = Field(default_factory=list, description='Полный список проанализированных процессов')
731
732 class ForensicsActivityReport(BaseModel):
733     """Поведенческая телеметрия и форензика активности пользователя."""
734     timestamp: str = Field(default_factory=lambda: datetime.now(timezone.utc).isoformat(), description='Время фиксации')
735     foreground_window: Dict[str, Any] = Field(default_factory=dict, description='Активное окно в фокусе (заголовок, процесс, PID, время)')
736     user_idle_seconds: float = Field(default=0.0, description='Время бездействия пользователя (без ввода с клавиатуры/мыши)')
737     camera_active_apps: List[Dict[str, Any]] = Field(default_factory=list, description='Приложения, использующие или недавно вызывавшие камеру')
738     microphone_active_apps: List[Dict[str, Any]] = Field(default_factory=list, description='Приложения, использующие микрофон')
739     userassist_top_apps: List[Dict[str, Any]] = Field(default_factory=list, description='История запусков из реестра UserAssist (счетчик и фокусное время)')
740
741 class KernelThrottlingReport(BaseModel):
742     """Качество работы ядра, прерываний и аппаратного троттлинга."""
743     timestamp: str = Field(default_factory=lambda: datetime.now(
```

```

w(timezone.utc).isoformat(), description='Время среза')
744     dpc_latency_pct: float = Field(default=0.0, description='П
процент времени ЦП в DPC (отложенных вызовах процедур)')
745     interrupt_latency_pct: float = Field(default=0.0, descript
ion='Процент времени ЦП в аппаратных прерываниях (ISR)')
746     dpc_status: str = Field(default='optimal', description='Ст
атус латентности прерываний: optimal, elevated, severe')
747     thermal_throttling_detected: bool = Field(default=False, d
escription='Обнаружен термический троттинг ядер (PROCHOT)')
748     power_limit_throttling_detected: bool = Field(default=False,
description='Обнаружено ограничение по питанию (PL1/PL2 Lim
it)')
749     system_uptime_seconds: float = Field(default=0.0, descript
ion='Время непрерывной работы системы (Uptime)')
750     uptime_formatted: str = Field(default='0 дн 0 ч', descript
ion='Отформатированное время Uptime')
751     last_bsod_crashes: List[Dict[str, Any]] = Field(default_fa
ctory=list, description='Последние зафиксированные падения сис
темы (BugCheck / Minidump)')
752     gpu_pcie_link: Dict[str, Any] = Field(default_factory=dict
, description='Режим шины PCIe для видеокарты (Link Speed / Wi
dth)')
753
754 class StorageBatteryWearReport(BaseModel):
755     """Телеметрия износа накопителей SSD/NVMe и батареи питани
я."""
756     timestamp: str = Field(default_factory=lambda: datetime.no
w(timezone.utc).isoformat(), description='Время формирования')
757     disks_wear: List[Dict[str, Any]] = Field(default_factory=l
ist, description='Метрики износа SSD, TBW и здоровье накопител
ей')
758     battery_wear: Dict[str, Any] = Field(default_factory=dict,
description='Глубокая телеметрия износа аккумулятора (Design
vs Full Charge)')
759
760 class PeripheralsNetworkReport(BaseModel):
761     """Телеметрия сети (Wi-Fi, интерфейсы) и периферийных устр
ойств (USB, Audio)."""

```

```

762     timestamp: str = Field(default_factory=lambda: datetime.now(
    timezone.utc).isoformat(), description='Время сбора')
763     usb_devices: List[Dict[str, Any]] = Field(default_factory=
    list, description='Подключенные USB-устройства и контроллеры')
764     wifi_telemetry: Dict[str, Any] = Field(default_factory=dict,
    description='Параметры текущей беспроводной сети Wi-Fi (SSID,
    BSSID, RSSI, канал)')
765     audio_endpoints: List[Dict[str, Any]] = Field(default_factory=
    list, description='Аудиоустройства и активные конечные точки')
766
767 class SystemCoreMetrics(BaseModel):
768     """Быстрые базовые метрики хоста (CPU, RAM, GPU, Disk I/O,
    Battery)."""
769     cpu: CpuMetrics
770     memory: MemoryMetrics
771     gpus: List[GpuMetrics] = Field(default_factory=list, description=
    'Метрики графических ускорителей')
772     disk_io: DiskIoMetrics
773     battery: BatteryMetrics
774     uptime_seconds: float = Field(default=0.0, description='Вре
    мя непрерывной работы в секундах')
775
776 class SystemHardwareQuick(BaseModel):
777     """Сводка состояния физических компонентов (диски, слоты п
    амяти, открытые сокет)."""
778     ram_sticks: List[RamStickInfo] = Field(default_factory=list,
    description='Конфигурация планок RAM (SPD)')
779     physical_disks: List[PhysicalDiskHealth] = Field(default_f
    actory=list, description='Состояние физических накопителей (SM
    ART)')
780     listening_ports: List[NetworkPortMetrics] = Field(default_
    factory=list, description='Список активных слушающих сокетов')
781     alerts: SystemHealthAlerts = Field(default_factory=SystemH
    ealthAlerts, description='Системные предупреждения и флаг пере
    загрузки')
782
783 class W64SystemEvent(BaseModel):

```

```

784     """Модель отдельного системного события AI W64 Collector."
785     """
786     event_id: str = Field(default='', description='Уникальный
идентификатор события')
787     timestamp: str = Field(default_factory=lambda: datetime.no
w(timezone.utc).isoformat(), description='Время события UTC')
788     event_type: str = Field(..., description='Тип события (pro
cess_start, file_modified, registry_change, network_open и др.
)')
789     path: Optional[str] = Field(default=None, description='Пут
ь к файлу или ветке реестра')
790     pid: Optional[int] = Field(default=None, description='Иден
тификатор процесса')
791     name: Optional[str] = Field(default=None, description='Имя
процесса/файла/параметра')
792     details: Dict[str, Any] = Field(default_factory=dict, desc
ription='Дополнительные атрибуты события')
793
794     def to_dict(self) -> Dict[str, Any]:
795         """Сериализация события в словарь."""
796         res = self.model_dump()
797         if self.details:
798             res.update(self.details)
799         return res
800
801 class ETWTraceEvent(BaseModel):
802     """Модель низкоуровневого события ETW (Event Tracing for W
indows)."""
803     event_id: str = Field(default='', description='Уникальный
идентификатор события')
804     timestamp: str = Field(default_factory=lambda: datetime.no
w(timezone.utc).isoformat(), description='Время события UTC')
805     event_type: str = Field(..., description='Тип ETW трассиро
вки (etw_process_create, etw_file_io, etw_network и др.)')
806     provider: str = Field(default='Microsoft-Windows-Security-
Auditing', description='Провайдер ETW')
807     payload: Dict[str, Any] = Field(default_factory=dict, desc
ription='Сырые поля события из ETW / Windows Event Log')

```

```
807
808     def to_dict(self) -> Dict[str, Any]:
809         """Сериализация ETW события в словарь."""
810         return self.model_dump()
811
812 class W64CollectorStatus(BaseModel):
813     """Статус и статистика работы сборщиков W64/ETW."""
814     running: bool = Field(default=False, description='Флаг активности сборщика')
815     events_count: int = Field(default=0, description='Общее количество зафиксированных событий')
816     last_event_time: Optional[str] = Field(default=None, description='Время последнего зафиксированного события')
817
818
819 class SamplingMode(str, Enum):
820     """Режимы сбора телеметрии и адаптивного масштабирования частоты."""
821     NORMAL = 'normal'
822     MONITORING = 'monitoring'
823     BOOT_AGGRESSIVE = 'boot_aggressive'
824     INCIDENT = 'incident'
825     FORENSIC = 'forensic'
826
827
828 class SystemMetricRollup(BaseModel):
829     """Сжатый временной срез системных метрик с расширенной статистикой (avg, min, max, p95, total)."""
830     period_start: str = Field(..., description='Начало периода агрегации (ISO timestamp)')
831     period_end: str = Field(..., description='Окончание периода агрегации (ISO timestamp)')
832     duration_seconds: float = Field(default=60.0, description='Длительность интервала в секундах')
833     sample_count: int = Field(default=0, description='Количество объединенных замеров')
834     tier: str = Field(default='1m', description='Уровень сжатия: 1s, 2s, 5s, 10s, 30s, 1m, 5m, 15m, 1h, 6h, 24h')
```



```
835 |
836 |     # CPU
837 |     cpu_avg: float = Field(default=0.0, description='Средняя н
агрузка CPU %')
838 |     cpu_min: float = Field(default=0.0, description='Минималън
ая нагрузка CPU %')
839 |     cpu_max: float = Field(default=0.0, description='Пиковая н
агрузка CPU %')
840 |     cpu_p95: float = Field(default=0.0, description='95-й перц
ентиль нагрузки CPU %')
841 |
842 |     # RAM
843 |     ram_avg_gb: float = Field(default=0.0, description='Средни
й объем занятой RAM в GB')
844 |     ram_min_gb: float = Field(default=0.0, description='Минима
льный объем занятой RAM в GB')
845 |     ram_max_gb: float = Field(default=0.0, description='Максим
альный объем занятой RAM в GB')
846 |     ram_percent_avg: float = Field(default=0.0, description='С
редний процент занятой памяти')
847 |     ram_percent_max: float = Field(default=0.0, description='М
аксимальный процент занятой памяти')
848 |
849 |     # Disk
850 |     disk_read_avg_mbs: float = Field(default=0.0, description=
'Sредняя скорость чтения с диска в МБ/с')
851 |     disk_read_max_mbs: float = Field(default=0.0, description=
'Пиковая скорость чтения с диска в МБ/с')
852 |     disk_read_total_mb: float = Field(default=0.0, description
='Суммарно прочитано за период в МБ')
853 |     disk_write_avg_mbs: float = Field(default=0.0, description
='Средняя скорость записи на диск в МБ/с')
854 |     disk_write_max_mbs: float = Field(default=0.0, description
='Пиковая скорость записи на диск в МБ/с')
855 |     disk_write_total_mb: float = Field(default=0.0, descriptio
n='Суммарно записано за период в МБ')
856 |
857 |     # Network
```

```
858     network_rx_avg_mbs: float = Field(default=0.0, description
    = 'Средняя скорость входящего трафика в МБ/с')
859     network_rx_max_mbs: float = Field(default=0.0, description
    = 'Пиковая скорость входящего трафика в МБ/с')
860     network_rx_total_mb: float = Field(default=0.0, description
    = 'Суммарно получено входящего трафика в МБ')
861     network_tx_avg_mbs: float = Field(default=0.0, description
    = 'Средняя скорость исходящего трафика в МБ/с')
862     network_tx_max_mbs: float = Field(default=0.0, description
    = 'Пиковая скорость исходящего трафика в МБ/с')
863     network_tx_total_mb: float = Field(default=0.0, description
    = 'Суммарно отправлено исходящего трафика в МБ')
864
865
866 class ProcessMetricRollup(BaseModel):
867     """Сжатый временной срез per-process метрик за период набл
    юдения."""
868     name: str = Field(..., description='Имя исполняемого файла
    процесса')
869     pid: Optional[int] = Field(default=None, description='PID
    процесса (если отслеживался конкретный экземпляр)')
870     period_start: str = Field(..., description='Начало периода
    (ISO timestamp)')
871     period_end: str = Field(..., description='Конец периода (I
    SO timestamp)')
872     duration_seconds: float = Field(default=60.0, description=
    'Длительность наблюдения')
873     sample_count: int = Field(default=0, description='Число за
    мер')
874     path: Optional[str] = Field(default=None, description='Пут
    ь к исполняемому файлу')
875
876     cpu_avg: float = Field(default=0.0, description='Средний C
    PU %')
877     cpu_max: float = Field(default=0.0, description='Пиковый C
    PU %')
878     cpu_p95: float = Field(default=0.0, description='95-й перц
    ентиль CPU %')
```

```
879
880     ram_avg_mb: float = Field(default=0.0, description='Средни
    й объем RAM в МБ')
881     ram_max_mb: float = Field(default=0.0, description='Пиковы
    й объем RAM в МБ')
882
883     disk_read_total_mb: float = Field(default=0.0, description
    ='Суммарно прочитано с диска в МБ')
884     disk_write_total_mb: float = Field(default=0.0, descriptio
    n='Суммарно записано на диск в МБ')
885     network_rx_total_mb: float = Field(default=0.0, descriptio
    n='Суммарно получено по сети в МБ')
886     network_tx_total_mb: float = Field(default=0.0, descriptio
    n='Суммарно отправлено по сети в МБ')
887
888
889 class TelemetryIncident(BaseModel):
890     """Модель диагностического инцидента аномалии с привязкой
    контекста и кольцевого буфера."""
891     incident_id: str = Field(..., description='Уникальный иден
    тификатор инцидента (напр. INC-20261001-01)')
892     timestamp: str = Field(default_factory=lambda: datetime.no
    w(timezone.utc).isoformat(), description='Время детекции инцид
    ента')
893     trigger_type: str = Field(..., description='Тип триггера:
    cpu_spike, disk_burst, network_burst, suspicious_process, dpc_
    degradation, defender_alert')
894     severity: str = Field(default='warning', description='Крит
    ичность: info, warning, critical')
895     title: str = Field(..., description='Краткое название инци
    дента на русском языке')
896     description: str = Field(default='', description='Подробно
    е описание выявленной аномалии')
897
898     trigger_metrics: Dict[str, Any] = Field(default_factory=dict,
    description='Значения метрик в момент срабатывания триггера')
899     suspect_processes: List[Dict[str, Any]] = Field(default_fa
```

```

    story=list, description='Список подозрительных процессов, связ
анных с аномалией')
900     related_events: List[Dict[str, Any]] = Field(default_facto
ry=list, description='Дискретные события (запуск процессов, ET
W, Defender) за период')
901     metrics_summary: Dict[str, Any] = Field(default_factory=di
ct, description='Агрегированная сводка метрик до и после инцид
ента')
902     raw_window: List[Dict[str, Any]] = Field(default_factory=l
ist, description='Срез сырых секундных замеров из кольцевого б
уфера (pre/post window)')
903
904
905 class ShutdownType(str, Enum):
906     """Типы завершения работы и перезагрузки Windows."""
907     PLANNED = 'PLANNED'
908     UNEXPECTED = 'UNEXPECTED'
909     CRASH_BSOD = 'CRASH_BSOD'
910     UPDATE_RESTART = 'UPDATE_RESTART'
911     USER_INITIATED = 'USER_INITIATED'
912     HARDWARE_POWER_BUTTON = 'HARDWARE_POWER_BUTTON'
913     DIRTY_POWER_LOSS = 'DIRTY_POWER_LOSS'
914     HYBRID_SHUTDOWN = 'HYBRID_SHUTDOWN'
915     UNKNOWN = 'UNKNOWN'
916
917
918 class RebootCorrelatedEvent(BaseModel):
919     """Событие, связанное с перезагрузкой или завершением рабо
ты."""
920     event_id: int = Field(..., description='Event ID (например
, 1074, 41, 6008, 1001)')
921     provider: str = Field(..., description='Источник/провайдер
события (User32, Kernel-Power, EventLog и др.)')
922     timestamp: str = Field(..., description='Время регистрации
события в формате ISO/строка')
923     level: str = Field(default='Information', description='Уро
вень события (Critical, Error, Warning, Information)')
924     description: str = Field(default='', description='Краткое

```

```
описание смысла события на русском языке')
925     details: Dict[str, Any] = Field(default_factory=dict, description='Извлеченные структурированные поля события')
926
927
928 class RebootSession(BaseModel):
929     """Структурированная запись об одной сессии перезагрузки /
    выключения Windows."""
930     boot_id: str = Field(..., description='Уникальный идентификатор сессии перезагрузки (напр. boot-20261001-014218)')
931     boot_time: str = Field(..., description='Время запуска системы (OS Startup)')
932     previous_boot_time: Optional[str] = Field(default=None, description='Время предыдущего запуска системы')
933     uptime_seconds: Optional[float] = Field(default=None, description='Время непрерывной работы предыдущей сессии в секундах')
934     uptime_human: Optional[str] = Field(default=None, description='Человекочитаемое время работы (напр. "7h 25m")')
935     shutdown_type: ShutdownType = Field(default=ShutdownType.UNKNOWN, description='Классифицированный тип завершения работы')
936     likely_class: str = Field(default='unknown', description='Короткий машинный класс причины')
937     conclusion: str = Field(default='', description='Итоговое аналитическое заключение на русском языке')
938     initiating_process: Optional[str] = Field(default=None, description='Процесс, инициировавший перезагрузку (из Event 1074)')
939     initiating_user: Optional[str] = Field(default=None, description='Учетная запись, от имени которой инициирован рестарт')
940     shutdown_action: Optional[str] = Field(default=None, description='Действие: restart, power off, shutdown, hybrid')
941     reason_text: Optional[str] = Field(default=None, description='Текстовая причина перезапуска')
942     reason_code: Optional[str] = Field(default=None, description='Шестнадцатеричный код причины (напр. 0x80020010)')
943     bugcheck_code: Optional[str] = Field(default=None, descrip
```

```

tion='Код остановки BSOD (напр. 0x000000D1)')
944     bugcheck_params: List[str] = Field(default_factory=list, d
description='Параметры BugCheck (p1..p4)')
945     power_button_timestamp: Optional[int] = Field(default=None
, description='Метка нажатия кнопки питания из Event 41')
946     windows_update_kb: Optional[str] = Field(default=None, des
cription='Номер установленного обновления Windows (Event 19)')
947     service_installed: Optional[str] = Field(default=None, des
cription='Имя недавно установленной службы (Event 7045)')
948     evidence: List[str] = Field(default_factory=list, descript
ion='Список доказательных фактов и событий')
949     events_chain: List[RebootCorrelatedEvent] = Field(default_
factory=list, description='Хронологическая цепочка коррелирова
нных событий')
950
951
952 class RebootAnalysisReport(BaseModel):
953     """Сводный отчет анализа стабильности и причин перезагрузо
к системы."""
954     generated_at: str = Field(default_factory=lambda: datetime
.now(timezone.utc).isoformat(), description='Время формирова
ния отчета')
955     hostname: str = Field(default='', description='Имя компьют
ера')
956     current_boot_time: str = Field(default='', description='Вр
емя текущей загрузки Windows')
957     current_uptime_seconds: float = Field(default=0.0, descrip
tion='Текущий аптайм в секундах')
958     current_uptime_human: str = Field(default='', description=
'Текущий аптайм в читаемом формате')
959     total_reboots_analyzed: int = Field(default=0, description
='Количество проанализированных перезагрузок')
960     planned_count: int = Field(default=0, description='Количес
тво плановых перезагрузок')
961     unexpected_count: int = Field(default=0, description='Коли
чество неожиданных/аварийных выключений')
962     bsod_count: int = Field(default=0, description='Количество
сбоев с синим экраном (BSOD)')

```

```

963     update_reboot_count: int = Field(default=0, description='К
| оличество перезагрузок из-за обновлений Windows')
964     stability_score: float = Field(default=100.0, description=
| 'Оценка стабильности завершений работы (0..100)')
965     sessions: List[RebootSession] = Field(default_factory=list
| , description='Список сессий перезагрузок (от новых к старым)'
| )
966     summary_ru: str = Field(default='', description='Общее рез
| юме надежности на русском языке')
967
968
969 class DefenderTelemetrySummary(BaseModel):
970     """Сводка телеметрии встроенного защитника Windows Defende
| r."""
971     cfa_enabled: bool = Field(default=False, description='Вклю
| чена ли защита Controlled Folder Access (от Ransomware)')
972     cfa_mode: str = Field(default='Disabled', description='Реж
| им CFA: Enabled, Disabled, Audit, Warn')
973     cfa_protected_folders_count: int = Field(default=0, descri
| ption='Количество защищенных папок')
974     asr_rules_count: int = Field(default=0, description='Колич
| ество настроенных правил ASR')
975     asr_enabled_count: int = Field(default=0, description='Кол
| ичество активных правил ASR')
976     exclusions_count: int = Field(default=0, description='Обще
| е количество исключений антивируса')
977     path_exclusions: List[str] = Field(default_factory=list, d
| escription='Список путей, исключенных из проверки')
978     process_exclusions: List[str] = Field(default_factory=list
| , description='Список процессов, исключенных из проверки')
979     active_threats_count: int = Field(default=0, description='
| Количество активных необработанных угроз')
980     threat_history_count: int = Field(default=0, description='
| Количество зафиксированных угроз в истории')
981
982
983 class StartupTelemetrySummary(BaseModel):
984     """Сводка телеметрии точек автозапуска и персистентности п

```

```

программ."""
985     total_entries: int = Field(default=0, description='Общее к
оличество программ и элементов в автозагрузке')
986     registry_run_count: int = Field(default=0, description='За
писи автозапуска в реестре Run/RunOnce')
987     startup_folders_count: int = Field(default=0, description=
'Ярлыки в папках Startup (пользователь/система)')
988     scheduled_tasks_count: int = Field(default=0, description=
'Задачи планировщика со стартом при входе')
989     startup_services_count: int = Field(default=0, description
='Службы Windows с автозапуском')
990     entries: List[Dict[str, Any]] = Field(default_factory=list
, description='Список записей автозапуска с параметрами')
991
992 class VssTelemetrySummary(BaseModel):
993     """Сводка состояния теневых копий томов VSS и точек восста
новления."""
994     total_snapshots_count: int = Field(default=0, description=
'Количество существующих теневых копий')
995     protected_volumes: List[str] = Field(default_factory=list,
description='Список защищенных томов (буквы дисков)')
996     latest_snapshot_time: Optional[str] = Field(default=None,
description='Время создания последней теневой копии')
997     snapshots: List[Dict[str, Any]] = Field(default_factory=li
st, description='Список снимков VSS')
998
999
1000 class UserAccountsTelemetrySummary(BaseModel):
1001     """Сводка аудита локальных пользователей и привилегий."""
1002     total_users_count: int = Field(default=0, description='Общ
ее число локальных учетных записей')
1003     admin_users_count: int = Field(default=0, description='Чис
ло учетных записей с правами администратора')
1004     locked_users_count: int = Field(default=0, description='Чи
сло заблокированных учетных записей')
1005     admin_usernames: List[str] = Field(default_factory=list, d
escription='Имена локальных администраторов')
1006     logged_in_users: List[str] = Field(default_factory=list, d

```



```
description='Активные вошедшие пользователи')
1007     users: List[Dict[str, Any]] = Field(default_factory=list,
description='Сводный список учетных записей')
1008
1009
1010 class ExtendedSystemAuditReport(BaseModel):
1011     """Комплексный отчет расширенного аудита безопасности, авт
оагрузки, VSS и пользователей."""
1012     timestamp: str = Field(default_factory=lambda: datetime.no
w(timezone.utc).isoformat(), description='Время аудита')
1013     hostname: str = Field(default='', description='Имя компьют
ера')
1014     defender: DefenderTelemetrySummary = Field(default_factory
=DefenderTelemetrySummary, description='Сводка Windows Defende
r')
1015     startup: StartupTelemetrySummary = Field(default_factory=S
tartupTelemetrySummary, description='Сводка автозагрузки')
1016     vss: VssTelemetrySummary = Field(default_factory=VssTeleme
trySummary, description='Сводка теневых копий VSS')
1017     users: UserAccountsTelemetrySummary = Field(default_factor
y=UserAccountsTelemetrySummary, description='Сводка локальных
пользователей')
1018
1019
1020 class SecurityEventItem(BaseModel):
1021     """Нормализованная запись журнала безопасности Windows (Se
curity Event Log)."""
1022     id: Optional[int] = Field(default=None, description='Идент
ификатор в базе данных')
1023     event_record_id: int = Field(default=0, description='Event
RecordID в журнале Windows')
1024     event_id: int = Field(..., description='Идентификатор типа
события (Event ID, например 4688, 4624)')
1025     timestamp: str = Field(default='', description='Время возн
икновения события (ISO/локальное)')
1026     created_at: float = Field(default_factory=lambda: datetime
.now(timezone.utc).timestamp(), description='Unix epoch timest
amp')
```

```
1027     computer: str = Field(default='', description='Имя компьютера')
1028     channel: str = Field(default='Security', description='Канал журнала')
1029     level: str = Field(default='Information', description='Уровень важности')
1030
1031     # Субъект (кто инициировал действие)
1032     subject_user: str = Field(default='', description='Имя пользователя инициатора')
1033     subject_domain: str = Field(default='', description='Домен инициатора')
1034     subject_sid: str = Field(default='', description='SID инициатора')
1035
1036     # Целевой субъект (над кем/чем действие)
1037     target_user: str = Field(default='', description='Имя целевого пользователя')
1038     target_domain: str = Field(default='', description='Домен целевого пользователя')
1039     target_sid: str = Field(default='', description='SID целевого пользователя')
1040
1041     # Процессы и запуск (Event ID 4688, 4689)
1042     process_id: int = Field(default=0, description='PID процесса')
1043     process_name: str = Field(default='', description='Имя или путь процесса')
1044     parent_process_id: int = Field(default=0, description='PID родительского процесса')
1045     parent_process_name: str = Field(default='', description='Имя родительского процесса')
1046     command_line: str = Field(default='', description='Командная строка запуска')
1047
1048     # Сессия и сеть (4624, 4625, Kerberos)
1049     logon_id: str = Field(default='', description='Идентификатор сеанса входа LogonID')
```

```
1050     logon_type: Optional[int] = Field(default=None, description='Тип входа (2=Interactive, 3=Network, 10=RDP и т.д.)')
1051     elevated_token: Optional[int] = Field(default=None, description='Признак повышенных прав (Elevation)')
1052     source_ip: str = Field(default='', description='Исходный IP адрес')
1053     source_port: Optional[int] = Field(default=None, description='Исходный сетевой порт')
1054
1055     # Объекты и статус
1056     object_name: str = Field(default='', description='Имя затронутого объекта или службы')
1057     status_code: str = Field(default='', description='Код статуса/ошибки (Status/SubStatus)')
1058     message: str = Field(default='', description='Человекочитаемое описание события')
1059     event_data: Dict[str, Any] = Field(default_factory=dict, description='Словарь всех извлеченных полей события')
1060     ingested_at: str = Field(default_factory=lambda: datetime.now(timezone.utc).isoformat(), description='Время сохранения в телеметрию')
1061
1062
1063 class SecurityEventRaw(BaseModel):
1064     """Сырая запись события журнала безопасности для повторного анализа."""
1065     id: Optional[int] = Field(default=None, description='Идентификатор в базе')
1066     event_record_id: int = Field(default=0, description='Event RecordID в журнале Windows')
1067     event_id: int = Field(..., description='Идентификатор типа события')
1068     timestamp: str = Field(default='', description='Время события')
1069     created_at: float = Field(default_factory=lambda: datetime.now(timezone.utc).timestamp(), description='Unix epoch timestamp')
1070     channel: str = Field(default='Security', description='Кана
```

```
    л журнала')
1071     raw_xml: str = Field(default='', description='Исходный XML
    события')
1072     ingested_at: str = Field(default_factory=lambda: datetime.
    now(timezone.utc).isoformat(), description='Время захвата')
1073
1074
1075 class SecurityBookmarkState(BaseModel):
1076     """Состояние закладки инкрементального сбора."""
1077     channel: str = Field(default='Security', description='Имя
    канала')
1078     last_record_id: int = Field(default=0, description='Послед
    ний обработанный EventRecordID')
1079     bookmark_xml: Optional[str] = Field(default=None, descript
    ion='XML закладки WevtAPI')
1080     last_timestamp: str = Field(default='', description='Время
    последнего события')
1081     updated_at: float = Field(default_factory=lambda: datetime
    .now(timezone.utc).timestamp(), description='Время обновления'
    )
1082
1083
1084 class SecurityAuditStatus(BaseModel):
1085     """Статус аудита безопасности Windows и доступности журнал
    а Security."""
1086     accessible: bool = Field(default=False, description='Досту
    пен ли журнал Security текущему процессу')
1087     record_count: int = Field(default=0, description='Всего за
    писей в журнале Security')
1088     process_creation_audit_enabled: bool = Field(default=False
    , description='Включен ли аудит создания процессов (Event 4688
    )')
1089     command_line_audit_enabled: bool = Field(default=False, de
    scription='Включено ли включение CommandLine в Event 4688')
1090     error: Optional[str] = Field(default=None, description='Ош
    ибка доступа при наличии')
1091     last_record_id: int = Field(default=0, description='Послед
    ний обработанный EventRecordID в телеметрии')
```

```
1092     total_ingested_events: int = Field(default=0, description=
| 'Количество событий в telemetry.db')
1093
1094
1095 class SecurityCorrelationItem(BaseModel):
1096     """Результат корреляции событий безопасности с метриками т
| елеметрии."""
1097     timestamp: str = Field(default='', description='Временная
| метка')
1098     event_type: str = Field(default='', description='Тип событ
| ия (ProcessCreate, Logon, etc.)')
1099     user: str = Field(default='', description='Пользователь')
1100     process_name: str = Field(default='', description='Процесс
| ')
1101     pid: int = Field(default=0, description='PID')
1102     parent_process_name: str = Field(default='', description='
| Родительский процесс')
1103     command_line: str = Field(default='', description='Команда
| ')
1104     cpu_percent: Optional[float] = Field(default=None, descrip
| tion='Нагрузка CPU в этот момент')
1105     ram_mb: Optional[float] = Field(default=None, description=
| 'Использование RAM процессом')
1106     disk_spike: bool = Field(default=False, description='Вспле
| ск дисковой активности')
1107     network_spike: bool = Field(default=False, description='Вс
| плеск сетевой активности')
1108     notes: List[str] = Field(default_factory=list, description
| ='Аналитические заметки')
1109
1110
1111 class SecurityCollectorReport(BaseModel):
1112     """Сводный отчет работы сборщика событий безопасности."""
1113     channel: str = Field(default='Security', description='Кана
| л')
1114     total_events_ingested: int = Field(default=0, description=
| 'Количество сохраненных событий')
1115     last_record_id: int = Field(default=0, description='Послед
```

```

    ний EventRecordID')
1116     bookmark_xml: Optional[str] = Field(default=None, description='XML закладки')
1117     events_by_id: Dict[int, int] = Field(default_factory=dict, description='Распределение событий по Event ID')
1118     errors: List[str] = Field(default_factory=list, description='Ошибки во время сбора')
1119     events: List[SecurityEventItem] = Field(default_factory=list, description='Список собранных событий')
1120
1121
1122 class PowerEventRecord(BaseModel):
1123     """Модель отдельного события жизненного цикла и питания Windows."""
1124     id: Optional[int] = Field(default=None, description='Идентификатор в базе данных')
1125     event_id: int = Field(..., description='Event ID (1074, 13, 6006, 6008, 41, 12, 6005, 6009, 1001, 19, 42, 107)')
1126     provider: str = Field(default='', description='Имя провайдера (User32, Kernel-General, EventLog, Kernel-Power, etc.)')
1127     channel: str = Field(default='System', description='Имя канала')
1128     timestamp: str = Field(default='', description='Временная метка ISO/текст')
1129     created_at: float = Field(default_factory=lambda: datetime.now(timezone.utc).timestamp(), description='Epoch timestamp')
1130     event_type: str = Field(default='event', description='Тип события: boot, shutdown_clean, shutdown_planned, shutdown_unexpected, sleep, wake, bsod, update_install')
1131     shutdown_type: Optional[str] = Field(default=None, description='Тип выключения: restart, power off, shutdown, unexpected, etc.')
1132     user: Optional[str] = Field(default=None, description='Пользователь / инициатор (User)')
1133     domain: Optional[str] = Field(default=None, description='Домен или имя хоста инициатора')
1134     process: Optional[str] = Field(default=None, description='Исполняемый процесс (shutdown.exe, explorer.exe, wmiprvse.exe)

```

```

')
1135     process_id: Optional[int] = Field(default=None, description='Идентификатор процесса (PID)')
1136     reason: Optional[str] = Field(default=None, description='Причина выключения / перезагрузки')
1137     reason_code: Optional[str] = Field(default=None, description='Код причины (например, 0x80020010)')
1138     comment: Optional[str] = Field(default=None, description='Пользовательский комментарий к выключению')
1139     unexpected: bool = Field(default=False, description='Флаг неожиданного выключения / потери питания')
1140     bugcheck_code: Optional[str] = Field(default=None, description='Код BugCheck (BSOD)')
1141     bugcheck_params_json: Optional[str] = Field(default=None, description='Параметры BugCheck')
1142     boot_id: Optional[str] = Field(default=None, description='Идентификатор загрузочной сессии')
1143     details_json: Union[Dict[str, Any], str] = Field(default_factory=dict, description='Дополнительные извлеченные атрибуты')
1144     raw_xml: Optional[str] = Field(default=None, description='Исходный XML события')
1145
1146
1147 class PowerSessionRecord(BaseModel):
1148     """Модель реконструированной сессии работы операционной системы (Power Session)."""
1149     id: Optional[int] = Field(default=None, description='Идентификатор в базе данных')
1150     session_id: str = Field(..., description='Уникальный идентификатор сессии (например PS-20261006-042606)')
1151     boot_time: str = Field(..., description='Время старта/загрузки операционной системы')
1152     boot_timestamp: float = Field(default=0.0, description='Epoch timestamp старта')
1153     shutdown_time: Optional[str] = Field(default=None, description='Время выключения/перезагрузки (None для активной сессии)')

```

```
1154 shutdown_timestamp: Optional[float] = Field(default=None,
description='Epoch timestamp выключения')
1155 uptime_seconds: float = Field(default=0.0, description='Дл
ительность сессии в секундах')
1156 uptime_human: str = Field(default='', description='Человек
очитаемый аптайм (например: 14ч 35м 31с)')
1157 shutdown_type: str = Field(default='Active', description='
Тип завершения: Active, Restart, Shutdown, Unexpected, Sleep/W
ake')
1158 initiator: Optional[str] = Field(default=None, description
='Инициатор выключения (DOMAIN\\User или SYSTEM)')
1159 process: Optional[str] = Field(default=None, description='
Процесс инициатор (shutdown.exe, explorer.exe, etc.)')
1160 reason: Optional[str] = Field(default=None, description='Т
екст причины выключения')
1161 reason_code: Optional[str] = Field(default=None, descripti
on='Шестнадцатеричный код причины')
1162 comment: Optional[str] = Field(default=None, description='
Комментарий')
1163 clean_shutdown: bool = Field(default=True, description='Шт
атное корректное выключение')
1164 unexpected_shutdown: bool = Field(default=False, descripti
on='Внезапное выключение / потеря питания')
1165 bugcheck: Optional[str] = Field(default=None, description=
'Информация о BSOD / BugCheck')
1166 boot_event_id: int = Field(default=12, description='Event
ID события загрузки (12, 6005, etc.)')
1167 shutdown_event_id: Optional[int] = Field(default=None, des
cription='Event ID события выключения (1074, 13, 6006, 41, 600
8)')
1168 initiator_chain: List[str] = Field(default_factory=list, d
escription='Цепочка корреляции инициатора (User -> WMI -> wmi
rvse -> shutdown)')
1169 events: List[Dict[str, Any]] = Field(default_factory=list,
description='Список связанных событий в этой сессии')
1170 created_at: float = Field(default_factory=lambda: datetime
.now(timezone.utc).timestamp(), description='Время создания за
писи')
```



```

1171
1172
1173 class PowerLifecycleSummary(BaseModel):
1174     """Сводные метрики жизненного цикла питания и аптайма."""
1175     current_boot_time: str = Field(default='', description='Вр
        емя текущей загрузки ОС')
1176     current_uptime_seconds: float = Field(default=0.0, descrip
        tion='Текущий аптайм в секундах')
1177     current_uptime_human: str = Field(default='', description=
        'Текущий аптайм в строковом формате')
1178     total_sessions_count: int = Field(default=0, description='
        Всего зафиксировано сессий')
1179     clean_shutdowns_count: int = Field(default=0, description=
        'Штатных выключений/перезагрузок')
1180     unexpected_shutdowns_count: int = Field(default=0, descrip
        tion='Внезапных сбоев питания / падений')
1181     bsod_count: int = Field(default=0, description='Количество
        критических BSOD BugCheck')
1182     last_shutdown_type: Optional[str] = Field(default=None, de
        scription='Тип последнего выключения')
1183     last_initiator: Optional[str] = Field(default=None, descri
        ption='Инициатор последнего выключения')
1184     last_reason: Optional[str] = Field(default=None, descripti
        on='Причина последнего выключения')
1185     sessions: List[PowerSessionRecord] = Field(default_factory
        =list, description='Список последних сессий питания')
1186
1187
1188 # =====
        =====
1189 # Process Intelligence & Deep Dive Data Models (PID Recycling
        Protection)
1190 # =====
        =====
1191
1192 class ProcessDefinitionRecord(BaseModel):
1193     """Паспорт программы (Process Definition) для Process Inte
        lligence."""

```

```
1194     definition_id: Optional[int] = Field(default=None, descrip
1195 |     tion='Идентификатор описания')
1196     name: str = Field(..., description='Имя исполняемого файла
1197 |     программы')
1198     executable_path: str = Field(..., description='Полный уник
1199 |     альный путь к исполняемому файлу')
1200     sha256: Optional[str] = Field(default=None, description='S
1201 |     HA256 хеш исполняемого файла')
1202     company_name: Optional[str] = Field(default=None, descript
1203 |     ion='Разработчик/компания')
1204     file_description: Optional[str] = Field(default=None, desc
1205 |     ription='Описание из ресурсов файла')
1206     icon_base64: Optional[str] = Field(default=None, descripti
1207 |     on='Base64 иконка процесса')
1208     first_seen: str = Field(default='', description='Время пер
1209 |     вого обнаружения программы')
1210     last_seen: str = Field(default='', description='Время посл
1211 |     еднего запуска программы')
1212
1213
1214 class ProcessInstanceRecord(BaseModel):
1215     """Единичный уникальный инстанс процесса (Process Instance
1216 |     ) с защитой от PID Recycling."""
1217     instance_id: Optional[int] = Field(default=None, descripti
1218 |     on='Уникальный первичный ключ инстанса')
1219     definition_id: int = Field(..., description='Ссылка на пас
1220 |     порт программы')
1221     pid: int = Field(..., description='Process ID операционной
1222 |     системы')
1223     parent_instance_id: Optional[int] = Field(default=None, de
1224 |     scription='Уникальный instance_id родительского процесса')
1225     name: str = Field(..., description='Имя процесса')
1226     executable_path: str = Field(..., description='Полный путь
1227 |     к исполняемому файлу')
1228     command_line: Optional[str] = Field(default=None, descript
1229 |     ion='Полная командная строка запуска')
1230     start_time: str = Field(..., description='Время запуска пр
1231 |     оцесса ядра ОС (ISO-8601)')
```

```
1215     exit_time: Optional[str] = Field(default=None, description
    = 'Время завершения процесса')
1216     exit_code: Optional[int] = Field(default=None, description
    = 'Код завершения процесса')
1217     session_id: int = Field(default=1, description='Идентифика
    тор пользовательской сессии Windows')
1218     user_name: Optional[str] = Field(default=None, description
    = 'Имя пользователя учетной записи')
1219     integrity_level: Optional[str] = Field(default=None, descr
    iption='Уровень целостности (Low, Medium, High, System)')
1220     status: str = Field(default='RUNNING', description='Статус
    инстанса: RUNNING, EXITED, SUSPENDED')
1221     cpu_percent: float = Field(default=0.0, description='Текущ
    ая/последняя нагрузка CPU %')
1222     memory_mb: float = Field(default=0.0, description='Текущая
    /последняя память Working Set в МБ')
1223     uptime_seconds: float = Field(default=0.0, description='Вр
    емя работы инстанса в секундах')
1224     ppid: Optional[int] = Field(default=None, description='PID
    родителя на момент запуска')
1225     icon_base64: Optional[str] = Field(default=None, descripti
    on='Base64 иконка процесса')
1226
1227
1228 class ProcessSampleRecord(BaseModel):
1229     """Временной срез телеметрии ресурсов инстанса процесса."""
    "
1230     sample_id: Optional[int] = Field(default=None, description
    = 'Идентификатор среза')
1231     instance_id: int = Field(..., description='Уникальный inst
    ance_id процесса')
1232     timestamp: str = Field(..., description='ISO-8601 временна
    я метка')
1233     created_at: float = Field(..., description='Epoch timestam
    p среза')
1234     cpu_percent: float = Field(default=0.0, description='Сумма
    рный % загрузки CPU')
1235     cpu_user_time: float = Field(default=0.0, description='Use
```

```

r mode CPU time')
1236     cpu_kernel_time: float = Field(default=0.0, description='K
kernel mode CPU time')
1237     thread_count: int = Field(default=1, description='Количес
во потоков процесса')
1238     working_set_mb: float = Field(default=0.0, description='Фи
зическая память Working Set в МБ')
1239     private_bytes_mb: float = Field(default=0.0, description='
Выделенная память Private Bytes в МБ')
1240     page_faults_sec: float = Field(default=0.0, description='O
шибки страниц (Page Faults / sec)')
1241     gpu_load_percent: float = Field(default=0.0, description='
Нагрузка GPU в %')
1242     gpu_vram_mb: float = Field(default=0.0, description='Испол
ьзование видеопамати VRAM в МБ')
1243     disk_read_bytes_sec: float = Field(default=0.0, descriptio
n='Скорость чтения с диска в байт/с')
1244     disk_write_bytes_sec: float = Field(default=0.0, descripti
on='Скорость записи на диск в байт/с')
1245     disk_iops: float = Field(default=0.0, description='Количес
тво операций ввода-вывода (IOPS)')
1246     handle_count: int = Field(default=0, description='Количес
то открытых дескрипторов (Handles)')
1247     gdi_objects: int = Field(default=0, description='Количеств
о GDI объектов (кисти, шрифты, битмапы)')
1248     user_objects: int = Field(default=0, description='Количес
то USER объектов (окна, меню)')
1249     sockets_count: int = Field(default=0, description='Количес
то активных сетевых сокетов')
1250     sockets_json: Optional[str] = Field(default=None, descripti
on='JSON список сетевых подключений')
1251
1252
1253 class ProcessFileEventRecord(BaseModel):
1254     """Событие файловой операции процесса."""
1255     id: Optional[int] = Field(default=None, description='Идент
ификатор события')
1256     instance_id: Optional[int] = Field(default=None, descripti

```

```

on='Instance ID процесса')
1257     timestamp: str = Field(..., description='Время события')
1258     action: str = Field(..., description='Действие: CREATE, MO
DIFY, DELETE, RENAME')
1259     target_folder: Optional[str] = Field(default=None, descrip
tion='Целевая папка')
1260     file_path: str = Field(..., description='Полный путь к фай
лу')
1261     bytes_count: int = Field(default=0, description='Объем зат
ронутых данных в байтах')
1262     created_at: float = Field(..., description='Epoch timestam
p')
1263
1264
1265 class ProcessSocketRecord(BaseModel):
1266     """Запись о сетевом сокете процесса."""
1267     protocol: str = Field(default='TCP', description='Протокол
: TCP, UDP')
1268     local_address: str = Field(..., description='Локальный IP:
порт')
1269     remote_address: str = Field(..., description='Удаленный IP
:порт')
1270     status: str = Field(default='ESTABLISHED', description='Ст
атус сокета: ESTABLISHED, LISTEN, CLOSE_WAIT, etc.')
1271     sent_kbs: float = Field(default=0.0, description='Скорость
отправки (КБ/с)')
1272     recv_kbs: float = Field(default=0.0, description='Скорость
приема (КБ/с)')
1273     dns_name: Optional[str] = Field(default=None, description=
'DNS имя удаленного сервера')
1274     country: Optional[str] = Field(default=None, description='
Страна назначения')
1275
1276
1277 class ProcessLineageNode(BaseModel):
1278     """Узел интерактивного графа происхождения процесса (Proce
ss Lineage)."""
1279     instance_id: int = Field(..., description='Уникальный inst

```

```

    ance_id')
1280     pid: int = Field(..., description='PID процесса')
1281     name: str = Field(..., description='Имя процесса')
1282     status: str = Field(default='RUNNING', description='Статус
инстанса')
1283     start_time: str = Field(..., description='Время старта')
1284     executable_path: str = Field(..., description='Путь к испо
лняемому файлу')
1285     is_current: bool = Field(default=False, description='Флаг
текущего выбранного процесса')
1286     parent: Optional[Dict[str, Any]] = Field(default=None, des
cription='Информация о родительском инстансе')
1287     children: List[Dict[str, Any]] = Field(default_factory=lis
t, description='Дочерние инстансы')
1288
1289
1290 class ProcessSafeOpsRequest(BaseModel):
1291     """Запрос на безопасное администрирование инстанса процесс
а."""
1292     action: str = Field(..., description='Действие: kill, susp
end, resume, dump, priority')
1293     priority_class: Optional[str] = Field(default=None, descri
ption='Приоритет: Idle, BelowNormal, Normal, AboveNormal, High
, Realtime')
1294     reason: Optional[str] = Field(default='Manual admin action
from UI', description='Причина выполнения действия')
1295     dump_folder: Optional[str] = Field(default=None, descripti
on='Пользовательская папка для дампа')
1296
1297
1298 class ProcessSafeOpsResponse(BaseModel):
1299     """Ответ на безопасное действие над процессом."""
1300     instance_id: int = Field(..., description='Идентификатор и
нстанса')
1301     pid: int = Field(..., description='PID процесса')
1302     action: str = Field(..., description='Выполненное действие
')
1303     success: bool = Field(..., description='Успешность операци

```

```

и')
1304     message: str = Field(..., description='Сообщение о результ
ате')
1305     details: Dict[str, Any] = Field(default_factory=dict, desc
ription='Дополнительные метаданные')
1306
1307
1308 # =====
=====
1309 # Per-PID Telemetry Engine Models (T3 TZ_PER_PID_RESOURCE_TELE
METRY_COMPLETION)
1310 # =====
=====
1311
1312 class CpuDetail(BaseModel):
1313     """Детализация процессора для попроцессной телеметрии."""
1314     percent: float = Field(default=0.0, description='Загрузка
процессора в процентах')
1315     user_time_ms: int = Field(default=0, description='Время вы
полнения в пользовательском режиме (мс)')
1316     kernel_time_ms: int = Field(default=0, description='Время
выполнения в режиме ядра (мс)')
1317     thread_count: int = Field(default=1, description='Количес
тво активных потоков')
1318
1319
1320 class MemoryDetail(BaseModel):
1321     """Детализация оперативной памяти для попроцессной телемет
рии."""
1322     working_set_mb: float = Field(default=0.0, description='Па
бочий набор памяти (Working Set) в МБ')
1323     private_bytes_mb: float = Field(default=0.0, description='
Приватная память (Private Bytes) в МБ')
1324     page_faults: int = Field(default=0, description='Количеств
о ошибок страниц памяти (Page Faults)')
1325
1326
1327 class GpuDetail(BaseModel):

```

```
1328     """Детализация GPU и видеопамяти для попроцессной телеметр
ии."""
1329     vram_dedicated_mb: float = Field(default=0.0, description=
'Выделенная видеопамять (VRAM) в МБ')
1330     utilization_percent: float = Field(default=0.0, descriptio
n='Утилизация GPU движка в процентах')
1331
1332
1333 class IoDetail(BaseModel):
1334     """Детализация дискового ввода-вывода и дескрипторов."""
1335     read_bytes_sec: int = Field(default=0, description='Скорос
ть чтения с диска (байт/с)')
1336     write_bytes_sec: int = Field(default=0, description='Скоро
сть записи на диск (байт/с)')
1337     handles_count: int = Field(default=0, description='Количес
тво открытых дескрипторов')
1338     read_ops_total: int = Field(default=0, description='Всего
операций чтения')
1339     write_ops_total: int = Field(default=0, description='Всего
операций записи')
1340
1341
1342 class NetworkDetail(BaseModel):
1343     """Детализация сетевой активности процесса."""
1344     bytes_sent_sec: int = Field(default=0, description='Скорос
ть отправки по сети (байт/с)')
1345     bytes_recv_sec: int = Field(default=0, description='Скорос
ть приема по сети (байт/с)')
1346     active_sockets: int = Field(default=0, description='Количе
ство активных сетевых сокетов')
1347
1348
1349 class ProcessTelemetryResponse(BaseModel):
1350     """Строгий канонический REST API контракт телеметрии проце
сса по PID."""
1351     pid: int = Field(..., description='Идентификатор процесса
(PID)')
1352     process_name: str = Field(..., description='Имя исполняемо
```



```

    го файла')
1353     executable_path: Optional[str] = Field(default=None, description='Полный путь к исполняемому файлу')
1354     cpu: CpuDetail = Field(default_factory=CpuDetail, description='Метрики процессора')
1355     memory: MemoryDetail = Field(default_factory=MemoryDetail, description='Метрики памяти')
1356     gpu: GpuDetail = Field(default_factory=GpuDetail, description='Метрики GPU/VRAM')
1357     io: IoDetail = Field(default_factory=IoDetail, description='Метрики дискового I/O и дескрипторов')
1358     network: NetworkDetail = Field(default_factory=NetworkDetail, description='Метрики сети')
1359     timestamp: str = Field(default='', description='Время фиксации снимка')
1360
1361
1362 class ProcessFileEventItem(BaseModel):
1363     """Запись события файловой активности процесса."""
1364     event_id: int = Field(..., description='Уникальный ID события')
1365     pid: int = Field(..., description='Идентификатор процесса (PID)')
1366     process_name: str = Field(..., description='Имя процесса')
1367     action_type: str = Field(..., description='Тип действия (CREATE, MODIFY, DELETE, RENAME)')
1368     target_directory: str = Field(..., description='Отслеживаемая целевая директория')
1369     file_path: str = Field(..., description='Полный путь к затронутому файлу')
1370     bytes_affected: int = Field(default=0, description='Объем затронутых байт')
1371     timestamp: str = Field(..., description='Время регистрации события')
1372
1373
1374 class TrackedDirectoryRequest(BaseModel):
1375     """Запрос на управление отслеживаемой директорией."""

```

```
1376     action: str = Field(..., description="Действие: 'add' или  
1377     'remove'")  
1378     directory_path: str = Field(..., description='Абсолютный п  
1379     уть к директории')  
1380 class TrackedDirectoryResponse(BaseModel):  
1381     """Ответ на операцию с отслеживаемой директорией."""  
1382     status: str = Field(default='ok', description='Статус опер  
1383     ации')  
1384     action: str = Field(..., description='Выполненное действие  
1385     ')  
1386     directory_path: str = Field(..., description='Путь к дирек  
1387     тории')  
1388     tracked_directories: List[str] = Field(default_factory=lis  
1389     t, description='Актуальный список отслеживаемых директорий')  
1390 class ProcessPidSnapshot(BaseModel):  
1391     """Каноническая модель единого снимка ресурсов процесса по  
1392     PID для SQLite."""  
1393     snapshot_id: Optional[int] = Field(default=None, descripti  
1394     on='ID снимка')  
1395     pid: int = Field(..., description='PID процесса')  
1396     process_name: str = Field(..., description='Имя процесса')  
1397     executable_path: Optional[str] = Field(default=None, descr  
1398     iption='Путь к исполняемому файлу')  
1399     cpu_percent: float = Field(default=0.0, description='Загру  
1400    зка CPU %')  
1401     user_time_ms: int = Field(default=0, description='User tim  
1402     e ms')  
1403     kernel_time_ms: int = Field(default=0, description='Kernel  
1404     time ms')  
1405     thread_count: int = Field(default=1, description='Количес  
1406     тво потоков')  
1407     working_set_bytes: int = Field(default=0, description='Wor  
1408     king set байт')  
1409     private_bytes: int = Field(default=0, description='Private
```

```
bytes')
1400     page_faults_count: int = Field(default=0, description='Pag
e faults')
1401     gpu_vram_bytes: int = Field(default=0, description='VRAM б
айт')
1402     gpu_utilization: float = Field(default=0.0, description='G
PU load %')
1403     read_bytes_total: int = Field(default=0, description='Всег
о прочитано байт')
1404     write_bytes_total: int = Field(default=0, description='Всег
о записано байт')
1405     read_ops_total: int = Field(default=0, description='Всего
операций чтения')
1406     write_ops_total: int = Field(default=0, description='Всего
операций записи')
1407     net_bytes_sent_total: int = Field(default=0, description='
Всего отправлено байт по сети')
1408     net_bytes_recv_total: int = Field(default=0, description='
Всего получено байт по сети')
1409     handle_count: int = Field(default=0, description='Дескрипт
оры')
1410     gdi_objects: int = Field(default=0, description='GDI объек
ты')
1411     user_objects: int = Field(default=0, description='USER объ
екты')
1412     timestamp: str = Field(default='', description='Время сним
ка')
1413
1414
1415
1416
1417
1418
1419
1420
```

power_lifecycle.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - Power L
  ifecycle
4 # =====
5 # Description:
6 #   Движок сбора, разбора и корреляции событий жизненного цикл
  а питания Windows
7 #   (Boot, Clean Shutdown, Planned Restart, Unexpected Power L
  oss, BSOD, Sleep/Wake)
8 #   и автоматической реконструкции сессий питания (Power Sessi
  ons).
9 #
10 # Usage Examples:
11 #   Python API:
12 #       from apps.windows.telemetry.power_lifecycle import Power
  LifecycleEngine
13 #
14 #       engine = PowerLifecycleEngine()
15 #       summary = engine.scan_and_reconstruct(hours=720)
16 #       sessions = engine.get_sessions(limit=20)
17 #
18 # File: power_lifecycle.py
19 # Project: ai-breadboard
20 # Package: apps.windows.telemetry
21 # Author: hypo69
22 # Copyright: © 2026 hypo69
23 # Updated: 2026-10-08 02:51:00
24 # =====
25
26 from __future__ import annotations
27 """Движок анализа событий питания и реконструкции сессий жизне
  нного цикла Windows."""
```

```
28 |
29 | import json
30 | import os
31 | import re
32 | import socket
33 | import subprocess
34 | import time
35 | from datetime import datetime, timezone, timedelta
36 | from typing import Any, Dict, List, Optional, Tuple
37 |
38 | try:
39 |     from logger import logger
40 | except ImportError:
41 |     import logging
42 |     logger = logging.getLogger(__name__)
43 |
44 | from apps.windows.telemetry.win32_fffi.wevtapi import WevtAPI
45 | from apps.windows.telemetry.win32_fffi.error_decoder import decode_bugcheck_code, format_system_error
46 | from apps.windows.telemetry.models import (
47 |     PowerEventRecord,
48 |     PowerSessionRecord,
49 |     PowerLifecycleSummary,
50 | )
51 | from apps.windows.telemetry.sqlite import TelemetryStorage
52 |
53 |
54 | # Целевые Event ID для жизненного цикла питания и выключений
55 | POWER_LIFECYCLE_EVENT_IDS = {
56 |     1074: ('User32', 'Плановое выключение / перезагрузка иници
ирована пользователем или процессом'),
57 |     13: ('Microsoft-Windows-Kernel-General', 'Операционная сис
тема начала завершение работы'),
58 |     6006: ('EventLog', 'Служба Event Log корректно остановлена
(штатное чистое выключение)'),
59 |     6008: ('EventLog', 'Предыдущее завершение работы системы б
ыло неожиданным (Unexpected Shutdown)'),
60 |     41: ('Microsoft-Windows-Kernel-Power', 'КРИТИЧЕСКИЙ СБОЙ:
```

```

Перезагрузка без предварительного чистого выключения'),
61     12: ('Microsoft-Windows-Kernel-General', 'Операционная сис
тема запущена (Boot)'),
62     6005: ('EventLog', 'Служба Event Log запущена (ОС готова к
работе)'),
63     6009: ('EventLog', 'Windows записала информацию о версии я
дра при загрузке'),
64     1001: ('WER-SystemErrorReporting', 'Аварийный сбой ядра /
BSOD (BugCheck)'),
65     19: ('Microsoft-Windows-WindowsUpdateClient', 'Успешно уст
ановлено обновление Windows Update'),
66     42: ('Microsoft-Windows-Kernel-Power', 'Переход системы в
спящий режим (Sleep)'),
67     107: ('Microsoft-Windows-Kernel-Power', 'Пробуждение систе
мы из спящего режима (Resume/Wake)'),
68 }
69
70
71 def format_uptime_human(seconds: float) -> str:
72     """Форматирует секунды аптайма в человекочитаемую строку н
а русском языке.
73
74     Args:
75         seconds: Длительность в секундах.
76
77     Returns:
78         str: Строка вида '3д 14ч 35м 31с' или '14ч 35м 31с'.
79     """
80     total_sec = max(0, int(seconds))
81     days = total_sec // 86400
82     hours = (total_sec % 86400) // 3600
83     minutes = (total_sec % 3600) // 60
84     sec = total_sec % 60
85
86     parts = []
87     if days > 0:
88         parts.append(f"{days}д")
89     if hours > 0 or days > 0:

```

```

90         parts.append(f"{hours}ч")
91     if minutes > 0 or hours > 0 or days > 0:
92         parts.append(f"{minutes}м")
93     parts.append(f"{sec}с")
94
95     return ' '.join(parts)
96
97
98 class PowerLifecycleEngine:
99     """Движок глубокого анализа и реконструкции жизненного цик
100 ла питания ОС."""
101
102     def __init__(
103         self,
104         storage: Optional[TelemetryStorage] = None,
105         wevtapi: Optional[WevtAPI] = None,
106     ) -> None:
107         """Инициализация движка жизненного цикла питания.
108
109         Args:
110             storage: Экземпляр TelemetryStorage SQLite хранили
111 ща.
112             wevtapi: Низкоуровневая Ctypes-обертка WevtAPI.
113         """
114         self.storage = storage or TelemetryStorage.get_instanc
115 e()
116         self.wevtapi = wevtapi or WevtAPI()
117
118     def get_current_system_boot(self) -> Tuple[datetime, float
119 ]:
120         """Определяет точное время последней загрузки и текущи
121 й аптайм.
122
123         Returns:
124             Tuple[datetime, float]: (boot_time UTC, uptime в с
125 екундах).
126         """
127         try:

```

```

122         import psutil
123         b_epoch = psutil.boot_time()
124         boot_dt = datetime.fromtimestamp(b_epoch, tz=timezone.
one.utc)
125         uptime_sec = max(0.0, time.time() - b_epoch)
126         return boot_dt, uptime_sec
127     except Exception:
128         boot_dt = datetime.now(timezone.utc)
129         return boot_dt, 0.0
130
131     def parse_event_1074(self, ev: Dict[str, Any]) -> Dict[str
, Any]:
132         """Глубокий разбор события User32 1074 (инициатор, про
цесс, тип, причина).
133
134         Args:
135             ev: Словарь события из WeventAPI.
136
137         Returns:
138             Dict[str, Any]: Извлеченные нормализованные параме
тры.
139         """
140         msg = ev.get('message', '') or ev.get('formatted_messa
ge', '')
141         data = ev.get('event_data', {}) or {}
142
143         # 1. Попытка извлечения из XML Data (param1 ... param7
)
144         process = data.get('param1') or data.get('Process') or
''
145         computer = data.get('param2') or data.get('Computer')
or ev.get('computer', '')
146         reason = data.get('param3') or data.get('Reason') or '
'
147         reason_code = data.get('param4') or data.get('ReasonCo
de') or ''
148         shutdown_type = data.get('param5') or data.get('Shutdo
wnType') or ''

```



```

149 |         comment = data.get('param6') or data.get('Comment') or
150 |         ''
151 |
152 |         # 2. Если параметры пустые, применяем регулярные выраж
153 |         ения к сообщению
154 |         if not process and msg:
155 |             # Process regex
156 |             m_proc = re.search(r'(:process|процесс|процессом)
157 | \s+([^\s]+)', msg, re.IGNORECASE)
158 |             if m_proc:
159 |                 process = m_proc.group(1).rstrip('.,;:')
160 |
161 |             # User regex
162 |             m_user = re.search(r'(:user|пользователя|пользо
163 | ватель|имени)\s+([^\s]+)', msg, re.IGNORECASE)
164 |             if m_user:
165 |                 user = m_user.group(1).rstrip('.,;:')
166 |
167 |             # Reason Code regex
168 |             m_code = re.search(r'(:Reason Code|код причины|ко
169 | дом причины)[:\s]+(0x[0-9a-fA-F]+)', msg, re.IGNORECASE)
170 |             if m_code:
171 |                 reason_code = m_code.group(1)
172 |
173 |             # Shutdown Type regex
174 |             m_type = re.search(r'(:Shutdown Type|тип выключен
175 | ия|тип завершения)[:\s]+([a-zA-Za-яA-Я0-9_-]+)', msg, re.IGNOR
176 | ECASE)
177 |             if m_type:
178 |                 shutdown_type = m_type.group(1)
179 |
180 |             # Reason text regex
181 |             m_reason = re.search(r'(:reason|причине|причина)[
182 | :\s]+(.*?)(:Reason Code|код причины|$)', msg, re.IGNORECASE)
183 |             if m_reason:
184 |                 reason = m_reason.group(1).strip()

```

```

179         # Comment regex
180         m_comment = re.search(r'(?::comment|комментарий)[:\s]+(.*?)$', msg, re.IGNORECASE)
181         if m_comment:
182             comment = m_comment.group(1).strip()
183
184         # Нормализация типа выключения
185         norm_type = 'Shutdown'
186         st_lower = str(shutdown_type).lower()
187         if 'restart' in st_lower or 'перезагр' in st_lower:
188             norm_type = 'Restart'
189         elif 'power off' in st_lower or 'выключ' in st_lower or 'poweroff' in st_lower:
190             norm_type = 'PowerOff'
191         elif 'hybrid' in st_lower or 'fast' in st_lower:
192             norm_type = 'HybridShutdown'
193
194         domain = ''
195         if '\\' in user:
196             domain, user_name = user.split('\\', 1)
197         else:
198             user_name = user
199
200         # Корреляция цепочки инициатора
201         initiator_chain: List[str] = []
202         proc_lower = process.lower()
203         if 'wmiprvse' in proc_lower:
204             initiator_chain = [
205                 f"Пользователь/Скрипт ({user or 'WMI Client'})",
206                 "WMI Remote/Local Call",
207                 "wmiprvse.exe",
208                 "Windows ExitWindowsEx API",
209             ]
210         elif 'shutdown.exe' in proc_lower:
211             initiator_chain = [
212                 f"Пользователь ({user or 'Console'})",
213                 "Командная строка / Task Scheduler",

```

```

214         "shutdown.exe",
215         "InitiateSystemShutdownEx API",
216     ]
217     elif 'explorer.exe' in proc_lower:
218         initiator_chain = [
219             f"Пользователь ({user or 'Interactive'})",
220             "Меню Пуск / Win+X / Alt+F4",
221             "explorer.exe",
222             "Shell Shutdown API",
223         ]
224     elif 'svchost.exe' in proc_lower:
225         initiator_chain = [
226             "Служба обновления Windows / System Service",
227             "svchost.exe",
228             "Automatic Maintenance Restart",
229         ]
230     elif process:
231         initiator_chain = [
232             f"Пользователь ({user or 'User'})",
233             f"Процесс ({process})",
234             "Shutdown API",
235         ]
236
237     return {
238         'process': process,
239         'computer': computer,
240         'reason': reason or 'Плановое завершение работы',
241         'reason_code': reason_code,
242         'shutdown_type': norm_type,
243         'comment': comment,
244         'user': user,
245         'user_name': user_name,
246         'domain': domain,
247         'initiator_chain': initiator_chain,
248     }
249
250     def read_raw_power_events(self, hours: int = 720, limit: i
nt = 500) -> List[Dict[str, Any]]:

```

```

251 |         """Извлекает сырые события питания из журнала System W
      | indows.
252 |
253 |     Args:
254 |         hours: Глубина выборки в часах.
255 |         limit: Максимальное количество событий каждого тип
      | а.
256 |
257 |     Returns:
258 |         List[Dict[str, Any]]: Список событий, упорядоченны
      | х хронологически.
259 |         """
260 |         target_ids = list(POWER_LIFECYCLE_EVENT_IDS.keys())
261 |         raw_events = self.wevtapi.read_events(
262 |             channel='System',
263 |             limit=limit,
264 |             hours=hours,
265 |             event_id=target_ids,
266 |             format_message=True,
267 |         )
268 |
269 |         normalized: List[Dict[str, Any]] = []
270 |         for ev in raw_events:
271 |             eid = ev.get('event_id', 0)
272 |             if eid not in POWER_LIFECYCLE_EVENT_IDS:
273 |                 continue
274 |
275 |             provider_info = POWER_LIFECYCLE_EVENT_IDS.get(eid,
      | ('System', ''))
276 |             provider = ev.get('provider') or provider_info[0]
277 |             ts = ev.get('timestamp') or ''
278 |             raw_xml = ev.get('raw_data') or ''
279 |             msg = ev.get('message') or ev.get('formatted_messa
      | ge') or provider_info[1]
280 |
281 |             event_type = 'event'
282 |             shutdown_type = None
283 |             user = None

```

```

284 |         domain = None
285 |         process = None
286 |         pid = ev.get('process_id')
287 |         reason = None
288 |         reason_code = None
289 |         comment = None
290 |         unexpected = False
291 |         bugcheck_code = None
292 |         bugcheck_params = None
293 |         details: Dict[str, Any] = {'message': msg}
294 |
295 |         if eid == 1074:
296 |             parsed_1074 = self.parse_event_1074(ev)
297 |             event_type = 'shutdown_planned'
298 |             shutdown_type = parsed_1074['shutdown_type']
299 |             user = parsed_1074['user']
300 |             domain = parsed_1074['domain']
301 |             process = parsed_1074['process']
302 |             reason = parsed_1074['reason']
303 |             reason_code = parsed_1074['reason_code']
304 |             comment = parsed_1074['comment']
305 |             details['initiator_chain'] = parsed_1074['init
iator_chain']
306 |             details['computer'] = parsed_1074['computer']
307 |
308 |         elif eid == 13:
309 |             event_type = 'shutdown_clean'
310 |             shutdown_type = 'Shutdown'
311 |             reason = 'Операционная система завершает работ
у ядра'
312 |
313 |         elif eid == 6006:
314 |             event_type = 'shutdown_clean'
315 |             shutdown_type = 'Shutdown'
316 |             reason = 'Служба Event Log корректно остановле
на'
317 |
318 |         elif eid in (41, 6008):

```

```

319 |         event_type = 'shutdown_unexpected'
320 |         unexpected = True
321 |         shutdown_type = 'Unexpected'
322 |         reason = 'Внезапное завершение работы (потеря
питания / зависание / сбой)'
323 |         if eid == 41:
324 |             ev_data = ev.get('event_data', {})
325 |             bc = ev_data.get('BugcheckCode')
326 |             if bc and str(bc) != '0':
327 |                 try:
328 |                     bc_int = int(bc)
329 |                     bugcheck_code = hex(bc_int)
330 |                     decoded = decode_bugcheck_code(bc_
int)
331 |                     details['bugcheck_info'] = decoded
332 |                     details['bugcheck_symbol'] = decod
ed.get('symbol', '')
333 |                 except Exception:
334 |                     bugcheck_code = str(bc)
335 |                     details['power_button_timestamp'] = ev_dat
a.get('PowerButtonTimestamp', '0')
336 |
337 |             elif eid in (12, 6005, 6009):
338 |                 event_type = 'boot'
339 |                 reason = 'Запуск операционной системы и систем
ных служб'
340 |
341 |             elif eid == 1001:
342 |                 event_type = 'bsod'
343 |                 reason = 'Критический сбой операционной систем
ы (BugCheck / BSOD)'
344 |                 ev_data = ev.get('event_data', {})
345 |                 bugcheck_code = ev_data.get('param1') or ev_da
ta.get('BugcheckCode')
346 |                 details['bugcheck_params'] = [ev_data.get(f'pa
ram{i}') for i in range(1, 6)]
347 |
348 |             elif eid == 19:

```

```

349         event_type = 'update_install'
350         reason = 'Успешно установлено обновление Windo
ws Update'
351
352     elif eid == 42:
353         event_type = 'sleep'
354         reason = 'Переход системы в спящий режим'
355
356     elif eid == 107:
357         event_type = 'wake'
358         reason = 'Выход системы из спящего режима'
359
360     record = {
361         'event_id': eid,
362         'provider': provider,
363         'channel': ev.get('channel', 'System'),
364         'timestamp': ts,
365         'created_at': self._parse_iso_to_epoch(ts),
366         'event_type': event_type,
367         'shutdown_type': shutdown_type,
368         'user': user,
369         'domain': domain,
370         'process': process,
371         'process_id': pid,
372         'reason': reason,
373         'reason_code': reason_code,
374         'comment': comment,
375         'unexpected': unexpected,
376         'bugcheck_code': bugcheck_code,
377         'bugcheck_params_json': json.dumps(bugcheck_pa
rams, ensure_ascii=False) if bugcheck_params else None,
378         'details_json': json.dumps(details, ensure_asc
ii=False),
379         'raw_xml': raw_xml,
380     }
381     normalized.append(record)
382
383     # Сортировка от старых к новым для корректной реконстр

```

укции

```
384         normalized.sort(key=lambda x: x['created_at'])
385         return normalized
386
387     def _parse_iso_to_epoch(self, ts_str: str) -> float:
388         """Преобразует строку даты/времени в unix epoch timestamp."""
389         if not ts_str:
390             return time.time()
391         for fmt in ('%Y-%m-%d %H:%M:%S', '%Y-%m-%dT%H:%M:%S',
392                    '%Y-%m-%d %H:%M:%S.%f', '%Y-%m-%dT%H:%M:%S.%fZ'):
393             try:
394                 dt = datetime.strptime(ts_str.split('.')[0].replace('T', ' '), '%Y-%m-%d %H:%M:%S')
395                 return dt.replace(tzinfo=timezone.utc).timestamp()
396             except Exception:
397                 pass
398         return time.time()
399
400     def reconstruct_power_sessions(self, events: List[Dict[str, Any]]) -> List[PowerSessionRecord]:
401         """Реконструирует цепочки Boot -> Power Session -> Shutdown/Crash.
402
403         Args:
404             events: Хронологически отсортированный список нормализованных событий.
405
406         Returns:
407             List[PowerSessionRecord]: Список собранных сессий питания.
408
409         """
410         sessions: List[PowerSessionRecord] = []
411         current_session: Optional[Dict[str, Any]] = None
412
413         for ev in events:
414             eid = ev.get('event_id', 0)
```



```

413         e_type = ev.get('event_type', '')
414         ts = ev.get('timestamp', '')
415         epoch = ev.get('created_at', 0.0)
416
417         # Событие запуска ОС (Boot)
418         if eid in (12, 6005, 6009) or e_type == 'boot':
419             # Если уже была открыта сессия без штатного за-
420             # вершения, закрываем её как Unexpected
421             if current_session:
422                 s_id = current_session['session_id']
423                 uptime_sec = max(0.0, epoch - current_sess
424                 ion['boot_timestamp'])
425                 current_session['shutdown_time'] = ts
426                 current_session['shutdown_timestamp'] = ep
427                 och
428                 current_session['uptime_seconds'] = uptime
429                 _sec
430                 current_session['uptime_human'] = format_u
431                 ptime_human(uptime_sec)
432                 if not current_session.get('shutdown_type'
433                 ) or current_session.get('shutdown_type') == 'Active':
434                     current_session['shutdown_type'] = 'Un
435                     expected'
436                     current_session['unexpected_shutdown']
437                     = True
438                     current_session['clean_shutdown'] = Fa
439                     lse
440                     current_session['reason'] = current_se
441                     ssion['reason'] or 'Внезапный перезапуск без выключения (Kerne
442                     l-Power 41 / 6008)'
443                     sessions.append(PowerSessionRecord(**curre
444                     nt_session))
445
446             # Открываем новую сессию
447             session_id = f"PS-{ts.replace(':', '').replace
448             ('-', '').replace(' ', '-')}"
449             current_session = {
450                 'session_id': session_id,

```

```

438         'boot_time': ts,
439         'boot_timestamp': epoch,
440         'shutdown_time': None,
441         'shutdown_timestamp': None,
442         'uptime_seconds': 0.0,
443         'uptime_human': '',
444         'shutdown_type': 'Active',
445         'initiator': None,
446         'process': None,
447         'reason': None,
448         'reason_code': None,
449         'comment': None,
450         'clean_shutdown': True,
451         'unexpected_shutdown': False,
452         'bugcheck': None,
453         'boot_event_id': eid,
454         'shutdown_event_id': None,
455         'initiator_chain': [],
456         'events': [ev],
457         'created_at': epoch,
458     }
459
460     elif current_session:
461         current_session['events'].append(ev)
462
463         # Завершение сессии (1074, 13, 6006)
464         if eid == 1074:
465             current_session['shutdown_event_id'] = 107
466             current_session['shutdown_type'] = ev.get(
467                 'shutdown_type') or 'Restart'
468             current_session['initiator'] = ev.get('use
469             r')
470             current_session['process'] = ev.get('proce
471             ss')
472             current_session['reason'] = ev.get('reason
473             ')
474             current_session['reason_code'] = ev.get('r

```

```

eason_code')
471         current_session['comment'] = ev.get('comment')
472         current_session['clean_shutdown'] = True
473         details = json.loads(ev.get('details_json') or '{}') if isinstance(ev.get('details_json'), str) else (ev.get('details_json') or {})
474         if details.get('initiator_chain'):
475             current_session['initiator_chain'] = details['initiator_chain']
476
477         elif eid in (13, 6006):
478             if not current_session.get('shutdown_event_id'):
479                 current_session['shutdown_event_id'] = eid
480                 if current_session.get('shutdown_type') == 'Active':
481                     current_session['shutdown_type'] = 'Shutdown'
482                 current_session['clean_shutdown'] = True
483
484         elif eid in (41, 6008):
485             current_session['unexpected_shutdown'] = True
486             current_session['clean_shutdown'] = False
487             current_session['shutdown_type'] = 'Unexpected'
488             current_session['shutdown_event_id'] = eid
489             if ev.get('bugcheck_code'):
490                 current_session['bugcheck'] = ev.get('bugcheck_code')
491             current_session['reason'] = ev.get('reason') or 'Внезапное выключение питания (Kernel-Power 41)'
492
493         elif eid == 1001:
494             current_session['unexpected_shutdown'] = True
495     return

```

```

495         current_session['clean_shutdown'] = False
496         current_session['shutdown_type'] = 'BSOD'
497         current_session['bugcheck'] = ev.get('bugc
heck_code') or 'CRITICAL_PROCESS_DIED'
498
499         # Финализация последней активной сессии
500         if current_session:
501             cur_dt, cur_uptime = self.get_current_system_boot(
)
502             now_epoch = time.time()
503             if not current_session.get('shutdown_time'):
504                 current_session['shutdown_type'] = 'Active'
505                 current_session['uptime_seconds'] = max(0.0, n
ow_epoch - current_session['boot_timestamp'])
506                 current_session['uptime_human'] = format_uptime_human(current_session['uptime_seconds'])
507                 sessions.append(PowerSessionRecord(**current_session))
508
509         # Сортировка сессий от самых свежих к старым
510         sessions.sort(key=lambda s: s.boot_timestamp, reverse=
True)
511         return sessions
512
513     def scan_and_reconstruct(self, hours: int = 720, force: bo
ol = False) -> PowerLifecycleSummary:
514         """Выполняет сканирование журналов событий, парсинг и
сохранение сессий в SQLite.
515
516         Args:
517             hours: Глубина выборки в часах (по умолчанию 720ч
= 30 дней).
518             force: Принудительно выполнить пересчет.
519
520         Returns:
521             PowerLifecycleSummary: Сводный отчет жизненного ци
кла питания.
522         """

```

```

523         logger.info(f'[PowerLifecycle] Сканирование событий пи
        тания Windows за последние {hours}ч...')
524         raw_events = self.read_raw_power_events(hours=hours)
525
526         if raw_events:
527             self.storage.save_power_events(raw_events)
528             sessions = self.reconstruct_power_sessions(raw_eve
        nts)
529             self.storage.save_power_sessions(sessions)
530             logger.info(f'[PowerLifecycle] Реконструировано {l
        en(sessions)} сессий питания из {len(raw_events)} событий.')
531         else:
532             sessions = []
533
534         return self.get_summary()
535
536     def get_summary(self) -> PowerLifecycleSummary:
537         """Получает сводную статистику и последние сессии пита
        ния из БД SQLite.
538
539         Returns:
540             PowerLifecycleSummary: Сводный объект метрик.
541         """
542         db_summary = self.storage.get_power_summary()
543         db_sessions_raw = self.storage.get_power_sessions(limi
        t=30)
544         sessions_records = [PowerSessionRecord(**s) for s in d
        b_sessions_raw]
545
546         cur_dt, cur_uptime = self.get_current_system_boot()
547         cur_boot_str = cur_dt.strftime('%Y-%m-%d %H:%M:%S')
548         cur_uptime_human = format_uptime_human(cur_uptime)
549
550         return PowerLifecycleSummary(
551             current_boot_time=db_summary.get('current_boot_tim
        e') or cur_boot_str,
552             current_uptime_seconds=cur_uptime,
553             current_uptime_human=cur_uptime_human,

```

```

554         total_sessions_count=db_summary.get('total_session
| s_count', 0),
555         clean_shutdowns_count=db_summary.get('clean_shutdo
| wns_count', 0),
556         unexpected_shutdowns_count=db_summary.get('unexpec
| ted_shutdowns_count', 0),
557         bsod_count=db_summary.get('bsod_count', 0),
558         last_shutdown_type=db_summary.get('last_shutdown_t
| ype'),
559         last_initiator=db_summary.get('last_initiator'),
560         last_reason=db_summary.get('last_reason'),
561         sessions=sessions_records,
562     )
563
564     def get_sessions(
565         self,
566         limit: int = 50,
567         offset: int = 0,
568         shutdown_type: Optional[str] = None,
569         unexpected_only: bool = False,
570     ) -> List[PowerSessionRecord]:
571         """Возвращает список сессий питания с фильтрацией."""
572         items = self.storage.get_power_sessions(
573             limit=limit,
574             offset=offset,
575             shutdown_type=shutdown_type,
576             unexpected_only=unexpected_only,
577         )
578         return [PowerSessionRecord(**it) for it in items]
579
580     def get_events(
581         self,
582         limit: int = 100,
583         offset: int = 0,
584         event_id: Optional[Union[int, List[int]]] = None,
585     ) -> List[PowerEventRecord]:
586         """Возвращает список событий питания из БД."""
587         items = self.storage.get_power_events(limit=limit, off

```

```
    set=offset, event_id=event_id)
588         return [PowerEventRecord(**it) for it in items]
589
590
591 __all__ = [
592     'PowerLifecycleEngine',
593     'POWER_LIFECYCLE_EVENT_IDS',
594     'format_uptime_human',
595 ]
596
```

process_token_collector.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - Process
  Token Collector
4 # =====
5 # Description:
6 #   Коллектор токенов процессов, уровней целостности, прав UAC
  , SID и Session ID.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.process_token_collector impo
  rt ProcessTokenCollector
11 #
12 #       collector = ProcessTokenCollector()
13 #       tokens = collector.collect()
14 #
15 # File: process_token_collector.py
16 # Project: ai-breadboard
17 # Package: apps.windows.telemetry
18 # Author: hypo69
19 # Copyright: © 2026 hypo69
20 # Updated: 2026-10-04 01:45:00
21 # =====
22
23 """Коллектор токенов процессов, уровней целостности, прав UAC,
  SID и Session ID."""
24
25 from __future__ import annotations
26
27 import ctypes
28 import ctypes.wintypes as wtypes
29 import os
```



```

30 from typing import Dict, List, Optional
31 try:
32     import psutil
33     PSUTIL_AVAILABLE = True
34 except ImportError:
35     psutil = None
36     PSUTIL_AVAILABLE = False
37
38 from .models import ProcessTokenInfo
39
40 TOKEN_QUERY = 8
41 TokenUser = 1
42 TokenGroups = 2
43 TokenPrivileges = 3
44 TokenElevation = 20
45 TokenIntegrityLevel = 25
46
47
48 class TOKEN_ELEVATION(ctypes.Structure):
49     _fields_ = [('TokenIsElevated', ctypes.DWORD)]
50
51
52 class LUID(ctypes.Structure):
53     _fields_ = [('LowPart', ctypes.DWORD), ('HighPart', ctypes
54         .LONG)]
55
56 class LUID_AND_ATTRIBUTES(ctypes.Structure):
57     _fields_ = [('Luid', LUID), ('Attributes', ctypes.DWORD)]
58
59
60 class TOKEN_MANDATORY_LABEL(ctypes.Structure):
61     _fields_ = [('Label', LUID_AND_ATTRIBUTES)]
62
63
64 class SID_AND_ATTRIBUTES(ctypes.Structure):
65     _fields_ = [('Sid', ctypes.c_void_p), ('Attributes', ctypes
66         .DWORD)]

```

```

66 |
67 |
68 | class TOKEN_USER(ctypes.Structure):
69 |     _fields_ = [('User', SID_AND_ATTRIBUTES)]
70 |
71 |
72 | def _get_process_token(handle: Any) -> wtypes.HANDLE:
73 |     """Получить дескриптор токена процесса."""
74 |     token = wtypes.HANDLE()
75 |     if not ctypes.windll.advapi32.OpenProcessToken(handle, TOK
76 | EN_QUERY, ctypes.byref(token)):
77 |         raise ctypes.WinError()
78 |     return token
79 |
80 | def _get_token_elevation(token: Any) -> bool:
81 |     """Определить статус повышения прав UAC."""
82 |     elevation = TOKEN_ELEVATION()
83 |     size = wtypes.DWORD(ctypes.sizeof(elevation))
84 |     ret = ctypes.windll.advapi32.GetTokenInformation(
85 |         token, TokenElevation, ctypes.byref(elevation), size,
86 | ctypes.byref(size)
87 |     )
88 |     if not ret:
89 |         raise ctypes.WinError()
90 |     return bool(elevation.TokenIsElevated)
91 |
92 | def _get_integrity_level(token: Any) -> str:
93 |     """Получить уровень целостности процесса (Untrusted, Low,
94 | Medium, High, System)."""
95 |     size = wtypes.DWORD(0)
96 |     ctypes.windll.advapi32.GetTokenInformation(token, TokenInt
97 | egrityLevel, None, 0, ctypes.byref(size))
98 |     if size.value == 0:
99 |         return 'Medium'
100 |     buf = ctypes.create_string_buffer(size.value)
101 |     if not ctypes.windll.advapi32.GetTokenInformation(token, T

```

```

    okenIntegrityLevel, buf, size, ctypes.byref(size)):
100         raise ctypes.WinError()
101     label = ctypes.cast(buf, ctypes.POINTER(TOKEN_MANDATORY_LA
    BEL)).contents
102     integrity = label.Label.Luid.LowPart
103     if integrity < 4096:
104         return 'Untrusted'
105     elif integrity < 8192:
106         return 'Low'
107     elif integrity < 12288:
108         return 'Medium'
109     elif integrity < 16384:
110         return 'High'
111     else:
112         return 'System'
113
114
115 def _get_token_sid(token: Any) -> Optional[str]:
116     """Извлечь строковый SID пользователя из токена процесса."""
117     try:
118         size = wtypes.DWORD(0)
119         ctypes.windll.advapi32.GetTokenInformation(token, Toke
    nUser, None, 0, ctypes.byref(size))
120         if size.value == 0:
121             return None
122         buf = ctypes.create_string_buffer(size.value)
123         if not ctypes.windll.advapi32.GetTokenInformation(toke
    n, TokenUser, buf, size, ctypes.byref(size)):
124             return None
125         user_struct = ctypes.cast(buf, ctypes.POINTER(TOKEN_US
    ER)).contents
126         sid_ptr = user_struct.User.Sid
127         if not sid_ptr:
128             return None
129         str_sid = wtypes.LPWSTR()
130         if ctypes.windll.advapi32.ConvertSidToStringSidW(sid_p
    tr, ctypes.byref(str_sid)):

```

```

131         sid_val = str_sid.value
132         ctypes.windll.kernel32.LocalFree(str_sid)
133         return sid_val
134     except Exception:
135         pass
136     return None
137
138
139 def get_process_session_id(pid: int) -> Optional[int]:
140     """Получить Session ID процесса (Session 0=Службы, Session
141     1+=Интерактивный пользователь)."""
142     if os.name != 'nt':
143         return None
144     try:
145         session_id = ctypes.DWORD(0)
146         if ctypes.windll.kernel32.ProcessIdToSessionId(pid, ct
147         ypes.byref(session_id)):
148             return int(session_id.value)
149     except Exception:
150         pass
151     return None
152
153 class ProcessTokenCollector:
154     """Коллектор сведений о безопасности токенов, правах UAC,
155     SID и сессиях процессов Windows."""
156
157     def get_process_token_info(self, pid: int, name: str = '')
158     -> Optional[ProcessTokenInfo]:
159         """Собрать информацию о токене для конкретного PID.
160
161         Args:
162             pid: Идентификатор процесса.
163             name: Имя процесса (опционально).
164
165         Returns:
166             Optional[ProcessTokenInfo]: Данные токена или None
167             при отсутствии доступа.

```

```

164         """
165         if os.name != 'nt':
166             return ProcessTokenInfo(pid=pid, name=name, elevat
ion=False, integrity_level='Medium', sid=None, session_id=1)
167
168         session_id = get_process_session_id(pid)
169         handle = ctypes.windll.kernel32.OpenProcess(1024, Fals
e, pid)
170         if not handle:
171             return ProcessTokenInfo(
172                 pid=pid,
173                 name=name,
174                 elevation=None,
175                 integrity_level=None,
176                 privileges=[],
177                 sid=None,
178                 session_id=session_id,
179             )
180         try:
181             token = _get_process_token(handle)
182             elevated = _get_token_elevation(token)
183             integrity = _get_integrity_level(token)
184             sid = _get_token_sid(token)
185             privileges: List[str] = []
186             return ProcessTokenInfo(
187                 pid=pid,
188                 name=name,
189                 elevation=bool(elevated),
190                 integrity_level=integrity,
191                 privileges=privileges,
192                 sid=sid,
193                 session_id=session_id,
194             )
195         except Exception:
196             return ProcessTokenInfo(
197                 pid=pid,
198                 name=name,
199                 elevation=None,

```

```

200         integrity_level=None,
201         privileges=[],
202         sid=None,
203         session_id=session_id,
204     )
205     finally:
206         ctypes.windll.kernel32.CloseHandle(handle)
207
208     def collect(self) -> List[ProcessTokenInfo]:
209         """Собрать сведения о токенах для всех активных процес
210         сов в системе.
211
212         Returns:
213             List[ProcessTokenInfo]: Список токенов процессов.
214         """
215         results: List[ProcessTokenInfo] = []
216         if not PSUTIL_AVAILABLE:
217             current_pid = os.getpid()
218             info = self.get_process_token_info(current_pid, 'p
219             ython.exe')
220             if info:
221                 results.append(info)
222             return results
223
224         for proc in psutil.process_iter(attrs=['pid', 'name']):
225             :
226             try:
227                 pid = proc.info['pid']
228                 name = proc.info['name'] or 'unknown'
229                 info = self.get_process_token_info(pid, name)
230                 if info:
231                     results.append(info)
232             except Exception:
233                 continue
234         return results

```

ring_buffer.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - Ring Bu
  ffer
4 # =====
5 # Description:
6 #   Потокобезопасный кольцевой буфер в оперативной памяти для
  удержания
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.ring_buffer import Telemetry
  RingBuffer
11 #
12 #       service = TelemetryRingBuffer()
13 #
14 # File: ring_buffer.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-06 05:32:00
20 # =====
21
22 from __future__ import annotations
23 """Потокобезопасный кольцевой буфер в оперативной памяти для у
  держания сырой телеметрии."""
24
25 import collections
26 import json
27 import sys
28 import threading
29 import time
```

```

30 from datetime import datetime, timezone
31 from typing import Any, Dict, List, Optional, Tuple
32
33
34 class TelemetryRingBuffer:
35     """Потокобезопасный кольцевой буфер для хранения сырых мет
    | рик телеметрии в RAM с ограничением по объему памяти."""
36
37     def __init__(self, capacity: int = 600, max_bytes: int = 3
    | 2 * 1024 * 1024) -> None:
38         """Инициализирует кольцевой буфер заданной емкости и м
    | аксимального объема RAM.
39
40         Args:
41             capacity: Максимальное количество удерживаемых зап
    | исей (по умолчанию 600 = 10 мин при 1с).
42             max_bytes: Максимальный совокупный объем памяти бу
    | фера в байтах (по умолчанию 32 МБ).
43         """
44         self._capacity = max(10, capacity)
45         self._max_bytes = max(1024 * 1024, max_bytes)
46         self._current_bytes = 0
47         self._buffer: collections.deque[Tuple[float, Dict[str,
    | Any], int]] = collections.deque(maxlen=self._capacity)
48         self._lock = threading.Lock()
49
50     @property
51     def capacity(self) -> int:
52         """Возвращает максимальную емкость буфера в количестве
    | записей."""
53         return self._capacity
54
55     @property
56     def max_bytes(self) -> int:
57         """Возвращает максимальный лимит памяти буфера в байта
    | х."""
58         return self._max_bytes
59

```



```

60 |     @property
61 |     def current_bytes(self) -> int:
62 |         """Возвращает текущий ориентировочный объем занимаемой
        |         памяти буфера в байтах."""
63 |         with self._lock:
64 |             return self._current_bytes
65 |
66 |     def _estimate_size(self, item: Dict[str, Any]) -> int:
67 |         """Оценивает размер словаря с метриками в байтах."""
68 |         try:
69 |             return len(json.dumps(item, ensure_ascii=False, de
        |             fault=str).encode('utf-8')) + 128
70 |         except Exception:
71 |             return sys.getsizeof(item) + 256
72 |
73 |     def append(self, item: Dict[str, Any], epoch_timestamp: Op
        |     tional[float] = None) -> None:
74 |         """Добавляет запись телеметрии в кольцевой буфер с кон
        |         тролем лимитов RAM и количества.
75 |
76 |         Args:
77 |             item: Словарь с телеметрией или системным снимком.
78 |             epoch_timestamp: Unix epoch timestamp в секундах (
        |             если None, берется текущее время).
79 |         """
80 |         ts = epoch_timestamp if epoch_timestamp is not None el
        |         se time.time()
81 |         item_size = self._estimate_size(item)
82 |         with self._lock:
83 |             # Если буфер заполнен по длине, deque сам вытеснит
        |             старый элемент, но нам нужно учесть его размер
84 |             if len(self._buffer) == self._capacity and self._b
        |             uffer:
85 |                 old_ts, old_item, old_size = self._buffer[0]
86 |                 self._current_bytes = max(0, self._current_byt
        |                 es - old_size)
87 |
88 |                 self._buffer.append((ts, item, item_size))

```

```

89         self._current_bytes += item_size
90
91         # Контроль жесткого байтового лимита RAM: вытеснен
    ие самых старых записей при переполнении
92         while self._current_bytes > self._max_bytes and le
    n(self._buffer) > 1:
93             old_ts, old_item, old_size = self._buffer.popl
    eft()
94             self._current_bytes = max(0, self._current_byt
    es - old_size)
95
96     def get_recent(self, seconds: float = 300.0) -> List[Dict[
    str, Any]]:
97         """Возвращает список записей за последние N секунд.
98
99         Args:
100             seconds: Длительность временного окна назад от тек
    ущего момента.
101
102         Returns:
103             List[Dict[str, Any]]: Список замеров от старых к н
    ОВЫМ.
104
105         """
106         now = time.time()
107         cutoff = now - max(0.0, seconds)
108         with self._lock:
109             return [item for ts, item, _ in self._buffer if ts
    >= cutoff]
110
111     def get_window(
112         self,
113         center_timestamp: float,
114         before_seconds: float = 300.0,
115         after_seconds: float = 0.0,
116     ) -> List[Dict[str, Any]]:
117         """Извлекает срез телеметрии вокруг заданного момента
    времени (например, T-5m .. T+20m).

```

```
118 |         Args:
119 |             center_timestamp: Центральный момент времени событ
    | ия (Unix epoch seconds).
120 |             before_seconds: Длительность окна ДО события в сек
    | ундах.
121 |             after_seconds: Длительность окна ПОСЛЕ события в с
    | екундах.
122 |
123 |         Returns:
124 |             List[Dict[str, Any]]: Список записей, попавших в и
    | нтервал.
125 |             """
126 |             start_ts = center_timestamp - max(0.0, before_seconds)
127 |             end_ts = center_timestamp + max(0.0, after_seconds)
128 |             with self._lock:
129 |                 return [item for ts, item, _ in self._buffer if st
    | art_ts <= ts <= end_ts]
130 |
131 |     def get_all(self) -> List[Dict[str, Any]]:
132 |         """Возвращает все текущие записи в буфере.
133 |
134 |         Returns:
135 |             List[Dict[str, Any]]: Полный текущий срез буфера.
136 |             """
137 |             with self._lock:
138 |                 return [item for _, item, _ in self._buffer]
139 |
140 |     def count(self) -> int:
141 |         """Возвращает текущее число записей в буфере."""
142 |         with self._lock:
143 |             return len(self._buffer)
144 |
145 |     def clear(self) -> None:
146 |         """Очищает буфер."""
147 |         with self._lock:
148 |             self._buffer.clear()
149 |             self._current_bytes = 0
150 |
```

sampling_controller.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - Sampling
  # Controller
4 # =====
5 # Description:
6 #   Контроллер адаптивного изменения частоты сбора телеметрии
  # (Sampling Controller).
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.sampling_controller import S
  # amplingController
11 #
12 #       service = SamplingController()
13 #
14 # File: sampling_controller.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-06 05:34:00
20 # =====
21
22 from __future__ import annotations
23 """Контроллер адаптивного изменения частоты сбора телеметрии (
  # Sampling Controller)."""
24
25 import time
26 from datetime import datetime, timezone
27 from typing import Any, Dict, Optional
28
29 try:
```

```

30     import psutil
31     BOOT_TIME = psutil.boot_time()
32 except Exception:
33     BOOT_TIME = time.time()
34
35 try:
36     from logger import logger
37 except ImportError:
38     from logger import logger
39
40 from .models import SamplingMode
41
42
43 class SamplingController:
44     """Контроллер режимов сбора телеметрии и адаптивного интер
45 вала."""
46
47     def __init__(
48         self,
49         initial_mode: SamplingMode = SamplingMode.BOOT_AGGRESS
50 IVE,
51         boot_epoch: Optional[float] = None,
52     ) -> None:
53         """Инициализирует контроллер режимов сбора.
54
55         Args:
56             initial_mode: Начальный режим работы (по умолчанию
57 BOOT_AGGRESSIVE).
58             boot_epoch: Время старта ОС или сервиса в секундах
59 Unix epoch.
60         """
61         self._mode = initial_mode
62         self._boot_epoch = boot_epoch if boot_epoch is not Non
63 e else BOOT_TIME
64         self._start_epoch = time.time()
65         self._incident_expiry_epoch: Optional[float] = None
66         self._previous_mode: SamplingMode = SamplingMode.MONIT
67 ORING

```

```

62 |
63 |     @property
64 |     def mode(self) -> SamplingMode:
65 |         """Возвращает текущий режим сбора с учетом авто-истече
        ния инцидентов."""
66 |         self._check_incident_expiration()
67 |         return self._mode
68 |
69 |     def set_mode(self, mode: SamplingMode) -> None:
70 |         """Принудительно устанавливает режим сбора телеметрии.
71 |
72 |         Args:
73 |             mode: Новый режим работы.
74 |         """
75 |         self._mode = mode
76 |         if mode not in (SamplingMode.INCIDENT, SamplingMode.FO
        RENSIC):
77 |             self._incident_expiry_epoch = None
78 |             logger.info(f"Режим сбора телеметрии переключен на: {s
        elf._mode.value}")
79 |
80 |     def trigger_incident_mode(self, duration_seconds: float =
        1200.0) -> None:
81 |         """Переводит сборщик в режим INCIDENT (1-секундный зах
        ват) на заданную длительность.
82 |
83 |         Args:
84 |             duration_seconds: Длительность удержания режима ин
        цидента (по умолчанию 20 минут = 1200 сек).
85 |         """
86 |         if self._mode not in (SamplingMode.INCIDENT, SamplingM
        ode.FORENSIC):
87 |             self._previous_mode = self._mode
88 |             self._mode = SamplingMode.INCIDENT
89 |             self._incident_expiry_epoch = time.time() + max(0.01,
        duration_seconds)
90 |             logger.warning(f"Активирован режим INCIDENT на {durati
        on_seconds} сек.")

```

```

91
92     def trigger_forensic_mode(self, duration_seconds: float =
93     300.0) -> None:
94         """Переводит сборщик в режим FORENSIC (250-мс глубокий
95         захват) на заданную длительность.
96
97         Args:
98             duration_seconds: Длительность удержания форензик-
99             режима в секундах (по умолчанию 5 минут = 300 сек).
100         """
101         if self._mode not in (SamplingMode.INCIDENT, SamplingM
102         ode.FORENSIC):
103             self._previous_mode = self._mode
104             self._mode = SamplingMode.FORENSIC
105             self._incident_expiry_epoch = time.time() + max(0.01,
106             duration_seconds)
107             logger.warning(f"Активирован высокоскоростной режим FO
108             RENSIC (250мс) на {duration_seconds} сек.")
109
110     def get_current_interval(self) -> float:
111         """Возвращает текущий рекомендуемый интервал опроса те
112         леметрии в секундах.
113
114         Returns:
115             float: Интервал в секундах.
116         """
117         self._check_incident_expiration()
118
119         if self._mode == SamplingMode.FORENSIC:
120             return 0.25
121
122         if self._mode == SamplingMode.INCIDENT:
123             return 1.0
124
125         if self._mode == SamplingMode.BOOT_AGGRESSIVE:
126             # Адаптивная кривая после старта Windows / сервиса
127             uptime = time.time() - self._boot_epoch
128             if uptime < 0:

```

```

122         uptime = time.time() - self._start_epoch
123
124         if uptime <= 600.0:  # 0-10 минут: 1-секундные дан
ные
125             return 1.0
126         elif uptime <= 3600.0:  # 10-60 минут: 5-секундные
127             return 5.0
128         elif uptime <= 86400.0:  # 1-24 часа: 30-секундные
129             return 30.0
130         else:
131             # После 24 часов переходим в дежурный монитори
нг
132             return 60.0
133
134         if self._mode == SamplingMode.MONITORING:
135             return 30.0
136
137         if self._mode == SamplingMode.NORMAL:
138             return 60.0
139
140         return 30.0
141
142     def should_collect_per_process_details(self) -> bool:
143         """Определяет, требуется ли собирать глубокий срез по
всем процессам на текущем тике."""
144         return True
145
146     def _check_incident_expiration(self) -> None:
147         """Проверяет истечение времени режима инцидента/форенз
ики и возвращает предыдущий режим."""
148         if self._mode in (SamplingMode.INCIDENT, SamplingMode.
FORENSIC) and self._incident_expiry_epoch is not None:
149             if time.time() >= self._incident_expiry_epoch:
150                 logger.info("Время фиксации инцидента/форензик
и истекло. Возврат в штатный режим.")
151                 self._mode = self._previous_mode
152                 self._incident_expiry_epoch = None
153

```



```
154 |     def get_status(self) -> Dict[str, Any]:
155 |         """Возвращает статус контроллера опроса.
156 |
157 |         Returns:
158 |             Dict[str, Any]: Словарь с текущим режимом, интерва
159 | лом и временем старта.
160 |         """
161 |         return {
162 |             "mode": self.mode.value,
163 |             "current_interval_sec": self.get_current_interval(
164 | ),
165 |             "boot_epoch": self._boot_epoch,
166 |             "incident_active": self._mode in (SamplingMode.INC
167 | IDENT, SamplingMode.FORENSIC),
168 |             "incident_remaining_sec": max(0.0, (self._incident
169 | _expiry_epoch - time.time())) if self._incident_expiry_epoch e
170 | lse 0.0,
171 |         }
```

security_collector.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - Security Collector
4 # =====
5 # Description:
6 #   Коллектор событий журнала безопасности Windows Security Event Log.
7 #   Реализует инкрементальный сбор на основе закладок (Bookmark), XPath-фильтрацию,
8 #   нормализацию и корреляцию с телеметрией системы.
9 #
10 # Usage Examples:
11 #   Python API:
12 #       from apps.windows.telemetry.security_collector import WindowsSecurityCollector
13 #
14 #       collector = WindowsSecurityCollector()
15 #       status = collector.check_access_and_audit()
16 #       report = collector.collect_incremental(batch_size=200)
17 #
18 # File: security_collector.py
19 # Project: ai-breadboard
20 # Package: apps.windows.telemetry
21 # Author: hypo69
22 # Copyright: © 2026 hypo69
23 # Updated: 2026-10-10 05:40:00
24 # =====
25
26 from __future__ import annotations
27 """Коллектор событий журнала безопасности Windows Security Event Log и Microsoft Defender."""
28
```

```
29 import os
30 import sys
31 import time
32 import winreg
33 from datetime import datetime, timezone
34 from typing import Any, Callable, Dict, List, Optional, Set, T
   tuple
35
36 try:
37     from logger import logger
38 except ImportError:
39     import logging
40     logger = logging.getLogger(__name__)
41
42 from .models import (
43     SecurityAuditStatus,
44     SecurityBookmarkState,
45     SecurityCollectorReport,
46     SecurityCorrelationItem,
47     SecurityEventItem,
48     SecurityEventRaw,
49 )
50 from .security_normalizer import SecurityEventNormalizer
51 from .sqlite import TelemetryStorage
52 from .win32_fffi.wevtapi import WevtAPI
53
54
55 DEFAULT_SECURITY_EVENT_IDS: List[int] = [
56     4624, # Успешный вход
57     4625, # Неудачный вход
58     4634, # Завершение logon
59     4647, # Пользователь инициировал logoff
60     4672, # Специальные привилегии
61     4688, # Создание процесса
62     4689, # Завершение процесса
63     4697, # Установка службы
64     4698, # Создание Scheduled Task
65     4702, # Изменение Scheduled Task
```

```

66 | 4719, # Изменение аудита
67 | 4720, # Создание пользователя
68 | 4722, # Включение пользователя
69 | 4723, # Попытка изменения пароля
70 | 4724, # Сброс пароля
71 | 4725, # Отключение пользователя
72 | 4726, # Удаление пользователя
73 | 4732, # Добавление в локальную группу
74 | 4738, # Изменение пользователя
75 | 4740, # Блокировка пользователя
76 | 4768, # Kerberos TGT
77 | 4769, # Kerberos service ticket
78 | 4771, # Kerberos pre-auth failure
79 | 4776, # NTLM authentication
80 | 1102, # Очищен Security log
81 | ]
82 |
83 | DEFENDER_CHANNEL: str = 'Microsoft-Windows-Windows Defender/Operational'
84 |
85 | DEFAULT_DEFENDER_EVENT_IDS: List[int] = [
86 |     1000, # Запуск сканирования
87 |     1001, # Завершение сканирования
88 |     1002, # Отмена сканирования
89 |     1005, # Пауза сканирования
90 |     1013, # Очистка истории угроз
91 |     1116, # Обнаружение вредоносной программы / угрозы
92 |     1117, # Выполнение действия по угрозе (Quarantine/Clean/Block)
93 |     1118, # Сбой действия по угрозе
94 |     1119, # Критическая ошибка
95 |     1121, # Блокировка CFA (Controlled Folder Access)
96 |     1122, # Аудит CFA
97 |     1123, # Блокировка ASR (Attack Surface Reduction)
98 |     1124, # Аудит ASR
99 |     1127, # Блокировка Network Protection
100 |     1128, # Аудит Network Protection
101 |     1150, # Служба защиты работает штатно (Healthy)

```

```

102 |     1151, # Отчет о работоспособности компонентов
103 |     2000, # Обновление сигнатур успешно
104 |     2001, # Ошибка обновления сигнатур
105 |     2010, # Облачная защита/аналитика
106 |     5000, # Включение защиты в реальном времени
107 |     5001, # Отключение защиты в реальном времени
108 |     5004, # Изменение конфигурации
109 |     5007, # Изменение параметров защиты
110 | ]
111 |
112 |
113 | class WindowsSecurityCollector:
114 |     """Инкрементальный сборщик событий безопасности Windows (S
115 | security.evtx и Defender Operational)."""
116 |
117 |     def __init__(
118 |         self,
119 |         storage: Optional[TelemetryStorage] = None,
120 |         wevtapi: Optional[WevtAPI] = None,
121 |         normalizer: Optional[SecurityEventNormalizer] = None,
122 |     ) -> None:
123 |         """Инициализация сборщика безопасности.
124 |
125 |         Args:
126 |             storage: Экземпляр хранилища TelemetryStorage (если
127 | и None, берется синглтон).
128 |             wevtapi: Экземпляр обертки WevtAPI.
129 |             normalizer: Экземпляр нормализатора событий.
130 |         """
131 |         self.storage = storage or TelemetryStorage.get_instance()
132 |         self.wevtapi = wevtapi or WevtAPI()
133 |         self.normalizer = normalizer or SecurityEventNormalizer()
134 |
135 |     def check_access_and_audit(self) -> SecurityAuditStatus:
136 |         """Проверить доступность журнала Security и статус политик аудита.

```

```

135
136         Returns:
137             SecurityAuditStatus: Структурированный статус прав
            и настроек.
138         """
139         access_info = self.wevtapi.check_channel_access('Security')
140         accessible = access_info.get('accessible', False)
141         record_count = access_info.get('record_count', 0)
142         error_msg = access_info.get('error')
143
144         # Проверка включения аудита командной строки в реестре
145         cmdline_audit = False
146         try:
147             with winreg.OpenKey(
148                 winreg.HKEY_LOCAL_MACHINE,
149                 r'SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System\Audit',
150                 0,
151                 winreg.KEY_READ,
152             ) as key:
153                 val, _ = winreg.QueryValueEx(key, 'ProcessCreationIncludeCmdLine_Enabled')
154                 cmdline_audit = bool(val)
155         except OSError:
156             cmdline_audit = False
157
158         # Получаем статистику из базы данных
159         stats = self.storage.get_security_stats() if self.storage else {}
160         last_rec = stats.get('max_record_id', 0)
161         total_ingested = stats.get('total_events', 0)
162
163         return SecurityAuditStatus(
164             accessible=accessible,
165             record_count=record_count,
166             process_creation_audit_enabled=True, # Базовый аудит включен в современных редакциях Windows

```

```

167         command_line_audit_enabled=cmdline_audit,
168         error=error_msg,
169         last_record_id=last_rec,
170         total_ingested_events=total_ingested,
171     )
172
173     def _build_xpath_query(
174         self,
175         channel: str = 'Security',
176         event_ids: Optional[List[int]] = None,
177     ) -> str:
178         """Построить эффективный XPath фильтр для выборки целе
179         вых Event ID.
180
181         Предотвращает переполнение предиката Win32 WevtAPI (ко
182         д 15001) при длинных списках,
183         используя диапазонную фильтрацию.
184         """
185         if event_ids is None:
186             if 'Defender' in channel:
187                 return (
188                     '*[System[((EventID >= 1000 and EventID <=
189                     1160) or '
190                     '(EventID >= 2000 and EventID <= 2020) or
191                     '(EventID >= 5000 and EventID <= 5020))]]'
192                 )
193             # Стандартный диапазон охватывает все ключевые соб
194             ытия аудита безопасности (4624-4776 + 1102)
195             return '*[System[(EventID >= 4624 and EventID <= 4
196             776) or EventID=1102]]'
197             if not event_ids:
198                 return '*'
199             if len(event_ids) <= 15:
200                 clauses = ' or '.join(f'EventID={eid}' for eid in
201                 event_ids)
202                 return f'*[System[({clauses})]]'
203             min_id = min(event_ids)

```

```

198         max_id = max(event_ids)
199         return f'*[System[(EventID >= {min_id} and EventID <=
| {max_id})]]'
200
201     def collect_incremental(
202         self,
203         channel: str = 'Security',
204         batch_size: int = 500,
205         max_records: int = 5000,
206         filter_event_ids: Optional[List[int]] = None,
207         save_raw: bool = False,
208     ) -> SecurityCollectorReport:
209         """Выполнить инкрементальный сбор новых событий из ука
| занного журнала (Security или Defender).
210
211         Args:
212             channel: Имя канала ('Security' или 'Microsoft-Win
| dows-Windows Defender/Operational').
213             batch_size: Размер порции за один вызов EvtNext.
214             max_records: Максимальное количество событий для с
| бора за один вызов.
215             filter_event_ids: Список опрашиваемых Event ID или
| None для дефолтного набора.
216             save_raw: Сохранять ли параллельно сырой XML в sec
| urity_events_raw.
217
218         Returns:
219             SecurityCollectorReport: Отчет о результатах сбора
| .
220         """
221         report = SecurityCollectorReport(channel=channel)
222
223         # Проверка доступа к журналу
224         access_info = self.wevtapi.check_channel_access(channe
| l)
225
226         if not access_info.get('accessible', False):
227             err = access_info.get('error') or f'Доступ к канал
| у {channel} запрещен'

```



```

227         report.errors.append(err)
228         logger.warning(f'[SecurityCollector] {err}')
229         return report
230
231         # Загрузка сохраненной закладки
232         bm_dict = self.storage.get_security_bookmark(channel)
233         if self.storage else None
234         bookmark_xml = bm_dict.get('bookmark_xml') if bm_dict
235         else None
236         last_saved_record_id = bm_dict.get('last_record_id', 0
237         ) if bm_dict else 0
238
239         xpath = self._build_xpath_query(channel=channel, event
240         _ids=filter_event_ids)
241         collected_items: List[SecurityEventItem] = []
242         collected_raw: List[SecurityEventRaw] = []
243         events_by_id: Dict[int, int] = {}
244         max_record_id = last_saved_record_id
245         latest_timestamp = ''
246         total_fetched = 0
247
248         # Если закладки еще нет (первый запуск), читаем истори
249         ческий хвост последних событий в обратном порядке
250         if not bookmark_xml:
251             tail_limit = min(batch_size, max_records)
252             events_raw, init_bm, highest_rec_id = self.wevtapi
253             .read_events_incremental(
254                 channel=channel,
255                 query_xpath=xpath,
256                 bookmark_xml=None,
257                 batch_size=tail_limit,
258                 format_message=False,
259                 forward=False,
260             )
261             # Разворачиваем в хронологический порядок
262             events_raw = list(reversed(events_raw))
263             bookmark_xml = init_bm
264             max_record_id = highest_rec_id

```

```

259 |
260 |         for raw_dict in events_raw:
261 |             item, raw_item = self.normalizer.normalize_event(
262 |                 raw_dict, extract_raw=save_raw)
263 |             collected_items.append(item)
264 |             if raw_item:
265 |                 collected_raw.append(raw_item)
266 |                 events_by_id[item.event_id] = events_by_id.get(
267 |                     (item.event_id, 0) + 1
268 |                     if item.timestamp:
269 |                         latest_timestamp = item.timestamp
270 |                         rec_id = item.event_record_id
271 |                         if rec_id > max_record_id:
272 |                             max_record_id = rec_id
273 |
274 |         if self.storage and collected_items:
275 |             self.storage.save_security_events(collected_items,
276 |                 save_raw=save_raw, raw_events=collected_raw)
277 |             if bookmark_xml:
278 |                 self.storage.save_security_bookmark(
279 |                     channel=channel,
280 |                     last_record_id=max_record_id,
281 |                     bookmark_xml=bookmark_xml,
282 |                     last_timestamp=latest_timestamp,
283 |                 )
284 |             total_fetched += len(events_raw)
285 |
286 |         # Последующие запуски (или продолжение) читают строго
287 |         # вперед от закладки
288 |         while total_fetched < max_records and bookmark_xml:
289 |             cur_batch_limit = min(batch_size, max_records - total_fetched)
290 |             events_raw, new_bookmark_xml, highest_rec_id = self.wevtapi.read_events_incremental(
291 |                 channel=channel,
292 |                 query_xpath=xpath,
293 |                 bookmark_xml=bookmark_xml,
294 |                 batch_size=cur_batch_limit,

```

```
291         format_message=False,
292         forward=True,
293     )
294
295     if not events_raw:
296         break
297
298     batch_items: List[SecurityEventItem] = []
299     batch_raw: List[SecurityEventRaw] = []
300
301     for raw_dict in events_raw:
302         item, raw_item = self.normalizer.normalize_event(
303             raw_dict, extract_raw=save_raw)
304         batch_items.append(item)
305         if raw_item:
306             batch_raw.append(raw_item)
307             events_by_id[item.event_id] = events_by_id.get(
308                 (item.event_id, 0) + 1
309                 if item.timestamp:
310                     latest_timestamp = item.timestamp
311                     rec_id = item.event_record_id
312                     if rec_id > max_record_id:
313                         max_record_id = rec_id
314
315     if highest_rec_id > max_record_id:
316         max_record_id = highest_rec_id
317
318     # Сохраняем порцию в SQLite
319     if self.storage and batch_items:
320         self.storage.save_security_events(batch_items,
321             save_raw=save_raw, raw_events=batch_raw)
322         if new_bookmark_xml:
323             self.storage.save_security_bookmark(
324                 channel=channel,
325                 last_record_id=max_record_id,
326                 bookmark_xml=new_bookmark_xml,
327                 last_timestamp=latest_timestamp,
328             )
```

```

326
327         collected_items.extend(batch_items)
328         if save_raw:
329             collected_raw.extend(batch_raw)
330
331         total_fetched += len(events_raw)
332         bookmark_xml = new_bookmark_xml
333
334         # Если вернулось меньше запрошенного batch_size, з
начит достигли текущего конца журнала
335         if len(events_raw) < cur_batch_limit:
336             break
337
338         report.total_events_ingested = len(collected_items)
339         report.last_record_id = max_record_id
340         report.bookmark_xml = bookmark_xml
341         report.events_by_id = events_by_id
342         report.events = collected_items[:100] # Ограничиваем
возвращаемый срез в отчете
343
344         logger.info(
345             f'[SecurityCollector] Инкрементально собрано {len(
collected_items)} событий канала {channel}. '
346             f'Последний RecordID: {max_record_id}'
347         )
348         return report
349
350     def collect_defender_events(
351         self,
352         batch_size: int = 200,
353         max_records: int = 2000,
354         filter_event_ids: Optional[List[int]] = None,
355         save_raw: bool = False,
356     ) -> SecurityCollectorReport:
357         """Выполнить инкрементальный сбор новых событий Micros
oft Defender (Defender/Operational).
358
359         Args:

```

```

360         batch_size: Размер порции за один вызов.
361         max_records: Максимальное число событий за запуск.
362         filter_event_ids: Опциональный список Event ID.
363         save_raw: Сохранять ли сырой XML.
364
365     Returns:
366         SecurityCollectorReport: Отчет с собранными событиями Defender.
367         """
368     return self.collect_incremental(
369         channel=DEFENDER_CHANNEL,
370         batch_size=batch_size,
371         max_records=max_records,
372         filter_event_ids=filter_event_ids,
373         save_raw=save_raw,
374     )
375
376     def collect_all_security_and_defender(
377         self,
378         batch_size: int = 500,
379         max_records: int = 2000,
380         save_raw: bool = False,
381     ) -> Dict[str, SecurityCollectorReport]:
382         """Инкрементально собрать события обоих журналов: Security и Defender.
383
384     Args:
385         batch_size: Размер выборки.
386         max_records: Лимит событий на канал.
387         save_raw: Сохранение сырого XML.
388
389     Returns:
390         Dict[str, SecurityCollectorReport]: Отчеты по каждому каналу.
391         """
392     sec_rep = self.collect_incremental(
393         channel='Security',
394         batch_size=batch_size,

```

```

395         max_records=max_records,
396         save_raw=save_raw,
397     )
398     def_rep = self.collect_defender_events(
399         batch_size=batch_size,
400         max_records=max_records,
401         save_raw=save_raw,
402     )
403     return {
404         'Security': sec_rep,
405         DEFENDER_CHANNEL: def_rep,
406     }
407
408     def get_live_tail(
409         self,
410         limit: int = 20,
411         event_id: Optional[int] = None,
412         search: str = '',
413         channel: str = 'Security',
414     ) -> List[SecurityEventItem]:
415         """Получить самые свежие события безопасности в реальн
ом времени (live tail).
416
417         Args:
418             limit: Количество событий.
419             event_id: Опциональный фильтр по Event ID.
420             search: Поисковая подстрока.
421             channel: Канал ('Security' или DEFENDER_CHANNEL).
422
423         Returns:
424             List[SecurityEventItem]: Список нормализованных со
бытий.
425         """
426         raw_events = self.wevtapi.read_events(
427             channel=channel,
428             limit=limit,
429             event_id=event_id or 0,
430             search=search,

```

```

431         format_message=False,
432     )
433     items: List[SecurityEventItem] = []
434     for raw in raw_events:
435         item, _ = self.normalizer.normalize_event(raw, extract_raw=False)
436         items.append(item)
437     return items
438
439     def subscribe_defender_stream(
440         self,
441         callback: Callable[[SecurityEventItem], None],
442         filter_event_ids: Optional[List[int]] = None,
443     ) -> Optional[Any]:
444         """Оформить потоковую подписку на входящие события Microsoft Defender.
445
446         Args:
447             callback: Функция обратного вызова при возникновении события.
448             filter_event_ids: Список Event ID для фильтрации.
449
450         Returns:
451             Optional[HANDLE]: Дескриптор подписки или None.
452         """
453         xpath = self._build_xpath_query(channel=DEFENDER_CHANNEL, event_ids=filter_event_ids)
454
455         def _on_event(raw_dict: Dict[str, Any]) -> None:
456             item, raw_item = self.normalizer.normalize_event(raw_dict, extract_raw=True)
457             if self.storage:
458                 self.storage.save_security_events([item], save_raw=True, raw_events=[raw_item] if raw_item else None)
459             callback(item)
460
461         return self.wevtapi.subscribe_events(
462             channel=DEFENDER_CHANNEL,

```

```

463         query_xpath=xpath,
464         callback=_on_event,
465         start_at_oldest=False,
466     )
467
468     def subscribe_stream(
469         self,
470         callback: Callable[[SecurityEventItem], None],
471         filter_event_ids: Optional[List[int]] = None,
472     ) -> Optional[Any]:
473         """Оформить потоковую подписку на входящие события без
опасности через EvtSubscribe.
474
475         Args:
476             callback: Функция обратного вызова при получении с
обытия.
477             filter_event_ids: Список Event ID для фильтрации.
478
479         Returns:
480             Optional[HANDLE]: Дескриптор подписки или None.
481         """
482         xpath = self._build_xpath_query(filter_event_ids)
483
484         def _on_event(raw_dict: Dict[str, Any]) -> None:
485             item, raw_item = self.normalizer.normalize_event(r
aw_dict, extract_raw=True)
486             if self.storage:
487                 self.storage.save_security_events([item], save
_raw=True, raw_events=[raw_item] if raw_item else None)
488                 callback(item)
489
490         return self.wevtapi.subscribe_events(
491             channel='Security',
492             query_xpath=xpath,
493             callback=_on_event,
494             start_at_oldest=False,
495         )
496

```



```

497     def correlate_security_with_telemetry(
498         self,
499         time_window_seconds: float = 60.0,
500         pid: Optional[int] = None,
501         user: Optional[str] = None,
502         limit: int = 50,
503     ) -> List[SecurityCorrelationItem]:
504         """Выполнить корреляцию событий безопасности с метрика
ми телеметрии и процессами.
505
506         Связывает события создания процессов (4688), входов по
льзователей (4624) и
507         событий служб (4697) с реальной загрузкой CPU/RAM/диск
а и снимками процессов.
508
509         Args:
510             time_window_seconds: Временное окно корреляции в с
екундах.
511             pid: Фильтр по PID.
512             user: Фильтр по пользователю.
513             limit: Лимит записей.
514
515         Returns:
516             List[SecurityCorrelationItem]: Список коррелирован
ных событий с аналитикой.
517         """
518         if not self.storage:
519             return []
520
521         # 1. Извлекаем события безопасности
522         sec_events = self.storage.get_security_events(
523             user=user,
524             pid=pid,
525             limit=limit,
526         )
527         if not sec_events:
528             return []
529

```

```

530 |         # 2. Извлекаем последние снимки системы
531 |         snapshots = self.storage.get_snapshots(limit=30)
532 |         snap_by_time = {s.get('timestamp', ''): s for s in snapshots}
533 |
534 |         correlations: List[SecurityCorrelationItem] = []
535 |
536 |         for ev in sec_events:
537 |             eid = int(ev.get('event_id', 0) or 0)
538 |             p_name = ev.get('process_name', '')
539 |             p_short = p_name.replace('\\', '/').split('/')[-1]
540 |             if p_name else ''
541 |             p_pid = int(ev.get('process_id', 0) or 0)
542 |             u_name = ev.get('subject_user', '') or ev.get('target_user', '')
543 |             ts = ev.get('timestamp', '')
544 |             cmd = ev.get('command_line', '')
545 |             parent_name = ev.get('parent_process_name', '')
546 |             parent_short = parent_name.replace('\\', '/').split('/')[-1] if parent_name else ''
547 |
548 |             notes: List[str] = []
549 |             cpu_val: Optional[float] = None
550 |             ram_val: Optional[float] = None
551 |             disk_spike = False
552 |             net_spike = False
553 |
554 |             # Поиск близкого по времени снимка телеметрии
555 |             for s_ts, snap in snap_by_time.items():
556 |                 if ts and s_ts and abs(len(ts) - len(s_ts)) <
557 |                 5:
558 |                     cpu_total = float(snap.get('cpu_total_percent', 0.0) or 0.0)
559 |                     if cpu_total > 50.0:
560 |                         cpu_val = cpu_total
561 |                         notes.append(f'Повышенная загрузка CPU
562 | системы в момент события: {cpu_total:.1f}%')

```

```
561 |             d_write = float(snap.get('disk_write_bytes
| _sec', 0.0) or 0.0)
562 |             if d_write > 50 * 1024 * 1024:
563 |                 disk_spike = True
564 |                 notes.append(f'Всплеск дисковой записи
| : {d_write / 1024 / 1024:.1f} МБ/с')
565 |
566 |             n_recv = float(snap.get('network_recv_byte
| s_sec', 0.0) or 0.0)
567 |             if n_recv > 10 * 1024 * 1024:
568 |                 net_spike = True
569 |                 notes.append(f'Всплеск сетевого трафик
| а: {n_recv / 1024 / 1024:.1f} МБ/с')
570 |                 break
571 |
572 |         if eid == 4688:
573 |             ev_type = 'ProcessCreate'
574 |             if parent_short and ('powershell' in parent_sh
| ort.lower() or 'cmd' in parent_short.lower()):
575 |                 notes.append(f'Запуск из интерактивной обо
| лочки {parent_short}')
576 |                 if 'admin' in u_name.lower() or ev.get('elevat
| ed_token') == 1:
577 |                     notes.append('Запуск с повышенными привиле
| гиями (Elevated Token)')
578 |             elif eid == 4624:
579 |                 ev_type = 'UserLogon'
580 |                 ltype = ev.get('logon_type')
581 |                 if ltype == 10:
582 |                     notes.append('Удаленный вход по протоколу
| RDP (LogonType 10)')
583 |                 elif ltype == 2:
584 |                     notes.append('Локальный интерактивный вход
| (LogonType 2)')
585 |                 elif ltype == 3:
586 |                     notes.append('Сетевой вход SMB/RPC (LogonT
| ype 3)')
587 |             elif eid == 4625:
```

```

588         ev_type = 'FailedLogon'
589         notes.append(f"Неудачная попытка входа: {ev.get(
    t('status_code', '')}}")
590         elif eid == 4697:
591             ev_type = 'ServiceInstall'
592             notes.append(f"Установка новой службы {ev.get(
    'object_name', '')}}")
593         else:
594             ev_type = f'Event_{eid}'
595
596         correlations.append(
597             SecurityCorrelationItem(
598                 timestamp=ts,
599                 event_type=ev_type,
600                 user=u_name,
601                 process_name=p_short or p_name,
602                 pid=p_pid,
603                 parent_process_name=parent_short or parent
    _name,
604                 command_line=cmd,
605                 cpu_percent=cpu_val,
606                 ram_mb=ram_val,
607                 disk_spike=disk_spike,
608                 network_spike=net_spike,
609                 notes=notes,
610             )
611         )
612
613         return correlations
614
615
616 __all__ = [
617     'WindowsSecurityCollector',
618     'DEFAULT_SECURITY_EVENT_IDS',
619     'DEFAULT_DEFENDER_EVENT_IDS',
620     'DEFENDER_CHANNEL',
621 ]
622

```

security_normalizer.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - Security Normalizer
4 # =====
5 # Description:
6 #   Нормализатор событий журнала безопасности Windows Security Event Log.
7 #   Извлекает и структурирует атрибуты пользователей, процессов, сессий и политик.
8 #
9 # Usage Examples:
10 #   Python API:
11 #       from apps.windows.telemetry.security_normalizer import SecurityEventNormalizer
12 #
13 #       normalizer = SecurityEventNormalizer()
14 #       item, raw = normalizer.normalize_event(event_dict)
15 #
16 # File: security_normalizer.py
17 # Project: ai-breadboard
18 # Package: apps.windows.telemetry
19 # Author: hypo69
20 # Copyright: © 2026 hypo69
21 # Updated: 2026-10-10 05:40:00
22 # =====
23
24 from __future__ import annotations
25 """Нормализатор событий журнала безопасности Windows Security Event Log и Microsoft Defender."""
26
27 import json
28 import time
```

```

29 import xml.etree.ElementTree as ET
30 from datetime import datetime, timezone
31 from typing import Any, Dict, List, Optional, Tuple
32
33 try:
34     from logger import logger
35 except ImportError:
36     import logging
37     logger = logging.getLogger(__name__)
38
39 from .models import SecurityEventItem, SecurityEventRaw
40
41
42 def _parse_int(val: Any, default: int = 0) -> int:
43     """Безопасный парсинг целого числа из строки, включая шест
    надцатеричные значения (0x...)."""
44     if val is None:
45         return default
46     if isinstance(val, int):
47         return val
48     s = str(val).strip()
49     if not s or s == '-':
50         return default
51     try:
52         if s.startswith(('0x', '0X')):
53             return int(s, 16)
54         return int(s)
55     except ValueError:
56         try:
57             return int(float(s))
58         except ValueError:
59             return default
60
61
62 class SecurityEventNormalizer:
63     """Нормализатор сырых XML и словарей событий Windows Secur
    ity и Defender в типизированные модели."""
64

```

```

65 |     EVENT_DESCRIPTIONS = {
66 |         # Windows Security Event Log (Security.evtx)
67 |         4624: 'Успешный вход пользователя в систему',
68 |         4625: 'Неудачная попытка входа в систему',
69 |         4634: 'Завершение сеанса пользователя (Logoff)',
70 |         4647: 'Пользователь инициировал выход из системы',
71 |         4672: 'Назначение специальных привилегий администратор
    | а',
72 |         4688: 'Создание нового процесса',
73 |         4689: 'Завершение процесса',
74 |         4697: 'Установка новой системной службы',
75 |         4698: 'Создание запланированной задачи (Task Scheduler
    | )',
76 |         4702: 'Изменение параметров запланированной задачи',
77 |         4719: 'Изменение системной политики аудита безопасност
    | и',
78 |         4720: 'Создание учетной записи пользователя',
79 |         4722: 'Включение учетной записи пользователя',
80 |         4723: 'Попытка изменения пароля пользователя',
81 |         4724: 'Сброс пароля учетной записи пользователя',
82 |         4725: 'Отключение учетной записи пользователя',
83 |         4726: 'Удаление учетной записи пользователя',
84 |         4732: 'Добавление пользователя в локальную группу безо
    | пасности',
85 |         4738: 'Изменение параметров учетной записи пользовател
    | я',
86 |         4740: 'Блокировка учетной записи пользователя',
87 |         4768: 'Запрос билета Kerberos TGT (Authentication Tick
    | et)',
88 |         4769: 'Запрос сервисного билета Kerberos (Service Tick
    | et)',
89 |         4771: 'Сбой предварительной проверки подлинности Kerbe
    | ros',
90 |         4776: 'Проверка учетных данных контроллером (NTLM/MSV1
    | _0)',
91 |         1102: 'ВНИМАНИЕ: Журнал аудита безопасности был очищен
    | ',
92 |

```

93 # Microsoft-Windows-Windows Defender/Operational
94 1000: 'Запуск сканирования Microsoft Defender',
95 1001: 'Сканирование Microsoft Defender успешно заверше
но',
96 1002: 'Сканирование Microsoft Defender отменено пользо
вателем или системой',
97 1005: 'Сканирование Microsoft Defender приостановлено'
,
98 1013: 'Очистка истории обнаруженных угроз Defender',
99 1116: 'ВНИМАНИЕ: Microsoft Defender обнаружил вредонос
ную программу / угрозу',
100 1117: 'Microsoft Defender выполнил действие по нейтрал
изации угрозы',
101 1118: 'СБОЙ: Microsoft Defender не смог выполнить дейс
твие над угрозой',
102 1119: 'Критическая ошибка при обработке угрозы Microso
ft Defender',
103 1121: 'CFA (Controlled Folder Access) заблокировал нес
анкционированное изменение файла',
104 1122: 'CFA (Controlled Folder Access) зафиксировал ауд
ит изменения файла',
105 1123: 'Правило ASR (Attack Surface Reduction) заблокир
овало потенциально опасное действие',
106 1124: 'Правило ASR (Attack Surface Reduction) сработал
о в режиме аудита',
107 1127: 'Сетевая защита Microsoft Defender заблокировала
сетевое соединение',
108 1128: 'Сетевая защита Microsoft Defender зафиксировала
аудит сетевого соединения',
109 1150: 'Служба защиты Microsoft Defender работает в шта
тном режиме (Healthy)',
110 1151: 'Отчет о состоянии и работоспособности компонент
ов Microsoft Defender',
111 2000: 'Антивирусные сигнатуры Microsoft Defender успеш
но обновлены',
112 2001: 'Ошибка обновления антивирусных сигнатур Microso
ft Defender',
113 2010: 'Microsoft Defender использовал облачную защиту


```

    для получения сведений о безопасности',
114         5000: 'Защита в реальном времени Microsoft Defender вк
    лючена',
115         5001: 'ВНИМАНИЕ: Защита в реальном времени Microsoft D
    efender отключена',
116         5004: 'Изменена конфигурация антивируса Microsoft Defe
    nder',
117         5007: 'Изменены параметры защиты Microsoft Defender',
118     }
119
120     def normalize_event(
121         self,
122         event_dict: Dict[str, Any],
123         extract_raw: bool = True,
124     ) -> Tuple[SecurityEventItem, Optional[SecurityEventRaw]]:
125         """Нормализовать срез события безопасности в типизиров
    анную структуру.
126
127         Args:
128             event_dict: Словарь события от WevtAPI.
129             extract_raw: Формировать ли параллельно объект сыр
    ого XML.
130
131         Returns:
132             Tuple[SecurityEventItem, Optional[SecurityEventRaw
    ]]:
133             Кортеж нормализованного события и опциональног
    о сырого объекта.
134         """
135         event_id = _parse_int(event_dict.get('event_id', 0))
136         record_id = _parse_int(event_dict.get('record_id', 0))
137         timestamp = str(event_dict.get('timestamp', '') or '')
138         computer = str(event_dict.get('computer', '') or '')
139         channel = str(event_dict.get('channel', 'Security') or
    'Security')
140         level = str(event_dict.get('level', 'Information') or
    'Information')
141         raw_xml = str(event_dict.get('raw_data', '') or '')

```

```
142
143     # Получаем данные EventData
144     ed = event_dict.get('event_data', {})
145     if not ed and raw_xml:
146         ed = self._parse_event_data_xml(raw_xml)
147
148     now_iso = datetime.now(timezone.utc).isoformat()
149     now_epoch = time.time()
150
151     # Извлечение общих субъектов
152     subject_user = ed.get('SubjectUserName') or ed.get('User
erName') or ''
153     subject_domain = ed.get('SubjectDomainName') or ed.get
('UserDomain') or ''
154     subject_sid = ed.get('SubjectUserSid') or ed.get('User
Sid') or ''
155
156     target_user = ed.get('TargetUserName') or ed.get('Memb
erName') or ''
157     target_domain = ed.get('TargetDomainName') or ''
158     target_sid = ed.get('TargetUserSid') or ed.get('Member
Sid') or ''
159
160     # Извлечение процессов
161     process_id = 0
162     process_name = ''
163     parent_process_id = 0
164     parent_process_name = ''
165     command_line = ''
166
167     if event_id == 4688:
168         process_id = _parse_int(ed.get('NewProcessId') or
ed.get('ProcessId', 0))
169         process_name = str(ed.get('NewProcessName') or '')
170         parent_process_id = _parse_int(ed.get('ProcessId')
or ed.get('ParentProcessId', 0))
171         parent_process_name = str(ed.get('ParentProcessNam
e') or '')
```

```

172         command_line = str(ed.get('CommandLine') or '')
173     elif event_id == 4689:
174         process_id = _parse_int(ed.get('ProcessId', 0))
175         process_name = str(ed.get('ProcessName') or '')
176     else:
177         process_id = _parse_int(ed.get('ProcessId', 0))
178         process_name = str(ed.get('ProcessName') or '')
179
180     # Logon сессия и сеть
181     logon_id = str(ed.get('TargetLogonId') or ed.get('SubjectLogonId') or '')
182     logon_type_val = ed.get('LogonType')
183     logon_type = _parse_int(logon_type_val, -1) if logon_type_val is not None else None
184     if logon_type == -1:
185         logon_type = None
186
187     elevated_token_val = ed.get('ElevatedToken') or ed.get('TokenElevationType')
188     elevated_token = _parse_int(elevated_token_val, -1) if elevated_token_val is not None else None
189     if elevated_token == -1:
190         elevated_token = None
191
192     source_ip = str(ed.get('IpAddress') or ed.get('WorkstationName') or '')
193     if source_ip == '-':
194         source_ip = ''
195     source_port_val = ed.get('IpPort')
196     source_port = _parse_int(source_port_val, -1) if source_port_val is not None else None
197     if source_port == -1:
198         source_port = None
199
200     # Объект и статусы
201     object_name = (
202         ed.get('ObjectName')
203         or ed.get('ServiceName')

```

```

204         or ed.get('TaskName')
205         or ed.get('TargetName')
206         or ''
207     )
208     status_code = str(ed.get('Status') or ed.get('SubStatus') or ed.get('ExitStatus') or '')
209
210     # Извлечение специфичных атрибутов Microsoft Defender
211     if 'Defender' in channel or 'Threat Name' in ed or 'ThreatName' in ed or event_id in range(1000, 1160) or event_id in range(2000, 2020) or event_id in range(5000, 5020):
212         threat_name = ed.get('Threat Name') or ed.get('ThreatName') or ''
213         threat_path = ed.get('Path') or ed.get('ThreatPath') or ''
214         p_name_def = ed.get('Process Name') or ed.get('ProcessName') or ''
215         def_user = ed.get('Detection User') or ed.get('User') or ''
216         severity = ed.get('Severity Name') or ed.get('SeverityName') or ''
217         action_name = ed.get('Action Name') or ed.get('ActionName') or ''
218         sig_ver = ed.get('Security intelligence Version') or ed.get('Signature Version') or ''
219
220         if threat_name and not object_name:
221             object_name = threat_name
222         if threat_path and not command_line:
223             command_line = threat_path
224         if p_name_def and not process_name:
225             process_name = p_name_def
226         if def_user and not subject_user:
227             subject_user = def_user
228         if severity and not status_code:
229             status_code = f"Severity:{severity}"
230         if action_name and 'Action:' not in status_code:
231             status_code = f"{status_code} Action:{action_n

```

```

    ame}".strip()
232         if sig_ver and not object_name:
233             object_name = f"Sig:{sig_ver}"
234
235     # Формирование русскоязычного сообщения
236     message = self._build_human_message(
237         event_id=event_id,
238         subject_user=subject_user,
239         target_user=target_user,
240         process_name=process_name,
241         process_id=process_id,
242         parent_process_name=parent_process_name,
243         parent_process_id=parent_process_id,
244         command_line=command_line,
245         logon_type=logon_type,
246         source_ip=source_ip,
247         object_name=object_name,
248         status_code=status_code,
249         default_msg=event_dict.get('message', ''),
250     )
251
252     item = SecurityEventItem(
253         event_record_id=record_id,
254         event_id=event_id,
255         timestamp=timestamp,
256         created_at=now_epoch,
257         computer=computer,
258         channel=channel,
259         level=level,
260         subject_user=subject_user,
261         subject_domain=subject_domain,
262         subject_sid=subject_sid,
263         target_user=target_user,
264         target_domain=target_domain,
265         target_sid=target_sid,
266         process_id=process_id,
267         process_name=process_name,
268         parent_process_id=parent_process_id,

```

```
269         parent_process_name=parent_process_name,
270         command_line=command_line,
271         logon_id=logon_id,
272         logon_type=logon_type,
273         elevated_token=elevated_token,
274         source_ip=source_ip,
275         source_port=source_port,
276         object_name=object_name,
277         status_code=status_code,
278         message=message,
279         event_data=ed,
280         ingested_at=now_iso,
281     )
282
283     raw_item = None
284     if extract_raw and raw_xml:
285         raw_item = SecurityEventRaw(
286             event_record_id=record_id,
287             event_id=event_id,
288             timestamp=timestamp,
289             created_at=now_epoch,
290             channel=channel,
291             raw_xml=raw_xml,
292             ingested_at=now_iso,
293         )
294
295     return item, raw_item
296
297     def _build_human_message(
298         self,
299         event_id: int,
300         subject_user: str,
301         target_user: str,
302         process_name: str,
303         process_id: int,
304         parent_process_name: str,
305         parent_process_id: int,
306         command_line: str,
```

```

307         logon_type: Optional[int],
308         source_ip: str,
309         object_name: str,
310         status_code: str,
311         default_msg: str = '',
312     ) -> str:
313         """Сформировать понятное русскоязычное описание события."""
314         p_short = process_name.replace('\\', '/').split('/')[-1] if process_name else ''
315         parent_short = parent_process_name.replace('\\', '/').split('/')[-1] if parent_process_name else ''
316
317         if event_id == 4688:
318             user_part = f"пользователем {subject_user}" if subject_user and subject_user != '-' else ""
319             parent_part = f", родитель: {parent_short} (PID {parent_process_id})" if parent_short else ""
320             cmd_part = f" [{command_line[:120]}...]" if len(command_line) > 120 else (f" [{command_line}]" if command_line else "")
321             return f"Создание процесса: {p_short} (PID {process_id}) {user_part}{parent_part}{cmd_part}".strip()
322
323         if event_id == 4689:
324             status_part = f", код завершения {status_code}" if status_code else ""
325             return f"Завершение процесса: {p_short} (PID {process_id}){status_part}"
326
327         if event_id == 4624:
328             user = target_user or subject_user or 'Неизвестно'
329             type_str = f", тип входа {logon_type}" if logon_type is not None else ""
330             ip_str = f", IP: {source_ip}" if source_ip else ""
331             return f"Успешный вход пользователя: {user}{type_str}{ip_str}"
332

```

```
333         if event_id == 4625:
334             user = target_user or subject_user or 'Неизвестно'
335             status_str = f", статус ошибки {status_code}" if s
|tatus_code else ""
336             ip_str = f", IP: {source_ip}" if source_ip else ""
337             return f"Неудачная попытка входа: {user}{status_st
|r}{ip_str}"
338
339         if event_id in (4634, 4647):
340             user = target_user or subject_user or 'Пользовател
|ь'
341             return f"Выход пользователя из системы: {user}"
342
343         if event_id == 4672:
344             return f"Назначение привилегий администратора поль
|зователю {subject_user}"
345
346         if event_id == 4697:
347             return f"Установка новой службы Windows: {object_n
|ame} ({p_short})"
348
349         if event_id == 4698:
350             return f"Создание запланированной задачи: {object_
|name} пользователем {subject_user}"
351
352         if event_id == 4702:
353             return f"Обновление запланированной задачи: {objec
|t_name}"
354
355         if event_id == 4719:
356             return f"Изменение системной политики аудита безоп
|асности администратором {subject_user}"
357
358         if event_id == 4720:
359             return f"Создание учетной записи пользователя: {ta
|rget_user} (инициатор: {subject_user})"
360
361         if event_id == 4722:
```



```
362         return f"Включение учетной записи: {target_user}"
363
364     if event_id in (4723, 4724):
365         return f"Сброс/изменение пароля пользователя: {target_user}"
366
367     if event_id == 4725:
368         return f"Отключение учетной записи пользователя: {target_user}"
369
370     if event_id == 4726:
371         return f"Удаление учетной записи пользователя: {target_user}"
372
373     if event_id == 4732:
374         return f"Добавление пользователя {target_user} в группу безопасности {object_name}"
375
376     if event_id == 4740:
377         return f"Блокировка учетной записи пользователя: {target_user}"
378
379     if event_id == 4768:
380         return f"Запрос Kerberos TGT для {target_user or subject_user}"
381
382     if event_id == 4771:
383         return f"Сбой пре-аутентификации Kerberos для {target_user} (Код: {status_code})"
384
385     if event_id == 4776:
386         return f"Проверка учетных данных NTLM для {target_user}"
387
388     if event_id == 1102:
389         return f"ВНИМАНИЕ: Журнал аудита безопасности был очищен пользователем {subject_user}!"
390
```

```

391 |         # События Microsoft Defender
392 |         if event_id == 1116:
393 |             threat = object_name or 'Вредоносная программа'
394 |             target = f" в '{command_line or process_name}'" if
| (command_line or process_name) else ""
395 |             status_p = f" [{status_code}]" if status_code else
| ""
396 |             return f"␣ Обнаружена угроза Defender: {threat}{t
| arget}{status_p}"
397 |
398 |         if event_id == 1117:
399 |             threat = object_name or 'Угроза'
400 |             action = status_code or 'Обезврежено'
401 |             return f"␣ Угроза нейтрализована Defender: {threa
| t} ({action})"
402 |
403 |         if event_id == 1118:
404 |             threat = object_name or 'Угроза'
405 |             return f"␣ Ошибка нейтрализации угрозы Defender: {
| threat} ({status_code})"
406 |
407 |         if event_id in (1121, 1122):
408 |             mode_str = "заблокировано" if event_id == 1121 els
| e "аудит"
409 |             proc = p_short or 'Процесс'
410 |             target = f" -> {command_line}" if command_line els
| e ""
411 |             return f"␣ Защита папок от шифровальщиков CFA ({m
| ode_str}): {proc}{target}"
412 |
413 |         if event_id in (1123, 1124):
414 |             mode_str = "блокировка" if event_id == 1123 else "
| аудит"
415 |             proc = p_short or 'Процесс'
416 |             return f"␣ Срабатывание правила ASR ({mode_str}):
| {proc} [{object_name}]"
417 |
418 |         if event_id in (1127, 1128):

```

```
419 |         mode_str = "заблокировано" if event_id == 1127 els
    | e "аудит"
420 |         return f"□ Сетевая защита Defender ({mode_str}):
    | соединение {source_ip or command_line}"
421 |
422 |         if event_id == 1150:
423 |             return f"□ Служба Microsoft Defender здорова и акт
    | ивна (Healthy) [{object_name or 'OK'}]"
424 |
425 |         if event_id == 1151:
426 |             return "i Отчет о работоспособности компонентов Mi
    | crosoft Defender"
427 |
428 |         if event_id in (2000, 2010):
429 |             return f"□ Обновление аналитики безопасности / си
    | гнатур Defender [{object_name or 'Успешно'}]"
430 |
431 |         if event_id == 2001:
432 |             return f"□ Ошибка обновления сигнатур Defender ({s
    | tatus_code})"
433 |
434 |         if event_id == 5000:
435 |             return "□ Защита в реальном времени Microsoft Defe
    | nder включена"
436 |
437 |         if event_id == 5001:
438 |             return "□ ВНИМАНИЕ: Защита в реальном времени Mic
    | rosoft Defender была отключена!"
439 |
440 |         if event_id in (5004, 5007):
441 |             return f"□ Изменена конфигурация/параметры Microso
    | ft Defender ({object_name or status_code})"
442 |
443 |         if event_id in (1000, 1001, 1002, 1005):
444 |             scan_desc = self.EVENT_DESCRIPTIONS.get(event_id,
    | f"Сканирование Defender ({event_id})")
445 |             return f"□ {scan_desc}"
446 |
```

```

447         if default_msg:
448             return default_msg
449
450         desc = self.EVENT_DESCRIPTIONS.get(event_id, f'Событие
безопасности {event_id}')
451         return f"{desc} (субъект: {subject_user or target_user
or '-'})"
452
453     def _parse_event_data_xml(self, raw_xml: str) -> Dict[str,
str]:
454         """Извлечь структурированные пары ключ-значение из Eve
ntData XML."""
455         res: Dict[str, str] = {}
456         try:
457             root = ET.fromstring(raw_xml)
458             for elem in root.iter():
459                 tag = elem.tag.split('}')[1]
460                 if tag in ('Data', 'string', 'Value') and elem
.text:
461                     name_attr = elem.attrib.get('Name')
462                     if name_attr:
463                         res[name_attr] = elem.text.strip()
464         except Exception:
465             pass
466         return res
467
468
469 __all__ = ['SecurityEventNormalizer', '_parse_int']
470

```

sensors.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - Sensors
4 # =====
5 # Description:
6 #   Движок считывания показаний температурных датчиков, напряж
   ений и вентиляторов.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.sensors import get_hardware_
   sensors
11 #
12 #       res = get_hardware_sensors()
13 #
14 # File: sensors.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-04 07:00:00
20 # =====
21
22 from __future__ import annotations
23 """Движок считывания показаний аппаратных сенсоров, сетевых ме
   трик и скорости FAST.com."""
24
25 import json
26 import os
27 import subprocess
28 from typing import List, Optional
29 try:
30     import wmi
```

```

31 |     _WMI_AVAILABLE = True
32 | except ImportError:
33 |     _WMI_AVAILABLE = False
34 | from logger import logger
35 | from .models import HardwareSensor
36 | from .internet_speed import InternetSpeedSensor
37 | from .sensor_registry import SensorProvider
38 |
39 | # NOTE: GPU temperature, power, and fan sensors are sourced fr
    om:
40 | # 1. GpuProber._probe_nvidia() -> HardwareMonitor.get_gpu_me
    trics()
41 | # 2. sensor_collector.extract_sensor_readings() normalizes G
    puMetrics to sensor_id format
42 |
43 | def _probe_acpi_thermal_zones() -> List[HardwareSensor]:
44 |     """Сбор данных температурных зон Windows ACPI через CIM с
    fallback на WMI.
45 |
46 |     Returns:
47 |         List[HardwareSensor]: Список показаний термальных зон
    ACPI.
48 |     """
49 |     sensors: List[HardwareSensor] = []
50 |     if os.name != 'nt':
51 |         return sensors
52 |
53 |     # 1. Первичный сбор через CIM (Get-CimInstance)
54 |     try:
55 |         cmd = [
56 |             'powershell.exe', '-NoProfile', '-NonInteractive',
57 |             '-Command',
58 |             'Get-CimInstance -Namespace root/wmi -ClassName MS
    Acpi_ThermalZoneTemperature -ErrorAction SilentlyContinue | '
59 |             'Select-Object InstanceName, CurrentTemperature |
    ConvertTo-Json -Compress'
60 |         ]
61 |         res = subprocess.run(cmd, capture_output=True, text=Tr

```

```

ue, timeout=5)
61         if res.returncode == 0 and res.stdout.strip():
62             raw_data = json.loads(res.stdout.strip())
63             items = [raw_data] if isinstance(raw_data, dict) e
lse (raw_data if isinstance(raw_data, list) else [])
64             for idx, zone in enumerate(items):
65                 raw_temp = zone.get('CurrentTemperature')
66                 if raw_temp and raw_temp > 0:
67                     celsius = round(float(raw_temp) / 10.0 - 2
73.15, 1)
68                     if -20.0 <= celsius <= 125.0:
69                         zone_name = zone.get('InstanceName') o
r f'Thermal Zone {idx}'
70                         sensor = HardwareSensor(
71                             sensor_id=f'acpi_thermal_{idx}',
72                             name=f'ACPI {zone_name}',
73                             category='temperature',
74                             value=celsius,
75                             unit='°C'
76                         )
77                         sensor.provider = SensorProvider.CIM_S
YSTEM
78                         sensors.append(sensor)
79                     if sensors:
80                         return sensors
81             except Exception as ex:
82                 logger.debug(f'CIM ACPI thermal probe skipped or error
: {ex}')
83
84     # 2. Fallback / слой совместимости через WMI
85     try:
86         import wmi
87         import pythoncom
88         pythoncom.CoInitialize()
89         w = wmi.WMI(namespace='root\\wmi')
90         for idx, zone in enumerate(w.MSAcpi_ThermalZoneTempera
ture()):
91             raw_temp = getattr(zone, 'CurrentTemperature', Non

```

```

e)
92         if raw_temp and raw_temp > 0:
93             celsius = round(float(raw_temp) / 10.0 - 273.1
5, 1)
94             if -20.0 <= celsius <= 125.0:
95                 zone_name = getattr(zone, 'InstanceName',
f'Thermal Zone {idx}')
96                 sensor = HardwareSensor(
97                     sensor_id=f'acpi_thermal_{idx}',
98                     name=f'ACPI {zone_name}',
99                     category='temperature',
100                     value=celsius,
101                     unit='°C'
102                 )
103                 sensor.provider = SensorProvider.WMI_FALLB
ACK
104                 sensors.append(sensor)
105     except Exception as ex:
106         logger.debug(f'WMI ACPI thermal fallback probe skipped
: {ex}')
107
108     return sensors
109
110 def _probe_network_sensors() -> List[HardwareSensor]:
111     """Probe network interface metrics using Windows native to
ols.
112
113     Returns:
114         List[HardwareSensor]: Network interface telemetry as s
ensors with provider marker.
115     """
116     sensors: List[HardwareSensor] = []
117     try:
118         import psutil
119         current_net_io = psutil.net_io_counters(pernic=True)
120         if current_net_io:
121             for name, stats in current_net_io.items():
122                 # Create sensors with provider marker

```



```
123         s1 = HardwareSensor(sensor_id=f'net_{name}_byt
    es_recv', name=f'Network {name} Bytes Received', category='net
    work', value=round(stats.bytes_recv, 2), unit='B')
124         s1.provider = SensorProvider.NETWORK_SENSOR
125         sensors.append(s1)
126
127         s2 = HardwareSensor(sensor_id=f'net_{name}_byt
    es_sent', name=f'Network {name} Bytes Sent', category='network
    ', value=round(stats.bytes_sent, 2), unit='B')
128         s2.provider = SensorProvider.NETWORK_SENSOR
129         sensors.append(s2)
130
131         s3 = HardwareSensor(sensor_id=f'net_{name}_pac
    kets_recv', name=f'Network {name} Packets Received', category=
    'network', value=stats.packets_recv, unit='pkts')
132         s3.provider = SensorProvider.NETWORK_SENSOR
133         sensors.append(s3)
134
135         s4 = HardwareSensor(sensor_id=f'net_{name}_pac
    kets_sent', name=f'Network {name} Packets Sent', category='net
    work', value=stats.packets_sent, unit='pkts')
136         s4.provider = SensorProvider.NETWORK_SENSOR
137         sensors.append(s4)
138
139         s5 = HardwareSensor(sensor_id=f'net_{name}_err
    ors_recv', name=f'Network {name} Errors Received', category='n
    etwork', value=stats.errin, unit='errors')
140         s5.provider = SensorProvider.NETWORK_SENSOR
141         sensors.append(s5)
142
143         s6 = HardwareSensor(sensor_id=f'net_{name}_err
    ors_sent', name=f'Network {name} Errors Sent', category='netwo
    rk', value=stats.errout, unit='errors')
144         s6.provider = SensorProvider.NETWORK_SENSOR
145         sensors.append(s6)
146
147         s7 = HardwareSensor(sensor_id=f'net_{name}_dro
    pped_recv', name=f'Network {name} Dropped Received', category=
```

```

    'network', value=stats.dropin, unit='pkts')
148         s7.provider = SensorProvider.NETWORK_SENSOR
149         sensors.append(s7)
150
151         s8 = HardwareSensor(sensor_id=f'net_{name}_dro
pped_sent', name=f'Network {name} Dropped Sent', category='net
work', value=stats.dropout, unit='pkts')
152         s8.provider = SensorProvider.NETWORK_SENSOR
153         sensors.append(s8)
154     except Exception as ex:
155         logger.debug(f'Network sensor probe failed: {ex}')
156     return sensors
157
158 def _probe_internet_speed_sensors() -> List[HardwareSensor]:
159     """Собрать показания скорости интернета и характеристик се
ти FAST.com.
160
161     Returns:
162         List[HardwareSensor]: Метрики скорости интернета и зад
ержки с маркером провайдера.
163     """
164     sensors: List[HardwareSensor] = []
165     try:
166         sensor = InternetSpeedSensor()
167         metrics = sensor.measure_internet_speed()
168
169         s1 = HardwareSensor(
170             sensor_id='internet_ping',
171             name='Internet Ping',
172             category='network',
173             value=round(metrics.get('ping_ms', 0.0), 2),
174             unit='ms'
175         )
176         s1.provider = SensorProvider.INTERNET_SPEED
177         sensors.append(s1)
178
179         s2 = HardwareSensor(
180             sensor_id='internet_download',

```

```

181         name='Internet Download Speed',
182         category='network',
183         value=round(metrics.get('download_mbps', 0.0), 2),
184         unit='Mbps'
185     )
186     s2.provider = SensorProvider.INTERNET_SPEED
187     sensors.append(s2)
188
189     s3 = HardwareSensor(
190         sensor_id='internet_upload',
191         name='Internet Upload Speed',
192         category='network',
193         value=round(metrics.get('upload_mbps', 0.0), 2),
194         unit='Mbps'
195     )
196     s3.provider = SensorProvider.INTERNET_SPEED
197     sensors.append(s3)
198
199     s4 = HardwareSensor(
200         sensor_id='internet_bufferbloat',
201         name='Internet Bufferbloat',
202         category='network',
203         value=round(metrics.get('bufferbloat_ms', 0.0), 2)
204         ,
205         unit='ms'
206     )
207     s4.provider = SensorProvider.INTERNET_SPEED
208     sensors.append(s4)
209
210     s5 = HardwareSensor(
211         sensor_id='internet_latency_loaded',
212         name='Internet Loaded Latency',
213         category='network',
214         value=round(metrics.get('latency_loaded_ms', 0.0),
215         2),
216         unit='ms'
217     )
218     s5.provider = SensorProvider.INTERNET_SPEED

```

```

217         sensors.append(s5)
218     except Exception as ex:
219         logger.debug(f'Ошибка опроса сенсора скорости интернет
a: {ex}')
220     return sensors
221
222 def _probe_storage_sensors() -> List[HardwareSensor]:
223     """Сбор сенсоров температуры и надежности накопителей.
224
225     :return: Список объектов HardwareSensor для дисков с provi
der marker.
226     """
227     sensors: List[HardwareSensor] = []
228     if os.name != 'nt':
229         return sensors
230     try:
231         from apps.windows.sdk.modules.storage_manager.core.win
dows_storage_sensor import WindowsStorageSensor
232         sensor = WindowsStorageSensor(timeout_sec=30)
233         disks = sensor.get_physical_disks()
234         for disk in disks:
235             disk_slug = disk.device_id.replace('\\', '_').repl
ace('.', '_')
236             if disk.temperature_c is not None:
237                 s1 = HardwareSensor(sensor_id=f'disk_temp_{dis
k_slug}', name=f'{disk.friendly_name} Temperature', category='
temperature', value=float(disk.temperature_c), unit='°C')
238                 s1.provider = SensorProvider.WINDOWS_STORAGE
239                 sensors.append(s1)
240             if disk.wear_percentage is not None:
241                 s2 = HardwareSensor(sensor_id=f'disk_wear_{dis
k_slug}', name=f'{disk.friendly_name} Wear', category='wear',
value=float(disk.wear_percentage), unit='%')
242                 s2.provider = SensorProvider.WINDOWS_STORAGE
243                 sensors.append(s2)
244     except Exception as ex:
245         logger.debug(f'Ошибка сбора сенсоров накопителей: {ex}
')

```

```

246     return sensors
247
248 def _probe_cloud_storage_sensors() -> List[HardwareSensor]:
249     """Сбор сенсоров свободного и занятого места для облачного
    хранилища OneDrive.
250
251     Returns:
252         List[HardwareSensor]: Список сенсоров для дисков OneDr
ive с provider marker.
253     """
254     import shutil
255     sensors: List[HardwareSensor] = []
256     od_paths: List[str] = []
257     for env_var in ('OneDrive', 'OneDriveConsumer', 'OneDriveC
ommerical'):
258         val = os.environ.get(env_var)
259         if val and os.path.exists(val) and (val not in od_path
s):
260             od_paths.append(val)
261     if not od_paths:
262         home = os.path.expanduser('~')
263         candidate = os.path.join(home, 'OneDrive')
264         if os.path.exists(candidate):
265             od_paths.append(candidate)
266     if os.name == 'nt':
267         try:
268             import winreg
269             for subkey in ('Software\\Microsoft\\OneDrive\\Acc
ounts\\Personal', 'Software\\Microsoft\\OneDrive\\Accounts\\Bu
siness1', 'Software\\Microsoft\\OneDrive'):
270                 try:
271                     with winreg.OpenKey(winreg.HKEY_CURRENT_US
ER, subkey) as k:
272                         val, _ = winreg.QueryValueEx(k, 'UserF
older')
273                         if val and os.path.exists(val) and (va
l not in od_paths):
274                             od_paths.append(val)

```

```

275         except OSError:
276             pass
277     except Exception:
278         pass
279     for idx, od_path in enumerate(od_paths):
280         try:
281             usage = shutil.disk_usage(od_path)
282             free_gb = round(usage.free / 1024 ** 3, 2)
283             total_gb = round(usage.total / 1024 ** 3, 2)
284             used_pct = round(usage.used / max(usage.total, 1)
* 100.0, 1)
285             slug = f'_{idx + 1}' if len(od_paths) > 1 else ''
286             label_suffix = f' ({os.path.basename(od_path)})' i
f len(od_paths) > 1 else ''
287
288             s1 = HardwareSensor(sensor_id=f'onedrive{slug}_fre
e_gb', name=f'OneDrive Free Space{label_suffix}', category='st
orage', value=free_gb, unit='GB')
289             s1.provider = SensorProvider.SENSOR_COLLECTOR
290             sensors.append(s1)
291
292             s2 = HardwareSensor(sensor_id=f'onedrive{slug}_use
d_percent', name=f'OneDrive Used Percent{label_suffix}', categ
ory='storage', value=used_pct, unit='%')
293             s2.provider = SensorProvider.SENSOR_COLLECTOR
294             sensors.append(s2)
295
296             s3 = HardwareSensor(sensor_id=f'onedrive{slug}_tot
al_gb', name=f'OneDrive Total Capacity{label_suffix}', categor
y='storage', value=total_gb, unit='GB')
297             s3.provider = SensorProvider.SENSOR_COLLECTOR
298             sensors.append(s3)
299         except Exception as ex:
300             logger.debug(f'Ошибка сбора сенсора для OneDrive (
{od_path}): {ex}')
301     return sensors
302
303 def get_hardware_sensors() -> List[HardwareSensor]:

```

```
304 |     """Retrieve all available hardware sensors across all supp
    | orted backends.
305 |
306 |     NOTE: GPU sensors (temperature, power, fan) are NOT includ
    | ed here to avoid
307 |     duplication. They are collected via GpuProber in HardwareM
    | onitor.get_gpu_metrics()
308 |     and then normalized by SensorCollector.extract_sensor_read
    | ings().
309 |
310 |     Returns:
311 |         List[HardwareSensor]: Consolidated list of active hard
    | ware sensors.
312 |     """
313 |     all_sensors: List[HardwareSensor] = []
314 |     # GPU sensors excluded - collected via GpuProber to avoid
    | duplicate nvidia-smi calls
315 |     all_sensors.extend(_probe_acpi_thermal_zones())
316 |     all_sensors.extend(_probe_storage_sensors())
317 |     all_sensors.extend(_probe_cloud_storage_sensors())
318 |     all_sensors.extend(_probe_network_sensors())
319 |     all_sensors.extend(_probe_internet_speed_sensors())
320 |     return all_sensors
```

sensor_collector.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - Sensor
  Collector
4 # =====
5 # Description:
6 #   Сбор данных со всех доступных сенсоров и аппаратных провай
  деров.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.sensor_collector import Sens
  orCollector
11 #
12 #       service = SensorCollector()
13 #
14 # File: sensor_collector.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-06 08:10:00
20 # =====
21
22 from __future__ import annotations
23 """Сбор данных со всех доступных сенсоров и аппаратных провай
  деров."""
24
25 import re
26 import threading
27 import time
28 from datetime import datetime, timezone
29 from typing import Any, Dict, List, Optional
```



```

30 try:
31     from logger import logger
32 except ImportError:
33     from logger import logger
34 from .telemetry_config import TelemetryConfigManager
35 from .sensor_registry import SensorDeduplicator, SensorProvide
36 r
37 try:
38     from apps.windows.sdk.modules.hardware.hardware_monitor im
39     port HardwareMonitor
40     from apps.windows.sdk.modules.hardware.lhm_service import
41     LhmService
42     from .internet_speed import InternetSpeedSensor
43 except ImportError as e:
44     logger.warning(f'Не удалось импортировать зависимости сенс
45     оров: {e}')
46     HardwareMonitor = None
47     LhmService = None
48     InternetSpeedSensor = None
49
50 class SensorCollector:
51     """Коллектор данных со всех сенсоров с нормализованной схе
52     мой датчиков."""
53
54     def __init__(
55         self,
56         config_manager: Optional[TelemetryConfigManager] = Non
57         e,
58         lhm_service: Optional[Any] = None,
59     ) -> None:
60         """Инициализирует коллектор сенсоров.
61
62         Args:
63             config_manager: Менеджер конфигурации для настроек
64             и сенсоров.
65             lhm_service: Сервис LibreHardwareMonitor (опционал
66             ьно).
67         """

```

```

60         self.config_manager = config_manager
61         self._lhm_service = lhm_service or (LhmService() if Lh
mService else None)
62         self._hardware_monitor = HardwareMonitor() if Hardware
Monitor else None
63         self._internet_speed_sensor = InternetSpeedSensor() if
InternetSpeedSensor else None
64         self._last_hardware_snapshot: Optional[Dict[str, Any]]
= None
65         self._lock = threading.Lock()
66
67     def _get_hardware_data(self) -> Dict[str, Any]:
68         """Получает данные от HardwareMonitor.
69
70         Returns:
71             Dict[str, Any]: Данные HardwareMonitor.
72         """
73         if self._hardware_monitor is None:
74             return {}
75         try:
76             snapshot = self._hardware_monitor.get_snapshot(inc
lude_smart=True)
77             return snapshot.to_dict()
78         except Exception as e:
79             logger.error(f'Ошибка получения данных от Hardware
Monitor: {e}')
80             return {}
81
82     def _get_internet_speed_data(self) -> Dict[str, Any]:
83         """Получает данные о скорости интернета и задержке чер
ез FAST.com.
84
85         Returns:
86             Dict[str, Any]: Данные скорости интернета (downloa
d, upload, ping, bufferbloat, loaded latency).
87         """
88         if self._internet_speed_sensor is None:
89             return {'available': False, 'error': 'InternetSpee

```

```

    dSensor not available'}
90         try:
91             metrics = self._internet_speed_sensor.measure_inte
internet_speed()
92             return {
93                 'available': metrics.get('available', True),
94                 'ping_ms': metrics.get('ping_ms', 0.0),
95                 'download_mbps': metrics.get('download_mbps',
0.0),
96                 'upload_mbps': metrics.get('upload_mbps', 0.0)
,
97                 'latency_unloaded_ms': metrics.get('latency_un
loaded_ms', metrics.get('ping_ms', 0.0)),
98                 'latency_loaded_ms': metrics.get('latency_load
ed_ms', 0.0),
99                 'bufferbloat_ms': metrics.get('bufferbloat_ms'
, 0.0),
100                 'provider': metrics.get('provider', 'fast.com'
),
101                 'server': metrics.get('server', ''),
102                 'ip': metrics.get('ip', ''),
103                 'isp': metrics.get('isp', ''),
104             }
105         except Exception as e:
106             logger.error(f'Ошибка получения данных скорости ин
тернета: {e}')
107             return {'available': False, 'error': str(e)}
108
109     def _detect_provider(self, sensor_id: str, category: str,
hardware_type: str) -> SensorProvider:
110         """Определяет провайдера сенсора на основе его ID, кат
егории и источника.
111
112         Args:
113             sensor_id: Идентификатор сенсора.
114             category: Категория сенсора (Temperatures, Load, e
tc.).
115             hardware_type: Тип оборудования (cpu, gpu, etc.).

```

```
116 |
117 |     Returns:
118 |         SensorProvider: Провайдер сенсора.
119 |     """
120 |     # GPU sensors from GpuProber
121 |     if sensor_id.startswith('gpu_') and ('_temp' in sensor
    | _id or '_load' in sensor_id):
122 |         return SensorProvider.GPU_PROBER
123 |
124 |     # Storage sensors
125 |     if sensor_id.startswith('disk_') or sensor_id.startswi
    | th('storage_'):
126 |         return SensorProvider.WINDOWS_STORAGE
127 |
128 |     # Internet speed sensors
129 |     if sensor_id.startswith('internet_'):
130 |         return SensorProvider.INTERNET_SPEED
131 |
132 |     # Network sensors
133 |     if sensor_id.startswith('net_'):
134 |         return SensorProvider.NETWORK_SENSOR
135 |
136 |     # CPU/RAM utilization from sensor_collector
137 |     if sensor_id in ('cpu_util_total', 'ram_util_pct', 'sw
    | ap_util_pct'):
138 |         return SensorProvider.SENSOR_COLLECTOR
139 |
140 |     if sensor_id.startswith('cpu_core_') and '_load' in se
    | nsor_id:
141 |         return SensorProvider.SENSOR_COLLECTOR
142 |
143 |     if sensor_id.startswith('cpu_freq'):
144 |         return SensorProvider.SENSOR_COLLECTOR
145 |
146 |     # OneDrive sensors
147 |     if sensor_id.startswith('onedrive_'):
148 |         return SensorProvider.SENSOR_COLLECTOR
149 |
```

```

150         # Network rates
151         if sensor_id in ('network_sent_rate', 'network_recv_ra
te'):
152             return SensorProvider.SENSOR_COLLECTOR
153
154         # Storage usage
155         if sensor_id.startswith('storage_usage_') or sensor_id
in ('storage_read_rate', 'storage_write_rate'):
156             return SensorProvider.SENSOR_COLLECTOR
157
158         # Default based on category
159         if category in ('Temperatures', 'temperature'):
160             return SensorProvider.ACPI_THERMAL
161
162         return SensorProvider.SENSOR_COLLECTOR
163
164     def extract_sensor_readings(
165         self, hardware_data: Dict[str, Any], *args: Any, inter
net_data: Optional[Dict[str, Any]] = None, **kwargs: Any
166     ) -> List[Dict[str, Any]]:
167         """Извлекает нормализованный список всех текущих показ
аний датчиков с дедупликацией.
168
169         Каждый элемент содержит: id, hardware_name, hardware_t
ype, sensor_category,
170         sensor_name, unit, value, _provider.
171
172         Args:
173             hardware_data: Данные HardwareMonitor.
174             *args: Опциональные extra_sensors_data и/или inter
net_data.
175             internet_data: Данные скорости интернета (если пер
еданы явно).
176
177         Returns:
178             List[Dict[str, Any]]: Нормализованный и дедуплицир
ованный список сенсоров.
179         """

```

```

180         extra_sensors_data = None
181         if len(args) == 1:
182             if isinstance(args[0], dict) and ("ping_ms" in args[0] or "download_mbps" in args[0] or "available" in args[0]):
183                 internet_data = args[0]
184             else:
185                 extra_sensors_data = args[0]
186         elif len(args) >= 2:
187             extra_sensors_data = args[0]
188             internet_data = args[1]
189
190         raw_readings: List[Dict[str, Any]] = []
191
192         # Дополнительные внешние сенсоры
193         if extra_sensors_data and isinstance(extra_sensors_data, dict) and "sensors" in extra_sensors_data:
194             for s in extra_sensors_data["sensors"]:
195                 if isinstance(s, dict):
196                     sensor_id = s.get("id", s.get("sensor_name", "sensor"))
197                     category = s.get("sensor_category", "Metric")
198                     hardware_type = s.get("hardware_type", "sensor")
199                     provider = self._detect_provider(str(sensor_id), category, hardware_type)
200                     raw_readings.append({
201                         "id": sensor_id,
202                         "hardware_name": s.get("hardware_name", "Hardware"),
203                         "hardware_type": hardware_type,
204                         "sensor_category": category,
205                         "sensor_name": s.get("sensor_name", "Sensor"),
206                         "unit": s.get("unit", ""),
207                         "value": s.get("value_num", s.get("value", 0.0)),
208                         "_provider": provider,

```

```

209         })
210
211         if 'cpu' in hardware_data:
212             cpu = hardware_data['cpu']
213             cpu_name = cpu.get('model') or cpu.get('model_name
214             ') or 'Intel Core Processor'
215             raw_readings.append({
216                 'id': 'cpu_util_total',
217                 'hardware_name': cpu_name,
218                 'hardware_type': 'cpu',
219                 'sensor_category': 'Load',
220                 'sensor_name': 'CPU Total',
221                 'unit': '%',
222                 'value': cpu.get('utilization_pct', 0.0),
223                 '_provider': SensorProvider.SENSOR_COLLECTOR
224             })
225             if 'per_core_pct' in cpu and isinstance(cpu['per_c
226             ore_pct'], list):
227                 for idx, core_val in enumerate(cpu['per_core_p
228                 ct']):
229                     raw_readings.append({
230                         'id': f'cpu_core_{idx}_load',
231                         'hardware_name': cpu_name,
232                         'hardware_type': 'cpu',
233                         'sensor_category': 'Load',
234                         'sensor_name': f'CPU Core #{idx}',
235                         'unit': '%',
236                         'value': core_val,
237                         '_provider': SensorProvider.SENSOR_COL
238                         LECTOR
239                     })
240             if 'frequency_current_mhz' in cpu and cpu['frequen
241             cy_current_mhz']:
242                 raw_readings.append({
243                     'id': 'cpu_freq_current',
244                     'hardware_name': cpu_name,
245                     'hardware_type': 'cpu',
246                     'sensor_category': 'Clocks',

```

```

242         'sensor_name': 'CPU Core Frequency',
243         'unit': 'MHz',
244         'value': cpu['frequency_current_mhz'],
245         '_provider': SensorProvider.SENSOR_COLLECT
OR
246     })
247
248     if 'memory' in hardware_data:
249         ram = hardware_data['memory']
250         raw_readings.append({
251             'id': 'ram_util_pct',
252             'hardware_name': 'System Memory',
253             'hardware_type': 'memory',
254             'sensor_category': 'Load',
255             'sensor_name': 'Memory Used',
256             'unit': '%',
257             'value': ram.get('utilization_pct', 0.0),
258             '_provider': SensorProvider.SENSOR_COLLECTOR
259         })
260         if 'swap_utilization_pct' in ram and ram['swap_utili
lization_pct'] is not None:
261             raw_readings.append({
262                 'id': 'swap_util_pct',
263                 'hardware_name': 'System Memory',
264                 'hardware_type': 'memory',
265                 'sensor_category': 'Load',
266                 'sensor_name': 'Swap Used',
267                 'unit': '%',
268                 'value': ram['swap_utilization_pct'],
269                 '_provider': SensorProvider.SENSOR_COLLECT
OR
270             })
271
272     # GPU metrics: temperature, load, memory, and subsystem engines
273     if 'gpus' in hardware_data and isinstance(hardware_data['gpus'], list):
274         for gpu in hardware_data['gpus']:

```



```

275 |         gpu_name = gpu.get('name', 'GPU')
276 |         gpu_idx = gpu.get('index', 0)
277 |         # GPU temperature - use hardware_data from Gpu
    | Prober
278 |         if 'temperature_gpu_c' in gpu and gpu['tempera
    | ture_gpu_c'] is not None:
279 |             raw_readings.append({
280 |                 'id': f'gpu_{gpu_idx}_temp',
281 |                 'hardware_name': gpu_name,
282 |                 'hardware_type': 'gpu',
283 |                 'sensor_category': 'Temperatures',
284 |                 'sensor_name': 'GPU Core Temperature',
285 |                 'unit': '°C',
286 |                 'value': gpu['temperature_gpu_c'],
287 |                 '_provider': SensorProvider.GPU_PROBER
288 |             })
289 |         # GPU load
290 |         if 'utilization_gpu_pct' in gpu and gpu['utili
    | zation_gpu_pct'] is not None:
291 |             raw_readings.append({
292 |                 'id': f'gpu_{gpu_idx}_load',
293 |                 'hardware_name': gpu_name,
294 |                 'hardware_type': 'gpu',
295 |                 'sensor_category': 'Load',
296 |                 'sensor_name': 'GPU Core Load',
297 |                 'unit': '%',
298 |                 'value': gpu['utilization_gpu_pct'],
299 |                 '_provider': SensorProvider.GPU_PROBER
300 |             })
301 |         # GPU VRAM / Memory
302 |         if 'memory_used_mb' in gpu and gpu['memory_use
    | d_mb'] is not None:
303 |             raw_readings.append({
304 |                 'id': f'gpu_{gpu_idx}_mem_used',
305 |                 'hardware_name': gpu_name,
306 |                 'hardware_type': 'gpu',
307 |                 'sensor_category': 'Data',
308 |                 'sensor_name': 'GPU Memory Used',

```

```

309         'unit': 'MB',
310         'value': float(gpu['memory_used_mb']),
311         '_provider': SensorProvider.GPU_PROBER
312     })
313     if 'memory_total_mb' in gpu and gpu['memory_to
tal_mb'] is not None:
314         raw_readings.append({
315             'id': f'gpu_{gpu_idx}_mem_total',
316             'hardware_name': gpu_name,
317             'hardware_type': 'gpu',
318             'sensor_category': 'Data',
319             'sensor_name': 'GPU Memory Total',
320             'unit': 'MB',
321             'value': float(gpu['memory_total_mb'])
322         },
323         '_provider': SensorProvider.GPU_PROBER
324     })
325     # GPU Subsystem Engines (3D, Video Decode, Vid
eo Processing, Copy, etc.)
326     engines = gpu.get('engines')
327     if engines and isinstance(engines, dict):
328         for eng_name, eng_val in engines.items():
329             clean_id = re.sub(r'^[a-zA-Z0-9]+', '_
', eng_name.lower()).strip('_')
330             raw_readings.append({
331                 'id': f'gpu_{gpu_idx}_eng_{clean_i
d}',
332                 'hardware_name': gpu_name,
333                 'hardware_type': 'gpu',
334                 'sensor_category': 'Load',
335                 'sensor_name': eng_name,
336                 'unit': '%',
337                 'value': float(eng_val),
338                 '_provider': SensorProvider.GPU_PR
OBER
339             })
340     if 'storage' in hardware_data:

```

```

341         storage = hardware_data['storage']
342         if 'partitions' in storage and isinstance(storage[
|'partitions'], list):
343             for part in storage['partitions']:
344                 dev = part.get('device', 'Disk')
345                 raw_readings.append({
346                     'id': f"storage_usage_{dev.replace(':', '
|, ' ').replace(chr(92), ' ')}",
347                     'hardware_name': f'Storage ({dev})',
348                     'hardware_type': 'storage',
349                     'sensor_category': 'Load',
350                     'sensor_name': f'Used Space {dev}',
351                     'unit': '%',
352                     'value': part.get('utilization_pct', 0
|.0),
353                     '_provider': SensorProvider.SENSOR_COL
|LECTOR
354                 })
355         if 'io_rates' in storage and storage['io_rates']:
356             io = storage['io_rates']
357             raw_readings.append({
358                 'id': 'storage_read_rate',
359                 'hardware_name': 'Storage',
360                 'hardware_type': 'storage',
361                 'sensor_category': 'Throughput',
362                 'sensor_name': 'Disk Read Rate',
363                 'unit': 'B/s',
364                 'value': io.get('read_bytes_sec', 0.0),
365                 '_provider': SensorProvider.SENSOR_COLLECT
|OR
366             })
367             raw_readings.append({
368                 'id': 'storage_write_rate',
369                 'hardware_name': 'Storage',
370                 'hardware_type': 'storage',
371                 'sensor_category': 'Throughput',
372                 'sensor_name': 'Disk Write Rate',
373                 'unit': 'B/s',

```

```

374         'value': io.get('write_bytes_sec', 0.0),
375         '_provider': SensorProvider.SENSOR_COLLECT
OR
376     })
377
378     # Network metrics: aggregated throughput rates
379     # Note: Detailed per-interface stats are collected by
    _probe_network_sensors() in sensors.py
380     if 'network' in hardware_data:
381         net = hardware_data['network']
382         raw_readings.append({
383             'id': 'network_sent_rate',
384             'hardware_name': 'Network',
385             'hardware_type': 'network',
386             'sensor_category': 'Throughput',
387             'sensor_name': 'Network Upload Speed',
388             'unit': 'B/s',
389             'value': net.get('bytes_sent_sec', 0.0),
390             '_provider': SensorProvider.SENSOR_COLLECTOR
391         })
392         raw_readings.append({
393             'id': 'network_recv_rate',
394             'hardware_name': 'Network',
395             'hardware_type': 'network',
396             'sensor_category': 'Throughput',
397             'sensor_name': 'Network Download Speed',
398             'unit': 'B/s',
399             'value': net.get('bytes_recv_sec', 0.0),
400             '_provider': SensorProvider.SENSOR_COLLECTOR
401         })
402
403     # Internet speed metrics
404     if internet_data and internet_data.get('available'):
405         raw_readings.append({
406             'id': 'internet_download_speed',
407             'hardware_name': 'Internet',
408             'hardware_type': 'network',
409             'sensor_category': 'Throughput',

```

```

410         'sensor_name': 'Internet Download Bandwidth',
411         'unit': 'Mbps',
412         'value': internet_data.get('download_mbps', 0.
0),
413         '_provider': SensorProvider.INTERNET_SPEED
414     })
415     raw_readings.append({
416         'id': 'internet_upload_speed',
417         'hardware_name': 'Internet',
418         'hardware_type': 'network',
419         'sensor_category': 'Throughput',
420         'sensor_name': 'Internet Upload Bandwidth',
421         'unit': 'Mbps',
422         'value': internet_data.get('upload_mbps', 0.0)
423     },
424         '_provider': SensorProvider.INTERNET_SPEED
425     })
426     raw_readings.append({
427         'id': 'internet_ping',
428         'hardware_name': 'Internet',
429         'hardware_type': 'network',
430         'sensor_category': 'Throughput',
431         'sensor_name': 'Internet Ping Latency',
432         'unit': 'ms',
433         'value': internet_data.get('ping_ms', 0.0),
434         '_provider': SensorProvider.INTERNET_SPEED
435     })
436     raw_readings.append({
437         'id': 'internet_bufferbloat',
438         'hardware_name': 'Internet',
439         'hardware_type': 'network',
440         'sensor_category': 'Throughput',
441         'sensor_name': 'Internet Bufferbloat',
442         'unit': 'ms',
443         'value': internet_data.get('bufferbloat_ms', 0
.0),
444         '_provider': SensorProvider.INTERNET_SPEED
445     })

```

```

445         raw_readings.append({
446             'id': 'internet_latency_loaded',
447             'hardware_name': 'Internet',
448             'hardware_type': 'network',
449             'sensor_category': 'Throughput',
450             'sensor_name': 'Internet Loaded Latency',
451             'unit': 'ms',
452             'value': internet_data.get('latency_loaded_ms'
    , 0.0),
453             '_provider': SensorProvider.INTERNET_SPEED
454         })
455
456     # Дедупликация базовых показаний по приоритету провайде
    ера
457     deduplicator = SensorDeduplicator()
458     unique_readings: List[Dict[str, Any]] = []
459
460     for reading in raw_readings:
461         sensor_id = str(reading.get('id', ''))
462         value = reading.get('value', 0.0)
463         provider = reading.get('_provider', SensorProvider
    .SENSOR_COLLECTOR)
464
465         if deduplicator.add(sensor_id, provider, value):
466             unique_readings.append(reading)
467
468     # Обогащение из LibreHardwareMonitor (добавляются ТОЛЬ
    КО сенсоры, отсутствующие в базовом пуле CIM/WMI)
469     if self._lhm_service and self._lhm_service.is_running(
    ):
470         try:
471             lhm_items = self._lhm_service.get_flattened_se
    nsors()
472             existing_ids = {str(r.get('id', '')) for r in
    unique_readings}
473             for item in lhm_items:
474                 hw_name = item.get("hardware_name", "Hardw
    are")

```

```

475         hw_type = item.get("hardware_type", "senso
r")
476         cat = item.get("sensor_category", "General
")
477         s_name = item.get("sensor_name", "Sensor")
478
479         if "Distance to TjMax" in s_name:
480             continue
481
482         val_num = item.get("value_num")
483         if val_num is None:
484             continue
485
486         vraw = str(item.get("value_raw") or "").lo
wer()
487         unit = ""
488         if "°c" in vraw or "c" in vraw:
489             unit = "°C"
490         elif "%" in vraw:
491             unit = "%"
492         elif "rpm" in vraw:
493             unit = "RPM"
494         elif "v" in vraw and "w" not in vraw:
495             unit = "V"
496         elif "w" in vraw:
497             unit = "W"
498         elif "mhz" in vraw:
499             unit = "MHz"
500         elif "b/s" in vraw:
501             unit = "B/s"
502         elif "gb" in vraw:
503             unit = "GB"
504
505         clean_hw = re.sub(r'^a-zA-Z0-9+', '_', h
w_name.lower()).strip('_')
506         clean_name = re.sub(r'^a-zA-Z0-9+', '_',
s_name.lower()).strip('_')
507         sid = f"lhm_{clean_hw}_{clean_name}"

```

```

508 |
509 |         # Пропускаем, если такой датчик уже есть и
    |         ли это дубликат общей нагрузки CPU/RAM
510 |         if sid in existing_ids or (cat == "Load" a
    |         nd "total" in clean_name and any("util_total" in eid for eid i
    |         n existing_ids)):
511 |             continue
512 |
513 |             existing_ids.add(sid)
514 |             unique_readings.append({
515 |                 'id': sid,
516 |                 'hardware_name': hw_name,
517 |                 'hardware_type': hw_type,
518 |                 'sensor_category': cat,
519 |                 'sensor_name': s_name,
520 |                 'unit': unit,
521 |                 'value': float(val_num),
522 |                 '_provider': SensorProvider.LHM_ENRICH
    |         ED,
523 |                 })
524 |             except Exception as lhm_err:
525 |                 logger.debug(f'Ошибка обогащения сенсорами LHM
    |         : {lhm_err}')
526 |
527 |         return unique_readings
528 |
529 |     def get_hardware_snapshot(self) -> Dict[str, Any]:
530 |         """Получает полный снапшот аппаратного состояния со сп
    |         иском сенсоров.
531 |
532 |         Returns:
533 |             Dict[str, Any]: Снапшот со списком показаний сенсо
    |         ров.
534 |         """
535 |         with self._lock:
536 |             now_iso = datetime.now(timezone.utc).isoformat()
537 |             hardware_data = self._get_hardware_data()
538 |             internet_data = self._get_internet_speed_data()

```



```

539         sensor_readings = self.extract_sensor_readings(har
|         dware_data, internet_data)
540         sensors_payload = []
541         for r in sensor_readings:
542             sensors_payload.append({'id': r.get('id'), 'ha
|             rdware_name': r.get('hardware_name'), 'hardware_type': r.get('
|             hardware_type'), 'sensor_category': r.get('sensor_category'),
|             'sensor_name': r.get('sensor_name'), 'unit': r.get('unit'), 'v
|             alues': [{'num': r.get('value', 0.0), 'time': now_iso}]})
543             result = {'timestamp': now_iso, 'sensors': sensors
|             _payload, 'hardware_monitor': hardware_data if hardware_data e
|             lse {}, 'internet_speed': internet_data if internet_data else
|             {}}
544             self._last_hardware_snapshot = result
545             return result
546
547     def get_last_snapshot(self) -> Optional[Dict[str, Any]]:
548         """Возвращает последний собранный снапшот.
549
550         Returns:
551             Optional[Dict[str, Any]]: Последний снапшот или No
|         ne.
552         """
553         return self._last_hardware_snapshot
554
555     def collect_all_sensors(self) -> List[Dict[str, Any]]:
556         """Собирает показания всех доступных датчиков.
557
558         Returns:
559             List[Dict[str, Any]]: Список прочитанных сенсоров.
560         """
561         with self._lock:
562             hardware_data = self._get_hardware_data()
563             internet_data = self._get_internet_speed_data()
564             return self.extract_sensor_readings(hardware_data,
|             internet_data)
565
566     def collect_lhm_cpu_readings(self) -> List[Dict[str, Any]]

```

```

:
567 |         """Извлекает нормализованные показания температур ядер
      | CPU и Package из LHM.
568 |
569 |         Returns:
570 |             List[Dict[str, Any]]: Список показаний датчиков те
      | мператур.
571 |         """
572 |         from apps.windows.sdk.modules.hardware.lhm_service imp
      | ort LhmService
573 |         lhm = self._lhm_service or LhmService()
574 |         items = lhm.get_flattened_sensors()
575 |         results: List[Dict[str, Any]] = []
576 |         core_idx = 0
577 |         for item in items:
578 |             if item.get("hardware_type") != "cpu" or item.get(
      | "sensor_category") != "Temperatures":
579 |                 continue
580 |             name = item.get("sensor_name", "")
581 |             if "Distance to TjMax" in name:
582 |                 continue
583 |             val = item.get("value_num")
584 |             if val is None:
585 |                 continue
586 |             if "Package" in name:
587 |                 results.append({
588 |                     "id": "cpu_package_temp",
589 |                     "hardware_name": item.get("hardware_name",
      | "CPU"),
590 |                     "hardware_type": "cpu",
591 |                     "sensor_category": "Temperatures",
592 |                     "sensor_name": name,
593 |                     "unit": "°C",
594 |                     "value": val,
595 |                     "value_num": val,
596 |                 })
597 |             elif "Core" in name:
598 |                 results.append({

```

```
599 |         "id": f"cpu_core_{core_idx}_temp",
600 |         "hardware_name": item.get("hardware_name",
    | "CPU"),
601 |         "hardware_type": "cpu",
602 |         "sensor_category": "Temperatures",
603 |         "sensor_name": name,
604 |         "unit": "°C",
605 |         "value": val,
606 |         "value_num": val,
607 |     })
608 |     core_idx += 1
609 | return results
610 |
```

sensor_registry.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - Sensor
  Registry
4 # =====
5 # Description:
6 #   Реестр сенсоров и политика дедупликации.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.sensor_registry import SensorProvider
11 #
12 #       service = SensorProvider()
13 #
14 # File: sensor_registry.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-04 07:00:00
20 # =====
21
22 from __future__ import annotations
23 """Реестр сенсоров и политика дедупликации."""
24
25 from dataclasses import dataclass, field
26 from enum import Enum
27 from typing import Dict, List, Optional, Set, Tuple
28 from datetime import datetime, timezone
29
30
31 class SensorProvider(Enum):
```

```

32 |     """Приоритеты и категории провайдеров сенсоров.
33 |
34 |     Приоритеты распределены так:
35 |     - INTERNET_SPEED (55): Выделенный speedtest
36 |     - CIM_SYSTEM (50): Основной современный нативный интерфейс
    |     Windows (CIM/MSFT)
37 |     - WINDOWS_STORAGE (45): WindowsStorageSensor (диски, SMART
    |     , температуры NVMe)
38 |     - GPU_PROBER (40): Прямой зонд GPU (NVIDIA/AMD/Intel)
39 |     - ACPI_THERMAL (35): Системные ACPI ThermalZones
40 |     - HARDWARE_MONITOR (35): Агрегатор аппаратных метрик
41 |     - SENSOR_COLLECTOR (30): Базовый коллектор системных метри
    |     к
42 |     - WMI_FALLBACK (30): WMI слой совместимости / fallback при
    |     недоступности CIM
43 |     - NETWORK_SENSOR (25): Сетевые интерфейсы psutil
44 |     - LHM_ENRICHED (20): Обогащение отсутствующими в CIM/WMI с
    |     енсорами из LibreHardwareMonitor
45 |     - NVIDIA_SMI_DIRECT (10): Прямой вызов утилит
46 |     """
47 |     INTERNET_SPEED = 55           # InternetSpeedSensor - приори
    |     тет для speedtest метрик
48 |     CIM_SYSTEM = 50               # Основной интерфейс CIM (Powe
    |     rShell / CimSession)
49 |     WINDOWS_STORAGE = 45         # WindowsStorageSensor (SMART,
    |     temps) - приоритет для storage
50 |     GPU_PROBER = 40               # GpuProber - приоритет для GP
    |     U метрик
51 |     ACPI_THERMAL = 35             # ACPI thermal zones
52 |     HARDWARE_MONITOR = 35        # HardwareMonitor (агрегатор)
53 |     SENSOR_COLLECTOR = 30        # SensorCollector.extract_sens
    |     or_readings()
54 |     WMI_FALLBACK = 30            # WMI слой совместимости
55 |     NETWORK_SENSOR = 25          # Network sensors через psutil
56 |     LHM_ENRICHED = 20            # Обогащение детальными датчик
    |     ами из LibreHardwareMonitor
57 |     NVIDIA_SMI_DIRECT = 10       # Прямой вызов nvidia-smi (leg
    |     асу)

```

```

58 |
59 |
60 | @dataclass
61 | class SensorDefinition:
62 |     """Определение сенсора с политикой дедупликации."""
63 |     sensor_id: str
64 |     category: str
65 |     preferred_provider: SensorProvider
66 |     description: str = ""
67 |     unit: str = ""
68 |     allow_multiple_sources: bool = False # Если True, дублика
ты разрешены с разными провайдерами
69 |     dedup_window_seconds: float = 1.0 # Окно дедупликации
по времени
70 |
71 |
72 | class SensorRegistry:
73 |     """Централизованный реестр сенсоров с политикой дедупликац
ии."""
74 |
75 |     # Реестр известных сенсоров с предпочтительными провайдера
ми
76 |     _registry: Dict[str, SensorDefinition] = {}
77 |
78 |     # Мappings категорий на предпочтительные провайдеры
79 |     _category_providers: Dict[str, SensorProvider] = {
80 |         "temperature": SensorProvider.ACPI_THERMAL,
81 |         "Temperatures": SensorProvider.ACPI_THERMAL,
82 |         "gpu_temp": SensorProvider.GPU_PROBER,
83 |         "gpu_load": SensorProvider.GPU_PROBER,
84 |         "gpu_power": SensorProvider.GPU_PROBER,
85 |         "gpu_fan": SensorProvider.GPU_PROBER,
86 |         "cpu_temp": SensorProvider.ACPI_THERMAL,
87 |         "cpu_load": SensorProvider.SENSOR_COLLECTOR,
88 |         "Load": SensorProvider.SENSOR_COLLECTOR, # CPU/RAM ut
ilization
89 |         "Clocks": SensorProvider.SENSOR_COLLECTOR,
90 |         "network_throughput": SensorProvider.SENSOR_COLLECTOR,

```

```

91         "network_bytes": SensorProvider.NETWORK_SENSOR,
92         "Throughput": SensorProvider.SENSOR_COLLECTOR,
93         "internet": SensorProvider.INTERNET_SPEED,
94         "storage_temp": SensorProvider.WINDOWS_STORAGE,
95         "storage_wear": SensorProvider.WINDOWS_STORAGE,
96         "storage_usage": SensorProvider.SENSOR_COLLECTOR,
97         "storage": SensorProvider.SENSOR_COLLECTOR,
98     }
99
100     # Паттерны дублирующихся sensor_id
101     _duplicate_patterns: Dict[str, List[str]] = {
102         # GPU метрики - собираются предпочтительно через GpuPr
103         "gpu_*": ["gpu_prober", "sensor_collector", "nvidia_s
104         mi_direct"],
105         # CPU метрики
106         "cpu_*": ["acpi_thermal", "sensor_collector"],
107         # Internet ping - дублируется в sensors.py и sensor_co
108         llector
109         "internet_ping": ["internet_speed", "sensor_collector"
110         ],
111         # Network throughput - дублируется в sensors.py и sens
112         or_collector
113         "network_*_rate": ["network_sensor", "sensor_collecto
114         r"],
115     }
116
117     @classmethod
118     def register(cls, definition: SensorDefinition) -> None:
119         """Регистрирует сенсор в реестре.
120
121         Args:
122             definition: Определение сенсора с политикой.
123         """
124         cls._registry[definition.sensor_id] = definition
125
126     @classmethod
127     def get_preferred_provider(cls, sensor_id: str, category:

```

```

    str) -> Optional[SensorProvider]:
123     """Возвращает предпочтительный провайдер для данного с
    енсора.
124
125     Args:
126         sensor_id: Идентификатор сенсора.
127         category: Категория сенсора.
128
129     Returns:
130         Preferred provider или None если не определен.
131     """
132     # 1. Проверяем явную регистрацию
133     if sensor_id in cls._registry:
134         return cls._registry[sensor_id].preferred_provider
135
136     # 2. Определяем по специфическому паттерну sensor_id
137     import re
138     for pattern, providers in cls._duplicate_patterns.item
s():
139         if re.match(pattern, sensor_id):
140             # Возвращаем первый из списка как предпочтител
141             ьный
142             provider_name = providers[0].upper()
143             try:
144                 return SensorProvider(provider_name)
145             except KeyError:
146                 pass
147
148     # 3. Проверяем по категории
149     if category in cls._category_providers:
150         return cls._category_providers[category]
151
152     return None
153
154     @classmethod
155     def should_keep_sensor(
156         cls,
157         sensor_id: str,

```



```

157         provider: SensorProvider,
158         existing_sensors: List[Tuple[str, SensorProvider]]
159     ) -> bool:
160         """Определяет, нужно ли сохранить показание сенсора ил
и отклонить как дубликат.
161
162     Args:
163         sensor_id: Идентификатор сенсора.
164         provider: Провайдер, предоставивший показание.
165         existing_sensors: Список уже собранных (sensor_id,
provider) пар.
166
167     Returns:
168         True если сенсор нужно сохранить, False если это д
убликат.
169         """
170         # Проверяем, есть ли уже такой sensor_id
171         for existing_id, existing_provider in existing_sensors
:
172             if existing_id == sensor_id:
173                 # Дубликат найден - сравниваем приоритеты пров
айдеров
174                 preferred = cls.get_preferred_provider(sensor_
id, "")
175                 if preferred:
176                     # Сохраняем только от предпочтительного пр
овайдера
177                     return provider == preferred
178                 else:
179                     # Нет явной политики - сохраняем от провай
дера с высшим приоритетом
180                     return provider.value > existing_provider.
value
181
182         # Дубликатов нет - сохраняем
183         return True
184
185

```

```

186 class SensorDeduplicator:
187     """Дедупликатор показаний сенсоров."""
188
189     def __init__(self) -> None:
190         """Инициализирует дедупликатор с пустым кешем."""
191         self._seen: Dict[str, Tuple[float, SensorProvider, flo
192 at]] = {} # sensor_id -> (timestamp, provider, value)
193
194     def add(
195         self,
196         sensor_id: str,
197         provider: SensorProvider,
198         value: float,
199         timestamp: Optional[float] = None,
200         dedup_window: float = 1.0
201     ) -> bool:
202         """Пытается добавить показание сенсора с проверкой на
203 дубликаты.
204
205     Args:
206         sensor_id: Идентификатор сенсора.
207         provider: Провайдер показания.
208         value: Значение показания.
209         timestamp: Временная метка (epoch seconds).
210         dedup_window: Окно дедупликации в секундах.
211
212     Returns:
213         True если показание добавлено, False если отклонен
214 о как дубликат.
215
216     """
217     if timestamp is None:
218         timestamp = datetime.now(timezone.utc).timestamp()
219
220     # Проверяем наличие в кеше
221     if sensor_id in self._seen:
222         cached_ts, cached_provider, cached_value = self._s
223 een[sensor_id]

```

```

220         # Проверяем временное окно
221         if abs(timestamp - cached_ts) < dedup_window:
222             # В пределах окна - проверяем приоритеты
223             preferred = SensorRegistry.get_preferred_provi
224             der(sensor_id, "")
225             if preferred:
226                 # Если текущий провайдер предпочтительнее
227                 - обновляем
228                 if provider == preferred:
229                     self._seen[sensor_id] = (timestamp, pr
230                     ovider, value)
231                     return True
232                 else:
233                     # Отклоняем
234                     return False
235             else:
236                 # Сравниваем по значению приоритета
237                 if provider.value > cached_provider.value:
238                     self._seen[sensor_id] = (timestamp, pr
239                     ovider, value)
240                     return True
241                 else:
242                     return False
243
244         # Нет дубликата или вне окна - добавляем
245         self._seen[sensor_id] = (timestamp, provider, value)
246         return True
247
248     def get_unique(self) -> Dict[str, Tuple[SensorProvider, fl
249     oat]]:
250         """Возвращает уникальные показания сенсоров.
251
252         Returns:
253             Словарь {sensor_id: (provider, value)}.
254         """
255         return {sid: (prov, val) for sid, (_, prov, val) in se
256         lf._seen.items()}

```

```
252     def clear(self) -> None:
253         """Очищает кеш дедупликатора."""
254         self._seen.clear()
255
256
257     def deduplicate_sensor_readings(
258         readings: List[Dict],
259         provider: SensorProvider = SensorProvider.SENSOR_COLLECTOR
260     ) -> List[Dict]:
261         """Дедуплицирует список показаний сенсоров.
262
263         Args:
264             readings: Список показаний с полями 'id' и 'value'.
265             provider: Провайдер показаний.
266
267         Returns:
268             Дедуплицированный список показаний.
269         """
270         deduplicator = SensorDeduplicator()
271         unique_readings: List[Dict] = []
272
273         for reading in readings:
274             sensor_id = reading.get('id', '')
275             value = reading.get('value', 0.0)
276
277             if deduplicator.add(sensor_id, provider, value):
278                 unique_readings.append(reading)
279
280         return unique_readings
281
```

```

service.py
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - Service
4 # =====
5 # Description:
6 #   Фоновый сервис посекундного сбора системных метрик с сохра
   нением в SQLite.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.service import TelemetryLogg
   erService
11 #
12 #       instance = TelemetryLoggerService.get_instance()
13 #
14 # File: service.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-10 05:40:00
20 # =====
21
22 from __future__ import annotations
23 """Фоновый сервис периодического сбора системных метрик с сохр
   анением в SQLite."""
24
25 import asyncio
26 import threading
27 import time
28 from typing import Any, Dict, List, Optional
29 try:
30     from logger import logger

```

```

31 except ImportError:
32     from logger import logger
33 from .collector import SystemCollector
34 from apps.windows.telemetry_research.hardware_history_manager
    import HardwareHistoryManager
35 from .sqlite import TelemetryStorage
36 from .telemetry_config import TelemetryConfigManager
37 from .w64_collector import AIW64Collector
38 from .w64_etw_collector import AIW64ETWCollector
39
40 class TelemetryLoggerService:
41     """Сервис периодического сбора системной телеметрии, аудит
    а оборудования и W64/ETW событий в SQLite."""
42     _instance: Optional[TelemetryLoggerService] = None
43
44     def __init__(
45         self,
46         interval_sec: Optional[float] = None,
47         top_processes: Optional[int] = None,
48         collector: Optional[SystemCollector] = None,
49         storage: Optional[TelemetryStorage] = None,
50         hardware_audit_interval_sec: Optional[float] = None,
51         rollup_interval_sec: Optional[float] = None,
52         enable_w64: Optional[bool] = None,
53         config_manager: Optional[TelemetryConfigManager] = Non
    e,
54         db_cleanup_interval_sec: Optional[float] = None,
55     ) -> None:
56         """Инициализирует сервис сбора системных метрик.
57
58         Args:
59             interval_sec: Интервал между замерами телеметрии в
    секундах (по умолчанию из config.json).
60             top_processes: Лимит сохраняемых активных процессо
    в (по умолчанию из config.json).
61             collector: Экземпляр сборщика SystemCollector (опц
    ионально).
62             storage: Экземпляр хранилища TelemetryStorage (опц

```

ионально).

```
63         hardware_audit_interval_sec: Интервал периодическо
го аудита железа (по умолчанию из config.json).
64         rollup_interval_sec: Интервал запуска обобщения ст
арых данных (по умолчанию из config.json).
65         enable_w64: Включение подсистемы сбора событий W64
/ETW (по умолчанию из config.json).
66         config_manager: Менеджер конфигурации.
67         db_cleanup_interval_sec: Интервал фонового контрол
я размера базы данных (сек).
68         """
69         self.config_manager = config_manager or TelemetryConfi
gManager()
70         self._custom_interval_set = (interval_sec is not None)
71         self.interval_sec = max(
72             0.1,
73             interval_sec if interval_sec is not None else self
.config_manager.get_interval_seconds()
74         )
75         self.top_processes = max(
76             1,
77             top_processes if top_processes is not None else se
lf.config_manager.get_effective_process_limit()
78         )
79         self.hardware_audit_interval_sec = max(
80             1.0,
81             hardware_audit_interval_sec if hardware_audit_inte
rval_sec is not None else self.config_manager.get_heavy_interv
al_seconds()
82         )
83         self.rollup_interval_sec = max(
84             1.0,
85             rollup_interval_sec if rollup_interval_sec is not
None else self.config_manager.get_aggregation_interval()
86         )
87         self.collector = collector or SystemCollector()
88         self.storage = storage or TelemetryStorage.get_instanc
e()
```

```

89         self.db_cleanup_interval_sec = max(
90             5.0,
91             db_cleanup_interval_sec if db_cleanup_interval_sec
is not None else self.config_manager.get_db_cleanup_interval_
seconds()
92         )
93
94         cfg = self.config_manager.get_config()
95         w64_cfg = cfg.get('w64_collector', {})
96         self.enable_w64 = enable_w64 if enable_w64 is not None
else w64_cfg.get('enabled', True)
97
98         self._w64_collector: Optional[AIW64Collector] = None
99         self._w64_etw_collector: Optional[AIW64ETWCollector] =
None
100
101         self._running = False
102         self._thread: Optional[threading.Thread] = None
103         self._stop_event = threading.Event()
104         self._ticks_count = 0
105         self._start_time: Optional[float] = None
106         self._last_tick_time: Optional[float] = None
107         self._last_hw_audit_time: Optional[float] = None
108         self._last_def_audit_time: Optional[float] = None
109         self._last_rollup_time: Optional[float] = None
110         self._last_db_cleanup_time: Optional[float] = None
111         self._last_error: Optional[str] = None
112         self._last_snapshot: Optional[Any] = None
113
114     @classmethod
115     def get_instance(cls) -> TelemetryLoggerService:
116         """Возвращает глобальный синглтон сервиса телеметрии.
117
118         Returns:
119             TelemetryLoggerService: Единственный экземпляр сер
виса.
120         """
121         if cls._instance is None:

```



```

122         cls._instance = TelemetryLoggerService()
123     return cls._instance
124
125     @property
126     def is_running(self) -> bool:
127         """Проверяет, запущен ли фоновый сбор телеметрии.
128
129         Returns:
130             bool: True если сбор активен, иначе False.
131         """
132         return self._running and self._thread is not None and
self._thread.is_alive()
133
134     @property
135     def history_manager(self) -> HardwareHistoryManager:
136         """Возвращает менеджер истории оборудования."""
137         return self.collector.history_manager
138
139     @property
140     def w64_collector(self) -> Optional[AIW64Collector]:
141         """Возвращает сборщик W64 событий (создается лениво пр
и enable_w64)."""
142         if self._w64_collector is None and self.enable_w64:
143             self._init_w64_collectors()
144         return self._w64_collector
145
146     @w64_collector.setter
147     def w64_collector(self, val: Optional[AIW64Collector]) ->
None:
148         self._w64_collector = val
149
150     @property
151     def w64_etw_collector(self) -> Optional[AIW64ETWCollector]
:
152         """Возвращает сборщик ETW событий (создается лениво пр
и enable_w64)."""
153         if self._w64_etw_collector is None and self.enable_w64
:

```

```

154         self._init_w64_collectors()
155     return self._w64_etw_collector
156
157     @w64_etw_collector.setter
158     def w64_etw_collector(self, val: Optional[AIW64ETWCollector]) -> None:
159         self._w64_etw_collector = val
160
161     def _init_w64_collectors(self) -> None:
162         """Инициализирует W64 и ETW сборщики при необходимости
163         ."""
164         if not self.enable_w64:
165             return
166         cfg = self.config_manager.get_config()
167         w64_cfg = cfg.get('w64_collector', {})
168         if self._w64_collector is None:
169             self._w64_collector = AIW64Collector(
170                 storage=self.storage,
171                 enable_file_monitoring=w64_cfg.get('enable_file_monitoring', True),
172                 enable_process_monitoring=w64_cfg.get('enable_process_monitoring', True),
173                 enable_registry_monitoring=w64_cfg.get('enable_registry_monitoring', True),
174                 enable_network_monitoring=w64_cfg.get('enable_network_monitoring', True),
175                 enable_event_log_monitoring=w64_cfg.get('enable_event_log_monitoring', True),
176             )
177         if self._w64_etw_collector is None:
178             self._w64_etw_collector = AIW64ETWCollector(
179                 storage=self.storage,
180                 enable_process_trace=w64_cfg.get('enable_process_trace', True),
181                 enable_disk_trace=w64_cfg.get('enable_disk_trace', True),
182                 enable_network_trace=w64_cfg.get('enable_network_trace', True),

```

```

182         enable_registry_trace=w64_cfg.get('enable_regi
|stry_trace', True),
183     )
184
185     def start(self) -> bool:
186         """Запускает фоновый поток сбора телеметрии, аудита об
|орудования и W64/ETW сборщиков.
187
188         Returns:
189             bool: True если сервис успешно запущен, False если
|уже работал.
190         """
191         if self.is_running:
192             logger.debug('Сервис сбора телеметрии уже запущен.
|')
193             return False
194         self._stop_event.clear()
195         self._running = True
196         self._start_time = time.time()
197         self._ticks_count = 0
198         self._last_error = None
199         self._last_hw_audit_time = None
200         self._last_db_cleanup_time = None
201
202         if self.enable_w64:
203             self._init_w64_collectors()
204             if self.w64_collector:
205                 self.w64_collector.start()
206             if self.w64_etw_collector:
207                 self.w64_etw_collector.start()
208
209         # При каждом старте телеметрии фиксируем снимок автоза
|пуска в хранилище
210         try:
211             startup_entry = self.collector.archive_startup_sta
|te(auto_diff=True)
212             if startup_entry:
213                 self.storage.save_startup_archive(startup_entr

```

```

y)
214         except Exception as st_init_ex:
215             logger.debug(f'Фиксация снимка автозапуска при ста
рте телеметрии: {st_init_ex}')
216
217         # При старте телеметрии фиксируем снимок состояния Def
ender и безопасности в БД
218         try:
219             threading.Thread(target=self.collect_and_save_defe
nder_state, name='DefenderInitCollector', daemon=True).start()
220         except Exception as def_init_ex:
221             logger.debug(f'Ошибка запуска инициализации Defend
er: {def_init_ex}')
222
223         self._thread = threading.Thread(target=self._worker_lo
op, name='TelemetryLoggerWorker', daemon=True)
224         self._thread.start()
225         logger.info(f'Фоновый сервис телеметрии запущен в БД (
интервал: {self.interval_sec}с, аудит железа: {self.hardware_a
udit_interval_sec}с, W64: {self.enable_w64}, БД: {self.storage
.db_path})')
226         return True
227
228     def stop(self) -> bool:
229         """Останавливает фоновый поток сбора телеметрии и W64/
ETW сборщиков.
230
231         Returns:
232             bool: True если сервис был остановлен, False если
он не работал.
233         """
234         if not self._running:
235             return False
236         self._running = False
237         self._stop_event.set()
238
239         if self.w64_collector:
240             self.w64_collector.stop()

```

```

241         if self.w64_etw_collector:
242             self.w64_etw_collector.stop()
243
244         if self._thread and self._thread.is_alive():
245             self._thread.join(timeout=3.0)
246         self._thread = None
247         try:
248             self.storage.flush()
249         except Exception:
250             pass
251         logger.info(f'Фоновый сервис телеметрии остановлен. Вс
его тиков: {self._ticks_count}')
252         return True
253
254     def sync_config_from_manager(self) -> bool:
255         """Синхронизирует интервалы и настройки сервиса из мен
еджера конфигурации при их изменении.
256
257         Returns:
258             bool: True если конфигурация изменилась и была при
менена, иначе False.
259         """
260         self.config_manager.check_tc_mode_auto_switch()
261         if self.config_manager.check_and_reload():
262             if not self._custom_interval_set:
263                 self.interval_sec = max(0.1, self.config_manag
er.get_interval_seconds())
264                 self.top_processes = max(1, self.config_manager.ge
t_effective_process_limit())
265                 self.hardware_audit_interval_sec = max(1.0, self.c
onfig_manager.get_heavy_interval_seconds())
266                 self.db_cleanup_interval_sec = max(5.0, self.conf
ig_manager.get_db_cleanup_interval_seconds())
267                 self.rollup_interval_sec = max(1.0, self.config_ma
nager.get_aggregation_interval())
268                 logger.info(
269                     f" [Сервис телеметрии] Конфигурация config.j
son обновлена 'на лету': "

```

```

270         f"базовый={self.interval_sec}с, аудит железа={
self.hardware_audit_interval_sec}с, "
271         f"очистка БД={self.db_cleanup_interval_sec}с,
топ процессов={self.top_processes}, mode={self.config_manager.
get_telemetry_mode()}"
272     )
273     return True
274     return False
275
276     def _worker_loop(self) -> None:
277         """Основной рабочий цикл фонового сбора метрик, записи
в БД и аудита изменений."""
278         while not self._stop_event.is_set():
279             loop_start = time.time()
280             try:
281                 # Проверка изменения config.json 'на лету'
282                 self.sync_config_from_manager()
283
284                 loop = None
285                 try:
286                     loop = asyncio.get_event_loop()
287                 except RuntimeError:
288                     loop = None
289                 if loop and loop.is_running():
290                     task = asyncio.run_coroutine_threadsafe(se
lf.collector.get_snapshot(process_limit=self.top_processes), l
oop)
291                     snapshot = task.result(timeout=30.0)
292                 else:
293                     snapshot = asyncio.run(self.collector.get_
snapshot(process_limit=self.top_processes))
294                     self._last_snapshot = snapshot
295                     self._ticks_count += 1
296                     self._last_tick_time = time.time()
297                 try:
298                     self.storage.save_snapshot(snapshot, top_n
=self.top_processes)
299                 except Exception as db_err:

```

```

300         logger.warning(f'Ошибка сохранения снимка
телеметрии в БД: {db_err}')
301         if self._last_hw_audit_time is None or loop_start - self._last_hw_audit_time >= self.hardware_audit_interval_sec:
302             try:
303                 archive_entry = self.collector.archive
hardware_state(auto_diff=True)
304                 if archive_entry:
305                     self.storage.save_hardware_archive
(archive_entry)
306             except Exception as hw_ex:
307                 logger.debug(f'Ошибка при периодическо
м аудите железа: {hw_ex}')
308             try:
309                 startup_archive = self.collector.archi
ve_startup_state(auto_diff=True)
310                 if startup_archive:
311                     self.storage.save_startup_archive(
startup_archive)
312             except Exception as st_ex:
313                 logger.debug(f'Ошибка при периодическо
м аудите автозапуска: {st_ex}')
314             try:
315                 self.collector.get_extended_system_aud
it()
316             except Exception as ext_ex:
317                 logger.debug(f'Ошибка при периодическо
м расширенном аудите: {ext_ex}')
318             try:
319                 self.collect_and_save_defender_state()
320             except Exception as def_ex:
321                 logger.debug(f'Ошибка при периодическо
м аудите Defender: {def_ex}')
322                 self._last_hw_audit_time = loop_start
323                 self._last_def_audit_time = loop_start
324                 if self._last_rollup_time is None or loop_start - self._last_rollup_time >= self.rollup_interval_sec:

```

```

325         try:
326             self.run_rollups()
327             self._last_rollup_time = loop_start
328         except Exception as roll_ex:
329             logger.debug(f'Ошибка при периодическо
м обобщении процессов: {roll_ex}')
330             if self._last_db_cleanup_time is None or loop_
start - self._last_db_cleanup_time >= self.db_cleanup_interval
_sec:
331                 try:
332                     self.cleanup_db()
333                     self._last_db_cleanup_time = loop_star
t
334                 except Exception as clean_ex:
335                     logger.debug(f'Ошибка при периодическо
м контроле размера БД: {clean_ex}')
336             except (RuntimeError, ValueError) as shut_ex:
337                 if 'shutdown' in str(shut_ex).lower() or 'clos
ed file' in str(shut_ex).lower():
338                     break
339                 self._last_error = str(shut_ex)
340                 logger.debug(f'Ошибка при сборе системной теле
метрии: {shut_ex}')
341             except Exception as ex:
342                 self._last_error = str(ex)
343                 logger.debug(f'Ошибка при сборе системной теле
метрии: {ex}')
344             elapsed = time.time() - loop_start
345             sleep_time = max(0.01, self.interval_sec - elapsed
)
346             if self._stop_event.wait(timeout=sleep_time):
347                 break
348
349     def collect_and_save_defender_state(self) -> None:
350         """Опрашивает текущее состояние Microsoft Defender и с
охраняет снимок в базу данных SQLite."""
351         try:
352             from apps.windows.sdk.modules.defender.core.defend

```



```

er_service import DefenderService
353         from apps.windows.sdk.modules.defender.core.asr_ma
nager import ASRManager
354         from apps.windows.sdk.modules.defender.core.exclus
ions_auditor import ExclusionsAuditor
355         from apps.windows.sdk.modules.defender.core.threat
_manager import ThreatManager
356
357         def_svc = DefenderService()
358         asr_mgr = ASRManager(def_svc)
359         excl_aud = ExclusionsAuditor(def_svc)
360         threat_mgr = ThreatManager(def_svc)
361
362         status = def_svc.get_defender_status()
363         asr_rules = asr_mgr.get_asr_rules()
364         excl_report = excl_aud.audit_exclusions()
365         threats = threat_mgr.get_threats_history(limit=50)
366
367         all_exclusions = excl_report.path_exclusions + exc
l_report.extension_exclusions + excl_report.process_exclusions
368         snap_id = f"snap_defender_{int(datetime.now(timezo
ne.utcnow().timestamp()))}"
369         self.storage.save_defender_snapshot(
370             snapshot_id=snap_id,
371             defender_status=status,
372             exclusions=all_exclusions,
373             asr_rules=asr_rules,
374             threats=threats,
375         )
376         logger.debug(f'Снимок состояния Defender зафиксиро
ван в БД ({snap_id})')
377
378         # Инкрементально опрашиваем операционные события D
efender и Security
379         try:
380             from .security_collector import WindowsSecurit
yCollector
381             sec_collector = WindowsSecurityCollector(stora

```

```

ge=self.storage)
382         sec_collector.collect_all_security_and_defender(batch_size=200, max_records=1000)
383         except Exception as ev_ex:
384             logger.debug(f'Ошибка сбора операционных событий Defender/Security: {ev_ex}')
385         except Exception as e:
386             logger.debug(f'Ошибка сбора и сохранения состояния Defender: {e}')
387
388     def run_rollups(self) -> Dict[str, Any]:
389         """Принудительно запускает процедуры обобщения устаревших метрик процессов.
390
391         Returns:
392             Dict[str, Any]: Сводная статистика обобщения за 2 минуты и за 1 день.
393         """
394         short_res = self.storage.aggregate_process_metrics_2min(cutoff_seconds=120)
395         daily_res = self.storage.aggregate_process_metrics_daily(cutoff_days=1)
396         return {'rollup_2min': short_res, 'rollup_daily': daily_res}
397
398     def cleanup_db(self) -> Dict[str, Any]:
399         """Запускает процедуру контроля размера и усечения базы данных телеметрии.
400
401         Returns:
402             Dict[str, Any]: Отчет о результатах контроля размера.
403         """
404         return self.storage.enforce_size_limit()
405
406     def get_status(self) -> Dict[str, Any]:
407         """Возвращает текущий статус сервиса телеметрии.
408

```

```

409         Returns:
410             Dict[str, Any]: Словарь с состоянием и статистикой
работы сервиса.
411         """
412         uptime = round(time.time() - self._start_time, 1) if s
elf._start_time and self.is_running else 0.0
413         return {'is_running': self.is_running, 'interval_sec':
self.interval_sec, 'top_processes_limit': self.top_processes,
'hardware_audit_interval_sec': self.hardware_audit_interval_s
ec, 'ticks_recorded': self._ticks_count, 'uptime_seconds': upt
ime, 'last_tick_epoch': self._last_tick_time, 'last_hw_audit_e
poch': self._last_hw_audit_time, 'last_error': self._last_erro
r, 'storage_stats': self.storage.get_storage_stats()}
414
415     def get_last_snapshot(self) -> Optional[Any]:
416         """Возвращает последний собранный снапшот.
417
418         Returns:
419             Optional[Any]: Последний снапшот или None.
420         """
421         return self._last_snapshot
422
423     def get_history(self, limit: int=60) -> List[Dict[str, Any
]]:
424         """Возвращает историю системных снимков из SQLite базы
данных.
425
426         Args:
427             limit: Количество последних записей.
428
429         Returns:
430             List[Dict[str, Any]]: Список исторических срезов.
431         """
432         return self.storage.get_snapshots(limit=limit)
433
434     def record_event(self, event_type: str, event_details: Dic
t[str, Any], severity: str='info') -> int:
435         """Записывает событие в SQLite базу данных телеметрии.

```

```
436 |
437 |     Args:
438 |         event_type: Тип события (hardware_change, process_
| start, driver_update).
439 |         event_details: Детали события.
440 |         severity: Уровень серьезности события.
441 |
442 |     Returns:
443 |         int: ID добавленной записи события.
444 |     """
445 |     logger.info(f'[Событие Telemetry] {event_type}: {event
| _details}')
446 |     try:
447 |         return self.storage.save_event(event_type, event_d
| etails, severity=severity)
448 |     except Exception as ex:
449 |         logger.error(f'Не удалось записать событие в базу
| данных: {ex}')
450 |         return 0
```

aggregator.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry Sqlite -
  Aggregator
4 # =====
5 # Description:
6 #   Многоуровневый SQL-агрегатор временных рядов телеметрии (r
  aw -> hourly ->
7 #   daily -> weekly -> monthly -> yearly) и планировщик агрега
  ции.
8 #
9 # Usage Examples:
10 #   Python API:
11 #       from apps.windows.telemetry.sqlite.aggregator import Tel
  emetrySqlAggregator
12 #
13 #       agg = TelemetrySqlAggregator(connection_manager)
14 #       agg.run_pipeline()
15 #
16 # File: aggregator.py
17 # Project: ai-breadboard
18 # Package: apps.windows.telemetry.sqlite
19 # Author: hypo69
20 # Copyright: © 2026 hypo69
21 # Updated: 2026-10-06 05:32:00
22 # =====
23
24 from __future__ import annotations
25
26 """Многоуровневый SQL-агрегатор временных рядов телеметрии и п
  ланировщик агрегации."""
27
28 import sqlite3
```

```

29 import threading
30 import time
31 from datetime import datetime, timezone
32 from enum import Enum
33 from typing import Any, Dict, List, Optional, Union
34
35 from .connection import TelemetryConnectionManager
36
37 try:
38     from logger import logger
39 except ImportError:
40     import logging
41     logger = logging.getLogger(__name__)
42
43
44 class AggregationLevel(str, Enum):
45     """Уровни иерархической агрегации временных рядов телеметр
46     ии."""
47
48     HOURLY = "hourly"
49     DAILY = "daily"
50     WEEKLY = "weekly"
51     MONTHLY = "monthly"
52     YEARLY = "yearly"
53
54 def sensors_aggregate(
55     conn: sqlite3.Connection,
56     level: Union[AggregationLevel, str],
57     start_epoch: Optional[float] = None,
58     end_epoch: Optional[float] = None,
59     detect_spikes: bool = True,
60 ) -> Dict[str, int]:
61     """Выполняет единую SQL-агрегацию сенсоров для указанного
62     уровня иерархии.
63
64     Args:
65         conn: Активное подключение к базе данных SQLite.

```

```
65 |         level: Уровень агрегации ('hourly', 'daily', 'weekly',
66 |         'monthly', 'yearly').
67 |         start_epoch: Начальная граница выборки в секундах epoch
68 |         h (опционально).
69 |         end_epoch: Конечная граница выборки в секундах epoch (
70 |         опционально).
71 |         detect_spikes: Флаг сохранения статистических всплеско
72 |         в/аномалий в telemetry_spikes.
73 |
74 |     Returns:
75 |     Dict[str, int]: Словарь с количеством обработанных бак
76 |     етов и сохраненных аномалий.
77 |     """
78 |     lvl_str = str(level.value if isinstance(level, Aggregation
79 |     Level) else level).lower()
80 |     cursor = conn.cursor()
81 |     now_epoch = time.time()
82 |     res = {"level": lvl_str, "buckets_aggregated": 0, "spikes_
83 |     recorded": 0}
84 |
85 |     if lvl_str == AggregationLevel.HOURLY.value:
86 |         s_epoch = float(start_epoch) if start_epoch is not Non
87 |         e else 0.0
88 |         e_epoch = float(end_epoch) if end_epoch is not None el
89 |         se (int(now_epoch // 3600) * 3600.0)
90 |
91 |     # 1. SQL агрегация сырых измерений в часовые бакеты
92 |     cursor.execute("""
93 |         INSERT INTO telemetry_hourly (
94 |             sensor_id,
95 |             bucket_start,
96 |             bucket_end,
97 |             sample_count,
98 |             value_min,
99 |             value_max,
100 |             value_avg,
101 |             value_sum,
102 |             value_sum_sq,
```

```

94         value_stddev,
95         value_first,
96         value_last
97     )
98     WITH raw_window AS (
99         SELECT
100             sensor_id,
101             CAST(created_at / 3600 AS INTEGER) * 3600
102         AS b_start,
103             created_at,
104             value,
105             FIRST_VALUE(value) OVER (
106                 PARTITION BY sensor_id, CAST(created_a
107             t / 3600 AS INTEGER) * 3600
108                 ORDER BY created_at ASC
109             ) AS f_val,
110             FIRST_VALUE(value) OVER (
111                 PARTITION BY sensor_id, CAST(created_a
112             t / 3600 AS INTEGER) * 3600
113                 ORDER BY created_at DESC
114             ) AS l_val
115         FROM sensor_polls
116         WHERE created_at >= ? AND created_at < ? AND v
117         alue IS NOT NULL
118     )
119     SELECT
120         sensor_id,
121         b_start AS bucket_start,
122         b_start + 3600 AS bucket_end,
123         COUNT(*) AS sample_count,
124         MIN(value) AS value_min,
125         MAX(value) AS value_max,
126         AVG(value) AS value_avg,
127         SUM(value) AS value_sum,
128         SUM(value * value) AS value_sum_sq,
129         SQRT(MAX(0.0, (SUM(value * value) - (SUM(value
130         ) * SUM(value)) / COUNT(*)) / COUNT(*))) AS value_stddev,
131         MIN(f_val) AS value_first,

```



```

127         MAX(l_val) AS value_last
128     FROM raw_window
129     GROUP BY sensor_id, b_start
130     ON CONFLICT(sensor_id, bucket_start) DO UPDATE SET
131         bucket_end = excluded.bucket_end,
132         sample_count = excluded.sample_count,
133         value_min = excluded.value_min,
134         value_max = excluded.value_max,
135         value_avg = excluded.value_avg,
136         value_sum = excluded.value_sum,
137         value_sum_sq = excluded.value_sum_sq,
138         value_stddev = excluded.value_stddev,
139         value_first = excluded.value_first,
140         value_last = excluded.value_last;
141     """ , (s_epoch, e_epoch))
142     res["buckets_aggregated"] = cursor.rowcount if cursor.
rowcount > 0 else 0
143
144     # 2. Выявление и сохранение аномальных всплесков
145     if detect_spikes:
146         cursor.execute("""
147             INSERT INTO telemetry_spikes (
148                 sensor_id, timestamp, created_at, value, s
pike_type,
149                 baseline_avg, baseline_stddev, details
150             )
151             SELECT
152                 p.sensor_id,
153                 p.timestamp,
154                 p.created_at,
155                 p.value,
156                 CASE WHEN p.value >= h.value_avg THEN 'hig
h' ELSE 'low' END,
157                 h.value_avg,
158                 h.value_stddev,
159                 'Всплеск: знач=' || ROUND(p.value, 2) || '
, avg=' || ROUND(h.value_avg, 2) || ', std=' || ROUND(h.value_
stddev, 2)

```

```

160         FROM sensor_polls p
161         JOIN telemetry_hourly h
162         ON p.sensor_id = h.sensor_id
163         AND (CAST(p.created_at / 3600 AS INTEGER) * 3
164         600) = h.bucket_start
165         WHERE h.value_stddev > 0.01
166         AND ABS(p.value - h.value_avg) >= 2.5 * h.va
167         lue_stddev
168         AND p.created_at >= ? AND p.created_at < ?
169         AND NOT EXISTS (
170             SELECT 1 FROM telemetry_spikes s
171             WHERE s.sensor_id = p.sensor_id AND s.cr
172             eated_at = p.created_at
173             );
174         "", (s_epoch, e_epoch))
175         res["spikes_recorded"] = cursor.rowcount if cursor
176         .rowcount > 0 else 0
177
178     elif lvl_str == AggregationLevel.DAILY.value:
179         s_epoch = float(start_epoch) if start_epoch is not Non
180         e else 0.0
181         e_epoch = float(end_epoch) if end_epoch is not None el
182         se (int(now_epoch // 86400) * 86400.0)
183
184         cursor.execute("""
185             INSERT INTO telemetry_daily (
186                 sensor_id,
187                 bucket_start,
188                 bucket_end,
189                 sample_count,
190                 value_min,
191                 value_max,
192                 value_avg,
193                 value_sum,
194                 value_sum_sq,
195                 value_stddev,
196                 value_first,
197                 value_last

```

```

192 |         )
193 |     WITH hourly_window AS (
194 |         SELECT
195 |             sensor_id,
196 |             CAST(bucket_start / 86400 AS INTEGER) * 86
197 | 400 AS b_start,
198 |             bucket_start,
199 |             sample_count,
200 |             value_min,
201 |             value_max,
202 |             value_sum,
203 |             value_sum_sq,
204 |             value_first,
205 |             value_last,
206 |             FIRST_VALUE(value_first) OVER (
207 |                 PARTITION BY sensor_id, CAST(bucket_start
208 | art / 86400 AS INTEGER) * 86400
209 |                 ORDER BY bucket_start ASC
210 |             ) AS f_val,
211 |             FIRST_VALUE(value_last) OVER (
212 |                 PARTITION BY sensor_id, CAST(bucket_start
213 | art / 86400 AS INTEGER) * 86400
214 |                 ORDER BY bucket_start DESC
215 |             ) AS l_val
216 |         FROM telemetry_hourly
217 |         WHERE bucket_start >= ? AND bucket_start < ?
218 |     )
219 |     SELECT
220 |         sensor_id,
221 |         b_start AS bucket_start,
222 |         b_start + 86400 AS bucket_end,
223 |         SUM(sample_count) AS sample_count,
224 |         MIN(value_min) AS value_min,
225 |         MAX(value_max) AS value_max,
226 |         SUM(value_sum) / SUM(sample_count) AS value_av
227 | g,
228 |         SUM(value_sum) AS value_sum,
229 |         SUM(value_sum_sq) AS value_sum_sq,

```

```

226 |             Sqrt(MAX(0.0, (SUM(value_sum_sq) - (SUM(value_
| sum) * SUM(value_sum)) / SUM(sample_count)) / SUM(sample_count
| ))) AS value_stddev,
227 |             MIN(f_val) AS value_first,
228 |             MAX(l_val) AS value_last
229 | FROM hourly_window
230 | GROUP BY sensor_id, b_start
231 | ON CONFLICT(sensor_id, bucket_start) DO UPDATE SET
232 |     bucket_end = excluded.bucket_end,
233 |     sample_count = excluded.sample_count,
234 |     value_min = excluded.value_min,
235 |     value_max = excluded.value_max,
236 |     value_avg = excluded.value_avg,
237 |     value_sum = excluded.value_sum,
238 |     value_sum_sq = excluded.value_sum_sq,
239 |     value_stddev = excluded.value_stddev,
240 |     value_first = excluded.value_first,
241 |     value_last = excluded.value_last;
242 |     """, (s_epoch, e_epoch))
243 |     res["buckets_aggregated"] = cursor.rowcount if cursor.
| rowcount > 0 else 0
244 |
245 |     elif lvl_str == AggregationLevel.WEEKLY.value:
246 |         s_epoch = float(start_epoch) if start_epoch is not Non
| e else 0.0
247 |         e_epoch = float(end_epoch) if end_epoch is not None el
| se (int(now_epoch // 604800) * 604800.0)
248 |
249 |         cursor.execute("""
250 |             INSERT INTO telemetry_weekly (
251 |                 sensor_id,
252 |                 bucket_start,
253 |                 bucket_end,
254 |                 sample_count,
255 |                 value_min,
256 |                 value_max,
257 |                 value_avg,
258 |                 value_sum,

```

```

259         value_sum_sq,
260         value_stddev,
261         value_first,
262         value_last
263     )
264     WITH daily_window AS (
265         SELECT
266             sensor_id,
267             CAST(bucket_start / 604800 AS INTEGER) * 6
268             04800 AS b_start,
269             bucket_start,
270             sample_count,
271             value_min,
272             value_max,
273             value_sum,
274             value_sum_sq,
275             value_first,
276             value_last,
277             FIRST_VALUE(value_first) OVER (
278                 PARTITION BY sensor_id, CAST(bucket_start / 604800 AS INTEGER) * 604800
279                 ORDER BY bucket_start ASC
280             ) AS f_val,
281             FIRST_VALUE(value_last) OVER (
282                 PARTITION BY sensor_id, CAST(bucket_start / 604800 AS INTEGER) * 604800
283                 ORDER BY bucket_start DESC
284             ) AS l_val
285         FROM telemetry_daily
286         WHERE bucket_start >= ? AND bucket_start < ?
287     )
288     SELECT
289         sensor_id,
290         b_start AS bucket_start,
291         b_start + 604800 AS bucket_end,
292         SUM(sample_count) AS sample_count,
293         MIN(value_min) AS value_min,
294         MAX(value_max) AS value_max,

```

```

294         SUM(value_sum) / SUM(sample_count) AS value_av
    g,
295         SUM(value_sum) AS value_sum,
296         SUM(value_sum_sq) AS value_sum_sq,
297         SQRT(MAX(0.0, (SUM(value_sum_sq) - (SUM(value_
sum) * SUM(value_sum)) / SUM(sample_count)) / SUM(sample_count
))) AS value_stddev,
298         MIN(f_val) AS value_first,
299         MAX(l_val) AS value_last
300     FROM daily_window
301     GROUP BY sensor_id, b_start
302     ON CONFLICT(sensor_id, bucket_start) DO UPDATE SET
303         bucket_end = excluded.bucket_end,
304         sample_count = excluded.sample_count,
305         value_min = excluded.value_min,
306         value_max = excluded.value_max,
307         value_avg = excluded.value_avg,
308         value_sum = excluded.value_sum,
309         value_sum_sq = excluded.value_sum_sq,
310         value_stddev = excluded.value_stddev,
311         value_first = excluded.value_first,
312         value_last = excluded.value_last;
313     """, (s_epoch, e_epoch))
314     res["buckets_aggregated"] = cursor.rowcount if cursor.
rowcount > 0 else 0
315
316     elif lvl_str == AggregationLevel.MONTHLY.value:
317         s_epoch = float(start_epoch) if start_epoch is not Non
e else 0.0
318         e_epoch = float(end_epoch) if end_epoch is not None el
se now_epoch
319
320         cursor.execute("""
321             INSERT INTO telemetry_monthly (
322                 sensor_id,
323                 bucket_start,
324                 bucket_end,
325                 sample_count,

```

```

326         value_min,
327         value_max,
328         value_avg,
329         value_sum,
330         value_sum_sq,
331         value_stddev,
332         value_first,
333         value_last
334     )
335     WITH daily_window AS (
336         SELECT
337             sensor_id,
338             CAST(strftime('%s', datetime(bucket_start,
339 'unixepoch', 'start of month')) AS INTEGER) AS b_start,
340             CAST(strftime('%s', datetime(bucket_start,
341 'unixepoch', 'start of month', '+1 month')) AS INTEGER) AS b_
342 end,
343             bucket_start,
344             sample_count,
345             value_min,
346             value_max,
347             value_sum,
348             value_sum_sq,
349             value_first,
350             value_last,
351             FIRST_VALUE(value_first) OVER (
352                 PARTITION BY sensor_id, strftime('%Y-%
353 m', datetime(bucket_start, 'unixepoch'))
354                 ORDER BY bucket_start ASC
355             ) AS f_val,
356             FIRST_VALUE(value_last) OVER (
357                 PARTITION BY sensor_id, strftime('%Y-%
358 m', datetime(bucket_start, 'unixepoch'))
359                 ORDER BY bucket_start DESC
360             ) AS l_val
361         FROM telemetry_daily
362         WHERE bucket_start >= ? AND bucket_start < ?
363     )

```

```

359         SELECT
360             sensor_id,
361             b_start AS bucket_start,
362             MAX(b_end) AS bucket_end,
363             SUM(sample_count) AS sample_count,
364             MIN(value_min) AS value_min,
365             MAX(value_max) AS value_max,
366             SUM(value_sum) / SUM(sample_count) AS value_av
367         g,
368             SUM(value_sum) AS value_sum,
369             SUM(value_sum_sq) AS value_sum_sq,
370             SQRT(MAX(0.0, (SUM(value_sum_sq) - (SUM(value_
371 sum) * SUM(value_sum)) / SUM(sample_count)) / SUM(sample_count
372 ))) AS value_stddev,
373             MIN(f_val) AS value_first,
374             MAX(l_val) AS value_last
375     FROM daily_window
376     GROUP BY sensor_id, b_start
377     ON CONFLICT(sensor_id, bucket_start) DO UPDATE SET
378         bucket_end = excluded.bucket_end,
379         sample_count = excluded.sample_count,
380         value_min = excluded.value_min,
381         value_max = excluded.value_max,
382         value_avg = excluded.value_avg,
383         value_sum = excluded.value_sum,
384         value_sum_sq = excluded.value_sum_sq,
385         value_stddev = excluded.value_stddev,
386         value_first = excluded.value_first,
387         value_last = excluded.value_last;
388     "", (s_epoch, e_epoch))
389     res["buckets_aggregated"] = cursor.rowcount if cursor.
390 rowcount > 0 else 0
391
392     elif lvl_str == AggregationLevel.YEARLY.value:
393         s_epoch = float(start_epoch) if start_epoch is not Non
394 e else 0.0
395         e_epoch = float(end_epoch) if end_epoch is not None el
396 se now_epoch

```



```

391 |
392 |         cursor.execute("""
393 |             INSERT INTO telemetry_yearly (
394 |                 sensor_id,
395 |                 bucket_start,
396 |                 bucket_end,
397 |                 sample_count,
398 |                 value_min,
399 |                 value_max,
400 |                 value_avg,
401 |                 value_sum,
402 |                 value_sum_sq,
403 |                 value_stddev,
404 |                 value_first,
405 |                 value_last
406 |             )
407 |             WITH monthly_window AS (
408 |                 SELECT
409 |                     sensor_id,
410 |                     CAST(strftime('%s', datetime(bucket_start,
411 | 'unixepoch', 'start of year')) AS INTEGER) AS b_start,
412 |                     CAST(strftime('%s', datetime(bucket_start,
413 | 'unixepoch', 'start of year', '+1 year')) AS INTEGER) AS b_end,
414 |                     bucket_start,
415 |                     sample_count,
416 |                     value_min,
417 |                     value_max,
418 |                     value_sum,
419 |                     value_sum_sq,
420 |                     value_first,
421 |                     value_last,
422 |                     FIRST_VALUE(value_first) OVER (
423 |                         PARTITION BY sensor_id, strftime('%Y',
424 | datetime(bucket_start, 'unixepoch'))
425 |                         ORDER BY bucket_start ASC
426 |                     ) AS f_val,
427 |                     FIRST_VALUE(value_last) OVER (

```

```

425 |             PARTITION BY sensor_id, strftime('%Y',
|             datetime(bucket_start, 'unixepoch'))
426 |             ORDER BY bucket_start DESC
427 |             ) AS l_val
428 |         FROM telemetry_monthly
429 |         WHERE bucket_start >= ? AND bucket_start < ?
430 |     )
431 |     SELECT
432 |         sensor_id,
433 |         b_start AS bucket_start,
434 |         MAX(b_end) AS bucket_end,
435 |         SUM(sample_count) AS sample_count,
436 |         MIN(value_min) AS value_min,
437 |         MAX(value_max) AS value_max,
438 |         SUM(value_sum) / SUM(sample_count) AS value_av
|     g,
439 |         SUM(value_sum) AS value_sum,
440 |         SUM(value_sum_sq) AS value_sum_sq,
441 |         SQRT(MAX(0.0, (SUM(value_sum_sq) - (SUM(value_
| sum) * SUM(value_sum)) / SUM(sample_count)) / SUM(sample_count
| ))) AS value_stddev,
442 |         MIN(f_val) AS value_first,
443 |         MAX(l_val) AS value_last
444 |     FROM monthly_window
445 |     GROUP BY sensor_id, b_start
446 |     ON CONFLICT(sensor_id, bucket_start) DO UPDATE SET
447 |         bucket_end = excluded.bucket_end,
448 |         sample_count = excluded.sample_count,
449 |         value_min = excluded.value_min,
450 |         value_max = excluded.value_max,
451 |         value_avg = excluded.value_avg,
452 |         value_sum = excluded.value_sum,
453 |         value_sum_sq = excluded.value_sum_sq,
454 |         value_stddev = excluded.value_stddev,
455 |         value_first = excluded.value_first,
456 |         value_last = excluded.value_last;
457 |     "", (s_epoch, e_epoch))
458 |     res["buckets_aggregated"] = cursor.rowcount if cursor.

```

```

    rowcount > 0 else 0
459
460     else:
461         raise ValueError(f"Неизвестный уровень агрегации: {lvl
    _str}")
462
463     return res
464
465
466 class TelemetrySqlAggregator:
467     """Управляет вызовами SQL-агрегации и каскадным выполнением
    цепочки агрегаторов."""
468
469     def __init__(self, connection_manager: TelemetryConnection
    Manager) -> None:
470         """Инициализирует SQL-агрегатор телеметрии.
471
472         Args:
473             connection_manager: Менеджер подключений SQLite.
474         """
475         self._cm = connection_manager
476         self._scheduler_timer: Optional[threading.Timer] = None
477
478         self._scheduler_interval: float = 300.0
479         self._scheduler_running: bool = False
480
481     def start_background_scheduler(self, interval_seconds: float = 300.0) -> None:
482         """Запускает фоновый регламентный планировщик агрегации.
483
484         Args:
485             interval_seconds: Периодичность запуска каскада агрегации в секундах (по умолчанию 300с = 5 мин).
486         """
487         if self._cm.read_only:
488             return
489         self.stop_background_scheduler()

```

```

489         self._scheduler_interval = max(10.0, float(interval_se
conds))
490         self._scheduler_running = True
491         self._schedule_next_run()
492         logger.info(f"Запущен фоновый регламентный планировщик
агрегации (квант: {self._scheduler_interval}с)")
493
494     def stop_background_scheduler(self) -> None:
495         """Останавливает фоновый регламентный планировщик агре
гации."""
496         self._scheduler_running = False
497         if self._scheduler_timer:
498             try:
499                 self._scheduler_timer.cancel()
500             except Exception:
501                 pass
502             self._scheduler_timer = None
503
504     def _schedule_next_run(self) -> None:
505         if not self._scheduler_running or self._cm.read_only:
506             return
507         self._scheduler_timer = threading.Timer(self._schedule
r_interval, self._on_scheduler_tick)
508         self._scheduler_timer.daemon = True
509         self._scheduler_timer.start()
510
511     def _on_scheduler_tick(self) -> None:
512         if not self._scheduler_running or self._cm.read_only:
513             return
514         try:
515             self.run_pipeline()
516         except Exception as ex:
517             logger.debug(f"Ошибка регламентной фоновой агрегац
ии: {ex}")
518         finally:
519             self._schedule_next_run()
520
521     def aggregate(

```

```

522         self,
523         level: Union[AggregationLevel, str],
524         start_epoch: Optional[float] = None,
525         end_epoch: Optional[float] = None,
526         detect_spikes: bool = True,
527     ) -> Dict[str, Any]:
528         """Запускает агрегацию указанного уровня в транзакции.
529
530         Args:
531             level: Уровень агрегации ('hourly', 'daily', 'weekly', 'monthly', 'yearly').
532             start_epoch: Начальная граница диапазона.
533             end_epoch: Конечная граница диапазона.
534             detect_spikes: Флаг фиксации всплесков.
535
536         Returns:
537             Dict[str, Any]: Результаты агрегации.
538         """
539         if self._cm.read_only:
540             return {"level": str(level), "buckets_aggregated":
541                     0, "spikes_recorded": 0, "read_only": True}
542
543         with self._cm.lock, self._cm.get_connection() as conn:
544             result = sensors_aggregate(
545                 conn=conn,
546                 level=level,
547                 start_epoch=start_epoch,
548                 end_epoch=end_epoch,
549                 detect_spikes=detect_spikes,
550             )
551             conn.commit()
552             return result
553
554     def aggregate_hourly(self, start_epoch: Optional[float] =
555         None, end_epoch: Optional[float] = None, detect_spikes: bool =

```

```

    poch=start_epoch, end_epoch=end_epoch, detect_spikes=detect_sp
    ikes)
556
557     def aggregate_daily(self, start_epoch: Optional[float] = N
one, end_epoch: Optional[float] = None) -> Dict[str, Any]:
558         """Выполняет суточную агрегацию из почасовых бакетов."
"""
559         return self.aggregate(AggregationLevel.DAILY, start_ep
och=start_epoch, end_epoch=end_epoch)
560
561     def aggregate_weekly(self, start_epoch: Optional[float] =
None, end_epoch: Optional[float] = None) -> Dict[str, Any]:
562         """Выполняет недельную агрегацию из суточных бакетов."
"""
563         return self.aggregate(AggregationLevel.WEEKLY, start_e
poch=start_epoch, end_epoch=end_epoch)
564
565     def aggregate_monthly(self, start_epoch: Optional[float] =
None, end_epoch: Optional[float] = None) -> Dict[str, Any]:
566         """Выполняет месячную агрегацию из суточных бакетов.""
"
567         return self.aggregate(AggregationLevel.MONTHLY, start_
epoch=start_epoch, end_epoch=end_epoch)
568
569     def aggregate_yearly(self, start_epoch: Optional[float] =
None, end_epoch: Optional[float] = None) -> Dict[str, Any]:
570         """Выполняет годовую агрегацию из месячных бакетов."""
571         return self.aggregate(AggregationLevel.YEARLY, start_e
poch=start_epoch, end_epoch=end_epoch)
572
573     def run_pipeline(
574         self,
575         start_epoch: Optional[float] = None,
576         end_epoch: Optional[float] = None,
577         detect_spikes: bool = True,
578     ) -> Dict[str, Any]:
579         """Выполняет полный каскад агрегации: raw -> hourly ->
daily -> weekly -> monthly -> yearly в одной транзакции.

```

```

580 |
581 |     Args:
582 |         start_epoch: Начальная граница диапазона.
583 |         end_epoch: Конечная граница диапазона.
584 |         detect_spikes: Флаг фиксации всплесков.
585 |
586 |     Returns:
587 |         Dict[str, Any]: Сводная статистика всех уровней аг-
    | регации.
588 |         """
589 |         if self._cm.read_only:
590 |             return {"read_only": True}
591 |
592 |         with self._cm.lock, self._cm.get_connection() as conn:
593 |             h_res = sensors_aggregate(conn, AggregationLevel.H
    | OURLY, start_epoch, end_epoch, detect_spikes)
594 |             d_res = sensors_aggregate(conn, AggregationLevel.D
    | AILY, start_epoch, end_epoch, False)
595 |             w_res = sensors_aggregate(conn, AggregationLevel.W
    | EEKLY, start_epoch, end_epoch, False)
596 |             m_res = sensors_aggregate(conn, AggregationLevel.M
    | ONTHLY, start_epoch, end_epoch, False)
597 |             y_res = sensors_aggregate(conn, AggregationLevel.Y
    | EARLY, start_epoch, end_epoch, False)
598 |             conn.commit()
599 |
600 |         return {
601 |             "hourly": h_res,
602 |             "daily": d_res,
603 |             "weekly": w_res,
604 |             "monthly": m_res,
605 |             "yearly": y_res,
606 |         }
607 |

```

buffer.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry Sqlite -
  Buffer
4 # =====
5 # Description:
6 #   Управление буферизацией и пакетной записью (batching) пото
  ка телеметрии в SQLite.
7 #
8 #   Зачем нужен этот модуль:
9 #       1. Снижение нагрузки на диск (I/O) и ускорение SQLite:
10 #          Объединяет одиночные события метрик в пачки, выполняя
  одну быструю
11 #          транзакцию вместо сотен мелких операций записи с част
  ыми fsync/commit.
12 #       2. Предотвращение блокировок базы данных:
13 #          Минимизирует время удержания эксклюзивных транзакций
  SQLite, предотвращая
14 #          ошибки 'database is locked' при одновременном чтении
  данных агентами и UI.
15 #       3. Защита от потери данных (Fail-Safe / Disaster Recover
  y):
16 #          При сбоях записи или аварийном завершении удерживает
  данные во внешнем
17 #          журнале и автоматически восстанавливает/дописывает их
  при следующем старте.
18 #
19 #   Поддерживаемые режимы буферизации (buffer_mode):
20 #       - 'memory' (По умолчанию):
21 #          Накапливает записи в оперативной памяти (список в RA
  M).
22 #          Сброс в БД происходит по достижении buffer_size или
  по таймеру flush_interval_seconds.
23 #          Обеспечивает максимальную скорость работы.
```



```
24 # - 'file':
25 #     Каждая входящая запись немедленно сериализуется и до
    # писывается на диск
26 #     в аварийный JSONL-файл (telemetry_buffer.jsonl) с вы
    # зовом fsync.
27 #     По достижении buffer_size или по таймеру весь файл с
    # брасывается в SQLite
28 #     и усекается. Защищает от потери данных при внезапном
    # падении процесса/питания.
29 # - 'direct':
30 #     Отключает фоновый периодический таймер автосброса. За
    # писи накапливаются в RAM
31 #     и сбрасываются только при заполнении buffer_size или
    # явном вызове flush().
32 #
33 # Usage Examples:
34 # Python API:
35 # from apps.windows.telemetry.sqlite.buffer import TelemetryBuffer
36 # from apps.windows.telemetry.sqlite.connection import TelemetryConnectionManager
37 # from apps.windows.telemetry.sqlite.writer import TelemetryWriter
38 #
39 # cm = TelemetryConnectionManager("telemetry.db")
40 # writer = TelemetryWriter(cm)
41 # buf = TelemetryBuffer(
42 #     connection_manager=cm,
43 #     writer=writer,
44 #     buffer_mode='memory', # 'memory' | 'file' | 'direct'
    #
45 #     buffer_size=50,
46 #     flush_interval_seconds=30.0
47 # )
48 # buf.enqueue({"timestamp": "...", "cpu_percent": 12.5})
49 #
50 # File: buffer.py
51 # Project: ai-breadboard
```

```

52 # Package: apps.windows.telemetry.sqlite
53 # Author: hypo69
54 # Copyright: © 2026 hypo69
55 # Updated: 2026-10-06 17:47:00
56 # =====
    =====
57
58 from __future__ import annotations
59
60 import json
61 import os
62 import sys
63 import threading
64 from pathlib import Path
65 from typing import Any, Dict, List, Optional, Union
66
67 from .connection import TelemetryConnectionManager
68 from .writer import TelemetryWriter
69
70 try:
71     from logger import logger
72 except ImportError:
73     import logging
74     logger = logging.getLogger(__name__)
75
76
77 class TelemetryBuffer:
78     """Управляет буферизацией событий телеметрии в памяти и аварийном JSONL файле с контролем лимитов RAM."""
79
80     def __init__(
81         self,
82         connection_manager: TelemetryConnectionManager,
83         writer: TelemetryWriter,
84         buffer_mode: str = 'memory',
85         buffer_size: int = 50,
86         flush_interval_seconds: float = 30.0,
87         buffer_file_path: Optional[Union[str, Path]] = None,

```

```

88         auto_flush: bool = True,
89         max_buffer_bytes: int = 32 * 1024 * 1024,
90     ) -> None:
91         """Инициализирует буфер сброса телеметрии.
92
93         Args:
94             connection_manager: Менеджер подключений SQLite.
95             writer: Писатель пакетов телеметрии.
96             buffer_mode: Режим буферизации ('memory', 'file',
97             'direct').
98             buffer_size: Максимальный размер пачки до сброса.
99             flush_interval_seconds: Интервал таймера сброса в
100             секундах.
101             buffer_file_path: Путь к файлу аварийного буфера.
102             auto_flush: Флаг автоматического запуска таймера с
103             броса.
104             max_buffer_bytes: Максимальный лимит памяти для за
105             писей буфера в байтах (по умолчанию 32 МБ).
106         """
107         self._cm = connection_manager
108         self._writer = writer
109         self._buffer_mode = buffer_mode.lower() if buffer_mode
110         in ('memory', 'file', 'direct') else 'memory'
111         self._buffer_size = max(1, int(buffer_size))
112         self._max_buffer_bytes = max(1024 * 1024, int(max_buff
113         er_bytes))
114         self._current_buffer_bytes: int = 0
115         self._flush_interval_seconds = max(0.5, float(flush_in
116         terval_seconds))
117         self._auto_flush = bool(auto_flush) and not self._cm.r
118         ead_only
119
120         if buffer_file_path:
121             self._buffer_file_path = Path(buffer_file_path)
122         else:
123             self._buffer_file_path = self._cm.db_path.parent /
124             'telemetry_buffer.jsonl'

```

```
117         if not self._cm.read_only:
118             self._buffer_file_path.parent.mkdir(parents=True,
119             exist_ok=True)
120
121         self._buffer: List[Dict[str, Any]] = []
122         self._file_buffer_count: int = 0
123         self._is_running = True
124         self._flush_timer: Optional[threading.Timer] = None
125
126         if not self._cm.read_only:
127             self._recover_file_buffer()
128             if self._auto_flush and self._buffer_mode != 'direct':
129                 self._start_flush_timer()
130
131     @property
132     def buffer_mode(self) -> str:
133         """Текущий режим буферизации."""
134         return self._buffer_mode
135
136     @property
137     def buffer_size(self) -> int:
138         """Максимальный размер буфера в количестве записей."""
139         return self._buffer_size
140
141     @property
142     def max_buffer_bytes(self) -> int:
143         """Максимальный лимит памяти буфера в байтах."""
144         return self._max_buffer_bytes
145
146     @property
147     def current_buffer_bytes(self) -> int:
148         """Текущий объем занимаемой памяти буфера в байтах."""
149         with self._cm.lock:
150             return self._current_buffer_bytes
151
152     @property
153     def flush_interval_seconds(self) -> float:
```

```

153         """Интервал таймера автосброса в секундах."""
154         return self._flush_interval_seconds
155
156     @property
157     def buffer_file_path(self) -> Path:
158         """Путь к файлу аварийного JSONL буфера."""
159         return self._buffer_file_path
160
161     def _estimate_size(self, record: Dict[str, Any]) -> int:
162         """Оценивает объем памяти записи в байтах."""
163         try:
164             return len(json.dumps(record, ensure_ascii=False,
165 default=str).encode('utf-8')) + 128
166         except Exception:
167             return sys.getsizeof(record) + 256
168
169     def set_flush_interval_seconds(self, interval: float) -> None:
170         """Динамически изменяет интервал периодического сброса
171         буфера в SQLite."""
172         new_interval = max(0.5, float(interval))
173         if abs(self._flush_interval_seconds - new_interval) >
174 0.01:
175             with self._cm.lock:
176                 self._flush_interval_seconds = new_interval
177                 if self._auto_flush and self._buffer_mode != '
178 direct' and not self._cm.read_only:
179                     self._start_flush_timer()
180                     logger.info(f"Интервал сброса буфера телеметри
181 и обновлен: {self._flush_interval_seconds}с")
182
183     def set_buffer_mode(self, mode: str) -> None:
184         """Переключает режим буферизации."""
185         m = mode.lower()
186         if m in ('memory', 'file', 'direct'):
187             with self._cm.lock:
188                 self.flush()
189                 self._buffer_mode = m

```

```

185         if m == 'direct':
186             self._stop_flush_timer()
187             elif self._auto_flush and not self._cm.read_on
ly:
188                 self._start_flush_timer()
189                 logger.info(f'Режим буферизации переключен на:
{m}')
190
191     def get_buffered_count(self) -> int:
192         """Возвращает число накопленных записей в буфере."""
193         with self._cm.lock:
194             return len(self._buffer) + self._file_buffer_count
195
196     def _start_flush_timer(self) -> None:
197         if self._cm.read_only or not self._is_running or self.
_buffer_mode == 'direct':
198             return
199         self._stop_flush_timer()
200         self._flush_timer = threading.Timer(self._flush_interv
al_seconds, self._on_flush_timer)
201         self._flush_timer.daemon = True
202         self._flush_timer.start()
203
204     def _stop_flush_timer(self) -> None:
205         if self._flush_timer is not None:
206             try:
207                 self._flush_timer.cancel()
208             except Exception:
209                 pass
210             self._flush_timer = None
211
212     def _on_flush_timer(self) -> None:
213         if not self._is_running or self._cm.read_only:
214             return
215         try:
216             self.flush()
217         except Exception as ex:
218             logger.debug(f'Ошибка фонового сброса буфера телем

```

```

    етрии: {ex}')
219         finally:
220             with self._cm.lock:
221                 if self._is_running and self._auto_flush and s
self._buffer_mode != 'direct':
222                     self._start_flush_timer()
223
224     def _recover_file_buffer(self) -> None:
225         if self._cm.read_only:
226             return
227         if self._buffer_file_path.exists() and self._buffer_fi
le_path.stat().st_size > 0:
228             try:
229                 with open(self._buffer_file_path, 'r', encodin
g='utf-8') as f:
230                     count = sum(1 for line in f if line.strip(
))
231                     self._file_buffer_count = count
232                     if count > 0:
233                         logger.info(f'Обнаружено {count} сохраненн
ых записей в {self._buffer_file_path.name}. Сброс в БД...')
234                         self.flush()
235                     except Exception as ex:
236                         logger.warning(f'Ошибка проверки файла буфера
телеметрии: {ex}')
```

```

237
238     def enqueue(self, record: Dict[str, Any]) -> None:
239         """Добавляет запись в буфер в зависимости от выбранног
о режима."""
240         if self._cm.read_only:
241             logger.warning('Попытка добавления записи в буфер
в режиме Read-Only')
242             return
243
244         rec_size = self._estimate_size(record)
245         with self._cm.lock:
246             if self._buffer_mode == 'file':
247                 try:

```

```

248         line = json.dumps(record, ensure_ascii=False, default=str)
249         with open(self._buffer_file_path, 'a', encoding='utf-8') as f:
250             f.write(line + '\n')
251             f.flush()
252             os.fsync(f.fileno())
253             self._file_buffer_count += 1
254         except Exception as ex:
255             logger.error(f'Ошибка записи в файл буфера
телеметрии: {ex}')
256             self._buffer.append(record)
257             self._current_buffer_bytes += rec_size
258
259             if self._file_buffer_count >= self._buffer_size or self._current_buffer_bytes >= self._max_buffer_bytes:
260                 self.flush()
261             else:
262                 self._buffer.append(record)
263                 self._current_buffer_bytes += rec_size
264                 if len(self._buffer) >= self._buffer_size or self._current_buffer_bytes >= self._max_buffer_bytes:
265                     self.flush()
266
267     def flush(self) -> int:
268         """Принудительно сбрасывает записи в SQLite."""
269         if self._cm.read_only:
270             return 0
271
272         with self._cm.lock:
273             records_to_save: List[Dict[str, Any]] = []
274
275             if self._buffer_file_path.exists() and self._buffer_file_path.stat().st_size > 0:
276                 try:
277                     with open(self._buffer_file_path, 'r', encoding='utf-8') as f:
278                         for line in f:

```



```

279 |             s = line.strip()
280 |             if s:
281 |                 try:
282 |                     records_to_save.append(jso
n.loads(s))
283 |                 except Exception:
284 |                     pass
285 |             except Exception as read_err:
286 |                 logger.error(f'Ошибка чтения файла буфера:
{read_err}')
287 |
288 |         if self._buffer:
289 |             records_to_save.extend(self._buffer)
290 |             self._buffer = []
291 |             self._current_buffer_bytes = 0
292 |
293 |         if not records_to_save:
294 |             return 0
295 |
296 |         inserted_count = self._writer.batch_insert_records
(records_to_save)
297 |
298 |         if self._buffer_file_path.exists():
299 |             try:
300 |                 with open(self._buffer_file_path, 'w', enc
oding='utf-8') as f:
301 |                     f.truncate(0)
302 |                     self._file_buffer_count = 0
303 |             except Exception as trunc_err:
304 |                 logger.warning(f'Ошибка усечения файла буф
ера: {trunc_err}')
305 |
306 |         if inserted_count > 0:
307 |             logger.debug(f'[Хранилище SQLite] Пакетный сброс буфера: {inserted_count} записей зафиксировано в {self._cm.db_path.name}')
308 |
309 |         return inserted_count

```

```
310 |
311 |     def close(self) -> None:
312 |         """Останавливает таймер и выполняет финальный сброс бу
313 |         фера."""
314 |         self._is_running = False
315 |         self._stop_flush_timer()
316 |         if not self._cm.read_only:
317 |             try:
318 |                 self.flush()
319 |             except Exception:
320 |                 pass
```

connection.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry Sqlite -
  Connection
4 # =====
5 # Description:
6 #   Менеджер подключений к SQLite базе данных с поддержкой реж
  има только чтение.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.sqlite.connection import Tel
  emetryConnectionManager
11 #
12 #       cm = TelemetryConnectionManager(db_path=Path("logs/telem
  etry.db"), read_only=True)
13 #       with cm.get_connection() as conn:
14 #           ...
15 #
16 # File: connection.py
17 # Project: ai-breadboard
18 # Package: apps.windows.telemetry.sqlite
19 # Author: hypo69
20 # Copyright: © 2026 hypo69
21 # Updated: 2026-10-06 05:32:00
22 # =====
23
24 from __future__ import annotations
25
26 """Управление подключениями к базе данных SQLite с поддержкой
  режимов Read-Only и WAL."""
27
28 import sqlite3
```

```
29 import threading
30 from pathlib import Path
31 from typing import Optional, Union
32
33 try:
34     from logger import logger
35 except ImportError:
36     import logging
37     logger = logging.getLogger(__name__)
38
39
40 class TelemetryConnectionManager:
41     """Менеджер создания и настройки соединений к SQLite базе
42     данных телеметрии."""
43
44     def __init__(
45         self,
46         db_path: Union[str, Path],
47         read_only: bool = False,
48         timeout: float = 15.0,
49         lock: Optional[threading.RLock] = None,
50     ) -> None:
51         """Инициализирует менеджер соединений.
52
53         Args:
54             db_path: Путь к файлу SQLite базы данных.
55             read_only: Флаг строгого режима только чтения (бло
56             кирует запись).
57             timeout: Таймаут ожидания освобождения базы данных
58             в секундах.
59             lock: Разделяемый RLock (опционально).
60
61         """
62         self.db_path = Path(db_path)
63         self.read_only = bool(read_only)
64         self.timeout = float(timeout)
65         self._lock = lock or threading.RLock()
66
67         if not self.read_only:
```

```

64         self.db_path.parent.mkdir(parents=True, exist_ok=True)
65
66     @property
67     def lock(self) -> threading.RLock:
68         """Возвращает разделяемый поток-безопасный лок."""
69         return self._lock
70
71     def get_connection(self) -> sqlite3.Connection:
72         """Создает и настраивает соединение SQLite с необходим
73         ыми PRAGMA директивами.
74
75         Returns:
76             sqlite3.Connection: Настроенное соединение.
77
78         """
79         if self.read_only:
80             db_uri = f"file:{self.db_path.as_posix()}?mode=ro"
81             try:
82                 conn = sqlite3.connect(db_uri, uri=True, timeout=self.timeout,
83                                         check_same_thread=False)
84             except sqlite3.OperationalError:
85                 conn = sqlite3.connect(str(self.db_path), timeout=self.timeout,
86                                         check_same_thread=False)
87             try:
88                 conn.execute('PRAGMA query_only = ON;')
89             except Exception:
90                 pass
91         else:
92             conn = sqlite3.connect(str(self.db_path), timeout=self.timeout,
93                                     check_same_thread=False)
94             conn.execute('PRAGMA journal_mode = WAL;')
95             conn.execute('PRAGMA synchronous = NORMAL;')
96             conn.execute('PRAGMA foreign_keys = ON;')
97             conn.execute('PRAGMA busy_timeout = 15000;')
98             conn.execute('PRAGMA cache_size = -64000;')
99             conn.execute('PRAGMA temp_store = MEMORY;')
100            conn.execute('PRAGMA mmap_size = 268435456;')
101            conn.execute('PRAGMA wal_autocheckpoint = 1000;')

```

```
97 |  
98 |     conn.row_factory = sqlite3.Row  
99 |     return conn  
100 |
```

maintenance.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry Sqlite -
  Maintenance
4 # =====
5 # Description:
6 #   Обслуживание, очистка, агрегация и контроль размера базы д
  анных SQLite.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.sqlite.maintenance import Te
    lemetryMaintenance
11 #
12 #       maint = TelemetryMaintenance(connection_manager)
13 #       stats = maint.get_storage_stats()
14 #
15 # File: maintenance.py
16 # Project: ai-breadboard
17 # Package: apps.windows.telemetry.sqlite
18 # Author: hypo69
19 # Copyright: © 2026 hypo69
20 # Updated: 2026-10-08 11:55:00
21 # =====
22
23 from __future__ import annotations
24
25 """Методы обслуживания, очистки старых записей, дефрагментации
  и агрегации телеметрии в SQLite."""
26
27 import sqlite3
28 import time
29 from collections import defaultdict
```

```

30 from datetime import datetime, timezone, timedelta
31 from pathlib import Path
32 from typing import Any, Dict, List, Optional
33
34 from .connection import TelemetryConnectionManager
35
36 try:
37     from logger import logger
38 except ImportError:
39     import logging
40     logger = logging.getLogger(__name__)
41
42
43 class TelemetryMaintenance:
44     """Выполняет регламентные операции: очистка, контроль разм
    ера, вакуумирование и агрегации."""
45
46     def __init__(
47         self,
48         connection_manager: TelemetryConnectionManager,
49         max_db_size_mb: float = 50.0,
50         retention_days: int = 7,
51         auto_vacuum_enabled: bool = True,
52         buffer_file_path: Optional[Path] = None,
53     ) -> None:
54         """Инициализирует менеджер обслуживания базы данных.
55
56         Args:
57             connection_manager: Менеджер соединений SQLite.
58             max_db_size_mb: Максимальный размер файлов БД в МБ
59             .
60             retention_days: Срок хранения сырых записей в днях
61             .
62             auto_vacuum_enabled: Флаг автоматического запуска
63             VACUUM.
64             buffer_file_path: Путь к файлу буфера.
65         """
66         self._cm = connection_manager

```



```

64         self.max_db_size_mb = max(1.0, float(max_db_size_mb))
65         self.retention_days = max(1, int(retention_days))
66         self.auto_vacuum_enabled = bool(auto_vacuum_enabled)
67         self.buffer_file_path = buffer_file_path or (self._cm.
db_path.parent / 'telemetry_buffer.jsonl')
68
69     def get_db_file_sizes(self) -> Dict[str, int]:
70         """Возвращает физические размеры файлов БД на диске (D
B, WAL, SHM) в байтах."""
71         db_path = self._cm.db_path
72         db_bytes = db_path.stat().st_size if db_path.exists()
else 0
73         wal_path = db_path.with_name(db_path.name + '-wal')
74         shm_path = db_path.with_name(db_path.name + '-shm')
75         wal_bytes = wal_path.stat().st_size if wal_path.exists
() else 0
76         shm_bytes = shm_path.stat().st_size if shm_path.exists
() else 0
77         return {
78             'db_bytes': db_bytes,
79             'wal_bytes': wal_bytes,
80             'shm_bytes': shm_bytes,
81             'total_bytes': db_bytes + wal_bytes + shm_bytes,
82         }
83
84     def get_total_db_size_mb(self) -> float:
85         """Возвращает совокупный физический размер файлов БД в
МБ."""
86         sizes = self.get_db_file_sizes()
87         return round(sizes['total_bytes'] / (1024.0 * 1024.0),
3)
88
89     def checkpoint_wal(self) -> bool:
90         """Сбрасывает WAL-журнал в основной файл базы данных."
""
91         if self._cm.read_only:
92             return False
93         with self._cm.lock:

```

```

94         try:
95             with self._cm.get_connection() as conn:
96                 conn.execute('PRAGMA wal_checkpoint(TRUNCATE);')
97             return True
98         except Exception as e:
99             logger.debug(f'Ошибка сброса WAL-журнала: {e}')
100     )
101
102     def vacuum(self) -> bool:
103         """Выполняет сжатие и дефрагментацию базы данных."""
104         if self._cm.read_only:
105             return False
106         with self._cm.lock:
107             self.checkpoint_wal()
108             try:
109                 conn = sqlite3.connect(str(self._cm.db_path),
110                                         timeout=30.0, isolation_level=None)
111                 try:
112                     conn.execute('VACUUM;')
113                     finally:
114                         conn.close()
115                 logger.info(f'Выполнен VACUUM для базы данных
116 телеметрии ({self._cm.db_path})')
117                 return True
118             except Exception as e:
119                 logger.warning(f'Ошибка при выполнении VACUUM:
120 {e}')
121             return False
122
123     def get_storage_stats(self, buffered_count: int = 0, buffer_mode: str = 'memory', buffer_size: int = 50, flush_interval_seconds: float = 30.0) -> Dict[str, Any]:
124         """Возвращает агрегированную статистику таблиц, размера и параметров."""
125         with self._cm.lock, self._cm.get_connection() as conn:
126             cursor = conn.cursor()

```

```

124 |         cursor.execute('SELECT COUNT(*) FROM system_snapsh
    | ots;')
125 |         snap_count = cursor.fetchone()[0]
126 |         cursor.execute('SELECT COUNT(*) FROM process_snaps
    | hots;')
127 |         proc_count = cursor.fetchone()[0]
128 |         cursor.execute('SELECT COUNT(*) FROM sensor_polls;
    | ')
129 |         sensors_count = cursor.fetchone()[0]
130 |         cursor.execute('SELECT COUNT(*) FROM telemetry_eve
    | nts;')
131 |         events_count = cursor.fetchone()[0]
132 |         cursor.execute('SELECT COUNT(*) FROM hardware_audi
    | ts;')
133 |         audits_count = cursor.fetchone()[0]
134 |         cursor.execute('SELECT COUNT(*) FROM app_polls;')
135 |         app_polls_count = cursor.fetchone()[0]
136 |         cursor.execute('SELECT COUNT(*) FROM app_events;')
137 |         app_events_count = cursor.fetchone()[0]
138 |         cursor.execute('SELECT COUNT(*) FROM app_param_cha
    | nges;')
139 |         app_params_count = cursor.fetchone()[0]
140 |         cursor.execute('SELECT COUNT(*) FROM custom_record
    | s;')
141 |         custom_count = cursor.fetchone()[0]
142 |         cursor.execute('SELECT COUNT(*) FROM device_events
    | ;')
143 |         device_events_count = cursor.fetchone()[0]
144 |         cursor.execute('SELECT COUNT(*) FROM device_invent
    | ory;')
145 |         device_inventory_count = cursor.fetchone()[0]
146 |         cursor.execute('SELECT COUNT(*) FROM w64_events;')
147 |         w64_events_count = cursor.fetchone()[0]
148 |         cursor.execute('SELECT COUNT(*) FROM process_outli
    | ers;')
149 |         outliers_count = cursor.fetchone()[0]
150 |         cursor.execute('SELECT COUNT(*) FROM process_rollu
    | ps_2min;')

```

```

151 |         rollups_2min_count = cursor.fetchone()[0]
152 |         cursor.execute('SELECT COUNT(*) FROM process_rollu
    | ps_daily;')
153 |         rollups_daily_count = cursor.fetchone()[0]
154 |         cursor.execute('SELECT COUNT(*) FROM incidents;')
155 |         incidents_count = cursor.fetchone()[0]
156 |         cursor.execute('SELECT COUNT(*) FROM telemetry_rol
    | lups;')
157 |         rollups_count = cursor.fetchone()[0]
158 |
159 |         def _safe_count(table: str) -> int:
160 |             try:
161 |                 cursor.execute(f'SELECT COUNT(*) FROM {tab
    | le};')
162 |                 return cursor.fetchone()[0]
163 |             except Exception:
164 |                 return 0
165 |
166 |         telemetry_hourly_count = _safe_count('telemetry_ho
    | urly')
167 |         telemetry_daily_count = _safe_count('telemetry_dai
    | ly')
168 |         telemetry_weekly_count = _safe_count('telemetry_we
    | ekly')
169 |         telemetry_monthly_count = _safe_count('telemetry_m
    | onthly')
170 |         telemetry_yearly_count = _safe_count('telemetry_ye
    | arly')
171 |         telemetry_spikes_count = _safe_count('telemetry_sp
    | ikes')
172 |         disk_inventory_count = _safe_count('disk_inventory
    | ')
173 |         volume_inventory_count = _safe_count('volume_inven
    | tory')
174 |         disk_health_count = _safe_count('disk_health_snaps
    | hots')
175 |         disk_perf_count = _safe_count('disk_performance_sa
    | mples')

```

```

176         disk_io_count = _safe_count('disk_io_events')
177         cpu_inventory_count = _safe_count('cpu_inventory')
178         ram_module_inventory_count = _safe_count('ram_modu
le_inventory')
179         gpu_inventory_count = _safe_count('gpu_inventory')
180         network_adapter_inventory_count = _safe_count('net
work_adapter_inventory')
181         cpu_samples_count = _safe_count('cpu_telemetry_sam
ples')
182         ram_samples_count = _safe_count('ram_telemetry_sam
ples')
183         gpu_samples_count = _safe_count('gpu_telemetry_sam
ples')
184         network_samples_count = _safe_count('network_adapt
er_samples')
185
186         file_sizes = self.get_db_file_sizes()
187         size_mb = round(file_sizes['db_bytes'] / (1024 * 1
024), 2)
188         wal_size_mb = round(file_sizes['wal_bytes'] / (102
4 * 1024), 2)
189         total_size_mb = round(file_sizes['total_bytes'] /
(1024 * 1024), 2)
190         buffer_file_size = self.buffer_file_path.stat().st
_size if self.buffer_file_path.exists() else 0
191
192         return {
193             'db_path': str(self._cm.db_path),
194             'snapshots_count': snap_count,
195             'process_snapshots_count': proc_count,
196             'process_outliers_count': outliers_count,
197             'process_rollups_2min_count': rollups_2min_cou
nt,
198             'process_rollups_daily_count': rollups_daily_c
ount,
199             'incidents_count': incidents_count,
200             'telemetry_rollups_count': rollups_count,
201             'telemetry_hourly_count': telemetry_hourly_cou

```

```

nt,
202         'telemetry_daily_count': telemetry_daily_count
    ,
203         'telemetry_weekly_count': telemetry_weekly_cou
nt,
204         'telemetry_monthly_count': telemetry_monthly_c
ount,
205         'telemetry_yearly_count': telemetry_yearly_cou
nt,
206         'telemetry_spikes_count': telemetry_spikes_cou
nt,
207         'disk_inventory_count': disk_inventory_count,
208         'volume_inventory_count': volume_inventory_cou
nt,
209         'disk_health_count': disk_health_count,
210         'disk_perf_count': disk_perf_count,
211         'disk_io_count': disk_io_count,
212         'cpu_inventory_count': cpu_inventory_count,
213         'ram_module_inventory_count': ram_module_inven
tory_count,
214         'gpu_inventory_count': gpu_inventory_count,
215         'network_adapter_inventory_count': network_ada
pter_inventory_count,
216         'cpu_samples_count': cpu_samples_count,
217         'ram_samples_count': ram_samples_count,
218         'gpu_samples_count': gpu_samples_count,
219         'network_samples_count': network_samples_count
    ,
220         'sensor_polls_count': sensors_count,
221         'events_count': events_count,
222         'hardware_audits_count': audits_count,
223         'app_polls_count': app_polls_count,
224         'app_events_count': app_events_count,
225         'app_param_changes_count': app_params_count,
226         'custom_records_count': custom_count,
227         'device_events_count': device_events_count,
228         'device_inventory_count': device_inventory_cou
nt,

```

```

229         'w64_events_count': w64_events_count,
230         'file_size_mb': size_mb,
231         'wal_size_mb': wal_size_mb,
232         'total_size_mb': total_size_mb,
233         'max_db_size_mb': self.max_db_size_mb,
234         'retention_days': self.retention_days,
235         'size_limit_exceeded': total_size_mb > self.ma
x_db_size_mb,
236         'buffer_mode': buffer_mode,
237         'buffered_count': buffered_count,
238         'buffer_size': buffer_size,
239         'flush_interval_seconds': flush_interval_secon
ds,
240         'buffer_file_path': str(self.buffer_file_path)
,
241         'buffer_file_size_bytes': buffer_file_size,
242     }
243
244     def cleanup_old_records(self, retention_days: Optional[int
] = None, vacuum_after: bool = False) -> int:
245         """Удаляет устаревшие записи телеметрии."""
246         if self._cm.read_only:
247             return 0
248         days = retention_days if retention_days is not None el
se self.retention_days
249         threshold_epoch = datetime.now(timezone.utc).timestamp
() - (float(days) * 86400.0 if days > 0 else -1.0)
250         with self._cm.lock, self._cm.get_connection() as conn:
251             cursor = conn.cursor()
252             cursor.execute('SELECT id FROM system_snapshots WH
ERE created_at < ?', (threshold_epoch,))
253             old_ids = [row[0] for row in cursor.fetchall()]
254             if old_ids:
255                 cursor.execute('DELETE FROM process_snapshots
WHERE snapshot_id IN (SELECT id FROM system_snapshots WHERE cr
eated_at < ?)', (threshold_epoch,))
256                 cursor.execute('DELETE FROM system_snapshots W
HERE created_at < ?', (threshold_epoch,))

```

```
257 |         cursor.execute('DELETE FROM sensor_polls WHERE cre
      | ated_at < ?', (threshold_epoch,))
258 |         cursor.execute('DELETE FROM telemetry_events WHERE
      | created_at < ?', (threshold_epoch,))
259 |         cursor.execute('DELETE FROM app_polls WHERE create
      | d_at < ?', (threshold_epoch,))
260 |         cursor.execute('DELETE FROM app_events WHERE creat
      | ed_at < ?', (threshold_epoch,))
261 |         cursor.execute('DELETE FROM app_param_changes WHER
      | E created_at < ?', (threshold_epoch,))
262 |         cursor.execute('DELETE FROM custom_records WHERE c
      | reated_at < ?', (threshold_epoch,))
263 |         cursor.execute('DELETE FROM w64_events WHERE creat
      | ed_at < ?', (threshold_epoch,))
264 |         cursor.execute('DELETE FROM process_outliers WHERE
      | created_at < ?', (threshold_epoch,))
265 |         cursor.execute('DELETE FROM device_events WHERE cr
      | eated_at < ?', (threshold_epoch,))
266 |         cursor.execute('DELETE FROM system_extended_audits
      | WHERE created_at < ?', (threshold_epoch,))
267 |         cursor.execute('DELETE FROM disk_io_events WHERE c
      | reated_at < ?', (threshold_epoch,))
268 |         cursor.execute('DELETE FROM disk_performance_sampl
      | es WHERE created_at < ?', (threshold_epoch,))
269 |         cursor.execute('DELETE FROM disk_health_snapshots
      | WHERE created_at < ?', (threshold_epoch,))
270 |         cursor.execute('DELETE FROM cpu_telemetry_samples
      | WHERE created_at < ?', (threshold_epoch,))
271 |         cursor.execute('DELETE FROM ram_telemetry_samples
      | WHERE created_at < ?', (threshold_epoch,))
272 |         cursor.execute('DELETE FROM gpu_telemetry_samples
      | WHERE created_at < ?', (threshold_epoch,))
273 |         cursor.execute('DELETE FROM network_adapter_sample
      | s WHERE created_at < ?', (threshold_epoch,))
274 |
275 |         # Очистка посекундных снимков process_pid_snapshot
      | s и файловых событий старше 24 часов
276 |         pid_snap_threshold_str = (datetime.now(timezone.ut
```



```

c) - timedelta(days=min(days, 1)).isoformat()
277         cursor.execute('DELETE FROM process_pid_snapshots
WHERE timestamp < ?', (pid_snap_threshold_str,))
278         cursor.execute('DELETE FROM process_file_events WH
ERE timestamp < ?', (pid_snap_threshold_str,))
279
280         conn.commit()
281         deleted_count = len(old_ids)
282         logger.info(f'Очищено {deleted_count} устаревших с
нимков телеметрии (старше {days} дн.)')
283
284         if vacuum_after and self.auto_vacuum_enabled:
285             self.vacuum()
286
287         return deleted_count
288
289     def enforce_size_limit(self, max_size_mb: Optional[float]
= None, target_ratio: float = 0.85) -> Dict[str, Any]:
290         """Ограничивает совокупный размер БД телеметрии."""
291         if self._cm.read_only:
292             return {'pruned': False, 'read_only': True}
293         limit_mb = max_size_mb if max_size_mb is not None else
self.max_db_size_mb
294         initial_sizes = self.get_db_file_sizes()
295         initial_size_mb = round(initial_sizes['total_bytes'] /
(1024.0 * 1024.0), 3)
296
297         if initial_size_mb <= limit_mb:
298             return {
299                 'pruned': False,
300                 'initial_size_mb': initial_size_mb,
301                 'final_size_mb': initial_size_mb,
302                 'max_size_mb': limit_mb,
303                 'deleted_snapshots': 0,
304                 'deleted_records': 0,
305             }
306
307         target_mb = max(0.5, limit_mb * target_ratio)

```

```

308         logger.warning(f'Превышен лимит размера базы данных:
    {initial_size_mb:.2f} МБ > {limit_mb:.2f} МБ.')
309
310         total_deleted_snaps = self.cleanup_old_records(retenti
on_days=self.retention_days, vacuum_after=False)
311         self.checkpoint_wal()
312
313         with self._cm.lock, self._cm.get_connection() as conn:
314             cursor = conn.cursor()
315             while True:
316                 cursor.execute('SELECT COUNT(*) FROM system_sn
    apshots;')
317                 snap_count = cursor.fetchone()[0]
318                 if not snap_count or snap_count <= 5:
319                     break
320                 batch_size = max(5, int(snap_count * 0.25))
321                 cursor.execute('SELECT id, created_at FROM sys
    tem_snapshots ORDER BY created_at ASC LIMIT ?', (batch_size,))
322                 rows = cursor.fetchall()
323                 if not rows:
324                     break
325                 ids_to_del = [r[0] for r in rows]
326                 cutoff_t = rows[-1][1]
327                 placeholders = ','.join('?' for _ in ids_to_de
    l)
328                 cursor.execute(f'DELETE FROM process_snapshots
    WHERE snapshot_id IN ({placeholders})', ids_to_del)
329                 cursor.execute(f'DELETE FROM system_snapshots
    WHERE id IN ({placeholders})', ids_to_del)
330                 cursor.execute('DELETE FROM sensor_polls WHERE
    created_at <= ?', (cutoff_t,))
331                 cursor.execute('DELETE FROM w64_events WHERE c
    reated_at <= ?', (cutoff_t,))
332                 cursor.execute('DELETE FROM app_polls WHERE cr
    eated_at <= ?', (cutoff_t,))
333                 cursor.execute('DELETE FROM custom_records WHE
    RE created_at <= ?', (cutoff_t,))
334                 conn.commit()

```

```

335         total_deleted_snaps += len(ids_to_del)
336         if snap_count - len(ids_to_del) <= 5:
337             break
338
339     if self.auto_vacuum_enabled:
340         self.vacuum()
341     else:
342         self.checkpoint_wal()
343
344     final_sizes = self.get_db_file_sizes()
345     final_size_mb = round(final_sizes['total_bytes'] / (10
346 24.0 * 1024.0), 3)
347
348     return {
349         'pruned': True,
350         'initial_size_mb': initial_size_mb,
351         'final_size_mb': final_size_mb,
352         'max_size_mb': limit_mb,
353         'target_size_mb': target_mb,
354         'deleted_snapshots': total_deleted_snaps,
355     }
356
357     def aggregate_process_metrics_2min(self, cutoff_seconds: i
358 nt = 120, outlier_cpu_threshold: float = 30.0) -> Dict[str, in
359 t]:
360         """Обобщает процессы старше двух минут по средним пока
361 зателям."""
362         if self._cm.read_only:
363             return {'rollups_created': 0, 'outliers_saved': 0,
364 'raw_deleted': 0}
365         now_epoch = datetime.now(timezone.utc).timestamp()
366         cutoff_epoch = now_epoch - cutoff_seconds
367         with self._cm.lock, self._cm.get_connection() as conn:
368             cursor = conn.cursor()
369             cursor.execute('''
370                 SELECT p.id, p.snapshot_id, p.timestamp, p.pid
371                 , p.name, p.status,
372                 p.cpu_percent, p.memory_mb, p.memory_pe

```

```

rcent, p.num_threads,
367         p.username, s.created_at as snap_epoch
368     FROM process_snapshots p
369     JOIN system_snapshots s ON p.snapshot_id = s.i
d
370         WHERE s.created_at < ?
371         ORDER BY p.name, p.pid, s.created_at ASC
372     '', (cutoff_epoch,))
373     raw_rows = cursor.fetchall()
374     if not raw_rows:
375         return {'rollups_created': 0, 'outliers_saved'
: 0, 'raw_deleted': 0}
376
377     outlier_rows = []
378     for r in raw_rows:
379         cpu_val = float(r['cpu_percent'] or 0.0)
380         if cpu_val >= outlier_cpu_threshold:
381             outlier_rows.append((
382                 r['snapshot_id'], r['timestamp'], r['s
nap_epoch'], r['pid'], r['name'],
383                 r['status'] or 'running', cpu_val, flo
at(r['memory_mb'] or 0.0),
384                 float(r['memory_percent'] or 0.0), int
(r['num_threads'] or 1),
385                 r['username'] or '', f'Outlier: CPU={c
pu_val:.1f}% ',
386                 ))
387     outliers_saved = 0
388     if outlier_rows:
389         cursor.executemany('''
390             INSERT INTO process_outliers (
391                 snapshot_id, timestamp, created_at, pi
d, name, status,
392                 cpu_percent, memory_mb, memory_percent
, num_threads, username, details
393             ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?,
?)
394             '', outlier_rows)

```

```

395         outliers_saved = len(outlier_rows)
396
397         grouped = defaultdict(list)
398         for r in raw_rows:
399             grouped[(r['name'], r['pid'])].append(r)
400
401         rollup_rows = []
402         for (p_name, p_pid), procs in grouped.items():
403             cnt = len(procs)
404             cpus = [float(p['cpu_percent'] or 0.0) for p in
n procs]
405             mems = [float(p['memory_mb'] or 0.0) for p in
procs]
406             threads = [int(p['num_threads'] or 1) for p in
procs]
407             proc_outliers = sum(1 for c in cpus if c >= ou
tlier_cpu_threshold)
408             rollup_rows.append((
409                 procs[0]['timestamp'], procs[-1]['timestam
p'], procs[0]['snap_epoch'], procs[-1]['snap_epoch'],
410                 p_name, p_pid, procs[-1]['username'] or ''
, cnt, round(sum(cpus) / cnt, 2),
411                 round(max(cpus), 2), round(min(cpus), 2),
round(sum(mems) / cnt, 2),
412                 round(max(mems), 2), round(min(mems), 2),
round(sum(threads) / cnt, 1), proc_outliers,
413                 ))
414
415         if rollup_rows:
416             cursor.executemany('''
417                 INSERT INTO process_rollups_2min (
418                     period_start, period_end, period_start
_epoch, period_end_epoch,
419                     name, pid, username, sample_count, avg
_cpu_percent, max_cpu_percent,
420                     min_cpu_percent, avg_memory_mb, max_me
mory_mb, min_memory_mb,
421                     avg_num_threads, outliers_count

```

```

422         ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?,
    |      ?, ?, ?, ?, ?)
423         '', rollup_rows)
424
425         raw_ids = [r['id'] for r in raw_rows]
426         cursor.execute(f'''
427             DELETE FROM process_snapshots WHERE id IN ({',
    | '.join(['?'] * len(raw_ids))})
428             ''', raw_ids)
429         conn.commit()
430         return {'rollups_created': len(rollup_rows), 'outl
    | iers_saved': outliers_saved, 'raw_deleted': len(raw_ids)}
431
432     def aggregate_process_metrics_daily(self, cutoff_days: int
    | = 1) -> Dict[str, int]:
433         """Обобщает агрегаты процессов в суточную статистику."
    | ""
434         if self._cm.read_only:
435             return {'daily_rollups_created': 0, '2min_rollups_
    | cleaned': 0}
436         now_epoch = datetime.now(timezone.utc).timestamp()
437         cutoff_epoch = now_epoch - cutoff_days * 86400
438         with self._cm.lock, self._cm.get_connection() as conn:
439             cursor = conn.cursor()
440             cursor.execute('SELECT * FROM process_rollups_2min
    | WHERE period_end_epoch < ? ORDER BY name, period_start_epoch
    | ASC', (cutoff_epoch,))
441             old_rollups = cursor.fetchall()
442             if not old_rollups:
443                 return {'daily_rollups_created': 0, '2min_roll
    | ups_cleaned': 0}
444
445             grouped_daily = defaultdict(list)
446             for r in old_rollups:
447                 d_str = r['period_start'][:10] if len(r['perio
    | d_start']) >= 10 else 'unknown'
448                 grouped_daily[(d_str, r['name'])].append(r)
449

```

```

450         daily_rows = []
451         for (d_str, p_name), items in grouped_daily.items(
452             ):
453             total_samples = sum(int(it['sample_count']) fo
454             r it in items)
455             if total_samples <= 0:
456                 continue
457             weighted_cpu = sum(float(it['avg_cpu_percent']
458             ) * int(it['sample_count']) for it in items) / total_samples
459             weighted_mem = sum(float(it['avg_memory_mb'])
460             * int(it['sample_count']) for it in items) / total_samples
461             daily_rows.append((
462                 d_str, p_name, total_samples, round(weight
463                 ed_cpu, 2), round(max(float(it['max_cpu_percent']) for it in i
464                 tems), 2),
465                 round(weighted_mem, 2), round(max(float(it
466                 ['max_memory_mb']) for it in items), 2),
467                 sum(int(it['outliers_count'] or 0) for it
468                 in items), max(it['period_end'] for it in items),
469                 ))
470
471         if daily_rows:
472             cursor.executemany('''
473                 INSERT INTO process_rollups_daily (
474                     date, name, sample_count, avg_cpu_perc
475                     ent, max_cpu_percent,
476                     avg_memory_mb, max_memory_mb, outliers
477                     _count, last_seen
478                     ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
479                 ''', daily_rows)
480
481         del_ids = [r['id'] for r in old_rollups]
482         cursor.execute(f'DELETE FROM process_rollups_2min
483         WHERE id IN ({','.join(['?'] * len(del_ids))})', del_ids)
484         conn.commit()
485         return {'daily_rollups_created': len(daily_rows),
486             '2min_rollups_cleaned': len(del_ids)}
487

```

reader.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry Sqlite -
  Reader
4 # =====
5 # Description:
6 #   Класс TelemetryReader для чтения и выборки данных телеметр
  ии из SQLite.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.sqlite.reader import Telemet
  ryReader
11 #
12 #       reader = TelemetryReader(connection_manager)
13 #       snapshots = reader.get_snapshots(limit=10)
14 #
15 # File: reader.py
16 # Project: ai-breadboard
17 # Package: apps.windows.telemetry.sqlite
18 # Author: hypo69
19 # Copyright: © 2026 hypo69
20 # Updated: 2026-10-10 05:41:00
21 # =====
22
23 from __future__ import annotations
24
25 """Класс TelemetryReader для выполнения выборок, аналитики и ч
  тения данных телеметрии."""
26
27 import csv
28 import json
29 import sqlite3
```



```

30 import time
31 from datetime import datetime, timezone
32 from pathlib import Path
33 from typing import Any, Dict, List, Optional, Union
34
35 from .connection import TelemetryConnectionManager
36
37 try:
38     from logger import logger
39 except ImportError:
40     import logging
41     logger = logging.getLogger(__name__)
42
43
44 class TelemetryReader:
45     """Выполняет запросы чтения данных телеметрии, процессов,
46     сенсоров и аудита из базы SQLite."""
47
48     def __init__(self, connection_manager: TelemetryConnection
49     Manager) -> None:
50         """Инициализирует ридер телеметрии.
51
52         Args:
53             connection_manager: Менеджер подключений SQLite.
54         """
55         self._cm = connection_manager
56
57     def get_snapshots(self, limit: int = 60, since_epoch: Opti
58     onal[float] = None) -> List[Dict[str, Any]]:
59         """Извлекает исторические срезы системной телеметрии."
60
61         ""
62
63         with self._cm.lock, self._cm.get_connection() as conn:
64             cursor = conn.cursor()
65             if since_epoch is not None:
66                 cursor.execute(
67                     'SELECT * FROM system_snapshots WHERE crea
68                     ted_at >= ? ORDER BY id DESC LIMIT ?',
69                     (since_epoch, limit)

```

```

63         )
64         else:
65             cursor.execute('SELECT * FROM system_snapshots
ORDER BY id DESC LIMIT ?', (limit,))
66             return [dict(row) for row in cursor.fetchall()]
67
68     def get_latest_cpu(self) -> Optional[Dict[str, Any]]:
69         """Извлекает последние метрики процессора из таблицы s
system_snapshots."""
70         with self._cm.lock, self._cm.get_connection() as conn:
71             cursor = conn.cursor()
72             cursor.execute('''
73                 SELECT id, timestamp, created_at, hostname, cp
u_total_percent, cpu_frequency_mhz
74                 FROM system_snapshots
75                 ORDER BY id DESC
76                 LIMIT 1;
77             ''')
78             row = cursor.fetchone()
79             return dict(row) if row else None
80
81     def get_cpu_history(self, limit: int = 60, since_epoch: Op
tional[float] = None) -> List[Dict[str, Any]]:
82         """Извлекает историю нагрузки процессора."""
83         with self._cm.lock, self._cm.get_connection() as conn:
84             cursor = conn.cursor()
85             if since_epoch is not None:
86                 cursor.execute('''
87                     SELECT id, timestamp, created_at, cpu_tota
l_percent, cpu_frequency_mhz
88                     FROM system_snapshots
89                     WHERE created_at >= ?
90                     ORDER BY id DESC
91                     LIMIT ?;
92                 ''', (since_epoch, limit))
93             else:
94                 cursor.execute('''
95                     SELECT id, timestamp, created_at, cpu_tota

```

```

    l_percent, cpu_frequency_mhz
96         FROM system_snapshots
97         ORDER BY id DESC
98         LIMIT ?;
99         ''' , (limit,))
100         return [dict(row) for row in cursor.fetchall()]
101
102     def get_latest_storage_data(self) -> Dict[str, Any]:
103         """Извлекает последние данные о накопителях и разделах
104         ."""
105         with self._cm.lock, self._cm.get_connection() as conn:
106             cursor = conn.cursor()
107             cursor.execute('''
108                 SELECT id, timestamp, created_at, disks_json,
109                 physical_disks_json
110                 FROM system_snapshots
111                 ORDER BY id DESC
112                 LIMIT 1;
113             ''')
114             row = cursor.fetchone()
115             if not row:
116                 return {'partitions': [], 'physical_disks': []}
117
118             partitions: List[Dict[str, Any]] = []
119             if row['disks_json']:
120                 try:
121                     partitions = json.loads(row['disks_json'])
122                 except Exception:
123                     partitions = []
124
125             physical_disks: List[Dict[str, Any]] = []
126             if row['physical_disks_json']:
127                 try:
128                     physical_disks = json.loads(row['physical_
129                     disks_json'])
130                 except Exception:
131                     physical_disks = []

```

```

129
130         return {
131             'id': row['id'],
132             'timestamp': row['timestamp'],
133             'created_at': row['created_at'],
134             'partitions': partitions,
135             'physical_disks': physical_disks,
136         }
137
138     def get_latest_snapshot_full(self) -> Optional[Dict[str, Any]]:
139         """Извлекает и десериализует последний системный снимок со всеми метаданными и процессами."""
140         with self._cm.lock, self._cm.get_connection() as conn:
141             cursor = conn.cursor()
142             cursor.execute('SELECT * FROM system_snapshots ORDER BY id DESC LIMIT 1;')
143             row = cursor.fetchone()
144             if not row:
145                 return None
146
147             snap_row = dict(row)
148             snap_id = snap_row.get('id')
149
150             # 1. Попытка десериализации raw_json
151             raw_json_str = snap_row.get('raw_json')
152             full_dict: Dict[str, Any] = {}
153             if raw_json_str and isinstance(raw_json_str, str):
154                 try:
155                     full_dict = json.loads(raw_json_str)
156                 except Exception:
157                     full_dict = {}
158
159             # 2. Обогащение/fallback из именованных колонок
160             for field in ('hostname', 'username', 'uptime_seconds', 'os_name', 'os_build', 'os_install_date',
161                         'system_language', 'os_install_language', 'user_locale', 'system_locale', 'timezone', 'codepage',

```

```

162         'cpu_total_percent', 'cpu_frequency_
| mhz', 'memory_total_gb', 'memory_used_gb', 'memory_percent',
163         'swap_percent', 'gpu_load_percent',
| 'gpu_temp_c', 'disk_read_bytes_sec', 'disk_write_bytes_sec',
164         'disk_read_count_sec', 'disk_write_c
| ount_sec', 'network_sent_bytes_sec', 'network_recv_bytes_sec',
| 'timestamp'):
165         if snap_row.get(field) is not None and (field
| not in full_dict or not full_dict[field]):
166             full_dict[field] = snap_row[field]
167
168     # 3. Десериализация JSON-колонок
169     json_column_mappings = {
170         'input_languages_json': 'input_languages',
171         'disks_json': 'disks',
172         'physical_disks_json': 'physical_disks',
173         'monitors_json': 'monitors',
174         'updates_json': 'updates',
175         'office_json': 'office',
176         'onedrive_json': 'onedrive',
177         'battery_json': 'battery',
178         'ram_sticks_json': 'ram_sticks',
179         'listening_ports_json': 'listening_ports',
180         'alerts_json': 'alerts',
181         'hardware_audit': 'hardware_audit',
182     }
183     for col_name, target_key in json_column_mappings.i
| tems():
184         val_str = snap_row.get(col_name)
185         if val_str and isinstance(val_str, str) and (t
| arget_key not in full_dict or not full_dict[target_key]):
186             try:
187                 full_dict[target_key] = json.loads(val
| _str)
188             except Exception:
189                 pass
190
191     # 4. Процессы (если не были в raw_json)

```

```

192         if 'top_processes' not in full_dict or not full_dict.get('top_processes'):
193             if snap_id:
194                 cursor.execute(
195                     'SELECT pid, name, status, cpu_percent
196                     , memory_mb, memory_percent, num_threads, num_handles, '
197                     'username, read_bytes_sec, write_bytes_sec, integrity_level, elevation, ppid, parent_name, '
198                     'executable as executable_path, cmdline, sid, session_id, creation_time, process_guid, ancestor_chain, launch_reason '
199                     'FROM process_snapshots WHERE snapshot_id = ? ORDER BY cpu_percent DESC LIMIT 50;',
200                     (snap_id,)
201                 )
202                 full_dict['top_processes'] = [dict(p) for p in cursor.fetchall()]
203
204             # 5. Структурирование CPU и Memory если отсутствуют объекты
205             if 'cpu' not in full_dict or not isinstance(full_dict.get('cpu'), dict):
206                 full_dict['cpu'] = {
207                     'total_percent': float(snap_row.get('cpu_total_percent') or 0.0),
208                     'frequency_mhz': float(snap_row.get('cpu_frequency_mhz') or 0.0),
209                     'model': full_dict.get('cpu_model', 'Intel / AMD Processor'),
210                     'architecture': 'x86_64',
211                 }
212             if 'memory' not in full_dict or not isinstance(full_dict.get('memory'), dict):
213                 tot_m = float(snap_row.get('memory_total_gb') or 0.0)
214                 used_m = float(snap_row.get('memory_used_gb') or 0.0)
215                 full_dict['memory'] = {

```

```

215         'total_gb': tot_m,
216         'used_gb': used_m,
217         'available_gb': max(0.0, tot_m - used_m),
218         'percent': float(snap_row.get('memory_perc
ent') or 0.0),
219         'swap_percent': float(snap_row.get('swap_p
ercent') or 0.0),
220     }
221     if 'disk_io' not in full_dict or not isinstance(fu
ll_dict.get('disk_io'), dict):
222         full_dict['disk_io'] = {
223             'read_bytes_per_sec': float(snap_row.get('
disk_read_bytes_sec') or 0.0),
224             'write_bytes_per_sec': float(snap_row.get(
'disk_write_bytes_sec') or 0.0),
225             'read_count_per_sec': float(snap_row.get('
disk_read_count_sec') or 0.0),
226             'write_count_per_sec': float(snap_row.get(
'disk_write_count_sec') or 0.0),
227         }
228
229     return full_dict
230
231     def get_snapshot_processes(self, snapshot_id: int) -> List
[Dict[str, Any]]:
232         """Извлекает список процессов, зафиксированных в снимк
е."""
233         with self._cm.lock, self._cm.get_connection() as conn:
234             cursor = conn.cursor()
235             cursor.execute(
236                 'SELECT * FROM process_snapshots WHERE snapsho
t_id = ? ORDER BY cpu_percent DESC, memory_mb DESC',
237                 (snapshot_id,)
238             )
239             return [dict(row) for row in cursor.fetchall()]
240
241     def get_process_history(
242         self,

```

```

243         name: Optional[str] = None,
244         pid: Optional[int] = None,
245         limit: int = 100,
246     ) -> List[Dict[str, Any]]:
247         """Извлекает историю поведения процесса по имени или P
ID."""
248         with self._cm.lock, self._cm.get_connection() as conn:
249             cursor = conn.cursor()
250             if pid is not None and name is not None:
251                 cursor.execute(
252                     'SELECT * FROM process_snapshots WHERE pid
= ? AND name LIKE ? ORDER BY id DESC LIMIT ?',
253                     (pid, f'%{name}%', limit)
254                 )
255             elif pid is not None:
256                 cursor.execute(
257                     'SELECT * FROM process_snapshots WHERE pid
= ? ORDER BY id DESC LIMIT ?',
258                     (pid, limit)
259                 )
260             elif name is not None:
261                 cursor.execute(
262                     'SELECT * FROM process_snapshots WHERE nam
e LIKE ? ORDER BY id DESC LIMIT ?',
263                     (f'%{name}%', limit)
264                 )
265             else:
266                 cursor.execute('SELECT * FROM process_snapshots
ORDER BY id DESC LIMIT ?', (limit,))
267             return [dict(row) for row in cursor.fetchall()]
268
269     def get_latest_processes(self, limit: int = 50, sort_by: str = 'cpu') -> List[Dict[str, Any]]:
270         """Извлекает процессы из самого последнего зафиксированного снимка."""
271         sort_key = (sort_by or 'cpu').lower()
272         if sort_key in ('handles', 'descriptors'):
273             order_col = 'num_handles DESC, cpu_percent DESC'

```



```

274         elif sort_key in ('memory', 'ram', 'memory_mb'):
275             order_col = 'memory_mb DESC, cpu_percent DESC'
276         elif sort_key == 'memory_percent':
277             order_col = 'memory_percent DESC, cpu_percent DESC'
278         elif sort_key == 'name':
279             order_col = 'name ASC'
280         elif sort_key == 'pid':
281             order_col = 'pid ASC'
282         else:
283             order_col = 'cpu_percent DESC, memory_mb DESC'
284
285         with self._cm.lock, self._cm.get_connection() as conn:
286             cursor = conn.cursor()
287             cursor.execute('SELECT MAX(id) FROM system_snapshots;')
288             row = cursor.fetchone()
289             last_snap_id = row[0] if row and row[0] is not None else None
290             if last_snap_id is None:
291                 cursor.execute('SELECT MAX(snapshot_id) FROM process_snapshots;')
292                 row = cursor.fetchone()
293                 last_snap_id = row[0] if row and row[0] is not None else None
294             if last_snap_id is None:
295                 return []
296
297             sql = f'''
298                 SELECT pid, name, status, cpu_percent, memory_
299                 mb, memory_percent,
300                 num_threads, num_handles, username, rea
301                 d_bytes_sec, write_bytes_sec,
302                 integrity_level, elevation, ppid, paren
303                 t_name, executable, cmdline, timestamp
304                 FROM process_snapshots
305                 WHERE snapshot_id = ?
306                 ORDER BY {order_col}

```

```

304         '''
305         if limit is not None and limit > 0:
306             cursor.execute(sql + ' LIMIT ?', (last_snap_id
307 | , limit))
308         else:
309             cursor.execute(sql, (last_snap_id,))
310             return [dict(r) for r in cursor.fetchall()]
311
312     def get_process_stats(self, name: Optional[str] = None, li
313 | mit: int = 50) -> Dict[str, Any]:
314         """Возвращает агрегированную статистику процессов и вы
315 | бросы."""
316         with self._cm.lock, self._cm.get_connection() as conn:
317             cursor = conn.cursor()
318             if name:
319                 cursor.execute('SELECT * FROM process_rollups_
320 | 2min WHERE name LIKE ? ORDER BY id DESC LIMIT ?', (f'%{name}%'
321 | , limit))
322             else:
323                 cursor.execute('SELECT * FROM process_rollups_
324 | 2min ORDER BY id DESC LIMIT ?', (limit,))
325             rollups_2min = [dict(r) for r in cursor.fetchall()
326 | ]
327
328         if name:
329             cursor.execute('SELECT * FROM process_rollups_
330 | daily WHERE name LIKE ? ORDER BY date DESC LIMIT ?', (f'%{name}%'
331 | , limit))
332         else:
333             cursor.execute('SELECT * FROM process_rollups_
334 | daily ORDER BY date DESC LIMIT ?', (limit,))
335             daily_stats = [dict(r) for r in cursor.fetchall()]
336
337         if name:
338             cursor.execute('SELECT * FROM process_outliers
339 | WHERE name LIKE ? ORDER BY id DESC LIMIT ?', (f'%{name}%', li
340 | mit))
341         else:

```

```

330         cursor.execute('SELECT * FROM process_outliers
ORDER BY id DESC LIMIT ?', (limit,))
331         outliers = [dict(r) for r in cursor.fetchall()]
332
333         return {
334             'name_filter': name,
335             'rollups_2min': rollups_2min,
336             'daily_stats': daily_stats,
337             'outliers': outliers,
338             'total_rollups_count': len(rollups_2min),
339             'total_outliers_count': len(outliers),
340         }
341
342     def get_events(self, event_type: Optional[str] = None, limit: int = 100) -> List[Dict[str, Any]]:
343         """Извлекает список зафиксированных событий телеметрии
344         ."""
345         with self._cm.lock, self._cm.get_connection() as conn:
346             cursor = conn.cursor()
347             if event_type:
348                 cursor.execute('SELECT * FROM telemetry_events
WHERE event_type = ? ORDER BY id DESC LIMIT ?', (event_type,
limit))
349             else:
350                 cursor.execute('SELECT * FROM telemetry_events
ORDER BY id DESC LIMIT ?', (limit,))
351             return [dict(row) for row in cursor.fetchall()]
352
353     def get_sensor_history(
354         self,
355         sensor_id: Optional[str] = None,
356         category: Optional[str] = None,
357         limit: int = 100,
358     ) -> List[Dict[str, Any]]:
359         """Извлекает историю показаний сенсоров."""
360         with self._cm.lock, self._cm.get_connection() as conn:
361             cursor = conn.cursor()
362             if sensor_id and category:

```

```

362         cursor.execute(
363             'SELECT * FROM sensor_polls WHERE sensor_id = ? AND sensor_category = ? ORDER BY id DESC LIMIT ?',
364             (sensor_id, category, limit)
365         )
366         elif sensor_id:
367             cursor.execute('SELECT * FROM sensor_polls WHERE sensor_id = ? ORDER BY id DESC LIMIT ?', (sensor_id, limit)
368             )
369         elif category:
370             cursor.execute('SELECT * FROM sensor_polls WHERE sensor_category = ? ORDER BY id DESC LIMIT ?', (category, limit))
371         else:
372             cursor.execute('SELECT * FROM sensor_polls ORDER BY id DESC LIMIT ?', (limit,))
373         return [dict(row) for row in cursor.fetchall()]
374
375     def get_latest_sensors(self, limit: Optional[int] = None) -> List[Dict[str, Any]]:
376         """Извлекает последнее актуальное значение каждого сенсора."""
377         with self._cm.lock, self._cm.get_connection() as conn:
378             cursor = conn.cursor()
379             query = '''
380                 SELECT s1.* FROM sensor_polls s1
381                 INNER JOIN (
382                     SELECT sensor_id, MAX(id) as max_id
383                     FROM sensor_polls
384                     GROUP BY sensor_id
385                 ) s2 ON s1.id = s2.max_id
386                 ORDER BY s1.sensor_category, s1.sensor_name
387             '''
388             if limit is not None:
389                 cursor.execute(query + ' LIMIT ?', (limit,))
390             else:
391                 cursor.execute(query)
392             return [dict(row) for row in cursor.fetchall()]

```

```

392
393     def get_cpu_hierarchy_metrics(self) -> Dict[str, Any]:
394         """Извлекает структурированные иерархические метрики п
процессора из SQLite.
395
396         Выбирает последние замеры сенсоров CPU, системные сним
ки и историю нагрузки/температур.
397
398         Returns:
399             Dict[str, Any]: Словарь с сенсорами, снимками, пас
портом и историей.
400         """
401         with self._cm.lock, self._cm.get_connection() as conn:
402             cursor = conn.cursor()
403
404             # 1. Последние сенсоры CPU из sensor_polls (исключ
ая дисковые и GPU метрики)
405             query_sensors = '''
406                 SELECT s1.* FROM sensor_polls s1
407                 INNER JOIN (
408                     SELECT sensor_id, MAX(id) as max_id
409                     FROM sensor_polls
410                     WHERE hardware_type = 'cpu' OR (
411                         hardware_name NOT LIKE '%GPU%'
412                         AND hardware_name NOT LIKE '%Storage%'
413                         AND hardware_name NOT LIKE '%Disk%'
414                         AND sensor_id NOT LIKE 'disk_speed_%'
415                         AND (hardware_type = 'system' OR senso
r_category IN ('Voltages', 'Powers', 'Clocks', 'Temperatures',
'Load', 'voltages', 'powers', 'clocks', 'temperatures', 'load
'))
416                     )
417                 GROUP BY sensor_id
418                 ) s2 ON s1.id = s2.max_id
419                 ORDER BY s1.sensor_category, s1.sensor_name
420             '''
421             cursor.execute(query_sensors)
422             sensors = [dict(row) for row in cursor.fetchall()]

```

```

423 |
424 |         # 2. Последний системный снимок
425 |         cursor.execute('SELECT * FROM system_snapshots ORD
| ER BY id DESC LIMIT 1')
426 |         snap_row = cursor.fetchone()
427 |         latest_snapshot = dict(snap_row) if snap_row else
| None
428 |
429 |         # 3. Данные инвентаризации CPU
430 |         cursor.execute('SELECT * FROM cpu_inventory ORDER
| BY processor_id ASC LIMIT 1')
431 |         inv_row = cursor.fetchone()
432 |         if inv_row:
433 |             cpu_inv = dict(inv_row)
434 |         else:
435 |             cpu_inv = self._probe_and_persist_cpu_inventor
| y(cursor)
436 |
437 |         # 4. История снимков для графиков
438 |         cursor.execute('''
439 |             SELECT timestamp, created_at, cpu_total_perce
| nt, cpu_frequency_mhz
440 |             FROM system_snapshots
441 |             ORDER BY id DESC
442 |             LIMIT 60
443 |         ''')
444 |         history_rows = [dict(r) for r in cursor.fetchall()
| ]
445 |         history_rows.reverse()
446 |
447 |         return {
448 |             'sensors': sensors,
449 |             'snapshot': latest_snapshot,
450 |             'inventory': cpu_inv,
451 |             'history': history_rows,
452 |         }
453 |
454 |     def _probe_and_persist_cpu_inventory(self, cursor: sqlite3

```

```

.Cursor) -> Dict[str, Any]:
455     """Опрашивает спецификации CPU и сохраняет их в cpu_in
    ventory."""
456     import os
457     import platform
458     import time
459     from datetime import datetime, timezone
460     try:
461         import psutil
462     except ImportError:
463         psutil = None
464
465     vendor = 'Intel'
466     model = platform.processor() or 'Processor'
467     arch = 'x86_64' if platform.machine() in ('AMD64', 'x8
6_64') else platform.machine()
468     phys_cores = psutil.cpu_count(logical=False) if psutil
    else 1
469     log_cores = psutil.cpu_count(logical=True) if psutil e
    lse 1
470     freq = psutil.cpu_freq() if psutil else None
471     base_mhz = float(freq.current if freq else 2900.0)
472     max_mhz = float(freq.max if (freq and freq.max) else b
    ase_mhz)
473     socket = ''
474     l2_kb = None
475     l3_kb = None
476     stepping = ''
477     features: List[str] = []
478
479     if os.name == 'nt':
480         try:
481             import winreg
482             with winreg.OpenKey(winreg.HKEY_LOCAL_MACHINE,
                r'HARDWARE\DESCRIPTION\System\CentralProcessor\0') as k:
483                 reg_name, _ = winreg.QueryValueEx(k, 'Proc
                essorNameString')
484                 reg_vendor, _ = winreg.QueryValueEx(k, 'Ve

```

```

ndorIdentifier')
485         reg_mhz, _ = winreg.QueryValueEx(k, '~MHz'
    )
486         reg_ident, _ = winreg.QueryValueEx(k, 'Ide
ntifier')
487         if reg_name:
488             model = str(reg_name).strip()
489         if reg_vendor:
490             v_str = str(reg_vendor).strip()
491             vendor = 'Intel' if 'intel' in v_str.l
ower() else ('AMD' if 'amd' in v_str.lower() else v_str)
492         if reg_mhz:
493             base_mhz = float(reg_mhz)
494         if reg_ident:
495             stepping = str(reg_ident).strip()
496     except Exception:
497         pass
498
499     try:
500         import win32com.client
501         wmi_obj = win32com.client.GetObject('winmgmts:
    ')
502         for p in wmi_obj.InstancesOf('Win32_Processor'
    ):
503             if getattr(p, 'Name', None):
504                 model = str(p.Name).strip()
505             if getattr(p, 'SocketDesignation', None):
506                 socket = str(p.SocketDesignation).stri
p()
507             if getattr(p, 'L2CacheSize', None):
508                 l2_kb = int(p.L2CacheSize)
509             if getattr(p, 'L3CacheSize', None):
510                 l3_kb = int(p.L3CacheSize)
511             if getattr(p, 'MaxClockSpeed', None):
512                 max_mhz = float(p.MaxClockSpeed)
513             if getattr(p, 'Manufacturer', None):
514                 m_str = str(p.Manufacturer).strip()
515                 vendor = 'Intel' if 'intel' in m_str.l

```



```

    ower() else ('AMD' if 'amd' in m_str.lower() else m_str)
516         if getattr(p, 'NumberOfCores', None):
517             phys_cores = int(p.NumberOfCores)
518             if getattr(p, 'NumberOfLogicalProcessors',
None):
519                 log_cores = int(p.NumberOfLogicalProce
ssors)
520                 break
521         except Exception:
522             pass
523
524         now_iso = datetime.now(timezone.utc).isoformat()
525         now_epoch = time.time()
526
527         cpu_inv = {
528             'processor_id': 0,
529             'name': model,
530             'vendor': vendor,
531             'architecture': arch,
532             'physical_cores': phys_cores or 1,
533             'logical_cores': log_cores or 1,
534             'base_frequency_mhz': base_mhz,
535             'max_frequency_mhz': max_mhz,
536             'l2_cache_kb': l2_kb,
537             'l3_cache_kb': l3_kb,
538             'socket': socket,
539             'stepping': stepping,
540             'features_json': json.dumps(features),
541             'first_seen': now_iso,
542             'last_seen': now_iso,
543             'updated_at': now_epoch,
544         }
545
546         try:
547             cursor.execute('''
548                 INSERT OR REPLACE INTO cpu_inventory (
549                     processor_id, name, vendor, architecture,
physical_cores, logical_cores,

```

```

550         base_frequency_mhz, max_frequency_mhz, l2_
    cache_kb, l3_cache_kb, socket,
551         features_json, first_seen, last_seen, upda
    ted_at
552         ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?,
    ?, ?, ?)
553         ''' , (
554             cpu_inv['processor_id'], cpu_inv['name'], cpu_
    inv['vendor'], cpu_inv['architecture'],
555             cpu_inv['physical_cores'], cpu_inv['logical_co
    res'], cpu_inv['base_frequency_mhz'],
556             cpu_inv['max_frequency_mhz'], cpu_inv['l2_cach
    e_kb'], cpu_inv['l3_cache_kb'],
557             cpu_inv['socket'], cpu_inv['features_json'], c
    pu_inv['first_seen'],
558             cpu_inv['last_seen'], cpu_inv['updated_at']
559         ))
560     except Exception as ex:
561         logger.debug(f"[reader] Не удалось сохранить cpu_i
    nventory: {ex}")
562
563     return cpu_inv
564
565     def get_app_polls(self, app: Optional[str] = None, metric_
    name: Optional[str] = None, limit: int = 100) -> List[Dict[str
    , Any]]:
566         """Извлекает историю опросов приложений."""
567         with self._cm.lock, self._cm.get_connection() as conn:
568             cursor = conn.cursor()
569             if app and metric_name:
570                 cursor.execute('SELECT * FROM app_polls WHERE
    app = ? AND metric_name = ? ORDER BY id DESC LIMIT ?', (app, m
    etric_name, limit))
571             elif app:
572                 cursor.execute('SELECT * FROM app_polls WHERE
    app = ? ORDER BY id DESC LIMIT ?', (app, limit))
573             else:
574                 cursor.execute('SELECT * FROM app_polls ORDER

```

```

        BY id DESC LIMIT ?', (limit,))
575         return [dict(row) for row in cursor.fetchall()]
576
577     def get_app_events(self, app: Optional[str] = None, event_
type: Optional[str] = None, limit: int = 100) -> List[Dict[str
, Any]]:
578         """Извлекает события приложений."""
579         with self._cm.lock, self._cm.get_connection() as conn:
580             cursor = conn.cursor()
581             if app and event_type:
582                 cursor.execute('SELECT * FROM app_events WHERE
app = ? AND event_type = ? ORDER BY id DESC LIMIT ?', (app, e
vent_type, limit))
583             elif app:
584                 cursor.execute('SELECT * FROM app_events WHERE
app = ? ORDER BY id DESC LIMIT ?', (app, limit))
585             else:
586                 cursor.execute('SELECT * FROM app_events ORDER
BY id DESC LIMIT ?', (limit,))
587             return [dict(row) for row in cursor.fetchall()]
588
589     def get_app_param_changes(self, app: Optional[str] = None,
limit: int = 100) -> List[Dict[str, Any]]:
590         """Извлекает изменения параметров приложений."""
591         with self._cm.lock, self._cm.get_connection() as conn:
592             cursor = conn.cursor()
593             if app:
594                 cursor.execute('SELECT * FROM app_param_change
s WHERE app = ? ORDER BY id DESC LIMIT ?', (app, limit))
595             else:
596                 cursor.execute('SELECT * FROM app_param_change
s ORDER BY id DESC LIMIT ?', (limit,))
597             return [dict(row) for row in cursor.fetchall()]
598
599     def export_app_polls_to_csv(self, app: Optional[str] = Non
e, output_path: Optional[Union[str, Path]] = None, limit: int
= 50000) -> Path:
600         """Экспортирует историю опросов приложений в CSV-файл.

```

```

"""
601         polls = self.get_app_polls(app=app, limit=limit)
602         target = Path(output_path) if output_path else Path('a
pp_polls.csv')
603         target.parent.mkdir(parents=True, exist_ok=True)
604         headers = ['timestamp', 'app', 'poll_type', 'metric_na
me', 'value', 'unit', 'status', 'details']
605         with open(target, 'w', newline='', encoding='utf-8-sig
') as f:
606             writer = csv.writer(f)
607             writer.writerow(headers)
608             for p in polls:
609                 writer.writerow([
610                     p.get('timestamp', ''), p.get('app', ''),
p.get('poll_type', ''),
611                     p.get('metric_name', ''), p.get('value', '
'), p.get('unit', ''),
612                     p.get('status', ''), p.get('details', ''),
613                 ])
614         return target
615
616     def export_app_events_to_csv(self, app: Optional[str] = No
ne, output_path: Optional[Union[str, Path]] = None, limit: int
= 50000) -> Path:
617         """Экспортирует историю событий приложений в CSV-файл.
"""
618         events = self.get_app_events(app=app, limit=limit)
619         target = Path(output_path) if output_path else Path('a
pp_events.csv')
620         target.parent.mkdir(parents=True, exist_ok=True)
621         headers = ['timestamp', 'app', 'event_type', 'status',
'user', 'details']
622         with open(target, 'w', newline='', encoding='utf-8-sig
') as f:
623             writer = csv.writer(f)
624             writer.writerow(headers)
625             for e in events:
626                 writer.writerow([

```

```

627         e.get('timestamp', ''), e.get('app', ''),
        e.get('event_type', ''),
628         e.get('status', ''), e.get('user', ''), e.
        get('details', ''),
629     ])
630     return target
631
632     def export_app_param_changes_to_csv(self, app: Optional[str] = None, output_path: Optional[Union[str, Path]] = None, limit: int = 50000) -> Path:
633         """Экспортирует историю изменения параметров в CSV-файл."""
634         params = self.get_app_param_changes(app=app, limit=limit)
635         target = Path(output_path) if output_path else Path('app_param_changes.csv')
636         target.parent.mkdir(parents=True, exist_ok=True)
637         headers = ['timestamp', 'app', 'param_name', 'old_value', 'new_value', 'status', 'user', 'details']
638         with open(target, 'w', newline='', encoding='utf-8-sig') as f:
639             writer = csv.writer(f)
640             writer.writerow(headers)
641             for p in params:
642                 writer.writerow([
643                     p.get('timestamp', ''), p.get('app', ''),
        p.get('param_name', ''),
644                     p.get('old_value', ''), p.get('new_value',
        ''), p.get('status', ''),
645                     p.get('user', ''), p.get('details', ''),
646                 ])
647             return target
648
649     def get_custom_records(self, source_file: Optional[str] = None, limit: int = 100) -> List[Dict[str, Any]]:
650         """Извлекает кастомные записи логов."""
651         with self._cm.lock, self._cm.get_connection() as conn:
652             cursor = conn.cursor()

```

```

653         if source_file:
654             cursor.execute('SELECT * FROM custom_records W
HERE source_file = ? ORDER BY id DESC LIMIT ?', (source_file,
limit))
655         else:
656             cursor.execute('SELECT * FROM custom_records O
RDER BY id DESC LIMIT ?', (limit,))
657         return [dict(row) for row in cursor.fetchall()]
658
659     def get_device_events(self, event_type: Optional[str] = No
ne, device_instance_id: Optional[str] = None, limit: int = 100
) -> List[Dict[str, Any]]:
660         """Извлекает события устройств."""
661         with self._cm.lock, self._cm.get_connection() as conn:
662             cursor = conn.cursor()
663             if event_type and device_instance_id:
664                 cursor.execute('SELECT * FROM device_events WH
ERE event_type = ? AND device_instance_id = ? ORDER BY id DESC
LIMIT ?', (event_type, device_instance_id, limit))
665             elif event_type:
666                 cursor.execute('SELECT * FROM device_events WH
ERE event_type = ? ORDER BY id DESC LIMIT ?', (event_type, lim
it))
667             elif device_instance_id:
668                 cursor.execute('SELECT * FROM device_events WH
ERE device_instance_id = ? ORDER BY id DESC LIMIT ?', (device_
instance_id, limit))
669             else:
670                 cursor.execute('SELECT * FROM device_events OR
DER BY id DESC LIMIT ?', (limit,))
671             return [dict(row) for row in cursor.fetchall()]
672
673     def get_device_install_date(self, device_instance_id: str)
-> Optional[str]:
674         """Возвращает дату установки устройства из таблицы dev
ice_inventory."""
675         norm_id = str(device_instance_id).strip().upper()
676         if not norm_id:

```

```

677         return None
678     with self._cm.lock, self._cm.get_connection() as conn:
679         cursor = conn.cursor()
680         cursor.execute('SELECT install_date FROM device_in
ventory WHERE UPPER(device_instance_id) = ?', (norm_id,))
681         row = cursor.fetchone()
682         return str(row[0]) if (row and row[0]) else None
683
684     def get_w64_events(self, event_type: Optional[str] = None,
provider: Optional[str] = None, limit: int = 100) -> List[Dic
t[str, Any]]:
685         """Извлекает системные события W64/ETW."""
686         query = 'SELECT raw_json FROM w64_events WHERE 1=1'
687         params: List[Any] = []
688         if event_type:
689             query += ' AND event_type = ?'
690             params.append(event_type)
691         if provider:
692             query += ' AND provider = ?'
693             params.append(provider)
694         query += ' ORDER BY created_at DESC LIMIT ?'
695         params.append(max(1, limit))
696
697         with self._cm.lock, self._cm.get_connection() as conn:
698             cursor = conn.cursor()
699             cursor.execute(query, tuple(params))
700             results = []
701             for row in cursor.fetchall():
702                 try:
703                     results.append(json.loads(row[0]))
704                 except Exception:
705                     continue
706             return results
707
708     def get_incidents(self, limit: int = 50, trigger_type: Opt
ional[str] = None, severity: Optional[str] = None) -> List[Dic
t[str, Any]]:
709         """Возвращает список зафиксированных инцидентов."""

```

```

710 |         query = 'SELECT * FROM incidents WHERE 1=1'
711 |         params: List[Any] = []
712 |         if trigger_type:
713 |             query += ' AND trigger_type = ?'
714 |             params.append(trigger_type)
715 |         if severity:
716 |             query += ' AND severity = ?'
717 |             params.append(severity)
718 |         query += ' ORDER BY created_at DESC LIMIT ?'
719 |         params.append(limit)
720 |
721 |         results: List[Dict[str, Any]] = []
722 |         try:
723 |             with self._cm.lock, self._cm.get_connection() as c
724 |             conn:
725 |                 cursor = conn.cursor()
726 |                 cursor.execute(query, tuple(params))
727 |                 for row in cursor.fetchall():
728 |                     item = dict(row)
729 |                     for f in ('trigger_metrics_json', 'suspect
730 | _processes_json', 'related_events_json', 'metrics_summary_json
731 | ', 'raw_window_json'):
732 |                         if item.get(f):
733 |                             try:
734 |                                 item[f.replace('_json', '')] =
735 | json.loads(item[f])
736 |                             except Exception:
737 |                                 item[f.replace('_json', '')] =
738 | []
739 |                         results.append(item)
740 |         except Exception as ex:
741 |             logger.error(f'Ошибка при получении инцидентов: {e
742 | x}')
743 |         return results
744 |
745 |     def get_system_rollups(self, tier: str = '1m', start_epoch
746 | : Optional[float] = None, end_epoch: Optional[float] = None, l
747 | imit: int = 100) -> List[Dict[str, Any]]:

```



```

740 |         """Возвращает сжатые бакеты метрик за период."""
741 |         query = 'SELECT * FROM telemetry_rollups WHERE tier =
|         ?'
742 |         params: List[Any] = [tier]
743 |         if start_epoch is not None:
744 |             query += ' AND period_start_epoch >= ?'
745 |             params.append(start_epoch)
746 |         if end_epoch is not None:
747 |             query += ' AND period_end_epoch <= ?'
748 |             params.append(end_epoch)
749 |         query += ' ORDER BY period_start_epoch DESC LIMIT ?'
750 |         params.append(limit)
751 |
752 |         results: List[Dict[str, Any]] = []
753 |         try:
754 |             with self._cm.lock, self._cm.get_connection() as c
|         onn:
755 |                 cursor = conn.cursor()
756 |                 cursor.execute(query, tuple(params))
757 |                 for row in cursor.fetchall():
758 |                     results.append(dict(row))
759 |         except Exception as ex:
760 |             logger.error(f'Ошибка при получении rollups: {ex}'
|         )
761 |         return results
762 |
763 |     def get_reboot_history(self, limit: int = 50) -> List[Dict
|         [str, Any]]:
764 |         """Извлекает историю сессий перезагрузок."""
765 |         results: List[Dict[str, Any]] = []
766 |         try:
767 |             with self._cm.lock, self._cm.get_connection() as c
|         onn:
768 |                 cursor = conn.cursor()
769 |                 cursor.execute('SELECT * FROM reboot_history O
|         RDER BY boot_time DESC, created_at DESC LIMIT ?', (limit,))
770 |                 for row in cursor.fetchall():
771 |                     item = dict(row)

```

```

772         for f in ('bugcheck_params_json', 'evidenc
e_json', 'events_chain_json', 'raw_json'):
773             if item.get(f):
774                 try:
775                     item[f.replace('_json', '')] =
json.loads(item[f])
776                 except Exception:
777                     pass
778                 results.append(item)
779             except Exception as ex:
780                 logger.error(f'Ошибка извлечения истории перезагру
зок: {ex}')
781         return results
782
783     def get_latest_reboot_session(self) -> Optional[Dict[str,
Any]]:
784         """Возвращает самую последнюю запись сессии перезагруз
ки."""
785         history = self.get_reboot_history(limit=1)
786         return history[0] if history else None
787
788     def get_extended_audits(self, limit: int = 50) -> List[Dic
t[str, Any]]:
789         """Возвращает историю расширенных системных аудитов."""
790
791         results: List[Dict[str, Any]] = []
792         try:
793             with self._cm.lock, self._cm.get_connection() as c
onn:
794                 cursor = conn.cursor()
795                 cursor.execute('SELECT * FROM system_extended_
audits ORDER BY created_at DESC LIMIT ?', (limit,))
796                 for row in cursor.fetchall():
797                     item = dict(row)
798                     if item.get('raw_json'):
799                         try:
800                             item['data'] = json.loads(item['ra
w_json'])

```

```

800 |             except Exception:
801 |                 pass
802 |                 results.append(item)
803 |         except Exception as ex:
804 |             logger.error(f'Ошибка извлечения расширенных аудит
| ов: {ex}')
805 |         return results
806 |
807 |     def get_latest_extended_audit(self) -> Optional[Dict[str,
| Any]]:
808 |         """Возвращает последний расширенный системный аудит."""
809 |         audits = self.get_extended_audits(limit=1)
810 |         return audits[0] if audits else None
811 |
812 |     def get_hardware_audits(self, limit: int = 50) -> List[Dic
| t[str, Any]]:
813 |         """Возвращает историю аудитов аппаратного обеспечения
| из таблицы hardware_audits."""
814 |         results: List[Dict[str, Any]] = []
815 |         try:
816 |             with self._cm.lock, self._cm.get_connection() as c
| onn:
817 |                 cursor = conn.cursor()
818 |                 cursor.execute('SELECT * FROM hardware_audits
| ORDER BY created_at DESC LIMIT ?', (limit,))
819 |                 for row in cursor.fetchall():
820 |                     item = dict(row)
821 |                     if item.get('raw_json'):
822 |                         try:
823 |                             item['data'] = json.loads(item['ra
| w_json'])
824 |                         except Exception:
825 |                             pass
826 |                             results.append(item)
827 |                     except Exception as ex:
828 |                         logger.error(f'Ошибка извлечения аудитов оборудова
| ния: {ex}')
```

```

829         return results
830
831     def get_latest_hardware_audit(self) -> Optional[Dict[str,
Any]]:
832         """Возвращает самый последний сохраненный аудит аппара
тного обеспечения."""
833         audits = self.get_hardware_audits(limit=1)
834         return audits[0] if audits else None
835
836     def get_startup_archives(self, limit: int = 50) -> List[Di
ct[str, Any]]:
837         """Возвращает историю архивных снимков автозапуска из
таблицы startup_audit_archives."""
838         results: List[Dict[str, Any]] = []
839         try:
840             with self._cm.lock, self._cm.get_connection() as c
onn:
841                 cursor = conn.cursor()
842                 cursor.execute('SELECT * FROM startup_audit_ar
chives ORDER BY created_at DESC LIMIT ?', (limit,))
843                 for row in cursor.fetchall():
844                     item = dict(row)
845                     if item.get('raw_json'):
846                         try:
847                             item['data'] = json.loads(item['ra
w_json'])
848                         except Exception:
849                             pass
850                     results.append(item)
851         except Exception as ex:
852             logger.error(f'Ошибка извлечения архивов автозапус
ка: {ex}')
853         return results
854
855     def get_latest_startup_archive(self) -> Optional[Dict[str,
Any]]:
856         """Возвращает самый последний сохраненный снимок автоз
апуска."""

```

```

857         audits = self.get_startup_archives(limit=1)
858         return audits[0] if audits else None
859
860     def get_startup_changes(self, limit: int = 100, archive_id
: Optional[str] = None) -> List[Dict[str, Any]]:
861         """Возвращает историю зафиксированных отличий/изменени
й в автозапуске."""
862         results: List[Dict[str, Any]] = []
863         try:
864             with self._cm.lock, self._cm.get_connection() as c
onn:
865                 cursor = conn.cursor()
866                 if archive_id:
867                     cursor.execute(
868                         'SELECT * FROM startup_changes WHERE a
rchive_id = ? ORDER BY created_at DESC, id DESC LIMIT ?',
869                         (archive_id, limit)
870                     )
871                 else:
872                     cursor.execute(
873                         'SELECT * FROM startup_changes ORDER B
Y created_at DESC, id DESC LIMIT ?',
874                         (limit,)
875                     )
876                 for row in cursor.fetchall():
877                     item = dict(row)
878                     if item.get('previous_value'):
879                         try:
880                             item['previous_value'] = json.load
s(item['previous_value'])
881                         except Exception:
882                             pass
883                     if item.get('current_value'):
884                         try:
885                             item['current_value'] = json.loads
(item['current_value'])
886                         except Exception:
887                             pass

```

```

888         results.append(item)
889     except Exception as ex:
890         logger.error(f'Ошибка извлечения изменений автозап
узка: {ex}')
891     return results
892
893     def get_process_provenance_history(
894         self,
895         guid: Optional[str] = None,
896         pid: Optional[int] = None,
897         name: Optional[str] = None,
898         user: Optional[str] = None,
899         event_type: Optional[str] = None,
900         hours: Optional[int] = None,
901         limit: int = 100,
902     ) -> List[Dict[str, Any]]:
903         """Извлечение исторических событий provenance процессо
В. """
904         results: List[Dict[str, Any]] = []
905         query = 'SELECT * FROM process_provenance_events WHERE
1=1'
906         params: List[Any] = []
907
908         if hours is not None:
909             cutoff = time.time() - (hours * 3600)
910             query += ' AND created_at >= ?'
911             params.append(cutoff)
912         if guid:
913             query += ' AND process_guid = ?'
914             params.append(guid)
915         if pid is not None:
916             query += ' AND pid = ?'
917             params.append(pid)
918         if name:
919             query += ' AND name LIKE ?'
920             params.append(f'%{name}%')
921         if user:
922             query += ' AND (user LIKE ? OR sid LIKE ?)'

```

```

923         params.extend([f'%{user}%', f'%{user}%'])
924     if event_type:
925         query += ' AND event_type = ?'
926         params.append(event_type)
927
928     query += ' ORDER BY created_at DESC LIMIT ?'
929     params.append(limit)
930
931     try:
932         with self._cm.lock, self._cm.get_connection() as c
933         onn:
934             cursor = conn.cursor()
935             cursor.execute(query, tuple(params))
936             for row in cursor.fetchall():
937                 item = dict(row)
938                 if item.get('details_json'):
939                     try:
940                         item['details'] = json.loads(item[
941                         'details_json'])
942                     except Exception:
943                         item['details'] = item['details_js
944                         on']
945                 results.append(item)
946     except Exception as ex:
947         logger.error(f'Ошибка извлечения provenance процес
948         сов: {ex}')
949     return results
950
951     def get_process_lineage(self, guid_or_pid: Union[str, int]
952     ) -> List[Dict[str, Any]]:
953         """Построение полной восходящей цепочки предков (linea
954         ge) процесса."""
955         str_val = str(guid_or_pid)
956         chain: List[Dict[str, Any]] = []
957         visited_guids: set = set()
958         visited_pids: set = set()
959
960         history = self.get_process_provenance_history(guid=str

```

```

    _val, limit=1)
955         if not history and str_val.isdigit():
956             history = self.get_process_provenance_history(pid=
int(str_val), limit=1)
957
958         current_node: Optional[Dict[str, Any]] = history[0] if
history else None
959
960         while current_node:
961             node_guid = current_node.get('process_guid')
962             node_pid = current_node.get('pid')
963
964             if (node_guid and node_guid in visited_guids) or (
node_pid and node_pid in visited_pids):
965                 break
966
967             if node_guid:
968                 visited_guids.add(node_guid)
969             if node_pid:
970                 visited_pids.add(node_pid)
971
972             chain.append(current_node)
973             parent_guid = current_node.get('parent_guid')
974             parent_pid = current_node.get('ppid') or current_n
ode.get('parent_pid')
975
976             parent_node = None
977             if parent_guid:
978                 parent_hist = self.get_process_provenance_hist
ory(guid=parent_guid, limit=1)
979                 if parent_hist:
980                     parent_node = parent_hist[0]
981
982             if not parent_node and parent_pid:
983                 parent_hist = self.get_process_provenance_hist
ory(pid=int(parent_pid), limit=1)
984                 if parent_hist:
985                     parent_node = parent_hist[0]

```



```

986
987         current_node = parent_node
988
989         chain.reverse()
990         return chain
991
992     def get_telemetry_rollups(
993         self,
994         level: str = 'hourly',
995         sensor_id: Optional[str] = None,
996         start_epoch: Optional[float] = None,
997         end_epoch: Optional[float] = None,
998         limit: int = 100,
999     ) -> List[Dict[str, Any]]:
1000         """Извлекает агрегированные показатели сенсоров для за
данного уровня (hourly, daily, weekly, monthly, yearly).
1001
1002         Args:
1003             level: Уровень иерархии ('hourly', 'daily', 'weekl
y', 'monthly', 'yearly').
1004             sensor_id: Фильтр по идентификатору сенсора (опцио
нально).
1005             start_epoch: Нижняя граница времени бакета bucket_
start (опционально).
1006             end_epoch: Верхняя граница времени бакета bucket_s
tart (опционально).
1007             limit: Максимальное число возвращаемых записей.
1008
1009         Returns:
1010             List[Dict[str, Any]]: Список агрегированных бакето
в со статистикой.
1011         """
1012         lvl_clean = str(level).strip().lower()
1013         table_name = f'telemetry_{lvl_clean}'
1014         valid_tables = {'telemetry_hourly', 'telemetry_daily',
'telemetry_weekly', 'telemetry_monthly', 'telemetry_yearly'}
1015         if table_name not in valid_tables:
1016             table_name = 'telemetry_hourly'

```

```

1017
1018     conditions = []
1019     params: List[Any] = []
1020
1021     if sensor_id is not None:
1022         conditions.append('sensor_id = ?')
1023         params.append(sensor_id)
1024     if start_epoch is not None:
1025         conditions.append('bucket_start >= ?')
1026         params.append(float(start_epoch))
1027     if end_epoch is not None:
1028         conditions.append('bucket_start < ?')
1029         params.append(float(end_epoch))
1030
1031     where_clause = f"WHERE {' AND ' .join(conditions)}" if
conditions else ""
1032     query = f"SELECT * FROM {table_name} {where_clause} OR
DER BY bucket_start DESC LIMIT ?"
1033     params.append(int(limit))
1034
1035     with self._cm.lock, self._cm.get_connection() as conn:
1036         cursor = conn.cursor()
1037         cursor.execute(query, params)
1038         return [dict(row) for row in cursor.fetchall()]
1039
1040     def get_telemetry_spikes(
1041         self,
1042         sensor_id: Optional[str] = None,
1043         start_epoch: Optional[float] = None,
1044         limit: int = 100,
1045     ) -> List[Dict[str, Any]]:
1046         """Извлекает зафиксированные статистические всплески и
аномалии сенсоров.
1047
1048         Args:
1049             sensor_id: Фильтр по идентификатору сенсора (опцио
нально).
1050             start_epoch: Нижняя граница времени регистрации (о

```

пционально).

```
1051         limit: Максимальное число возвращаемых записей.
1052
1053     Returns:
1054         List[Dict[str, Any]]: Список аномальных записей.
1055     """
1056     conditions = []
1057     params: List[Any] = []
1058
1059     if sensor_id is not None:
1060         conditions.append('sensor_id = ?')
1061         params.append(sensor_id)
1062     if start_epoch is not None:
1063         conditions.append('created_at >= ?')
1064         params.append(float(start_epoch))
1065
1066     where_clause = f"WHERE {' AND '.join(conditions)}" if
conditions else ""
1067     query = f"SELECT * FROM telemetry_spikes {where_clause}
ORDER BY created_at DESC LIMIT ?"
1068     params.append(int(limit))
1069
1070     with self._cm.lock, self._cm.get_connection() as conn:
1071         cursor = conn.cursor()
1072         cursor.execute(query, params)
1073         return [dict(row) for row in cursor.fetchall()]
1074
1075     def get_disk_inventory(self, disk_id: Optional[int] = None
) -> List[Dict[str, Any]]:
1076         """Извлекает паспортные данные физических дисков из та
блицы disk_inventory."""
1077         with self._cm.lock, self._cm.get_connection() as conn:
1078             cursor = conn.cursor()
1079             if disk_id is not None:
1080                 cursor.execute('SELECT * FROM disk_inventory W
HERE disk_id = ? ORDER BY disk_id ASC', (disk_id,))
1081             else:
1082                 cursor.execute('SELECT * FROM disk_inventory 0
```

```

1082 |         ORDER BY disk_id ASC')
1083 |         return [dict(row) for row in cursor.fetchall()]
1084 |
1085 |     def get_volume_inventory(self) -> List[Dict[str, Any]]:
1086 |         """Извлекает паспортные данные логических томов из таб
лицы volume_inventory."""
1087 |         with self._cm.lock, self._cm.get_connection() as conn:
1088 |             cursor = conn.cursor()
1089 |             cursor.execute('SELECT * FROM volume_inventory ORD
ER BY drive_letter ASC')
1090 |             return [dict(row) for row in cursor.fetchall()]
1091 |
1092 |     def get_disk_health_history(
1093 |         self,
1094 |         disk_id: Optional[int] = None,
1095 |         serial_number: Optional[str] = None,
1096 |         limit: int = 100,
1097 |         since_epoch: Optional[float] = None,
1098 |     ) -> List[Dict[str, Any]]:
1099 |         """Извлекает историю показателей здоровья и износа нак
опителей."""
1100 |         conditions = []
1101 |         params: List[Any] = []
1102 |
1103 |         if disk_id is not None:
1104 |             conditions.append('disk_id = ?')
1105 |             params.append(disk_id)
1106 |         if serial_number is not None:
1107 |             conditions.append('serial_number = ?')
1108 |             params.append(serial_number)
1109 |         if since_epoch is not None:
1110 |             conditions.append('created_at >= ?')
1111 |             params.append(float(since_epoch))
1112 |
1113 |         where_clause = f"WHERE {' AND '.join(conditions)}" if
conditions else ""
1114 |         query = f"SELECT * FROM disk_health_snapshots {where_c
lause} ORDER BY created_at DESC LIMIT ?"

```

```

1115         params.append(int(limit))
1116
1117         with self._cm.lock, self._cm.get_connection() as conn:
1118             cursor = conn.cursor()
1119             cursor.execute(query, params)
1120             return [dict(row) for row in cursor.fetchall()]
1121
1122     def get_disk_performance_samples(
1123         self,
1124         disk_name: Optional[str] = None,
1125         limit: int = 100,
1126         since_epoch: Optional[float] = None,
1127     ) -> List[Dict[str, Any]]:
1128         """Извлекает историю показателей производительности ди
1129         сков (IOPS, MB/s)."""
1130         conditions = []
1131         params: List[Any] = []
1132
1133         if disk_name is not None:
1134             conditions.append('disk_name = ?')
1135             params.append(disk_name)
1136         if since_epoch is not None:
1137             conditions.append('created_at >= ?')
1138             params.append(float(since_epoch))
1139
1140         where_clause = f"WHERE {' AND '}.join(conditions))" if
1141         conditions else ""
1142         query = f"SELECT * FROM disk_performance_samples {wher
1143         e_clause} ORDER BY created_at DESC LIMIT ?"
1144         params.append(int(limit))
1145
1146         with self._cm.lock, self._cm.get_connection() as conn:
1147             cursor = conn.cursor()
1148             cursor.execute(query, params)
1149             return [dict(row) for row in cursor.fetchall()]
1150
1151     def get_disk_io_events(
1152         self,

```

```

1150         pid: Optional[int] = None,
1151         disk_id: Optional[int] = None,
1152         limit: int = 100,
1153         since_epoch: Optional[float] = None,
1154     ) -> List[Dict[str, Any]]:
1155         """Извлекает события прямого ввода-вывода процессов."""
1156
1157         conditions = []
1158         params: List[Any] = []
1159
1160         if pid is not None:
1161             conditions.append('pid = ?')
1162             params.append(pid)
1163         if disk_id is not None:
1164             conditions.append('disk_id = ?')
1165             params.append(disk_id)
1166         if since_epoch is not None:
1167             conditions.append('created_at >= ?')
1168             params.append(float(since_epoch))
1169
1170         where_clause = f"WHERE {' AND '}.join(conditions)" if
conditions else ""
1171         query = f"SELECT * FROM disk_io_events {where_clause}
ORDER BY created_at DESC LIMIT ?"
1172         params.append(int(limit))
1173
1174         with self._cm.lock, self._cm.get_connection() as conn:
1175             cursor = conn.cursor()
1176             cursor.execute(query, params)
1177             return [dict(row) for row in cursor.fetchall()]
1178
1179     def get_cpu_inventory(self, processor_id: Optional[int] =
None) -> List[Dict[str, Any]]:
1180         """Извлекает паспортные данные процессоров из таблицы
cpu_inventory."""
1181
1182         with self._cm.lock, self._cm.get_connection() as conn:
1183             cursor = conn.cursor()
1184             if processor_id is not None:

```

```
1183 |         cursor.execute('SELECT * FROM cpu_inventory WH
| ERE processor_id = ? ORDER BY processor_id ASC', (processor_id
| ,))
1184 |         else:
1185 |             cursor.execute('SELECT * FROM cpu_inventory OR
| DER BY processor_id ASC')
1186 |             return [dict(row) for row in cursor.fetchall()]
1187 |
1188 |     def get_ram_module_inventory(self) -> List[Dict[str, Any]]
| :
1189 |         """Извлекает паспортные данные физических планок памят
| и из таблицы ram_module_inventory."""
1190 |         with self._cm.lock, self._cm.get_connection() as conn:
1191 |             cursor = conn.cursor()
1192 |             cursor.execute('SELECT * FROM ram_module_inventory
| ORDER BY slot_id ASC')
1193 |             return [dict(row) for row in cursor.fetchall()]
1194 |
1195 |     def get_gpu_inventory(self, gpu_id: Optional[int] = None)
| -> List[Dict[str, Any]]:
1196 |         """Извлекает паспортные данные графических ускорителей
| из таблицы gpu_inventory."""
1197 |         with self._cm.lock, self._cm.get_connection() as conn:
1198 |             cursor = conn.cursor()
1199 |             if gpu_id is not None:
1200 |                 cursor.execute('SELECT * FROM gpu_inventory WH
| ERE gpu_id = ? ORDER BY gpu_id ASC', (gpu_id,))
1201 |             else:
1202 |                 cursor.execute('SELECT * FROM gpu_inventory OR
| DER BY gpu_id ASC')
1203 |             return [dict(row) for row in cursor.fetchall()]
1204 |
1205 |     def get_network_adapter_inventory(self) -> List[Dict[str,
| Any]]:
1206 |         """Извлекает паспортные данные сетевых интерфейсов из
| таблицы network_adapter_inventory."""
1207 |         with self._cm.lock, self._cm.get_connection() as conn:
1208 |             cursor = conn.cursor()
```

```

1209         cursor.execute('SELECT * FROM network_adapter_inve
ntory ORDER BY adapter_id ASC')
1210         return [dict(row) for row in cursor.fetchall()]
1211
1212     def get_cpu_telemetry_samples(
1213         self,
1214         processor_id: Optional[int] = None,
1215         limit: int = 100,
1216         since_epoch: Optional[float] = None,
1217     ) -> List[Dict[str, Any]]:
1218         """Извлекает историю показателей нагрузки и температур
процессоров."""
1219         conditions = []
1220         params: List[Any] = []
1221
1222         if processor_id is not None:
1223             conditions.append('processor_id = ?')
1224             params.append(processor_id)
1225         if since_epoch is not None:
1226             conditions.append('created_at >= ?')
1227             params.append(float(since_epoch))
1228
1229         where_clause = f"WHERE {' AND '.join(conditions)}" if
conditions else ""
1230         query = f"SELECT * FROM cpu_telemetry_samples {where_c
lause} ORDER BY created_at DESC LIMIT ?"
1231         params.append(int(limit))
1232
1233         with self._cm.lock, self._cm.get_connection() as conn:
1234             cursor = conn.cursor()
1235             cursor.execute(query, params)
1236             return [dict(row) for row in cursor.fetchall()]
1237
1238     def get_ram_telemetry_samples(
1239         self,
1240         limit: int = 100,
1241         since_epoch: Optional[float] = None,
1242     ) -> List[Dict[str, Any]]:

```



```

1243 |         """Извлекает историю показателей использования оперативной
1244 |         памяти."""
1245 |         conditions = []
1246 |         params: List[Any] = []
1247 |
1248 |         if since_epoch is not None:
1249 |             conditions.append('created_at >= ?')
1250 |             params.append(float(since_epoch))
1251 |
1252 |         where_clause = f"WHERE {' AND '}.join(conditions)}" if conditions else ""
1253 |         query = f"SELECT * FROM ram_telemetry_samples {where_clause} ORDER BY created_at DESC LIMIT ?"
1254 |         params.append(int(limit))
1255 |
1256 |         with self._cm.lock, self._cm.get_connection() as conn:
1257 |             cursor = conn.cursor()
1258 |             cursor.execute(query, params)
1259 |             return [dict(row) for row in cursor.fetchall()]
1260 |
1261 |     def get_gpu_telemetry_samples(
1262 |         self,
1263 |         gpu_id: Optional[int] = None,
1264 |         name: Optional[str] = None,
1265 |         limit: int = 100,
1266 |         since_epoch: Optional[float] = None,
1267 |     ) -> List[Dict[str, Any]]:
1268 |         """Извлекает историю показателей нагрузки, памяти и температуры GPU."""
1269 |         conditions = []
1270 |         params: List[Any] = []
1271 |
1272 |         if gpu_id is not None:
1273 |             conditions.append('gpu_id = ?')
1274 |             params.append(gpu_id)
1275 |
1276 |         if name is not None:
1277 |             conditions.append('name = ?')
1278 |             params.append(name)

```

```

1277         if since_epoch is not None:
1278             conditions.append('created_at >= ?')
1279             params.append(float(since_epoch))
1280
1281         where_clause = f"WHERE {' AND '}.join(conditions)}" if
conditions else ""
1282         query = f"SELECT * FROM gpu_telemetry_samples {where_c
lause} ORDER BY created_at DESC LIMIT ?"
1283         params.append(int(limit))
1284
1285         with self._cm.lock, self._cm.get_connection() as conn:
1286             cursor = conn.cursor()
1287             cursor.execute(query, params)
1288             return [dict(row) for row in cursor.fetchall()]
1289
1290     def get_network_adapter_samples(
1291         self,
1292         adapter_name: Optional[str] = None,
1293         limit: int = 100,
1294         since_epoch: Optional[float] = None,
1295     ) -> List[Dict[str, Any]]:
1296         """Извлекает историю показателей сетевой активности ад
аптеров."""
1297         conditions = []
1298         params: List[Any] = []
1299
1300         if adapter_name is not None:
1301             conditions.append('adapter_name = ?')
1302             params.append(adapter_name)
1303         if since_epoch is not None:
1304             conditions.append('created_at >= ?')
1305             params.append(float(since_epoch))
1306
1307         where_clause = f"WHERE {' AND '}.join(conditions)}" if
conditions else ""
1308         query = f"SELECT * FROM network_adapter_samples {where
_clause} ORDER BY created_at DESC LIMIT ?"
1309         params.append(int(limit))

```

```

1310
1311         with self._cm.lock, self._cm.get_connection() as conn:
1312             cursor = conn.cursor()
1313             cursor.execute(query, params)
1314             return [dict(row) for row in cursor.fetchall()]
1315
1316         # =====
1317         # Подсистемы рефакторинга: Data-First SQLite Readers
1318         # =====
1319
1320         # 1. Windows Event Logs
1321         def get_latest_event_log_channels(self) -> List[Dict[str,
1322         Any]]:
1323             """Извлечение последнего снимка каналов журналов событ
1324             ий."""
1325             with self._cm.lock, self._cm.get_connection() as conn:
1326                 cursor = conn.cursor()
1327                 cursor.execute('''
1328                     SELECT snapshot_id FROM event_log_channel_snap
1329                     shots
1330                     ORDER BY id DESC LIMIT 1
1331                 ''')
1332                 row = cursor.fetchone()
1333                 if not row:
1334                     return []
1335                 snap_id = row['snapshot_id'] if isinstance(row, sq
1336                 lite3.Row) else row[0]
1337                 cursor.execute('''
1338                     SELECT channel_name AS name, is_enabled AS ena
1339                     bled, record_count,
1340                     size_bytes, channel_type, display_name,
1341                     description
1342                     FROM event_log_channel_snapshots
1343                     WHERE snapshot_id = ?
1344                     ORDER BY record_count DESC
1345                 ''', (snap_id,))

```

```

1340         return [dict(r) for r in cursor.fetchall()]
1341
1342     def get_event_log_entries(
1343         self,
1344         channel: str = 'System',
1345         level: str = '',
1346         limit: int = 50,
1347         hours: int = 24,
1348     ) -> List[Dict[str, Any]]:
1349         """Выборка записей журнала событий из кеша по каналу и
1350         уровню."""
1351         conditions = ['channel = ?']
1352         params: List[Any] = [channel]
1353
1354         if level:
1355             conditions.append('LOWER(level) = LOWER(?)')
1356             params.append(level)
1357
1358         if hours > 0:
1359             since_time = time.time() - (hours * 3600)
1360             conditions.append('created_at >= ?')
1361             params.append(since_time)
1362
1363         where_clause = f"WHERE {' AND '.join(conditions)}"
1364         query = f'''
1365             SELECT channel, event_id, level, provider_name, ti
1366             me_created, message
1367             FROM event_log_entries_cache
1368             {where_clause}
1369             ORDER BY id DESC
1370             LIMIT ?
1371         '''
1372         params.append(int(limit))
1373
1374         with self._cm.lock, self._cm.get_connection() as conn:
1375             cursor = conn.cursor()
1376             cursor.execute(query, params)
1377             rows = [dict(r) for r in cursor.fetchall()]

```

```

1376         if not rows and hours > 0:
1377             # Fallback без фильтра по времени, если база б
        ыла заполнена раньше
1378             params[-2] = 0.0
1379             cursor.execute(query, params)
1380             rows = [dict(r) for r in cursor.fetchall()]
1381         return rows
1382
1383     def get_latest_event_log_intelligence_profile(self, channel: str = 'System') -> Optional[Dict[str, Any]]:
1384         """Извлечение последнего сохраненного профиля Log Intelligence."""
1385         with self._cm.lock, self._cm.get_connection() as conn:
1386             cursor = conn.cursor()
1387             cursor.execute('''
1388                 SELECT * FROM event_log_intelligence_profiles
1389                 WHERE channel = ?
1390                 ORDER BY id DESC LIMIT 1
1391             ''', (channel,))
1392             row = cursor.fetchone()
1393             if not row:
1394                 return None
1395             d = dict(row)
1396             d['critical_incidents'] = json.loads(d['critical_incidents_json']) if d.get('critical_incidents_json') else []
1397             d['top_clusters'] = json.loads(d['top_clusters_json']) if d.get('top_clusters_json') else []
1398             d['bursts'] = json.loads(d['bursts_json']) if d.get('bursts_json') else []
1399             decision_gate = json.loads(d['decision_gate_json']) if d.get('decision_gate_json') else {}
1400             profile_data = {
1401                 'total_events': d.get('total_analyzed', 0),
1402                 'unique_templates_count': d.get('unique_patterns_count', 0),
1403                 'critical_count': len(d['critical_incidents'])
1404             },
            'health_score': 100 - min(100, len(d['critical

```

```

    _incidents']) * 10),
1405         'top_patterns': d['top_clusters'],
1406         'bursts': d['bursts'],
1407     }
1408     d['profile'] = profile_data
1409     d['decision'] = decision_gate
1410     return d
1411
1412     def get_latest_event_log_report(self) -> Optional[Dict[str
, Any]]:
1413         """Формирование сводного отчета о журналах и ошибках и
з базы данных."""
1414         channels = self.get_latest_event_log_channels()
1415         if not channels:
1416             return None
1417         with self._cm.lock, self._cm.get_connection() as conn:
1418             cursor = conn.cursor()
1419             cursor.execute('''
1420                 SELECT channel, event_id, level, provider_name
, time_created, message
1421                 FROM event_log_entries_cache
1422                 WHERE LOWER(level) IN ('error', 'critical', 'w
arning')
1423                 ORDER BY id DESC LIMIT 20
1424             ''')
1425             errors = [dict(r) for r in cursor.fetchall()]
1426
1427             crit_count = sum(1 for e in errors if str(e.get('l
evel', ''')).lower() == 'critical')
1428             err_count = sum(1 for e in errors if str(e.get('le
vel', ''')).lower() == 'error')
1429             warn_count = sum(1 for e in errors if str(e.get('l
evel', ''')).lower() == 'warning')
1430
1431             return {
1432                 'total_channels': len(channels),
1433                 'critical_events_24h': crit_count,
1434                 'error_events_24h': err_count,

```

```

1435         'warning_events_24h': warn_count,
1436         'channels': channels,
1437         'recent_errors': errors,
1438         'timestamp': datetime.now(timezone.utc).isofor
mat(),
1439     }
1440
1441     # 2. Windows Firewall Manager
1442     def get_latest_firewall_profiles(self) -> Optional[Dict[str, Any]]:
1443         """Извлечение состояния профилей брандмауэра Windows."""
1444         with self._cm.lock, self._cm.get_connection() as conn:
1445             cursor = conn.cursor()
1446             cursor.execute('''
1447                 SELECT domain_enabled, private_enabled, public
_enabled,
1448                        domain_default_inbound, private_default
_inbound, public_default_inbound,
1449                        stealth_mode_enabled, timestamp
1450                 FROM firewall_profile_snapshots
1451                 ORDER BY id DESC LIMIT 1
1452             ''')
1453             row = cursor.fetchone()
1454             return dict(row) if row else None
1455
1456     def get_latest_firewall_rules(self, direction: Optional[str] = None, limit: int = 200) -> List[Dict[str, Any]]:
1457         """Выборка правил фильтрации брандмауэра Windows из по
следнего снимка."""
1458         with self._cm.lock, self._cm.get_connection() as conn:
1459             cursor = conn.cursor()
1460             cursor.execute('SELECT snapshot_id FROM firewall_r
ule_snapshots ORDER BY id DESC LIMIT 1')
1461             row = cursor.fetchone()
1462             if not row:
1463                 return []
1464             snap_id = row['snapshot_id'] if isinstance(row, sq

```

```

lite3.Row) else row[0]
1465
1466         conditions = ['snapshot_id = ?']
1467         params: List[Any] = [snap_id]
1468         if direction:
1469             conditions.append('LOWER(direction) = LOWER(?)
1470         ')
1471             params.append(direction)
1472
1473         where_clause = f"WHERE {' AND '}.join(conditions)}"
1474         query = f'''
1475             SELECT rule_name AS name, display_name, direct
1476             ion, action, enabled,
1477             protocol, local_port, remote_port, prog
1478             ram_path, profile_mask
1479             FROM firewall_rule_snapshots
1480             {where_clause}
1481             ORDER BY id ASC
1482             LIMIT ?
1483             '''
1484         params.append(int(limit))
1485         cursor.execute(query, params)
1486         return [dict(r) for r in cursor.fetchall()]
1487
1488     # 3. Windows Services Manager
1489     def get_latest_services_list(self, status: Optional[str] =
1490     None, limit: int = 500) -> List[Dict[str, Any]]:
1491         """Извлечение списка служб Windows из последнего снимк
1492         а."""
1493         with self._cm.lock, self._cm.get_connection() as conn:
1494             cursor = conn.cursor()
1495             cursor.execute('SELECT snapshot_id FROM services_s
1496             napshots ORDER BY id DESC LIMIT 1')
1497             row = cursor.fetchone()
1498             if not row:
1499                 return []
1500             snap_id = row['snapshot_id'] if isinstance(row, sq
1501             lite3.Row) else row[0]

```



```

1495
1496         conditions = ['snapshot_id = ?']
1497         params: List[Any] = [snap_id]
1498         if status:
1499             conditions.append('LOWER(state) = LOWER(?)')
1500             params.append(status)
1501
1502         where_clause = f"WHERE {' AND '}.join(conditions)}"
1503         query = f'''
1504             SELECT service_name AS name, display_name, sta
1505 te AS status,
1506             start_type AS startup_type, pid, binary
1507 _path AS binpath,
1508             account AS user_account, is_orphaned
1509 FROM services_snapshots
1510 {where_clause}
1511 ORDER BY service_name ASC
1512 LIMIT ?
1513 '''
1514         params.append(int(limit))
1515         cursor.execute(query, params)
1516         return [dict(r) for r in cursor.fetchall()]
1517
1518     def get_latest_services_report(self) -> Optional[Dict[str,
1519 Any]]:
1520         """Формирование сводного отчета о службах Windows."""
1521         services = self.get_latest_services_list(limit=1000)
1522         if not services:
1523             return None
1524         total = len(services)
1525         running = sum(1 for s in services if str(s.get('status
1526 ', '')).upper() in ('RUNNING', '4', 'SERVICE_RUNNING'))
1527         stopped = sum(1 for s in services if str(s.get('status
1528 ', '')).upper() in ('STOPPED', '1', 'SERVICE_STOPPED'))
1529         orphaned = sum(1 for s in services if s.get('is_orphan
1530 ed'))
1531
1532         return {

```

```

1527         'total_services': total,
1528         'running_services': running,
1529         'stopped_services': stopped,
1530         'orphaned_services': orphaned,
1531         'services': services,
1532         'timestamp': datetime.now(timezone.utc).isoformat(
1533     ),
1534     }
1535
1536     def get_service_change_events(self, limit: int = 100) -> List[Dict[str, Any]]:
1537         """Извлечение истории изменений служб."""
1538         with self._cm.lock, self._cm.get_connection() as conn:
1539             cursor = conn.cursor()
1540             cursor.execute('''
1541                 SELECT timestamp, service_name, display_name,
1542                 action,
1543                 old_state, new_state, performed_by, details_json
1544                 FROM service_change_events
1545                 ORDER BY id DESC LIMIT ?
1546             ''', (int(limit),))
1547             rows = []
1548             for r in cursor.fetchall():
1549                 d = dict(r)
1550                 if d.get('details_json'):
1551                     d['details'] = json.loads(d['details_json'])
1552             rows.append(d)
1553             return rows
1554
1555     # 4. CPU Throttling & Power Limits
1556     def get_latest_throttling_snapshot(self) -> Optional[Dict[str, Any]]:
1557         """Извлечение последнего снимка троттлинга процессора
и термических зон."""
1558         with self._cm.lock, self._cm.get_connection() as conn:
1559             cursor = conn.cursor()

```

```

1558         cursor.execute('''
1559             SELECT snapshot_id, timestamp, prochot_active,
1560             pl1_limit_watts,
1561             pl2_limit_watts, current_power_watts, m
1562             ax_core_temp_c,
1563             package_temp_c, dpc_latency_us, isr_lat
1564             ency_us,
1565             throttling_reasons_json
1566             FROM cpu_throttling_snapshots
1567             ORDER BY id DESC LIMIT 1
1568             ''')
1569         row = cursor.fetchone()
1570         if not row:
1571             return None
1572         d = dict(row)
1573         d['throttling_reasons'] = json.loads(d['throttling
1574         _reasons_json']) if d.get('throttling_reasons_json') else []
1575
1576         snap_id = d['snapshot_id']
1577         cursor.execute('''
1578             SELECT zone_name, temperature_c, critical_limi
1579             t_c, throttling_limit_c, sensor_provider
1580             FROM thermal_zone_snapshots
1581             WHERE snapshot_id = ?
1582             ''', (snap_id,))
1583         d['thermal_zones'] = [dict(z) for z in cursor.fetc
1584         hall()]
1585
1586         return d
1587
1588     # 5. Behavioral Forensics
1589     def get_latest_forensics_snapshot(self) -> Optional[Dict[s
1590     tr, Any]]:
1591         """Извлечение последнего снимка поведенческой форензик
1592         и."""
1593
1594         with self._cm.lock, self._cm.get_connection() as conn:
1595             cursor = conn.cursor()
1596             cursor.execute('''
1597                 SELECT foreground_window_title, foreground_pro

```

```

cess_name,
1588         foreground_pid, user_idle_seconds, came
ra_active_apps_json,
1589         microphone_active_apps_json, userassist
_top_apps_json, timestamp
1590         FROM forensics_snapshots
1591         ORDER BY id DESC LIMIT 1
1592         '', )
1593         row = cursor.fetchone()
1594         if not row:
1595             return None
1596         d = dict(row)
1597         d['camera_active_apps'] = json.loads(d['camera_act
ive_apps_json']) if d.get('camera_active_apps_json') else []
1598         d['microphone_active_apps'] = json.loads(d['microp
hone_active_apps_json']) if d.get('microphone_active_apps_json
') else []
1599         d['userassist_top_apps'] = json.loads(d['userassis
t_top_apps_json']) if d.get('userassist_top_apps_json') else [
]
1600         return d
1601
1602     # 6. Process Leaks & Resource Starvation
1603     def get_latest_process_leak_report(self, limit: int = 20)
-> Optional[Dict[str, Any]]:
1604         """Извлечение последнего снимка утечек ресурсов процес
сов."""
1605         with self._cm.lock, self._cm.get_connection() as conn:
1606             cursor = conn.cursor()
1607             cursor.execute('''
1608                 SELECT snapshot_id, timestamp, total_processes
, suspicious_count
1609                 FROM process_leak_snapshots
1610                 ORDER BY id DESC LIMIT 1
1611             ''')
1612             row = cursor.fetchone()
1613             if not row:
1614                 return None

```

```

1615         snap_id = row['snapshot_id'] if isinstance(row, sq
| lite3.Row) else row[0]
1616         report = dict(row)
1617
1618         cursor.execute('''
1619             SELECT pid, name, handles_count, gdi_objects,
| user_objects,
1620             page_faults, working_set_mb, leak_risk_
| score, leak_risk_reasons_json
1621             FROM process_leak_items
1622             WHERE snapshot_id = ?
1623             ORDER BY handles_count DESC
1624             LIMIT ?
1625             ''', (snap_id, int(limit)))
1626         items = []
1627         for r in cursor.fetchall():
1628             d = dict(r)
1629             d['leak_risk_reasons'] = json.loads(d['leak_r
| sk_reasons_json']) if d.get('leak_risk_reasons_json') else []
1630             items.append(d)
1631             report['all_processes'] = items
1632             report['top_handle_hogs'] = sorted(items, key=lamb
| da x: x.get('handles_count', 0), reverse=True)[:10]
1633             report['top_gdi_hogs'] = sorted(items, key=lambda
| x: x.get('gdi_objects', 0), reverse=True)[:10]
1634             report['top_page_fault_hogs'] = sorted(items, key=
| lambda x: x.get('page_faults', 0), reverse=True)[:10]
1635             report['suspicious_processes'] = [x for x in items
| if x.get('leak_risk_score') != 'normal']
1636             return report
1637
1638     # 7. Defender Security
1639     def get_latest_defender_status(self) -> Optional[Dict[str,
| Any]]:
1640         """Извлечение статуса Защитника Windows."""
1641         with self._cm.lock, self._cm.get_connection() as conn:
1642             cursor = conn.cursor()
1643             cursor.execute('''

```

```

1644         SELECT * FROM defender_snapshots
1645         ORDER BY id DESC LIMIT 1
1646     '')
1647     row = cursor.fetchone()
1648     if not row:
1649         return None
1650     d = dict(row)
1651     if d.get('raw_status_json'):
1652         try:
1653             d.update(json.loads(d['raw_status_json']))
1654         except Exception:
1655             pass
1656     return d
1657
1658     def get_latest_defender_exclusions(self, limit: int = 200)
-> List[Dict[str, Any]]:
1659         """Извлечение исключений антивируса Defender."""
1660         with self._cm.lock, self._cm.get_connection() as conn:
1661             cursor = conn.cursor()
1662             cursor.execute('SELECT snapshot_id FROM defender_e
xclusions ORDER BY id DESC LIMIT 1')
1663             row = cursor.fetchone()
1664             if not row:
1665                 return []
1666             snap_id = row['snapshot_id'] if isinstance(row, sq
lite3.Row) else row[0]
1667             cursor.execute(''
1668                 SELECT exclusion_type, exclusion_value, risk_l
evel
1669                 FROM defender_exclusions
1670                 WHERE snapshot_id = ?
1671                 LIMIT ?
1672             ''', (snap_id, limit))
1673             items = []
1674             for r in cursor.fetchall():
1675                 d = dict(r)
1676                 d['type'] = d.get('exclusion_type', 'path')
1677                 d['value'] = d.get('exclusion_value', '')

```

```

1678         items.append(d)
1679     return items
1680
1681     def get_latest_defender_asr_rules(self) -> List[Dict[str,
Any]]:
1682         """Извлечение правил ASR Защитника Windows."""
1683         with self._cm.lock, self._cm.get_connection() as conn:
1684             cursor = conn.cursor()
1685             cursor.execute('SELECT snapshot_id FROM defender_a
sr_rules ORDER BY id DESC LIMIT 1')
1686             row = cursor.fetchone()
1687             if not row:
1688                 return []
1689             snap_id = row['snapshot_id'] if isinstance(row, sq
lite3.Row) else row[0]
1690             cursor.execute('''
1691                 SELECT rule_guid, rule_name, rule_action
1692                 FROM defender_asr_rules
1693                 WHERE snapshot_id = ?
1694             ''', (snap_id,))
1695             items = []
1696             for r in cursor.fetchall():
1697                 d = dict(r)
1698                 d['guid'] = d.get('rule_guid', '')
1699                 d['name'] = d.get('rule_name', '')
1700                 d['state'] = d.get('rule_action', 'not_configu
red')
1701                 items.append(d)
1702     return items
1703
1704     def get_latest_defender_threats(self, limit: int = 50) ->
List[Dict[str, Any]]:
1705         """Извлечение обнаруженных угроз Defender."""
1706         with self._cm.lock, self._cm.get_connection() as conn:
1707             cursor = conn.cursor()
1708             cursor.execute('''
1709                 SELECT threat_id, threat_name, severity, categ
ory, resources_json, detection_time

```

```

1710         FROM defender_threats
1711         ORDER BY id DESC LIMIT ?
1712         ''' , (limit,))
1713         rows = []
1714         for r in cursor.fetchall():
1715             d = dict(r)
1716             d['resources'] = json.loads(d['resources_json']
1717 ])) if d.get('resources_json') else []
1718             d['status'] = d.get('status', 'Cleaned')
1719             d['initial_detection_time'] = d.get('detection
1720 _time')
1721             rows.append(d)
1722         return rows
1723
1724     def get_latest_defender_diagnostics(self) -> Dict[str, Any
1725 ]:
1726         """Сводная диагностика безопасности Defender."""
1727         status = self.get_latest_defender_status() or {}
1728         exclusions = self.get_latest_defender_exclusions()
1729         asr_rules = self.get_latest_defender_asr_rules()
1730         threats = self.get_latest_defender_threats()
1731         return {
1732             'status': status,
1733             'exclusions': exclusions,
1734             'asr_rules': asr_rules,
1735             'threats': threats,
1736             'total_exclusions': len(exclusions),
1737             'total_asr_rules': len(asr_rules),
1738             'active_threats_count': len(threats),
1739         }
1740
1741     # 8. Process Network & Process Manager
1742     def get_latest_process_network_activity(
1743         self,
1744         limit: int = 100,
1745         only_internet: bool = True,
1746     ) -> List[Dict[str, Any]]:
1747         """Извлечение сетевой активности процессов из process_

```



```

network_snapshots."""
1745         with self._cm.lock, self._cm.get_connection() as conn:
1746             cursor = conn.cursor()
1747             cursor.execute('SELECT snapshot_id FROM process_ne
network_snapshots ORDER BY id DESC LIMIT 1')
1748             row = cursor.fetchone()
1749             if not row:
1750                 return []
1751             snap_id = row['snapshot_id'] if isinstance(row, sq
lite3.Row) else row[0]
1752
1753             conditions = ['snapshot_id = ?']
1754             params: List[Any] = [snap_id]
1755             if only_internet:
1756                 conditions.append("LOWER(service_type) = 'inte
rnet' OR remote_address != '')")
1757
1758             where_clause = f"WHERE {' AND '.join(conditions)}"
1759             query = f'''
1760                 SELECT pid, process_name, local_address, remot
e_address, protocol,
1761                     status, service_type, sent_kb, recv_kb,
read_speed_kbs, write_speed_kbs
1762                 FROM process_network_snapshots
1763                 {where_clause}
1764                 ORDER BY sent_kb + recv_kb DESC
1765                 LIMIT ?
1766             '''
1767             params.append(int(limit))
1768             cursor.execute(query, params)
1769             return [dict(r) for r in cursor.fetchall()]
1770
1771     # 9. Software Transparency Scanner
1772     def get_software_inventory_from_db(self, limit: int = 200,
offset: int = 0) -> List[Dict[str, Any]]:
1773         """Извлечение инвентаря установленного ПО с агрегацией
данных из SQLite."""
1774         with self._cm.lock, self._cm.get_connection() as conn:

```

```

1775         cursor = conn.cursor()
1776         cursor.execute('''
1777             SELECT s.*,
1778                 (SELECT COUNT(*) FROM software_storage_
locations l WHERE l.app_id = s.app_id) AS storage_locations_co
unt,
1779                 (SELECT COALESCE(SUM(size_bytes), 0) FR
OM software_storage_locations l WHERE l.app_id = s.app_id) AS
total_storage_bytes,
1780                 (SELECT summary FROM software_ai_resear
ch r WHERE r.app_id = s.app_id) AS ai_summary,
1781                 (SELECT confidence_level FROM software_
ai_research r WHERE r.app_id = s.app_id) AS ai_confidence
1782             FROM software_inventory s
1783             ORDER BY s.display_name ASC
1784             LIMIT ? OFFSET ?
1785             ''', (int(limit), int(offset)))
1786         return [dict(r) for r in cursor.fetchall()]
1787
1788     def get_software_app_details_from_db(self, app_id: str) ->
Optional[Dict[str, Any]]:
1789         """Извлечение полной карточки установленной программы
со всеми связанными данными."""
1790         with self._cm.lock, self._cm.get_connection() as conn:
1791             cursor = conn.cursor()
1792             cursor.execute('SELECT * FROM software_inventory W
HERE app_id = ?', (app_id,))
1793             row = cursor.fetchone()
1794             if not row:
1795                 return None
1796             app_data = dict(row)
1797
1798             cursor.execute('SELECT category, path, size_bytes,
file_count, last_updated FROM software_storage_locations WHER
E app_id = ?', (app_id,))
1799             app_data['storage_locations'] = [dict(r) for r in
cursor.fetchall()]
1800

```

```

1801         cursor.execute('SELECT file_path, format, size_bytes, snippet FROM software_config_files WHERE app_id = ?', (app_id,))
1802         app_data['config_files'] = [dict(r) for r in cursor.fetchall()]
1803
1804         cursor.execute('SELECT * FROM software_ai_research WHERE app_id = ?', (app_id,))
1805         ai_row = cursor.fetchone()
1806         if ai_row:
1807             ai_dict = dict(ai_row)
1808             ai_dict['confirmed_facts'] = json.loads(ai_dict['confirmed_facts_json']) if ai_dict.get('confirmed_facts_json') else []
1809             ai_dict['inferred_facts'] = json.loads(ai_dict['inferred_facts_json']) if ai_dict.get('inferred_facts_json') else []
1810             app_data['ai_research'] = ai_dict
1811         else:
1812             app_data['ai_research'] = None
1813
1814         return app_data
1815
1816     def get_security_events(
1817         self,
1818         event_id: Optional[int] = None,
1819         user: Optional[str] = None,
1820         process_name: Optional[str] = None,
1821         pid: Optional[int] = None,
1822         channel: Optional[str] = None,
1823         limit: int = 100,
1824         offset: int = 0,
1825         since_epoch: Optional[float] = None,
1826     ) -> List[Dict[str, Any]]:
1827         """Выборка нормализованных событий журнала безопасности Windows с фильтрами.
1828
1829         Args:

```

```
1830         event_id: Фильтр по Event ID (например, 4688, 4624
1831         , 1116).
1832         user: Фильтр по имени субъекта или целевого пользо
1833         вателя (подстрока).
1834         process_name: Фильтр по имени процесса.
1835         pid: Фильтр по идентификатору процесса PID.
1836         channel: Фильтр по имени журнала (Security или Def
1837         ender).
1838         limit: Максимальное количество возвращаемых записе
1839         й.
1840         offset: Смещение выборки.
1841         since_epoch: Нижняя граница времени epoch.
1842
1843     Returns:
1844         List[Dict[str, Any]]: Список событий безопасности.
1845
1846     """
1847     conditions: List[str] = []
1848     params: List[Any] = []
1849
1850     if channel:
1851         conditions.append('channel = ?')
1852         params.append(str(channel))
1853
1854     if event_id is not None:
1855         conditions.append('event_id = ?')
1856         params.append(int(event_id))
1857
1858     if user:
1859         conditions.append('(subject_user LIKE ? OR target_
1860         user LIKE ?)')
1861         params.extend([f'"{user}"', f'"{user}"'])
1862
1863     if process_name:
1864         conditions.append('(process_name LIKE ? OR parent_
1865         process_name LIKE ?)')
1866         params.extend([f'"{process_name}"', f'"{process_na
1867         me}"'])
```

```

1861         if pid is not None:
1862             conditions.append('(process_id = ? OR parent_proce
1863 ss_id = ?)')
1864             params.extend([int(pid), int(pid)])
1865
1866         if since_epoch is not None:
1867             conditions.append('created_at >= ?')
1868             params.append(float(since_epoch))
1869
1870         where_clause = f"WHERE {' AND '.join(conditions)}" if
1871 conditions else ""
1872         query = f'''
1873             SELECT * FROM security_events
1874             {where_clause}
1875             ORDER BY id DESC
1876             LIMIT ? OFFSET ?
1877         '''
1878         params.extend([int(limit), int(offset)])
1879
1880         with self._cm.lock, self._cm.get_connection() as conn:
1881             cursor = conn.cursor()
1882             cursor.execute(query, params)
1883             rows = [dict(r) for r in cursor.fetchall()]
1884             for r in rows:
1885                 if r.get('event_data_json'):
1886                     try:
1887                         r['event_data'] = json.loads(r['event_
1888 data_json'])
1889                     except Exception:
1890                         r['event_data'] = {}
1891                 else:
1892                     r['event_data'] = {}
1893             return rows
1894
1895     def get_security_failed_logons(self, limit: int = 50, hour
1896 s: int = 24) -> List[Dict[str, Any]]:
1897         """Выборка событий неудачных попыток входа и сбоев аут
1898 ентификации (4625, 4771).

```

```

1894
1895     Args:
1896         limit: Лимит записей.
1897         hours: Глубина выборки в часах.
1898
1899     Returns:
1900         List[Dict[str, Any]]: Список неудачных попыток вхо
1901 да.
1902         """
1903         since_time = time.time() - (hours * 3600) if hours > 0
1904     else 0.0
1905         query = '''
1906             SELECT * FROM security_events
1907             WHERE event_id IN (4625, 4771) AND created_at >= ?
1908             ORDER BY id DESC
1909             LIMIT ?
1910         '''
1911         with self._cm.lock, self._cm.get_connection() as conn:
1912             cursor = conn.cursor()
1913             cursor.execute(query, (since_time, int(limit)))
1914             return [dict(r) for r in cursor.fetchall()]
1915
1916     def get_security_process_creations(
1917         self,
1918         process_name: Optional[str] = None,
1919         user: Optional[str] = None,
1920         limit: int = 50,
1921     ) -> List[Dict[str, Any]]:
1922         """Выборка событий создания процессов (Event ID 4688)
1923 с информацией о родителях и командной строке.
1924
1925     Args:
1926         process_name: Подстрока имени процесса.
1927         user: Имя пользователя инициатора.
1928         limit: Лимит записей.
1929
1930     Returns:
1931         List[Dict[str, Any]]: Список запусков процессов.

```

```

1929 |         """
1930 |         conditions = ['event_id = 4688']
1931 |         params: List[Any] = []
1932 |
1933 |         if process_name:
1934 |             conditions.append('(process_name LIKE ? OR command
1935 | _line LIKE ?)')
1936 |             params.extend([f'%{process_name}%', f'%{process_na
1937 | me}%'])
1938 |
1939 |         if user:
1940 |             conditions.append('subject_user LIKE ?')
1941 |             params.append(f'%{user}%')
1942 |
1943 |         where_clause = f"WHERE {' AND '}.join(conditions)"
1944 |         query = f'''
1945 |             SELECT id, event_record_id, timestamp, subject_use
1946 | r, process_id,
1947 |             process_name, parent_process_id, parent_pro
1948 | cess_name,
1949 |             command_line, logon_id, message
1950 |             FROM security_events
1951 |             {where_clause}
1952 |             ORDER BY id DESC
1953 |             LIMIT ?
1954 |         '''
1955 |         params.append(int(limit))
1956 |
1957 |         with self._cm.lock, self._cm.get_connection() as conn:
1958 |             cursor = conn.cursor()
1959 |             cursor.execute(query, params)
1960 |             return [dict(r) for r in cursor.fetchall()]

```

1958 | def get_security_bookmark(self, channel: str = 'Security')
1959 | -> Optional[Dict[str, Any]]:
1960 | """Извлечение последней сохраненной закладки для канал
а журнала событий.

```

1961 |         Args:
1962 |             channel: Имя канала (по умолчанию 'Security').
1963 |
1964 |         Returns:
1965 |             Optional[Dict[str, Any]]: Словарь с состоянием за-
1966 | ладки или None.
1967 |         """
1968 |         with self._cm.lock, self._cm.get_connection() as conn:
1969 |             cursor = conn.cursor()
1970 |             cursor.execute('SELECT * FROM security_collector_b
1971 | ookmarks WHERE channel = ?', (channel,))
1972 |             row = cursor.fetchone()
1973 |             return dict(row) if row else None
1974 |
1975 |     def get_security_stats(self) -> Dict[str, Any]:
1976 |         """Получить сводную статистику по событиям безопасност
1977 | и в базе данных SQLite.
1978 |
1979 |         Returns:
1980 |             Dict[str, Any]: Словарь со счетчиками и последним
1981 | record_id.
1982 |         """
1983 |         with self._cm.lock, self._cm.get_connection() as conn:
1984 |             cursor = conn.cursor()
1985 |             cursor.execute('SELECT COUNT(*), MAX(event_record_
1986 | id), MIN(timestamp), MAX(timestamp) FROM security_events')
1987 |             row = cursor.fetchone()
1988 |             total_events = row[0] if row else 0
1989 |             max_rec_id = row[1] if row and row[1] else 0
1990 |             min_ts = row[2] if row and row[2] else ''
1991 |             max_ts = row[3] if row and row[3] else ''
1992 |
1993 |             cursor.execute(''
1994 |                 SELECT event_id, COUNT(*) as cnt
1995 |                 FROM security_events
1996 |                 GROUP BY event_id
1997 |                 ORDER BY cnt DESC
1998 |                 LIMIT 10

```



```

1994         '''
1995         top_ids = {r[0]: r[1] for r in cursor.fetchall()}
1996
1997         cursor.execute('SELECT COUNT(*) FROM security_events WHERE event_id = 4688')
1998         proc_events = cursor.fetchone()[0]
1999
2000         cursor.execute('SELECT COUNT(*) FROM security_events WHERE event_id = 4625')
2001         failed_logons = cursor.fetchone()[0]
2002
2003         return {
2004             'total_events': total_events,
2005             'max_record_id': max_rec_id,
2006             'first_event_time': min_ts,
2007             'latest_event_time': max_ts,
2008             'process_creation_count': proc_events,
2009             'failed_logon_count': failed_logons,
2010             'top_event_ids': top_ids,
2011         }
2012
2013     def get_power_sessions(
2014         self,
2015         limit: int = 50,
2016         offset: int = 0,
2017         shutdown_type: Optional[str] = None,
2018         unexpected_only: bool = False,
2019         clean_only: bool = False,
2020     ) -> List[Dict[str, Any]]:
2021         """Извлечь список сессий питания операционной системы
из SQLite.
2022
2023         Args:
2024             limit: Максимальное количество записей.
2025             offset: Смещение выборки.
2026             shutdown_type: Фильтрация по типу выключения (Restart, Shutdown, Unexpected, etc.).
2027             unexpected_only: Только внезапные завершения работ

```

ы.

```
2028         clean_only: Только штатные исключения.
2029
2030     Returns:
2031         List[Dict[str, Any]]: Список словарей с полями PowerSessionRecord.
2032         """
2033         where_clauses: List[str] = []
2034         params: List[Any] = []
2035
2036         if shutdown_type:
2037             where_clauses.append("LOWER(shutdown_type) = LOWER(
2038                 (?))")
2039             params.append(shutdown_type)
2040
2041         if unexpected_only:
2042             where_clauses.append("unexpected_shutdown = 1")
2043         elif clean_only:
2044             where_clauses.append("clean_shutdown = 1 AND unexpected_shutdown = 0")
2045
2046         sql = "SELECT * FROM power_sessions"
2047         if where_clauses:
2048             sql += " WHERE " + " AND ".join(where_clauses)
2049         sql += " ORDER BY boot_time DESC LIMIT ? OFFSET ?"
2050         params.extend([max(1, limit), max(0, offset)])
2051
2052         with self._cm.lock, self._cm.get_connection() as conn:
2053             cursor = conn.cursor()
2054             cursor.execute(sql, params)
2055             rows = cursor.fetchall()
2056             results: List[Dict[str, Any]] = []
2057             for row in rows:
2058                 item = dict(row)
2059                 if item.get('initiator_chain_json'):
2060                     try:
2061                         item['initiator_chain'] = json.loads(item['initiator_chain_json'])
```

```

2061         except Exception:
2062             item['initiator_chain'] = []
2063     else:
2064         item['initiator_chain'] = []
2065
2066     if item.get('events_json'):
2067         try:
2068             item['events'] = json.loads(item['even
2069 ts_json'])
2070         except Exception:
2071             item['events'] = []
2072     else:
2073         item['events'] = []
2074
2075     item['clean_shutdown'] = bool(item.get('clean_
2076 shutdown'))
2077     item['unexpected_shutdown'] = bool(item.get('u
2078 nexpected_shutdown'))
2079     results.append(item)
2080     return results
2081
2082     def get_power_session_by_id(self, session_id: str) -> Opti
2083 onal[Dict[str, Any]]:
2084         """Извлечь детальную информацию по конкретной сессии п
2085 итания.
2086
2087     Args:
2088         session_id: Идентификатор сессии (session_id или i
2089 d).
2090
2091     Returns:
2092         Optional[Dict[str, Any]]: Словарь с полями сессии
2093 или None.
2094
2095     """
2096     with self._cm.lock, self._cm.get_connection() as conn:
2097         cursor = conn.cursor()
2098         if session_id.isdigit():
2099             cursor.execute("SELECT * FROM power_sessions W

```

```

HERE id = ? OR session_id = ?", (int(session_id), session_id))
2092         else:
2093             cursor.execute("SELECT * FROM power_sessions W
HERE session_id = ?", (session_id,))
2094             row = cursor.fetchone()
2095             if not row:
2096                 return None
2097             item = dict(row)
2098             if item.get('initiator_chain_json'):
2099                 try:
2100                     item['initiator_chain'] = json.loads(item[
'initiator_chain_json'])
2101                 except Exception:
2102                     item['initiator_chain'] = []
2103             else:
2104                 item['initiator_chain'] = []
2105
2106             if item.get('events_json'):
2107                 try:
2108                     item['events'] = json.loads(item['events_j
son'])
2109                 except Exception:
2110                     item['events'] = []
2111             else:
2112                 item['events'] = []
2113
2114             item['clean_shutdown'] = bool(item.get('clean_shut
down'))
2115             item['unexpected_shutdown'] = bool(item.get('unexp
ected_shutdown'))
2116             return item
2117
2118     def get_power_events(
2119         self,
2120         limit: int = 100,
2121         offset: int = 0,
2122         event_id: Optional[Union[int, List[int]]] = None,
2123         event_type: Optional[str] = None,

```

```

2124         hours: Optional[int] = None,
2125     ) -> List[Dict[str, Any]]:
2126         """Извлечь события питания и жизненного цикла из табли
цы power_events.
2127
2128     Args:
2129         limit: Лимит записей.
2130         offset: Смещение.
2131         event_id: Фильтр по Event ID или списку ID.
2132         event_type: Фильтр по типу события (boot, shutdown
, etc.).
2133         hours: Ограничение глубины в часах.
2134
2135     Returns:
2136         List[Dict[str, Any]]: Список событий.
2137     """
2138     where_clauses: List[str] = []
2139     params: List[Any] = []
2140
2141     if isinstance(event_id, list) and event_id:
2142         placeholders = ', '.join(['?'] * len(event_id))
2143         where_clauses.append(f"event_id IN ({placeholders}
)")
2144         params.extend(event_id)
2145     elif isinstance(event_id, int) and event_id > 0:
2146         where_clauses.append("event_id = ?")
2147         params.append(event_id)
2148
2149     if event_type:
2150         where_clauses.append("LOWER(event_type) = LOWER(?)
")
2151         params.append(event_type)
2152
2153     if hours and hours > 0:
2154         cutoff = time.time() - (hours * 3600)
2155         where_clauses.append("created_at >= ?")
2156         params.append(cutoff)
2157

```

```

2158         sql = "SELECT * FROM power_events"
2159         if where_clauses:
2160             sql += " WHERE " + " AND ".join(where_clauses)
2161         sql += " ORDER BY timestamp DESC, id DESC LIMIT ? OFFSET ?"
2162         params.extend([max(1, limit), max(0, offset)])
2163
2164         with self._cm.lock, self._cm.get_connection() as conn:
2165             cursor = conn.cursor()
2166             cursor.execute(sql, params)
2167             rows = cursor.fetchall()
2168             results: List[Dict[str, Any]] = []
2169             for row in rows:
2170                 item = dict(row)
2171                 if item.get('details_json'):
2172                     try:
2173                         item['details'] = json.loads(item['details_json'])
2174                     except Exception:
2175                         item['details'] = {}
2176                 else:
2177                     item['details'] = {}
2178                 item['unexpected'] = bool(item.get('unexpected'))
2179                 results.append(item)
2180             return results
2181
2182     def get_power_summary(self) -> Dict[str, Any]:
2183         """Получить сводную статистику по сессиям и событиям питания из SQLite.
2184
2185         Returns:
2186             Dict[str, Any]: Сводная статистика (аптайм, число сессий, сбои, последнее выключение).
2187         """
2188         with self._cm.lock, self._cm.get_connection() as conn:
2189             cursor = conn.cursor()
2190             cursor.execute("SELECT COUNT(*) FROM power_session

```

```

s")
2191         total_sessions = cursor.fetchone()[0]
2192
2193         cursor.execute("SELECT COUNT(*) FROM power_session
s WHERE clean_shutdown = 1 AND unexpected_shutdown = 0")
2194         clean_count = cursor.fetchone()[0]
2195
2196         cursor.execute("SELECT COUNT(*) FROM power_session
s WHERE unexpected_shutdown = 1")
2197         unexpected_count = cursor.fetchone()[0]
2198
2199         cursor.execute("SELECT COUNT(*) FROM power_events
WHERE event_id = 1001 OR bugcheck_code IS NOT NULL")
2200         bsod_count = cursor.fetchone()[0]
2201
2202         cursor.execute("SELECT * FROM power_sessions WHERE
shutdown_time IS NOT NULL ORDER BY boot_time DESC LIMIT 1")
2203         last_shutdown_row = cursor.fetchone()
2204
2205         last_shutdown_type = last_shutdown_row['shutdown_t
ype'] if last_shutdown_row else None
2206         last_initiator = last_shutdown_row['initiator'] if
last_shutdown_row else None
2207         last_reason = last_shutdown_row['reason'] if last_
shutdown_row else None
2208
2209         # Текущая активная сессия
2210         cursor.execute("SELECT * FROM power_sessions WHERE
shutdown_time IS NULL ORDER BY boot_time DESC LIMIT 1")
2211         active_row = cursor.fetchone()
2212
2213         cur_boot = active_row['boot_time'] if active_row e
lse ''
2214         cur_uptime_sec = active_row['uptime_seconds'] if a
ctive_row else 0.0
2215         cur_uptime_human = active_row['uptime_human'] if a
ctive_row else ''
2216

```

```

2217         return {
2218             'total_sessions_count': total_sessions,
2219             'clean_shutdowns_count': clean_count,
2220             'unexpected_shutdowns_count': unexpected_count
2221         },
2222         {
2223             'bsod_count': bsod_count,
2224             'current_boot_time': cur_boot,
2225             'current_uptime_seconds': cur_uptime_sec,
2226             'current_uptime_human': cur_uptime_human,
2227             'last_shutdown_type': last_shutdown_type,
2228             'last_initiator': last_initiator,
2229             'last_reason': last_reason,
2230         }
2231
2232     def get_process_pid_snapshot(self, pid: int) -> Optional[Dict[str, Any]]:
2233         """Извлекает последний канонический снимок метрик процесса по PID (< 5 мс)."""
2234         with self._cm.lock, self._cm.get_connection() as conn:
2235             cursor = conn.cursor()
2236             cursor.execute(
2237                 'SELECT * FROM process_pid_snapshots WHERE pid = ? ORDER BY snapshot_id DESC LIMIT 1',
2238                 (pid,)
2239             )
2240             row = cursor.fetchone()
2241             return dict(row) if row else None
2242
2243     def get_process_file_events(
2244         self,
2245         pid: Optional[int] = None,
2246         directory: Optional[str] = None,
2247         limit: int = 50
2248     ) -> List[Dict[str, Any]]:
2249         """Извлекает события файловой активности с фильтрацией по PID и/или директории."""
2250         with self._cm.lock, self._cm.get_connection() as conn:
2251             cursor = conn.cursor()

```



```

2250         conditions = []
2251         params: List[Any] = []
2252         if pid is not None:
2253             conditions.append('pid = ?')
2254             params.append(pid)
2255         if directory:
2256             conditions.append('target_directory LIKE ?')
2257             params.append(f'%{directory}%')
2258
2259         where_clause = f"WHERE {' AND ' .join(conditions)}"
2260         if conditions else ''
2261         query = f"SELECT * FROM process_file_events {where_clause} ORDER BY event_id DESC LIMIT ?"
2262         params.append(limit)
2263
2264         cursor.execute(query, tuple(params))
2265         return [dict(row) for row in cursor.fetchall()]
2266
2267     def get_telemetry_time_ranges(self) -> Dict[str, Any]:
2268         """Определяет временные границы данных в таблице system_snapshots и формирует список доступных интервалов.
2269
2270         Returns:
2271             Dict[str, Any]: Метаданные диапазона времени и доступные интервалы (секунды, минуты, часы, дни, недели, месяцы, все).
2272
2273         """
2274         import time
2275         with self._cm.lock, self._cm.get_connection() as conn:
2276             cursor = conn.cursor()
2277             cursor.execute(
2278                 'SELECT MIN(created_at) as min_ts, MAX(created_at) as max_ts, COUNT(*) as cnt '
2279                 'FROM system_snapshots;'
2280             )
2281             row = cursor.fetchone()
2282
2283             min_ts = row['min_ts'] if row and row['min_ts'] is

```

```
not None else None
2282         max_ts = row['max_ts'] if row and row['max_ts'] is
not None else None
2283         total_cnt = int(row['cnt']) if row and row['cnt']
is not None else 0
2284
2285         now = time.time()
2286         span_seconds = 0.0
2287         if min_ts is not None and max_ts is not None:
2288             span_seconds = max(0.0, float(max_ts - min_ts)
, float(now - min_ts))
2289
2290         # Полное определение временных интервалов
2291         intervals_definition = [
2292             {'id': 'seconds', 'label': 'Секунды', 'seconds
': 120, 'min_span': 0},
2293             {'id': 'minutes', 'label': 'Минуты', 'seconds'
: 3600, 'min_span': 60},
2294             {'id': 'hours', 'label': 'Часы', 'seconds': 86
400, 'min_span': 3600},
2295             {'id': 'days', 'label': 'Дни', 'seconds': 8640
0 * 7, 'min_span': 86400},
2296             {'id': 'weeks', 'label': 'Недели', 'seconds':
86400 * 30, 'min_span': 86400 * 7},
2297             {'id': 'months', 'label': 'Месяцы', 'seconds':
86400 * 365, 'min_span': 86400 * 30},
2298             {'id': 'all', 'label': 'Все', 'seconds': None,
'min_span': 0},
2299         ]
2300
2301         available_intervals = []
2302         for item in intervals_definition:
2303             is_avail = (span_seconds >= item['min_span'])
if item['min_span'] > 0 else True
2304             if item['id'] == 'all':
2305                 is_avail = True
2306
2307             if is_avail:
```

```

2308         available_intervals.append({
2309             'id': item['id'],
2310             'label': item['label'],
2311             'seconds': item['seconds'],
2312             'is_available': True,
2313         })
2314
2315     return {
2316         'status': 'ok',
2317         'min_created_at': min_ts,
2318         'max_created_at': max_ts,
2319         'span_seconds': round(span_seconds, 1),
2320         'total_snapshots': total_cnt,
2321         'intervals': available_intervals,
2322         'available_ids': [i['id'] for i in available_i
2323     intervals],
2324     }
2325
2326     def get_history_by_interval(
2327         self,
2328         interval: str = 'seconds',
2329         metric: str = 'all',
2330         limit: int = 120
2331     ) -> List[Dict[str, Any]]:
2332         """Выбирает исторические точки системных снимков под у
2333         казанный интервал с downsampling.
2334
2335         Args:
2336             interval: Временной интервал ('seconds', 'minutes'
2337             , 'hours', 'days', 'weeks', 'months', 'all').
2338             metric: Категория метрики ('all', 'cpu', 'gpu', 'r
2339             am', 'net', 'storage').
2340             limit: Максимальное количество возвращаемых точек.
2341
2342         Returns:
2343             List[Dict[str, Any]]: Хронологический список точек
2344             телеметрии от старых к новым.
2345         """

```

```

2341 |         import time
2342 |         with self._cm.lock, self._cm.get_connection() as conn:
2343 |             cursor = conn.cursor()
2344 |             now = time.time()
2345 |             interval_lower = (interval or 'seconds').lower().s
trip()
2346 |
2347 |             since_ts = None
2348 |             if interval_lower == 'seconds':
2349 |                 since_ts = now - 120
2350 |             elif interval_lower == 'minutes':
2351 |                 since_ts = now - 3600
2352 |             elif interval_lower == 'hours':
2353 |                 since_ts = now - 86400
2354 |             elif interval_lower == 'days':
2355 |                 since_ts = now - (86400 * 7)
2356 |             elif interval_lower == 'weeks':
2357 |                 since_ts = now - (86400 * 30)
2358 |             elif interval_lower == 'months':
2359 |                 since_ts = now - (86400 * 365)
2360 |             elif interval_lower == 'all':
2361 |                 since_ts = None
2362 |
2363 |             if since_ts is not None:
2364 |                 cursor.execute(
2365 |                     'SELECT id, timestamp, created_at, cpu_tot
al_percent, cpu_frequency_mhz, '
2366 |                     'memory_total_gb, memory_used_gb, memory_p
ercent, swap_percent, '
2367 |                     'gpu_load_percent, gpu_temp_c, disk_read_b
ytes_sec, disk_write_bytes_sec, '
2368 |                     'network_sent_bytes_sec, network_recv_byte
s_sec '
2369 |                     'FROM system_snapshots WHERE created_at >=
? ORDER BY id ASC;',
2370 |                     (since_ts,)
2371 |                 )
2372 |             else:

```

```

2373         cursor.execute(
2374             'SELECT id, timestamp, created_at, cpu_tot
al_percent, cpu_frequency_mhz, '
2375             'memory_total_gb, memory_used_gb, memory_p
ercent, swap_percent, '
2376             'gpu_load_percent, gpu_temp_c, disk_read_b
ytes_sec, disk_write_bytes_sec, '
2377             'network_sent_bytes_sec, network_recv_byte
s_sec '
2378             'FROM system_snapshots ORDER BY id ASC;'
2379         )
2380
2381         rows = [dict(r) for r in cursor.fetchall()]
2382
2383         if not rows:
2384             cursor.execute(
2385                 'SELECT id, timestamp, created_at, cpu_tot
al_percent, cpu_frequency_mhz, '
2386                 'memory_total_gb, memory_used_gb, memory_p
ercent, swap_percent, '
2387                 'gpu_load_percent, gpu_temp_c, disk_read_b
ytes_sec, disk_write_bytes_sec, '
2388                 'network_sent_bytes_sec, network_recv_byte
s_sec '
2389                 'FROM system_snapshots ORDER BY id DESC LI
MIT ?;',
2390                 (limit,)
2391             )
2392             rows = [dict(r) for r in cursor.fetchall()]
2393             rows.reverse()
2394
2395             if len(rows) > limit:
2396                 step = len(rows) / float(limit)
2397                 downsampled = [rows[int(i * step)] for i in ra
nge(limit)]
2398                 if rows[-1] not in downsampled:
2399                     downsampled[-1] = rows[-1]
2400                 rows = downsampled

```

2401 |
2402 |
2403 |
2404 |
2405 |
2406 |
2407 |
2408 |
2409 |

return rows

schema.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry Sqlite -
  Schema
4 # =====
5 # Description:
6 #   Схема таблиц, индексов и миграций SQLite базы данных систе
  мной телеметрии.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.sqlite.schema import init_da
  tabase_schema
11 #
12 #       init_database_schema(connection)
13 #
14 # File: schema.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry.sqlite
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-10 05:49:00
20 # =====
21
22 from __future__ import annotations
23
24 """Схема таблиц, индексов и процедур миграции базы данных теле
  метрии SQLite."""
25
26 import sqlite3
27 from typing import Set
28
29 try:
```

```
30     from logger import logger
31 except ImportError:
32     import logging
33     logger = logging.getLogger(__name__)
34
35
36 def init_database_schema(conn: sqlite3.Connection) -> None:
37     """Инициализирует таблицы, индексы и выполняет миграции дл
    я базы данных телеметрии.
38
39     Args:
40         conn: Активное подключение к базе данных SQLite.
41     """
42     cursor = conn.cursor()
43
44     # 1. Системные снимки (основная таблица метрик)
45     cursor.execute('''
46         CREATE TABLE IF NOT EXISTS system_snapshots (
47             id INTEGER PRIMARY KEY AUTOINCREMENT,
48             timestamp TEXT NOT NULL,
49             created_at REAL NOT NULL,
50             hostname TEXT,
51             username TEXT,
52             uptime_seconds REAL,
53             os_name TEXT,
54             os_build TEXT,
55             os_install_date TEXT,
56             system_language TEXT,
57             os_install_language TEXT,
58             user_locale TEXT,
59             system_locale TEXT,
60             timezone TEXT,
61             codepage TEXT,
62             input_languages_json TEXT,
63             disks_json TEXT,
64             physical_disks_json TEXT,
65             monitors_json TEXT,
66             updates_json TEXT,
```



```

67         office_json TEXT,
68         onedrive_json TEXT,
69         battery_json TEXT,
70         ram_sticks_json TEXT,
71         listening_ports_json TEXT,
72         alerts_json TEXT,
73         cpu_total_percent REAL,
74         cpu_frequency_mhz REAL,
75         memory_total_gb REAL,
76         memory_used_gb REAL,
77         memory_percent REAL,
78         swap_percent REAL,
79         gpu_load_percent REAL,
80         gpu_temp_c REAL,
81         disk_read_bytes_sec REAL,
82         disk_write_bytes_sec REAL,
83         disk_read_count_sec REAL,
84         disk_write_count_sec REAL,
85         network_sent_bytes_sec REAL,
86         network_recv_bytes_sec REAL,
87         hardware_audit TEXT,
88         raw_json TEXT
89     );
90     ''')
91
92     # 2. Сенсоры (аппаратные датчики)
93     cursor.execute('''
94         CREATE TABLE IF NOT EXISTS sensor_polls (
95             id INTEGER PRIMARY KEY AUTOINCREMENT,
96             sensor_id TEXT NOT NULL,
97             timestamp TEXT NOT NULL,
98             created_at REAL NOT NULL,
99             hardware_name TEXT,
100             hardware_type TEXT,
101             sensor_category TEXT,
102             sensor_name TEXT,
103             unit TEXT,
104             value REAL,

```

```

105         provider TEXT,
106         provider_priority INTEGER DEFAULT 30,
107         raw_json TEXT,
108         UNIQUE(sensor_id, timestamp)
109     );
110     ''')
111
112 # 3. Снимки процессов
113 cursor.execute('''
114     CREATE TABLE IF NOT EXISTS process_snapshots (
115         id INTEGER PRIMARY KEY AUTOINCREMENT,
116         snapshot_id INTEGER NOT NULL,
117         timestamp TEXT NOT NULL,
118         pid INTEGER NOT NULL,
119         name TEXT NOT NULL,
120         status TEXT,
121         cpu_percent REAL,
122         memory_mb REAL,
123         memory_percent REAL,
124         num_threads INTEGER,
125         num_handles INTEGER DEFAULT 0,
126         username TEXT,
127         read_bytes_sec REAL,
128         write_bytes_sec REAL,
129         integrity_level TEXT,
130         elevation INTEGER DEFAULT 0,
131         ppid INTEGER,
132         parent_name TEXT,
133         executable TEXT,
134         cmdline TEXT,
135         sid TEXT,
136         session_id INTEGER,
137         creation_time TEXT,
138         process_guid TEXT,
139         ancestor_chain TEXT,
140         launch_reason TEXT,
141         FOREIGN KEY (snapshot_id) REFERENCES system_snapsh
ots(id) ON DELETE CASCADE

```

```

142         );
143     '')
144
145     # 4. Происхождение и жизненный цикл процессов
146     cursor.execute('''
147         CREATE TABLE IF NOT EXISTS process_provenance_events (
148             id INTEGER PRIMARY KEY AUTOINCREMENT,
149             event_id TEXT NOT NULL,
150             timestamp TEXT NOT NULL,
151             created_at REAL NOT NULL,
152             event_type TEXT NOT NULL,
153             pid INTEGER NOT NULL,
154             ppid INTEGER,
155             process_guid TEXT NOT NULL,
156             parent_guid TEXT,
157             name TEXT NOT NULL,
158             executable_path TEXT,
159             command_line TEXT,
160             user TEXT,
161             sid TEXT,
162             session_id INTEGER,
163             integrity_level TEXT,
164             elevation INTEGER,
165             parent_name TEXT,
166             parent_cmdline TEXT,
167             ancestor_chain TEXT,
168             launch_reason TEXT,
169             source TEXT DEFAULT 'system',
170             details_json TEXT,
171             raw_json TEXT
172         );
173     '')
174
175     # 5. Почасовые агрегаты сенсоров
176     cursor.execute('''
177         CREATE TABLE IF NOT EXISTS sensor_aggregates_hourly (
178             id INTEGER PRIMARY KEY AUTOINCREMENT,
179             sensor_id TEXT NOT NULL,

```

```

180         period_start TEXT NOT NULL,
181         period_end TEXT NOT NULL,
182         avg_value REAL,
183         min_value REAL,
184         max_value REAL,
185         count INTEGER,
186         raw_json TEXT
187     );
188 '''
189
190 # 6. Двухминутные агрегаты процессов
191 cursor.execute('''
192     CREATE TABLE IF NOT EXISTS process_rollups_2min (
193         id INTEGER PRIMARY KEY AUTOINCREMENT,
194         period_start TEXT NOT NULL,
195         period_end TEXT NOT NULL,
196         period_start_epoch REAL,
197         period_end_epoch REAL,
198         name TEXT NOT NULL,
199         pid INTEGER,
200         username TEXT,
201         sample_count INTEGER NOT NULL,
202         avg_cpu_percent REAL NOT NULL,
203         max_cpu_percent REAL NOT NULL,
204         min_cpu_percent REAL,
205         avg_memory_mb REAL NOT NULL,
206         max_memory_mb REAL NOT NULL,
207         min_memory_mb REAL,
208         avg_num_threads REAL,
209         outliers_count INTEGER DEFAULT 0
210     );
211 '''
212
213 # 7. Суточные агрегаты процессов
214 cursor.execute('''
215     CREATE TABLE IF NOT EXISTS process_rollups_daily (
216         id INTEGER PRIMARY KEY AUTOINCREMENT,
217         date TEXT NOT NULL,

```

```

218         name TEXT NOT NULL,
219         sample_count INTEGER NOT NULL,
220         avg_cpu_percent REAL NOT NULL,
221         max_cpu_percent REAL NOT NULL,
222         avg_memory_mb REAL NOT NULL,
223         max_memory_mb REAL NOT NULL,
224         outliers_count INTEGER DEFAULT 0,
225         last_seen TEXT
226     );
227 '''
228
229 # 8. Бакеты сжатия временных рядов телеметрии
230 cursor.execute('''
231     CREATE TABLE IF NOT EXISTS telemetry_rollups (
232         id INTEGER PRIMARY KEY AUTOINCREMENT,
233         period_start TEXT NOT NULL,
234         period_end TEXT NOT NULL,
235         period_start_epoch REAL NOT NULL,
236         period_end_epoch REAL NOT NULL,
237         duration_seconds REAL NOT NULL,
238         sample_count INTEGER NOT NULL,
239         tier TEXT NOT NULL,
240         cpu_avg REAL,
241         cpu_min REAL,
242         cpu_max REAL,
243         cpu_p95 REAL,
244         ram_avg_gb REAL,
245         ram_min_gb REAL,
246         ram_max_gb REAL,
247         ram_percent_avg REAL,
248         ram_percent_max REAL,
249         disk_read_avg_mbs REAL,
250         disk_read_max_mbs REAL,
251         disk_read_total_mb REAL,
252         disk_write_avg_mbs REAL,
253         disk_write_max_mbs REAL,
254         disk_write_total_mb REAL,
255         network_rx_avg_mbs REAL,

```

```

256         network_rx_max_mbs REAL,
257         network_rx_total_mb REAL,
258         network_tx_avg_mbs REAL,
259         network_tx_max_mbs REAL,
260         network_tx_total_mb REAL,
261         raw_json TEXT
262     );
263 ''')
264
265 # 9. Аудит оборудования
266 cursor.execute('''
267     CREATE TABLE IF NOT EXISTS hardware_audits (
268         id INTEGER PRIMARY KEY AUTOINCREMENT,
269         archive_id TEXT UNIQUE NOT NULL,
270         timestamp TEXT NOT NULL,
271         created_at REAL NOT NULL,
272         devices_count INTEGER DEFAULT 0,
273         problem_devices_count INTEGER DEFAULT 0,
274         outdated_drivers_count INTEGER DEFAULT 0,
275         changes_count INTEGER DEFAULT 0,
276         raw_json TEXT
277     );
278 ''')
279
280 # 10. История перезагрузок
281 cursor.execute('''
282     CREATE TABLE IF NOT EXISTS reboot_history (
283         id INTEGER PRIMARY KEY AUTOINCREMENT,
284         boot_id TEXT UNIQUE NOT NULL,
285         boot_time TEXT NOT NULL,
286         previous_boot_time TEXT,
287         uptime_seconds REAL,
288         uptime_human TEXT,
289         shutdown_type TEXT NOT NULL,
290         likely_class TEXT,
291         conclusion TEXT,
292         initiating_process TEXT,
293         initiating_user TEXT,

```

```

294         shutdown_action TEXT,
295         reason_text TEXT,
296         reason_code INTEGER,
297         bugcheck_code TEXT,
298         bugcheck_params_json TEXT,
299         power_button_timestamp TEXT,
300         windows_update_kb TEXT,
301         service_installed TEXT,
302         evidence_json TEXT,
303         events_chain_json TEXT,
304         raw_json TEXT,
305         created_at REAL NOT NULL
306     );
307 '''
308
309 # 11. Инциденты аномалий
310 cursor.execute('''
311     CREATE TABLE IF NOT EXISTS incidents (
312         id INTEGER PRIMARY KEY AUTOINCREMENT,
313         incident_id TEXT UNIQUE NOT NULL,
314         timestamp TEXT NOT NULL,
315         created_at REAL NOT NULL,
316         trigger_type TEXT NOT NULL,
317         severity TEXT DEFAULT 'warning',
318         title TEXT NOT NULL,
319         description TEXT,
320         trigger_metrics_json TEXT,
321         suspect_processes_json TEXT,
322         related_events_json TEXT,
323         metrics_summary_json TEXT,
324         raw_window_json TEXT
325     );
326 '''
327
328 # 12. События телеметрии
329 cursor.execute('''
330     CREATE TABLE IF NOT EXISTS telemetry_events (
331         id INTEGER PRIMARY KEY AUTOINCREMENT,

```

```

332         timestamp TEXT NOT NULL,
333         created_at REAL NOT NULL,
334         event_type TEXT NOT NULL,
335         severity TEXT DEFAULT 'info',
336         event_details TEXT,
337         raw_json TEXT
338     );
339     '''
340
341     # 13. Опросы приложений
342     cursor.execute('''
343         CREATE TABLE IF NOT EXISTS app_polls (
344             id INTEGER PRIMARY KEY AUTOINCREMENT,
345             timestamp TEXT NOT NULL,
346             created_at REAL NOT NULL,
347             app TEXT NOT NULL,
348             poll_type TEXT,
349             metric_name TEXT,
350             value REAL,
351             unit TEXT,
352             status TEXT,
353             details TEXT,
354             raw_json TEXT
355         );
356     ''')
357
358     # 14. События приложений
359     cursor.execute('''
360         CREATE TABLE IF NOT EXISTS app_events (
361             id INTEGER PRIMARY KEY AUTOINCREMENT,
362             timestamp TEXT NOT NULL,
363             created_at REAL NOT NULL,
364             app TEXT NOT NULL,
365             event_type TEXT NOT NULL,
366             status TEXT,
367             message TEXT,
368             details TEXT,
369             raw_json TEXT

```



```

370         );
371     '')
372
373     # 15. Изменения параметров приложений
374     cursor.execute('''
375         CREATE TABLE IF NOT EXISTS app_param_changes (
376             id INTEGER PRIMARY KEY AUTOINCREMENT,
377             timestamp TEXT NOT NULL,
378             created_at REAL NOT NULL,
379             app TEXT NOT NULL,
380             param_name TEXT NOT NULL,
381             old_value TEXT,
382             new_value TEXT,
383             status TEXT,
384             user TEXT,
385             details TEXT,
386             raw_json TEXT
387         );
388     '')
389
390     # 16. Пользовательские записи
391     cursor.execute('''
392         CREATE TABLE IF NOT EXISTS custom_records (
393             id INTEGER PRIMARY KEY AUTOINCREMENT,
394             timestamp TEXT NOT NULL,
395             created_at REAL NOT NULL,
396             source_file TEXT,
397             payload_json TEXT
398         );
399     '')
400
401     # 17. События устройств (PnP / Device Events)
402     cursor.execute('''
403         CREATE TABLE IF NOT EXISTS device_events (
404             id INTEGER PRIMARY KEY AUTOINCREMENT,
405             timestamp TEXT NOT NULL,
406             created_at REAL NOT NULL,
407             event_type TEXT NOT NULL,

```

```

408         device_instance_id TEXT,
409         friendly_name TEXT,
410         device_class TEXT,
411         category TEXT,
412         has_problem INTEGER DEFAULT 0,
413         problem_code INTEGER DEFAULT 0,
414         status_code INTEGER DEFAULT 0,
415         manufacturer TEXT,
416         flapping_count INTEGER DEFAULT 0,
417         uptime_seconds REAL DEFAULT 0.0,
418         raw_json TEXT
419     );
420 '''
421
422 # 18. Инвентарь устройств
423 cursor.execute('''
424     CREATE TABLE IF NOT EXISTS device_inventory (
425         id INTEGER PRIMARY KEY AUTOINCREMENT,
426         device_instance_id TEXT UNIQUE NOT NULL,
427         friendly_name TEXT,
428         device_class TEXT,
429         install_date TEXT NOT NULL,
430         created_at REAL NOT NULL,
431         updated_at REAL NOT NULL
432     );
433 '''
434
435 # 19. Системные события W64/ETW
436 cursor.execute('''
437     CREATE TABLE IF NOT EXISTS w64_events (
438         id INTEGER PRIMARY KEY AUTOINCREMENT,
439         event_id INTEGER,
440         timestamp TEXT NOT NULL,
441         created_at REAL NOT NULL,
442         event_type TEXT,
443         path TEXT,
444         pid INTEGER,
445         name TEXT,

```

```
446         provider TEXT,
447         raw_json TEXT
448     );
449     '''
450
451 # 20. Аномальные процессы
452 cursor.execute('''
453     CREATE TABLE IF NOT EXISTS process_outliers (
454         id INTEGER PRIMARY KEY AUTOINCREMENT,
455         snapshot_id INTEGER,
456         timestamp TEXT NOT NULL,
457         created_at REAL NOT NULL,
458         pid INTEGER,
459         name TEXT,
460         status TEXT,
461         cpu_percent REAL,
462         memory_mb REAL,
463         memory_percent REAL,
464         num_threads INTEGER,
465         username TEXT,
466         details TEXT
467     );
468     ''')
469
470 # 21. Расширенный системный аудит
471 cursor.execute('''
472     CREATE TABLE IF NOT EXISTS system_extended_audits (
473         id INTEGER PRIMARY KEY AUTOINCREMENT,
474         timestamp TEXT NOT NULL,
475         created_at REAL NOT NULL,
476         hostname TEXT,
477         defender_cfa_enabled INTEGER DEFAULT 0,
478         defender_asr_count INTEGER DEFAULT 0,
479         defender_exclusions_count INTEGER DEFAULT 0,
480         defender_threats_count INTEGER DEFAULT 0,
481         startup_entries_count INTEGER DEFAULT 0,
482         vss_snapshots_count INTEGER DEFAULT 0,
483         users_total_count INTEGER DEFAULT 0,
```

```

484         users_admin_count INTEGER DEFAULT 0,
485         raw_json TEXT
486     );
487     '')
488
489 # 22. Почасовые агрегаты сенсоров (telemetry_hourly)
490 cursor.execute('''
491     CREATE TABLE IF NOT EXISTS telemetry_hourly (
492         sensor_id TEXT NOT NULL,
493         bucket_start INTEGER NOT NULL,
494         bucket_end INTEGER NOT NULL,
495         sample_count INTEGER NOT NULL,
496         value_min REAL,
497         value_max REAL,
498         value_avg REAL,
499         value_sum REAL,
500         value_sum_sq REAL,
501         value_stddev REAL,
502         value_first REAL,
503         value_last REAL,
504         PRIMARY KEY (sensor_id, bucket_start)
505     );
506     '')
507
508 # 23. Суточные агрегаты сенсоров (telemetry_daily)
509 cursor.execute('''
510     CREATE TABLE IF NOT EXISTS telemetry_daily (
511         sensor_id TEXT NOT NULL,
512         bucket_start INTEGER NOT NULL,
513         bucket_end INTEGER NOT NULL,
514         sample_count INTEGER NOT NULL,
515         value_min REAL,
516         value_max REAL,
517         value_avg REAL,
518         value_sum REAL,
519         value_sum_sq REAL,
520         value_stddev REAL,
521         value_first REAL,

```

```

522         value_last REAL,
523         PRIMARY KEY (sensor_id, bucket_start)
524     );
525     '')
526
527 # 24. Недельные агрегаты сенсоров (telemetry_weekly)
528 cursor.execute('''
529     CREATE TABLE IF NOT EXISTS telemetry_weekly (
530         sensor_id TEXT NOT NULL,
531         bucket_start INTEGER NOT NULL,
532         bucket_end INTEGER NOT NULL,
533         sample_count INTEGER NOT NULL,
534         value_min REAL,
535         value_max REAL,
536         value_avg REAL,
537         value_sum REAL,
538         value_sum_sq REAL,
539         value_stddev REAL,
540         value_first REAL,
541         value_last REAL,
542         PRIMARY KEY (sensor_id, bucket_start)
543     );
544     ''')
545
546 # 25. Месячные агрегаты сенсоров (telemetry_monthly)
547 cursor.execute('''
548     CREATE TABLE IF NOT EXISTS telemetry_monthly (
549         sensor_id TEXT NOT NULL,
550         bucket_start INTEGER NOT NULL,
551         bucket_end INTEGER NOT NULL,
552         sample_count INTEGER NOT NULL,
553         value_min REAL,
554         value_max REAL,
555         value_avg REAL,
556         value_sum REAL,
557         value_sum_sq REAL,
558         value_stddev REAL,
559         value_first REAL,

```

```

560         value_last REAL,
561         PRIMARY KEY (sensor_id, bucket_start)
562     );
563     '')
564
565 # 26. Годовые агрегаты сенсоров (telemetry_yearly)
566 cursor.execute('''
567     CREATE TABLE IF NOT EXISTS telemetry_yearly (
568         sensor_id TEXT NOT NULL,
569         bucket_start INTEGER NOT NULL,
570         bucket_end INTEGER NOT NULL,
571         sample_count INTEGER NOT NULL,
572         value_min REAL,
573         value_max REAL,
574         value_avg REAL,
575         value_sum REAL,
576         value_sum_sq REAL,
577         value_stddev REAL,
578         value_first REAL,
579         value_last REAL,
580         PRIMARY KEY (sensor_id, bucket_start)
581     );
582     '')
583
584 # 27. Фиксация аномальных всплесков сенсоров (telemetry_sp
ikes)
585 cursor.execute('''
586     CREATE TABLE IF NOT EXISTS telemetry_spikes (
587         id INTEGER PRIMARY KEY AUTOINCREMENT,
588         sensor_id TEXT NOT NULL,
589         timestamp TEXT NOT NULL,
590         created_at REAL NOT NULL,
591         value REAL NOT NULL,
592         spike_type TEXT DEFAULT 'high',
593         baseline_avg REAL,
594         baseline_stddev REAL,
595         details TEXT,
596         raw_json TEXT

```

```

597         );
598     '')
599
600     # 28. События прямого ввода-вывода (disk_io_events / ETW /
        Process I/O)
601     cursor.execute('''
602         CREATE TABLE IF NOT EXISTS disk_io_events (
603             id INTEGER PRIMARY KEY AUTOINCREMENT,
604             timestamp TEXT NOT NULL,
605             created_at REAL NOT NULL,
606             pid INTEGER NOT NULL,
607             process_name TEXT NOT NULL,
608             disk_id INTEGER NOT NULL,
609             operation TEXT NOT NULL,
610             bytes_count INTEGER NOT NULL,
611             duration_ms REAL,
612             offset INTEGER,
613             file_path TEXT
614         );
615     '')
616
617     # 29. Снимки здоровья и износа накопителей (disk_health_sn
        apshots / NVMe SMART / TBW)
618     cursor.execute('''
619         CREATE TABLE IF NOT EXISTS disk_health_snapshots (
620             id INTEGER PRIMARY KEY AUTOINCREMENT,
621             timestamp TEXT NOT NULL,
622             created_at REAL NOT NULL,
623             disk_id INTEGER NOT NULL,
624             serial_number TEXT,
625             model TEXT,
626             bus_type TEXT,
627             temperature_c REAL,
628             wear_percent REAL,
629             available_spare REAL,
630             tbw_written_tb REAL,
631             tbw_read_tb REAL,
632             power_on_hours INTEGER,

```

```

633         power_cycles INTEGER,
634         unsafe_shutdowns INTEGER,
635         media_errors INTEGER,
636         error_log_entries INTEGER,
637         health_status TEXT DEFAULT 'Healthy',
638         raw_smart_json TEXT
639     );
640     ''')
641
642     # 30. Паспорт и конфигурация физических накопителей (disk_
inventory)
643     cursor.execute('''
644         CREATE TABLE IF NOT EXISTS disk_inventory (
645             id INTEGER PRIMARY KEY AUTOINCREMENT,
646             disk_id INTEGER NOT NULL,
647             device_path TEXT NOT NULL,
648             serial_number TEXT UNIQUE,
649             vendor TEXT,
650             product TEXT,
651             revision TEXT,
652             bus_type TEXT,
653             media_type TEXT,
654             size_bytes INTEGER,
655             size_gb REAL,
656             sector_size INTEGER,
657             removable INTEGER DEFAULT 0,
658             partition_style TEXT,
659             gpt_guid TEXT,
660             partitions_json TEXT,
661             first_seen TEXT NOT NULL,
662             last_seen TEXT NOT NULL,
663             updated_at REAL NOT NULL
664         );
665     ''')
666
667     # 31. Паспорт логических томов (volume_inventory)
668     cursor.execute('''
669         CREATE TABLE IF NOT EXISTS volume_inventory (

```



```

670         id INTEGER PRIMARY KEY AUTOINCREMENT,
671         volume_guid TEXT UNIQUE NOT NULL,
672         drive_letter TEXT,
673         mount_point TEXT,
674         label TEXT,
675         filesystem TEXT,
676         filesystem_flags INTEGER,
677         cluster_size_bytes INTEGER,
678         sector_size_bytes INTEGER,
679         total_bytes INTEGER,
680         total_gb REAL,
681         disk_id INTEGER,
682         first_seen TEXT NOT NULL,
683         last_seen TEXT NOT NULL,
684         updated_at REAL NOT NULL
685     );
686 '''
687
688 # 32. Сэмплы производительности дисков (disk_performance_s
amples)
689 cursor.execute('''
690     CREATE TABLE IF NOT EXISTS disk_performance_samples (
691         id INTEGER PRIMARY KEY AUTOINCREMENT,
692         timestamp TEXT NOT NULL,
693         created_at REAL NOT NULL,
694         disk_name TEXT NOT NULL,
695         read_bytes_sec REAL,
696         write_bytes_sec REAL,
697         read_iops REAL,
698         write_iops REAL,
699         read_time_ms REAL,
700         write_time_ms REAL,
701         percent_disk_time REAL,
702         queue_length REAL
703     );
704 '''
705
706 # 33. Паспорт процессоров (cpu_inventory)

```

```

707 cursor.execute('''
708     CREATE TABLE IF NOT EXISTS cpu_inventory (
709         id INTEGER PRIMARY KEY AUTOINCREMENT,
710         processor_id INTEGER NOT NULL,
711         name TEXT UNIQUE NOT NULL,
712         vendor TEXT,
713         architecture TEXT,
714         physical_cores INTEGER,
715         logical_cores INTEGER,
716         base_frequency_mhz REAL,
717         max_frequency_mhz REAL,
718         l2_cache_kb INTEGER,
719         l3_cache_kb INTEGER,
720         socket TEXT,
721         features_json TEXT,
722         first_seen TEXT NOT NULL,
723         last_seen TEXT NOT NULL,
724         updated_at REAL NOT NULL
725     );
726 ''')
727
728 # 34. Паспорт планок памяти (ram_module_inventory)
729 cursor.execute('''
730     CREATE TABLE IF NOT EXISTS ram_module_inventory (
731         id INTEGER PRIMARY KEY AUTOINCREMENT,
732         slot_id INTEGER NOT NULL,
733         bank_label TEXT NOT NULL,
734         device_locator TEXT,
735         serial_number TEXT UNIQUE,
736         part_number TEXT,
737         manufacturer TEXT,
738         capacity_bytes INTEGER,
739         capacity_gb REAL,
740         speed_mhz INTEGER,
741         memory_type TEXT,
742         form_factor TEXT,
743         configured_voltage REAL,
744         first_seen TEXT NOT NULL,

```

```

745         last_seen TEXT NOT NULL,
746         updated_at REAL NOT NULL
747     );
748     '')
749
750 # 35. Паспорт графических ускорителей (gpu_inventory)
751 cursor.execute('''
752     CREATE TABLE IF NOT EXISTS gpu_inventory (
753         id INTEGER PRIMARY KEY AUTOINCREMENT,
754         gpu_id INTEGER NOT NULL,
755         pci_device_id TEXT UNIQUE,
756         name TEXT NOT NULL,
757         vendor TEXT,
758         driver_version TEXT,
759         driver_date TEXT,
760         vram_bytes INTEGER,
761         vram_gb REAL,
762         pci_bus_id TEXT,
763         bios_version TEXT,
764         cuda_cores INTEGER,
765         directml_supported INTEGER DEFAULT 1,
766         first_seen TEXT NOT NULL,
767         last_seen TEXT NOT NULL,
768         updated_at REAL NOT NULL
769     );
770     '')
771
772 # 36. Паспорт сетевых интерфейсов (network_adapter_invento
ry)
773 cursor.execute('''
774     CREATE TABLE IF NOT EXISTS network_adapter_inventory (
775         id INTEGER PRIMARY KEY AUTOINCREMENT,
776         adapter_id INTEGER NOT NULL,
777         adapter_guid TEXT UNIQUE NOT NULL,
778         name TEXT NOT NULL,
779         interface_name TEXT,
780         mac_address TEXT,
781         adapter_type TEXT,

```

```

782         is_physical INTEGER DEFAULT 1,
783         is_wireless INTEGER DEFAULT 0,
784         max_speed_mbps INTEGER,
785         driver_name TEXT,
786         driver_version TEXT,
787         first_seen TEXT NOT NULL,
788         last_seen TEXT NOT NULL,
789         updated_at REAL NOT NULL
790     );
791 '''
792
793 # 37. Сэмплы телеметрии процессора (cpu_telemetry_samples)
794 cursor.execute('''
795     CREATE TABLE IF NOT EXISTS cpu_telemetry_samples (
796         id INTEGER PRIMARY KEY AUTOINCREMENT,
797         timestamp TEXT NOT NULL,
798         created_at REAL NOT NULL,
799         processor_id INTEGER DEFAULT 0,
800         total_percent REAL,
801         user_percent REAL,
802         kernel_percent REAL,
803         frequency_mhz REAL,
804         temperature_c REAL,
805         package_power_w REAL,
806         core_utilization_json TEXT,
807         core_temperatures_json TEXT
808     );
809 '''
810
811 # 38. Сэмплы телеметрии оперативной памяти (ram_telemetry_
812 samples)
813 cursor.execute('''
814     CREATE TABLE IF NOT EXISTS ram_telemetry_samples (
815         id INTEGER PRIMARY KEY AUTOINCREMENT,
816         timestamp TEXT NOT NULL,
817         created_at REAL NOT NULL,
818         total_bytes INTEGER,
819         total_gb REAL,

```

```

819         used_bytes INTEGER,
820         used_gb REAL,
821         available_bytes INTEGER,
822         available_gb REAL,
823         percent_used REAL,
824         swap_total_gb REAL,
825         swap_used_gb REAL,
826         swap_percent REAL,
827         pool_paged_mb REAL,
828         pool_nonpaged_mb REAL
829     );
830 '''
831
832 # 39. Сэмплы телеметрии GPU (gpu_telemetry_samples)
833 cursor.execute('''
834     CREATE TABLE IF NOT EXISTS gpu_telemetry_samples (
835         id INTEGER PRIMARY KEY AUTOINCREMENT,
836         timestamp TEXT NOT NULL,
837         created_at REAL NOT NULL,
838         gpu_id INTEGER DEFAULT 0,
839         name TEXT NOT NULL,
840         load_percent REAL,
841         memory_used_mb REAL,
842         memory_total_mb REAL,
843         memory_percent REAL,
844         temperature_gpu_c REAL,
845         temperature_memory_c REAL,
846         fan_speed_pct REAL,
847         power_draw_w REAL,
848         clock_graphics_mhz REAL,
849         clock_memory_mhz REAL
850     );
851 '''
852
853 # 40. Сэмплы сетевых интерфейсов (network_adapter_samples)
854 cursor.execute('''
855     CREATE TABLE IF NOT EXISTS network_adapter_samples (
856         id INTEGER PRIMARY KEY AUTOINCREMENT,

```

```

857         timestamp TEXT NOT NULL,
858         created_at REAL NOT NULL,
859         adapter_name TEXT NOT NULL,
860         bytes_recv_sec REAL,
861         bytes_sent_sec REAL,
862         packets_recv_sec REAL,
863         packets_sent_sec REAL,
864         errors_in_sec REAL,
865         errors_out_sec REAL,
866         link_speed_mbps INTEGER,
867         is_connected INTEGER DEFAULT 1
868     );
869     '''
870
871     # 41. Архивные снимки автозапуска Windows (startup_audit_a
archives)
872     cursor.execute('''
873         CREATE TABLE IF NOT EXISTS startup_audit_archives (
874             id INTEGER PRIMARY KEY AUTOINCREMENT,
875             archive_id TEXT UNIQUE NOT NULL,
876             timestamp TEXT NOT NULL,
877             created_at REAL NOT NULL,
878             total_entries INTEGER DEFAULT 0,
879             active_entries INTEGER DEFAULT 0,
880             disabled_entries INTEGER DEFAULT 0,
881             broken_entries INTEGER DEFAULT 0,
882             health_score INTEGER DEFAULT 100,
883             changes_count INTEGER DEFAULT 0,
884             raw_json TEXT
885         );
886     ''')
887
888     # 42. Отличия и изменения в автозапуске между снимками (st
artup_changes)
889     cursor.execute('''
890         CREATE TABLE IF NOT EXISTS startup_changes (
891             id INTEGER PRIMARY KEY AUTOINCREMENT,
892             archive_id TEXT NOT NULL,

```

```

893         timestamp TEXT NOT NULL,
894         created_at REAL NOT NULL,
895         change_type TEXT NOT NULL,
896         entry_id TEXT NOT NULL,
897         name TEXT NOT NULL,
898         description TEXT,
899         previous_value TEXT,
900         current_value TEXT
901     );
902     '''
903
904     # 43. Снимки каналов журналов событий (event_log_channel_s
napshots)
905     cursor.execute('''
906         CREATE TABLE IF NOT EXISTS event_log_channel_snapshots
907     (
908         id INTEGER PRIMARY KEY AUTOINCREMENT,
909         snapshot_id TEXT NOT NULL,
910         timestamp DATETIME NOT NULL,
911         channel_name TEXT NOT NULL,
912         display_name TEXT,
913         description TEXT,
914         record_count INTEGER DEFAULT 0,
915         size_bytes INTEGER DEFAULT 0,
916         channel_type TEXT DEFAULT 'Admin',
917         is_enabled INTEGER DEFAULT 1,
918         created_at REAL NOT NULL
919     );
920     '''
921
922     # 44. Кеш записей журналов событий (event_log_entries_cach
e)
923     cursor.execute('''
924         CREATE TABLE IF NOT EXISTS event_log_entries_cache (
925             id INTEGER PRIMARY KEY AUTOINCREMENT,
926             snapshot_id TEXT NOT NULL,
927             timestamp DATETIME NOT NULL,
928             channel TEXT NOT NULL,

```

```

928         event_id INTEGER DEFAULT 0,
929         level TEXT DEFAULT 'Information',
930         provider_name TEXT,
931         time_created TEXT,
932         message TEXT,
933         raw_json TEXT,
934         created_at REAL NOT NULL
935     );
936     '''
937
938     # 45. Скомпилированные профили Log Intelligence (event_log
939     _intelligence_profiles)
940     cursor.execute('''
941         CREATE TABLE IF NOT EXISTS event_log_intelligence_prof
942         iles (
943             id INTEGER PRIMARY KEY AUTOINCREMENT,
944             snapshot_id TEXT NOT NULL,
945             timestamp DATETIME NOT NULL,
946             channel TEXT NOT NULL,
947             total_analyzed INTEGER DEFAULT 0,
948             unique_patterns_count INTEGER DEFAULT 0,
949             critical_incidents_json TEXT,
950             top_clusters_json TEXT,
951             bursts_json TEXT,
952             decision_gate_json TEXT,
953             created_at REAL NOT NULL
954         );
955     ''')
956
957     # 46. Профили брандмауэра (firewall_profile_snapshots)
958     cursor.execute('''
959         CREATE TABLE IF NOT EXISTS firewall_profile_snapshots
960         (
961             id INTEGER PRIMARY KEY AUTOINCREMENT,
962             snapshot_id TEXT NOT NULL,
963             timestamp DATETIME NOT NULL,
964             domain_enabled INTEGER NOT NULL,
965             private_enabled INTEGER NOT NULL,

```



```

963         public_enabled INTEGER NOT NULL,
964         domain_default_inbound TEXT,
965         private_default_inbound TEXT,
966         public_default_inbound TEXT,
967         stealth_mode_enabled INTEGER DEFAULT 1,
968         created_at REAL NOT NULL
969     );
970     ''')
971
972 # 47. Правила брандмауэра (firewall_rule_snapshots)
973 cursor.execute('''
974     CREATE TABLE IF NOT EXISTS firewall_rule_snapshots (
975         id INTEGER PRIMARY KEY AUTOINCREMENT,
976         snapshot_id TEXT NOT NULL,
977         rule_name TEXT NOT NULL,
978         display_name TEXT,
979         direction TEXT NOT NULL,
980         action TEXT NOT NULL,
981         enabled INTEGER NOT NULL,
982         protocol TEXT,
983         local_port TEXT,
984         remote_port TEXT,
985         program_path TEXT,
986         profile_mask INTEGER,
987         created_at REAL NOT NULL
988     );
989     ''')
990
991 # 48. Снимки состояния служб Windows (services_snapshots)
992 cursor.execute('''
993     CREATE TABLE IF NOT EXISTS services_snapshots (
994         id INTEGER PRIMARY KEY AUTOINCREMENT,
995         snapshot_id TEXT NOT NULL,
996         timestamp DATETIME NOT NULL,
997         service_name TEXT NOT NULL,
998         display_name TEXT,
999         state TEXT NOT NULL,
1000         state_code INTEGER,

```

```

1001         start_type TEXT,
1002         pid INTEGER,
1003         binary_path TEXT,
1004         account TEXT,
1005         is_orphaned INTEGER DEFAULT 0,
1006         created_at REAL NOT NULL
1007     );
1008 ''')
1009
1010 # 49. События изменений служб (service_change_events)
1011 cursor.execute('''
1012     CREATE TABLE IF NOT EXISTS service_change_events (
1013         id INTEGER PRIMARY KEY AUTOINCREMENT,
1014         timestamp DATETIME NOT NULL,
1015         service_name TEXT NOT NULL,
1016         display_name TEXT,
1017         action TEXT NOT NULL,
1018         old_state TEXT,
1019         new_state TEXT,
1020         performed_by TEXT DEFAULT 'SYSTEM',
1021         details_json TEXT,
1022         created_at REAL NOT NULL
1023     );
1024 ''')
1025
1026 # 50. Снимки троттлинга и электропитания CPU (cpu_throttling_snapshots)
1027 cursor.execute('''
1028     CREATE TABLE IF NOT EXISTS cpu_throttling_snapshots (
1029         id INTEGER PRIMARY KEY AUTOINCREMENT,
1030         snapshot_id TEXT NOT NULL,
1031         timestamp DATETIME NOT NULL,
1032         prohot_active BOOLEAN NOT NULL DEFAULT 0,
1033         pl1_limit_watts REAL,
1034         pl2_limit_watts REAL,
1035         current_power_watts REAL,
1036         max_core_temp_c REAL,
1037         package_temp_c REAL,

```

```

1038         dpc_latency_us INTEGER DEFAULT 0,
1039         isr_latency_us INTEGER DEFAULT 0,
1040         throttling_reasons_json TEXT,
1041         created_at REAL NOT NULL
1042     );
1043 '''
1044
1045 # 51. Снимки температурных зон (thermal_zone_snapshots)
1046 cursor.execute('''
1047     CREATE TABLE IF NOT EXISTS thermal_zone_snapshots (
1048         id INTEGER PRIMARY KEY AUTOINCREMENT,
1049         snapshot_id TEXT NOT NULL,
1050         timestamp DATETIME NOT NULL,
1051         zone_name TEXT NOT NULL,
1052         temperature_c REAL NOT NULL,
1053         critical_limit_c REAL,
1054         throttling_limit_c REAL,
1055         sensor_provider TEXT NOT NULL,
1056         created_at REAL NOT NULL
1057     );
1058 '''
1059
1060 # 52. Снимки поведенческой форе́нзики (forensics_snapshots)
1061 cursor.execute('''
1062     CREATE TABLE IF NOT EXISTS forensics_snapshots (
1063         id INTEGER PRIMARY KEY AUTOINCREMENT,
1064         snapshot_id TEXT NOT NULL,
1065         timestamp DATETIME NOT NULL,
1066         foreground_window_title TEXT,
1067         foreground_process_name TEXT,
1068         foreground_pid INTEGER,
1069         user_idle_seconds INTEGER DEFAULT 0,
1070         camera_active_apps_json TEXT,
1071         microphone_active_apps_json TEXT,
1072         userassist_top_apps_json TEXT,
1073         created_at REAL NOT NULL
1074     );
1075 '''

```

```

1076
1077 # 53. Сводные снимки утечек ресурсов (process_leak_snapsho
ts)
1078 cursor.execute('''
1079     CREATE TABLE IF NOT EXISTS process_leak_snapshots (
1080         id INTEGER PRIMARY KEY AUTOINCREMENT,
1081         snapshot_id TEXT NOT NULL UNIQUE,
1082         timestamp DATETIME NOT NULL,
1083         total_processes INTEGER NOT NULL,
1084         suspicious_count INTEGER NOT NULL,
1085         created_at REAL NOT NULL
1086     );
1087 ''')
1088
1089 # 54. Детализация процессов с утечками (process_leak_items
)
1090 cursor.execute('''
1091     CREATE TABLE IF NOT EXISTS process_leak_items (
1092         id INTEGER PRIMARY KEY AUTOINCREMENT,
1093         snapshot_id TEXT NOT NULL,
1094         pid INTEGER NOT NULL,
1095         name TEXT NOT NULL,
1096         handles_count INTEGER NOT NULL,
1097         gdi_objects INTEGER NOT NULL,
1098         user_objects INTEGER NOT NULL,
1099         page_faults INTEGER NOT NULL,
1100         working_set_mb REAL NOT NULL,
1101         leak_risk_score TEXT NOT NULL,
1102         leak_risk_reasons_json TEXT,
1103         created_at REAL NOT NULL
1104     );
1105 ''')
1106
1107 # 55. Снимки защитника Defender (defender_snapshots)
1108 cursor.execute('''
1109     CREATE TABLE IF NOT EXISTS defender_snapshots (
1110         id INTEGER PRIMARY KEY AUTOINCREMENT,
1111         snapshot_id TEXT NOT NULL,

```

```

1112         timestamp DATETIME NOT NULL,
1113         real_time_protection_enabled INTEGER NOT NULL,
1114         cloud_protection_enabled INTEGER NOT NULL,
1115         behavior_monitoring_enabled INTEGER NOT NULL,
1116         tamper_protection_enabled INTEGER NOT NULL,
1117         pua_protection_enabled INTEGER NOT NULL,
1118         antivirus_enabled INTEGER NOT NULL,
1119         antispyware_enabled INTEGER NOT NULL,
1120         engine_version TEXT,
1121         av_signature_version TEXT,
1122         last_quick_scan_datetime DATETIME,
1123         last_full_scan_datetime DATETIME,
1124         security_score INTEGER DEFAULT 100,
1125         cfa_state INTEGER DEFAULT 0,
1126         raw_status_json TEXT,
1127         created_at REAL NOT NULL
1128     );
1129 '''
1130
1131 # 56. Исключения защитника (defender_exclusions)
1132 cursor.execute('''
1133     CREATE TABLE IF NOT EXISTS defender_exclusions (
1134         id INTEGER PRIMARY KEY AUTOINCREMENT,
1135         snapshot_id TEXT NOT NULL,
1136         exclusion_type TEXT NOT NULL,
1137         exclusion_value TEXT NOT NULL,
1138         risk_level TEXT DEFAULT 'SAFE',
1139         created_at REAL NOT NULL
1140     );
1141 '''
1142
1143 # 57. Правила ASR (defender_asr_rules)
1144 cursor.execute('''
1145     CREATE TABLE IF NOT EXISTS defender_asr_rules (
1146         id INTEGER PRIMARY KEY AUTOINCREMENT,
1147         snapshot_id TEXT NOT NULL,
1148         rule_guid TEXT NOT NULL,
1149         rule_name TEXT NOT NULL,

```

```

1150         rule_action TEXT NOT NULL,
1151         created_at REAL NOT NULL
1152     );
1153     '''
1154
1155     # 58. Обнаруженные угрозы (defender_threats)
1156     cursor.execute('''
1157         CREATE TABLE IF NOT EXISTS defender_threats (
1158             id INTEGER PRIMARY KEY AUTOINCREMENT,
1159             snapshot_id TEXT NOT NULL,
1160             threat_id TEXT,
1161             threat_name TEXT NOT NULL,
1162             severity TEXT NOT NULL,
1163             category TEXT,
1164             resources_json TEXT,
1165             detection_time DATETIME,
1166             created_at REAL NOT NULL
1167         );
1168     ''')
1169
1170     # 59. Снимки сетевых сокетов процессов (process_network_snapshots)
1171     cursor.execute('''
1172         CREATE TABLE IF NOT EXISTS process_network_snapshots (
1173             id INTEGER PRIMARY KEY AUTOINCREMENT,
1174             snapshot_id TEXT NOT NULL,
1175             timestamp DATETIME NOT NULL,
1176             pid INTEGER NOT NULL,
1177             process_name TEXT NOT NULL,
1178             local_address TEXT,
1179             remote_address TEXT,
1180             protocol TEXT,
1181             status TEXT,
1182             service_type TEXT,
1183             sent_kb REAL DEFAULT 0.0,
1184             recv_kb REAL DEFAULT 0.0,
1185             read_speed_kbs REAL DEFAULT 0.0,
1186             write_speed_kbs REAL DEFAULT 0.0,

```

```

1187         created_at REAL NOT NULL
1188     );
1189     '''
1190
1191     # 60. Инвентарь установленного ПО (software_inventory)
1192     cursor.execute('''
1193         CREATE TABLE IF NOT EXISTS software_inventory (
1194             app_id TEXT PRIMARY KEY,
1195             display_name TEXT NOT NULL,
1196             version TEXT,
1197             publisher TEXT,
1198             install_date TEXT,
1199             install_location TEXT,
1200             architecture TEXT,
1201             is_system_component INTEGER DEFAULT 0,
1202             first_seen DATETIME NOT NULL,
1203             last_scanned_at DATETIME NOT NULL
1204         );
1205     '''
1206
1207     # 61. Категоризированные места хранения ПО (software_storage_locations)
1208     cursor.execute('''
1209         CREATE TABLE IF NOT EXISTS software_storage_locations
1210     (
1211         id INTEGER PRIMARY KEY AUTOINCREMENT,
1212         app_id TEXT NOT NULL,
1213         category TEXT NOT NULL,
1214         path TEXT NOT NULL,
1215         size_bytes INTEGER DEFAULT 0,
1216         file_count INTEGER DEFAULT 0,
1217         last_updated DATETIME NOT NULL,
1218         FOREIGN KEY (app_id) REFERENCES software_inventory
1219         (app_id) ON DELETE CASCADE
1220     );
1221     '''
1222
1223     # 62. Файлы конфигурации ПО (software_config_files)

```

```

1222 cursor.execute('''
1223     CREATE TABLE IF NOT EXISTS software_config_files (
1224         id INTEGER PRIMARY KEY AUTOINCREMENT,
1225         app_id TEXT NOT NULL,
1226         file_path TEXT NOT NULL,
1227         format TEXT,
1228         size_bytes INTEGER DEFAULT 0,
1229         snippet TEXT,
1230         FOREIGN KEY (app_id) REFERENCES software_inventory
1231         (app_id) ON DELETE CASCADE
1232     );
1233     ''')
1234
1235 # 63. Сетевые снимки ПО (software_network_snapshots)
1236 cursor.execute('''
1237     CREATE TABLE IF NOT EXISTS software_network_snapshots
1238     (
1239         id INTEGER PRIMARY KEY AUTOINCREMENT,
1240         snapshot_id TEXT NOT NULL,
1241         timestamp DATETIME NOT NULL,
1242         app_id TEXT NOT NULL,
1243         pid INTEGER NOT NULL,
1244         local_address TEXT,
1245         remote_address TEXT,
1246         protocol TEXT,
1247         domain_name TEXT,
1248         sent_kb REAL DEFAULT 0.0,
1249         recv_kb REAL DEFAULT 0.0,
1250         FOREIGN KEY (app_id) REFERENCES software_inventory
1251         (app_id) ON DELETE CASCADE
1252     );
1253     ''')
1254
1255 # 64. Анализ ПО с помощью ИИ (software_ai_research)
1256 cursor.execute('''
1257     CREATE TABLE IF NOT EXISTS software_ai_research (
1258         app_id TEXT PRIMARY KEY,
1259         summary TEXT,

```



```

1257         config_purpose_explanation TEXT,
1258         data_storage_explanation TEXT,
1259         network_activity_explanation TEXT,
1260         confirmed_facts_json TEXT,
1261         inferred_facts_json TEXT,
1262         confidence_level REAL DEFAULT 0.0,
1263         updated_at DATETIME NOT NULL,
1264         FOREIGN KEY (app_id) REFERENCES software_inventory
    (app_id) ON DELETE CASCADE
1265     );
1266     '')
1267
1268     # 65. Нормализованные события безопасности Windows (security_events)
1269     cursor.execute('''
1270         CREATE TABLE IF NOT EXISTS security_events (
1271             id INTEGER PRIMARY KEY AUTOINCREMENT,
1272             event_record_id INTEGER DEFAULT 0,
1273             event_id INTEGER NOT NULL,
1274             timestamp TEXT NOT NULL,
1275             created_at REAL NOT NULL,
1276             computer TEXT,
1277             channel TEXT DEFAULT 'Security',
1278             level TEXT DEFAULT 'Information',
1279             subject_user TEXT,
1280             subject_domain TEXT,
1281             subject_sid TEXT,
1282             target_user TEXT,
1283             target_domain TEXT,
1284             target_sid TEXT,
1285             process_id INTEGER DEFAULT 0,
1286             process_name TEXT,
1287             parent_process_id INTEGER DEFAULT 0,
1288             parent_process_name TEXT,
1289             command_line TEXT,
1290             logon_id TEXT,
1291             logon_type INTEGER,
1292             elevated_token INTEGER,

```

```

1293         source_ip TEXT,
1294         source_port INTEGER,
1295         object_name TEXT,
1296         status_code TEXT,
1297         message TEXT,
1298         event_data_json TEXT,
1299         ingested_at TEXT NOT NULL
1300     );
1301     '''
1302
1303     # 66. Сырые события безопасности Windows (security_events_
raw)
1304     cursor.execute('''
1305         CREATE TABLE IF NOT EXISTS security_events_raw (
1306             id INTEGER PRIMARY KEY AUTOINCREMENT,
1307             event_record_id INTEGER DEFAULT 0,
1308             event_id INTEGER NOT NULL,
1309             timestamp TEXT NOT NULL,
1310             created_at REAL NOT NULL,
1311             channel TEXT DEFAULT 'Security',
1312             raw_xml TEXT NOT NULL,
1313             ingested_at TEXT NOT NULL
1314         );
1315     '''
1316
1317     # 67. Закладки инкрементального сбора событий (security_co
llector_bookmarks)
1318     cursor.execute('''
1319         CREATE TABLE IF NOT EXISTS security_collector_bookmark
s (
1320             id INTEGER PRIMARY KEY AUTOINCREMENT,
1321             channel TEXT NOT NULL UNIQUE,
1322             last_record_id INTEGER DEFAULT 0,
1323             bookmark_xml TEXT,
1324             last_timestamp TEXT,
1325             updated_at REAL NOT NULL
1326         );
1327     '''

```

```

1328
1329     # 68. События питания и жизненного цикла Windows (power_ev
ents)
1330     cursor.execute('''
1331         CREATE TABLE IF NOT EXISTS power_events (
1332             id INTEGER PRIMARY KEY AUTOINCREMENT,
1333             event_id INTEGER NOT NULL,
1334             provider TEXT,
1335             channel TEXT DEFAULT 'System',
1336             timestamp TEXT NOT NULL,
1337             created_at REAL NOT NULL,
1338             event_type TEXT NOT NULL,
1339             shutdown_type TEXT,
1340             user TEXT,
1341             domain TEXT,
1342             process TEXT,
1343             process_id INTEGER,
1344             reason TEXT,
1345             reason_code TEXT,
1346             comment TEXT,
1347             unexpected INTEGER DEFAULT 0,
1348             bugcheck_code TEXT,
1349             bugcheck_params_json TEXT,
1350             boot_id TEXT,
1351             details_json TEXT,
1352             raw_xml TEXT,
1353             UNIQUE(event_id, timestamp, provider) ON CONFLICT
REPLACE
1354         );
1355     ''')
1356
1357     # 69. Реконструированные сессии питания ОС (power_sessions
)
1358     cursor.execute('''
1359         CREATE TABLE IF NOT EXISTS power_sessions (
1360             id INTEGER PRIMARY KEY AUTOINCREMENT,
1361             session_id TEXT UNIQUE NOT NULL,
1362             boot_time TEXT NOT NULL,

```

```

1363         boot_timestamp REAL,
1364         shutdown_time TEXT,
1365         shutdown_timestamp REAL,
1366         uptime_seconds REAL,
1367         uptime_human TEXT,
1368         shutdown_type TEXT DEFAULT 'Active',
1369         initiator TEXT,
1370         process TEXT,
1371         reason TEXT,
1372         reason_code TEXT,
1373         comment TEXT,
1374         clean_shutdown INTEGER DEFAULT 1,
1375         unexpected_shutdown INTEGER DEFAULT 0,
1376         bugcheck TEXT,
1377         boot_event_id INTEGER DEFAULT 12,
1378         shutdown_event_id INTEGER,
1379         initiator_chain_json TEXT,
1380         events_json TEXT,
1381         created_at REAL NOT NULL
1382     );
1383     '')
1384
1385     # 69.1. Разрывы и паузы телеметрии при сне (telemetry_gaps
1386     )
1387     cursor.execute('''
1388         CREATE TABLE IF NOT EXISTS telemetry_gaps (
1389             id INTEGER PRIMARY KEY AUTOINCREMENT,
1390             start_utc TEXT NOT NULL,
1391             end_utc TEXT NOT NULL,
1392             gap_type TEXT NOT NULL,
1393             duration_seconds REAL NOT NULL,
1394             confidence TEXT NOT NULL,
1395             evidence_json TEXT,
1396             created_at REAL NOT NULL
1397         );
1398     ''')
1399     cursor.execute('CREATE INDEX IF NOT EXISTS idx_telemetry_gaps_start ON telemetry_gaps(start_utc);')

```

```

1399
1400 # 70. Паспорт программы (process_definition)
1401 cursor.execute('''
1402     CREATE TABLE IF NOT EXISTS process_definition (
1403         definition_id INTEGER PRIMARY KEY AUTOINCREMENT,
1404         name TEXT NOT NULL,
1405         executable_path TEXT NOT NULL UNIQUE,
1406         sha256 TEXT,
1407         company_name TEXT,
1408         file_description TEXT,
1409         icon_base64 TEXT,
1410         first_seen TEXT NOT NULL,
1411         last_seen TEXT NOT NULL
1412     );
1413 ''')
1414
1415 # 71. Единичный инстанс процесса (process_instance - Process Intelligence & PID Recycling Resolution)
1416 cursor.execute('''
1417     CREATE TABLE IF NOT EXISTS process_instance (
1418         instance_id INTEGER PRIMARY KEY AUTOINCREMENT,
1419         definition_id INTEGER NOT NULL,
1420         pid INTEGER NOT NULL,
1421         parent_instance_id INTEGER,
1422         name TEXT NOT NULL,
1423         executable_path TEXT NOT NULL,
1424         command_line TEXT,
1425         start_time TEXT NOT NULL,
1426         exit_time TEXT,
1427         exit_code INTEGER,
1428         session_id INTEGER DEFAULT 1,
1429         user_name TEXT,
1430         integrity_level TEXT,
1431         status TEXT NOT NULL DEFAULT 'RUNNING',
1432         UNIQUE (pid, start_time),
1433         FOREIGN KEY (parent_instance_id) REFERENCES process_instance(instance_id),
1434         FOREIGN KEY (definition_id) REFERENCES process_def

```

```

initiation(definition_id)
1435     );
1436     '')
1437
1438     # 72. Срезы телеметрии во времени (process_sample)
1439     cursor.execute('''
1440         CREATE TABLE IF NOT EXISTS process_sample (
1441             sample_id INTEGER PRIMARY KEY AUTOINCREMENT,
1442             instance_id INTEGER NOT NULL,
1443             timestamp TEXT NOT NULL,
1444             created_at REAL NOT NULL,
1445             cpu_percent REAL DEFAULT 0.0,
1446             cpu_user_time REAL DEFAULT 0.0,
1447             cpu_kernel_time REAL DEFAULT 0.0,
1448             thread_count INTEGER DEFAULT 1,
1449             working_set_mb REAL DEFAULT 0.0,
1450             private_bytes_mb REAL DEFAULT 0.0,
1451             page_faults_sec REAL DEFAULT 0.0,
1452             gpu_load_percent REAL DEFAULT 0.0,
1453             gpu_vram_mb REAL DEFAULT 0.0,
1454             disk_read_bytes_sec REAL DEFAULT 0.0,
1455             disk_write_bytes_sec REAL DEFAULT 0.0,
1456             disk_iops REAL DEFAULT 0.0,
1457             handle_count INTEGER DEFAULT 0,
1458             gdi_objects INTEGER DEFAULT 0,
1459             user_objects INTEGER DEFAULT 0,
1460             sockets_count INTEGER DEFAULT 0,
1461             sockets_json TEXT,
1462             FOREIGN KEY (instance_id) REFERENCES process_insta
nce(instance_id) ON DELETE CASCADE
1463         );
1464     '')
1465
1466     # 73. Лента файловых операций процесса (process_file_event
s)
1467     cursor.execute('''
1468         CREATE TABLE IF NOT EXISTS process_file_events (
1469             event_id INTEGER PRIMARY KEY AUTOINCREMENT,

```

```

1470         pid INTEGER NOT NULL DEFAULT 0,
1471         process_name TEXT NOT NULL DEFAULT '',
1472         action_type TEXT NOT NULL DEFAULT '', -- 'CREATE',
        'MODIFY', 'DELETE', 'RENAME'
1473         target_directory TEXT NOT NULL DEFAULT '',
1474         file_path TEXT NOT NULL,
1475         bytes_affected INTEGER DEFAULT 0,
1476         instance_id INTEGER,
1477         timestamp TEXT NOT NULL DEFAULT (datetime('now')),
1478         created_at REAL DEFAULT 0.0,
1479         FOREIGN KEY (instance_id) REFERENCES process_insta
nce(instance_id) ON DELETE CASCADE
1480     );
1481     '')
1482
1483     # 74. Единая таблица снимков ресурсов и активности процесс
ов по PID (Per-PID Telemetry Engine)
1484     cursor.execute('''
1485         CREATE TABLE IF NOT EXISTS process_pid_snapshots (
1486             snapshot_id INTEGER PRIMARY KEY AUTOINCREMENT,
1487             pid INTEGER NOT NULL,
1488             process_name TEXT NOT NULL,
1489             executable_path TEXT,
1490
1491             -- Процессор (CPU)
1492             cpu_percent REAL NOT NULL DEFAULT 0.0,
1493             user_time_ms INTEGER NOT NULL DEFAULT 0,
1494             kernel_time_ms INTEGER NOT NULL DEFAULT 0,
1495             thread_count INTEGER NOT NULL DEFAULT 1,
1496
1497             -- Память (RAM)
1498             working_set_bytes INTEGER NOT NULL DEFAULT 0,
1499             private_bytes INTEGER NOT NULL DEFAULT 0,
1500             page_faults_count INTEGER DEFAULT 0,
1501
1502             -- Графика (GPU)
1503             gpu_vram_bytes INTEGER DEFAULT 0,
1504             gpu_utilization REAL DEFAULT 0.0,

```

```

1505
1506         -- Дискоский I/O
1507         read_bytes_total INTEGER DEFAULT 0,
1508         write_bytes_total INTEGER DEFAULT 0,
1509         read_ops_total INTEGER DEFAULT 0,
1510         write_ops_total INTEGER DEFAULT 0,
1511
1512         -- Сетевой трафик
1513         net_bytes_sent_total INTEGER DEFAULT 0,
1514         net_bytes_recv_total INTEGER DEFAULT 0,
1515
1516         -- Системные дескрипторы и GUI-ресурсы
1517         handle_count INTEGER DEFAULT 0,
1518         gdi_objects INTEGER DEFAULT 0,
1519         user_objects INTEGER DEFAULT 0,
1520
1521         timestamp TEXT NOT NULL DEFAULT (datetime('now'))
1522     );
1523     '')
1524
1525     # Индексы для ускорения выборок
1526     cursor.execute('CREATE INDEX IF NOT EXISTS idx_proc_def_pa
1527 th ON process_definition(executable_path);')
1528     cursor.execute('CREATE INDEX IF NOT EXISTS idx_proc_inst_p
1529 id_start ON process_instance(pid, start_time);')
1530     cursor.execute('CREATE INDEX IF NOT EXISTS idx_proc_inst_s
1531 tatus ON process_instance(status);')
1532     cursor.execute('CREATE INDEX IF NOT EXISTS idx_proc_inst_p
1533 arent ON process_instance(parent_instance_id);')
1534     cursor.execute('CREATE INDEX IF NOT EXISTS idx_proc_inst_n
1535 ame ON process_instance(name);')
1536     cursor.execute('CREATE INDEX IF NOT EXISTS idx_proc_sample
1537 _inst_time ON process_sample(instance_id, created_at);')
1538     cursor.execute('CREATE INDEX IF NOT EXISTS idx_proc_sample
1539 _time ON process_sample(created_at);')
1540     cursor.execute('CREATE INDEX IF NOT EXISTS idx_proc_fe_ins
1541 t_time ON process_file_events(instance_id, created_at);')
1542     cursor.execute('CREATE INDEX IF NOT EXISTS idx_snapshots_c

```



```
reated_at ON system_snapshots(created_at);')
1535 cursor.execute('CREATE INDEX IF NOT EXISTS idx_startup_arc
hives_created_at ON startup_audit_archives(created_at);')
1536 cursor.execute('CREATE INDEX IF NOT EXISTS idx_startup_arc
hives_archive_id ON startup_audit_archives(archive_id);')
1537 cursor.execute('CREATE INDEX IF NOT EXISTS idx_startup_cha
nges_created_at ON startup_changes(created_at);')
1538 cursor.execute('CREATE INDEX IF NOT EXISTS idx_startup_cha
nges_archive_id ON startup_changes(archive_id);')
1539 cursor.execute('CREATE INDEX IF NOT EXISTS idx_snapshots_t
imestamp ON system_snapshots(timestamp);')
1540 cursor.execute('CREATE INDEX IF NOT EXISTS idx_snapshots_c
reated_host ON system_snapshots(created_at, hostname);')
1541 cursor.execute('CREATE INDEX IF NOT EXISTS idx_sensor_poll
s_sensor_time ON sensor_polls(sensor_id, created_at);')
1542 cursor.execute('CREATE INDEX IF NOT EXISTS idx_sensor_poll
s_category ON sensor_polls(sensor_category, created_at);')
1543 cursor.execute('CREATE INDEX IF NOT EXISTS idx_processes_s
napshot_id ON process_snapshots(snapshot_id);')
1544 cursor.execute('CREATE INDEX IF NOT EXISTS idx_processes_n
ame_pid ON process_snapshots(name, pid);')
1545 cursor.execute('CREATE INDEX IF NOT EXISTS idx_proc_snap_s
napshot_cpu ON process_snapshots(snapshot_id, cpu_percent DESC
);')
1546 cursor.execute('CREATE INDEX IF NOT EXISTS idx_aggregates_
sensor_time ON sensor_aggregates_hourly(sensor_id, period_star
t);')
1547 cursor.execute('CREATE INDEX IF NOT EXISTS idx_rollups_dai
ly_date ON process_rollups_daily(date, name);')
1548 cursor.execute('CREATE INDEX IF NOT EXISTS idx_rollups_tie
r_start_end ON telemetry_rollups(tier, period_start_epoch, per
iod_end_epoch);')
1549 cursor.execute('CREATE INDEX IF NOT EXISTS idx_hardware_au
dits_archive_id ON hardware_audits(archive_id);')
1550 cursor.execute('CREATE INDEX IF NOT EXISTS idx_reboot_hist
ory_boot_time ON reboot_history(boot_time);')
1551 cursor.execute('CREATE INDEX IF NOT EXISTS idx_incidents_c
reated_at ON incidents(created_at);')
```

```
1552     cursor.execute('CREATE INDEX IF NOT EXISTS idx_incidents_t  
rigger ON incidents(trigger_type);')  
1553     cursor.execute('CREATE INDEX IF NOT EXISTS idx_incidents_s  
ev_time ON incidents(severity, created_at);')  
1554     cursor.execute('CREATE INDEX IF NOT EXISTS idx_proc_prov_g  
uid ON process_provenance_events(process_guid);')  
1555     cursor.execute('CREATE INDEX IF NOT EXISTS idx_proc_prov_c  
reated_at ON process_provenance_events(created_at);')  
1556     cursor.execute('CREATE INDEX IF NOT EXISTS idx_proc_prov_n  
ame_pid ON process_provenance_events(name, pid);')  
1557     cursor.execute('CREATE INDEX IF NOT EXISTS idx_device_even  
ts_created_at ON device_events(created_at);')  
1558     cursor.execute('CREATE INDEX IF NOT EXISTS idx_device_inve  
ntory_dev_id ON device_inventory(device_instance_id);')  
1559     cursor.execute('CREATE INDEX IF NOT EXISTS idx_w64_events_  
created_at ON w64_events(created_at);')  
1560     cursor.execute('CREATE INDEX IF NOT EXISTS idx_process_out  
liers_created_at ON process_outliers(created_at);')  
1561     cursor.execute('CREATE INDEX IF NOT EXISTS idx_extended_au  
dits_created_at ON system_extended_audits(created_at);')  
1562     cursor.execute('CREATE INDEX IF NOT EXISTS idx_telemetry_h  
ourly_time ON telemetry_hourly(bucket_start);')  
1563     cursor.execute('CREATE INDEX IF NOT EXISTS idx_telemetry_h  
ourly_sensor ON telemetry_hourly(sensor_id, bucket_start);')  
1564     cursor.execute('CREATE INDEX IF NOT EXISTS idx_telemetry_d  
aily_time ON telemetry_daily(bucket_start);')  
1565     cursor.execute('CREATE INDEX IF NOT EXISTS idx_telemetry_d  
aily_sensor ON telemetry_daily(sensor_id, bucket_start);')  
1566     cursor.execute('CREATE INDEX IF NOT EXISTS idx_telemetry_w  
eekly_time ON telemetry_weekly(bucket_start);')  
1567     cursor.execute('CREATE INDEX IF NOT EXISTS idx_telemetry_w  
eekly_sensor ON telemetry_weekly(sensor_id, bucket_start);')  
1568     cursor.execute('CREATE INDEX IF NOT EXISTS idx_telemetry_m  
onthly_time ON telemetry_monthly(bucket_start);')  
1569     cursor.execute('CREATE INDEX IF NOT EXISTS idx_telemetry_m  
onthly_sensor ON telemetry_monthly(sensor_id, bucket_start);')  
1570     cursor.execute('CREATE INDEX IF NOT EXISTS idx_telemetry_y  
early_time ON telemetry_yearly(bucket_start);')
```

```
1571 cursor.execute('CREATE INDEX IF NOT EXISTS idx_telemetry_y
early_sensor ON telemetry_yearly(sensor_id, bucket_start);')
1572 cursor.execute('CREATE INDEX IF NOT EXISTS idx_telemetry_s
pikes_time ON telemetry_spikes(created_at, sensor_id);')
1573 cursor.execute('CREATE INDEX IF NOT EXISTS idx_disk_io_pid
_time ON disk_io_events(pid, created_at);')
1574 cursor.execute('CREATE INDEX IF NOT EXISTS idx_disk_io_dis
k_time ON disk_io_events(disk_id, created_at);')
1575 cursor.execute('CREATE INDEX IF NOT EXISTS idx_disk_health
_disk_time ON disk_health_snapshots(disk_id, created_at);')
1576 cursor.execute('CREATE INDEX IF NOT EXISTS idx_disk_health
_serial_time ON disk_health_snapshots(serial_number, created_a
t);')
1577 cursor.execute('CREATE INDEX IF NOT EXISTS idx_disk_inv_se
rial ON disk_inventory(serial_number);')
1578 cursor.execute('CREATE INDEX IF NOT EXISTS idx_volume_inv_
guid ON volume_inventory(volume_guid);')
1579 cursor.execute('CREATE INDEX IF NOT EXISTS idx_disk_perf_n
ame_time ON disk_performance_samples(disk_name, created_at);')
1580 cursor.execute('CREATE INDEX IF NOT EXISTS idx_cpu_inv_nam
e ON cpu_inventory(name);')
1581 cursor.execute('CREATE INDEX IF NOT EXISTS idx_ram_mod_ser
ial ON ram_module_inventory(serial_number);')
1582 cursor.execute('CREATE INDEX IF NOT EXISTS idx_gpu_inv_pci
ON gpu_inventory(pci_device_id);')
1583 cursor.execute('CREATE INDEX IF NOT EXISTS idx_net_inv_gui
d ON network_adapter_inventory(adapter_guid);')
1584 cursor.execute('CREATE INDEX IF NOT EXISTS idx_cpu_samples
_time ON cpu_telemetry_samples(created_at);')
1585 cursor.execute('CREATE INDEX IF NOT EXISTS idx_ram_samples
_time ON ram_telemetry_samples(created_at);')
1586 cursor.execute('CREATE INDEX IF NOT EXISTS idx_gpu_samples
_time ON gpu_telemetry_samples(created_at);')
1587 cursor.execute('CREATE INDEX IF NOT EXISTS idx_net_samples
_time ON network_adapter_samples(created_at);')
1588
1589 # Индексы для новых подсистем
1590 cursor.execute('CREATE INDEX IF NOT EXISTS idx_evt_chan_sn
```

```
ap_id ON event_log_channel_snapshots(snapshot_id);')
1591 cursor.execute('CREATE INDEX IF NOT EXISTS idx_evt_chan_na
me ON event_log_channel_snapshots(channel_name);')
1592 cursor.execute('CREATE INDEX IF NOT EXISTS idx_evt_entries
_snap_id ON event_log_entries_cache(snapshot_id);')
1593 cursor.execute('CREATE INDEX IF NOT EXISTS idx_evt_entries
_chan_lvl ON event_log_entries_cache(channel, level);')
1594 cursor.execute('CREATE INDEX IF NOT EXISTS idx_evt_intel_c
han ON event_log_intelligence_profiles(channel);')
1595
1596 cursor.execute('CREATE INDEX IF NOT EXISTS idx_fw_prof_sna
pshot ON firewall_profile_snapshots(snapshot_id);')
1597 cursor.execute('CREATE INDEX IF NOT EXISTS idx_fw_rule_sna
pshot ON firewall_rule_snapshots(snapshot_id);')
1598 cursor.execute('CREATE INDEX IF NOT EXISTS idx_fw_rule_nam
e ON firewall_rule_snapshots(rule_name);')
1599
1600 cursor.execute('CREATE INDEX IF NOT EXISTS idx_services_sn
ap_id ON services_snapshots(snapshot_id);')
1601 cursor.execute('CREATE INDEX IF NOT EXISTS idx_services_na
me ON services_snapshots(service_name);')
1602 cursor.execute('CREATE INDEX IF NOT EXISTS idx_services_st
ate ON services_snapshots(state);')
1603 cursor.execute('CREATE INDEX IF NOT EXISTS idx_svc_changes
_ts ON service_change_events(timestamp);')
1604 cursor.execute('CREATE INDEX IF NOT EXISTS idx_svc_changes
_name ON service_change_events(service_name);')
1605
1606 cursor.execute('CREATE INDEX IF NOT EXISTS idx_throttling_
snapshot_id ON cpu_throttling_snapshots(snapshot_id);')
1607 cursor.execute('CREATE INDEX IF NOT EXISTS idx_throttling_
timestamp ON cpu_throttling_snapshots(timestamp);')
1608 cursor.execute('CREATE INDEX IF NOT EXISTS idx_thermal_zon
e_snapshot ON thermal_zone_snapshots(snapshot_id);')
1609
1610 cursor.execute('CREATE INDEX IF NOT EXISTS idx_forensics_s
napshot_id ON forensics_snapshots(snapshot_id);')
1611 cursor.execute('CREATE INDEX IF NOT EXISTS idx_forensics_t
```

```
timestamp ON forensics_snapshots(timestamp);')
1612
1613     cursor.execute('CREATE INDEX IF NOT EXISTS idx_proc_leak_s
napshot_id ON process_leak_snapshots(snapshot_id);')
1614     cursor.execute('CREATE INDEX IF NOT EXISTS idx_proc_leak_t
imestamp ON process_leak_snapshots(timestamp);')
1615     cursor.execute('CREATE INDEX IF NOT EXISTS idx_proc_leak_i
tem_snapshot ON process_leak_items(snapshot_id);')
1616     cursor.execute('CREATE INDEX IF NOT EXISTS idx_proc_leak_i
tem_pid ON process_leak_items(pid);')
1617
1618     cursor.execute('CREATE INDEX IF NOT EXISTS idx_defender_sn
apshot_id ON defender_snapshots(snapshot_id);')
1619     cursor.execute('CREATE INDEX IF NOT EXISTS idx_defender_ti
mestamp ON defender_snapshots(timestamp);')
1620     cursor.execute('CREATE INDEX IF NOT EXISTS idx_defender_ex
clusions_snap ON defender_exclusions(snapshot_id);')
1621     cursor.execute('CREATE INDEX IF NOT EXISTS idx_defender_as
r_snap ON defender_asr_rules(snapshot_id);')
1622     cursor.execute('CREATE INDEX IF NOT EXISTS idx_defender_th
reats_snap ON defender_threats(snapshot_id);')
1623
1624     cursor.execute('CREATE INDEX IF NOT EXISTS idx_proc_net_sn
apshot_id ON process_network_snapshots(snapshot_id);')
1625     cursor.execute('CREATE INDEX IF NOT EXISTS idx_proc_net_pi
d ON process_network_snapshots(pid);')
1626
1627     cursor.execute('CREATE INDEX IF NOT EXISTS idx_sw_storage_
app ON software_storage_locations(app_id);')
1628     cursor.execute('CREATE INDEX IF NOT EXISTS idx_sw_config_a
pp ON software_config_files(app_id);')
1629     cursor.execute('CREATE INDEX IF NOT EXISTS idx_sw_net_app_
ts ON software_network_snapshots(timestamp, app_id);')
1630
1631     # Индексы для подсистемы безопасности Security Event Log
1632     cursor.execute('CREATE INDEX IF NOT EXISTS idx_security_ev
ent_id ON security_events(event_id);')
1633     cursor.execute('CREATE INDEX IF NOT EXISTS idx_security_ti
```

```

mestamp ON security_events(timestamp);')
1634 cursor.execute('CREATE INDEX IF NOT EXISTS idx_security_cr
eated_at ON security_events(created_at);')
1635 cursor.execute('CREATE INDEX IF NOT EXISTS idx_security_us
er ON security_events(subject_user);')
1636 cursor.execute('CREATE INDEX IF NOT EXISTS idx_security_ta
rget_user ON security_events(target_user);')
1637 cursor.execute('CREATE INDEX IF NOT EXISTS idx_security_pr
ocess ON security_events(process_name);')
1638 cursor.execute('CREATE INDEX IF NOT EXISTS idx_security_pi
d ON security_events(process_id);')
1639 cursor.execute('CREATE INDEX IF NOT EXISTS idx_security_re
cord_id ON security_events(event_record_id);')
1640 cursor.execute('CREATE INDEX IF NOT EXISTS idx_sec_raw_rec
ord_id ON security_events_raw(event_record_id);')
1641 cursor.execute('CREATE INDEX IF NOT EXISTS idx_sec_raw_cre
ated_at ON security_events_raw(created_at);')
1642 cursor.execute('CREATE INDEX IF NOT EXISTS idx_sec_bm_chan
nel ON security_collector_bookmarks(channel);')
1643
1644 # Индексы для подсистемы событий питания и сессий
1645 cursor.execute('CREATE INDEX IF NOT EXISTS idx_power_event
s_eid ON power_events(event_id);')
1646 cursor.execute('CREATE INDEX IF NOT EXISTS idx_power_event
s_ts ON power_events(timestamp);')
1647 cursor.execute('CREATE INDEX IF NOT EXISTS idx_power_event
s_type ON power_events(event_type);')
1648 cursor.execute('CREATE INDEX IF NOT EXISTS idx_power_event
s_user ON power_events(user);')
1649 cursor.execute('CREATE INDEX IF NOT EXISTS idx_power_event
s_proc ON power_events(process);')
1650 cursor.execute('CREATE INDEX IF NOT EXISTS idx_power_sessi
ons_boot_time ON power_sessions(boot_time);')
1651 cursor.execute('CREATE INDEX IF NOT EXISTS idx_power_sessi
ons_shutdown_type ON power_sessions(shutdown_type);')
1652 cursor.execute('CREATE INDEX IF NOT EXISTS idx_power_sessi
ons_clean ON power_sessions(clean_shutdown, unexpected_shutdow
n);')

```

```

1653     cursor.execute('CREATE INDEX IF NOT EXISTS idx_power_sessions_initiator ON power_sessions(initiator);')
1654
1655     _run_migrations(cursor)
1656
1657     # Индексы для Per-PID Telemetry Engine и отслеживания файлов
1658     cursor.execute('CREATE INDEX IF NOT EXISTS idx_proc_pid_time ON process_pid_snapshots(pid, timestamp);')
1659     cursor.execute('CREATE INDEX IF NOT EXISTS idx_proc_name_time ON process_pid_snapshots(process_name, timestamp);')
1660     cursor.execute('CREATE INDEX IF NOT EXISTS idx_file_events_pid ON process_file_events(pid);')
1661     cursor.execute('CREATE INDEX IF NOT EXISTS idx_file_events_dir ON process_file_events(target_directory, timestamp);')
1662
1663     conn.commit()
1664
1665
1666 def _run_migrations(cursor: sqlite3.Cursor) -> None:
1667     """Выполняет безопасные миграции добавления новых колонок
1668     в существующие таблицы."""
1669     # Миграция: telemetry_events
1670     try:
1671         cursor.execute("PRAGMA table_info(telemetry_events)")
1672         columns = [row[1] for row in cursor.fetchall()]
1673         if 'severity' not in columns:
1674             cursor.execute("ALTER TABLE telemetry_events ADD COLUMN severity TEXT DEFAULT 'info'")
1675         if 'event_details' not in columns:
1676             cursor.execute("ALTER TABLE telemetry_events ADD COLUMN event_details TEXT")
1677         if 'raw_json' not in columns:
1678             cursor.execute("ALTER TABLE telemetry_events ADD COLUMN raw_json TEXT")
1679     except Exception as exc:
1680         logger.warning(f'Миграция telemetry_events: {exc}')

```

```
1681 # Миграция: system_snapshots
1682 try:
1683     cursor.execute("PRAGMA table_info(system_snapshots)")
1684     snap_cols = {row[1] for row in cursor.fetchall()}
1685     expected_snap_cols = {
1686         'hostname': 'TEXT',
1687         'username': 'TEXT',
1688         'uptime_seconds': 'REAL',
1689         'os_name': 'TEXT',
1690         'os_build': 'TEXT',
1691         'os_install_date': 'TEXT',
1692         'system_language': 'TEXT',
1693         'os_install_language': 'TEXT',
1694         'user_locale': 'TEXT',
1695         'system_locale': 'TEXT',
1696         'timezone': 'TEXT',
1697         'codepage': 'TEXT',
1698         'input_languages_json': 'TEXT',
1699         'disks_json': 'TEXT',
1700         'physical_disks_json': 'TEXT',
1701         'monitors_json': 'TEXT',
1702         'updates_json': 'TEXT',
1703         'office_json': 'TEXT',
1704         'onedrive_json': 'TEXT',
1705         'battery_json': 'TEXT',
1706         'ram_sticks_json': 'TEXT',
1707         'listening_ports_json': 'TEXT',
1708         'alerts_json': 'TEXT',
1709         'cpu_total_percent': 'REAL',
1710         'cpu_frequency_mhz': 'REAL',
1711         'memory_total_gb': 'REAL',
1712         'memory_used_gb': 'REAL',
1713         'memory_percent': 'REAL',
1714         'swap_percent': 'REAL',
1715         'gpu_load_percent': 'REAL',
1716         'gpu_temp_c': 'REAL',
1717         'disk_read_bytes_sec': 'REAL',
1718         'disk_write_bytes_sec': 'REAL',
```



```

1719         'disk_read_count_sec': 'REAL',
1720         'disk_write_count_sec': 'REAL',
1721         'network_sent_bytes_sec': 'REAL',
1722         'network_recv_bytes_sec': 'REAL',
1723         'hardware_audit': 'TEXT',
1724         'raw_json': 'TEXT',
1725     }
1726     for col_name, col_type in expected_snap_cols.items():
1727         if col_name not in snap_cols:
1728             cursor.execute(f"ALTER TABLE system_snapshots
1729 ADD COLUMN {col_name} {col_type}")
1729     except Exception as exc:
1730         logger.warning(f'Миграция system_snapshots: {exc}')
1731
1732     # Миграция: process_snapshots
1733     try:
1734         cursor.execute("PRAGMA table_info(process_snapshots)")
1735         proc_cols = {row[1] for row in cursor.fetchall()}
1736         new_proc_columns = {
1737             'num_handles': 'INTEGER DEFAULT 0',
1738             'read_bytes_sec': 'REAL',
1739             'write_bytes_sec': 'REAL',
1740             'integrity_level': 'TEXT',
1741             'elevation': 'INTEGER DEFAULT 0',
1742             'ppid': 'INTEGER',
1743             'parent_name': 'TEXT',
1744             'executable': 'TEXT',
1745             'cmdline': 'TEXT',
1746             'sid': 'TEXT',
1747             'session_id': 'INTEGER',
1748             'creation_time': 'TEXT',
1749             'process_guid': 'TEXT',
1750             'ancestor_chain': 'TEXT',
1751             'launch_reason': 'TEXT',
1752         }
1753         for col_name, col_def in new_proc_columns.items():
1754             if col_name not in proc_cols:
1755                 cursor.execute(f"ALTER TABLE process_snapshots

```

```

    ADD COLUMN {col_name} {col_def}")
1756     except Exception as exc:
1757         logger.warning(f'Миграция process_snapshots: {exc}')
1758
1759     # Миграция: process_file_events
1760     try:
1761         cursor.execute("PRAGMA table_info(process_file_events)
1762 ")
1763         fe_cols = {row[1] for row in cursor.fetchall()}
1764         if fe_cols:
1765             # Безопасное переименование устаревших имен колоно
1766 к при их наличии
1767             legacy_renames = [
1768                 ('id', 'event_id'),
1769                 ('action', 'action_type'),
1770                 ('target_folder', 'target_directory'),
1771                 ('bytes_count', 'bytes_affected'),
1772             ]
1773             for old_col, new_col in legacy_renames:
1774                 if old_col in fe_cols and new_col not in fe_co
1775 ls:
1776                 cursor.execute(f"ALTER TABLE process_file_
1777 events RENAME COLUMN {old_col} TO {new_col}")
1778                 fe_cols.remove(old_col)
1779                 fe_cols.add(new_col)
1780
1781             target_fe_columns = {
1782                 'pid': "INTEGER NOT NULL DEFAULT 0",
1783                 'process_name': "TEXT NOT NULL DEFAULT ''",
1784                 'action_type': "TEXT NOT NULL DEFAULT ''",
1785                 'target_directory': "TEXT NOT NULL DEFAULT ''"
1786 ,
1787                 'file_path': "TEXT NOT NULL DEFAULT ''",
1788                 'bytes_affected': "INTEGER DEFAULT 0",
1789                 'instance_id': "INTEGER",
1790                 'timestamp': "TEXT NOT NULL DEFAULT (datetime(
1791 'now'))",
1792                 'created_at': "REAL DEFAULT 0.0",

```

```
1787 |         }
1788 |         for col_name, col_def in target_fe_columns.items()
      | :
1789 |             if col_name not in fe_cols:
1790 |                 cursor.execute(f"ALTER TABLE process_file_
      | events ADD COLUMN {col_name} {col_def}")
1791 |     except Exception as exc:
1792 |         logger.warning(f'Миграция process_file_events: {exc}')
1793 |
1794 |
1795 |
```

storage.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry Sqlite -
  Storage
4 # =====
5 # Description:
6 #   Главный фасад TelemetryStorage базы данных SQLite телеметр
  ии.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.sqlite import TelemetryStora
    ge
11 #
12 #       storage = TelemetryStorage.get_instance(read_only=True)
13 #       snapshots = storage.get_snapshots(limit=10)
14 #
15 # File: storage.py
16 # Project: ai-breadboard
17 # Package: apps.windows.telemetry.sqlite
18 # Author: hypo69
19 # Copyright: © 2026 hypo69
20 # Updated: 2026-10-10 05:41:00
21 # =====
22
23 from __future__ import annotations
24
25 """Главный класс TelemetryStorage, объединяющий подсистемы чте
  ния, записи, буферизации и обслуживания."""
26
27 import atexit
28 import os
29 import sqlite3
```

```
30 import threading
31 from pathlib import Path
32 from typing import Any, Dict, List, Optional, Union
33
34 from .aggregator import AggregationLevel, TelemetrySqlAggregator
35 from .buffer import TelemetryBuffer
36 from .connection import TelemetryConnectionManager
37 from .maintenance import TelemetryMaintenance
38 from .reader import TelemetryReader
39 from .schema import init_database_schema
40 from .writer import TelemetryWriter
41 from ..models import (
42     HardwareArchiveEntry,
43     ProcessLifecycleEvent,
44     ProcessProvenanceInfo,
45     StartupArchiveEntry,
46     SystemMetricRollup,
47     SystemSnapshot,
48     TelemetryIncident,
49 )
50 from ..telemetry_config import TelemetryConfigManager
51
52 try:
53     from logger import logger
54 except ImportError:
55     import logging
56     logger = logging.getLogger(__name__)
57
58
59 class TelemetryStorage:
60     """Универсальный фасад персистентного хранения и выборки т
    елеметрии в SQLite."""
61
62     _instance: Optional[TelemetryStorage] = None
63     _ro_instance: Optional[TelemetryStorage] = None
64     _lock = threading.RLock()
65
```

```

66     def __init__(
67         self,
68         db_path: Optional[Union[str, Path]] = None,
69         buffer_mode: Optional[str] = None,
70         buffer_size: Optional[int] = None,
71         flush_interval_seconds: Optional[float] = None,
72         buffer_file_path: Optional[Union[str, Path]] = None,
73         auto_flush: bool = True,
74         max_db_size_mb: Optional[float] = None,
75         retention_days: Optional[int] = None,
76         auto_vacuum: Optional[bool] = None,
77         read_only: bool = False,
78     ) -> None:
79         """Инициализирует подключение к SQLite базе данных тел
80         еметрии."""
81
82         self.read_only = bool(read_only)
83
84         # 1. Определение путей базы данных
85         if db_path is None:
86             programdata = os.environ.get('ProgramData') or os.
87             environ.get('ALLUSERSPROFILE')
88             if programdata and os.path.exists(programdata):
89                 base_dir = Path(programdata)
90             else:
91                 appdata = os.environ.get('APPDATA') or os.envi
92                 ron.get('LOCALAPPDATA')
93                 base_dir = Path(appdata) if appdata and os.pat
94                 h.exists(appdata) else (Path.home() / '.config')
95
96                 ai_tel_dir = base_dir / 'AITelemetry' / 'data'
97                 if (ai_tel_dir / 'telemetry.db').is_file() or ai_t
98                 el_dir.is_dir():
99                     target_dir = ai_tel_dir
100                 else:
101                     target_dir = base_dir / 'AI-Breadboard' / 'app
102                     s' / 'windows' / 'telemetry' / 'logs'
103
104                 if not self.read_only:

```

```

98         target_dir.mkdir(parents=True, exist_ok=True)
99         self.db_path = target_dir / 'telemetry.db'
100     else:
101         self.db_path = Path(db_path)
102         if not self.read_only:
103             self.db_path.parent.mkdir(parents=True, exist_
ok=True)
104
105     # 2. Конфигурация
106     cfg_manager = None
107     try:
108         cfg_manager = TelemetryConfigManager()
109     except Exception:
110         cfg_manager = None
111
112     raw_mode = buffer_mode or (cfg_manager.get_buffer_mode
() if cfg_manager else 'memory')
113     b_mode = raw_mode.lower() if raw_mode in ('memory', 'f
ile', 'direct') else 'memory'
114     b_size = int(buffer_size if buffer_size is not None el
se (cfg_manager.get_buffer_size() if cfg_manager else 50))
115     f_interval = float(flush_interval_seconds if flush_int
erval_seconds is not None else (cfg_manager.get_flush_interval
_seconds() if cfg_manager else 30.0))
116     m_size = float(max_db_size_mb if max_db_size_mb is not
None else (cfg_manager.get_max_db_size_mb() if cfg_manager el
se 50.0))
117     r_days = int(retention_days if retention_days is not N
one else (cfg_manager.get_retention_days() if cfg_manager else
7))
118     a_vacuum = bool(auto_vacuum if auto_vacuum is not None
else (cfg_manager.is_auto_vacuum_enabled() if cfg_manager els
e True))
119
120     # 3. Инициализация менеджера подключений
121     self._cm = TelemetryConnectionManager(db_path=self.db_
path, read_only=self.read_only, lock=self._lock)
122

```

```

123 |         # 4. Инициализация схемы (только в режиме записи)
124 |         if not self.read_only:
125 |             try:
126 |                 with self._cm.lock, self._cm.get_connection()
| as conn:
127 |                     init_database_schema(conn)
128 |             except Exception as schema_err:
129 |                 logger.warning(f'Ошибка инициализации схемы ба
| зы данных: {schema_err}')
130 |
131 |         # 5. Инициализация подсистем
132 |         self._reader = TelemetryReader(self._cm)
133 |         self._writer = TelemetryWriter(self._cm)
134 |         self._buffer = TelemetryBuffer(
135 |             connection_manager=self._cm,
136 |             writer=self._writer,
137 |             buffer_mode=b_mode,
138 |             buffer_size=b_size,
139 |             flush_interval_seconds=f_interval,
140 |             buffer_file_path=buffer_file_path,
141 |             auto_flush=auto_flush,
142 |         )
143 |         self._maintenance = TelemetryMaintenance(
144 |             connection_manager=self._cm,
145 |             max_db_size_mb=m_size,
146 |             retention_days=r_days,
147 |             auto_vacuum_enabled=a_vacuum,
148 |             buffer_file_path=self._buffer.buffer_file_path,
149 |         )
150 |         self._aggregator = TelemetrySqlAggregator(self._cm)
151 |
152 |         if not self.read_only:
153 |             try:
154 |                 atexit.register(self.close)
155 |             except Exception:
156 |                 pass
157 |
158 |         @classmethod

```



```

159     def get_instance(
160         cls,
161         db_path: Optional[Union[str, Path]] = None,
162         read_only: bool = False,
163     ) -> TelemetryStorage:
164         """Возвращает синглтон-экземпляр хранилища телеметрии
165         (для записи или только для чтения)."""
166         with cls._lock:
167             if read_only:
168                 if cls._ro_instance is None or (db_path is not
169                 None and cls._ro_instance.db_path != Path(db_path)):
170                     cls._ro_instance = TelemetryStorage(db_path=
171                     db_path, read_only=True, auto_flush=False)
172                 return cls._ro_instance
173             else:
174                 if cls._instance is None or (db_path is not No
175                 ne and cls._instance.db_path != Path(db_path)):
176                     cls._instance = TelemetryStorage(db_path=d
177                     b_path, read_only=False)
178                 return cls._instance
179
180     def _get_connection(self) -> sqlite3.Connection:
181         """Создает и настраивает соединение с базой данных SQL
182         ite."""
183         return self._cm.get_connection()
184
185     def get_connection(self) -> sqlite3.Connection:
186         """Возвращает активное подключение к SQLite базе данны
187         х."""
188         return self._cm.get_connection()
189
190     @property
191     def connection(self) -> sqlite3.Connection:
192         """Свойство прямого доступа к подключению SQLite."""
193         return self._cm.get_connection()
194
195     @property
196     def reader(self) -> TelemetryReader:

```

```

190         """Подсистема чтения данных."""
191         return self._reader
192
193     @property
194     def writer(self) -> TelemetryWriter:
195         """Подсистема записи данных."""
196         return self._writer
197
198     @property
199     def buffer(self) -> TelemetryBuffer:
200         """Подсистема буферизации."""
201         return self._buffer
202
203     @property
204     def maintenance(self) -> TelemetryMaintenance:
205         """Подсистема регламентного обслуживания."""
206         return self._maintenance
207
208     @property
209     def aggregator(self) -> TelemetrySqlAggregator:
210         """Подсистема многоуровневой SQL-агрегации."""
211         return self._aggregator
212
213     # -----
214     # Делегирование методов буфера
215     # -----
216
217     def set_buffer_mode(self, mode: str) -> None:
218         self._buffer.set_buffer_mode(mode)
219
220     def get_buffer_mode(self) -> str:
221         return self._buffer.buffer_mode
222
223     def set_flush_interval_seconds(self, interval: float) -> N
one:
224         """Обновляет интервал таймера сброса буфера в SQLite."""

```

```

"""
225         self._buffer.set_flush_interval_seconds(interval)
226
227     def get_buffered_count(self) -> int:
228         return self._buffer.get_buffered_count()
229
230     def flush(self) -> int:
231         return self._buffer.flush()
232
233     def close(self) -> None:
234         self._aggregator.stop_background_scheduler()
235         self._buffer.close()
236
237     # -----
238     # Делегирование методов сохранения (Writer / Buffer)
239     # -----
240
241     def save_snapshot(self, snapshot: SystemSnapshot, top_n: i
nt = 20) -> int:
242         if self._buffer.buffer_mode == 'direct':
243             with self._cm.lock, self._cm.get_connection() as c
onn:
244                 snap_id = self._writer.insert_snapshot_row(con
n.cursor(), {'data': snapshot, 'top_n': top_n})
245                 conn.commit()
246                 return snap_id
247             snap_dict = snapshot if isinstance(snapshot, dict) els
e (snapshot.model_dump() if hasattr(snapshot, 'model_dump') el
se (vars(snapshot) if hasattr(snapshot, '__dict__') else snaps
hot))
248             self._buffer.enqueue({'type': 'snapshot', 'data': snap
_dict, 'top_n': top_n})
249             return 1
250
251     def save_sensor_poll(self, sensor_item: Dict[str, Any], ti
mestamp: Optional[str] = None) -> int:

```

```

252         if self._buffer.buffer_mode == 'direct':
253             with self._cm.lock, self._cm.get_connection() as c
conn:
254                 row = self._writer.prepare_sensor_poll_row(sensor_item, timestamp)
255                 conn.cursor().execute('''
256                     INSERT INTO sensor_polls (
257                         sensor_id, timestamp, created_at, hardware_name, hardware_type,
258                         sensor_category, sensor_name, unit, value, provider, provider_priority, raw_json
259                     ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
260                     ON CONFLICT(sensor_id, timestamp) DO UPDATE SET
261                         created_at = excluded.created_at, hardware_name = excluded.hardware_name,
262                         hardware_type = excluded.hardware_type, sensor_category = excluded.sensor_category,
263                         sensor_name = excluded.sensor_name, unit = excluded.unit, value = excluded.value,
264                         provider = excluded.provider, provider_priority = excluded.provider_priority,
265                         raw_json = excluded.raw_json
266                     ''', row)
267                 conn.commit()
268                 return 1
269             self._buffer.enqueue({'type': 'sensor_poll', 'data': sensor_item, 'timestamp': timestamp})
270             return 1
271
272     def save_sensor_polls_batch(self, items: List[Dict[str, Any]], timestamp: Optional[str] = None, deduplicate: bool = True) -> int:
273         if not items:
274             return 0
275         if deduplicate:
276             from ..sensor_registry import SensorDeduplicator,

```

```

SensorProvider
277         deduplicator = SensorDeduplicator()
278         for item in items:
279             sensor_id = str(item.get('id', ''))
280             value = item.get('value', 0.0)
281             provider_raw = item.get('_provider', SensorPro
vider.SENSOR_COLLECTOR)
282             if isinstance(provider_raw, str):
283                 try:
284                     provider = SensorProvider[provider_raw
.upper()]
285                 except KeyError:
286                     provider = SensorProvider.SENSOR_COLLE
CTOR
287             elif isinstance(provider_raw, SensorProvider):
288                 provider = provider_raw
289             else:
290                 provider = SensorProvider.SENSOR_COLLECTOR
291                 deduplicator.add(sensor_id, provider, value)
292             unique_providers = deduplicator.get_unique()
293             items = [it for it in items if str(it.get('id', ''
)) in unique_providers]
294
295         if not items:
296             return 0
297         if self._buffer.buffer_mode == 'direct':
298             return self._writer.batch_insert_records([{'type':
'sensor_polls_batch', 'data': items, 'timestamp': timestamp}]
)
299         self._buffer.enqueue({'type': 'sensor_polls_batch', 'd
ata': items, 'timestamp': timestamp})
300         return len(items)
301
302         save_sensor_polls = save_sensor_polls_batch
303
304         def save_sensor_aggregates_batch(self, aggregates: List[Di
ct[str, Any]]) -> int:
305             if not aggregates:

```

```

306         return 0
307         if self._buffer.buffer_mode == 'direct':
308             return self._writer.batch_insert_records([{'type':
    'sensor_aggregates_batch', 'data': aggregates}])
309             self._buffer.enqueue({'type': 'sensor_aggregates_batch
    ', 'data': aggregates})
310             return len(aggregates)
311
312     def save_event(self, event_type: str, event_details: Dict[
    str, Any], severity: str = 'info', timestamp: Optional[str] =
    None) -> int:
313         rec = {'type': 'event', 'event_type': event_type, 'det
    ails': event_details, 'severity': severity, 'timestamp': times
    tamp}
314         if self._buffer.buffer_mode == 'direct':
315             return self._writer.batch_insert_records([rec])
316             self._buffer.enqueue(rec)
317             return 1
318
319     def save_hardware_archive(self, archive_entry: HardwareArc
    hiveEntry) -> int:
320         entry_dict = archive_entry if isinstance(archive_entry
    , dict) else (archive_entry.model_dump() if hasattr(archive_en
    try, 'model_dump') else (vars(archive_entry) if hasattr(archiv
    e_entry, '__dict__') else archive_entry))
321         if self._buffer.buffer_mode == 'direct':
322             return self._writer.batch_insert_records([{'type':
    'hardware_archive', 'data': entry_dict}])
323             self._buffer.enqueue({'type': 'hardware_archive', 'dat
    a': entry_dict})
324             return 1
325
326     def save_startup_archive(self, archive_entry: StartupArch
    iveEntry) -> int:
327         """Сохраняет снимок аудита автозапуска и зафиксированн
    ые изменения в БД."""
328         entry_dict = archive_entry if isinstance(archive_entry
    , dict) else (archive_entry.model_dump() if hasattr(archive_en

```

```

try, 'model_dump') else (vars(archive_entry) if hasattr(archive_entry, '__dict__') else archive_entry))
329         if self._buffer.buffer_mode == 'direct':
330             return self._writer.batch_insert_records([{'type': 'startup_archive', 'data': entry_dict}])
331             self._buffer.enqueue({'type': 'startup_archive', 'data': entry_dict})
332             return 1
333
334     def save_app_poll(self, app: str, poll_type: str, metric_name: str, value: Any, unit: str = '', status: str = 'OK', details: Any = '', timestamp: Optional[str] = None) -> int:
335         rec = {'type': 'app_poll', 'app': app, 'poll_type': poll_type, 'metric_name': metric_name, 'value': value, 'unit': unit, 'status': status, 'details': details, 'timestamp': timestamp}
336         if self._buffer.buffer_mode == 'direct':
337             return self._writer.batch_insert_records([rec])
338             self._buffer.enqueue(rec)
339             return 1
340
341     def save_app_event(self, app: str, event_type: str, status: str = 'OK', details: Any = '', timestamp: Optional[str] = None) -> int:
342         rec = {'type': 'app_event', 'app': app, 'event_type': event_type, 'status': status, 'details': details, 'timestamp': timestamp}
343         if self._buffer.buffer_mode == 'direct':
344             return self._writer.batch_insert_records([rec])
345             self._buffer.enqueue(rec)
346             return 1
347
348     def save_app_param_change(self, app: str, param_name: str, old_value: Any, new_value: Any, status: str = 'SUCCESS', user: str = 'system', details: Any = '', timestamp: Optional[str] = None) -> int:
349         rec = {'type': 'app_param_change', 'app': app, 'param_name': param_name, 'old_value': old_value, 'new_value': new_value}

```

```

    lue, 'status': status, 'user': user, 'details': details, 'time
stamp': timestamp}
350         if self._buffer.buffer_mode == 'direct':
351             return self._writer.batch_insert_records([rec])
352         self._buffer.enqueue(rec)
353         return 1
354
355     def save_app_polls_batch(self, polls: List[Dict[str, Any]]
) -> int:
356         if not polls:
357             return 0
358         if self._buffer.buffer_mode == 'direct':
359             return self._writer.batch_insert_records([{'type':
'app_polls_batch', 'data': polls}])
360         self._buffer.enqueue({'type': 'app_polls_batch', 'data
': polls})
361         return len(polls)
362
363     def save_app_events_batch(self, events: List[Dict[str, Any
]]) -> int:
364         if not events:
365             return 0
366         if self._buffer.buffer_mode == 'direct':
367             return self._writer.batch_insert_records([{'type':
'app_events_batch', 'data': events}])
368         self._buffer.enqueue({'type': 'app_events_batch', 'dat
a': events})
369         return len(events)
370
371     def save_app_param_changes_batch(self, param_changes: List
[Dict[str, Any]]) -> int:
372         if not param_changes:
373             return 0
374         if self._buffer.buffer_mode == 'direct':
375             return self._writer.batch_insert_records([{'type':
'app_param_changes_batch', 'data': param_changes}])
376         self._buffer.enqueue({'type': 'app_param_changes_batch
', 'data': param_changes})

```



```

377         return len(param_changes)
378
379     def save_custom_record(self, source_file: str, payload: Dict[str, Any], timestamp: Optional[str] = None) -> int:
380         rec = {'type': 'custom_record', 'source_file': source_file, 'payload': payload, 'timestamp': timestamp}
381         if self._buffer.buffer_mode == 'direct':
382             return self._writer.batch_insert_records([rec])
383         self._buffer.enqueue(rec)
384         return 1
385
386     def save_device_event(self, event: Dict[str, Any], timestamp: Optional[str] = None) -> int:
387         rec = {'type': 'device_event', 'data': event, 'timestamp': timestamp}
388         if self._buffer.buffer_mode == 'direct':
389             return self._writer.batch_insert_records([rec])
390         self._buffer.enqueue(rec)
391         return 1
392
393     def save_w64_event(self, event_data: Dict[str, Any], provider: str = 'w64_collector') -> int:
394         rec = {'type': 'w64_event', 'data': event_data, 'provider': provider}
395         if self._buffer.buffer_mode == 'direct':
396             return self._writer.batch_insert_records([rec])
397         self._buffer.enqueue(rec)
398         return 1
399
400     def save_process_provenance_event(self, event: Union[ProcessLifecycleEvent, ProcessProvenanceInfo, Dict[str, Any]]) -> bool:
401         ev_dict = event if isinstance(event, dict) else (event.model_dump() if hasattr(event, 'model_dump') else (vars(event) if hasattr(event, '__dict__') else event))
402         if self._buffer.buffer_mode == 'direct':
403             return self._writer.batch_insert_records([{'type': 'process_provenance_event', 'data': ev_dict}]) > 0

```

```

404         self._buffer.enqueue({'type': 'process_provenance_event', 'data': ev_dict})
405         return True
406
407     def save_process_provenance_batch(self, events: List[Union[ProcessLifecycleEvent, ProcessProvenanceInfo, Dict[str, Any]]]) -> int:
408         if not events:
409             return 0
410         ev_dicts = [e if isinstance(e, dict) else (e.model_dump() if hasattr(e, 'model_dump') else (vars(e) if hasattr(e, '__dict__') else e)) for e in events]
411         if self._buffer.buffer_mode == 'direct':
412             return self._writer.batch_insert_records([{'type': 'process_provenance_events_batch', 'data': ev_dicts}])
413         self._buffer.enqueue({'type': 'process_provenance_events_batch', 'data': ev_dicts})
414         return len(events)
415
416     def save_incident(self, incident: TelemetryIncident) -> bool:
417         return self._writer.batch_insert_records([{'type': 'incident', 'data': incident if isinstance(incident, dict) else (incident.model_dump() if hasattr(incident, 'model_dump') else (vars(incident) if hasattr(incident, '__dict__') else incident))}] > 0 or True
421
422     def save_system_rollup(self, rollup: SystemMetricRollup) -> bool:
423         return self._writer.batch_insert_records([{'type': 'system_rollup', 'data': rollup if isinstance(rollup, dict) else (rollup.model_dump() if hasattr(rollup, 'model_dump') else (vars(rollup) if hasattr(rollup, '__dict__') else rollup))}] > 0 or True
426
427

```

```

428     def save_reboot_session(self, session: Any) -> int:
429         return self._writer.batch_insert_records([
430             'type': 'reboot_session',
431             'data': session if isinstance(session, dict) else
432             (session.model_dump() if hasattr(session, 'model_dump') else (
433             vars(session) if hasattr(session, '__dict__') else session))
434             ])) or 1
435
436     def save_extended_audit(self, audit: Any, timestamp: Optional[str] = None) -> int:
437         return self._writer.batch_insert_records([
438             'type': 'extended_audit',
439             'data': audit if isinstance(audit, dict) else (audit.model_dump() if
440             hasattr(audit, 'model_dump') else (vars(audit) if
441             hasattr(audit, '__dict__') else audit)),
442             'timestamp': timestamp,
443             ])) or 1
444
445     # -----
446     # -----
447     # Делегирование методов чтения (Reader)
448     # -----
449     # -----
450
451     def get_snapshots(self, limit: int = 60, since_epoch: Optional[float] = None) -> List[Dict[str, Any]]:
452         return self._reader.get_snapshots(limit=limit, since_epoch=since_epoch)
453
454     def get_latest_snapshot_full(self) -> Optional[Dict[str, Any]]:
455         """Извлекает и десериализует последний системный снимок со всеми метаданными и процессами."""
456         return self._reader.get_latest_snapshot_full()
457
458     def get_latest_cpu(self) -> Optional[Dict[str, Any]]:
459         return self._reader.get_latest_cpu()
460

```

```
455     def get_cpu_history(self, limit: int = 60, since_epoch: Op-
| tional[float] = None) -> List[Dict[str, Any]]:
456         return self._reader.get_cpu_history(limit=limit, since
| _epoch=since_epoch)
457
458     def get_latest_storage_data(self) -> Dict[str, Any]:
459         return self._reader.get_latest_storage_data()
460
461     def get_snapshot_processes(self, snapshot_id: int) -> List
| [Dict[str, Any]]:
462         return self._reader.get_snapshot_processes(snapshot_id
| =snapshot_id)
463
464     def get_process_history(self, name: Optional[str] = None,
| pid: Optional[int] = None, limit: int = 100) -> List[Dict[str,
| Any]]:
465         return self._reader.get_process_history(name=name, pid
| =pid, limit=limit)
466
467     def get_process_provenance_history(self, **kwargs: Any) ->
| List[Dict[str, Any]]:
468         return self._reader.get_process_provenance_history(**k
| wargs)
469
470     def get_process_lineage(self, guid_or_pid: Union[str, int]
| ) -> List[Dict[str, Any]]:
471         return self._reader.get_process_lineage(guid_or_pid=gu
| id_or_pid)
472
473     def get_latest_processes(self, limit: int = 50, sort_by: s
| tr = 'cpu') -> List[Dict[str, Any]]:
474         return self._reader.get_latest_processes(limit=limit,
| sort_by=sort_by)
475
476     def get_process_stats(self, name: Optional[str] = None, li
| mit: int = 50) -> Dict[str, Any]:
477         return self._reader.get_process_stats(name=name, limit
| =limit)
```

```

478
479     def get_events(self, event_type: Optional[str] = None, limit: int = 100) -> List[Dict[str, Any]]:
480         return self._reader.get_events(event_type=event_type, limit=limit)
481
482     def get_sensor_history(self, sensor_id: Optional[str] = None, category: Optional[str] = None, limit: int = 100) -> List[Dict[str, Any]]:
483         return self._reader.get_sensor_history(sensor_id=sensor_id, category=category, limit=limit)
484
485     def get_latest_sensors(self, limit: Optional[int] = None) -> List[Dict[str, Any]]:
486         return self._reader.get_latest_sensors(limit=limit)
487
488     def get_cpu_hierarchy_metrics(self) -> Dict[str, Any]:
489         """Извлекает иерархические метрики процессора, ядер, потоков и сенсоров из базы телеметрии."""
490         return self._reader.get_cpu_hierarchy_metrics()
491
492     def get_app_polls(self, app: Optional[str] = None, metric_name: Optional[str] = None, limit: int = 100) -> List[Dict[str, Any]]:
493         return self._reader.get_app_polls(app=app, metric_name=metric_name, limit=limit)
494
495     def get_app_events(self, app: Optional[str] = None, event_type: Optional[str] = None, limit: int = 100) -> List[Dict[str, Any]]:
496         return self._reader.get_app_events(app=app, event_type=event_type, limit=limit)
497
498     def get_app_param_changes(self, app: Optional[str] = None, limit: int = 100) -> List[Dict[str, Any]]:
499         return self._reader.get_app_param_changes(app=app, limit=limit)
500

```

```
501     def export_app_polls_to_csv(self, app: Optional[str] = None, output_path: Optional[Union[str, Path]] = None, limit: int = 50000) -> Path:
502         return self._reader.export_app_polls_to_csv(app=app, output_path=output_path, limit=limit)
503
504     def export_app_events_to_csv(self, app: Optional[str] = None, output_path: Optional[Union[str, Path]] = None, limit: int = 50000) -> Path:
505         return self._reader.export_app_events_to_csv(app=app, output_path=output_path, limit=limit)
506
507     def export_app_param_changes_to_csv(self, app: Optional[str] = None, output_path: Optional[Union[str, Path]] = None, limit: int = 50000) -> Path:
508         return self._reader.export_app_param_changes_to_csv(app=app, output_path=output_path, limit=limit)
509
510     def get_custom_records(self, source_file: Optional[str] = None, limit: int = 100) -> List[Dict[str, Any]]:
511         return self._reader.get_custom_records(source_file=source_file, limit=limit)
512
513     def get_device_events(self, event_type: Optional[str] = None, device_instance_id: Optional[str] = None, limit: int = 100) -> List[Dict[str, Any]]:
514         return self._reader.get_device_events(event_type=event_type, device_instance_id=device_instance_id, limit=limit)
515
516     def get_device_install_date(self, device_instance_id: str) -> Optional[str]:
517         return self._reader.get_device_install_date(device_instance_id=device_instance_id)
518
519     def get_or_create_device_install_date(
520         self,
521         device_instance_id: str,
522         friendly_name: Optional[str] = None,
```

```

523         device_class: Optional[str] = None,
524         default_date: Optional[str] = None,
525     ) -> str:
526         """Возвращает существующую дату установки устройства и
ли сохраняет новую."""
527         existing = self._reader.get_device_install_date(device
_instance_id)
528         if existing:
529             return existing
530         date_to_save = default_date or datetime.now(timezone.u
tc).strftime('%d.%m.%Y %H:%M')
531         if not self.read_only:
532             try:
533                 with self._cm.lock, self._cm.get_connection()
as conn:
534                     cursor = conn.cursor()
535                     self._writer.upsert_device_inventory(
536                         cursor=cursor,
537                         device_instance_id=device_instance_id,
538                         friendly_name=friendly_name,
539                         device_class=device_class,
540                         install_date=date_to_save,
541                     )
542                     conn.commit()
543             except Exception as ex:
544                 logger.debug(f'Ошибка сохранения инвентаря уст
ройства: {ex}')
545         return date_to_save
546
547     def get_w64_events(self, event_type: Optional[str] = None,
provider: Optional[str] = None, limit: int = 100) -> List[Dic
t[str, Any]]:
548         return self._reader.get_w64_events(event_type=event_ty
pe, provider=provider, limit=limit)
549
550     def get_incidents(self, limit: int = 50, trigger_type: Opt
ional[str] = None, severity: Optional[str] = None) -> List[Dic
t[str, Any]]:

```

```
551         return self._reader.get_incidents(limit=limit, trigger
|_type=trigger_type, severity=severity)
552
553     def get_system_rollups(self, tier: str = '1m', start_epoch
|: Optional[float] = None, end_epoch: Optional[float] = None, l
|imit: int = 100) -> List[Dict[str, Any]]:
554         return self._reader.get_system_rollups(tier=tier, star
|t_epoch=start_epoch, end_epoch=end_epoch, limit=limit)
555
556     def get_reboot_history(self, limit: int = 50) -> List[Dict
|[str, Any]]:
557         return self._reader.get_reboot_history(limit=limit)
558
559     def get_latest_reboot_session(self) -> Optional[Dict[str,
|Any]]:
560         return self._reader.get_latest_reboot_session()
561
562     def get_extended_audits(self, limit: int = 50) -> List[Dic
|t[str, Any]]:
563         return self._reader.get_extended_audits(limit=limit)
564
565     def get_latest_extended_audit(self) -> Optional[Dict[str,
|Any]]:
566         return self._reader.get_latest_extended_audit()
567
568     def get_hardware_audits(self, limit: int = 50) -> List[Dic
|t[str, Any]]:
569         return self._reader.get_hardware_audits(limit=limit)
570
571     def get_latest_hardware_audit(self) -> Optional[Dict[str,
|Any]]:
572         return self._reader.get_latest_hardware_audit()
573
574     def get_startup_archives(self, limit: int = 50) -> List[Di
|ct[str, Any]]:
575         return self._reader.get_startup_archives(limit=limit)
576
577     def get_latest_startup_archive(self) -> Optional[Dict[str,
```



```

Any]]:
578         return self._reader.get_latest_startup_archive()
579
580     def get_startup_changes(self, limit: int = 100, archive_id
: Optional[str] = None) -> List[Dict[str, Any]]:
581         return self._reader.get_startup_changes(limit=limit, a
rchive_id=archive_id)
582
583     def get_telemetry_rollups(self, level: str = 'hourly', sen
sor_id: Optional[str] = None, start_epoch: Optional[float] = N
one, end_epoch: Optional[float] = None, limit: int = 100) -> L
ist[Dict[str, Any]]:
584         return self._reader.get_telemetry_rollups(level=level,
sensor_id=sensor_id, start_epoch=start_epoch, end_epoch=end_e
poch, limit=limit)
585
586     def get_telemetry_spikes(self, sensor_id: Optional[str] =
None, start_epoch: Optional[float] = None, limit: int = 100) -
> List[Dict[str, Any]]:
587         return self._reader.get_telemetry_spikes(sensor_id=sen
sor_id, start_epoch=start_epoch, limit=limit)
588
589     def get_telemetry_time_ranges(self) -> Dict[str, Any]:
590         """Возвращает временные диапазоны и доступные интервал
ы из БД телеметрии."""
591         return self._reader.get_telemetry_time_ranges()
592
593     def get_history_by_interval(self, interval: str = 'seconds
', metric: str = 'all', limit: int = 120) -> List[Dict[str, An
y]]:
594         """Возвращает исторические срезы снимков под указанный
интервал."""
595         return self._reader.get_history_by_interval(interval=i
nterval, metric=metric, limit=limit)
596
597     # -----
-----
598     # Делегирование методов хранилища накопителей (Storage Sub

```

```

system)
599     # -----
      -----
600
601     def save_disk_inventory(self, disk_data: Any) -> None:
602         """Сохранить или обновить паспорт физического накопите
        ля."""
603         rec = {'type': 'disk_inventory', 'data': disk_data}
604         if self._buffer.buffer_mode == 'direct':
605             self._writer.batch_insert_records([rec])
606             return
607         self._buffer.enqueue(rec)
608
609     def save_volume_inventory(self, volume_data: Any) -> None:
610         """Сохранить или обновить паспорт логического тома."""
611         rec = {'type': 'volume_inventory', 'data': volume_data
        }
612         if self._buffer.buffer_mode == 'direct':
613             self._writer.batch_insert_records([rec])
614             return
615         self._buffer.enqueue(rec)
616
617     def save_disk_health_snapshot(self, health_data: Any) -> N
one:
618         """Сохранить снимок здоровья / SMART / TBW накопителя.
        """
619         rec = {'type': 'disk_health', 'data': health_data}
620         if self._buffer.buffer_mode == 'direct':
621             self._writer.batch_insert_records([rec])
622             return
623         self._buffer.enqueue(rec)
624
625     def save_disk_performance_sample(self, perf_data: Any) ->
None:
626         """Сохранить сэмпл производительности диска."""
627         rec = {'type': 'disk_performance', 'data': perf_data}
628         if self._buffer.buffer_mode == 'direct':
629             self._writer.batch_insert_records([rec])

```

```

630         return
631         self._buffer.enqueue(rec)
632
633     def save_disk_io_event(self, io_event_data: Any) -> None:
634         """Сохранить событие прямого ввода-вывода."""
635         rec = {'type': 'disk_io_event', 'data': io_event_data}
636         if self._buffer.buffer_mode == 'direct':
637             self._writer.batch_insert_records([rec])
638             return
639         self._buffer.enqueue(rec)
640
641     def get_disk_inventory(self, disk_id: Optional[int] = None
642 ) -> List[Dict[str, Any]]:
643         """Извлечь паспортные данные физических дисков."""
644         return self._reader.get_disk_inventory(disk_id=disk_id
645 )
646
647     def get_volume_inventory(self) -> List[Dict[str, Any]]:
648         """Извлечь паспортные данные логических томов."""
649         return self._reader.get_volume_inventory()
650
651     def get_disk_health_history(
652         self,
653         disk_id: Optional[int] = None,
654         serial_number: Optional[str] = None,
655         limit: int = 100,
656         since_epoch: Optional[float] = None,
657     ) -> List[Dict[str, Any]]:
658         """Извлечь историю здоровья и SMART/TBW накопителей."""
659
660         return self._reader.get_disk_health_history(disk_id=di
661 sk_id, serial_number=serial_number, limit=limit, since_epoch=s
662 ince_epoch)
663
664     def get_disk_performance_samples(
665         self,
666         disk_name: Optional[str] = None,
667         limit: int = 100,

```

```

663         since_epoch: Optional[float] = None,
664     ) -> List[Dict[str, Any]]:
665         """Извлечь сэмплы производительности дисков."""
666         return self._reader.get_disk_performance_samples(disk_
name=disk_name, limit=limit, since_epoch=since_epoch)
667
668     def get_disk_io_events(
669         self,
670         pid: Optional[int] = None,
671         disk_id: Optional[int] = None,
672         limit: int = 100,
673         since_epoch: Optional[float] = None,
674     ) -> List[Dict[str, Any]]:
675         """Извлечь события ввода-вывода процессов."""
676         return self._reader.get_disk_io_events(pid=pid, disk_i
d=disk_id, limit=limit, since_epoch=since_epoch)
677
678     # -----
679     # Делегирование методов хранилища процессоров (CPU Subsystem)
680     # -----
681
682     def save_cpu_inventory(self, cpu_data: Any) -> None:
683         """Сохранить или обновить паспорт процессора."""
684         rec = {'type': 'cpu_inventory', 'data': cpu_data}
685         if self._buffer.buffer_mode == 'direct':
686             self._writer.batch_insert_records([rec])
687             return
688         self._buffer.enqueue(rec)
689
690     def save_cpu_telemetry_sample(self, sample_data: Any) -> N
one:
691         """Сохранить снимок телеметрии процессора."""
692         rec = {'type': 'cpu_telemetry', 'data': sample_data}
693         if self._buffer.buffer_mode == 'direct':
694             self._writer.batch_insert_records([rec])

```

```

695         return
696         self._buffer.enqueue(rec)
697
698     def get_cpu_inventory(self, processor_id: Optional[int] =
None) -> List[Dict[str, Any]]:
699         """Извлечь паспортные данные процессоров."""
700         return self._reader.get_cpu_inventory(processor_id=pro
cessor_id)
701
702     def get_cpu_telemetry_samples(
703         self,
704         processor_id: Optional[int] = None,
705         limit: int = 100,
706         since_epoch: Optional[float] = None,
707     ) -> List[Dict[str, Any]]:
708         """Извлечь историю метрик процессора."""
709         return self._reader.get_cpu_telemetry_samples(processo
r_id=processor_id, limit=limit, since_epoch=since_epoch)
710
711     # -----
712     # Делегирование методов хранилища памяти (RAM Module / SPD
Subsystem)
713     # -----
714
715     def save_ram_module_inventory(self, module_data: Any) -> N
one:
716         """Сохранить или обновить паспорт планки оперативной п
амяти."""
717         rec = {'type': 'ram_module_inventory', 'data': module_
data}
718         if self._buffer.buffer_mode == 'direct':
719             self._writer.batch_insert_records([rec])
720             return
721         self._buffer.enqueue(rec)
722
723     def save_ram_telemetry_sample(self, sample_data: Any) -> N

```

```

one:
724     """Сохранить снимок телеметрии оперативной памяти."""
725     rec = {'type': 'ram_telemetry', 'data': sample_data}
726     if self._buffer.buffer_mode == 'direct':
727         self._writer.batch_insert_records([rec])
728         return
729     self._buffer.enqueue(rec)
730
731     def get_ram_module_inventory(self) -> List[Dict[str, Any]]
732     :
733         """Извлечь паспортные данные физических планок оперативной памяти."""
734         return self._reader.get_ram_module_inventory()
735
736     def get_ram_telemetry_samples(
737     self,
738     limit: int = 100,
739     since_epoch: Optional[float] = None,
740     ) -> List[Dict[str, Any]]:
741         """Извлечь историю метрик оперативной памяти."""
742         return self._reader.get_ram_telemetry_samples(limit=limit, since_epoch=since_epoch)
743
744     # -----
745     # Делегирование методов хранилища графики (GPU Subsystem)
746     # -----
747
748     def save_gpu_inventory(self, gpu_data: Any) -> None:
749         """Сохранить или обновить паспорт графического ускорителя."""
750         rec = {'type': 'gpu_inventory', 'data': gpu_data}
751         if self._buffer.buffer_mode == 'direct':
752             self._writer.batch_insert_records([rec])
753             return
754         self._buffer.enqueue(rec)

```

```

755     def save_gpu_telemetry_sample(self, sample_data: Any) -> N
one:
756         """Сохранить снимок телеметрии графического ускорителя
. """
757         rec = {'type': 'gpu_telemetry', 'data': sample_data}
758         if self._buffer.buffer_mode == 'direct':
759             self._writer.batch_insert_records([rec])
760             return
761         self._buffer.enqueue(rec)
762
763     def get_gpu_inventory(self, gpu_id: Optional[int] = None)
-> List[Dict[str, Any]]:
764         """Извлечь паспортные данные GPU."""
765         return self._reader.get_gpu_inventory(gpu_id=gpu_id)
766
767     def get_gpu_telemetry_samples(
768         self,
769         gpu_id: Optional[int] = None,
770         name: Optional[str] = None,
771         limit: int = 100,
772         since_epoch: Optional[float] = None,
773     ) -> List[Dict[str, Any]]:
774         """Извлечь историю метрик GPU."""
775         return self._reader.get_gpu_telemetry_samples(gpu_id=g
pu_id, name=name, limit=limit, since_epoch=since_epoch)
776
777     # -----
-----
778     # Делегирование методов хранилища сети (Network Adapter Su
bsystem)
779     # -----
-----
780
781     def save_network_adapter_inventory(self, adapter_data: Any
) -> None:
782         """Сохранить или обновить паспорт сетевого адаптера."""
"
783         rec = {'type': 'network_adapter_inventory', 'data': ad

```

```

apter_data}
784         if self._buffer.buffer_mode == 'direct':
785             self._writer.batch_insert_records([rec])
786             return
787         self._buffer.enqueue(rec)
788
789     def save_network_adapter_sample(self, sample_data: Any) ->
None:
790         """Сохранить сэмпл показателей сетевого адаптера."""
791         rec = {'type': 'network_adapter_sample', 'data': sampl
e_data}
792         if self._buffer.buffer_mode == 'direct':
793             self._writer.batch_insert_records([rec])
794             return
795         self._buffer.enqueue(rec)
796
797     def get_network_adapter_inventory(self) -> List[Dict[str,
Any]]:
798         """Извлечь паспортные данные сетевых адаптеров."""
799         return self._reader.get_network_adapter_inventory()
800
801     def get_network_adapter_samples(
802         self,
803         adapter_name: Optional[str] = None,
804         limit: int = 100,
805         since_epoch: Optional[float] = None,
806     ) -> List[Dict[str, Any]]:
807         """Извлечь сэмплы показателей сетевых адаптеров."""
808         return self._reader.get_network_adapter_samples(adapte
r_name=adapter_name, limit=limit, since_epoch=since_epoch)
809
810
811     # -----
-----
812     # Делегирование методов обслуживания (Maintenance)
813     # -----
-----
814

```



```

815 |     def get_db_file_sizes(self) -> Dict[str, int]:
816 |         return self._maintenance.get_db_file_sizes()
817 |
818 |     def get_total_db_size_mb(self) -> float:
819 |         return self._maintenance.get_total_db_size_mb()
820 |
821 |     def checkpoint_wal(self) -> bool:
822 |         return self._maintenance.checkpoint_wal()
823 |
824 |     def vacuum(self) -> bool:
825 |         return self._maintenance.vacuum()
826 |
827 |     def get_storage_stats(self) -> Dict[str, Any]:
828 |         return self._maintenance.get_storage_stats(
829 |             buffered_count=self.get_buffered_count(),
830 |             buffer_mode=self.get_buffer_mode(),
831 |             buffer_size=self._buffer.buffer_size,
832 |             flush_interval_seconds=self._buffer.flush_interval
|_seconds,
833 |         )
834 |
835 |     get_stats = get_storage_stats
836 |
837 |     def cleanup_old_records(self, retention_days: Optional[int
| ] = None, vacuum_after: bool = False) -> int:
838 |         return self._maintenance.cleanup_old_records(retention
|_days=retention_days, vacuum_after=vacuum_after)
839 |
840 |     def enforce_size_limit(self, max_size_mb: Optional[float]
| = None, target_ratio: float = 0.85) -> Dict[str, Any]:
841 |         return self._maintenance.enforce_size_limit(max_size_m
|b=max_size_mb, target_ratio=target_ratio)
842 |
843 |     def prune_to_size(self, target_size_mb: float) -> Dict[str
| , Any]:
844 |         return self.enforce_size_limit(max_size_mb=target_size
|_mb, target_ratio=0.95)
845 |

```

```

846     def aggregate_process_metrics_2min(self, cutoff_seconds: i
      nt = 120, outlier_cpu_threshold: float = 30.0) -> Dict[str, in
      t]:
847         return self._maintenance.aggregate_process_metrics_2mi
      n(cutoff_seconds=cutoff_seconds, outlier_cpu_threshold=outlier
      _cpu_threshold)
848
849     def aggregate_process_metrics_daily(self, cutoff_days: int
      = 1) -> Dict[str, int]:
850         return self._maintenance.aggregate_process_metrics_dai
      ly(cutoff_days=cutoff_days)
851
852     # -----
      -----
853     # Делегирование методов SQL-агрегации сенсоров (Aggregator
      )
854     # -----
      -----
855
856     def aggregate_sensors(self, level: Union[AggregationLevel,
      str], start_epoch: Optional[float] = None, end_epoch: Optiona
      l[float] = None, detect_spikes: bool = True) -> Dict[str, Any]
      :
857         return self._aggregator.aggregate(level=level, start_e
      poch=start_epoch, end_epoch=end_epoch, detect_spikes=detect_sp
      ikes)
858
859     def aggregate_hourly(self, start_epoch: Optional[float] =
      None, end_epoch: Optional[float] = None, detect_spikes: bool =
      True) -> Dict[str, Any]:
860         return self._aggregator.aggregate_hourly(start_epoch=s
      tart_epoch, end_epoch=end_epoch, detect_spikes=detect_spikes)
861
862     def aggregate_daily(self, start_epoch: Optional[float] = N
      one, end_epoch: Optional[float] = None) -> Dict[str, Any]:
863         return self._aggregator.aggregate_daily(start_epoch=st
      art_epoch, end_epoch=end_epoch)
864

```

```

865     def aggregate_weekly(self, start_epoch: Optional[float] =
| None, end_epoch: Optional[float] = None) -> Dict[str, Any]:
866         return self._aggregator.aggregate_weekly(start_epoch=s
| tart_epoch, end_epoch=end_epoch)
867
868     def aggregate_monthly(self, start_epoch: Optional[float] =
| None, end_epoch: Optional[float] = None) -> Dict[str, Any]:
869         return self._aggregator.aggregate_monthly(start_epoch=
| start_epoch, end_epoch=end_epoch)
870
871     def aggregate_yearly(self, start_epoch: Optional[float] =
| None, end_epoch: Optional[float] = None) -> Dict[str, Any]:
872         return self._aggregator.aggregate_yearly(start_epoch=s
| tart_epoch, end_epoch=end_epoch)
873
874     def run_aggregation_pipeline(self, start_epoch: Optional[f
| loat] = None, end_epoch: Optional[float] = None, detect_spikes
| : bool = True) -> Dict[str, Any]:
875         return self._aggregator.run_pipeline(start_epoch=start
| _epoch, end_epoch=end_epoch, detect_spikes=detect_spikes)
876
877     def start_background_aggregator(self, interval_seconds: fl
| oat = 300.0) -> None:
878         """Запускает фоновый регламентный планировщик агрегаци
| и."""
879         self._aggregator.start_background_scheduler(interval_s
| econds=interval_seconds)
880
881     def stop_background_aggregator(self) -> None:
882         """Останавливает фоновый регламентный планировщик агре
| гации."""
883         self._aggregator.stop_background_scheduler()
884
885     # -----
| -----
886     # Делегирование методов подсистем рефакторинга: Data-First
| SQLite
887     # -----

```

```

-----
888
889     # 1. Windows Event Logs
890     def save_event_log_snapshot(self, snapshot_id: str, channels: List[Any], entries: List[Any], intelligence_profiles: Optional[List[Any]] = None) -> int:
891         return self._writer.save_event_log_snapshot(snapshot_id, channels, entries, intelligence_profiles)
892
893     def get_latest_event_log_channels(self) -> List[Dict[str, Any]]:
894         return self._reader.get_latest_event_log_channels()
895
896     def get_event_log_entries(self, channel: str = 'System', level: str = '', limit: int = 50, hours: int = 24) -> List[Dict[str, Any]]:
897         return self._reader.get_event_log_entries(channel=channel, level=level, limit=limit, hours=hours)
898
899     def get_latest_event_log_intelligence_profile(self, channel: str = 'System') -> Optional[Dict[str, Any]]:
900         return self._reader.get_latest_event_log_intelligence_profile(channel=channel)
901
902     def get_latest_event_log_report(self) -> Optional[Dict[str, Any]]:
903         return self._reader.get_latest_event_log_report()
904
905     # 2. Windows Firewall
906     def save_firewall_snapshot(self, snapshot_id: str, profile: Any, rules: List[Any]) -> int:
907         return self._writer.save_firewall_snapshot(snapshot_id, profile, rules)
908
909     def update_firewall_rule_state(self, rule_name: str, enabled: bool) -> bool:
910         return self._writer.update_firewall_rule_state(rule_name, enabled)

```

```

911
912     def get_latest_firewall_profiles(self) -> Optional[Dict[str, Any]]:
913         return self._reader.get_latest_firewall_profiles()
914
915     def get_latest_firewall_rules(self, direction: Optional[str] = None, limit: int = 200) -> List[Dict[str, Any]]:
916         return self._reader.get_latest_firewall_rules(direction=direction, limit=limit)
917
918     # 3. Windows Services
919     def save_services_snapshot(self, snapshot_id: str, services: List[Any]) -> int:
920         return self._writer.save_services_snapshot(snapshot_id, services)
921
922     def record_service_change(self, service_name: str, display_name: str, action: str, old_state: Optional[str] = None, new_state: Optional[str] = None, performed_by: str = 'SYSTEM', details: Optional[Dict[str, Any]] = None) -> int:
923         return self._writer.record_service_change(service_name, display_name, action, old_state, new_state, performed_by, details)
924
925     def get_latest_services_list(self, status: Optional[str] = None, limit: int = 500) -> List[Dict[str, Any]]:
926         return self._reader.get_latest_services_list(status=status, limit=limit)
927
928     def get_latest_services_report(self) -> Optional[Dict[str, Any]]:
929         return self._reader.get_latest_services_report()
930
931     def get_service_change_events(self, limit: int = 100) -> List[Dict[str, Any]]:
932         return self._reader.get_service_change_events(limit=limit)
933

```

```

934     # 4. Throttling & Power
935     def save_throttling_snapshot(self, snapshot_id: str, throttling_data: Any, thermal_zones: Optional[List[Any]] = None) -> int:
936         return self._writer.save_throttling_snapshot(snapshot_id, throttling_data, thermal_zones)
937
938     def get_latest_throttling_snapshot(self) -> Optional[Dict[str, Any]]:
939         return self._reader.get_latest_throttling_snapshot()
940
941     # 5. Forensics
942     def save_forensics_snapshot(self, snapshot_id: str, forensics_data: Any) -> int:
943         return self._writer.save_forensics_snapshot(snapshot_id, forensics_data)
944
945     def get_latest_forensics_snapshot(self) -> Optional[Dict[str, Any]]:
946         return self._reader.get_latest_forensics_snapshot()
947
948     # 6. Process Leaks
949     def save_process_leak_snapshot(self, snapshot_id: str, total_processes: Any, suspicious_count: int = 0, leak_items: Optional[List[Any]] = None) -> int:
950         return self._writer.save_process_leak_snapshot(snapshot_id, total_processes, suspicious_count, leak_items)
951
952     def get_latest_process_leak_report(self, limit: int = 20) -> Optional[Dict[str, Any]]:
953         return self._reader.get_latest_process_leak_report(limit=limit)
954
955     # 7. Defender
956     def save_defender_snapshot(self, snapshot_id: str, defender_status: Any, exclusions: Optional[List[Any]] = None, asr_rules: Optional[List[Any]] = None, threats: Optional[List[Any]] = None) -> int:

```

```

957         return self._writer.save_defender_snapshot(snapshot_id
, defender_status, exclusions, asr_rules, threats)
958
959     def get_latest_defender_status(self) -> Optional[Dict[str,
Any]]:
960         return self._reader.get_latest_defender_status()
961
962     def get_latest_defender_exclusions(self, limit: int = 200)
-> List[Dict[str, Any]]:
963         return self._reader.get_latest_defender_exclusions(lim
it=limit)
964
965     def get_latest_defender_asr_rules(self) -> List[Dict[str,
Any]]:
966         return self._reader.get_latest_defender_asr_rules()
967
968     def get_latest_defender_threats(self, limit: int = 50) ->
List[Dict[str, Any]]:
969         return self._reader.get_latest_defender_threats(limit=
limit)
970
971     def get_latest_defender_diagnostics(self) -> Dict[str, Any
]:
972         return self._reader.get_latest_defender_diagnostics()
973
974     # 8. Process Network & Process Manager
975     def save_process_network_snapshot(self, snapshot_id: str,
network_items: List[Any]) -> int:
976         return self._writer.save_process_network_snapshot(snap
shot_id, network_items)
977
978     def get_latest_process_network_activity(self, limit: int =
100, only_internet: bool = True) -> List[Dict[str, Any]]:
979         return self._reader.get_latest_process_network_activit
y(limit=limit, only_internet=only_internet)
980
981     # 9. Software Transparency
982     def save_software_inventory(self, apps: List[Any], storage

```

```

    _locations: Optional[List[Any]] = None, config_files: Optional
[List[Any]] = None, ai_research: Optional[List[Any]] = None) -
> int:
983         return self._writer.save_software_inventory(apps, stor
age_locations, config_files, ai_research)
984
985     def save_software_network_snapshots(self, snapshot_id: str
, net_items: List[Any]) -> int:
986         return self._writer.save_software_network_snapshots(sn
apshot_id, net_items)
987
988     def get_software_inventory_from_db(self, limit: int = 200,
offset: int = 0) -> List[Dict[str, Any]]:
989         return self._reader.get_software_inventory_from_db(lim
it=limit, offset=offset)
990
991     def get_software_app_details_from_db(self, app_id: str) ->
Optional[Dict[str, Any]]:
992         return self._reader.get_software_app_details_from_db(a
pp_id=app_id)
993
994     # 10. Security Events & Audit
995     def save_security_events(
996         self,
997         events: List[Any],
998         save_raw: bool = False,
999         raw_events: Optional[List[Any]] = None,
1000     ) -> int:
1001         """Сохраняет события безопасности Windows в SQLite."""
1002         return self._writer.save_security_events(events, save_
raw=save_raw, raw_events=raw_events)
1003
1004     def save_security_bookmark(
1005         self,
1006         channel: str,
1007         last_record_id: int,
1008         bookmark_xml: Optional[str] = None,
1009         last_timestamp: str = '',

```



```

1010         ) -> None:
1011             """Сохраняет состояние закладки сбора событий в SQLite
1012             ."""
1013             self._writer.save_security_bookmark(
1014                 channel=channel,
1015                 last_record_id=last_record_id,
1016                 bookmark_xml=bookmark_xml,
1017                 last_timestamp=last_timestamp,
1018             )
1019
1020     def get_security_events(
1021         self,
1022         event_id: Optional[int] = None,
1023         user: Optional[str] = None,
1024         process_name: Optional[str] = None,
1025         pid: Optional[int] = None,
1026         channel: Optional[str] = None,
1027         limit: int = 100,
1028         offset: int = 0,
1029         since_epoch: Optional[float] = None,
1030     ) -> List[Dict[str, Any]]:
1031         """Выборка нормализованных событий безопасности Window
1032         s."""
1033         return self._reader.get_security_events(
1034             event_id=event_id,
1035             user=user,
1036             process_name=process_name,
1037             pid=pid,
1038             channel=channel,
1039             limit=limit,
1040             offset=offset,
1041             since_epoch=since_epoch,
1042         )
1043
1044     def get_security_failed_logons(self, limit: int = 50, hour
1045         s: int = 24) -> List[Dict[str, Any]]:
1046         """Выборка неудачных попыток входа (Event 4625, 4771).
1047         """

```

```

1044         return self._reader.get_security_failed_logons(limit=limit, hours=hours)
1045
1046     def get_security_process_creations(
1047         self,
1048         process_name: Optional[str] = None,
1049         user: Optional[str] = None,
1050         limit: int = 50,
1051     ) -> List[Dict[str, Any]]:
1052         """Выборка запусков процессов (Event 4688) с командной
строкой."""
1053         return self._reader.get_security_process_creations(
1054             process_name=process_name,
1055             user=user,
1056             limit=limit,
1057         )
1058
1059     def get_security_bookmark(self, channel: str = 'Security')
-> Optional[Dict[str, Any]]:
1060         """Получить сохраненную закладку канала."""
1061         return self._reader.get_security_bookmark(channel=channel)
1062
1063     def get_security_stats(self) -> Dict[str, Any]:
1064         """Получить сводную статистику по событиям безопасност
и."""
1065         return self._reader.get_security_stats()
1066
1067     def save_power_events(self, events: List[Any]) -> int:
1068         """Сохраняет события питания в SQLite."""
1069         return self._writer.save_power_events(events)
1070
1071     def save_power_sessions(self, sessions: List[Any]) -> int:
1072         """Сохраняет реконструированные сессии питания в SQLit
е."""
1073         return self._writer.save_power_sessions(sessions)
1074
1075     def get_power_sessions(

```

```

1076         self,
1077         limit: int = 50,
1078         offset: int = 0,
1079         shutdown_type: Optional[str] = None,
1080         unexpected_only: bool = False,
1081         clean_only: bool = False,
1082     ) -> List[Dict[str, Any]]:
1083         """Извлечь список сессий питания операционной системы.
1084         """
1085         return self._reader.get_power_sessions(
1086             limit=limit,
1087             offset=offset,
1088             shutdown_type=shutdown_type,
1089             unexpected_only=unexpected_only,
1090             clean_only=clean_only,
1091         )
1092
1093     def get_power_session_by_id(self, session_id: str) -> Optional[Dict[str, Any]]:
1094         """Извлечь детальную информацию по конкретной сессии питания."""
1095         return self._reader.get_power_session_by_id(session_id=session_id)
1096
1097     def get_power_events(
1098         self,
1099         limit: int = 100,
1100         offset: int = 0,
1101         event_id: Optional[Union[int, List[int]]] = None,
1102         event_type: Optional[str] = None,
1103         hours: Optional[int] = None,
1104     ) -> List[Dict[str, Any]]:
1105         """Извлечь события питания и жизненного цикла."""
1106         return self._reader.get_power_events(
1107             limit=limit,
1108             offset=offset,
1109             event_id=event_id,
1110             event_type=event_type,

```

```

1110         hours=hours,
1111     )
1112
1113     def get_power_summary(self) -> Dict[str, Any]:
1114         """Получить сводную статистику по сессиям и событиям п
итания."""
1115         return self._reader.get_power_summary()
1116
1117     def get_process_pid_snapshot(self, pid: int) -> Optional[D
ict[str, Any]]:
1118         """Получить последний снимок метрик процесса по PID (<
5 мс)."""
1119         return self._reader.get_process_pid_snapshot(pid=pid)
1120
1121     def get_process_file_events(
1122         self,
1123         pid: Optional[int] = None,
1124         directory: Optional[str] = None,
1125         limit: int = 50
1126     ) -> List[Dict[str, Any]]:
1127         """Получить события файловой активности процесса."""
1128         return self._reader.get_process_file_events(pid=pid, d
irectory=directory, limit=limit)
1129
1130     def insert_process_pid_snapshots(self, snapshots: List[Uni
on[Dict[str, Any], Any]]) -> int:
1131         """Сохранить снимки метрик процессов по PID."""
1132         return self._writer.insert_process_pid_snapshots(snaps
hots=snapshots)
1133
1134     def insert_process_file_events(self, events: List[Union[Di
ct[str, Any], Any]]) -> int:
1135         """Сохранить события файловой активности."""
1136         return self._writer.insert_process_file_events(events=
events)
1137
1138
1139

```

1140 |

1141 |

1142 |

writer.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry Sqlite -
  Writer
4 # =====
5 # Description:
6 #   Класс TelemetryWriter для сериализации и записи телеметрии
  в SQLite.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.sqlite.writer import Telemet
  ryWriter
11 #
12 #       writer = TelemetryWriter(connection_manager)
13 #       saved = writer.batch_insert_records(records)
14 #
15 # File: writer.py
16 # Project: ai-breadboard
17 # Package: apps.windows.telemetry.sqlite
18 # Author: hypo69
19 # Copyright: © 2026 hypo69
20 # Updated: 2026-10-08 11:54:00
21 # =====
22
23 from __future__ import annotations
24
25 """Класс TelemetryWriter для сериализации, подготовки строк и
  транзакционной записи в SQLite."""
26
27 import json
28 import sqlite3
29 import time
```

```

30 from datetime import datetime, timezone
31 from pathlib import Path
32 from typing import Any, Dict, List, Optional, Union
33
34 from .connection import TelemetryConnectionManager
35 from ..models import (
36     HardwareArchiveEntry,
37     ProcessLifecycleEvent,
38     ProcessProvenanceInfo,
39     StartupArchiveEntry,
40     SystemMetricRollup,
41     SystemSnapshot,
42     TelemetryIncident,
43 )
44
45 try:
46     from logger import logger
47 except ImportError:
48     import logging
49     logger = logging.getLogger(__name__)
50
51
52 class TelemetryWriter:
53     """Выполняет пакетную и прямую вставку записей телеметрии
54     в базу данных SQLite."""
55
56     def __init__(self, connection_manager: TelemetryConnection
57     Manager) -> None:
58         """Инициализирует писатель телеметрии.
59
60         Args:
61             connection_manager: Менеджер соединений SQLite.
62
63         """
64         self._cm = connection_manager
65
66     def batch_insert_records(self, records: List[Dict[str, Any
67     ]]) -> int:
68         """Выполняет пакетную вставку разнородных записей в SQ

```

Lite в единой транзакции.

```
65 |
66 |     Args:
67 |         records: Список словарей записей.
68 |
69 |     Returns:
70 |         int: Общее число сохраненных записей.
71 |     """
72 |     if not records or self._cm.read_only:
73 |         return 0
74 |
75 |     saved_total = 0
76 |     with self._cm.lock, self._cm.get_connection() as conn:
77 |         cursor = conn.cursor()
78 |         sensor_poll_rows: List[tuple] = []
79 |         sensor_agg_rows: List[tuple] = []
80 |
81 |         for item in records:
82 |             rec_type = item.get('type')
83 |
84 |             if rec_type == 'snapshot':
85 |                 self.insert_snapshot_row(cursor, item)
86 |                 saved_total += 1
87 |
88 |             elif rec_type == 'sensor_poll':
89 |                 sensor_poll_rows.append(self.prepare_senso
90 | r_poll_row(item.get('data', {}), item.get('timestamp')))
91 |
92 |             elif rec_type == 'sensor_polls_batch':
93 |                 ts = item.get('timestamp')
94 |                 for s in item.get('data', []):
95 |                     sensor_poll_rows.append(self.prepare_s
96 | ensor_poll_row(s, ts))
97 |
98 |             elif rec_type == 'sensor_aggregates_batch':
99 |                 for a in item.get('data', []):
100 |                     sensor_agg_rows.append((
101 |                         a.get('sensor_id', ''),
```



```

100         a.get('period_start', ''),
101         a.get('period_end', ''),
102         a.get('avg', None),
103         a.get('min', None),
104         a.get('max', None),
105         a.get('count', 0),
106         json.dumps(a, ensure_ascii=False,
default=str),
107     ))
108
109     elif rec_type == 'hardware_archive':
110         self.insert_hardware_archive_row(cursor, item.get('data', {}))
111         saved_total += 1
112
113     elif rec_type == 'startup_archive':
114         self.insert_startup_archive_row(cursor, item.get('data', {}))
115         saved_total += 1
116
117     elif rec_type == 'event':
118         ts_str = item.get('timestamp') or datetime
.now(timezone.utc).isoformat()
119         try:
120             now_epoch = datetime.fromisoformat(ts_str).timestamp()
121         except Exception:
122             now_epoch = datetime.now(timezone.utc).timestamp()
123         details_str = json.dumps(item.get('details', {}), ensure_ascii=False, default=str)
124         cursor.execute('''
125             INSERT INTO telemetry_events (
126                 timestamp, created_at, event_type,
severity, event_details, raw_json
127             ) VALUES (?, ?, ?, ?, ?, ?)
128             ''', (ts_str, now_epoch, item.get('event_type', ''), item.get('severity', 'info'), details_str, details_

```

```
str))
129         saved_total += 1
130
131         elif rec_type == 'disk_inventory':
132             self.upsert_disk_inventory(cursor, item.get(
133                 'data') or item)
134             saved_total += 1
135
136         elif rec_type == 'volume_inventory':
137             self.upsert_volume_inventory(cursor, item.
138                 get('data') or item)
139             saved_total += 1
140
141         elif rec_type == 'disk_health':
142             self.insert_disk_health_snapshot(cursor, i
143                 tem.get('data') or item)
144             saved_total += 1
145
146         elif rec_type == 'disk_performance':
147             self.insert_disk_performance_sample(cursor
148                 , item.get('data') or item)
149             saved_total += 1
150
151         elif rec_type == 'disk_io_event':
152             self.insert_disk_io_event(cursor, item.get
153                 ('data') or item)
154             saved_total += 1
155
156         elif rec_type in ('cpu_inventory', 'cpu_inv'):
157             self.upsert_cpu_inventory(cursor, item.get
158                 ('data') or item)
159             saved_total += 1
160
161         elif rec_type in ('ram_module_inventory', 'ram
162             _inventory', 'ram_module'):
163             self.upsert_ram_module_inventory(cursor, i
164                 tem.get('data') or item)
165             saved_total += 1
```

```
158|
159|         elif rec_type in ('gpu_inventory', 'gpu_inv'):
160|             self.upsert_gpu_inventory(cursor, item.get
161| ('data') or item)
162|             saved_total += 1
163|
164|         elif rec_type in ('network_adapter_inventory',
165| 'net_inventory', 'net_adapter_inv'):
166|             self.upsert_network_adapter_inventory(curs
167| or, item.get('data') or item)
168|             saved_total += 1
169|
170|         elif rec_type in ('cpu_telemetry', 'cpu_sample
171| '):
172|             self.insert_cpu_telemetry_sample(cursor, i
173| tem.get('data') or item)
174|             saved_total += 1
175|
176|         elif rec_type in ('ram_telemetry', 'ram_sample
177| '):
178|             self.insert_ram_telemetry_sample(cursor, i
179| tem.get('data') or item)
180|             saved_total += 1
181|
182|         elif rec_type in ('gpu_telemetry', 'gpu_sample
183| '):
184|             self.insert_gpu_telemetry_sample(cursor, i
185| tem.get('data') or item)
186|             saved_total += 1
187|
188|         elif rec_type in ('network_adapter_sample', 'n
189| etwork_sample', 'net_sample'):
190|             self.insert_network_adapter_sample(cursor,
191| item.get('data') or item)
192|             saved_total += 1
193|
194|         elif rec_type == 'app_poll':
195|             cursor.execute(''
```

```

185         INSERT INTO app_polls (
186             timestamp, created_at, app, poll_t
ype, metric_name, value, unit, status, details, raw_json
187         ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?
)
188         '', self.prepare_app_poll_row(item.get('d
ata') or item))
189         saved_total += 1
190
191         elif rec_type == 'app_polls_batch':
192             rows = [self.prepare_app_poll_row(p) for p
in item.get('data', [])]
193             if rows:
194                 cursor.executemany(''
195                     INSERT INTO app_polls (
196                         timestamp, created_at, app, po
ll_type, metric_name, value, unit, status, details, raw_json
197                     ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?,
?, ?)
198                 '', rows)
199                 saved_total += len(rows)
200
201         elif rec_type == 'app_event':
202             cursor.execute(''
203                 INSERT INTO app_events (
204                     timestamp, created_at, app, event_
type, status, details, raw_json
205                 ) VALUES (?, ?, ?, ?, ?, ?, ?, ?)
206             '', self.prepare_app_event_row(item.get('
data') or item))
207             saved_total += 1
208
209         elif rec_type == 'app_events_batch':
210             rows = [self.prepare_app_event_row(ev) for
ev in item.get('data', [])]
211             if rows:
212                 cursor.executemany(''
213                     INSERT INTO app_events (

```

```

214         timestamp, created_at, app, ev
ent_type, status, details, raw_json
215         ) VALUES (?, ?, ?, ?, ?, ?, ?, ?)
216         '', rows)
217         saved_total += len(rows)
218
219         elif rec_type == 'app_param_change':
220             cursor.execute(''
221                 INSERT INTO app_param_changes (
222                     timestamp, created_at, app, param_
name, old_value, new_value, status, user, details, raw_json
223                     ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?
)
224             '', self.prepare_app_param_row(item.get('
data') or item))
225             saved_total += 1
226
227         elif rec_type == 'app_param_changes_batch':
228             rows = [self.prepare_app_param_row(pc) for
pc in item.get('data', [])]
229             if rows:
230                 cursor.executemany(''
231                     INSERT INTO app_param_changes (
232                         timestamp, created_at, app, pa
ram_name, old_value, new_value, status, user, details, raw_jso
n
233                         ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?,
?, ?)
234                     '', rows)
235             saved_total += len(rows)
236
237         elif rec_type == 'custom_record':
238             ts_str = item.get('timestamp') or datetime
.now(timezone.utc).isoformat()
239             try:
240                 now_epoch = datetime.fromisoformat(ts_
str).timestamp()
241             except Exception:

```

```

242         now_epoch = datetime.now(timezone.utc)
243         .timestamp()
244         payload_str = json.dumps(item.get('payload
245         ', {}), ensure_ascii=False, default=str)
246         cursor.execute('''
247             INSERT INTO custom_records (timestamp,
248             created_at, source_file, payload_json)
249             VALUES (?, ?, ?, ?)
250             ''', (ts_str, now_epoch, item.get('source_
251             file', ''), payload_str))
252             saved_total += 1
253
254         elif rec_type == 'device_event':
255             cursor.execute('''
256                 INSERT INTO device_events (
257                 timestamp, created_at, event_type,
258                 device_instance_id,
259                 friendly_name, device_class, categ
260                 ory, has_problem,
261                 problem_code, status_code, manufac
262                 turer,
263                 flapping_count, uptime_seconds, ra
264                 w_json
265                 ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?
266                 , ?, ?, ?, ?)
267                 ''', self.prepare_device_event_row(item.ge
268                 t('data', {}), item.get('timestamp')))
269                 saved_total += 1
270
271             elif rec_type == 'w64_event':
272                 cursor.execute('''
273                     INSERT INTO w64_events (
274                     event_id, timestamp, created_at, e
275                     vent_type, path, pid, name, provider, raw_json
276                     ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
277                     ''', self.prepare_w64_event_row(item.get('
278                     data', {}), item.get('provider', 'w64_collector')))
279                     saved_total += 1

```



```

291         saved_total += len(rows)
292
293         elif rec_type == 'incident':
294             inc_data = item.get('data', {})
295             now_dt = datetime.now(timezone.utc)
296             ts_str = inc_data.get('timestamp') or item
297             .get('timestamp') or now_dt.isoformat()
298             try:
299                 now_epoch = datetime.fromisoformat(ts_
300 str).timestamp()
301             except Exception:
302                 now_epoch = now_dt.timestamp()
303                 inc_id = inc_data.get('incident_id') or f"
304 INC-{int(now_epoch)}"
305                 cursor.execute('''
306                     INSERT OR REPLACE INTO incidents (
307                         incident_id, timestamp, created_at
308                         , trigger_type, severity, title,
309                         description, trigger_metrics_json,
310                         suspect_processes_json,
311                         related_events_json, metrics_summa
312 ry_json, raw_window_json
313                     ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?
314 , ?, ?)
315                 ''', (
316                     inc_id, ts_str, now_epoch,
317                     inc_data.get('trigger_type', ''),
318                     inc_data.get('severity', 'warning'),
319                     inc_data.get('title', ''),
320                     inc_data.get('description', ''),
321                     json.dumps(inc_data.get('trigger_metri
322 cs', {}), ensure_ascii=False, default=str),
323                     json.dumps(inc_data.get('suspect_proce
324 sses', []), ensure_ascii=False, default=str),
325                     json.dumps(inc_data.get('related_event
326 s', []), ensure_ascii=False, default=str),
327                     json.dumps(inc_data.get('metrics_summa
328 ry', {}), ensure_ascii=False, default=str),

```



```

318         json.dumps(inc_data.get('raw_window',
    | [], ensure_ascii=False, default=str),
319         ))
320         saved_total += 1
321
322         elif rec_type == 'system_rollup':
323             r_data = item.get('data', {})
324             p_start = r_data.get('period_start', '')
325             p_end = r_data.get('period_end', '')
326             try:
327                 p_start_epoch = datetime.fromisoformat
    | (p_start).timestamp()
328             except Exception:
329                 p_start_epoch = 0.0
330             try:
331                 p_end_epoch = datetime.fromisoformat(p
    | _end).timestamp()
332             except Exception:
333                 p_end_epoch = 0.0
334             cursor.execute('''
335                 INSERT INTO telemetry_rollups (
336                     period_start, period_end, period_s
    | tart_epoch, period_end_epoch,
337                     duration_seconds, sample_count, ti
    | er,
338                     cpu_avg, cpu_min, cpu_max, cpu_p95
    | ,
339                     ram_avg_gb, ram_min_gb, ram_max_gb
    | , ram_percent_avg, ram_percent_max,
340                     disk_read_avg_mbs, disk_read_max_m
    | bs, disk_read_total_mb,
341                     disk_write_avg_mbs, disk_write_max
    | _mbs, disk_write_total_mb,
342                     network_rx_avg_mbs, network_rx_max
    | _mbs, network_rx_total_mb,
343                     network_tx_avg_mbs, network_tx_max
    | _mbs, network_tx_total_mb,
344                     raw_json

```

```

345 |             ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?
346 |             , ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
347 |             ''' , (
348 |                 p_start, p_end, p_start_epoch, p_end_e
349 | poch,
350 |                 float(r_data.get('duration_seconds', 0
351 | .0) or 0.0),
352 |                 int(r_data.get('sample_count', 0) or 0
353 | ),
354 |                 str(r_data.get('tier', '1m')),
355 |                 r_data.get('cpu_avg'), r_data.get('cpu
356 | _min'), r_data.get('cpu_max'), r_data.get('cpu_p95'),
357 |                 r_data.get('ram_avg_gb'), r_data.get('
358 | ram_min_gb'), r_data.get('ram_max_gb'),
359 |                 r_data.get('ram_percent_avg'), r_data.
360 | get('ram_percent_max'),
361 |                 r_data.get('disk_read_avg_mbs'), r_dat
362 | a.get('disk_read_max_mbs'), r_data.get('disk_read_total_mb'),
363 |                 r_data.get('disk_write_avg_mbs'), r_da
364 | ta.get('disk_write_max_mbs'), r_data.get('disk_write_total_mb'
365 | ),
366 |                 r_data.get('network_rx_avg_mbs'), r_da
367 | ta.get('network_rx_max_mbs'), r_data.get('network_rx_total_mb'
368 | ),
369 |                 r_data.get('network_tx_avg_mbs'), r_da
370 | ta.get('network_tx_max_mbs'), r_data.get('network_tx_total_mb'
371 | ),
372 |                 json.dumps(r_data, ensure_ascii=False,
373 | default=str),
374 |             ))
375 |         saved_total += 1
376 |
377 |     elif rec_type == 'reboot_session':
378 |         s_data = item.get('data', {})
379 |         now_dt = datetime.now(timezone.utc)
380 |         boot_time = s_data.get('boot_time') or now
381 | _dt.isoformat()
382 |
383 |     try:

```

```

367         now_epoch = datetime.fromisoformat(boot_time).timestamp()
368     except Exception:
369         now_epoch = now_dt.timestamp()
370     cursor.execute('''
371         INSERT OR REPLACE INTO reboot_history
372         (
373             boot_id, boot_time, previous_boot_time, uptime_seconds, uptime_human,
374             shutdown_type, likely_class, conclusion, initiating_process, initiating_user,
375             shutdown_action, reason_text, reason_code, bugcheck_code, bugcheck_params_json,
376             power_button_timestamp, windows_update_kb, service_installed, evidence_json,
377             events_chain_json, raw_json, created_at
378             ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?,
379             , ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
380     ''', (
381         s_data.get('boot_id') or f"boot_{int(now_epoch)}",
382         boot_time,
383         s_data.get('previous_boot_time'),
384         float(s_data.get('uptime_seconds', 0.0) or 0.0),
385         s_data.get('uptime_human'),
386         s_data.get('shutdown_type', 'unknown')
387         ,
388         s_data.get('likely_class'),
389         s_data.get('conclusion'),
390         s_data.get('initiating_process'),
391         s_data.get('initiating_user'),
392         s_data.get('shutdown_action'),
393         s_data.get('reason_text'),
394         s_data.get('reason_code'),
395         s_data.get('bugcheck_code'),
396         json.dumps(s_data.get('bugcheck_params

```

```

    ', {}), ensure_ascii=False, default=str),
394         s_data.get('power_button_timestamp'),
395         s_data.get('windows_update_kb'),
396         s_data.get('service_installed'),
397         json.dumps(s_data.get('evidence', {})),
    ensure_ascii=False, default=str),
398         json.dumps(s_data.get('events_chain',
    []), ensure_ascii=False, default=str),
399         json.dumps(s_data, ensure_ascii=False,
    default=str),
400         now_epoch,
401     ))
402     saved_total += 1
403
404     elif rec_type == 'extended_audit':
405         audit_data = item.get('data', {})
406         now_dt = datetime.now(timezone.utc)
407         ts_str = item.get('timestamp') or audit_data.get('timestamp') or now_dt.isoformat()
408         try:
409             now_epoch = datetime.fromisoformat(ts_str).timestamp()
410         except Exception:
411             now_epoch = now_dt.timestamp()
412
413     def _get_f(obj: Any, key: str, default: Any = None) -> Any:
414         if isinstance(obj, dict):
415             return obj.get(key, default)
416         return getattr(obj, key, default)
417
418     def_obj = _get_f(audit_data, 'defender', {})
419     startup_obj = _get_f(audit_data, 'startup', {})
420     vss_obj = _get_f(audit_data, 'vss', {})
421     users_obj = _get_f(audit_data, 'users', {})
    )

```

```

422
423         cursor.execute('''
424             INSERT INTO system_extended_audits (
425                 timestamp, created_at, hostname,
426                 defender_cfa_enabled, defender_asr
427                 _count, defender_exclusions_count, defender_threats_count,
428                 startup_entries_count, vss_snapsho
429                 ts_count, users_total_count, users_admin_count,
430                 raw_json
431                 ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?
432                 , ?, ?)
433             ''', (
434                 ts_str, now_epoch, _get_f(audit_data,
435                 'hostname', ''),
436                 int(bool(_get_f(def_obj, 'cfa_enabled'
437                 , False))),
438                 int(_get_f(def_obj, 'asr_rules_count',
439                 0) or 0),
440                 int(_get_f(def_obj, 'exclusions_count'
441                 , len(_get_f(def_obj, 'path_exclusions', []) or [])) or 0),
442                 int(_get_f(def_obj, 'active_threats_co
443                 unt', 0) or 0),
444                 int(_get_f(startup_obj, 'total_entries
445                 ', len(_get_f(startup_obj, 'entries', []) or [])) or 0),
446                 int(_get_f(vss_obj, 'total_snapshots_c
447                 ount', len(_get_f(vss_obj, 'snapshots', []) or [])) or 0),
448                 int(_get_f(users_obj, 'total_users_cou
449                 nt', len(_get_f(users_obj, 'users', []) or [])) or 0),
450                 int(len(_get_f(users_obj, 'admin_usern
451                 ames', []) or []) or _get_f(users_obj, 'users_admin_count', 0)
452                 or 0),
453                 json.dumps(audit_data, ensure_ascii=Fa
454                 lse, default=str),
455                 ))
456         saved_total += 1
457
458     if sensor_poll_rows:
459         cursor.executemany('''

```

```

446         INSERT INTO sensor_polls (
447             sensor_id, timestamp, created_at, hard
ware_name, hardware_type,
448             sensor_category, sensor_name, unit, va
lue, provider, provider_priority, raw_json
449         ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?,
?)
450         ON CONFLICT(sensor_id, timestamp) DO UPDAT
E SET
451             created_at = excluded.created_at,
452             hardware_name = excluded.hardware_name
,
453             hardware_type = excluded.hardware_type
,
454             sensor_category = excluded.sensor_cate
gory,
455             sensor_name = excluded.sensor_name,
456             unit = excluded.unit,
457             value = excluded.value,
458             provider = excluded.provider,
459             provider_priority = excluded.provider_
priority,
460             raw_json = excluded.raw_json
'', sensor_poll_rows)
461         saved_total += len(sensor_poll_rows)
462
463
464         if sensor_agg_rows:
465             cursor.executemany(''
INSERT INTO sensor_aggregates_hourly (
466                 sensor_id, period_start, period_end, a
vg_value, min_value, max_value, count, raw_json
467             ) VALUES (?, ?, ?, ?, ?, ?, ?, ?)
'', sensor_agg_rows)
468         saved_total += len(sensor_agg_rows)
469
470
471         conn.commit()
472     return saved_total
473
474

```

```

475     def prepare_sensor_poll_row(self, s: Dict[str, Any], times
|tamp: Optional[str] = None) -> tuple:
476         """Подготавливает кортеж для вставки в sensor_polls."""
|
477         now_dt = datetime.now(timezone.utc)
478         ts_str = timestamp or s.get('timestamp') or now_dt.iso
|format()
479         try:
480             now_epoch = datetime.fromisoformat(ts_str).timesta
|mp()
481         except Exception:
482             now_epoch = now_dt.timestamp()
483             sid = str(s.get('id') or s.get('sensor_id') or '')
484             s_name = s.get('sensor_name') or s.get('name') or (f'S
|ensor #{sid}' if sid else 'Sensor')
485             cat = s.get('sensor_category') or s.get('category') or
|'General'
486
487             # Определение типа оборудования с fallback
488             hw_type = s.get('hardware_type')
489             if not hw_type:
490                 sid_l = sid.lower()
491                 cat_l = cat.lower()
492                 if 'disk' in sid_l or 'storage' in sid_l or 'stora
|ge' in cat_l:
493                     hw_type = 'storage'
494                 elif 'gpu' in sid_l or 'vram' in sid_l or 'graphic
|s' in sid_l:
495                     hw_type = 'gpu'
496                 elif 'ram' in sid_l or 'memory' in sid_l:
497                     hw_type = 'memory'
498                 elif 'net' in sid_l or 'nic' in sid_l or 'network'
|in cat_l:
499                     hw_type = 'network'
500                 else:
501                     hw_type = 'cpu'
502
503             hw_name = s.get('hardware_name') or ('Storage' if hw_t

```

```

    type == 'storage' else 'System')
504     unit = s.get('unit', '')
505     raw_val = s.get('value', s.get('value_num', 0.0))
506     try:
507         val_float = round(float(raw_val), 2)
508     except (ValueError, TypeError):
509         val_float = 0.0
510
511     provider_raw = s.get('_provider') or s.get('provider')
    or 'CIM_SYSTEM'
512     if hasattr(provider_raw, 'name'):
513         provider_str = provider_raw.name
514         provider_priority = provider_raw.value
515     elif isinstance(provider_raw, str):
516         provider_str = provider_raw.upper()
517         try:
518             from ..sensor_registry import SensorProvider
519             provider_priority = SensorProvider[provider_ra
w.upper()].value
520         except (KeyError, ImportError):
521             provider_priority = 30
522     else:
523         provider_str = 'CIM_SYSTEM'
524         provider_priority = 30
525
526     raw_json_str = s.get('raw_json')
527     if not raw_json_str:
528         try:
529             raw_json_str = json.dumps(s, ensure_ascii=False
e, default=str)
530         except Exception:
531             raw_json_str = None
532
533     return (sid, ts_str, now_epoch, hw_name, hw_type, cat,
s_name, unit, val_float, provider_str, provider_priority, raw
_json_str)
534
535     def prepare_app_poll_row(self, p: Dict[str, Any]) -> tuple

```



```

:
536     now_dt = datetime.now(timezone.utc)
537     ts_str = p.get('timestamp') or now_dt.isoformat()
538     try:
539         now_epoch = datetime.fromisoformat(ts_str).timestamp()
540     except Exception:
541         now_epoch = now_dt.timestamp()
542     app = p.get('app', 'unknown')
543     poll_type = p.get('poll_type', 'poll')
544     metric_name = p.get('metric_name', 'metric')
545     val = p.get('value')
546     try:
547         val_num = float(val) if val is not None and not isinstance(val, (dict, list)) else None
548     except (ValueError, TypeError):
549         val_num = None
550     unit = p.get('unit', '')
551     status = p.get('status', 'OK')
552     details = p.get('details', '')
553     details_str = json.dumps(details, ensure_ascii=False)
554     if isinstance(details, (dict, list)) else str(details or '')
555     raw_json = json.dumps({'app': app, 'poll_type': poll_type, 'metric_name': metric_name, 'value': val, 'unit': unit, 'status': status, 'details': details}, ensure_ascii=False)
556     return (ts_str, now_epoch, app, poll_type, metric_name, val_num, unit, status, details_str, raw_json)
557
558 def prepare_app_event_row(self, ev: Dict[str, Any]) -> tuple:
559     now_dt = datetime.now(timezone.utc)
560     ts_str = ev.get('timestamp') or now_dt.isoformat()
561     try:
562         now_epoch = datetime.fromisoformat(ts_str).timestamp()
563     except Exception:
564         now_epoch = now_dt.timestamp()
565     app = ev.get('app', 'unknown')

```

```

565         event_type = ev.get('event_type', 'event')
566         status = ev.get('status', 'OK')
567         details = ev.get('details', '')
568         details_str = json.dumps(details, ensure_ascii=False)
569         if isinstance(details, (dict, list)) else str(details or '')
570         raw_json = json.dumps({'app': app, 'event_type': event
571                                _type, 'status': status, 'details': details}, ensure_ascii=False)
572         return (ts_str, now_epoch, app, event_type, status, de
573                tails_str, raw_json)
574
575     def prepare_app_param_row(self, pc: Dict[str, Any]) -> tuple:
576
577         now_dt = datetime.now(timezone.utc)
578         ts_str = pc.get('timestamp') or now_dt.isoformat()
579         try:
580             now_epoch = datetime.fromisoformat(ts_str).timestamp()
581         except Exception:
582             now_epoch = now_dt.timestamp()
583         app = pc.get('app', 'unknown')
584         param_name = pc.get('param_name', 'param')
585         old_val = pc.get('old_value')
586         new_val = pc.get('new_value')
587         old_str = json.dumps(old_val, ensure_ascii=False) if i
588         isinstance(old_val, (dict, list)) else str(old_val or '')
589         new_str = json.dumps(new_val, ensure_ascii=False) if i
590         isinstance(new_val, (dict, list)) else str(new_val or '')
591         status = pc.get('status', 'SUCCESS')
592         user = pc.get('user', 'system')
593         details = pc.get('details', '')
594         details_str = json.dumps(details, ensure_ascii=False)
595         if isinstance(details, (dict, list)) else str(details or '')
596         raw_json = json.dumps({'app': app, 'param_name': param
597                                _name, 'old_value': old_val, 'new_value': new_val, 'status': s
598                                tatus, 'user': user, 'details': details}, ensure_ascii=False)
599         return (ts_str, now_epoch, app, param_name, old_str, n
600                ew_str, status, user, details_str, raw_json)

```

```

591
592     def prepare_device_event_row(self, event: Dict[str, Any],
timestamp: Optional[str] = None) -> tuple:
593         now_dt = datetime.now(timezone.utc)
594         ts_str = timestamp or event.get('timestamp') or now_dt
.isoformat()
595         try:
596             now_epoch = datetime.fromisoformat(ts_str).timesta
mp()
597         except Exception:
598             now_epoch = now_dt.timestamp()
599         raw_json = json.dumps(event, ensure_ascii=False, defau
lt=str)
600         return (
601             ts_str, now_epoch, event.get('event_type', ''), ev
ent.get('device_instance_id', ''),
602             event.get('friendly_name', ''), event.get('device_
class', ''), event.get('category', ''),
603             int(bool(event.get('has_problem', False))), event.
get('problem_code', 0),
604             event.get('status_code', 0), event.get('manufactur
er', ''),
605             event.get('flapping_count_in_window', 0), event.ge
t('uptime_seconds', 0.0), raw_json,
606         )
607
608     def prepare_w64_event_row(self, event_data: Dict[str, Any]
, provider: str = 'w64_collector') -> tuple:
609         now_dt = datetime.now(timezone.utc)
610         ts_str = event_data.get('timestamp') or now_dt.isoform
at()
611         try:
612             now_epoch = datetime.fromisoformat(ts_str).timesta
mp()
613         except Exception:
614             now_epoch = now_dt.timestamp()
615         event_id = str(event_data.get('event_id') or f'evt_{in
t(now_epoch)})')

```

```

616         event_type = str(event_data.get('event_type', 'unknown
        '))
617         path_val = event_data.get('path')
618         pid_val = event_data.get('pid')
619         if pid_val is not None:
620             try:
621                 pid_val = int(pid_val)
622             except (ValueError, TypeError):
623                 pid_val = None
624         name_val = event_data.get('name')
625         raw_json = json.dumps(event_data, ensure_ascii=False,
        default=str)
626         return (event_id, ts_str, now_epoch, event_type, path_
        val, pid_val, name_val, provider, raw_json)
627
628     def prepare_process_provenance_row(self, p: Any) -> tuple:
629         """Подготавливает кортеж для вставки в process_provena
        nce_events."""
630         p_dict = p if isinstance(p, dict) else (p.model_dump()
        if hasattr(p, 'model_dump') else (vars(p) if hasattr(p, '__di
        ct__') else p))
631         now_dt = datetime.now(timezone.utc)
632         ts_str = p_dict.get('timestamp') or now_dt.isoformat()
633         try:
634             now_epoch = datetime.fromisoformat(ts_str).timesta
        mp()
635         except Exception:
636             now_epoch = now_dt.timestamp()
637
638         pid = int(p_dict.get('pid') or 0)
639         event_id = p_dict.get('event_id') or f"pp_{int(now_epo
        ch * 1000)}_{pid}"
640         event_type = p_dict.get('event_type') or 'ProcessCreat
        ed'
641         ppid = int(p_dict['ppid']) if p_dict.get('ppid') is no
        t None else None
642         name = p_dict.get('name') or 'unknown'
643         exe_path = p_dict.get('executable_path') or p_dict.get

```

```

    ('executable')
644     cmdline = p_dict.get('command_line') or p_dict.get('cm
    dline')
645     user = p_dict.get('user') or p_dict.get('username')
646     sid = p_dict.get('sid')
647     session_id = int(p_dict['session_id']) if p_dict.get('
    session_id') is not None else None
648     integrity = p_dict.get('integrity_level')
649     elevation = int(bool(p_dict.get('elevation', False)))
650     parent_name = p_dict.get('parent_name')
651     parent_cmdline = p_dict.get('parent_cmdline')
652     ancestor_chain = p_dict.get('ancestor_chain') or p_dic
    t.get('ancestor_chain_str')
653     if isinstance(ancestor_chain, list):
654         ancestor_chain = ' -> '.join(ancestor_chain)
655     launch_reason = p_dict.get('launch_reason') or p_dict.
    get('why_was_it_created')
656     source = p_dict.get('source') or 'system'
657     creation_time = p_dict.get('creation_time') or ts_str
658     process_guid = p_dict.get('process_guid') or f"proc_{p
    id}_{creation_time}"
659     parent_guid = p_dict.get('parent_guid')
660     details = p_dict.get('details') or p_dict.get('details
    _json')
661     details_json = json.dumps(details, ensure_ascii=False)
    if isinstance(details, (dict, list)) else (str(details) if de
    tails else None)
662     raw_json = json.dumps(p_dict, ensure_ascii=False, defa
    ult=str)
663
664     return (
665         event_id, ts_str, now_epoch, event_type, process_g
    uid, parent_guid,
666         pid, ppid, name, exe_path, cmdline, user, sid, ses
    sion_id,
667         integrity, elevation, parent_name, parent_cmdline,
668         ancestor_chain, launch_reason, source, details_jso
    n, raw_json

```

```

669         )
670
671     def insert_snapshot_row(self, cursor: sqlite3.Cursor, item
: Dict[str, Any]) -> int:
672         """Вставляет строку снимка и процессы в базу данных со
всеми расширенными параметрами."""
673         snap_data = item.get('data', {})
674         top_n = item.get('top_n', 20)
675         snapshot = snap_data if isinstance(snap_data, SystemSn
apshot) else None
676
677         if snapshot:
678             hostname = snapshot.hostname
679             username = snapshot.username or ''
680             uptime = snapshot.uptime_seconds
681             os_name = snapshot.os_name or 'Windows'
682             os_build = str(snapshot.os_build or '')
683             os_install_date = str(snapshot.os_install_date or
'')
684             system_lang = str(snapshot.system_language or '')
685             os_install_lang = str(snapshot.os_install_language
or '')
686             user_locale = str(snapshot.user_locale or '')
687             sys_locale = str(snapshot.system_locale or '')
688             timezone_val = str(snapshot.timezone or '')
689             codepage_val = str(snapshot.codepage or '')
690             input_langs = snapshot.input_languages or []
691             input_langs_json = json.dumps(input_langs, ensure_
ascii=False) if input_langs else None
692
693             cpu_pct = snapshot.cpu.total_percent if snapshot.c
pu else 0.0
694             cpu_freq = snapshot.cpu.frequency_mhz if snapshot.
cpu else 0.0
695             mem_tot = snapshot.memory.total_gb if snapshot.mem
ory else 0.0
696             mem_used = snapshot.memory.used_gb if snapshot.mem
ory else 0.0

```

```

697         mem_pct = snapshot.memory.percent if snapshot.memo
ry else 0.0
698         swap_pct = snapshot.memory.swap_percent if snapsho
t.memory else 0.0
699         disk_rb = snapshot.disk_io.read_bytes_per_sec if s
napshot.disk_io else 0.0
700         disk_wb = snapshot.disk_io.write_bytes_per_sec if
snapshot.disk_io else 0.0
701         disk_rc = snapshot.disk_io.read_count_per_sec if s
napshot.disk_io else 0.0
702         disk_wc = snapshot.disk_io.write_count_per_sec if
snapshot.disk_io else 0.0
703         net_sent = sum(getattr(i, 'bytes_sent_sec', 0.0) o
r 0.0 for i in (snapshot.network or []))
704         net_recv = sum(getattr(i, 'bytes_recv_sec', 0.0) o
r 0.0 for i in (snapshot.network or []))
705         gpu_load = 0.0
706         gpu_temp = 0.0
707         if snapshot.gpus:
708             gpu = snapshot.gpus[0]
709             gpu_load = getattr(gpu, 'load_percent', None)
or getattr(gpu, 'utilization_gpu_pct', 0.0) or 0.0
710             gpu_temp = getattr(gpu, 'temperature_celsius',
None) or getattr(gpu, 'temperature_gpu_c', 0.0) or 0.0
711             raw_procs = snapshot.top_processes or []
712
713         disks_list = [d if isinstance(d, dict) else (d.mod
el_dump() if hasattr(d, 'model_dump') else (vars(d) if ha
sattr(d, '__dict__') else d)) for d in (snapshot.disks or [])]
714         disks_json = json.dumps(disks_list, ensure_ascii=F
alse, default=str) if disks_list else None
715         phys_disks_list = [d if isinstance(d, dict) else (
d.model_dump() if hasattr(d, 'model_dump') else (vars(d) if ha
sattr(d, '__dict__') else d)) for d in (snapshot.physical_disk
s or [])]
716         physical_disks_json = json.dumps(phys_disks_list,
ensure_ascii=False, default=str) if phys_disks_list else None
717

```

```
718         monitors_list = [m if isinstance(m, dict) else (m.
| model_dump() if hasattr(m, 'model_dump') else (vars(m) if hasa
| ttr(m, '__dict__') else m)) for m in (snapshot.monitors or [])
| ]
719         monitors_json = json.dumps(monitors_list, ensure_a
| scii=False, default=str) if monitors_list else None
720
721         updates_obj = snapshot.updates.model_dump() if has
| attr(snapshot.updates, 'model_dump') else (snapshot.updates if
| isinstance(snapshot.updates, dict) else (vars(snapshot.update
| s) if hasattr(snapshot.updates, '__dict__') else None))
722         updates_json = json.dumps(updates_obj, ensure_asci
| i=False, default=str) if updates_obj else None
723
724         office_obj = snapshot.office.model_dump() if hasat
| tr(snapshot.office, 'model_dump') else (snapshot.office if isi
| nstance(snapshot.office, dict) else (vars(snapshot.office) if
| hasattr(snapshot.office, '__dict__') else None))
725         office_json = json.dumps(office_obj, ensure_ascii=
| False, default=str) if office_obj else None
726
727         onedrive_obj = snapshot.onedrive.model_dump() if h
| asattr(snapshot.onedrive, 'model_dump') else (snapshot.onedriv
| e if isinstance(snapshot.onedrive, dict) else (vars(snapshot.o
| nedrive) if hasattr(snapshot.onedrive, '__dict__') else None))
728         onedrive_json = json.dumps(onedrive_obj, ensure_as
| cii=False, default=str) if onedrive_obj else None
729
730         battery_obj = snapshot.battery.model_dump() if has
| attr(snapshot.battery, 'model_dump') else (snapshot.battery if
| isinstance(snapshot.battery, dict) else (vars(snapshot.batter
| y) if hasattr(snapshot.battery, '__dict__') else None))
731         battery_json = json.dumps(battery_obj, ensure_asci
| i=False, default=str) if battery_obj else None
732
733         ram_sticks_list = [r if isinstance(r, dict) else (
| r.model_dump() if hasattr(r, 'model_dump') else (vars(r) if ha
| sattr(r, '__dict__') else r)) for r in (snapshot.ram_sticks or
```



```

    []])
734         ram_sticks_json = json.dumps(ram_sticks_list, ensu
re_ascii=False, default=str) if ram_sticks_list else None
735
736         ports_list = [p if isinstance(p, dict) else (p.mod
el_dump() if hasattr(p, 'model_dump') else (vars(p) if hasattr
(p, '__dict__') else p)) for p in (snapshot.listening_ports or
    [])]
737         listening_ports_json = json.dumps(ports_list, ensu
re_ascii=False, default=str) if ports_list else None
738
739         alerts_list = [a if isinstance(a, dict) else (a.mo
del_dump() if hasattr(a, 'model_dump') else (vars(a) if hasatt
r(a, '__dict__') else a)) for a in (snapshot.alerts or [])]
740         alerts_json = json.dumps(alerts_list, ensure_ascii
=False, default=str) if alerts_list else None
741
742         ts_str = snapshot.timestamp
743         hw_audit = getattr(snapshot, 'hardware_audit', Non
e)
744         raw_full_dict = snapshot.model_dump() if hasattr(s
napshot, 'model_dump') else (vars(snapshot) if hasattr(snapshot,
 '__dict__') else snapshot)
745         raw_json_str = json.dumps(raw_full_dict, ensure_as
cii=False, default=str)
746         else:
747             hostname = snap_data.get('hostname')
748             username = snap_data.get('username') or ''
749             uptime = snap_data.get('uptime_seconds')
750             os_name = snap_data.get('os_name') or 'Windows'
751             os_build = str(snap_data.get('os_build') or '')
752             os_install_date = str(snap_data.get('os_install_da
te') or '')
753             system_lang = str(snap_data.get('system_language')
or '')
754             os_install_lang = str(snap_data.get('os_install_la
nguage') or '')
755             user_locale = str(snap_data.get('user_locale') or

```

```

756 |         sys_locale = str(snap_data.get('system_locale') or
757 |         '')
758 |         timezone_val = str(snap_data.get('timezone') or '')
759 |         codepage_val = str(snap_data.get('codepage') or '')
760 |         input_langs = snap_data.get('input_languages', [])
761 |         input_langs_json = json.dumps(input_langs, ensure_
762 |         ascii=False) if input_langs else None
763 |         disks_val = snap_data.get('disks', [])
764 |         disks_list = [d if isinstance(d, dict) else (d.mod
765 |         el_dump() if hasattr(d, 'model_dump') else (vars(d) if hasattr
766 |         (d, '__dict__') else d)) for d in disks_val] if isinstance(dis
767 |         ks_val, list) else []
768 |         disks_json = json.dumps(disks_list, ensure_ascii=F
769 |         alse, default=str) if disks_list else None
770 |         phys_disks_val = snap_data.get('physical_disks', [
771 |         ])
772 |         phys_disks_list = [d if isinstance(d, dict) else (
773 |         d.model_dump() if hasattr(d, 'model_dump') else (vars(d) if ha
774 |         sattr(d, '__dict__') else d)) for d in phys_disks_val] if isin
775 |         stance(phys_disks_val, list) else []
776 |         physical_disks_json = json.dumps(phys_disks_list,
777 |         ensure_ascii=False, default=str) if phys_disks_list else None
778 |         mon_val = snap_data.get('monitors', [])
779 |         monitors_json = json.dumps(mon_val, ensure_ascii=F
780 |         alse, default=str) if mon_val else None
781 |         upd_val = snap_data.get('updates')
782 |         updates_json = json.dumps(upd_val, ensure_ascii=Fa
783 |         lse, default=str) if upd_val else None
784 |         off_val = snap_data.get('office')
785 |         office_json = json.dumps(off_val, ensure_ascii=Fal
786 |         se, default=str) if off_val else None

```

```

777
778         od_val = snap_data.get('onedrive')
779         onedrive_json = json.dumps(od_val, ensure_ascii=False,
    | default=str) if od_val else None
780
781         bat_val = snap_data.get('battery')
782         battery_json = json.dumps(bat_val, ensure_ascii=False,
    | default=str) if bat_val else None
783
784         ram_val = snap_data.get('ram_sticks', [])
785         ram_sticks_json = json.dumps(ram_val, ensure_ascii
    | =False, default=str) if ram_val else None
786
787         ports_val = snap_data.get('listening_ports', [])
788         listening_ports_json = json.dumps(ports_val, ensur
    | e_ascii=False, default=str) if ports_val else None
789
790         alerts_val = snap_data.get('alerts', [])
791         alerts_json = json.dumps(alerts_val, ensure_ascii=
    | False, default=str) if alerts_val else None
792
793         cpu_obj = snap_data.get('cpu', {})
794         cpu_pct = cpu_obj.get('total_percent', 0.0) if isi
    | nstance(cpu_obj, dict) else getattr(cpu_obj, 'total_percent',
    | 0.0)
795         cpu_freq = cpu_obj.get('frequency_mhz', 0.0) if is
    | instance(cpu_obj, dict) else getattr(cpu_obj, 'frequency_mhz',
    | 0.0)
796         mem_obj = snap_data.get('memory', {})
797         mem_tot = mem_obj.get('total_gb', 0.0) if isinstan
    | ce(mem_obj, dict) else getattr(mem_obj, 'total_gb', 0.0)
798         mem_used = mem_obj.get('used_gb', 0.0) if isinstan
    | ce(mem_obj, dict) else getattr(mem_obj, 'used_gb', 0.0)
799         mem_pct = mem_obj.get('percent', 0.0) if isinstanc
    | e(mem_obj, dict) else getattr(mem_obj, 'percent', 0.0)
800         swap_pct = mem_obj.get('swap_percent', 0.0) if isi
    | nstance(mem_obj, dict) else getattr(mem_obj, 'swap_percent', 0
    | .0)

```

```

801         disk_obj = snap_data.get('disk_io', {})
802         disk_rb = disk_obj.get('read_bytes_per_sec', 0.0)
803         if isinstance(disk_obj, dict) else getattr(disk_obj, 'read_bytes_per_sec', 0.0)
804         disk_wb = disk_obj.get('write_bytes_per_sec', 0.0)
805         if isinstance(disk_obj, dict) else getattr(disk_obj, 'write_bytes_per_sec', 0.0)
806         disk_rc = disk_obj.get('read_count_per_sec', 0.0)
807         if isinstance(disk_obj, dict) else getattr(disk_obj, 'read_count_per_sec', 0.0)
808         disk_wc = disk_obj.get('write_count_per_sec', 0.0)
809         if isinstance(disk_obj, dict) else getattr(disk_obj, 'write_count_per_sec', 0.0)
810         net_list = snap_data.get('network', [])
811         net_sent = sum((n.get('bytes_sent_sec', 0.0) if isinstance(n, dict) else getattr(n, 'bytes_sent_sec', 0.0)) or 0.0 for n in net_list)
812         net_recv = sum((n.get('bytes_recv_sec', 0.0) if isinstance(n, dict) else getattr(n, 'bytes_recv_sec', 0.0)) or 0.0 for n in net_list)
813         gpus = snap_data.get('gpus', [])
814         gpu_load = 0.0
815         gpu_temp = 0.0
816         if gpus:
817             g0 = gpus[0]
818             gpu_load = (g0.get('load_percent', 0.0) if isinstance(g0, dict) else getattr(g0, 'load_percent', 0.0)) or 0.0
819             gpu_temp = (g0.get('temperature_celsius', 0.0) if isinstance(g0, dict) else getattr(g0, 'temperature_celsius', 0.0)) or 0.0
820         raw_procs = snap_data.get('top_processes', [])
821         ts_str = snap_data.get('timestamp')
822         hw_audit = snap_data.get('hardware_audit')
823         raw_json_str = json.dumps(snap_data, ensure_ascii=False, default=str)
824         now_dt = datetime.now(timezone.utc)

```

```
822         ts_str = ts_str or now_dt.isoformat()
823         try:
824             now_epoch = datetime.fromisoformat(ts_str).timestamp()
825         except Exception:
826             now_epoch = now_dt.timestamp()
827
828         hardware_audit_json = json.dumps(hw_audit or {}, ensure_ascii=False, default=str)
829
830         cursor.execute('''
831             INSERT INTO system_snapshots (
832                 timestamp, created_at, hostname, username, uptime_seconds, os_name, os_build, os_install_date,
833                 system_language, os_install_language, user_locale, system_locale, timezone, codepage,
834                 input_languages_json, disks_json, physical_disks_json, monitors_json, updates_json, office_json,
835                 onedrive_json, battery_json, ram_sticks_json, listening_ports_json, alerts_json,
836                 cpu_total_percent, cpu_frequency_mhz, memory_total_gb, memory_used_gb,
837                 memory_percent, swap_percent, gpu_load_percent, gpu_temp_c,
838                 disk_read_bytes_sec, disk_write_bytes_sec, disk_read_count_sec,
839                 disk_write_count_sec, network_sent_bytes_sec, network_recv_bytes_sec, hardware_audit, raw_json
840             ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
841         ''', (
842             ts_str, now_epoch, hostname, username, uptime, os_name, os_build, os_install_date,
843             system_lang, os_install_lang, user_locale, sys_locale, timezone_val, codepage_val,
844             input_langs_json, disks_json, physical_disks_json, monitors_json, updates_json, office_json,
```

```

845 |         onedrive_json, battery_json, ram_sticks_json, list
      | ening_ports_json, alerts_json,
846 |         cpu_pct, cpu_freq, mem_tot, mem_used,
847 |         mem_pct, swap_pct, gpu_load, gpu_temp,
848 |         disk_rb, disk_wb, disk_rc,
849 |         disk_wc, net_sent, net_recv, hardware_audit_json,
      | raw_json_str
850 |     ))
851 |     snapshot_id = cursor.lastrowid or 0
852 |
853 |     processes = raw_procs if top_n is None or top_n <= 0 e
      | lse raw_procs[:top_n]
854 |     proc_rows = []
855 |     pid_snapshot_rows = []
856 |     for p in processes:
857 |         p_dict = p if isinstance(p, dict) else (p.model_du
      | mp() if hasattr(p, 'model_dump') else (vars(p) if hasattr(p, '
      | __dict__') else p))
858 |         proc_rows.append((
859 |             snapshot_id, ts_str, p_dict.get('pid', 0), p_d
      | ict.get('name', 'unknown'),
860 |             p_dict.get('status', ''), float(p_dict.get('cp
      | u_percent', 0.0) or 0.0),
861 |             float(p_dict.get('memory_mb', 0.0) or 0.0), fl
      | oat(p_dict.get('memory_percent', 0.0) or 0.0),
862 |             int(p_dict.get('num_threads', 0) or 0), int(p_
      | dict.get('num_handles', 0) or 0),
863 |             str(p_dict.get('username', '') or ''), float(p
      | _dict.get('read_bytes_sec', 0.0) or 0.0),
864 |             float(p_dict.get('write_bytes_sec', 0.0) or 0.
      | 0), p_dict.get('integrity_level', None),
865 |             int(bool(p_dict.get('elevation', False))), p_d
      | ict.get('ppid', None),
866 |             p_dict.get('parent_name', None), p_dict.get('e
      | xecutable_path', p_dict.get('executable', None)),
867 |             p_dict.get('cmdline', p_dict.get('command_line
      | ', None)), p_dict.get('sid', None),
868 |             p_dict.get('session_id', None), p_dict.get('cr

```

```

eation_time', None),
869         p_dict.get('process_guid', None), p_dict.get('
ancestor_chain', None),
870         p_dict.get('launch_reason', None),
871     ))
872
873     # Снимок канонического Per-PID Telemetry Engine
874     mem_mb = float(p_dict.get('memory_mb', 0.0) or 0.0
)
875     ws_bytes = int(p_dict.get('working_set_bytes') or
(mem_mb * 1024 * 1024))
876     pid_snapshot_rows.append((
877         int(p_dict.get('pid', 0)),
878         str(p_dict.get('name', p_dict.get('process_name', 'unknown'))),
879         p_dict.get('executable_path', p_dict.get('executable', None)),
880         float(p_dict.get('cpu_percent', 0.0) or 0.0),
881         int(p_dict.get('user_time_ms', 0) or 0),
882         int(p_dict.get('kernel_time_ms', 0) or 0),
883         int(p_dict.get('num_threads', p_dict.get('thread_count', 1)) or 1),
884         ws_bytes,
885         int(p_dict.get('private_bytes', 0) or 0),
886         int(p_dict.get('page_faults_count', p_dict.get('page_faults', 0)) or 0),
887         int(p_dict.get('gpu_vram_bytes', 0) or 0),
888         float(p_dict.get('gpu_utilization', 0.0) or 0.0),
889         int(p_dict.get('read_bytes_total', p_dict.get('read_bytes_sec', 0)) or 0),
890         int(p_dict.get('write_bytes_total', p_dict.get('write_bytes_sec', 0)) or 0),
891         int(p_dict.get('read_ops_total', 0) or 0),
892         int(p_dict.get('write_ops_total', 0) or 0),
893         int(p_dict.get('net_bytes_sent_total', 0) or 0
),
894         int(p_dict.get('net_bytes_recv_total', 0) or 0

```

```

),
895         int(p_dict.get('num_handles', p_dict.get('hand
le_count', 0)) or 0),
896         int(p_dict.get('gdi_objects', 0) or 0),
897         int(p_dict.get('user_objects', 0) or 0),
898         ts_str
899     ))
900
901     if proc_rows:
902         cursor.executemany('''
903             INSERT INTO process_snapshots (
904                 snapshot_id, timestamp, pid, name, status,
905                 cpu_percent,
906                 memory_mb, memory_percent, num_threads, nu
m_handles,
907                 username, read_bytes_sec, write_bytes_sec,
908                 integrity_level, elevation,
909                 ppid, parent_name, executable, cmdline, si
d, session_id,
910                 creation_time, process_guid, ancestor_chai
n, launch_reason
911             ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?,
?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
912         ''', proc_rows)
913
914     if pid_snapshot_rows:
915         cursor.executemany('''
916             INSERT INTO process_pid_snapshots (
917                 pid, process_name, executable_path,
918                 cpu_percent, user_time_ms, kernel_time_ms,
919                 thread_count,
920                 working_set_bytes, private_bytes, page_fau
lts_count,
921                 gpu_vram_bytes, gpu_utilization,
922                 read_bytes_total, write_bytes_total, read_
ops_total, write_ops_total,
923                 net_bytes_sent_total, net_bytes_recv_total
,

```



```

921 |             handle_count, gdi_objects, user_objects, t
| timestamp
922 |             ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?,
| ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
923 |             '', pid_snapshot_rows)
924 |
925 |         # Синхронизация нормализованных таблиц оборудования и
| метрик
926 |         try:
927 |             # 1. CPU
928 |             cpu_data = getattr(snapshot, 'cpu', None) or snap_
| data.get('cpu')
929 |             if cpu_data:
930 |                 c_dict = cpu_data if isinstance(cpu_data, dict
| ) else (cpu_data.model_dump() if hasattr(cpu_data, 'model_dump
| ') else vars(cpu_data))
931 |                 if c_dict.get('model') or c_dict.get('name'):
932 |                     self.upsert_cpu_inventory(cursor, {
933 |                         'processor_id': 0,
934 |                         'name': c_dict.get('model') or c_dict.
| get('name', 'CPU'),
935 |                         'architecture': c_dict.get('architectu
| re', 'x86_64'),
936 |                         'physical_cores': c_dict.get('physical
| _cores', 1),
937 |                         'logical_cores': c_dict.get('logical_c
| ores', 1),
938 |                         'base_frequency_mhz': c_dict.get('freq
| uency_mhz', 0.0),
939 |                     })
940 |                 self.insert_cpu_telemetry_sample(cursor, {
941 |                     'timestamp': ts_str,
942 |                     'created_at': now_epoch,
943 |                     'processor_id': 0,
944 |                     'total_percent': cpu_pct,
945 |                     'frequency_mhz': cpu_freq,
946 |                     'temperature_c': c_dict.get('temperature_c
| elsius') or c_dict.get('temperature_c'),

```

```

947         'core_utilization': c_dict.get('per_core_p
| percent', []),
948     })
949
950     # 2. RAM
951     mem_data = getattr(snapshot, 'memory', None) or sn
| ap_data.get('memory')
952     if mem_data:
953         m_dict = mem_data if isinstance(mem_data, dict
| ) else (mem_data.model_dump() if hasattr(mem_data, 'model_dump
| ') else vars(mem_data))
954         self.insert_ram_telemetry_sample(cursor, {
955             'timestamp': ts_str,
956             'created_at': now_epoch,
957             'total_gb': mem_tot,
958             'used_gb': mem_used,
959             'available_gb': m_dict.get('available_gb',
| max(0.0, mem_tot - mem_used)),
960             'percent_used': mem_pct,
961             'swap_total_gb': m_dict.get('swap_total_gb
| ', 0.0),
962             'swap_used_gb': m_dict.get('swap_used_gb',
| 0.0),
963             'swap_percent': swap_pct,
964         })
965
966     # 3. RAM Sticks (SPD)
967     sticks = getattr(snapshot, 'ram_sticks', None) or
| snap_data.get('ram_sticks') or []
968     for s in sticks:
969         self.upsert_ram_module_inventory(cursor, s)
970
971     # 4. GPUs
972     gpus_list = getattr(snapshot, 'gpus', None) or sna
| p_data.get('gpus') or []
973     for g_idx, g_item in enumerate(gpus_list):
974         g_dict = g_item if isinstance(g_item, dict) el
| se (g_item.model_dump() if hasattr(g_item, 'model_dump') else

```

```

vars(g_item))
975         g_name = g_dict.get('name', f'GPU {g_idx}')
976         name_l = g_name.lower()
977         if 'nvidia' in name_l or 'geforce' in name_l o
r g_dict.get('has_cuda'):
978             vendor = 'NVIDIA'
979             elif 'amd' in name_l or 'radeon' in name_l:
980                 vendor = 'AMD'
981             elif 'intel' in name_l or 'arc' in name_l or '
uhd' in name_l or 'hd graphics' in name_l:
982                 vendor = 'Intel'
983             else:
984                 vendor = g_dict.get('vendor') or 'Generic'
985
986         self.upsert_gpu_inventory(cursor, {
987             'gpu_id': g_idx,
988             'name': g_name,
989             'vendor': vendor,
990             'vram_gb': g_dict.get('memory_total_gb', 0
.0),
991             'directml_supported': bool(g_dict.get('has
_directml', True)),
992         })
993         self.insert_gpu_telemetry_sample(cursor, {
994             'timestamp': ts_str,
995             'created_at': now_epoch,
996             'gpu_id': g_idx,
997             'name': g_dict.get('name', f'GPU {g_idx}')
,
998             'load_percent': g_dict.get('load_percent')
,
999             'memory_used_mb': (g_dict.get('memory_used
_gb', 0.0) or 0.0) * 1024.0,
1000             'memory_total_mb': (g_dict.get('memory_tot
al_gb', 0.0) or 0.0) * 1024.0,
1001             'temperature_gpu_c': g_dict.get('temperatu
re_celsius') or g_dict.get('temperature_gpu_c'),
1002         })

```

```

1003
1004         # 5. Network
1005         net_items = getattr(snapshot, 'network', None) or
snap_data.get('network') or []
1006         for n_idx, n_item in enumerate(net_items):
1007             n_dict = n_item if isinstance(n_item, dict) el
se (n_item.model_dump() if hasattr(n_item, 'model_dump') else
vars(n_item))
1008             self.upsert_network_adapter_inventory(cursor,
{
1009                 'adapter_id': n_idx,
1010                 'adapter_guid': n_dict.get('name', f'Adapt
er_{n_idx}'),
1011                 'name': n_dict.get('name', f'Adapter {n_id
x}'),
1012                 'max_speed_mbps': n_dict.get('speed_mbps',
0),
1013             })
1014             self.insert_network_adapter_sample(cursor, {
1015                 'timestamp': ts_str,
1016                 'created_at': now_epoch,
1017                 'adapter_name': n_dict.get('name', f'Adapt
er {n_idx}'),
1018                 'bytes_recv_sec': n_dict.get('bytes_recv_p
er_sec', 0.0),
1019                 'bytes_sent_sec': n_dict.get('bytes_sent_p
er_sec', 0.0),
1020                 'link_speed_mbps': n_dict.get('speed_mbps'
, 0),
1021                 'is_connected': bool(n_dict.get('is_up', T
rue)),
1022             })
1023         except Exception as norm_err:
1024             logger.debug(f'Предупреждение синхронизации нормал
изованных таблиц оборудования: {norm_err}')
1025
1026         return snapshot_id
1027

```

```

1028     def insert_hardware_archive_row(self, cursor: sqlite3.Curs
or, data: Any) -> int:
1029         now_dt = datetime.now(timezone.utc)
1030         if isinstance(data, HardwareArchiveEntry):
1031             raw_json = data.model_dump_json()
1032             report = data.report
1033             prob_count = report.problem_devices_count if repor
t else 0
1034             outdated_count = report.outdated_drivers_count if
report else 0
1035             archive_id = data.archive_id
1036             timestamp = data.timestamp
1037             devices_count = data.devices_count
1038             changes_count = data.changes_count
1039         else:
1040             raw_json = json.dumps(data, ensure_ascii=False, de
fault=str)
1041             report = data.get('report', {})
1042             prob_count = report.get('problem_devices_count', 0
) if isinstance(report, dict) else getattr(report, 'problem_de
vices_count', 0)
1043             outdated_count = report.get('outdated_drivers_coun
t', 0) if isinstance(report, dict) else getattr(report, 'outda
ted_drivers_count', 0)
1044             archive_id = data.get('archive_id', f'arch_{int(no
w_dt.timestamp())}')
1045             timestamp = data.get('timestamp', now_dt.isoformat
())
1046             devices_count = data.get('devices_count', 0)
1047             changes_count = data.get('changes_count', 0)
1048
1049         try:
1050             now_epoch = datetime.fromisoformat(timestamp).time
stamp()
1051         except Exception:
1052             now_epoch = now_dt.timestamp()
1053
1054         cursor.execute(''

```

```

1055         INSERT OR REPLACE INTO hardware_audits (
1056             archive_id, timestamp, created_at, devices_cou
1057 nt,
1058             problem_devices_count, outdated_drivers_count,
1059 changes_count, raw_json
1060         ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
1061         '''', (archive_id, timestamp, now_epoch, devices_count,
1062 prob_count, outdated_count, changes_count, raw_json))
1063         return cursor.lastrowid or 0
1064
1065     def insert_startup_archive_row(self, cursor: sqlite3.Curso
1066 r, data: Any) -> int:
1067         """Сохранение снимка аудита автозапуска и зафиксирован
1068 ных изменений в БД."""
1069         now_dt = datetime.now(timezone.utc)
1070         if isinstance(data, StartupArchiveEntry):
1071             raw_json = data.model_dump_json()
1072             archive_id = data.archive_id
1073             timestamp = data.timestamp
1074             total_entries = data.total_entries
1075             health_score = data.health_score
1076             changes_count = data.changes_count
1077             report = data.report
1078             changes = data.changes
1079             if hasattr(report, 'summary') and report.summary:
1080                 active_entries = report.summary.active_entries
1081                 disabled_entries = report.summary.disabled_ent
1082 ries
1083                 broken_entries = report.summary.broken_entries
1084             else:
1085                 active_entries = 0
1086                 disabled_entries = 0
1087                 broken_entries = 0
1088         else:
1089             raw_json = json.dumps(data, ensure_ascii=False, de
1090 fault=str)
1091             archive_id = data.get('archive_id', f'startup_{int
1092 (now_dt.timestamp())}')

```

```

1085         timestamp = data.get('timestamp', now_dt.isoformat
    | ())
1086         total_entries = data.get('total_entries', 0)
1087         health_score = data.get('health_score', 100)
1088         changes_count = data.get('changes_count', 0)
1089         report = data.get('report', {})
1090         summary = report.get('summary', {}) if isinstance(
    | report, dict) else getattr(report, 'summary', None)
1091         if isinstance(summary, dict):
1092             active_entries = summary.get('active_entries',
    | 0)
1093             disabled_entries = summary.get('disabled_entri
    | es', 0)
1094             broken_entries = summary.get('broken_entries',
    | 0)
1095         else:
1096             active_entries = getattr(summary, 'active_entr
    | ies', 0) if summary else 0
1097             disabled_entries = getattr(summary, 'disabled_
    | entries', 0) if summary else 0
1098             broken_entries = getattr(summary, 'broken_entr
    | ies', 0) if summary else 0
1099             changes = data.get('changes', [])
1100
1101         try:
1102             now_epoch = datetime.fromisoformat(timestamp).time
    | stamp()
1103         except Exception:
1104             now_epoch = now_dt.timestamp()
1105
1106         cursor.execute('''
1107             INSERT OR REPLACE INTO startup_audit_archives (
1108                 archive_id, timestamp, created_at, total_entri
    | es,
1109                 active_entries, disabled_entries, broken_entr
    | ies,
1110                 health_score, changes_count, raw_json
1111             ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)

```

```

1112         '', (
1113             archive_id, timestamp, now_epoch, total_entries,
1114             active_entries, disabled_entries, broken_entries,
1115             health_score, changes_count, raw_json
1116         ))
1117
1118     if changes:
1119         for ch in changes:
1120             ch_dict = ch.model_dump(mode='json') if hasattr(ch, 'model_dump') else ch
1121             cursor.execute('''
1122                 INSERT INTO startup_changes (
1123                     archive_id, timestamp, created_at, change_type,
1124                     entry_id, name, description, previous_value, current_value
1125                 ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
1126             ''', (
1127                 archive_id,
1128                 ch_dict.get('timestamp', timestamp),
1129                 now_epoch,
1130                 ch_dict.get('change_type', 'modified'),
1131                 ch_dict.get('entry_id', ''),
1132                 ch_dict.get('name', ''),
1133                 ch_dict.get('description', ''),
1134                 json.dumps(ch_dict.get('previous_value'), ensure_ascii=False) if ch_dict.get('previous_value') is not None else None,
1135                 json.dumps(ch_dict.get('current_value'), ensure_ascii=False) if ch_dict.get('current_value') is not None else None,
1136             ))
1137
1138     return cursor.lastrowid or 0
1139
1140 def upsert_device_inventory(
1141     self,
1142     cursor: sqlite3.Cursor,

```



```

1143         device_instance_id: str,
1144         friendly_name: Optional[str] = None,
1145         device_class: Optional[str] = None,
1146         install_date: Optional[str] = None,
1147     ) -> int:
1148         """Вставка или обновление записи в таблице device_inve
ntory."""
1149         now_dt = datetime.now(timezone.utc)
1150         now_epoch = now_dt.timestamp()
1151         norm_id = str(device_instance_id).strip().upper()
1152         date_val = install_date or now_dt.strftime('%d.%m.%Y %
H:%M')
1153         cursor.execute('''
1154             INSERT INTO device_inventory (
1155                 device_instance_id, friendly_name, device_clas
s, install_date, created_at, updated_at
1156             ) VALUES (?, ?, ?, ?, ?, ?)
1157             ON CONFLICT(device_instance_id) DO UPDATE SET
1158                 friendly_name = COALESCE(excluded.friendly_nam
e, device_inventory.friendly_name),
1159                 device_class = COALESCE(excluded.device_class,
device_inventory.device_class),
1160                 updated_at = excluded.updated_at
1161             ''', (norm_id, friendly_name or '', device_class or ''
, date_val, now_epoch, now_epoch))
1162         return cursor.lastrowid or 0
1163
1164     def upsert_disk_inventory(self, cursor: sqlite3.Cursor, da
ta: Any) -> int:
1165         """Вставка или обновление паспорта физического накопит
еля (disk_inventory)."""
1166         d = data if isinstance(data, dict) else (data.model_du
mp() if hasattr(data, 'model_dump') else vars(data))
1167         now_dt = datetime.now(timezone.utc)
1168         ts_str = d.get('timestamp') or now_dt.isoformat()
1169         now_epoch = now_dt.timestamp()
1170         serial = str(d.get('serial_number') or f"DISK_{d.get('
disk_id', 0)}").strip()

```

```

1171 |
1172 |         cursor.execute('''
1173 |             INSERT INTO disk_inventory (
1174 |                 disk_id, device_path, serial_number, vendor, p
1175 | roduct,
1176 |                 revision, bus_type, media_type, size_bytes, si
1177 | ze_gb,
1178 |                 sector_size, removable, partition_style, gpt_g
1179 | uid,
1180 |                 partitions_json, first_seen, last_seen, update
1181 | d_at
1182 |             ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?
1183 | , ?, ?, ?, ?)
1184 |
1185 |             ON CONFLICT(serial_number) DO UPDATE SET
1186 |                 disk_id = excluded.disk_id,
1187 |                 device_path = excluded.device_path,
1188 |                 vendor = excluded.vendor,
1189 |                 product = excluded.product,
1190 |                 revision = excluded.revision,
1191 |                 bus_type = excluded.bus_type,
1192 |                 media_type = excluded.media_type,
1193 |                 size_bytes = excluded.size_bytes,
1194 |                 size_gb = excluded.size_gb,
1195 |                 sector_size = excluded.sector_size,
1196 |                 removable = excluded.removable,
1197 |                 partition_style = excluded.partition_style,
1198 |                 gpt_guid = excluded.gpt_guid,
1199 |                 partitions_json = excluded.partitions_json,
1200 |                 last_seen = excluded.last_seen,
1201 |                 updated_at = excluded.updated_at
1202 |         ''', (
1203 |             int(d.get('disk_id', 0)),
1204 |             d.get('device_path') or f"\\\\\\\\.\\\\PhysicalDrive{d.g
1205 | et('disk_id', 0)}",
1206 |             serial,
1207 |             d.get('vendor', ''),
1208 |             d.get('product') or d.get('name', ''),
1209 |             d.get('revision', ''),

```

```

1203         d.get('bus_type', 'Unknown'),
1204         d.get('media_type', 'SSD'),
1205         int(d.get('size_bytes', 0)),
1206         float(d.get('size_gb', 0.0)),
1207         int(d.get('sector_size', 512)),
1208         int(bool(d.get('removable', False))),
1209         d.get('partition_style', 'GPT'),
1210         d.get('gpt_guid', ''),
1211         json.dumps(d.get('partitions', []), ensure_ascii=F
    else, default=str),
1212         ts_str,
1213         ts_str,
1214         now_epoch,
1215     ))
1216     return cursor.lastrowid or 0
1217
1218     def upsert_volume_inventory(self, cursor: sqlite3.Cursor,
    data: Any) -> int:
1219         """Вставка или обновление паспорта логического тома (v
    olume_inventory)."""
1220         v = data if isinstance(data, dict) else (data.model_du
    mp() if hasattr(data, 'model_dump') else vars(data))
1221         now_dt = datetime.now(timezone.utc)
1222         ts_str = v.get('timestamp') or now_dt.isoformat()
1223         now_epoch = now_dt.timestamp()
1224         guid = str(v.get('volume_guid') or v.get('volume_id')
    or v.get('drive_letter', 'UNKNOWN')).strip()
1225
1226         cursor.execute('''
1227             INSERT INTO volume_inventory (
1228                 volume_guid, drive_letter, mount_point, label,
    filesystem,
1229                 filesystem_flags, cluster_size_bytes, sector_s
    ize_bytes,
1230                 total_bytes, total_gb, disk_id, first_seen, la
    st_seen, updated_at
1231             ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?
    )

```

```

1232         ON CONFLICT(volume_guid) DO UPDATE SET
1233             drive_letter = excluded.drive_letter,
1234             mount_point = excluded.mount_point,
1235             label = excluded.label,
1236             filesystem = excluded.filesystem,
1237             filesystem_flags = excluded.filesystem_flags,
1238             cluster_size_bytes = excluded.cluster_size_byt
1239 es,
1240             sector_size_bytes = excluded.sector_size_bytes
1241 ,
1242             total_bytes = excluded.total_bytes,
1243             total_gb = excluded.total_gb,
1244             disk_id = excluded.disk_id,
1245             last_seen = excluded.last_seen,
1246             updated_at = excluded.updated_at
1247     ''' , (
1248         guid,
1249         v.get('drive_letter', ''),
1250         v.get('mount_point', ''),
1251         v.get('label', ''),
1252         v.get('filesystem', 'NTFS'),
1253         int(v.get('filesystem_flags', 0) or 0),
1254         int(v.get('cluster_size_bytes', 4096) or 4096),
1255         int(v.get('sector_size_bytes', 512) or 512),
1256         int(v.get('total_bytes', 0) or 0),
1257         float(v.get('total_gb', 0.0) or 0.0),
1258         v.get('disk_id'),
1259         ts_str,
1260         ts_str,
1261         now_epoch,
1262     ))
1263     return cursor.lastrowid or 0
1264
1265     def insert_disk_health_snapshot(self, cursor: sqlite3.Curs
1266 or, data: Any) -> int:
1267         """Вставка снимка здоровья / SMART / TBW накопителя (d
1268 isk_health_snapshots)."""
1269         h = data if isinstance(data, dict) else (data.model_du

```

```

mp() if hasattr(data, 'model_dump') else vars(data))
1266     now_dt = datetime.now(timezone.utc)
1267     ts_str = h.get('timestamp') or now_dt.isoformat()
1268     try:
1269         created_at = float(h.get('created_at') or datetime
1270 .fromisoformat(ts_str).timestamp())
1271     except Exception:
1272         created_at = now_dt.timestamp()
1273
1274     cursor.execute('''
1275         INSERT INTO disk_health_snapshots (
1276             timestamp, created_at, disk_id, serial_number,
1277             model, bus_type,
1278             temperature_c, wear_percent, available_spare,
1279             tbw_written_tb,
1280             tbw_read_tb, power_on_hours, power_cycles, uns
1281             afe_shutdowns,
1282             media_errors, error_log_entries, health_status
1283             , raw_smart_json
1284             ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?
1285             , ?, ?, ?, ?)
1286     ''', (
1287         ts_str,
1288         created_at,
1289         int(h.get('disk_id', 0)),
1290         h.get('serial_number'),
1291         h.get('model', ''),
1292         h.get('bus_type', 'Unknown'),
1293         h.get('temperature_c'),
1294         h.get('wear_percentage') or h.get('wear_percent'),
1295         h.get('available_spare_percent') or h.get('availab
le_spare'),
1296         h.get('tbw_written_tb') or h.get('lifetime_write_t
b'),
1297         h.get('tbw_read_tb') or h.get('lifetime_read_tb'),
1298         h.get('power_on_hours'),
1299         h.get('power_cycles'),
1300         h.get('unsafe_shutdowns'),

```

```

1295         h.get('media_errors'),
1296         h.get('error_log_entries'),
1297         h.get('health_status', 'Healthy'),
1298         json.dumps(h, ensure_ascii=False, default=str),
1299     ))
1300     return cursor.lastrowid or 0
1301
1302     def insert_disk_performance_sample(self, cursor: sqlite3.C
1303         ursor, data: Any) -> int:
1304         """Вставка сэмпла производительности диска (disk_perfo
1305         rmance_samples)."""
1306         p = data if isinstance(data, dict) else (data.model_du
1307         mp() if hasattr(data, 'model_dump') else vars(data))
1308         now_dt = datetime.now(timezone.utc)
1309         ts_str = p.get('timestamp') or now_dt.isoformat()
1310         try:
1311             created_at = float(p.get('created_at') or datetime
1312             .fromisoformat(ts_str).timestamp())
1313         except Exception:
1314             created_at = now_dt.timestamp()
1315
1316         cursor.execute('''
1317             INSERT INTO disk_performance_samples (
1318                 timestamp, created_at, disk_name, read_bytes_s
1319                 ec, write_bytes_sec,
1320                 read_iops, write_iops, read_time_ms, write_tim
1321                 e_ms, percent_disk_time, queue_length
1322             ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
1323         ''', (
1324             ts_str,
1325             created_at,
1326             p.get('disk_name') or f"PhysicalDrive{p.get('disk_
1327             id', 0)}",
1328             float(p.get('read_bytes_sec', 0.0) or 0.0),
1329             float(p.get('write_bytes_sec', 0.0) or 0.0),
1330             float(p.get('read_iops', 0.0) or 0.0),
1331             float(p.get('write_iops', 0.0) or 0.0),
1332             float(p.get('read_time_ms', 0.0) or p.get('avg_rea

```

```

    d_latency_ms', 0.0) or 0.0),
1326         float(p.get('write_time_ms', 0.0) or p.get('avg_wr
    ite_latency_ms', 0.0) or 0.0),
1327         float(p.get('percent_disk_time', 0.0) or p.get('di
    sk_time_percent', 0.0) or 0.0),
1328         float(p.get('queue_length', 0.0) or 0.0),
1329     ))
1330     return cursor.lastrowid or 0
1331
1332     def insert_disk_io_event(self, cursor: sqlite3.Cursor, dat
    a: Any) -> int:
1333         """Вставка события ввода-вывода (disk_io_events)."""
1334         ev = data if isinstance(data, dict) else (data.model_d
    ump() if hasattr(data, 'model_dump') else vars(data))
1335         now_dt = datetime.now(timezone.utc)
1336         ts_str = ev.get('timestamp') or now_dt.isoformat()
1337         try:
1338             created_at = float(ev.get('created_at') or datetim
    e.fromisoformat(ts_str).timestamp())
1339         except Exception:
1340             created_at = now_dt.timestamp()
1341
1342         cursor.execute('''
1343             INSERT INTO disk_io_events (
1344                 timestamp, created_at, pid, process_name, disk
    _id,
1345                 operation, bytes_count, duration_ms, offset, f
    ile_path
1346             ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
1347         ''', (
1348             ts_str,
1349             created_at,
1350             int(ev.get('pid', 0)),
1351             ev.get('process_name', ''),
1352             int(ev.get('disk_id', 0)),
1353             ev.get('operation', 'READ'),
1354             int(ev.get('bytes_count', 0)),
1355             ev.get('duration_ms'),

```

```

1356         ev.get('offset'),
1357         ev.get('file_path'),
1358     ))
1359     return cursor.lastrowid or 0
1360
1361     def upsert_cpu_inventory(self, cursor: sqlite3.Cursor, data: Any) -> int:
1362         """Вставка или обновление паспорта физического процессора (cpu_inventory)."""
1363         c = data if isinstance(data, dict) else (data.model_dump() if hasattr(data, 'model_dump') else vars(data))
1364         now_dt = datetime.now(timezone.utc)
1365         ts_str = c.get('timestamp') or now_dt.isoformat()
1366         now_epoch = now_dt.timestamp()
1367         name = str(c.get('name') or f"CPU_{c.get('processor_id', 0)}").strip()
1368
1369         cursor.execute('''
1370             INSERT INTO cpu_inventory (
1371                 processor_id, name, vendor, architecture, physical_cores, logical_cores,
1372                 base_frequency_mhz, max_frequency_mhz, l2_cache_kb, l3_cache_kb,
1373                 socket, features_json, first_seen, last_seen, updated_at
1374             ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?
1375             , ?)
1376
1377             ON CONFLICT(name) DO UPDATE SET
1378                 processor_id = excluded.processor_id,
1379                 vendor = excluded.vendor,
1380                 architecture = excluded.architecture,
1381                 physical_cores = excluded.physical_cores,
1382                 logical_cores = excluded.logical_cores,
1383                 base_frequency_mhz = excluded.base_frequency_mhz,
1384                 max_frequency_mhz = excluded.max_frequency_mhz,
1385                 l2_cache_kb = excluded.l2_cache_kb,

```



```

1384         l3_cache_kb = excluded.l3_cache_kb,
1385         socket = excluded.socket,
1386         features_json = excluded.features_json,
1387         last_seen = excluded.last_seen,
1388         updated_at = excluded.updated_at
1389     ''' , (
1390         int(c.get('processor_id', 0)),
1391         name,
1392         c.get('vendor', ''),
1393         c.get('architecture', 'x86_64'),
1394         int(c.get('physical_cores', 1) or 1),
1395         int(c.get('logical_cores', 1) or 1),
1396         float(c.get('base_frequency_mhz', 0.0) or 0.0),
1397         float(c.get('max_frequency_mhz', 0.0) or 0.0),
1398         c.get('l2_cache_kb'),
1399         c.get('l3_cache_kb'),
1400         c.get('socket', ''),
1401         json.dumps(c.get('features', []), ensure_ascii=False, default=str),
1402         ts_str,
1403         ts_str,
1404         now_epoch,
1405     ))
1406     return cursor.lastrowid or 0
1407
1408     def upsert_ram_module_inventory(self, cursor: sqlite3.Cursor, data: Any) -> int:
1409         """Вставка или обновление паспорта планки оперативной
1410         памяти (ram_module_inventory)."""
1411         r = data if isinstance(data, dict) else (data.model_dump() if hasattr(data, 'model_dump') else vars(data))
1412         now_dt = datetime.now(timezone.utc)
1413         ts_str = r.get('timestamp') or now_dt.isoformat()
1414         now_epoch = now_dt.timestamp()
1415         serial = str(r.get('serial_number') or f"RAM_{r.get('bank_label', '')}_{r.get('slot_id', 0)}").strip()
1416         cursor.execute(''

```

```

1417         INSERT INTO ram_module_inventory (
1418             slot_id, bank_label, device_locator, serial_nu
1419             mber, part_number,
1420             manufacturer, capacity_bytes, capacity_gb, spe
1421             ed_mhz, memory_type,
1422             form_factor, configured_voltage, first_seen, l
1423             ast_seen, updated_at
1424         ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?
1425         , ?)
1426
1427         ON CONFLICT(serial_number) DO UPDATE SET
1428             slot_id = excluded.slot_id,
1429             bank_label = excluded.bank_label,
1430             device_locator = excluded.device_locator,
1431             part_number = excluded.part_number,
1432             manufacturer = excluded.manufacturer,
1433             capacity_bytes = excluded.capacity_bytes,
1434             capacity_gb = excluded.capacity_gb,
1435             speed_mhz = excluded.speed_mhz,
1436             memory_type = excluded.memory_type,
1437             form_factor = excluded.form_factor,
1438             configured_voltage = excluded.configured_volta
1439             ge,
1440             last_seen = excluded.last_seen,
1441             updated_at = excluded.updated_at
1442     ''' , (
1443         int(r.get('slot_id', 0)),
1444         r.get('bank_label', 'DIMM'),
1445         r.get('device_locator', ''),
1446         serial,
1447         r.get('part_number', ''),
1448         r.get('manufacturer', ''),
1449         int(r.get('capacity_bytes', 0) or 0),
1450         float(r.get('capacity_gb', 0.0) or 0.0),
1451         int(r.get('speed_mhz', 0) or 0),
1452         r.get('memory_type', 'DDR4'),
1453         r.get('form_factor', 'DIMM'),
1454         r.get('configured_voltage'),
1455         ts_str,

```

```

1450         ts_str,
1451         now_epoch,
1452     ))
1453     return cursor.lastrowid or 0
1454
1455     def upsert_gpu_inventory(self, cursor: sqlite3.Cursor, data: Any) -> int:
1456         """Вставка или обновление паспорта графического ускорителя (gpu_inventory)."""
1457         g = data if isinstance(data, dict) else (data.model_dump() if hasattr(data, 'model_dump') else vars(data))
1458         now_dt = datetime.now(timezone.utc)
1459         ts_str = g.get('timestamp') or now_dt.isoformat()
1460         now_epoch = now_dt.timestamp()
1461         pci_id = str(g.get('pci_device_id') or g.get('pci_bus_id') or f"GPU_{g.get('gpu_id', 0)}_{g.get('name', '')}").strip()
1462
1463         cursor.execute('''
1464             INSERT INTO gpu_inventory (
1465                 gpu_id, pci_device_id, name, vendor, driver_version, driver_date,
1466                 vram_bytes, vram_gb, pci_bus_id, bios_version, cuda_cores,
1467                 directml_supported, first_seen, last_seen, updated_at
1468             ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?
1469             , ?)
1470
1471             ON CONFLICT(pci_device_id) DO UPDATE SET
1472                 gpu_id = excluded.gpu_id,
1473                 name = excluded.name,
1474                 vendor = excluded.vendor,
1475                 driver_version = excluded.driver_version,
1476                 driver_date = excluded.driver_date,
1477                 vram_bytes = excluded.vram_bytes,
1478                 vram_gb = excluded.vram_gb,
1479                 pci_bus_id = excluded.pci_bus_id,
1480                 bios_version = excluded.bios_version,

```

```

1479         cuda_cores = excluded.cuda_cores,
1480         directml_supported = excluded.directml_support
1481     ed,
1482         last_seen = excluded.last_seen,
1483         updated_at = excluded.updated_at
1484     ''' , (
1485         int(g.get('gpu_id', 0)),
1486         pci_id,
1487         g.get('name', 'Graphics Adapter'),
1488         g.get('vendor', ''),
1489         g.get('driver_version', ''),
1490         g.get('driver_date', ''),
1491         int(g.get('vram_bytes', 0) or 0),
1492         float(g.get('vram_gb', 0.0) or (g.get('memory_total_
1493         l_mb', 0.0) / 1024.0 if g.get('memory_total_mb') else 0.0)),
1494         g.get('pci_bus_id', ''),
1495         g.get('bios_version', ''),
1496         g.get('cuda_cores'),
1497         int(bool(g.get('directml_supported', True))),
1498         ts_str,
1499         ts_str,
1500         now_epoch,
1501     ))
1502     return cursor.lastrowid or 0
1503
1504     def upsert_network_adapter_inventory(self, cursor: sqlite3
1505     .Cursor, data: Any) -> int:
1506         """Вставка или обновление паспорта сетевого адаптера (
1507         network_adapter_inventory)."""
1508         n = data if isinstance(data, dict) else (data.model_du
1509         mp() if hasattr(data, 'model_dump') else vars(data))
1510         now_dt = datetime.now(timezone.utc)
1511         ts_str = n.get('timestamp') or now_dt.isoformat()
1512         now_epoch = now_dt.timestamp()
1513         guid = str(n.get('adapter_guid') or n.get('mac_address
1514         ') or n.get('name', 'UNKNOWN')).strip()
1515         cursor.execute(''

```

```

1511         INSERT INTO network_adapter_inventory (
1512             adapter_id, adapter_guid, name, interface_name
1513             , mac_address,
1514             adapter_type, is_physical, is_wireless, max_sp
1515             eed_mbps,
1516             driver_name, driver_version, first_seen, last_
1517             seen, updated_at
1518         ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?
1519         )
1520
1521         ON CONFLICT(adapter_guid) DO UPDATE SET
1522             adapter_id = excluded.adapter_id,
1523             name = excluded.name,
1524             interface_name = excluded.interface_name,
1525             mac_address = excluded.mac_address,
1526             adapter_type = excluded.adapter_type,
1527             is_physical = excluded.is_physical,
1528             is_wireless = excluded.is_wireless,
1529             max_speed_mbps = excluded.max_speed_mbps,
1530             driver_name = excluded.driver_name,
1531             driver_version = excluded.driver_version,
1532             last_seen = excluded.last_seen,
1533             updated_at = excluded.updated_at
1534
1535         '', (
1536             int(n.get('adapter_id', 0)),
1537             guid,
1538             n.get('name', 'Network Adapter'),
1539             n.get('interface_name', ''),
1540             n.get('mac_address', ''),
1541             n.get('adapter_type', 'Ethernet'),
1542             int(bool(n.get('is_physical', True))),
1543             int(bool(n.get('is_wireless', False))),
1544             int(n.get('max_speed_mbps', 0) or 0),
1545             n.get('driver_name', ''),
1546             n.get('driver_version', ''),
1547             ts_str,
1548             ts_str,
1549             now_epoch,
1550         ))

```

```

1545         return cursor.lastrowid or 0
1546
1547     def insert_cpu_telemetry_sample(self, cursor: sqlite3.Curs
or, data: Any) -> int:
1548         """Вставка снимка метрик нагрузки и температур процесс
opa (cpu_telemetry_samples)."""
1549         c = data if isinstance(data, dict) else (data.model_du
mp() if hasattr(data, 'model_dump') else vars(data))
1550         now_dt = datetime.now(timezone.utc)
1551         ts_str = c.get('timestamp') or now_dt.isoformat()
1552         try:
1553             created_at = float(c.get('created_at') or datetime
.fromisoformat(ts_str).timestamp())
1554         except Exception:
1555             created_at = now_dt.timestamp()
1556
1557         cursor.execute('''
1558             INSERT INTO cpu_telemetry_samples (
1559                 timestamp, created_at, processor_id, total_per
cent, user_percent,
1560                 kernel_percent, frequency_mhz, temperature_c,
package_power_w,
1561                 core_utilization_json, core_temperatures_json
1562             ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
1563         ''', (
1564             ts_str,
1565             created_at,
1566             int(c.get('processor_id', 0)),
1567             float(c.get('total_percent', 0.0) or 0.0),
1568             float(c.get('user_percent', 0.0) or 0.0),
1569             float(c.get('kernel_percent', 0.0) or 0.0),
1570             float(c.get('frequency_mhz', 0.0) or 0.0),
1571             c.get('temperature_c') or c.get('temperature_celsi
us'),
1572             c.get('package_power_w'),
1573             json.dumps(c.get('core_utilization') or c.get('per
_core_percent', []), ensure_ascii=False, default=str),
1574             json.dumps(c.get('core_temperatures', []), ensure_

```

```

        ascii=False, default=str),
1575         ))
1576         return cursor.lastrowid or 0
1577
1578     def insert_ram_telemetry_sample(self, cursor: sqlite3.Curs
or, data: Any) -> int:
1579         """Вставка снимка метрик использования оперативной пам
яти (ram_telemetry_samples)."""
1580         r = data if isinstance(data, dict) else (data.model_du
mp() if hasattr(data, 'model_dump') else vars(data))
1581         now_dt = datetime.now(timezone.utc)
1582         ts_str = r.get('timestamp') or now_dt.isoformat()
1583         try:
1584             created_at = float(r.get('created_at') or datetime
.fromisoformat(ts_str).timestamp())
1585         except Exception:
1586             created_at = now_dt.timestamp()
1587
1588         cursor.execute('''
1589             INSERT INTO ram_telemetry_samples (
1590                 timestamp, created_at, total_bytes, total_gb,
used_bytes, used_gb,
1591                 available_bytes, available_gb, percent_used, s
wap_total_gb,
1592                 swap_used_gb, swap_percent, pool_paged_mb, poo
l_nonpaged_mb
1593             ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?
)
1594         ''', (
1595             ts_str,
1596             created_at,
1597             int(r.get('total_bytes', 0) or 0),
1598             float(r.get('total_gb', 0.0) or 0.0),
1599             int(r.get('used_bytes', 0) or 0),
1600             float(r.get('used_gb', 0.0) or 0.0),
1601             int(r.get('available_bytes', 0) or 0),
1602             float(r.get('available_gb', 0.0) or 0.0),
1603             float(r.get('percent_used', 0.0) or r.get('percent

```

```

        ', 0.0) or 0.0),
1604         float(r.get('swap_total_gb', 0.0) or 0.0),
1605         float(r.get('swap_used_gb', 0.0) or 0.0),
1606         float(r.get('swap_percent', 0.0) or 0.0),
1607         r.get('pool_paged_mb'),
1608         r.get('pool_nonpaged_mb'),
1609     ))
1610     return cursor.lastrowid or 0
1611
1612     def insert_gpu_telemetry_sample(self, cursor: sqlite3.Curs
or, data: Any) -> int:
1613         """Вставка снимка метрик GPU (gpu_telemetry_samples)."""
1614
1615         g = data if isinstance(data, dict) else (data.model_du
mp() if hasattr(data, 'model_dump') else vars(data))
1616         now_dt = datetime.now(timezone.utc)
1617         ts_str = g.get('timestamp') or now_dt.isoformat()
1618         try:
1619             created_at = float(g.get('created_at') or datetime
.fromisoformat(ts_str).timestamp())
1620         except Exception:
1621             created_at = now_dt.timestamp()
1622
1623         cursor.execute('''
1624             INSERT INTO gpu_telemetry_samples (
1625                 timestamp, created_at, gpu_id, name, load_perc
ent, memory_used_mb,
1626                 memory_total_mb, memory_percent, temperature_g
pu_c, temperature_memory_c,
1627                 fan_speed_pct, power_draw_w, clock_graphics_mh
z, clock_memory_mhz
1628             ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?
)
1629         ''', (
1630             ts_str,
1631             created_at,
1632             int(g.get('gpu_id', 0) or g.get('index', 0)),
            g.get('name', 'GPU'),

```



```

1633         g.get('load_percent') or g.get('utilization_gpu_percent'),
1634         float(g.get('memory_used_mb', 0.0) or (g.get('memory_used_gb', 0.0) * 1024.0 if g.get('memory_used_gb') else 0.0)),
1635         float(g.get('memory_total_mb', 0.0) or (g.get('memory_total_gb', 0.0) * 1024.0 if g.get('memory_total_gb') else 0.0)),
1636         g.get('memory_percent') or g.get('utilization_memory_pct'),
1637         g.get('temperature_gpu_c') or g.get('temperature_celsius'),
1638         g.get('temperature_memory_c'),
1639         g.get('fan_speed_pct'),
1640         g.get('power_draw_w'),
1641         g.get('clock_graphics_mhz'),
1642         g.get('clock_memory_mhz'),
1643     ))
1644     return cursor.lastrowid or 0
1645
1646     def insert_network_adapter_sample(self, cursor: sqlite3.Cursor, data: Any) -> int:
1647         """Вставка снимка метрик сетевого адаптера (network_adapter_samples)."""
1648         n = data if isinstance(data, dict) else (data.model_dump() if hasattr(data, 'model_dump') else vars(data))
1649         now_dt = datetime.now(timezone.utc)
1650         ts_str = n.get('timestamp') or now_dt.isoformat()
1651         try:
1652             created_at = float(n.get('created_at') or datetime.fromisoformat(ts_str).timestamp())
1653         except Exception:
1654             created_at = now_dt.timestamp()
1655
1656         cursor.execute('''
1657             INSERT INTO network_adapter_samples (
1658                 timestamp, created_at, adapter_name, bytes_received_sec, bytes_sent_sec,

```

```

1659         packets_recv_sec, packets_sent_sec, errors_in_
1660         sec, errors_out_sec,
1661         link_speed_mbps, is_connected
1662     ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
1663     ''' , (
1664         ts_str,
1665         created_at,
1666         n.get('adapter_name') or n.get('name', 'Network Ad
1667         apter'),
1668         float(n.get('bytes_recv_sec', 0.0) or n.get('bytes
1669         _recv_per_sec', 0.0) or 0.0),
1670         float(n.get('bytes_sent_sec', 0.0) or n.get('bytes
1671         _sent_per_sec', 0.0) or 0.0),
1672         float(n.get('packets_recv_sec', 0.0) or 0.0),
1673         float(n.get('packets_sent_sec', 0.0) or 0.0),
1674         float(n.get('errors_in_sec', 0.0) or 0.0),
1675         float(n.get('errors_out_sec', 0.0) or 0.0),
1676         int(n.get('link_speed_mbps', 0) or n.get('speed_mb
1677         ps', 0) or 0),
1678         int(bool(n.get('is_connected', True) if n.get('is_
1679         connected') is not None else n.get('is_up', True))),
1680     ))
1681     return cursor.lastrowid or 0
1682
1683     # =====
1684     # =====
1685     # Подсистемы рефакторинга: Data-First SQLite Writers
1686     # =====
1687     # =====
1688
1689     def save_event_log_snapshot(
1690         self,
1691         snapshot_id: str,
1692         channels: List[Any],
1693         entries: List[Any],
1694         intelligence_profiles: Optional[List[Any]] = None,
1695     ) -> int:
1696         """Сохранение снимка каналов, кэша событий и профилей

```

```

интеллекта логов. """
1689         if self._cm.read_only:
1690             return 0
1691         now_dt = datetime.now(timezone.utc)
1692         ts_str = now_dt.isoformat()
1693         created_at = now_dt.timestamp()
1694
1695         with self._cm.lock, self._cm.get_connection() as conn:
1696             cursor = conn.cursor()
1697             for ch in channels:
1698                 d = ch if isinstance(ch, dict) else (ch.model_
dump() if hasattr(ch, 'model_dump') else vars(ch))
1699                 cursor.execute('''
1700                     INSERT INTO event_log_channel_snapshots (
1701                         snapshot_id, timestamp, channel_name,
display_name, description,
1702                         record_count, size_bytes, channel_type
, is_enabled, created_at
1703                         ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
1704                     ''', (
1705                         snapshot_id, ts_str, d.get('name') or d.ge
t('channel_name', ''),
1706                         d.get('display_name'), d.get('description'
),
1707                         int(d.get('record_count', 0) or 0), int(d.
get('size_bytes', 0) or 0),
1708                         d.get('channel_type', 'Admin'), int(bool(d
.get('enabled', True) if d.get('enabled') is not None else d.g
et('is_enabled', True))),
1709                         created_at,
1710                     ))
1711
1712             for ev in entries:
1713                 d = ev if isinstance(ev, dict) else (ev.model_
dump() if hasattr(ev, 'model_dump') else vars(ev))
1714                 cursor.execute('''
1715                     INSERT INTO event_log_entries_cache (
1716                         snapshot_id, timestamp, channel, event

```

```

_id, level, provider_name,
1717         time_created, message, raw_json, creat
ed_at
1718         ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
1719         '', (
1720         snapshot_id, ts_str, d.get('channel', 'Sys
tem'),
1721         int(d.get('event_id', 0) or 0), d.get('lev
el', 'Information'),
1722         d.get('provider_name') or d.get('provider'
) or d.get('source', ''),
1723         d.get('time_created') or d.get('timestamp'
, ts_str),
1724         d.get('message', ''), json.dumps(d, ensure
_ascii=False) if isinstance(d, dict) else None,
1725         created_at,
1726         ))
1727
1728         if intelligence_profiles:
1729             for prof in intelligence_profiles:
1730                 p = prof if isinstance(prof, dict) else (p
rof.model_dump() if hasattr(prof, 'model_dump') else vars(prof
))
1731                 prof_data = p.get('profile') if isinstance
(p.get('profile'), dict) else p
1732                 cursor.execute(''
1733                     INSERT INTO event_log_intelligence_pro
files (
1734                         snapshot_id, timestamp, channel, t
otal_analyzed, unique_patterns_count,
1735                         critical_incidents_json, top_clust
ers_json, bursts_json, decision_gate_json, created_at
1736                     ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?
)
1737                     '', (
1738                         snapshot_id, ts_str, p.get('channel',
'System'),
1739                         int(p.get('total_analyzed', 0) or prof

```

```

_data.get('total_events', 0) or 0),
1740         int(p.get('unique_patterns_count', 0)
or prof_data.get('unique_templates_count', 0) or 0),
1741         json.dumps(p.get('critical_incidents')
or p.get('anomalies') or prof_data.get('critical_incidents',
[], ensure_ascii=False),
1742         json.dumps(p.get('top_clusters') or p.
get('top_patterns') or prof_data.get('top_patterns', []), ensu
re_ascii=False),
1743         json.dumps(p.get('bursts') or prof_dat
a.get('bursts', []), ensure_ascii=False),
1744         json.dumps(p.get('decision') or p.get(
'decision_gate') or {
1745             'strategy': p.get('strategy'),
1746             'strategy_rationale': p.get('strat
egy_rationale'),
1747             'recommended_llm_action': p.get('r
ecommanded_llm_action'),
1748             }, ensure_ascii=False),
1749         created_at,
1750     ))
1751
1752     conn.commit()
1753     return len(channels) + len(entries)
1754
1755     def save_firewall_snapshot(self, snapshot_id: str, profile
s: Any, rules: List[Any]) -> int:
1756         """Сохранение снимка профилей и правил брандмауэра Win
dows."""
1757         if self._cm.read_only:
1758             return 0
1759         now_dt = datetime.now(timezone.utc)
1760         ts_str = now_dt.isoformat()
1761         created_at = now_dt.timestamp()
1762
1763         p = profiles if isinstance(profiles, dict) else (profi
les.model_dump() if hasattr(profiles, 'model_dump') else vars(
profiles))

```

```

1764
1765         with self._cm.lock, self._cm.get_connection() as conn:
1766             cursor = conn.cursor()
1767             cursor.execute('''
1768                 INSERT INTO firewall_profile_snapshots (
1769                     snapshot_id, timestamp, domain_enabled, pr
1770                     ivate_enabled, public_enabled,
1771                     domain_default_inbound, private_default_in
1772                     bound, public_default_inbound,
1773                     stealth_mode_enabled, created_at
1774                     ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
1775             ''', (
1776                 snapshot_id, ts_str,
1777                 int(bool(p.get('domain_enabled', True))),
1778                 int(bool(p.get('private_enabled', True))),
1779                 int(bool(p.get('public_enabled', True))),
1780                 p.get('domain_default_inbound', 'Block'),
1781                 p.get('private_default_inbound', 'Block'),
1782                 p.get('public_default_inbound', 'Block'),
1783                 int(bool(p.get('stealth_mode_enabled', True)))
1784             ),
1785                 created_at,
1786             ))
1787
1788         for r in rules:
1789             d = r if isinstance(r, dict) else (r.model_dump() if hasattr(r, 'model_dump') else vars(r))
1790             cursor.execute('''
1791                 INSERT INTO firewall_rule_snapshots (
1792                     snapshot_id, rule_name, display_name,
1793                     direction, action, enabled,
1794                     protocol, local_port, remote_port, pro
1795                     gram_path, profile_mask, created_at
1796                     ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
1797             ''', (
1798                 snapshot_id, d.get('name') or d.get('rule_
1799                 name', ''),

```

```

1794         d.get('display_name') or d.get('name', '')
1795     ,
1796         d.get('direction', 'Inbound'),
1797         d.get('action', 'Allow'),
1798         int(bool(d.get('enabled', True))),
1799         d.get('protocol', 'Any'),
1800         str(d.get('local_port', '') or ''),
1801         str(d.get('remote_port', '') or ''),
1802         d.get('program_path') or d.get('program',
1803         ''),
1804         int(d.get('profile_mask', 0) or 0),
1805         created_at,
1806     ))
1807
1808     conn.commit()
1809     return len(rules) + 1
1810
1811     def update_firewall_rule_state(self, rule_name: str, enabled: bool) -> bool:
1812         """Атомарное обновление состояния правила брандмауэра
1813         в базе данных."""
1814         if self._cm.read_only:
1815             return False
1816         with self._cm.lock, self._cm.get_connection() as conn:
1817             cursor = conn.cursor()
1818             cursor.execute(
1819                 'UPDATE firewall_rule_snapshots SET enabled =
1820                 ? WHERE rule_name = ?',
1821                 (int(bool(enabled)), rule_name)
1822             )
1823             conn.commit()
1824             return cursor.rowcount > 0
1825
1826     def save_services_snapshot(self, snapshot_id: str, services: List[Any]) -> int:
1827         """Сохранение снимка состояния служб Windows."""
1828         if self._cm.read_only or not services:
1829             return 0

```

```

1826         now_dt = datetime.now(timezone.utc)
1827         ts_str = now_dt.isoformat()
1828         created_at = now_dt.timestamp()
1829
1830         with self._cm.lock, self._cm.get_connection() as conn:
1831             cursor = conn.cursor()
1832             for s in services:
1833                 d = s if isinstance(s, dict) else (s.model_dump
1834 | p() if hasattr(s, 'model_dump') else vars(s))
1835                 cursor.execute('''
1836 |             INSERT INTO services_snapshots (
1837 |                 snapshot_id, timestamp, service_name,
1838 | display_name, state,
1839 |                 state_code, start_type, pid, binary_pa
1840 | th, account, is_orphaned, created_at
1841 |             ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?,
1842 | ?)
1843 |             ', (
1844 |                 snapshot_id, ts_str,
1845 |                 d.get('name') or d.get('service_name', ''),
1846 |                 d.get('display_name'),
1847 |                 d.get('state') or d.get('status', 'STOPPED
1848 | '),
1849 |                 int(d.get('state_code', 0) or 0),
1850 |                 d.get('start_type') or d.get('startup_type
1851 | ', 'Manual'),
1852 |                 int(d.get('pid', 0) or 0) if d.get('pid')
1853 | else None,
1854 |                 d.get('binary_path') or d.get('binpath', '
1855 | '),
1856 |                 d.get('account') or d.get('user_account',
1857 | ''),
1858 |                 int(bool(d.get('is_orphaned', False))),
1859 |                 created_at,
1860 |             ))
1861             conn.commit()
1862             return len(services)

```



```

1854
1855     def record_service_change(
1856         self,
1857         service_name: str,
1858         display_name: str,
1859         action: str,
1860         old_state: Optional[str] = None,
1861         new_state: Optional[str] = None,
1862         performed_by: str = 'SYSTEM',
1863         details: Optional[Dict[str, Any]] = None,
1864     ) -> int:
1865         """Фиксация события изменения состояния службы."""
1866         if self._cm.read_only:
1867             return 0
1868         now_dt = datetime.now(timezone.utc)
1869         ts_str = now_dt.isoformat()
1870         created_at = now_dt.timestamp()
1871
1872         with self._cm.lock, self._cm.get_connection() as conn:
1873             cursor = conn.cursor()
1874             cursor.execute('''
1875                 INSERT INTO service_change_events (
1876                     timestamp, service_name, display_name, act
1877                     ion, old_state,
1878                     new_state, performed_by, details_json, cre
1879                     ated_at
1880                     ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
1881                 ''', (
1882                     ts_str, service_name, display_name, action, ol
1883                     d_state,
1884                     new_state, performed_by,
1885                     json.dumps(details, ensure_ascii=False) if det
1886                     ails else None,
1887                     created_at,

```

```

1888     def save_throttling_snapshot(
1889         self,
1890         snapshot_id: str,
1891         throttling_data: Any,
1892         thermal_zones: Optional[List[Any]] = None,
1893     ) -> int:
1894         """Сохранение снимка троттлинга процессора и ACPI терм
1895         ических зон."""
1896         if self._cm.read_only:
1897             return 0
1898         now_dt = datetime.now(timezone.utc)
1899         ts_str = now_dt.isoformat()
1900         created_at = now_dt.timestamp()
1901
1902         t = throttling_data if isinstance(throttling_data, dict) else (throttling_data.model_dump() if hasattr(throttling_data, 'model_dump') else vars(throttling_data))
1903
1904         with self._cm.lock, self._cm.get_connection() as conn:
1905             cursor = conn.cursor()
1906             cursor.execute('''
1907                 INSERT INTO cpu_throttling_snapshots (
1908                     snapshot_id, timestamp, prochot_active, pl1_limit_watts, pl2_limit_watts,
1909                     current_power_watts, max_core_temp_c, package_temp_c, dpc_latency_us,
1910                     isr_latency_us, throttling_reasons_json, created_at
1911                 ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
1912             ''', (
1913                 snapshot_id, ts_str,
1914                 int(bool(t.get('prochot_active', False))),
1915                 float(t.get('pl1_limit_watts', 0.0) or 0.0) if t.get('pl1_limit_watts') is not None else None,
1916                 float(t.get('pl2_limit_watts', 0.0) or 0.0) if t.get('pl2_limit_watts') is not None else None,
1917                 float(t.get('current_power_watts', 0.0) or 0.0) if t.get('current_power_watts') is not None else None,

```

```

1917 |         float(t.get('max_core_temp_c', 0.0) or 0.0) if
      | t.get('max_core_temp_c') is not None else None,
1918 |         float(t.get('package_temp_c', 0.0) or 0.0) if
      | t.get('package_temp_c') is not None else None,
1919 |         int(t.get('dpc_latency_us', 0) or 0),
1920 |         int(t.get('isr_latency_us', 0) or 0),
1921 |         json.dumps(t.get('throttling_reasons') or t.ge
      | t('reasons', []), ensure_ascii=False),
1922 |         created_at,
1923 |     ))
1924 |
1925 |     if thermal_zones:
1926 |         for z in thermal_zones:
1927 |             zd = z if isinstance(z, dict) else (z.mode
      | l_dump() if hasattr(z, 'model_dump') else vars(z))
1928 |             cursor.execute('''
1929 |                 INSERT INTO thermal_zone_snapshots (
1930 |                     snapshot_id, timestamp, zone_name,
      | temperature_c,
1931 |                     critical_limit_c, throttling_limit
      | _c, sensor_provider, created_at
1932 |                 ) VALUES (?, ?, ?, ?, ?, ?, ?, ?)
1933 |             ''', (
1934 |                 snapshot_id, ts_str,
1935 |                 zd.get('zone_name') or zd.get('name',
      | 'ThermalZone'),
1936 |                 float(zd.get('temperature_c', 0.0) or
      | 0.0),
1937 |                 float(zd.get('critical_limit_c', 100.0
      | ) or 100.0) if zd.get('critical_limit_c') is not None else Non
      | e,
1938 |                 float(zd.get('throttling_limit_c', 90.
      | 0) or 90.0) if zd.get('throttling_limit_c') is not None else N
      | one,
1939 |                 zd.get('sensor_provider', 'ACPI'),
1940 |                 created_at,
1941 |             ))
1942 |

```

```

1943         conn.commit()
1944         return 1 + (len(thermal_zones) if thermal_zones else 0)
1945
1946     def save_forensics_snapshot(self, snapshot_id: str, forensics_data: Any) -> int:
1947         """Сохранение снимка поведенческой форензики."""
1948         if self._cm.read_only:
1949             return 0
1950         now_dt = datetime.now(timezone.utc)
1951         ts_str = now_dt.isoformat()
1952         created_at = now_dt.timestamp()
1953
1954         f = forensics_data if isinstance(forensics_data, dict)
1955         else (forensics_data.model_dump() if hasattr(forensics_data, 'model_dump') else vars(forensics_data))
1956
1957         with self._cm.lock, self._cm.get_connection() as conn:
1958             cursor = conn.cursor()
1959             cursor.execute('''
1960                 INSERT INTO forensics_snapshots (
1961                     snapshot_id, timestamp, foreground_window_title, foreground_process_name,
1962                     foreground_pid, user_idle_seconds, camera_active_apps_json,
1963                     microphone_active_apps_json, userassist_to_p_apps_json, created_at
1964                     ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
1965                 ''', (
1966                     snapshot_id, ts_str,
1967                     f.get('foreground_window_title') or f.get('active_window_title', ''),
1968                     f.get('foreground_process_name') or f.get('active_process_name', ''),
1969                     int(f.get('foreground_pid', 0) or f.get('active_pid', 0) or 0),
1970                     int(f.get('user_idle_seconds', 0) or f.get('idle_seconds', 0) or 0),

```

```

1970         json.dumps(f.get('camera_active_apps') or f.ge
1971         t('camera_apps', []), ensure_ascii=False),
1972         json.dumps(f.get('microphone_active_apps') or
1973         f.get('microphone_apps', []), ensure_ascii=False),
1974         json.dumps(f.get('userassist_top_apps') or f.g
1975         et('userassist_apps', []), ensure_ascii=False),
1976         created_at,
1977     ))
1978     conn.commit()
1979     return cursor.lastrowid or 0
1980
1981 def save_process_leak_snapshot(
1982     self,
1983     snapshot_id: str,
1984     total_processes: Any,
1985     suspicious_count: int = 0,
1986     leak_items: Optional[List[Any]] = None,
1987 ) -> int:
1988     """Сохранение снимка аудита утечек ресурсов процессов.
1989     """
1990     if self._cm.read_only:
1991         return 0
1992     now_dt = datetime.now(timezone.utc)
1993     ts_str = now_dt.isoformat()
1994     created_at = now_dt.timestamp()
1995
1996     # Если передан объект/словарь отчета
1997     if hasattr(total_processes, 'model_dump') or (isinstan
1998     ce(total_processes, dict) and 'total_processes' in total_proce
1999     sses):
2000         rep_dict = total_processes.model_dump() if hasattr
2001         (total_processes, 'model_dump') else total_processes
2002         tot_proc = int(rep_dict.get('total_processes', 0))
2003         susp_cnt = int(rep_dict.get('suspicious_count', 0))
2004     )
2005     items_list = rep_dict.get('all_processes') or rep_
2006     dict.get('top_handle_hogs') or []
2007     else:

```

```

1999         tot_proc = int(total_processes or 0)
2000         susp_cnt = int(suspicious_count or 0)
2001         items_list = leak_items or []
2002
2003         with self._cm.lock, self._cm.get_connection() as conn:
2004             cursor = conn.cursor()
2005             cursor.execute('''
2006                 INSERT INTO process_leak_snapshots (
2007                     snapshot_id, timestamp, total_processes, s
uspicious_count, created_at
2008                     ) VALUES (?, ?, ?, ?, ?)
2009             ''', (
2010                 snapshot_id, ts_str, tot_proc, susp_cnt, creat
ed_at
2011             ))
2012
2013         for it in items_list:
2014             d = it if isinstance(it, dict) else (it.model_
dump() if hasattr(it, 'model_dump') else vars(it))
2015             cursor.execute('''
2016                 INSERT INTO process_leak_items (
2017                     snapshot_id, pid, name, handles_count,
gdi_objects, user_objects,
2018                     page_faults, working_set_mb, leak_risk
_score, leak_risk_reasons_json, created_at
2019                     ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
2020             ''', (
2021                 snapshot_id,
2022                 int(d.get('pid', 0) or 0),
2023                 d.get('name') or d.get('process_name', '')
,
2024                 int(d.get('handles_count', 0) or d.get('nu
m_handles', 0) or 0),
2025                 int(d.get('gdi_objects', 0) or 0),
2026                 int(d.get('user_objects', 0) or 0),
2027                 int(d.get('page_faults', 0) or 0),
2028                 float(d.get('working_set_mb', 0.0) or d.ge
t('memory_mb', 0.0) or 0.0),

```

```

2029         d.get('leak_risk_score', 'LOW'),
2030         json.dumps(d.get('leak_risk_reasons') or d
2031         .get('reasons', []), ensure_ascii=False),
2032         created_at,
2033     ))
2034     conn.commit()
2035     return len(leak_items) + 1
2036
2037     def save_defender_snapshot(
2038         self,
2039         snapshot_id: str,
2040         defender_status: Any,
2041         exclusions: Optional[List[Any]] = None,
2042         asr_rules: Optional[List[Any]] = None,
2043         threats: Optional[List[Any]] = None,
2044     ) -> int:
2045         """Сохранение снимка состояния Защитника Windows, искл
2046         ючений, ASR и угроз."""
2047         if self._cm.read_only:
2048             return 0
2049         now_dt = datetime.now(timezone.utc)
2050         ts_str = now_dt.isoformat()
2051         created_at = now_dt.timestamp()
2052
2053         s = defender_status if isinstance(defender_status, dic
2054         t) else (defender_status.model_dump() if hasattr(defender_stat
2055         us, 'model_dump') else vars(defender_status))
2056
2057         with self._cm.lock, self._cm.get_connection() as conn:
2058             cursor = conn.cursor()
2059             cursor.execute('''
2060                 INSERT INTO defender_snapshots (
2061                     snapshot_id, timestamp, real_time_protecti
2062                     on_enabled, cloud_protection_enabled,
2063                     behavior_monitoring_enabled, tamper_protecti
2064                     on_enabled, pua_protection_enabled, en
2065                     antivirus_enabled, antispyware_enabled, en

```

```

gine_version, av_signature_version,
2061         last_quick_scan_datetime, last_full_scan_d
atetime, security_score,
2062         cfa_state, raw_status_json, created_at
2063     ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?,
?, ?, ?, ?, ?)
2064     '', (
2065         snapshot_id, ts_str,
2066         int(bool(s.get('real_time_protection_enabled',
True))),
2067         int(bool(s.get('cloud_protection_enabled', Tru
e))),
2068         int(bool(s.get('behavior_monitoring_enabled',
True))),
2069         int(bool(s.get('tamper_protection_enabled', Tr
ue))),
2070         int(bool(s.get('pua_protection_enabled', True)
)),
2071         int(bool(s.get('antivirus_enabled', True))),
2072         int(bool(s.get('antispyware_enabled', True))),
2073         s.get('engine_version', ''),
2074         s.get('av_signature_version') or s.get('signat
ure_version', ''),
2075         s.get('last_quick_scan_datetime'),
2076         s.get('last_full_scan_datetime'),
2077         int(s.get('security_score', 100) or 100),
2078         int(s.get('cfa_state', 0) or 0),
2079         json.dumps(s, ensure_ascii=False) if isinstanc
e(s, dict) else None,
2080         created_at,
2081     ))
2082
2083     if exclusions:
2084         for exc in exclusions:
2085             d = exc if isinstance(exc, dict) else (exc
.model_dump() if hasattr(exc, 'model_dump') else vars(exc))
2086             cursor.execute(''
2087                 INSERT INTO defender_exclusions (

```



```

2088 |             snapshot_id, exclusion_type, exclu
| sion_value, risk_level, created_at
2089 |             ) VALUES (?, ?, ?, ?, ?)
2090 |             '', (
2091 |                 snapshot_id,
2092 |                 d.get('type') or d.get('exclusion_type
| ', 'path'),
2093 |                 d.get('value') or d.get('exclusion_val
| ue', ''),
2094 |                 d.get('risk_level', 'SAFE'),
2095 |                 created_at,
2096 |             ))
2097 |
2098 |         if asr_rules:
2099 |             for asr in asr_rules:
2100 |                 d = asr if isinstance(asr, dict) else (asr
| .model_dump() if hasattr(asr, 'model_dump') else vars(asr))
2101 |                 cursor.execute('''
2102 |                     INSERT INTO defender_asr_rules (
2103 |                         snapshot_id, rule_guid, rule_name,
| rule_action, created_at
2104 |                     ) VALUES (?, ?, ?, ?, ?)
2105 |                     '', (
2106 |                         snapshot_id,
2107 |                         d.get('guid') or d.get('rule_guid', ''
| ),
2108 |                         d.get('name') or d.get('rule_name', ''
| ),
2109 |                         d.get('action') or d.get('rule_action'
| , 'Block'),
2110 |                         created_at,
2111 |                     ))
2112 |
2113 |         if threats:
2114 |             for th in threats:
2115 |                 d = th if isinstance(th, dict) else (th.mo
| del_dump() if hasattr(th, 'model_dump') else vars(th))
2116 |                 cursor.execute('''

```

```

2117         INSERT INTO defender_threats (
2118             snapshot_id, threat_id, threat_name,
2119             severity, category, resources_json, detection_time, created_at
2120         ) VALUES (?, ?, ?, ?, ?, ?, ?, ?)
2121         ''' , (
2122             snapshot_id,
2123             d.get('threat_id') or d.get('id', ''),
2124             d.get('threat_name') or d.get('name',
2125             ''),
2126             d.get('severity', 'High'),
2127             d.get('category', 'Trojan'),
2128             json.dumps(d.get('resources', []), ensure_ascii=False),
2129             d.get('detection_time'),
2130             created_at,
2131         ))
2132
2133     conn.commit()
2134     return 1 + (len(exclusions) if exclusions else 0)
2135
2136     def save_process_network_snapshot(self, snapshot_id: str,
2137     network_items: List[Any]) -> int:
2138         """Сохранение снимка сетевой активности процессов."""
2139         if self._cm.read_only or not network_items:
2140             return 0
2141         now_dt = datetime.now(timezone.utc)
2142         ts_str = now_dt.isoformat()
2143         created_at = now_dt.timestamp()
2144
2145         with self._cm.lock, self._cm.get_connection() as conn:
2146             cursor = conn.cursor()
2147             for it in network_items:
2148                 d = it if isinstance(it, dict) else (it.model_dump() if hasattr(it, 'model_dump') else vars(it))
2149                 cursor.execute(''
2150                     INSERT INTO process_network_snapshots (
2151                         snapshot_id, timestamp, pid, process_n

```

```

ame, local_address,
2149         remote_address, protocol, status, serv
ice_type, sent_kb,
2150         recv_kb, read_speed_kbs, write_speed_k
bs, created_at
2151         ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?,
?, ?, ?)
2152         '', (
2153         snapshot_id, ts_str,
2154         int(d.get('pid', 0) or 0),
2155         d.get('process_name') or d.get('name', ''
),
2156         d.get('local_address', ''),
2157         d.get('remote_address', ''),
2158         d.get('protocol', 'TCP'),
2159         d.get('status', 'ESTABLISHED'),
2160         d.get('service_type', 'Internet'),
2161         float(d.get('sent_kb', 0.0) or 0.0),
2162         float(d.get('recv_kb', 0.0) or 0.0),
2163         float(d.get('read_speed_kbs', 0.0) or 0.0)
),
2164         float(d.get('write_speed_kbs', 0.0) or 0.0
),
2165         created_at,
2166         ))
2167         conn.commit()
2168         return len(network_items)
2169
2170     def save_software_inventory(
2171         self,
2172         apps: List[Any],
2173         storage_locations: Optional[List[Any]] = None,
2174         config_files: Optional[List[Any]] = None,
2175         ai_research: Optional[List[Any]] = None,
2176     ) -> int:
2177         """Сохранение инвентаря установленного ПО, мест хранен
ия, конфигов и AI-аналитики."""
2178         if self._cm.read_only or not apps:

```

```

2179         return 0
2180     now_dt = datetime.now(timezone.utc)
2181     ts_str = now_dt.isoformat()
2182
2183     with self._cm.lock, self._cm.get_connection() as conn:
2184         cursor = conn.cursor()
2185         for app in apps:
2186             d = app if isinstance(app, dict) else (app.model_dump() if hasattr(app, 'model_dump') else vars(app))
2187             app_id = d.get('app_id') or d.get('id') or d.get('name', '')
2188             cursor.execute('''
2189                 INSERT OR REPLACE INTO software_inventory
2190                 (
2191                     app_id, display_name, version, publisher, install_date,
2192                     install_location, architecture, is_system_component, first_seen, last_scanned_at
2193                     ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, COALESCE
2194                     ((SELECT first_seen FROM software_inventory WHERE app_id = ?),
2195                     ?), ?)
2196                 ''', (
2197                     app_id,
2198                     d.get('display_name') or d.get('name', app_id),
2199                     d.get('version', ''),
2200                     d.get('publisher', ''),
2201                     d.get('install_date', ''),
2202                     d.get('install_location') or d.get('path', ''),
2203                     d.get('architecture', 'x64'),
2204                     int(bool(d.get('is_system_component', False)))
2205                 ))
2206             cursor.execute('UPDATE software_inventory SET first_seen = ?, last_scanned_at = ? WHERE app_id = ?',
2207                             (app_id, ts_str, ts_str, app_id))
2208         if storage_locations:
2209             for loc in storage_locations:

```

```

2207         d = loc if isinstance(loc, dict) else (loc
2208         .model_dump() if hasattr(loc, 'model_dump') else vars(loc))
2209         cursor.execute('''
2210             INSERT INTO software_storage_locations
2211             (
2212                 app_id, category, path, size_bytes
2213                 , file_count, last_updated
2214             ) VALUES (?, ?, ?, ?, ?, ?)
2215             ''', (
2216                 d.get('app_id', ''),
2217                 d.get('category', 'data'),
2218                 d.get('path', ''),
2219                 int(d.get('size_bytes', 0) or 0),
2220                 int(d.get('file_count', 0) or 0),
2221                 ts_str,
2222             ))
2223
2224     if config_files:
2225         for cf in config_files:
2226             d = cf if isinstance(cf, dict) else (cf.mo
2227             del_dump() if hasattr(cf, 'model_dump') else vars(cf))
2228             cursor.execute('''
2229                 INSERT INTO software_config_files (
2230                     app_id, file_path, format, size_by
2231                     tes, snippet
2232                 ) VALUES (?, ?, ?, ?, ?)
2233                 ''', (
2234                     d.get('app_id', ''),
2235                     d.get('file_path') or d.get('path', ''
2236                     ),
2237                     d.get('format', 'json'),
2238                     int(d.get('size_bytes', 0) or 0),
2239                     d.get('snippet', ''),
2240                 ))
2241
2242     if ai_research:
2243         for ai in ai_research:
2244             d = ai if isinstance(ai, dict) else (ai.mo

```

```

del_dump() if hasattr(ai, 'model_dump') else vars(ai))
2239         app_id = d.get('app_id', '')
2240         cursor.execute('''
2241             INSERT OR REPLACE INTO software_ai_res
search (
2242                 app_id, summary, config_purpose_ex
planation, data_storage_explanation,
2243                 network_activity_explanation, conf
irmed_facts_json, inferred_facts_json,
2244                 confidence_level, updated_at
2245             ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
2246         ''', (
2247             app_id,
2248             d.get('summary', ''),
2249             d.get('config_purpose_explanation', ''
),
2250             d.get('data_storage_explanation', ''),
2251             d.get('network_activity_explanation',
''),
2252             json.dumps(d.get('confirmed_facts', []
), ensure_ascii=False),
2253             json.dumps(d.get('inferred_facts', []
), ensure_ascii=False),
2254             float(d.get('confidence_level', 0.0) o
r 0.0),
2255             ts_str,
2256         ))
2257
2258         conn.commit()
2259         return len(apps)
2260
2261     def save_software_network_snapshots(self, snapshot_id: str
, net_items: List[Any]) -> int:
2262         """Сохранение снимков сетевой активности ПО."""
2263         if self._cm.read_only or not net_items:
2264             return 0
2265         now_dt = datetime.now(timezone.utc)
2266         ts_str = now_dt.isoformat()

```

```

2267
2268         with self._cm.lock, self._cm.get_connection() as conn:
2269             cursor = conn.cursor()
2270             for it in net_items:
2271                 d = it if isinstance(it, dict) else (it.model_
dump() if hasattr(it, 'model_dump') else vars(it))
2272                 cursor.execute('''
2273                     INSERT INTO software_network_snapshots (
2274                         snapshot_id, timestamp, app_id, pid, l
ocal_address,
2275                         remote_address, protocol, domain_name,
sent_kb, recv_kb
2276                     ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
2277                 ''', (
2278                     snapshot_id, ts_str,
2279                     d.get('app_id', ''),
2280                     int(d.get('pid', 0) or 0),
2281                     d.get('local_address', ''),
2282                     d.get('remote_address', ''),
2283                     d.get('protocol', 'TCP'),
2284                     d.get('domain_name', ''),
2285                     float(d.get('sent_kb', 0.0) or 0.0),
2286                     float(d.get('recv_kb', 0.0) or 0.0),
2287                 ))
2288             conn.commit()
2289             return len(net_items)
2290
2291     def save_security_events(
2292         self,
2293         events: List[Any],
2294         save_raw: bool = False,
2295         raw_events: Optional[List[Any]] = None,
2296     ) -> int:
2297         """Сохраняет список нормализованных событий журнала бе
зопасности в SQLite.
2298
2299         Args:
2300             events: Список объектов SecurityEventItem или слов

```

арей.

2301 save_raw: Сохранять ли параллельно сырые XML событ
ия в security_events_raw.

2302 raw_events: Список объектов SecurityEventRaw или с
ловарей.

2303

2304 Returns:

2305 int: Количество сохраненных событий.

2306 """

2307 if self._cm.read_only or not events:

2308 return 0

2309

2310 with self._cm.lock, self._cm.get_connection() as conn:

2311 cursor = conn.cursor()

2312 for ev in events:

2313 d = ev if isinstance(ev, dict) else (ev.model_
dump() if hasattr(ev, 'model_dump') else vars(ev))

2314 ed_json = json.dumps(d.get('event_data', {}),
ensure_ascii=False) if d.get('event_data') else None

2315

2316 cursor.execute('''

2317 INSERT INTO security_events (

2318 event_record_id, event_id, timestamp,
created_at, computer,

2319 channel, level, subject_user, subject_
domain, subject_sid,

2320 target_user, target_domain, target_sid
, process_id,

2321 process_name, parent_process_id, paren
t_process_name,

2322 command_line, logon_id, logon_type, el
evated_token,

2323 source_ip, source_port, object_name, s
tatus_code,

2324 message, event_data_json, ingested_at

2325) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?,
?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)

2326 ''', (


```

2327         int(d.get('event_record_id', 0) or 0),
2328         int(d.get('event_id', 0) or 0),
2329         str(d.get('timestamp', '')),
2330         float(d.get('created_at', time.time()) or
time.time()),
2331         str(d.get('computer', '')),
2332         str(d.get('channel', 'Security')),
2333         str(d.get('level', 'Information')),
2334         str(d.get('subject_user', '')),
2335         str(d.get('subject_domain', '')),
2336         str(d.get('subject_sid', '')),
2337         str(d.get('target_user', '')),
2338         str(d.get('target_domain', '')),
2339         str(d.get('target_sid', '')),
2340         int(d.get('process_id', 0) or 0),
2341         str(d.get('process_name', '')),
2342         int(d.get('parent_process_id', 0) or 0),
2343         str(d.get('parent_process_name', '')),
2344         str(d.get('command_line', '')),
2345         str(d.get('logon_id', '')),
2346         d.get('logon_type'),
2347         d.get('elevated_token'),
2348         str(d.get('source_ip', '')),
2349         d.get('source_port'),
2350         str(d.get('object_name', '')),
2351         str(d.get('status_code', '')),
2352         str(d.get('message', '')),
2353         ed_json,
2354         str(d.get('ingested_at', datetime.now(time
zone.utc).isoformat()))),
2355     ))
2356
2357     if save_raw and raw_events:
2358         for raw in raw_events:
2359             rd = raw if isinstance(raw, dict) else (ra
w.model_dump() if hasattr(raw, 'model_dump') else vars(raw))
2360             cursor.execute(''
2361                 INSERT INTO security_events_raw (

```

```

2362 |                                     event_record_id, event_id, timesta
      | mp, created_at,
2363 |                                     channel, raw_xml, ingested_at
2364 |                                     ) VALUES (?, ?, ?, ?, ?, ?, ?)
2365 |                                     ''' , (
2366 |                                     int(rd.get('event_record_id', 0) or 0)
      | ,
2367 |                                     int(rd.get('event_id', 0) or 0),
2368 |                                     str(rd.get('timestamp', '')),
2369 |                                     float(rd.get('created_at', time.time()
      | ) or time.time()),
2370 |                                     str(rd.get('channel', 'Security')),
2371 |                                     str(rd.get('raw_xml', '')),
2372 |                                     str(rd.get('ingested_at', datetime.now
      | (timezone.utc).isoformat()))),
2373 |                                     ))
2374 |
2375 |                                     conn.commit()
2376 |                                     return len(events)
2377 |
2378 |     def save_security_bookmark(
2379 |         self,
2380 |         channel: str,
2381 |         last_record_id: int,
2382 |         bookmark_xml: Optional[str] = None,
2383 |         last_timestamp: str = '',
2384 |     ) -> None:
2385 |         """Сохраняет или обновляет состояние закладки инкремен
      | тального сбора в SQLite.
2386 |
2387 |         Args:
2388 |             channel: Имя канала событий (например, 'Security')
      | .
2389 |             last_record_id: Последний обработанный EventRecord
      | ID.
2390 |             bookmark_xml: XML-представление закладки WevtAPI.
2391 |             last_timestamp: Временная метка последнего обработ
      | анного события.

```

```

2392 |         """
2393 |         if self._cm.read_only:
2394 |             return
2395 |
2396 |         now_epoch = time.time()
2397 |         with self._cm.lock, self._cm.get_connection() as conn:
2398 |             cursor = conn.cursor()
2399 |             cursor.execute('''
2400 |                 INSERT INTO security_collector_bookmarks (
2401 |                     channel, last_record_id, bookmark_xml, las
2402 | t_timestamp, updated_at
2403 |                 ) VALUES (?, ?, ?, ?, ?)
2404 |                 ON CONFLICT(channel) DO UPDATE SET
2405 |                     last_record_id = excluded.last_record_id,
2406 |                     bookmark_xml = CASE WHEN excluded.bookmark
2407 | _xml IS NOT NULL THEN excluded.bookmark_xml ELSE security_coll
2408 | ector_bookmarks.bookmark_xml END,
2409 |                     last_timestamp = excluded.last_timestamp,
2410 |                     updated_at = excluded.updated_at
2411 |             ''', (
2412 |                 channel,
2413 |                 int(last_record_id or 0),
2414 |                 bookmark_xml,
2415 |                 last_timestamp,
2416 |                 now_epoch,
2417 |             ))
2418 |             conn.commit()
2419 |
2420 |     def save_power_events(self, events: List[Any]) -> int:
2421 |         """Сохраняет пакет сырых/нормализованных событий питан
2422 | ия в таблицу power_events.
2423 |
2424 |         Args:
2425 |             events: Список объектов PowerEventRecord или слова
2426 | рей.
2427 |
2428 |         Returns:
2429 |             int: Количество сохраненных записей.

```

```

2425 |         """
2426 |         if not events or self._cm.read_only:
2427 |             return 0
2428 |
2429 |         now_epoch = time.time()
2430 |         saved = 0
2431 |         with self._cm.lock, self._cm.get_connection() as conn:
2432 |             cursor = conn.cursor()
2433 |             for ev in events:
2434 |                 if hasattr(ev, 'model_dump'):
2435 |                     d = ev.model_dump()
2436 |                 elif isinstance(ev, dict):
2437 |                     d = ev
2438 |                 else:
2439 |                     continue
2440 |
2441 |                 eid = int(d.get('event_id', 0))
2442 |                 provider = str(d.get('provider') or '')
2443 |                 channel = str(d.get('channel') or 'System')
2444 |                 ts = str(d.get('timestamp') or '')
2445 |                 c_at = float(d.get('created_at') or now_epoch)
2446 |                 event_type = str(d.get('event_type') or 'event
2447 | ')
2448 |
2449 |                 shutdown_type = d.get('shutdown_type')
2450 |                 user = d.get('user')
2451 |                 domain = d.get('domain')
2452 |                 process = d.get('process')
2453 |                 pid = d.get('process_id')
2454 |                 reason = d.get('reason')
2455 |                 reason_code = d.get('reason_code')
2456 |                 comment = d.get('comment')
2457 |                 unexpected = 1 if d.get('unexpected') else 0
2458 |                 bugcheck_code = d.get('bugcheck_code')
2459 |                 bugcheck_params = d.get('bugcheck_params_json'
2460 | )
2461 |
2462 |                 if isinstance(bugcheck_params, (dict, list)):
2463 |                     bugcheck_params = json.dumps(bugcheck_params, ensure_ascii=False)

```

```

2460         boot_id = d.get('boot_id')
2461         details_json = d.get('details_json')
2462         if isinstance(details_json, (dict, list)):
2463             details_json = json.dumps(details_json, en
sure_ascii=False)
2464         raw_xml = d.get('raw_xml')
2465
2466         cursor.execute('''
2467             INSERT INTO power_events (
2468                 event_id, provider, channel, timestamp
, created_at,
2469                 event_type, shutdown_type, user, domai
n, process,
2470                 process_id, reason, reason_code, comme
nt, unexpected,
2471                 bugcheck_code, bugcheck_params_json, b
oot_id, details_json, raw_xml
2472             ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?,
?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
2473             ON CONFLICT(event_id, timestamp, provider)
DO UPDATE SET
2474                 event_type = excluded.event_type,
2475                 shutdown_type = excluded.shutdown_type
,
2476                 user = excluded.user,
2477                 domain = excluded.domain,
2478                 process = excluded.process,
2479                 process_id = excluded.process_id,
2480                 reason = excluded.reason,
2481                 reason_code = excluded.reason_code,
2482                 comment = excluded.comment,
2483                 unexpected = excluded.unexpected,
2484                 bugcheck_code = excluded.bugcheck_code
,
2485                 bugcheck_params_json = excluded.bugche
ck_params_json,
2486                 boot_id = excluded.boot_id,
2487                 details_json = excluded.details_json,

```

```

2488         raw_xml = excluded.raw_xml
2489         ''' , (
2490             eid, provider, channel, ts, c_at,
2491             event_type, shutdown_type, user, domain, p
process,
2492             pid, reason, reason_code, comment, unexpec
ted,
2493             bugcheck_code, bugcheck_params, boot_id, d
etails_json, raw_xml
2494         ))
2495         saved += 1
2496
2497         conn.commit()
2498         return saved
2499
2500     def save_power_sessions(self, sessions: List[Any]) -> int:
2501         """Сохраняет пакет реконструированных сессий питания в
таблицу power_sessions.
2502
2503         Args:
2504             sessions: Список объектов PowerSessionRecord или с
ловарей.
2505
2506         Returns:
2507             int: Количество сохраненных/обновленных сессий.
2508         """
2509         if not sessions or self._cm.read_only:
2510             return 0
2511
2512         now_epoch = time.time()
2513         saved = 0
2514         with self._cm.lock, self._cm.get_connection() as conn:
2515             cursor = conn.cursor()
2516             for s in sessions:
2517                 if hasattr(s, 'model_dump'):
2518                     d = s.model_dump()
2519                 elif isinstance(s, dict):
2520                     d = s

```

```

2521         else:
2522             continue
2523
2524         session_id = str(d.get('session_id') or '')
2525         if not session_id:
2526             continue
2527
2528         boot_time = str(d.get('boot_time') or '')
2529         boot_ts = float(d.get('boot_timestamp') or 0.0
2530     )
2531         shutdown_time = d.get('shutdown_time')
2532         shutdown_ts = float(d.get('shutdown_timestamp'
2533     ) or 0.0) if d.get('shutdown_timestamp') is not None else None
2534         uptime_sec = float(d.get('uptime_seconds') or
2535     0.0)
2536         uptime_human = str(d.get('uptime_human') or ''
2537     )
2538         shutdown_type = str(d.get('shutdown_type') or
2539     'Active')
2540         initiator = d.get('initiator')
2541         process = d.get('process')
2542         reason = d.get('reason')
2543         reason_code = d.get('reason_code')
2544         comment = d.get('comment')
2545         clean = 1 if d.get('clean_shutdown', True) else
2546     0
2547         unexpected = 1 if d.get('unexpected_shutdown',
2548     False) else 0
2549         bugcheck = d.get('bugcheck')
2550         boot_eid = int(d.get('boot_event_id', 12))
2551         shutdown_eid = int(d.get('shutdown_event_id'))
2552         if d.get('shutdown_event_id') is not None else None
2553
2554         initiator_chain = d.get('initiator_chain') or
2555     []
2556         if isinstance(initiator_chain, list):
2557             chain_json = json.dumps(initiator_chain, e
2558     nsure_ascii=False)

```

```

2549         else:
2550             chain_json = str(initiator_chain)
2551
2552             events = d.get('events') or []
2553             if isinstance(events, (list, dict)):
2554                 events_json = json.dumps(events, ensure_as
cii=False)
2555             else:
2556                 events_json = str(events)
2557
2558             c_at = float(d.get('created_at') or now_epoch)
2559
2560             cursor.execute('''
2561                 INSERT INTO power_sessions (
2562                     session_id, boot_time, boot_timestamp,
shutdown_time, shutdown_timestamp,
2563                     uptime_seconds, uptime_human, shutdown
_type, initiator, process,
2564                     reason, reason_code, comment, clean_sh
utdown, unexpected_shutdown,
2565                     bugcheck, boot_event_id, shutdown_even
t_id, initiator_chain_json,
2566                     events_json, created_at
2567                 ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?,
?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
2568                 ON CONFLICT(session_id) DO UPDATE SET
2569                     boot_time = excluded.boot_time,
2570                     boot_timestamp = excluded.boot_timesta
mp,
2571                     shutdown_time = excluded.shutdown_time
,
2572                     shutdown_timestamp = excluded.shutdown
_timestamp,
2573                     uptime_seconds = excluded.uptime_secon
ds,
2574                     uptime_human = excluded.uptime_human,
2575                     shutdown_type = excluded.shutdown_type
,

```



```

2576         initiator = excluded.initiator,
2577         process = excluded.process,
2578         reason = excluded.reason,
2579         reason_code = excluded.reason_code,
2580         comment = excluded.comment,
2581         clean_shutdown = excluded.clean_shutdown,
2582         unexpected_shutdown = excluded.unexpected_shutdown,
2583         bugcheck = excluded.bugcheck,
2584         boot_event_id = excluded.boot_event_id
2585     ,
2586         shutdown_event_id = excluded.shutdown_event_id,
2587         initiator_chain_json = excluded.initiator_chain_json,
2588         events_json = excluded.events_json,
2589         created_at = excluded.created_at
2590     ''' , (
2591         session_id, boot_time, boot_ts, shutdown_time, shutdown_ts,
2592         uptime_sec, uptime_human, shutdown_type, initiator, process,
2593         reason, reason_code, comment, clean, unexpected,
2594         bugcheck, boot_eid, shutdown_eid, chain_json,
2595         events_json, c_at
2596     ))
2597     saved += 1
2598     conn.commit()
2599     return saved
2600
2601     def insert_process_pid_snapshots(self, snapshots: List[Union[Dict[str, Any], Any]]) -> int:
2602         """Пакетная вставка снимков ресурсов процессов по PID в process_pid_snapshots."""

```

```

2603         if not snapshots:
2604             return 0
2605         now_ts = datetime.now(timezone.utc).isoformat()
2606         rows = []
2607         for s in snapshots:
2608             d = s if isinstance(s, dict) else (s.model_dump()
2609 if hasattr(s, 'model_dump') else vars(s))
2610             mem_mb = float(d.get('memory_mb', 0.0) or 0.0)
2611             ws_bytes = int(d.get('working_set_bytes') or (mem_
2612 mb * 1024 * 1024))
2613             rows.append((
2614                 int(d.get('pid', 0)),
2615                 str(d.get('process_name', d.get('name', 'unkno
2616 wn'))),
2617                 d.get('executable_path', d.get('executable', N
2618 one)),
2619                 float(d.get('cpu_percent', 0.0) or 0.0),
2620                 int(d.get('user_time_ms', 0) or 0),
2621                 int(d.get('kernel_time_ms', 0) or 0),
2622                 int(d.get('thread_count', d.get('num_threads',
2623 1)) or 1),
2624                 ws_bytes,
2625                 int(d.get('private_bytes', 0) or 0),
2626                 int(d.get('page_faults_count', d.get('page_fau
2627 lts', 0)) or 0),
2628                 int(d.get('gpu_vram_bytes', 0) or 0),
2629                 float(d.get('gpu_utilization', 0.0) or 0.0),
2630                 int(d.get('read_bytes_total', d.get('read_byte
2631 s_sec', 0)) or 0),
2632                 int(d.get('write_bytes_total', d.get('write_by
2633 tes_sec', 0)) or 0),
2634                 int(d.get('read_ops_total', 0) or 0),
2635                 int(d.get('write_ops_total', 0) or 0),
2636                 int(d.get('net_bytes_sent_total', 0) or 0),
2637                 int(d.get('net_bytes_recv_total', 0) or 0),
2638                 int(d.get('handle_count', d.get('num_handles',
2639 0)) or 0),
2640                 int(d.get('gdi_objects', 0) or 0),

```

```

2632         int(d.get('user_objects', 0) or 0),
2633         d.get('timestamp') or now_ts
2634     ))
2635
2636     with self._cm.lock, self._cm.get_connection() as conn:
2637         cursor = conn.cursor()
2638         cursor.executemany('''
2639             INSERT INTO process_pid_snapshots (
2640                 pid, process_name, executable_path,
2641                 cpu_percent, user_time_ms, kernel_time_ms,
2642                 thread_count,
2643                 working_set_bytes, private_bytes, page_faults_count,
2644                 gpu_vram_bytes, gpu_utilization,
2645                 read_bytes_total, write_bytes_total, read_ops_total, write_ops_total,
2646                 net_bytes_sent_total, net_bytes_recv_total,
2647                 handle_count, gdi_objects, user_objects, timestamp
2648                 ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?,
2649                 ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
2650             ''', rows)
2651         conn.commit()
2652     return len(rows)
2653
2654     def insert_process_file_events(self, events: List[Union[Dict[str, Any], Any]]) -> int:
2655         """Пакетная вставка событий файловой активности в process_file_events."""
2656         if not events:
2657             return 0
2658         now_ts = datetime.now(timezone.utc).isoformat()
2659         now_epoch = time.time()
2660         rows = []
2661         for ev in events:
2662             d = ev if isinstance(ev, dict) else (ev.model_dump() if hasattr(ev, 'model_dump') else vars(ev))

```

```

2661         rows.append((
2662             int(d.get('pid', 0)),
2663             str(d.get('process_name', d.get('name', ''))),
2664             str(d.get('action_type', d.get('action', 'MODI
FY')))),
2665             str(d.get('target_directory', d.get('target_fo
lder', ''))),
2666             str(d.get('file_path', '')),
2667             int(d.get('bytes_affected', d.get('bytes_count
', 0)) or 0),
2668             d.get('instance_id', None),
2669             str(d.get('timestamp') or now_ts),
2670             float(d.get('created_at') or now_epoch)
2671         ))
2672
2673     with self._cm.lock, self._cm.get_connection() as conn:
2674         cursor = conn.cursor()
2675         cursor.executemany('''
2676             INSERT INTO process_file_events (
2677                 pid, process_name, action_type, target_dir
ectory,
2678                 file_path, bytes_affected, instance_id, ti
mestamp, created_at
2679                 ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
2680             ''', rows)
2681         conn.commit()
2682     return len(rows)
2683
2684
2685
2686
2687

```

__init__.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - Sqlite
  Package
4 # =====
5 # Description:
6 #   Пакет персистентного хранения и выборки системной телеметр
  ии в SQLite.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.sqlite import TelemetryStora
  ge, TelemetryReader
11 #
12 #       storage = TelemetryStorage.get_instance(read_only=True)
13 #       cpu = storage.get_latest_cpu()
14 #
15 # File: __init__.py
16 # Project: ai-breadboard
17 # Package: apps.windows.telemetry.sqlite
18 # Author: hypo69
19 # Copyright: © 2026 hypo69
20 # Updated: 2026-10-06 00:20:00
21 # =====
22
23 from __future__ import annotations
24
25 """Пакет персистентного хранения, буферизации и выборки систем
  ной телеметрии в SQLite."""
26
27 from .aggregator import AggregationLevel, TelemetrySqlAggregat
  or, sensors_aggregate
28 from .buffer import TelemetryBuffer
```

```
29 | from .connection import TelemetryConnectionManager
30 | from .maintenance import TelemetryMaintenance
31 | from .reader import TelemetryReader
32 | from .schema import init_database_schema
33 | from .storage import TelemetryStorage
34 | from .writer import TelemetryWriter
35 |
36 | SQLiteTelemetryStorage = TelemetryStorage
37 |
38 | __all__ = [
39 |     'TelemetryStorage',
40 |     'SQLiteTelemetryStorage',
41 |     'TelemetryReader',
42 |     'TelemetryWriter',
43 |     'TelemetryBuffer',
44 |     'TelemetryMaintenance',
45 |     'TelemetryConnectionManager',
46 |     'TelemetrySqlAggregator',
47 |     'AggregationLevel',
48 |     'sensors_aggregate',
49 |     'init_database_schema',
50 | ]
51 |
52 |
```

init_db.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - Init Db
4 # =====
5 # Description:
6 #   Модуль проверки, создания и валидации целостности структур
7 #   ы базы данных
8 # Usage Examples:
9 #   CLI:
10 #       python -m apps.windows.telemetry.init_db
11 #   Python API:
12 #       from apps.windows.telemetry.init_db import get_default_t
13 #       telemetry_db_path
14 #       res = get_default_telemetry_db_path()
15 #
16 # File: init_db.py
17 # Project: ai-breadboard
18 # Package: apps.windows.telemetry
19 # Author: hypo69
20 # Copyright: © 2026 hypo69
21 # Updated: 2026-10-06 07:50:00
22 # =====
23
24 from __future__ import annotations
25 """Модуль проверки, создания и валидации целостности структуры
26   базы данных"""
27
28 import argparse
29 import json
30 import os
31 from pathlib import Path
```

```

31 import sqlite3
32 import sys
33 from typing import Any, Dict, List, Optional, Union
34
35 _project_root = str(Path(__file__).resolve().parents[3])
36 if _project_root not in sys.path:
37     sys.path.insert(0, _project_root)
38 if __package__ in (None, ''):
39     __package__ = 'apps.windows.telemetry'
40
41 try:
42     from logger import logger
43 except ImportError:
44     import logging
45     logger = logging.getLogger(__name__)
46
47 from apps.windows.telemetry.sqlite import TelemetryStorage
48
49
50 def get_default_telemetry_db_path() -> Path:
51     """Возвращает стандартный путь к файлу базы данных telemetry.db.
52
53     Returns:
54         Path: Путь к файлу telemetry.db в каталоге логов приложения.
55     """
56     programdata = os.environ.get('ProgramData') or os.environ.get('ALLUSERSPROFILE')
57     if programdata and os.path.exists(programdata):
58         base_dir = Path(programdata)
59     else:
60         appdata = os.environ.get('APPDATA') or os.environ.get('LOCALAPPDATA')
61         if appdata and os.path.exists(appdata):
62             base_dir = Path(appdata)
63         else:
64             base_dir = Path.home() / '.config'

```



```

65 |     return base_dir / 'AI-Breadboard' / 'apps' / 'windows' / '
    | telemetry' / 'logs' / 'telemetry.db'
66 |
67 |
68 | def init_telemetry_database(
69 |     db_path: Optional[Union[str, Path]] = None,
70 |     force: bool = False,
71 |     check_integrity: bool = True,
72 | ) -> Dict[str, Any]:
73 |     """Инициализирует базу данных telemetry.db, создавая все т
    | аблицы и индексы при отсутствии.
74 |
75 |     Args:
76 |         db_path: Путь к целевому файлу SQLite базы данных (по
    | умолчанию стандартный путь).
77 |         force: Принудительно переинициализировать структуру та
    | блиц.
78 |         check_integrity: Выполнить проверку целостности PRAGMA
    | integrity_check.
79 |
80 |     Returns:
81 |         Dict[str, Any]: Словарь с результатами инициализации и
    | метаданными БД.
82 |     """
83 |     target_path = Path(db_path) if db_path else get_default_te
    |lemetry_db_path()
84 |     target_path.parent.mkdir(parents=True, exist_ok=True)
85 |
86 |     was_created = False
87 |     if not target_path.exists() or force:
88 |         logger.debug(f'Инициализация новой базы данных телемет
    |рии по пути: {target_path}')
89 |         storage = TelemetryStorage(db_path=target_path)
90 |         was_created = True
91 |     else:
92 |         storage = TelemetryStorage.get_instance(db_path=target
    |_path)
93 |

```

```

94 |     # Получение списка созданных таблиц
95 |     tables: List[str] = []
96 |     integrity_ok = True
97 |     integrity_msg = 'ok'
98 |
99 |     try:
100 |         with sqlite3.connect(str(target_path)) as conn:
101 |             cursor = conn.cursor()
102 |             cursor.execute("SELECT name FROM sqlite_master WHE
RE type='table' AND name NOT LIKE 'sqlite_%';")
103 |             tables = sorted([row[0] for row in cursor.fetchall
104 |                             (())])
105 |
106 |             if check_integrity:
107 |                 cursor.execute('PRAGMA integrity_check;')
108 |                 res = cursor.fetchone()
109 |                 if res and res[0] != 'ok':
110 |                     integrity_ok = False
111 |                     integrity_msg = str(res[0])
112 |     except Exception as ex:
113 |         logger.error(f'Ошибка проверки схемы и целостности БД
114 | {target_path}: {ex}')
115 |         integrity_ok = False
116 |         integrity_msg = str(ex)
117 |
118 |     size_bytes = target_path.stat().st_size if target_path.exi
119 | sts() else 0
120 |     size_mb = round(size_bytes / (1024 * 1024), 3)
121 |
122 |     result = {
123 |         'status': 'success' if integrity_ok else 'warning',
124 |         'db_path': str(target_path),
125 |         'exists': target_path.exists(),
126 |         'created': was_created,
127 |         'size_bytes': size_bytes,
128 |         'size_mb': size_mb,
129 |         'tables_count': len(tables),
130 |         'tables': tables,

```

```
128         'integrity_ok': integrity_ok,
129         'integrity_message': integrity_msg,
130     }
131
132     if was_created:
133         logger.info(f'    База данных telemetry.db успешно созда
на: {len(tables)} таблиц ({size_mb} МБ)')
134     else:
135         logger.debug(f'База данных telemetry.db подтверждена:
{len(tables)} таблиц ({size_mb} МБ)')
136
137     return result
138
139
140 def main() -> int:
141     """Точка входа CLI для инициализации и проверки базы данны
х телеметрии."""
142     parser = argparse.ArgumentParser(
143         description='Инициализатор и верификатор SQLite базы д
анных системной телеметрии (telemetry.db)'
144     )
145     parser.add_argument('--db-path', type=str, default=None, h
elp='Пользовательский путь к telemetry.db')
146     parser.add_argument('--force', action='store_true', help='
Принудительно переинициализировать структуру')
147     parser.add_argument('--check-only', action='store_true', h
elp='Только проверить существующую БД без создания')
148     parser.add_argument('--json', action='store_true', help='В
ывод результата в формате JSON')
149
150     args = parser.parse_args()
151
152     target_path = Path(args.db_path) if args.db_path else get_
default_telemetry_db_path()
153
154     if args.check_only and not target_path.exists():
155         if args.json:
156             print(json.dumps({'status': 'error', 'message': f'
```

```

    Файл не найден: {target_path}', 'exists': False}, ensure_ascii
    =False))
157         else:
158             print(f'❌ База данных телеметрии не найдена: {target_path}')
159             return 1
160
161     res = init_telemetry_database(db_path=args.db_path, force=
    args.force)
162
163     if args.json:
164         print(json.dumps(res, ensure_ascii=False, indent=2))
165     else:
166         status_icon = '✅ ' if res['integrity_ok'] else '❌ '
167         print(f"\n{status_icon} База данных телеметрии: {res['
    db_path']}")
168         print(f"    • Существует:      {res['exists']}")
169         print(f"    • Вновь создана:  {res['created']}")
170         print(f"    • Размер файла:   {res['size_mb']} МБ ({res
    ['size_bytes']} байт)")
171         print(f"    • Таблиц в схеме: {res['tables_count']}")
172         print(f"    • Целостность:   {res['integrity_message']
    }")
173         if res['tables']:
174             print(f"    • Таблицы:           {'', '.join(res['table
    s'][:8])){'...' if len(res['tables']) > 8 else ''}")
175             print('')
176
177     return 0 if res['integrity_ok'] else 1
178
179
180 init_telemetry_db = init_telemetry_database
181
182 __all__ = [
183     "init_telemetry_database",
184     "init_telemetry_db",
185     "get_default_telemetry_db_path",
186     "verify_telemetry_db_integrity",

```

```
187 | ]
188 |
189 |
190 | if __name__ == '__main__':
191 |     sys.exit(main())
192 |
193 |
```

ring_buffer.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - Ring Bu
  ffer
4 # =====
5 # Description:
6 #   Потокбезопасный кольцевой буфер в оперативной памяти для
  удержания
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.ring_buffer import Telemetry
  RingBuffer
11 #
12 #       service = TelemetryRingBuffer()
13 #
14 # File: ring_buffer.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-06 05:32:00
20 # =====
21
22 from __future__ import annotations
23 """Потокбезопасный кольцевой буфер в оперативной памяти для у
  держания сырой телеметрии."""
24
25 import collections
26 import json
27 import sys
28 import threading
29 import time
```

```

30 from datetime import datetime, timezone
31 from typing import Any, Dict, List, Optional, Tuple
32
33
34 class TelemetryRingBuffer:
35     """Потокобезопасный кольцевой буфер для хранения сырых мет
    | рик телеметрии в RAM с ограничением по объему памяти."""
36
37     def __init__(self, capacity: int = 600, max_bytes: int = 3
    | 2 * 1024 * 1024) -> None:
38         """Инициализирует кольцевой буфер заданной емкости и м
    | аксимального объема RAM.
39
40         Args:
41             capacity: Максимальное количество удерживаемых зап
    | исей (по умолчанию 600 = 10 мин при 1с).
42             max_bytes: Максимальный совокупный объем памяти бу
    | фера в байтах (по умолчанию 32 МБ).
43         """
44         self._capacity = max(10, capacity)
45         self._max_bytes = max(1024 * 1024, max_bytes)
46         self._current_bytes = 0
47         self._buffer: collections.deque[Tuple[float, Dict[str,
    | Any], int]] = collections.deque(maxlen=self._capacity)
48         self._lock = threading.Lock()
49
50     @property
51     def capacity(self) -> int:
52         """Возвращает максимальную емкость буфера в количестве
    | записей."""
53         return self._capacity
54
55     @property
56     def max_bytes(self) -> int:
57         """Возвращает максимальный лимит памяти буфера в байта
    | х."""
58         return self._max_bytes
59

```

```

60 |     @property
61 |     def current_bytes(self) -> int:
62 |         """Возвращает текущий ориентировочный объем занимаемой
        |         памяти буфера в байтах."""
63 |         with self._lock:
64 |             return self._current_bytes
65 |
66 |     def _estimate_size(self, item: Dict[str, Any]) -> int:
67 |         """Оценивает размер словаря с метриками в байтах."""
68 |         try:
69 |             return len(json.dumps(item, ensure_ascii=False, de
        |             fault=str).encode('utf-8')) + 128
70 |         except Exception:
71 |             return sys.getsizeof(item) + 256
72 |
73 |     def append(self, item: Dict[str, Any], epoch_timestamp: Op
        |     tional[float] = None) -> None:
74 |         """Добавляет запись телеметрии в кольцевой буфер с кон
        |         тролем лимитов RAM и количества.
75 |
76 |         Args:
77 |             item: Словарь с телеметрией или системным снимком.
78 |             epoch_timestamp: Unix epoch timestamp в секундах (
        |             если None, берется текущее время).
79 |         """
80 |         ts = epoch_timestamp if epoch_timestamp is not None el
        |         se time.time()
81 |         item_size = self._estimate_size(item)
82 |         with self._lock:
83 |             # Если буфер заполнен по длине, deque сам вытеснит
        |             старый элемент, но нам нужно учесть его размер
84 |             if len(self._buffer) == self._capacity and self._b
        |             uffer:
85 |                 old_ts, old_item, old_size = self._buffer[0]
86 |                 self._current_bytes = max(0, self._current_byt
        |                 es - old_size)
87 |
88 |                 self._buffer.append((ts, item, item_size))

```



```

89         self._current_bytes += item_size
90
91         # Контроль жесткого байтового лимита RAM: вытеснен
    ие самых старых записей при переполнении
92         while self._current_bytes > self._max_bytes and le
    n(self._buffer) > 1:
93             old_ts, old_item, old_size = self._buffer.popl
    eft()
94             self._current_bytes = max(0, self._current_byt
    es - old_size)
95
96     def get_recent(self, seconds: float = 300.0) -> List[Dict[
    str, Any]]:
97         """Возвращает список записей за последние N секунд.
98
99         Args:
100             seconds: Длительность временного окна назад от тек
    ущего момента.
101
102         Returns:
103             List[Dict[str, Any]]: Список замеров от старых к н
    ОВЫМ.
104
105         """
106         now = time.time()
107         cutoff = now - max(0.0, seconds)
108         with self._lock:
109             return [item for ts, item, _ in self._buffer if ts
    >= cutoff]
110
111     def get_window(
112         self,
113         center_timestamp: float,
114         before_seconds: float = 300.0,
115         after_seconds: float = 0.0,
116     ) -> List[Dict[str, Any]]:
117         """Извлекает срез телеметрии вокруг заданного момента
    времени (например, T-5m .. T+20m).

```

```
118 |         Args:
119 |             center_timestamp: Центральный момент времени событ
    | ия (Unix epoch seconds).
120 |             before_seconds: Длительность окна ДО события в сек
    | ундах.
121 |             after_seconds: Длительность окна ПОСЛЕ события в с
    | екундах.
122 |
123 |         Returns:
124 |             List[Dict[str, Any]]: Список записей, попавших в и
    | нтервал.
125 |             """
126 |             start_ts = center_timestamp - max(0.0, before_seconds)
127 |             end_ts = center_timestamp + max(0.0, after_seconds)
128 |             with self._lock:
129 |                 return [item for ts, item, _ in self._buffer if st
    | art_ts <= ts <= end_ts]
130 |
131 |     def get_all(self) -> List[Dict[str, Any]]:
132 |         """Возвращает все текущие записи в буфере.
133 |
134 |         Returns:
135 |             List[Dict[str, Any]]: Полный текущий срез буфера.
136 |             """
137 |             with self._lock:
138 |                 return [item for _, item, _ in self._buffer]
139 |
140 |     def count(self) -> int:
141 |         """Возвращает текущее число записей в буфере."""
142 |         with self._lock:
143 |             return len(self._buffer)
144 |
145 |     def clear(self) -> None:
146 |         """Очищает буфер."""
147 |         with self._lock:
148 |             self._buffer.clear()
149 |             self._current_bytes = 0
150 |
```

__init__.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry Storage -
  Package Root
4 # =====
5 # Description:
6 #   Хранилище телеметрии: кольцевой буфер в памяти, SQLite баз
  а и WAL-режим.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.storage.ring_buffer import T
  elemetryRingBuffer
11 #
12 # File: __init__.py
13 # Project: ai-breadboard
14 # Package: apps.windows.telemetry.storage
15 # Author: hypo69
16 # Copyright: © 2026 hypo69
17 # Updated: 2026-10-08 09:36:00
18 # =====
19
20 from __future__ import annotations
21 """Подсистема хранения телеметрии SQLite и кольцевого буфера."""
22
23 from apps.windows.telemetry.storage.ring_buffer import Telemet
  ryRingBuffer
24 from apps.windows.telemetry.storage.init_db import init_teleme
  try_db
25
26 __all__ = [
27     "TelemetryRingBuffer",
```

```
28 |     "init_telemetry_db",
29 | ]
30 |
```

storage_usage.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - Storage
  Usage
4 # =====
5 # Description:
6 #   Четырехуровневый стек сбора телеметрии накопителей Windows
  :
7 #   1. DiskInventoryCollector (Win32 + IOCTL Storage Property
  & Volumes)
8 #   2. DiskHealthCollector (NVMe SMART / Health / TBW)
9 #   3. DiskPerformanceCollector (IOPS, Latency, Throughput, Qu
  eue)
10 #   4. DiskProcessIOCollector (Попроцессный I/O анализ и агрег
  аты)
11 #
12 # Usage Examples:
13 #   Python API:
14 #       from apps.windows.telemetry.storage_usage import (
15 #           DiskInventoryCollector,
16 #           DiskHealthCollector,
17 #           DiskPerformanceCollector,
18 #           DiskProcessIOCollector,
19 #           WindowsStorageUsageCollector,
20 #       )
21 #
22 #       collector = WindowsStorageUsageCollector()
23 #       disks = collector.get_physical_disks()
24 #       report = collector.get_disk_usage_period_report()
25 #
26 # File: storage_usage.py
27 # Project: ai-breadboard
28 # Package: apps.windows.telemetry
29 # Author: hypo69
```

```
30 # Copyright: © 2026 hypo69
31 # Updated: 2026-10-06 01:35:00
32 # =====
33 |=====
34 from __future__ import annotations
35 """Четырехуровневый стек сбора телеметрии накопителей Windows.
36 |"""
37 import math
38 import os
39 import time
40 from typing import Any, Dict, List, Optional, Tuple
41 |
42 try:
43 |     import psutil
44 |     PSUTIL_AVAILABLE = True
45 except ImportError:
46 |     psutil = None
47 |     PSUTIL_AVAILABLE = False
48 |
49 from logger import logger
50 from apps.windows.sdk.modules.storage_manager.core.raw_disk_io
51 | import WindowsRawDiskIO
52 from apps.windows.sdk.modules.storage_manager.core.windows_storage_sensor import WindowsStorageSensor
53 from apps.windows.telemetry.models import (
54 |     AppDiskUsageItem,
55 |     DiskUsagePeriodReport,
56 |     PhysicalDiskHealth,
57 | )
58 |
59 def _format_bytes(num_bytes: float) -> str:
60 |     """Форматирование объема байт в человекочитаемый вид (B, K
61 |     B, MB, GB, TB).
62 |
63 |     Args:
```

```

63         num_bytes: Количество байт.
64
65     Returns:
66         str: Отформатированная строка.
67     """
68     if num_bytes <= 0 or math.isnan(num_bytes):
69         return '0 B'
70     units = ['B', 'KB', 'MB', 'GB', 'TB', 'PB']
71     idx = 0
72     val = float(num_bytes)
73     while val >= 1024.0 and idx < len(units) - 1:
74         val /= 1024.0
75         idx += 1
76     return f'{val:.2f} {units[idx]}' if idx > 0 else f'{int(val)} B'
77
78
79 class DiskInventoryCollector:
80     """Слой 1: Коллектор инвентаря физических накопителей, разделов и логических томов."""
81
82     def __init__(self, raw_io: Optional[WindowsRawDiskIO] = None) -> None:
83         """Инициализация коллектора инвентаря дисков."""
84         self.raw_io = raw_io or WindowsRawDiskIO()
85
86     def collect(self) -> Dict[str, Any]:
87         """Сбор полного инвентаря накопителей и томов.
88
89         Returns:
90             Dict[str, Any]: Данные физических дисков и логических томов.
91         """
92         disks = self.raw_io.list_physical_disks()
93         volumes = self.raw_io.get_logical_volumes()
94         return {
95             'disks_count': len(disks),
96             'disks': disks,

```

```

97         'volumes_count': len(volumes),
98         'volumes': volumes,
99         'timestamp': time.time(),
100     }
101
102
103 class DiskHealthCollector:
104     """Слой 2: Коллектор SMART-показателей здоровья, износа и
105     TBW накопителей."""
106
107     def __init__(
108         self,
109         raw_io: Optional[WindowsRawDiskIO] = None,
110         storage_sensor: Optional[WindowsStorageSensor] = None,
111     ) -> None:
112         """Инициализация коллектора здоровья дисков."""
113         self.raw_io = raw_io or WindowsRawDiskIO()
114         self.storage_sensor = storage_sensor or WindowsStorage
115         Sensor()
116
117     def collect(self, force_refresh: bool = False) -> List[Dic
118     t[str, Any]]:
119         """Сбор показателей здоровья всех доступных накопите
120         лей.
121
122         Returns:
123             List[Dict[str, Any]]: Показатели SMART, TBW, темпе
124             ратуры и ошибок.
125         """
126         results: List[Dict[str, Any]] = []
127         raw_disks = self.raw_io.list_physical_disks()
128
129         for d in raw_disks:
130             disk_id = d.get('disk_id', 0)
131             smart = d.get('smart_health', {})
132             results.append({
133                 'disk_id': disk_id,
134                 'device_path': d.get('device_path'),

```



```

130         'model': d.get('product'),
131         'bus_type': d.get('bus_type'),
132         'temperature_c': smart.get('temperature_c'),
133         'wear_percentage': smart.get('percentage_used'
    ),
134         'available_spare_percent': smart.get('available_spare_percent'),
135         'tbw_written_tb': smart.get('data_units_written_tb'),
136         'tbw_read_tb': smart.get('data_units_read_tb')
    ,
137         'power_on_hours': smart.get('power_on_hours'),
138         'unsafe_shutdowns': smart.get('unsafe_shutdowns'),
139         'media_errors': smart.get('media_errors'),
140         'raw_available': smart.get('raw_nvme_available', False),
141     })
142
143     # Fallback / дополнение через WindowsStorageSensor CIM
    если нативный IOCTL вернул None
144     needs_cim = any(r['temperature_c'] is None and r['wear_percentage'] is None for r in results)
145     if needs_cim:
146         try:
147             cim_disks = self.storage_sensor.get_physical_disks(force_refresh=force_refresh)
148             cim_map = {str(cd.device_id).replace(r'\\.\PHYSICALDRIVE', '').replace('Disk', ''): cd for cd in cim_disks}
149             for r in results:
150                 str_id = str(r['disk_id'])
151                 if str_id in cim_map:
152                     cd = cim_map[str_id]
153                     if r['temperature_c'] is None and cd.temperature_c:
154                         r['temperature_c'] = cd.temperature_c
155                     if r['wear_percentage'] is None and cd

```

```

155 |         .wear_percentage:
156 |             r['wear_percentage'] = cd.wear_per
157 |             centage
158 |             if r['power_on_hours'] is None and cd.
159 |             power_on_hours:
160 |                 r['power_on_hours'] = cd.power_on_
161 |                 hours
162 |                 if r['tbw_written_tb'] is None and cd.
163 |                 lifetime_write_tb:
164 |                     r['tbw_written_tb'] = cd.lifetime_
165 |                     write_tb
166 |                     if r['tbw_read_tb'] is None and cd.lif
167 |                     etime_read_tb:
168 |                         r['tbw_read_tb'] = cd.lifetime_rea
169 |                         d_tb
170 |             except Exception as exc:
171 |                 logger.debug(f"Ошибка дополнения здоровья чере
172 | з CIM: {exc}")
173 |
174 |         return results
175 |
176 |
177 | class DiskPerformanceCollector:
178 |     """Слой 3: Коллектор текущей скорости, IOPS и времени откл
179 | ика дисков."""
180 |
181 |     def __init__(self) -> None:
182 |         """Инициализация коллектора производительности дисков.
183 | """
184 |
185 |         self._last_sample_time: float = time.time()
186 |         self._last_io_counters: Dict[str, Any] = {}
187 |
188 |     def collect(self) -> Dict[str, Any]:
189 |         """Сбор показателей пропускной способности и IOPS нако
190 | пителей.
191 |
192 | Returns:
193 | Dict[str, Any]: Метрики скорости чтения/записи и о

```

пераций в секунду.

```
182         """
183         now = time.time()
184         dt = max(0.001, now - self._last_sample_time)
185         res: Dict[str, Any] = {
186             'total_read_bytes_sec': 0.0,
187             'total_write_bytes_sec': 0.0,
188             'total_read_iops': 0.0,
189             'total_write_iops': 0.0,
190             'per_disk': {},
191             'dt_sec': dt,
192         }
193
194         if not PSUTIL_AVAILABLE:
195             self._last_sample_time = now
196             return res
197
198         try:
199             per_disk_counters = psutil.disk_io_counters(perdis
200 k=True)
201             if per_disk_counters:
202                 for disk_name, cur in per_disk_counters.items(
203 ):
204                     prev = self._last_io_counters.get(disk_name)
205                     if prev:
206                         r_bytes = max(0, cur.read_bytes - prev
207 .read_bytes) / dt
208                         w_bytes = max(0, cur.write_bytes - prev
209 v.write_bytes) / dt
210                         r_count = max(0, cur.read_count - prev
211 .read_count) / dt
212                         w_count = max(0, cur.write_count - prev
213 v.write_count) / dt
214                         r_time = max(0, cur.read_time - prev.r
215 ead_time)
216                         w_time = max(0, cur.write_time - prev.
217 write_time)
```

```

210 |
211 |         res['total_read_bytes_sec'] += r_bytes
212 |         res['total_write_bytes_sec'] += w_byte
    | s
213 |         res['total_read_iops'] += r_count
214 |         res['total_write_iops'] += w_count
215 |
216 |         res['per_disk'][disk_name] = {
217 |             'read_bytes_sec': round(r_bytes, 1
    | ),
218 |             'write_bytes_sec': round(w_bytes,
    | 1),
219 |             'read_iops': round(r_count, 1),
220 |             'write_iops': round(w_count, 1),
221 |             'read_time_ms': r_time,
222 |             'write_time_ms': w_time,
223 |         }
224 |         self._last_io_counters = per_disk_counters
225 |     except Exception as exc:
226 |         logger.debug(f"Ошибка сбора счетчиков производител
    | ьности дисков: {exc}")
227 |
228 |         self._last_sample_time = now
229 |         return res
230 |
231 |
232 | class DiskProcessIOCollector:
233 |     """Слой 4: Коллектор активности ввода-вывода процессов (To
    | p Writers/Readers)."""
234 |
235 |     def __init__(self) -> None:
236 |         """Инициализация коллектора I/O процессов."""
237 |         self._last_process_io: Dict[int, Dict[str, Any]] = {}
238 |         self._last_poll_time: float = time.time()
239 |
240 |     def collect(self) -> Dict[str, Dict[str, Any]]:
241 |         """Опрос I/O всех активных процессов и вычисление дель
    | т чтения/записи.

```

```

242
243     Returns:
244         Dict[str, Dict[str, Any]]: Агрегированные дельты п
о именам процессов.
245         """
246         if not PSUTIL_AVAILABLE:
247             return {}
248
249         now = time.time()
250         aggregated_by_name: Dict[str, Dict[str, Any]] = {}
251
252         try:
253             for p in psutil.process_iter(attrs=['pid', 'name']
):
254                 try:
255                     pid = p.info['pid']
256                     name = p.info.get('name') or f'PID_{pid}'
257                     io_cnt = p.io_counters()
258                     cur_r = io_cnt.read_bytes
259                     cur_w = io_cnt.write_bytes
260
261                     prev = self._last_process_io.get(pid)
262                     if prev:
263                         delta_r = max(0, cur_r - prev['read_by
tes'])
264                         delta_w = max(0, cur_w - prev['write_b
ytes'])
265                     else:
266                         delta_r = cur_r
267                         delta_w = cur_w
268
269                     self._last_process_io[pid] = {
270                         'name': name,
271                         'read_bytes': cur_r,
272                         'write_bytes': cur_w,
273                         'time': now,
274                     }
275

```

```

276         if name not in aggregated_by_name:
277             aggregated_by_name[name] = {
278                 'pid': pid,
279                 'read_bytes': 0,
280                 'write_bytes': 0,
281             }
282             aggregated_by_name[name]['read_bytes'] +=
283             delta_r
284             aggregated_by_name[name]['write_bytes'] +=
285             delta_w
286         except (psutil.NoSuchProcess, psutil.AccessDen
287         ired, AttributeError):
288             continue
289         except Exception as ex:
290             logger.debug(f'Ошибка выборки process I/O deltas:
291             {ex}')
292
293     self._last_poll_time = now
294     return aggregated_by_name
295
296 class WindowsStorageUsageCollector:
297     """Унифицированный диспетчер дисковой телеметрии, объединя
298     ющий все 4 слоя."""
299
300     def __init__(
301         self,
302         raw_io: Optional[WindowsRawDiskIO] = None,
303         storage_sensor: Optional[WindowsStorageSensor] = None,
304     ) -> None:
305         """Инициализация унифицированного коллектора дисковой
306         телеметрии."""
307
308         self.raw_io = raw_io or WindowsRawDiskIO()
309         self.storage_sensor = storage_sensor or WindowsStorage
310         Sensor()
311
312         self.inventory_collector = DiskInventoryCollector(self
313         .raw_io)

```

```

306         self.health_collector = DiskHealthCollector(self.raw_i
o, self.storage_sensor)
307         self.performance_collector = DiskPerformanceCollector(
)
308         self.process_io_collector = DiskProcessIOCollector()
309
310     def get_physical_disks(self, force_refresh: bool = False)
-> List[PhysicalDiskHealth]:
311         """Получить актуальный список физических накопителей с
показателями SMART и Lifetime."""
312         results: List[PhysicalDiskHealth] = []
313         try:
314             health_list = self.health_collector.collect(force_
refresh=force_refresh)
315             for h in health_list:
316                 results.append(PhysicalDiskHealth(
317                     device_id=f"Disk {h.get('disk_id', 0)}",
318                     model=h.get('model') or f"PhysicalDrive{h.
get('disk_id', 0)}",
319                     media_type='SSD' if 'SSD' in (h.get('model
', '')) + h.get('bus_type', '').upper() or h.get('bus_type') =
= 'NVMe' else 'HDD',
320                     size_gb=0.0,
321                     health_status='Healthy',
322                     operational_status='OK',
323                     temperature_celsius=h.get('temperature_c')
,
324                     interface_type=h.get('bus_type') or 'NVMe'
,
325                     lifetime_read_bytes=int(h['tbw_read_tb'] *
(1024 ** 4)) if h.get('tbw_read_tb') else None,
326                     lifetime_write_bytes=int(h['tbw_written_tb
'] * (1024 ** 4)) if h.get('tbw_written_tb') else None,
327                     lifetime_read_tb=h.get('tbw_read_tb'),
328                     lifetime_write_tb=h.get('tbw_written_tb'),
329                     power_on_hours=h.get('power_on_hours'),
330                     wear_percentage=h.get('wear_percentage'),
331                 ))

```

```

332         except Exception as ex:
333             logger.debug(f'Ошибка получения физических накопит
елей: {ex}')
334
335         if not results:
336             results.append(PhysicalDiskHealth(
337                 device_id='Disk 0',
338                 model='System Drive (NVMe/SSD)',
339                 media_type='SSD',
340                 size_gb=512.0,
341                 health_status='Healthy',
342                 operational_status='OK',
343                 interface_type='NVMe',
344             ))
345         return results
346
347     def poll_process_io_deltas(self) -> Dict[str, Dict[str, An
y]]:
348         """Опросить текущий I/O процессов и рассчитать накопле
нные дельты."""
349         return self.process_io_collector.collect()
350
351     def get_disk_usage_period_report(
352         self,
353         period_minutes: int = 1440,
354         force_refresh_smart: bool = False,
355     ) -> DiskUsagePeriodReport:
356         """Сформировать агрегированный отчет об использовании
диска за интервал."""
357         phys_disks = self.get_physical_disks(force_refresh=for
ce_refresh_smart)
358         proc_deltas = self.poll_process_io_deltas()
359
360         writers_list: List[AppDiskUsageItem] = []
361         readers_list: List[AppDiskUsageItem] = []
362
363         total_r = sum(item['read_bytes'] for item in proc_delt
as.values())

```



```

364         total_w = sum(item['write_bytes'] for item in proc_deltas.values())
365         total_all = total_r + total_w
366
367         # Ранжирование по записи
368         sorted_writers = sorted(proc_deltas.items(), key=lambda x: x[1]['write_bytes'], reverse=True)
369         for name, item in sorted_writers[:15]:
370             wb = item['write_bytes']
371             rb = item['read_bytes']
372             tot = wb + rb
373             if wb <= 0:
374                 continue
375             pct = round((wb / total_w) * 100.0, 1) if total_w > 0 else 0.0
376             writers_list.append(AppDiskUsageItem(
377                 process_name=name,
378                 pid=item.get('pid'),
379                 read_bytes=rb,
380                 write_bytes=wb,
381                 total_bytes=tot,
382                 read_formatted=_format_bytes(rb),
383                 write_formatted=_format_bytes(wb),
384                 share_percent=pct,
385             ))
386
387         # Ранжирование по чтению
388         sorted_readers = sorted(proc_deltas.items(), key=lambda x: x[1]['read_bytes'], reverse=True)
389         for name, item in sorted_readers[:15]:
390             wb = item['write_bytes']
391             rb = item['read_bytes']
392             tot = wb + rb
393             if rb <= 0:
394                 continue
395             pct = round((rb / total_r) * 100.0, 1) if total_r > 0 else 0.0
396             readers_list.append(AppDiskUsageItem(

```

```

397         process_name=name,
398         pid=item.get('pid'),
399         read_bytes=rb,
400         write_bytes=wb,
401         total_bytes=tot,
402         read_formatted=_format_bytes(rb),
403         write_formatted=_format_bytes(wb),
404         share_percent=pct,
405     ))
406
407     top_writer = sorted_writers[0][0] if sorted_writers and sorted_writers[0][1]['write_bytes'] > 0 else 'N/A'
408     top_reader = sorted_readers[0][0] if sorted_readers and sorted_readers[0][1]['read_bytes'] > 0 else 'N/A'
409
410     diag = (
411         f"За анализируемый период ({period_minutes} мин) записано {_format_bytes(total_w)}, "
412         f"прочитано {_format_bytes(total_r)}. Главный источник записи: {top_writer}, чтения: {top_reader}."
413     )
414
415     return DiskUsagePeriodReport(
416         period_minutes=period_minutes,
417         total_read_bytes=total_r,
418         total_write_bytes=total_w,
419         total_read_formatted=_format_bytes(total_r),
420         total_write_formatted=_format_bytes(total_w),
421         total_formatted=_format_bytes(total_all),
422         top_writers=writers_list,
423         top_readers=readers_list,
424         physical_disks=phys_disks,
425         summary_text=diag,
426     )
427
428
429 __all__ = [
430     'DiskInventoryCollector',

```

```
431 | 'DiskHealthCollector',  
432 | 'DiskPerformanceCollector',  
433 | 'DiskProcessIOCollector',  
434 | 'WindowsStorageUsageCollector',  
435 | ]  
436 |
```

telemetry_config.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - Telemetry Config
4 # =====
5 # Description:
6 #   Менеджер конфигурации сбора телеметрии и сенсоров с поддержкой
7 #   динамического перечитывания config.json на лету.
8 #
9 # Usage Examples:
10 #   Python API:
11 #       from apps.windows.telemetry.telemetry_config import TelemetryConfigManager
12 #
13 #       config_mgr = TelemetryConfigManager()
14 #       interval = config_mgr.get_interval_seconds()
15 #       if config_mgr.check_and_reload():
16 #           print("Конфигурация обновлена!")
17 #
18 # File: telemetry_config.py
19 # Project: ai-breadboard
20 # Package: apps.windows.telemetry
21 # Author: hypo69
22 # Copyright: © 2026 hypo69
23 # Updated: 2026-10-08 10:25:00
24 # =====
25
26 from __future__ import annotations
27 """Менеджер конфигурации сбора телеметрии и сенсоров."""
28
29 import json
30 import os
```

```
31 import re
32 import shutil
33 from pathlib import Path
34 from typing import Any, Dict, List, Optional
35 try:
36     from logger import logger
37 except ImportError:
38     import logging
39     logger = logging.getLogger(__name__)
40
41
42 def get_default_telemetry_config_path() -> Path:
43     """Возвращает путь по умолчанию к файлу конфигурации телем
44     етрии в %ProgramData%.
45
46     Проверяет сначала %ProgramData%/AITelemetry/config/config.
47     json,
48     затем %ProgramData%/AI-Breadboard/apps/windows/telemetry/c
49     onfig.json.
50     Если целевой файл отсутствует, выполняется попытка скопиро
51     вать шаблон из пакета модуля.
52
53     Returns:
54         Path: Путь к файлу конфигурации в %ProgramData%.
55     """
56     programdata = os.environ.get('ProgramData') or os.environ.
57     get('ALLUSERSPROFILE')
58     if programdata and os.path.exists(programdata):
59         base_dir = Path(programdata)
60     else:
61         appdata = os.environ.get('APPDATA') or os.environ.get(
62         'LOCALAPPDATA')
63         if appdata and os.path.exists(appdata):
64             base_dir = Path(appdata)
65         else:
66             base_dir = Path.home() / '.config'
67
68     ai_tel_config = base_dir / 'AITelemetry' / 'config' / 'con
```

```

fig.json'
63     if ai_tel_config.is_file():
64         return ai_tel_config
65
66     target_path = base_dir / 'AI-Breadboard' / 'apps' / 'windo
ws' / 'telemetry' / 'config.json'
67
68     if not target_path.exists():
69         bundled_template = Path(__file__).parent / 'config.jso
n'
70         if bundled_template.is_file():
71             try:
72                 target_path.parent.mkdir(parents=True, exist_o
k=True)
73                 shutil.copy2(bundled_template, target_path)
74             except Exception:
75                 return bundled_template
76     return target_path
77
78
79 def parse_interval_to_seconds(val: Any, default: float = 5.0)
-> float:
80     """Преобразует строковое или числовое значение интервала в
секунды (float).
81
82     Поддерживает:
83     - Числа (int, float): 5 -> 5.0, 0.5 -> 0.5
84     - Строковые числа: "5", "5.0" -> 5.0
85     - Строки с единицами времени:
86         - Секунды: "5s", "5 seconds", "5 сек", "5 секунд"
-> 5.0
87         - Минуты: "1m", "1 minute", "5 minutes", "2 мин" -
> 60.0 / 300.0 / 120.0
88         - Часы: "1h", "1 hour", "6 hours", "1 час" -> 3600
.0 / 21600.0
89         - Дни: "1d", "1 day", "7 days", "1 день" -> 86400.
0 / 604800.0
90

```

```

91 |     Args:
92 |         val: Значение интервала из конфигурации.
93 |         default: Значение по умолчанию в секундах.
94 |
95 |     Returns:
96 |         float: Интервал в секундах (минимум 0.001 при валидном
положительном значении).
97 |         ""
98 |     if val is None:
99 |         return float(default)
100 |     if isinstance(val, (int, float)):
101 |         return max(0.001, float(val))
102 |     if isinstance(val, str):
103 |         val_str = val.strip().lower()
104 |         if not val_str:
105 |             return float(default)
106 |         try:
107 |             return max(0.001, float(val_str))
108 |         except ValueError:
109 |             pass
110 |         match = re.match(r"^([\d.]+\s*([a-za-я]+)?$", val_str
)
111 |         if match:
112 |             try:
113 |                 num = float(match.group(1))
114 |                 unit = match.group(2) or "s"
115 |                 if unit in ("s", "sec", "second", "seconds", "
сек", "секунд", "секунды", "секунда"):
116 |                     return max(0.001, num)
117 |                 elif unit in ("m", "min", "minute", "minutes",
"мин", "минут", "минуты", "минута"):
118 |                     return max(0.001, num * 60.0)
119 |                 elif unit in ("h", "hr", "hour", "hours", "ч",
"час", "часа", "часов"):
120 |                     return max(0.001, num * 3600.0)
121 |                 elif unit in ("d", "day", "days", "д", "день",
"дня", "дней"):
122 |                     return max(0.001, num * 86400.0)

```

```

123         except ValueError:
124             pass
125     return float(default)
126
127
128 class TelemetryConfigManager:
129     """Менеджер конфигурации для централизованного управления
130     сенсорами и интервалами."""
131
132     def __init__(self, config_path: Optional[str] = None) -> None:
133         """Инициализирует менеджер конфигурации.
134
135         Args:
136             config_path: Путь к файлу конфигурации (по умолчанию: %APPDATA%/AI-Breadboard/apps/windows/telemetry/config.json).
137
138         """
139         if config_path:
140             self._config_path = str(config_path)
141         else:
142             self._config_path = str(get_default_telemetry_config_path())
143
144         self._config: Dict[str, Any] = {}
145         self._sensors_config: Dict[str, Dict[str, Any]] = {}
146         self._loggers_config: Dict[str, Dict[str, Any]] = {}
147         self._default_interval = 5.0
148         self._fast_interval = 30.0
149         self._fast_duration_days = 2
150         self._aggregation_interval = 3600.0
151         self._heavy_interval = 60.0
152         self._heavy_disk_scan_interval = 43200.0
153         self._heavy_mode_max_duration_days = 5.0
154         self._heavy_mode_auto_switch_enabled = True
155         self._mode = 'hybrid'
156         self._telemetry_mode = 'telemetry'
157         self._standard_flush_interval_seconds = 30.0
158         self._tc_flush_interval_seconds = 5.0

```



```

156         self._tc_mode_timeout_seconds = 300.0
157         self._tc_mode_activated_at: Optional[float] = None
158         self._max_db_size_mb = 100.0
159         self._retention_days = 7
160         self._db_cleanup_interval_seconds = 300.0
161         self._flush_interval_seconds = 30.0
162         self._auto_vacuum_enabled = True
163         self._last_mtime_ns: int = 0
164         self._load_config()
165
166     def _load_config(self) -> None:
167         """Загружает конфигурацию из JSON файла с фиксацией вр
168         емени модификации."""
169         try:
170             cfg_path = Path(self._config_path)
171             if cfg_path.is_file():
172                 self._last_mtime_ns = cfg_path.stat().st_mtime
173                 _ns
174             with open(self._config_path, 'r', encoding='utf-8'
175 ) as f:
176                 new_config = json.load(f)
177
178                 self._config = new_config
179                 if 'sensors' in self._config:
180                     self._sensors_config = self._config.get('senso
181 rs', {})
182                 elif 'telemetry' in self._config and 'sensors' in
183 self._config['telemetry']:
184                     self._sensors_config = self._config['telemetry
185 '].get('sensors', {})
186                 else:
187                     self._sensors_config = {}
188
189                 self._loggers_config = self._config.get('loggers',
190 )
191                 {}
192
193                 raw_interval = self._config.get('interval_seconds'
194 , self._config.get('interval', self._config.get('telemetry', {

```

```

    }).get('interval_seconds', 5.0)))
186         self._default_interval = parse_interval_to_seconds
    (raw_interval, 5.0)
187
188         raw_heavy = self._config.get('heavy_interval_seconds', self._config.get('heavy_interval', self._config.get('telemetry', {})).get('heavy_interval_seconds', 60.0)))
189         self._heavy_interval = parse_interval_to_seconds(raw_heavy, 60.0)
190
191         raw_disk_interval = self._config.get('heavy_disk_scan_interval_seconds')
192         self._heavy_disk_scan_interval = parse_interval_to_seconds(raw_disk_interval, 43200.0)
193
194         raw_fast_interval = self._config.get('fast_interval_seconds')
195         self._fast_interval = parse_interval_to_seconds(raw_fast_interval, 30.0)
196
197         self._fast_duration_days = int(self._config.get('fast_duration_days', 2))
198
199         raw_agg = self._config.get('aggregation_interval_seconds', self._config.get('rollup_interval_seconds'))
200         self._aggregation_interval = parse_interval_to_seconds(raw_agg, 3600.0)
201
202         raw_max_days = self._config.get('heavy_mode_max_duration_days')
203         self._heavy_mode_max_duration_days = float(raw_max_days) if raw_max_days is not None else 5.0
204
205         self._heavy_mode_auto_switch_enabled = bool(self._config.get('heavy_mode_auto_switch_enabled', True))
206         self._mode = str(self._config.get('mode', self._config.get('telemetry', {})).get('mode', 'hybrid')).lower()
207

```

```

208         raw_tel_mode = str(self._config.get('telemetry_mod
    e', self._config.get('profile', 'telemetry'))).lower()
209         self._telemetry_mode = raw_tel_mode if raw_tel_mod
    e in ('telemetry', 'tc') else 'telemetry'
210
211         raw_std_flush = self._config.get('standard_flush_i
    nterval_seconds', self._config.get('flush_interval_seconds', 3
    0.0))
212         self._standard_flush_interval_seconds = parse_inte
    rval_to_seconds(raw_std_flush, 30.0)
213
214         raw_tc_flush = self._config.get('tc_flush_interval
    _seconds', 5.0)
215         self._tc_flush_interval_seconds = parse_interval_t
    o_seconds(raw_tc_flush, 5.0)
216
217         raw_tc_timeout = self._config.get('tc_mode_timeout
    _seconds', self._config.get('tc_timeout_seconds', 300.0))
218         self._tc_mode_timeout_seconds = parse_interval_to_
    seconds(raw_tc_timeout, 300.0)
219
220         raw_tc_act = self._config.get('tc_mode_activated_a
    t')
221         try:
222             self._tc_mode_activated_at = float(raw_tc_act)
    if raw_tc_act is not None else None
223         except (ValueError, TypeError):
224             self._tc_mode_activated_at = None
225
226         raw_max_db = self._config.get('max_db_size_mb', se
    lf._config.get('max_file_size_mb', 100.0))
227         self._max_db_size_mb = float(raw_max_db) if raw_ma
    x_db is not None else 100.0
228
229         raw_retention = self._config.get('retention_days',
    7)
230         self._retention_days = int(raw_retention) if raw_r
    etention is not None else 7

```

```

231 |
232 |         raw_cleanup_interval = self._config.get('db_cleanup_interval_seconds')
233 |         self._db_cleanup_interval_seconds = parse_interval_to_seconds(raw_cleanup_interval, 300.0)
234 |
235 |         if self._telemetry_mode == 'tc':
236 |             self._flush_interval_seconds = self._tc_flush_interval_seconds
237 |         else:
238 |             self._flush_interval_seconds = self._standard_flush_interval_seconds
239 |
240 |         self._auto_vacuum_enabled = bool(self._config.get('auto_vacuum_enabled', True))
241 |     except FileNotFoundError:
242 |         if not self._config:
243 |             self._config = {}
244 |             self._sensors_config = {}
245 |             self._loggers_config = {}
246 |             self._default_interval = 5.0
247 |             self._heavy_interval = 60.0
248 |             self._heavy_disk_scan_interval = 43200.0
249 |             self._heavy_mode_max_duration_days = 5.0
250 |             self._heavy_mode_auto_switch_enabled = True
251 |             self._mode = 'hybrid'
252 |             self._max_db_size_mb = 50.0
253 |             self._retention_days = 7
254 |             self._db_cleanup_interval_seconds = 300.0
255 |             self._flush_interval_seconds = 30.0
256 |             self._auto_vacuum_enabled = True
257 |     except (json.JSONDecodeError, OSError, PermissionError) as e:
258 |         if not self._config:
259 |             raise ValueError(f'Ошибка парсинга {self._config_path}: {e}')
260 |         logger.warning(f'Временная ошибка считывания {self._config_path} на лету: {e}')

```

```

261
262     @property
263     def config_path(self) -> str:
264         """Возвращает путь к активному файлу конфигурации."""
265         return self._config_path
266
267     def check_and_reload(self) -> bool:
268         """Проверяет модификацию файла config.json на диске и
269         перезагружает параметры 'на лету'.
270
271         Returns:
272             bool: True если конфигурация изменилась и успешно
273             перезагружена, иначе False.
274
275         """
276         try:
277             cfg_path = Path(self._config_path)
278             if not cfg_path.is_file():
279                 return False
280             current_mtime_ns = cfg_path.stat().st_mtime_ns
281             if current_mtime_ns != self._last_mtime_ns:
282                 self._load_config()
283                 return True
284             except Exception as e:
285                 logger.debug(f'Проверка модификации config.json: {
286                     e}')
287                 return False
288
289     def reload(self) -> None:
290         """Принудительно перезагружает конфигурацию из файла."
291
292         """
293         self._load_config()
294
295     def get_data_path(self) -> Path:
296         """Возвращает настроенный или системный путь к директо
297         рии хранения данных телеметрии."""
298         custom = self._config.get('data_path') or self._config
299         .get('data_dir')
300         if not custom and isinstance(self._config.get('storage

```

```

    '), dict):
293         custom = self._config['storage'].get('data_path')
    or self._config['storage'].get('data_dir')
294         if custom:
295             return Path(os.path.expandvars(str(custom)))
296         programdata = os.environ.get('ProgramData') or os.envi
    ron.get('ALLUSERSPROFILE')
297         if programdata and os.path.exists(programdata):
298             base_dir = Path(programdata)
299         else:
300             appdata = os.environ.get('APPDATA') or os.environ.
    get('LOCALAPPDATA')
301             if appdata and os.path.exists(appdata):
302                 base_dir = Path(appdata)
303             else:
304                 base_dir = Path.home() / '.config'
305             return base_dir / 'AI-Breadboard' / 'apps' / 'windows'
    / 'telemetry' / 'logs'
306
307     def get_db_path(self) -> Path:
308         """Возвращает настроенный или системный путь к файлу S
    QLite базы данных телеметрии."""
309         if isinstance(self._config.get('storage'), dict):
310             custom_db = self._config['storage'].get('db_path')
311             if custom_db:
312                 return Path(os.path.expandvars(str(custom_db))
    )
313             custom_db = self._config.get('db_path')
314             if custom_db:
315                 return Path(os.path.expandvars(str(custom_db)))
316             return self.get_data_path() / 'telemetry.db'
317
318     def get_fast_interval(self) -> float:
319         """Возвращает интервал быстрого режима (сек)."""
320         return getattr(self, '_fast_interval', 30.0)
321
322     def get_fast_duration_days(self) -> int:
323         """Возвращает длительность быстрого режима в днях."""

```

```

324         return getattr(self, '_fast_duration_days', 2)
325
326     def get_aggregation_interval(self) -> float:
327         """Возвращает интервал агрегации (сек)."""
328         return getattr(self, '_aggregation_interval', 3600.0)
329
330     def get_mode(self) -> str:
331         """Возвращает режим сбора: minimal, hybrid, full."""
332         return self._mode if self._mode in ('minimal', 'hybrid', 'full') else 'hybrid'
333
334     def get_interval_seconds(self) -> float:
335         """Возвращает быстрый базовый интервал сбора телеметрии."""
336         return self._default_interval
337
338     def get_heavy_interval_seconds(self) -> float:
339         """Возвращает периодический интервал тяжелых сенсоров."""
340         return self._heavy_interval
341
342     def get_heavy_disk_scan_interval(self) -> float:
343         """Возвращает интервал тяжелой проверки дисков (в секундах)."""
344         return getattr(self, '_heavy_disk_scan_interval', 43200.0)
345
346     def get_heavy_mode_max_duration_days(self) -> float:
347         """Возвращает максимальную непрерывную длительность тяжелого режима (в днях)."""
348         return getattr(self, '_heavy_mode_max_duration_days', 5.0)
349
350     def is_heavy_auto_switch_enabled(self) -> bool:
351         """Возвращает флаг разрешения автопереключения тяжелого режима в легкий."""
352         return getattr(self, '_heavy_mode_auto_switch_enabled', True)

```

```

353
354     def get_heavy_collectors(self) -> Dict[str, bool]:
355         """Возвращает словарь флагов включения тяжелых коллект
опров."""
356         default_heavy = {
357             'hardware_sensors': True,
358             'deep_disk_smart': True,
359             'system_snapshot': True,
360             'hardware_tree': True,
361         }
362         custom = self._config.get('heavy_collectors', {})
363         if isinstance(custom, dict):
364             default_heavy.update(custom)
365         return default_heavy
366
367     def get_top_processes(self) -> int:
368         """Возвращает число сохраняемых процессов с наибольшей
нагрузкой."""
369         return int(self._config.get('top_processes', 25))
370
371     def get_buffer_mode(self) -> str:
372         """Возвращает режим буферизации телеметрии ('memory',
'file', 'direct')."""
373         mode = str(self._config.get('buffer_mode', 'memory')).
lower()
374         return mode if mode in ('memory', 'file', 'direct') el
se 'memory'
375
376     def get_buffer_size(self) -> int:
377         """Возвращает максимальный размер буфера перед принуди
тельным сбросом в БД."""
378         try:
379             val = int(self._config.get('buffer_size', 50))
380             return max(1, val)
381         except (ValueError, TypeError):
382             return 50
383
384     def get_telemetry_mode(self) -> str:

```



```

385 |         """Возвращает режим телеметрии: 'telemetry' (стандартн
    |   ый) или 'tc' (реальное время)."""
386 |         self.check_tc_mode_auto_switch()
387 |         return getattr(self, '_telemetry_mode', 'telemetry')
388 |
389 |     def check_tc_mode_auto_switch(self) -> bool:
390 |         """Проверяет таймаут активного режима 'tc' (реальное в
    |   ремя) и переключает в 'telemetry'.
391 |
392 |         Returns:
393 |             bool: True если произошло автопереключение в 'tele
    |   metry', иначе False.
394 |         """
395 |         if getattr(self, '_telemetry_mode', 'telemetry') == 't
    |   c' and getattr(self, '_tc_mode_activated_at', None):
396 |             import time
397 |             elapsed = time.time() - self._tc_mode_activated_at
398 |             timeout = getattr(self, '_tc_mode_timeout_seconds'
    |   , 300.0)
399 |             if elapsed >= timeout:
400 |                 self._telemetry_mode = 'telemetry'
401 |                 self._tc_mode_activated_at = None
402 |                 self._flush_interval_seconds = getattr(self, '_
    |   _standard_flush_interval_seconds', 30.0)
403 |                 self.save_config({
404 |                     'telemetry_mode': 'telemetry',
405 |                     'tc_mode_activated_at': None
406 |                 })
407 |                 logger.info(
408 |                     f" [Телеметрия] Режим реального времени '
    |   tc' завершен по таймауту ({int(timeout)}с). "
409 |                     f"Автоматически переключен в стандартный р
    |   ежим 'telemetry' (flush: {self._flush_interval_seconds}с, max_
    |   db: {self.get_max_db_size_mb()}МБ)"
410 |                 )
411 |                 return True
412 |             return False
413 |

```

```

414     def activate_tc_mode(self, duration_seconds: Optional[float] = None, flush_interval_seconds: Optional[float] = None) ->
    bool:
415         """Активирует режим отображения телеметрии реального в
    ремени ('tc') с задержкой сброса в N сек.
416
417         Args:
418             duration_seconds: Длительность режима ТС в секунда
    х до автопереключения в telemetry (по умолчанию 300с = 5 мин).
419             flush_interval_seconds: Интервал сброса в БД в сек
    ундах (по умолчанию 5.0с).
420
421         Returns:
422             bool: True при успешной активации.
423         """
424         import time
425         self._telemetry_mode = 'tc'
426         self._tc_mode_activated_at = time.time()
427         if flush_interval_seconds is not None:
428             self._tc_flush_interval_seconds = max(0.1, float(f
    lush_interval_seconds))
429         if duration_seconds is not None:
430             self._tc_mode_timeout_seconds = max(1.0, float(dur
    ation_seconds))
431         self._flush_interval_seconds = self._tc_flush_interval
    _seconds
432
433         updates: Dict[str, Any] = {
434             'telemetry_mode': 'tc',
435             'tc_mode_activated_at': self._tc_mode_activated_at
    ,
436             'tc_flush_interval_seconds': self._tc_flush_interv
    al_seconds,
437             'tc_mode_timeout_seconds': self._tc_mode_timeout_s
    econds,
438         }
439         logger.info(f" [Телеметрия] Активирован режим реально
    го времени 'tc' (flush: {self._tc_flush_interval_seconds}с, та

```

```

    ймаут: {self._tc_mode_timeout_seconds}c)")
440         return self.save_config(updates)
441
442     def deactivate_tc_mode(self) -> bool:
443         """Деактивирует режим 'tc' и возвращает телеметрию в с
тандартный режим 'telemetry'."""
444         self._telemetry_mode = 'telemetry'
445         self._tc_mode_activated_at = None
446         self._flush_interval_seconds = getattr(self, '_standar
d_flush_interval_seconds', 30.0)
447         updates = {
448             'telemetry_mode': 'telemetry',
449             'tc_mode_activated_at': None,
450         }
451         logger.info(f" [Телеметрия] Переключено в стандартны
й режим 'telemetry' (flush: {self._flush_interval_seconds}c)")
452         return self.save_config(updates)
453
454     def get_standard_flush_interval_seconds(self) -> float:
455         """Возвращает стандартный интервал сброса буфера в БД
(по умолчанию 30с)."""
456         return getattr(self, '_standard_flush_interval_seconds
', 30.0)
457
458     def get_tc_flush_interval_seconds(self) -> float:
459         """Возвращает интервал сброса в режиме реального време
ни ТС (по умолчанию 5с)."""
460         return getattr(self, '_tc_flush_interval_seconds', 5.0
)
461
462     def get_tc_mode_timeout_seconds(self) -> float:
463         """Возвращает таймаут действия режима ТС до автоперекл
ючения в стандартный (в секундах, по умолчанию 300с)."""
464         return getattr(self, '_tc_mode_timeout_seconds', 300.0
)
465
466     def get_flush_interval_seconds(self) -> float:
467         """Возвращает периодический интервал сброса буфера в Б

```

```

Д с учетом активного режима."""
468         self.check_tc_mode_auto_switch()
469         if getattr(self, '_telemetry_mode', 'telemetry') == 't
с':
470             return getattr(self, '_tc_flush_interval_seconds',
5.0)
471         return getattr(self, '_standard_flush_interval_seconds
', 30.0)
472
473     def get_multi_rate_config(self) -> Dict[str, Any]:
474         """Возвращает настройки асинхронного многоскоростного
ядра сбора телеметрии."""
475         return self._config.get('multi_rate_engine', {
476             'enabled': True,
477             'level3_realtime_interval_seconds': 1.0,
478             'level2_adaptive_min_interval_seconds': 0.5,
479             'level2_adaptive_max_interval_seconds': 30.0,
480             'level1_smart_interval_seconds': 43200.0,
481             'level1_battery_interval_seconds': 900.0,
482             'level1_security_interval_seconds': 3600.0,
483             'level1_scm_interval_seconds': 1800.0,
484             'deadband': {
485                 'temperature_c': 1.0,
486                 'cpu_load_pct': 3.0,
487                 'freq_mhz': 50.0,
488                 'power_w': 2.0,
489                 'fan_rpm': 100.0,
490                 'heartbeat_interval_seconds': 60.0,
491             }
492         })
493
494     def get_deadband_config(self) -> Dict[str, float]:
495         """Возвращает пороги Deadband для сенсоров."""
496         mr = self.get_multi_rate_config()
497         return mr.get('deadband', {
498             'temperature_c': 1.0,
499             'cpu_load_pct': 3.0,
500             'freq_mhz': 50.0,

```

```
501         'power_w': 2.0,
502         'fan_rpm': 100.0,
503         'heartbeat_interval_seconds': 60.0,
504     })
505
506     def get_max_db_size_mb(self) -> float:
507         """Возвращает максимальный разрешенный размер SQLite б
508 азы данных (в МБ, по умолчанию 100.0)."""
509         try:
510             val = float(self._config.get('max_db_size_mb', sel
511 f._config.get('max_file_size_mb', 100.0)))
512             return max(1.0, val)
513         except (ValueError, TypeError):
514             return 100.0
515
516     def get_retention_days(self) -> int:
517         """Возвращает срок хранения сырых записей телеметрии (
518 в днях)."""
519         try:
520             val = int(self._config.get('retention_days', 7))
521             return max(1, val)
522         except (ValueError, TypeError):
523             return 7
524
525     def get_db_cleanup_interval_seconds(self) -> float:
526         """Возвращает интервал периодической проверки и очистк
527 и БД (в секундах)."""
528         return getattr(self, '_db_cleanup_interval_seconds', 3
529 00.0)
530
531     def is_auto_vacuum_enabled(self) -> bool:
532         """Возвращает флаг разрешения авто-вакуума (VACUUM) SQ
533 Lite при усечении."""
534         return bool(self._config.get('auto_vacuum_enabled', Tr
535 ue))
536
537     def get_process_mode(self) -> str:
538         """Возвращает режим фильтрации процессов ('top_n' или
```

```

'all'))."""
532         return str(self._config.get('process_mode', 'top_n')).
lower()
533
534     def get_top_processes_limit(self) -> int:
535         """Возвращает настроенный лимит количества Топ-процесс
ов (по умолчанию 10)."""
536         try:
537             val = int(self._config.get('top_processes', 10))
538             return max(1, val)
539         except (ValueError, TypeError):
540             return 10
541
542     def get_effective_process_limit(self) -> int:
543         """Возвращает эффективный лимит процессов для сохранен
ия в БД телеметрии."""
544         pm = self.get_process_mode()
545         if pm == 'all':
546             return 500
547         return self.get_top_processes_limit()
548
549     def get_sensor_config(self, sensor_name: str) -> Dict[str,
Any]:
550         """Возвращает конфигурацию конкретного сенсора."""
551         return self._sensors_config.get(sensor_name, {'enabled
': False})
552
553     def is_sensor_enabled(self, sensor_name: str) -> bool:
554         """Проверяет, включен ли сенсор."""
555         config = self.get_sensor_config(sensor_name)
556         return bool(config.get('enabled', False))
557
558     def get_sensor_interval(self, sensor_name: str) -> float:
559         """Возвращает интервал опроса сенсора в секундах."""
560         config = self.get_sensor_config(sensor_name)
561         raw = config.get('interval_seconds', config.get('inter
val', self._default_interval))
562         return parse_interval_to_seconds(raw, self._default_in

```

```

    terval)
563
564     def get_sensor_metrics(self, sensor_name: str) -> List[str
    ]:
565         """Возвращает список метрик для сенсора."""
566         config = self.get_sensor_config(sensor_name)
567         return config.get('metrics', [])
568
569     def get_all_sensor_names(self) -> List[str]:
570         """Возвращает список всех имен сенсоров."""
571         return list(self._sensors_config.keys())
572
573     def get_enabled_sensors(self) -> List[str]:
574         """Возвращает список включенных сенсоров."""
575         return [name for name, config in self._sensors_config.
    items() if config.get('enabled', False)]
576
577     def get_logger_interval(self, logger_name: str, default: f
    loat = 60.0) -> float:
578         """Возвращает интервал опроса логгера приложения в сек
    ундах."""
579         logger_info = self._loggers_config.get(logger_name, {}
    )
580         if isinstance(logger_info, dict):
581             raw = logger_info.get('interval', self._config.get
    ('default_interval'))
582             return parse_interval_to_seconds(raw, default)
583             return float(default)
584
585     def is_logger_enabled(self, logger_name: str, default: boo
    l = True) -> bool:
586         """Проверяет активность логгера приложения."""
587         logger_info = self._loggers_config.get(logger_name, {}
    )
588         if isinstance(logger_info, dict):
589             return bool(logger_info.get('enabled', default))
590         return default
591

```

```

592     def get_all_intervals(self) -> Dict[str, Any]:
593         """Возвращает сводный словарь всех настроенных интерва
ЛОВ СИСТЕМЫ."""
594         return {
595             'telemetry_mode': self.get_telemetry_mode(),
596             'default_interval_seconds': self.get_interval_seco
nds(),
597             'heavy_interval_seconds': self.get_heavy_interval_
seconds(),
598             'fast_interval_seconds': self.get_fast_interval(),
599             'aggregation_interval_seconds': self.get_aggregati
on_interval(),
600             'heavy_disk_scan_interval_seconds': self.get_heavy
_disk_scan_interval(),
601             'db_cleanup_interval_seconds': self.get_db_cleanup
_interval_seconds(),
602             'flush_interval_seconds': self.get_flush_interval_
seconds(),
603             'standard_flush_interval_seconds': self.get_standar
d_flush_interval_seconds(),
604             'tc_flush_interval_seconds': self.get_tc_flush_int
erval_seconds(),
605             'tc_mode_timeout_seconds': self.get_tc_mode_timeou
t_seconds(),
606             'sensors': {
607                 name: self.get_sensor_interval(name)
608                 for name in self.get_all_sensor_names()
609             },
610             'loggers': {
611                 name: self.get_logger_interval(name)
612                 for name in self._loggers_config.keys()
613             }
614         }
615
616     def get_general_config(self) -> Dict[str, Any]:
617         """Возвращает общую конфигурацию (без сенсоров)."""
618         return {k: v for k, v in self._config.items() if k !=
'sensors'}

```



```

619
620     def get_config(self) -> Dict[str, Any]:
621         """Возвращает полную конфигурацию телеметрии."""
622         return self._config.copy()
623
624     @property
625     def config(self) -> Dict[str, Any]:
626         """Свойство для получения полной конфигурации."""
627         return self.get_config()
628
629     def save_config(self, new_config: Optional[Dict[str, Any]]
630 = None) -> bool:
631         """Сохраняет текущую или переданную конфигурацию в фай
632 л.
633
634     Args:
635         new_config: Опциональный словарь новых параметров.
636
637     Returns:
638         bool: True при успешном сохранении.
639
640     """
641     if new_config:
642         self._config.update(new_config)
643         if 'sensors' in new_config:
644             self._sensors_config = new_config['sensors']
645         if 'loggers' in new_config:
646             self._loggers_config = new_config['loggers']
647         if 'interval_seconds' in new_config:
648             self._default_interval = parse_interval_to_sec
649 onds(new_config['interval_seconds'], 5.0)
650         if 'heavy_interval_seconds' in new_config:
651             self._heavy_interval = parse_interval_to_secon
652 ds(new_config['heavy_interval_seconds'], 60.0)
653         if 'heavy_disk_scan_interval_seconds' in new_conf
654 ig:
655             self._heavy_disk_scan_interval = parse_interva
656 l_to_seconds(new_config['heavy_disk_scan_interval_seconds'], 4
657 3200.0)

```

```
650         if 'fast_interval_seconds' in new_config:
651             self._fast_interval = parse_interval_to_seconds(new_config['fast_interval_seconds'], 30.0)
652         if 'aggregation_interval_seconds' in new_config:
653             self._aggregation_interval = parse_interval_to_seconds(new_config['aggregation_interval_seconds'], 3600.0)
654         if 'heavy_mode_max_duration_days' in new_config:
655             self._heavy_mode_max_duration_days = float(new_config['heavy_mode_max_duration_days'])
656         if 'heavy_mode_auto_switch_enabled' in new_config:
657             self._heavy_mode_auto_switch_enabled = bool(new_config['heavy_mode_auto_switch_enabled'])
658         if 'mode' in new_config:
659             self._mode = str(new_config['mode']).lower()
660         if 'telemetry_mode' in new_config:
661             self._telemetry_mode = str(new_config['telemetry_mode']).lower()
662         if 'standard_flush_interval_seconds' in new_config:
663             self._standard_flush_interval_seconds = parse_interval_to_seconds(new_config['standard_flush_interval_seconds'], 30.0)
664         if 'tc_flush_interval_seconds' in new_config:
665             self._tc_flush_interval_seconds = parse_interval_to_seconds(new_config['tc_flush_interval_seconds'], 5.0)
666         if 'tc_mode_timeout_seconds' in new_config:
667             self._tc_mode_timeout_seconds = parse_interval_to_seconds(new_config['tc_mode_timeout_seconds'], 300.0)
668         if 'tc_mode_activated_at' in new_config:
669             self._tc_mode_activated_at = float(new_config['tc_mode_activated_at']) if new_config['tc_mode_activated_at'] is not None else None
670         if 'max_db_size_mb' in new_config:
671             self._max_db_size_mb = float(new_config['max_db_size_mb'])
672         if 'retention_days' in new_config:
673             self._retention_days = int(new_config['retention_days'])
```

```

674         if 'db_cleanup_interval_seconds' in new_config:
675             self._db_cleanup_interval_seconds = parse_inte
rval_to_seconds(new_config['db_cleanup_interval_seconds'], 300
|.0)
676         if 'flush_interval_seconds' in new_config:
677             self._flush_interval_seconds = parse_interv
al_to_seconds(new_config['flush_interval_seconds'], 30.0)
678         if 'auto_vacuum_enabled' in new_config:
679             self._auto_vacuum_enabled = bool(new_config['a
uto_vacuum_enabled'])
680         try:
681             target_path = Path(self._config_path)
682             target_path.parent.mkdir(parents=True, exist_ok=Tr
ue)
683             with open(target_path, 'w', encoding='utf-8') as f
:
684                 json.dump(self._config, f, indent=2, ensure_as
cii=False)
685             if target_path.is_file():
686                 self._last_mtime_ns = target_path.stat().st_mt
ime_ns
687             return True
688         except Exception:
689             return False

```

telemetry_engine.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - Engine
4 # =====
5 # Description:
6 #   Асинхронное многоскоростное ядро сбора телеметрии (Multi-Rate Decoupled Telemetry Engine).
7 #   Реализует 4-уровневое разделение контуров сбора с адаптивным управлением частотой,
8 #   Zero-Delay запуском (< 2 мс) и дедупликацией записи в SQLite (Deadband + Heartbeat).
9 #
10 # Usage Examples:
11 #   Python API:
12 #       from apps.windows.telemetry.telemetry_engine import TelemetryEngine
13 #
14 #       engine = TelemetryEngine()
15 #       await engine.start()
16 #
17 # File: telemetry_engine.py
18 # Project: ai-breadboard
19 # Package: apps.windows.telemetry
20 # Author: hypo69
21 # Copyright: © 2026 hypo69
22 # Updated: 2026-10-08 10:25:00
23 # =====
24
25 from __future__ import annotations
26 """Асинхронное многоскоростное ядро сбора телеметрии и адаптивного управления интервалами."""
27
28 import asyncio
```

```

29 import time
30 from datetime import datetime, timezone
31 from typing import Any, Callable, Dict, List, Optional, Set, T
   tuple
32
33 try:
34     import psutil
35     PSUTIL_AVAILABLE = True
36 except ImportError:
37     psutil = None
38     PSUTIL_AVAILABLE = False
39
40 try:
41     from logger import logger
42 except ImportError:
43     import logging
44     logger = logging.getLogger(__name__)
45
46 from .collector import SystemCollector
47 from .models import (
48     CpuMetrics,
49     GpuMetrics,
50     MemoryMetrics,
51     NetworkInterfaceMetrics,
52     ProcessMetrics,
53     SystemSnapshot,
54 )
55 from .sqlite import TelemetryStorage
56 from .telemetry_config import TelemetryConfigManager
57 from .win32_ffl.nethelper import IPHelperAPI, NetworkSocketInfo
58
59
60 class DeadbandTracker:
61     """Трекер зон нечувствительности (Deadband) для адаптивной
   фильтрации сенсоров."""
62
63     def __init__(

```

```

64 |         self,
65 |         temp_threshold: float = 1.0,
66 |         cpu_load_threshold: float = 3.0,
67 |         freq_threshold_mhz: float = 50.0,
68 |         power_threshold_w: float = 2.0,
69 |         fan_threshold_rpm: float = 100.0,
70 |         heartbeat_interval_sec: float = 60.0,
71 |     ) -> None:
72 |         """Инициализирует пороги фильтрации.
73 |
74 |         Args:
75 |             temp_threshold: Порог температуры (°C).
76 |             cpu_load_threshold: Порог загрузки CPU (%).
77 |             freq_threshold_mhz: Порог частоты (МГц).
78 |             power_threshold_w: Порог мощности (Вт).
79 |             fan_threshold_rpm: Порог скорости вентилятора (RPM
80 |             heartbeat_interval_sec: Интервал контрольного кадр
81 |             a Heartbeat (сек).
82 |             """
83 |         self.temp_threshold = temp_threshold
84 |         self.cpu_load_threshold = cpu_load_threshold
85 |         self.freq_threshold_mhz = freq_threshold_mhz
86 |         self.power_threshold_w = power_threshold_w
87 |         self.fan_threshold_rpm = fan_threshold_rpm
88 |         self.heartbeat_interval_sec = heartbeat_interval_sec
89 |
90 |         self._last_values: Dict[str, float] = {}
91 |         self._last_saved_values: Dict[str, float] = {}
92 |         self._last_saved_time: Dict[str, float] = {}
93 |
94 |     def _get_threshold_for_sensor(self, sensor_id: str, category: str, unit: str) -> float:
95 |         """Определяет порог Deadband для конкретного сенсора."
96 |
97 |         cat_lower = category.lower()
98 |         unit_lower = unit.lower()
99 |         sid_lower = sensor_id.lower()

```

```

98
99         if 'temp' in cat_lower or '°c' in unit_lower or 'temp'
|         in sid_lower:
100             return self.temp_threshold
101         if 'load' in cat_lower or '%' in unit_lower or 'util'
|         in sid_lower:
102             return self.cpu_load_threshold
103         if 'clock' in cat_lower or 'mhz' in unit_lower or 'fre
|         q' in sid_lower:
104             return self.freq_threshold_mhz
105         if 'power' in cat_lower or 'w' in unit_lower or 'power
|         ' in sid_lower:
106             return self.power_threshold_w
107         if 'fan' in cat_lower or 'rpm' in unit_lower or 'fan'
|         in sid_lower:
108             return self.fan_threshold_rpm
109         return 2.0
110
111     def check_spike(self, readings: List[Dict[str, Any]]) -> b
|     ool:
112         """Проверяет, превысил ли хотя бы один параметр порог
|         Deadband с прошлого замера.
113
114         Args:
115             readings: Список текущих показаний датчиков.
116
117         Returns:
118             bool: True если зафиксирован всплеск (Trigger-Up),
|             иначе False.
119         """
120         has_spike = False
121         for r in readings:
122             sid = str(r.get('id') or r.get('sensor_id') or '')
123             if not sid:
124                 continue
125             raw_val = r.get('value', r.get('value_num', 0.0))
126             try:
127                 val = float(raw_val)

```

```

128         except (TypeError, ValueError):
129             continue
130
131         prev_val = self._last_values.get(sid)
132         threshold = self._get_threshold_for_sensor(
133             sid,
134             str(r.get('sensor_category') or ''),
135             str(r.get('unit') or ''),
136         )
137
138         if prev_val is not None:
139             delta = abs(val - prev_val)
140             if delta >= threshold:
141                 has_spike = True
142
143         self._last_values[sid] = val
144
145     return has_spike
146
147     def filter_records_for_storage(
148         self, readings: List[Dict[str, Any]], now_epoch: Optional[float] = None
149     ) -> List[Dict[str, Any]]:
150         """Фильтрует записи для записи в БД: сохраняет только
151         изменения > Deadband или Heartbeat.
152
153         Args:
154             readings: Список текущих показаний датчиков.
155             now_epoch: Текущее время Unix epoch.
156
157         Returns:
158             List[Dict[str, Any]]: Отфильтрованные записи для с
159             хранения в SQLite.
160         """
161         now = now_epoch if now_epoch is not None else time.time()
162
163         to_store: List[Dict[str, Any]] = []

```



```

162         for r in readings:
163             sid = str(r.get('id') or r.get('sensor_id') or '')
164             if not sid:
165                 continue
166             raw_val = r.get('value', r.get('value_num', 0.0))
167             try:
168                 val = float(raw_val)
169             except (TypeError, ValueError):
170                 continue
171
172             last_saved = self._last_saved_values.get(sid)
173             last_time = self._last_saved_time.get(sid, 0.0)
174             threshold = self._get_threshold_for_sensor(
175                 sid,
176                 str(r.get('sensor_category') or ''),
177                 str(r.get('unit') or ''),
178             )
179
180             should_save = False
181             if last_saved is None:
182                 should_save = True
183             elif abs(val - last_saved) >= threshold:
184                 should_save = True
185             elif (now - last_time) >= self.heartbeat_interval_
sec:
186                 should_save = True
187
188             if should_save:
189                 self._last_saved_values[sid] = val
190                 self._last_saved_time[sid] = now
191                 to_store.append(r)
192
193         return to_store
194
195
196 class TelemetryEngine:
197     """Асинхронное ядро сбора системной телеметрии (Multi-Rate
    Decoupled Telemetry Engine)."""

```

```
198 |
199 |     def __init__(
200 |         self,
201 |         collector: Optional[SystemCollector] = None,
202 |         storage: Optional[TelemetryStorage] = None,
203 |         config_manager: Optional[TelemetryConfigManager] = Non
    | e,
204 |     ) -> None:
205 |         """Инициализирует ядро телеметрии с независимыми конту
    | рами.
206 |
207 |         Args:
208 |             collector: Экземпляр SystemCollector для системных
    | вызовов.
209 |             storage: Экземпляр TelemetryStorage для персистент
    | ности.
210 |             config_manager: Экземпляр TelemetryConfigManager.
211 |         """
212 |         self.collector = collector or SystemCollector()
213 |         self.storage = storage or TelemetryStorage.get_instanc
    | e()
214 |         self.config_manager = config_manager or TelemetryConfi
    | gManager()
215 |
216 |         self.is_running = False
217 |         self.static_inventory_loaded = False
218 |         self.static_catalog: Dict[str, Any] = {}
219 |
220 |         self.deadband_tracker = DeadbandTracker()
221 |         self.ip_helper = IPHelperAPI() if hasattr(IPHelperAPI,
    | '__init__') else None
222 |
223 |         self._tasks: List[asyncio.Task] = []
224 |         self._current_sensor_interval: float = 2.0
225 |         self._cooldown_ticks: int = 0
226 |         self._main_tick_count: int = 0
227 |         self._last_snapshot: Optional[SystemSnapshot] = None
228 |         self._last_smart_scan_time: float = 0.0
```

```

229         self._last_battery_scan_time: float = 0.0
230         self._last_security_scan_time: float = 0.0
231         self._last_scm_scan_time: float = 0.0
232
233     async def start(self) -> None:
234         """Единая асинхронная точка входа запуска службы телем
етрии.
235
236         Обеспечивает Zero-Delay старт (< 2 мс) и разделение ко
нтуров по 4 приоритетам.
237         """
238         if self.is_running:
239             logger.debug("[TelemetryEngine] Сервис уже запущен
.")
240             return
241
242         self.is_running = True
243         logger.info("[TelemetryEngine] □ Запуск Multi-Rate De
coupled Telemetry Engine...")
244
245         # 1. ЗАПУСК ГЛАВНОГО REAL-TIME ЦИКЛА (LEVEL 3) - Старт
ует мгновенно (< 2 мс)
246         main_task = asyncio.create_task(
247             self._main_realtime_tick_loop(),
248             name="MainRealtimeTickTask",
249         )
250         self._tasks.append(main_task)
251
252         # 2. ОТЛОЖЕННЫЙ ЗАПУСК СТАТИЧЕСКОГО ПАСПОРТА (LEVEL 0)
- Задержка 200 мс
253         deferred_task = asyncio.create_task(
254             self._deferred_static_inventory_task(),
255             name="DeferredStaticInventoryTask",
256         )
257         self._tasks.append(deferred_task)
258
259         # 3. АДАПТИВНЫЙ ДИНАМИЧЕСКИЙ КОНТУР СЕНСОРОВ (LEVEL 2)
- Задержка 500 мс

```

```

260         adaptive_task = asyncio.create_task(
261             self._adaptive_sensors_loop(),
262             name="AdaptiveSensorsTask",
263         )
264         self._tasks.append(adaptive_task)
265
266         # 4. НИЗКОЧАСТОТНЫЙ ФОНОВЫЙ ОПРОС (LEVEL 1) - Задержка
267         5.0 с
268         low_freq_task = asyncio.create_task(
269             self._low_frequency_background_loop(),
270             name="LowFrequencyTask",
271         )
272         self._tasks.append(low_freq_task)
273
274         # 5. ПАКЕТНЫЙ WAL-СБРОС БУФЕРА В SQLITE (Раз в 5.0 с)
275         wal_writer_task = asyncio.create_task(
276             self._batch_wal_flusher_loop(),
277             name="BatchWalFlusherTask",
278         )
279         self._tasks.append(wal_writer_task)
280
281         logger.info("[TelemetryEngine] □ Все контуры телеметрии
и инициализированы.")
282
283         async def stop(self) -> None:
284             """Корректно останавливает все асинхронные задачи и сб
285             расывает буферы."""
286             if not self.is_running:
287                 return
288
289             self.is_running = False
290             logger.info("[TelemetryEngine] Остановка ядра телеметр
291             ии...")
292
293             for task in self._tasks:
294                 if not task.done():
295                     task.cancel()

```

```

294         await asyncio.gather(*self._tasks, return_exceptions=True)
295         self._tasks.clear()
296
297         # Финальный сброс буфера в SQLite
298         try:
299             self.storage.flush()
300         except Exception as ex:
301             logger.debug(f"[TelemetryEngine] Ошибка при финаль
ном сбросе буфера: {ex}")
302
303             logger.info(f"[TelemetryEngine] Сервис остановлен. Вып
олнено главных тиков: {self._main_tick_count}")
304
305     async def _main_realtime_tick_loop(self) -> None:
306         """Главный высокоприоритетный Real-Time контур (Level
3): PID, сокеты, файловый I/O.
307
308         СТРОГОЕ ПРАВИЛО: Только нативные быстрые C-FFI вызовы
(iphlapi, ReadDirectoryChangesW, psutil).
309         Никаких WMI, COM, heavy IOCTL или синхронных блокировок!
310
311         """
312         logger.info("[MainTick] □ Главный Real-Time контур (Le
vel 3) успешно запущен!")
313         self._main_tick_count = 0
314
315         while self.is_running:
316             t0 = time.perf_counter()
317             self._main_tick_count += 1
318
319             try:
320                 # 1. Попроцессные метрики по PID (CPU, RAM, ha
ndles, threads)
321                 top_procs = await self._collect_pid_metrics_fa
st()
322
323                 # 2. Сетевые сокеты и соединения по PID через

```

```

|iphlpapi.dll
323         sockets = await self._collect_network_sockets_
|fast()
324
325         # 3. Базовые системные метрики (CPU, RAM, Диск
|I/O, Сеть I/O)
326         cpu_m = self._get_fast_cpu_metrics()
327         mem_m = self.collector.get_memory_metrics()
328         parts, disk_io = self.collector.get_disk_metri
|cs()
329         net_m = self.collector.get_network_metrics()
330
331         snapshot = SystemSnapshot(
332             hostname=self.static_catalog.get('hostname
|') or 'localhost',
333             uptime_seconds=round(time.time() - (psutil
|.boot_time() if PSUTIL_AVAILABLE else time.time()), 1),
334             cpu=cpu_m,
335             memory=mem_m,
336             disks=parts,
337             disk_io=disk_io,
338             network=net_m,
339             top_processes=top_procs,
340         )
341         self._last_snapshot = snapshot
342
343         # Сохраняем в буфер SQLite без блокирующего fs
|ync
344         self.storage.save_snapshot(snapshot, top_n=len
|(top_procs))
345
346         except asyncio.CancelledError:
347             break
348         except Exception as e:
349             logger.error(f"[MainTick] Ошибка в главном тик
|e #{self._main_tick_count}: {e}", exc_info=True)
350
351         dt = time.perf_counter() - t0

```

```

352         sleep_time = max(0.05, 1.0 - dt)
353         try:
354             await asyncio.sleep(sleep_time)
355         except asyncio.CancelledError:
356             break
357
358     async def _deferred_static_inventory_task(self) -> None:
359         """Отложенный сбор статического паспорта системы (Level 0).
360
361         Запускается асинхронно СТРОГО ПОСЛЕ старта главного цикла (пауза 200 мс).
362
363         Тяжелые вызовы WMI/COM/реестра выносятся в ThreadPoolExecutor.
364
365         """
366         try:
367             await asyncio.sleep(0.2)
368         except asyncio.CancelledError:
369             return
370
371         logger.info("[Level-0] □ Начало фонового сбора статического паспорта системы...")
372         t0 = time.perf_counter()
373
374         loop = asyncio.get_running_loop()
375         try:
376             self.static_catalog = await loop.run_in_executor(None, self._heavy_hardware_inventory_probe)
377             self.static_inventory_loaded = True
378
379             # Фиксируем паспорт и снимки в БД
380             try:
381                 archive_entry = self.collector.archive_hardware_state(auto_diff=True)
382                 if archive_entry:
383                     self.storage.save_hardware_archive(archive_entry)
384             except Exception as hw_err:

```

```

383 |             logger.debug(f"[Level-0] Ошибка архивирования
| оборудование: {hw_err}")
384 |
385 |             try:
386 |                 startup_entry = self.collector.archive_startup
| _state(auto_diff=True)
387 |                 if startup_entry:
388 |                     self.storage.save_startup_archive(startup_
| entry)
389 |             except Exception as st_err:
390 |                 logger.debug(f"[Level-0] Ошибка архивирования
| автозапуска: {st_err}")
391 |
392 |                 dt = time.perf_counter() - t0
393 |                 logger.info(f"[Level-0] □ Паспорт системы сформиро
| ван за {dt:.2f}s и сохранен в памяти.")
394 |             except asyncio.CancelledError:
395 |                 pass
396 |             except Exception as ex:
397 |                 logger.error(f"[Level-0] Ошибка сбора статического
| паспорта: {ex}")
398 |
399 |     def _heavy_hardware_inventory_probe(self) -> Dict[str, Any
| ]:
400 |         """Синхронный тяжелый опрос WMI, SPD памяти, специфика
| ций GPU и дисков."""
401 |         ident = self.collector.get_system_identity()
402 |         ram_sticks = self.collector.get_ram_sticks_sync() if h
| asattr(self.collector, 'get_ram_sticks_sync') else []
403 |         if not ram_sticks and hasattr(self.collector, 'get_ram
| _sticks'):
404 |             try:
405 |                 ram_sticks = asyncio.run(self.collector.get_ra
| m_sticks())
406 |             except Exception:
407 |                 ram_sticks = []
408 |
409 |         phys_disks = self.collector.get_physical_disks_health(

```



```

    force=True)
410         gpus = self.collector.get_gpu_metrics()
411
412         return {
413             "hostname": ident.get("hostname", ""),
414             "username": ident.get("username", ""),
415             "os_build": ident.get("os_build", ""),
416             "os_install_date": ident.get("os_install_date", ""
        ),
417             "system_language": ident.get("system_language", ""
        ),
418             "user_locale": ident.get("user_locale", ""),
419             "timezone": ident.get("timezone", ""),
420             "ram_spd": [f"{r.manufacturer} {r.speed_mhz}MHz ({
r.part_number})" for r in (ram_sticks or [])],
421             "disks_passport": [f"{d.model} ({d.size_gb}GB, {d.
media_type})" for d in (phys_disks or [])],
422             "gpu_specs": [f"{g.name} ({g.vendor}, {g.memory_to
tal_gb}GB)" for g in (gpus or [])],
423         }
424
425     async def _adaptive_sensors_loop(self) -> None:
426         """Адаптивный динамический контур (Level 2): опрос сен
соров от 500 мс до 30.0 с.
427
428         Использует Deadband, Trigger-Up (500 мс при всплеске),
Cooldown (10 тиков)
429         и Exponential Back-Off (* 1.5 при штиле).
430         """
431         try:
432             await asyncio.sleep(0.5)
433         except asyncio.CancelledError:
434             return
435
436         self._current_sensor_interval = 2.0
437         self._cooldown_ticks = 0
438
439         while self.is_running:

```

```

440         t0 = time.perf_counter()
441         try:
442             loop = asyncio.get_running_loop()
443             readings = await loop.run_in_executor(None, se
lf._poll_sensors_sync)
444
445             # Проверка всплеска по Deadband
446             has_spike = self.deadband_tracker.check_spike(
readings)
447
448             if has_spike:
449                 self._current_sensor_interval = 0.5 # Мгн
овенный сброс (Trigger-Up)
450                 self._cooldown_ticks = 10 # 10 т
иков удержания высокой частоты
451                 elif self._cooldown_ticks > 0:
452                     self._current_sensor_interval = 0.5 # Уде
ржание режима остывания
453                     self._cooldown_ticks -= 1
454                 else:
455                     # Плавное замедление при штиле
456                     self._current_sensor_interval = min(30.0,
self._current_sensor_interval * 1.5)
457
458                 # Дедупликация и запись в SQLite (только измен
ения > Deadband или Heartbeat)
459                 filtered_readings = self.deadband_tracker.filt
er_records_for_storage(readings)
460                 if filtered_readings:
461                     self.storage.save_sensor_polls(filtered_re
adings)
462
463             except asyncio.CancelledError:
464                 break
465             except Exception as e:
466                 logger.error(f"[AdaptiveSensors] Ошибка: {e}")
467
468         dt = time.perf_counter() - t0

```

```

469         sleep_time = max(0.05, self._current_sensor_interv
al - dt)
470         try:
471             await asyncio.sleep(sleep_time)
472         except asyncio.CancelledError:
473             break
474
475     def _poll_sensors_sync(self) -> List[Dict[str, Any]]:
476         """Синхронный сбор доступных сенсоров."""
477         try:
478             from .sensors import get_hardware_sensors
479             sensors = get_hardware_sensors()
480             return [
481                 {
482                     "id": s.sensor_id,
483                     "sensor_name": s.name,
484                     "sensor_category": s.category,
485                     "unit": s.unit,
486                     "value": s.value,
487                     "hardware_type": getattr(s, "hardware_type
", "cpu"),
488                     "hardware_name": getattr(s, "hardware_name
", "System"),
489                 }
490                 for s in sensors
491             ]
492         except Exception as ex:
493             logger.debug(f"[AdaptiveSensors] Ошибка сбора сенс
оров: {ex}")
494             return []
495
496     async def _low_frequency_background_loop(self) -> None:
497         """Низкочастотный фоновый опрос (Level 1): SMART (12ч)
, Аккумулятор (15мин), Defender/Безопасность (1ч)."""
498         try:
499             await asyncio.sleep(5.0)
500         except asyncio.CancelledError:
501             return

```

```
502 |
503 |     while self.is_running:
504 |         now = time.time()
505 |         loop = asyncio.get_running_loop()
506 |
507 |         try:
508 |             # 1. SMART SSD/NVMe (каждые 12 часов = 43200 с
509 |             )
510 |             if now - self._last_smart_scan_time >= 43200.0
511 |             :
512 |                 await loop.run_in_executor(None, self._pol
513 |                 l_smart_sync)
514 |                 self._last_smart_scan_time = now
515 |
516 |             # 2. Аккумулятор и профиль питания (каждые 15
517 |             минут = 900 с)
518 |             if now - self._last_battery_scan_time >= 900.0
519 |             :
520 |                 await loop.run_in_executor(None, self._pol
521 |                 l_battery_sync)
522 |                 self._last_battery_scan_time = now
523 |
524 |             # 3. Аудит Defender и безопасности (каждый 1 ч
525 |             ас = 3600 с)
526 |             if now - self._last_security_scan_time >= 3600
527 |             .0:
528 |                 await loop.run_in_executor(None, self._pol
529 |                 l_security_sync)
530 |                 self._last_security_scan_time = now
531 |
532 |             # 4. Реестр служб SCM и задач планировщика (ка
533 |             ждые 30 минут = 1800 с)
534 |             if now - self._last_scm_scan_time >= 1800.0:
535 |                 await loop.run_in_executor(None, self._pol
536 |                 l_scm_tasks_sync)
537 |                 self._last_scm_scan_time = now
538 |
539 |         except asyncio.CancelledError:
```

```

529         break
530     except Exception as e:
531         logger.error(f"[LowFreq] Ошибка: {e}")
532
533     try:
534         await asyncio.sleep(60.0) # Проверка расписан
ия каждую минуту
535     except asyncio.CancelledError:
536         break
537
538     def _poll_smart_sync(self) -> None:
539         """Опрос SMART накопителей."""
540         try:
541             self.collector.get_physical_disks_health(force=True
e)
542             logger.info("[LowFreq] □ S.M.A.R.T. опрос физичес
ких накопителей успешно выполнен.")
543         except Exception as ex:
544             logger.debug(f"[LowFreq] SMART error: {ex}")
545
546     def _poll_battery_sync(self) -> None:
547         """Опрос состояния батареи."""
548         try:
549             self.collector.get_battery_metrics()
550         except Exception as ex:
551             logger.debug(f"[LowFreq] Battery error: {ex}")
552
553     def _poll_security_sync(self) -> None:
554         """Опрос состояния безопасности и Defender."""
555         try:
556             from .service import TelemetryLoggerService
557             svc = TelemetryLoggerService.get_instance()
558             svc.collect_and_save_defender_state()
559             logger.info("[LowFreq] □ Аудит безопасности Defen
der зафиксирован в БД.")
560         except Exception as ex:
561             logger.debug(f"[LowFreq] Security audit error: {ex
}")

```

```

562
563     def _poll_scm_tasks_sync(self) -> None:
564         """Опрос состояния служб и планировщика."""
565         try:
566             self.collector.get_extended_system_audit()
567         except Exception as ex:
568             logger.debug(f"[LowFreq] SCM/Tasks audit error: {e
x}")
569
570     async def _batch_wal_flusher_loop(self) -> None:
571         """Периодический сброс буфера в SQLite транзакцией каж
дые 5.0 с."""
572         while self.is_running:
573             try:
574                 await asyncio.sleep(5.0)
575                 self.storage.flush()
576             except asyncio.CancelledError:
577                 break
578             except Exception as e:
579                 logger.debug(f"[WalFlusher] Ошибка сброса буфе
ра: {e}")
580
581     async def _collect_pid_metrics_fast(self) -> List[ProcessM
etrics]:
582         """Быстрый сбор метрик активных процессов через psutil
без блокировок."""
583         if not PSUTIL_AVAILABLE:
584             return []
585         loop = asyncio.get_running_loop()
586         return await loop.run_in_executor(None, self.collector
.get_top_processes, 10, 'cpu')
587
588     async def _collect_network_sockets_fast(self) -> List[Netw
orkSocketInfo]:
589         """Быстрый опрос сокетов через iphlapi.dll."""
590         if not self.ip_helper:
591             return []
592         loop = asyncio.get_running_loop()

```

```
593 |         try:
594 |             return await loop.run_in_executor(None, self.ip_he
| lper.get_tcp_connections)
595 |         except Exception:
596 |             return []
597 |
598 |     def _get_fast_cpu_metrics(self) -> CpuMetrics:
599 |         """Получение базовых метрик CPU."""
600 |         total_p = 0.0
601 |         freq_mhz = 0.0
602 |         if PSUTIL_AVAILABLE:
603 |             try:
604 |                 total_p = psutil.cpu_percent(interval=None)
605 |                 cf = psutil.cpu_freq()
606 |                 if cf:
607 |                     freq_mhz = round(cf.current, 1)
608 |             except Exception:
609 |                 pass
610 |         return CpuMetrics(
611 |             model=self.static_catalog.get("cpu_model", "x86_64
| "),
612 |             total_percent=total_p,
613 |             frequency_mhz=freq_mhz,
614 |             physical_cores=psutil.cpu_count(logical=False) or
| 1 if PSUTIL_AVAILABLE else 1,
615 |             logical_cores=psutil.cpu_count(logical=True) or 1
| if PSUTIL_AVAILABLE else 1,
616 |         )
617 |
```

w64_collector.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - W64 Collector
4 # =====
5 # Description:
6 #   AI Windows 64-bit Collector - модуль сбора детальных системных событий телеметрии.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.w64_collector import AIW64Collector
11 #
12 #       service = AIW64Collector()
13 #
14 # File: w64_collector.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-06 07:35:00
20 # =====
21
22 from __future__ import annotations
23 """AI Windows 64-bit Collector - модуль сбора детальных системных событий телеметрии."""
24
25 import json
26 import os
27 import re
28 import subprocess
29 import threading
```



```

30 import time
31 from datetime import datetime, timezone
32 from pathlib import Path
33 from typing import Any, Callable, Dict, List, Optional
34 from logger import logger
35 from .models import W64CollectorStatus, W64SystemEvent
36 from .sqlite import TelemetryStorage
37
38
39 class AIW64Collector:
40     """Максимально детальный сборщик событий Windows 64-bit дл
я подсистемы телеметрии.
41
42     Фиксирует непрерывные изменения системного состояния:
43     - Процессы (создание, завершение, активность)
44     - Файловая система (изменения в отслеживаемых путях)
45     - Реестр Windows (модификации ключей)
46     - Сетевая активность (новые соединения и открытые порты)
47     """
48
49     def __init__(
50         self,
51         enable_file_monitoring: bool = True,
52         enable_process_monitoring: bool = True,
53         enable_registry_monitoring: bool = True,
54         enable_network_monitoring: bool = True,
55         enable_event_log_monitoring: bool = True,
56         monitored_paths: Optional[List[str]] = None,
57         poll_interval_sec: float = 1.0,
58         storage: Optional[TelemetryStorage] = None,
59         on_event_callback: Optional[Callable[[Dict[str, Any]],
None]] = None,
60         **kwargs: Any,
61     ) -> None:
62         """Инициализация сборщика системных событий W64.
63
64         Args:
65             enable_file_monitoring: Флаг мониторинга изменений

```

```

        файлов.
66         enable_process_monitoring: Флаг мониторинга процес
сов.
67         enable_registry_monitoring: Флаг мониторинга реест
ра.
68         enable_network_monitoring: Флаг мониторинга сетевы
х соединений.
69         enable_event_log_monitoring: Флаг мониторинга журн
алов Windows.
70         monitored_paths: Список путей файловой системы для
отслеживания.
71         poll_interval_sec: Интервал между циклами сканиров
ания изменений (сек).
72         storage: Экземпляр хранилища TelemetryStorage для
сохранения в SQLite через буфер.
73         on_event_callback: Опциональный callback-обработчи
к каждого зафиксированного события.
74         **kwargs: Дополнительные параметры конфигурации.
75         """
76         self.enable_file_monitoring = enable_file_monitoring
77         self.enable_process_monitoring = enable_process_monito
ring
78         self.enable_registry_monitoring = enable_registry_moni
toring
79         self.enable_network_monitoring = enable_network_monito
ring
80         self.enable_event_log_monitoring = enable_event_log_mo
nitoring
81         self.poll_interval = max(0.1, poll_interval_sec)
82         self.storage = storage or TelemetryStorage.get_instanc
e()
83         self.on_event_callback = on_event_callback
84
85         self.monitored_paths = monitored_paths or [
86             os.path.expandvars(r'%USERPROFILE%\Documents'),
87             os.path.expandvars(r'%USERPROFILE%\Downloads'),
88             os.path.expandvars(r'%USERPROFILE%\Desktop'),
89             r'C:\Program Files',

```

```

90         r'C:\Program Files (x86)',
91         r'C:\Windows\System32',
92         r'C:\Windows\SysWOW64',
93     ]
94
95     self._running = False
96     self._thread: Optional[threading.Thread] = None
97     self._stop_event = threading.Event()
98     self._event_counter = 0
99     self._last_event_time: Optional[str] = None
100     self._last_baseline: Dict[str, Any] = {}
101     logger.debug('AIW64Collector инициализирован (буфериза
ция в оперативной памяти)')
102
103     def _log_event(self, event_type: str, event_data: Dict[str
, Any]) -> None:
104         """Зафиксировать событие телеметрии в оперативной памя
ти с пакетным сбросом в SQLite.
105
106         Args:
107             event_type: Тип события.
108             event_data: Словарь с атрибутами события.
109         """
110         self._event_counter += 1
111         now_iso = datetime.now(timezone.utc).isoformat()
112         event_data['event_id'] = f'evt_{int(time.time())}_{sel
f._event_counter}'
113         event_data['timestamp'] = now_iso
114         event_data['event_type'] = event_type
115         self._last_event_time = now_iso
116
117         # 1. Сохранение в буфер памяти TelemetryStorage
118         if self.storage:
119             try:
120                 self.storage.save_w64_event(event_data, provid
er='w64_collector')
121             except Exception as ex:
122                 logger.debug(f'Ошибка сохранения W64 события в

```

```

    БД: {ex}')
123
124     # 2. Передача в callback при наличии
125     if self.on_event_callback:
126         try:
127             self.on_event_callback(event_data)
128         except Exception as ex:
129             logger.debug(f'Ошибка в on_event_callback: {ex
    }')
130
131     def start(self) -> bool:
132         """Запустить фоновый поток мониторинга событий.
133
134         Returns:
135             bool: True если сборщик успешно запущен, False есл
и уже работал.
136         """
137         if self._running:
138             logger.warning('AIW64Collector уже запущен')
139             return False
140
141         self._running = True
142         self._stop_event.clear()
143         self._update_baseline()
144         self._thread = threading.Thread(target=self._monitorin
g_loop, name='AIW64CollectorWorker', daemon=True)
145         self._thread.start()
146         logger.info('AIW64Collector успешно запущен')
147         return True
148
149     def stop(self) -> bool:
150         """Остановить фоновый поток мониторинга.
151
152         Returns:
153             bool: True если сборщик остановлен, False если не
был активен.
154         """
155         if not self._running:

```

```

156         return False
157
158     self._running = False
159     self._stop_event.set()
160     if self._thread and self._thread.is_alive():
161         self._thread.join(timeout=5.0)
162     logger.info(f'AIW64Collector остановлен. Всего зафиксир
163 ровано событий: {self._event_counter}')
164     return True
165
166     def _update_baseline(self) -> None:
167         """Сформировать начальный снимок состояния системы для
168 сравнения."""
169         self._last_baseline = {
170             'processes': self._get_current_processes(),
171             'network_connections': self._get_current_network_c
172 onnections(),
173             'registry_keys': self._get_registry_keys(),
174             'file_hashes': self._get_file_hashes(),
175         }
176
177     def _monitoring_loop(self) -> None:
178         """Основной рабочий цикл мониторинга и выявления измен
179 ений."""
180         last_check = time.time()
181         while not self._stop_event.is_set():
182             try:
183                 current_time = time.time()
184                 if current_time - last_check >= self.poll_inte
185 rval:
186                     current_state = {
187                         'processes': self._get_current_process
188 es(),
189                         'network_connections': self._get_curre
190 nt_network_connections(),
191                         'registry_keys': self._get_registry_ke
192 ys(),
193                         'file_hashes': self._get_file_hashes()

```

```

186         }
187         self._detect_changes(self._last_baseline,
current_state)
188         self._last_baseline = current_state
189         last_check = current_time
190
191         self._stop_event.wait(timeout=min(0.5, self.po
ll_interval))
192         except Exception as ex:
193             logger.error(f'Ошибка в цикле мониторинга AIW6
4Collector: {ex}')
194
195     def _detect_changes(self, old_state: Dict[str, Any], new_s
tate: Dict[str, Any]) -> None:
196         """Сравнить предыдущее и текущее состояние и залогиров
ать изменения.
197
198         Args:
199             old_state: Предыдущий системный срез.
200             new_state: Текущий системный срез.
201         """
202         # Мониторинг процессов
203         if self.enable_process_monitoring:
204             old_procs = {p['pid']: p for p in old_state.get('p
rocesses', [])}
205             new_procs = {p['pid']: p for p in new_state.get('p
rocesses', [])}
206
207             for pid, proc in new_procs.items():
208                 if pid not in old_procs:
209                     self._log_event('process_start', proc)
210
211             for pid, proc in old_procs.items():
212                 if pid not in new_procs:
213                     self._log_event('process_stop', proc)
214
215         # Мониторинг сетевых соединений

```

```

216         if self.enable_network_monitoring:
217             old_conns = {c.get('key'): c for c in old_state.ge
t('network_connections', []) if c.get('key')}
218             new_conns = {c.get('key'): c for c in new_state.ge
t('network_connections', []) if c.get('key')}
219
220             for key, conn in new_conns.items():
221                 if key not in old_conns:
222                     self._log_event('network_open', conn)
223
224             for key, conn in old_conns.items():
225                 if key not in new_conns:
226                     self._log_event('network_close', conn)
227
228         # Мониторинг файлов
229         if self.enable_file_monitoring:
230             old_files = old_state.get('file_hashes', {})
231             new_files = new_state.get('file_hashes', {})
232
233             for path, h in new_files.items():
234                 if path not in old_files:
235                     self._log_event('file_created', {'path': p
ath, 'hash': h})
236                 elif old_files[path] != h:
237                     self._log_event('file_modified', {'path':
path, 'old_hash': old_files[path], 'new_hash': h})
238
239             for path, h in old_files.items():
240                 if path not in new_files:
241                     self._log_event('file_deleted', {'path': p
ath, 'hash': h})
242
243         # Мониторинг реестра
244         if self.enable_registry_monitoring:
245             old_reg = old_state.get('registry_keys', {})
246             new_reg = new_state.get('registry_keys', {})
247
248             for key, val in new_reg.items():

```

```

249         if key not in old_reg:
250             self._log_event('registry_created', {'key'
: key, 'value': val})
251         elif old_reg[key] != val:
252             self._log_event('registry_modified', {'key
': key, 'old_value': old_reg[key], 'new_value': val})
253
254         for key, val in old_reg.items():
255             if key not in new_reg:
256                 self._log_event('registry_deleted', {'key'
: key, 'value': val})
257
258     def _get_current_processes(self) -> List[Dict[str, Any]]:
259         """Получить текущий список активных процессов в систем
e."""
260         processes = []
261         try:
262             import psutil
263             for proc in psutil.process_iter(['pid', 'name', 'e
xe', 'cmdline', 'username', 'create_time']):
264                 try:
265                     info = proc.info
266                     processes.append({
267                         'pid': info.get('pid'),
268                         'name': info.get('name') or '',
269                         'path': info.get('exe') or '',
270                         'cmdline': ' '.join(info.get('cmdline'
) or []) if info.get('cmdline') else '',
271                         'username': info.get('username') or ''
272                         ,
273                         'create_time': info.get('create_time')
or 0.0,
274                     })
275                 except (psutil.NoSuchProcess, psutil.AccessDen
ied):
276                     continue
277             except Exception as ex:
278                 logger.debug(f'Ошибка при получении списка процесс

```



```

    ов: {ex}')
278         return processes
279
280     def _get_current_network_connections(self) -> List[Dict[str, Any]]:
281         """Получить текущие активные сетевые подключения."""
282         connections = []
283         try:
284             import psutil
285             for conn in psutil.net_connections(kind='inet'):
286                 try:
287                     laddr = f'{conn.laddr.ip}:{conn.laddr.port}' if conn.laddr else ''
288                     raddr = f'{conn.raddr.ip}:{conn.raddr.port}' if conn.raddr else ''
289                     key = f'{conn.type}_{laddr}_{raddr}_{conn.pid}'
290                     connections.append({
291                         'key': key,
292                         'fd': conn.fd,
293                         'family': conn.family.name if hasattr(conn.family, 'name') else str(conn.family),
294                         'type': conn.type.name if hasattr(conn.type, 'name') else str(conn.type),
295                         'laddr': laddr,
296                         'raddr': raddr,
297                         'status': conn.status,
298                         'pid': conn.pid,
299                     })
300                 except Exception:
301                     continue
302             except Exception as ex:
303                 logger.debug(f'Ошибка при получении сетевых соединений: {ex}')
304         return connections
305
306     def _get_registry_keys(self) -> Dict[str, Any]:
307         """Получить контрольные ключи реестра автозагрузки и с

```

```

        ИСТЕМНЫХ ПОЛИТИК. """
308         keys = {}
309         if os.name != 'nt':
310             return keys
311         try:
312             import winreg
313             hives = [
314                 (winreg.HKEY_CURRENT_USER, r'Software\Microsof
315                 t\Windows\CurrentVersion\Run'),
316                 (winreg.HKEY_LOCAL_MACHINE, r'Software\Microso
317                 ft\Windows\CurrentVersion\Run'),
318                 (winreg.HKEY_LOCAL_MACHINE, r'Software\Microso
319                 ft\Windows\CurrentVersion\RunOnce'),
320             ]
321             for hive, subkey in hives:
322                 try:
323                     with winreg.OpenKey(hive, subkey) as k:
324                         idx = 0
325                         while True:
326                             try:
327                                 name, val, _ = winreg.EnumValu
328                                 e(k, idx)
329                                 key_path = f'{subkey}\\{name}'
330                                 keys[key_path] = str(val)
331                                 idx += 1
332                             except OSError:
333                                 break
334                         except OSError:
335                             continue
336                     except Exception as ex:
337                         logger.debug(f'Ошибка при чтении ключей реестра: {
338                         ex}')
339             return keys
340
341     def _get_file_hashes(self) -> Dict[str, str]:
342         """Получить отпечатки и размеры отслеживаемых файлов д
343         ля быстрого детекта изменений. """
344         hashes = {}

```

```

339         for root_path in self.monitored_paths:
340             p = Path(root_path)
341             if not p.exists():
342                 continue
343             try:
344                 # Ограничиваем сканирование первыми 100 файлам
и для предотвращения перегрузки
345                 count = 0
346                 for item in p.rglob('*'):
347                     if count > 100:
348                         break
349                     if item.is_file():
350                         try:
351                             stat = item.stat()
352                             hashes[str(item)] = f'{stat.st_siz
e}_{stat.st_mtime}'
353                             count += 1
354                         except OSError:
355                             continue
356                     except Exception as ex:
357                         logger.debug(f'Ошибка при проверке файлов в {r
oot_path}: {ex}')
358                 return hashes
359
360     def get_status(self) -> Dict[str, Any]:
361         """Получить диагностический статус сборщика.
362
363         Returns:
364             Dict[str, Any]: Словарь со статусом и счетчиками.
365         """
366         status = W64CollectorStatus(
367             running=self._running,
368             events_count=self._event_counter,
369             last_event_time=self._last_event_time,
370         )
371         return status.model_dump()
372
373     def get_events(self, event_type: Optional[str] = None, lim

```

```

        it: int = 100) -> List[Dict[str, Any]]:
374         """Получить список последних событий из SQLite хранилища телеметрии.
375
376         Args:
377             event_type: Фильтр по типу события.
378             limit: Максимальное число записей.
379
380         Returns:
381             List[Dict[str, Any]]: Список событий.
382         """
383         if self.storage:
384             return self.storage.get_w64_events(event_type=event_type, provider='w64_collector', limit=limit)
385         return []
386
387
388 _global_w64_collector: Optional[AIW64Collector] = None
389 _lock = threading.Lock()
390
391 def get_w64_collector(**kwargs: Any) -> AIW64Collector:
392     """Получить глобальный синглтон сборщика AIW64Collector.
393
394     Args:
395         **kwargs: Аргументы инициализации при первом создании.
396
397     Returns:
398         AIW64Collector: Синглтон сборщика.
399     """
400     global _global_w64_collector
401     with _lock:
402         if _global_w64_collector is None:
403             _global_w64_collector = AIW64Collector(**kwargs)
404         return _global_w64_collector
405
406 def start_w64_collector(**kwargs: Any) -> bool:
407     """Запустить глобальный сборщик AIW64Collector."""
408     collector = get_w64_collector(**kwargs)

```

```
409 |     return collector.start()
410 |
411 | def stop_w64_collector() -> bool:
412 |     """Остановить глобальный сборщик AIW64Collector."""
413 |     global _global_w64_collector
414 |     with _lock:
415 |         if _global_w64_collector:
416 |             res = _global_w64_collector.stop()
417 |             _global_w64_collector = None
418 |             return res
419 |     return False
420 |
```

w64_etw_collector.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - W64 Etw
  Collector
4 # =====
5 # Description:
6 #   ETW Collector для сбора низкоуровневых событий ядра и журна
  лов аудита Windows.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.w64_etw_collector import AIW
  64ETWCollector
11 #
12 #       service = AIW64ETWCollector()
13 #
14 # File: w64_etw_collector.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-06 07:35:00
20 # =====
21
22 from __future__ import annotations
23 """ETW Collector для сбора низкоуровневых событий ядра и журна
  лов аудита Windows."""
24
25 import json
26 import os
27 import threading
28 import time
29 from collections import deque
```

```

30 from datetime import datetime, timezone
31 from pathlib import Path
32 from typing import Any, Callable, Dict, List, Optional
33 from logger import logger
34 from .models import ETWTraceEvent, ProcessLifecycleEvent, W64C
    collectorStatus
35 from .sqlite import TelemetryStorage
36 from .win32_fffi.wevtapi import WevtAPI
37
38
39 class AIW64ETWCollector:
40     """Сборщик низкоуровневых событий через ETW и журналы ауди
    та Windows."""
41
42     def __init__(
43         self,
44         enable_process_trace: bool = True,
45         enable_disk_trace: bool = True,
46         enable_network_trace: bool = True,
47         enable_registry_trace: bool = True,
48         poll_interval_sec: float = 5.0,
49         storage: Optional[TelemetryStorage] = None,
50         on_event_callback: Optional[Callable[[Dict[str, Any]],
    None]] = None,
51         **kwargs: Any,
52     ) -> None:
53         """Инициализация ETW сборщика телеметрии.
54
55         Args:
56             enable_process_trace: Трассировка создания/заверше
    ния процессов (ID 4688).
57             enable_disk_trace: Трассировка операций файлового
    аудита (ID 4663).
58             enable_network_trace: Трассировка сетевых подключе
    ний Windows Filtering Platform.
59             enable_registry_trace: Трассировка изменений ключе
    й аудита реестра.
60             poll_interval_sec: Интервал между выборками событи

```

й ETW (сек).

```
61         storage: Экземпляр хранилища TelemetryStorage.
62         on_event_callback: Callback-обработчик каждого зафиксированного события.
63         **kwargs: Дополнительные параметры конфигурации.
64         """
65         self.enable_process_trace = enable_process_trace
66         self.enable_disk_trace = enable_disk_trace
67         self.enable_network_trace = enable_network_trace
68         self.enable_registry_trace = enable_registry_trace
69         self.poll_interval = max(1.0, poll_interval_sec)
70         self.storage = storage or TelemetryStorage.get_instance()
71         self.on_event_callback = on_event_callback
72
73         self.wevt_api = WevtAPI() if os.name == 'nt' else None
74         self._seen_records: set[tuple[str, int]] = set()
75         self._seen_queue: deque[tuple[str, int]] = deque(maxlen=5000)
76
77         self._running = False
78         self._thread: Optional[threading.Thread] = None
79         self._stop_event = threading.Event()
80         self._event_counter = 0
81         self._last_event_time: Optional[str] = None
82         logger.debug('AIW64ETWCollector инициализирован (буферизация в оперативной памяти)')
83
84     def start(self) -> bool:
85         """Запустить фоновый поток сбора ETW событий.
86
87         Returns:
88             bool: True если сборщик успешно запущен, False если уже работал.
89         """
90         if self._running:
91             logger.warning('AIW64ETWCollector уже запущен')
92             return False
```



```

93
94     self._running = True
95     self._stop_event.clear()
96     self._thread = threading.Thread(target=self._etw_loop,
name='AIW64ETWWorker', daemon=True)
97     self._thread.start()
98     logger.info('AIW64ETWCollector успешно запущен')
99     return True
100
101     def stop(self) -> bool:
102         """Остановить фоновый поток ETW сбора.
103
104         Returns:
105             bool: True если сборщик остановлен, False если не
был активен.
106         """
107         if not self._running:
108             return False
109
110         self._running = False
111         self._stop_event.set()
112         if self._thread and self._thread.is_alive():
113             self._thread.join(timeout=5.0)
114         logger.info(f'AIW64ETWCollector остановлен. Всего зафи
ксировано событий: {self._event_counter}')
115         return True
116
117     def _etw_loop(self) -> None:
118         """Основной рабочий цикл выборки событий ETW."""
119         while not self._stop_event.is_set():
120             try:
121                 self._collect_etw_events()
122                 self._stop_event.wait(timeout=self.poll_interv
al)
123             except Exception as ex:
124                 logger.error(f'Ошибка в цикле ETW сборщика: {e
x}')
125

```

```

126     def _collect_etw_events(self) -> None:
127         """Собрать порцию событий из включенных провайдеров."""
128         "
129         if os.name != 'nt' or not self.wevt_api or not self.we
130         vt_api.is_available():
131             return
132
133         if self.enable_process_trace:
134             self._collect_process_events()
135         if self.enable_disk_trace:
136             self._collect_file_events()
137         if self.enable_network_trace:
138             self._collect_network_events()
139         if self.enable_registry_trace:
140             self._collect_registry_events()
141
142     def _collect_security_events(self, event_ids: int | list[i
143     nt], event_type: str, max_events: int = 50) -> None:
144         """Собрать порцию событий из Security журнала Windows
145         через WevtAPI.
146
147         Args:
148             event_ids: Идентификатор или список идентификаторо
149             в событий.
150             event_type: Название типа события для логирования.
151             max_events: Максимальное количество событий в выбо
152             рке.
153
154         """
155         if not self.wevt_api or not self.wevt_api.is_available
156         ():
157             return
158
159         try:
160             raw_events = self.wevt_api.read_events(
161                 channel='Security',
162                 limit=max_events,
163                 event_id=event_ids,
164             )

```

```

157         for ev in raw_events:
158             rec_id = ev.get('record_id')
159             if rec_id:
160                 key = ('Security', rec_id)
161                 if key in self._seen_records:
162                     continue
163                 if len(self._seen_queue) == self._seen_queue.
ue.maxlen:
164                     oldest = self._seen_queue[0]
165                     self._seen_records.discard(oldest)
166                     self._seen_queue.append(key)
167                     self._seen_records.add(key)
168
169                 event_data = ev.get('event_data', {})
170                 if event_data:
171                     self._log_event(event_type, event_data)
172             except Exception as ex:
173                 logger.debug(f'Ошибка сбора событий {event_type} E
TW: {ex}')
174
175     def _collect_process_events(self) -> None:
176         """Собрать события создания процессов (Security ID 468
8)."""
177         self._collect_security_events(4688, 'etw_process_creat
e')
178
179     def _collect_file_events(self) -> None:
180         """Собрать события файловых операций (Security ID 4663
)."""
181         self._collect_security_events(4663, 'etw_file_access')
182
183     def _collect_network_events(self) -> None:
184         """Собрать события сетевых блокировок/подключений WFP
(Security ID 5156/5158)."""
185         self._collect_security_events([5156, 5158], 'etw_netwo
rk_connection')
186
187     def _collect_registry_events(self) -> None:

```

```

188 |         """Собрать события аудита реестра (Security ID 4657)."""
189 |         self._collect_security_events(4657, 'etw_registry_access')
190 |
191 |     def _log_event(self, event_type: str, event_data: Dict[str, Any]) -> None:
192 |         """Зафиксировать событие ETW в оперативной памяти с пакетным сбросом в SQLite.
193 |
194 |         Args:
195 |             event_type: Тип события ETW.
196 |             event_data: Словарь с полями из журнала событий.
197 |         """
198 |         self._event_counter += 1
199 |         now_iso = datetime.now(timezone.utc).isoformat()
200 |
201 |         payload = {
202 |             'event_id': f'etw_{int(time.time())}_{self._event_counter}',
203 |             'timestamp': now_iso,
204 |             'event_type': event_type,
205 |             'provider': 'Microsoft-Windows-Security-Auditing',
206 |             'data': event_data,
207 |         }
208 |         self._last_event_time = now_iso
209 |
210 |         # 1. Сохранение в SQLite TelemetryStorage через буфер
211 |         # памяти
212 |         if self.storage:
213 |             try:
214 |                 db_record = {
215 |                     'event_id': payload['event_id'],
216 |                     'timestamp': now_iso,
217 |                     'event_type': event_type,
218 |                     'path': event_data.get('ObjectName') or event_data.get('NewProcessName'),
219 |                     'pid': event_data.get('ProcessId') or even

```

```

t_data.get('NewProcessId'),
219         'name': event_data.get('ProcessName') or e
vent_data.get('SubjectUserName'),
220         'details': event_data,
221     }
222     self.storage.save_w64_event(db_record, provide
r='w64_etw_collector')
223
224     # Специфическая фиксация создания процессов дл
я Process Provenance
225     if event_type == 'etw_process_create':
226         def _safe_pid(val: Any) -> int:
227             if val is None:
228                 return 0
229             if isinstance(val, int):
230                 return val
231             s = str(val).strip()
232             if s.startswith(('0x', '0X')):
233                 try:
234                     return int(s, 16)
235                 except ValueError:
236                     return 0
237             try:
238                 return int(s)
239             except ValueError:
240                 return 0
241
242         new_pid = _safe_pid(event_data.get('NewPro
cessId') or event_data.get('ProcessId'))
243         parent_pid = _safe_pid(event_data.get('Pro
cessId') or event_data.get('ParentProcessId'))
244         exe_path = event_data.get('NewProcessName'
) or event_data.get('ProcessName') or ''
245         proc_name = os.path.basename(exe_path) if
exe_path else 'unknown'
246         parent_exe = event_data.get('ParentProcess
Name') or ''
247         parent_name = os.path.basename(parent_exe)

```

```

    if parent_exe else None
248         u_name = event_data.get('SubjectUserName')
249         d_name = event_data.get('SubjectDomainName
    ')
250         full_user = f'{d_name}\\{u_name}' if d_name and u_name and d_name != '-' else u_name
251         u_sid = event_data.get('SubjectUserSid')
252         cmdline = event_data.get('CommandLine')
253         integrity = event_data.get('MandatoryLabel
    ')
254
255         prov_event = ProcessLifecycleEvent(
256             event_type='ProcessCreated',
257             process_guid=f'proc_{new_pid}_{int(time.time())}',
258             pid=new_pid,
259             name=proc_name,
260             executable_path=exe_path,
261             command_line=cmdline,
262             user=full_user,
263             sid=u_sid,
264             integrity_level=integrity,
265             parent_pid=parent_pid,
266             parent_name=parent_name,
267             created_at=now_iso,
268             details=event_data,
269         )
270         self.storage.save_process_provenance_event
    (prov_event)
271     except Exception as ex:
272         logger.debug(f'Ошибка сохранения ETW события в
    БД: {ex}')
273
274     # 2. Передача в callback
275     if self.on_event_callback:
276         try:
277             self.on_event_callback(payload)
278         except Exception as ex:

```

```

279         logger.debug(f'Ошибка в ETW on_event_callback:
    {ex}')
280
281     def get_status(self) -> Dict[str, Any]:
282         """Получить диагностический статус ETW сборщика.
283
284         Returns:
285             Dict[str, Any]: Словарь со статусом и счетчиками.
286         """
287         status = W64CollectorStatus(
288             running=self._running,
289             events_count=self._event_counter,
290             last_event_time=self._last_event_time,
291         )
292         return status.model_dump()
293
294     def get_events(self, event_type: Optional[str] = None, limit: int = 100) -> List[Dict[str, Any]]:
295         """Получить список последних событий ETW из SQLite хранилища.
296
297         Args:
298             event_type: Фильтр по типу события.
299             limit: Максимальное число записей.
300
301         Returns:
302             List[Dict[str, Any]]: Список событий.
303         """
304         if self.storage:
305             return self.storage.get_w64_events(event_type=event_type, provider='w64_etw_collector', limit=limit)
306         return []
307

```

advapi32.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry Win32_Ffi
  - Advapi32
4 # =====
5 # Description:
6 #   advapi32.dll wrapper for registry, services, and security
  functions.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.win32_ffadvapi32 import SY
  STEMTIME
11 #
12 #       service = SYSTEMTIME()
13 #
14 # File: advapi32.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry.win32_ffadvapi32
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-01 13:28:28
20 # =====
21
22 """advapi32.dll wrapper for registry, services, and security f
  unctions."""
23
24 import ctypes
25 import ctypes.wintypes as wintypes
26 from typing import List, Optional, Dict, Any
27 from dataclasses import dataclass
28 from enum import IntFlag
29 HKEY_CLASSES_ROOT = 2147483648
```



```

30 HKEY_CURRENT_USER = 2147483649
31 HKEY_LOCAL_MACHINE = 2147483650
32 HKEY_USERS = 2147483651
33 HKEY_PERFORMANCE_DATA = 2147483652
34 HKEY_CURRENT_CONFIG = 2147483653
35 KEY_READ = 131097
36 KEY_WRITE = 131078
37 KEY_QUERY_VALUE = 1
38 KEY_ENUMERATE_SUB_KEYS = 8
39 SERVICE_QUERY_STATUS = 4
40 SERVICE_QUERY_CONFIG = 1
41 SERVICE_AUTO_START = 2
42 SERVICE_DEMAND_START = 3
43 SERVICE_DISABLED = 4
44 SERVICE_RUNNING = 4
45 SERVICE_STOPPED = 1
46
47 class SYSTEMTIME(ctypes.Structure):
48     _fields_ = [('wYear', wintypes.WORD), ('wMonth', wintypes.
WORD), ('wDayOfWeek', wintypes.WORD), ('wDay', wintypes.WORD),
    ('wHour', wintypes.WORD), ('wMinute', wintypes.WORD), ('wSeco
nd', wintypes.WORD), ('wMilliseconds', wintypes.WORD)]
49
50 class SERVICE_STATUS(ctypes.Structure):
51     _fields_ = [('dwServiceType', wintypes.DWORD), ('dwCurrent
State', wintypes.DWORD), ('dwControlsAccepted', wintypes.DWORD
), ('dwWin32ExitCode', wintypes.DWORD), ('dwServiceSpecificExi
tCode', wintypes.DWORD), ('dwCheckPoint', wintypes.DWORD), ('d
wWaitHint', wintypes.DWORD)]
52
53 class ENUM_SERVICE_STATUS(ctypes.Structure):
54     _fields_ = [('lpServiceName', wintypes.LPWSTR), ('lpDispla
yName', wintypes.LPWSTR), ('ServiceStatus', SERVICE_STATUS)]
55
56 @dataclass
57 class RegistryValue:
58     """Registry value information."""
59     name: str

```

```

60     value_type: int
61     data: Any
62
63 @dataclass
64 class ServiceInfo:
65     """Service information."""
66     name: str
67     display_name: str
68     state: str
69     service_type: str
70     start_type: str
71
72 class Advapi32API:
73     """Wrapper for advapi32 registry and service functions."""
74
75     def __init__(self):
76         self.advapi32 = ctypes.windll.advapi32
77         self._setup_function_signatures()
78
79     def _setup_function_signatures(self):
80         """Configure ctypes function signatures."""
81         self.advapi32.RegOpenKeyExW.argtypes = [wintypes.HKEY,
82         wintypes.LPCWSTR, wintypes.DWORD, wintypes.DWORD, ctypes.POIN
83         TER(wintypes.HKEY)]
84         self.advapi32.RegOpenKeyExW.restype = wintypes.LONG
85         self.advapi32.RegQueryValueExW.argtypes = [wintypes.HK
86         EY, wintypes.LPCWSTR, wintypes.LPVOID, ctypes.POINTER(wintypes
87         .DWORD), wintypes.LPVOID, ctypes.POINTER(wintypes.DWORD)]
88         self.advapi32.RegQueryValueExW.restype = wintypes.LONG
89         self.advapi32.RegEnumKeyExW.argtypes = [wintypes.HKEY,
90         wintypes.DWORD, wintypes.LPWSTR, ctypes.POINTER(wintypes.DWOR
91         D), ctypes.POINTER(wintypes.DWORD), wintypes.LPWSTR, ctypes.PO
92         INTER(wintypes.DWORD), ctypes.POINTER(SYSTEMTIME)]
93         self.advapi32.RegEnumKeyExW.restype = wintypes.LONG
94         self.advapi32.RegCloseKey.argtypes = [wintypes.HKEY]
95         self.advapi32.RegCloseKey.restype = wintypes.LONG
96         self.advapi32.OpenSCManagerW.argtypes = [wintypes.LPCW
97         STR, wintypes.LPCWSTR, wintypes.DWORD]

```

```

90         self.advapi32.OpenSCManagerW.restype = wintypes.SC_HAN
DLE
91         self.advapi32.OpenServiceW.argtypes = [wintypes.SC_HAN
DLE, wintypes.LPCWSTR, wintypes.DWORD]
92         self.advapi32.OpenServiceW.restype = wintypes.SC_HAN
DLE
93         self.advapi32.QueryServiceStatus.argtypes = [wintypes.
SC_HANDLE, ctypes.POINTER(SERVICE_STATUS)]
94         self.advapi32.QueryServiceStatus.restype = wintypes.BO
OL
95         self.advapi32.EnumServicesStatusW.argtypes = [wintypes
.SC_HANDLE, wintypes.DWORD, wintypes.DWORD, ctypes.POINTER(ENU
M_SERVICE_STATUS), wintypes.DWORD, ctypes.POINTER(wintypes.DWO
RD), ctypes.POINTER(wintypes.DWORD), wintypes.LPVOID]
96         self.advapi32.EnumServicesStatusW.restype = wintypes.B
OOL
97         self.advapi32.CloseServiceHandle.argtypes = [wintypes.
SC_HANDLE]
98         self.advapi32.CloseServiceHandle.restype = wintypes.BO
OL
99
100     def read_registry_value(self, hive: int, path: str, value_
name: str) -> Optional[Any]:
101         """
102         Read a registry value.
103
104         Args:
105             hive: Registry hive (HKEY_*)
106             path: Registry path
107             value_name: Value name
108
109         Returns:
110             Registry value or None
111         """
112         try:
113             key = wintypes.HKEY()
114             result = self.advapi32.RegOpenKeyExW(hive, path, 0
, KEY_READ, ctypes.byref(key))

```

```

115         if result != 0:
116             return None
117         try:
118             value_type = wintypes.DWORD()
119             data_size = wintypes.DWORD(4096)
120             data = ctypes.create_unicode_buffer(4096)
121             result = self.advapi32.RegQueryValueExW(key, v
alue_name, None, ctypes.byref(value_type), data, ctypes.byref(
data_size))
122             if result == 0:
123                 return RegistryValue(name=value_name, valu
e_type=value_type.value, data=data.value)
124             return None
125         finally:
126             self.advapi32.RegCloseKey(key)
127     except Exception:
128         return None
129
130     def enumerate_registry_values(self, hive: int, path: str)
-> List[RegistryValue]:
131         """
132         Enumerate all values in a registry key.
133
134         Args:
135             hive: Registry hive
136             path: Registry path
137
138         Returns:
139             List of RegistryValue objects
140         """
141         values = []
142         try:
143             key = wintypes.HKEY()
144             result = self.advapi32.RegOpenKeyExW(hive, path, 0
, KEY_READ, ctypes.byref(key))
145             if result != 0:
146                 return values
147             try:

```

```

148         index = 0
149         while True:
150             value_name_buffer = ctypes.create_unicode_
buffer(256)
151             value_name_size = wintypes.DWORD(len(value
_name_buffer))
152             value_type = wintypes.DWORD()
153             data_size = wintypes.DWORD(4096)
154             data = ctypes.create_unicode_buffer(4096)
155             result = self.advapi32.RegEnumKeyExW(key,
index, value_name_buffer, ctypes.byref(value_name_size), None,
None, None, None)
156             if result != 0:
157                 break
158             values.append(RegistryValue(name=value_nam
e_buffer.value, value_type=value_type.value, data=data.value))
159             index += 1
160         finally:
161             self.advapi32.RegCloseKey(key)
162     except Exception:
163         pass
164     return values
165
166     def enumerate_services(self) -> List[ServiceInfo]:
167         """
168         Enumerate all services.
169
170         Returns:
171             List of ServiceInfo objects
172         """
173         services = []
174         try:
175             scm_handle = self.advapi32.OpenSCManagerW(None, No
ne, 1)
176             if not scm_handle:
177                 return services
178             try:
179                 max_services = 256

```

```

180         services_buffer = (ENUM_SERVICE_STATUS * max_s
| services)()
181         bytes_needed = wintypes.DWORD()
182         services_returned = wintypes.DWORD()
183         result = self.advapi32.EnumServicesStatusW(scm
| _handle, 48, 4, services_buffer, ctypes.sizeof(services_buffer
| ), ctypes.byref(bytes_needed), ctypes.byref(services_returned)
| , None)
184         if result:
185             for i in range(services_returned.value):
186                 svc = services_buffer[i]
187                 service_info = ServiceInfo(name=svc.lp
| ServiceName if svc.lpServiceName else 'Unknown', display_name=
| svc.lpDisplayName if svc.lpDisplayName else 'Unknown', state='
| Running' if svc.ServiceStatus.dwCurrentState == SERVICE_RUNNIN
| G else 'Stopped', service_type='Service' if svc.ServiceStatus.
| dwServiceType == 16 else 'Driver', start_type='Auto Start' if
| svc.ServiceStatus.dwCurrentState == SERVICE_AUTO_START else 'M
| anual')
188                 services.append(service_info)
189         finally:
190             self.advapi32.CloseServiceHandle(scm_handle)
191     except Exception:
192         pass
193     return services

```

error_decoder.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry Win32_Ffi
4 # - Error Decoder
5 # =====
6 # Description:
7 #   Модуль нативного декодирования системных кодов возврата Win32, HRESULT,
8 #
9 # Usage Examples:
10 #   Python API:
11 #       from apps.windows.telemetry.win32_fffi.error_decoder import WindowsErrorDecoder
12 #
13 #       service = WindowsErrorDecoder()
14 #
15 # File: error_decoder.py
16 # Project: ai-breadboard
17 # Package: apps.windows.telemetry.win32_fffi
18 # Author: hypo69
19 # Copyright: © 2026 hypo69
20 # Updated: 2026-10-01 13:28:28
21 # =====
22
23 from __future__ import annotations
24
25 """Модуль нативного декодирования системных кодов возврата Win32, HRESULT, """
26
27 import ctypes
28 import ctypes.wintypes as wintypes
29 import os
30 import sys
31 from typing import Any, Dict, Optional, Tuple, Union
```

```
30
31 try:
32     from logger import logger
33 except ImportError:
34     import logging
35     logger = logging.getLogger(__name__)
36
37 # Флаги для FormatMessageW
38 FORMAT_MESSAGE_ALLOCATE_BUFFER = 0x00000100
39 FORMAT_MESSAGE_IGNORE_INSERTS = 0x00000200
40 FORMAT_MESSAGE_FROM_SYSTEM = 0x00001000
41 FORMAT_MESSAGE_FROM_HMODULE = 0x00000800
42
43 # Известные стандартные Stop-коды ядра Windows (BugCheck Codes
44 )
45 BUGCHECK_CODES_MAP: Dict[int, Tuple[str, str]] = {
46     0x0000000A: ('IRQL_NOT_LESS_OR_EQUAL', 'Процесс или драйве
47     р попытался обратиться к адресу памяти при недопустимо высоком
48     уровне IRQL. '),
49     0x0000001E: ('KMODE_EXCEPTION_NOT_HANDLED', 'Программа ядр
50     а или драйвер сгенерировали исключение, которое не было перехв
51     ачено обработчиком. '),
52     0x00000024: ('NTFS_FILE_SYSTEM', 'Критический сбой или пов
53     реждение в системном драйвере файловой системы ntfs.sys. '),
54     0x0000003B: ('SYSTEM_SERVICE_EXCEPTION', 'Исключение при в
55     ыполнении подпрограммы системной службы Windows. '),
56     0x00000050: ('PAGE_FAULT_IN_NONPAGED_AREA', 'Запрос недопу
57     стимой системной памяти. Ошибка вызвана некорректным адресом и
58     ли сбоем RAM. '),
59     0x0000007E: ('SYSTEM_THREAD_EXCEPTION_NOT_HANDLED', 'Систе
60     мный поток сгенерировал неперехваченное исключение (часто связ
61     ано с несовместимым драйвером). '),
62     0x0000007F: ('UNEXPECTED_KERNEL_MODE_TRAP', 'Аппаратная ло
63     вушка процессора (деление на ноль, повреждение стека или перег
64     рузка оборудования). '),
65     0x0000009F: ('DRIVER_POWER_STATE_FAILURE', 'Драйвер находи
66     тся в несогласованном или недействительном состоянии управлени
67     я электропитанием (сна/пробуждения). '),
```


53 0x000000BE: ('ATTEMPTED_WRITE_TO_READONLY_MEMORY', 'Попытка
а драйвера выполнить запись в сегмент памяти, доступный только
для чтения.'),

54 0x000000C2: ('BAD_POOL_CALLER', 'Текущий поток выполнил не
корректный запрос к пулу системной памяти ядра.'),

55 0x000000D1: ('DRIVER_IRQL_NOT_LESS_OR_EQUAL', 'Драйвер поп
ытался обратиться к выгружаемой памяти на повышенном уровне пр
ерываний IRQL.'),

56 0x000000EF: ('CRITICAL_PROCESS_DIED', 'Критически важный с
истемный процесс (csrss.exe, wininit.exe и др.) неожиданно зав
ершил свою работу.'),

57 0x00000109: ('CRITICAL_STRUCTURE_CORRUPTION', 'Ядро обнару
жило повреждение критических структур данных или кода ядра (Pa
tchGuard).'),

58 0x00000116: ('VIDEO_TDR_FAILURE', 'Сбой видеодрайвера: поп
ытка сброса зависшего графического драйвера завершилась тайм-а
утом.'),

59 0x00000117: ('VIDEO_TDR_TIMEOUT_DETECTED', 'Графический др
айвер не ответил вовремя в пределах интервала обнаружения тайм
-аута TDR.'),

60 0x00000124: ('WHEA_UNCORRECTABLE_ERROR', 'Критическая неус
траняемая аппаратная ошибка архитектуры Windows Hardware Error
Architecture (CPU/RAM/PCIe).'),

61 0x00000133: ('DPC_WATCHDOG_VIOLATION', 'Сторожевой таймер
DPC зафиксировал зависание отложенного вызова процедуры драйве
ром.'),

62 0x00000139: ('KERNEL_SECURITY_CHECK_FAILURE', 'Ядро обнару
жило повреждение критической структуры данных безопасности (Bu
ffer Overflow / Stack Cookie).'),

63 0x00000154: ('UNEXPECTED_STORE_EXCEPTION', 'Сбой подсистем
ы хранения страниц памяти ядра (компрессия памяти / повреждени
е накопителя).'),

64 0x00000192: ('KERNEL_AUTO_BOOST_LOCK_ACQUISITION_WITH_RAIS
ED_IRQL', 'Попытка захвата блокировки AutoBoost ядра на повыше
нном уровне IRQL.'),

65 0x000001A7: ('STORE_DATA_STRUCTURE_CORRUPTION', 'Поврежден
ие внутренней структуры данных диспетчера хранилища страниц па
мяти.'),

```

66     0xC000021A: ('STATUS_SYSTEM_PROCESS_TERMINATED', 'Подсисте
ма пользовательского режима (WinLogon или Csrss) аварийно заве
ршилась с нарушением безопасности. '),
67     0xC0000221: ('STATUS_IMAGE_CHECKSUM_MISMATCH', 'Контрольна
я сумма системного драйвера или DLL не совпадает с заголовком
файла. '),
68 }
69
70
71 class WindowsErrorDecoder:
72     """Класс нативного декодирования ошибок Windows через Form
atMessageW."""
73
74     def __init__(self) -> None:
75         """Инициализация дескрипторов библиотек."""
76         self._kernel32 = getattr(ctypes.windll, 'kernel32', No
ne) if os.name == 'nt' else None
77         self._ntdll_handle: Optional[int] = None
78         if self._kernel32 and os.name == 'nt':
79             try:
80                 self._kernel32.GetModuleHandleW.argtypes = [wi
ntypes.LPCWSTR]
81                 self._kernel32.GetModuleHandleW.restype = wint
ypes.HMODULE
82                 self._ntdll_handle = self._kernel32.GetModuleH
andleW('ntdll.dll')
83             except Exception as ex:
84                 logger.debug(f'Не удалось получить дескриптор
ntdll.dll: {ex}')
85
86         def parse_error_code(self, code: Union[int, str]) -> Tuple
[int, str]:
87             """Преобразовать входной код ошибки (число, hex-строку
или отрицательный int) в uint32.
88
89             Args:
90                 code: Числовое или строковое представление кода ош
ибки.

```

```

91 |
92 |     Returns:
93 |         Tuple[int, str]: (нормализованный uint32 код, форм
    | атированная hex-строка 0XXXXXXXXX).
94 |     """
95 |     if isinstance(code, str):
96 |         c_str = code.strip()
97 |         try:
98 |             if c_str.lower().startswith('0x'):
99 |                 val = int(c_str, 16)
100 |             else:
101 |                 val = int(c_str, 10)
102 |         except ValueError:
103 |             return 0, '0x00000000'
104 |     else:
105 |         val = int(code)
106 |
107 |     # Преобразуем отрицательные 32-битные числа (например
    | из ctypes signed HRESULT) в беззнаковый uint32
108 |     uint_val = val & 0xFFFFFFFF
109 |     return uint_val, f'0x{uint_val:08X}'
110 |
111 |     def format_system_error(self, code: Union[int, str], lang_
    | id: int = 0) -> str:
112 |         """Получить локализованное системное описание ошибки и
    | з Windows API (Win32 / HRESULT / NTSTATUS).
113 |
114 |     Args:
115 |         code: Код ошибки (например, 5, 0x80070005, 0xC000
    | 005, '0x80240020').
116 |         lang_id: Идентификатор языка (0 - язык текущего по
    | тока/системы).
117 |
118 |     Returns:
119 |         str: Локализованное текстовое описание ошибки или
    | пустая строка, если код не распознан.
120 |     """
121 |     if not self._kernel32 or os.name != 'nt':

```

```

122         return ''
123
124     uint_code, _ = self.parse_error_code(code)
125     if uint_code == 0:
126         return 'Операция успешно завершена.'
127
128     # 1. Попытка чтения из системного репозитория (FORMAT_
MESSAGE_FROM_SYSTEM для Win32 и HRESULT)
129     buf = wintypes.LPWSTR()
130     flags = FORMAT_MESSAGE_ALLOCATE_BUFFER | FORMAT_MESSAG
E_FROM_SYSTEM | FORMAT_MESSAGE_IGNORE_INSERTS
131     chars = self._kernel32.FormatMessageW(
132         flags,
133         None,
134         wintypes.DWORD(uint_code),
135         wintypes.DWORD(lang_id),
136         ctypes.byref(buf),
137         0,
138         None,
139     )
140
141     if chars > 0 and buf.value:
142         msg = buf.value.strip().rstrip('\r\n.')
143         if self._kernel32.LocalFree:
144             self._kernel32.LocalFree(buf)
145         return msg
146
147     # 2. Если код лежит в диапазоне NTSTATUS (0xC0000000..
0xDFFFFFFF или 0x40000000..0x7FFFFFFF)
148     # запрашиваем системные сообщения из ntdll.dll
149     if self._ntdll_handle:
150         buf_nt = wintypes.LPWSTR()
151         flags_nt = FORMAT_MESSAGE_ALLOCATE_BUFFER | FORMAT
_MESSAGE_FROM_HMODULE | FORMAT_MESSAGE_IGNORE_INSERTS
152         chars_nt = self._kernel32.FormatMessageW(
153             flags_nt,
154             wintypes.HMODULE(self._ntdll_handle),
155             wintypes.DWORD(uint_code),

```

```

156         wintypes.DWORD(lang_id),
157         ctypes.byref(buf_nt),
158         0,
159         None,
160     )
161     if chars_nt > 0 and buf_nt.value:
162         msg_nt = buf_nt.value.strip().rstrip('\r\n.')
163         if self._kernel32.LocalFree:
164             self._kernel32.LocalFree(buf_nt)
165         return msg_nt
166
167     # 3. Если это HRESULT вида 0x8007XXXX (Win32 Facility
168     7), пробуем распаковать Win32 код ошибки
169     if (uint_code & 0xFFFF0000) == 0x80070000:
170         win32_subcode = uint_code & 0xFFFF
171         return self.format_system_error(win32_subcode, lan
172         g_id=lang_id)
173
174     return ''
175
176     def decode_bugcheck_code(self, code: Union[int, str]) -> D
177     ict[str, str]:
178         """Расшифровать Stop-код синего экрана (BSOD BugCheck
179         Code).
180
181         Args:
182             code: Числовой или строковый код BugCheck (наприме
183             п, 209, 0xD1, '0x000000D1').
184
185         Returns:
186             Dict[str, str]: Словарь с полями 'code_hex', 'symb
187             ol', 'description_ru'.
188
189         """
190         uint_code, hex_str = self.parse_error_code(code)
191         symbol = 'UNKNOWN_BUGCHECK'
192         desc = 'Критический сбой ядра Windows.'
193
194         if uint_code in BUGCHECK_CODES_MAP:

```

```
188         symbol, desc = BUGCHECK_CODES_MAP[uint_code]
189     else:
190         # Попробуем запросить описание через ntdll
191         nt_desc = self.format_system_error(uint_code)
192         if nt_desc:
193             desc = nt_desc
194
195     return {
196         'code_hex': hex_str,
197         'code_dec': str(uint_code),
198         'symbol': symbol,
199         'description_ru': desc,
200     }
201
202
203 # Глобальный синглтон декодера
204 _decoder_instance: Optional[WindowsErrorDecoder] = None
205
206
207 def get_error_decoder() -> WindowsErrorDecoder:
208     """Получить глобальный экземпляр WindowsErrorDecoder."""
209     global _decoder_instance
210     if _decoder_instance is None:
211         _decoder_instance = WindowsErrorDecoder()
212     return _decoder_instance
213
214
215 def format_system_error(code: Union[int, str], lang_id: int =
216     0) -> str:
217     """Удобный хелпер для форматирования системного кода ошибок
218     и."""
219     return get_error_decoder().format_system_error(code, lang_
220     id=lang_id)
221
222
223 def decode_bugcheck_code(code: Union[int, str]) -> Dict[str, s
224     tr]:
225     """Удобный хелпер для расшифровки Stop-кода ядра (BugCheck
```

```
    )."""
222     return get_error_decoder().decode_bugcheck_code(code)
223
224
225 __all__ = [
226     'WindowsErrorDecoder',
227     'get_error_decoder',
228     'format_system_error',
229     'decode_bugcheck_code',
230     'BUGCHECK_CODES_MAP',
231 ]
232
```

etw.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry Win32_Ffi
4 # - Etw
5 # Description:
6 #   Event Tracing for Windows (ETW) wrapper for real-time even
7 #   t monitoring.
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.win32_fffi.etw import GUID
11 #
12 #       service = GUID()
13 #
14 # File: etw.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry.win32_fffi
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-01 13:28:28
20 # =====
21
22 """Event Tracing for Windows (ETW) wrapper for real-time event
23   monitoring."""
24
25 import ctypes
26 import ctypes.wintypes as wintypes
27 from typing import Optional, List, Callable, Dict, Any
28 from dataclasses import dataclass
29 from enum import IntFlag
30 import threading
31
32 EVENT_TRACE_FLAG_PROCESS = 1
```



```

31 EVENT_TRACE_FLAG_THREAD = 2
32 EVENT_TRACE_FLAG_IMAGE_LOAD = 4
33 EVENT_TRACE_FLAG_DISK_IO = 256
34 EVENT_TRACE_FLAG_DISK_FILE_IO = 512
35 EVENT_TRACE_FLAG_MEMORY_PAGE_FAULTS = 4096
36 EVENT_TRACE_FLAG_MEMORY_HARD_FAULTS = 8192
37 EVENT_TRACE_FLAG_NETWORK_TCPIP = 65536
38 EVENT_TRACE_FLAG_REGISTRY = 131072
39 EVENT_TRACE_FLAG_FILE_IO = 262144
40 EVENT_TRACE_FLAG_FILE_IO_INIT = 524288
41 EVENT_TRACE_REAL_TIME_MODE = 256
42 EVENT_TRACE_FILE_MODE_SEQUENTIAL = 1
43 EVENT_TRACE_FILE_MODE_CIRCULAR = 2
44 EVENT_TRACE_USE_GLOBAL_SEQUENCE = 16384
45 WNODE_FLAG_TRACED_GUID = 131072
46 WNODE_FLAG_USE_MOF_PTR = 1048576
47 PEVENT_TRACE_BUFFER_CALLBACK = ctypes.CFUNCTYPE(wintypes.ULONG
    , wintypes.LPVOID)
48 PEVENT_CALLBACK = ctypes.CFUNCTYPE(None, wintypes.LPVOID)
49
50 class GUID(ctypes.Structure):
51     _fields_ = [('Data1', wintypes.DWORD), ('Data2', wintypes.
    WORD), ('Data3', wintypes.WORD), ('Data4', ctypes.c_ubyte * 8)
    ]
52
53 class WNODE_HEADER(ctypes.Structure):
54     _fields_ = [('BufferSize', wintypes.DWORD), ('ProviderId',
    wintypes.DWORD), ('ClientContext', ctypes.c_uint64), ('Timest
    amp', ctypes.c_uint64), ('Guid', GUID), ('KernelHandle', winty
    pes.HANDLE), ('Flags', wintypes.DWORD)]
55
56 class EVENT_TRACE_PROPERTIES(ctypes.Structure):
57     _fields_ = [('Wnode', WNODE_HEADER), ('BufferSize', wintyp
    es.DWORD), ('MinimumBuffers', wintypes.DWORD), ('MaximumBuffer
    s', wintypes.DWORD), ('MaximumFileSize', wintypes.DWORD), ('Lo
    gFileMode', wintypes.DWORD), ('FlushTimer', wintypes.DWORD), (
    'EnableFlags', wintypes.DWORD), ('AgeLimit', wintypes.LONG), (
    'NumberOfBuffers', wintypes.DWORD), ('FreeBuffers', wintypes.D

```

```

WORD), ('EventsLost', wintypes.DWORD), ('BuffersWritten', wintypes.DWORD), ('LogBuffersLost', wintypes.DWORD), ('RealTimeBuffersLost', wintypes.DWORD), ('LoggerThreadId', wintypes.HANDLE)]
58
59 class EVENT_TRACE_HEADER(ctypes.Structure):
60     _fields_ = [('Size', wintypes.USHORT), ('Reserved', wintypes.USHORT), ('Class', ctypes.c_ubyte), ('Type', ctypes.c_ubyte), ('Level', ctypes.c_ubyte), ('Version', ctypes.c_ubyte), ('Marker', wintypes.DWORD), ('ThreadId', wintypes.DWORD), ('ProcessId', wintypes.DWORD), ('TimeStamp', ctypes.c_uint64), ('ProviderId', GUID), ('ClientContext', wintypes.DWORD), ('Flags', wintypes.DWORD)]
61
62 @dataclass
63 class EtwEvent:
64     """ETW event information."""
65     timestamp: int
66     pid: int
67     tid: int
68     event_class: int
69     event_type: int
70     level: int
71     data: bytes
72
73 class EtwAPI:
74     """Wrapper for ETW event tracing functions."""
75
76     def __init__(self):
77         self.advapi32 = ctypes.windll.advapi32
78         self.ntdll = ctypes.windll.ntdll
79         self._setup_function_signatures()
80         self._session_handle = None
81         self._event_callbacks: Dict[int, List[Callable]] = {}
82
83     def _setup_function_signatures(self):
84         """Configure ctypes function signatures."""
85         self.advapi32.StartTraceW.argtypes = [ctypes.POINTER(w

```

```

intypes.HANDLE), wintypes.LPCWSTR, ctypes.POINTER(EVENT_TRACE_
PROPERTIES)])
86         self.advapi32.StartTraceW.restype = wintypes.ULONG
87         self.advapi32.StopTraceW.argtypes = [wintypes.HANDLE,
wintypes.LPCWSTR, ctypes.POINTER(EVENT_TRACE_PROPERTIES)]
88         self.advapi32.StopTraceW.restype = wintypes.ULONG
89         self.advapi32.EnableTrace.argtypes = [wintypes.ULONG,
wintypes.ULONG, wintypes.ULONG, ctypes.POINTER(GUID), wintypes
.HANDLE]
90         self.advapi32.EnableTrace.restype = wintypes.ULONG
91         self.advapi32.ControlTraceW.argtypes = [wintypes.HANDL
E, wintypes.LPCWSTR, ctypes.POINTER(EVENT_TRACE_PROPERTIES), w
intypes.DWORD]
92         self.advapi32.ControlTraceW.restype = wintypes.ULONG
93
94     def start_kernel_logger(self, enable_flags: int=EVENT_TRAC
E_FLAG_PROCESS) -> bool:
95         """
96         Start the Kernel Logger session for real-time event tr
acing.
97
98         Args:
99             enable_flags: Flags for events to capture
100
101         Returns:
102             True if successful
103         """
104         try:
105             properties = EVENT_TRACE_PROPERTIES()
106             properties.Wnode.BufferSize = ctypes.sizeof(EVENT_
TRACE_PROPERTIES)
107             properties.Wnode.Guid = GUID()
108             properties.Wnode.ClientContext = 1
109             properties.Wnode.Flags = WNODE_FLAG_TRACED_GUID
110             properties.BufferSize = 128
111             properties.MinimumBuffers = 4
112             properties.MaximumBuffers = 64
113             properties.LogFileMode = EVENT_TRACE_REAL_TIME_MOD

```

```

E | EVENT_TRACE_USE_GLOBAL_SEQUENCE
114 |         properties.FlushTimer = 100
115 |         properties.EnableFlags = enable_flags
116 |         properties.AgeLimit = 0
117 |         session_handle = wintypes.HANDLE()
118 |         status = self.advapi32.StartTraceW(ctypes.byref(session_handle), 'NT Kernel Logger', ctypes.byref(properties))
119 |         if status == 0:
120 |             self._session_handle = session_handle
121 |             return True
122 |         return False
123 |     except Exception:
124 |         return False
125 |
126 |     def stop_kernel_logger(self) -> bool:
127 |         """
128 |         Stop the Kernel Logger session.
129 |
130 |         Returns:
131 |             True if successful
132 |         """
133 |         try:
134 |             if not self._session_handle:
135 |                 return False
136 |             properties = EVENT_TRACE_PROPERTIES()
137 |             properties.Wnode.BufferSize = ctypes.sizeof(EVENT_TRACE_PROPERTIES)
138 |             status = self.advapi32.StopTraceW(self._session_handle, 'NT Kernel Logger', ctypes.byref(properties))
139 |             if status == 0:
140 |                 self._session_handle = None
141 |                 return True
142 |             return False
143 |         except Exception:
144 |             return False
145 |
146 |     def register_event_callback(self, event_type: int, callback: Callable) -> None:

```

```

147 |         """
148 |         Register a callback for a specific event type.
149 |
150 |         Args:
151 |             event_type: ETW event type
152 |             callback: Callback function to invoke
153 |         """
154 |         if event_type not in self._event_callbacks:
155 |             self._event_callbacks[event_type] = []
156 |         self._event_callbacks[event_type].append(callback)
157 |
158 |     def unregister_event_callback(self, event_type: int, callback: Callable) -> None:
159 |         """
160 |         Unregister a callback.
161 |
162 |         Args:
163 |             event_type: ETW event type
164 |             callback: Callback function to remove
165 |         """
166 |         if event_type in self._event_callbacks:
167 |             try:
168 |                 self._event_callbacks[event_type].remove(callback)
169 |             except ValueError:
170 |                 pass
171 |
172 |     def get_session_handle(self) -> Optional[wintypes.HANDLE]:
173 |         """Get the current session handle."""
174 |         return self._session_handle

```

kernel32.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry Win32_Ffi
  - Kernel32
4 # =====
5 # Description:
6 #   kernel32.dll wrapper for process enumeration and snapshot
  operations.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.win32_fffi.kernel32 import PR
  OCESSENTRY32
11 #
12 #       service = PROCESSIONT32()
13 #
14 # File: kernel32.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry.win32_fffi
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-01 13:28:28
20 # =====
21
22 """kernel32.dll wrapper for process enumeration and snapshot o
  perations."""
23
24 import ctypes
25 import ctypes.wintypes as wintypes
26 from typing import List, Optional, Tuple
27 from dataclasses import dataclass
28 from enum import IntFlag
29 TH32CS_SNAPPROCESS = 2
```

```

30 TH32CS_SNAPTHREAD = 4
31 TH32CS_SNAPMODULE = 8
32 TH32CS_SNAPMODULE32 = 16
33 ABOVE_NORMAL_PRIORITY_CLASS = 32768
34 BELOW_NORMAL_PRIORITY_CLASS = 16384
35 HIGH_PRIORITY_CLASS = 128
36 IDLE_PRIORITY_CLASS = 64
37 NORMAL_PRIORITY_CLASS = 32
38 REALTIME_PRIORITY_CLASS = 256
39
40 class PROCESSENTRY32(ctypes.Structure):
41     _fields_ = [('dwSize', wintypes.DWORD), ('th32ProcessID',
wintypes.DWORD), ('th32ParentProcessID', wintypes.DWORD), ('dw
Priority', wintypes.LONG), ('dwThreadCount', wintypes.DWORD),
('th32ModuleID', wintypes.DWORD), ('pcbUsage', wintypes.DWORD)
, ('th32AccessKey', wintypes.LPVOID), ('szExeFile', ctypes.c_c
har * 260)]
42
43 class THREADENTRY32(ctypes.Structure):
44     _fields_ = [('dwSize', wintypes.DWORD), ('th32ThreadID', w
intypes.DWORD), ('th32OwnerProcessID', wintypes.DWORD), ('tpBa
sePri', wintypes.LONG), ('tpDeltaPri', wintypes.LONG), ('dwFla
gs', wintypes.DWORD)]
45
46 class MODULEENTRY32(ctypes.Structure):
47     _fields_ = [('dwSize', wintypes.DWORD), ('th32ModuleID', w
intypes.DWORD), ('th32ProcessID', wintypes.DWORD), ('GlpvBase'
, wintypes.LPVOID), ('dwSize_', wintypes.DWORD), ('hModule', w
intypes.HMODULE), ('szModule', ctypes.c_char * 256), ('szExePa
th', ctypes.c_char * 260)]
48
49 class FILETIME(ctypes.Structure):
50     _fields_ = [('dwLowDateTime', wintypes.DWORD), ('dwHighDat
eTime', wintypes.DWORD)]
51
52 @dataclass
53 class ProcessInfo:
54     """Process snapshot information from Toolhelp32."""

```

```

55     pid: int
56     ppid: int
57     name: str
58     priority: int
59     thread_count: int
60
61 @dataclass
62 class ThreadInfo:
63     """Thread snapshot information from Toolhelp32."""
64     tid: int
65     pid: int
66     base_priority: int
67     delta_priority: int
68
69 @dataclass
70 class ModuleInfo:
71     """Module snapshot information from Toolhelp32."""
72     name: str
73     path: str
74     base_address: int
75     size: int
76
77 class Kernel32API:
78     """Wrapper for kernel32 process and thread enumeration fun
79     ctions."""
80
81     def __init__(self):
82         self.kernel32 = ctypes.windll.kernel32
83         self._setup_function_signatures()
84
85     def _setup_function_signatures(self):
86         """Configure ctypes function signatures for proper typ
87         e checking."""
88         self.kernel32.CreateToolhelp32Snapshot.argtypes = [win
89         types.DWORD, wintypes.DWORD]
90         self.kernel32.CreateToolhelp32Snapshot.restype = winty
91         pes.HANDLE
92         self.kernel32.Process32First.argtypes = [wintypes.HAND

```



```

    LE, ctypes.POINTER(PROCESSENTRY32)]
89         self.kernel32.Process32First.restype = wintypes.BOOL
90         self.kernel32.Process32Next.argtypes = [wintypes.HANDL
    E, ctypes.POINTER(PROCESSENTRY32)]
91         self.kernel32.Process32Next.restype = wintypes.BOOL
92         self.kernel32.Thread32First.argtypes = [wintypes.HANDL
    E, ctypes.POINTER(THREADENTRY32)]
93         self.kernel32.Thread32First.restype = wintypes.BOOL
94         self.kernel32.Thread32Next.argtypes = [wintypes.HANDLE
    , ctypes.POINTER(THREADENTRY32)]
95         self.kernel32.Thread32Next.restype = wintypes.BOOL
96         self.kernel32.Module32First.argtypes = [wintypes.HANDL
    E, ctypes.POINTER(MODULEENTRY32)]
97         self.kernel32.Module32First.restype = wintypes.BOOL
98         self.kernel32.Module32Next.argtypes = [wintypes.HANDLE
    , ctypes.POINTER(MODULEENTRY32)]
99         self.kernel32.Module32Next.restype = wintypes.BOOL
100        self.kernel32.CloseHandle.argtypes = [wintypes.HANDLE]
101        self.kernel32.CloseHandle.restype = wintypes.BOOL
102        self.kernel32.GetCurrentProcess.restype = wintypes.HAN
    DLE
103        self.kernel32.GetProcessTimes.argtypes = [wintypes.HAN
    DLE, ctypes.POINTER(FILETIME), ctypes.POINTER(FILETIME), ctype
    s.POINTER(FILETIME), ctypes.POINTER(FILETIME)]
104        self.kernel32.GetProcessTimes.restype = wintypes.BOOL
105        self.kernel32.GetPriorityClass.argtypes = [wintypes.HA
    NDLE]
106        self.kernel32.GetPriorityClass.restype = wintypes.DWOR
    D
107
108    def enumerate_processes(self) -> List[ProcessInfo]:
109        """
110        Enumerate all running processes using Toolhelp32.
111
112        Returns:
113            List of ProcessInfo objects
114        """
115        processes = []

```

```

116         snapshot = self.kernel32.CreateToolhelp32Snapshot(TH32
| CS_SNAPPROCESS, 0)
117         if snapshot == -1:
118             return processes
119         try:
120             pe = PROCESSENTRY32()
121             pe.dwSize = ctypes.sizeof(PROCESSENTRY32)
122             if self.kernel32.Process32First(snapshot, ctypes.b
| yref(pe)):
123                 while True:
124                     try:
125                         process = ProcessInfo(pid=pe.th32Proce
| ssID, ppid=pe.th32ParentProcessID, name=pe.szExeFile.decode('u
| tf-8', errors='ignore'), priority=pe.dwPriority, thread_count=
| pe.dwThreadCount)
126                         processes.append(process)
127                     except Exception:
128                         pass
129                     if not self.kernel32.Process32Next(snapsho
| t, ctypes.byref(pe)):
130                         break
131                 finally:
132                     self.kernel32.CloseHandle(snapshot)
133             return processes
134
135     def enumerate_threads(self, pid: Optional[int]=None) -> Li
| st[ThreadInfo]:
136         """
137         Enumerate threads for a process or all threads if pid
| is None.
138
139         Args:
140             pid: Process ID to filter threads, or None for all
| threads
141
142         Returns:
143             List of ThreadInfo objects
144         """

```

```

145 |         threads = []
146 |         snapshot = self.kernel32.CreateToolhelp32Snapshot(TH32
| CS_SNAPTHREAD, pid or 0)
147 |         if snapshot == -1:
148 |             return threads
149 |         try:
150 |             te = THREADENTRY32()
151 |             te.dwSize = ctypes.sizeof(THREADENTRY32)
152 |             if self.kernel32.Thread32First(snapshot, ctypes.by
| ref(te)):
153 |                 while True:
154 |                     try:
155 |                         if pid is None or te.th32OwnerProcessI
| D == pid:
156 |                             thread = ThreadInfo(tid=te.th32Thr
| eadID, pid=te.th32OwnerProcessID, base_priority=te.tpBasePri,
| delta_priority=te.tpDeltaPri)
157 |                             threads.append(thread)
158 |                         except Exception:
159 |                             pass
160 |                         if not self.kernel32.Thread32Next(snapshot
| , ctypes.byref(te)):
161 |                             break
162 |             finally:
163 |                 self.kernel32.CloseHandle(snapshot)
164 |             return threads
165 |
166 |     def enumerate_modules(self, pid: int) -> List[ModuleInfo]:
167 |         """
168 |         Enumerate modules (DLLs) loaded in a process.
169 |
170 |         Args:
171 |             pid: Process ID
172 |
173 |         Returns:
174 |             List of ModuleInfo objects
175 |         """
176 |         modules = []

```

```

177 |         snapshot = self.kernel32.CreateToolhelp32Snapshot(TH32
| CS_SNAPMODULE | TH32CS_SNAPMODULE32, pid)
178 |         if snapshot == -1:
179 |             return modules
180 |         try:
181 |             me = MODULEENTRY32()
182 |             me.dwSize = ctypes.sizeof(MODULEENTRY32)
183 |             if self.kernel32.Module32First(snapshot, ctypes.by
| ref(me)):
184 |                 while True:
185 |                     try:
186 |                         module = ModuleInfo(name=me.szModule.d
| ecode('utf-8', errors='ignore'), path=me.szExePath.decode('utf
| -8', errors='ignore'), base_address=int(me.GlpvBase), size=me.
| dwSize_)
187 |                         modules.append(module)
188 |                     except Exception:
189 |                         pass
190 |                     if not self.kernel32.Module32Next(snapshot
| , ctypes.byref(me)):
191 |                         break
192 |             finally:
193 |                 self.kernel32.CloseHandle(snapshot)
194 |             return modules
195 |
196 |     def get_process_priority_class(self, pid: int) -> Optional
| [str]:
197 |         """
198 |         Get the priority class of a process.
199 |
200 |         Args:
201 |             pid: Process ID
202 |
203 |         Returns:
204 |             Priority class name or None if error
205 |         """
206 |         try:
207 |             handle = ctypes.windll.kernel32.OpenProcess(1024,

```

```

False, pid)
208         if not handle:
209             return None
210         try:
211             priority = self.kernel32.GetPriorityClass(handle)
212             priority_map = {ABOVE_NORMAL_PRIORITY_CLASS: 'Above Normal', BELOW_NORMAL_PRIORITY_CLASS: 'Below Normal', HIGH_PRIORITY_CLASS: 'High', IDLE_PRIORITY_CLASS: 'Idle', NORMAL_PRIORITY_CLASS: 'Normal', REALTIME_PRIORITY_CLASS: 'Realtime'}
213             return priority_map.get(priority, f'Unknown ({priority})')
214         finally:
215             self.kernel32.CloseHandle(handle)
216     except Exception:
217         return None
218
219     def get_process_times(self, pid: int) -> Optional[Tuple[int, int]]:
220         """
221         Get process creation and exit times.
222
223         Args:
224             pid: Process ID
225
226         Returns:
227             Tuple of (creation_time_filetime, exit_time_filetime) or None
228         """
229         try:
230             handle = ctypes.windll.kernel32.OpenProcess(1024, False, pid)
231             if not handle:
232                 return None
233             try:
234                 creation_time = FILETIME()
235                 exit_time = FILETIME()

```

```
236 |         kernel_time = FILETIME()
237 |         user_time = FILETIME()
238 |         if self.kernel32.GetProcessTimes(handle, ctypes
|         s.byref(creation_time), ctypes.byref(exit_time), ctypes.byref(
|         kernel_time), ctypes.byref(user_time)):
239 |             return (creation_time.dwLowDateTime, exit_
|         time.dwLowDateTime)
240 |         return None
241 |     finally:
242 |         self.kernel32.CloseHandle(handle)
243 | except Exception:
244 |     return None
```

nethelper.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry Win32_Ffi
  # - Nethelper
4 # =====
5 # Description:
6 #   Низкоуровневый интерфейс к IP Helper API (iphlpapi.dll) Wi
  ndows.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.win32_ffnethelper import M
  IB_TCPROW_OWNER_PID
11 #
12 #       service = MIB_TCPROW_OWNER_PID()
13 #
14 # File: nethelper.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry.win32_ffnethelper
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-01 13:28:28
20 # =====
21
22 from __future__ import annotations
23 """Низкоуровневый интерфейс к IP Helper API (iphlpapi.dll) Win
  dows."""
24
25 import ctypes
26 import ctypes.wintypes as wintypes
27 import socket
28 import struct
29 from dataclasses import dataclass
```

```
30 from typing import Any, Dict, List, Optional
31 from logger import logger
32 AF_INET = 2
33 AF_INET6 = 23
34 TCP_TABLE_BASIC_LISTENER = 0
35 TCP_TABLE_BASIC_CONNECTIONS = 1
36 TCP_TABLE_BASIC_ALL = 2
37 TCP_TABLE_OWNER_PID_LISTENER = 3
38 TCP_TABLE_OWNER_PID_CONNECTIONS = 4
39 TCP_TABLE_OWNER_PID_ALL = 5
40 TCP_TABLE_OWNER_MODULE_LISTENER = 6
41 TCP_TABLE_OWNER_MODULE_CONNECTIONS = 7
42 TCP_TABLE_OWNER_MODULE_ALL = 8
43 UDP_TABLE_BASIC = 0
44 UDP_TABLE_OWNER_PID = 1
45 UDP_TABLE_OWNER_MODULE = 2
46 MIB_TCP_STATE_CLOSED = 1
47 MIB_TCP_STATE_LISTEN = 2
48 MIB_TCP_STATE_SYN_SENT = 3
49 MIB_TCP_STATE_SYN_RCVD = 4
50 MIB_TCP_STATE_ESTAB = 5
51 MIB_TCP_STATE_FIN_WAIT1 = 6
52 MIB_TCP_STATE_FIN_WAIT2 = 7
53 MIB_TCP_STATE_CLOSE_WAIT = 8
54 MIB_TCP_STATE_CLOSING = 9
55 MIB_TCP_STATE_LAST_ACK = 10
56 MIB_TCP_STATE_TIME_WAIT = 11
57 MIB_TCP_STATE_DELETE_TCB = 12
58 TCP_STATE_MAP = {MIB_TCP_STATE_CLOSED: 'CLOSED', MIB_TCP_STATE
    _LISTEN: 'LISTEN', MIB_TCP_STATE_SYN_SENT: 'SYN_SENT', MIB_TCP
    _STATE_SYN_RCVD: 'SYN_RCVD', MIB_TCP_STATE_ESTAB: 'ESTABLISHED
    ', MIB_TCP_STATE_FIN_WAIT1: 'FIN_WAIT1', MIB_TCP_STATE_FIN_WAI
    T2: 'FIN_WAIT2', MIB_TCP_STATE_CLOSE_WAIT: 'CLOSE_WAIT', MIB_T
    CP_STATE_CLOSING: 'CLOSING', MIB_TCP_STATE_LAST_ACK: 'LAST_ACK
    ', MIB_TCP_STATE_TIME_WAIT: 'TIME_WAIT', MIB_TCP_STATE_DELETE_
    TCB: 'DELETE_TCB'}
59 ERROR_INSUFFICIENT_BUFFER = 122
60
```



```
61 class MIB_TCPROW_OWNER_PID(ctypes.Structure):
62     """Структура записи TCP соединения с PID."""
63     _fields_ = [('dwState', wintypes.DWORD), ('dwLocalAddr', w
intypes.DWORD), ('dwLocalPort', wintypes.DWORD), ('dwRemoteAdd
r', wintypes.DWORD), ('dwRemotePort', wintypes.DWORD), ('dwOwn
ingPid', wintypes.DWORD)]
64
65 class MIB_UDPROW_OWNER_PID(ctypes.Structure):
66     """Структура записи UDP сокета с PID."""
67     _fields_ = [('dwLocalAddr', wintypes.DWORD), ('dwLocalPort
', wintypes.DWORD), ('dwOwningPid', wintypes.DWORD)]
68
69 class MIB_IF_ROW2(ctypes.Structure):
70     """Нативная структура MIB_IF_ROW2 из iphlpapi.dll."""
71     _fields_ = [
72         ('InterfaceLuid', ctypes.c_uint64),
73         ('InterfaceIndex', wintypes.DWORD),
74         ('InterfaceGuid', ctypes.c_byte * 16),
75         ('Alias', ctypes.c_wchar * 257),
76         ('Description', ctypes.c_wchar * 257),
77         ('PhysicalAddressLength', wintypes.DWORD),
78         ('PhysicalAddress', ctypes.c_byte * 32),
79         ('PermanentPhysicalAddress', ctypes.c_byte * 32),
80         ('Mtu', wintypes.DWORD),
81         ('Type', wintypes.DWORD),
82         ('TunnelType', ctypes.c_int),
83         ('MediaType', ctypes.c_int),
84         ('PhysicalMediumType', ctypes.c_int),
85         ('AccessType', ctypes.c_int),
86         ('DirectionType', ctypes.c_int),
87         ('InterfaceAndOperStatusFlags', ctypes.c_byte),
88         ('OperStatus', ctypes.c_int),
89         ('AdminStatus', ctypes.c_int),
90         ('MediaConnectState', ctypes.c_int),
91         ('NetworkGuid', ctypes.c_byte * 16),
92         ('ConnectionType', ctypes.c_int),
93         ('TransmitLinkSpeed', ctypes.c_uint64),
94         ('ReceiveLinkSpeed', ctypes.c_uint64),
```

```

95 |         ('InOctets', ctypes.c_uint64),
96 |         ('InUcastPkts', ctypes.c_uint64),
97 |         ('InNUcastPkts', ctypes.c_uint64),
98 |         ('InDiscards', ctypes.c_uint64),
99 |         ('InErrors', ctypes.c_uint64),
100 |         ('InUnknownProtos', ctypes.c_uint64),
101 |         ('InUcastOctets', ctypes.c_uint64),
102 |         ('InMulticastOctets', ctypes.c_uint64),
103 |         ('InBroadcastOctets', ctypes.c_uint64),
104 |         ('OutOctets', ctypes.c_uint64),
105 |         ('OutUcastPkts', ctypes.c_uint64),
106 |         ('OutNUcastPkts', ctypes.c_uint64),
107 |         ('OutDiscards', ctypes.c_uint64),
108 |         ('OutErrors', ctypes.c_uint64),
109 |         ('OutUcastOctets', ctypes.c_uint64),
110 |         ('OutMulticastOctets', ctypes.c_uint64),
111 |         ('OutBroadcastOctets', ctypes.c_uint64),
112 |         ('OutQLen', ctypes.c_uint64),
113 |     ]
114 |
115 | class MIB_IF_TABLE2(ctypes.Structure):
116 |     """Таблица сетевых интерфейсов MIB_IF_TABLE2."""
117 |     _fields_ = [
118 |         ('NumEntries', wintypes.DWORD),
119 |         ('Table', MIB_IF_ROW2 * 1),
120 |     ]
121 |
122 | class MIB_IPNETROW(ctypes.Structure):
123 |     """Структура записи таблицы ARP (MIB_IPNETROW)."""
124 |     _fields_ = [
125 |         ('dwIndex', wintypes.DWORD),
126 |         ('dwPhysAddrLen', wintypes.DWORD),
127 |         ('bPhysAddr', ctypes.c_ubyte * 8),
128 |         ('dwAddr', wintypes.DWORD),
129 |         ('dwType', wintypes.DWORD),
130 |     ]
131 |
132 | class MIB_IPNETTABLE(ctypes.Structure):

```

```
133     """Таблица записей ARP (MIB_IPNETTABLE)."""
134     _fields_ = [
135         ('dwNumEntries', wintypes.DWORD),
136         ('table', MIB_IPNETROW * 1),
137     ]
138
139 IP_NET_TYPE_MAP = {
140     1: 'OTHER',
141     2: 'INVALID',
142     3: 'DYNAMIC',
143     4: 'STATIC',
144 }
145
146 @dataclass
147 class NetworkSocketInfo:
148     """Нормализованные данные о сетевом соединении/сожете."""
149     protocol: str
150     local_address: str
151     local_port: int
152     remote_address: str
153     remote_port: int
154     state: str
155     pid: int
156
157     def to_dict(self) -> Dict[str, Any]:
158         """Преобразование в словарь."""
159         return {'protocol': self.protocol, 'local_address': self.local_address, 'local_port': self.local_port, 'remote_address': self.remote_address, 'remote_port': self.remote_port, 'state': self.state, 'pid': self.pid}
160
161 class IPHelperAPI:
162     """Нативный интерфейс к iphlpapi.dll."""
163
164     def __init__(self) -> None:
165         """Инициализация функций iphlpapi.dll."""
166         self._iphlpapi = ctypes.windll.iphlpapi
167         self._GetExtendedTcpTable = self._iphlpapi.GetExtended
```

```

    TcpTable
168         self._GetExtendedTcpTable.argtypes = [ctypes.c_void_p,
    ctypes.POINTER(wintypes.DWORD), wintypes.BOOL, wintypes.ULONG
    , ctypes.c_int, wintypes.ULONG]
169         self._GetExtendedTcpTable.restype = wintypes.DWORD
170         self._GetExtendedUdpTable = self._iphlpapi.GetExtended
    UdpTable
171         self._GetExtendedUdpTable.argtypes = [ctypes.c_void_p,
    ctypes.POINTER(wintypes.DWORD), wintypes.BOOL, wintypes.ULONG
    , ctypes.c_int, wintypes.ULONG]
172         self._GetExtendedUdpTable.restype = wintypes.DWORD
173
174         # SendARP
175         self._SendARP = getattr(self._iphlpapi, 'SendARP', Non
    e)
176         if self._SendARP:
177             self._SendARP.argtypes = [wintypes.ULONG, wintypes
    .ULONG, ctypes.c_void_p, ctypes.POINTER(wintypes.ULONG)]
178             self._SendARP.restype = wintypes.DWORD
179
180         # GetIpNetTable
181         self._GetIpNetTable = getattr(self._iphlpapi, 'GetIpNe
    tTable', None)
182         if self._GetIpNetTable:
183             self._GetIpNetTable.argtypes = [ctypes.c_void_p, c
    ctypes.POINTER(wintypes.ULONG), wintypes.BOOL]
184             self._GetIpNetTable.restype = wintypes.DWORD
185
186         if hasattr(self._iphlpapi, 'GetIfTable2'):
187             self._GetIfTable2 = self._iphlpapi.GetIfTable2
188             self._GetIfTable2.argtypes = [ctypes.POINTER(ctype
    s.c_void_p)]
189             self._GetIfTable2.restype = wintypes.DWORD
190             self._FreeMibTable = getattr(self._iphlpapi, 'Free
    MibTable', None)
191             if self._FreeMibTable:
192                 self._FreeMibTable.argtypes = [ctypes.c_void_p
    ]

```

```

193         else:
194             self._GetIfTable2 = None
195
196     @staticmethod
197     def _ip_to_str(ip_int: int) -> str:
198         """Преобразование сетевого порядка байт IP в строку."""
199
200         try:
201             return socket.inet_ntoa(struct.pack('<L', ip_int))
202         except Exception:
203             return '0.0.0.0'
204
205     @staticmethod
206     def _port_to_int(port_int: int) -> int:
207         """Преобразование сетевого порядка байт порта в int."""
208
209         return port_int >> 8 & 255 | (port_int & 255) << 8
210
211     def get_adapter_statistics(self) -> List[Dict[str, Any]]:
212         """Получение накопительной статистики по всем сетевым
213         адаптерам через GetIfTable2.
214
215         Returns:
216             List[Dict[str, Any]]: Список словарей со статистик
217             ой интерфейсов.
218         """
219         if not self._GetIfTable2:
220             return []
221         p_table = ctypes.c_void_p()
222         res = self._GetIfTable2(ctypes.byref(p_table))
223         if res != 0 or not p_table:
224             return []
225         try:
226             num_entries = ctypes.cast(p_table, ctypes.POINTER(
227                 wintypes.DWORD)).contents.value
228             offset = ctypes.sizeof(wintypes.DWORD)
229             row_size = ctypes.sizeof(MIB_IF_ROW2)
230             base_addr = p_table.value or 0

```

```

226         if not base_addr:
227             return []
228         stats: List[Dict[str, Any]] = []
229         for i in range(num_entries):
230             row_addr = base_addr + offset + i * row_size
231             row = MIB_IF_ROW2.from_address(row_addr)
232             name = row.Alias or row.Description or f'Inter
face_{row.InterfaceIndex}'
233             stats.append({
234                 'name': name,
235                 'alias': row.Alias,
236                 'description': row.Description,
237                 'interface_index': row.InterfaceIndex,
238                 'received_bytes': row.InOctets,
239                 'sent_bytes': row.OutOctets,
240                 'received_packets': row.InUcastPkts + row.
InNUcastPkts,
241                 'sent_packets': row.OutUcastPkts + row.Out
NUcastPkts,
242                 'received_discarded': row.InDiscards,
243                 'received_errors': row.InErrors,
244                 'outbound_discarded': row.OutDiscards,
245                 'outbound_errors': row.OutErrors,
246                 'transmit_speed_bps': row.TransmitLinkSpee
d,
247                 'receive_speed_bps': row.ReceiveLinkSpeed,
248                 'oper_status': row.OperStatus,
249             })
250         return stats
251     finally:
252         if hasattr(self, '_FreeMibTable') and self._FreeMi
bTable and p_table:
253             self._FreeMibTable(p_table)
254
255     def get_tcp_connections(self) -> List[NetworkSocketInfo]:
256         """Получение всех активных TCP соединений и слушающих
портов с PID.
257

```

```

258         Returns:
259             List[NetworkSocketInfo]: Список соединений.
260         """
261         size = wintypes.DWORD(0)
262         self._GetExtendedTcpTable(None, ctypes.byref(size), Tr
ue, AF_INET, TCP_TABLE_OWNER_PID_ALL, 0)
263         if size.value == 0:
264             return []
265         buf = (ctypes.c_byte * size.value)()
266         res = self._GetExtendedTcpTable(buf, ctypes.byref(size
), True, AF_INET, TCP_TABLE_OWNER_PID_ALL, 0)
267         if res != 0:
268             return []
269         num_entries = ctypes.cast(buf, ctypes.POINTER(wintypes
.DWORD)).contents.value
270         row_size = ctypes.sizeof(MIB_TCPROW_OWNER_PID)
271         offset = ctypes.sizeof(wintypes.DWORD)
272         connections: List[NetworkSocketInfo] = []
273         for i in range(num_entries):
274             row_ptr = ctypes.byref(buf, offset + i * row_size)
275             row = ctypes.cast(row_ptr, ctypes.POINTER(MIB_TCPR
OW_OWNER_PID)).contents
276             state_str = TCP_STATE_MAP.get(row.dwState, f'STATE
_{row.dwState}')
277             local_ip = self._ip_to_str(row.dwLocalAddr)
278             local_port = self._port_to_int(row.dwLocalPort)
279             remote_ip = self._ip_to_str(row.dwRemoteAddr)
280             remote_port = self._port_to_int(row.dwRemotePort)
281             if row.dwState != MIB_TCP_STATE_LISTEN else 0
282                 connections.append(NetworkSocketInfo(protocol='TCP
', local_address=local_ip, local_port=local_port, remote_addre
ss=remote_ip, remote_port=remote_port, state=state_str, pid=ro
w.dwOwningPid))
283
284         return connections
285
286     def get_udp_sockets(self) -> List[NetworkSocketInfo]:
287         """Получение всех UDP слушающих сокетов с привязкой к
PID.

```

```

286
287     Returns:
288         List[NetworkSocketInfo]: Список UDP сокетов.
289         """
290         size = wintypes.DWORD(0)
291         self._GetExtendedUdpTable(None, ctypes.byref(size), True, AF_INET, UDP_TABLE_OWNER_PID, 0)
292         if size.value == 0:
293             return []
294         buf = (ctypes.c_byte * size.value)()
295         res = self._GetExtendedUdpTable(buf, ctypes.byref(size), True, AF_INET, UDP_TABLE_OWNER_PID, 0)
296         if res != 0:
297             return []
298         num_entries = ctypes.cast(buf, ctypes.POINTER(wintypes.DWORD)).contents.value
299         row_size = ctypes.sizeof(MIB_UDPROW_OWNER_PID)
300         offset = ctypes.sizeof(wintypes.DWORD)
301         sockets_list: List[NetworkSocketInfo] = []
302         for i in range(num_entries):
303             row_ptr = ctypes.byref(buf, offset + i * row_size)
304             row = ctypes.cast(row_ptr, ctypes.POINTER(MIB_UDPROW_OWNER_PID)).contents
305             local_ip = self._ip_to_str(row.dwLocalAddr)
306             local_port = self._port_to_int(row.dwLocalPort)
307             sockets_list.append(NetworkSocketInfo(protocol='UDP', local_address=local_ip, local_port=local_port, remote_address='*.*.*.*', remote_port=0, state='LISTEN', pid=row.dwOwningPid))
308         return sockets_list
309
310     def send_arp(self, dest_ip: str, src_ip: Optional[str] = None) -> Optional[str]:
311         """Отправка ARP-запроса через SendARP API Windows и получение MAC-адреса.
312
313         Args:
314             dest_ip: Целевой IPv4-адрес.

```



```

315 |         src_ip: Исходный IPv4-адрес (опционально, 0 по умо
| лчанию).
316 |
317 |     Returns:
318 |         Optional[str]: Отформатированный MAC-адрес (e.g. '
| 00:1A:2B:3C:4D:5E') или None при отсутствии ответа.
319 |         """
320 |         if not getattr(self, '_SendARP', None):
321 |             return None
322 |         try:
323 |             dest_addr = struct.unpack('<L', socket.inet_aton(d
| est_ip))[0]
324 |             src_addr = struct.unpack('<L', socket.inet_aton(sr
| c_ip))[0] if src_ip else 0
325 |             mac_buf = (ctypes.c_ubyte * 6)()
326 |             mac_len = wintypes.ULONG(6)
327 |             res = self._SendARP(dest_addr, src_addr, ctypes.by
| ref(mac_buf), ctypes.byref(mac_len))
328 |             if res == 0 and mac_len.value == 6:
329 |                 return ':'.join(f'{b:02X}' for b in mac_buf)
330 |             return None
331 |         except Exception as ex:
332 |             logger.debug(f'SendARP error for {dest_ip}: {ex}')
333 |             return None
334 |
335 |     def get_ip_net_table(self) -> List[Dict[str, Any]]:
336 |         """Получение таблицы ARP-записей соседей ядра Windows
| через GetIpNetTable.
337 |
338 |     Returns:
339 |         List[Dict[str, Any]]: Список записей ARP соседей (
| ip, mac, interface_index, type).
340 |         """
341 |         if not getattr(self, '_GetIpNetTable', None):
342 |             return []
343 |         try:
344 |             size = wintypes.ULONG(0)
345 |             self._GetIpNetTable(None, ctypes.byref(size), True

```

```

    )
346         if size.value == 0:
347             return []
348         buf = (ctypes.c_byte * size.value)()
349         res = self._GetIpNetTable(buf, ctypes.byref(size),
    True)
350         if res != 0:
351             return []
352         num_entries = ctypes.cast(buf, ctypes.POINTER(wint
types.DWORD)).contents.value
353         row_size = ctypes.sizeof(MIB_IPNETROW)
354         offset = ctypes.sizeof(wintypes.DWORD)
355         neighbors: List[Dict[str, Any]] = []
356         for i in range(num_entries):
357             row_ptr = ctypes.byref(buf, offset + i * row_s
ize)
358             row = ctypes.cast(row_ptr, ctypes.POINTER(MIB_
IPNETROW)).contents
359             ip = self._ip_to_str(row.dwAddr)
360             if row.dwPhysAddrLen == 6:
361                 mac = ':'.join(f'{b:02X}' for b in row.bPh
ysAddr[:6])
362             else:
363                 mac = ''
364             net_type = IP_NET_TYPE_MAP.get(row.dwType, f'T
YPE_{row.dwType}')
365             if ip and ip != '0.0.0.0' and not ip.startswit
h('255.255.255') and not ip.startswith('224.') and net_type !=
'INVALID':
366                 neighbors.append({
367                     'ip': ip,
368                     'mac': mac,
369                     'interface_index': row.dwIndex,
370                     'type': net_type,
371                 })
372         return neighbors
373     except Exception as ex:
374         logger.debug(f'GetIpNetTable error: {ex}')

```


ntdll.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry Win32_Ffi
4 # - Ntdll
5 # Description:
6 #   ntdll.dll wrapper for native NT API functions.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.win32_fffi.ntdll import SYSTEM_
11 #       M_INFO
12 #       service = SYSTEM_INFO()
13 #
14 # File: ntdll.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry.win32_fffi
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-01 13:28:28
20 # =====
21
22 """ntdll.dll wrapper for native NT API functions."""
23
24 import ctypes
25 import ctypes.wintypes as wintypes
26 from typing import Optional, Any
27 from dataclasses import dataclass
28 STATUS_SUCCESS = 0
29 STATUS_BUFFER_TOO_SMALL = 3221225507
30 SystemBasicInformation = 0
31 SystemProcessorInformation = 1
```

32 SystemPerformanceInformation = 2
33 SystemTimeOfDayInformation = 3
34 SystemPathInformation = 4
35 SystemProcessInformation = 5
36 SystemCallCountInformation = 6
37 SystemDeviceInformation = 7
38 SystemProcessorPerformanceInformation = 8
39 SystemFlagsInformation = 9
40 SystemCallTimeInformation = 10
41 SystemModuleInformation = 11
42 SystemLocksInformation = 12
43 SystemStackTraceInformation = 13
44 SystemPagedPoolInformation = 14
45 SystemNonPagedPoolInformation = 15
46 SystemHandleInformation = 16
47 SystemObjectInformation = 17
48 SystemPageFileInformation = 18
49 SystemVdmInstemulInformation = 19
50 SystemVdmBopInformation = 20
51 SystemFileCacheInformation = 21
52 SystemPoolTagInformation = 22
53 SystemInterruptInformation = 23
54 SystemDpcBehaviorInformation = 24
55 SystemFullMemoryInformation = 25
56 SystemLoadGdiDriverInformation = 26
57 SystemUnloadGdiDriverInformation = 27
58 SystemTimeAdjustmentInformation = 28
59 SystemSummaryMemoryInformation = 29
60 SystemMirrorInformation = 30
61 SystemPerformanceTraceInformation = 31
62 SystemCrashDumpInformation = 32
63 SystemExceptionInformation = 33
64 SystemCrashDumpStateInformation = 34
65 SystemDebuggerInformation = 35
66 SystemProcessorSpeedInformation = 36
67 SystemNUMAProximityNodeInformation = 37
68 SystemDpcWatchdogInformation = 38
69 SystemDpcWatchdogInformation2 = 39

```

70 SystemInvalidProcessorSpeedInformation = 40
71
72 class SYSTEM_INFO(ctypes.Structure):
73     _fields_ = [('dwOemId', wintypes.DWORD), ('dwPageSize', wintypes.DWORD), ('lpMinimumApplicationAddress', wintypes.LPVOID), ('lpMaximumApplicationAddress', wintypes.LPVOID), ('dwActiveProcessorMask', wintypes.DWORD), ('dwNumberOfProcessors', wintypes.DWORD), ('dwProcessorType', wintypes.DWORD), ('dwAllocationGranularity', wintypes.DWORD), ('wProcessorLevel', wintypes.WORD), ('wProcessorRevision', wintypes.WORD)]
74
75 class UNICODE_STRING(ctypes.Structure):
76     _fields_ = [('Length', wintypes.USHORT), ('MaximumLength', wintypes.USHORT), ('Buffer', wintypes.LPWSTR)]
77
78 class OBJECT_NAME_INFORMATION(ctypes.Structure):
79     _fields_ = [('Name', UNICODE_STRING)]
80
81 class PROCESS_BASIC_INFORMATION(ctypes.Structure):
82     _fields_ = [('Reserved1', wintypes.LPVOID), ('PebBaseAddress', wintypes.LPVOID), ('Reserved2_0', wintypes.LPVOID), ('Reserved2_1', wintypes.LPVOID), ('UniqueProcessId', wintypes.LPVOID), ('Reserved3', wintypes.LPVOID)]
83
84 class RTL_OSVERSIONINFOEXW(ctypes.Structure):
85     _fields_ = [('dwOSVersionInfoSize', wintypes.DWORD), ('dwMajorVersion', wintypes.DWORD), ('dwMinorVersion', wintypes.DWORD), ('dwBuildNumber', wintypes.DWORD), ('dwPlatformId', wintypes.DWORD), ('szCSDVersion', ctypes.c_wchar * 128), ('wServicePackMajor', wintypes.WORD), ('wServicePackMinor', wintypes.WORD), ('wSuiteMask', wintypes.WORD), ('wProductType', wintypes.BYTE), ('wReserved', wintypes.BYTE)]
86
87 @dataclass
88 class VersionInfo:
89     """Windows version information."""
90     major: int
91     minor: int

```

```

92     build: int
93     platform: str
94
95 class NtdllAPI:
96     """Wrapper for ntdll native API functions."""
97
98     def __init__(self):
99         self.ntdll = ctypes.windll.ntdll
100         self._setup_function_signatures()
101
102     def _setup_function_signatures(self):
103         """Configure ctypes function signatures."""
104         self.ntdll.NtQuerySystemInformation.argtypes = [wintypes.DWORD, wintypes.LPVOID, wintypes.ULONG, ctypes.POINTER(wintypes.ULONG)]
105         self.ntdll.NtQuerySystemInformation.restype = wintypes.LONG
106         self.ntdll.NtQueryObject.argtypes = [wintypes.HANDLE, wintypes.DWORD, wintypes.LPVOID, wintypes.ULONG, ctypes.POINTER(wintypes.ULONG)]
107         self.ntdll.NtQueryObject.restype = wintypes.LONG
108         self.ntdll.NtQueryInformationProcess.argtypes = [wintypes.HANDLE, wintypes.DWORD, wintypes.LPVOID, wintypes.ULONG, ctypes.POINTER(wintypes.ULONG)]
109         self.ntdll.NtQueryInformationProcess.restype = wintypes.LONG
110         self.ntdll.RtlGetVersion.argtypes = [ctypes.POINTER(RTL_OSVERSIONINFOEXW)]
111         self.ntdll.RtlGetVersion.restype = wintypes.LONG
112
113     def get_windows_version(self) -> Optional[VersionInfo]:
114         """
115         Get Windows version information.
116
117         Returns:
118             VersionInfo object or None
119         """
120         try:

```

```

121         version_info = RTL_OSVERSIONINFOEXW()
122         version_info.dwOSVersionInfoSize = ctypes.sizeof(R
TL_OSVERSIONINFOEXW)
123         status = self.ntdll.RtlGetVersion(ctypes.byref(ver
sion_info))
124         if status == STATUS_SUCCESS:
125             platform_map = {0: 'Win32s', 1: 'Windows 9x',
2: 'Windows NT/2000/XP/Vista/7/8/10/11'}
126             return VersionInfo(major=version_info.dwMajorV
ersion, minor=version_info.dwMinorVersion, build=version_info.
dwBuildNumber, platform=platform_map.get(version_info.dwPlatfo
rmId, 'Unknown'))
127         return None
128     except Exception:
129         return None
130
131     def query_system_information(self, info_class: int) -> Opt
ional[bytes]:
132         """
133         Query system information.
134
135         Args:
136             info_class: System information class
137
138         Returns:
139             System information bytes or None
140         """
141         try:
142             buffer_size = 8192
143             buffer = ctypes.create_string_buffer(buffer_size)
144             return_length = wintypes.ULONG()
145             status = self.ntdll.NtQuerySystemInformation(info_
class, buffer, buffer_size, ctypes.byref(return_length))
146             if status == STATUS_SUCCESS:
147                 return buffer.raw[:return_length.value]
148             return None
149         except Exception:
150             return None

```



```

151
152     def query_object_name(self, handle: int) -> Optional[str]:
153         """
154         Query the name of a kernel object.
155
156         Args:
157             handle: Object handle
158
159         Returns:
160             Object name or None
161         """
162         try:
163             buffer_size = 1024
164             buffer = ctypes.create_string_buffer(buffer_size)
165             return_length = wintypes.ULONG()
166             status = self.ntdll.NtQueryObject(handle, 1, buffer,
167 r, buffer_size, ctypes.byref(return_length))
168             if status == STATUS_SUCCESS:
169                 obj_name_info = ctypes.cast(buffer, ctypes.POINTER(OBJECT_NAME_INFORMATION))
170                 if obj_name_info.contents.Name.Buffer:
171                     return obj_name_info.contents.Name.Buffer
172                 return None
173             except Exception:
174                 return None
175
176     def query_process_basic_info(self, pid: int) -> Optional[PROCESS_BASIC_INFORMATION]:
177         """
178         Query basic process information.
179
180         Args:
181             pid: Process ID
182
183         Returns:
184             PROCESS_BASIC_INFORMATION structure or None
185         """
186         try:

```

```
186 |         handle = ctypes.windll.kernel32.OpenProcess(1024,  
    | False, pid)  
187 |         if not handle:  
188 |             return None  
189 |         try:  
190 |             pbi = PROCESS_BASIC_INFORMATION()  
191 |             return_length = wintypes.ULONG()  
192 |             status = self.ntdll.NtQueryInformationProcess(  
    | handle, 0, ctypes.byref(pbi), ctypes.sizeof(pbi), ctypes.byref  
    | (return_length))  
193 |             if status == STATUS_SUCCESS:  
194 |                 return pbi  
195 |             return None  
196 |         finally:  
197 |             ctypes.windll.kernel32.CloseHandle(handle)  
198 |     except Exception:  
199 |         return None
```

psapi.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry Win32_Ffi
4 # - Psapi
5 # Description:
6 #   psapi.dll wrapper for process memory and module enumeration.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.win32_ffapi import PROCESS_MEMORY_COUNTERS
11 #
12 #       service = PROCESS_MEMORY_COUNTERS()
13 #
14 # File: psapi.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry.win32_ffapi
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-01 13:28:28
20 # =====
21
22 """psapi.dll wrapper for process memory and module enumeration
23 ."""
24
25 import ctypes
26 import ctypes.wintypes as wintypes
27 from typing import List, Optional, Tuple
28 from dataclasses import dataclass
29 LIST_MODULES_DEFAULT = 0
30 LIST_MODULES_32BIT = 1
```

```

30 LIST_MODULES_64BIT = 2
31 LIST_MODULES_ALL = 3
32 SIZE_T = getattr(wintypes, 'SIZE_T', ctypes.c_size_t)
33
34 class PROCESS_MEMORY_COUNTERS(ctypes.Structure):
35     _fields_ = [('cb', wintypes.DWORD), ('PageFaultCount', win
types.DWORD), ('PeakWorkingSetSize', SIZE_T), ('WorkingSetSize
', SIZE_T), ('QuotaPeakPagedPoolUsage', SIZE_T), ('QuotaPagedP
oolUsage', SIZE_T), ('QuotaPeakNonPagedPoolUsage', SIZE_T), ('
QuotaNonPagedPoolUsage', SIZE_T), ('PagefileUsage', SIZE_T), (
'PeakPagefileUsage', SIZE_T)]
36
37 @dataclass
38 class MemoryInfo:
39     """Process memory statistics from psapi."""
40     page_faults: int
41     peak_working_set: int
42     working_set: int
43     paged_pool_peak: int
44     paged_pool_usage: int
45     nonpaged_pool_peak: int
46     nonpaged_pool_usage: int
47     pagefile_usage: int
48     pagefile_peak: int
49
50 class PsapiAPI:
51     """Wrapper for psapi process memory and module functions."
    ""
52
53     def __init__(self):
54         self.psapi = ctypes.windll.psapi
55         self._setup_function_signatures()
56
57     def _setup_function_signatures(self):
58         """Configure ctypes function signatures."""
59         self.psapi.EnumProcesses.argtypes = [ctypes.POINTER(wi
ntypes.DWORD), wintypes.DWORD, ctypes.POINTER(wintypes.DWORD)]
60         self.psapi.EnumProcesses.restype = wintypes.BOOL

```

```

61         self.psapi.GetProcessMemoryInfo.argtypes = [wintypes.H
HANDLE, ctypes.POINTER(PROCESS_MEMORY_COUNTERS), wintypes.DWORD
]
62         self.psapi.GetProcessMemoryInfo.restype = wintypes.BOOL
63         self.psapi.EnumProcessModules.argtypes = [wintypes.HAN
DLE, ctypes.POINTER(wintypes.HMODULE), wintypes.DWORD, ctypes.
POINTER(wintypes.DWORD)]
64         self.psapi.EnumProcessModules.restype = wintypes.BOOL
65         self.psapi.EnumProcessModulesEx.argtypes = [wintypes.H
HANDLE, ctypes.POINTER(wintypes.HMODULE), wintypes.DWORD, ctype
s.POINTER(wintypes.DWORD), wintypes.DWORD]
66         self.psapi.EnumProcessModulesEx.restype = wintypes.BOOL
67         self.psapi.GetModuleFileNameExW.argtypes = [wintypes.H
HANDLE, wintypes.HMODULE, wintypes.LPWSTR, wintypes.DWORD]
68         self.psapi.GetModuleFileNameExW.restype = wintypes.DWORD
69         self.psapi.GetProcessImageFileNameW.argtypes = [wintyp
es.HANDLE, wintypes.LPWSTR, wintypes.DWORD]
70         self.psapi.GetProcessImageFileNameW.restype = wintypes
.DWORD
71
72     def enumerate_processes(self) -> List[int]:
73         """
74         Get all process IDs.
75
76         Returns:
77             List of process IDs
78         """
79         max_pids = 1024
80         pids = (wintypes.DWORD * max_pids)()
81         count_ptr = wintypes.DWORD()
82         if self.psapi.EnumProcesses(pids, ctypes.sizeof(pids),
ctypes.byref(count_ptr)):
83             count = count_ptr.value // ctypes.sizeof(wintypes.
DWORD)
84             return list(pids[:count])

```

```

85         return []
86
87     def get_process_memory_info(self, pid: int) -> Optional[Me
memoryInfo]:
88         """
89         Get memory statistics for a process.
90
91         Args:
92             pid: Process ID
93
94         Returns:
95             MemoryInfo object or None
96         """
97         try:
98             handle = ctypes.windll.kernel32.OpenProcess(1024,
False, pid)
99             if not handle:
100                 return None
101             try:
102                 pmc = PROCESS_MEMORY_COUNTERS()
103                 pmc.cb = ctypes.sizeof(PROCESS_MEMORY_COUNTERS
)
104                 if self.psapi.GetProcessMemoryInfo(handle, cty
pes.byref(pmc), ctypes.sizeof(pmc)):
105                     return MemoryInfo(page_faults=pmc.PageFaul
tCount, peak_working_set=pmc.PeakWorkingSetSize, working_set=p
mc.WorkingSetSize, paged_pool_peak=pmc.QuotaPeakPagedPoolUsage
, paged_pool_usage=pmc.QuotaPagedPoolUsage, nonpaged_pool_peak
=pmc.QuotaPeakNonPagedPoolUsage, nonpaged_pool_usage=pmc.Quota
NonPagedPoolUsage, pagefile_usage=pmc.PagefileUsage, pagefile_
peak=pmc.PeakPagefileUsage)
106                 return None
107             finally:
108                 ctypes.windll.kernel32.CloseHandle(handle)
109         except Exception:
110             return None
111
112     def enumerate_process_modules(self, pid: int, filter_type:

```

```

    int=LIST_MODULES_ALL) -> List[str]:
113         """
114         Get loaded modules for a process.
115
116         Args:
117             pid: Process ID
118             filter_type: Module filter (32-bit, 64-bit, or all
119         )
120
121         Returns:
122             List of module paths
123         """
124         modules = []
125         try:
126             handle = ctypes.windll.kernel32.OpenProcess(4096 |
127                 1024, False, pid)
128             if not handle:
129                 return modules
130             try:
131                 max_modules = 256
132                 module_handles = (wintypes.HMODULE * max_modul
133             es)()
134                 count_ptr = wintypes.DWORD()
135                 if self.psapi.EnumProcessModulesEx(handle, mod
136             ule_handles, ctypes.sizeof(module_handles), ctypes.byref(count
137             _ptr), filter_type):
138                     count = count_ptr.value // ctypes.sizeof(w
139             intypes.HMODULE)
140                     for i in range(count):
141                         filename_buffer = ctypes.create_unicod
142             e_buffer(260)
143                         if self.psapi.GetModuleFileNameExW(han
144             dle, module_handles[i], filename_buffer, len(filename_buffer))
145             :
146                             modules.append(filename_buffer.val
147             ue)
148             finally:
149                 ctypes.windll.kernel32.CloseHandle(handle)

```

```
140         except Exception:
141             pass
142         return modules
143
144     def get_process_image_filename(self, pid: int) -> Optional
145     [str]:
146         """
147         Get the executable path of a process.
148
149         Args:
150             pid: Process ID
151
152         Returns:
153             Process image filename or None
154         """
155         try:
156             handle = ctypes.windll.kernel32.OpenProcess(1024,
157             False, pid)
158             if not handle:
159                 return None
160             try:
161                 filename_buffer = ctypes.create_unicode_buffer
162                 (260)
163                 if self.psapi.GetProcessImageFileNameW(handle,
164                 filename_buffer, len(filename_buffer)):
165                     return filename_buffer.value
166                 return None
167             finally:
168                 ctypes.windll.kernel32.CloseHandle(handle)
169         except Exception:
170             return None
```


scm.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry Win32_Ffi
4 # - Scm
5 # Description:
6 # Низкоуровневый интерфейс к Service Control Manager (advapi
7 # 32.dll) Windows.
8 # Usage Examples:
9 # Python API:
10 # from apps.windows.telemetry.win32_fffi.scm import SERVICE
11 # _STATUS_PROCESS
12 # service = SERVICE_STATUS_PROCESS()
13 #
14 # File: scm.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry.win32_fffi
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-01 13:28:28
20 # =====
21
22 from __future__ import annotations
23 """Низкоуровневый интерфейс к Service Control Manager (advapi3
24 2.dll) Windows."""
25
26 import ctypes
27 import ctypes.wintypes as wintypes
28 from dataclasses import dataclass
29 from typing import Any, Dict, List, Optional
30 from logger import logger
```

```

30 SC_MANAGER_CONNECT = 1
31 SC_MANAGER_CREATE_SERVICE = 2
32 SC_MANAGER_ENUMERATE_SERVICE = 4
33 SC_MANAGER_LOCK = 8
34 SC_MANAGER_QUERY_LOCK_STATUS = 16
35 SC_MANAGER_MODIFY_BOOT_CONFIG = 32
36 SC_MANAGER_ALL_ACCESS = 983103
37 SERVICE_WIN32_OWN_PROCESS = 16
38 SERVICE_WIN32_SHARE_PROCESS = 32
39 SERVICE_DRIVER = 11
40 SERVICE_TYPE_ALL = SERVICE_WIN32_OWN_PROCESS | SERVICE_WIN32_S
    HARE_PROCESS | SERVICE_DRIVER
41 SERVICE_ACTIVE = 1
42 SERVICE_INACTIVE = 2
43 SERVICE_STATE_ALL = SERVICE_ACTIVE | SERVICE_INACTIVE
44 SERVICE_STOPPED = 1
45 SERVICE_START_PENDING = 2
46 SERVICE_STOP_PENDING = 3
47 SERVICE_RUNNING = 4
48 SERVICE_CONTINUE_PENDING = 5
49 SERVICE_PAUSE_PENDING = 6
50 SERVICE_PAUSED = 7
51 SERVICE_QUERY_CONFIG = 1
52 SERVICE_CHANGE_CONFIG = 2
53 SERVICE_QUERY_STATUS = 4
54 SERVICE_ENUMERATE_DEPENDENTS = 8
55 SERVICE_START = 16
56 SERVICE_STOP = 32
57 SERVICE_PAUSE_CONTINUE = 64
58 SERVICE_INTERROGATE = 128
59 SERVICE_USER_DEFINED_CONTROL = 256
60 SERVICE_ALL_ACCESS = 983551
61 SC_ENUM_PROCESS_INFO = 0
62 ERROR_MORE_DATA = 234
63
64 class SERVICE_STATUS_PROCESS(ctypes.Structure):
65     """Структура статуса и процесса службы."""
66     _fields_ = [('dwServiceType', wintypes.DWORD), ('dwCurrent

```

```
State', wintypes.DWORD), ('dwControlsAccepted', wintypes.DWORD), ('dwWin32ExitCode', wintypes.DWORD), ('dwServiceSpecificExitCode', wintypes.DWORD), ('dwCheckPoint', wintypes.DWORD), ('dwWaitHint', wintypes.DWORD), ('dwProcessId', wintypes.DWORD), ('dwServiceFlags', wintypes.DWORD)]
```

67

```
68 class ENUM_SERVICE_STATUS_PROCESSW(ctypes.Structure):
```

```
69     """Структура элемента перечисления служб с PID."""
```

```
70     _fields_ = [('lpServiceName', wintypes.LPWSTR), ('lpDisplayName', wintypes.LPWSTR), ('ServiceStatusProcess', SERVICE_STATUS_PROCESS)]
```

71

```
72 class QUERY_SERVICE_CONFIGW(ctypes.Structure):
```

```
73     """Структура детальной конфигурации службы."""
```

```
74     _fields_ = [('dwServiceType', wintypes.DWORD), ('dwStartType', wintypes.DWORD), ('dwErrorControl', wintypes.DWORD), ('lpBinaryPathName', wintypes.LPWSTR), ('lpLoadOrderGroup', wintypes.LPWSTR), ('dwTagId', wintypes.DWORD), ('lpDependencies', wintypes.LPWSTR), ('lpServiceStartName', wintypes.LPWSTR), ('lpDisplayName', wintypes.LPWSTR)]
```

75

```
76 @dataclass
```

```
77 class ServiceInfo:
```

```
78     """Нормализованные данные о службе Windows."""
```

```
79     name: str
```

```
80     display_name: str
```

```
81     state: str
```

```
82     state_code: int
```

```
83     pid: int
```

```
84     service_type: int
```

```
85     binary_path: str = ''
```

```
86     start_type: str = ''
```

```
87     account: str = ''
```

88

```
89     def to_dict(self) -> Dict[str, Any]:
```

```
90         """Преобразование в словарь."""
```

```
91         return {'name': self.name, 'display_name': self.display_name, 'state': self.state, 'state_code': self.state_code, 'p
```

```

id': self.pid, 'service_type': self.service_type, 'binary_path':
self.binary_path, 'start_type': self.start_type, 'account':
self.account}

92
93 class ServiceControlManager:
94     """Низкоуровневый клиент к Service Control Manager (advapi
32.dll)."""
95     STATE_MAP = {SERVICE_STOPPED: 'STOPPED', SERVICE_START_PEN
DING: 'START_PENDING', SERVICE_STOP_PENDING: 'STOP_PENDING', S
ERVICE_RUNNING: 'RUNNING', SERVICE_CONTINUE_PENDING: 'CONTINUE
_PENDING', SERVICE_PAUSE_PENDING: 'PAUSE_PENDING', SERVICE_PAU
SED: 'PAUSED'}
96     START_TYPE_MAP = {0: 'BOOT', 1: 'SYSTEM', 2: 'AUTO_START',
3: 'DEMAND_START', 4: 'DISABLED'}
97
98     def __init__(self) -> None:
99         """Инициализация функций advapi32.dll."""
100         self._advapi32 = ctypes.windll.advapi32
101         self._OpenSCManagerW = self._advapi32.OpenSCManagerW
102         self._OpenSCManagerW.argtypes = [wintypes.LPCWSTR, win
types.LPCWSTR, wintypes.DWORD]
103         self._OpenSCManagerW.restype = wintypes.SC_HANDLE
104         self._EnumServicesStatusExW = self._advapi32.EnumServi
cesStatusExW
105         self._EnumServicesStatusExW.argtypes = [wintypes.SC_HA
NDLE, wintypes.DWORD, wintypes.DWORD, wintypes.DWORD, wintypes
.LPBYTE, wintypes.DWORD, ctypes.POINTER(wintypes.DWORD), ctype
s.POINTER(wintypes.DWORD), ctypes.POINTER(wintypes.DWORD), win
types.LPCWSTR]
106         self._EnumServicesStatusExW.restype = wintypes.BOOL
107         self._OpenServiceW = self._advapi32.OpenServiceW
108         self._OpenServiceW.argtypes = [wintypes.SC_HANDLE, win
types.LPCWSTR, wintypes.DWORD]
109         self._OpenServiceW.restype = wintypes.SC_HANDLE
110         self._QueryServiceConfigW = self._advapi32.QueryServic
eConfigW
111         self._QueryServiceConfigW.argtypes = [wintypes.SC_HAN
DLE, wintypes.LPBYTE, wintypes.DWORD, ctypes.POINTER(wintypes.D

```

```

WORD)]
112         self._QueryServiceConfigW.restype = wintypes.BOOL
113         self._CloseServiceHandle = self._advapi32.CloseService
Handle
114         self._CloseServiceHandle.argtypes = [wintypes.SC_HANDL
E]
115         self._CloseServiceHandle.restype = wintypes.BOOL
116
117         def enum_services(self, service_type: int=SERVICE_WIN32_OW
N_PROCESS | SERVICE_WIN32_SHARE_PROCESS) -> List[ServiceInfo]:
118             """Быстрое пакетное перечисление всех служб через Enum
ServicesStatusExW.
119
120             Args:
121                 service_type: Маска типов служб (Win32, Driver, et
с.).
122
123             Returns:
124                 List[ServiceInfo]: Список обнаруженных служб с PID
и статусом.
125             """
126             h_scm = self._OpenSCManagerW(None, None, SC_MANAGER_EN
UMERATE_SERVICE | SC_MANAGER_CONNECT)
127             if not h_scm:
128                 logger.debug('Не удалось открыть SCManager через W
inAPI')
129                 return []
130             services: List[ServiceInfo] = []
131             try:
132                 bytes_needed = wintypes.DWORD(0)
133                 services_returned = wintypes.DWORD(0)
134                 resume_handle = wintypes.DWORD(0)
135                 self._EnumServicesStatusExW(h_scm, SC_ENUM_PROCESS
_INFO, service_type, SERVICE_STATE_ALL, None, 0, ctypes.byref(
bytes_needed), ctypes.byref(services_returned), ctypes.byref(r
esume_handle), None)
136                 if bytes_needed.value == 0:
137                     return []

```

```

138         buf = (ctypes.c_byte * bytes_needed.value)()
139         success = self._EnumServicesStatusExW(h_scm, SC_EN
UM_PROCESS_INFO, service_type, SERVICE_STATE_ALL, ctypes.cast(
buf, wintypes.LPBYTE), bytes_needed.value, ctypes.byref(bytes_
needed), ctypes.byref(services_returned), ctypes.byref(resume_
handle), None)
140         if not success and ctypes.GetLastError() != ERROR_
MORE_DATA:
141             return []
142             entry_ptr = ctypes.cast(buf, ctypes.POINTER(ENUM_S
ERVICE_STATUS_PROCESSW))
143             for i in range(services_returned.value):
144                 entry = entry_ptr[i]
145                 state_code = entry.ServiceStatusProcess.dwCurr
entState
146                 state_str = self.STATE_MAP.get(state_code, 'UN
KNOWN')
147                 pid = entry.ServiceStatusProcess.dwProcessId
148                 svc = ServiceInfo(name=entry.lpServiceName or
'', display_name=entry.lpDisplayName or entry.lpServiceName or
'', state=state_str, state_code=state_code, pid=pid, service_
type=entry.ServiceStatusProcess.dwServiceType)
149                 services.append(svc)
150             except Exception as ex:
151                 logger.debug(f'Исключение при нативном перечислени
и служб SCM: {ex}')
152             finally:
153                 self._CloseServiceHandle(h_scm)
154             return services
155
156     def get_service_config(self, service_name: str) -> Optiona
l[Dict[str, Any]]:
157         """Получение подробной конфигурации службы через Query
ServiceConfigW.
158
159         Args:
160             service_name: Системное имя службы.
161

```

```

162         Returns:
163             Optional[Dict[str, Any]]: Данные конфигурации или
None при ошибке.
164         """
165         h_scm = self._OpenSCManagerW(None, None, SC_MANAGER_CO
NNECT)
166         if not h_scm:
167             return None
168         try:
169             h_svc = self._OpenServiceW(h_scm, service_name, SE
RVICE_QUERY_CONFIG)
170             if not h_svc:
171                 return None
172             try:
173                 bytes_needed = wintypes.DWORD(0)
174                 self._QueryServiceConfigW(h_svc, None, 0, ctypes
es.byref(bytes_needed))
175                 if bytes_needed.value == 0:
176                     return None
177                 buf = (ctypes.c_byte * bytes_needed.value)()
178                 if not self._QueryServiceConfigW(h_svc, ctypes
.cast(buf, wintypes.LPBYTE), bytes_needed.value, ctypes.byref(
bytes_needed)):
179                     return None
180                 cfg = ctypes.cast(buf, ctypes.POINTER(QUERY_SE
RVICE_CONFIGW)).contents
181                 return {'binary_path': cfg.lpBinaryPathName or
'', 'start_type': self.START_TYPE_MAP.get(cfg.dwStartType, st
r(cfg.dwStartType)), 'start_type_code': cfg.dwStartType, 'acco
unt': cfg.lpServiceStartName or '', 'display_name': cfg.lpDisp
layName or service_name, 'dependencies': cfg.lpDependencies or
''}
182             finally:
183                 self._CloseServiceHandle(h_svc)
184         except Exception as ex:
185             logger.debug(f"Ошибка чтения конфигурации службы '{
{service_name}': {ex}")
186             return None

```

```
187 |         finally:  
188 |             self._CloseServiceHandle(h_scm)
```


setupapi.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry Win32_Ffi
  - Setupapi
4 # =====
5 # Description:
6 #   Низкоуровневый интерфейс к SetupAPI и Configuration Manager PnP.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.win32_ff.setupapi import GUID
11 #
12 #       service = GUID()
13 #
14 # File: setupapi.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry.win32_ff
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-01 13:28:28
20 # =====
21
22 from __future__ import annotations
23 """Низкоуровневый интерфейс к SetupAPI и Configuration Manager
   PnP."""
24
25 import ctypes
26 import ctypes.wintypes as wintypes
27 from dataclasses import dataclass
28 from typing import Any, Dict, List, Optional
29 from logger import logger
```

```
30 DIGCF_DEFAULT = 1
31 DIGCF_PRESENT = 2
32 DIGCF_ALLCLASSES = 4
33 DIGCF_PROFILE = 8
34 DIGCF_DEVICEINTERFACE = 16
35 SPDRP_DEVICEDESC = 0
36 SPDRP_HARDWAREID = 1
37 SPDRP_CLASS = 7
38 SPDRP_CLASSGUID = 8
39 SPDRP_DRIVER = 9
40 SPDRP_FRIENDLYNAME = 12
41 SPDRP_LOCATION_INFORMATION = 13
42 SPDRP_MFG = 11
43 DN_ROOT_ENUMERATED = 1
44 DN_DRIVER_LOADED = 2
45 DN_ENUM_LOADED = 4
46 DN_STARTED = 8
47 DN_MANUAL = 16
48 DN_NEED_TO_ENUM = 32
49 DN_NOT_FIRST_TIME = 64
50 DN_HARDWARE_ENUM = 128
51 DN_LIBCONFIG = 256
52 DN_FILTERED = 512
53 DN_MOVED = 1024
54 DN_DISABLEABLE = 2048
55 DN_REMOVABLE = 4096
56 DN_HAS_PROBLEM = 1024
57 DN_PRIVATE_PROBLEM = 32768
58 CR_SUCCESS = 0
59 INVALID_HANDLE_VALUE = ctypes.c_void_p(-1).value
60
61 class GUID(ctypes.Structure):
62     """Структура Windows GUID."""
63     _fields_ = [('Data1', wintypes.DWORD), ('Data2', wintypes.
64         WORD), ('Data3', wintypes.WORD), ('Data4', wintypes.BYTE * 8)]
65
66 class SP_DEVINFO_DATA(ctypes.Structure):
67     """Структура информации об устройстве SetupAPI."""
```

```

67     _fields_ = [('cbSize', wintypes.DWORD), ('ClassGuid', GUID
    ), ('DevInst', wintypes.DWORD), ('Reserved', ctypes.c_void_p)]
68
69 @dataclass
70 class PnPDeviceInfo:
71     """Нормализованные данные об аппаратном устройстве PnP."""
72     device_instance_id: str
73     friendly_name: str
74     hardware_id: str
75     device_class: str
76     manufacturer: str
77     has_problem: bool
78     status_code: int
79     problem_code: int
80
81     def to_dict(self) -> Dict[str, Any]:
82         """Преобразование в словарь."""
83         return {'device_instance_id': self.device_instance_id,
84             'friendly_name': self.friendly_name, 'hardware_id': self.hard
85             ware_id, 'device_class': self.device_class, 'manufacturer': se
86             lf.manufacturer, 'has_problem': self.has_problem, 'status_code
87             ': self.status_code, 'problem_code': self.problem_code}
88
89 class SetupAPI:
90     """Низкоуровневый клиент к SetupAPI и CfgMgr32."""
91
92     def __init__(self) -> None:
93         """Инициализация библиотек setupapi.dll и cfgmgr32.dll
94         ."""
95         self._setupapi = ctypes.windll.setupapi
96         self._cfgmgr32 = ctypes.windll.cfmgr32
97         self._SetupDiGetClassDevsW = self._setupapi.SetupDiGet
98         ClassDevsW
99         self._SetupDiGetClassDevsW.argtypes = [ctypes.c_void_p
100         , wintypes.LPCWSTR, wintypes.HWND, wintypes.DWORD]
101         self._SetupDiGetClassDevsW.restype = ctypes.c_void_p
102         self._SetupDiEnumDeviceInfo = self._setupapi.SetupDiEn
103         umDeviceInfo

```

```

96         self._SetupDiEnumDeviceInfo.argtypes = [ctypes.c_void_p, wintypes.DWORD, ctypes.POINTER(SP_DEVINFO_DATA)]
97         self._SetupDiEnumDeviceInfo.restype = wintypes.BOOL
98         self._SetupDiGetDeviceRegistryPropertyW = self._setupapi.SetupDiGetDeviceRegistryPropertyW
99         self._SetupDiGetDeviceRegistryPropertyW.argtypes = [ctypes.c_void_p, ctypes.POINTER(SP_DEVINFO_DATA), wintypes.DWORD, ctypes.POINTER(wintypes.DWORD), wintypes.LPBYTE, wintypes.DWORD, ctypes.POINTER(wintypes.DWORD)]
100        self._SetupDiGetDeviceRegistryPropertyW.restype = wintypes.BOOL
101        self._SetupDiDestroyDeviceInfoList = self._setupapi.SetupDiDestroyDeviceInfoList
102        self._SetupDiDestroyDeviceInfoList.argtypes = [ctypes.c_void_p]
103        self._SetupDiDestroyDeviceInfoList.restype = wintypes.BOOL
104        self._CM_Get_DevNode_Status = self._cfgmgr32.CM_Get_DevNode_Status
105        self._CM_Get_DevNode_Status.argtypes = [ctypes.POINTER(wintypes.ULONG), ctypes.POINTER(wintypes.ULONG), wintypes.DWORD, wintypes.ULONG]
106        self._CM_Get_DevNode_Status.restype = wintypes.DWORD
107        self._CM_Get_Device_IDW = self._cfgmgr32.CM_Get_Device_IDW
108        self._CM_Get_Device_IDW.argtypes = [wintypes.DWORD, wintypes.LPWSTR, wintypes.ULONG, wintypes.ULONG]
109        self._CM_Get_Device_IDW.restype = wintypes.DWORD
110
111        def _get_property_string(self, h_dev_info: ctypes.c_void_p, dev_data: SP_DEVINFO_DATA, prop: int) -> str:
112            """Получение строкового свойства устройства из реестра SetupAPI."""
113            reg_type = wintypes.DWORD(0)
114            req_size = wintypes.DWORD(0)
115            self._SetupDiGetDeviceRegistryPropertyW(h_dev_info, ctypes.byref(dev_data), prop, ctypes.byref(reg_type), None, 0, ctypes.byref(req_size))

```

```

116         if req_size.value == 0:
117             return ''
118         buf = (ctypes.c_byte * req_size.value)()
119         if not self._SetupDiGetDeviceRegistryPropertyW(h_dev_i
nfo, ctypes.byref(dev_data), prop, ctypes.byref(reg_type), cty
pes.cast(buf, wintypes.LPBYTE), req_size.value, ctypes.byref(r
eq_size)):
120             return ''
121         try:
122             return ctypes.wstring_at(buf)
123         except Exception:
124             return ''
125
126     def get_all_devices(self, only_present: bool=True) -> List
[PnPDeviceInfo]:
127         """Получение списка всех устройств PnP с их статусом и
кодами проблем.
128
129         Args:
130             only_present: Фильтровать только подключенные устр
ойства.
131
132         Returns:
133             List[PnPDeviceInfo]: Список PnP устройств.
134         """
135         flags = DIGCF_ALLCLASSES
136         if only_present:
137             flags |= DIGCF_PRESENT
138         h_dev_info = self._SetupDiGetClassDevsW(None, None, No
ne, flags)
139         if not h_dev_info or h_dev_info == INVALID_HANDLE_VALU
E:
140             logger.debug('Не удалось получить дескриптор Setup
DiGetClassDevsW')
141             return []
142         devices: List[PnPDeviceInfo] = []
143         try:
144             dev_data = SP_DEVINFO_DATA()

```

```

145         dev_data.cbSize = ctypes.sizeof(SP_DEVINFO_DATA)
146         idx = 0
147         while self._SetupDiEnumDeviceInfo(h_dev_info, idx,
        ctypes.byref(dev_data)):
148             idx += 1
149             try:
150                 friendly_name = self._get_property_string(
        h_dev_info, dev_data, SPDRP_FRIENDLYNAME)
151                 if not friendly_name:
152                     friendly_name = self._get_property_str
        ing(h_dev_info, dev_data, SPDRP_DEVICEDESC)
153                 hw_id = self._get_property_string(h_dev_in
        fo, dev_data, SPDRP_HARDWAREID)
154                 cls_name = self._get_property_string(h_dev
        _info, dev_data, SPDRP_CLASS)
155                 mfg = self._get_property_string(h_dev_info
        , dev_data, SPDRP_MFG)
156                 id_buf = (ctypes.c_wchar * 512)()
157                 cm_res = self._CM_Get_Device_IDW(dev_data.
        DevInst, id_buf, 512, 0)
158                 device_instance_id = id_buf.value if cm_re
        s == CR_SUCCESS else ''
159                 status = wintypes.ULONG(0)
160                 prob_code = wintypes.ULONG(0)
161                 cm_status = self._CM_Get_DevNode_Status(ct
        ypes.byref(status), ctypes.byref(prob_code), dev_data.DevInst,
        0)
162                 has_problem = False
163                 problem_num = 0
164                 if cm_status == CR_SUCCESS:
165                     problem_num = prob_code.value
166                     has_problem = status.value & DN_HAS_PR
        OBLEM != 0 or problem_num != 0
167                 devices.append(PnPDeviceInfo(device instan
        ce_id=device_instance_id, friendly_name=friendly_name or devic
        e_instance_id or 'Устройство без имени', hardware_id=hw_id, de
        vice_class=cls_name, manufacturer=mfg, has_problem=has_problem
        , status_code=status.value if cm_status == CR_SUCCESS else 0,

```

```
        problem_code=problem_num))
168 |             except Exception:
169 |                 continue
170 |         finally:
171 |             self._SetupDiDestroyDeviceInfoList(h_dev_info)
172 |         return devices
173 |
174 |     def get_problem_devices(self) -> List[PnPDeviceInfo]:
175 |         """Быстрый поиск только устройств с аппаратными сбоями
176 |         (Code 10/43/28).
177 |
178 |         Returns:
179 |             List[PnPDeviceInfo]: Список проблемных устройств.
180 |
181 |         all_devs = self.get_all_devices(only_present=True)
181 |         return [d for d in all_devs if d.has_problem and d.pro
blem_code > 0]
```

tasksched.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry Win32_Ffi
4 # - Tasksched
5 # =====
6 # Description:
7 #   Интерфейс к Task Scheduler 2.0 COM API Windows.
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.win32_fffi.tasksched import T
11 #       askSchedulerAPI
12 #
13 #       service = TaskSchedulerAPI()
14 #
15 # File: tasksched.py
16 # Project: ai-breadboard
17 # Package: apps.windows.telemetry.win32_fffi
18 # Author: hypo69
19 # Copyright: © 2026 hypo69
20 # Updated: 2026-10-06 12:35:00
21 # =====
22
23 from __future__ import annotations
24
25 """Интерфейс к Task Scheduler 2.0 COM API Windows."""
26
27 import sys
28 from datetime import datetime
29 from typing import Any, Dict, List, Optional
30 from logger import logger
31
32 TASK_STATE_UNKNOWN = 0
33 TASK_STATE_DISABLED = 1
```



```

32 TASK_STATE_QUEUED = 2
33 TASK_STATE_READY = 3
34 TASK_STATE_RUNNING = 4
35 TASK_STATE_MAP = {
36     TASK_STATE_UNKNOWN: 'Unknown',
37     TASK_STATE_DISABLED: 'Disabled',
38     TASK_STATE_QUEUED: 'Queued',
39     TASK_STATE_READY: 'Ready',
40     TASK_STATE_RUNNING: 'Running'
41 }
42
43 TASK_ACTION_EXEC = 0
44 TASK_ACTION_COM_HANDLER = 5
45 TASK_ACTION_SEND_EMAIL = 6
46 TASK_ACTION_SHOW_MESSAGE = 7
47
48 TASK_TRIGGER_TYPE_MAP = {
49     0: 'On event',
50     1: 'Time',
51     2: 'Daily',
52     3: 'Weekly',
53     4: 'Monthly',
54     5: 'Monthly DOW',
55     6: 'On idle',
56     7: 'On registration',
57     8: 'At startup',
58     9: 'At logon',
59     11: 'Session state change',
60 }
61
62
63 def _format_com_date(dt_obj: Any) -> Optional[str]:
64     """Форматирует дату из COM API в читаемую строку.
65
66     Args:
67         dt_obj: COM объект даты или datetime.
68
69     Returns:

```

```

70         Optional[str]: Отформатированная дата или None.
71     """
72     if dt_obj is None:
73         return None
74     try:
75         # COM дата 1899 года означает отсутствие даты
76         if hasattr(dt_obj, 'year'):
77             if dt_obj.year < 1910 or dt_obj.year > 2100:
78                 return None
79             return dt_obj.strftime('%Y-%m-%d %H:%M:%S')
80         s = str(dt_obj).strip()
81         if '1899' in s or s == '' or s == 'None':
82             return None
83         return s
84     except Exception:
85         return None
86
87
88 class TaskSchedulerAPI:
89     """Обертка над COM-интерфейсом Task Scheduler 2.0 (Schedule.Service)."""
90
91     def __init__(self) -> None:
92         """Инициализация COM-объекта планировщика."""
93         self._service: Optional[Any] = None
94         self._is_available = self._connect()
95
96     def _connect(self) -> bool:
97         """Подключение к локальной службе планировщика через COM."""
98         try:
99             import win32com.client
100             self._service = win32com.client.Dispatch('Schedule
101             .Service')
102             self._service.Connect()
103             return True
104         except Exception as ex:
105             logger.debug(f'COM-интерфейс Schedule.Service недо

```

```

    ступен: {ex}')
105         self._service = None
106         return False
107
108     @property
109     def is_available(self) -> bool:
110         """Доступность нативного COM-интерфейса планировщика."
111         ""
112         return self._is_available and self._service is not None
113
114     def get_all_tasks(self, max_tasks: int = 500) -> List[Dict
115     [str, Any]]:
116         """Рекурсивный сбор всех задач планировщика из всех па
117         пок.
118
119         Args:
120             max_tasks: Максимальное количество задач для выбор
121             ки.
122
123         Returns:
124             List[Dict[str, Any]]: Список задач с путями, дейст
125             виями и статусами.
126         """
127         if not self.is_available or self._service is None:
128             return []
129         tasks: List[Dict[str, Any]] = []
130         try:
131             root_folder = self._service.GetFolder('\\')
132             self._scan_folder(root_folder, tasks, max_tasks)
133         except Exception as ex:
134             logger.debug(f'Ошибка при рекурсивном чтении задач
135             планировщика COM: {ex}')
136         return tasks
137
138     def _scan_folder(self, folder: Any, tasks: List[Dict[str,
139     Any]], max_tasks: int) -> None:
140         """Рекурсивный обход папки планировщика.

```

```

134
135     Args:
136         folder: COM-объект ITaskFolder.
137         tasks: Результирующий список задач.
138         max_tasks: Лимит задач.
139     """
140     if len(tasks) >= max_tasks:
141         return
142     try:
143         folder_tasks = folder.GetTasks(0)
144         for task in folder_tasks:
145             if len(tasks) >= max_tasks:
146                 break
147             try:
148                 task_info = self._parse_task(task)
149                 if task_info:
150                     tasks.append(task_info)
151             except Exception:
152                 continue
153         subfolders = folder.GetFolders(0)
154         for sub in subfolders:
155             if len(tasks) >= max_tasks:
156                 break
157             self._scan_folder(sub, tasks, max_tasks)
158     except Exception as ex:
159         logger.debug(f"Ошибка при сканировании папки '{get
160 attr(folder, 'Path', '')}': {ex}")
161
162     def _parse_task(self, task: Any) -> Optional[Dict[str, Any
163 ]]:
164         """Парсинг свойств зарегистрированной задачи (IRegiste
165 redTask)."""
166         name = getattr(task, 'Name', '') or ''
167         path = getattr(task, 'Path', '') or ''
168         state_code = getattr(task, 'State', 0)
169         state_name = TASK_STATE_MAP.get(state_code, 'Unknown')
170         enabled = getattr(task, 'Enabled', True)
171         next_run_raw = getattr(task, 'NextRunTime', None)

```

```

169 |         last_run_raw = getattr(task, 'LastRunTime', None)
170 |         last_result = getattr(task, 'LastTaskResult', 0) or 0
171 |
172 |         next_run_str = _format_com_date(next_run_raw)
173 |         last_run_str = _format_com_date(last_run_raw)
174 |
175 |         actions_list: List[Dict[str, Any]] = []
176 |         action_strs: List[str] = []
177 |         author = ''
178 |         triggers_list: List[str] = []
179 |
180 |         try:
181 |             definition = task.Definition
182 |             if hasattr(definition, 'RegistrationInfo') and def
initiation.RegistrationInfo:
183 |                 author = getattr(definition.RegistrationInfo,
184 | 'Author', '') or ''
185 |
186 |             if hasattr(definition, 'Triggers') and definition.
187 | Triggers:
188 |                 for trig in definition.Triggers:
189 |                     t_type = getattr(trig, 'Type', -1)
190 |                     t_label = TASK_TRIGGER_TYPE_MAP.get(t_type
191 | , 'Custom')
192 |                     if t_label not in triggers_list:
193 |                         triggers_list.append(t_label)
194 |
195 |             if hasattr(definition, 'Actions') and definition.A
196 | ctions:
197 |                 for act in definition.Actions:
198 |                     act_type = getattr(act, 'Type', -1)
199 |                     if act_type == TASK_ACTION_EXEC:
200 |                         exec_path = getattr(act, 'Path', '') o
201 | r ''
202 |                         args = getattr(act, 'Arguments', '') o
203 | r ''
204 |                         working_dir = getattr(act, 'WorkingDir
205 | ectory', '') or ''

```

```

199         full_cmd = f'{exec_path} {args}'.strip
    |
    | ()
200         actions_list.append({
201             'type': 'Exec',
202             'path': exec_path,
203             'arguments': args,
204             'working_directory': working_dir,
205             'command': full_cmd
206         })
207         action_strs.append(full_cmd)
208         elif act_type == TASK_ACTION_COM_HANDLER:
209             clsid = getattr(act, 'ClassId', '')
210             actions_list.append({'type': 'ComHandl
    | er', 'class_id': clsid})
211             action_strs.append(f'ComHandler:{clsid
    | }')
212         except Exception:
213             pass
214
215         schedule_type = ', '.join(triggers_list) if triggers_l
    | ist else 'Custom'
216         action_summary = ' ; '.join(action_strs) if action_str
    | s else ''
217
218         return {
219             'TaskName': name,
220             'task_name': name,
221             'name': name,
222             'TaskPath': path,
223             'task_path': path,
224             'State': state_name,
225             'state': state_name,
226             'status': state_name,
227             'Enabled': enabled,
228             'enabled': enabled,
229             'NextRunTime': next_run_str,
230             'next_run_time': next_run_str,
231             'LastRunTime': last_run_str,

```

```
232 |         'last_run_time': last_run_str,  
233 |         'LastTaskResult': last_result,  
234 |         'last_task_result': last_result,  
235 |         'Author': author,  
236 |         'author': author,  
237 |         'ScheduleType': schedule_type,  
238 |         'schedule_type': schedule_type,  
239 |         'Actions': action_summary,  
240 |         'action': action_summary,  
241 |         'ActionDetails': actions_list,  
242 |     }
```

wevtapi.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry Win32_Ffi
4 # - Wevtapi
5 # Description:
6 # Низкоуровневый интерфейс к wevtapi.dll для работы с журнал
7 # ами событий Windows.
8 # Usage Examples:
9 # Python API:
10 # from apps.windows.telemetry.win32_fffi.wevtapi import Cha
11 # nnelMetadata
12 # service = ChannelMetadata()
13 #
14 # File: wevtapi.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry.win32_fffi
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-08 02:15:00
20 # =====
21
22 from __future__ import annotations
23 """Низкоуровневый интерфейс к wevtapi.dll для работы с журнала
24 ми событий Windows."""
25
26 import ctypes
27 import ctypes.wintypes as wintypes
28 import struct
29 import sys
30 import winreg
```



```

30 import xml.etree.ElementTree as ET
31 from dataclasses import dataclass
32 from typing import Any, Callable, Dict, List, Optional, Tuple
33 from logger import logger
34
35 # Константы EvtQuery и EvtRender
36 EVT_QUERY_CHANNEL_PATH = 1
37 EVT_QUERY_FILE_PATH = 2
38 EVT_QUERY_REVERSE_DIRECTION = 512
39 EVT_QUERY_FORWARD_DIRECTION = 256
40 EVT_RENDER_EVENT_XML = 1
41 EVT_RENDER_BOOKMARK_XML = 2
42
43 # Константы EvtSeek
44 EVT_SEEK_ORIGIN_SET = 1
45 EVT_SEEK_ORIGIN_CURRENT = 2
46 EVT_SEEK_ORIGIN_END = 3
47 EVT_SEEK_ORIGIN_BOOKMARK = 4
48 EVT_SEEK_RELATIVE_TO_BOOKMARK = 8
49 EVT_SEEK_AFTER_BOOKMARK = 16
50 EVT_SEEK_STRICT = 0x10000
51
52 # Константы EvtSubscribe
53 EVT_SUBSCRIBE_TO_FUTURE_EVENTS = 1
54 EVT_SUBSCRIBE_START_AT_OLDEST = 2
55 EVT_SUBSCRIBE_START_AFTER_BOOKMARK = 3
56 EVT_SUBSCRIBE_TOLERATE_QUERY_ERRORS = 0x1000
57 EVT_SUBSCRIBE_STRICT = 0x10000
58
59 # Callback тип для EvtSubscribe
60 EVT_SUBSCRIBE_CALLBACK = ctypes.WINFUNCTYPE(
61     wintypes.DWORD,
62     wintypes.DWORD,
63     wintypes.LPVOID,
64     wintypes.HANDLE
65 )
66 CHANNEL_DESCRIPTIONS: Dict[str, str] = {'System': 'Системный ж
урнал Windows: события ядра ОС, системных служб, драйверов уст

```

ройств, сбоев питания и сетевого стека.', 'Application': 'Журнал приложений: события, предупреждения, сбои и ошибки пользовательских и серверных программ (Crash, Faults).', 'Security': 'Журнал безопасности: аудит входов в систему (Logon/Logoff), права доступа, повышение привилегий UAC и политики безопасности.', 'Setup': 'Журнал установки: логи процесса установки операционной системы, обновлений Windows и накопительных пакетов KB.', 'Microsoft-Windows-WindowsUpdateClient/Operational': 'Клиент обновления Windows Update: загрузка, верификация и установка патчей, исправления безопасности и драйверов.', 'Microsoft-Windows-Kernel-PnP/Configuration': 'Конфигурация оборудования Plug and Play: подключение новых устройств, установка драйверов, коды ошибок 10/43.', 'Microsoft-Windows-Kernel-Power/Operational': 'Питание ядра Windows: переходы в сон/гибернацию, сбои питания (Event 41), изменение состояний батареи и ACPI.', 'Microsoft-Windows-TaskScheduler/Operational': 'Планировщик заданий: запуск, завершение, сбои и выполнение фоновых регламентных задач Windows.', 'Microsoft-Windows-Windows Defender/Operational': 'Антивирус Microsoft Defender: обнаружение угроз, сканирование в реальном времени, обновление антивирусных сигнатур.', 'Microsoft-Windows-Hyper-V-Compute-Operational': 'Платформа виртуализации Hyper-V: жизненный цикл виртуальных машин, контейнеров WSL2 и виртуальных адаптеров.', 'Microsoft-Windows-Windows Firewall With Advanced Security/Firewall': 'Брандмауэр Windows: блокировки входящего/исходящего трафика, сетевые правила фильтрации и аномалии сокетов.', 'Microsoft-Windows-Diagnostics-Performance/Operational': 'Диагностика производительности: анализ времени загрузки Windows, выключения и зависания системных компонентов.', 'Microsoft-Windows-Bits-Client/Operational': 'Фоновая интеллектуальная служба передачи (BITS): загрузка файлов и обновлений по сети с контролем канала.', 'Microsoft-Windows-DNS-Client/Operational': 'DNS-клиент Windows: резолвинг доменных имен, таймауты DNS серверов и сетевые кэширования.', 'Microsoft-Windows-CodeIntegrity/Operational': 'Контроль целостности кода: проверка цифровых подписей драйверов и исполняемых системных файлов.', 'Microsoft-Windows-PowerShell/Operational': 'Исполнение скриптов PowerShell: запуск командлетов, сценариев автоматизации и модулей администрирования.', 'Microsoft-Windo

```
ws-Sysmon/Operational': 'Microsoft System Monitor (Sysmon): гл  
убокая телеметрия создания процессов (Event 1), сети (Event 3)  
, файловых операций (11/23/26) и хэшей исполняемых файлов.'}
```

67

```
68 @dataclass
```

```
69 class ChannelMetadata:
```

```
70     """Метаданные о канале журнала Windows."""
```

```
71     channel_name: str
```

```
72     display_name: str
```

```
73     description: str = ''
```

```
74     is_enabled: bool = True
```

```
75     record_count: int = 0
```

```
76     category: str = 'Windows Event Log'
```

77

```
78 class WevtAPI:
```

```
79     """Ctypes-обертка для wevtapi.dll и реестра Windows Event  
Log."""
```

80

```
81     def __init__(self) -> None:
```

```
82         """Инициализация библиотеки wevtapi.dll."""
```

```
83         self._available = False
```

```
84         self.wevtapi: Optional[ctypes.WinDLL] = None
```

```
85         self._publisher_cache: Dict[str, Optional[wintypes.HAN  
DLE]] = {}
```

```
86         self._active_subscriptions: List[Tuple[wintypes.HANDLE  
, Any]] = []
```

```
87         if sys.platform == 'win32':
```

```
88             try:
```

```
89                 self.wevtapi = ctypes.windll.LoadLibrary('wevt  
api.dll')
```

```
90                 self._setup_signatures()
```

```
91                 self._available = True
```

```
92             except Exception as ex:
```

```
93                 logger.debug(f'[WevtAPI] Не удалось загрузить  
wevtapi.dll: {ex}')
```

94

```
95     def _setup_signatures(self) -> None:
```

```
96         """Настройка типов аргументов и возвращаемых значений
```

```

    функций. """
97         if not self.wevtapi:
98             return
99         self.wevtapi.EvtOpenChannelEnum.argtypes = [wintypes.H
HANDLE, wintypes.DWORD]
100         self.wevtapi.EvtOpenChannelEnum.restype = wintypes.HAN
DLE
101         self.wevtapi.EvtNextChannelPath.argtypes = [wintypes.H
HANDLE, wintypes.DWORD, wintypes.LPCWSTR, ctypes.POINTER(wintype
s.DWORD)]
102         self.wevtapi.EvtNextChannelPath.restype = wintypes.BOOL
L
103         self.wevtapi.EvtClose.argtypes = [wintypes.HANDLE]
104         self.wevtapi.EvtClose.restype = wintypes.BOOL
105         self.wevtapi.EvtOpenLog.argtypes = [wintypes.HANDLE, w
intypes.LPCWSTR, wintypes.DWORD]
106         self.wevtapi.EvtOpenLog.restype = wintypes.HANDLE
107         self.wevtapi.EvtGetLogInfo.argtypes = [wintypes.HANDLE
, wintypes.DWORD, wintypes.DWORD, ctypes.c_void_p, ctypes.POIN
TER(wintypes.DWORD)]
108         self.wevtapi.EvtGetLogInfo.restype = wintypes.BOOL
109         self.wevtapi.EvtQuery.argtypes = [wintypes.HANDLE, win
types.LPCWSTR, wintypes.LPCWSTR, wintypes.DWORD]
110         self.wevtapi.EvtQuery.restype = wintypes.HANDLE
111         self.wevtapi.EvtNext.argtypes = [wintypes.HANDLE, wint
ypes.DWORD, ctypes.POINTER(wintypes.HANDLE), wintypes.DWORD, w
intypes.DWORD, ctypes.POINTER(wintypes.DWORD)]
112         self.wevtapi.EvtNext.restype = wintypes.BOOL
113         self.wevtapi.EvtRender.argtypes = [wintypes.HANDLE, wi
ntypes.HANDLE, wintypes.DWORD, wintypes.DWORD, wintypes.LPVOID
, ctypes.POINTER(wintypes.DWORD), ctypes.POINTER(wintypes.DWOR
D)]
114         self.wevtapi.EvtRender.restype = wintypes.BOOL
115         self.wevtapi.EvtOpenPublisherMetadata.argtypes = [wint
ypes.HANDLE, wintypes.LPCWSTR, wintypes.LPCWSTR, wintypes.DWOR
D, wintypes.DWORD]
116         self.wevtapi.EvtOpenPublisherMetadata.restype = wintyp
es.HANDLE

```

```

117         self.wevtapi.EvtFormatMessage.argtypes = [wintypes.HAN
HANDLE, wintypes.HANDLE, wintypes.DWORD, wintypes.DWORD, ctypes.c
_void_p, wintypes.DWORD, wintypes.DWORD, wintypes.LPWSTR, ctyp
es.POINTER(wintypes.DWORD)]
118         self.wevtapi.EvtFormatMessage.restype = wintypes.BOOL
119         self.wevtapi.EvtCreateBookmark.argtypes = [wintypes.LP
CWSTR]
120         self.wevtapi.EvtCreateBookmark.restype = wintypes.HAND
LE
121         self.wevtapi.EvtUpdateBookmark.argtypes = [wintypes.HA
NDLE, wintypes.HANDLE]
122         self.wevtapi.EvtUpdateBookmark.restype = wintypes.BOOL
123         self.wevtapi.EvtSeek.argtypes = [wintypes.HANDLE, ctyp
es.c_int64, wintypes.HANDLE, wintypes.DWORD, wintypes.DWORD]
124         self.wevtapi.EvtSeek.restype = wintypes.BOOL
125         self.wevtapi.EvtSubscribe.argtypes = [wintypes.HANDLE,
wintypes.HANDLE, wintypes.LPCWSTR, wintypes.LPCWSTR, wintypes
.HANDLE, wintypes.LPVOID, EVT_SUBSCRIBE_CALLBACK, wintypes.DWO
RD]
126         self.wevtapi.EvtSubscribe.restype = wintypes.HANDLE
127
128     def is_available(self) -> bool:
129         """Проверить доступность wevtapi.dll в текущей системе
. """
130         return self._available
131
132     def get_channel_record_count(self, channel_name: str) -> i
nt:
133         """Мгновенно получить точное количество записей в кана
ле через EvtGetLogInfo."""
134         if not self._available or not self.wevtapi:
135             return 0
136         h_log = self.wevtapi.EvtOpenLog(None, channel_name, 1)
137         if not h_log:
138             return 0
139         try:
140             buf = (ctypes.c_byte * 128)()
141             used = wintypes.DWORD(0)

```

```
142         if self.wevtapi.EvtGetLogInfo(h_log, 5, 128, ctypes
| s.byref(buf), ctypes.byref(used)):
143             return struct.unpack('<Q', bytes(buf)[:8])[0]
144         except Exception:
145             pass
146         finally:
147             self.wevtapi.EvtClose(h_log)
148         return 0
149
150     def get_channel_description(self, channel_name: str) -> str:
151         """Получить человекочитаемое описание назначения канала."""
152         if channel_name in CHANNEL_DESCRIPTIONS:
153             return CHANNEL_DESCRIPTIONS[channel_name]
154         if 'Defender' in channel_name:
155             return 'Журнал защитных механизмов, антивирусного
| сканирования и предотвращения вторжений Microsoft Defender.'
156         if 'Kernel-Power' in channel_name:
157             return 'Журнал управления электропитанием, состоян
| ий ACPI, спящего режима и аварийных отключений питания.'
158         if 'Kernel-PnP' in channel_name:
159             return 'Журнал конфигурации оборудования, обнаруже
| ния и инициализации устройств Plug and Play.'
160         if 'WindowsUpdate' in channel_name:
161             return 'Журнал службы обновлений Windows, загрузки
| пакетов KB и применения хотфиксов.'
162         if 'TaskScheduler' in channel_name:
163             return 'Журнал планировщика задач Windows, запуска
| автоматических фоновых процессов и триггеров.'
164         if 'Hyper-V' in channel_name:
165             return 'Журнал подсистемы виртуализации Hyper-V, W
| SL2 контейнеров и гостевых служб.'
166         if 'PowerShell' in channel_name:
167             return 'Журнал среды PowerShell, выполнения скрипт
| ов автоматизации и команд администрирования.'
168         if 'Firewall' in channel_name:
169             return 'Журнал сетевого брандмауэра Windows, прави
```

```

    л сетевого экрана и событий фильтрации трафика.'
170         if 'Storage' in channel_name or 'Disk' in channel_name
    :
171         return 'Журнал дисковой подсистемы, контроллеров н
    акопителей, томов NTFS/ReFS и файловой системы.'
172         if 'Network' in channel_name or 'Wlan' in channel_name
    or 'NDIS' in channel_name:
173         return 'Журнал сетевых интерфейсов, Wi-Fi соединен
    ий, сетевых адаптеров и стека TCP/IP.'
174         if 'Audio' in channel_name or 'Sound' in channel_name:
175         return 'Журнал аудиоподсистемы, службы Windows Aud
    io и конечных звуковых устройств.'
176         if 'Print' in channel_name or 'Spooler' in channel_nam
    e:
177         return 'Журнал диспетчера очереди печати и драйвер
    ов принтеров.'
178         if 'Bluetooth' in channel_name or 'BTH' in channel_nam
    e:
179         return 'Журнал стека Bluetooth, сопряжения беспров
    одных устройств и протоколов передачи данных.'
180         if 'USB' in channel_name:
181         return 'Журнал контроллеров USB, хабов и подключен
    ных внешних периферийных устройств.'
182         return f"Специализированный системный журнал Windows д
    ля компонента '{channel_name}'."
183
184     def enumerate_channels(self) -> List[ChannelMetadata]:
185         """Получить исчерпывающий список всех каналов Windows
    Event Log с количеством записей."""
186         channels: List[ChannelMetadata] = []
187         seen: set[str] = set()
188         classic = [('System', 'System (Системный)'), ('Applica
    tion', 'Application (Приложения)'), ('Security', 'Security (Бе
    зопасность)'), ('Setup', 'Setup (Установка)'), ('Microsoft-Win
    dows-WindowsUpdateClient/Operational', 'Windows Update Client'
    ), ('Microsoft-Windows-Kernel-PnP/Configuration', 'Kernel PnP
    Configuration'), ('Microsoft-Windows-Kernel-Power/Operational'
    , 'Kernel Power'), ('Microsoft-Windows-TaskScheduler/Operation

```

```

al', 'Task Scheduler'), ('Microsoft-Windows-Windows Defender/O
perational', 'Windows Defender'), ('Microsoft-Windows-Hyper-V-
Compute-Operational', 'Hyper-V Compute'), ('Microsoft-Windows-
PowerShell/Operational', 'Windows PowerShell')]
189         for cname, dname in classic:
190             rec_cnt = self.get_channel_record_count(cname)
191             channels.append(ChannelMetadata(channel_name=cname
, display_name=dname, description=self.get_channel_description
(cname), record_count=rec_cnt, is_enabled=True))
192             seen.add(cname.lower())
193             if self._available and self.wevtapi:
194                 try:
195                     h_enum = self.wevtapi.EvtOpenChannelEnum(None,
0)
196                     if h_enum:
197                         buffer_size = 512
198                         buf = ctypes.create_unicode_buffer(buffer_
size)
199                         used = wintypes.DWORD()
200                         while True:
201                             success = self.wevtapi.EvtNextChannelP
ath(h_enum, buffer_size, buf, ctypes.byref(used))
202                             if not success:
203                                 err = ctypes.GetLastError()
204                                 if err == 122:
205                                     buffer_size = used.value + 10
206                                     buf = ctypes.create_unicode_bu
ffer(buffer_size)
207                                     continue
208                                     break
209                                     chan_str = buf.value.strip()
210                                     if chan_str and chan_str.lower() not i
n seen:
211                                         rec_cnt = self.get_channel_record_
count(chan_str)
212                                         channels.append(ChannelMetadata(ch
annel_name=chan_str, display_name=chan_str, description=self.g
et_channel_description(chan_str), record_count=rec_cnt, is_ena

```



```

        bled=True))
213 |                 seen.add(chan_str.lower())
214 |                 self.wevtapi.EvtClose(h_enum)
215 |             except Exception as ex:
216 |                 logger.debug(f'[WevtAPI] Ошибка при перечислен
ии через EvtOpenChannelEnum: {ex}')
217 |             try:
218 |                 reg_channels = self._enumerate_registry_channels()
219 |                 for r_chan in reg_channels:
220 |                     if r_chan.lower() not in seen:
221 |                         rec_cnt = self.get_channel_record_count(r_
chan)
222 |                         channels.append(ChannelMetadata(channel_na
me=r_chan, display_name=r_chan, description=self.get_channel_d
escription(r_chan), record_count=rec_cnt, is_enabled=True))
223 |                         seen.add(r_chan.lower())
224 |             except Exception as ex:
225 |                 logger.debug(f'[WevtAPI] Ошибка чтения каналов из
реестра: {ex}')
226 |             return channels
227 |
228 |     def _enumerate_registry_channels(self) -> List[str]:
229 |         """Считать имена зарегистрированных каналов из реестра
Windows."""
230 |         res: List[str] = []
231 |         key_path = 'SOFTWARE\\Microsoft\\Windows\\CurrentVersi
on\\WINEVT\\Channels'
232 |         try:
233 |             with winreg.OpenKey(winreg.HKEY_LOCAL_MACHINE, key
_path, 0, winreg.KEY_READ | winreg.KEY_WOW64_64KEY) as key:
234 |                 num_subkeys, _, _ = winreg.QueryInfoKey(key)
235 |                 for i in range(num_subkeys):
236 |                     try:
237 |                         subkey_name = winreg.EnumKey(key, i)
238 |                         if subkey_name:
239 |                             res.append(subkey_name)
240 |             except OSError:
241 |                 break

```

```

242         except Exception:
243             pass
244         return res
245
246     def read_events(self, channel: str='System', limit: int=10
247 0, level: str='', search: str='', event_id: int | list[int] |
248 tuple[int, ...] | set[int] = 0, hours: int=24, format_message:
249 bool=True) -> List[Dict[str, Any]]:
250         """Быстрое чтение событий из указанного канала через E
251 vtQuery / EvtRender."""
252         if not self._available or not self.wevtapi:
253             return []
254         limit = max(1, min(limit, 2000))
255         xpath_parts: List[str] = []
256         if level:
257             lvl_lower = level.lower()
258             if 'crit' in lvl_lower:
259                 xpath_parts.append('Level=1')
260             elif 'err' in lvl_lower:
261                 xpath_parts.append('Level=2')
262             elif 'warn' in lvl_lower:
263                 xpath_parts.append('Level=3')
264             elif 'inf' in lvl_lower:
265                 xpath_parts.append('Level=4')
266             elif 'verb' in lvl_lower:
267                 xpath_parts.append('Level=5')
268         if isinstance(event_id, (list, tuple, set)):
269             valid_eids = [e for e in event_id if isinstance(e,
270 int) and e > 0]
271             if valid_eids:
272                 clauses = ' or '.join(f'EventID={eid}' for eid
273 in valid_eids)
274                 xpath_parts.append(f'({clauses})')
275             elif isinstance(event_id, int) and event_id > 0:
276                 xpath_parts.append(f'EventID={event_id}')
277         if xpath_parts:
278             clause = ' and '.join(xpath_parts)
279             query_xpath = f'*[System[{clause}]]'

```

```

274         else:
275             query_xpath = '*'
276             events: List[Dict[str, Any]] = []
277             try:
278                 h_query = self.wevtapi.EvtQuery(None, channel, query_xpath, EVT_QUERY_CHANNEL_PATH | EVT_QUERY_REVERSE_DIRECTION)
279                 if not h_query:
280                     return []
281                 try:
282                     event_handles = (wintypes.HANDLE * limit)()
283                     returned = wintypes.DWORD(0)
284                     if self.wevtapi.EvtNext(h_query, limit, event_handles, 2000, 0, ctypes.byref(returned)):
285                         for i in range(returned.value):
286                             h_ev = event_handles[i]
287                             if h_ev:
288                                 ev_dict = self._render_event_xml(h_ev, channel, format_message=format_message)
289                                 if ev_dict:
290                                     if search:
291                                         s_lower = search.lower()
292                                         msg = str(ev_dict.get('message', '')).lower()
293                                         prov = str(ev_dict.get('provider', '')).lower()
294                                         eid = str(ev_dict.get('event_id', ''))
295                                         if s_lower not in msg and s_lower not in prov and (s_lower not in eid):
296                                             self.wevtapi.EvtClose(h_ev)
297                                     continue
298                                 events.append(ev_dict)
299                                 self.wevtapi.EvtClose(h_ev)
300                 finally:
301                     self.wevtapi.EvtClose(h_query)
302             except Exception as ex:

```

```

303         logger.debug(f'[WevtAPI] Ошибка при чтении событий
    ({channel}): {ex}')
304         return events
305
306     def _get_publisher_metadata(self, provider_name: str) -> Optional[wintypes.HANDLE]:
307         """Получить кешированный дескриптор метаданных провайдера событий."""
308         if not self._available or not self.wevtapi or not provider_name:
309             return None
310         if provider_name in self._publisher_cache:
311             return self._publisher_cache[provider_name]
312         try:
313             h_pub = self.wevtapi.EvtOpenPublisherMetadata(None, provider_name, None, 0, 0)
314             if h_pub:
315                 self._publisher_cache[provider_name] = h_pub
316                 return h_pub
317             self._publisher_cache[provider_name] = None
318         except Exception:
319             self._publisher_cache[provider_name] = None
320         return None
321
322     def format_event_message(self, h_event: wintypes.HANDLE, provider_name: str, flag: int = 1) -> Optional[str]:
323         """Сформировать нативное локализованное системное сообщение события через EvtFormatMessage.
324
325         Args:
326             h_event: Дескриптор события Windows.
327             provider_name: Имя издателя/провайдера (например, 'Microsoft-Windows-Kernel-Power').
328             flag: Флаг форматирования (1 = EvtFormatMessageEvent, 2 = Level, 3 = Task).
329
330         Returns:
331             Optional[str]: Отформатированная локализованная ст

```

```

    пока сообщения или None.
332     """
333     if not self._available or not self.wevtapi or not h_event or not provider_name:
334         return None
335     h_pub = self._get_publisher_metadata(provider_name)
336     if not h_pub:
337         return None
338     try:
339         used = wintypes.DWORD()
340         self.wevtapi.EvtFormatMessage(h_pub, h_event, 0, 0
, None, flag, 0, None, ctypes.byref(used))
341         err = ctypes.GetLastError()
342         if err != 122 or used.value == 0:
343             return None
344         buf = ctypes.create_unicode_buffer(used.value + 2)
345         if self.wevtapi.EvtFormatMessage(h_pub, h_event, 0
, 0, None, flag, used.value, buf, ctypes.byref(used)):
346             val = buf.value
347             if val:
348                 return val.strip()
349     except Exception:
350         pass
351     return None
352
353     def close(self) -> None:
354         """Освободить все открытые дескрипторы издателей."""
355         if self._available and self.wevtapi:
356             for h in list(self._publisher_cache.values()):
357                 if h:
358                     try:
359                         self.wevtapi.EvtClose(h)
360                     except Exception:
361                         pass
362             self._publisher_cache.clear()
363
364     def __del__(self) -> None:
365         """Деструктор для гарантированного освобождения ресурс

```

```

        об. """
366         self.close()
367
368     def _render_event_xml(self, h_event: wintypes.HANDLE, chan
nel: str, format_message: bool = True) -> Optional[Dict[str, A
ny]]:
369         """Отрендерить событие в XML строку и разобрать в слов
арь."""
370         if not self.wevtapi:
371             return None
372         used = wintypes.DWORD()
373         prop_count = wintypes.DWORD()
374         self.wevtapi.EvtRender(None, h_event, EVT_RENDER_EVENT
_XML, 0, None, ctypes.byref(used), ctypes.byref(prop_count))
375         err = ctypes.GetLastError()
376         if err != 122:
377             return None
378         buf = ctypes.create_unicode_buffer(used.value // 2 + 2
)
379         if not self.wevtapi.EvtRender(None, h_event, EVT_RENDE
R_EVENT_XML, used.value, ctypes.byref(buf), ctypes.byref(used)
, ctypes.byref(prop_count)):
380             return None
381         xml_str = buf.value
382         if not xml_str:
383             return None
384         res = self._parse_event_xml(xml_str, channel)
385         if res and format_message:
386             prov = res.get('provider', '')
387             if prov:
388                 formatted = self.format_event_message(h_event,
prov, flag=1)
389                 if formatted:
390                     res['formatted_message'] = formatted
391                     res['message'] = formatted
392         return res
393
394     def _parse_event_xml(self, xml_str: str, channel: str) ->

```

```

Dict[str, Any]:
395     """Разобрать XML представление Windows события в нормализованную структуру."""
396     res: Dict[str, Any] = {'timestamp': '', 'level': 'Information', 'event_id': 0, 'provider': '', 'computer': '', 'process_id': 0, 'thread_id': 0, 'channel': channel, 'message': '', 'raw_data': xml_str}
397     try:
398         clean_xml = ET.fromstring(xml_str)
399         system_elem = clean_xml.find('{http://schemas.microsoft.com/win/2004/08/events/event}System')
400         if system_elem is None:
401             system_elem = clean_xml.find('System')
402         if system_elem is not None:
403             for child in system_elem:
404                 tag = child.tag.split('}')[1]
405                 if tag == 'TimeCreated':
406                     raw_time = child.attrib.get('SystemTime', '')
407                     if raw_time:
408                         res['timestamp'] = raw_time.replace('T', ' ').split('.')[0]
409                     elif tag == 'EventID':
410                         try:
411                             res['event_id'] = int(child.text or '0')
412                         except ValueError:
413                             res['event_id'] = 0
414                     elif tag == 'EventRecordID':
415                         try:
416                             res['record_id'] = int(child.text or '0')
417                         except ValueError:
418                             res['record_id'] = 0
419                     elif tag == 'Level':
420                         lvl_map = {'1': 'Critical', '2': 'Error', '3': 'Warning', '4': 'Information', '5': 'Verbose'}
421                         res['level'] = lvl_map.get(str(child.t

```

```

    ext or '').strip(), 'Information')
422         elif tag == 'Provider':
423             res['provider'] = child.attrib.get('Name', '')
424         elif tag == 'Computer':
425             res['computer'] = child.text or ''
426         elif tag == 'Execution':
427             try:
428                 res['process_id'] = int(child.attrib.get('ProcessID', 0))
429                 res['thread_id'] = int(child.attrib.get('ThreadID', 0))
430             except ValueError:
431                 pass
432         elif tag == 'Channel':
433             if child.text:
434                 res['channel'] = child.text
435     data_dict: Dict[str, str] = {}
436     data_texts: List[str] = []
437     for d in clean_xml.iter():
438         tag = d.tag.split(':')[0]
439         if tag in ('Data', 'string', 'Value') and d.text:
440             t = d.text.strip()
441             name_attr = d.attrib.get('Name')
442             if name_attr:
443                 data_dict[name_attr] = t
444                 data_texts.append(f'{name_attr}: {t}')
445             elif t:
446                 data_texts.append(t)
447     res['event_data'] = data_dict
448     if data_texts:
449         res['message'] = ' | '.join(data_texts[:12])
450     else:
451         res['message'] = f"Event {res['event_id']} from {res['provider']}"
452     except Exception:
453         res['message'] = f'Event recorded in {channel}'

```



```

454         return res
455
456     def query_process_events(self, limit: int=100, filter_proc
ess: Optional[str]=None, filter_user: Optional[str]=None) -> L
ist[Dict[str, Any]]:
457         """
458         Запросить структурированные события создания процессов
из Sysmon (Event 1)
459         или Windows Security Log (Event 4688).
460
461         Args:
462             limit: Максимальное количество событий.
463             filter_process: Фильтр по имени/пути исполняемого
файла.
464             filter_user: Фильтр по имени учетной записи пользо
вателя.
465
466         Returns:
467             Список нормализованных словарей с информацией о за
пусках процессов.
468         """
469         results: List[Dict[str, Any]] = []
470         sysmon_events = self.read_events(channel='Microsoft-Wi
ndows-Sysmon/Operational', limit=limit, event_id=1)
471         for ev in sysmon_events:
472             ed = ev.get('event_data', {})
473             exe_path = ed.get('Image', '')
474             cmd_line = ed.get('CommandLine', '')
475             user = ed.get('User', '')
476             pid = int(ed.get('ProcessId', 0) or 0)
477             ppid = int(ed.get('ParentProcessId', 0) or 0)
478             parent_exe = ed.get('ParentImage', '')
479             parent_cmd = ed.get('ParentCommandLine', '')
480             hashes = ed.get('Hashes', '')
481             guid = ed.get('ProcessGuid', '')
482             parent_guid = ed.get('ParentProcessGuid', '')
483             if filter_process and filter_process.lower() not i
n exe_path.lower() and (filter_process.lower() not in cmd_line

```

```

        .lower()):
484             continue
485         if filter_user and filter_user.lower() not in user
        .lower():
486             continue
487         results.append({'source': 'Sysmon (Event 1)', 'tim
estamp': ev.get('timestamp'), 'event_id': 1, 'process_id': pid
, 'process_name': exe_path.replace('\\', '/').split('/')[-1] i
f exe_path else '', 'executable_path': exe_path, 'command_line
': cmd_line, 'user': user, 'parent_process_id': ppid, 'parent_
process_name': parent_exe.replace('\\', '/').split('/')[-1] if
parent_exe else '', 'parent_executable_path': parent_exe, 'pa
rent_command_line': parent_cmd, 'process_guid': guid, 'parent_
process_guid': parent_guid, 'hashes': hashes, 'raw_event': ev}
    )
488         if len(results) < limit:
489             remaining = limit - len(results)
490             sec_events = self.read_events(channel='Security',
limit=remaining, event_id=4688)
491             for ev in sec_events:
492                 ed = ev.get('event_data', {})
493                 exe_path = ed.get('NewProcessName', '')
494                 cmd_line = ed.get('CommandLine', '')
495                 user = ed.get('SubjectUserName', '')
496                 raw_pid = ed.get('NewProcessId', '0')
497                 try:
498                     pid = int(raw_pid, 16) if str(raw_pid).sta
rtswith('0x') else int(raw_pid or 0)
499                 except ValueError:
500                     pid = 0
501                 raw_ppid = ed.get('ProcessId', '0')
502                 try:
503                     ppid = int(raw_ppid, 16) if str(raw_ppid).
startswith('0x') else int(raw_ppid or 0)
504                 except ValueError:
505                     ppid = 0
506                 parent_exe = ed.get('ParentProcessName', '')
507                 if filter_process and filter_process.lower() n

```

```

ot in exe_path.lower() and (filter_process.lower() not in cmd_
line.lower())):
508         continue
509         if filter_user and filter_user.lower() not in
user.lower():
510             continue
511             results.append({'source': 'Security Audit (Eve
nt 4688)', 'timestamp': ev.get('timestamp'), 'event_id': 4688,
'process_id': pid, 'process_name': exe_path.replace('\\', '/')
).split('/')[-1] if exe_path else '', 'executable_path': exe_p
ath, 'command_line': cmd_line, 'user': user, 'parent_process_i
d': ppid, 'parent_process_name': parent_exe.replace('\\', '/')
.split('/')[-1] if parent_exe else '', 'parent_executable_path
': parent_exe, 'parent_command_line': '', 'process_guid': '',
'parent_process_guid': '', 'hashes': '', 'raw_event': ev})
512         return results
513
514     def query_powershell_script_blocks(self, limit: int = 100,
filter_text: Optional[str] = None) -> List[Dict[str, Any]]:
515         """Запросить блоки скриптов PowerShell (Script Block L
ogging, Event ID 4104).
516
517         Args:
518             limit: Максимальное количество записей.
519             filter_text: Подстрока для поиска в теле скрипта и
ли пути.
520
521         Returns:
522             List[Dict[str, Any]]: Нормализованные события с бл
оками кода скриптов.
523         """
524         results: List[Dict[str, Any]] = []
525         ps_events = self.read_events(channel='Microsoft-Window
s-PowerShell/Operational', limit=limit, event_id=4104)
526         for ev in ps_events:
527             ed = ev.get('event_data', {})
528             script_text = ed.get('ScriptBlockText', '') or ev.
get('message', '')

```

```

529         script_id = ed.get('ScriptBlockId', '')
530         path = ed.get('Path', '')
531         user_id = ed.get('UserId', '')
532         msg_total = ed.get('MessageTotal', '1')
533         msg_number = ed.get('MessageNumber', '1')
534
535         if filter_text and filter_text.lower() not in script_text.lower() and (filter_text.lower() not in path.lower()):
536             continue
537
538         results.append({
539             'source': 'PowerShell ScriptBlock (Event 4104)
540             ',
541             'timestamp': ev.get('timestamp'),
542             'event_id': 4104,
543             'script_block_id': script_id,
544             'script_text': script_text,
545             'path': path,
546             'user_id': user_id,
547             'message_number': int(msg_number) if str(msg_number).isdigit() else 1,
548             'message_total': int(msg_total) if str(msg_total).isdigit() else 1,
549             'raw_event': ev,
550         })
551     return results
552
553     def query_sysmon_events(
554         self,
555         limit: int = 100,
556         event_ids: Optional[List[int]] = None,
557         filter_process: Optional[str] = None,
558     ) -> List[Dict[str, Any]]:
559         """Запросить события безопасности Sysmon (Event ID 1,
560         7, 10, 11, 23, 26).
561
562         Args:
563             limit: Максимальное количество записей.

```

```

562         event_ids: Список опрашиваемых идентификаторов со-
563         бытий (по умолчанию: 1, 7, 10, 11, 23, 26).
564
565         filter_process: Фильтр по имени/пути исполняемого
566         файла или модуля.
567
568     Returns:
569         List[Dict[str, Any]]: Нормализованные события Sysmon.
570
571     """
572     target_ids = event_ids or [1, 7, 10, 11, 23, 26]
573     all_events: List[Dict[str, Any]] = []
574
575     for eid in target_ids:
576         evs = self.read_events(channel='Microsoft-Windows-
577         Sysmon/Operational', limit=limit, event_id=eid)
578
579         for ev in evs:
580             ed = ev.get('event_data', {})
581             img = ed.get('Image', '') or ed.get('SourceImage', '')
582
583             if filter_process and filter_process.lower() not in img.lower() and (filter_process.lower() not in str(ed).lower()):
584                 continue
585
586             item: Dict[str, Any] = {
587                 'source': f'Sysmon (Event {eid})',
588                 'timestamp': ev.get('timestamp'),
589                 'event_id': eid,
590                 'process_id': int(ed.get('ProcessId', 0) or ed.get('SourceProcessId', 0) or 0),
591                 'image': img,
592                 'process_guid': ed.get('ProcessGuid') or ed.get('SourceProcessGuid', ''),
593                 'user': ed.get('User', ''),
594                 'raw_event': ev,
595             }
596
597             if eid == 1:

```

```

591         item.update({
592             'event_type': 'ProcessCreate',
593             'command_line': ed.get('CommandLine',
    ''),
594             'parent_process_id': int(ed.get('ParentProcessId', 0) or 0),
595             'parent_image': ed.get('ParentImage',
    ''),
596             'hashes': ed.get('Hashes', ''),
597         })
598     elif eid == 7:
599         item.update({
600             'event_type': 'ImageLoaded',
601             'image_loaded': ed.get('ImageLoaded',
    ''),
602             'signed': ed.get('Signed', ''),
603             'signature': ed.get('Signature', ''),
604             'hashes': ed.get('Hashes', ''),
605         })
606     elif eid == 10:
607         item.update({
608             'event_type': 'ProcessAccess',
609             'target_process_id': int(ed.get('TargetProcessId', 0) or 0),
610             'target_image': ed.get('TargetImage',
    ''),
611             'granted_access': ed.get('GrantedAccess', ''),
612             'call_trace': ed.get('CallTrace', ''),
613         })
614     elif eid in (11, 23, 26):
615         item.update({
616             'event_type': 'FileDelete' if eid in (
    23, 26) else 'FileCreate',
617             'target_filename': ed.get('TargetFilename', ''),
618             'hashes': ed.get('Hashes', ''),
619         })

```

```

620
621         all_events.append(item)
622
623         all_events.sort(key=lambda x: str(x.get('timestamp', '
|')), reverse=True)
624         return all_events[:limit]
625
626     def create_bookmark(self, bookmark_xml: Optional[str] = No
|ne) -> Optional[wintypes.HANDLE]:
627         """Создать дескриптор закладки (Bookmark) для инкремен
|тального позиционирования.
628
629         Args:
630             bookmark_xml: Исходная XML-строка закладки или Non
|e для новой пустой закладки.
631
632         Returns:
633             Optional[wintypes.HANDLE]: Дескриптор закладки или
|None.
634         """
635         if not self._available or not self.wevtapi:
636             return None
637         try:
638             h_bookmark = self.wevtapi.EvtCreateBookmark(bookma
|rk_xml if bookmark_xml else None)
639             return h_bookmark if h_bookmark else None
640         except Exception as ex:
641             logger.debug(f'[WevtAPI] Ошибка создания закладки:
|{ex}')
642             return None
643
644     def update_bookmark(self, bookmark_handle: wintypes.HANDLE
|, event_handle: wintypes.HANDLE) -> bool:
645         """Обновить позицию закладки по текущему событию.
646
647         Args:
648             bookmark_handle: Дескриптор закладки.
649             event_handle: Дескриптор обработанного события.

```

```

650
651     Returns:
652         bool: True в случае успешного обновления.
653     """
654     if not self._available or not self.wevtapi or not bookmark_handle or not event_handle:
655         return False
656     try:
657         return bool(self.wevtapi.EvtUpdateBookmark(bookmark_handle, event_handle))
658     except Exception as ex:
659         logger.debug(f'[WevtAPI] Ошибка обновления закладк и: {ex}')
660         return False
661
662     def render_bookmark(self, bookmark_handle: wintypes.HANDLE) -> Optional[str]:
663         """Отрендерить текущее состояние закладки в XML строку
664         .
665     Args:
666         bookmark_handle: Дескриптор закладки.
667
668     Returns:
669         Optional[str]: XML-представление закладки для сохранения в SQLite/файле.
670     """
671     if not self._available or not self.wevtapi or not bookmark_handle:
672         return None
673     used = wintypes.DWORD()
674     prop_count = wintypes.DWORD()
675     self.wevtapi.EvtRender(None, bookmark_handle, EVT_RENDER_BOOKMARK_XML, 0, None, ctypes.byref(used), ctypes.byref(prop_count))
676     err = ctypes.GetLastError()
677     if err != 122 or used.value == 0:
678         return None

```



```

679         buf = ctypes.create_unicode_buffer(used.value // 2 + 2
        )
680         if self.wevtapi.EvtRender(None, bookmark_handle, EVT_R
        ENDER_BOOKMARK_XML, used.value, ctypes.byref(buf), ctypes.byre
        f(used), ctypes.byref(prop_count)):
681             return buf.value.strip()
682         return None
683
684     def seek_bookmark(self, h_query: wintypes.HANDLE, bookmark
        _handle: wintypes.HANDLE, flags: int = EVT_SEEK_AFTER_BOOKMARK
        ) -> bool:
685         """Переместить указатель чтения в выборке относительно
        сохраненной закладки.
686
687         Args:
688             h_query: Дескриптор открытой выборки EvtQuery.
689             bookmark_handle: Дескриптор закладки.
690             flags: Флаги позиционирования (по умолчанию EVT_SE
        EK_AFTER_BOOKMARK).
691
692         Returns:
693             bool: True в случае успешного позиционирования.
694             ""
695         if not self._available or not self.wevtapi or not h_qu
        ery or not bookmark_handle:
696             return False
697         try:
698             return bool(self.wevtapi.EvtSeek(h_query, 0, bookm
        ark_handle, 0, flags))
699         except Exception as ex:
700             logger.debug(f'[WevtAPI] Ошибка позиционирования п
        о закладке: {ex}')
701             return False
702
703     def read_events_incremental(
704         self,
705         channel: str = 'Security',
706         query_xpath: str = '*',

```

```

707         bookmark_xml: Optional[str] = None,
708         batch_size: int = 100,
709         format_message: bool = False,
710         forward: bool = True,
711     ) -> Tuple[List[Dict[str, Any]], Optional[str], int]:
712         """Инкрементальное чтение событий из журнала с использо-
713         ванием закладки (Bookmark).
714
715         Args:
716             channel: Имя канала Windows Event Log (например, '
717             Security', 'System').
718             query_xpath: XPath-фильтр для выборки событий.
719             bookmark_xml: XML сохраненной закладки с прошлого
720             сеанса сбора.
721             batch_size: Максимальное количество событий за оди-
722             н опрос.
723             format_message: Форматировать ли локализованные ст-
724             роки через publisher metadata.
725             forward: Читать вперед по хронологии (True) или на-
726             зад (False).
727
728         Returns:
729             Tuple[List[Dict[str, Any]], Optional[str], int]:
730             (список событий, обновленный bookmark_xml, мак-
731             симальный event_record_id).
732
733         """
734         if not self._available or not self.wevtapi:
735             return [], None, 0
736
737         flags = EVT_QUERY_CHANNEL_PATH
738         if not forward:
739             flags |= EVT_QUERY_REVERSE_DIRECTION
740
741         h_query = None
742         h_bookmark = None
743         events: List[Dict[str, Any]] = []
744         max_record_id = 0

```

[illegible]

```

768         new_bookmark_xml = self.render_bookmark(h_bookmark
769         ) if h_bookmark else None
770         return events, new_bookmark_xml, max_record_id
771
772     except Exception as ex:
773         logger.debug(f'[WevtAPI] Ошибка инкрементального ч
774         тения ({channel}): {ex}')
775         return [], None, 0
776     finally:
777         if h_bookmark:
778             self.wevtapi.EvtClose(h_bookmark)
779         if h_query:
780             self.wevtapi.EvtClose(h_query)
781
782     def subscribe_events(
783     self,
784     channel: str = 'Security',
785     query_xpath: str = '*',
786     callback: Optional[Callable[[Dict[str, Any]], None]] =
787     None,
788     bookmark_xml: Optional[str] = None,
789     start_at_oldest: bool = False,
790     ) -> Optional[wintypes.HANDLE]:
791         """Оформить подписку на входящий поток событий канала
792         через EvtSubscribe.
793
794         Args:
795             channel: Канал журнала событий (например, 'Securit
796             y').
797             query_xpath: XPath-фильтр подписки.
798             callback: Функция обратного вызова при получении с
799             обытия.
800             bookmark_xml: Закладка для возобновления подписки.
801             start_at_oldest: Читать с самого старого события п
802             ри отсутствии закладки.
803
804         Returns:
805             Optional[wintypes.HANDLE]: Дескриптор подписки или

```

```

None.
799     """
800     if not self._available or not self.wevtapi or not call
back:
801         return None
802
803     h_bookmark = self.create_bookmark(bookmark_xml) if boo
kmark_xml else None
804     flags = EVT_SUBSCRIBE_TOLERATE_QUERY_ERRORS
805     if h_bookmark:
806         flags |= EVT_SUBSCRIBE_START_AFTER_BOOKMARK
807     elif start_at_oldest:
808         flags |= EVT_SUBSCRIBE_START_AT_OLDEST
809     else:
810         flags |= EVT_SUBSCRIBE_TO_FUTURE_EVENTS
811
812     def _native_callback(action: int, user_context: wintyp
es.LPVOID, h_event: wintypes.HANDLE) -> int:
813         if action == 1 and h_event: # EvtSubscribeActionD
eliver
814             try:
815                 ev_dict = self._render_event_xml(h_event,
channel, format_message=False)
816                 if ev_dict:
817                     callback(ev_dict)
818             except Exception as ex:
819                 logger.debug(f'[WevtAPI] Ошибка в subscrib
e callback: {ex}')
820                 return 0
821
822     c_cb = EVT_SUBSCRIBE_CALLBACK(_native_callback)
823     try:
824         h_sub = self.wevtapi.EvtSubscribe(
825             None,
826             None,
827             channel,
828             query_xpath,
829             h_bookmark,

```

```

830         None,
831         c_cb,
832         flags
833     )
834     if h_sub:
835         self._active_subscriptions.append((h_sub, c_cb
836     ))
837     return h_sub
838 except Exception as ex:
839     logger.debug(f'[WevtAPI] Ошибка EvtSubscribe ({cha
840 nnel}): {ex}')
841 finally:
842     if h_bookmark:
843         self.wevtapi.EvtClose(h_bookmark)
844     return None
845
846 def unsubscribe(self, h_sub: wintypes.HANDLE) -> bool:
847     """Отменить активную подписку на события."""
848     if not self._available or not self.wevtapi or not h_su
849 b:
850         return False
851     try:
852         self._active_subscriptions = [s for s in self._act
853 ive_subscriptions if s[0] != h_sub]
854         return bool(self.wevtapi.EvtClose(h_sub))
855     except Exception:
856         return False
857
858 def check_channel_access(self, channel_name: str = 'Securi
859 ty') -> Dict[str, Any]:
860     """Проверить доступность и привилегии для чтения канал
861 а Windows Event Log.
862
863     Args:
864         channel_name: Имя канала (по умолчанию 'Security')
865
866     Returns:

```

```

861         Dict[str, Any]: Словарь с информацией о доступе, о
      | ошибках и количестве записей.
862         """
863         if not self._available or not self.wevtapi:
864             return {
865                 'accessible': False,
866                 'channel': channel_name,
867                 'error': 'wevtapi.dll недоступна в текущей сре
      | де',
868                 'record_count': 0,
869             }
870
871         h_log = self.wevtapi.EvtOpenLog(None, channel_name, 1)
872         if not h_log:
873             err = ctypes.GetLastError()
874             err_msg = f'Win32 Error {err}'
875             if err == 5:
876                 err_msg = 'Отказано в доступе (ERROR_ACCESS_DE
      | NIED, код 5). Требуется права Администратора или SeSecurityPri
      | vilege.'
877             elif err == 1314:
878                 err_msg = 'Клиент не обладает требуемыми приви
      | легиями (ERROR_PRIVILEGE_NOT_HELD, код 1314).'
879             return {
880                 'accessible': False,
881                 'channel': channel_name,
882                 'error': err_msg,
883                 'error_code': err,
884                 'record_count': 0,
885             }
886
887         rec_count = 0
888         try:
889             buf = (ctypes.c_byte * 128)()
890             used = wintypes.DWORD(0)
891             if self.wevtapi.EvtGetLogInfo(h_log, 5, 128, ctypes
      | s.byref(buf), ctypes.byref(used)):
892                 rec_count = struct.unpack('<Q', bytes(buf)[:8])

```

```
    ) [0]
893 |         except Exception:
894 |             pass
895 |         finally:
896 |             self.wevtapi.EvtClose(h_log)
897 |
898 |         return {
899 |             'accessible': True,
900 |             'channel': channel_name,
901 |             'error': None,
902 |             'error_code': 0,
903 |             'record_count': rec_count,
904 |         }
905 |
906 |
907 | __all__ = ['WevtAPI', 'ChannelMetadata', 'CHANNEL_DESCRIPTIONS']
```


__init__.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry Win32_Ffi
4 # - Init
5 # Description:
6 #   Модуль низкоуровневых биндингов Win32 FFI / ctypes.
7 #
8 # Usage Examples:
9 #
10 #
11 # File: __init__.py
12 # Project: ai-breadboard
13 # Package: apps.windows.telemetry.win32_fffi
14 # Author: hypo69
15 # Copyright: © 2026 hypo69
16 # Updated: 2026-10-01 13:28:28
17 # =====
18
19 """Модуль низкоуровневых биндингов Win32 FFI / ctypes."""
20
21 from .kernel32 import Kernel32API
22 from .psapi import PsapiAPI
23 from .advapi32 import Advapi32API
24 from .ntdll import NtdllAPI
25 from .etw import EtwAPI
26 from .wevtapi import WevtAPI, ChannelMetadata, CHANNEL_DESCRIPTIONS
27 from .scm import ServiceControlManager, ServiceInfo
28 from .tasksched import TaskSchedulerAPI
29 from .nethelper import IPHelperAPI, NetworkSocketInfo
30 from .setupapi import SetupAPI, PnPDeviceInfo
31 from .error_decoder import WindowsErrorDecoder, format_system_
```

```
error, decode_bugcheck_code
32
33 __all__ = [
34     'Kernel32API',
35     'PsapiAPI',
36     'Advapi32API',
37     'NtdllAPI',
38     'EtwAPI',
39     'WevtAPI',
40     'ChannelMetadata',
41     'CHANNEL_DESCRIPTIONS',
42     'ServiceControlManager',
43     'ServiceInfo',
44     'TaskSchedulerAPI',
45     'IPHelperAPI',
46     'NetworkSocketInfo',
47     'SetupAPI',
48     'PnPDeviceInfo',
49     'WindowsErrorDecoder',
50     'format_system_error',
51     'decode_bugcheck_code',
52 ]
53
```

win_service.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - Windows
  Service Entry
4 # =====
5 # Description:
6 #   Точка входа и обертка Windows Service для фоновой службы A
  ITelemetry.
7 #   Поддерживает исполнение под управлением Service Control Ma
  nager (SCM),
8 #   а также интерактивный запуск через консоль и CLI-команды w
  in32serviceutil.
9 #
10 # Usage Examples:
11 #   CLI:
12 #       AITelemetry.exe install
13 #       AITelemetry.exe start
14 #       AITelemetry.exe stop
15 #       AITelemetry.exe restart
16 #       AITelemetry.exe remove
17 #       AITelemetry.exe --mode hybrid
18 #
19 # File: win_service.py
20 # Project: ai-breadboard
21 # Package: apps.windows.telemetry
22 # Author: hypo69
23 # Copyright: © 2026 hypo69
24 # Updated: 2026-10-06 07:58:00
25 # =====
26
27 from __future__ import annotations
28 """Windows Service реализация для фонового демона сбора телеме
  трии AI-Breadboard."""
```

```
29 |
30 | import logging
31 | import os
32 | import sys
33 | import threading
34 | import time
35 | from pathlib import Path
36 | from typing import Any, Dict, List, Optional
37 |
38 | # Обеспечиваем корректные пути импорта для автономного и скомпилированного режима
39 | _current_dir = Path(__file__).resolve().parent
40 | _project_root = str(_current_dir.parents[2]) if len(_current_dir.parents) >= 3 else str(_current_dir)
41 | if _project_root not in sys.path:
42 |     sys.path.insert(0, _project_root)
43 |
44 | # Настройка системного логгера для службы
45 | def _setup_service_logging() -> None:
46 |     """Настраивает логирование службы в файл ProgramData и консоль."""
47 |     programdata = os.environ.get('ProgramData') or os.environ.get('ALLUSERSPROFILE')
48 |     if programdata and os.path.exists(programdata):
49 |         base_dir = Path(programdata)
50 |     else:
51 |         appdata = os.environ.get('APPDATA') or os.environ.get('LOCALAPPDATA')
52 |         base_dir = Path(appdata) if appdata and os.path.exists(appdata) else Path.home()
53 |
54 |     log_dir = base_dir / 'AITelemetry' / 'logs'
55 |     try:
56 |         log_dir.mkdir(parents=True, exist_ok=True)
57 |         log_file = log_dir / 'telemetry.log'
58 |         root_logger = logging.getLogger()
59 |         root_logger.setLevel(logging.INFO)
60 |
```

```

61         # File handler
62         file_handler = logging.FileHandler(str(log_file), encoding='utf-8')
63         file_handler.setFormatter(logging.Formatter(
64             '%(asctime)s [%(levelname)s] [%(name)s] %(message)s',
65             datefmt='%Y-%m-%d %H:%M:%S'
66         ))
67         root_logger.addHandler(file_handler)
68
69         # Stream handler for interactive execution
70         if not getattr(sys, 'frozen', False) or sys.stdout is not None:
71             console_handler = logging.StreamHandler()
72             console_handler.setFormatter(logging.Formatter(
73                 '%(asctime)s [%(levelname)s] %(message)s',
74                 datefmt='%H:%M:%S'
75             ))
76             root_logger.addHandler(console_handler)
77     except Exception:
78         pass
79
80
81 try:
82     from logger import logger
83 except ImportError:
84     _setup_service_logging()
85     logger = logging.getLogger('AITelemetryService')
86
87 try:
88     import win32serviceutil
89     import win32service
90     import win32event
91     import servicemanager
92     PYWIN32_AVAILABLE = True
93 except ImportError:
94     PYWIN32_AVAILABLE = False
95

```

```

96 |
97 | if PYWIN32_AVAILABLE:
98 |     class AITelemetryWindowsService(win32serviceutil.ServiceFramework):
99 |         """Windows Service адаптер для службы сбора системной
100 |         телеметрии AI-Breadboard."""
101 |         _svc_name_ = 'AITelemetry'
102 |         _svc_display_name_ = 'AI-Breadboard Telemetry Service'
103 |         _svc_description_ = 'Фоновая служба мониторинга, сбора
104 |         телеметрии и аналитики метрик AI-Breadboard'
105 |
106 |     def __init__(self, args: List[str]) -> None:
107 |         """Инициализирует экземпляр службы Windows Service
108 |         ."""
109 |         win32serviceutil.ServiceFramework.__init__(self, args)
110 |         self.hWaitStop = win32event.CreateEvent(None, 0, 0
111 |         , None)
112 |         self.is_alive = True
113 |         self._service_thread: Optional[threading.Thread] =
114 |         None
115 |
116 |     def SvcStop(self) -> None:
117 |         """Обрабатывает сигнал остановки службы от Service
118 |         Control Manager (SCM)."""
119 |         self.ReportServiceStatus(win32service.SERVICE_STOP
120 |         _PENDING)
121 |         logger.info('SCM запросил остановку службы AITelem
122 |         etry...')
123 |         self.is_alive = False
124 |         win32event.SetEvent(self.hWaitStop)
125 |
126 |         # Сигнализируем главному циклу
127 |         try:
128 |             from apps.windows.telemetry.main import _stop_
129 |             event
130 |             _stop_event.set()
131 |         except Exception:

```

```

123         pass
124
125     def SvcShutdown(self) -> None:
126         """Обрабатывает сигнал выключения ОС Windows."""
127         logger.info('Система выключается. Остановка службы
AITelemetry...')
128         self.SvcStop()
129
130     def SvcDoRun(self) -> None:
131         """Основной метод исполнения службы Windows Servic
e."""
132         _setup_service_logging()
133         logger.info('Служба AITelemetry запускается под уп
равлением Windows SCM...')
134         try:
135             servicemanager.LogMsg(
136                 servicemanager.EVENTLOG_INFORMATION_TYPE,
137                 servicemanager.PYS_SERVICE_STARTED,
138                 (self._svc_name_, ''))
139         )
140         except Exception:
141             pass
142
143         self.ReportServiceStatus(win32service.SERVICE_RUNN
ING)
144
145         try:
146             from apps.windows.telemetry.main import run_te
lemetry_service
147             from apps.windows.telemetry.telemetry_config i
mport TelemetryConfigManager
148
149             cfg_mgr = TelemetryConfigManager()
150             mode = cfg_mgr.get_mode()
151             interval = cfg_mgr.get_interval_seconds()
152             heavy_interval = cfg_mgr.get_heavy_interval_se
conds()
153             top_processes = cfg_mgr.get_effective_process_

```

```

limit()
154         heavy_collectors = cfg_mgr.get_heavy_collector
s()
155
156         logger.info(f'Запуск цикла телеметрии (режим:
{mode}, интервал: {interval}с, тяжелый: {heavy_interval}с)')
157         run_telemetry_service(
158             mode=mode,
159             interval=interval,
160             heavy_interval=heavy_interval,
161             top_processes=top_processes,
162             heavy_collectors=heavy_collectors,
163         )
164     except Exception as ex:
165         logger.error(f'Критическая ошибка при работе с
лужбы: {ex}', exc_info=True)
166     finally:
167         logger.info('Служба AITelemetry завершила рабо
ту. ')
168         self.ReportServiceStatus(win32service.SERVICE_
STOPPED)
169
170
171 def run_interactive() -> int:
172     """Запускает телеметрию в интерактивном консольном режиме.
"""
173     _setup_service_logging()
174     from apps.windows.telemetry.main import main
175     return main()
176
177
178 def main_entry() -> int:
179     """Главная точка входа исполняемого файла AITelemetry.exe.
180
181     Returns:
182         int: Код возврата.
183     """
184     _setup_service_logging()

```



```

185
186     # Проверяем запуск диспетчером SCM без параметров
187     if len(sys.argv) == 1:
188         if PYWIN32_AVAILABLE:
189             try:
190                 servicemanager.Initialize()
191                 servicemanager.PrepareToHostSingle(AITelemetry
WindowsService)
192                 servicemanager.StartServiceCtrlDispatcher()
193                 return 0
194             except Exception as ex:
195                 # Если запущен не из SCM, переключаемся в конс
ольный режим
196                 logger.debug(f'Запуск через SCM не удался ({ex
}), переход в консольный режим.')
197                 return run_interactive()
198         else:
199             return run_interactive()
200
201     cmd = sys.argv[1].lower() if len(sys.argv) > 1 else ''
202
203     # Команды управления службой
204     if cmd in ('install', 'update', 'remove', 'start', 'stop',
'restart', 'status'):
205         if PYWIN32_AVAILABLE:
206             win32serviceutil.HandleCommandLine(AITelemetryWind
owsService)
207             return 0
208         else:
209             logger.error('Модуль pywin32 недоступен. Управлени
е службой через SCM невозможно.')
210             return 1
211
212     if cmd in ('run', 'interactive', '--mode', '--minimal', '--
interval', '--once', '-v', '--verbose', '--help', '-h'):
213         return run_interactive()
214
215     # По умолчанию передаем аргументы в главный CLI

```

```
216 |     return run_interactive()
217 |
218 |
219 | if __name__ == '__main__':
220 |     sys.exit(main_entry())
221 |
```

__init__.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - Package
  Root
4 # =====
5 # Description:
6 #   Слой 3: Сбор, хранение, каталогизация и аналитика телеметр
  ии Windows.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry import SystemCollector, Diag
  nosticEngine
11 #
12 # File: __init__.py
13 # Project: ai-breadboard
14 # Package: apps.windows.telemetry
15 # Author: hypo69
16 # Copyright: © 2026 hypo69
17 # Updated: 2026-10-08 12:20:00
18 # =====
19
20 from __future__ import annotations
21 """Единый слой телеметрии Windows (Слой 3) платформы AI-Breadb
  oard."""
22
23 # Импорт перечислений из слоя apps.windows.contracts
24 from apps.windows.contracts import (
25     RiskLevel,
26     TelemetryTier,
27     ProcessState,
28     ThreadState,
29 )
```

```
30 |
31 | # Модели телеметрии
32 | from apps.windows.telemetry.models import (
33 |     AnomalyItem,
34 |     BatteryMetrics,
35 |     CloudStorageInfo,
36 |     CpuMetrics,
37 |     DiskIoMetrics,
38 |     DiskPartitionMetrics,
39 |     DriverInfo,
40 |     ForensicsActivityReport,
41 |     GpuMetrics,
42 |     HardwareArchiveEntry,
43 |     HardwareAuditReport,
44 |     HardwareChangeItem,
45 |     HardwareDeviceAudit,
46 |     HardwareNode,
47 |     HardwareSensor,
48 |     KernelThrottlingReport,
49 |     MemoryMetrics,
50 |     MonitorInfo,
51 |     NetworkInterfaceMetrics,
52 |     NetworkPortMetrics,
53 |     NpuMetrics,
54 |     OfficeSuiteInfo,
55 |     PeripheralsNetworkReport,
56 |     PhysicalDiskHealth,
57 |     ProcessLeakDiagnosticsReport,
58 |     ProcessLeakItem,
59 |     ProcessLifecycleEvent,
60 |     ProcessMetrics,
61 |     ProcessNetworkActivity,
62 |     ProcessProvenanceInfo,
63 |     ProcessProvenanceReport,
64 |     ProcessTokenInfo,
65 |     RamStickInfo,
66 |     StartupArchiveEntry,
67 |     StartupChangeItem,
```

```
68 |     StorageBatteryWearReport,
69 |     SystemCoreMetrics,
70 |     SystemDiagnosticReport,
71 |     SystemHardwareQuick,
72 |     SystemHealthAlerts,
73 |     SystemSnapshot,
74 |     TelemetryIncident,
75 |     W64CollectorStatus,
76 |     W64SystemEvent,
77 |     WindowsUpdateInfo,
78 |     ETWTraceEvent,
79 |     SecurityAuditStatus,
80 |     SecurityBookmarkState,
81 |     SecurityCollectorReport,
82 |     SecurityCorrelationItem,
83 |     SecurityEventItem,
84 |     SecurityEventRaw,
85 | )
86 |
87 | # Хранилище и база данных
88 | from apps.windows.telemetry.sqlite import (
89 |     AggregationLevel,
90 |     TelemetryBuffer,
91 |     TelemetryConnectionManager,
92 |     TelemetryMaintenance,
93 |     TelemetryReader,
94 |     TelemetrySqlAggregator,
95 |     TelemetryStorage,
96 |     TelemetryWriter,
97 |     sensors_aggregate,
98 | )
99 | from apps.windows.telemetry.storage.ring_buffer import TelemetryRingBuffer
100 | from apps.windows.telemetry.storage.init_db import (
101 |     init_telemetry_db,
102 |     init_telemetry_database,
103 |     get_default_telemetry_db_path,
104 | )
```

```
105 |
106 | # Коллекторы и датчики
107 | from apps.windows.telemetry.sensors import get_hardware_sensor
    | s
108 | from apps.windows.telemetry.collector import SystemCollector
109 | from apps.windows.telemetry.service import TelemetryLoggerService
    | ice
110 | from apps.windows.telemetry.telemetry_engine import TelemetryEngine, DeadbandTracker
    | engine, DeadbandTracker
111 | from apps.windows.telemetry.telemetry_config import TelemetryConfigManager, get_default_telemetry_config_path
    | onfigManager, get_default_telemetry_config_path
112 | from apps.windows.telemetry.json_logger import TelemetryJsonLogger
    | gger
113 | from apps.windows.telemetry.file_collector import FileCollector
    | r
114 | from apps.windows.telemetry.sensor_collector import SensorCollector
    | ector
115 | from apps.windows.telemetry.device_flapping_sensor import DeviceFlappingSensor, DeviceTransitionEvent
    | ceFlappingSensor, DeviceTransitionEvent
116 | from apps.windows.telemetry.storage_usage import WindowsStorageUsageCollector
    | eUsageCollector
117 | from apps.windows.telemetry.w64_collector import (
118 |     AIW64Collector,
119 |     get_w64_collector,
120 |     start_w64_collector,
121 |     stop_w64_collector,
122 | )
123 | from apps.windows.telemetry.w64_etw_collector import AIW64ETWCollector
    | ollector
124 | from apps.windows.telemetry.security_collector import WindowsSecurityCollector
    | ecurityCollector
125 | from apps.windows.telemetry.security_normalizer import SecurityEventNormalizer
    | yEventNormalizer
126 |
127 | # Аналитический контур (прямой импорт без lazy __getattr__)
128 | from apps.windows.telemetry.analytics.aggregator import TelemetryAggregator
    | tryAggregator
129 | from apps.windows.telemetry.analytics.analyzer import TelemetryAnalyzer
    | yAnalyzer
```

```
    yAnalyzer
130 from apps.windows.telemetry.analytics.compactor import Telemet
    ryCompactor
131 from apps.windows.telemetry.analytics.diagnostic_engine import
    DiagnosticEngine, SystemDiagnosticEngine
132 from apps.windows.telemetry.analytics.grouped_telemetry import
    GroupedTelemetryBuilder
133 from apps.windows.telemetry.analytics.incident_detector import
    IncidentDetector
134 from apps.windows.telemetry.analytics.reboot_analyzer import W
    indowsRebootAnalyzer, RebootAnalyzer
135 from apps.windows.telemetry.analytics.hardware_auditor import
    HardwareAuditor
136 from apps.windows.telemetry.analytics.deep_diagnostics import
    DeepDiagnosticsEngine
137 from apps.windows.telemetry.analytics.hardware_history_manager
    import HardwareHistoryManager
138
139 # Каталог событий
140 from apps.windows.telemetry.catalog.event_catalog import Windo
    wsEventCatalog
141
142 __all__ = [
143     # Контракты и модели
144     "RiskLevel",
145     "TelemetryTier",
146     "ProcessState",
147     "ThreadState",
148     "CpuMetrics",
149     "MemoryMetrics",
150     "GpuMetrics",
151     "HardwareSensor",
152     "HardwareNode",
153     "SystemSnapshot",
154     "ProcessMetrics",
155     "ProcessTokenInfo",
156     "TelemetryIncident",
157     "AnomalyItem",
```

```
158 | "BatteryMetrics",
159 | "DiskIoMetrics",
160 | "DiskPartitionMetrics",
161 | "DriverInfo",
162 | "ForensicsActivityReport",
163 | "HardwareArchiveEntry",
164 | "HardwareAuditReport",
165 | "HardwareChangeItem",
166 | "HardwareDeviceAudit",
167 | "KernelThrottlingReport",
168 | "NetworkInterfaceMetrics",
169 | "NetworkPortMetrics",
170 | "PeripheralsNetworkReport",
171 | "PhysicalDiskHealth",
172 | "ProcessLeakDiagnosticsReport",
173 | "ProcessLeakItem",
174 | "ProcessLifecycleEvent",
175 | "ProcessNetworkActivity",
176 | "ProcessProvenanceInfo",
177 | "ProcessProvenanceReport",
178 | "RamStickInfo",
179 | "StorageBatteryWearReport",
180 | "SystemDiagnosticReport",
181 | "SystemHealthAlerts",
182 | "W64CollectorStatus",
183 | "W64SystemEvent",
184 | "ETWTraceEvent",
185 | "SecurityAuditStatus",
186 | "SecurityBookmarkState",
187 | "SecurityCollectorReport",
188 | "SecurityCorrelationItem",
189 | "SecurityEventItem",
190 | "SecurityEventRaw",
191 | # Коллекторы
192 | "SystemCollector",
193 | "SensorCollector",
194 | "FileCollector",
195 | "WindowsSecurityCollector",
```



```
196 | "SecurityEventNormalizer",
197 | "WindowsStorageUsageCollector",
198 | "AIW64Collector",
199 | "AIW64ETWCollector",
200 | "get_w64_collector",
201 | "start_w64_collector",
202 | "stop_w64_collector",
203 | "get_hardware_sensors",
204 | "DeviceFlappingSensor",
205 | "DeviceTransitionEvent",
206 | # Хранилище
207 | "TelemetryRingBuffer",
208 | "init_telemetry_db",
209 | "init_telemetry_database",
210 | "get_default_telemetry_db_path",
211 | "TelemetryStorage",
212 | "TelemetrySqlAggregator",
213 | "AggregationLevel",
214 | "sensors_aggregate",
215 | "TelemetryLoggerService",
216 | "TelemetryEngine",
217 | "DeadbandTracker",
218 | "TelemetryConfigManager",
219 | "TelemetryJsonLogger",
220 | "get_default_telemetry_config_path",
221 | # Аналитика
222 | "DiagnosticEngine",
223 | "SystemDiagnosticEngine",
224 | "GroupedTelemetryBuilder",
225 | "IncidentDetector",
226 | "WindowsRebootAnalyzer",
227 | "RebootAnalyzer",
228 | "TelemetryAggregator",
229 | "TelemetryAnalyzer",
230 | "TelemetryCompactor",
231 | "HardwareAuditor",
232 | "DeepDiagnosticsEngine",
233 | "HardwareHistoryManager",
```

```
234 | # Каталог
235 | "WindowsEventCatalog",
236 | ]
```